Sommario

Prefazione XVII
Prefazione all'edizione italiana XXI

Capitolo 1 Introduzione 1

1.1 Approccio strutturale 2
1.1.1 Linguaggi, livelli e macchine virtuali 2
1.1.2 Attuali macchine multilivello 4
1.1.3 Evoluzione delle macchine multilivello 8

1.2 Pietre miliari nell'architettura dei computer 13
1.2.1 Generazione zero – Computer meccanici (1642-1945) 13
1.2.2 Prima generazione – Valvole (1945-1955) 16
1.2.3 Seconda generazione – Transistor (1955-1965) 19
1.2.4 Terza generazione – Circuiti integrati (1965-1980) 22
1.2.5 Quarta generazione – Integrazione a grandissima scala (1980-?) 24
1.2.6 Quinta generazione – Computer a basso consumo e computer invisibili 27

1.3 Tipologie di computer 29
1.3.1 Forze tecnologiche ed economiche 29
1.3.2 Tipologie di computer 31
1.3.3 Computer usa e getta 32
1.3.4 Microcontrollori 34
1.3.5 Dispositivi mobili e da gioco 36
1.3.6 Personal computer 37
1.3.7 Server 37
1.3.8 Mainframe 40

1.4 Esempi di famiglie di computer 40
1.4.1 Introduzione all'architettura x86 41
1.4.2 Introduzione all'architettura ARM 46
1.4.3 Introduzione all'arhitettura AVR 49

1.5 Unità metriche 50

1.6 Organizzazione del libro 51

Capitolo 2 Organizzazione dei sistemi di calcolo 55

2.1 Processori 55
2.1.1 Organizzazione della CPU 56

2.1.2 Esecuzione dell'istruzione 58
2.1.3 RISC contro CISC 62
2.1.4 Principi di progettazione dei calcolatori moderni 63
2.1.5 Parallelismo a livello d'istruzione 65
2.1.6 Parallelismo a livello di processore 69
2.2 Memoria principale 74
2.2.1 Bit 74
2.2.2 Indirizzi di memoria 75
2.2.3 Ordinamento dei byte 76
2.2.4 Codici correttori 78
2.2.5 Memoria cache 82
2.2.6 Assemblaggio e tipi di memoria 86
2.3 Memoria secondaria 87
2.3.1 Gerarchie di memoria 87
2.3.2 Dischi magnetici 88
2.3.3 Dischi IDE 92
2.3.4 Dischi SCSI 94
2.3.5 RAID 95
2.3.6 Dischi a stato solido 99
2.3.7 CD-ROM 101
2.3.8 CD-registrabili 105
2.3.9 CD-riscrivibili 108
2.3.10 DVD 108
2.3.11 Blu-Ray 111
2.4 Input/Output 111
2.4.1 Bus 111
2.4.2 Terminali 115
2.4.3 Mouse 121
2.4.4 Controller per videogiochi 123
2.4.5 Stampanti 125
2.4.5 Apparecchiature per le telecomunicazioni 130
2.4.6 Macchine fotografiche digitali 139
2.4.7 Codifica dei caratteri 141
2.5 Riepilogo 146

Capitolo 3 Livello logico digitale 151

3.1 Porte logiche e algebra di Boole 151
3.1.1 Porte logiche 152
3.1.2 Algebra di Boole 154
3.1.3 Implementazione delle funzioni booleane 156
3.1.4 Equivalenza di circuiti 158

3.2 Circuiti logici digitali elementari 162
3.2.1 Circuiti integrati 162
3.2.2 Reti combinatorie 163
3.2.3 Circuiti per l'aritmetica 169
3.2.4 Clock 173
3.3 Memoria 174
3.3.1 Latch 175
3.3.2 Flip-flop 177
3.3.3 Registri 180
3.3.4 Organizzazione della memoria 180
3.3.5 Chip di memoria 183
3.3.6 RAM e ROM 186
3.4 Chip della CPU e bus 191
3.4.1 Chip della CPU 192
3.4.2 Bus del calcolatore 194
3.4.3 Ampiezza del bus 196
3.4.4 Temporizzazione del bus 198
3.4.5 Arbitraggio del bus 202
3.4.6 Operazioni del bus 205
3.5 Esempi di chip della CPU 208
3.5.1 Intel Core i7 208
3.5.2 Texas Instruments OMAP4430 215
3.5.3 Il microcontrollore Atmel ATmega168 219
3.6 Esempi di bus 221
3.6.1 Bus PCI 222
3.6.2 PCI Express 230
3.6.3 Universal Serial Bus 235
3.7 Interfacce 239
3.7.1 Interfacce di I/O 239
3.7.2 Decodifica dell'indirizzo 240
3.8 Riepilogo 243

Capitolo 4 Livello di microarchitettura 249

4.1 Esempio di microarchitettura 249
4.1.1 Percorso dati 250
4.1.2 Microistruzioni 257
4.1.3 Unità di controllo microprogrammata: Mic-1 259
4.2 Esempio di ISA: IJVM 264
4.2.1 Stack 264
4.2.2 Modello della memoria di IJVM 267

4.2.3 Insieme d'istruzioni IJVM 268
4.2.4 Compilazione da Java a IJVM 273
4.3 Implementazione di esempio 274
4.3.1 Microistruzioni e notazione 274
4.3.2 Implementazione di IJVM con Mic-1 279
4.4 Progettazione del livello di microarchitettura 291
4.4.1 Velocità/costi 292
4.4.2 Riduzione della lunghezza del percorso di esecuzione 294
4.4.3 Architettura con prefetching: Mic-2 301
4.4.4 Architettura a pipeline: Mic-3 305
4.4.5 Pipeline a sette stadi: Mic-4 310
4.5 Miglioramento delle prestazioni 314
4.5.1 Memoria cache 314
4.5.2 Predizione dei salti 321
4.5.3 Esecuzione fuori sequenza e rinomina dei registri 326
4.5.4 Esecuzione speculativa 332
4.6 Esempi del livello di microarchitettura 334
4.6.1 Microarchitettura della CPU Core i7 335
4.6.2 Microarchitettura della CPU OMAP4430 341
4.6.3 Microarchitettura del microcontrollore ATmega168 346
4.7 Confronto tra i7, OMAP4430 e ATmega168 348
4.8 Riepilogo 349

Capitolo 5 Livello di architettura dell'insieme d'istruzioni 353

5.1 Panoramica del livello ISA 355
5.1.1 Proprietà del livello ISA 355
5.1.2 Modelli di memoria 357
5.1.3 Registri 359
5.1.4 Istruzioni 361
5.1.5 Panoramica del livello ISA del Core i7 361
5.1.6 Panoramica del livello ISA dell'OMAP4430 ARM 363
5.1.7 Panoramica del livello ISA dell'ATmega168 AVR 366
5.2 Tipi di dati 367
5.2.1 Tipi di dati numerici 368
5.2.2 Tipi di dati non numerici 369
5.2.3 Tipi di dati del Core i7 370
5.2.4 Tipi di dati dell'OMAP4430 ARM 370
5.2.5 Tipi di dati dell'ATmega168 371

5.3 Formati d'istruzione 371
5.3.1 Criteri progettuali per i formati d'istruzioni 372
5.3.2 Codice operativo espandibile 374
5.3.3 Formati delle istruzioni del Core i7 377
5.3.4 Formati delle istruzioni dell'OMAP4430 ARM 378
5.3.5 Formati delle istruzioni dell'ATmega168 AVR 380
5.4 Indirizzamento 381
5.4.1 Modalità d'indirizzamento 381
5.4.2 Indirizzamento immediato 381
5.4.3 Indirizzamento diretto 382
5.4.4 Indirizzamento a registro 382
5.4.5 **Indirizzamento a registro indiretto 382**
5.4.6 Indirizzamento indicizzato 384
5.4.7 Indirizzamento indicizzato esteso 385
5.4.8 Indirizzamento a stack 386
5.4.9 Modalità d'indirizzamento per istruzioni di salto 389
5.4.10 Ortogonalità dei codici operativi e delle modalità d'indirizzamento 390
5.4.11 Modalità d'indirizzamento del Core i7 392
5.4.12 Modalità d'indirizzamento dell'OMAP4430 394
5.4.13 Modalità d'indirizzamento dell'ATmega168 AVR 394
5.4.14 Analisi delle modalità d'indirizzamento 395
5.5 Tipi d'istruzioni 396
5.5.1 Istruzioni di trasferimento dati 396
5.5.2 Operazioni binarie 397
5.5.3 Operazioni unarie 398
5.5.4 Confronti e salti condizionati 400
5.5.5 Invocazione di procedura 402
5.5.6 Istruzioni di ciclo 403
5.5.7 Input/Output 404
5.5.8 Istruzioni del Core i7 407
5.5.9 Istruzioni della CPU ARM OMAP4430 410
5.5.10 Istruzioni dell'ATmega168 AVR 412
5.5.11 Insiemi d'istruzioni a confronto 414
5.6 Controllo del flusso 415
5.6.1 Flusso sequenziale e diramazioni 415
5.6.2 Procedure 416
5.6.3 Coroutine 421
5.6.4 Trap 423
5.6.5 Interrupt 424

5.7 Un esempio: le torri di Hanoi 428
5.7.1 Le torri di Hanoi nel linguaggio assemblativo del Core i7 428
5.7.2 Le torri di Hanoi nel linguaggio assemblativo dell'OMAP4430 ARM 430
5.8 Architettura IA-64 e Itanium 2 431
5.8.1 Il problema dell'ISA IA-32 432
5.8.2 Modello IA-64 e calcolo che utilizza il parallelismo esplicito 433
5.8.3 Riduzione degli accessi in memoria 433
5.8.4 Scheduling delle istruzioni 434
5.8.5 Riduzione dei salti condizionati: attribuzione di predicati 436
5.8.6 Caricamenti speculativi 439
5.9 Riepilogo 440

Capitolo 6 Livello macchina del sistema operativo 445
6.1 Memoria virtuale 446
6.1.1 Paginazione 447
6.1.2 Implementazione della paginazione 449
6.1.3 Paginazione a richiesta e working set 452
6.1.4 Politica di sostituzione delle pagine 454
6.1.5 Dimensione di pagina e frammentazione 456
6.1.6 Segmentazione 457
6.1.7 Implementazione della segmentazione 461
6.1.8 Memoria virtuale del Core i7 464
6.1.9 Memoria virtuale della CPU ARM OMAP4430 468
6.1.10 Memoria virtuale e caching 471
6.2 Virtualizzazione hardware 471
6.2.1 Virtualizzazione hardware nel Core i7 473
6.3 Istruzioni di I/O a livello OSM 473
6.3.1 File 474
6.3.2 Implementazione delle istruzioni di I/O a livello OSM 475
6.3.3 Istruzioni per la gestione di directory 479
6.4 Istruzioni per il calcolo parallelo a livello OSM 481
6.4.1 Creazione dei processi 481
6.4.2 Corsa critica 482
6.4.3 Sincronizzazione dei processi tramite semafori 486
6.5 Sistemi operativi di esempio 490
6.5.1 Introduzione 490
6.5.2 Esempi di memoria virtuale 496

6.5.3 Esempi di I/O a livello OS 500
6.5.4 Esempi di gestione dei processi 512
6.6 Riepilogo 518

Capitolo 7 Livello del linguaggio assemblativo 525
7.1 Introduzione al linguaggio assemblativo 526
7.1.1 Che cos'è un linguaggio assemblativo 526
7.1.2 Perché usare il linguaggio assemblativo 527
7.1.3 Formato delle istruzioni del linguaggio assemblativo 528
7.1.4 Pseudoistruzioni 530
7.2 Macroistruzioni 532
7.2.1 Definizione, chiamata ed espansione di macro 533
7.2.2 Macro con parametri 535
7.2.3 Caratteristiche avanzate 535
7.2.4 Implementazione delle funzionalità macro negli assemblatori 536
7.3 Processo di assemblaggio 537
7.3.1 Assemblatori a due passate 537
7.3.2 Prima passata 538
7.3.3 Seconda passata 542
7.3.4 Tabella dei simboli 544
7.4 Collegamento e caricamento 546
7.4.1 Compiti del linker 547
7.4.2 Struttura di un modulo oggetto 550
7.4.3 Rilocazione a tempo del binding e dinamica 551
7.4.4 Collegamento dinamico 554
7.5 Riepilogo 558

Capitolo 8 Architetture per il calcolo parallelo 561
8.1 Parallelismo nel chip 562
8.1.1 Parallelismo a livello delle istruzioni 563
8.1.2 Multithreading nel chip 570
8.1.3 Multiprocessori in un solo chip 576
8.2 Coprocessori 582
8.2.1 Processori di rete 583
8.2.2 Processori grafici 591
8.2.3 Crittoprocessori 594
8.3 Multiprocessori con memoria condivisa 594
8.3.1 Multiprocessori e multicomputer a confronto 594
8.3.2 Semantica della memoria 602

8.3.3 Architetture di multiprocessori simmetrici UMA 606
8.3.4 Multiprocessori NUMA 614
8.3.5 Multiprocessori COMA 623

8.4 Multicomputer a scambio di messaggi 625
8.4.1 Reti d'interconnessione 626
8.4.2 MPP: processori massicciamente paralleli 630
8.4.3 Cluster 640
8.4.4 Software di comunicazione per multicomputer 645
8.4.5 Scheduling 648
8.4.6 Memoria condivisa a livello applicativo 649
8.4.7 Prestazioni 657

8.5 Grid computing 663

8.6 Riepilogo 666

Capitolo 9 Bibliografia 671

Appendice A Aritmetica binaria 681

A.1 Numeri a precisione finita 681

A.2 Sistemi di numerazione in base fissa 683

A.3 Conversione tra basi 684

A.4 Numeri binari negativi 688

A.5 Aritmetica binaria 690

Appendice B Numeri in virgola mobile 693

B.1 Principi dell'aritmetica in virgola mobile 693

B.2 Standard in virgola mobile IEEE 754 696

Appendice C Programmazione in linguaggio assemblativo 703

C.1 Panoramica 704
C.1.1 Linguaggio assemblativo 704
C.1.2 Breve programma in linguaggio assemblativo 705

C.2 Processore 8088 706
C.2.1 Ciclo del processore 706
C.2.2 Registri d'uso generale 708
C.2.3 Registri puntatore 709

C.3 Memoria e indirizzamento 710
C.3.1 Organizzazione di memoria e segmenti 711
C.3.2 Indirizzamento 712

C.4 Istruzioni dell'8088 715
C.4.1 Trasferimento, copia e aritmetica 716
C.4.2 Operazioni logiche, su bit e di scorrimento 718
C.4.3 Cicli e operazioni iterative su stringhe 719
C.4.4 Istruzioni di salto e di chiamata 720
C.4.5 Chiamate di subroutine 722
C.4.6 Chiamate di sistema e subroutine di sistema 723
C.4.7 Osservazioni sull'insieme d'istruzioni 725

C.5 Assemblatore 726
C.5.1 Introduzione 726
C.5.2 as88, un assemblatore basato su ACK 727
C.5.3 Differenze tra assemblatori dell'8088 731

C.6 Tracer 732
C.6.1 Comandi del tracer 734

C.7 Installazione 735

C.8 Esempi 736
C.8.1 Esempio Hello World 737
C.8.2 Esempio con i registri d'uso generale 740
C.8.3 Istruzioni di chiamata e puntatori ai registri 741
C.8.4 Debugging di un programma per la stampa di array 744
C.8.5 Manipolazione di stringhe e istruzioni su stringhe 747
C.8.6 Tabella di salto 750
C.8.7 File: accesso diretto e accesso bufferizzato 752

Indice analitico 757

Prefazione

Le prime cinque edizioni di questo libro erano basate sull'idea che un computer può essere visto come una gerarchia di livelli, in cui ciascuno realizza una particolare e ben definita funzione. Questo fondamentale concetto è valido ancor oggi così come lo era quando venne pubblicata la prima edizione; per questo motivo è stato mantenuto come base anche di questa pubblicazione. Come nelle precedenti edizioni, sono trattati in modo dettagliato il livello logico digitale, il livello di micro-architettura, l'insieme delle istruzioni, il livello di architettura, il livello macchina del sistema operativo e quello del linguaggio assemblativo.

Pur mantenendo quindi la struttura di base, questa nuova edizione contiene molti cambiamenti, che permettono al testo di restare al passo con la rapida evoluzione dell'industria dei computer. Sono stati aggiornati anche i modelli utilizzati come casi di studio nel testo. Gli esempi presentati sono ora Intel Core i7, Texas Instrument OMAP4430 e Atmel ATmega168. Il Core i7 è un esempio di CPU largamente utilizzata su portatili, desktop e server, mentre l'OMAP4430 è una diffusissima CPU basata su ARM, molto utilizzata su smartphone e tablet.

Anche se probabilmente non avete mai sentito parlare del microcontrollore ATmega168, vi sarà spesso capitato di utilizzarne uno. L'ATmega168, basato su AVR, è presente in diversi sistemi integrati, a partire dalle radiosveglie digitali fino ai forni a microonde. L'interesse per i sistemi integrati è in continua crescita e l'ATmega168 rappresenta una scelta ampiamente diffusa grazie al suo costo estremamente contenuto (qualche centesimo di euro), alla disponibilità di software e periferiche e al gran numero di programmatori specializzati. Il numero di microcontrollori ATmega168 esistenti supera di gran lunga il numero di processori Pentium e Core i3, i5 e i7. L'ATmega168 è anche utilizzato sulla piattaforma integrata a scheda singola (*single-board embedded computer*) Arduino, un popolare sistema per hobbisti progettato in Italia, e costa meno di una cena in pizzeria.

Nel corso degli anni, molti docenti mi hanno richiesto a più riprese materiale sulla programmazione in linguaggio assemblativo. Con la sesta edizione il materiale è reso disponibile alla pagina web di questo libro (indicata in seguito), dove può essere facilmente aggiornato ed espanso. Il linguaggio assemblativo scelto è quello dell'8088, che rappresenta una versione molto ridotta del diffusissimo insieme d'istruzioni iA32 utilizzato nei processori Core i7. Avremmo potuto utilizzare ARM, AVR o qualsiasi altra architettura della quale nessuno ha mai sentito parlare, ma l'8088, come strumento motivazionale, rappresenta la scelta migliore, perché molti studenti hanno a disposizione una CPU compatibile con l'8088. L'intero insieme di istruzioni del Core i7 è troppo

complesso per essere compreso dagli studenti in dettaglio: l'8088 è simile, ma molto più semplice. Il Core i7, descritto in dettaglio in questa edizione, è inoltre in grado di eseguire codice 8088. Tuttavia, poiché effettuare il debug del codice assemblativo è particolarmente difficile, ho messo a disposizione un insieme di strumenti per facilitare l'apprendimento della programmazione assemblativa, tra cui un assemblatore 8088, un simulatore e un *tracer*. Questi strumenti sono forniti per Windows, Unix e Linux e si trovano sul sito web del libro.

Nel corso degli anni il testo si è allungato (la prima edizione contava 443 pagine, questa ne conta 769). Una crescita inevitabile in quanto la materia continua a svilupparsi e vi è un numero sempre maggiore di cose da conoscere. Di conseguenza, quando il libro è utilizzato come testo di un corso semestrale, potrebbe non essere possibile terminarlo. Un possibile approccio potrebbe allora essere quello di studiare, come minimo indispensabile, i Capitoli 1, 2 e 3 interamente, la prima parte del Capitolo 4 (fino al Paragrafo 4.4 compreso) e il Capitolo 5. L'eventuale tempo rimanente potrebbe essere impiegato per il resto del Capitolo 4 e per parti dei Capitoli 6, 7 e 8 a seconda delle preferenze di docente e studenti.

Vediamo ora rapidamente le principali novità di questa nuova edizione. Il primo capitolo presenta ancora una panoramica storica delle architetture dei computer nella quale si evidenzia come si sia raggiunto lo stato attuale presentando le pietre miliari che hanno marcato il cammino. Molti studenti resteranno stupiti scoprendo che i computer più potenti degli anni '60 costavano milioni di dollari, ma avevano molto meno dell'uno per cento della potenza di calcolo dei loro smartphone. Nel corso del capitolo viene presentato l'ampio spettro di computer oggi esistenti, tra cui FPGA (*field-programmable gate arrays*), smartphone, tablet e console per videogiochi, e vengono introdotti i nostri tre esempi di architetture (Core i7, OMAP4430 e ATmega168).

Nel Capitolo 2 è stato ampliato il materiale sugli stili di elaborazione per includere i processori con parallelismo sui dati, tra cui le unità di elaborazione grafica (GPU). La panoramica sui dispositivi di memorizzazione è stata allargata per includere le sempre più diffuse memorie flash. È stato inoltre aggiunto nuovo materiale alla sezione Input/Output, con dettagli sui moderni controller di gioco, tra cui il Wiimote e il Kinect e sugli schermi interattivi utilizzati su smartphone e tablet.

Il Capitolo 3 è stato sottoposto a una profonda revisione. Tuttavia si parte ancora dal funzionamento dei transistor di modo che anche gli studenti totalmente privi di conoscenze hardware siano in grado di capire in linea di principio come funziona un moderno computer. Forniremo nuove informazioni sugli FPGA, strutture hardware programmabili che abbassano i costi di progetto a livello di porta logica a tal punto da renderli utilizzabili nelle scuole. In questo capitolo diamo una descrizione ad alto livello delle tre nuove architetture di esempio.

Il Capitolo 4, che è sempre stato molto apprezzato per la chiara spiegazione del reale funzionamento di un computer, non ha subìto modifiche rispetto alla precedente edizione. Sono stati comunque aggiunti alcuni nuovi paragrafi che trattano il livello di microarchitettura di Core i7, OMAP4430 e ATmega168.

I Capitoli 5 e 6 sono stati aggiornati usando le nuove architetture d'esempio, in particolare con l'aggiunta di nuovi paragrafi che descrivono le istruzioni ARM e AVR. Il Capitolo 6 utilizza come esempio Windows 7 e non più Windows XP.

Il Capitolo 7, sulla programmazione in linguaggio assemblativo, resta in gran parte invariato rispetto al passato.

Per riflettere i nuovi sviluppi nell'ambito del calcolo parallelo, il Capitolo 8 ha subìto una notevole revisione. Sono stati aggiunti nuovi particolari sull'architettura multiprocessore del Core i7 ed è descritta in dettaglio l'architettura della GPU NVIDIA Fermi. Infine, i paragrafi sui supercomputer BlueGene e Red Storm sono stati aggiornati per includere le più recenti modifiche apportate a queste enormi macchine.

Anche il Capitolo 9 è cambiato. Le letture suggerite sono state trasferite sulla pagina web e ora il capitolo contiene solo i riferimenti citati nel libro, molti dei quali sono nuovi: l'architettura dei calcolatori è un settore dinamico.

Le Appendici A e B non sono state modificate rispetto all'edizione precedente: negli ultimi anni la notazione binaria e in virgola mobile non è molto cambiata. L'Appendice C è stata scritta dal Dott. Evert Wattel della Vrije Universiteit di Amsterdam. Il Dott. Wattel ha un'esperienza pluriennale d'insegnamento mediante questi strumenti. A lui vanno i miei ringraziamenti per aver scritto questa appendice. Ci sono poche variazioni rispetto alla quinta edizione, ma gli strumenti a disposizione sono ora sul web e non più su un CD-ROM allegato.

Oltre agli strumenti per la programmazione in linguaggio assemblativo, il sito web contiene un simulatore grafico da utilizzare parallelamente allo studio del quarto capitolo. Gli studenti possono utilizzarlo come aiuto per meglio cogliere i principi ivi discussi. Il simulatore grafico è stato scritto dal Prof. Richard Salter dell'Oberlin College, e a lui vanno i nostri ringraziamenti.

Il sito web è raggiungibile all'indirizzo

http://www.pearsonhighered.com/tanenbaum

Da qui selezionate il collegamento a questo testo e scegliete la pagina che state cercando. Le risorse per gli studenti includono:

- assemblatore e tracer;
- simulatore grafico;
- letture suggerite.

Le risorse per i docenti includono:

- slide PowerPoint con le figure e le tabelle del testo;
- soluzioni degli esercizi di fine capitolo (in lingua inglese).

Per accedere a queste risorse, i docenti che adottano il testo dovranno registrarsi nell'Area Docenti sul sito web http://hpe.pearson.it.

Diverse persone hanno letto questo volume (o una sua parte) e fornito consigli utili, o sono state in altro modo d'aiuto. Vogliamo ringraziare, in particolare, Anna Austin, Mark Austin, Livio Bertacco, Valeria Bertacco, Debapriya Chatterjee, Jason Clemons, Andrew DeOrio, Joseph Greathouse e Andrea Pellegrini.

Ringraziamo le seguenti seguenti persone, che hanno rivisto il testo e suggerito alcuni cambiamenti: Jason D. Bakos (University of South Carolina), Bob Brown (Southern Polytechnic State University), Andrew Chen (Minnesota State University, Moorhead), J. Archer Harris (James Madison University), Susan Krucke (James Madison University), A. Yavuz Oruc (University of Maryland), Frances Marsh (Jamestown Community College), Kris Schindler (University at Buffalo).

Varie persone ci hanno aiutato nella creazione di nuovi esercizi: Byron A. Jeff (Clayton University), Laura W. McFall (DePaul University), Taghi M. Mostafavi (University of North Carolina at Charlotte), James Nystrom (Ferris State University). Abbiamo apprezzato, ancora una volta, il loro aiuto.

Il nostro editor, Tracy Johnson, ci è stato d'aiuto in tutti i modi ed è stato molto paziente. L'assistenza di Carole Snyder nel coordinamento delle persone coinvolte nel progetto è stata molto apprezzata. Bob Englehardt ha fatto un ottimo lavoro nella produzione.

Io (AST) vorrei ringraziare ancora una volta Suzanne per il suo amore e la sua pazienza che non ha mai fine, nemmeno dopo 21 libri. Barbara e Marvin sono da sempre fonte di gioia; ora sanno cosa fanno i professori per guadagnarsi da vivere. Aron appartiene alla nuova generazione: quella dei bambini che usano regolarmente i computer ancora prima di iniziare la scuola materna. Nathan non è ancora così avanti, ma quando avrà imparato a camminare l'iPad sarà a un passo.

Infine, io (TA) vorrei approfittare di questa opportunità per ringraziare mia suocera Roberta, che mi ha aiutato a ritagliare un po' di tempo di qualità per lavorare a questo libro. La sua tavola da pranzo a Bassano Del Grappa mi ha offerto la giusta quantità di solitudine, di accoglienza e di vino, per portare a termine questo compito importante.

ANDREW S. TANENBAUM
TODD AUSTIN

Prefazione all'edizione italiana

Due sono le chiavi di lettura di questa nuova edizione *del Tanenbaum*. La prima apre una finestra sull'affascinante mondo dei sistemi di calcolo. Mondo sempre più variegato e difforme. Un mondo che va dal milione di server nei data center di Google ai microcontrollori a 8 bit (che, acquistati in grandi quantità, possono costare meno di 10 centesimi). Un mondo nel quale anche il più accorto viaggiatore è destinato a perdersi, a meno di non poter disporre di una solida e competente guida. E in effetti l'approccio strutturale del libro in questione ci accompagna passo passo nel viaggio che ci porta dal livello logico digitale a quello dei linguaggi orientati al tipo di problema; dal semplice (ma pionieristico) livello concettuale della macchina di Von Neumann agli Intel Core i7, alla piattaforma integrata di controllo Arduino. In questo ruolo, *il Tanenbaum* ricalca la tradizione di chiarezza e completezza delle precedenti edizioni e si riconferma un utilissimo strumento didattico.

Ma c'è un altro aspetto che spesso tende a sfuggire agli utenti informatici. Aspetto che è bene illustrare con le stesse parole dell'autore: "Il messaggio da cogliere è che il modello di calcolo più diffuso in una data epoca dipende molto dalla tecnologia, dall'economia e dalle applicazioni disponibili al momento e può cambiare nel momento in cui cambiano questi fattori". Se Babbage avesse avuto il silicio...

Quindi, e giustamente, questo testo non è soltanto un "libro di testo", ma ci informa anche su aspetti hardware che hanno potenzialità rivoluzionarie. Un esempio su tutti: la trattazione delle memorie flash: prestazioni in salita e costi in discesa.

Cosa aspettarci dalla prossima edizione *del Tanenbaum*?

Prof. Ottavio M. D'Antona
Dipartimento di Informatica
Università degli Studi di Milano

CAPITOLO 1

Introduzione

Un calcolatore digitale è una macchina in grado di svolgere dei compiti per le persone eseguendo le istruzioni che le vengono assegnate. Con il termine **programma** si intende una sequenza d'istruzioni che descrive come portare a termine un dato compito. I circuiti elettronici dei computer possono riconoscere ed eseguire direttamente soltanto un insieme limitato d'istruzioni semplici in cui tutti i programmi devono essere convertiti prima di poter essere eseguiti. Raramente queste istruzioni elementari sono molto più complicate delle seguenti:

- sommare due numeri;
- controllare se un numero vale zero;
- copiare una porzione di dati da una parte all'altra della memoria.

L'insieme di queste istruzioni primitive forma un linguaggio, chiamato **linguaggio macchina**, attraverso il quale è possibile comunicare con il computer. Chi progetta un calcolatore deve decidere quali istruzioni costituiranno il suo linguaggio macchina. Per ridurre la complessità e il costo dei dispositivi elettronici generalmente si cerca di progettare le più semplici istruzioni primitive che siano compatibili con il tipo di utilizzo e i requisiti prestazionali in base ai quali si progetta il computer. Dato che quasi tutti i linguaggi macchina sono semplici ed elementari, risulta difficile e noioso utilizzarli.

Nel corso del tempo, questa semplice osservazione ha portato a strutturare i computer come una serie di livelli di astrazione, costruiti l'uno sull'altro. In questo modo la complessità è gestibile più agevolmente e i computer possono essere progettati in modo sistematico e organizzato. Chiamiamo **approccio strutturale** questo modo di concepire l'architettura dei computer: il concetto sarà descritto in modo più preciso nel paragrafo successivo. Successivamente prenderemo in considerazione alcuni sviluppi storici, lo stato dell'arte e alcuni importanti esempi.

1.1 Approccio strutturale

Come abbiamo appena accennato esiste una gran differenza tra ciò che è adatto agli utenti e ciò che lo è per i computer. Gli utenti vogliono fare *X*, ma i computer possono fare soltanto *Y*: l'obiettivo del libro è spiegare in che modo sia possibile risolvere questo problema.

1.1.1 Linguaggi, livelli e macchine virtuali

Il problema può essere affrontato in due modi distinti: entrambi prevedono la definizione di un nuovo insieme d'istruzioni che sia più comodo da utilizzare rispetto alle istruzioni macchina predefinite. Considerate globalmente, anche queste nuove istruzioni formano un linguaggio, che chiamiamo L1, allo stesso modo in cui le istruzioni macchina formavano a loro volta un linguaggio, che chiamiamo L0. I due approcci differiscono nel modo in cui i programmi scritti in L1 possono essere eseguiti da un computer, in grado di eseguire soltanto quelli scritti nel proprio linguaggio, L0.

Un metodo per eseguire un programma scritto in L1 consiste nel sostituire, in una fase iniziale, ogni sua istruzione con un'equivalente sequenza di istruzioni in L0. Il programma che ne risulta è costituito interamente da istruzioni di L0 e può essere eseguito dal computer al posto del programma L1 originale. Questa tecnica è chiamata **traduzione**.

L'altra tecnica consiste invece nello scrivere un programma in L0 che accetta come dati d'ingresso programmi in L1; tale programma li esegue esaminando un'istruzione alla volta e sostituendola direttamente con l'equivalente sequenza di istruzioni L0. Questa tecnica, che non richiede la generazione preventiva di un nuovo programma L0, è chiamata **interpretazione** e il programma che la esegue è detto **interprete**.

Traduzione e interpretazione sono simili e in entrambi i casi il computer può trattare istruzioni L1 eseguendo le equivalenti sequenze di istruzioni L0. La differenza è che, nel caso della traduzione, il programma L1 viene, all'inizio, convertito interamente in un programma L0. Il programma L1 può quindi essere gettato via, mentre il programma L0 viene caricato nella memoria del computer per essere eseguito. Durante l'esecuzione il computer ha il controllo del nuovo programma L0.

Nell'interpretazione invece ciascuna istruzione L1 viene esaminata, decodificata ed eseguita direttamente, cioè senza alcuna traduzione. In questo caso il computer ha il controllo del programma interprete, mentre il programma L1 è visto come un insieme di dati. Entrambi i metodi, oltre a una sempre più frequente combinazione dei due, sono ampiamente utilizzati.

Invece di ragionare in termini di traduzione e interpretazione, spesso è più semplice immaginare l'esistenza di un ipotetico computer o **macchina virtuale**, il cui linguaggio macchina sia L1. Chiamiamo M1 questa macchina virtuale M1 (e M0 quella corrispondente al linguaggio L0). Se una tale macchina potesse essere costruita in modo sufficientemente economico, non ci sarebbe affatto bisogno del linguaggio L0 né di una macchina capace di eseguire programmi L0. Gli utenti potrebbero semplicemente scrivere i loro programmi in L1, che verrebbero eseguiti direttamente dal computer. Anche se una macchina virtuale con linguaggio macchina L1 è troppo costosa o complicata da essere costruita realmente, si possono tuttavia scrivere programmi dedicati. Questi programmi possono essere interpretati oppure tradotti da un programma scritto in L0, eseguibile direttamente sui computer esistenti. In altre parole, si possono scrivere programmi per macchine virtuali come se queste esistessero veramente.

Per rendere la traduzione o l'interpretazione utilizzabili in pratica, i linguaggi L0 e L1 non devono essere "troppo" diversi fra loro. Per la maggior parte delle applicazioni questo vincolo fa sì che L1, pur essendo migliore di L0, sia spesso ancora lontano dal linguaggio ideale. Questo risultato può sembrare forse scoraggiante rispetto alle intenzioni iniziali, secondo cui la creazione di L1 avrebbe dovuto sollevare il programmatore dall'onere di esprimere algoritmi in un linguaggio più adatto alle macchine che agli utenti. In ogni caso la situazione non è senza speranza.

L'approccio più ovvio consiste nell'inventare un nuovo insieme d'istruzioni che sia, rispetto a L1, maggiormente orientato agli utenti piuttosto che alle macchine. Anche questo terzo insieme forma a sua volta un linguaggio, che chiamiamo L2 (e la corrispondente macchina virtuale sarà M2). Si possono scrivere direttamente programmi in L2 come se esistesse realmente una macchina virtuale dotata di tale linguaggio macchina. Questi programmi possono essere tradotti in L1 oppure eseguiti da un interprete scritto in L1.

La definizione di una serie di linguaggi, ciascuno dei quali più pratico da utilizzare rispetto al precedente, può continuare indefinitamente finché non se ne ottenga uno sufficientemente adeguato. Ciascun linguaggio utilizza il precedente come base. Quindi un computer che usa questa tecnica può essere immaginato come una serie di **strati** o **livelli** disposti l'uno sopra l'altro, come mostrato nella Figura 1.1. Il livello, o linguaggio, che si trova più in basso è il più semplice e quello più in alto è il più sofisticato.

Tra linguaggi e macchine virtuali sussiste un'importante relazione. Una macchina ha un proprio linguaggio macchina, costituito da tutte le istruzioni che è in grado di eseguire. Di fatto, una macchina definisce un linguaggio. Nello stesso tempo, un linguaggio definisce una macchina: quella che è in grado di eseguire tutti i programmi scritti in quel linguaggio. Anche se una macchina definita da un linguaggio arbitrario potrebbe ovviamente essere enormemente complicata e costosa da realizzare, è tuttavia possibile immaginarla. Una macchina dotata di C, C++ o Java come linguaggio macchina risulterebbe sicuramente complessa ma, grazie alla tecnologia odierna, sarebbe in ogni caso realizzabile. Vi è tuttavia una buona ragione per non percorrere questa via: tale computer risulterebbe più costoso rispetto all'utilizzo di altre tecniche. Il semplice fatto che sia realizzabile non è sufficiente, dato che un progetto, per essere fattibile, deve anche essere economicamente conveniente.

In un certo senso, un computer con *n* livelli può essere interpretato come *n* distinte macchine virtuali. Osserviamo che, come succede per molti termini del linguaggio informatico, l'espressione "macchina virtuale" assume anche altri significati, uno dei quali sarà trattato più avanti in questo volume. Solo i programmi scritti nel linguaggio L0 possono essere eseguiti direttamente dalla componentistica digitale senza far ricorso alla traduzione o all'interpretazione, mentre i programmi scritti in L1, L2, ..., L*n* devono essere interpretati da un interprete eseguito a un livello inferiore oppure devono essere tradotti in un altro linguaggio, corrispondente a un livello più basso.

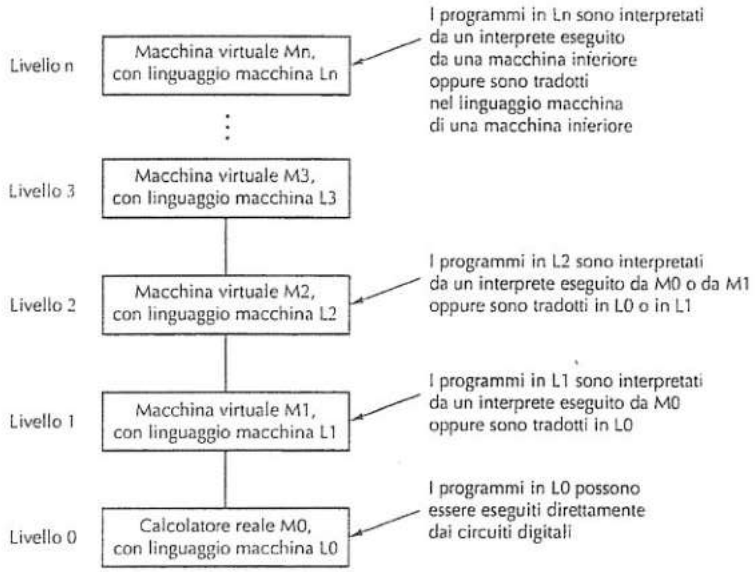


Figura 1.1 Macchina multilivello.

Se si scrivono programmi utilizzando la macchina di un dato livello non ci si deve preoccupare degli interpreti o dei traduttori sottostanti, dato che la struttura della macchina assicura che questi programmi saranno, in un modo o nell'altro, eseguiti. Non è di alcun interesse pratico il fatto che siano portati a termine da un interprete, eseguito a sua volta da un altro interprete, oppure che siano eseguiti direttamente dai circuiti digitali. In entrambi i casi si ottiene lo stesso risultato: i programmi vengono eseguiti.

La maggior parte dei programmatori che usa una macchina di un dato livello è interessata solamente al livello superiore, quello che meno assomiglia al linguaggio macchina del livello più basso. Tuttavia chi è interessato a comprendere il reale funzionamento di un computer li deve studiare tutti. Anche chi progetta nuovi computer o nuovi livelli deve avere la stessa familiarità con tutti i livelli e non solo con quello superiore. Il tema centrale di questo libro è costituito da concetti e tecniche inerenti la costruzione di macchine come gerarchie di livelli, nonché dai dettagli di quest'ultimi.

1.1.2 Attuali macchine multilivello

La maggior parte dei moderni computer consiste di due o più livelli. Come mostrato nella Figura 1.2 esistono anche macchine costituite da sei livelli. Il livello 0, che si trova alla base, rappresenta il vero e proprio hardware della macchina, i cui circuiti eseguono i programmi scritti nel linguaggio macchina del livello 1. Per completezza occorre dire che esiste un ulteriore livello al di sotto di quello che noi consideriamo livello 0. Questo livello è chiamato **livello dei dispositivi** e non è mostrato nella Figura 1.2, dato che

ricade nel campo dell'ingegneria elettronica (al di fuori quindi dell'ambito di questo libro). A questo livello i progettisti hanno a che fare con i singoli transistor, gli elementi primitivi nella progettazione dei computer. Se si vuole studiare il funzionamento interno dei transistor si deve entrare nel campo della fisica dello stato solido.

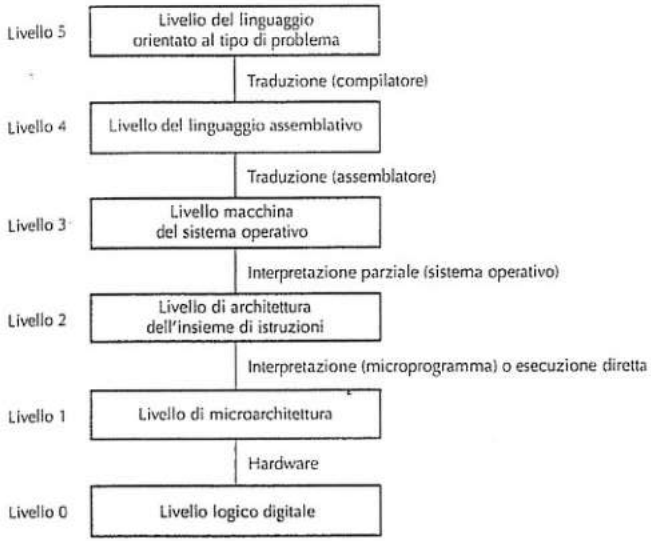


Figura 1.2 Computer a sei livelli. Sotto ciascun livello è indicato il metodo di supporto (oltre al nome del programma corrispondente).

Gli oggetti che analizzeremo nel livello più basso della nostra trattazione, il **livello logico digitale**, sono le **porte** (*gate*). Queste, pur essendo costruite utilizzando componenti analogici come i transistor, possono essere correttamente modellate come dispositivi digitali. Ciascuna porta, costituita al più da una manciata di transistor, è dotata di uno o più input digitali (segnali corrispondenti ai valori 0 o 1) e calcola in output una semplice funzione dei valori d'ingresso, per esempio AND od OR. È possibile combinare un piccolo numero di porte per formare una memoria a 1 bit, che può memorizzare i valori 0 e 1. Combinando le memorie a 1 bit in gruppi, per esempio di 16, 32 o 64, è possibile formare quelli che vengono chiamati registri. Ciascun **registro** può contenere un numero il cui valore può variare fino a un certo limite. È inoltre possibile combinare le porte per formare lo stesso motore computazionale principale. Nel Capitolo 3 esamineremo in dettaglio le porte e il livello logico digitale.

Subito dopo troviamo il **livello di micro-architettura**. Qui vi è una memoria locale, formata da un gruppo di registri (in genere da 8 a 32) e un circuito chiamato **ALU** (*Arithmetic Logic Unit*, unità aritmetico-logica), capace di effettuare semplici operazio-

ni aritmetiche. I registri sono connessi alla ALU per formare un **percorso dati** lungo il quale questi ultimi si spostano. L'operazione base del percorso dati consiste nel selezionare uno o due registri, permettere alla ALU di operare su di loro (per esempio sommandoli) e memorizzare infine il risultato in uno dei registri. In alcune macchine le operazioni del percorso dati sono controllate da un programma chiamato **microprogramma**, mentre su altre il percorso dati è controllato direttamente dall'hardware. Nelle prime edizioni di questo libro abbiamo chiamato tale livello il "livello di microprogrammazione", dato che in passato era quasi sempre rappresentato da un interprete software. Attualmente invece il percorso dati è spesso controllato in modo diretto dall'hardware (interamente o parzialmente). Per rispecchiare questo cambiamento abbiamo modificato il nome in "livello di micro-architettura".

Nelle macchine in cui il controllo del percorso dati avviene via software il microprogramma è un interprete per le istruzioni del livello 2, che utilizza il percorso dati per prelevare, esaminare ed eseguire un'istruzione alla volta. Nel caso per esempio di un'istruzione ADD, questa viene prelevata, i suoi operandi localizzati e portati nei registri, la somma calcolata dalla ALU e infine il risultato viene ricopiato nel registro appropriato. Passi analoghi si verificano anche in una macchina con controllo via hardware del percorso dati, senza però avere un programma memorizzato e dedicato all'interpretazione delle istruzioni del livello 2.

Il livello 2 è costituito da quello che chiamiamo il **livello ISA** (*Instruction Set Architecture Level*, livello di architettura dell'insieme d'istruzioni). Ogni produttore di computer pubblica un manuale per ciascuno dei propri modelli, un "Manuale di riferimento del linguaggio macchina" o "Principi di funzionamento del computer Western Wombat Model 100X", o qualcosa di simile. Questi manuali trattano il livello ISA, ma non quelli sottostanti; quando descrivono l'insieme d'istruzioni della macchina, presentano in realtà le istruzioni eseguite in modo interpretato dal microprogramma o dai circuiti elettronici. Se un produttore di computer dotasse una delle proprie macchine di due diversi interpreti, dovrebbe fornire due distinti manuali del "linguaggio macchina" dato che i due interpreti definirebbero due livelli ISA differenti.

Il livello successivo è generalmente un livello ibrido. La maggior parte delle istruzioni nel suo linguaggio fa parte anche del livello ISA (non vi è ragione per cui un'istruzione che appaia a un livello non possa essere presente anche in altri). Inoltre, vi è un insieme di nuove istruzioni, una diversa organizzazione della memoria e la capacità di eseguire programmi in modo concorrente, oltre a varie altre funzionalità. Fra i diversi modi di progettare il livello 3 vi è una maggior varietà rispetto a quanto non si riscontri nei livelli 1 e 2.

I nuovi servizi aggiunti nel livello 3 sono realizzati da un interprete eseguito al livello 2, storicamente chiamato sistema operativo. Le istruzioni del livello 3, identiche a quelle del livello 2, sono eseguite direttamente dal microprogramma (o dai circuiti elettronici) e non dal sistema operativo. In altre parole, alcune delle istruzioni del livello 3 sono interpretate dal sistema operativo, mentre altre sono interpretate in modo diretto dal microprogramma; per questo motivo tale livello viene detto "ibrido". Nel corso del libro ci si riferirà a questo livello come al **livello macchina del sistema operativo**.

Tra i livelli 3 e 4 vi è una spaccatura fondamentale. I tre livelli inferiori non sono progettati per essere utilizzati dal programmatore medio, ma concepiti principalmente per eseguire interpreti e traduttori necessari come supporto per i livelli più alti. Questi interpreti e traduttori sono scritti da professionisti chiamati **programmatori di sistema**, specializzati nella progettazione e nell'implementazione di nuove macchine virtuali. Il livello 4 e quelli superiori sono invece pensati per i programmatori che devono risolvere problemi applicativi.

Un altro cambiamento che si riscontra a partire dal livello 4 è il modo in cui vengono supportati i livelli superiori. Mentre i livelli 2 e 3 sono sempre interpretati, i livelli 4, 5 e quelli superiori sono generalmente supportati mediante traduzione.

Un'ulteriore differenza tra i primi tre livelli e gli altri riguarda la natura del linguaggio corrispondente. I linguaggi macchina dei livelli 1, 2 e 3 sono numerici; i loro programmi consistono quindi in lunghe serie di numeri, certamente più adatte alle macchine che agli esseri umani. A partire dal livello 4 i linguaggi contengono invece parole e abbreviazioni umanamente comprensibili.

Il livello 4, quello del linguaggio assemblativo, è in realtà una forma simbolica di uno dei linguaggi sottostanti. Questo livello fornisce ai programmatori un modo per scrivere programmi per i livelli 1, 2 e 3 in una forma meno difficoltosa rispetto a quella dei linguaggi delle rispettive macchine virtuali. I programmi in linguaggio assemblativo sono inizialmente tradotti nei linguaggi dei livelli 1, 2 o 3 e successivamente interpretati dall'appropriata macchina virtuale o reale. Il programma che esegue la traduzione è chiamato **assemblatore**.

Il livello 5 consiste generalmente di linguaggi, spesso chiamati **linguaggi ad alto livello**, definiti per essere utilizzati dai programmatori di applicazioni. Ne esistono letteralmente a centinaia; alcuni fra i più conosciuti sono C, C++, Java, Perl, Python e PHP. I programmi scritti in questi linguaggi sono generalmente tradotti al livello 3 o al livello 4 da un traduttore detto **compilatore**; in casi meno frequenti è anche possibile che essi siano interpretati. I programmi in Java, per esempio, di solito sono tradotti inizialmente in un linguaggio di tipo ISA chiamato *Java byte code*, che viene successivamente interpretato.

In alcuni casi, il livello 5 consiste di un interprete specifico per un determinato campo di applicazione, per esempio la matematica simbolica. Fornisce dati e operazioni per risolvere problemi in quell'area in termini facilmente comprensibili agli esperti di quel campo.

Riassumendo, il concetto chiave da ricordare è che i computer sono progettati come una serie di livelli, ciascuno costruito su quelli che lo precedono. Ciascun livello rappresenta una diversa astrazione, caratterizzata dalla presenza di oggetti e operazioni diversi. Progettando e analizzando i computer in questo modo è possibile tralasciare temporaneamente i dettagli irrilevanti, riducendo di conseguenza un soggetto complesso in qualcosa di più facile comprensione.

L'insieme dei tipi di dati, delle operazioni e delle funzionalità di ciascun livello è chiamato **architettura**. Questa tratta gli aspetti visibili agli utenti di quel determinato livello. Fanno quindi parte dell'architettura le caratteristiche che un programmatore può vedere, come la quantità di memoria disponibile, mentre non ne fanno parte gli aspetti

implementativi, come la tecnologia con cui è realizzata la memoria. Lo studio di come progettare le parti di un computer visibili ai programmatori è chiamato **architettura degli elaboratori**. Nella pratica comune architettura dei calcolatori e organizzazione dei computer significano essenzialmente la stessa cosa.

1.1.3 Evoluzione delle macchine multilivello

Per fornire una panoramica delle macchine multilivello, ne esamineremo brevemente gli sviluppi storici, mostrando come il numero e la natura dei livelli siano evoluti nel corso degli anni. I programmi scritti in un vero linguaggio macchina (livello 1) possono essere eseguiti direttamente dai circuiti del computer (livello 0), senza far ricorso ad alcuna traduzione o interpretazione. Questi circuiti formano, insieme alla memoria e ai dispositivi di input/output, l'**hardware** del computer. L'hardware consiste di oggetti tangibili (circuiti integrati, schede con circuiti stampati, cavi, trasformatori, memorie e stampanti) piuttosto che di idee astratte, algoritmi o istruzioni.

Al contrario, il **software** consiste di **algoritmi** (istruzioni dettagliate su come realizzare un determinato compito) e delle loro rappresentazioni per i computer, vale a dire i programmi. I programmi possono essere memorizzati su hard disk, floppy disk, CD-ROM o altri supporti, ma l'essenza del software è l'insieme delle istruzioni che costituiscono il programma, non il supporto fisico su cui sono registrate.

Nei primissimi computer, il confine tra hardware e software era chiaro e cristallino. Nel corso del tempo, man mano che i computer si sono evoluti, quel confine si è però sfocato in modo considerevole per via dell'aggiunta, della rimozione e dell'unione di livelli, tanto che oggi è spesso difficile tenerli separati (Vahid, 2003). E proprio per questo, un tema centrale di questo libro è:

hardware e software sono logicamente equivalenti.

Ogni operazione eseguita via software può anche essere costruita direttamente in hardware, preferibilmente dopo averne compreso in modo sufficientemente approfondito il funzionamento. Come ha affermato Karen Panetta Lentz: "L'hardware non è che software pietrificato". Ovviamente è anche vero il contrario: ogni istruzione eseguita dall'hardware può essere simulata via software. La scelta di collocare certe funzioni nell'hardware e altre nel software è basata su fattori quali il costo, la velocità, l'affidabilità e la frequenza di modifiche che ci si aspetta. Esistono poche regole ferree nello stabilire se X deve andare nell'hardware e Y deve essere programmato esplicitamente. Queste scelte cambiano a seconda del mercato delle tecnologie, della domanda e del modo di usare il computer.

Invenzione della microprogrammazione

I primi computer digitali, risalenti agli anni '40, avevano solamente due livelli: il livello ISA, in cui erano realizzati tutti i programmi, e il livello logico digitale, che li eseguiva. I circuiti del livello logico digitale erano complessi, difficili da comprendere e da realizzare e, oltretutto, inaffidabili.

Nel 1951 Maurice Wilkes, un ricercatore della Cambridge University, propose di progettare un computer a tre livelli, al fine di semplificare drasticamente l'hardware e ridurre così il numero di (poco affidabili) valvole necessarie (Wilkes, 1951). Questa macchina avrebbe dovuto avere un interprete predefinito e non modificabile (il microprogramma), la cui funzione sarebbe stata quella di eseguire programmi al livello ISA mediante interpretazione. Sarebbe stato necessario un minor numero di circuiti elettronici, dato che l'hardware avrebbe dovuto eseguire microprogrammi costituiti da un insieme limitato di istruzioni invece di programmi a livello ISA, basati su un insieme di istruzioni molto più ampio. Dato che i circuiti di allora erano costituiti da valvole[1], una tale semplificazione prometteva di ridurne il numero migliorando di conseguenza l'affidabilità (cioè diminuendo il numero giornaliero di guasti).

Durante gli anni '50 vennero costruite poche di queste macchine a tre livelli, mentre lo furono di più negli anni '60. A partire dal 1970 l'idea di avere un livello ISA interpretato da un microprogramma divenne dominante e tutte le principali macchine dell'epoca la implementarono.

Invenzione del sistema operativo

In questi primi anni la maggior parte dei computer era di tipo "fai da te", il che significava che il programmatore doveva far funzionare la macchina personalmente. Vicino a ogni macchina vi era un foglio su cui ogni programmatore prenotava la fascia oraria durante la quale desiderava far eseguire il proprio programma, per esempio dalle 3 alle 5 del mattino (molti programmatori preferivano lavorare quando vi era silenzio in sala macchine). Quando arrivava il momento, il programmatore si dirigeva verso la sala macchine con un mazzo di schede perforate da 80 colonne (uno dei primi supporti di input) in una mano e una matita appuntita nell'altra. Una volta arrivato nella stanza del computer spingeva gentilmente il precedente programmatore fuori dalla porta e prendeva il controllo del computer.

Se il programmatore desiderava eseguire un programma FORTRAN si verificavano i seguenti passi.

1. Guardava nell'armadietto dove erano tenuti tutti i programmi, estraeva il grande mazzo di schede verdi etichettato "compilatore FORTRAN", lo inseriva nel lettore delle schede perforate e premeva il pulsante START.
2. Inseriva il suo programma nel lettore delle schede e premeva il pulsante CONTINUE. Il programma veniva letto.
3. Quando il computer si arrestava, il programmatore faceva leggere il suo programma FORTRAN una seconda volta. Anche se alcuni compilatori richiedevano una sola passata sull'input, molti altri ne richiedevano due o più. A ogni passata doveva essere letto un gran numero di schede.
4. Infine la traduzione si avvicinava al termine. Spesso in questa fase il programmatore diventava nervoso, poiché, nel caso in cui il compilatore avesse trovato un errore nel

[1] I cosiddetti tubi a vuoto (*N.d.Reu*).

Anno	Nome	Realizzato da	Commento
1834	Analytical Engine	Babbage	Primo tentativo di costruire un computer digitale
1936	Z1	Zuse	Prima macchina calcolatrice funzionante basata su relé
1943	COLOSSUS	Governo britannico	Primo computer elettronico
1944	Mark I	Aiken	Primo computer americano di uso generale
1946	ENIAC I	Eckert/Mauchley	Inizio della moderna storia dei computer
1949	EDSAC	Wilkes	Primo computer a programma memorizzato
1951	Whirlwind I	M.I.T.	Primo computer con elaborazione in tempo reale
1952	IAS	Von Neumann	Progetto su cui si basa la maggior parte delle macchine odierne
1960	PDP-1	DEC	Primo minicomputer (venduto in 50 unità)
1961	1401	IBM	Piccola macchina aziendale di grande diffusione
1962	7094	IBM	Dominò il campo della computazione scientifica nei primi anni '60
1963	B5000	Burroughs	Prima macchina progettata per un linguaggio ad alto livello
1964	360	IBM	Prima linea di prodotti progettata come una famiglia
1964	6600	CDC	Primo supercomputer per calcoli scientifici
1965	PDP-8	DEC	Primo minicomputer con un mercato di massa (venduto in 50.000 unità)
1970	PDP-11	DEC	Dominò il mondo dei minicomputer negli anni '70
1974	8080	Intel	Primo computer a 8 bit di uso generale su un chip
1974	CRAY-1	Cray	Primo supercomputer vettoriale
1978	VAX	DEC	Primo superminicomputer a 32 bit
1981	IBM PC	IBM	Inizio dell'era moderna dei personal computer
1981	Osborne-1	Osborne	Primo computer portatile
1983	Lisa	Apple	Primo personal computer dotato di una GUI
1985	386	Intel	Progenitore a 32 bit della linea Pentium
1985	MIPS	MIPS	Prima macchina RISC commerciale
1985	XC2064	Xilinx	Primo FPGA
1987	SPARC	Sun	Prima workstation RISC basata su SPARC
1989	GridPad	Grid Systems	Primo tablet computer commerciale
1990	RS6000	IBM	Prima macchina superscalare
1992	Alpha	DEC	Primo personal computer a 64 bit
1992	Simon	IBM	Primo smartphone
1993	Newton	Apple	Primo computer palmare (PDA)
2001	POWER4	IBM	Primo multiprocessore dual-core

Figura 1.4 Alcune pietre miliari nello sviluppo del computer digitale moderno.

Per i successivi 150 anni non successe niente di significativo finché un professore di matematica della Cambridge University, Charles Babbage (1792-1871), inventore tra l'altro del tachimetro, progettò e costruì una macchina chiamata *difference engine* ("macchina differenziale"). Questo dispositivo meccanico, capace come quello di Pascal soltanto di sommare e sottrarre, fu progettato per calcolare tabelle di numeri utili per la navigazione. L'intera costruzione della macchina fu pensata per eseguire un solo algoritmo, il metodo matematico delle differenze finite. L'aspetto più interessante della *difference engine* fu però la sua forma di output: i risultati venivano incisi mediante una punta d'acciaio su una lastra di rame, anticipando quindi i futuri supporti a singola scrittura quali le schede perforate e i CD-ROM.

Sebbene la *difference engine* funzionasse in modo abbastanza soddisfacente, Babbage si stancò presto di una macchina capace di eseguire un solo algoritmo. Egli cominciò a profondere una sempre maggior quantità di tempo e di beni familiari (per non parlare delle 17.000 sterline di fondi governativi) nella progettazione e costruzione di una macchina che ne rappresentasse l'evoluzione: la *analytical engine* ("macchina analitica"). La *analytical engine* era composta da quattro componenti: il magazzino (la memoria), il mulino (l'unità computazionale), il dispositivo di input (le schede perforate) e il dispositivo di output (output stampato e perforato). Il magazzino era composto da 1000 parole da 50 cifre, utilizzate per memorizzare dati e risultati. Il mulino poteva prelevare gli operandi dal magazzino per sommarli, sottrarli, moltiplicarli o dividerli e infine memorizzarne nuovamente il risultato nel magazzino. Anche questa macchina, al pari della *difference engine*, era interamente meccanica.

Il grande avanzamento rappresentato dall'*analytical engine* consisteva nel fatto di essere di uso generale. Leggeva le istruzioni dalle schede perforate e le eseguiva. Alcune istruzioni comandavano la macchina in modo che prelevasse due numeri dal magazzino, li portasse nel mulino, eseguisse su di loro una data operazione (per esempio la somma) e ne rispedisse il risultato al magazzino. Altre istruzioni erano in grado di analizzare un numero e ramificare l'esecuzione del programma in base al suo segno. Perforando un diverso programma sulle schede di input era quindi possibile far eseguire all'*analytical engine* diverse computazioni, cosa che era invece impossibile con la *difference engine*.

Dato che la *analytical engine* era programmabile mediante un semplice linguaggio assemblativo, era necessario produrre il software. A tal fine, Babbage assunse Ada Augusta Lovelace, la giovane figlia del famoso poeta inglese, Lord Byron. Ada Lovelace fu quindi la prima programmatrice della storia e in suo onore fu chiamato il linguaggio di programmazione Ada.

Sfortunatamente, come avviene nel caso di molti progettisti moderni, Babbage non riuscì mai a ottenere un hardware completamente privo di errori. Il problema nasceva dalla necessità di avere migliaia e migliaia di rotelle e ingranaggi per ottenere un grado di precisione che la tecnologia del diciannovesimo secolo non era ancora in grado di fornire. Ciononostante le sue idee erano in largo anticipo sui tempi, al punto che la maggior parte dei computer moderni presenta oggi una struttura molto simile all'*analytical engine*. Per questo motivo è giusto affermare che Babbage fu il padre (o meglio, il nonno) degli odierni calcolatori.

Il successivo e significativo avanzamento avvenne verso la fine degli anni '30, quando uno studente tedesco di ingegneria, Konrad Zuse, costruì una serie di macchine calcolatrici automatiche utilizzando dei relé elettromagnetici. Una volta scoppiata la guerra, non riuscì a ottenere finanziamenti dal governo, poiché i burocrati si aspettavano di vincere il conflitto in così breve tempo da ritenere che la nuova macchina non sarebbe stata pronta prima della fine della guerra. Zuse non conosceva il lavoro di Babbage e le sue macchine furono distrutte nel 1944 durante un bombardamento degli Alleati su Berlino. Anche se per questi motivi il suo lavoro non poté influenzare lo sviluppo delle macchine successive, Zuse va comunque considerato come uno dei pionieri in questo campo.

Non molto tempo dopo, negli Stati Uniti altri due ricercatori, John Atanasoff dell'Iowa State College e George Stibbitz dei Bell Labs, si dedicarono alla progettazione di calcolatori. La macchina di Atanasoff era incredibilmente avanzata per l'epoca: era basata sull'aritmetica binaria e utilizzava dei condensatori per la memoria. Affinché la loro carica non si disperdesse, la memoria veniva periodicamente aggiornata, secondo un processo che Atanasoff chiamò *jogging the memory*. Le moderne memorie dinamiche (DRAM) funzionano secondo lo stesso principio. Sfortunatamente la macchina non divenne mai realmente operativa; in un certo senso Atanasoff fu come Babbage, cioè un precursore che non ebbe successo a causa dell'inadeguatezza tecnologica del proprio tempo.

Al contrario il computer di Stibbitz, sebbene meno evoluto di quello di Atanasoff, riuscì a funzionare e il suo inventore ne diede una dimostrazione pubblica nel 1940 durante una conferenza al Dartmouth College. Tra il pubblico c'era John Mauchley, a quel tempo ancora sconosciuto, ma di cui in seguito si sarebbe sentito parlare nel mondo dei computer. Mentre Zuse, Stibbitz e Atanasoff stavano progettando calcolatori automatici, un giovane di nome Howard Aiken si sforzava di svolgere a mano tediosi calcoli numerici, per la sua tesi di dottorato, alla Harvard University. Dopo la laurea, Aiken comprese l'importanza di poter svolgere calcoli per mezzo di una macchina. Frequentando una biblioteca, scoprì il lavoro di Babbage e decise di costruire per mezzo di relé il computer di uso generale che Babbage a suo tempo non riuscì a realizzare mediante ingranaggi.

Nel 1944, alla Harvard University, Aiken completò la sua prima macchina, il Mark I. Era composta da 72 parole di 23 cifre decimali, aveva un tempo d'istruzione di 6 secondi e usava un nastro di carta perforato per l'input e l'output. Dal momento in cui Aiken realizzò il suo successore, il Mark II, i computer basati su relé divennero obsoleti; era iniziata infatti l'era dell'elettronica.

1.2.2 Prima generazione – Valvole (1945-1955)

Lo stimolo per lo sviluppo dei computer elettronici venne dalla Seconda Guerra Mondiale. Nella prima parte della guerra i sottomarini tedeschi stavano decimando la flotta britannica. Da Berlino gli ammiragli tedeschi spedivano i comandi via radio ai sottomarini; questi ordini potevano quindi essere intercettati dai britannici. Il problema era che i messaggi venivano codificati per mezzo di un dispositivo chiamato **ENIGMA**; il precursore di questa macchina era stato progettato da Thomas Jefferson che, oltre a essere stato il terzo presidente degli Stati Uniti d'America, fu anche un inventore dilettante.

All'inizio della guerra lo spionaggio britannico riuscì a procurarsi una macchina ENIGMA grazie all'aiuto dei servizi segreti polacchi che erano riusciti a rubarla ai tedeschi. Tuttavia, per poter decifrare un messaggio codificato occorreva svolgere un'enorme quantità di calcoli, ed era necessario poterlo fare molto velocemente, non appena il comando veniva intercettato, affinché ciò potesse essere di una qualche utilità. Per decodificare questi messaggi il governo britannico creò un laboratorio segretissimo per la costruzione di un computer chiamato COLOSSUS. Il famoso matematico inglese Alan Turing diede il suo aiuto alla progettazione di questa macchina. COLOSSUS divenne operativo nel 1943 ma, dato che il governo britannico tenne sotto segreto militare per 30 anni praticamente ogni aspetto del progetto, lo sviluppo di COLOSSUS non ebbe alcun seguito. Vale la pena ricordarlo semplicemente perché fu il primo elaboratore digitale al mondo.

Oltre a distruggere le macchine di Zuse e incentivare la costruzione di COLOSSUS, la guerra condizionò il campo dei computer anche negli Stati Uniti. L'esercito aveva bisogno di tabelle per impostare la mira dell'artiglieria pesante e a tal scopo vennero assunte centinaia di donne (in quanto ritenute più precise degli uomini) affinché producessero in serie queste tabelle per mezzo di calcolatori manuali. Tuttavia la procedura richiedeva molto tempo, e vi era sempre il rischio dell'errore umano.

John Mauchley, che aveva studiato sia i lavori di Atanasoff sia quelli di Stibbitz, venne a sapere che l'esercito era interessato alla costruzione di calcolatori meccanici. Come molti altri ricercatori dopo di lui, avanzò all'esercito una richiesta di sovvenzione per finanziare la costruzione di un calcolatore elettronico. Nel 1943 la richiesta fu accettata e Mauchley cominciò a costruire, insieme al suo studente J. Presper Eckert, un computer elettronico che chiamarono **ENIAC** (*Electronic Numerical Integrator And Computer*). L'elaboratore ENIAC era costituito da 18.000 valvole termoioniche e 1500 relé, pesava 30 tonnellate e consumava 140 KW di energia. Dal punto di vista dell'architettura, la macchina era dotata di 20 registri, ciascuno dei quali in grado di memorizzare un numero decimale a 10 cifre. Un registro decimale è una memoria molto piccola che può mantenere un valore fino a un massimo numero di cifre digitali, analogamente al contachilometri che tiene traccia di quanta strada una macchina ha percorso nella sua vita. ENIAC veniva programmato regolando 6000 interruttori multi-posizione e connettendo una moltitudine di prese con una vera e propria foresta di cavi.

La macchina fu terminata solo nel 1946, quando ormai non poteva più essere utile per lo scopo originario. Tuttavia, dato che la guerra era finita, fu concesso a Mauchley e Eckert di organizzare una scuola estiva per presentare il loro lavoro ai colleghi. Questo evento segnò l'inizio di un'esplosione di interesse nella costruzione di grandi computer digitali.

Dopo quella scuola estiva molti altri ricercatori intrapresero la costruzione di computer elettronici. Il primo a essere operativo fu l'elaboratore EDSAC (1949), costruito presso la Cambridge University da Maurice Wilkes. Fra gli altri progetti, vi furono JOHNIAC alla Rand Corporation, ILLIAC alla Illinois University, MANIAC presso il Los Alamos Laboratory e WEIZAC al Weizmann Institute in Israele.

Eckert e Mauchley cominciarono presto a lavorare su un successore della loro macchina, chiamato **EDVAC** (*Electronic Discrete Variable Automatic Computer*). Quel

progetto morì però nel momento in cui decisero di lasciare la Pennsylvania University per formare a Philadelphia (la Silicon Valley non era ancora stata inventata) una propria società, la Eckert-Mauchley Computer Corporation. Dopo una serie di fusioni questa compagnia è diventata la moderna Unisys Corporation.

Facendo una piccola digressione in ambito legale è opportuno ricordare che Eckert e Mauchley registrarono un brevetto in cui sostenevano di essere gli inventori del computer digitale. A posteriori si può affermare che non sarebbe stato male possedere un tale brevetto. Dopo anni di controversie legali i tribunali decisero che il brevetto Eckert-Mauchley non era valido e che l'inventore del computer digitale, pur non avendolo mai brevettato, era stato John Atanasoff, che aveva effettivamente reso l'invenzione di dominio pubblico.

Mentre Eckert e Mauchley erano impegnati a lavorare su EDVAC, un membro del progetto ENIAC, John von Neumann, andò all'Institute of Advanced Studies di Princeton per costruire la propria versione dell'EDVAC, la macchina IAS. Von Neumann era un genio del livello di Leonardo da Vinci. Parlava molte lingue, era un esperto di fisica e matematica e aveva la capacità di ricordare perfettamente ogni cosa avesse sentito, visto o letto; era inoltre in grado di citare a memoria e alla lettera libri che aveva letto anni prima. Nel momento in cui cominciò a interessarsi ai computer era già il più importante matematico al mondo.

Una delle cose che gli apparvero subito evidenti fu che la programmazione dei computer mediante un gran numero di interruttori e cavi era lenta, tediosa e poco flessibile. Egli comprese che anche i programmi, al pari dei dati, potevano essere rappresentati in forma numerica all'interno della memoria del computer. Inoltre si rese conto che la contorta aritmetica decimale seriale usata nell'elaboratore ENIAC, in cui ogni cifra era rappresentata da 10 valvole (1 accesa e 9 spente), poteva essere sostituita da un'aritmetica binaria parallela, in modo simile a quanto aveva realizzato Atanasoff anni prima.

Il primo progetto elementare che descrisse è conosciuto al giorno d'oggi come **macchina di von Neumann**. Fu utilizzato nel computer EDSAC, il primo che memorizzava il programma in memoria, ed è ancora oggi, a ben mezzo secolo di distanza, alla base di quasi tutti i computer digitali. Questo progetto, così come la macchina IAS costruita in collaborazione con Herman Goldstine, ha avuto una tale influenza che vale la pena descriverlo brevemente. Nonostante si associ sempre il nome di Von Neumann a questo progetto, anche Goldstine e altri ricercatori diedero un contributo sostanziale. Uno schema dell'architettura è mostrato nella Figura 1.5.

La macchina di von Neumann era composta da cinque componenti principali: la memoria, l'unità aritmetico-logica, l'unità di controllo e i dispositivi di input e output. La memoria era costituita da 4096 locazioni, ciascuna delle quali conteneva 40 bit. Ciascuna parola poteva mantenere due istruzioni da 20 bit oppure un numero da 40 bit. Le istruzioni erano composte da 8 bit, che definivano il tipo d'istruzione, e dai restanti 12 bit, che specificavano una delle 4096 parole di memoria. L'unità aritmetico-logica e l'unità di controllo formavano insieme il "cervello" del computer; negli elaboratori moderni esse sono combinate in un unico chip chiamato **CPU** (*Central Processing Unit*, "unità centrale di calcolo").

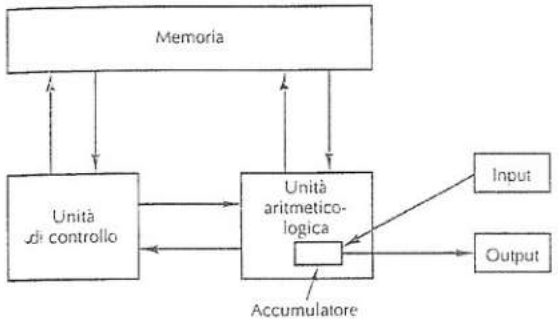


Figura 1.5 La macchina originale di von Neumann.

All'interno dell'unità aritmetico-logica vi era uno speciale registro da 40 bit, chiamato **accumulatore**. La tipica istruzione sommava una parola di memoria all'accumulatore oppure ne copiava il contenuto in memoria. La macchina non aveva un'aritmetica in virgola mobile dato che von Neumann riteneva che ogni matematico che si rispetti dovesse essere in grado di tener traccia a mente della posizione della virgola (per essere precisi, della virgola binaria).

Praticamente nello stesso periodo in cui von Neumann realizzava la macchina IAS, altri ricercatori del M.I.T. stavano costruendo un computer. Diversamente da IAS, ENIAC e altre macchine dello stesso tipo, dotate di lunghe parole e pensate per poter elaborare numeri molto grandi, la macchina del M.I.T., chiamata Whirlwind I, aveva parole di soli 16 bit ed era progettata per il controllo in tempo reale. Grazie a questo progetto si deve l'invenzione della memoria a nuclei magnetici da parte di Jay Forrester e, infine, la nascita del primo minicomputer commerciale.

Mentre avveniva tutto questo, IBM era una piccola società impegnata nel commercio di schede perforate e di macchine per il loro ordinamento. Inizialmente IBM non era fortemente interessata ai computer, sebbene avesse in parte finanziato il lavoro di Aiken. Cominciò a diventarlo nel 1953 dopo la produzione del modello 701 e quindi molto tempo dopo che la compagnia di Eckert e Mauchley era diventata, grazie al computer UNIVAC, il numero uno sul mercato. Il modello 701 era dotato di 2048 parole di 36 bit, con due istruzioni per parola, e fu la prima di una serie di macchine scientifiche che in una decade cominciarono a dominare il mercato. Tre anni dopo fu il momento del modello 704, che inizialmente aveva 4096 parole di memoria centrale, istruzioni a 36 bit e un nuovo, innovativo, hardware in virgola mobile. Nel 1958 IBM cominciò la produzione del modello 709, la sua ultima macchina che utilizzava le valvole, in pratica una miglioria del computer 704.

1.2.3 Seconda generazione – Transistor (1955-1965)

Il transistor è stato inventato nel 1948 presso i Bell Labs da John Bardeen, Walter Brattain e William Shockley, per la cui scoperta ricevettero nel 1956 il premio Nobel per la

fisica. In 10 anni il transistor rivoluzionò i computer al punto che nei tardi anni '50 i computer a valvole divennero obsoleti. Il primo computer a transistor fu costruito presso i Lincoln Laboratory del M.I.T. Si trattava di una macchina a 16 bit che seguiva la linea del Whirlwind I. Fu chiamato **TX-0** (*Transistorized eXperimental computer 0*), ideato semplicemente come dispositivo per testare il più evoluto TX-2.

Il computer TX-2 non prese mai il volo, ma nel 1957 Kenneth Olsen, uno degli ingegneri che lavoravano al Lincoln Laboratory, fondò una società, la Digital Equipment Corporation (DEC), per produrre una macchina commerciale molto simile al modello TX-0. Passarono ben quattro anni prima di veder apparire questa macchina, chiamata PDP-1; ciò fu dovuto principalmente al fatto che i finanziatori che avevano fondato la DEC erano fermamente convinti che non vi fosse mercato per i computer. Dopo tutto anche T.J. Watson, ex presidente di IBM, una volta ebbe a dire che il mercato mondiale di computer era all'incirca di sole quattro o cinque unità. Al contrario DEC si affermò principalmente per la vendita di schede di circuiti di piccole dimensioni alle imprese, da integrare nei loro prodotti.

Nel 1961, quando finalmente apparve, l'elaboratore PDP-1 era dotato di 4096 parole di memoria da 18 bit e poteva eseguire 200.000 istruzioni al secondo. Queste prestazioni corrispondevano alla metà di quelle dell'IBM 7090, successore a transistor del modello 709, nonché il più veloce computer al mondo dell'epoca. Il PDP-1 costava però 120.000 dollari, mentre l'IBM 7090 ne costava milioni. DEC vendette decine di PDP-1 e questo rappresentò la nascita dell'industria dei microcomputer.

Uno dei primi PDP-1 fu donato al M.I.T., dove attrasse rapidamente l'attenzione di alcuni dei giovani geni in erba, così comuni in quella università. Una delle principali innovazioni del PDP-1 era un display visuale in grado di disegnare punti in qualsiasi zona dello schermo da 512 per 512 pixel. Da lì a breve gli studenti programmarono la macchina PDP-1 per poter giocare a *spacewar*, facendo così conoscere al mondo il primo videogame della storia.

Pochi anni dopo DEC introdusse il modello PDP-8, una macchina a 12 bit, molto più economica del PDP-1 (16.000 dollari). La principale innovazione introdotta dal PDP-8 fu quella di avere un unico bus, chiamato *omnibus*, mostrato nella Figura 1.6. Un **bus** è un insieme di cavi paralleli utilizzati per connettere i diversi componenti di un computer. Questa architettura fu un grande passo in avanti rispetto alla macchina IAS, che era centralizzata rispetto alla memoria, e fu adottata da quasi tutti i successivi computer di piccole dimensioni. DEC riuscì a vendere 50.000 PDP-8, affermandosi così leader nel mercato dei microcomputer.

Nel frattempo la reazione di IBM all'avvento del transistor fu la costruzione del modello 7090, una versione a transistor del computer 709, citato precedentemente, e in seguito del modello 7094. Il computer 7094 aveva un ciclo di clock di 2 microsecondi e possedeva una memoria centrale di 32.536 parole da 36 bit. I modelli 7090 e 7094 segnarono la fine delle macchine in stile ENIAC e dominarono il mondo dell'elaborazione scientifica durante gli anni '60.

Nello stesso momento in cui, grazie al modello 7094, IBM diventava una delle principali forze del mercato, otteneva enormi profitti vendendo una piccola macchina per le aziende, chiamata 1401. Questa macchina era in grado di leggere e scrivere nastri

magnetici, leggere e perforare schede e stampare output a una velocità paragonabile a quella del modello 7094, il tutto a una frazione del prezzo. Era quindi perfetta per la gestione dei dati aziendali, ma aveva pessime prestazioni nei calcoli scientifici.

Figura 1.6 L'omnibus del computer PDP-8.

L'architettura del modello 1401 era insolita, dato che non aveva alcun registro e neppure una lunghezza fissa di parola. La sua memoria era composta da 4000 byte da 8 bit, sebbene modelli successivi supportassero fino a 16.000 byte, una dimensione che all'epoca era stupefacente. Ciascun byte conteneva un carattere da 6 bit, un bit di amministrazione e un bit usato per segnalare la fine di una parola. Per fare un esempio, un'istruzione `MOVE` specificava un indirizzo d'origine e uno di destinazione e iniziava a spostare byte dall'origine alla destinazione finché non ne incontrava uno il cui bit di fine parola era impostato a 1.

Nel 1964 una piccola e sconosciuta società, la Control Data Corporation (CDC), introdusse il modello 6600, una macchina che era quasi un ordine di grandezza più veloce dell'imponente 7094 e di ogni altra macchina dell'epoca. Per i matematici che si occupavano di analisi numerica, spesso chiamati *number cruncher*, "masticatori di numeri", fu amore a prima vista e il computer CDC fu lanciato verso il successo. Il segreto della sua velocità, e il motivo per cui fosse molto più del modello 7094, stava nel fatto che la CPU era, al suo interno, una macchina altamente parallela. Era dotata di varie unità funzionali che potevano lavorare contemporaneamente: alcune erano destinate a svolgere addizioni, altre moltiplicazioni e altre ancora divisioni. Anche se per ottenere il maggior guadagno possibile era richiesta un'attenta programmazione, con un po' di lavoro si riusciva a far eseguire contemporaneamente anche 10 istruzioni.

Come se non bastasse, il modello 6600 aveva al suo interno un certo numero di piccoli computer che lo aiutavano; un po' come i Sette Nani con Biancaneve, questo voleva dire che la CPU poteva spendere tutto il suo tempo a macinare numeri lasciando ai piccoli computer tutti i dettagli della gestione dei programmi e dell'input/output. A posteriori si può affermare che il 6600 era in anticipo di decenni sui tempi; molte delle idee chiave proprie dei computer moderni possono infatti essere ricondotte a questa macchina.

Il progettista del computer 6600, Seymour Cray, fu una figura leggendaria del livello di von Neumann. Dedicò la sua intera vita a costruire macchine sempre più veloci, ora chiamate **supercomputer**, tra cui gli elaboratori 6600, 7600 e Cray-1. Inventò inoltre un famoso algoritmo per l'acquisto di un'autovettura: si va dal rivenditore più vicino a casa propria, si indica la macchina più vicina alla porta e si dice: "Voglio questa". L'al-

goritmo impiega il minor tempo possibile per le cose non importanti (come comprare una macchina) per lasciare il maggior tempo possibile alle cose importanti (come progettare supercomputer).

Nella stessa epoca ci furono molti altri computer, ma uno si distinse per un motivo particolare e merita per questo di essere menzionato: il Burroughs B5000. I progettisti di macchine come i modelli PDP-1, 7094 e 6600 si concentravano esclusivamente sull'hardware, sia nel tentativo di renderlo economico (DEC) sia in quello di renderlo veloce (IBM e CDC), mentre ignoravano quasi completamente il software. I progettisti del B5000 adottarono invece un approccio diverso. Costruirono una macchina con il preciso intento di programmarla in Algol 60, un precursore di C e Java, e inclusero nell'hardware molte caratteristiche finalizzate a facilitare il compito del compilatore. Nacque così l'idea che anche il software ha la sua importanza; purtroppo questa considerazione fu quasi subito abbandonata.

1.2.4 Terza generazione – Circuiti integrati (1965-1980)

Nel 1958 l'invenzione dei circuiti integrati su silicio da parte di Jack Kilby e Robert Noyce (che lavorarono in maniera indipendente) permise di realizzare su un unico chip decine di transistor. Questo metodo di assemblaggio rese possibile la costruzione di computer più piccoli, più veloci e più economici rispetto ai loro predecessori basati su transistor. Alcuni dei computer più significativi di questa generazione sono descritti in seguito.

Fin dal 1964 IBM era la società leader nel mondo dei computer e si trovava di fronte a un grande problema che riguardava le sue due macchine di maggior successo e redditività. I modelli 7094 e 1401 erano totalmente incompatibili. Uno era un "macinatore di numeri" a grande velocità che usava aritmetica binaria parallela su registri a 36 bit, mentre l'altro era un diffuso processore di input/output che usava aritmetica decimale su parole di memoria a lunghezza variabile. Molte fra le società clienti di IBM avevano comprato entrambe le macchine e non amavano l'idea di dover avere due dipartimenti di programmazione separati senza nulla in comune.

Quando arrivò il momento di sostituire questi modelli, IBM fece un cambio radicale. Introdusse un'unica linea di prodotti, il System/360, basata su circuiti integrati e progettata sia per i calcoli scientifici sia per quelli commerciali. Il System/360 presentava molteplici innovazioni, ma la più importante fu che si trattava di una famiglia di circa mezza dozzina di macchine dotate dello stesso linguaggio assemblativo, ma di dimensione e potenza crescenti. I clienti potevano quindi sostituire i loro 1401 con i 360 Model 30 e i loro 7094 con i 360 Model 75. Il Model 75 era il più grande e il più veloce (e anche il più caro), ma il software scritto per uno di questi modelli poteva, in linea di principio, essere eseguito anche su uno differente. In pratica il software scritto per un piccolo modello funzionava senza problemi su un modello più grande, ma non valeva il viceversa: quando ci si spostava verso un modello di dimensioni inferiori il programma poteva non entrare in memoria. In ogni caso ciò rappresentava un grande avanzamento rispetto alla situazione delle macchine 7094 e 1401. L'idea di avere una *famiglia di computer* fu subito colta anche da molti altri produttori che, in pochi anni, misero sul mercato linee di macchine che ricoprivano un grande spettro di prezzi e prestazioni. La Figura 1.7 mostra alcune caratteristiche della famiglia 360 originaria (successivamente vennero introdotti anche altri modelli).

Un'altra grande innovazione della serie 360 fu la **multiprogrammazione**, grazie a cui è possibile avere più programmi in memoria allo stesso tempo, di modo che quando si è in attesa di completare un'operazione di input/output si possono eseguire dei calcoli. Il risultato che si ottiene con questa tecnica è un maggior utilizzo della CPU.

Il 360 fu anche il primo a essere capace di emulare (simulare) altri computer. Il modello più piccolo poteva emulare il computer 1401, mentre quelli più grandi erano in grado di emulare il modello 7094, di modo che, durante il passaggio ai computer 360, i clienti potessero continuare a utilizzare i loro vecchi programmi (binari) senza modificarli. Alcuni modelli eseguivano i programmi scritti per il 1401 in modo talmente più veloce che molti clienti non convertirono mai il proprio software.

Proprietà	Model 30	Model 40	Model 50	Model 65
Prestazioni relative	1	3,5	10	21
Ciclo di clock (in miliardesimi di secondo)	1000	625	500	250
Quantità massima di memoria (in byte)	65.536	262.144	262.144	524.288
Numero di byte prelevati per ciclo di clock	1	2	4	16
Massimo numero di canali per i dati	3	3	4	6

Figura 1.7 L'offerta iniziale della linea di prodotti IBM 360.

Sul 360 l'emulazione risultava facile, in quanto tutti i modelli iniziali, e molti di quelli successivi, erano microprogrammati. Tutto quello che IBM doveva fare era scrivere tre microprogrammi, uno per l'insieme d'istruzioni native del modello 360, uno per quelle del modello 1401 e uno per quelle del modello 7094. Questa flessibilità fu una delle ragioni principali per cui fu introdotta la microprogrammazione nel 360. La motivazione di Wilkes sulla riduzione del numero di valvole non contava più, ovviamente, perché il 360 non aveva valvole.

Il modello 360, grazie a un compromesso, risolse anche il dilemma dell'aritmetica binaria parallela e di quella decimale seriale: la macchina aveva 16 registri da 32 bit per l'aritmetica binaria, ma la sua memoria era orientata al byte, come quella del modello 1401. Era inoltre dotata d'istruzioni seriali simili a quelle del modello 1401 per lo spostamento in memoria di blocchi di dimensioni variabili.

Un'altra caratteristica saliente del computer 360 era un enorme (per l'epoca) spazio degli indirizzi di 2^{24}=16.777.216 byte. All'epoca, una tale quantità di memoria appariva praticamente come infinita, dato che il costo della memoria era di qualche dollaro al byte. Sfortunatamente la serie 360 fu seguita dalle serie 370, 4300, 3080, 3090, 390 e dalla serie z, tutte basate essenzialmente sulla stessa architettura. A metà degli anni '80 il limite della memoria divenne un problema reale e IBM fu costretta ad abbandonare in parte la compatibilità quando introdusse gli indirizzi a 32 bit, necessari per indirizzare una nuova memoria di 2^{32} byte.

Con il senno di poi si potrebbe discutere sul perché non si fosse pensato fin dall'inizio all'utilizzo di indirizzi a 32 bit, dato che le macchine erano state dotate di registri e parole a 32 bit; all'epoca però nessuno avrebbe mai potuto immaginare una macchina con 16 milioni di byte di memoria. Anche se per IBM il passaggio a indirizzi di 32 bit ebbe successo, si trattava ancora di una soluzione temporanea al problema dell'indirizzamento della memoria, perché i sistemi di calcolo avrebbero presto richiesto la capacità di indirizzare più di 2^32 (4.294.967.296) byte di memoria. In pochi anni sarebbero apparsi sulla scena computer con indirizzi di 64 bit.

Nella terza generazione, anche il mondo dei microcomputer fece un grande passo in avanti con l'introduzione da parte di DEC della serie PDP-11, un successore a 16 bit del modello PDP-8. Sotto molti punti di vista la serie PDP-11 era una sorta di piccolo fratello della serie 360 così come il computer PDP-1 lo era stato per il modello 7094. Entrambi i computer 360 e PDP-11 avevano i registri orientati alla parola e la memoria orientata al byte, e tutti e due vennero prodotti in una gamma in cui il rapporto prezzo/prestazioni variava. Il modello PDP-11 ebbe un enorme successo specialmente presso le università, permettendo a DEC di mantenere la leadership sugli altri produttori di microcomputer[2].

1.2.5 Quarta generazione – Integrazione a grandissima scala (1980-?)

Negli anni '80 la tecnologia **VLSI** (*Very Large Scale Integration*, "integrazione a larghissima scala") permise di inserire in un unico chip prima decine di migliaia, poi centinaia di migliaia e infine milioni di transistor. Questo sviluppo portò velocemente alla realizzazione di computer più piccoli e più veloci. Prima del PDP-1 i computer erano talmente grandi e costosi che, per farli funzionare, aziende e università dovevano disporre di speciali uffici chiamati **centri di calcolo**, mentre, con l'avvento dei microcomputer, ogni dipartimento poteva dotarsi di un proprio elaboratore. Dal 1980 i prezzi crollarono al punto che anche i privati potevano permettersi di possedere un computer: ebbe così inizio l'era del personal computer.

I personal computer furono utilizzati in modo molto diverso rispetto ai computer di grandi dimensioni. Vennero usati per l'elaborazione di testi, per l'amministrazione domestica (con i ben noti *spreadsheet*) e per numerose altre applicazioni fortemente interattive (come i videogiochi) che computer di dimensioni maggiori non potevano trattare in modo altrettanto pratico.

I primi personal computer venivano generalmente venduti in scatole di montaggio. Ciascun kit conteneva una scheda con circuito stampato, vari chip, tra cui di solito vi era un Intel 8080, alcuni cavi, un trasformatore e talvolta un floppy disk da 8 pollici. Era compito dell'acquirente montare le parti per costruire il computer, e il software non era incluso: se si voleva un qualsiasi programma occorreva scriverlo. Successivamente si diffuse per i processori 8080, il sistema operativo CP/M, scritto da Gary Kildall. Si trattava di un vero sistema operativo (su floppy disk), con un file system e una serie

[2] Questi modelli erano generalmente chiamati *minicomputer* (*N.d.Rev.*).

d'istruzioni che l'utente doveva scrivere con la tastiera per farle leggere da un interprete di comandi (*shell*).

Un altro dei primi personal computer fu l'Apple e in seguito l'Apple II, progettati da Steve Jobs e Steve Wozniak nel loro famoso garage. Questa macchina ebbe un'enorme diffusione tra i privati e nelle scuole, rendendo la Apple dall'oggi al domani un serio concorrente sul mercato.

IBM, la forza principale dell'industria dei computer, dopo aver osservato a lungo e considerato a fondo che cosa stavano facendo le aziende concorrenti, decise infine di entrare nel mercato dei personal computer. Dato che progettare da zero una nuova macchina usando solo componenti proprietarie, costruite con transistor proprietari fatti di materiali di proprietà, avrebbe richiesto troppo tempo, IBM scelse una strada alternativa e piuttosto inusuale. Diede a un suo dirigente, Philip Estridge, una grande borsa carica di soldi e gli affidò l'incarico di andare a costruire un personal computer lontano dalle interferenze dei burocrati della sede centrale di Armonk, nello stato di New York. Estridge iniziò a fare acquisti a 2000 Km di distanza, a Boca Raton (Florida), scelse come CPU l'Intel 8088 e costruì il Personal Computer IBM utilizzando componenti commerciali. Il PC IBM fu introdotto nel mercato nel 1981 e divenne subito il computer più venduto nella storia. Quando il PC ha compiuto i 30 anni sono apparsi diversi articoli sulla sua storia, tra cui quelli a firma Bradley (2011), Goth (2011), Bride (2011) e Singh (2011).

IBM fece anche qualcos'altro d'inusuale di cui in seguito si sarebbe pentita. Diversamente dal solito, invece di mantenere completamente segreto il progetto della macchina (o almeno di proteggerlo con un muro enorme e impenetrabile di brevetti) pubblicò, in un libro venduto a soli 49 dollari, tutti gli schemi al completo, compresi i diagrammi dei circuiti. L'idea era quella di permettere ad altre società di realizzare delle schede aggiuntive per il PC IBM, in modo da aumentarne la flessibilità e la popolarità. Sfortunatamente per IBM numerose altre società cominciarono a realizzare dei **cloni** del PC, dato che il progetto era completamente pubblico e i componenti facilmente reperibili sul mercato. Spesso questi computer erano molto più economici di quelli IBM; in questo modo prese il via una nuova industria.

Nonostante altre compagnie, tra cui Commodore, Apple e Atari, realizzassero personal computer senza utilizzare le CPU Intel, la forza del mercato dei PC IBM era talmente grande che gli altri ne vennero schiacciati. Solo in pochi sopravvissero e soltanto in mercati di nicchia.

Uno dei sopravvissuti, seppur a mala pena, fu il Macintosh di Apple. Il Macintosh era stato introdotto nel 1984 come successore dello sfortunato Apple Lisa, il primo computer dotato di una **GUI** (*Graphical User Interface*, "interfaccia grafica per l'utente"), simile alla ormai popolare interfaccia di Windows. Lisa fallì a causa del costo troppo elevato, mentre il più economico Macintosh, introdotto l'anno successivo, riscosse un enorme successo e ispirò amore e passione nei suoi numerosi ammiratori.

Il giovane mercato dei personal computer fece nascere anche il desiderio, fino ad allora senza precedenti, di avere computer portatili. A quel tempo un computer portatile aveva senso quanto può averne oggi un frigorifero portatile. Il primo vero personal computer portatile fu l'Osborne-1, che con i suoi 11 Kg fu più che altro un "computer da

bagagliaio". Eppure, dimostrò che era possibile realizzare computer portatili. L'Osborne-1 ebbe un successo commerciale modesto, ma un anno dopo Compaq produsse il suo primo clone di PC IBM portatile e divenne rapidamente il leader di tale mercato.

La versione iniziale del PC IBM veniva venduta insieme al sistema operativo MS-DOS, fornito dalla Microsoft Corporation, una società che all'epoca era ancora piccola. Visto che Intel riusciva a produrre CPU sempre più potenti, IBM e Microsoft poterono sviluppare un successore di MS-DOS, chiamato OS/2, dotato di un'interfaccia utente grafica simile a quella dell'Apple Macintosh. Allo stesso tempo Microsoft, pensando all'ipotesi che OS/2 potesse non prendere piede, cominciò a sviluppare un suo proprio sistema operativo, Windows, eseguito sopra MS-DOS. Per riassumere questa lunga vicenda basti dire che OS/2 non ebbe successo, che fra IBM e Microsoft vi fu un'accesa e pubblica disputa e che Microsoft continuò per la propria strada, facendo di Windows un enorme successo. Come la piccola Intel e la ancora più piccola Microsoft abbiano potuto detronizzare IBM, una delle più grandi, ricche e potenti società nella storia del mondo, è senza dubbio una lezione che viene attentamente studiata in tutte le scuole di economia del mondo.

Dopo il successo dell'8088, Intel continuò a realizzarne versioni via via migliori e più potenti. Particolarmente degno di nota fu il processore a 32 bit 80386, introdotto nel 1985, che fu seguito da una versione potenziata, chiamata naturalmente 80486. Le versioni successive sono andate sotto il nome di Pentium e Core e sono quelle utilizzate in quasi tutti i moderni PC. Il termine utilizzato da molte persone per descrivere l'architettura di tutti questi processori è x86. Il nome x86 è anche utilizzato per indicare i chip compatibili prodotti da AMD.

Alla metà degli anni '80 iniziò ad affermarsi un nuovo tipo di progettazione chiamata RISC, che consisteva nel sostituire le complicate architetture esistenti con altre molto più semplici (e più veloci). Queste macchine erano in grado di eseguire più istruzioni allo stesso tempo, spesso in un ordine diverso da quello in cui apparivano nel programma. I concetti di CISC, RISC e superscalare saranno introdotti nel Capitolo 2 e verranno analizzati approfonditamente nel corso del libro.

Sempre alla metà degli anni '80, Ross Freeman sviluppò con alcuni colleghi di Xilinx un approccio intelligente per costruire circuiti integrati senza bisogno di montagne di soldi né di un impianto di produzione di silicio. Questo nuovo tipo di circuito, chiamato FPGA (*field-programmable gate array*), conteneva una grande quantità di porte logiche generiche che potevano essere "programmate" in un qualsiasi circuito incorporabile nel dispositivo. Questo nuovo straordinario approccio al progetto di circuiti rese l'hardware FPGA malleabile come il software. Con l'utilizzo di un FPGA, al costo variabile da poche decine ad alcune centinaia di dollari, divenne possibile costruire sistemi informatici specializzati per applicazioni uniche al servizio di un numero ristretto di utenti. Fortunatamente, le imprese di fabbricazione di silicio potevano ancora produrre chip più veloci, con consumo ridotto e meno costosi per le applicazioni che richiedevano milioni di chip. Ma per applicazioni con pochi utenti, come la prototipazione, le applicazioni di progettazione a bassi volumi e l'istruzione, i componenti FPGA rimangono uno strumento importante per la costruzione di hardware.

I computer restarono a 8, 16 o 32 bit fino al 1992, quando DEC presentò il rivoluzionario Alpha, una vera macchina a 64 bit di tipo RISC che surclassava di gran lunga le prestazioni degli altri computer. La macchina ebbe un successo limitato, ma dovette passare un decennio prima che le macchine a 64 bit cominciassero ad affermarsi su grande scala, soprattutto fra i server di fascia alta.

Per tutti gli anni '90 i sistemi di calcolo sono diventati sempre più veloci grazie a una varietà di ottimizzazioni micro-architetturali, molte delle quali saranno esaminate in questo libro. Gli utenti di questi sistemi sono stati coccolati dai produttori di computer, perché ogni nuovo sistema comprato sarebbe stato in grado di eseguire i loro programmi molto più velocemente del loro vecchio sistema. Tuttavia, verso la fine degli anni '90 questa tendenza cominciava a svanire a causa di due importanti ostacoli di progetto: gli architetti erano a corto di trucchi per rendere i programmi più veloci e il raffreddamento dei processori stava diventando troppo costoso.

Nel disperato tentativo di continuare a costruire processori più veloci, la maggior parte delle società di computer ha iniziato a virare verso architetture parallele per ricavare maggiori prestazioni dai propri chip. Nel 2001 IBM ha introdotto l'architettura dual-core POWER4, il primo caso di CPU che incorporava due processori sullo stesso chip. Oggi la maggior parte dei processori di classe desktop e server, e anche alcuni processori integrati, incorporano più processori su un singolo chip. Le prestazioni di questi multiprocessori si sono rivelate purtroppo tutt'altro che stellari per l'utente medio, perché (come vedremo nei capitoli successivi) macchine parallele richiedono ai programmatori di parallelizzare esplicitamente i programmi, un compito difficile e soggetto a errori.

1.2.6 Quinta generazione – Computer a basso consumo e computer invisibili

Nel 1981 il governo giapponese annunciò di voler stanziare 500 milioni di dollari per aiutare le società locali nella costruzione della quinta generazione di computer, che si sarebbe basata sull'intelligenza artificiale e che avrebbe rappresentato un enorme balzo in avanti rispetto alla "stupida" quarta generazione. I produttori di computer americani ed europei, avendo visto le compagnie giapponesi prendere piede in molti mercati, come quello delle telecamere, degli stereo e dei televisori, piombarono improvvisamente nel panico più totale e cominciarono a chiedere, tra le altre cose, finanziamenti ai rispettivi governi. Il progetto giapponese di quinta generazione, nonostante tanta enfasi, sostanzialmente fallì e venne abbandonato nel silenzio. In un certo senso fu come per l'*analytical engine* di Babbage; si trattò di un'idea visionaria talmente avanti rispetto ai tempi che la tecnologia necessaria non era neanche prevedibile.

Quella che può essere definita la quinta generazione di computer vide comunque la luce, seppur in un modo inaspettato: i computer si rimpiccolirono. Nel 1989 la Grid Systems rilasciò il primo tablet computer, chiamato GridPad. Si trattava di un piccolo schermo su cui gli utenti potevano scrivere con una speciale penna. Sistemi come il GridPad hanno mostrato che i computer non avevano bisogno di essere allocati su una scrivania o in una sala macchine, ma potevano essere inseriti in un supporto facile da trasportare, con touchscreen e riconoscimento della scrittura per dargli un valore aggiunto.

L'Apple Newton, prodotto nel 1993, mostrò che un computer poteva essere costruito con dimensioni non più grandi di quelle di un lettore di cassette audio. Come il Grid Pad, anche il Newton utilizzava come dispositivo di input la scrittura a mano libera, e questo rappresentò un grande ostacolo al suo successo; in seguito però altre macchine della stessa tipologia, ora chiamate PDA (*Personal Digital Assistants*, "assistente digitale personale"), hanno migliorato l'interfaccia utente e hanno avuto un'enorme diffusione. L'evoluzione di questi dispositivi ha portato agli attuali smartphone.

L'interfaccia di scrittura del PDA fu messa a punto da Jeff Hawkins, che aveva fondato una società denominata Palm per sviluppare PDA economici rivolti al mercato di massa. Hawkins era un ingegnere elettronico di formazione, ma aveva un vivo interesse nel campo delle neuroscienze, la disciplina che studia il cervello umano. Si rese conto che il riconoscimento della scrittura a mano libera poteva essere reso più affidabile insegnando agli utenti a scrivere in un modo più facilmente leggibile dai computer, una tecnica di input che Hawkins ha chiamato "Graffiti". Era necessario un breve periodo di formazione per l'utente, ma alla fine la scrittura diventava più veloce e affidabile. Il primo PDA della Palm, chiamato Palm Pilot, è stato un enorme successo. Graffiti è uno dei più grandi successi nel settore informatico, avendo mostrato la potenza della mente umana che ... trae vantaggio dalla potenza della mente umana.

Gli utenti di PDA avevano piena fiducia nei loro dispositivi e li utilizzavano religiosamente per gestire appuntamenti e contatti. Quando i telefoni cellulari iniziarono a guadagnare popolarità nei primi anni '90, IBM colse al volo l'opportunità di integrare il telefono cellulare con il PDA, creando così lo "smartphone". Il primo smartphone, chiamato Simon, utilizzava un touchscreen per la gestione dell'input e offriva all'utente tutte le funzionalità di un PDA più il telefono, i giochi e l'email. Le dimensioni sempre più piccole dei componenti e il costo sempre più contenuto hanno portato a una grande diffusione degli smartphone, rappresentati dai popolari Apple iPhone e dalle piattaforme Google Android.

Tuttavia, neanche i PDA e gli smartphone possono essere considerati veramente rivoluzionari. Molto più importanti sono invece i computer cosiddetti "invisibili", poiché integrati in elettrodomestici, orologi, carte di credito e numerosi altri dispositivi (Bechini et al., 2004). In un ampio spettro di applicazioni questi processori garantiscono grandi funzionalità a basso costo. Anche se si può discutere sul fatto che questi processori siano veramente una nuova generazione (dato che sono in circolazione dagli anni '70), è indubbio che stiano rivoluzionando il funzionamento di migliaia di elettrodomestici e altri dispositivi. Stanno già cominciando ad avere un forte impatto sul mondo e nei prossimi anni la loro influenza crescerà rapidamente. Un aspetto non comune di questi computer integrati è che l'hardware e il software sono spesso **coprogettati** (Henkel et al., 2003). Più avanti nel libro torneremo a occuparci di loro.

Se la prima generazione è rappresentata dalle macchine a valvole (per esempio ENIAC), la seconda dalle macchine a transistor (per esempio, l'IBM 7094), la terza dalle macchine a circuiti integrati (per esempio, l'IBM 360) e la quarta dai personal computer (per esempio, le CPU Intel), la quinta generazione è in realtà caratterizzata più da un cambiamento di modello che da una specifica nuova architettura. In futuro i computer si troveranno ovunque e saranno integrati in ogni oggetto, in poche parole essi saranno invisibili: faranno parte dei comuni gesti della vita di ogni giorno, come aprire porte, accendere luci, spendere denaro e migliaia di altre azioni. Questo modello, ideato dallo scomparso Mark Weiser, venne originariamente chiamato *ubiquitous computing* ("computazione onnipresente"), anche se spesso ci si riferisce a esso con il termine *pervasive computing* ("computazione pervasiva") (Weiser, 2002): cambierà profondamente il mondo, così come lo fece la rivoluzione industriale. Nel libro non tratteremo ulteriormente l'argomento, ma per avere maggiori informazioni si consultino (Lyytinen e Yoo, 2002; Saha e Mukherjee, 2003; Sakamura, 2002).

1.3 Tipologie di computer

Mentre nel paragrafo precedente abbiamo affrontato brevemente la storia dei sistemi di elaborazione, in questo guarderemo al presente e rivolgeremo anche uno sguardo al futuro. Sebbene i personal computer siano gli elaboratori più conosciuti, al giorno d'oggi esistono anche altri tipi di macchine: per questo vale la pena dare un'occhiata a quanto c'è intorno a noi.

1.3.1 Forze tecnologiche ed economiche

L'industria dei computer avanza più velocemente di tutte le altre. La principale forza trainante è la capacità, che i produttori di circuiti hanno, di integrare di giorno in giorno sempre più transistor all'interno dei chip. Avere più transistor, cioè piccoli interruttori elettronici, significa avere memorie più grandi e processori più potenti. Gordon Moore, cofondatore ed ex presidente di Intel, un giorno scherzando disse che, se la tecnologia dell'aviazione fosse cresciuta velocemente quanto quella dei computer, ora un aeroplano costerebbe 500 euro e farebbe il giro della terra in soli 20 minuti con 20 litri di carburante. Per di più un tale aeroplano avrebbe le dimensioni di una scatola da scarpe. In particolare Moore, durante la preparazione di un discorso dinanzi a un gruppo industriale, sottolineò che ogni nuova generazione di chip veniva introdotta 3 anni dopo quella precedente. Dato che ogni nuova generazione ha una quantità di memoria quattro volte maggiore della precedente, comprese che il numero di transistor in un chip aumentava a una velocità costante e predisse che questa crescita sarebbe continuata allo stesso modo per decenni. Questa osservazione è conosciuta come la **legge di Moore**.

Attualmente con il termine "legge di Moore" si intende il fatto che il numero di transistor raddoppia ogni 18 mesi; questo è equivalente a dire che il numero di transistor aumenterebbe circa del 60% ogni anno. La Figura 1.8, che riporta le dimensioni di chip di memoria e la loro data di apparizione, conferma che la legge di Moore è rimasta valida per oltre tre decenni.

Ovviamente la legge di Moore non è una vera e propria legge, ma un'osservazione empirica sulla velocità con cui i fisici dello stato solido e gli ingegneri hanno fatto avanzare lo stato dell'arte, e una previsione che ciò continuerà anche in futuro. Alcuni osservatori del mondo dell'industria si aspettano che la legge di Moore continui a valere per almeno un altro decennio. Altri osservatori si aspettano che la dissipazione di energia, la dispersione di corrente e altri effetti si manifestino prima e causino gravi problemi che dovranno essere risolti (Bose, 2004, Kim et al., 2003).

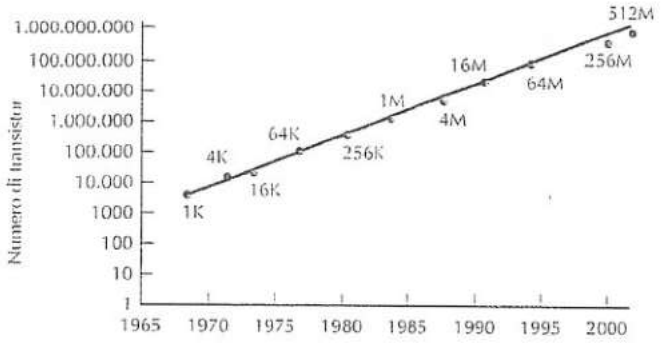


Figura 1.8 La legge di Moore prevede un aumento annuale del 60% del numero di transistor che può essere inserito su un chip. I valori indicati al di sopra e al di sotto della linea rappresentano le dimensioni della memoria, in bit.

La realtà sulla miniaturizzazione dei transistor è che il loro spessore sarà presto solo di pochi atomi. A quel punto i transistor saranno composti da troppo pochi atomi per essere affidabili, o più semplicemente si raggiungerà il punto in cui ulteriori diminuzioni di dimensione richiederanno l'utilizzo di componenti subatomiche. (Un buon consiglio a tutti coloro che lavorano in una fabbrica di produzione di silicio: si raccomanda di prendere un giorno libero quando decideranno di scomporre il transistor costituito da un solo atomo!)

Nonostante le molte sfide da affrontare per estendere l'andamento della legge di Moore, ci sono tecnologie promettenti all'orizzonte, tra le quali gli sviluppi in informatica quantistica (Oskin et al., 2002) e i nanotubi di carbonio (Heinze et al., 2002) che possono creare un'opportunità per spingere l'elettronica oltre i limiti imposti dal silicio.

La legge di Moore ha creato quello che gli economisti chiamano un **ciclo virtuoso**. Gli avanzamenti tecnologici (transistor/chip) portano a prodotti migliori e allo stesso tempo più economici. Prezzi inferiori portano a loro volta alla creazione di nuove applicazioni (nessuno programmava videogiochi per computer quando un elaboratore costava 10 milioni di dollari, anche se quando il prezzo scese a 120.000 dollari alcuni studenti del M.I.T. hanno raccolto la sfida). Nuove applicazioni creano nuovi mercati e fanno sorgere nuove società che cercano di trarne vantaggio. L'esistenza di tutte queste società genera competizione, che, a sua volta, crea la domanda economica per avere tecnologie migliori grazie a cui sconfiggere la concorrenza. In questo modo il cerchio si chiude.

Un altro fattore che guida lo sviluppo tecnologico è la prima legge del software di Nathan (dovuta a Nathan Myhrvold, un ex dirigente di Microsoft). Egli affermò che: "Il software è come un gas. Si espande fino a riempire il recipiente in cui è contenuto". Tornando agli anni '80 l'elaborazione di testi era fatta mediante programmi come *troff*. *Troff* occupa in memoria uno spazio dell'ordine dei KB, mentre i moderni elaboratori di testo occupano vari MB; senza alcun dubbio quelli futuri richiederanno GB di memoria (in prima approssimazione i prefissi K, M e G significano rispettivamente migliaia,

milioni e miliardi. si veda comunque il Paragrafo 1.5 per maggiori dettagli). Il software, continuando ad acquisire sempre più funzionalità (come crostacei che si attaccano via via alla chiglia di una barca), crea una richiesta costantemente crescente di processori più veloci, memorie più grandi e supporti di I/O di maggiore capacità.

Se nel corso degli anni l'aumento del numero di transistor per chip è stato sensazionale, i progressi delle altre tecnologie legate ai computer non sono stati da meno. Per fare un esempio, il PC IBM/XT introdotto nel 1982 aveva un hard disk da 10 MB, mentre a distanza di trent'anni i suoi successori sono comunemente dotati di hard disk da 1 TB. Questo aumento della capacità di cinque ordini di grandezza in 30 anni rappresenta un incremento annuale di circa il 50%. Misurare i progressi dei dischi è però un compito complesso, dato che oltre alla dimensione vi sono altri parametri da considerare, come la velocità di trasferimento, il tempo di ricerca e il prezzo. In ogni caso con quasi tutte queste metriche si verifica che dal 1982 il rapporto prezzo/prestazioni è migliorato ogni anno almeno del 50%. Questo enorme miglioramento delle prestazioni dei dischi, associato al fatto che il volume economico degli hard disk prodotti nella Silicon Valley ha superato quello delle CPU, ha spinto Al Hoagland ad affermare che il nome della famosa valle è sbagliato: andrebbe infatti chiamata Iron Oxide Valley (essendo questo il materiale – un ossido di ferro – su cui vengono registrati i dati nei dischi). La tendenza sta però lentamente ritornando a favore del silicio, dato che le memorie flash basate su silicio stanno rimpiazzando in molti sistemi i tradizionali dischi meccanici.

Un'altra area ad aver conosciuto uno spettacolare progresso è stata quella delle telecomunicazioni e delle reti. In meno di due decenni siamo passati dai modem a 300 bit/s, ai modem analogici a 56.000 bit/s fino alle fibre ottiche a 10^{12} bit/s. I cavi telefonici transatlantici in fibra ottica, come il TAT-12/13, costano circa 700 milioni di euro, durano 10 anni, e possono trasportare 300.000 chiamate simultanee; questi dati corrispondono a meno di un centesimo ogni 10 minuti di chiamata intercontinentale. È già stato dimostrato che si possono realizzare sistemi ottici di comunicazione a 10^{12} bit/s su distanze superiori a 100 km e senza l'utilizzo di amplificatori. La crescita esponenziale di Internet non richiede ulteriori commenti.

1.3.2 Tipologie di computer

Richard Hamming, ex-ricercatore dei Bell Labs, una volta osservò che un cambiamento della quantità di un ordine di grandezza produce un cambiamento della qualità. Secondo questa osservazione una macchina da corsa che può raggiungere i 1000 Km/h nel deserto del Nevada è un tipo di vettura fondamentalmente diverso da una comune automobile che va a 100 Km/h su un'autostrada. Analogamente un grattacielo da 100 piani non è semplicemente una versione ingrandita di un condominio da 10 piani. Nel caso dei computer non si parla di un fattore 10, ma addirittura di un fattore 1 milione nell'arco di tre decenni.

Il guadagno fornito dalla legge di Moore può essere sfruttato dai produttori dei chip in modi diversi. Uno di loro consiste nel costruire computer sempre più potenti allo stesso prezzo; l'altro nel costruire di anno in anno sempre lo stesso computer, ma a un prezzo inferiore. L'industria dei computer ha seguito entrambi gli approcci, e altri anco-

ra, al punto che oggi esiste un'ampia varietà di computer. La Figura 1.9 mostra una classificazione sommaria dei computer attuali.

Nei paragrafi seguenti esamineremo ciascuna di queste categorie e ne considereremo brevemente le proprietà.

Tipo	Prezzo ($)	Esempio di applicazione
Computer usa e getta	0,5	Cartoline d'auguri
Microcontrollore	5	Orologi, automobili, elettrodomestici
Dispositivi mobili e da gioco	50	Videogiochi e smartphone
Personal computer	500	Computer desktop o portatili
Server	5K	Server di rete
Raggruppamento di workstation	50-500K	Minisupercomputer di un dipartimento
Mainframe	5M	Elaborazione batch dei dati in una banca

Figura 1.9 L'attuale spettro di computer disponibili. I prezzi vanno interpretati "cum granu salis" (o meglio ancora, "cum saxo salis").

1.3.3 Computer usa e getta

Al gradino più basso troviamo i chip inseriti all'interno di cartoline d'auguri, che emettono la melodia di "Happy Birthday to You" o qualche altro motivetto altrettanto popolare. Gli autori non hanno ancora trovato una cartolina di condoglianze che suoni un canto funebre, ma ora che hanno reso pubblica questa idea si aspettano di vederne apparire presto una sul mercato. Per chi è cresciuto con grandi computer da milioni di dollari l'idea di un computer usa e getta ha senso quanto può averne quella di un aereo con lo stesso utilizzo.

Tuttavia i computer usa e getta fanno parte del nostro mondo e continueranno a farne parte anche in futuro. Probabilmente il più importante sviluppo nell'area di questi tipi di computer è rappresentato dai chip **RFID** (*Radio Frequency Identification*, "identificazione a radiofrequenza"). Oggi è possibile costruire, per pochi centesimi, chip RFID senza batteria, larghi meno di 0,5 mm, caratterizzati da un numero a 128 bit predefinito e univoco e contenenti al loro interno un piccolo radiotrasmettitore. Quando ricevono impulsi radio da un'antenna esterna i chip RFID si caricano grazie al segnale entrante sufficientemente a lungo da riuscire a ritrasmettere all'antenna il proprio numero identificativo. Anche se questi chip sono piccoli, le loro possibili implicazioni non lo sono affatto.

Partiamo da un'applicazione di uso mondano: eliminare il codice a barre dai prodotti. Sono già stati realizzati test sperimentali in cui i prodotti venduti in un negozio hanno, al posto del codice a barre, un chip RFID applicato dal produttore. Il cliente sceglie i prodotti desiderati, li ripone in un carrello della spesa e lo spinge semplicemente fuori dal negozio, evitando le casse. All'uscita un lettore dotato d'antenna spedisce un segnale richiedendo a ciascun prodotto di identificarsi, cosa che viene fatta attraverso una breve trasmissione radio. Anche il cliente viene identificato per mezzo di un chip

presente sul proprio bancomat o carta di credito, così che alla fine del mese il negozio potrà inviargli una dettagliata ricevuta di tutti gli acquisti effettuati. Se il cliente non ha un conto in una banca o una carta di credito che supporta RFID, scatta invece un allarme. Questo sistema non solo può sostituire i cassieri ed eliminare le relative attese in coda, ma può anche servire da sistema antifurto, dato che non è di alcuna utilità nascondere un prodotto in tasca o nello zainetto.

Una proprietà interessante di questo sistema è che, mentre il codice a barre identifica solo il tipo di prodotto, i chip RFID, grazie ai 128 bit a disposizione, possono identificare anche il prodotto stesso. Ne consegue che, per esempio, ogni pacchetto di aspirine presente sullo scaffale di un supermercato potrebbe avere un diverso codice RFID. Questo significa che se un produttore farmaceutico dovesse scoprire un difetto in un lotto di aspirine già distribuito, si potrebbe richiedere ai supermercati di tutto il mondo di far suonare un allarme se un cliente compra un pacchetto il cui numero RFID fa parte di tale gruppo. Ciò sarebbe attuabile anche se l'acquisto fosse effettuato in un paese lontano e a distanza di mesi; le aspirine non appartenenti al lotto difettato non provocherebbero invece alcun allarme.

Ma etichettare pacchetti di aspirine, biscotti o cibo per cani è soltanto l'inizio. Perché fermarsi a etichettare i biscotti per cani quando si può etichettare il cane stesso? Alcuni padroni stanno già chiedendo ai veterinari di impiantare nei loro animali un chip RFID, in modo da poterli rintracciare nel caso venissero rubati o si perdessero. Anche gli allevatori vogliono etichettare il loro bestiame. Il passo successivo che ci si aspetta è quello di genitori nervosi che richiedono al pediatra di impiantare un chip RFID nei loro figli, per poterli ritrovare nel caso venissero rapiti o si perdessero. Già che ci siamo, perché non obbligare gli ospedali a impiantarli in tutti i neonati, in modo da evitare gli scambi di bebè? Senza dubbio i governi e la polizia possono trovare ottime ragioni per identificare e tracciare i movimenti di tutti i cittadini in qualsiasi momento. Detto questo, le "implicazioni" dei chip RFID a cui si alludeva precedentemente risultano un po' più chiare.

Un'altra applicazione (leggermente meno controversa) dei chip RFID è l'identificazione di veicoli. Quando un treno, con chip RFID integrati, passa vicino a un lettore, il computer che vi è collegato ottiene una lista delle vetture che sono transitate. Questo sistema permette di localizzare agevolmente tutti i vagoni, il che può essere d'aiuto a fornitori, clienti e ferrovie. Uno schema simile sarebbe applicabile anche ai camion, mentre per le automobili l'idea è già utilizzata per il pagamento dei pedaggi autostradali per via elettronica (per esempio nel sistema E-Z Pass).

Anche la gestione dei bagagli delle compagnie aeree, così come molti altri sistemi di consegna, potrebbero trarre vantaggio dai chip RFID. All'areoporto di Heathrow, a Londra, è stato condotto un esperimento per sollevare i passeggeri in arrivo dal ritiro dei loro bagagli. Le borse dei passeggeri che avevano acquistato questo servizio venivano marcate mediante chip RFID, poi spedite separatamente all'interno dell'aeroporto e infine consegnate direttamente nei rispettivi hotel. Fra le svariate possibilità di utilizzo dei chip RFID si possono immaginare macchine che, giunte alla stazione di verniciatura di una linea di assemblaggio, comunicano il colore che deve essere loro applicato; ulteriori possibili impieghi comprendono lo studio delle migrazioni di animali, la pro-

duzione di vestiti che comunicano alla lavatrice la temperatura da utilizzare e molto altro ancora. Alcuni chip potrebbero essere integrati con sensori in modo che i bit inferiori del loro numero identificativo potrebbero specificare temperatura corrente, pressione, umidità o altre variabili ambientali.

Chip RFID più avanzati sono in grado di memorizzare dati in modo permanente. Questa possibilità ha spinto la Banca Centrale Europea a decidere di inserire tali chip nelle banconote di euro dei prossimi anni. I chip sarebbero in grado di memorizzare il loro percorso. Ciò non solo renderebbe virtualmente impossibile contraffare le banconote di euro, ma permetterebbe anche di individuare i riscatti pagati ai rapitori e i bottini delle rapine; inoltre sarebbe facile tener traccia del denaro sporco e invalidarne le banconote. In futuro, quando il denaro cesserà di essere anonimo, una procedura standard di polizia potrebbe essere quella di controllare dove sia stato di recente il denaro di un sospetto. Chi avrà bisogno di impiantare chip nelle persone quando i loro portafogli ne saranno già pieni? Molto probabilmente, quando l'opinione pubblica capirà che cosa permettono di fare i chip RFID, nasceranno dibattiti pubblici in materia.

La tecnologia utilizzata nei chip RFID si sta sviluppando rapidamente. I più piccoli sono passivi (non alimentati internamente) e capaci soltanto di trasmettere il loro numero univoco quando richiesto. Quelli più grandi invece sono attivi, possono contenere una piccola batteria e un computer elementare, e sono in grado di compiere alcuni calcoli. Le piccole carte utilizzate per le transazioni economiche rientrano in questa categoria.

I chip RFID si distinguono non solo per essere attivi o passivi, ma anche in base allo spettro di radiofrequenze a cui possono rispondere. Quelli che operano a basse frequenze hanno una velocità di trasferimento dati limitata, ma possono essere captati da un'antenna anche a grande distanza. Quelli che operano ad alte frequenze hanno invece una più alta velocità di trasferimento dati, ma un raggio d'azione più ristretto. I chip differiscono anche sotto altri aspetti e continuano a essere migliorati. Internet è piena d'informazioni riguardanti i chip RFID; un buon punto di partenza è il sito www.rfid.org.

1.3.4 Microcontrollori

Al gradino successivo troviamo i computer integrati in apparecchiature che non sono vendute come elaboratori. I computer integrati in un dispositivo, a volte chiamati **microcontrollori**, lo comandano e ne gestiscono l'interfaccia utente. I microcontrollori sono presenti in un gran numero di apparecchi molto diversi fra loro; in seguito sono elencate alcune di queste categorie:

1. elettrodomestici (radiosveglie, lavatrici, asciugatrici, forni a microonde, impianti antifurto);
2. dispositivi per la comunicazione (telefoni senza fili, cellulari, fax, cercapersone);
3. periferiche del computer (stampanti, scanner, modem, lettori CD-ROM);
4. apparecchi per l'intrattenimento (videoregistratori, DVD, stereo, lettori MP3, decoder video);
5. dispositivi per le immagini (TV, macchine fotografiche digitali, videocamere digitali, obiettivi, fotocopiatrici);

6. strumenti medicali (raggi X, MRI, elettrocardiogrammi, termometri digitali);
7. armi (missili, torpedini);
8. apparecchiature per gli acquisti (macchinette per la distribuzione automatica, bancomat, registratori di cassa);
9. giochi (bambole parlanti, console per videogame, auto o barche telecomandate).

Un'automobile può facilmente contenere 50 microcontrollori che comandano diversi sottosistemi tra cui l'antibloccaggio dei freni, l'iniezione della benzina, la radio e il GPS. Un jet ne può contenere anche più di 200, mentre una famiglia può facilmente possederne centinaia senza neanche saperlo. Fra pochi anni praticamente ogni dispositivo elettrico conterrà un microcontrollore. Il numero di microcontrollori venduti ogni anno surclassa di vari ordini di grandezza le vendite di tutti gli altri tipi di computer, fatta eccezione per quelli usa e getta.

Se i chip RFID sono sistemi minimali, i microcontrollori sono invece sistemi completi, anche se di piccole dimensioni. Ciascun microcontrollore è dotato di un processore, di una memoria e di capacità di I/O. Queste ultime di solito comprendono la gestione di pulsanti e interruttori del dispositivo e il controllo di luci, display e audio. Nella maggior parte dei casi il software è integrato nel chip sotto forma di una memoria di sola lettura creata durante la fabbricazione del microcontrollore. Generalmente i microcontrollori possono essere di due tipi: a uso generale, o specifico per un'applicazione. I primi non sono altro che piccoli, ma comuni, computer; mentre gli ultimi hanno un'architettura e un insieme d'istruzioni progettati specificatamente per una particolare applicazione, per esempio multimediale. I microcontrollori esistono in versioni da 4, 8, 16 e 32 bit.

Tuttavia anche i microcontrollori di uso generale differiscono per vari aspetti dai PC standard. Prima di tutto sono estremamente critici dal punto di vista dei costi. Un'azienda che deve comprare milioni di microcontrollori potrebbe basare la propria scelta su una differenza di prezzo di un centesimo per unità. Questo vincolo porta i produttori di microcontrollori a basare le proprie scelte architetturali molto più sui costi di produzione, rispetto ai produttori di chip da centinaia di euro. Il prezzo dei microcontrollori varia tuttavia in modo rilevante a seconda del loro numero di bit, della grandezza e tipo della memoria di cui sono dotati e di altri fattori; per farsi un'idea, un microcontrollore a 8 bit acquistato in grandi quantità può costare meno di 10 centesimi. Questo prezzo rende possibile l'inserimento di un computer all'interno di una radiosveglia da 9,95 euro.

In secondo luogo, praticamente tutti i microcontrollori lavorano in tempo reale; essi ricevono uno stimolo e ci si aspetta che diano una risposta immediata. Una comune situazione è l'accensione di una luce quando l'utente preme un pulsante; in tal caso non ci deve essere alcun ritardo tra il momento in cui il pulsante viene premuto e quello in cui la luce si accende. Spesso la necessità di lavorare in tempo reale influisce sul tipo di architettura.

In terzo luogo, i sistemi integrati sono generalmente soggetti a vincoli fisici legati a dimensioni, peso, consumo della batteria e ad altri limiti elettrici e meccanici. I microcontrollori al loro interno devono essere progettati tenendo ben presenti queste restrizioni.

Un esempio particolarmente piacevole di microcontrollore è la piattaforma integrata di controllo Arduino, progettata a Ivrea da Massimo Banzi e David Cuartielles. L'obiet-

tivo del progetto era di produrre una piattaforma di calcolo integrata che costasse meno di una pizza quattro stagioni, rendendo così l'hardware facilmente accessibile a studenti e a hobbisti. È stato un compito difficile, perché in Italia le pizze costano veramente poco! Comunque lo scopo è stato raggiunto: un sistema completo Arduino costa meno di 20 dollari!

Questo sistema è un progetto hardware open-source, il che significa che tutti i dettagli sono pubblicati e disponibili gratuitamente in modo che chiunque può costruire (e anche vendere) un sistema Arduino. Arduino è basato sul microcontrollore Atmel AVR 8-bit RISC e la maggior parte delle schede include anche il supporto I / O di base. La scheda è programmata con un linguaggio di programmazione integrato, chiamato Wiring, dotato di tutte le funzioni necessarie per controllare dispositivi real-time. Ciò che rende la piattaforma Arduino divertente da usare è la sua comunità di sviluppo attiva e di grandi dimensioni. Ci sono migliaia di progetti pubblicati che utilizzano Arduino e che vanno da un rilevatore elettronico d'inquinamento ambientale, a una giacca per ciclisti dotata di indicatori di direzione, a un rilevatore di umidità che invia un'email quando una pianta ha bisogno di essere innaffiata, a un aeroplano senza equipaggio. Per saperne di più su Arduino e darvi da fare con progetti personali, visitate www.arduino.cc.

1.3.5 Dispositivi mobili e da gioco

A un gradino più in alto si trovano i dispositivi mobili e le console per videogiochi. Si tratta di normali computer spesso dotati di speciali capacità grafiche e sonore, ma poco espandibili e forniti di software limitato. Inizialmente erano basati su CPU a basso costo per gestire la telefonia e per poter giocare, tramite il televisore, a semplici giochi come il ping pong. Negli anni si sono evoluti in sistemi molto più potenti, rivaleggiando in alcuni aspetti con i personal computer e superandone talvolta le prestazioni.

Per avere un'idea di che cosa ci sia all'interno di questi sistemi consideriamo le specifiche tecniche di tre prodotti diffusi. Come primo esempio, esaminiamo la PlayStation 3 di Sony. È dotata di una CPU multicore proprietaria a 3,2 GHz (chiamata Cell) basata sulla CPU RISC PowerPC e di sette SPE (*Synergistic Processing Elements*, "elementi di elaborazione sinergici") a 128 bit. La PlayStation 3 dispone inoltre di 512 MB di RAM, un chip grafico dedicato Nvidia a 550 MHz e un lettore Blu-ray. Come secondo esempio, consideriamo la console Microsoft Xbox 360. Quest'ultima è dotata di una CPU IBM PowerPC triple-core a 3,2 GHz, di 512 Mb di RAM, di un chip grafico dedicato ATI a 500 MHz, di un lettore DVD e di un hard disk. Il terzo esempio è il tablet Samsung Galaxy. All'interno di questo dispositivo vi sono 2 core ARM a 1 GHz e un processore grafico (integrati sul SoC, *System-on-a-chip*, Nvidia Tegra 2), 1 GB di RAM, una doppia telecamera, un giroscopio a 3 assi e una memoria flash.

Anche se queste macchine non sono potenti quanto i personal computer di fascia alta prodotti nello stesso periodo, esse non sono molto distanti da questi ultimi e sotto alcuni aspetti li superano (un esempio è l'SPE a 128 bit della PlayStation 3, più evoluta della CPU di un qualsiasi PC). La differenza principale tra queste macchine e un PC non risiede tanto nella CPU quanto nel fatto che le prime sono sistemi chiusi. Anche se di solito le console hanno interfacce USB e FireWire, gli utenti non possono espanderle attraverso schede aggiuntive. Inoltre, e forse questo è l'aspetto più importante, queste

piattaforme sono attentamente ottimizzate per un particolare tipo di applicazioni, quelle altamente interattive con grafica 3D e output multimediale. Tutto il resto è secondario. Queste restrizioni hardware e software, la scarsa possibilità di espansione, le memorie di piccole dimensioni, l'assenza di un monitor ad alta risoluzione e un disco fisso piccolo o del tutto assente fanno sì che queste macchine possano essere vendute a prezzi più bassi rispetto ai personal computer. Nonostante queste restrizioni, nel mondo sono stati venduti milioni di questi dispositivi e i volumi di vendita sono in continua crescita.

I dispositivi mobili hanno il requisito aggiuntivo di dover consumare meno energia possibile per funzionare: meno energia usano, più a lungo dura la loro batteria. Questo requisito costituisce un'importante sfida in fase di progetto, perché tablet e smartphone devono essere parsimoniosi nel consumo di energia, ma, allo stesso tempo, soddisfare gli utenti che si aspettano elevate prestazioni, come la grafica 3D, l'elaborazione multimediale in alta definizione e il gioco.

1.3.6 Personal computer

Continuando a salire i gradini della scala degli elaboratori arriviamo ai personal computer, cioè quelle macchine a cui perlopiù si pensa quando si sente pronunciare il termine "computer". Essi comprendono i modelli desktop e quelli portatili. Generalmente sono dotati di alcuni gigabyte di memoria, di un disco fisso capace di immagazzinare terabyte di dati, di un lettore CD-ROM/DVD/Blu-ray, di una scheda audio, di un'interfaccia di rete, di un monitor ad alta risoluzione e di altre periferiche. Sono dotati di sistemi operativi sofisticati, hanno molte possibilità di espansione e dispongono di un'ampia gamma di software disponibile.

Il cuore di ogni personal computer è costituito da una scheda di circuiti stampati che contiene la CPU, la memoria, vari dispositivi di I/O (come un chip audio e talvolta un modem), oltre alle interfacce per tastiera, mouse, disco, rete e ad alcuni slot di espansione. La Figura 1.10 mostra un'immagine di una di queste schede.

I computer portatili sono fondamentalmente dei PC realizzati in dimensioni ridotte. Usano le stesse componenti hardware, seppur costruite in dimensioni inferiori, e possono eseguire lo stesso software dei PC da scrivania. Poiché la maggior parte dei lettori avrà probabilmente una certa familiarità con PC e notebook, non è necessario ulteriore materiale introduttivo.

Ancora un'altra variante di questo tema sono i tablet, come il ben noto iPad. Questi dispositivi non sono altro che normali PC in un corpo più piccolo, con un disco allo stato solido al posto di un disco tradizionale, un touchscreen e una CPU diversa dalle x86. Ma dal punto di vista architettonico, i tablet sono solo notebook con un diverso fattore di forma.

1.3.7 Server

Spesso vengono impiegate versioni potenziate dei personal computer o delle workstation come server di rete, sia per reti locali (di solito all'interno di un'azienda) sia per Internet.

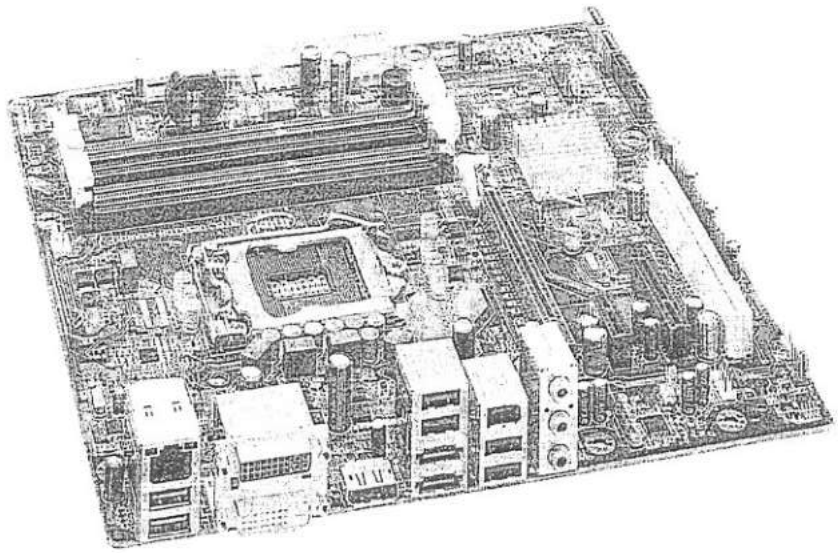


Figura 1.10 Nel cuore di ogni personal computer c'è una scheda di circuiti stampati. Questa immagine è una fotografia della scheda Intel DQ67SW. La fotografia è di proprietà di Intel Coorporation (copyright 2011) e pubblicata con autorizzazione.

Esistono configurazioni mono o multiprocessore, dotate di gigabyte di memoria, terabyte di spazio su hard disk e connessioni di rete ad alta velocità. Alcuni server sono in grado di gestire migliaia di transazioni al secondo.

Dal punto di vista dell'architettura un server monoprocessore non è però molto diverso da un personal computer monoprocessore. È semplicemente più veloce e più grande, ha una maggiore memoria di massa e generalmente ha una connessione alla rete più veloce. I server eseguono gli stessi sistemi operativi dei personal computer, di solito uno dei dialetti di UNIX oppure di Windows.

Cluster

Dato che il rapporto prezzo/prestazioni dei server continua progressivamente a migliorare, negli ultimi anni i progettisti di sistemi hanno cominciato a connettere fra loro un gran numero di queste macchine per formare i cosiddetti cluster. Essi consistono in più sistemi di classe server connessi fra loro mediante reti la cui velocità è dell'ordine dei gigabyte al secondo. Queste macchine eseguono software speciale che permette loro di lavorare in modo congiunto su uno stesso problema, spesso di tipo aziendale, scientifico o ingegneristico. Generalmente le macchine utilizzate vengono chiamate COTS (*Commodity Off The Shelf*, "prodotti pronti per l'uso") cioè elaboratori che chiunque può acquistare in un normale negozio di computer. La principale aggiunta che viene fatta è una connessione di rete ad alta velocità, anche se talvolta si utilizzano normali schede di rete commerciali.

In genere i cluster di grandi dimensioni si trovano in appositi locali o edifici dedicati denominati data center. Questi impianti possono comprendere, da una decina di macchine fino a 100.000, o più. Solitamente, la limitazione è imposta dalla quantità di denaro disponibile. Grazie ai bassi costi, le singole società possono ora possedere cluster per uso interno. Spesso i termini "cluster" e "data center" sono usati in maniera intercambiabile, anche se tecnicamente il primo indica l'insieme dei server, mentre il secondo indica il locale o l'edificio che li ospita.

Il tipico utilizzo di un cluster è quello di Internet Web server. Quando un sito web riceve migliaia di richieste al secondo per le sue pagine, la soluzione più economica è spesso quella di costruire un data center con centinaia, o addirittura migliaia, di server. Le richieste in arrivo vengono suddivise tra i server per consentirne l'elaborazione parallela. Per esempio, Google possiede data center sparsi in tutto il mondo per gestire le funzioni di ricerca; la più grande, a The Dalles in Oregon, è una struttura grande come due campi di calcio. La posizione è stata scelta perché i data center consumano grandi quantità di energia elettrica e The Dalles è sede di una centrale idroelettrica da 2 GW sul fiume Columbia, che può fornire una tale quantità di energia. Si pensa che Google disponga complessivamente di più di 1.000.000 di server nei suoi data center.

Il mercato dei computer è dinamico, con cose che cambiano continuamente. Nel 1960, l'informatica era dominata da giganteschi mainframe (vedi successivamente) che costavano decine di milioni di dollari, a cui gli utenti si collegavano mediante piccoli terminali remoti. Questo modello era fortemente centralizzato. Più avanti, negli anni '80, quando sono comparsi sulla scena i personal computer, milioni di persone ne hanno acquistato uno e il calcolo è stato decentralizzato.

Con l'avvento dei data center, stiamo iniziando a rivivere il passato sotto forma del cloud computing, il mainframe V2.0. L'idea è che tutti avranno uno o più dispositivi semplici (come PC, notebook, tablet e smartphone) che fungono essenzialmente da interfaccia al cloud (cioè, al data center) in cui sono memorizzate tutte le foto, i video, la musica e altri dati dell'utente. In questo modello, i dati sono accessibili da diversi dispositivi ovunque e in qualsiasi momento senza che l'utente debba preoccuparsi di dove siano. Qui, un data center pieno di server ha sostituito un unico grande computer centrale, ma il paradigma è ritornato di nuovo quello di un tempo: gli utenti dispongono di terminali semplici, mentre i dati e la potenza di calcolo sono centralizzati altrove.

Chi può sapere per quanto tempo questo modello sarà valido? Tra una decina d'anni potrebbe facilmente accadere che talmente tante persone avranno memorizzato così tante canzoni, foto e video nel cloud da far diventare l'infrastruttura (wireless) per la comunicazione un vero pantano. Questo potrebbe portare a una nuova rivoluzione: personal computer, dove la gente memorizza i propri dati nelle proprie macchine a livello locale, evitando così il traffico via etere.

Il messaggio da cogliere è che il modello di calcolo più diffuso in una data epoca dipende molto dalla tecnologia, dall'economia e dalle applicazioni disponibili al momento e può cambiare nel momento in cui cambiano questi fattori.

1.3.8 Mainframe

In cima alla scala di computer troviamo i mainframe, computer grandi come una stanza che rievocano gli elaboratori degli anni '60. In molti casi queste macchine sono i diretti discendenti dei mainframe IBM 360. La maggior parte di loro ha una velocità che non è molto più elevata rispetto a quella dei server più potenti, ma tutti i mainframe hanno capacità di I/O maggiori e sono spesso equipaggiati con un vasto numero di dischi, per la memorizzazione di migliaia di gigabyte di dati. Pur essendo costosi vengono ancora utilizzati per via dell'enorme investimento che rappresentano in termini di software, dati, procedure operative e personale specializzato. Molte società ritengono che sia più conveniente pagare una volta ogni tanto alcuni milioni di euro per comprarne uno nuovo, piuttosto che prendere in considerazione lo sforzo necessario a riprogrammare tutte le loro applicazioni per macchine più piccole.

È questa classe di computer che ha portato all'ormai famoso bug del millennio, provocato dal fatto che i programmatori (principalmente in COBOL) degli anni '60 e '70 usavano (per risparmiare memoria) solo due cifre decimali per rappresentare gli anni. Non avrebbero mai pensato che il loro software sarebbe durato trenta o quarant'anni. Grazie all'enorme lavoro profuso per risolvere il problema, il disastro previsto non si è verificato; ciononostante molte società sono ricadute nello stesso errore aggiungendo agli anni altre due cifre decimali. Gli autori prevedono quindi che la fine della civiltà così come l'abbiamo conosciuta avverrà il 31 dicembre 9999, quando l'equivalente di 8000 anni di vecchio codice COBOL smetterà simultaneamente di funzionare.

Negli ultimi anni, Internet ha dato nuova linfa ai mainframe, fino ad allora usati principalmente per l'esecuzione di software vecchio di 40 anni. Queste macchine hanno trovato una nuova nicchia come potenti server Internet gestendo per esempio un massiccio numero di transazioni di commercio elettronico al secondo, in particolare laddove sono richieste enormi basi di dati.

Fino a poco tempo fa esisteva anche un'ulteriore categoria di computer, ancora più potente dei mainframe: i **supercomputer**. Avevano CPU incredibilmente veloci, molti gigabyte di memoria principale e dischi e reti ad alta velocità. Queste macchine venivano utilizzate per imponenti calcoli scientifici e ingegneristici come la simulazione della collisione fra galassie, la sintesi di nuove medicine o la simulazione del flusso di aria attorno alle ali degli aerei. Negli ultimi anni i data center costituiti da componenti commerciali hanno però offerto la stessa potenza computazionale a un costo decisamente inferiore, e i veri supercomputer sono diventati una razza in via d'estinzione.

1.4 Esempi di famiglie di computer

In questo libro focalizzeremo l'attenzione su tre ben note tipologie di architetture: x86, ARM e AVR. La prima è presente in quasi tutti i personal computer (inclusi i PC Windows e Linux, e i Mac) e suoi sistemi server. L'interesse verso i primi è motivato dal fatto che ogni lettore ne ha senza dubbio utilizzato uno, mentre quello verso i server è dovuto al fatto che questi eseguono tutti i servizi di Internet. L'architettura ARM domina il mercato *mobile*; per esempio, la maggior parte degli smartphone e dei tablet si basa su

questi processori. Infine, l'architettura AVR è diffusa nei microcontrollori economici presenti in molti computer integrati. I computer integrati sono invisibili agli utenti, ma controllano macchine, televisori, forni a microonde, lavatrici e praticamente qualsiasi altro dispositivo che costi più di 50 euro.

In questo paragrafo introdurremo brevemente le tre ISA (architetture dell'insieme d'istruzioni) che verranno usate come esempio nel resto di questo libro.

1.4.1 Introduzione all'architettura x86

Nel 1968 Robert Noyce, inventore dell'integrazione dei circuiti su silicio, Gordon Moore, famoso per la legge omonima, e Arthur Rock, un imprenditore di San Francisco, fondarono la Intel Corporation per produrre chip di memoria. Durante il primo anno di attività il fatturato Intel fu di soli 3000 dollari, ma da allora gli introiti sono decollati (Intel è ora il più grande produttore di CPU al mondo).

Nei tardi anni '60 i calcolatori erano grandi macchine elettromeccaniche della grandezza di una moderna stampante laser e del peso di 20 Kg. Nel settembre del 1969 una società giapponese, la Busicom, si rivolse a Intel per richiedere la produzione di 12 chip appositamente progettati per un nuovo calcolatore elettronico. Ted Hoff, l'ingegnere di Intel cui fu affidato l'incarico, analizzò il progetto e si rese conto che, per realizzare le stesse operazioni, avrebbe potuto mettere su un singolo chip una CPU a 4 bit di uso generale e che questa sarebbe stata allo stesso tempo più semplice ed economica. Così nel 1970 nacque il processore 4004, la prima CPU su un chip costituita da 2300 transistor (Faggin et al., 1996).

Occorre notare che né Intel né Busicom si resero subito conto di che cosa avessero appena realizzato. Quando Intel decise che valeva la pena provare a utilizzare il 4004 in altri progetti, propose a Busicom di ricomprare tutti i diritti, restituendole i 60.000 dollari che la società giapponese aveva pagato per lo sviluppo della CPU. L'offerta fu subito accettata e da quel momento Intel poté cominciare a lavorare su una versione a 8 bit del processore, l'8008, introdotto nel 1972. La famiglia Intel, iniziata con il 4004 e l'8008 è illustrata nella Figura 1.11 che riporta per ogni chip la data di rilascio, la frequenza di clock, il numero di transistor e la dimensione della memoria.

Intel non si aspettava una grande richiesta per l'8008 e quindi impostò una linea di produzione su un basso volume. Fra lo stupore generale, il chip riscosse un enorme interesse, tale da spingere Intel a progettare una nuova CPU che aggirasse il principale problema dell'8008, cioè il limite di 16 KB di memoria (dovuto al numero di contatti esterni del chip). Il progetto terminò nel 1974 con la nascita dell'8080, una piccola CPU di uso generale. Esattamente come avvenne per il PDP-8, questo prodotto conquistò di colpo il mercato e divenne improvvisamente un articolo con un mercato di massa. La differenza è che, invece di venderne migliaia come fece DEC, Intel riuscì a venderne milioni.

Nel 1978 comparve l'8086, un'autentica CPU a 16 bit su un solo chip. L'8086 fu progettato per essere simile all'8080, pur senza la completa compatibilità. L'8086 fu seguito dall'8088, che aveva la sua stessa architettura e poteva eseguire gli stessi programmi, ma era dotato di un bus a 8 invece che a 16 bit, rendendolo più lento e allo stesso tempo più economico dell'8086. Quando IBM scelse l'8088 come CPU per il

primo PC IBM, questo processore divenne rapidamente lo standard nel mercato dei personal computer.

Chip	Data	MHz	N. transistor	Memoria	Descrizione
4004	4/1971	0.108	2300	640	Primo microprocessore su un solo chip
8008	4/1972	0.108	3500	16 KB	Primo microprocessore a 8 bit
8080	4/1974	2	6000	64 KB	Prima CPU di uso generale su un solo chip
8086	6/1978	5-10	29.000	1 MB	Prima CPU a 16 bit su un solo chip
8088	6/1979	5-8	29.000	1 MB	Usato nel PC IBM
80286	2/1982	8-12	134.000	16 MB	Introduzione della modalità protetta
80386	10/1985	16-33	275.000	4 GB	Prima CPU a 32 bit
80486	4/1989	25-100	1,2 M	4 GB	Memoria cache da 8 KB integrata
Pentium	3/1993	60-233	3,1 M	4 GB	Due pipeline; istruzioni MMX nei modelli successivi
Pentium Pro	3/1995	150-200	5.5 M	4 GB	Cache integrata a due livelli
Pentium II	5/1997	233-450	7.5 M	4 GB	Pentium Pro con istruzioni MMX
Pentium III	2/1999	650-1400	9,5 M	4 GB	Istruzioni SSE per la grafica 3D
Pentium 4	11/2000	1300-3800	42 M	4 GB	Hyperthreading; ulteriori istruzioni SSE
Core Duo	1/2006	1600-3200	152 M	2 GB	Due core in un singolo circuito stampato
Core	7/2006	1200-3200	410 M	64 GB	Architettura quad-core a 64 bit
Core i7	1/2011	1100-3300	1160 M	24 GB	Processore grafico integrato

Figura 1.11 I membri principali della famiglia di CPU Intel. Le velocità del clock sono misurate in MHz (megahertz) dove 1 Mhz corrisponde a 1 milione di cicli/s.

Né l'8088 né l'8086 potevano indirizzare più di 1 MB di memoria, il che nei primi anni '80 divenne via via un serio problema, al punto che Intel decise di progettare l'80286, una versione più potente, ma ancora compatibile con l'8086. L'insieme base d'istruzioni era essenzialmente lo stesso dell'8086 e dell'8088, ma l'organizzazione della memoria era molto differente e piuttosto complessa, a causa dei requisiti di compatibilità con gli altri processori. L'80286 fu utilizzato nel PC IBM/AT e nei modelli PS/2 di gamma media. Come l'8088 fu anch'esso un grande successo, soprattutto perché era in genere considerato come un 8088 più veloce.

Il successivo passo fu logicamente l'80386, una vera CPU a 32 bit su un chip che vide la luce nel 1985. Come il precedente, questo processore era una versione più o meno compatibile con tutto ciò che risaliva all'8080. Essendo retrocompatibile fu una benedizione per chi aveva necessità di eseguire vecchio software, ma un fastidio per chi avrebbe preferito un'architettura moderna, semplice, ben progettata e non appesantita dagli errori e dalla tecnologia degli anni precedenti.

Quattro anni dopo apparve l'80486. Era in sostanza una versione più veloce dell'80386 con in più un'unità in virgola mobile e una memoria cache di 8 KB sul chip. La **memoria cache** era usata per conservare, all'interno o vicino alla CPU, le parole di memoria utilizzate con più frequenza, di modo da evitare l'accesso (lento) alla memoria centrale. L'80486 aveva anche un supporto multiprocessore predefinito per permettere ai produttori di costruire sistemi composti da più CPU che condividessero una memoria comune.

A quel punto Intel scoprì nel modo peggiore (perdendo una causa legale per violazione di diritti) che i numeri (come 80486) non potevano essere registrati come marchio, e per questo alla generazione successiva fu dato un nome: Pentium (dal termine greco "cinque", ΠΕΝΤΑ). Diversamente dall'80486, che aveva una sola pipeline interna, il Pentium ne aveva due, il che lo aiutò a essere due volte più veloce (le pipeline verranno illustrate in dettaglio nel Capitolo 2).

In seguito, durante la sua corsa produttiva, Intel aggiunse l'insieme d'istruzioni speciali **MMX** (*MultiMedia eXtension*, "estensioni multimediali"). Queste istruzioni furono ideate per velocizzare i calcoli necessari per l'elaborazione di audio e video, rendendo così superflua la presenza di coprocessori multimediali.

Quando apparve la generazione successiva, la gente che sperava di vedere il Sexium (*sex* significa sei in Latino) rimase delusa. Ormai il nome Pentium era talmente conosciuto che gli addetti al marketing lo vollero mantenere e il nuovo chip prese il nome di Pentium Pro.

Nonostante il piccolo cambiamento nel nome, questo processore rappresentò una maggiore discontinuità rispetto al passato. Invece di avere due o più pipeline il Pentium Pro aveva un'organizzazione interna molto diversa e poteva eseguire fino a cinque istruzioni allo stesso tempo.

Un'altra innovazione introdotta dal Pentium Pro fu la memoria cache a due livelli. Nel chip stesso del processore vi erano 8 KB di memoria veloce contenenti le istruzioni usate più di frequente e altri 8 KB dedicati in modo analogo ai dati. All'interno della stesso alloggiamento del Pentium Pro (ma non nel chip stesso) vi era una seconda memoria cache di 256 KB.

Anche se il Pentium Pro era dotato di una cache di grandi dimensioni, mancavano le istruzioni MMX (dato che Intel non riuscì a produrre un così grande chip a un costo accettabile). Quando i progressi tecnologici permisero di avere contemporaneamente sullo stesso chip sia le istruzioni MMX sia la cache, il risultato fu la nascita del Pentium II. Successivamente, per migliorare le prestazioni con la grafica 3D, furono aggiunte ulteriori istruzioni multimediali, chiamate **SSE** (*Streaming SIMD Extensions*, "estensioni per stream SIMD") (Raman et al., 2000). Al nuovo chip fu dato il nome di Pentium III, anche se internamente era in sostanza un Pentium II.

Il Pentium successivo, rilasciato nel novembre 2000, era basato su un'architettura interna differente, ma aveva lo stesso insieme di istruzioni dei precedenti Pentium. Per celebrare l'evento Intel passò dalla numerazione in numeri romani a quella in numeri arabi e lo chiamò Pentium 4. Come al solito il Pentium 4 era più veloce di tutti i suoi

predecessori: la versione a 3.06 GHz introdusse in più una nuova e interessante caratteristica: l'*hyperthreading*. Questa potenzialità permetteva ai programmi di dividere il loro lavoro in due flussi di controllo che il Pentium 4 poteva eseguire in parallelo, accelerandone l'esecuzione. Inoltre vennero aggiunte altre istruzioni SSE per velocizzare ulteriormente l'elaborazione audio e video.

Nel 2006 Intel ha cambiato il marchio da Pentium a Core e ha lanciato un chip dual core, il **Core 2 Duo**. Quando Intel decise di volere anche una versione a core singolo del chip, più economica, iniziò semplicemente a vendere i Core 2 Duo con un core disabilitato, perché sprecare un po' di silicio su ogni chip prodotto era in ultima analisi più economico rispetto a sostenere enormi spese per il progetto e il test di un nuovo chip prodotto da zero. La serie Core ha continuato a evolversi, con l'i3, l'i5, e l'i7 come varianti per i computer di bassa, media e alta fascia, rispettivamente, ai quali seguiranno sicuramente altri modelli. La Figura 1.12 mostra una foto del Core i7. Questo chip contiene, in realtà, otto core, ma, fatta eccezione per la versione Xeon, ne sono abilitati soltanto sei. Grazie a questo approccio un chip con uno o due core difettosi può essere ancora venduto disabilitando gli eventuali core malfunzionanti. Ogni core ha la propria cache di primo e di secondo livello, ma vi è anche una cache condivisa di livello 3 (L3), utilizzata da tutti i core. Parleremo della cache in dettaglio più avanti nel libro.

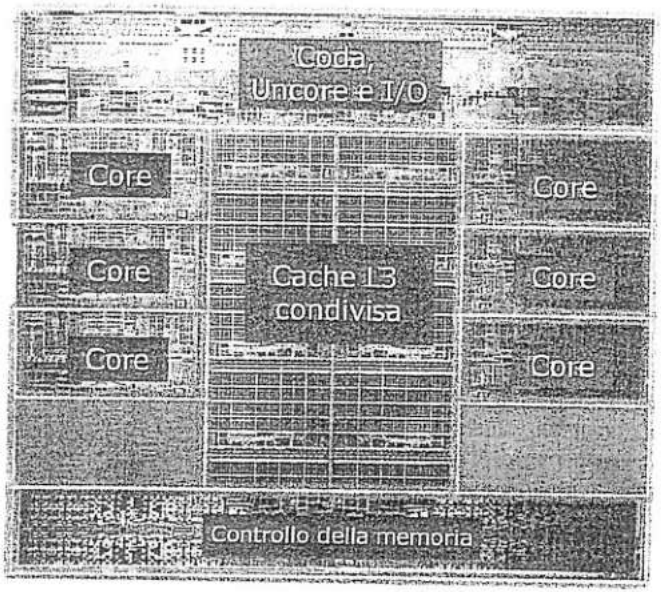


Figura 1.12 Il microprocessore Intel Core i7-3960X, © 2011 Intel Corporation. Il chip misura 21×21 mm² e contiene 2,27 miliardi di transistor (con "Uncore" si intendono le funzionalità esterne al core).

Intel, oltre alla linea principale di CPU per computer desktop di cui abbiamo parlato sinora, ha prodotto varianti di alcuni processori Pentium dedicate a mercati specifici. Nel 1998 ha introdotto una nuova linea di prodotti chiamata **Celeron**, in sostanza una versione più economica e con prestazioni ridotte del Pentium II, pensata per i PC di fascia bassa. Dato che l'architettura del Celeron è la stessa del Pentium II, non verrà trattata nel corso del libro. Nel giugno del 1998 Intel ha introdotto anche una versione speciale del Pentium II dedicata al mercato professionale. Questo processore, chiamato **Xeon**, aveva una cache più grande, un bus più veloce e un miglior supporto multiprocessore; a parte questi miglioramenti si trattava di un normale Pentium II, quindi neanch'esso verrà trattato specificatamente. Anche del Pentium III è stata prodotta una versione Xeon, come per i chip più recenti. Sugli ultimi processori una delle funzionalità aggiuntive dello Xeon è la presenza di più core.

Nel 2003 Intel ha messo in commercio il Pentium M (M per Mobile), progettato per i computer portatili. Questo chip faceva parte dell'architettura Centrino, i cui obiettivi erano la riduzione del consumo energetico per prolungare la durata delle batterie, la produzione di computer più piccoli e leggeri e la predisposizione alla connessione di rete senza fili secondo lo standard IEEE 802.11 (WiFi). Il Pentium M consumava molto meno ed era più piccolo del Pentium 4, due caratteristiche che avrebbero presto permesso a lui e ai suoi successori di soppiantare la micro-architettura Pentium 4 nei futuri progetti Intel.

Tutti i processori Intel sono retrôcompatibili con i loro predecessori fino all'8086. In altre parole un Pentium 4 o un Core possono eseguire programmi dell'8086 senza alcuna modifica. Per Intel questa compatibilità è sempre stata un requisito di progettazione, al fine di non far perdere agli utenti i loro investimenti in software. Il Core è però quattro ordini di grandezza più complesso dell'8086 ed è quindi naturale che possa fare molte cose in più. L'aggiunta graduale di tutte queste estensioni ha portato a un'architettura che non è così elegante come quella che si otterrebbe dando a qualcuno 42 milioni di transistor e il compito di progettarla ex novo.

È interessante notare che la legge di Moore, sebbene sia stata a lungo associata al numero di bit di memoria, risulta valida anche per i chip della CPU. Il grafico nella Figura 1.13 rappresenta in scala semi-logaritmica il numero di transistor di un chip (Figura 1.8) rispetto alla sua data di apparizione e mostra che la legge di Moore viene rispettata anche in base a questi dati.

Anche se con ogni probabilità la legge di Moore continuerà a essere valida per alcuni anni a venire, cominciano a profilarsi delle nuvole all'orizzonte per via dei problemi legati alla dissipazione del calore. L'uso di transistor più piccoli permette di raggiungere frequenze più alte, ma allo stesso tempo richiede una tensione più elevata. L'energia consumata e il calore dissipato sono proporzionali al quadrato della tensione, ragion per cui raggiungere velocità più elevate significa avere più calore da smaltire. Un processore Pentium 4 a 3,6 GHz consuma 115 W, il che significa che diventa caldo all'incirca quanto una lampadina da 100 W. Aumentare la frequenza di clock peggiora il problema.

Nel novembre del 2004 Intel ha dovuto rinunciare al Pentium 4 a 4 GHz a causa dei problemi dipendenti dalla dissipazione del calore. L'uso di ventole di grandi dimensioni può aiutare, ma il loro rumore infastidisce gli utenti e il raffreddamento ad acqua, usato nei mainframe di grandi dimensioni, non è un'opzione attualmente praticabile per i desktop (e ancor meno per quelli portatili).

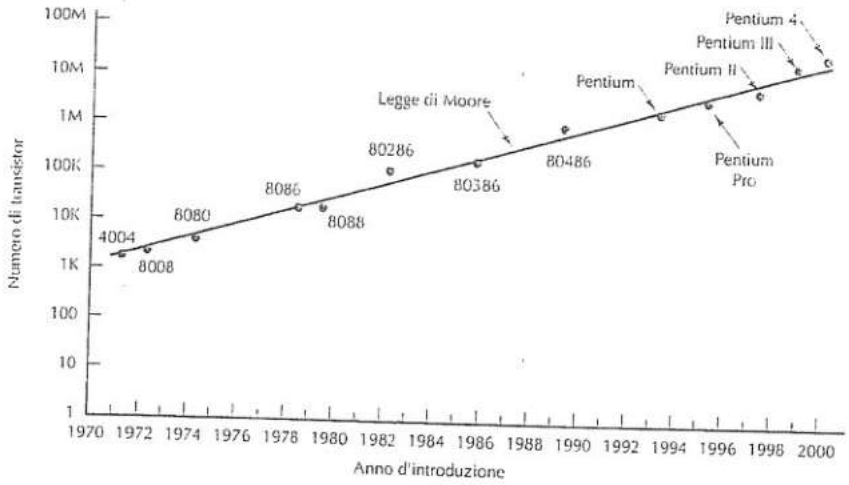


Figura 1.13 La legge di Moore per i chip della CPU (Intel).

Per questo motivo la finora inarrestabile marcia delle velocità di clock potrebbe essersi conclusa, almeno finché gli ingegneri di Intel non escogiteranno un modo per liberarsi efficientemente del calore generato. Nei progetti attuali di Intel vengono collocate due o più CPU su un singolo chip, insieme a una grande cache condivisa. Per via della relazione tra tensione e frequenza di clock, due CPU sullo stesso chip consumano molta meno potenza di una sola CPU con velocità doppia. Ne consegue che in futuro il guadagno fornito dalla legge di Moore potrebbe essere via via sfruttato per inserire nel chip un numero maggiore di core e cache sempre più grandi, piuttosto che per raggiungere velocità di clock sempre più elevate. Lo sfruttamento delle potenzialità dei multiprocessori mette i programmatori di fronte a grandi sfide, perché a differenza delle sofisticate micro-architetture monoprocessore del passato che permettevano di ottenere migliori prestazioni dai programmi esistenti, i multiprocessori richiedono al programmatore di orchestrare in modo esplicito l'esecuzione parallela, utilizzando thread, semafori, memoria condivisa e altre tecnologie portatrici di bug... e di mal di testa.

1.4.2 Introduzione all'architettura ARM

Nei primi anni '80, la società britannica Acorn Computer, facendo seguito al successo ottenuto dal loro personal computer a 8 bit BBC Micro, iniziò a lavorare su una seconda macchina con la speranza di competere con il PC IBM che da poco era apparso sul mercato. La BBC Micro era basata sul processore a 8-bit 6502. Steve Furber e i suoi colleghi della Acorn ritennero che il 6502 non aveva abbastanza muscoli per competere con il PC IBM a 16 bit 8086; iniziarono così a guardare le opzioni offerte dal mercato e decisero che erano troppo limitate.

Ispirati dal progetto Berkeley RISC, in cui un piccolo team aveva progettato un processore molto veloce (che alla fine portò all'architettura SPARC), decisero di costruire per il progetto una propria CPU e lo chiamarono Acorn RISC Machine (o ARM, che sarà poi ribattezzata Advanced RISC machine dopo la separazione di ARM da Acorn). Il progetto fu completato nel 1985, aveva istruzioni e dati a 32 bit e uno spazio di indirizzamento a 26 bit, ed era prodotto dalla VLSI Technology.

La prima architettura ARM (chiamata ARM2) fece la sua apparizione nel personal computer Acorn Archimedes. Questo PC era molto veloce ed economico per i suoi tempi, viaggiava a 2 MIPS (milioni di istruzioni al secondo) e costava solo 899 sterline inglesi al momento del lancio. La macchina ebbe una grande diffusione nel Regno Unito, in Irlanda, in Australia e in Nuova Zelanda, in particolare nelle scuole.

Dopo il successo di Archimedes, Apple contattò Acorn per sviluppare un processore ARM per l'imminente progetto Apple Newton, il primo computer palmare. Per focalizzarsi meglio sul progetto, il team di ARM lasciò Acorn per creare una nuova società, denominata Advanced RISC Machines (ARM). Il nuovo processore ARM fu chiamato ARM 610 e alimentava l'Apple Newton nel momento del suo lancio, nel 1993. A differenza del progetto ARM originale, questo nuovo processore ARM integrava 4 KB di memoria cache, migliorando significativamente le prestazioni. Sebbene l'Apple Newton non sia stato un grande successo, l'ARM 610 ha visto altre applicazioni vincenti, tra cui il PC RISC Acorn.

Nella metà degli anni '90, ARM collaborò con Digital Equipment Corporation per sviluppare una versione di ARM ad alta velocità e basso consumo, destinata ad applicazioni mobili che non potevano permettersi alti consumi, come i PDA. Insieme realizzarono il progetto StrongARM, che fin dalla sua prima apparizione si mise in evidenza nel settore per la sua alta velocità (233 MHz) e per la bassissima potenza richiesta (1 Watt). Questa efficienza è stata ottenuta attraverso un progetto semplice ed elegante che includeva due cache da 16 KB per istruzioni e dati. Il processore StrongARM e i suoi successori DEC hanno avuto un discreto successo sul mercato, trovando applicazione in un buon numero di PDA, set-top box, dispositivi multimediali e router.

Il progetto ARM più degno di attenzione è probabilmente l'ARM7, rilasciato da ARM nel 1994 e ancora oggi largamente utilizzato. Il progetto utilizzava cache separate per istruzioni e dati, e incorporava l'insieme di istruzioni a 16-bit denominato Thumb. Questo insieme di istruzioni è una versione ristretta dell'intero insieme di istruzioni ARM a 32 bit che consente ai programmatori di codificare molte delle operazioni più comuni in piccole istruzioni a 16 bit, riducendo in modo significativo la quantità di memoria utilizzata dai programmi. Il processore ha ben figurato in una vasta gamma di applicazioni embedded di bassa-media fascia, quali tostapane e controllo motori, ma anche nella console portatile Nintendo Game Boy Advance.

A differenza di molte società di computer, ARM non produce nessuno dei suoi microprocessori, ma crea piuttosto progetti, librerie e strumenti di sviluppo per ARM, che concede in licenza a progettisti di sistemi e produttori di chip. Per esempio, la CPU utilizzata nel tablet Android Samsung Galaxy Tab è un processore basato su ARM. Il Galaxy Tab utilizza il processore system-on-chip Tegra 2, che contiene due processori ARM Cortex-A9 e una unità di elaborazione grafica Nvidia GeForce. I core del Tegra 2

sono stati progettati da ARM, integrati in un SoC di progettazione Nvidia e prodotti dalla Taiwan Semiconductor Manufacturing Company (TSMC). Si tratta di una impressionante collaborazione fra società di paesi diversi in cui tutte le aziende hanno contribuito al valore finale del prodotto.

La Figura 1.14 mostra una foto dello stampo del SoC Tegra 2 di Nvidia. Il progetto contiene tre processori ARM: due core ARM Cortex-A9 a 1.2-GHz più un core ARM7. I Cortex-A9 sono core a doppia emissione, con esecuzione fuori sequenza, dotati di 1 MB di cache L2 e di supporto per multiprocessing con memoria condivisa. Abbiamo citato molte parole chiave che incontreremo nei prossimi capitoli. Per ora, è sufficiente sapere che queste caratteristiche rendono il design molto veloce! L'ARM7 è un core ARM più vecchio e più piccolo utilizzato per la configurazione del sistema e la gestione del consumo energetico. La grafica è gestita da un'unità di elaborazione grafica (GPU) GeForce a 333 MHz ottimizzata per il funzionamento a bassa potenza. Sono inoltre inclusi nel chip Tegra 2 encoder/decoder video, un processore audio e un interfaccia video HDMI.

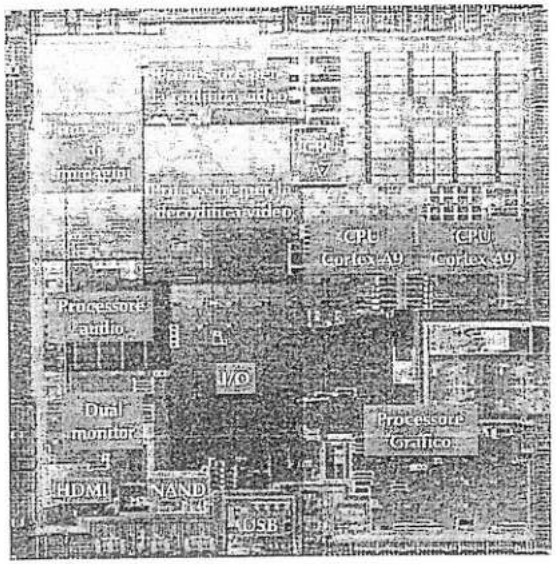


Figura 1.14 Il SoC Nvidia Tegra 2. Copyright 2011, Nvidia Corporation. Fotografia pubblicata con autorizzazione.

L'architettura ARM ha avuto grande successo nel mercato dei dispositivi a basso consumo, mobili e integrati. Nel gennaio 2011, ARM ha annunciato di aver venduto 15 miliardi di processori dall'inizio della sua attività e ha comunicato che le vendite continuavano a crescere. Anche se creata su misura per i mercati di fascia bassa, l'architettura ARM ha le capacità computazionali necessarie per fare bene in qualunque mercato e ci sono indizi che i suoi orizzonti si stiano espandendo. Per esempio, nel mese di ottobre 2011 è stato annunciato un processore ARM a 64-bit, mentre nel gennaio 2011 Nvidia ha annunciato il "Progetto Denver", un system-on-a-chip basato su ARM in fase di sviluppo destinato ai server e ad altri mercati. Il progetto utilizzerà più processori ARM a 64 bit e una GPU a uso generale (GPGPU). Gli aspetti del progetto rivolti al basso consumo contribuiranno a ridurre le esigenze di raffreddamento in *server farm* "fattoria di server" e data center.

1.4.3 Introduzione all'architettura AVR

Il nostro terzo esempio è molto diverso dal primo (l'architettura x86, utilizzata in personal computer e server) e dal secondo (l'architettura ARM, utilizzati in PDA e smartphone). Si tratta dell'architettura AVR ed è utilizzata in sistemi integrati di fascia bassa. La storia di AVR iniziò nel 1996 al Norwegian Institute of Technology, dove gli studenti Alf Egil-Bogen e Vegard Wollan progettarono una CPU RISC a 8-bit chiamata AVR. E' stato riportato che il nome deriva da "(A)lf e (V)egard (R)ISC processor". Poco dopo che il progetto fu completato, Atmel lo acquistò e diede vita alla Atmel Norway, dove due architetti continuarono a perfezionare la progettazione del processore. Atmel lanciò il suo primo microcontrollore AVR, l'AT90S1200, nel 1997. Per facilitare la sua adozione da parte dei progettisti di sistemi, fecero in modo che la piedinatura (*pinout*) fosse esattamente la stessa di quella del processore Intel 8051, uno dei microcontrollori più popolari del momento. Oggi c'è un grande interesse per l'architettura AVR, perché è il cuore della diffusa piattaforma integrata open source Arduino.

L'architettura AVR viene realizzata in tre classi di microcontrollori, come mostra la Figura 1.15. La classe più bassa, la tinyAVR, è progettata per le applicazioni con maggiori vincoli in termini di spazio, potenza e costi. Essa comprende una CPU a 8-bit, il supporto di base per I/O digitale e il supporto per un ingresso analogico (per esempio, la lettura della temperatura da un termostato). Il tinyAVR è così piccolo che i suoi pin devono assolvere un doppio compito; possono cioè essere riprogrammati in fase di esecuzione per una qualsiasi delle funzioni digitali o analogiche supportate dal microcontrollore. Il megaAVR, che si trova nel sistema open-source Arduino, è dotato anche di supporto per l'I/O seriale, di orologi interni e di uscite analogiche programmabili. Il top della gamma, in questa fascia bassa, è il microcontrollore AVR XMEGA, che incorpora anche un acceleratore per operazioni di crittografia e il supporto integrato per interfacce USB.

Chip	Flash	EEPROM	RAM	Pin	Caratteristiche
tinyAVR	0,5–16 KB	0–512B	32–512B	6–32	Molto piccolo, I/O digitale, ingresso analogico
megaAVR	8–256 KB	0,5–4 KB	0,25–8 KB	28-100	Molte periferiche, uscita analogica
AVR XMEGA	16–256 KB	1–4 KB	2–16 KB	44–100	Crittografia accelerata, USB

Figura 1.15 Classi di microcontrollori della famiglia AVR.

Insieme a varie periferiche aggiuntive, ogni classe di processori AVR include varie risorse di memoria. I microcontrollori montano generalmente tre tipi di memoria: flash, EEPROM e RAM. La memoria flash è programmabile con l'ausilio di un'interfaccia esterna e di alte tensioni: è qui dove il codice del programma e i dati sono memorizzati. La RAM Flash è non volatile, quindi mantiene ciò che in essa è stato scritto anche se il sistema viene spento. Come la memoria flash, anche la EEPROM è non volatile, ma a differenza della RAM flash, può essere modificata dal programma durante l'esecuzione. In questa memoria un sistema integrato mantiene le informazioni di configurazione dell'utente, per esempio l'impostazione della visualizzazione dell'ora in formato 12 o 24 ore. Infine, la RAM è la memoria in cui sono mantenute le variabili dei programmi in esecuzione. Questa memoria è volatile, quindi ogni valore memorizzato viene perso quando si toglie l'alimentazione al sistema. Studieremo in dettaglio le tipologie di RAM, volatili e non volatili, nel prossimo capitolo.

La ricetta per il successo nel business dei microcontrollori è di stipare nel chip tutto quello di cui ci potrebbe essere bisogno (anche il lavello della cucina, se si riuscisse a ridurlo a un millimetro quadrato) e poi inserirlo in un supporto economico, di piccole dimensioni e con pochi pin. Integrando molte funzioni nel microcontrollore sarà possibile farlo lavorare su diverse applicazioni, rendendolo piccolo ed economico potrà essere utilizzato in diversi contesti. Per avere un'idea di quante cose vengono cablate in un microcontrollore moderno, diamo uno sguardo alle periferiche incluse nell'Atmel megaAVR-168.

1. Tre timer (due a 8 bit e uno a 16).
2. Orologio in tempo reale con oscillatore.
3. Sei canali di modulazione in ampiezza di impulsi usati, per esempio, per controllare l'intensità luminosa o la velocità del motore.
4. Otto canali di conversione analogico/digitale utilizzati per leggere i livelli di tensione.
5. Ricetrasmettitore seriale universale.
6. Interfaccia seriale I2C, uno standard comune per l'interfacciamento ai sensori.
7. Timer "Watchdog" programmabile che rileva quando il sistema è bloccato.
8. Comparatore analogico integrato che confronta due tensioni di ingresso.
9. Rilevatore di sbalzi di tensione che interrompe il sistema quando viene a mancare l'alimentazione.
10. Oscillatore interno programmabile per pilotare il clock della CPU.

1.5 Unità metriche

Per evitare ogni tipo di confusione, vale la pena precisare che in questo libro, come di solito avviene nell'informatica, si usano le unità metriche e non le tradizionali unità di misura inglesi.

Nella Figura 1.16 sono elencati i principali prefissi metrici. Di solito sono abbreviati utilizzando le loro lettere iniziali e si usano lettere maiuscole per i multipli, come KB, MB, e così via, e minuscole per i sottomultipli. Quindi, per esempio, una linea di comunicazione a 1 Mbps trasmette 10^6 bit/s e un clock a 100 ps ha un tic ogni 10^{-10} secondi. Dato che sia *milli* sia *micro* iniziano con la lettera "m" si utilizza "m" per milli e "μ" (la lettera greca mu) per micro.

Occorre inoltre sottolineare che, nella pratica comune, queste unità hanno un significato leggermente diverso quando vengono utilizzate come misure della dimensione di memorie, dischi, file e database. In questi casi, dato che le dimensioni delle memorie sono sempre potenze di due, K assume il significato di 2^{10} cioè 1024 invece che di 10^3 = 1000. Quindi una memoria da 1 KB contiene 1024 e non 1000 byte. Analogamente una memoria da 1 MB contiene 2^{20} = 1.048.576 byte, una memoria da 1 GB ne contiene 2^{30} = 1.073.741.824, e un database da 1 Tbyte contiene 2^{40} = 1.099.511.627.776 byte. Tuttavia, una linea di comunicazione a 1 Kbps può trasmettere 1000 bit al secondo e una LAN a 10 Mbps funziona a 10.000.000 bit/s, dato che questi tassi non sono potenze di due. Sfortunatamente molti tendono a confondere questi due sistemi, specialmente per le dimensioni dei dischi. Per eliminare queste ambiguità le organizzazioni di standardizzazione hanno introdotto i nuovi termini kibibyte per 2^10 byte, mebibyte per 2^20 byte, gibibyte per 2^30 byte e tebibyte per 2^40 byte, ma l'industria sta rallentando il processo di adozione di questa terminologia. Noi pensiamo che fino a quando queste nuove parole non diventano di uso comune sia meglio continuare a usare i simboli KB, MB, GB e TB per indicare rispettivamente 2^{10}, 2^{20}, 2^{30} e 2^{40} byte e i simboli Kbps, Mbps, Gbps e Tbps per indicare rispettivamente 10^3, 10^6, 10^9 e 10^{12} bit/s.

Esp.	Valore esplicito	Prefisso	Esp.	Valore esplicito	Prefisso
10^{-3}	0,001	milli	10^{3}	1.000	Kilo
10^{-6}	0,000001	micro	10^{6}	1.000.000	Mega
10^{-9}	0,000000001	nano	10^{9}	1.000.000.000	Giga
10^{-12}	0,000000000001	pico	10^{12}	1.000.000.000.000	Tera
10^{-15}	0,000000000000001	femto	10^{15}	1.000.000.000.000.000	Peta
10^{-18}	0,000000000000000001	atto	10^{18}	1.000.000.000.000.000.000	Exa
10^{-21}	0,000000000000000000001	zepto	10^{21}	1.000.000.000.000.000.000.000	Zetta
10^{-24}	0,000000000000000000000001	yocto	10^{24}	1.000.000.000.000.000.000.000.000	Yotta

Figura 1.16 Principali prefissi metrici.

1.6 Organizzazione del libro

Questo libro tratta dei computer multilivello (che comprendono praticamente tutti i computer moderni) e del modo in cui sono organizzati. Verranno esaminati in notevole dettaglio quattro livelli: il livello logico digitale, il livello di micro-architettura, il livello ISA e il livello macchina del sistema operativo. Alcuni degli aspetti che esamineremo saranno la progettazione complessiva del livello (e il motivo per cui è stato progettato in quel modo), i tipi d'istruzioni e dati disponibili, l'organizzazione e l'indirizzamento della memoria e il metodo tramite il quale il livello è stato implementato. Lo studio di questi argomenti, e altri simili, è chiamato organizzazione, o architettura, del computer.

Ci preoccuperemo principalmente dei concetti piuttosto che dei dettagli e degli aspetti matematici. Per questa ragione alcuni degli esempi saranno fortemente semplificati, al fine di porre l'enfasi sulle idee centrali.

Nel corso del libro useremo le architetture x86, ARM e AVR come esempi pratici, in modo da far capire come i principi presentati nel testo possono essere, e sono, messi in pratica nella realtà. Questi tre esempi sono stati scelti in base a molteplici ragioni. In primo luogo sono tutti ampiamente utilizzati e con ogni probabilità il lettore può avere accesso ad almeno uno di loro. In secondo luogo, ciascuno ha una sua propria e determinata architettura, il che pone le basi per effettuare confronti e incoraggia un approccio basato su domande come: "Quali sono le alternative?". Spesso i libri che trattano di una sola macchina lasciano al lettore la sensazione che gli sia stata svelata "l'unica e corretta progettazione di un computer"; ciò è assurdo alla luce dei molti compromessi e delle decisioni arbitrarie che un progettista è obbligato a compiere. Si incoraggia il lettore a studiare questi computer, ed eventualmente altri, con un occhio critico; lo si spinge a cercare di capire il motivo per cui le cose sono in un determinato modo, oltre che a chiedersi come sarebbero potute essere diversamente, invece di accettarle passivamente.

Occorre chiarire fin dall'inizio che questo libro non spiega come programmare le architetture x86, ARM o AVR. Quando sarà necessario, queste macchine saranno usate a scopo dimostrativo, senza alcuna pretesa di essere completi. I lettori interessati a un'introduzione esauriente a una di queste macchine dovrebbero consultare le pubblicazioni specifiche.

Il Capitolo 2 è un'introduzione ai componenti base dei computer: processori, memorie e dispositivi di input/output. Si intende dare una panoramica dell'architettura di un sistema oltre a un'introduzione ai capitoli successivi.

I Capitoli da 3 a 6 trattano individualmente i livelli mostrati nella Figura 1.2. La trattazione sarà effettuata dal basso verso l'alto, dato che le macchine sono state tradizionalmente progettate in questo modo.

La progettazione di un generico livello è in gran parte determinata dalle proprietà del livello sottostante e per questo è difficile capire un livello a meno di non avere prima compreso a fondo i precedenti. Inoltre sembra più istruttivo procedere dai livelli più semplici fino a quelli più complessi, e non viceversa.

Il Capitolo 3 riguarda il livello logico digitale, il vero e proprio hardware della macchina. Si spiega che cosa sono le porte logiche e come possono essere combinate per formare circuiti funzionali. Si introduce anche l'algebra booleana, uno strumento per l'analisi dei circuiti digitali. Vengono spiegati inoltre i bus del computer e in particolare il popolare bus PCI. Nel capitolo vengono presentati numerosi esempi tratti dal mondo reale, tra cui i tre casi di studio sopracitati.

Il Capitolo 4 introduce l'architettura e il controllo del livello di micro-architettura. Ricorrendo anche a vari esempi ci si concentrerà sulla funzione principale di questo livello, che consiste nell'interpretare le istruzioni del livello 2 per quello superiore. Il capitolo contiene anche una trattazione del livello di micro-architettura di alcune macchine reali.

Il Capitolo 5 esamina il livello ISA, quello che molti rivenditori di computer definiscono come il linguaggio macchina. In questo capitolo analizzeremo in dettaglio le tre macchine d'esempio.

Il Capitolo 6 si occupa delle istruzioni, dell'organizzazione della memoria e dei meccanismi di controllo presenti nel livello macchina del sistema operativo. Gli esempi utilizzati sono Windows (diffuso sui sistemi desktop basati su x86) e UNIX, usato su molti sistemi basati su x86 e ARM.

Il Capitolo 7 riguarda il livello del linguaggio assemblativo e tratta sia del linguaggio sia del processo di assemblaggio. Viene inoltre introdotto l'argomento del collegamento (*linking*).

Il Capitolo 8 tratta dei computer paralleli, un argomento la cui importanza al giorno d'oggi è sempre più grande. Alcuni di questi computer hanno più CPU che condividono una memoria comune. Altri invece sono costituiti da varie CPU, ma senza condivisione di memoria.

Alcuni sono supercomputer, altri sono sistemi su singolo chip, mentre altri ancora sono cluster.

Il Capitolo 9 contiene un elenco alfabetico di riferimenti bibliografici. Le letture consigliate sono riportate sul sito web del libro, all'indirizzo: www.prenhall.com/tanenbaum.

PROBLEMI

1. Si spieghino con parole proprie i seguenti termini:
   a. traduttore (compilatore)
   b. interprete
   c. macchina virtuale.
2. Ha significato che un compilatore generi output per il livello di micro-architettura invece che per il livello ISA? Si analizzino i pro e i contro di tale ipotesi.
3. È possibile immaginare un computer multilivello in cui il livello dei dispositivi e i livelli logico digitali non siano i livelli più bassi? Si motivi la risposta.
4. Si consideri un computer in cui tutti i livelli siano diversi. Ciascun livello possiede istruzioni che sono m volte più potenti di quelle del livello sottostante; cioè un'istruzione del livello r può compiere il lavoro di m istruzioni del livello $r-1$. Se un programma del livello 1 richiede k secondi per essere eseguito, quanto tempo impiegheranno gli equivalenti programmi dei livelli 2, 3 e 4, assumendo che siano necessarie n istruzioni del livello r per interpretare una singola istruzione del livello $r+1$?
5. Alcune istruzioni del livello macchina del sistema operativo sono identiche alle istruzioni del linguaggio ISA. Queste istruzioni sono eseguite direttamente dal microprogramma o dall'hardware invece che dal sistema operativo. Alla luce della risposta data al problema precedente, perché questo avviene?
6. Si consideri un computer con interpreti identici ai livelli 1, 2 e 3. Un interprete impiega n istruzioni per prelevare, esaminare ed eseguire un'istruzione. Se un'istruzione del livello 1 richiede k nanosecondi per essere eseguita, quanto tempo impiega un'istruzione ai livelli 2, 3 e 4?
7. In che senso l'hardware e il software sono equivalenti? E in che senso non lo sono?
8. La *difference engine* di Babbage aveva un unico programma, fisso, che non poteva essere modificato. È essenzialmente la stessa cosa di un moderno CD-ROM il cui contenuto non può essere cambiato? Si motivi la risposta.
9. Una delle conseguenze dell'idea di von Neumann di memorizzare i programmi in memoria è che anch'essi possono essere modificati, esattamente come i dati. È possibile immaginare un esempio in cui questa funzione potrebbe essere utile? (Suggerimento: si pensi a operazioni aritmetiche sugli array).

10. Il rapporto tra le prestazioni del 360 model 75 e del 360 model 30 era di 50 volte, mentre il ciclo di clock era solo 5 volte più veloce. Come si spiega questa differenza?

11. Nella Figura 1.5 e nella Figura 1.6 sono mostrati due elementari progetti di un sistema. Si descriva come potrebbero avvenire gli input e output in ciascuno di loro. Quale dei due può potenzialmente fornire migliori prestazioni generali del sistema?

12. Si supponga che ogni giorno ciascuno dei 300 milioni di cittadini americani consumi due confezioni di un certo articolo a cui è associata un'etichetta RFID. Quante etichette RFID devono essere prodotte annualmente per soddisfare la domanda? Qual è il costo totale delle etichette, se ognuna di loro costa un penny? Considerando l'ammontare del Prodotto Interno Lordo questa quantità di denaro potrebbe oppure no essere un ostacolo all'impiego di etichette RFID per ogni confezione messa in vendita?

13. Si citino tre elettrodomestici che potrebbero funzionare mediante una CPU integrata.

14. A un certo momento della storia un transistor su un microprocessore arrivò ad avere un diametro di 1 dm. Secondo la legge di Moore, che dimensioni avrebbe assunto nell'anno successivo?

15. È stato mostrato che la legge di Moore non si applica solo alla densità dei semiconduttori, ma predice anche l'aumento dello spazio occupato e la diminuzione del tempo d'esecuzione di una simulazione. Si mostri che per una simulazione di meccanica dei fluidi che impiega 4 ore per essere eseguita su una macchina di oggi sarà necessaria 1 ora di esecuzione su una macchina costruita tra 3 anni e solo 15 minuti su una macchina costruita tra 6 anni. Si mostri poi che una simulazione molto onerosa con un tempo di esecuzione stimato di 5 anni terminerebbe prima se aspettassimo 3 anni prima di iniziare l'esecuzione.

16. Nel 1959, un IBM 7090 poteva eseguire circa 500.000 istruzioni al secondo, aveva una memoria di 32.768 parole di 36 bit e costava 3 milioni di dollari. Si confronti questa macchina con una macchina recente e si determini quanto migliore è il computer attuale moltiplicando il rapporto tra le dimensioni della memoria con quello tra le velocità e dividendo questa quantità per il rapporto tra i prezzi. Vediamo ora ciò che lo stesso guadagno avrebbe portato nell'aviazione nello stesso periodo di tempo. Il Boeing 707 è stato consegnato alle compagnie aeree, in notevoli quantità, nel 1959. La sua velocità era di 950 Km/h, poteva inizialmente trasportare 180 passeggeri e costava 4 milioni di dollari. Quali sarebbero ora la velocità, la capacità e il costo di un aeromobile se avesse ottenuto gli stessi guadagni che ha ottenuto un computer? Si indichino le assunzioni fatte su velocità, dimensione della memoria e prezzo.

17. L'evoluzione del settore dei computer è spesso ciclica. In origine, gli insiemi delle istruzioni erano cablati, poi si è passati al microprogrammata, quindi, con l'avvento delle macchine RISC si è ripreso a cablarli. In origine, il calcolo era centralizzato su mainframe di grandi dimensioni. Si citino due sviluppi allo scopo di mostrare che anche in questo caso si è avuto un comportamento ciclico.

18. La diatriba legale su chi ha inventato il computer fu risolta nell'aprile del 1973 dal giudice Earl Larson; egli si occupò della causa legale di violazione di brevetto intentata dalla Sperry Rand Corporation, società che aveva comprato i brevetti dell'elaboratore ENIAC. La posizione della Sperry Rand era che chiunque avesse prodotto un computer avrebbe dovuto pagarle i diritti, in quanto proprietaria dei brevetti chiave. Il caso arrivò in giudizio nel giugno 1971 e furono depositati più di 30.000 documenti. La trascrizione degli atti superò le 20.000 pagine. Utilizzando le vaste informazioni disponibili su Internet si studi il caso più approfonditamente e si scriva una relazione sui suoi aspetti tecnici. Che cosa sosteneva il brevetto di Eckert e Mauchley e perché il giudice ritenne che il loro sistema era basato sul precedente lavoro di Atanasoff?

19. Si scelgano le tre persone che si ritiene abbiano avuto la maggiore influenza nella creazione dell'hardware moderno; si scriva quindi una breve relazione che descriva il loro contributo e motivi la scelta effettuata.

20. Si scelgano le tre persone che si ritiene abbiano avuto la maggiore influenza nella creazione del software di sistema moderno; si scriva quindi una breve relazione che descriva il loro contributo e motivi la scelta effettuata.

21. Si scelgano le tre persone che si ritiene abbiano avuto la maggiore influenza nella creazione dei più trafficati siti web moderni; si scriva quindi una breve relazione che descriva il loro contributo e motivi la scelta effettuata.

CAPITOLO 2

Organizzazione dei sistemi di calcolo

Un calcolatore digitale è un sistema in cui processori, memorie e dispositivi periferici sono connessi tra loro. Questo capitolo fornisce un'introduzione a questi tre componenti e alle loro interconnessioni, servendo da base per l'analisi dettagliata dei singoli livelli che verrà data nei cinque capitoli successivi. Dato che i processori, le memorie e le periferiche sono concetti chiave che ritorneranno durante la trattazione di ciascun livello, iniziamo lo studio dell'architettura degli elaboratori esaminandoli uno alla volta.

2.1 Processori

La Figura 2.1 mostra l'organizzazione di un semplice calcolatore *bus-oriented*. La CPU (*Central Processing Unit*, "unità centrale di calcolo") è il "cervello" del computer e la sua funzione è quella di eseguire i programmi contenuti nella memoria principale prelevando le loro istruzioni, esaminandole ed eseguendole una dopo l'altra. I componenti sono connessi fra loro mediante un bus, cioè un insieme di cavi paralleli sui quali vengono trasmessi indirizzi, dati e segnali di controllo. I bus possono essere esterni alla CPU, per connetterla alla memoria e ai dispositivi di I/O, oppure interni, come vedremo tra poco.

La CPU è composta da parti distinte, tra cui l'unità di controllo e l'unità aritmetico-logica. La prima si occupa di prelevare le istruzioni dalla memoria principale e di determinarne il tipo, mentre la seconda esegue le operazioni, come l'addizione e l'AND, necessarie per portare a termine l'esecuzione delle istruzioni.

La CPU contiene anche una piccola memoria ad alta velocità, utilizzata per memorizzare i risultati temporanei e alcune informazioni di controllo. Questa memoria è costituita da un certo numero di registri, ciascuno dei quali ha una funzione e una dimensione predefinite. Ognuno di loro può contenere un numero, il cui valore può variare fino a un massimo che dipende dalla dimensione del registro, che di solito è uguale per tutti i registri. Dato che sono interni alla CPU possono essere letti e scritti a velocità elevate.

Il registro più importante è il **contatore d'istruzioni**, o **Program Counter (PC)**, che punta alla successiva istruzione che dovrà essere prelevata per l'esecuzione (il termine "contatore d'istruzioni", pur essendo universalmente accettato, è in qualche modo ambiguo, dato che tale registro non effettua alcun *conteggio*). Un altro registro particolarmente importante è il **registro istruzione corrente**, o *Instruction Register* (**IR**), contenente l'istruzione che si trova in fase di esecuzione. La maggior parte dei calcolatori possiede molti altri registri, alcuni di uso generale, altri dedicati a un uso specifico, altri ancora utilizzati dal sistema operativo per controllare il computer.

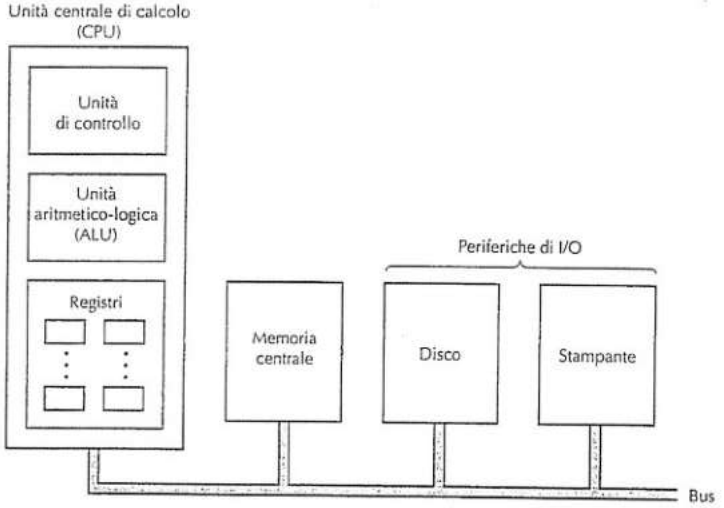


Figura 2.1 Organizzazione di un semplice calcolatore con una CPU e due periferiche di I/O.

2.1.1 Organizzazione della CPU

La Figura 2.2 mostra com'è organizzata internamente una parte, chiamata **percorso dati** (*data path*), di una tipica CPU di von Neumann; essa è composta dai registri (generalmente da 1 a 32), dalla ALU (*Arithmetic Logic Unit*, "unità aritmetico-logica") e da alcuni bus che connettono fra loro le diverse parti. I registri alimentano due registri di input della ALU (indicati nella figura con le lettere A e B) che mantengono i dati d'ingresso della ALU mentre questa è occupata nell'esecuzione di alcune computazioni. Il percorso dati riveste una grande importanza in tutte le macchine e nel corso del libro lo tratteremo in modo approfondito.

La ALU esegue alcune semplici operazioni sui suoi input, come addizioni e sottrazioni, e genera un risultato che viene memorizzato in un suo apposito registro di output. Questo valore può essere successivamente immagazzinato in uno dei registri della CPU che, volendo, può essere copiato in memoria in un secondo momento. Non tutti i tipi di

progettazione prevedono la presenza dei registri A, B e di quello di output. Nell'esempio è illustrata l'addizione, ma l'ALU può eseguire anche altre operazioni.

La maggior parte delle istruzioni può essere divisa in due categorie principali: le istruzioni registro-memoria e quelle registro-registro. Le istruzioni registro-memoria permettono di prelevare parole di memoria per portarle all'interno dei registri, dove sono utilizzabili, per esempio, come input della ALU per effettuare istruzioni successive. (Le "parole" sono le unità di dati che vengono spostate tra la memoria e i registri; una parola potrebbe essere un intero. Più avanti nel capitolo analizzeremo l'organizzazione della memoria.) Altre istruzioni registro-memoria permettono invece di copiare i valori dei registri nella memoria.

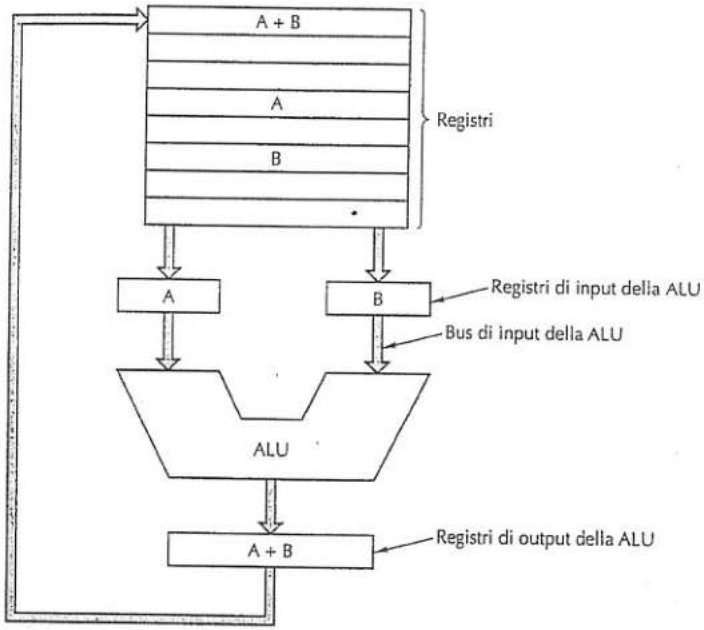


Figura 2.2 Percorso dati di una tipica macchina di von Neumann.

L'altra classe d'istruzioni è quella registro-registro. Una tipica istruzione di questo tipo preleva due operandi dai registri, li porta all'interno dei registri di input della ALU, esegue su di loro una qualche operazione (per esempio l'addizione o l'AND) e ne memorizza il risultato in uno dei registri. Il processo che consiste nel portare i due operandi attraverso la ALU e nel memorizzare il risultato è chiamato **ciclo del percorso dati** e rappresenta il cuore della maggior parte delle CPU. Con buona approssimazione si può dire che definisca che cosa sia in grado di fare una macchina. I moderni computer hanno

più ALU che operano in parallelo, specializzate per funzioni diverse. Più veloce è il ciclo del percorso dati, maggiore risulta la velocità del calcolatore.

2.1.2 Esecuzione dell'istruzione

La CPU esegue ogni istruzione compiendo una serie di piccoli passi che, in linea generale, possono essere descritti nel seguente modo:

1. prelevare la successiva istruzione dalla memoria per portarla nell'IR;
2. modificare il PC per farlo puntare all'istruzione seguente;
3. determinare il tipo dell'istruzione appena prelevata;
4. se l'istruzione usa una parola in memoria, determinare dove si trova;
5. se necessario, prelevare la parola per portarla in un registro della CPU;
6. eseguire l'istruzione;
7. tornare al punto 1 per iniziare l'esecuzione dell'istruzione successiva.

Spesso ci si riferisce a questa sequenza di passi con il termine di ciclo **esecutivo delle istruzioni**, o ciclo di **prelievo-decodifica-esecuzione**. Essa ha un'importanza centrale nel funzionamento di qualsiasi calcolatore.

La precedente descrizione del funzionamento della CPU assomiglia osservandola attentamente a un programma scritto in italiano. La Figura 2.3 mostra come sia possibile formalizzare questo programma in termini di un metodo (cioè una procedura) Java, che prende il nome di *interprete*. La macchina che viene interpretata possiede due registri visibili all'utente: il contatore di programma (PC), per tener traccia dell'indirizzo dell'istruzione successiva da prelevare, e l'accumulatore (AC), per contenere il risultato delle operazioni aritmetiche. Essa possiede anche alcuni registri interni per mantenere l'istruzione corrente durante la sua esecuzione (`instr`), il tipo d'istruzione corrente (`instr_type`), l'indirizzo dell'operando dell'istruzione (`data_loc`) e l'operando stesso (`data`). Si assume che le istruzioni contengano un unico indirizzo di memoria e che nella locazione di memoria corrispondente sia memorizzato l'operando, che può essere, per esempio, un dato da sommare nell'accumulatore.

Il fatto che sia possibile scrivere un programma che simula il comportamento della CPU mostra che esso non deve per forza essere eseguito da una scatola piena di componenti elettronici, cioè dalla CPU fisica. Un programma può infatti essere portato a termine anche da un altro programma, che preleva, esamina ed esegue le proprie istruzioni; come abbiamo già detto nel corso del Capitolo 1, quest'ultimo programma viene chiamato **interprete**.

Questa equivalenza tra i processori hardware e gli interpreti ha alcune importanti implicazioni nell'organizzazione e nel progetto dei sistemi di calcolatori. Un gruppo di progettisti, dopo aver specificato il linguaggio macchina, *L*, per un nuovo calcolatore, può scegliere se realizzare un processore hardware capace di eseguire direttamente i programmi in *L* oppure scrivere un programma che interpreti tali programmi. Se si sceglie di scrivere un interprete occorre anche prevedere i componenti hardware che permettano di eseguirlo. È inoltre possibile avere delle costruzioni ibride, nelle quali alcune esecuzioni vengono svolte dall'hardware, mentre altre dall'interpretazione software.


```
public class Interp
 { static int PC;                    // il program counter contiene l'indirizzo dell'istruzione successiva
  static int AC;                     // l'accumulatore, un registro per i calcoli aritmetici
  static int instr;                  // un registro che contiene l'istruzione corrente
  static int instr-type;             // il tipo d'istruzione (opcode)
  static int data-loc;               // l'indirizzo del dato, o -1 se non c'è
  static int data;                   // contiene l'operando corrente
  static boolean run-bit = true;     // un bit che può essere impostato a 0 per arrestare la macchina

public static void interpret(int memory[ ], int starting-address  {
  // Questa procedura interpreta programmi per una semplice macchina le cui istruzioni hanno
  // un solo operando in memoria. La macchina ha un registro AC (accumulatore), utilizzato per
  // i calcoli aritmetici. L'istruzione ADD, per esempio, somma un intero memorizzato in memoria al registro AC.
  // L'interprete continua finché il bit di esecuzione non viene impostato a 0 dall'istruzione HALT.
  // Lo stato di un processo eseguito su questa macchina è costituito dalla memoria, dal
  // contatore di programma, dal bit di esecuzione e dal registro AC. I parametri di input consistono
  // nell'immagine della memoria e nell'indirizzo di partenza.

PC = starting-address;
while (run-bit) {
  instr = memory[PC];                  / / preleva l'istruzione successiva  e la memorizza in instr
  PC = PC + 1;                         // incrementa il program counter
  instr-type = get-instr-type(instr);  // determina il tipo d'istruzione
  data-loc = find-data(instr,  instr-type);  // trova la posizione del dato (-1 se non c'è)
  if (data-loc >= 0)                   //•se "data-loc" vale -1, allora non c'è nessun operando
    data = memory[data-loc]            // preleva il dato
  execute(instr-type,  data);          // esegue l'istruzione
}

}

private static int get-instr-type(int addr) { ... }
private static int find-data(int  instr, int type) { ... }
private static void execute(int type, int data) { ... }
}
```

Figura 2.3 Interprete Java per un semplice calcolatore.

Dato che un interprete scompone le istruzioni della propria macchina obiettivo in piccoli passi, il calcolatore sul quale viene eseguito può essere molto più semplice e meno costoso rispetto a quanto lo sarebbe un processore hardware appositamente progettato per la macchina obiettivo. Tale convenienza risulta ancor più significativa se la macchina obiettivo possiede molte istruzioni e se queste sono particolarmente complesse e caratterizzate da molte opzioni. Il vantaggio deriva essenzialmente dal fatto che si sostituisce parte dell'hardware con il software (l'interprete), molto meno costoso da riprodurre.

I primi calcolatori avevano un piccolo insieme di semplici istruzioni, ma la richiesta di calcolatori con prestazioni più elevate portò, fra le altre cose, all'introduzione d'istruzioni più potenti. In una fase iniziale si notò che spesso l'uso d'istruzioni più complesse aumentava la velocità di esecuzione dei programmi, anche se le singole istruzioni richiedevano più tempo per essere eseguite. Alcuni esempi di queste istruzioni più complesse comprendono quelle in virgola mobile e quelle per l'accesso diretto agli elementi di un

array. In alcuni casi si osservò semplicemente che due istruzioni apparivano spesso consecutivamente e che quindi una singola istruzione avrebbe potuto svolgere il lavoro di entrambe.

Le istruzioni più complesse erano preferibili in quanto le singole esecuzioni delle operazioni elementari potevano essere sovrapposte oppure eseguite in parallelo utilizzando parti diverse dell'hardware. Dato che i calcolatori ad alte prestazioni giustificavano pienamente il costo di questo hardware aggiuntivo, essi arrivarono ad avere un numero molto maggiore d'istruzioni rispetto alle macchine di costo inferiore. Tuttavia, a causa della crescita del costo dello sviluppo del software e per via dei requisiti di compatibilità, sorse la necessità d'introdurre istruzioni complesse anche nelle macchine più economiche, per le quali il costo era più importante della velocità.

Alla fine degli anni '50 IBM (la società di computer più potente dell'epoca) comprese che mantenere una singola famiglia di macchine, tutte capaci di eseguire le stesse istruzioni, risultava molto vantaggioso non solo per IBM stessa, ma anche per i suoi clienti. IBM introdusse il termine **architettura** per descrivere questo livello di compatibilità. Una nuova famiglia di calcolatori avrebbe dovuto avere un'unica architettura, ma varie e distinte implementazioni che, pur distinguendosi nel prezzo e nelle prestazioni, sarebbero state in grado di eseguire lo stesso programma. Com'è possibile tuttavia costruire una macchina economica capace di eseguire tutte le complicate istruzioni di macchine costose e ad alte prestazioni?

La risposta risiede nell'interpretazione. Questa tecnica, suggerita inizialmente da Maurice Wilkes (1951), ha permesso di progettare calcolatori semplici ed economici, in grado tuttavia di eseguire un gran numero d'istruzioni. Il risultato dell'interpretazione fu la nascita dell'architettura IBM System/360, una famiglia di calcolatori fra loro compatibili e che variavano di quasi due ordini di grandezza sia nel prezzo sia nelle prestazioni. Soltanto nei modelli più costosi si ricorse a un'implementazione diretta dell'hardware (cioè senza interpretazione).

I calcolatori semplici che utilizzano istruzioni interpretate presentano anche altri vantaggi, tra i quali:

1. la capacità di correggere agevolmente le istruzioni implementate in modo errato, o addirittura di compensare le mancanze progettuali dell'hardware di base;
2. la possibilità di aggiungere nuove istruzioni a un costo minimo, anche dopo aver distribuito la macchina;
3. un progetto strutturato che permetta di sviluppare, testare e documentare efficientemente le istruzioni complesse.

Negli anni '70, quando il mercato dei computer esplose in modo impressionante e la potenza di calcolo cominciò a crescere rapidamente, la richiesta di calcolatori a basso costo favorì la progettazione di macchine che usavano interpreti. La possibilità di adattare l'hardware e l'interprete per un determinato insieme d'istruzioni apparve come una modalità di progettazione dei processori altamente conveniente dal punto di vista dei costi. Dato che la tecnologia dei semiconduttori avanzava rapidamente, i vantaggi sui costi acquisirono un peso maggiore rispetto alla possibilità di avere prestazioni più elevate; le architetture basate sugli interpreti divennero così il modo convenzionale di

progettare calcolatori. Quasi tutti i computer costruiti negli anni '70, dai minicomputer fino ai mainframe, si basavano infatti sull'interpretazione.

Nei tardi anni '70 divenne molto diffuso l'uso di processori semplici che eseguivano interpreti, fatta eccezione per i modelli più costosi e dalle prestazioni più elevate, come il Cray-1 e la serie Control Data Cyber. L'uso di un interprete eliminò i problemi di costo che avevano limitato l'uso d'istruzioni complesse e i progettisti iniziarono a esplorare istruzioni ancor più complicate, soprattutto nel modo in cui venivano specificati gli operandi da utilizzare.

Questa tendenza raggiunse il suo apice con il calcolatore VAX della Digital Equipment Corporation, che aveva varie centinaia d'istruzioni e più di 200 modi diversi di specificare gli operandi da utilizzare in ciascuna istruzione. Sfortunatamente l'architettura VAX fu concepita fin dall'inizio per essere implementata attraverso un interprete e con poca attenzione alla realizzazione di un modello ad alte prestazioni. Questo approccio portò all'inclusione di un grandissimo numero d'istruzioni la cui importanza era marginale e che era difficile poter eseguire direttamente. Questo errore risultò fatale alla macchina VAX e, in ultima analisi, alla DEC stessa (nel 1998 Compaq acquistò DEC e nel 2001 Hewlett-Packard acquistò Compaq).

Se i primi microprocessori a 8-bit erano macchine con un insieme d'istruzioni molto semplice, alla fine degli anni '70 ogni microprocessore era passato invece a una progettazione basata sull'interpretazione. Durante questo periodo una delle maggiori sfide che si presentavano ai progettisti dei microprocessori era quella di gestire la crescente complessità resa possibile dai circuiti integrati. Uno dei maggiori vantaggi dell'approccio basato sull'interpretazione era la possibilità di progettare un processore semplice, la cui complessità era in gran parte confinata nell'interprete contenuto in memoria; in questo modo era possibile portare la complessità progettuale dall'hardware al software.

Il successo del Motorola 68000, che disponeva di un ampio insieme d'istruzioni interpretate, e il contemporaneo fallimento dello Zilog Z8000 (che aveva anch'esso un grande insieme d'istruzioni, ma senza interprete) mostrò quali vantaggi offriva un interprete nel mettere velocemente sul mercato un nuovo microprocessore. Questo successo fu ancora più sorprendente se si considera il vantaggio che all'epoca aveva la società Zilog (il processore Z80, predecessore dello Z8000, era stato infatti molto più diffuso del 6800, predecessore del 68000). Ovviamente vi furono anche altre concause, non ultima la lunga esperienza di Motorola come produttore di processori e quella di Exxon (la società che controllava la Zilog) come società petrolifera, e non certo come produttore di processori.

Un altro fattore che all'epoca favorì l'interpretazione fu l'esistenza di veloci memorie di sola lettura, chiamate **memorie di controllo** (*control store*), usate per memorizzare l'interprete. Supponiamo che una normale istruzione interpretata richiedesse 10 istruzioni dell'interprete, chiamate **microistruzioni**, ciascuna da 100 ns, oltre a due riferimenti alla memoria centrale da 500 ns l'uno. L'esecuzione totale richiedeva quindi 2000 ns, un valore solo due volte peggiore del miglior risultato raggiungibile tramite l'esecuzione diretta; senza l'utilizzo della memoria di controllo, le istruzioni avrebbero invece impiegato 6000 ns. È evidente che un fattore di penalizzazione di sei volte è molto più difficile da accettare rispetto a un fattore due.

2.1.3 RISC contro CISC

Durante i tardi anni '70 gli interpreti permisero di sperimentare istruzioni molto complesse; il tentativo inseguito dai progettisti era quello di ridurre il "gap semantico" che divideva ciò che erano in grado di fare le macchine da quello che invece richiedevano i linguaggi di programmazione ad alto livello. Quasi mai si pensava di progettare macchine più semplici, esattamente come oggi non sono molti i ricercatori che si impegnano nel progettare fogli di calcolo, reti, server web e così via. meno potenti (e forse ciò è un peccato).

Ciononostante un gruppo guidato da John Cocke di IBM andò contro questa tendenza e provò a implementare alcune delle idee di Seymour Cray all'interno di un minicomputer ad alte prestazioni. Il risultato di questo lavoro fu un minicomputer sperimentale, chiamato **801**[1]. Anche se IBM non mise mai in commercio questa macchina e i risultati furono pubblicati soltanto alcuni anni dopo (Radin, 1982), trapelarono alcune voci riguardanti il progetto e altri ricercatori iniziarono a studiare architetture simili. Nel 1980, a Berkley, un gruppo guidato da David Patterson e Carlo Séquin cominciò la progettazione di chip VLSI per CPU che non utilizzavano l'interpretazione (Patterson, 1985; Patterson e Séquin, 1982). Per indicare questo progetto essi coniarono l'acronimo **RISC** e chiamarono la loro CPU RISC I, che fu seguita a breve dal modello RISC II. Poco dopo, nel 1981, a Stanford, sull'altra sponda della baia di San Francisco, John Hennessy progettò e costruì un chip, chiamato **MIPS**, leggermente differente sotto alcuni aspetti (Hennessy, 1984). Entrambi i chip evolsero in due importanti prodotti commerciali, rispettivamente SPARC e MIPS.

Questi nuovi processori erano significativamente diversi rispetto a quelli in commercio all'epoca. Dato che non dovevano essere retrocompatibili con alcun prodotto esistente, i loro progettisti furono liberi di scegliere nuovi insiemi d'istruzioni che massimizzassero le prestazioni generali del sistema. Mentre l'enfasi iniziale fu posta su istruzioni semplici la cui esecuzione potesse essere completata velocemente, presto si capì che la chiave per ottenere buone prestazioni consisteva nel progettare istruzioni che potessero essere emesse (iniziate) velocemente. Il tempo impiegato effettivamente per l'esecuzione di un'istruzione aveva un'importanza minore rispetto a quante se ne potevano iniziare in un secondo.

Quando vennero progettati questi semplici processori, la caratteristica che colpì maggiormente l'attenzione di tutti fu che il numero d'istruzioni disponibili era relativamente basso, generalmente attorno a 50. Questo valore era nettamente inferiore alle 200-300 dei computer convenzionali come il VAX e i grandi mainframe IBM. L'acronimo RISC significa infatti **computer con un insieme ridotto d'istruzioni** (*Reduced Instruction Set Computer*), scelto in contrapposizione a CISC, che significa **computer con un insieme d'istruzioni complesso** (*Complex Instruction Set Computer*): un sottile riferimento al VAX, che aveva dominato i dipartimenti universitari di quegli anni. L'uso del nome si è radicato ed è utilizzato ancora oggi, sebbene attualmente siano in pochi a ritenere che la dimensione dell'insieme d'istruzioni sia un problema importante.

[1] Pare che questo nome derivi dal numero dell'edificio del centro IBM in cui si svolse il progetto (*N.d.Rev.*).

In sintesi si può affermare che si scatenò una guerra di religione che vedeva i sostenitori di RISC attaccare l'ordine prestabilito (VAX, Intel e i grandi mainframe IBM). Essi sostenevano che il miglior modo per progettare un calcolatore fosse quello di avere poche istruzioni semplici che potevano essere eseguite in un ciclo del percorso dati mostrato nella Figura 2.2, per esempio leggendo due registri, combinandoli in qualche modo (come sommandoli o calcolandone il prodotto logico) e memorizzando infine il risultato all'interno di un registro. La loro tesi era che una macchina RISC prevaleva nei confronti di una CISC perché, anche se impiegava quattro o cinque istruzioni per fare ciò che una macchina CISC realizzava con una sola, le sue istruzioni erano 10 volte più veloci (in quanto non interpretate). Vale la pena sottolineare che a quel tempo la velocità delle memorie centrali aveva raggiunto quella delle memorie di controllo di sola lettura, rendendo ancora più marcata la penalizzazione dovuta all'interpretazione; ciò contribuì a favorire fortemente le macchine RISC.

Si potrebbe pensare che, grazie ai vantaggi offerti dalla tecnologia RISC, queste macchine (come la Sun UltraSPARC) abbiano fatto scomparire dal mercato le macchine CISC (come gli Intel Pentium), e invece ciò non è avvenuto: perché?

Primo fra tutti va considerato il problema della retrocompatibilità e con esso i miliardi di dollari investiti dalle società per produrre il software per le macchine Intel. Secondo, Intel è riuscita sorprendentemente a impiegare le stesse idee anche all'interno dell'architettura CISC. A partire dal processore 486, le CPU di Intel contengono un nucleo di tipo RISC che esegue le istruzioni più semplici (e spesso più comuni) in un unico ciclo del percorso dati, mentre interpreta le istruzioni più complesse secondo la classica modalità CISC. Il risultato finale è che le istruzioni più comuni sono veloci, mentre quelle meno comuni sono più lente.

Anche se questo approccio non è altrettanto veloce quanto una pura architettura RISC, esso fornisce prestazioni generali competitive, permettendo allo stesso tempo di eseguire senza alcuna modifica il software progettato precedentemente.

2.1.4 Principi di progettazione dei calcolatori moderni

Ora che sono passati più di due decenni dall'introduzione delle prime macchine RISC, alcuni principi di progettazione sono stati accettati come un buon modo di progettare un calcolatore, tenendo in considerazione lo stato attuale della tecnologia dell'hardware. Se dovesse verificarsi un importante cambiamento tecnologico (per esempio un nuovo processo di produzione tale da rendere il ciclo della memoria 10 volte più veloce di quello della CPU), molte delle scommesse finora fatte risulterebbero perdenti. I progettisti di calcolatori devono quindi tenere sempre d'occhio i cambiamenti tecnologici, in quanto possono modificare l'equilibrio fra i vari componenti.

Detto questo, esiste un insieme di principi di progettazione, talvolta chiamati **principi di progettazione RISC**, che i progettisti delle CPU cercano di seguire il più possibile. Anche se vincoli esterni, come i requisiti di retrocompatibilità con alcune architetture esistenti, richiedono di tanto in tanto alcuni compromessi, questi principi sono comunque degli obiettivi che i progettisti si sforzano di raggiungere. Di seguito ne analizzeremo i principali.

Tutte le istruzioni sono eseguite direttamente dall'hardware

Tutte le comuni istruzioni sono eseguite direttamente dall'hardware, e non sono interpretate mediante microistruzioni. L'eliminazione di un livello di interpretazione garantisce velocità più alte per la maggior parte delle istruzioni. Per i calcolatori che implementano insiemi d'istruzioni CISC, quelle più complesse devono essere divise in varie parti che possono essere eseguite come una sequenza di microistruzioni. Questo passo aggiuntivo rallenta la macchina, ma può tuttavia essere accettabile per le istruzioni utilizzate con minor frequenza.

Massimizzare la frequenza di emissione delle istruzioni

I calcolatori moderni ricorrono a molti trucchi per massimizzare le loro prestazioni, il principale dei quali consiste nel cercare di iniziare a eseguire più istruzioni al secondo. Dopo tutto, se si riescono a emettere 500 milioni d'istruzioni al secondo, si ottiene un processore da 500 MIPS, indipendentemente da quanto tempo impieghino quelle istruzioni per essere completate[2]. Questo principio suggerisce che il parallelismo può giocare un ruolo importante nelle prestazioni; infatti è possibile emettere un gran numero di lente istruzioni in un breve intervallo di tempo solo se si riescono a eseguire più istruzioni allo stesso tempo.

Anche se le istruzioni vengono sempre incontrate nell'ordine in cui appaiono nel programma, non sempre sono emesse rispettando questa sequenza (dato che alcune risorse necessarie potrebbero essere occupate) e non è neanche necessario che terminino in tale ordine. Ovviamente, se l'istruzione 1 imposta il valore di un registro e l'istruzione 2 usa lo stesso registro, occorre prestare grande attenzione per evitare che l'istruzione 2 lo legga prima che contenga il giusto valore. Per far ciò in modo corretto è necessario tener conto di molti aspetti; ciononostante questo approccio garantisce un guadagno nelle prestazioni, dato che è possibile eseguire più istruzioni allo stesso tempo.

Le istruzioni devono essere facili da decodificare

Un limite critico sulla frequenza di emissione delle istruzioni è dato dal processo di decodifica, che deve essere effettuato per ogni singola istruzione allo scopo di determinare le risorse necessarie. Tutto ciò che può aiutare questo processo si rivela utile: per esempio rendere le istruzioni regolari, di lunghezza fissa e con pochi campi. Meno formati d'istruzioni ci sono, meglio è.

Solo le istruzioni Load e Store fanno riferimento alla memoria

Uno dei modi più semplici per spezzare le operazioni in passi separati è quello di richiedere che, per la maggior parte delle istruzioni, gli operandi vengano prelevati dai registri e siano memorizzati al loro interno. L'operazione di spostamento degli operandi dalla memoria ai registri può essere invece compiuta separatamente mediante apposite istruzioni. Dato che l'accesso alla memoria può richiedere un tempo considerevole, il cui ritardo non è prevedibile, queste operazioni, a patto che non facciano niente altro se non muovere operandi tra registri e memoria, possono essere efficientemente sovrapposte all'esecuzione di altre istruzioni. Tale osservazione porta alla conclusione che soltanto le istruzioni `LOAD` e `STORE` dovrebbero far riferimento alla memoria, mentre tutte le altre dovrebbero operare esclusivamente sui registri.

[2] Tra l'altro, MIPS – acronimo di *Millions of Instructions Per Second* – sta per "milioni d'istruzioni al secondo", e il processore MIPS fu così chiamato in questo modo come gioco di parole in relazione all'acronimo. Ufficialmente il nome del processore sta per *Microprocessor without Interlocked Pipeline Stages* (*N.d.Rev.*).

Molti registri disponibili

Dato che l'accesso alla memoria è relativamente lento occorre prevedere molti registri (almeno 32) di modo che, una volta prelevata la parola, possa essere mantenuta nel registro fintanto sia necessario. È particolarmente inefficiente trovarsi senza registri liberi, in quanto ciò obbliga a scaricare in memoria tutti i valori dei registri per poi ricaricarli. Il miglior modo per evitare il più possibile questa operazione consiste nel disporre di un numero sufficiente di registri.

2.1.5 Parallelismo a livello d'istruzione

I progettisti di calcolatori si sforzano costantemente di migliorare le prestazioni delle loro macchine. Aumentare la velocità di clock per rendere i processori più veloci è una possibilità, ma per qualsiasi nuova architettura esiste un limite, dipendente dal momento storico, su che cosa sia possibile ottenere mediante la semplice forza bruta. Per questo motivo molti progettisti di computer vedono nel parallelismo (cioè nel compiere più azioni allo stesso tempo) un modo per ottenere prestazioni più elevate con una data velocità di clock.

Il parallelismo può essere presente in due forme principali: a livello d'istruzione e a livello di processore. Nella prima forma il parallelismo è sfruttato all'interno delle singole istruzioni per far sì che la macchina possa elaborarne un maggior numero al secondo, mentre nella seconda sono presenti più CPU che lavorano congiuntamente su uno stesso problema. Entrambi gli approcci hanno i propri pregi; in questo paragrafo analizzeremo il primo tipo di parallelismo, mentre nel prossimo studieremo quello a livello di processore.

Pipelining

Da anni ormai è assodato che uno dei maggiori colli di bottiglia nella velocità di esecuzione delle istruzioni è rappresentato dal prelievo delle istruzioni dalla memoria. Fin dal modello IBM Stretch (1959), per alleviare questo problema, i calcolatori sono stati dotati della capacità di poter prelevare in anticipo le istruzioni dalla memoria, in modo da averle già a disposizione nel momento in cui dovessero rendersi necessarie. Le istruzioni venivano memorizzate in un insieme di registri chiamati **buffer di prefetch**, dai quali potevano essere prese nel momento in cui venivano richieste, senza dover attendere che si completasse una lettura della memoria.

In pratica la tecnica di *prefetching* divide l'esecuzione dell'istruzione in due parti: il prelievo dell'istruzione e la sua esecuzione effettiva. Il concetto di **pipeline** spinge questa strategia molto più avanti; invece di dividere l'esecuzione di un'istruzione solamente in due fasi, la si divide in un numero maggiore di parti (spesso una dozzina o più) che

Tutte le istruzioni sono eseguite direttamente dall'hardware

Tutte le comuni istruzioni sono eseguite direttamente dall'hardware, e non sono interpretate mediante microistruzioni. L'eliminazione di un livello di interpretazione garantisce velocità più alte per la maggior parte delle istruzioni. Per i calcolatori che implementano insiemi d'istruzioni CISC, quelle più complesse devono essere divise in varie parti che possono essere eseguite come una sequenza di microistruzioni. Questo passo aggiuntivo rallenta la macchina, ma può tuttavia essere accettabile per le istruzioni utilizzate con minor frequenza.

Massimizzare la frequenza di emissione delle istruzioni

I calcolatori moderni ricorrono a molti trucchi per massimizzare le loro prestazioni, il principale dei quali consiste nel cercare di iniziare a eseguire più istruzioni al secondo. Dopo tutto, se si riescono a emettere 500 milioni d'istruzioni al secondo, si ottiene un processore da 500 MIPS, indipendentemente da quanto tempo impieghino quelle istruzioni per essere completate[2]. Questo principio suggerisce che il parallelismo può giocare un ruolo importante nelle prestazioni; infatti è possibile emettere un gran numero di lente istruzioni in un breve intervallo di tempo solo se si riescono a eseguire più istruzioni allo stesso tempo.

Anche se le istruzioni vengono sempre incontrate nell'ordine in cui appaiono nel programma, non sempre sono emesse rispettando questa sequenza (dato che alcune risorse necessarie potrebbero essere occupate) e non è neanche necessario che terminino in tale ordine. Ovviamente, se l'istruzione 1 imposta il valore di un registro e l'istruzione 2 usa lo stesso registro, occorre prestare grande attenzione per evitare che l'istruzione 2 lo legga prima che contenga il giusto valore. Per far ciò in modo corretto è necessario tener conto di molti aspetti; ciononostante questo approccio garantisce un guadagno nelle prestazioni, dato che è possibile eseguire più istruzioni allo stesso tempo.

Le istruzioni devono essere facili da decodificare

Un limite critico sulla frequenza di emissione delle istruzioni è dato dal processo di decodifica, che deve essere effettuato per ogni singola istruzione allo scopo di determinare le risorse necessarie. Tutto ciò che può aiutare questo processo si rivela utile: per esempio rendere le istruzioni regolari, di lunghezza fissa e con pochi campi. Meno formati d'istruzioni ci sono, meglio è.

Solo le istruzioni Load e Store fanno riferimento alla memoria

Uno dei modi più semplici per spezzare le operazioni in passi separati è quello di richiedere che, per la maggior parte delle istruzioni, gli operandi vengano prelevati dai registri e siano memorizzati al loro interno. L'operazione di spostamento degli operandi dalla memoria ai registri può essere invece compiuta separatamente mediante apposite istruzioni. Dato che l'accesso alla memoria può richiedere un tempo considerevole, il cui ritardo non è prevedibile, queste operazioni, a patto che non facciano niente altro se non muovere operandi tra registri e memoria, possono essere efficientemente sovrapposte all'esecuzione di altre istruzioni. Tale osservazione porta alla conclusione che soltanto le istruzioni `LOAD` e `STORE` dovrebbero far riferimento alla memoria, mentre tutte le altre dovrebbero operare esclusivamente sui registri.

[2] Tra l'altro, **MIPS** – acronimo di *Millions of Instructions Per Second* – sta per "milioni d'istruzioni al secondo", e il processore MIPS fu così chiamato in questo modo come gioco di parole in relazione all'acronimo. Ufficialmente il nome del processore sta per *Microprocessor without Interlocked Pipeline Stages* (*N.d.Rev.*).

Molti registri disponibili

Dato che l'accesso alla memoria è relativamente lento occorre prevedere molti registri (almeno 32) di modo che, una volta prelevata la parola, possa essere mantenuta nel registro fintanto sia necessario. È particolarmente inefficiente trovarsi senza registri liberi, in quanto ciò obbliga a scaricare in memoria tutti i valori dei registri per poi ricaricarli. Il miglior modo per evitare il più possibile questa operazione consiste nel disporre di un numero sufficiente di registri.

2.1.5 Parallelismo a livello d'istruzione

I progettisti di calcolatori si sforzano costantemente di migliorare le prestazioni delle loro macchine. Aumentare la velocità di clock per rendere i processori più veloci è una possibilità, ma per qualsiasi nuova architettura esiste un limite, dipendente dal momento storico, su che cosa sia possibile ottenere mediante la semplice forza bruta. Per questo motivo molti progettisti di computer vedono nel parallelismo (cioè nel compiere più azioni allo stesso tempo) un modo per ottenere prestazioni più elevate con una data velocità di clock.

Il parallelismo può essere presente in due forme principali: a livello d'istruzione e a livello di processore. Nella prima forma il parallelismo è sfruttato all'interno delle singole istruzioni per far sì che la macchina possa elaborarne un maggior numero al secondo, mentre nella seconda sono presenti più CPU che lavorano congiuntamente su uno stesso problema. Entrambi gli approcci hanno i propri pregi; in questo paragrafo analizzeremo il primo tipo di parallelismo, mentre nel prossimo studieremo quello a livello di processore.

Pipelining

Da anni ormai è assodato che uno dei maggiori colli di bottiglia nella velocità di esecuzione delle istruzioni è rappresentato dal prelievo delle istruzioni dalla memoria. Fin dal modello IBM Stretch (1959), per alleviare questo problema, i calcolatori sono stati dotati della capacità di poter prelevare in anticipo le istruzioni dalla memoria, in modo da averle già a disposizione nel momento in cui dovessero rendersi necessarie. Le istruzioni venivano memorizzate in un insieme di registri chiamati **buffer di prefetch**, dai quali potevano essere prese nel momento in cui venivano richieste, senza dover attendere che si completasse una lettura della memoria.

In pratica la tecnica di *prefetching* divide l'esecuzione dell'istruzione in due parti: il prelievo dell'istruzione e la sua esecuzione effettiva. Il concetto di **pipeline** spinge questa strategia molto più avanti; invece di dividere l'esecuzione di un'istruzione solamente in due fasi, la si divide in un numero maggiore di parti (spesso una dozzina o più) che

possono essere eseguite in parallelo; ciascuna di queste parti è gestita da componenti hardware dedicati.

La Figura 2.4(b) illustra il modello di pipeline. Durante il primo ciclo di clock lo stadio S1 sta lavorando sull'istruzione 1, prelevandola dalla memoria. Durante il ciclo di clock 2 lo stadio S2 decodifica l'istruzione 1, mentre lo stadio S1 preleva l'istruzione 2. Durante il ciclo 3 lo stadio S3 preleva gli operandi per l'istruzione 1, lo stadio S2 decodifica l'istruzione 2 e lo stadio S1 preleva la terza istruzione. Durante il quarto ciclo lo stadio S4 esegue l'istruzione 1, S3 preleva gli operandi per l'istruzione 2, S2 decodifica l'istruzione 3 e S1 preleva l'istruzione 4. Infine, durante l'ultimo ciclo S5 scrive il risultato dell'istruzione 1, mentre gli altri componenti lavorano sulle istruzioni successive.

Per rendere più chiaro il concetto di *pipelining* consideriamo un'analogia. Immaginiamo una fabbrica di dolciumi in cui i reparti di cottura e di confezionamento sono separati.

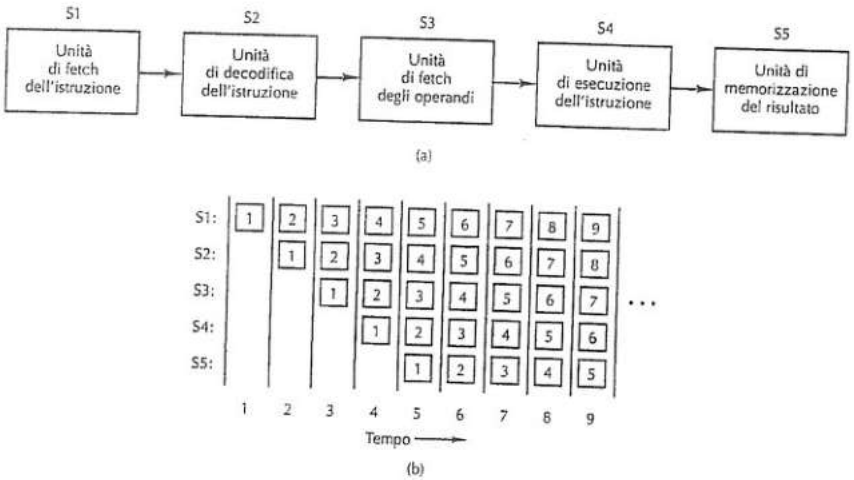


Figura 2.4 (a) Pipeline a cinque stadi. (b) Lo stato degli stadi in funzione del tempo. Sono mostrati nove cicli di clock.

Supponiamo che nel reparto spedizioni ci siano cinque operai (le unità di elaborazione) allineati lungo un nastro trasportatore. Ogni 10 secondi (il ciclo di clock), il primo operaio mette sul nastro una scatola vuota. La scatola viene passata al secondo operaio che inserisce al suo interno una torta. Poco dopo la scatola arriva alla postazione successiva dove viene chiusa e sigillata. Poi la scatola giunge al quarto operaio che vi attacca un'etichetta. Infine l'ultimo operaio rimuove la scatola dal nastro e la ripone in un grande contenitore per la spedizione a un supermercato. In sostanza questo è il modo in cui funziona la pipeline di un calcolatore: ogni istruzione (torta) attraversa vari passi di elaborazione prima di uscire, una volta completata, all'estremità opposta.

Tornando alla Figura 2.4 supponiamo che il ciclo di clock della macchina sia di 2 ns e che quindi un'istruzione impieghi 10 ns per percorrere i cinque stadi della pipeline. A prima vista, si potrebbe dire che la macchina abbia una velocità di 100 MIPS, ma in realtà le sue prestazioni sono molto migliori. Dato che in ogni ciclo di clock (2 ns) viene completata una nuova istruzione, la velocità reale di elaborazione è di 500 MIPS e non di soli 100.

L'uso della pipeline permette di bilanciare la **latenza** (il tempo che un'istruzione impiega per essere elaborata) e la **larghezza di banda del processore** (i MIPS della CPU). Con un ciclo di clock di T ns e una pipeline a n stadi, la latenza è di nT ns, poiché ogni istruzione attraversa n stadi, ognuno dei quali richiede T ns.

Dato che a ogni ciclo di clock viene completata un'istruzione e visto che vi sono $10^9/T$ cicli di clock al secondo, il numero d'istruzioni eseguite al secondo è $10^9/T$. Se per esempio $T = 2$ ns, vengono eseguite 500 milioni d'istruzioni al secondo. Dividendo la velocità di esecuzione delle istruzioni per un milione si ottiene il numero di MIPS, cioè $(10^9/T)/10^6 = 1000/T$ MIPS. In teoria potremmo misurare la velocità di esecuzione delle istruzioni in BIPS[3], al posto di MIPS, ma dato che nessuno lo fa non lo faremo neanche noi.

Architetture superscalari

Se è bene avere una pipeline, averne due è sicuramente meglio. La Figura 2.5 mostra un ipotetico progetto di una CPU con due pipeline, entrambe basate sullo schema della Figura 2.4. In questa situazione una singola unità di *fetch* preleva due istruzioni alla volta e le inserisce nelle pipeline, ognuna delle quali è dotata di una ALU. Affinché le due istruzioni possano essere eseguite in parallelo, non devono però esserci conflitti nell'uso delle risorse (cioè i registri) e nessuna delle due istruzioni deve dipendere dal risultato dell'altra. Come nel caso della singola pipeline, o è il compilatore a occuparsi di gestire correttamente questa situazione (l'hardware non effettua quindi alcun controllo e se le istruzioni sono incompatibili restituisce un risultato errato) oppure i conflitti sono rilevati ed eliminati durante l'esecuzione per mezzo di componenti hardware ad hoc.

Anche se le pipeline, singole o doppie, sono principalmente usate sulle macchine RISC, a partire dal 486 (il 386 e i suoi predecessori ne erano privi), Intel ha cominciato a introdurre le pipeline anche nelle proprie CPU. Il 486 aveva una sola pipeline, mentre il primo Pentium ne aveva due, entrambe a cinque stadi e simili a quelle mostrate nella Figura 2.5, sebbene la divisione esatta del lavoro tra gli stadi 2 e 3 (chiamati *decode-1* e *decode-2*) fosse leggermente diversa rispetto al nostro esempio. La pipeline principale, chiamata **pipeline u**, poteva eseguire una qualsiasi istruzione Pentium. La seconda pipeline, chiamata **pipeline v**, poteva invece eseguire solamente semplici istruzioni su interi (oltre a una semplice istruzione in virgola mobile, FXCH).

Alcune regole prestabilite determinavano se due istruzioni potevano essere eseguite in parallelo. Se tali istruzioni non erano sufficientemente semplici oppure erano incompatibili, soltanto la prima delle due veniva eseguita (nella pipeline u), mentre la seconda veniva trattenuta per essere successivamente accoppiata all'istruzione seguente. Le istruzioni erano sempre eseguite in ordine; per questo motivo compilatori specifici per

[3] Qui la B sta per *Billions*, cioè miliardi (*N.d.T.*).

il Pentium, che erano in grado di produrre coppie d'istruzioni compatibili, potevano generare programmi la cui esecuzione era più veloce rispetto a quella dei vecchi compilatori. Nel caso di programmi che lavoravano su interi, varie misurazioni mostrarono che su un Pentium il codice appositamente ottimizzato per la sua architettura veniva eseguito esattamente due volte più velocemente che su un 486 funzionante alla stessa velocità di clock (Pountain, 1993). Questo guadagno era da attribuire interamente alla seconda pipeline.

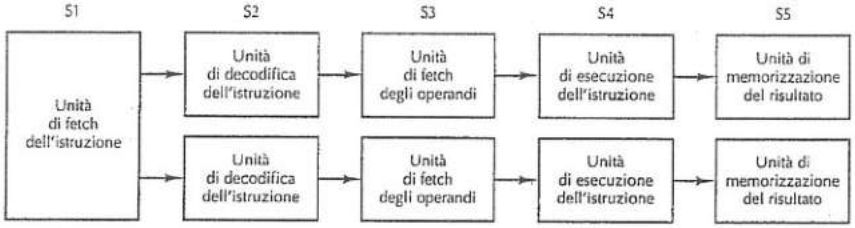


Figura 2.5 Doppia pipeline a cinque stadi con l'unità di *fetch* dell'istruzione in comune.

Anche se si potrebbe immaginare un'architettura a quattro pipeline, ciò obbligherebbe a duplicare troppi componenti hardware (gli informatici non credono nella magia del numero tre); per questo motivo nelle CPU di gamma alta si usa un approccio di tipo diverso. L'idea base consiste nell'avere una singola pipeline, ma di associarle più unità funzionali, com'è mostrato nella Figura 2.6. L'architettura Intel Core, per esempio, ha una struttura simile a quella della figura e sarà esaminata nel corso del Capitolo 4. Nel 1987 fu coniato il termine **architettura superscalare** (Agerwala e Cocke, 1987) per indicare questo approccio; le sue radici risalgono però al calcolatore CDC 6600, ideato più di 40 anni prima. Il 6600 prelevava un'istruzione ogni 100 ns e la passava a una delle 10 unità funzionali che lavoravano in parallelo mentre la CPU era occupata a lanciare l'istruzione successiva.

Nel corso del tempo la definizione di "superscalare" si è in qualche modo evoluta; ora è utilizzata per descrivere processori che lanciano più istruzioni (spesso quattro o sei) durante un ciclo di clock. Ovviamente una CPU superscalare, per poter gestire tutte queste istruzioni, deve avere più unità funzionali. Dato che i processori superscalari hanno generalmente una sola pipeline, essi assomigliano allo schema della Figura 2.6.

Se si utilizza questa definizione, allora da un punto di vista tecnico il 6600 non era superscalare, in quanto lanciava una sola istruzione ogni ciclo. Tuttavia il risultato finale era praticamente lo stesso: le istruzioni venivano lanciate a una frequenza molto più elevata rispetto a quella a cui potevano essere eseguite. Vi è una differenza concettuale molto piccola tra una CPU con ciclo di clock di 100 ns che lancia un'istruzione per ciclo a un gruppo di unità funzionali e una CPU con ciclo di clock di 400 ns che lancia allo stesso gruppo di unità funzionali quattro istruzioni per ciclo. In entrambi i casi l'idea chiave è che il tasso di lancio è molto più elevato del tasso di esecuzione, con il carico di lavoro che viene distribuito su un gruppo di unità funzionali.

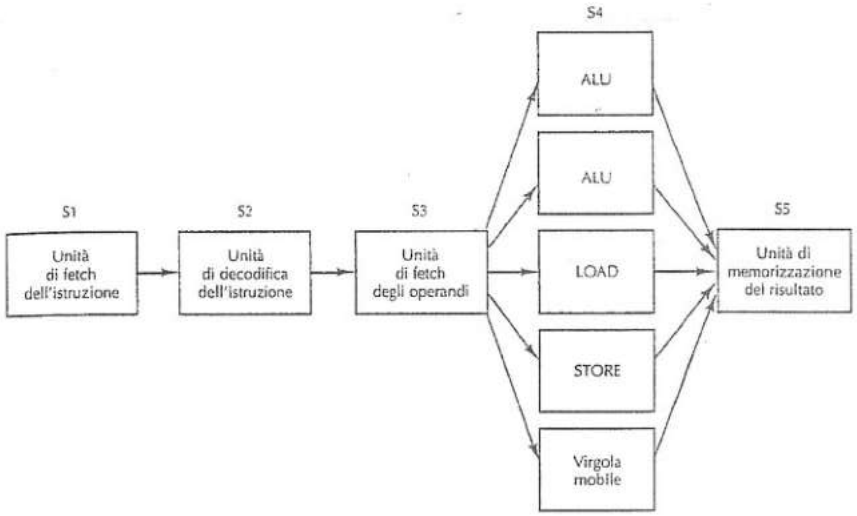


Figura 2.6 Processore superscalare con cinque unità funzionali.

Nell'idea di un processore superscalare è implicito il fatto che lo stadio S3 può lanciare istruzioni a una velocità significativamente superiore rispetto a quella a cui lo stadio S4 riesce a eseguirle. Se lo stadio S3 lanciasse un'istruzione ogni 10 ns e tutte le unità funzionali potessero compiere il loro lavoro in 10 ns, allora non più di un'istruzione alla volta sarebbe in esecuzione, facendo così crollare l'idea base. In realtà la maggior parte delle unità funzionali dello stadio 4 richiede un tempo di esecuzione significativamente maggiore di un singolo ciclo di clock, e ciò è in particolare vero per quelle che accedono alla memoria o che compiono calcoli aritmetici in virgola mobile. Come si può vedere nella figura è possibile avere più unità ALU nello stadio S4.

2.1.6 Parallelismo a livello di processore

La richiesta di calcolatori sempre più veloci sembra inarrestabile. Gli astronomi vogliono simulare che cosa successe durante il primo microsecondo successivo al big bang, gli economisti vogliono modellare l'economia su scala mondiale e gli adolescenti vogliono giocare su Internet con i loro amici virtuali con giochi 3D multimediali e interattivi. Dato che le CPU continuano a diventare più veloci, prima o poi ci si scontrerà con i problemi legati alla velocità della luce, il cui ritardo di propagazione è di 20 cm/ns sia nei cavi di rame sia nelle fibre ottiche, indipendentemente dall'abilità degli ingegneri di Intel. Inoltre, chip più veloci producono anche più calore, e la sua dissipazione costituisce un problema. È proprio la difficoltà di dissipare il calore prodotto a costituire il principale motivo per cui la velocità di clock delle CPU negli ultimi dieci anni ha subito una stagnazione.

Il parallelismo a livello d'istruzione aiuta in parte, ma difficilmente l'uso della pipeline e le operazioni superscalari possono aumentare le prestazioni di un fattore cinque o dieci. Per ottenere guadagni di 50, 100, e più, l'unica soluzione è quella di progettare calcolatori con più CPU; per questo motivo analizzeremo ora come sono organizzati alcuni di questi sistemi.

Computer con parallelismo sui dati

Un gran numero di problemi in ambiti computazionali come la fisica, l'ingegneria e la computer graphic vengono trattati con l'utilizzo di cicli e array, o comunque hanno una struttura altamente regolare. Spesso gli stessi calcoli vengono eseguiti ripetutamente su diversi insiemi di dati. La regolarità e la struttura di questi programmi li rende particolamente adatti a una esecuzione parallela che ne incrementi le prestazioni. Sono stati utilizzati principalmente due metodi per eseguire questi programmi altamente regolari in modo rapido ed efficiente: i processori SIMD e i processori vettoriali. Anche se questi due sistemi sono per molti versi piuttosto simili, il primo è in genere visto come un calcolatore parallelo mentre il secondo è stranamente considerato come un'estensione di un singolo processore.

I computer con parallelismo sui dati, grazie alla loro efficienza, hanno trovato molte applicazioni di successo. Essi sono in grado di offrire una grande potenza computazionale con l'utilizzo di un minor numero di transistor rispetto ad altri approcci. Gordon Moore (l'inventore della omonima legge) ha notoriamente ricordato che il costo del silicio è di circa 1 miliardo di dollari per ogni acro (equivalente a 4047 metri quadrati). Maggiore è la potenza calcolo che può essere ottenuta da un acro di silicio, più soldi una società di computer può guadagnare vendendolo. I processori con parallelismo sui dati sono uno dei mezzi più efficaci per ottenere alte prestazioni dal silicio. Poiché tutti i processori sono in esecuzione sulla stessa istruzione, il sistema ha bisogno di un solo "cervello" per controllare il computer. Il processore necessita dunque soltanto di uno stadio di prelievo, uno di decodifica e di una logica di controllo. Si tratta di un risparmio in termini di silicio che mette questi computer in condizioni di grande vantaggio sugli altri processori, a patto che il software eseguito sia altamente regolare e con molto parallelismo.

Un processore SIMD (*Single Instruction-stream Multiple Datastream*, "istruzioni singole, dati multipli") consiste di un elevato numero di processori identici che eseguono la stessa sequenza d'istruzioni su insiemi diversi di dati. Il primo sistema SIMD fu il calcolatore ILLIAC IV della University of Illinois (Bouknight et al., 1972). Il progetto originale prevedeva di costruire una macchina costituita da quattro quadranti, ciascuno formato da una griglia quadrata di 8 coppie di processori e memorie per lato. Una singola unità di controllo per ogni quadrante trasmetteva in broadcast le istruzioni, che venivano eseguite a passi sincronizzati da tutti i processori, ciascuno dei quali utilizzava i dati letti dalla propria memoria. Anche se venne costruito un solo quadrante per limitarne i costi, le prestazioni di ILLIAC IV raggiunsero i 50 megaflop (milioni di operazioni in virgola mobile al secondo). Si dice che, se la macchina fosse stata costruita interamente e se le sue prestazioni avessero raggiunto l'obiettivo originario (1 gigaflop), essa avrebbe avuto una potenza computazionale pari a due volte quella del mondo intero.

Le moderne unità di elaborazione grafica (GPU) si affidano in larga misura all'elaborazione SIMD per offrire grande potenza di calcolo con pochi transistor. L'elaborazione grafica si presta a processori SIMD perché la maggior parte degli algoritmi sono altamente regolari, con operazioni ripetute su pixel, vertici, texture e contorni. La Figura 2.7 mostra il processore SIMD della GPU Nvidia Fermi. Una GPU Fermi contiene fino a 16 multiprocessori streaming (SM) di tipo SIMD, e ogni SM contiene 32 processori SIMD. A ogni ciclo, lo scheduler sceglie due thread da eseguire sul processore SIMD.

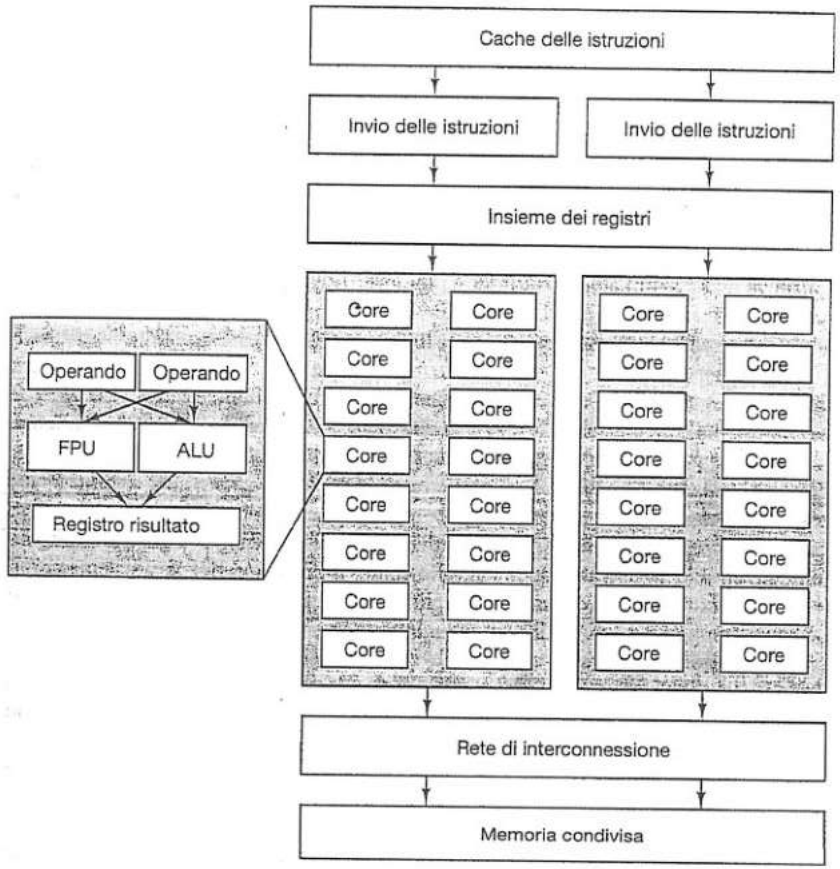


Figura 2.7 Il processore SIMD della GPU Fermi.

L'istruzione successiva di ogni thread verrà poi eseguita da 16 processori SIMD (o meno, se non c'è abbastanza parallelismo dei dati). Se ogni thread è in grado di esegui-

re 16 operazioni per ciclo, una GPU Fermi con 32 SM a pieno carico eseguirà ben 512 operazioni per ciclo. Si tratta di un risultato imponente, considerando che una CPU quad-core di uso generale di simili dimensioni dovrebbe lottare per raggiungere anche solo un trentaduesimo di tale capacità di elaborazione.

Agli occhi di un programmatore, i **processori vettoriali** risultano molto simili a un processore SIMD. Anche questi processori eseguono in modo molto efficiente una stessa sequenza di operazioni su coppie di dati, anche se, a differenza dei processori SIMD, tutte le operazioni di addizione sono eseguite da un unico sommatore, altamente strutturato a pipeline. L'azienda fondata da Seymour Cray, la Cray Research (che oggi fa parte di SGI), ha prodotto un gran numero di processori vettoriali, a partire dal modello Cray-1 nel 1974, fino ai modelli attuali.

Sia i processori SIMD sia i processori vettoriali lavorano su array di dati. Entrambi eseguono singole istruzioni che, per esempio, sommano a coppie gli elementi di due vettori ma, mentre un processore SIMD lo fa usando tanti sommatori quanti sono gli elementi dei vettori, in un processore vettoriale si utilizza invece un **registro vettoriale**, che consiste di un insieme di registri convenzionali caricabili dalla memoria in una singola istruzione (che, in realtà, li carica in modo sequenziale). Un'istruzione di somma tra vettori viene quindi eseguita su coppie di elementi prelevati da due registri vettoriali per alimentare un sommatore strutturato a pipeline. Il risultato è un altro vettore che può essere memorizzato in un registro vettoriale oppure utilizzato direttamente come operando per una nuova operazione vettoriale.

Le istruzioni SSE (*Streaming SIMD Extension*) dell'architettura Intel Core utilizzano questo modello di esecuzione per velocizzare i programmi altamente regolari, come le applicazioni multimediali o il software scientifico; sotto questo aspetto il calcolatore ILLIAC IV va considerato come uno degli antenati dell'architettura Intel Core.

Multiprocessori

In un processore parallelo sui dati le unità di elaborazione non sono delle CPU indipendenti, dato che c'è un'unica unità di controllo condivisa fra tutte. Il primo sistema parallelo che analizziamo e che è composto da più CPU complete è il **multiprocessore**, cioè un sistema composto da più CPU con una memoria in comune (così come un gruppo di persone condivide la stessa lavagna). Dato che ogni CPU può leggere e scrivere una qualsiasi parte della memoria, esse devono coordinarsi (via software) per evitare di ostacolarsi a vicenda. Quando due o più CPU hanno la possibilità di interagire in modo così profondo si dice che sono *tighly coupled* (cioè legate strettamente).

Sono possibili vari schemi d'implementazione, il più semplice dei quali consiste nell'avere un singolo bus con più CPU, tutte connesse a un'unica memoria. La Figura 2.8(a) mostra un diagramma di un simile multiprocessore.

Non serve una gran fantasia per capire che si verificano dei conflitti se un gran numero di processori veloci tenta costantemente di accedere alla memoria attraverso lo stesso bus. Per ridurre queste contese e migliorare le prestazioni i progettisti di multiprocessori hanno ideato vari schemi. La Figura 2.8 mostra un'architettura in cui ogni processore possiede una propria memoria locale, non accessibile agli altri. Questa memoria è utilizzabile per contenere il codice del programma e quei dati che non devono essere condivisi. L'accesso a questa memoria privata non utilizza il bus principale, riducendo quindi in modo considerevole il traffico sul bus. Oltre a questo schema ne esistono anche degli altri (per esempio, quello detto *caching* si veda più avanti).

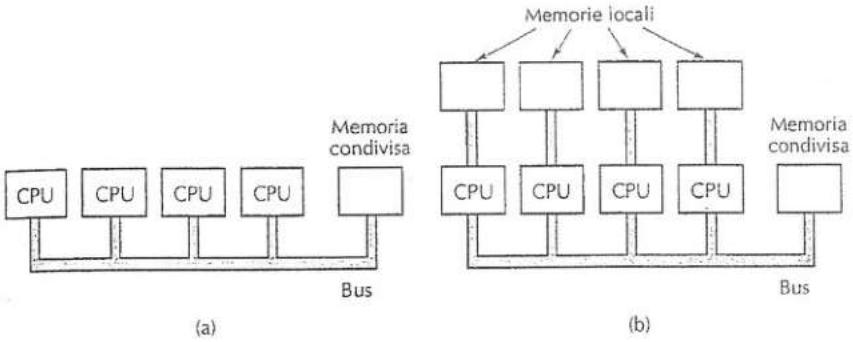


Figura 2.8 (a) Multiprocessore a bus singolo. (b) Multicomputer con memorie locali.

Rispetto ad altri tipi di calcolatori paralleli i multiprocessori hanno il vantaggio che è facile lavorare con un modello di programmazione basato su una memoria condivisa. Si immagini per esempio un programma che ricerca le cellule tumorali in un'immagine di un certo tessuto presa al microscopio. La fotografia digitalizzata potrebbe essere mantenuta in memoria comune e a ciascun processore potrebbe essere assegnata una particolare regione dell'immagine al cui interno effettuare la ricerca. Dato che ciascun processore ha accesso all'intera memoria, non vi è alcun problema se lo studio di una cellula, che inizia nella regione a esso assegnata, scavalca il limite della regione successiva.

Multicomputer

Se da un lato è relativamente semplice costruire multiprocessori composti da un modesto numero di processori (non più di 256), è invece decisamente più complicato realizzarne di più grandi. La difficoltà risiede nel connettere tutti i processori alla memoria. Per aggirare questi problemi molti progettisti hanno semplicemente abbandonato l'idea di avere una memoria condivisa e hanno costruito sistemi composti da un gran numero di calcolatori interconnessi, ciascuno dotato di una memoria privata. Questi sistemi sono detti **multicomputer**. In questi sistemi le CPU sono dette con legame lasco (*loosely coupled*), in contrapposizione con quelle che compongono i multiprocessori.

Le CPU dei multicomputer comunicano fra loro inviandosi messaggi, simili alle e-mail, ma molto più veloci. Nel caso di grandi sistemi, dato che non è efficiente connettere mutualmente tutti i calcolatori, si utilizzano topologie diverse come griglie 2D e 3D, alberi e anelli. Ne consegue che i messaggi tra due calcolatori per spostarsi dalla sorgente alla destinazione spesso devono passare attraverso uno o più macchine intermedie oppure attraverso dei commutatori. Ciononostante è possibile ottenere che lo

scambio di messaggi richieda un tempo nell'ordine di pochi microsecondi. Sono stati costruiti dei multicomputer dotati di oltre 250.000 CPU, come l'IBM Blue Gene/P.

Visto che è facile programmare i multiprocessori, mentre i multicomputer sono facili da costruire, molte ricerche sono indirizzate alla realizzazione di sistemi ibridi che uniscano le qualità di entrambi. Tali calcolatori cercano di dare l'illusione che esista una memoria condivisa, senza però averne realmente una in quanto troppo costosa. Nel Capitolo 8 analizzeremo in dettaglio multiprocessori e multicomputer.

2.2 Memoria principale

La **memoria** è quella parte del calcolatore in cui sono depositati programmi e dati. Alcuni informatici (specialmente quelli britannici) utilizzano il termine inglese ***store*** o ***storage*** ("immagazzinare", "immagazzinamento") al posto di memoria, anche se il termine *storage* viene utilizzato sempre più frequentemente per riferirsi alla registrazione dei dati nei dischi. Se non ci fosse una memoria da cui il processore potesse leggere e scrivere informazioni, non esisterebbero i calcolatori digitali a programma memorizzato.

2.2.1 Bit

L'unità base della memoria è la cifra binaria, chiamata **bit**. Un bit può avere valore 0 oppure 1 ed è l'unità più semplice possibile. (Difficilmente un dispositivo in grado di memorizzare soltanto 0 potrebbe essere alla base di un sistema di memorizzazione; è infatti necessario che ci siano almeno due valori distinti.)

Quando si dice che i calcolatori utilizzano l'aritmetica binaria perché è "efficiente", s'intende (anche se spesso non ci si rende conto) che l'informazione digitale può essere memorizzata utilizzando dei valori di una certa quantità fisica continua, come la tensione o la corrente. Se occorre distinguere più valori, allora ci deve essere una minor separazione tra valori adiacenti e la memoria risulta di conseguenza meno affidabile. Dato che la numerazione binaria (Appendice A) richiede due soli valori distinti, risulta il metodo più affidabile per codificare l'informazione digitale.

Alcuni calcolatori, tra i quali i mainframe IBM, sono pubblicizzati affermando che sono dotati di aritmetica decimale oltre di quella binaria. Il trucco utilizzato consiste nell'usare 4 bit per memorizzare una cifra decimale mediante un codice chiamato **BCD** (*Binary Coded Decimal*, "decimale codificato in binario"). Quattro bit forniscono 16 combinazioni, di cui 10 sono utilizzate per le cifre da 0 a 9, mentre 6 sono inutilizzate. Di seguito mostriamo la codifica decimale e binaria del numero 1944, utilizzando 16 bit in entrambi i casi:

```
decimale: 0001 1001 0100 0100     binaria: 0000011110011000
```

Nel formato decimale sedici bit possono memorizzare i numeri da 0 a 9999, permettendo solo 10.000 combinazioni, mentre una stringa binaria di 16 bit può avere 65.536 combinazioni distinte. Per questa ragione si dice che il sistema binario è più efficiente.

Consideriamo tuttavia che cosa accadrebbe se un giovane e brillante ingegnere elettronico inventasse un dispositivo altamente affidabile in grado di memorizzare direttamente le cifre da 0 a 9 dividendo in 10 intervalli la regione compresa tra 0 e 10 Volt. Quattro di questi dispositivi potrebbero memorizzare un qualsiasi numero decimale tra 0 e 9999, fornendo così 10.000 combinazioni. Essi potrebbero essere utilizzati anche per memorizzare numeri binari usando solamente le cifre 0 e 1, ma in questo caso, con quattro dispositivi, si otterrebbero soltanto 16 combinazioni. Con un simile dispositivo il sistema decimale risulterebbe ovviamente più efficiente.

2.2.2 Indirizzi di memoria

Le memorie sono costituite da un certo numero di **celle** (o **locazioni**) ciascuna delle quali può memorizzare informazioni. Ciascuna cella ha un numero, chiamato **indirizzo**, attraverso il quale il programma può riferirsi a essa. Se una memoria ha n celle, i suoi indirizzi varieranno da 0 a $n - 1$. Tutte le celle di una memoria contengono lo stesso numero di bit; se una cella è costituita da k bit, essa può contenere una qualsiasi delle 2^k diverse combinazioni di bit. La Figura 2.9 mostra tre diverse organizzazioni di una memoria a 96 bit; si noti che (per definizione) le celle adiacenti hanno indirizzi consecutivi.

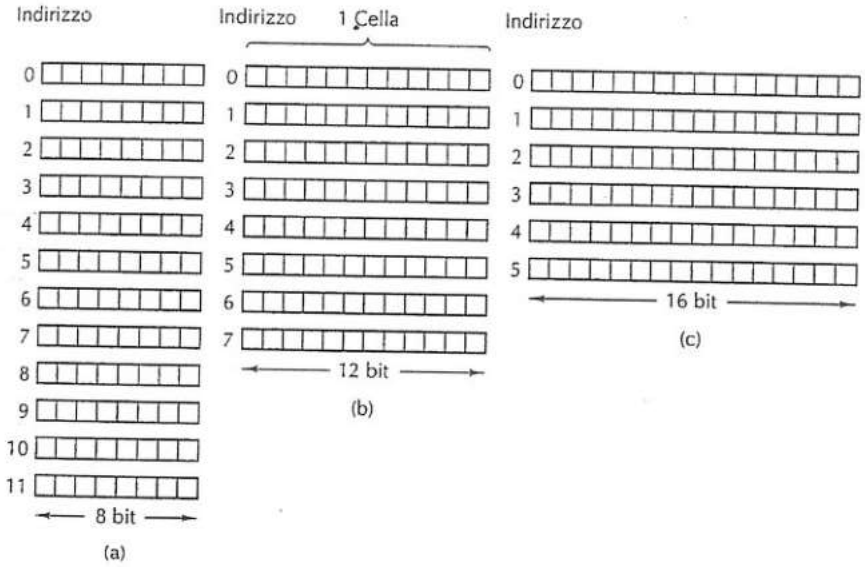


Figura 2.9 Tre modi di organizzare una memoria a 96 bit.

I calcolatori che usano il sistema numerico binario (comprese la notazione ottale ed esadecimale) esprimono gli indirizzi di memoria in notazione binaria. Se un indirizzo ha m bit, il massimo numero di celle indirizzabili è 2^m. Per esempio un indirizzo usato

che modifica per esempio la parola 0000000000 nella parola 0000000111 non può essere corretto.

Immaginiamo di voler progettare un codice con m bit di dati e r bit di controllo, capace di correggere tutti gli errori singoli. Ciascuna delle 2^m parole di memoria legali ha n parole di codice illegali a distanza 1 da essa. Queste sono formate invertendo sistematicamente ciascuno degli n bit nella parola di codice generata dalla parola di memoria considerata. Ciascuna delle 2^m parole di memoria legali richiede quindi $n + 1$ stringhe di bit a essa dedicate (n per i possibili errori e 1 per la combinazione corretta). Dato che il numero totale di combinazioni di bit è 2^n deve valere $(n + 1)2^m \leq 2^n$; questa disequazione può essere riscritta come $(m + r + 1) \leq 2^r$, dato che $n = m + r$. Conoscendo m, essa fornisce un limite inferiore al numero di bit di controllo richiesti per correggere errori singoli. La Figura 2.13 mostra il numero di bit di controllo richiesti per parole di memoria di varie lunghezze.

È possibile ottenere questo limite teorico utilizzando un metodo ideato da Richard Hamming. Prima di analizzare l'algoritmo di Hamming, guardiamo una semplice rappresentazione grafica che illustra chiaramente l'idea di un codice a correzione di errore per parole a 4 bit. Il diagramma di Venn della Figura 2.14(a) contiene tre cerchi, A, B e C, che insieme formano sette regioni. Come esempio codifichiamo la parola di memoria a 4 bit 1100 nelle regioni AB, ABC, AC e BC, inserendo un bit in ogni regione (in ordine alfabetico). Questa codifica è mostrata nella Figura 2.14(a).

Dimensione parola	Bit di controllo	Dimensione totale	Percentuale di overhead
8	4	12	50
16	5	21	31
32	6	38	19
64	7	71	11
128	8	136	6
256	9	265	4
512	10	522	2

Figura 2.13 Numero di bit di controllo per un codice che può correggere errori singoli.

Aggiungiamo poi un bit di parità a ciascuna delle tre regioni vuote per ottenere al loro interno una parità pari, come illustrato nella Figura 2.14(b). Per costruzione la somma dei bit in ciascuno dei tre cerchi, A, B e C, è ora un numero pari. Nel cerchio A abbiamo i quattro numeri 0, 0, 1 e 1, la cui somma fa 2, un numero pari. Nel cerchio B i numeri sono 1, 1, 0 e 0, che sommati fanno ancora 2, un numero pari. Infine nel cerchio C abbiamo nuovamente la stessa situazione. In questo esempio tutti i cerchi hanno la stessa somma, ma, in altre situazioni, è possibile ottenere somme diverse, come 0 e 4. Questa figura corrisponde a una parola di codice con 4 bit di dati e 3 bit di parità.

Supponiamo ora che il bit nella regione AC diventi errato, passando dal valore 0 al valore 1, come mostrato nella Figura 2.14(c). A questo punto il calcolatore può vedere

che i cerchi A e C hanno la parità errata (dispari). L'unico cambiamento di un singolo bit che li può correggere corrisponde a riportare AC al valore 0; questa modifica corregge correttamente l'errore. Seguendo questa strategia il calcolatore può riparare automaticamente gli errori di memoria che coinvolgono un solo bit.

Vediamo ora come l'algoritmo di Hamming può essere usato per costruire codici a correzione di errore per parole di memoria dalla dimensione arbitraria. In un codice di Hamming gli r bit di parità sono aggiunti a una parola a m bit, formando una nuova parola di lunghezza $m + r$ bit. I bit sono numerati a partire da 1, non 0, con il bit 1 nella posizione più a sinistra (più significativa). Tutti i bit la cui posizione è una potenza di 2 sono bit di parità; quelli restanti sono usati invece per i dati. Per esempio, con una parola a 16 bit, vengono aggiunti 5 bit di parità nelle posizioni 1, 2, 4, 8 e 16, mentre le altre contengono bit di dati. In totale la parola di memoria diventa di 21 bit (16 di dati, 5 di parità). In questo esempio useremo (arbitrariamente) la parità pari.

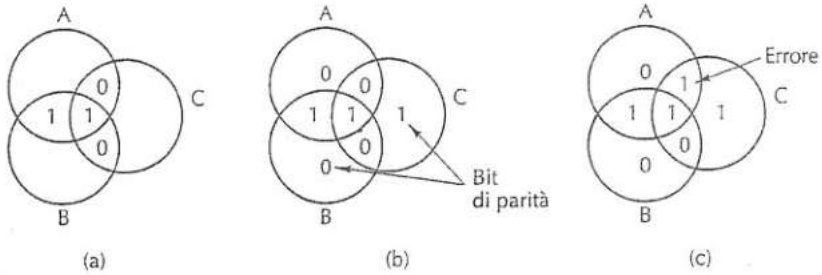


Figura 2.14 (a) Codifica di 1100. (b) Aggiunta della parità pari. (c) Errore in AC.

Ciascun bit di parità controlla alcune posizioni dei bit specifiche ed è impostato in modo che sia pari il numero totale di bit che hanno valore 1 nelle posizioni controllate. Ogni bit di parità controlla le seguenti posizioni:

```
il bit 1 controlla i bit 1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21
il bit 2 controlla i bit 2, 3, 6, 7, 10, 11, 14, 15, 18, 19
il bit 4 controlla i bit 4, 5, 6, 7, 12, 13, 14, 15, 20, 21
il bit 8 controlla i bit 8, 9, 10, 11, 12, 13, 14, 15
il bit 16 controlla i bit 16, 17, 18, 19, 20, 21.
```

In generale il bit di posto b è controllato dai bit $b_1, b_2, \ldots, b_j$ tali che $b_1 + b_2 + \ldots + b_j = b$. Per esempio il bit 5 è controllato dai bit 1 e 4 dato che $1 + 4 = 5$. Il bit 6 è controllato dai bit 2 e 4 in quanto $2 + 4 = 6$, e così via.

La Figura 2.15 mostra la correzione di un codice di Hamming per la parola di memoria a 16 bit 1111000010101110. La parola di codice a 21 bit corrispondente è 001011100000101101110. Per vedere come funziona la correzione di errore consideriamo che cosa accadrebbe se il bit 5 fosse invertito da un sovraccarico di tensione sulla linea di alimentazione. La nuova parola di codice diventerebbe 001001100000101101110

(invece di 00101110000010110110). Controllando i 5 bit di parità si otterrebbero i seguenti risultati:

```
bit di parità 1 non corretto (1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21
contengono cinque 1)
bit di parità 2 corretto (2, 3, 6, 7, 10, 11, 14, 15, 18, 19
contengono sei 1)
bit di parità 4 non corretto (4, 5, 6, 7, 12, 13, 14, 15, 20, 21
contengono cinque 1)
bit di parità 8 corretto (8, 9, 10, 11, 12, 13, 14, 15
contengono due 1)
bit di parità 16 corretto (16, 17, 18, 19, 20, 21
contengono quattro 1).
```

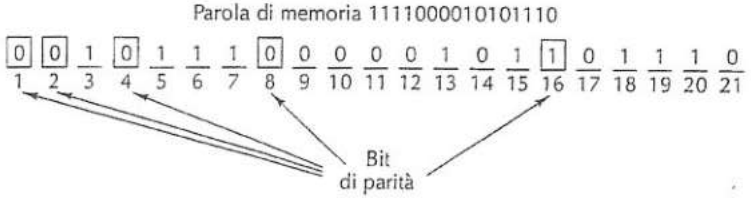


Figura 2.15 Costruzione del codice di Hamming per la parola di memoria 1111000010101110 aggiungendo 5 bit di controllo ai 16 bit di dati.

Il numero totale di 1 nelle posizioni 1, 3, 5, 7, 9, 11, 13, 15, 17, 19 e 21 dovrebbe essere un numero pari dato che si sta usando la parità pari. Il bit non corretto deve essere uno dei bit controllati dal bit di parità 1, cioè uno fra i bit 1, 3, 5, 7, 9, 11, 13, 15, 17, 19 e 21. Il bit di parità 4 è errato, il che significa che uno fra i bit 4, 5, 6, 7, 12, 13, 14, 15, 20 e 21 non è corretto. L'errore deve essere uno dei bit presenti in entrambe le liste, cioè il bit 5, 7, 13, 15 oppure 21. Tuttavia il bit 2 è errato e quindi i bit 7 e 15 sono eliminati. Analogamente il bit 8 è corretto, eliminando così il bit 13. Infine, dato che anche il bit 16 è corretto, va eliminato pure il 21. Il solo bit rimasto è il bit 5, che è quindi quello in cui si è verificato l'errore; dato che è stato letto come 1, esso deve assumere il valore 0. Questa successione di considerazioni consente di correggere gli errori.

Un semplice metodo per trovare i bit errati consiste nel calcolare inizialmente tutti i bit di parità. Se sono tutti corretti allora non si è verificato alcun errore (oppure più di uno). Successivamente si sommano tutti i bit di parità errati, contando 1 per il bit 1, 2 per il bit 2, 4 per il bit 4, e così via, e la somma risultante corrisponde alla posizione del bit errato. Se per esempio i bit di parità 1 e 4 sono errati, ma 2, 8 e 16 sono corretti, significa che il bit 5 (1 + 4) è stato invertito.

2.2.5 Memoria cache

Storicamente le CPU sono sempre state più veloci delle memorie; i miglioramenti di quest'ultime hanno contribuito a incremetare anche le prestazioni delle CPU, preservando di fatto lo squilibrio. Infatti, dato che diventa possibile collocare sempre più circuiti su un chip, i progettisti delle CPU stanno usando queste nuove possibilità per sviluppare le architetture a pipeline e superscalari, migliorando ulteriormente le velocità delle CPU. Al contrario i progettisti delle memorie hanno generalmente usato questa nuova tecnologia per aumentare la capacità dei loro chip, ma non la loro velocità, facendo peggiorare con il tempo lo squilibrio tra i due componenti. Il significato pratico di questa differenza di prestazioni è che, quando la CPU lancia una richiesta alla memoria, essa non otterrà la parola desiderata se non dopo molti cicli di CPU. Più lenta è la memoria, più cicli dovrà attendere la CPU.

Come abbiamo precisato precedentemente esistono due modi per trattare questo problema. Il metodo più semplice consiste nel far iniziare le istruzioni di lettura dalla memoria non appena vengono incontrate, permettendo al contempo di continuare l'esecuzione e bloccando la CPU quando un'istruzione tenta di usare una parola di memoria non ancora arrivata. Più lenta è la memoria, più frequentemente si verifica questo problema e maggiore è la penalizzazione che ne consegue. Per esempio, se un'istruzione su cinque accede alla memoria e il ritardo della memoria è di cinque cicli, il tempo di esecuzione sarà il doppio di quanto sarebbe stato con memoria istantanea. Se però il tempo di accesso alla memoria è di 50 cicli, il tempo di esecuzione crescerà di un fattore 11 (5 cicli per l'esecuzione delle istruzioni, più 50 cicli per l'attesa per la memoria).

L'altra soluzione consiste nel richiedere ai compilatori di non generare codice che utilizzi parole ancor prima che queste siano arrivate, il che consente di avere macchine che non si bloccano. Il problema di questo approccio è che è molto più facile a dirsi che a farsi. Spesso dopo una LOAD non vi è nient'altro da fare e quindi il compilatore è forzato a inserire delle istruzioni fasulle, dette NOP (cioè nessuna operazione), che non eseguono alcuna azione, ma occupano uno slot di tempo. In sostanza questo approccio è uno stallo software invece che hardware; il peggioramento delle prestazioni non cambia.

Attualmente il problema non sta nella tecnologia, ma piuttosto in considerazioni economiche. Gli ingegneri sanno come costruire memorie veloci quanto le CPU, ma, per poter andare alla stessa velocità, esse devono essere collocate sul chip della CPU (dato che utilizzare il bus per la memoria è troppo lento). Inserire una grande memoria sul chip della CPU la rende più grande, il che significa più costosa; inoltre, anche se il costo non fosse un problema, esistono comunque dei limiti alla dimensione di un chip di CPU. La scelta si riduce quindi tra avere una piccola quantità di memoria veloce o una grande quantità di memoria più lenta, anche se, idealmente, vorremmo avere una grande quantità di memoria veloce a un prezzo basso.

Nonostante queste limitazioni, ci sono delle tecniche interessanti che permettono di combinare una piccola quantità di memoria veloce con una grande quantità di memoria lenta al fine di ottenere a un prezzo modesto sia la velocità della memoria veloce (quasi) sia la capacità della memoria più grande. La piccola e veloce memoria è chiamata *cache* (dal francese *cacher*, che significa "nascondere")[4]. Ora descriveremo brevemente come

[4] In inglese la parola *cache* indica un deposito, una scorta di beni da utilizzare al momento opportuno (*N.d.T.*).

si usano queste memorie e come funzionano, mentre una descrizione più dettagliata sarà data nel Capitolo 4.

L'idea di base è semplice: le parole di memoria usate più di frequente sono mantenute all'interno della cache. Quando la CPU necessita di una parola, la cerca nella cache e, solo nel caso in cui essa non sia presente, la richiede alla memoria centrale. È possibile ridurre drasticamente il tempo medio di accesso se una frazione significativa delle parole è presente nella cache.

Il successo o il fallimento dipendono quindi da quali parole sono presenti nella cache. Da anni si sa che i programmi non accedono alle loro memorie in modo completamente casuale. Se a un certo istante la memoria fa un riferimento all'indirizzo A è molto probabile che il successivo riferimento alla memoria si troverà nelle vicinanze di A. Un semplice esempio è il programma stesso, in quanto, fatta eccezione per i salti e le chiamate a procedura, le istruzioni sono prelevate da locazioni contigue di memoria. Inoltre la maggior parte del tempo di esecuzione di un programma è spesa nei cicli, nei quali si esegue ripetutamente un numero limitato d'istruzioni. Analogamente, è molto probabile che un programma per la manipolazione di matrici effettuerà svariati riferimenti alla stessa matrice prima di spostarsi su altri dati.

L'osservazione secondo la quale i riferimenti alla memoria fatti in un breve intervallo temporale tendono a utilizzare solo una piccola frazione della memoria totale è chiamata **principio di località** ed è alla base di tutti i sistemi di cache. L'idea generale prevede che quando una parola viene referenziata, la parola stessa e alcune parole vicine sono portate dalla grande e lenta memoria all'interno della cache, in modo che sia possibile accedervi velocemente in un secondo momento. La Figura 2.16 mostra una tipica organizzazione di CPU, cache e memoria centrale. Se durante un piccolo intervallo una parola è letta o scritta k volte, il calcolatore dovrà effettuare un solo riferimento alla memoria lenta e $k - 1$ riferimenti alla memoria veloce. Maggiore è k, migliori sono le prestazioni complessive.

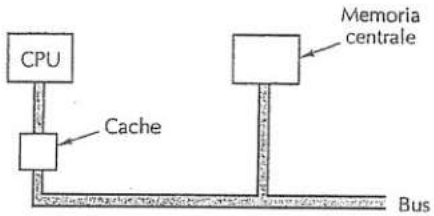


Figura 2.16 Da un punto di vista logico la cache si trova tra la CPU e la memoria centrale. Fisicamente può essere collocata in varie posizioni.

Possiamo formalizzare questo calcolo introducendo, il tempo di accesso alla cache, il tempo di accesso alla memoria centrale, la frequenza di successi (*hit ratio*), che corrisponde alla frazione di riferimenti che può essere soddisfatta dalla cache. Indicheremo

tali grandezze rispettivamente con le lettere c, m, e h. Nel semplice esempio del paragrafo precedente si ottiene $h = (k - 1)/k$. Alcuni autori definiscono anche la frequenza di fallimento (*miss ratio*), che vale $1 - h$.

È quindi possibile calcolare il tempo medio di accesso utilizzando la seguente relazione:

$$\text{Tempo medio di accesso} = c + (1 - h)\, m$$

Se h si avvicina a 1, tutti i riferimenti possono essere soddisfatti dalla cache e il tempo medio di accesso si approssima a c. Al contrario, quando h tende a 0 è necessario effettuare ogni volta un riferimento alla memoria e quindi il tempo di accesso medio si avvicina a $c + m$, ovvero alla somma del tempo c necessario per controllare inizialmente (senza successo) la cache e del tempo m per fare successivamente riferimento alla memoria. Su alcuni sistemi il riferimento alla memoria può essere fatto partire parallelamente alla ricerca nella cache, di modo che, se si verifica un fallimento della cache, il ciclo di memoria sia già iniziato. Tuttavia questa strategia richiede che la memoria possa essere bloccata in ogni momento quando si verifica un successo della cache, rendendo l'implementazione più complicata.

Le memorie centrali e le cache sono divise in blocchi di grandezza fissa per trarre vantaggio dal principio di località. Solitamente quando si parla di questi blocchi all'interno della cache, ci si riferisce a essi con il termine **linea di cache**. Quando si verifica un fallimento della cache, è l'intera linea a essere caricata dalla memoria centrale all'interno della cache, e non soltanto la parola richiesta. Per esempio con una linea di 64 byte, un riferimento all'indirizzo di memoria 260 porterà la linea compresa tra i byte 256 e 319 all'interno della linea di cache. Con un po' di fortuna alcune delle altre parole presenti nella linea di cache diventeranno necessarie nel giro di poche istruzioni. È più efficiente seguire questa strategia piuttosto che prelevare singole parole, dato che richiede meno tempo prelevare k parole tutte in una volta che prelevarle una a una in k momenti diversi. Inoltre il fatto di avere elementi di cache più grandi di una singola parola significa che ce n'è un numero minore, il che richiede un overhead inferiore. Infine, molti computer sono in grado di trasferire 64 o 128 bit in parallelo in un singolo ciclo di bus, anche se si tratta di macchine a 32 bit.

La progettazione delle cache sta acquisendo un'importanza sempre maggiore nelle CPU ad alte prestazioni. Un problema riguarda la grandezza della cache; più grande è, migliori sono le sue prestazioni, ma anche il suo costo è maggiore. Un secondo problema è la dimensione della linea di cache. Una cache da 16 KB può essere infatti divisa in 1024 linee di 16 byte, 2048 linee di 8 byte, o in altri modi. Un terzo problema è come deve essere organizzata la cache, cioè come tener traccia di quali parole di memoria sono memorizzate al suo interno in un dato momento.

Un quarto problema consiste nella scelta se mantenere istruzioni e dati in una sola cache oppure in due. Il fatto di avere una **cache unificata** (istruzioni e dati usano la stessa cache) implica una progettazione più semplice e bilancia automaticamente i prelievi delle istruzioni e dei dati. Al giorno d'oggi tuttavia si tende a favorire la **cache specializzata**, in cui le istruzioni sono memorizzate in una cache e i dati in un'altra. Questa organizzazione è chiamata **architettura Harvard**, in riferimento al vecchio

calcolatore Mark III di Howard Aiken, che aveva memorie separate per le istruzioni e per i dati. Ciò che spinge i progettisti in questa direzione è l'utilizzo molto diffuso delle CPU con pipeline in cui l'unità di prelievo dell'istruzione richiede l'accesso alle istruzioni nello stesso momento in cui l'unità di prelievo dell'operando richiede l'accesso ai dati. Le cache specializzate permettono che questi due accessi avvengano in parallelo, mentre con una unificata ciò non è possibile. Inoltre dato che le istruzioni non sono modificate durante l'esecuzione, non vi è mai bisogno di riscrivere in memoria il contenuto della cache delle istruzioni.

Infine un quinto problema è rappresentato dal numero delle cache. Al giorno d'oggi è comune avere chip contenenti una cache primaria, avere una cache secondaria fuori dal chip, ma assemblata sullo stesso circuito della CPU, e avere anche una terza cache un po' più lontana.

2.2.6 Assemblaggio e tipi di memoria

A partire dalle prime memorie basate su semiconduttori e fino ai primi anni '90, la memoria era prodotta, venduta e installata in un unico chip. La densità dei chip passò da 1 Kbit a oltre 1 Mbit, ma ogni chip veniva venduto come unità separata. Spesso i primi PC erano dotati di prese nelle quali era possibile inserire chip di memoria aggiuntivi, se e quando l'utente ne avesse avuto bisogno.

A partire dall'inizio degli anni '90 viene utilizzata una diversa organizzazione. Vari chip, in genere di 8 o 16 elementi, sono montati su una piccola scheda a circuiti stampati venduta singolarmente. Essa è chiamata **SIMM** (*Single Inline Memory Module*, "modulo di memoria con connettore singolo") oppure **DIMM** (*Dual Inline Memory Module*, "modulo di memoria con doppia linea di contatti"), a seconda che abbia i connettori allineati su uno o su due lati della scheda. Le SIMM, poco utilizzate ai giorni nostri, hanno un connettore laterale con 72 contatti e trasferiscono 32 bit per ciclo di clock; le DIMM invece hanno generalmente connettori con 120 contatti su ogni lato, per un totale di 240 contatti, e trasferiscono 64 bit per ciclo di clock. Le DIMM più diffuse oggi sono le DDR3, la terza versione delle memorie a doppia velocità (*Double Data Rate*). La Figura 2.17 mostra lo schema tipico di una DIMM.

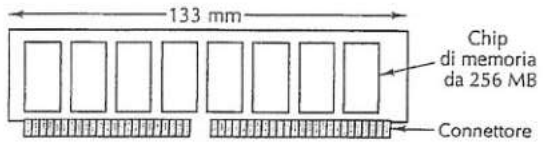


Figura 2.17 Vista di un modulo DIMM da 4GB con otto chip da 256 MB su ogni lato. L'altro lato è identico.

Una tipica configurazione di SIMM o DIMM può avere otto chip di dati da 256 MB ciascuno, per un totale di 2 GB sull'intero modulo. Molti calcolatori hanno spazio per quattro moduli, permettendo una capacità totale di 8 GB in caso di utilizzo di moduli da 2 GB, o maggiore se si utilizzano memorie più grandi.

Nei calcolatori portatili si usano DIMM più piccole, chiamate **SO-DIMM** (*Small Outline DIMM*, "DIMM dal profilo compatto"). Alle SIMM e alle DIMM è possibile aggiungere un bit di parità o la correzione d'errore, ma, dato che la frequenza media degli errori di un modulo è di uno ogni 10 anni, spesso nella maggior parte dei calcolatori ordinari le funzionalità di rilevazione e correzione degli errori non sono presenti.

2.3 Memoria secondaria

La memoria centrale non è mai troppo grande. Si vorrebbero sempre memorizzare più informazioni rispetto alle reali capacità; la ragione principale è che, grazie ai miglioramenti della tecnologia, si comincia a voler memorizzare cose che precedentemente ricadevano interamente nel regno della fantascienza. Per esempio, visto che il governo degli USA obbliga gli enti governativi a pubblicare i propri bilanci, si potrebbe immaginare che la Biblioteca del Congresso decida di digitalizzare e mettere in commercio tutto il proprio contenuto ("Tutta la conoscenza umana a soli $ 299,95"). Memorizzare all'incirca 50 milioni di libri, ciascuno con 1 MB di testo e 1 MB di immagini compresse, richiede 10^{14} byte, vale a dire 100 Terabyte. La memorizzazione di tutti i 50.000 film finora prodotti richiederebbe più o meno lo stesso spazio. Questa quantità d'informazione, almeno per alcuni decenni, sarà troppo grande per poter essere contenuta in memoria centrale.

2.3.1 Gerarchie di memoria

La soluzione che viene tradizionalmente adottata per memorizzare una gran mole di dati consiste nell'organizzare gerarchicamente la memoria, com'è mostrato nella Figura 2.18. Nella parte alta della gerarchia si trovano i registri della CPU, ai quali si può accedere alla stessa velocità della CPU. Più sotto vi è la memoria cache, la cui dimensione può variare da 32 KB fino ad alcuni megabyte. La memoria centrale è il passo successivo e la sua dimensione è compresa tra 1 (per i sistemi più economici e centinaia di gigabyte (per quelli professionali). Troviamo poi i dischi magnetici, la vera forza lavoro della memorizzazione permanente. Infine ci sono i nastri magnetici e i dischi ottici utilizzati per l'archiviazione.

Muovendosi verso il basso della gerarchia aumentano tre parametri chiave. Innanzitutto, il tempo di accesso diventa via via più grande. Ai registri della CPU è possibile accedere in un nanosecondo, o meno. Le memorie cache richiedono invece un tempo che è un piccolo multiplo di quello dei registri della CPU, mentre alle memorie centrali è generalmente possibile accedere in una decina di nanosecondi. A questo punto si incontra un grande salto, dato che il tempo di accesso ai dischi è maggiore di almeno 10 volte nel caso dei dischi a stato solido e di centinaia di volte nel caso dei dischi magnetici. L'accesso ai nastri e ai dischi ottici può essere misurato in secondi se i supporti devono essere presi e inseriti in un lettore.

Secondariamente, la capacità di memorizzazione aumenta man mano si scende verso il basso: i registri della CPU vanno bene per immagazzinare circa 128 byte, le cache per decine di megabyte, le memorie centrali per pochi gigabyte, i dischi a stato solido per

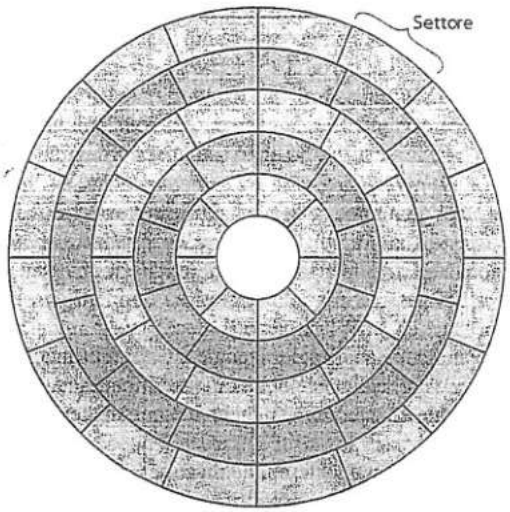


Figura 2.21 Disco con cinque zone. Ciascuna zona ha più tracce.

2.3.3 Dischi IDE

I dischi dei moderni personal computer sono un'evoluzione di quello che era presente nel PC XT della IBM; un disco da 10 MB della Seagate gestito da un controllore prodotto dalla Xebec collocato su una scheda aggiuntiva. Il disco della Seagate aveva 4 testine, 306 cilindri e 17 settori per traccia e il controllore era in grado di gestire due unità. Il sistema operativo leggeva dal disco e scriveva su di esso inserendo alcuni parametri nei registri della CPU e chiamando successivamente il **BIOS** (*Basic Input Output System*, "sistema elementare di input/output"), situato in una memoria di sola lettura integrata nel PC. Il BIOS lanciava le istruzioni macchina necessarie per caricare i registri del controllore, che iniziava il trasferimento dei dati.

La tecnologia evolse rapidamente e a metà degli anni '80 si passò alle prime unità **IDE** (*Integrated Drive Electronics*, "memoria di massa con elettronica integrata") in cui il controllo era strettamente integrato con l'unità, mentre prima si trovava su una scheda separata. Tuttavia, per ragioni di retrocompatibilità, le chiamate al BIOS non furono modificate. Secondo queste convenzioni l'indirizzamento dei settori avveniva specificando i numeri della loro testina, del cilindro e del settore, con la numerazione delle testine e dei cilindri che partiva da 0, mentre quella dei settori partiva da 1. Probabilmente questa scelta fu dovuta a un errore da parte del programmatore del BIOS originario, che scrisse il suo capolavoro usando l'assemblatore dell'8088. Utilizzando 4 bit per la testina, 6 bit per il settore e 10 bit per il cilindro, un'unità poteva avere al massimo 16 testine, 63 settori e 1024 cilindri, per un totale di 1.032.192 settori. Una simile unità poteva avere una capacità massima di 504 MB, che all'epoca poteva sembrare praticamente infinita, ma che al giorno d'oggi non lo è certamente. (Si criticherebbe oggi una macchina che non sia in grado di gestire dei dischi più grandi di un 1000 TB?)

Sfortunatamente, quando in seguito apparvero unità più capaci, la loro struttura (per esempio 4 testine, 32 settori, 2000 cilindri, cioè 256.000 settori) non poteva essere sfruttata; a causa delle convenzioni sulle chiamate al BIOS, rimaste per lungo tempo immutate, il sistema operativo non poteva infatti indirizzare questi dischi. Per aggirare il problema i controllori dei dischi cominciarono a imbrogliare, richiedendo che la geometria fosse all'interno dei limiti del BIOS, ma rimappando in realtà la geometria virtuale su quella reale. Pur funzionando, questo approccio rovinava però il lavoro del sistema operativo, che prestava attenzione nel collocare i dati in modo da minimizzare i tempi di ricerca.

Le unità IDE evolsero successivamente nelle unità **EIDE** (*Extended IDE*) che supportavano anche un secondo schema d'indirizzamento chiamato **LBA** (*Logical Block Addressing*, "indirizzamento logico dei blocchi"), che semplicemente numerava i settori a partire da 0 e fino a un massimo di $2^{28} - 1$. Questo schema, anche se richiedeva che il controllore convertisse gli indirizzi LBA negli indirizzi della testina, del settore e del cilindro, permetteva di superare il limite dei 504 MB. Sfortunatamente questa modalità d'indirizzamento creò un nuovo collo di bottiglia corrispondente a $2^{28} \times 2^9$ byte (128 GB). Quando nel 1994 fu adottato lo standard EIDE, nessuno, nemmeno con la più fervida immaginazione, poteva immaginare dischi di quelle dimensioni. I comitati degli standard, allo stesso modo dei politici, hanno la tendenza a rinviare nel tempo i problemi, in modo che sia il comitato successivo a doverli risolvere.

Le unità e i controllori EIDE presentavano anche altri miglioramenti. I controllori EIDE, per esempio, potevano avere due canali, ciascuno dei quali poteva avere un disco primario e uno secondario. Questa organizzazione ammetteva un massimo di quattro unità per controllore. Vennero supportati anche i lettori di CD-ROM e DVD e la velocità di trasferimento dati passò da 4 MB/s a 16,67 MB/s.

Mentre la tecnologia dei dischi continuava a migliorare, evolveva anche lo standard EIDE; per qualche strana ragione il suo successore fu però chiamato **ATA-3** (*AT Attachment*, "tecnologia avanzata per la connessione"), con un riferimento al PC/AT IBM (dove AT si riferiva alla allora Tecnologia Avanzata di una CPU a 16 bit che andava alla velocità di 8 MHz). Nella versione successiva lo standard fu chiamato **ATAPI-4** (*ATA Packet Interface*, "interfaccia per pacchetti ATA") e la velocità fu aumentata fino a 33 MB/s. Nello standard ATAPI-5 la velocità raggiunse invece i 66 MB/s.

A questo punto il limite dei 128 GB imposto dagli indirizzi LBA a 28 bit cominciò a profilarsi sempre più minaccioso, e quindi in ATAPI-6 la dimensione di LBA fu portata a 48 bit. Il nuovo standard finirà nei guai quando i dischi raggiungeranno la dimensione di $2^{48} \times 2^9$ byte. Con un aumento annuale del 50% della capacità, il limite dovuto ai 48 bit dovrebbe sopravvivere fino al 2035 circa (per scoprire la soluzione a questo problema si consulti l'undicesima edizione di questo libro!). I più astuti stanno già scommettendo su un aumento della dimensione di LBA a 64 bit. Lo standard ATAPI-6 ha anche aumentato la velocità di trasferimento dati a 100 MB/s e ha affrontato per la prima volta il problema del rumore dei dischi.

Lo standard ATAPI-7 rappresenta una rottura radicale con il passato. Questo standard, invece di aumentare la dimensione del connettore del disco (per migliorare la velocità di trasferimento), usa quella che viene chiamata interfaccia **serial ATA** per

trasferire 1 bit alla volta su un connettore a 7 pin a una velocità che parte da 150 MB/s, ma che nel tempo dovrebbe raggiungere 1.5 GB/s. La sostituzione di un cavo piatto a 80 fili con un cavo rotondo di pochi millimetri di spessore migliora la circolazione dell'aria all'interno del calcolatore. Inoltre l'interfaccia serial ATA utilizza 0,5 volt per i segnali (rispetto ai 5 V delle unità ATAPI-6), riducendo il consumo di energia. Il problema del consumo energetico dei dischi sta acquistando una grande importanza, sia nei sistemi di fascia alta, sia in quelli di fascia bassa, dove i portatili hanno un'alimentazione limitata (Gurumurthi et al., 2003).

2.3.4 Dischi SCSI

Per quanto riguarda l'organizzazione di cilindri, tracce e settori, i dischi SCSI (si pronuncia *scasi*) non sono differenti da quelli IDE, ma hanno una diversa interfaccia e una velocità di trasferimento dati molto più elevata. Le origini dei dischi SCSI risalgono a Howard Shugart, inventore dei floppy disk, utilizzati nei primi modelli di personal computer negli anni '80. La sua società introdusse nel 1979 il disco SASI (*Shugart Associates System Interface*, "sistema d'interfaccia Shugart Associates"). Nel 1986, dopo alcune modifiche e molte polemiche, venne standardizzato dalla ANSI e il suo nome cambiò in **SCSI** (*Small Computer System Interface*, "sistema d'interfaccia per piccoli sistemi"). A partire da quel momento sono state standardizzate versioni sempre più veloci con il nome di Fast SCSI (10 MHz), Ultra SCSI (20 MHz), Ultra2 SCSI (40 MHz), Ultra3 SCSI (80 MHz), Ultra4 SCSI (160 MHz) e Ultra5 SCSI (320 MHz). Ciascuna di queste interfacce aveva anche una versione *wide* ("larga") a 16 bit, l'unica utilizzata ai giorni nostri; la Figura 2.22 mostra le combinazioni principali.

Nome	Bit di dati	MHz del bus	MB/s
SCSI-1	8	5	5
Fast SCSI	8	10	10
Wide Fast SCSI	16	10	20
Ultra SCSI	8	20	20
Wide Ultra SCSI	16	20	40
Ultra2 SCSI	8	40	40
Wide Ultra2 SCSI	16	40	80
Ultra3 SCSI	8	80	80
Wide Ultra3 SCSI	16	80	160
Ultra4 SCSI	8	160	160
Wide Ultra4 SCSI	16	160	320
Wide Ultra5 SCSI	16	320	640

Figura 2.22 Parametri dei modelli SCSI.

Per via della velocità di trasferimento più elevata, i dischi SCSI sono lo standard in workstation e server di fascia alta, in particolare nelle macchine con configurazione RAID (si veda più avanti).

SCSI non è soltanto un'interfaccia per hard disk, ma è anche un bus al quale possono essere collegati un controllore e sette dispositivi. Questi possono comprendere uno o più hard disk SCSI, CD-ROM, masterizzatori, scanner, unità a nastro e altre periferiche SCSI. Ciascun dispositivo SCSI possiede un ID, compreso tra 0 e 7 (15 nel caso di *wide* SCSI), e due connettori, uno per l'input e uno per l'output. I cavi connettono in serie l'output di un dispositivo con l'input del successivo, come un filo di lucine natalizie. L'ultimo dispositivo della fila deve avere una terminazione per evitare che le riflessioni nell'estremità del bus SCSI interferiscano con altri dati lungo il bus. In genere il controllore si trova su una scheda aggiuntiva all'inizio della catena, anche se ciò non è esplicitamente richiesto dallo standard.

Il più comune cavo per lo standard SCSI a 8 bit ha 50 contatti, 25 dei quali sono cavi di terra accoppiati uno a uno con altri 25 in modo da fornire un'eccellente immunità al rumore, necessaria per poter operare ad alte velocità. Dei 25 cavi 8 sono per i dati, 1 per la parità, 9 per il controllo, mentre i rimanenti sono per l'alimentazione oppure riservati per usi futuri. I dispositivi a 16 bit (e quello a 32 bit) richiedono un secondo cavo per trasportare i segnali aggiuntivi. I cavi possono avere una lunghezza di vari metri, permettendo l'utilizzo di hard disk, scanner e altre periferiche esterne.

I controllori SCSI possono funzionare come iniziatori o come destinatari di comunicazione. Di solito il controllore funziona come iniziatore e lancia comandi ai dischi e ad altre periferiche; questi comandi sono blocchi della dimensione massima di 16 byte che indicano al destinatario che cosa deve fare. I comandi e le risposte avvengono in fase, utilizzando vari segnali di controllo per delimitare la fase e arbitrare l'accesso al bus quando più dispositivi tentano di usarlo nello stesso momento. Questo arbitraggio è importante in quanto SCSI consente a tutti i dispositivi di funzionare contemporaneamente, aumentando potenzialmente le prestazioni in un ambiente in cui vi possono essere più processi attivi allo stesso tempo (per esempio UNIX o Windows XP); IDE ed EIDE ammettono invece un solo dispositivo attivo a ogni istante.

2.3.5 RAID

Nel corso dell'ultimo decennio le prestazioni delle CPU sono aumentate in modo esponenziale, raddoppiando all'incirca ogni 18 mesi, mentre la stessa cosa non si è verificata per i dischi. Negli anni '70 il tempo medio di ricerca nei dischi dei minicomputer era compreso tra 50 e 100 ms, mentre ora è di 10 ms. In molti settori dell'industria (come quella automobilistica o aeronautica) un miglioramento delle prestazioni di un fattore 5 o 10 nell'arco di due decenni sarebbe una notizia di rilievo, ma nel settore dei computer è un dato imbarazzante. Per questo motivo la differenza tra le prestazioni della CPU e quella dei dischi è aumentata con il tempo in modo considerevole.

Come già visto, spesso si utilizza l'elaborazione parallela per velocizzare le prestazioni delle CPU. Quindi anche l'I/O parallelo è sembrato ad alcuni una buona idea. In un articolo del 1988 (Patterson et al., 1988) fu suggerita un'organizzazione a sei dischi per migliorare le prestazioni, l'affidabilità oppure entrambe allo stesso tempo. Questa

idea fu adottata velocemente dal mercato e portò a una nuova classe di dispositvi di I/O chiamata RAID. Patterson e i suoi colleghi definirono **RAID** come **Redundant Array of Inexpensive Disks** ("insieme ridondante di dischi economici"), ma il mercato ridefinì la "I" di *Inexpensive* ("economico") con il termine *Independent* ("indipendente"): forse per poter utilizzare dischi costosi? Dato che c'è sempre bisogno di un nemico (come in CISC contro RISC, anch'esso dovuto a Patterson), il ruolo dei cattivi fu assunto da **SLED** (*Single Large Expensive Disk*, "singolo disco capiente e costoso").

L'idea su cui si basa RAID è quella di installare vicino al calcolatore, di solito un server di grandi dimensioni, un contenitore pieno di dischi, di sostituire la scheda contenente il controllore del disco con un controllore RAID, di copiare i dati sul RAID e di continuare quindi le normali operazioni. In altre parole un RAID dovrebbe apparire agli occhi del sistema operativo come uno SLED, seppur con prestazioni e affidabilità più elevate. Dato che i dischi SCSI offrono buone prestazioni, prezzo basso e possibilità di avere fino a 7 unità su un singolo controllore (15 per l'interfaccia *wide* SCSI) è naturale che la maggior parte dei sistemi RAID consista di un controllore RAID SCSI, più un insieme di dischi SCSI che appaiono al sistema operativo come un'unica e grande unità. In questo modo non sono necessarie modifiche al software per poter usare RAID, il che rappresenta un grande vantaggio per gli amministratori di sistema.

Tutti i RAID, oltre ad apparire al software come un singolo disco, hanno la proprietà di distribuire i dati sulle diverse unità per permetterne la gestione parallela. Il gruppo di Patterson ha definito diversi schemi RAID, conosciuti con nomi che vanno da RAID livello 0 a RAID livello 5; esistono anche altri livelli di minor importanza che però non tratteremo. Il termine "livello" è in un certo modo fuorviante, in quanto non esiste alcuna gerarchia; si tratta semplicemente di sei diverse possibili organizzazioni, ognuna con un diverso mix di caratteristiche di affidabilità e prestazioni.

Il RAID livello 0 è mostrato nella Figura 2.23(a). Secondo questa organizzazione il disco virtuale simulato dal RAID è visto come se fosse suddiviso in *strip* ("strisce") di k settori ciascuna, con i settori da 0 a $k-1$ che compongono la *strip* 0, i settori da k a $2k-1$ la *strip* 1 e così via; se $k = 1$ ogni *strip* è un settore, se $k = 2$ una *strip* è composta da due settori e così via. L'organizzazione RAID livello 0 scrive sulle *strip* consecutive in modo ciclico (modalità *round robin*), come mostra la Figura 2.23(a) nel caso di un RAID con quattro unità. Questo modo di distribuire dati su più unità è chiamato **striping**. Se per esempio il software lancia un comando per leggere un blocco di dati costituito da quattro *strip* consecutive a partire dall'inizio di una *strip*, il controllore RAID spezzerà questo comando in quattro comandi separati, uno per ciascuno dei quattro dischi, e li farà eseguire in parallelo. In questo modo si ottiene un I/O parallelo senza che il software ne sia a conoscenza.

Il RAID livello 0 lavora meglio quando le richieste sono di grandi dimensioni. Se la dimensione di una richiesta supera il numero di unità moltiplicato per la dimensione di una *strip*, allora alcune unità riceveranno più richieste, che verranno soddisfatte una dopo l'altra. Spetta al controllore separare la richiesta e assegnare alle unità appropriate i comandi corretti nella giusta sequenza, oltre ad assemblare poi, correttamente, i risultati in memoria. Le prestazioni sono eccellenti e l'implementazione è semplice.

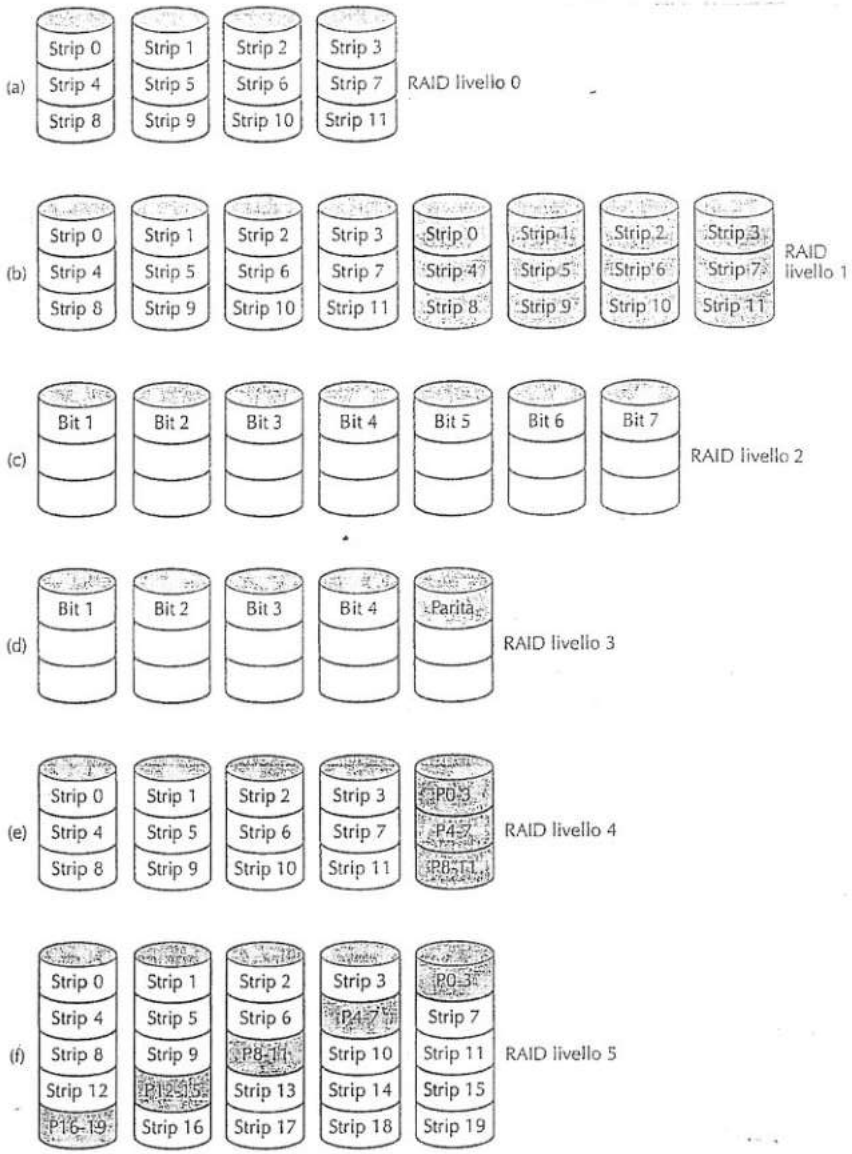


Figura 2.23 RAID dal livello 0 al livello 5. I dischi di backup e di parità sono in grigio.

Il RAID livello 0 lavora peggio con i sistemi operativi che tendono a richiedere i dati un settore alla volta. In una simile situazione il risultato sarà comunque corretto, ma non vi sarà parallelismo da sfruttare e quindi non si otterrà alcun guadagno nelle prestazioni. Un altro svantaggio di questa organizzazione è che l'affidabilità è potenzialmente peggiore di quella che si ha con uno SLED. Se un RAID consiste di quattro dischi, ciascuno con un intervallo medio tra guasti di 20.000 ore, all'incirca ogni 5000 ore un disco si guasterà e i dati saranno interamente persi; uno SLED con un intervallo medio tra guasti di 20.000 ore sarebbe quattro volte più affidabile. Inoltre questo tipo di progettazione non è in realt un vero e proprio RAID, in quanto non c'è alcuna ridondanza.

Il RAID livello 1, mostrato nella Figura 2.23(b), è invece un vero RAID in quanto duplica tutti i dischi, risultando così composto. in questo esempio, da quattro dischi primari e da quattro di backup, ma è anche possibile un qualsiasi altro numero pari di dischi. Nel caso di una scrittura ogni *strip* viene scritta due volte, mentre nel caso di una lettura è possibile usare entrambe le copie, distribuendo il carico di lavoro su più unità. Ne consegue che le prestazioni in scrittura non sono migliori di quelle che si otterrebbero con una singola unità, mentre quelle in lettura possono essere fino a due volte migliori. La tolleranza ai guasti è eccellente: se un disco si rompe, si può semplicemente utilizzare la copia. La riparazione consiste nell'installare un nuovo disco e copiarvi l'intero disco di backup.

Diversamente dai livelli 0 e 1 che lavorano con *strip* di settori, il RAID livello 2 funziona sulla base di una parola o, in alcuni casi, anche sulla base di un byte. Immaginiamo di dividere ciascun byte del singolo disco virtuale in una coppia di *nibble* (mezzo byte) di 4 bit, e di aggiungere a ciascuno di loro un codice di Hamming per formare una parola di 7 bit, in cui i bit 1, 2 e 4 sono bit di parità. Immaginiamo inoltre che i sette dischi della Figura 2.23(c) siano sincronizzati per quanto riguarda le posizioni dei bracci e delle rotazioni. In questo modo sarebbe possibile scrivere una parola del codice di Hamming a 7 bit sui sette dischi, usando un bit per ogni disco.

Il calcolatore Thinking Machines CM-2 utilizzava questo schema, prendendo parole di dati a 32 bit, aggiungendovi 6 bit di parità per formare una parola di Hamming a 38 bit, oltre a un bit extra per la parità dell'intera parola, e distribuendo ciascuna parola su 39 dischi. Il *throughput* totale era estremamente elevato, dato che nel tempo necessario per accedere a un settore era possibile scrivere una quantità di dati pari a 32 settori. Inoltre la perdita di un disco non causava problemi, in quanto comportava la perdita di 1 solo bit per ciascuna parola a 39 bit che veniva letta, perdita che il codice di Hamming riesce a gestire autonomamente.

Questa organizzazione presenta però alcuni difetti: la rotazione dei dischi deve essere sincronizzata e lo schema ha senso soltanto se si utilizza un numero significativo di unità (anche con 32 dischi di dati e 6 dischi di parità l'overhead è del 19%). Inoltre si richiede molto lavoro al controllore, dato che deve calcolare la *checksum* del codice di Hamming a ogni tempo di bit.

Il RAID livello 3, mostrato nella Figura 2.23(d), è una versione semplificata del RAID livello 2. Il bit di parità viene calcolato per ogni parola di dati e poi scritto su un apposito disco. Dato che le parole di dati sono distribuite su più unità, anche in questo caso, come per il RAID livello 2, i dischi devono essere sincronizzati. A prima vista potrebbe sembrare che un solo bit di parità possa permettere la sola rilevazione degli errori, ma non la loro correzione. Nel caso di errori casuali, questa osservazione è vera; tuttavia, nel caso di una rottura di un disco, lo schema permette la correzione completa degli errori singoli, in quanto la posizione del bit errato è conosciuta. Se un disco si guasta il controllore assume semplicemente che tutti i suoi bit valgano 0; se una parola ha un valore errato di parità, allora il bit del disco guastatosi doveva per forza valere 1 e poteva quindi essere corretto. Anche se i RAID livello 2 e livello 3 permettono un tasso di trasferimento dati elevato, il numero di richieste di I/O al secondo è lo stesso che per un singolo disco.

Anche i RAID livello 4 e livello 5 lavorano in base alle *strip*, e non a singole parole con parità; inoltre non richiedono la sincronizzazione tra i dischi. Il RAID livello 4 [vedi Figura 2.23(e)] è come il RAID livello 0, con una parità *strip*-per-*strip* scritta su un disco aggiuntivo. Per esempio, se ogni *strip* è lunga k byte, si calcola l'OR ESCLUSIVO fra tutte, ottenendo una *strip* di parità lunga k byte. Se un disco si guasta è possibile ricalcolare i byte persi grazie al disco di parità.

Questo schema protegge dalla perdita di un disco, ma ha prestazioni scarse quando si aggiornano piccole quantità di dati. Infatti, se si modifica un settore è necessario leggere tutti i dischi, in quanto occorre ricalcolare e riscrivere la parità. Una soluzione alternativa consiste nel leggere i precedenti dati e valori di parità e, a partire da questi, calcolare quelli nuovi. Anche ricorrendo a questa ottimizzazione una piccola modifica dei dati richiede comunque due letture e due scritture; un'organizzazione dei dischi di questo tipo è chiaramente inefficiente.

Il disco di parità può inoltre diventare un collo di bottiglia, a causa del grande carico di lavoro che pesa su di esso. Il RAID livello 5 elimina questo collo di bottiglia distribuendo uniformemente i bit di parità su tutti i dischi, in modalità *round robin*, com'è mostrato nella Figura 2.23(f). Tuttavia, quando si verifica un guasto a un disco, la ricostruzione del suo contenuto è un processo complesso.

2.3.6 Dischi a stato solido

I dischi basati su memoria flash non volatile, spesso chiamati dischi a stato solido (SSD), stanno diffondendosi sempre di più come un'alternativa ad alta velocità alle tradizionali tecnologie a disco magnetico. L'invenzione della SSD è una classica storia del tipo "Quando ti danno i limoni, fai una limonata". Anche se l'elettronica moderna può sembrare totalmente affidabile, in realtà i transistor si consumano lentamente con l'utilizzo: ogni volta che entrano in funzione si consumano un po' di più e si avvicina il momento della fine del loro ciclo di vita. Un guasto di cui può soffrire un transistor è dovuto alla cosiddetta "iniezione a caldo" (*hot carrier injection*), un meccanismo di avaria per cui una carica di elettroni viene rinchiusa all'interno del transistor, lasciandolo in uno stato permanente di blocco, acceso o spento. Nonostante questo meccanismo sia stato generalmente considerato come una condanna a morte per un (presunto) innocente transistor, Fujio Masuoka, mentre lavorava per Toshiba, ha scoperto un modo per sfruttarlo per creare una nuova memoria non volatile e, nei primi anni 1980, ha inventato la prima memoria flash.

I CD-R permettono ai privati e alle società di copiare facilmente i CD-ROM (e i CD audio), spesso in violazione dei diritti d'autore. Sono stati ideati vari schemi per ostacolare questa pirateria e far sì che sia difficile leggere un CD-ROM se non utilizzando il software fornito dall'autore dello stesso. Uno di questi sistemi prevede di registrare le lunghezze di tutti i file del CD-ROM come se fossero di vari gigabyte, ostacolando ogni tentativo da parte dei normali software di copiatura di trasferirli su hard disk. Le vere lunghezze sono memorizzate all'interno del software del produttore oppure sono nascoste (possibilmente criptate) in un'area inaspettata all'interno del CD-ROM. Un altro schema utilizza intenzionalmente ECC errati in specifici settori, con l'aspettativa che il programma di copiatura li "correggerà". Il software dell'applicazione controllerà anch'esso gli ECC e si rifiuterà di funzionare se essi sono corretti. Altre possibilità prevedono l'utilizzo di intervalli non standard tra le tracce e lo sfruttamento di altri "difetti" fisici.

2.3.9 CD-riscrivibili

Anche se abitualmente vengono usati supporti che permettono un'unica scrittura, come la carta o la pellicola fotografica, c'è una grande richiesta di CD-ROM riscrivibili. Una tecnologia oggi disponibile è quella dei **CD-RW** (*CD-ReWritable*, CD-Riscrivibili), che usano supporti della stessa dimensione dei CD-R. I CD-RW però, invece di usare la cianina o la ftalocianina per lo strato sul quale registrare, impiegano una lega di argento, indio, antimonio e tellurio. Questa lega ha due stati stabili, quello cristallino e quello amorfo, con diverse proprietà riflettenti.

I lettori CD-RW utilizzano laser che funzionano a tre potenze distinte. Alla potenza più elevata il laser scioglie la lega portandola dallo stato cristallino altamente riflettente a quello amorfo, dotato di una riflettività minore e che rappresenta un *pit*. Alla potenza media la lega si scioglie e si ricompone nello stato cristallino naturale per tornare a rappresentare un *land*. Alla potenza più bassa è possibile rilevare lo stato del materiale (per operazioni di lettura), senza però indurre alcuna trasformazione.

I CD-RW vergini sono molto più costosi dei CD-R vergini e per questo non li hanno sostituiti completamente. Inoltre il fatto che, una volta scritto, un CD-R non possa essere cancellato accidentalmente, rappresenta un gran vantaggio per il backup dei dischi, e non un difetto.

2.3.10 DVD

Il formato base dei CD/CD-ROM è stato definito attorno al 1980. Dalla metà degli anni '90 la tecnologia è notevolmente migliorata, al punto che oggi è possibile realizzare dischi ottici di maggiore capacità in modo economicamente conveniente. Nello stesso periodo, Hollywood stava cercando un modo per sostituire le videocassette analogiche con dischi a tecnologia ottica che offrissero una qualità più elevata, che fossero più economici da produrre, che durassero più a lungo, che occupassero meno spazio sugli scaffali dei negozi e che non dovessero essere riavvolti. Sembrava che per i dischi ottici la ruota del progresso stesse per girare ancora una volta.

Questa combinazione tra tecnologia e domanda proveniente da tre mercati estremamente ricchi e potenti ha portato al **DVD**, che originalmente era l'acronimo di *Digital Video Disk* (disco video digitale), mentre ora significa ufficialmente *Digital Versatile Disk* (disco digitale versatile). I DVD sono progettati in modo simile ai CD: si inietta policarbonato in uno stampo e sulla superficie si possono distinguere *pit* e *land* che vengono illuminati da un diodo laser e letti da un fotorilevatore. Le novità riguardano l'uso di:

1. *pit* più piccoli (0,4 μ contro gli 0,8 dei CD);
2. una spirale più stretta (0,74 μ di distanza tra le tracce rispetto agli 1,6 dei CD);
3. un laser rosso (a 0,65 μ invece che a 0,78 come nei CD).

Insieme, questi miglioramenti permettono una capacità sette volte maggiore, che arriva fino a 4,7 GB. Un DVD 1x ha una velocità di 1,4 MB/s (contro i 150 KB/s dei CD). Sfortunatamente il passaggio ai laser rossi, come quelli usati nei supermercati, obbliga i lettori DVD ad avere un secondo laser per poter leggere i CD e i CD-ROM esistenti, cosa che aggiunge un po' di complessità e aumenta i costi.

La capacità di 4,7 GB è sufficiente? Forse. Utilizzando la compressione MPEG-2 (standardizzata in IS 13346) un disco DVD da 4,7 GB può memorizzare 133 minuti di video a schermo intero in alta risoluzione (720 × 480), più di otto diverse colonne sonore e sottotitoli in 32 lingue diverse. Circa il 92% dei film che Hollywood ha finora prodotto ha una durata inferiore ai 133 minuti. Tuttavia, dato che alcune applicazioni, come i giochi multimediali o le enciclopedie, potrebbero richiedere quantità di dati più elevate e dato che Hollywood vorrebbe inserire più film sullo stesso disco, sono stati definiti quattro formati:

1. singolo lato, singolo strato (4,7 GB);
2. singolo lato, doppio strato (8,5 GB);
3. doppio lato, singolo strato (9,4 GB);
4. doppio lato, doppio strato (17 GB).

Perché così tanti formati? In una parola sola: politica. Philips e Sony volevano dischi a singolo lato e doppio strato per le versioni ad alta capacità, mentre Toshiba e Time Warner volevano dischi a doppio lato e doppio strato. Philips e Sony ritenevano che la gente non fosse disposta a girare i dischi per cambiar lato, mentre Time Warner non credeva che potesse funzionare l'utilizzo di due strati su un solo lato. Il compromesso fu quello di adottare tutte le soluzioni e di lasciare al mercato il compito di determinare quale sarebbe sopravvissuta. Alla fine il mercato emise il suo verdetto: Philips e Sony avevano ragione. Mai scommettere contro la tecnologia.

La tecnologia a doppio strato ha uno strato riflettente nella parte bassa, coperto da uno semiriflettente. A seconda del punto in cui il laser è messo a fuoco, esso rimbalza su un livello oppure sull'altro. Il livello più basso richiede che i *pit* e i *land* siano leggermente più grandi per essere leggibili in modo affidabile, e per questo motivo la sua capacità è leggermente inferiore rispetto a quella dello strato superiore.

I dischi a doppio lato sono creati prendendo due dischi a singolo lato da 0,6 mm di spessore e incollandoli l'uno contro l'altro, cioè "schiena contro schiena". Per far sì che tutte le versioni abbiano lo stesso spessore, un disco a singolo lato è composto da un disco spesso 0,6 mm incollato con un sottostrato vuoto (o forse, in futuro, con uno contenente 133 minuti di pubblicità, nella speranza che la gente sia curiosa e giri il disco per vedere che cosa c'è sull'altro lato). La Figura 2.28 illustra la struttura del disco a doppio strato e doppio lato.

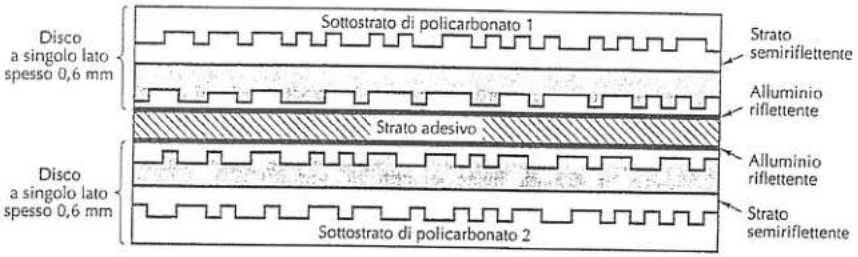


Figura 2.28 Disco DVD a doppio lato e doppio strato.

Il DVD è stato ideato da un consorzio di 10 aziende produttrici, sette delle quali giapponesi, in stretta collaborazione con i principali studi di produzione di Hollywood (alcuni dei quali sono di proprietà delle aziende giapponesi facenti parte del consorzio). Le società di computer e delle telecomunicazioni non furono invitate a questo "picnic" e l'attenzione finì per concentrarsi sull'utilizzo del DVD per il noleggio di film. Tra le funzionalità standard sono presenti, per esempio, l'omissione in tempo reale delle scene scabrose (per permettere ai genitori di rendere adatto ai bambini un film vietato ai minorenni), l'audio a sei canali e il supporto per il *Pan-and-Scan*. Quest'ultima funzionalità permette di decidere dinamicamente come tagliare le bande laterali dei film (il cui rapporto larghezza-altezza è di 3:2) per essere visualizzate sugli attuali schermi televisivi (che hanno un rapporto di forma di 4:3).

Un altro aspetto che molto probabilmente non sarebbe stato tenuto in conto dalle società di computer è la scelta intenzionale di rendere incompatibili i dischi previsti per gli Stati Uniti rispetto a quelli previsti per l'Europa, e anche per gli altri continenti. Hollywood ha preteso questa "caratteristica" poiché i nuovi film escono sempre prima negli Stati Uniti e vengono distribuiti in Europa solo quando negli Stati Uniti sono ormai in vendita i video.

L'idea era quella di impedire che i negozi di video europei potessero comprare i video negli Stati Uniti prima dell'uscita in Europa, riducendo quindi gli introiti delle sale cinematografiche. Se Hollywood fosse stata il motore del mercato dei calcolatori avremmo avuto floppy disk da 3,5 pollici negli Stati Uniti e floppy disk da 9 cm in Europa.

2.3.11 Blu-Ray

Nel mercato dei calcolatori nulla rimane fermo, tanto meno la tecnologia per la memorizzazione dei dati. Il DVD è stato appena introdotto e già il suo successore minaccia di renderlo obsoleto; si tratta del **Blu-Ray**, chiamato in questo modo poiché utilizza un laser blu invece di quello rosso dei DVD. I laser blu hanno una lunghezza d'onda più piccola di quelli rossi, il che permette una messa a fuoco più accurata e l'uso di *pit* e *land* più piccoli. I dischi Blu-Ray a singolo lato contengono circa 25 GB di dati; quelli a doppio lato 50 GB. La velocità di trasferimento dati è di circa 4,5 MB/s, buona per essere un disco ottico, ma ancora molto lontana da quella dei dischi magnetici (per esempio ATAPI-6 a 100 MB/s e Ultra5 SCSI a 640 MB/s). Ci si aspetta che alla fine Blu-Ray sostituirà i CD-ROM e i DVD, ma questo passaggio potrebbe richiedere degli anni.

2.4 Input/Output

Com'è stato detto all'inizio di questo capitolo, un calcolatore è composto da tre componenti principali: la CPU, le memorie e i dispositivi di **I/O**. Finora abbiamo analizzato la CPU e le memorie; adesso passiamo allo studio dei dispositivi di I/O e alla loro connessione con il resto del sistema.

2.4.1 Bus

Da un punto di vista fisico la struttura della maggior parte dei calcolatori e delle workstation è simile a quella mostrata nella Figura 2.29. L'organizzazione più comune consiste in una scatola metallica contenente una grande scheda di circuiti stampati nella parte bassa, chiamata **scheda madre** (scheda genitore, sembra più *politically correct*). La scheda madre contiene il chip della CPU, alcuni slot nei quali è possibile inserire i moduli DIMM e vari altri chip di supporto. Contiene inoltre un bus stampato lungo la sua lunghezza e alcune prese nelle quali è possibile inserire i connettori delle schede di I/O (il bus PCI). I PC più vecchi hanno anche un secondo bus (il bus ISA) per connettere le schede di I/O costruite molto tempo prima; di solito nei calcolatori più moderni questo bus non è presente e il suo utilizzo sta rapidamente scomparendo.

La struttura logica di un semplice personal computer di fascia bassa è mostrata nella Figura 2.30. Questo sistema ha un unico bus utilizzato per connettere la CPU, la memoria e i dispositivi di I/O, mentre la maggior parte dei sistemi ha invece due o più bus. Ciascun dispositivo di I/O è composto da due parti: una contenente la maggior parte dell'elettronica, chiamata il **controllore**, e un'altra contenente il dispositivo stesso, per esempio un lettore di dischi. Di solito il controllore è integrato direttamente sulla scheda madre, qualche volta è collocato su una scheda inserita in uno slot. Anche se lo schermo (il monitor) non è opzionale, il controllore video è talvolta situato su una scheda aggiuntiva per permettere all'utente di scegliere tra schede con o senza acceleratori hardware, memoria aggiuntiva e altre caratteristiche avanzate. Il controllore è connesso al suo dispositivo mediante un cavo che si collega al connettore nella parte posteriore della scatola metallica.

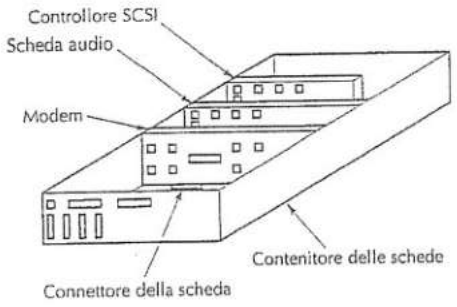


Figura 2.29 Struttura fisica di un personal computer.

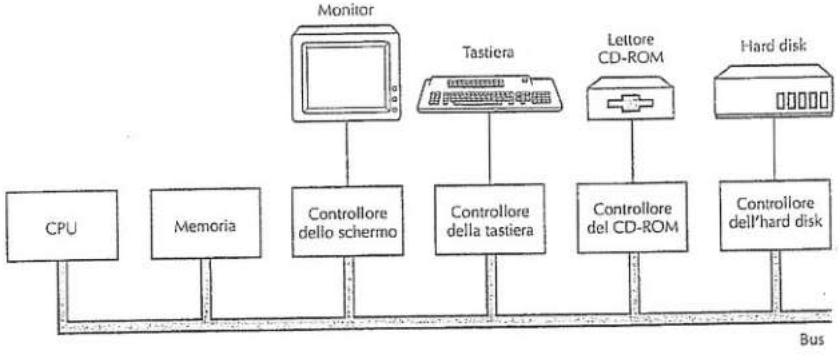


Figura 2.30 Struttura logica di un semplice personal computer.

Il compito di un controllore è di governare il proprio dispositivo di I/O e gestire il suo accesso al bus. Per esempio, quando un programma deve ottenere dei dati dal disco, manda un comando al suo controllore, che spedisce delle istruzioni al lettore, tra cui quella per la ricerca su disco. Il lettore, dopo aver individuato la traccia e il settore corretti, restituisce i dati al controllore sotto forma di un flusso seriale di bit. È compito del controllore spezzare il flusso di bit in unità, generalmente costituite da una o più parole, e scriverle successivamente in memoria. Quando un controllore legge e scrive dati in memoria senza l'intervento della CPU si dice che effettua un **Direct Memory Access** ("accesso diretto alla memoria"), meglio conosciuto con il suo acronimo **DMA**. Di solito, una volta che il trasferimento è completato, il controllore produce un **interrupt** che obbliga la CPU a sospendere immediatamente l'esecuzione del programma corrente e a far iniziare una speciale procedura, chiamata **gestore dell'interrupt** (*interrupt handler*) che controlla se ci sono errori, compie le azioni eventualmente necessarie e informa il sistema operativo che l'I/O è terminato. Dopo che il gestore dell'interrupt ha terminato

il proprio lavoro, la CPU riprende l'esecuzione del programma precedentemente sospeso all'arrivo dell'interrupt.

Il bus non è utilizzato solamente dai controllori di I/O, ma anche dalla CPU quando deve prelevare istruzioni o dati. Che cosa succede se la CPU e un controllore di I/O cercano di usare il bus allo stesso tempo? Un chip chiamato **arbitro del bus** stabilisce i turni. Generalmente le periferiche di I/O hanno la precedenza sulla CPU, dato che i dischi e altri dispositivi mobili non possono essere fermati e obbligarli ad aspettare vorrebbe dire perdere dati. Quando nessuna operazione di I/O è in esecuzione, la CPU può utilizzare tutti i cicli del bus per accedere alla memoria. Tuttavia quando un dispositivo di I/O è già in funzione, se richiede l'accesso al bus, questo gli viene subito concesso. Questo processo si chiama **furto di cicli** e rallenta le prestazioni del calcolatore.

Questo schema funzionava bene per i primi personal computer, dato che tutti i componenti erano più o meno bilanciati. Tuttavia, con l'aumento della velocità di CPU, memorie e dispositivi di I/O, sorse un problema: il bus non era più in grado di gestire il carico affidatogli. In un sistema chiuso, come le workstation adibite a calcoli ingegneristici, la soluzione fu quella di progettare un nuovo bus più veloce per il modello successivo. Questo approccio funzionava correttamente, dato che nessuno pensava di riutilizzare i vecchi dispositivi di I/O con il nuovo sistema.

Nel mondo dei PC però gli utenti aggiornavano spesso le loro CPU, ma allo stesso tempo volevano continuare a utilizzare stampanti, scanner e modem anche con il nuovo sistema. Era nato inoltre un enorme mercato attorno alla fornitura di periferiche di ogni tipo per il bus del PC IBM e non si voleva certo buttar via tutti gli investimenti per ricominciare da capo. IBM interpretò questo fatto nel modo peggiore quando lanciò il successore del PC IBM, la linea PS/2. Il PS/2 era dotato di un nuovo bus più veloce, ma molti produttori di cloni continuarono a utilizzare il bus dei vecchi PC, ora chiamato bus **ISA** (*Industry Standard Architecture*, "standard industriale di architettura"). Molti produttori di dischi e di altre periferiche continuarono a costruire controllori per quel bus e così IBM si trovò nella paradossale situazione di essere l'unico produttore di PC non più compatibili con IBM. Alla fine fu obbligata a tornare a supportare il bus ISA. Oggi il bus ISA è relegato a vecchi sistemi e musei dei computer ed è stato rimpiazzato da nuovi e più veloci standard. Aprendo una parentesi, occorre notare che ISA se utilizzato nel contesto dei livelli di una macchina significa *Instruction Set Architecture* ("architettura dell'insieme d'istruzioni"), mentre *Industry Standard Architecture* quando si parla di bus.

I bus PCI e PCIe

Anche se il mercato esercitava pressioni affinché nulla venisse modificato, i vecchi bus divennero realmente troppo lenti. Questa situazione portò altre società a sviluppare macchine dotate di più bus, tra le quali c'era ancora il vecchio bus ISA o il suo successore retrocompatibile **EISA** (*Extended ISA*, "ISA esteso"). Alla fine il vincitore fu il bus **PCI** (*Peripheral Component Interconnect*, "interconnessione di componenti periferici"), progettato da Intel. Al fine di incoraggiare l'intero mercato (compresi i propri concorrenti) ad adottarlo, Intel decise però di rendere di dominio pubblico tutti i brevetti.

Il bus PCI può essere utilizzato in varie configurazioni, di cui la più comune è illustrata nella Figura 2.31. In questo schema la CPU comunica con il controllore della

memoria lungo una connessione dedicata ad alta velocità. Il controllore comunica con la memoria e con il bus PCI in modo diretto, per far sì che il traffico tra CPU e memoria non occupi il bus PCI. Altre periferiche sono connesse direttamente al bus PCI. Una macchina così progettata contiene di solito due o tre slot PCI liberi, per permettere all'utente di inserire le proprie schede PCI di I/O per nuove periferiche.

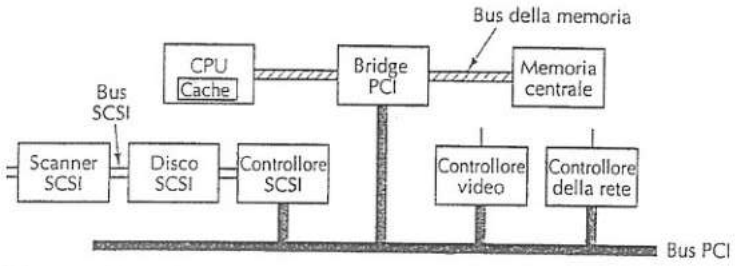


Figura 2.31 Tipico PC odierno con bus PCI. Il controllore SCSI è un dispositivo PCI.

Nel mondo dei computer non importa quanto veloce sia una cosa: un sacco di persone penseranno che è troppo lenta. Questo destino è toccato anche ai bus PCI, sostituiti con i bus PCIe. La maggioranza dei computer moderni supporta entrambi gli standard, così che gli utenti possano connettere dispositivi nuovi e veloci al bus PCIe e dispositivi più vecchi e più lenti al bus PCI.

Mentre quest'ultimo è stato solo un aggiornamento del vecchio ISA con velocità più alte e la possibilità di trasferire un maggior numero di bit in parallelo, il PCIe rappresenta un cambiamento radicale rispetto al PCI. In effetti non è nemmeno un bus, ma una rete punto a punto che utilizza linee seriali di bit e commutazione di pacchetto, più simile a Internet che a un bus tradizionale. L'architettura PCIe è rappresentata nella Figura 2.32.

Sono diverse le cose che balzano all'occhio riguardo al PCIe. Prima di tutto la connessione seriale tra le periferiche, ovvero un'ampiezza di 1 bit piuttosto che di 8, 16, 32 o 64. Anche se si può pensare che una connessione a 64 bit offra maggior larghezza di banda rispetto a una connessione a 1 bit, nella pratica le differenze nel tempo di propagazione di 64 bit, dette *skew*, portano a dover utilizzare velocità relativamente basse.

Nella comunicazione seriale possono invece essere usate velocità molto alte e questo compensa di gran lunga la mancanza di parallelismo. I bus PCI lavorano a una frequenza massima di 66 MHz; con il trasferimento di 64 bit per ciclo la massima velocità di traferimento dati risulta di 528 MB/sec. Con un clock rate di 8Gbps, anche se la comunicazione è seriale, la velocità di trasferimento di PCIe è di 1GB/sec. Inoltre, la comunicazione tra i dispositivi e la root complex, o lo switch, non è limitata a una sola coppia di fili. Un dispositivo può disporre di 32 coppie di fili, chiamate corsie (*lanes*). Siccome le corsie non sono sincrone, lo skew non gioca alcun ruolo. La maggior parte delle schede madri ha uno slot a 16 corsie per la scheda grafica, che con l'uso di PCIe 3.0 offrono

alle schede video una larghezza di banda di 16 GB/sec, circa 30 volte la velocità che le schede video PCI potevano raggiungere. Questa velocità è necessaria per le esigenze sempre crescenti di alcune applicazioni, come il 3D.

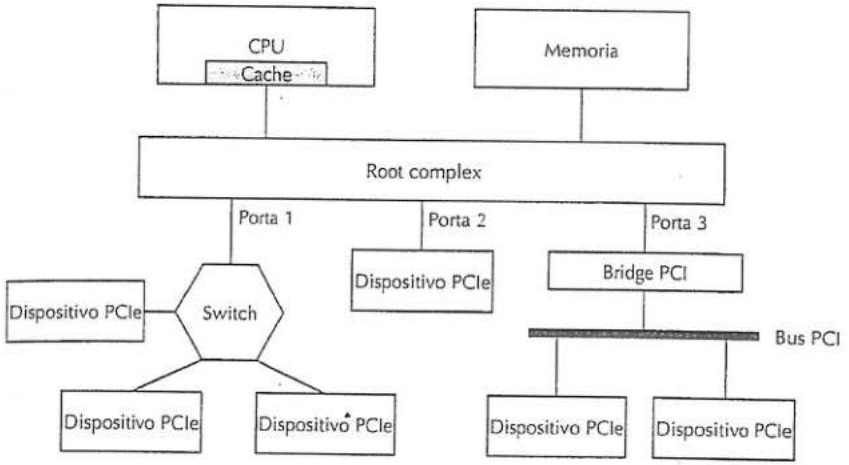


Figura 2.32 Esempio di architettura di un sistema PCIe con tre porte PCIe.

In secondo luogo, tutte le comunicazioni sono punto a punto. Quando la CPU deve comunicare con un dispositivo, invia un pacchetto a quel dispositivo dal quale riceve, generalmente, una risposta. Il pacchetto passa attraverso la root complex, che sta sulla scheda madre, e quindi va al dispositivo, talvolta passando per lo switch (o, se il dispositivo è PCI, per il bridge PCI). Questa evoluzione da un sistema in cui tutti i dispositivi restavano in ascolto sullo stesso bus a un sistema che utilizza una comunicazione punto a punto richiama lo sviluppo di Ethernet (una rete locale molto popolare), che agli inizi trasmetteva in broadcast e ora utilizza degli switch per permettere una comunicazione punto a punto.

2.4.2 Terminali

Diverse tipologie di dispositivi di I/O sono oggi disponibili. Alcune delle più comuni sono trattate in questo paragrafo. I terminali sono composti da due parti principali: la tastiera e il monitor. Nel mondo dei mainframe spesso questi componenti sono integrati in un unico dispositivo, collegato al calcolatore principale mediante una linea seriale o un cavo telefonico. Ancor oggi questi dispositivi sono ampiamente usati per le prenotazioni delle compagnie aeree, nelle banche e, più in generale, nei sistemi basati su mainframe. Nel mondo dei personal computer la tastiera e il monitor sono invece dei dispositivi indipendenti; la tecnologia è in ogni caso la stessa.

Tastiere

Esistono vari tipi di tastiere. Il PC IBM originario era dotato di una tastiera che aveva, sotto ogni tasto, un pulsante a scatto che dava un feedback tattile e produceva un clic quando il tasto veniva premuto sufficientemente. Oggi i tasti delle tastiere più economiche hanno semplicemente dei contatti meccanici, mentre quelle di qualità più elevata hanno un foglio di materiale elastometrico (una specie di gomma) tra i tasti e il circuito stampato sottostante. In questi modelli vi è, sotto ogni tasto, una piccola calotta che si piega quando il tasto viene premuto a sufficienza; una minuscola regione di materiale conduttivo presente al suo interno permette di chiudere il circuito. In alcune tastiere, la pressione di un tasto fa scorrere un piccolo corpo metallico all'interno di una bobina conduttrice inducendo in essa una corrente rilevabile. Vengono utilizzati anche vari altri metodi, di tipo meccanico o elettromagnetico.

La pressione di un tasto di un PC genera un interrupt che stimola una parte del sistema operativo, chiamata *gestore dell'interrupt della tastiera*. Questa routine, leggendo un registro hardware della tastiera, ricava il numero (da 1 a 102) associato al tasto premuto. Anche quando si rilascia un tasto viene generato un interrupt. In questo modo se l'utente preme il tasto SHIFT, poi preme e rilascia il tasto M e infine rilascia il tasto SHIFT, il sistema operativo può capire che l'utente voleva digitare una "M" maiuscola invece che una "m" minuscola. La gestione di sequenze multi-tasto che coinvolgono i tasti SHIFT, CTRL e ALT è fatta interamente via software (compresa la famosa e detestata sequenza CTRL-ALT-CANC utilizzata per riavviare i PC).

Touch screen

Nonostante le tastiere non corrano alcun rischio di fare la fine delle macchine da scrivere, c'è un nuovo arrivato nel mondo dei dispositivi di input: il touch screen. Anche se questi dispositivi sono entrati nel mercato di massa con l'avvento degli iPhone di Apple nel 2007, la loro storia inizia molto prima. Il primo touch screen è stato sviluppato al Royal Radar Establishment a Malvern, Regno Unito, nel 1965. Anche la tanto proclamata possibilità di pinching dell'iPhone risale in realtà a un lavoro fatto nell'Università di Toronto nel 1982. Da allora, sono state sviluppate e commercializzate diverse tecnologie. I dispositivi tattili possono essere classificati in due gruppi: gli opachi e i trasparenti. Un tipico esempio di dispositivo tattile opaco è il touchpad di un computer portatile. Un tipico dispositivo trasparente è lo schermo di uno smartphone o di un tablet. Considereremo qui solo quest'ultimo tipo. Tali dispositivi sono noti come **touch screen**. I touch screen più diffusi sono di tipo resistivo, capacitivo o a infrarossi.

Gli schermi a infrarossi hanno dei trasmettitori di raggi infrarossi, come diodi a emissione di luce infrarossa o laser, sul lato sinistro per esempio e sul lato superiore della smussatura attorno allo schermo e dei ricevitori sul lato destro e sul lato inferiore. Quando un dito, una penna o un qualunque oggetto opaco blocca uno o più fasci di luce il ricevitore corrispondente rileva la caduta del segnale e l'hardware del dispositivo comunica al sistema operativo quali fasci sono stati interrotti, permettendogli così di calcolare le coordinate del dito o della penna. Anche se questi dispositivi hanno una lunga tradizione e sono ancora usati in alcune applicazioni, non vengono utilizzati per i dispositivi mobili.

Un'altra vecchia tecnologia è quella dei **touch screen resistivi**. Questi sono formati da due strati, di cui il superiore è flessibile e contiene un gran numero di fili orizzontali, mentre quello inferiore contiene i fili verticali. Quando un dito o un altro oggetto fa pressione su un punto dello schermo, uno o più fili dello strato superiore viene a contatto con i quelli del livello inferiore, oppure vi si avvicina. L'elettronica del dispositivo fa si che si possa localizzare l'area che ha subito la depressione. La produzione di questi schermi è molto economica, e sono quindi molto utilizzati in ambiti applicativi particolarmente sensibili ai costi.

Le due tecnologie fin qui presentate funzionano bene quando lo schermo è premuto da un solo dito, ma hanno problemi in caso di utilizzo con due dita. Descriveremo il problema per gli schermi a infrarossi, ma lo stesso problema si ha per gli schermi resistivi. Si immagini che due dita siano posizionate alle coordinate (3,3) e (8,8). In questa condizione risultano interrotti i fasci di luce orizzontali x=3 e x=8 e i fasci di luce verticali y=3 e y=8. Si consideri ora la situazione, differente dalla precedente, in cui le dita sono posizionate nei punti (3,8) e (8,3), ovvero ai vertici opposti del rettangolo (3,3), (8,3), (8,8), (3,8). In questo caso risultano interrotti esattamente gli stessi fasci e il software non può in alcun modo riconoscere quale dei due casi descritti si sia verificato. Questo problema è noto come *ghosting*.

Per poter riconoscere la pressione contemporanea di più dita, una proprietà richiesta per l'utilizzo di gesti come espansione e pizzicamento (pinching), si ricorre a un'altra tecnologia. Quella utilizzata in gran parte degli smartphone e dei tablet (ma non nelle macchine fotografiche e in altri dispositivi) è il touch screen capacitivo a proiezione. Ci sono diverse tipologie di questi schermi, ma il tipo più diffuso è quello a **mutua capacitanza**. Tutti i touch screen in grado di riconoscere due o più punti di contatto contemporanei sono detti schermi multitouch. Vediamo brevemente come funzionano.

Ricordiamo che un condensatore è un dispositivo in grado di immagazzinare carica elettrica. Un semplice condensatore è formato da due conduttori separati da un isolante. Nei touch screen moderni, una griglia di "fili" sottili disposti verticalmente è separata da una griglia orizzontale per mezzo di un sottile strato isolante. Quando un dito tocca lo schermo modifica la capacitanza in tutte le intersezioni toccate (anche distanti) e questo cambiamento può essere misurato. Per verificare che i moderni touch screen sono diversi dai più datati schermi resistivi o a infrarossi, basta provare a toccarli con una penna, una matita, una graffetta o un dito ricoperto da un guanto per scoprire che non succede nulla. Il corpo umano è in grado di immagazzinare cariche elettriche, come chiunque abbia sfregato un tappeto in una giornata fredda e secca e quindi toccato una maniglia di metallo può dolorosamente testimoniare. Gli oggetti in plastica, legno e metallo non godono delle stesse proprietà.

I "fili" in un touch screen non sono i tradizionali fili di rame che si trovano nei normali dispositivi elettrici, che bloccherebbero la luce dalla schermata, ma sono sottili (tipicamente 50 micron) strisce di ossido di indio e stagno trasparenti, legate ai lati opposti di una piastra di vetro sottile, insieme alla quale formano i condensatori. In alcuni modelli più recenti, la lastra di vetro isolante viene sostituita da un sottile strato di diossido di silicio (sabbia!), con i tre strati atomizzati (spruzzati, atomo per atomo) su un substrato. In entrambi i casi, i condensatori sono protetti da polvere e graffi da una

lastra di vetro sovrapposta che forma la superficie dello schermo da toccare. Più sottile è la lastra di vetro superiore e più sensibile è lo schermo, ma la fragilità del dispositivo aumenta.

Durante il funzionamento, le tensioni vengono applicate alternativamente ai "fili" orizzontali e verticali mentre i valori di tensione, che sono influenzati dalla capacitanza di ciascuna intersezione, vengono letti lungo l'altra direzione. Questa operazione è ripetuta diverse volte al secondo, con le coordinate toccate che alimentano il driver del dispositivo con un flusso di coppie (x, y). Ulteriori operazioni, per esempio determinare un puntamento, un pizzicamento, un'espansione o una strisciata in atto, sono gestite dal sistema operativo. Se si utilizzano tutte e dieci le dita e si invita un amico ad aggiungere altri tocchi il sistema operativo potrebbe essere incapace di gestire l'operazione, anche se l'hardware del multitouch sarà all'altezza del compito.

Schermi piatti

I primi schermi dei computer utilizzavano tubi catodici (CRT) esattamente come le vecchie televisioni. Questi schermi erano troppo ingombranti e pesanti per poter essere utilizzati nei computer portatili e così si rese necessario sviluppare una nuova tecnologia: quella degli schermi piatti, in grado di offrire il fattore di forma adatto ai notebook e di consumare meno potenza. Al giorno d'oggi, i benefici in termini di consumo elettrico e dimensioni offerti dagli schermi piatti hanno soppiantato la tecnologia CRT.

La tecnologia più comune è quella dello schermo a cristalli liquidi, **LCD** (*Liquid Crystal Display*). La sua descrizione verrà fortemente semplificata, dato che si tratta di una tecnologia in continuo mutamento e particolarmente complessa, di cui esistono molte varianti.

I cristalli liquidi sono molecole organiche viscose che si muovono come un liquido, ma che hanno anche una struttura spaziale simile a quella di un cristallo. Scoperti nel 1888 da un botanico austriaco (Friedrich Reinitzer), negli anni '60 sono stati applicati per la prima volta agli schermi (per esempio in quelli di calcolatrici e orologi). Quando tutte le molecole sono allineate nella stessa direzione le proprietà ottiche del cristallo dipendono dalla direzione e dalla polarizzazione della luce incidente. È possibile modificare l'allineamento delle molecole, e quindi le proprietà ottiche, tramite l'applicazione di un campo elettrico. In particolare è possibile controllare elettricamente l'intensità della luce uscente facendola passare attraverso un cristallo liquido. Questa proprietà può essere sfruttata per costruire schermi piatti.

Uno schermo LCD consiste in due lastre di vetro parallele tra le quali è sigillato un cristallo liquido. Alle lastre sono attaccati degli elettrodi trasparenti. Una luce (naturale o artificiale) proveniente da dietro la lastra posteriore illumina lo schermo. Gli elettrodi trasparenti collegati a ciascuna lastra sono utilizzati per creare campi elettrici all'interno del cristallo liquido e, variando la tensione sulle diverse parti dello schermo, si può controllare l'immagine da rappresentare. Per la visualizzazione sono anche necessari dei filtri di polarizzazione incollati allo schermo frontale e a quello posteriore. Lo schema generale è mostrato nella Figura 2.33(a).

Anche se vengono usati diversi tipi di schermi LCD, ne considereremo come esempio solo uno: lo schermo **TN** (*Twisted Nematic*). In questo schermo la lastra posteriore contiene piccoli solchi orizzontali, mentre in quella frontale i solchi sono disposti verticalmente, come mostra la Figura 2.33(b). In assenza di campo elettrico le molecole del LCD tendono ad allinearsi lungo i solchi e, dato che gli allineamenti sulle due lastre differiscono di 90 gradi, le molecole (e quindi la struttura del cristallo) effettuano una torsione nello spazio intermedio.

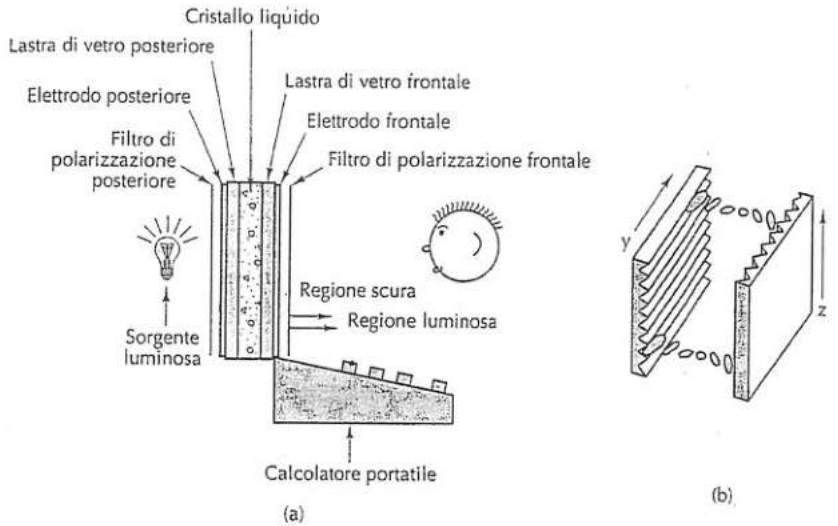


Figura 2.33 (a) Struttura di uno schermo LCD. (b) I solchi sulla lastra frontale e su quella posteriore sono perpendicolari tra loro.

Dietro allo schermo è collocato un filtro di polarizzazione orizzontale, che permette esclusivamente il passaggio di luce polarizzata in quella direzione, mentre nella parte frontale c'è un filtro di polarizzazione verticale. Se tra le lastre non ci fosse del liquido, la luce polarizzata orizzontalmente lasciata passare dal filtro polarizzatore posteriore sarebbe bloccata da quello anteriore, rendendo lo schermo uniformemente nero.

La torsione della struttura del cristallo delle molecole guida la luce nel suo passaggio e ruota la sua polarizzazione rendendola verticale. Quindi, in assenza di un campo elettrico, lo schermo LCD risulta uniformemente luminoso. Applicando tensione a particolari regioni della lastra si distrugge la struttura cristallina e la sua torsione, bloccando localmente il passaggio della luce.

È possibile utilizzare due schemi per l'applicazione della tensione. Negli **schermi a matrice passiva** (i più economici) entrambi gli elettrodi contengono fili paralleli. Per esempio, in uno schermo 1920 x 1080 (la dimensione del Full HD) l'elettrodo posteriore potrebbe avere 1920 fili verticali, e quello frontale 1080 orizzontali. Applicando una tensione a uno dei cavi verticali e un impulso a uno dei cavi orizzontali si modifica il

voltaggio di un pixel in una particolare posizione, facendolo diventare scuro per un breve intervallo di tempo. Il pixel (da *picture element*, ovvero elemento del disegno), è il punto colorato a partire dal quale si realizzano tutte le immagini digitali. Applicando lo stesso impulso al pixel successivo e poi a quello ancora seguente si può disegnare sullo schermo una linea scura. Solitamente lo schermo viene interamente ridisegnato 60 volte in un secondo, in modo da ingannare l'occhio e fargli credere che stia guardando un'immagine costante.

L'altro schema utilizzato estensivamente fa uso di **schermi a matrice attiva**: è più costoso, ma produce un'immagine di qualità superiore. In questo caso, invece di avere solamente due insiemi di fili paralleli, su uno degli elettrodi è presente, in corrispondenza di ciascun pixel, un piccolo elemento di commutazione. Accendendo o spegnendo questi elementi si può creare sullo schermo uno schema di tensioni, che corrisponde a un arbitrario pattern di bit. Gli elementi di commutazione sono chiamati **thin film transistor** ("transistor a pellicola sottile") e gli schermi piatti che li utilizzano sono spesso chiamati **schermi TFT**. Al giorno d'oggi la maggior parte dei calcolatori portatili e dei monitor a schermo piatto per computer desktop utilizza la tecnologia TFT.

Finora abbiamo descritto il funzionamento di uno schermo monocromatico. Anche se i dettagli sono molto più complicati, è sufficiente dire che gli schermi a colori si basano sugli stessi principi. Si usano filtri ottici per separare, in corrispondenza di ogni pixel, la luce bianca nelle componenti rossa, verde e blu, in modo da poterle visualizzare indipendentemente.

Nuove tecnologie di schermi sono all'orizzonte. Una delle più promettenti è lo schermo OLED (*Organic Led Emitting Diode*), che consiste in uno strato di molecole organiche cariche elettricamente schiacciate tra due elettrodi. Le variazioni di tensione eccitano le molecole e le portano in stati energetici più alti. Quando le molecole tornano allo stato normale emettono luce. Fornire maggiori dettagli su questa tecnologia va oltre gli scopi di questo libro (e dei suoi autori).

RAM della scheda video

La maggior parte dei monitor vengono ridisegnati dalle 60 alle 100 volte al secondo, accedendo a una memoria speciale, chiamata **RAM della scheda video** (*Video RAM*), che si trova sulla scheda del controllore dello schermo. Questa memoria ha una o più bitmap che rappresentano l'immagine dello schermo; per esempio su uno schermo di 1920×1080 **pixel** la RAM della scheda video deve contenere 1920×1080 valori, uno per ogni pixel. In realtà, per permettere un veloce passaggio da un'immagine dello schermo a un'altra, questa memoria potrebbe contenere più bitmap di questo tipo.

Su un'ordinario schermo ogni pixel dovrebbe essere rappresentato da un valore RGB (da *Red*, *Green* e *Blue*) composto da 3 byte, uno per l'intensità di ciascun componente (rosso, verde e blu) del colore del pixel (gli schermi di fascia alta utilizzano 10 o più bit per colore). Le leggi dell'ottica ci dicono che qualsiasi colore può essere costruito combinando linearmente luci rosse, verdi e blu.

Una RAM della scheda video con 1920×1080 pixel e 3 byte per pixel richiede oltre 6,2 MB per memorizzare l'immagine e una quantità non indifferente di tempo di CPU per operare su di essa. Per questa ragione alcuni calcolatori scendono al compromesso

di utilizzare un unico numero a 8 bit per indicare il colore desiderato. Questo numero è utilizzato come indice di una tabella hardware, chiamata **tavolozza** (*color palette*), contenente 256 elementi, ciascuno dei quali memorizza un valore RGB a 24 bit. Questo tipo di strategia, chiamata **colore indicizzato**, riduce di 2/3 i requisiti della RAM della scheda video, ma permette di visualizzare su schermo solo 256 colori per volta. Spesso, quando sullo schermo sono mostrate contemporaneamente più finestre, solo i colori di una di queste vengono *renderizzati* correttamente; infatti, anche se di solito ogni finestra ha la sua propria mappatura, esiste una sola tavolozza hardware. Vengono anche utilizzate tavolozze da 2^{16} colori, ma in questo caso il guadagno si riduce a 1/3.

Gli schermi basati su bitmap richiedono una grande quantità di larghezza di banda.

Per visualizzare dati multimediali a schermo intero su uno schermo 1920×1080 occorre trasferire a ogni fotogramma 6,2 MB di dati. Nel caso di video in movimento (*full-motion*) sono necessari almeno 25 fotogrammi al secondo, che corrispondono a una velocità totale di trasferimento dati di 155 MB/s. Questo carico supera le capacità di un bus PCI (132 MB/sec), ma il bus PCIe è in grado di gestirlo tranquillamente.

2.4.3 Mouse

Più il tempo passa, più i calcolatori sono utilizzati da persone con sempre minor conoscenza del loro funzionamento. I calcolatori della generazione ENIAC erano usati da coloro che li avevano costruiti, mentre negli anni '50 i computer venivano utilizzati soltanto da programmatori altamente specializzati. Oggigiorno invece i calcolatori sono largamente impiegati da persone che non sanno (o talvolta non vogliono sapere) troppi dettagli sul funzionamento dei calcolatori o sulla loro programmazione.

Agli inizi, la maggior parte dei calcolatori era dotata di un'interfaccia a linea di comando, attraverso la quale gli utenti impartivano i comandi. Dato che chi non è uno specialista dei calcolatori reputa che le interfacce a linea di comando siano spesso poco amichevoli, se non addirittura ostili, molti produttori di calcolatori, in seguito, hanno sviluppato interfacce punta-e-clicca, come quelle del Macintosh e di Windows. Per poter utilizzare questo tipo d'interfaccia è necessario avere uno strumento per puntare sullo schermo, e il metodo più comune consiste nell'usare un mouse.

Il **mouse** è un piccolo oggetto di plastica situato a fianco della tastiera. Quando viene mosso sul tavolo, un piccolo puntatore si sposta, in corrispondenza, sullo schermo, permettendo agli utenti di puntare agli elementi desiderati. Il mouse, sulla parte superiore, ha uno, due o tre pulsanti che consentono all'utente di selezionare elementi e voci dei menu. Gli utenti più inesperti preferiscono i mouse con un unico tasto (è infatti difficile premere quello sbagliato se ce n'è soltanto uno), mentre quelli più abili preferiscono la potenza di un mouse a più tasti che permette azioni più sofisticate.

Sono stati prodotti tre tipi di mouse: meccanici, ottici e opto-meccanici. Il primo mouse aveva sul fondo due rotelle di gomma che sporgevano all'esterno; gli assi delle due rotelle erano disposti perpendicolarmente, di modo che, quando il mouse veniva spostato parallelamente a uno degli assi principali, ruotava soltanto una rotella. Ciascuna rotella guidava una resistenza variabile o un potenziometro, in modo che misurando il loro cambiamento era possibile calcolare la rotazione e, di conseguenza, la distanza percorsa dal mouse lungo una particolare direzione. Negli ultimi anni questo tipo di

mouse è stato quasi completamente sostituito da un modello, mostrato nella Figura 2.34, che al posto delle due rotelle fa uso di una pallina leggermente sporgente sul fondo.

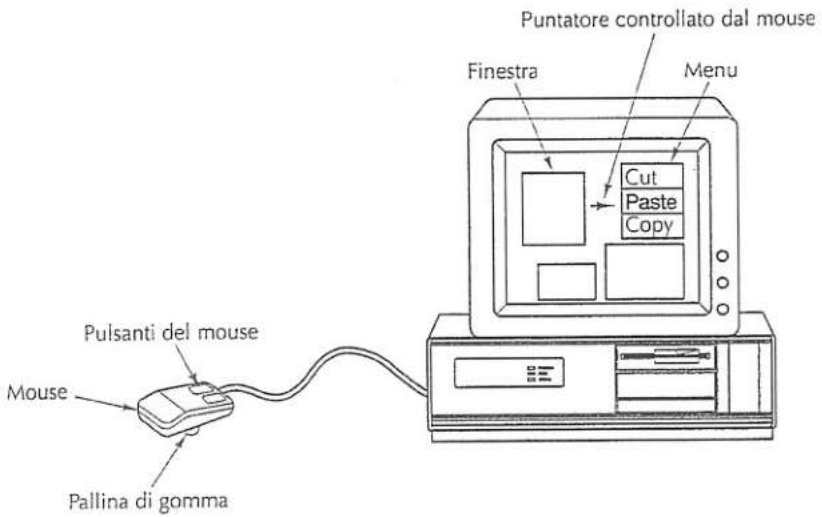


Figura 2.34 Mouse utilizzato per puntare a voci di un menu.

Il secondo tipo di mouse è quello ottico, che non ha né rotelle né pallina. Al loro posto ha, sul fondo, un LED (*Light Emitting Diode*, "diodo luminescente") e un fotorilevatore. I primi mouse ottici richiedevano un mouse pad in cui era raffigurata una griglia di linee molto vicine tra loro per determinare quante linee venivano attraversate e quindi l'entità dello spostamento del mouse. I mouse ottici moderni contengono un LED che illumina le imperfezioni sottostanti e una sottile fotocamera che registra piccole immagini (di solito di 18x18 pixel) circa 1000 volte al secondo. Le immagini consecutive vengono quindi analizzate per determinare di quanto il mouse si è eventualmente spostato. Alcuni mouse utilizzano un laser al posto del LED e sono più precisi, ma anche più costosi.

Il terzo tipo di mouse è quello opto-meccanico, anch'esso dotato di una pallina che fa ruotare due cilindretti perpendicolari tra loro. I cilindretti sono collegati a codificatori che hanno una serie di fori attraverso i quali può passare la luce; quando il mouse si sposta, i cilindretti ruotano e la luce colpisce il rilevatore ogni volta che un foro si trova allineato con il LED e il suo fotorilevatore. Il numero di impulsi che vengono rilevati è proporzionale allo spostamento effettuato.

Anche se sono possibili diverse soluzioni, in genere un mouse spedisce al calcolatore una sequenza di 3 byte ogni qualvolta si muove di una certa distanza minima (per esempio 0,01 pollici), chiamata in alcuni casi *mickey*. Di solito questi caratteri vengono spediti lungo una linea seriale, un bit alla volta. Il primo byte contiene un intero con segno che

indica quante unità sono state percorse dal mouse nella direzione x rispetto all'ultima rilevazione, mentre il secondo byte fornisce la stessa informazione per quanto riguarda il movimento lungo y; il terzo byte contiene invece lo stato corrente dei pulsanti del mouse. A volte si utilizzano 2 byte per i valori di spostamento di ogni coordinata.

Il software del calcolatore riceve queste informazioni sui movimenti relativi del mouse, le converte in coordinate assolute e, in corrispondenza della posizione calcolata, visualizza una piccola freccia. Per selezionare un elemento sullo schermo l'utente deve soltanto posizionare la freccia sulla regione desiderata e premere un pulsante: il calcolatore può determinare quale elemento è stato selezionato dato che sa in ogni momento in che posizione dello schermo si trova la freccia.

2.4.4 Controller per videogiochi

Grazie alle molteplici esigenze degli utilizzatori di videogiochi il mercato delle console di gioco ha sviluppato dispositivi di input specializzati. In questo paragrafo consideriamo due recenti sviluppi tra i controller per videogiochi: in Nintendo Wiimote e il Microsoft Kinect.

Il controller Wiimote

Apparso nel 2006 insieme alla console Nintendo Wii, il controller Wiimote aggiunge ai tradizionali pulsanti dei gamepad la capacità di rilevare i movimenti. Tutte le interazioni con il Wiimote sono inviate in tempo reale alla console di gioco tramite una sistema bluetooth interno. I sensori di movimento presenti in Wiimote gli permettono di rilevare movimenti in tre dimensioni. Inoltre, se rivolto verso il televisore, il controller è in grado di offrire una precisa funzione di puntamento.

La Figura 2.35 mostra come il Wiimote implementa la funzione di rilevazione dei movimenti. Il monitoraggio dei movimenti del Wiimote lungo tre direzioni è realizzato da un accelerometro a tre assi. Questo dispositivo contiene tre piccole masse, ciascuna in grado di muoversi lungo uno dei tre assi x, y e z (rispetto alla posizione del chip dell'accelerometro). Il movimento di ogni massa è proporzionale all'accelerazione lungo il proprio asse e modifica la capacitanza della massa rispetto a una parete fissa di metallo. Misurando le tre variazioni di capacitanza diventa possibile rilevare l'accelerazione lungo le tre direzioni. Grazie a questa tecnologia e alla matematica classica, la console Wii può seguire i movimenti del Wiimote nello spazio. Mentre si tenta di colpire con il Wiimote una pallina da tennis virtuale, il movimento del controller viene monitorato; se si ruota il polso all'ultimo momento per cercare di dare un effetto alla pallina, l'accelerometro del Wiimote sarà in grado di rilevare questo ulteriore movimento.

Gli accelerometri danno buoni risultati nella rilevazione dei movimenti tridimensionali del Wiimote, ma non possono offrire la precisione necessaria per controllare un puntatore sullo schermo televisivo. Gli accelerometri soffrono infatti di un'inevitabile margine di errore nella rilevazione dei movimenti. Con il passare del tempo, il calcolo della posizione esatta del Wiimote (basato sull'integrazione delle sue accelerazioni) diventa sempre meno accurato.

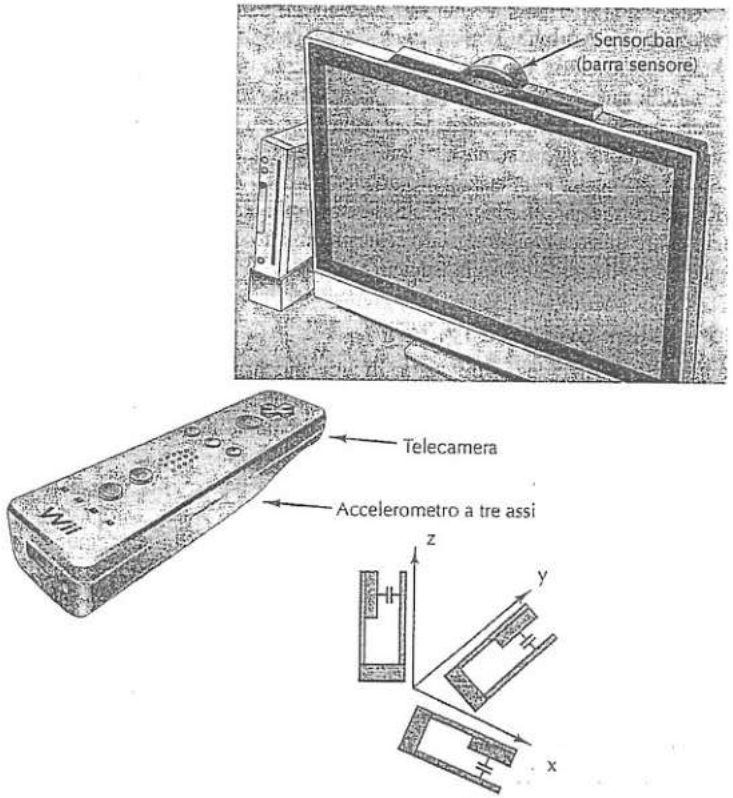


Figura 2.35 I sensori di movimento del controller Wiimote.

Per offrire una rilevazione di movimento più fine il Wiimote utilizza tecnologie intelligenti di computer vision. Posta sopra il televisore vi è una *sensor bar* ("barra sensore") che contiene dei LED a distanza fissata. Il Wiimote contiene una videocamera che, se puntata verso il televisore, è in grado di dedurre la distanza e l'orientamento del controller rispetto allo schermo. Visto che i LED della sensor bar sono a distanza fissa, la loro distanza vista dal Wiimote è proporzionale alla distanza del controller dalla sensor bar. La posizione della sensor bar nel campo visivo del Wiimote indica la direzione da cui il controller sta puntando il televisore. Monitorando continuamente questo orientamento è possibile ottenere una capacità di puntamento preciso senza soffrire degli errori degli accelerometri.

Il controller Kinect

Microsoft Kinect porta le capacità di computer vision dei controller per videogiochi a un nuovo livello. Questo dispositivo utilizza solamente tecniche di computer vison per determinare le interazioni dell'utente con la console di gioco. Il suo funzionamento si basa sulla rilevazione della posizione dell'utente nella stanza, sul suo orientamento e sui movimenti del suo corpo. Il controllo dei giochi avviene mediante movimenti predeterminati effettuati dalle mani, dalle gambe e da qualsiasi parte del corpo che il programmatore del videogioco vuole far muovere per il controllo del gioco.

La capacità di rilevazione di Kinect si basa su una fotocamera di profondità e una video camera. La camera di profondità misura la distanza dell'oggetto nel campo visivo di Kinect emettendo un fascio di raggi laser infrarossi e catturando poi i loro riflessi grazie a una camera a infrarossi. Usando una tecnica di computer vision nota come "structured light", Kinect può determinare la distanza degli oggetti che vede basandosi sul disturbo che il fascio infrarosso subisce dalla superficie illuminata.

L'informazione sulla profondità viene combinata con le informazioni strutturali restituite dalla videocamera per produrre una mappa di profondità della struttura. Questa mappa può quindi essere processata da un algoritmo di computer vision per localizzare le persone nella stanza (riconoscendo persino il loro volto) e l'orientamento e i movimenti del loro corpo. Dopo questa fase le informazioni sulle persone presenti nella stanza vengono inviate alla console di gioco che utilizza i dati raccolti per il controllo del videogioco.

2.4.5 Stampanti

Dopo aver preparato un documento o scaricato una pagina da Internet, gli utenti vogliono spesso stampare il lavoro fatto. Per questa ragione molti computer dispongono di una stampante. In questo paragrafo descriveremo alcune delle più diffuse tipologie di stampanti.

Stampanti laser

Dopo l'invenzione della stampa, da parte di Johann Gutenberg nel quindicesimo secolo, lo sviluppo più significativo nella riproduzione dei testi è probabilmente rappresentato dalla **stampante laser**. Questo dispositivo combina in un'unica periferica alta qualità dell'immagine, eccellente flessibilità, grande velocità e costo limitato. Le stampanti laser usano una tecnologia molto simile a quella delle fotocopiatrici; difatti molte società producono periferiche che permettono sia di effettuare fotocopie sia di stampare (e talvolta di spedire e ricevere fax).

La Figura 2.36 illustra la tecnologia di base. Il cuore della stampante è un tamburo rotante molto preciso (o, in alcuni sistemi di fascia alta, un nastro). All'inizio di ciascun ciclo di pagina il tamburo è caricato fino a circa 1000 volt e rivestito con un materiale fotosensibile. Successivamente la luce generata dal laser passa lungo tutta lunghezza del tamburo e si riflette su uno specchio ottagonale rotante. Il fascio di luce viene modulato per produrre regioni luminose e regioni scure: le parti del tamburo colpite dal raggio perdono la loro carica elettrica.

Il tamburo, dopo aver disegnato una linea di punti, ruota di una frazione di grado per permettere il disegno della linea successiva. A questo punto la prima linea di punti raggiunge il *toner*, un contenitore di polvere nera elettrostaticamente sensibile. Il toner è attirato dai punti che hanno ancora una carica elettrica, formando così un'immagine

visiva sulla linea del tamburo. Un istante dopo, ruotando, il tamburo ricoperto di *toner* viene premuto contro il foglio di carta, trasferendo su questo la polvere nera. Il foglio passa quindi attraverso dei rulli riscaldati per fissare l'immagine, fondendo in modo permanente il *toner* sulla carta. Nel seguito della rotazione il cilindro viene scaricato e ripulito di ogni residuo del *toner*, per poter essere nuovamente caricato e ricoperto di materiale fotosensibile, pronto per la stampa della pagina successiva.

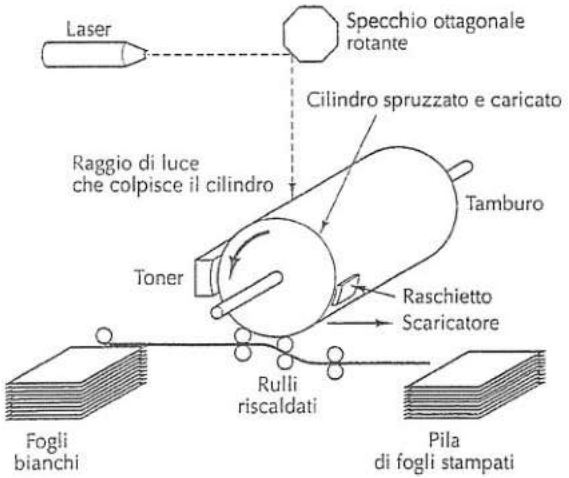


Figura 2.36 Funzionamento di una stampante laser.

È necessario precisare che questo processo è possibile grazie a una combinazione estremamente complessa di fisica, chimica, meccanica e ottica. Nonostante questa complessità alcuni produttori vendono dei meccanismi completi, chiamati **motori di stampa**, che comprendono tutte queste fasi. Per realizzare una stampante completa i produttori di stampanti laser combinano i motori di stampa con la propria elettronica e il proprio software. L'elettronica consiste in una veloce CPU integrata, alcuni megabyte di memoria per memorizzare la mappa dei bit di un'intera pagina e numerosi set di caratteri, alcuni dei quali predefiniti.

La maggior parte delle stampanti accetta comandi che descrivono come devono essere stampate le pagine. Questi comandi sono espressi in linguaggi come PCL di HP, PostScript di Adobe o PDF, dei veri e propri linguaggi di programmazione completi, anche se specializzati.

Le stampanti laser da 600 dpi e più possono stampare in modo accettabile fotografie in bianco e nero, ma la tecnologia è più complessa di quanto potrebbe sembrare. Si consideri una fotografia scannerizzata a 600 dpi che deve essere stampata con una stampante a 600 dpi. L'immagine scannerizzata contiene 600 × 600 pixel/pollice, ciascuno dei

quali consiste in un valore di grigio compreso fra 0 (bianco) e 255 (nero). Anche la stampante può stampare a 600 dpi. ma ciascun pixel stampato è o nero (*toner* presente) o bianco (*toner* assente): i grigi non possono essere stampati.

Per riprodurre la scala dei grigi la tecnica più comune è quella dei **mezzitoni**, la stessa utilizzata per i poster commerciali. L'immagine è divisa in celle, solitamente di 6 × 6 pixel: ogni cella può quindi contenere tra 0 e 36 pixel neri. Le celle con più pixel sono percepite dall'occhio come più scure rispetto alle altre. I valori di grigio compresi tra 0 e 255 sono rappresentati dividendo questo intervallo in 37 zone. I valori da 0 a 6 appartengono alla zona 0, i valori da 7 a 13 alla zona 1 e così via (la zona 37 è leggermente più piccola delle altre in quanto 256 non è divisibile esattamente per 37). Ogni qualvolta si incontra un grigio appartenente alla zona 0, si lascia la sua cella dei mezzitoni bianca, come mostra la Figura 2.37(a). Un valore appartenente alla zona 1 è stampato con un pixel nero; uno della zona 2 con due pixel neri, come mostra la Figura 2.37(b). Le Figure 2.37(c)-(f) mostrano altre zone con valori diversi. Se si prende una fotografia scannerizzata a 600 dpi e la si rappresenta mediante questa tecnica, la sua risoluzione effettiva si riduce ovviamente a 100 celle/pollice; questa risoluzione viene chiamata **frequenza del retino dei mezzitoni**, ed è misurata convenzionalmente in **lpi** (*lines per inch*, "linee per pollice").

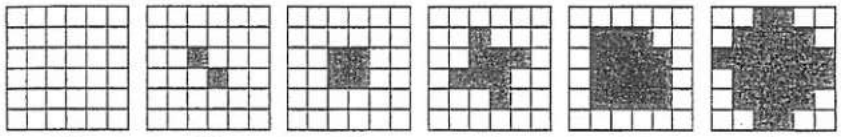


Figura 2.37 Mezzitoni per alcuni intervalli di livelli di grigio. (a) 0-6. (b) 14-20. (c) 28-34. (d) 56-62. (e) 105-111. (f) 161-167.

La stampa a colori

Sebbene la maggior parte delle stampanti laser sia monocromatica, le stampanti laser a colori stanno diventando sempre più comuni. Per questa ragione riteniamo sia utile dare qualche spiegazione di questa tecnica (valida anche per stampanti a getto d'inchiostro e per altre stampanti). Come potete immaginare, non è una tecnica banale. Le immagini a colori vengono viste in due modi distinti: per luce trasmessa o per luce riflessa. Le immagini per luce trasmessa, come quelle prodotte sui monitor, sono create mediante la sovrapposizione dei tre colori primari additivi: il rosso, il verde e il blu. Al contrario, le immagini per luce riflessa, come le fotografie a colori e le immagini di riviste su carta lucida, assorbono alcune lunghezze d'onda di luce e riflettono le restanti. Queste immagini sono create dalla sovrapposizione di tre colori primari sottrattivi, il ciano (completamente assorbito dal rosso), il giallo (completamente assorbito dal blu) e il magenta (completamente assorbito dal verde). In teoria ogni colore può essere riprodotto mischiando i colori ciano, giallo e magenta, ma in pratica è difficile produrre inchiostri

sufficientemente puri da ottenere un vero nero. Per questo motivo quasi tutti i sistemi di stampa a colori utilizzano quattro colori: il ciano, il giallo, il magenta e il nero, e sono chiamati **stampanti CMYK.** La K è a volte attribuita al nero (blacK), ma in realtà fa riferimento alla lastra chiave (*key plate*) alla quale le lastre dei colori sono allineate in una macchina da stampa convenzionale in quadricromia. Per rappresentare i colori i monitor usano, al contrario, luce trasmessa e il sistema RGB.

L'insieme completo di colori rappresentabili da uno schermo o da una stampante è chiamato **gamut**, cioè **gamma dei colori.** Nessun dispositivo ha una gamma di colori che corrisponde a quelli del mondo reale, dato che ogni colore assume tipicamente 256 intensità, che producono in totale solo 16.777.216 colori discreti. A causa di alcune imperfezioni dovute alla tecnologia, il numero totale di colori si riduce ulteriormente e quelli rimanenti non sono neanche distribuiti in modo uniforme nello spettro dei colori. Inoltre la percezione dei colori non dipende solo dalle leggi della fisica, ma è determinata in buona parte anche dal funzionamento dei coni e dei bastoncelli dell'occhio umano. Di conseguenza si vede che non è affatto banale convertire un'immagine a colori che appare correttamente sullo schermo in un'identica immagine stampata. I problemi principali che si incontrano in questo processo sono i seguenti:

1. i monitor a colori usano luce trasmessa, mentre le stampanti a colori usano luce riflessa;
2. i CRT generano 256 intensità per colore, mentre le stampanti usano i mezzitoni;
3. i monitor hanno un fondo nero, mentre la carta ha un colore chiaro;
4. le gamme dei colori RGB e CMYK sono diverse.

Per ottenere immagini stampate che corrispondano alla realtà (o anche alle immagini a schermo) è necessario calibrare i dispositivi, utilizzare software sofisticati per la costruzione e l'utilizzo dei profili ICC (*International Color Consortium*) e affidarsi a una considerevole esperienza da parte dell'utente.

Stampanti a getto d'inchiostro

Per le stampe casalinghe a basso costo molti preferiscono le **stampanti a getto d'inchiostro (inkjet).** La testina mobile di stampa, che contiene le cartucce di inchiostro, viene trascinata orizzontalmente da un nastro lungo la carta mentre l'inchiostro viene spruzzato da ugelli molto piccoli. Le goccioline di inchiostro hanno un volume di circa 1 picolitro, quindi 100 milioni di queste gocce formano una sola goccia d'acqua. Le stampanti a getto d'inchiostro sono di due tipi: piezoelettriche (utilizzate da Epson) e termiche (utilizzate da Canon, HP e Lexmark). Le prime hanno un particolare tipo di cristallo a fianco della camera contenente l'inchiostro. Quando gli viene applicata una tensione il cristallo si deforma leggermente, forzando l'uscita di una gocciolina di inchiostro dall'ugello. Il software può controllare la dimensione delle gocce regolando la tensione: maggiore è la tensione applicata, più grande è la goccia.

Le stampanti termiche (chiamate anche stampanti BubbleJet) contengono una minuscola resistenza all'interno di ciascun ugello. Quando viene applicata una tensione alla resistenza, questa si scalda molto velocemente, aumentando istantaneamente la temperatura dell'inchiostro fino al punto di ebollizione; a questo punto l'inchiostro evapora e forma una bolla di gas. La bolla di gas occupa un volume maggiore rispetto all'inchiostro che l'ha formata e preme sull'ugello. L'unico posto da cui l'inchiostro può fuoriuscire è la parte anteriore dell'ugello, verso la carta. L'ugello viene quindi raffreddato e il conseguente vuoto aspira un'altra goccia di inchiostro dal serbatoio. La velocità di stampa è limitata dalla velocità del ciclo di ebollizione/raffreddamento. Le goccioline sono tutte delle stesse dimensioni, ma inferiori a quelle utilizzate dalle stampanti piezoelettriche.

Le stampanti a getto d'inchiostro economiche hanno in genere risoluzioni di almeno 1200 dpi (punti per pollice), quelle di fascia alta di 4800 dpi. Queste stampanti sono economiche e di buona qualità, ma sono anche lente e utilizzano cartucce d'inchiostro costose. Quando le versioni migliori delle stampanti a getto d'inchiostro vengono utilizzate per stampare una foto professionale ad alta risoluzione su una carta fotografica con speciale rivestimento, i risultati non si distinguono dalle stampe convenzionali, anche fino a dimensioni di 8 × 10 pollici.

Per ottenere risultati migliori occorre utilizzare carte e inchiostri speciali. Esistono due tipi d'inchiostro: **a base di coloranti** e **a pigmenti.** I primi consistono in coloranti diluiti in un liquido, producono colori luminosi e fluiscono facilmente, ma il loro principale svantaggio è quello di sbiadirsi quando sono esposti a radiazioni ultraviolette, come quelle della luce solare. I secondi contengono invece delle particelle solide di pigmento sospese in un liquido che evapora dalla carta depositando il colore. Questi inchiostri non si sbiadiscono con il tempo, ma non sono luminosi quanto quelli basati su coloranti e le particelle di pigmento hanno la tendenza a bloccare gli ugelli, rendendone necessaria una periodica pulizia. Per stampare fotografie è necessario usare un tipo di carta lucida o patinata appositamente progettata per trattenere le gocce d'inchiostro e impedire che si spandano eccessivamente.

Stampanti speciali

Mentre le stampanti laser e a getto d'inchiostro dominano il mercato delle stampanti domestiche e da ufficio, in altre situazioni, dove sono necessari particolari requisiti in termini di qualità del colore, costi e altro, sono utilizzati altri tipi di stampanti. Una variante di quelle a getto d'inchiostro è costituita dalle **stampanti a inchiostro solido.** Questo tipo di stampante utilizza quattro speciali inchiostri a cera solidificati in blocchi che vengono successivamente sciolti all'interno di serbatoi riscaldati. Per queste stampanti il tempo di attesa all'accensione può raggiungere i 10 minuti, per consentire lo scioglimento dei blocchi d'inchiostro. L'inchiostro caldo viene spruzzato sulla carta, dove si solidifica e si fonde con essa grazie all'utilizzo di due rulli rigidi. In un certo senso vengono combinate le idee di spruzzare l'inchiostro, come nelle stampanti a getto d'inchiostro, e di fondere l'inchiostro sulla carta con rulli di gomma dura, come nelle stampanti laser.

Un altro tipo di stampante a colori è la **stampante a getto di cera.** Essa ha un ampio nastro rivestito di quattro inchiostri a cera e suddiviso in settori lunghi quanto la larghezza della pagina. Quando la carta passa sotto il nastro migliaia di elementi riscaldanti sciolgono la cera, che si fonde con la carta sotto forma di pixel utilizzando il sistema CMYK. Una volta le stampanti a cera erano la principale tecnologia di stampa a colori,

ma ora stanno per essere sostituite da altri tipi i cui materiali di consumo sono più economici.

Abbiamo poi la **stampante a sublimazione**. Anche se il termine ha un che di freudiano, fisicamente indica il passaggio dallo stato solido a quello gassoso, senza passare da quello liquido. Un noto materiale che sublima è il ghiaccio secco (diossido di carbonio ghiacciato), un altro è lo zolfo. In una stampante a sublimazione un elemento mobile contenente i coloranti CMYK passa sopra una testina di stampa in cui vi sono migliaia di elementi riscaldanti programmabili; i coloranti vengono vaporizzati e assorbiti da una carta speciale collocata vicino alla testina. Ciascun elemento riscaldante può produrre 256 livelli di temperatura; maggiore è la temperatura, maggiore è la quantità di colorante che si deposita e quindi l'intensità del colore. Diversamente da tutte le altre stampanti a colori non è necessario ricorrere alla tecnica dei mezzitoni, dato che è possibile creare una gamma quasi continua di colori. Di solito le stampati fotografiche di piccole dimensioni usano il processo di sublimazione per creare immagini fotografiche altamente realistiche su carta speciale (e costosa).

Infine, veniamo alla **stampante termica**, che contiene una piccola testina di stampa su cui vi sono un certo numero di piccoli aghi. Quando una corrente elettrica passa attraverso un ago, questo si scalda molto e molto in fretta. Nel momento in cui una carta speciale termosensibile passa sotto la testina di stampa, vengono disegnati sulla carta dei punti in corrispondenza degli aghi caldi. In effetti, una stampante termica è simile a una vecchia stampante ad aghi in cui gli aghi premevano contro un nastro da macchina da scrivere per disegnare punti sulla carta dietro il nastro. Le stampanti termiche sono ampiamente utilizzate per stampare gli scontrini nei negozi, nei bancomat, nelle stazioni di servizio automatiche e così via.

2.4.5 Apparecchiature per le telecomunicazioni

Al giorno d'oggi la maggior parte dei calcolatori è connessa a una rete di calcolatori, che spesso è Internet. In questo paragrafo descriveremo come funzionano le apparecchiature che permettono l'accesso a queste reti.

Modem

Dato che l'uso dei calcolatori si è diffuso fortemente negli ultimi anni, è sempre più comune che due computer debbano comunicare fra loro. Molti, per esempio, hanno un PC a casa e lo utilizzano per comunicare con il loro calcolatore al lavoro, con un Internet provider oppure con un sistema bancario on-line. In molti casi è la linea telefonica a permettere la comunicazione fisica tra due sistemi.

Una linea telefonica non è adatta a trasmettere i segnali di un calcolatore, che di solito rappresentano il bit 0 con 0 volt e il bit 1 con 3 o 5 volt, come mostra la Figura 2.38(a). Quando sono trasmessi su una linea telefonica progettata per la voce, i segnali a due livelli subiscono una notevole distorsione, provocando di conseguenza errori di trasmissione. Tuttavia un segnale dalla forma sinusoidale e con una frequenza che varia tra 1000 e 2000 Hz può essere trasmesso con una distorsione relativamente piccola; l'uso di questo segnale, chiamato **portante**, è alla base della maggior parte dei sistemi di telecomunicazione.

Dato che le pulsazioni di un'onda sinusoidale sono conosciute a priori, una sinusoide perfetta non trasmette alcuna informazione. Variandone però l'ampiezza, la frequenza o la fase è possibile trasmettere una sequenza di 1 o di 0, come mostra la Figura 2.38. Questo processo è conosciuto con il termine di **modulazione** e i dispositivi in grado di attuarlo si chiamano modem, dalla prime lettere delle parole inglesi MOdulator DEModulator. Nella **modulazione d'ampiezza** [vedi Figura 2.38(b)] si utilizzano due diverse tensioni per rappresentare rispettivamente i valori 0 e 1. Se si ascoltasse la trasmissione di dati digitali a una velocità molto bassa si sentirebbe un forte rumore per i valori 1 e nessun rumore per i valori 0.

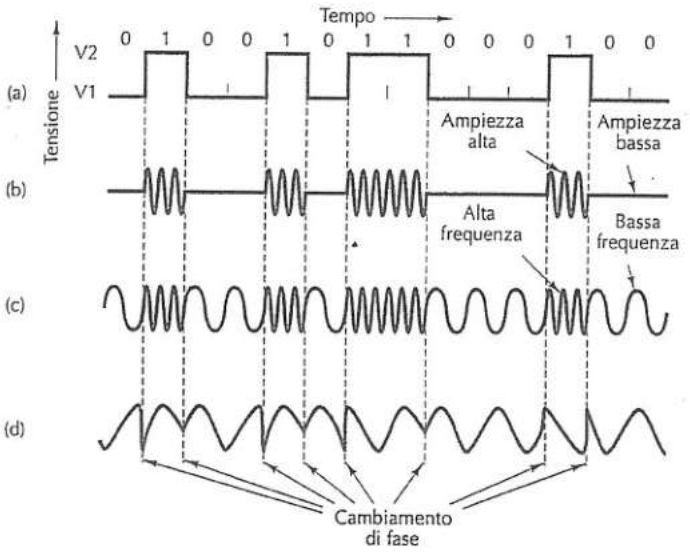


Figura 2.38 Trasmissione bit a bit della stringa 0100101 1000100 su una linea telefonica. (a) Segnale a due livelli. (b) Modulazione d'ampiezza. (c) Modulazione di frequenza. (d) Modulazione di fase.

Nella **modulazione di frequenza** [vedi Figura 2.38(c)] la tensione rimane a un livello costante, ma la frequenza della portante cambia in corrispondenza dei valori 1 e 0. Se si ascoltassero i dati digitali modulati rispetto alla frequenza si sentirebbero due toni distinti, in corrispondenza dei valori 0 e 1. La modulazione di frequenza è spesso indicata con il termine **modulazione (numerica) di frequenza** (*frequency shift keying*).

Nella **modulazione di fase** [vedi Figura 2.38(d)] l'ampiezza e la frequenza non cambiano, ma la fase della portante è invertita di 180 gradi quando i dati passano da 0 a 1 o viceversa. In sistemi a modulazione di fase più sofisticati la fase della portante cambia improvvisamente di 45, 135, 225 o 315 gradi all'inizio di ogni intervallo di tempo, in

modo da poter rappresentare 2 bit per ogni intervallo; questa codifica viene chiamata **modulazione a coppia di bit**. Uno sfasamento della fase di 45 gradi potrebbe rappresentare per esempio 00, uno di 135 gradi potrebbe rappresentare 01, e così via. Esistono anche altri schemi che permettono di trasmettere 3 o più bit per intervallo. Il numero di intervalli di tempo (cioè il numero di potenziali cambiamenti del segnale al secondo) è chiamato **baud**. Se si utilizzano 2 o più bit per intervallo, il bit rate (il valore della velocità in bit al secondo) sarà maggiore del baud rate (il valore della velocità in baud al secondo); è un errore molto comune confondere queste due unità di misura. Ripetiamo per l'ennesima volta: il baud rate esprime il numero di variazioni del segnale per secondo, il bit rate esprime il numero di bit trasmessi per secondo. Il bit rate è generalmente un multiplo del baud rate, anche se teoricamente potrebbe essere più basso.

Se i dati da trasmettere consistono in una serie di caratteri a 8 bit, potrebbe essere comodo disporre di una connessione in grado di trasmettere simultaneamente 8 bit. Per ottenere ciò servirebbero 8 coppie di fili, ma, dato che le linee telefoniche forniscono un solo canale, i bit devono essere spediti in modo seriale, uno dopo l'altro (o in gruppi di due se si usa la modulazione a coppia di bit). Con il termine modem si indica il dispositivo che riceve da un calcolatore i caratteri sotto forma di segnali a due livelli, un bit alla volta, e che trasmette i bit in gruppi di uno o due utilizzando una modulazione di ampiezza, di frequenza o di fase. Di solito si fa precedere ogni carattere a 8 bit da un bit d'inizio e lo si fa seguire da un bit di fine, in modo da marcarne le estremità; in questo modo ogni carattere richiede 10 bit.

Un modem in fase di trasmissione spedisce i singoli bit di un carattere a intervalli di tempo regolari. Una velocità di 9600 baud significa per esempio che il segnale cambia ogni 104 μs. Si utilizza poi un modem in ricezione per convertire una portante modulata in stringhe binarie. Dato che i bit giungono al ricevente a intervalli di tempo uniformi, il modem determina l'inizio del carattere e poi sincronizza il suo orologio per sapere quando deve campionare la linea per leggere i bit in entrata.

I modem moderni lavorano a 56 Kbps e, solitamente, a un baud rate molto più basso, utilizzando una combinazione di tecniche diverse, modulando ampiezza, frequenza e fase. Tutti questi modem sono **full-duplex**, il che significa che possono trasmettere allo stesso tempo in entrambe le direzioni (utilizzando frequenze diverse). Al contrario i modem e le linee di trasmissione che in un dato istante possono trasmettere in una sola direzione sono chiamate **half-duplex**. Le linee che possono trasmettere in una sola direzione sono invece chiamate **simplex**.

Digital subscriber line

Quando le società telefoniche riuscirono a raggiungere la velocità di 56 Kbps, si complimentarono tra loro con grande entusiasmo. Nel frattempo però l'industria della TV via cavo offriva già velocità superiori a 10 Mbps su cavi condivisi e la TV satellitare stava raggiungendo sistemi da 50 Mbps. Dato che l'accesso a Internet aveva assunto un'importanza sempre maggiore nei loro affari, le cosiddette **telcos** (da *telephone companies*, compagnie telefoniche) cominciarono a rendersi conto che avevano bisogno di un prodotto più competitivo rispetto alle linee *dial-up*. La loro risposta fu l'offerta di un nuovo servizio di accesso digitale a Internet; i servizi che offrono una larghezza di banda

maggiore rispetto al servizio telefonico standard sono spesso chiamati a **banda larga** (*broadband*), anche se in realtà questo termine è più che altro utilizzato a fini promozionali. Da un punto di vista squisitamente tecnico broadband indica la presenza di più canali di trasmissione mentre baseband indica che esiste un solo canale. Quindi, in teoria, una rete Ethernet a 10 Gbit, più veloce di qualunque servizio "broadband" offerto dalle compagnie telefoniche, non è per niente broadband, visto che utilizza un solo canale.

All'inizio vi erano molte offerte che si sovrapponevano e che rientravano, al variare di *x*, sotto il nome generale di **xDSL** (*Digital Subscriber Line*). In seguito descriveremo il servizio **ADSL** (*Asymmetric DSL*, "DSL asimmetrica"), destinato con ogni probabilità a diventare il più diffuso. Dato che lo sviluppo della ADSL è ancora in corso e non sono stati definiti tutti gli standard, alcuni dei dettagli seguenti potrebbero cambiare in futuro; ciononostante l'immagine che ne verrà data dovrebbe rimanere sostanzialmente valida. Per maggiori informazioni riguardo la ADSL si veda (Summers, 1999 e Vetter et al., 2000).

La lentezza dei modem deriva dal fatto che i telefoni sono stati inventati per trasportare la voce umana e l'intero sistema è stato attentamente ottimizzato a questo scopo e i dati hanno sempre interpretato il ruolo dei "fratellastri". Il cavo che collega un abbonato alla compagnia telefonica, chiamato **ciclo locale**, di solito viene limitato a 3000 Hz. Questo vincolo, che limita anche la velocità di trasferimento dati, viene ottenuto mediante un filtro che si trova nell'ufficio della compagnia telefonica. La larghezza di banda reale del ciclo locale dipende dalla sua lunghezza e può raggiungere 1,1 MHz per distanze di pochi chilometri.

La Figura 2.39 mostra l'approccio più comune nell'offerta del servizio ADSL. Quello che in pratica viene fatto consiste nel suddividere il ciclo locale in 256 canali indipendenti di 4312,5 Hz ciascuno. Il canale 0 è usato per il servizio telefonico tradizionale, **POTS** (*Plain Old Telephone Service*). I canali 1-5 non sono utilizzati e servono a mantenere separati la voce e i dati in modo che non interferiscano tra loro. Dei 250 canali rimanenti, uno è usato per il controllo del traffico in uscita, un altro per il controllo del traffico in entrata, mentre gli altri sono a disposizione per la trasmissione dei dati. Una ADSL corrisponde quindi a 250 modem.

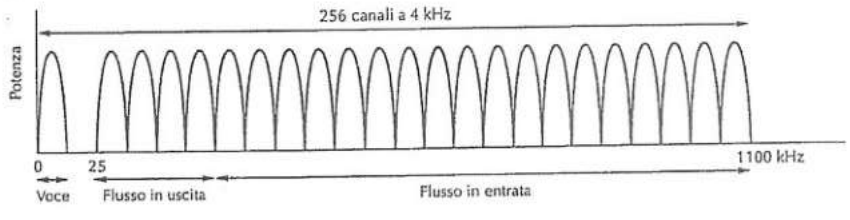


Figura 2.39 Funzionamento di una ADSL.

In teoria ciascuno dei canali rimanenti può essere utilizzato per un flusso dati fullduplex, ma in pratica le armoniche, le interferenze e altri effetti mantengono i sistemi reali molto al di sotto di questo dato teorico. Spetta al fornitore di servizi Internet determinare quanti canali sono usati in uscita e quanti in entrata. Dato che la maggior parte degli utenti scarica più dati di quanti non ne trasmetta, la maggior parte dei fornitori di servizi alloca l'80%-90% della larghezza di banda al canale in entrata (anche se una suddivisione 50-50 tra i due canali sarebbe tecnicamente possibile). A questa scelta si deve la lettera "A" nell'acronimo ADSL. Una suddivisione piuttosto comune prevede di utilizzare 32 canali in uscita e gli altri in entrata.

All'interno di ogni canale si controlla costantemente la qualità della linea e, se necessario, si modifica la velocità di trasferimento dati: per questo motivo canali differenti possono avere velocità diverse. I dati vengono spediti con una combinazione di modulazione di ampiezza e di fase, che permette di trasmettere fino a 15 bit per baud. Per esempio, con 224 canali in entrata e 15 bit/baud a 4000 baud, si ottiene una larghezza di banda in entrata di 13,44 Mbps. In pratica il rapporto tra segnale e rumore non è mai abbastanza buono per ottenere questa velocità, ma su cicli ad alta qualità e su distanze brevi è possibile raggiungere velocità di 8 Mbps.

La Figura 2.40 mostra una tipica organizzazione di una ADSL. In questo schema l'utente o un tecnico della compagnia telefonica devono installare un dispositivo d'interfaccia, **NID** (*Network Interface Device*) presso l'edificio del cliente. Questa piccola scatola di plastica segna la fine della proprietà della compagnia telefonica e l'inizio di quella dell'utente. Vicino al NID (o talvolta combinato con quest'ultimo) è presente un **divisore** (*splitter*), cioè un filtro analogico che divide la banda 0-4000 Hz (per il servizio telefonico tradizionale) dai dati. Il segnale POTS è instradato verso il telefono o il fax esistenti, mentre il segnale dei dati è instradato verso un modem ADSL. Il modem ADSL è un processore di segnale digitale impostato appositamente per funzionare come 250 modem in parallelo a diverse frequenze. Dato che la maggior parte dei modem ADSL attuali è esterna, il calcolatore deve essere connesso attraverso un collegamento ad alta velocità. Di solito ciò si ottiene inserendo una scheda Ethernet (la più diffusa tra le reti locali) nel calcolatore e creando una piccola Ethernet a due nodi che comprende soltanto il calcolatore e il modem ADSL. A volte si utilizza una porta USB al posto della Ethernet, ma in futuro saranno disponibili schede specifiche per modem ADSL.

All'altra estremità del cavo, presso la compagnia telefonica, è installato un altro divisore. Qui la parte del segnale relativa alla voce è separata tramite un filtro e spedita verso un normale commutatore per la voce. I segnali a frequenza maggiore di 26 kHz sono instradati verso un nuovo tipo di dispositivo, chiamato **DSLAM** (*Digital Subscriber Line Access Multiplexer*), che contiene lo stesso tipo di processore di segnale digitale all'interno del modem ADSL. Dopo che il segnale digitale è stato riconvertito in un flusso di bit, si creano i pacchetti di dati da spedire all'ISP.

Internet via cavo

Oggi molte società di TV via cavo offrono l'accesso a Internet attraverso i propri cavi. Dato che la tecnologia differisce significativamente da quella della ADSL, vale la pena analizzarla brevemente. In ogni grande città c'è una sede principale dell'operatore via

cavo, mentre, sparse sul territorio, ci sono un gran numero di scatole piene di componenti elettronici, chiamate **stazioni di testa**. Le stazioni di testa sono connesse alla sede principale da fibre ottiche o da cavi con un'alta larghezza di banda.

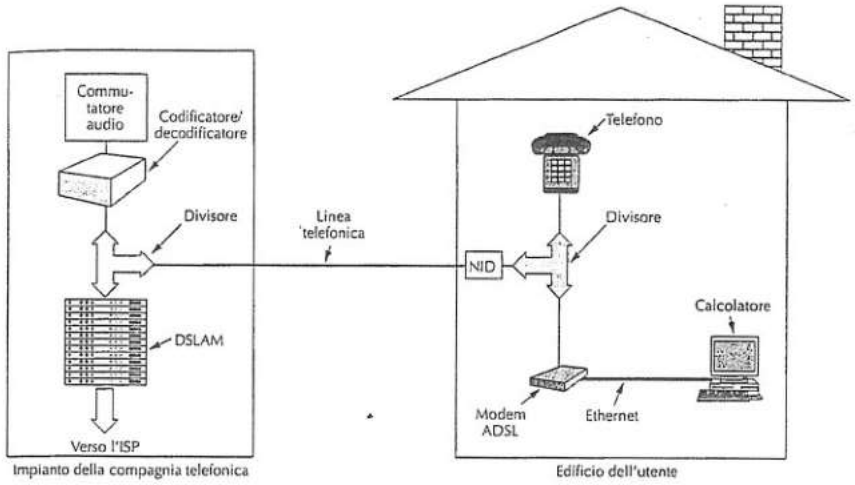


Figura 2.40 Tipica configurazione dei dispositivi per la ADSL.

Da ciascuna stazione di testa partono dei cavi che raggiungono centinaia di case e uffici e i clienti si collegano al cavo che passa vicino alla propria abitazione. Centinaia di utenti condividono quindi uno stesso cavo che parte dalla stazione di testa e la cui larghezza di banda, in genere, è di circa 750 MHz. Questo sistema è radicalmente differente dalla ADSL dove gli utenti hanno un cavo privato (cioè non condiviso) che li connette alla sede della compagnia telefonica. Tuttavia, all'atto pratico, non vi è una gran differenza tra avere il proprio canale a 1,1 MHz verso la sede della compagnia telefonica e dover condividere lo spettro di 200 MHz di un cavo connesso a una stazione di testa con altri 400 utenti, metà dei quali non lo usa nello stesso istante. Ciò significa però che un utente di Internet via cavo avrà un servizio migliore alle 4 di mattina, piuttosto che alle 4 di pomeriggio, mentre le prestazioni del servizio ADSL sono costanti per tutto il giorno. Chi voglia avere un servizio Internet via cavo dalle ottime prestazioni dovrebbe considerare l'ipotesi di trasferirsi in un quartiere residenziale (gli edifici sono lontani gli uni dagli altri e quindi ci sono pochi clienti per cavo) oppure in un quartiere povero (nessuno può permettersi un servizio Internet).

Il fatto che il cavo sia un mezzo condiviso pone il grande problema di determinare chi può spedire dati, quando e a quale frequenza. Per vedere come funziona tutto ciò, occorre prima descrivere brevemente come funziona la TV via cavo. Nel Nord America i canali televisivi via cavo occupano di solito l'intervallo tra 50 e 550 MHz (tranne per

le frequenze delle radio FM comprese tra 88 e 108 MHz). Questi canali hanno una larghezza di 6 MHz, comprese le bande di sicurezza necessarie per prevenire la sovrapposizione tra canali contigui. In Europa lo schema di allocazione è simile, anche se la banda parte da 65 MHz e ogni canale occupa tra i 6 e gli 8 MHz, per via della maggiore risoluzione richiesta dai sistemi PAL e SECAM. La parte inferiore della banda non è utilizzata per la trasmissione televisiva.

Per aggiungere il servizio di Internet, le società via cavo devono risolvere due problemi:

1. come aggiungere l'accesso a Internet senza interferire con i programmi TV;
2. come ottenere un traffico bidirezionale, dato che gli amplificatori sono per loro natura unidirezionali.

Le soluzioni adottate sono le seguenti. I cavi moderni offrono una banda minima di 550 MHz, ma arrivano spesso a 750 MHz e oltre. I canali in uscita (cioè dall'utente alla stazione di testa) sono collocati nella banda compresa tra 5 e 42 MHz (leggermente più alta in Europa), mentre il traffico in entrata (cioè dalla stazione di testa all'utente) utilizza la parte alta della banda totale (Figura 2.41).

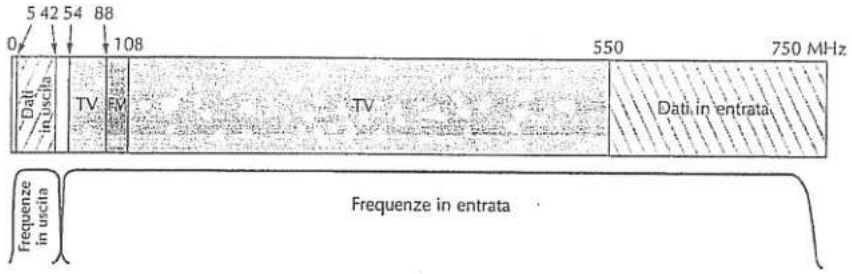


Figura 2.41 Allocazione delle frequenze in un classico sistema di TV via cavo utilizzato per l'accesso a Internet.

Occorre notare che, dato che i segnali televisivi sono tutti in entrata, è possibile utilizzare amplificatori in uscita solo nella regione tra 5 e 42 MHz e amplificatori in entrata che lavorano dai 54 MHz in su, com'è mostrato nella figura. Di conseguenza si ottiene un'asimmetria tra le bande in entrata e quelle in uscita, dato che è disponibile una regione di spettro sopra le frequenza dedicata alla televisione maggiore che non al di sotto.

D'altra parte il traffico è principalmente in entrata e quindi ciò non impensierisce né rattrista gli operatori via cavo. Inoltre, come abbiamo già visto, anche le compagnie telefoniche forniscono spesso un servizio di DSL asimmetrico, senza che ve ne sia alcuna esigenza tecnica.

L'accesso a Internet richiede un apposito modem, che abbia due interfacce: una verso il calcolatore e l'altra verso il cavo della rete. L'interfaccia tra il calcolatore e il modem via cavo è semplice e di solito si tratta di una scheda Ethernet, come nel caso della

ADSL. In futuro l'intero modem potrebbe essere una piccola scheda inserita all'interno del calcolatore, esattamente come per i vecchi modem telefonici.

L'altra interfaccia è invece più complicata. Una gran parte dello standard via cavo è legata all'ingegneria delle trasmissioni radio, un argomento che esula dallo scopo di questo libro. Il solo aspetto che vale la pena sottolineare è che i modem via cavo, come quelli ADSL, sono sempre attivi: stabiliscono una connessione non appena la macchina viene accesa e la mantengono finché non viene spenta, dato che le tariffe degli operatori via cavo non dipendono dal tempo di connessione.

Per comprendere meglio il loro funzionamento, vediamo che cosa succede quando un modem via cavo viene collegato e acceso. Il modem effettua una scansione dei canali in entrata per cercare uno speciale pacchetto spedito periodicamente dalla stazione di testa e che fornisce alcuni parametri di sistema. Non appena trova tale pacchetto il modem comunica la propria presenza utilizzando uno dei canali in uscita. A questo punto la stazione di testa risponde assegnando al modem alcuni canali in entrata e in uscita; la stazione di testa può modificare in un secondo momento tale assegnamento per bilanciare il carico di traffico quando lo reputa necessario.

Il modem determina quindi la propria distanza dalla stazione di testa spedendo uno speciale pacchetto e calcolando quanto tempo impieghi per ricevere una risposta; questo processo è chiamato **ranging**. È importante che il modem conosca questa distanza affinché possa regolare appropriatamente l'utilizzo di canali in uscita e per impostarne correttamente la temporizzazione. I canali sono divisi in intervalli temporali chiamati **minislot**; ogni pacchetto di dati in uscita deve entrare in uno o più *minislot* consecutivi. La stazione di testa comunica periodicamente l'inizio di una nuova serie di *minislot*, ma, per via dei tempi di propagazione sul cavo, questo segnale non è sentito simultaneamente da tutti i modem. Conoscendo la distanza che lo separa dalla stazione di testa ogni modem può calcolare quanto tempo prima era stato in realtà spedito il primo minislot. La lunghezza dei *minislot* dipende dalla rete, ma una tipica lunghezza del campo dati è di 8 byte.

Durante l'inizializzazione la stazione di testa assegna a ciascun modem un *minislot* da usare per richiedere una parte della larghezza di banda in uscita. In genere uno stesso *minislot* viene assegnato a più modem, generando problemi di contesa. Quando un calcolatore vuole spedire un pacchetto, lo trasferisce al modem, che richiede il numero di *minislot* necessario; se la stazione di testa accetta la richiesta, spedisce una conferma sul canale in entrata indicando al modem quali *minislot* sono stati riservati per il suo pacchetto. Il pacchetto viene quindi spedito a partire dal *minislot* che gli è stato allocato ed è inoltre possibile utilizzare un campo dell'intestazione per richiedere pacchetti aggiuntivi.

Se invece si verifica una contesa per la richiesta dei *minislot*, non verrà spedita alcuna conferma e il modem aspetterà un tempo casuale prima di riprovare. Dopo ciascun fallimento, il tempo casuale di attesa viene raddoppiato di modo che il carico si distribuisca meglio nel tempo quando il traffico è molto intenso.

I canali in entrata sono gestiti in maniera differente da quelli in uscita. Per prima cosa, esiste un solo mittente (la stazione di testa) e quindi non ci può essere contesa, e non c'è bisogno dei *minislot*, che in realtà non sono altro che una multiplazione statisti-

ca a divisione di tempo. In secondo luogo, poiché il traffico in entrata è solitamente molto più grande di quello in uscita, si utilizza una dimensione di 204 byte per tutti i pacchetti. Una parte di questi byte contengono un codice a correzione di errore Reed-Solomon e altre informazioni di servizio, lasciando solo 184 byte ai dati dell'utente. Questi valori sono stati scelti per essere compatibili con la televisione terrestre basata su MPEG-2, in modo che la formattazione dei canali della TV e di quelli del traffico in entrata sia la stessa. La Figura 2.42 mostra l'organizzazione logica delle connessioni.

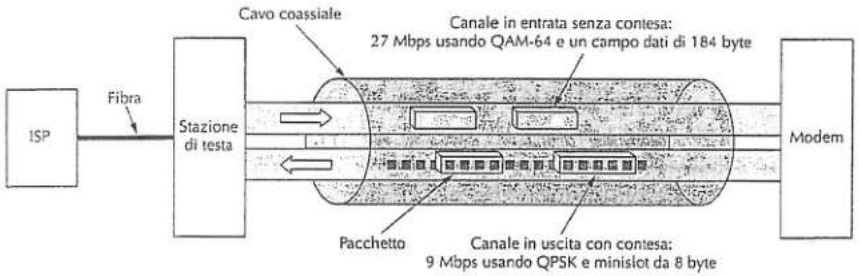


Figura 2.42 Tipici dettagli dei canali in entrata e in uscita nel Nord America.

Tornando all'inizializzazione del modem, dopo aver terminato l'operazione di ranging e ottenuto i propri canali in uscita, quelli in entrata e l'assegnamento dei minislot, il modem è libero di iniziare a spedire pacchetti. Questi giungono alla stazione di testa, che li trasmette via cavo su un canale dedicato alla sede principale della compagnia e quindi al fornitore di servizi (che potrebbe essere la compagnia stessa). Il primo pacchetto serve a richiedere all'ISP un indirizzo di rete (tecnicamente un indirizzo IP), assegnato dinamicamente. Il modem inoltre richiede, e ottiene, l'ora del giorno con grande precisione.

Il passo successivo riguarda la sicurezza. Dato che i cavi sono condivisi, chi lo volesse potrebbe leggere tutto il traffico che passa da lui. Per questo motivo il traffico viene interamente criptato in entrambe le direzioni, per evitare che qualcuno ficchi il naso (letteralmente) nei dati dei propri vicini. Una parte della procedura d'inizializzazione serve a stabilire le chiavi di cifratura. A prima vista si potrebbe pensare che è impossibile che due estranei, la stazione di testa e il modem, possano stabilire una chiave segreta alla luce del sole e davanti a migliaia di persone che li osservano. In realtà ciò è possibile, ma la tecnica utilizzata (l'algoritmo Diffie-Hellman) va oltre lo scopo di questo libro. Per un'analisi al riguardo si veda (Kaufman et al. 2002).

Infine il modem deve farsi identificare e fornire il proprio numero identificativo univoco sul canale sicuro. A questo punto l'inizializzazione è completa e l'utente può registrarsi presso l'ISP e cominciare a utilizzare il servizio.

Ci sarebbero molte altre cose da dire a proposito dei modem via cavo; alcuni significativi riferimenti bibliografici sono (Adams e Dulchinos, 2001; Donaldson e Jones, 2001 e Dutta-Roy, 2001).

2.4.6 Macchine fotografiche digitali

Le macchine fotografiche digitali sono ormai diventate una periferica del calcolatore, dato che i computer vengono sempre più frequentemente usati anche per la fotografia digitale. Spieghiamo brevemente come funziona. Tutte le macchine fotografiche hanno una lente che forma nella parte posteriore, internamente all'apparecchio, un'immagine del soggetto inquadrato. In una macchina fotografica di tipo classico la parte posteriore è coperta da una pellicola su cui si forma un'immagine latente nel momento in cui viene colpita dalla luce; durante lo sviluppo delle foto si può rendere visibile questa immagine utilizzando dei composti chimici. Una macchina fotografica digitale funziona nello stesso modo tranne per il fatto che al posto della pellicola c'è una griglia rettangolare di CCD (*Charge-Coupled Device*, "dispositivo ad accoppiamento di carica"), sensibili alla luce (alcune macchine fotografiche digitali usano i CMOS, ma noi ci concentreremo sui più comuni CCD).

Quando un CCD è colpito dalla luce si carica elettricamente in modo proporzionale alla quantità di luce ricevuta. Questa carica può essere letta da un convertitore analogico-digitale e trasformata in un valore compreso tra 0 e 255 (per gli apparecchi economici) oppure tra 0 e 4095 (per le macchine fotografiche reflex digitali con un'unica lente). Il funzionamento è schematizzato nella Figura 2.43.

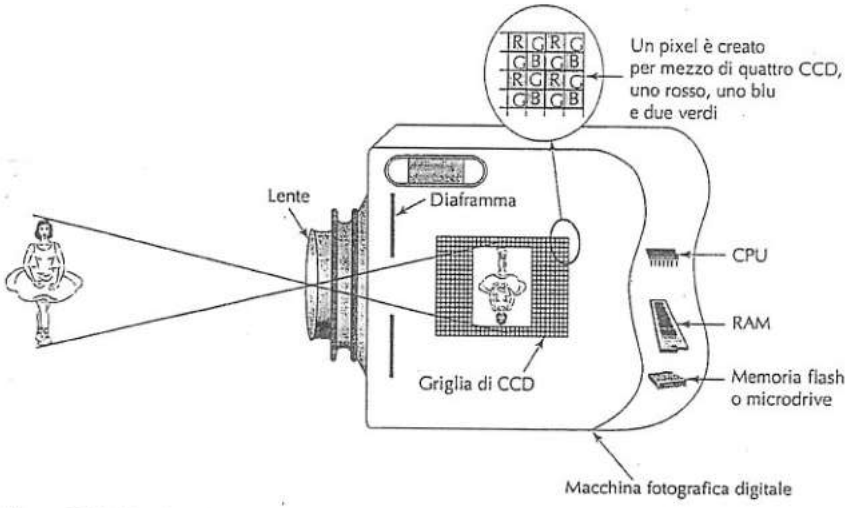


Figura 2.43 Macchina fotografica digitale.

Ciascun CCD produce un unico valore, indipendentemente dal colore della luce che lo colpisce. Per formare un'immagine a colori i CCD sono organizzati in gruppi di quattro elementi. Un **filtro di Bayer** è posizionato sopra il CCD per permettere che la luce rossa

colpisca solo uno dei quattro CCD in ciascun gruppo, la luce blu ne colpisca soltanto unaltro e la luce verde i due restanti. Si utilizzano due elementi per il verde, in quanto è molto più pratico utilizzare quattro CCD per ogni pixel al posto di tre e perché l'occhio umano è più sensibile alla luce verde che alle luci rosse e blu. Quando un produttore di macchine fotografiche digitali afferma che un modello ha, per esempio, 6 milioni di pixel, sta mentendo. La macchina ha 6 milioni di CCD, che corrispondono a 1,5 milioni di pixel. L'immagine sarà letta come una griglia di 2828 × 2121 pixel (sulle macchine a basso costo) oppure di 3000 × 2000 pixel (sulle SLR digitali), mentre i pixel aggiuntivi sono prodotti per interpolazione dal software interno alla macchina fotografica.

Quando viene premuto il pulsante per scattare una foto il software compie tre azioni: imposta la messa a fuoco, determina l'esposizione e calcola il bilanciamento del bianco. La messa a fuoco automatica funziona analizzando le alte frequenze dell'immagine e muovendo la lente finché non vengono massimizzate al fine di ottenere il maggiore dettaglio possibile. L'esposizione è determinata misurando la luce che raggiunge i CCD e regolando di conseguenza il diaframma della lente e il tempo di esposizione, di modo che l'intensità luminosa corrisponda al punto medio dell'intervallo gestito dai CCD. Il bilanciamento del bianco consiste nel misurare lo spettro della luce incidente e nell'effettuare, successivamente, le necessarie correzioni cromatiche.

Successivamente l'immagine viene letta dai CCD e memorizzata come una griglia di pixel nella memoria interna della macchina digitale. Le macchine digitali SLR di livello professionale, utilizzate dai fotografi professionisti, sono in grado di scattare otto fotogrammi ad alta risoluzione al secondo per 5 secondi; esse richiedono circa 1 GB di RAM interna per salvare le immagini prima di poterle elaborare e memorizzare in modo permanente. Le macchine fotografiche meno costose hanno una quantità di memoria RAM leggermente minore.

Nella fase successiva alla cattura dell'immagine, il software della macchina effettua la correzione per il bilanciamento del bianco, in modo da compensare le componenti rossastre o bluastre della luce (dovute per esempio a un soggetto in ombra o all'uso del flash). In seguito applica un algoritmo per la riduzione del rumore e un altro per compensare i CCD difettosi. Tenta poi di rendere l'immagine più definita e meno sfocata (a meno che questa funzione non sia stata disabilitata) cercando i contorni e aumentando l'intensità del gradiente intorno a loro.

Infine l'immagine può essere compressa per ridurre la quantità di spazio richiesta. Un formato comunemente utilizzato è **JPEG** (*Joint Photographic Experts Group*), che prevede l'applicazione della trasformata di Fourier bidimensionale e il taglio di alcune componenti ad alta frequenza. Questa trasformazione riduce il numero di bit necessari a rappresentare l'immagine, sacrificando però i dettagli più fini.

Quando sono terminate tutte le elaborazioni interne alla macchina fotografica, l'immagine viene memorizzata, generalmente su una memoria flash o un hard disk rimovibile di piccole dimensioni chiamato ***microdrive***. La fase di post-elaborazione e la scrittura possono richiedere alcuni secondi per ciascuna immagine.

Quando l'utente torna a casa può collegare la macchina fotografica digitale al calcolatore utilizzando solitamente un cavo USB. Le immagini vengono così trasferite dalla macchina fotografica al disco del calcolatore. Utilizzando programmi appositi, come

Adobe Photoshop, l'utente può ritagliare l'immagine, regolarne luminosità, contrasto, bilanciamento cromatico, nitidezza e fuoco, rimuovere parti dell'immagine e applicare numerosi filtri. Quando è soddisfatto del risultato l'utente può stampare i file delle immagini con una stampante a colori, spedirle via Internet a un sito di condivisione di immagini o a un negozio di stampa oppure può riversarle su un CD-ROM o un DVD per archiviarle ed eventualmente stamparle in un secondo momento. In una macchina fotografica SLR la quantità di potenza computazionale, di RAM, di spazio su hard disk e di software necessario è notevole. Il calcolatore non solo deve compiere tutte le azioni descritte, ma anche comunicare in tempo reale con la CPU della lente e con la CPU del flash, ridisegnare l'immagine sullo schermo LCD e gestire tutti i pulsanti, rotelle, luci, display e tasti della macchina. Si tratta di un sistema integrato estremamente potente, che spesso, in termini di prestazioni, è paragonabile ai calcolatori desktop di qualche anno fa.

2.4.7 Codifica dei caratteri

Ogni calcolatore ha un insieme di caratteri che, come minimo indispensabile, comprende le 26 lettere maiuscole, le 26 lettere minuscole, le cifre da 0 a 9 e un insieme di simboli speciali, come spazio, punto, virgola, segno meno e ritorno a capo.

Per poter utilizzare questi caratteri nel calcolatore occorre assegnare loro un numero: per esempio a = 1, b = 2, ..., z = 26, + = 27, − = 28. La corrispondenza tra caratteri e numeri naturali costituisce un **codice di caratteri**. È necessario che due calcolatori che comunicano fra loro utilizzino lo stesso codice, altrimenti non saranno in grado di capirsi. Per questa ragione sono stati definiti alcuni standard, di cui esamineremo i due più importanti.

ASCII

Un codice ampiamente utilizzato si chiama **ASCII** (*American Standard Code for Information Interchange*, "standard americano per lo scambio d'informazioni"). I caratteri ASCII sono definiti da 7 bit, permettendo così un totale di 128 caratteri distinti. Ciononostante, visto che i computer sono orientati ai byte, ogni carattere ASCII viene normalmente memorizzato in un byte distinto. La Figura 2.44 mostra il codice ASCII. I codici compresi tra 0 e 1F (in esadecimale) sono caratteri di controllo e non vengono stampati. I codici da 128 a 255 non fanno parte della codifica ASCII, ma i PC IBM li hanno utilizzati per caratteri speciali come le *emoticon* e la maggior parte dei computer attuali li utilizza ancora.

Molti dei caratteri di controllo ASCII sono pensati per la trasmissione di dati. Un messaggio potrebbe per esempio consistere di un carattere SOH (*Start of Header*, "inizio intestazione"), un'intestazione, un carattere STX (*Start of Text*, "inizio testo"), il testo stesso, un ETX (*End of Text*, "fine testo") e infine un carattere EOT (*End of Transmission*, "fine trasmissione"). In realtà i messaggi spediti sulle linee telefoniche e sulle reti sono formattati in maniera leggermente differente e i caratteri di controllo ASCII pensati per la trasmissione non sono praticamente più utilizzati.

I caratteri ASCII stampabili comprendono lettere maiuscole e minuscole, cifre, simboli di punteggiatura e alcuni simboli matematici.

Esa	Nome	Significato	Esa	Nome	Significato
0	NUL	Nullo	10	DLE	Uscita trasmissione (Data Link Escape)
1	SOH	Inizio intestazione (Start Of Heading)	11	DC1	Controllo periferica 1
2	STX	Inizio testo (Start Of Text)	12	DC2	Controllo periferica 2
3	ETX	Fine testo (End Of Text)	13	DC3	Controllo periferica 3
4	EOT	Fine trasmissione (End Of Transmission)	14	DC4	Controllo periferica 4
5	ENQ	Interrogazione (Enquiry)	15	NAK	Riconoscimento negativo (Negative AcKnowledgement)
6	ACK	Riconoscimento (ACKnowledgement)	16	SYN	Annulla (SYNchronous Idle)
7	BEL	Campanello (BELL)	17	ETB	End of Transmission Block
8	BS	BackSpace	18	CAN	CANcel
9	HT	Tabulazione orizzontale (Horizontal Tab))	19	EM	Fine supporto (End of Medium)
A	LF	Riga nuova (Line Feed)	1A	SUB	Sostituisci (SUBstitute)
B	VT	Tabulazione verticale (Vertical Tab)	1B	ESC	Esc (ESCape)
C	FF	Avanzamento carta/nuova pagina (Form Feed)	1C	FS	Separatore di file (File Separator)
D	CR	Ritorno a capo (Carriage Return)	1D	GS	Separatore di gruppi (Group Separator)
E	SO	Disinserzione (Shift Out)	1E	RS	Separatore di record (Record Separator)
F	SI	Inserzione (Shift In)	1F	US	Separatore di unità (Unit Separator)

Esa	Car	Esa	Car	Esa	Car	Esa	Car	Esa	Car	Esa	Car
20	Spazio	30	0	40	@	50	P	60	`	70	p
21	!	31	1	41	A	51	Q	61	a	71	q
22	"	32	2	42	B	52	R	62	b	72	r
23	#	33	3	43	C	53	S	63	c	73	s
24	$	34	4	44	D	54	T	64	d	74	t
25	%	35	5	45	E	55	U	65	e	75	u
26	&	36	6	46	F	56	V	66	f	76	v
27	'	37	7	47	G	57	W	67	g	77	w
28	(	38	8	48	H	58	X	68	h	78	x
29	)	39	9	49	I	59	Y	69	i	79	y
2A	*	3A	:	4A	J	5A	Z	6A	j	7A	z
2B	+	3B	;	4B	K	5B	[	6B	k	7B	{
2C	,	3C	<	4C	L	5C	\	6C	l	7C	|
2D	-	3D	=	4D	M	5D	]	6D	m	7D	}
2E	.	3E	>	4E	N	5E	^	6E	n	7E	~
2F	/	3F	?	4F	O	5F	_	6F	o	7F	DEL

Figura 2.44 Caratteri ASCII.

UNICODE

Il mercato dei calcolatori è nato e cresciuto principalmente negli Stati Uniti, il che ha portato a un insieme di caratteri, quello ASCII, adatto alla lingua inglese, ma meno per altre lingue. I francesi hanno bisogno degli accenti (per esempio, *système*), i tedeschi dei segni diacritici (per esempio, *für*), e così via. Alcune lingue europee hanno delle lettere che non sono comprese nel codice ASCII, come la tedesca ß e la danese ø. Altre hanno alfabeti completamente differenti (per esempio, il russo o l'arabo) e alcune lingue addirittura non hanno alfabeto (per esempio, il cinese). Dato che i computer si sono diffusi nei quattro angoli del mondo e i rivenditori di software desiderano vendere i propri prodotti anche nei paesi in cui la maggior parte della gente non parla inglese, è sorta la necessità di un diverso insieme di caratteri.

Il primo tentativo di espandere il codice ASCII fu il codice IS 646, che aggiungeva ulteriori 128 caratteri, trasformandolo così in un codice a 8 bit chiamato **Latin-1**. I caratteri aggiuntivi erano per lo più lettere latine accentate o con segni diacritici. Il tentativo successivo fu il codice IS 8859 che introdusse il concetto di **code page**, un insieme di 256 caratteri specifico di una particolare lingua o di un gruppo di lingue. Il codice IS 8859-1 corrisponde a Latin-1, il codice IS 8859-2 gestisce alcune lingue slave (per esempio, il ceco e il polacco), mentre il codice IS 8859-3 contiene i caratteri richiesti dal turco, dal maltese, dall'esperanto, dal galiziano e da altre lingue. Il problema principale del *code page* è che il software deve tener traccia di quale pagina è attiva, rendendo tra l'altro impossibile mischiare lingue diverse in uno stesso documento; inoltre questo schema non copre né il giapponese né il cinese.

Un gruppo di produttori di calcolatori ha deciso di risolvere il problema formando un consorzio per creare un nuovo sistema, chiamato **UNICODE**, che successivamente è diventato uno Standard Internazionale (IS 10646).

Oggi UNICODE è supportato da alcuni linguaggi di programmazione (per esempio, Java), da alcuni sistemi operativi (per esempio, Windows XP) e da molte applicazioni. Dato che il mercato dei calcolatori tende a diventare sempre più globale è molto probabile che questo standard venga accettato sempre di più.

L'idea alla base di UNICODE consiste nell'assegnare a ogni carattere un valore a 16 bit, chiamato **code point**; non esistono quindi caratteri o sequenze speciali composte da più byte. Il fatto che ogni simbolo sia composto da 16 bit rende più semplice la scrittura di programmi.

Utilizzando simboli a 16 bit, UNICODE ha 65.536 *code point*. Dato che le lingue di tutto il mondo usano complessivamente circa 200.000 simboli, i *code point* sono una risorsa scarsa che deve essere allocata con molta attenzione. Circa metà dei code point sono già stati assegnati e il consorzio UNICODE valuta in continuazione nuove proposte per i rimanenti. Per far sì che il codice UNICODE fosse accettato più velocemente, il consorzio ha deciso intelligentemente di utilizzare Latin-1 per i *code point* compresi tra 0 e 255, rendendo più facile la conversione tra ASCII e UNICODE. Inoltre, per evitare uno spreco eccessivo di *code point*, ogni segno diacritico ha il suo proprio *code point* e spetta al software combinarlo con la lettera vicina per formare un nuovo carattere. Anche se ciò dà più lavoro al software si risparmiano prezioni code point.

Lo spazio dei *code point* è diviso in blocchi, multipli di 16. All'interno di UNICODE a ciascun alfabeto principale è assegnata una sequenza di zone consecutive; alcuni

esempi (e il numero di *code point* allocati) sono il latino (336), il greco (144), il cirillico (256), l'armeno (96), l'ebraico (112), il devanagari (128), il gurmukhi (128), l'oriya (128), il telugu (128) e il kannada (128). Si noti che a queste lingue sono stati assegnati più *code point* delle loro lettere. Questa scelta deriva in parte dal fatto che molte lingue hanno più forme per ciascuna lettera; ciascuna lettera inglese ha, per esempio, due forme: quella minuscola e quella MAIUSCOLA. Alcune lingue hanno tre o quattro forme, talvolta dipendenti dalla posizione della lettera all'interno di una parola, se si trova cioè all'inizio, nel mezzo o alla fine.

I *code point*, oltre alle lettere di questi alfabeti, sono stati assegnati anche ai segni diacritici (112), ai simboli di punteggiatura (112), ai caratteri soprascritti e sottoscritti (48), ai simboli matematici (256), alle forme geometriche (96) e ai simboli ornamentali (192).

Dopo questi *code point* ci sono i simboli richiesti dal cinese, dal giapponese e dal coreano. Prima ci sono 1024 simboli fonetici (cioè il katakana e il bopomofo) e poi gli ideogrammi Han (20.992), usati sia dal cinese sia dal giapponese, e infine le sillabe coreane Hangul (11.156).

6400 *code point* sono inoltre stati allocati per uso locale, di modo che gli utenti possano definire caratteri speciali per usi particolari.

Anche se UNICODE risolve molti problemi legati all'internazionalizzazione, esso non risolve (e non cerca di risolvere) tutti i problemi globali. Per esempio, mentre le lettere dell'alfabeto latino sono nel loro ordine corretto, gli ideogrammi Han non sono ordinati come nel dizionario. Da ciò ne consegue che un programma inglese può ordinare alfabeticamente le parole *cat* e *dog* controllando i valori UNICODE delle loro iniziali, mentre lo stesso non può però essere fatto con un programma giapponese, che necessita di tabelle esterne per capire quale simbolo preceda l'altro.

Un altro problema è che con il tempo continuano ad apparire nuove parole. Cinquant'anni fa nessuno parlava di *app, chatroom, cyberspazio, emoticon, gigabyte, laser, modem, smiley o videocassette*. Se in inglese l'aggiunta di nuove parole non richiede nuovi *code point*, in giapponese sì. Oltre a nuove parole tecniche, c'è la richiesta di aggiungere almeno 20.000 nuovi nomi personali (soprattutto cinesi) e di luoghi. I non vedenti ritengono che il linguaggio Braille dovrebbe far parte di UNICODE, così come altri gruppi con particolari interessi (di tutti i tipi) vorrebbero che ciò che percepiscono fosse codificato, di loro diritto, in *code point*. Il consorzio UNICODE analizza e valuta tutte le nuove proposte.

UNICODE utilizza lo stesso *code point* per i caratteri che, in giapponese e cinese, hanno un aspetto simile, ma un significato differente o una scrittura leggermente diversa (come se gli elaboratori di testo inglesi scrivessero "*blue*" come "*blew*" solo perché suonano allo stesso modo). Alcune persone interpretano questo fatto come un'ottimizzazione dovuta alla scarsità di *code point*; altri come un esempio di imperialismo culturale anglosassone (assegnare ai caratteri valori a 16 bit non era forse una scelta altamente politica?). Per rendere la cosa ancora peggiore si pensi che un dizionario giapponese completo contiene 50.000 kanji (nomi esclusi); avendo a disposizione soltanto 20.992 *code point* per gli ideogrammi Han, è necessario compiere delle scelte. Non tutti i giapponesi ritengono che un consorzio di produttori di calcolatori, anche se alcuni sono giapponesi, sia il foro ideale per prendere queste decisioni.

Immaginate che neanche 65.536 *code point* sono bastati per soddisfare tutti! E così nel 1996 furono aggiunti 16 **plane** da 16 bit, portando il numero totale dei caratteri a 1.114.112.

UTF-8

Anche se migliore della codifica ASCII, Unicode alla fine ha esaurito i *code point*. Inoltre Unicode usa 16 bit per carattere per rappresentare testo ASCII puro, il che è uno spreco. Come conseguenza, per rispondere a questi problemi è stato sviluppato un altro schema di codifica: il sistema chiamato **UTF-8 UCS Transformation Format**, dove UCS è l'acronimo di *Universal Character Set* (insieme universale di caratteri), in sostanza Unicode. I codici UTF-8 sono di lunghezza variabile, da 1 a 4 byte, e possono codificare circa due miliardi di caratteri. UTF-8 è l'insieme di caratteri dominante nel World Wide Web.

Una delle vantaggiose proprietà di UTF-8 è che codici da 0 a 127 corrispondono ai caratteri ASCII, che possono così essere espressi in 1 byte (rispetto ai 2 di Unicode). Per gli altri caratteri, il bit più significativo del primo byte è impostato a 1, a indicare che seguono uno o più byte aggiuntivi. Sono utilizzati in tutto sei formati differenti, come illustrato nella Figura 2.45. I bit contrassegnati con "d" sono bit di dati.

Bit	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6
7	0ddddddd					
11	110ddddd	10dddddd				
16	1110dddd	10dddddd	10dddddd			
21	11110ddd	10dddddd	10dddddd	10dddddd		
26	111110dd	10dddddd	10dddddd	10dddddd	10dddddd	
31	1111110x	10dddddd	10dddddd	10dddddd	10dddddd	10dddddd

Figura 2.45 Lo schema di codifica UTF-8.

UTF-8 ha diversi vantaggi rispetto a Unicode e ad altri sistemi di codifica. Primo, se un programma o un documento utilizza solo i caratteri che fanno parte della codifica ASCII, ogni carattere può essere rappresentato con 8 bit. In secondo luogo, il primo byte di ogni carattere UTF-8 determina univocamente il numero di byte nel carattere. Terzo, i byte successivi al primo in un carattere UTF-8 iniziano sempre con 10, cosa mai vera per il byte iniziale, rendendo il codice auto sincronizzante. In particolare, in caso di errore di comunicazione o di memoria, è sempre possibile andare avanti e trovare l'inizio del carattere successivo (ammesso che non sia stato danneggiato).

Normalmente UTF-8 è usato per codificare soltanto i 17 plane di Unicode, anche se lo schema ammette molto di più dei 1.114.112 *code point*. Tuttavia, se gli antropologi scoprissero nuove tribù in Nuova Guinea o altrove le cui lingue non sono attualmente note (o se in futuro entreremo in contatto con gli extraterrestri), UTF-8 sarebbe pronto per l'inserimento dei loro alfabeti o ideogrammi.

2.5 Riepilogo

I sistemi di calcolo sono costituiti da tre tipi di componenti: processori, memorie e dispositivi di I/O. Il compito di un processore è quello di prelevare istruzioni, decodificarle ed eseguirle. Il ciclo preleva-decodifica-esegui può sempre essere descritto come un algoritmo e, difatti, è spesso eseguito da un interprete software che funziona a un livello inferiore. Per guadagnare velocità molti calcolatori hanno una o più pipeline oppure un'architettura superscalare con più unità funzionali che lavorano in parallelo. La pipeline permette di spezzare le istruzioni in più parti e di eseguire parti di istruzioni diverse in contemporanea. L'uso di unità funzionali multiple è un altro modo per ottenere un parallelismo senza influenzare l'insieme delle istruzioni o l'architettura visibili dal programmatore o dal compilatore.

I sistemi con più processori sono sempre più diffusi. I calcolatori paralleli comprendono gli array di processori, nei quali la stessa operazione è calcolata su più insiemi di dati allo stesso tempo, i multiprocessori, nei quali più CPU condividono una stessa memoria, e i multicomputer, nei quali più calcolatori, ciascuno dotato di una propria memoria, comunicano scambiandosi messaggi.

Le memorie possono essere classificate come primarie o secondarie. La memoria primaria è utilizzata per contenere il programma che viene eseguito in un dato momento. Il suo tempo di accesso è breve, di poche decine di nanosecondi al massimo, e indipendente dall'indirizzo al quale si vuole accedere. Le cache riducono ulteriormente questo tempo di accesso e sono utili perché essendo la velocità del processore maggiore di quella della memoria, l'attesa per l'accesso alla memoria rallenta notevolmente l'esecuzione dei processi. Alcune memorie sono inoltre equipaggiate con codici a correzione d'errore per migliorarne l'affidabilità.

Le memorie secondarie, al contrario, hanno tempi di accesso molto più lunghi (millisecondi e più) e dipendenti dalla posizione del dato che deve essere letto o scritto. Le più comuni memorie secondarie sono i nastri, le memorie flash, i dischi magnetici e i dischi ottici. Esistono diversi tipi di dischi magnetici, tra i quali gli IDE, gli SCSI e i RAID. I dischi ottici comprendono i CD-ROM, i CD-R, i DVD e i Blu-Ray.

I dispositivi di I/O sono utilizzati per trasferire informazioni dal e nel calcolatore, e sono collegati al processore e alla memoria mediante uno o più bus. Alcuni esempi comprendono i terminali, i mouse, le stampanti e i modem. La maggior parte dei dispositivi di I/O utilizza il codice di caratteri ASCII, ma viene utilizzato anche UNICODE e UTF-8 sta rapidamente guadagnando sempre più consenso man mano che il mondo dei computer diventa sempre più web-centrico.

PROBLEMI

1. Consideriamo il funzionamento di una macchina con il percorso dati mostrato nella Figura 2.2. Supponiamo che il caricamento dei registri di input richieda 5 ns, l'esecuzione della ALU 10 ns e la memorizzazione del risultato nel registro 5 ns. Qual è il numero massimo di MIPS che può raggiungere questa macchina senza far ricorso alla pipeline?
2. Qual è lo scopo del passo 2 nella lista del Paragrafo 2.1.2? Che cosa succederebbe se questo passo fosse omesso?

3. Sul calcolatore 1 tutte le istruzioni richiedono 10 ns per essere eseguite. Sul calcolatore 2 esse richiedono invece 5 ns. Si può dire con certezza che il calcolatore 2 è più veloce? Si argomenti la risposta.
4. Immaginiamo di progettare un calcolatore a singolo chip per un sistema integrato. Il chip conterrà anche tutta la memoria e funzionerà alla stessa velocità della CPU senza alcuna penalizzazione per l'accesso ai dati. Si esaminino i principi illustrati nel Paragrafo 2.1.4 e si dica se essi sono così importanti (ipotizzando che si desideri ottenere alte prestazioni).
5. Un certo calcolo è altamente sequenziale, in quanto ogni passo dipende dai precedenti. Per questa elaborazione sarebbe più appropriato un array di processori o un processore a pipeline? Si spieghi la scelta.
6. Per competere con le macchine tipografiche appena inventate, un monastero medioevale decise di riprodurre libri scritti a mano in un gran numero di copie facendo lavorare insieme, in un'enorme stanza, numerosi amanuensi. L'abate pronunciava ad alta voce la prima parola del libro da riprodurre e tutti gli amanuensi la trascrivevano. L'abate pronunciava quindi la seconda parola e gli amanuensi la trascrivevano. Questo processo veniva ripetuto finché l'intero libro non era letto ad alta voce e copiato. Quale fra i sistemi con parallelismo a livello di processore descritti nel Paragrafo 2.1.6 assomiglia più da vicino a questo sistema?
7. Scendendo nei cinque livelli della gerarchia di memorie trattati nel testo, i tempi di accesso aumentano. Si faccia una ragionevole stima del rapporto del tempo di accesso dei dischi ottici rispetto a quello dei registri della memoria. Si assuma che i dischi siano sempre caricati nel lettore.
8. Alla tipica domanda: "Credi al topolino dei dentini caduti?" i sociologi possono fornire tre risposte, sì, no e non so. In base a ciò la Società di Calcolatori Sociomagnetici ha deciso di costruire un calcolatore per elaborare i dati dei sondaggi. Questo computer ha una memoria trinaria, cioè ciascun byte (tryte?) consiste di 8 *trit*, ognuno dei quali può contenere i valori 0, 1 oppure 2. Quanti trit sono necessari per memorizzare un numero a 6 bit? Si dia un'espressione per il numero di trit necessari per memorizzare n bit.
9. Si calcoli la velocità di trasferimento dati dell'occhio umano utilizzando le seguenti informazioni. Il campo visivo è composto da circa 106 elementi (pixel). Ciascun pixel può essere ridotto alla sovrapposizione di tre colori primari, ciascuno dei quali ha 64 intensità. La risoluzione temporale è di 100 ms.
10. Si calcoli la velocità di trasferimento dati dell'orecchio umano usando le seguenti informazioni. Le persone possono sentire frequenze fino a 22 KHz. Per catturare tutta l'informazione contenuta in un segnale sonoro a 22 KHz è necessario campionare il suono al doppio della frequenza, cioè a 44 KHz. Con ogni probabilità un campione a 16 bit è sufficiente per catturare la maggior parte delle informazioni uditive (cioè l'orecchio non può distinguere più di 65.536 livelli d'intensità).
11. In tutti gli esseri viventi l'informazione genetica è codificata nelle molecole di DNA. Una molecola di DNA è una sequenza lineare dei quattro nucleotidi base: A, C, G e T. Il genoma umano contiene approssimativamente 3 × 109 nucleotidi sotto forma di circa 30.000 geni. Qual è la capacità totale (in bit) dell'informazione del genoma umano? Qual è la massima capacità d'informazione (in bit) del gene medio?
12. Un certo calcolatore può essere equipaggiato con 1.073.741.824 di byte di memoria. Perché un produttore dovrebbe scegliere un numero così particolare, invece di un numero più facile da ricordare come 1.000.000.000?
13. Si definisca un codice di Hamming a 7 bit con parità pari per le cifre comprese tra 0 e 9.
14. Si definisca un codice per le cifre comprese tra 0 e 9 la cui distanza di Hamming sia 2.
15. In un codice di Hamming alcuni bit vengono "sprecati", nel senso che sono utilizzati per effettuare il controllo e non per memorizzare l'informazione. Qual è la percentuale di bit sprecati per messaggi la cui lunghezza totale (dati + bit di controllo) è $2^n - 1$? Si valuti numericamente questa espressione per valori di n compresi tra 3 e 10.
16. Un carattere ASCII esteso è rappresentato su 8 bit. La codifica di Hamming associata a ogni carattere può dunque essere rappresentata da una stringa di 3 cifre esadecimali. Si codifichi il seguente testo composto da 5 caratteri ASCII estesi con l'utilizzo di un codice di Hamming a parità pari: Earth. Si rappresenti la risposta come stringa di cifre esadecimali.

17. La seguente stringa di cifre esadecimali codifica alcuni caratteri ASCII estesi in un codice di Hamming a parità pari: 0D3 DD3 0F2 5C1 1C5 CE3. Si decodifichino queste stringhe e si scrivano i caratteri che erano codificati.

18. Il disco mostrato nella Figura 2.19 ha 1024 settori per traccia e una velocità di rotazione di 7200 RPM. Qual è la velocità di trasferimento dati del disco per una singola traccia?

19. Un calcolatore ha un bus con un tempo di ciclo di 5 ns, durante il quale può leggere o scrivere dalla memoria una parola a 32 bit. Il calcolatore ha un disco Ultra4-SCSI che utilizza un bus e funziona a 160 MB/s. Solitamente la CPU preleva ed esegue un'istruzione a 32 bit a ogni intervallo di 1 ns. In che misura il disco rallenta la CPU?

20. Si immagini di dover scrivere la parte di un sistema operativo dedicata alla gestione del disco. Dal il punto di vista logico il disco sarà rappresentato come una sequenza di blocchi, a partire da 0 nella parte interna fino a un certo valore massimo nella parte esterna. Quando i file vengono creati occorre allocare settori liberi, cosa che si può fare a partire dall'interno oppure dal l'esterno. Ha importanza la scelta della strategia? Si motivi la risposta.

21. Quanto tempo è necessario per leggere un disco con 10.000 cilindri, ciascuno dei quali contiene quattro tracce di 2048 settori? Prima devono essere letti tutti i settori della traccia 0 a partire dal settore 0, poi tutti i settori della traccia 1 a partire dal settore 0, e così via. Il tempo di rotazione è di 10 ms e una ricerca richiede 1 ms tra cilindri adiacenti e 20 ms nel caso peggiore. Lo spostamento da una traccia all'altra dello stesso cilindro può essere fatta istantaneamente.

22. Il RAID livello 3 è in grado di correggere errori singoli utilizzando solo un disco di parità. Qual è la particolarità del RAID livello 2? Dopo tutto anch'esso è in grado di correggere solo gli errori singoli, ma necessita di un numero maggiore di dischi.

23. Qual è l'esatta capacità (in byte) di un CD-ROM Modo 2 per il supporto, ormai standardizzato, da 80 minuti? Qual è la capacità disponibile per i dati degli utenti utilizzando il Modo 1?

24. Per masterizzare un CD-R il laser deve pulsare ad alta frequenza. Qual è la lunghezza di un impulso, in nanosecondi, quando la frequenza è 10x e si utilizza il Modo 1?

25. Per poter inserire 133 minuti di video su un DVD a singolo lato e singolo strato è necessario utilizzare un'elevata compressione. Si calcoli il fattore di compressione richiesto, assumendo che sia disponibile uno spazio di 3,5 GB per la traccia video, che la risoluzione dell'immagine sia di 720 × 480 pixel con colori a 24 bit (RGB, 8 bit per colore) e che le immagini siano visualizzate a 30 frame/s.

26. I Blu-Ray hanno una velocità di 4,5 MB/s e una capacità di 25 GB. Quanto tempo è necessario per leggere l'intero disco?

27. La velocità di trasferimento dati tra la CPU e la sua memoria associata è di vari ordini di grandezza maggiore rispetto alla velocità delle componenti meccaniche di I/O. In che modo questo sbilanciamento può causare inefficienze? Com'è possibile alleviarle?

28. Un produttore afferma che il proprio terminale può visualizzare una bitmap con 2^{24} colori diversi. Se l'hardware ha 1 solo byte per ciascun pixel, com'è possibile ottenere questo valore?

29. Siete parte di un team internazionale segreto di scienziati al quale è stato appena affidato il compito di studiare un essere chiamato Herb, un extra-terrestre proveniente dal pianeta 10 recentemente arrivato sulla terra. Herb vi ha fornito le seguenti informazioni sul funzionamento dei suoi occhi. Il suo campo visivo consiste in 10^8 pixel. Ogni pixel è essenzialmente una sovrapposizione di 5 "colori" (per esempio, infrarosso, rosso, verde, blu e ultravioletto) ognuno dei quali ha 32 intensità. Il tempo di risoluzione del campo visivo di Herb è di 10msec. Calcolare la velocità, espressa in GB/s, degli occhi di Herb.

30. Un terminale ha uno schermo con una risoluzione di 1920x1080, che viene ridisegnato 75 volte al secondo. Quanto è lungo l'impulso che corrisponde a un pixel?

31. Una stampante monocromatica può stampare in ogni pagina 50 linee da 80 caratteri. Un carattere occupa in media un riquadro di 4 mm², di cui circa il 25% è ricoperto da *toner*, mentre il resto ne è privo. Uno strato di *toner* ha uno spessore di 25 μ e una cartuccia di *toner* per la stampante misura 25 × 8 × 2 cm. Per quante pagine può essere utilizzata una cartuccia?

32. Quando un testo ASCII con parità dispari viene trasmesso in modo asincrono alla velocità di 5600 caratteri/s su un modem da 56.000 bps, qual è la percentuale dei bit ricevuti che contiene effettivamente dati (e non è overhead)?

33. La società di Modem Hi-Fi ha appena progettato un nuovo modem a modulazione di frequenza che usa 64 frequenze invece di solo 2. Ciascun secondo è diviso in *n* intervalli temporali uguali, ognuno dei quali contiene uno dei 64 possibili toni. Quanti bit per secondo può trasmettere questo modem, utilizzando la trasmissione asincrona?

34. Un utente Internet si è abbonato a un servizio ADSL a 2 Mbps. La sua vicina si è abbonata a un servizio di Internet via cavo che utilizza una banda condivisa di 12 MHz. Lo schema di modulazione utilizzato è QAM-64 e ci sono *n* case per cavo, ciascuna con un calcolatore. Solo una frazione *f* di questi calcolatori è in linea nello stesso momento. In quali condizioni l'utente via cavo ottiene un servizio migliore dell'utente ADSL?

35. Una macchina fotografica digitale ha una risoluzione di 3000 × 2000 pixel e usa 3 byte/pixel per rappresentare i colori RGB. Il produttore della macchina fotografica vorrebbe poter scrivere su una memoria flash, in soli 2 s, un'immagine JPEG con un fattore di compressione 5x. Quale velocità di trasferimento dati è richiesta?

36. Una macchina fotografica digitale professionale ha un sensore con 24 milioni di pixel, ciascuno con 6 byte/pixel. Quante immagini possono essere memorizzate su una memoria flash da 8 GB se il fattore di compressione è 5x? Si assuma che 1 GB valga 2^{30} byte.

37. Si stimi quanti caratteri, spazi compresi, contiene in genere un libro di informatica. Quanti bit sono necessari per codificare un libro usando il codice ASCII con parità? Quanti CD-ROM servono per immagazzinare una libreria di 10.000 libri? Quanti DVD a doppio lato e doppio strato sono necessari per memorizzare la stessa libreria?

38. Si scriva una procedura *hamming* (*ascii*, *codificata*) che converta i 7 bit meno significativi di *ascii*in una parola di codice che sia un intero a 11 bit e che venga memorizzato in *codificata*.

39. Si scriva una funzione *distanza* (*codice*, *n*, *k*) che prenda come input un array *codice* di *n* caratteri di *k* bit ciascuno e restituisca come output la distanza dell'insieme di caratteri.

CAPITOLO 3

Livello logico digitale

Alla base della gerarchia mostrata nella Figura 1.2 si trova il livello logico digitale, ovvero il vero e proprio hardware del calcolatore. In questo capitolo approfondiremo quegli aspetti della logica digitale che serviranno poi da base per lo studio dei livelli superiori. Anche se gli argomenti affrontati si trovano al confine tra l'informatica e l'elettronica il materiale è "autocontenuto": per poterne seguire la trattazione non sono quindi necessarie specifiche conoscenze ingegneristiche o relative all'hardware.

Gli elementi base a partire dai quali sono costruiti tutti i calcolatori digitali sono incredibilmente semplici. Inizieremo il nostro studio partendo da questi componenti elementari e dalla particolare algebra a due valori (algebra di Boole) che si utilizza per analizzarli. Successivamente esamineremo alcuni circuiti fondamentali, tra cui quelli per eseguire calcoli aritmetici costruiti mediante semplici combinazioni di porte logiche. L'argomento successivo sarà l'organizzazione della memoria, ovvero com'è possibile combinare le porte logiche per memorizzare informazioni. In seguito verrà affrontato il tema delle CPU e in particolare il modo in cui CPU composte da un unico chip si interfacciano con memoria e periferiche. Nella parte finale del capitolo saranno presentati, a titolo di esempio, numerosi prodotti reali.

3.1 Porte logiche e algebra di Boole

I circuiti digitali possono essere costruiti combinando tra loro un piccolo numero di componenti elementari. Nei paragrafi successivi descriveremo questi elementi base, mostreremo come possono essere combinati e introdurremo una potente tecnica matematica utilizzabile per analizzarne il comportamento.

3.1.1 Porte logiche

Un circuito digitale è un circuito in cui sono presenti solo due valori logici[1]. Generalmente un valore (per esempio il numero binario 0) è rappresentato da un segnale compreso tra 0 e 0,5 volt, mentre l'altro valore (per esempio il numero binario 1) è rappresentato da un segnale compreso tra 1 e 1,5 volt; le tensioni al di fuori di questi intervalli non sono ammesse. La base hardware di tutti i calcolatori digitali è costituita da alcuni piccoli dispositivi elettronici, chiamati **porte logiche** (*gate*), ciascuna delle quali calcola una diversa funzione di questi segnali.

I dettagli sul funzionamento interno delle porte logiche esulano dallo scopo di questo libro, in quanto rientrano nel campo del **livello dei dispositivi**, che si trova al di sotto del nostro livello 0. Ciononostante, faremo ora una piccola digressione al riguardo, senza però entrare nei dettagli più complessi, in modo da fornire un'immagine d'insieme dell'idea base. Tutta la moderna logica digitale si fonda, in ultima analisi, sul fatto che un transistor può essere costruito in modo da funzionare come un velocissimo interruttore binario. Nella Figura 3.1(a) è possibile vedere un transistor integrato in un semplice circuito. I transistor hanno tre connessioni verso il mondo esterno: il **collettore**, la **base** e l'**emettitore**. Quando la tensione in ingresso scende sotto un valore critico, chiamato V_{in}, il transistor viene disabilitato e si comporta come una resistenza infinita[2]. La conseguenza è che l'output del circuito, V_{out}, assume un valore vicino a V_{cc}: una tensione regolata esternamente che, per questo tipo di transistor, vale generalmente +5 volt. Quando, al contrario, V_{in} supera il valore critico, il transistor si attiva e si comporta come un conduttore ideale[3], facendo scaricare V_{out} a terra (per convenzione, 0 volt).

È importante notare che quando V_{in} è basso, V_{out} è alto e viceversa. Questo circuito è quindi un invertitore che converte un valore logico 0 in un valore logico 1 e un valore logico 1 in un valore logico 0. La resistenza (la linea a zig zag) è necessaria per limitare la quantità di corrente nel transistor ed evitare che esso si fonda. In genere il tempo richiesto per passare da uno stato a un altro è di meno di un nanosecondo.

La Figura 3.1(b) mostra due transistor collegati in serie. Se V_1 e V_2 sono alte, allora entrambi i transistor saranno in conduzione e V_{out} sarà portato al valore basso, mentre, se almeno uno degli ingressi è basso, allora i transistor corrispondenti saranno disattivati e l'uscita sarà alta. In altre parole V_{out} sarà basso se e soltanto se sia V_1 sia V_2 sono alte.

Nella Figura 3.1(c) i due transistor sono collegati in parallelo, invece che in serie. In questa configurazione se uno dei due ingressi è alto, allora il transistor corrispondente sarà attivato e l'uscita sarà scaricata a terra, mentre, se entrambi gli ingressi sono bassi, l'uscita rimarrà alta.

Questi tre circuiti, così come altri con lo stesso funzionamento, formano le tre porte logiche più semplici, chiamate rispettivamente NOT, NAND e NOR. Dato che spesso le porte NOT vengono chiamate **invertitori**, nel corso del testo utilizzeremo i due termini in modo equivalente. Se assumiamo la convenzione di interpretare "alto" (Vcc

[1] In perfetta analogia a quanto accade nella logica classica (*N.d.Rev.*).
[2] Circuito aperto (*N.d.T.*).
[3] Corto circuito (*N.d.T.*).

volt) come un valore logico 1 e "basso" (terra) come un valore logico 0, allora è possibile esprimere il valore in uscita come una funzione dei valori in ingresso. La Figura 3.2(a)-(c) mostra i simboli usati per indicare queste tre porte logiche e il loro comportamento funzionale. Nelle figure, A e B rappresentano gli ingressi, X l'uscita e ogni riga specifica il valore in uscita data una particolare combinazione dei valori in ingresso. Se si fa passare il valore in uscita della Figura 3.1(b) in un circuito invertitore, si ottiene un nuovo circuito il cui comportamento è esattamente l'opposto di quello della portalogica NAND. In questo nuovo circuito l'uscita vale 1 se e solo se entrambi gli ingressi valgono 1; esso realizza la porta logica chiamata AND, il cui simbolo e la cui descrizione funzionale sono dati nella Figura 3.2(c). Analogamente è possibile collegare la porta logica NOR a un invertitore, in modo da ottenere un circuito la cui uscita valga 1 se almeno uno dei due ingressi vale 1, mentre valga 0 se entrambi gli ingressi valgono 0. La Figura 3.2(e) mostra il simbolo e la descrizione funzionale di questo circuito, chiamato porta logica OR. I piccoli cerchietti, utilizzati come parte dei simboli dell'invertitore, della porta NAND e della porta NOR, rappresentano sempre un'inversione.

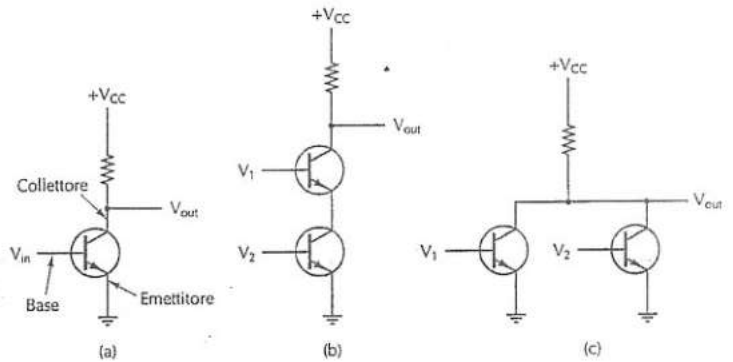
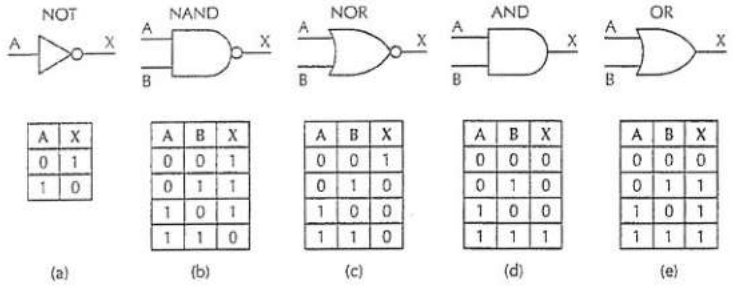


Figura 3.1 (a) Un invertitore. (b) Una porta NAND. (c) Una porta NOR.

Le porte logiche della Figura 3.2 sono i principali elementi costitutivi del livello logico digitale. Dall'analisi precedente è chiaro che le porte NAND e NOR necessitano di due transistor ciascuna, mentre le porte AND e OR ne richiedono tre; per questa ragione molti calcolatori sono basati sulle porte logiche NAND e NOR piuttosto che sulle più familiari porte AND e OR. In realtà tutte le porte logiche sono implementate in modo piuttosto diverso, ma in ogni caso le porte NAND e NOR rimangono più semplici di quelle AND e OR. Parlando di porte logiche vale la pena precisare che esse possono avere anche più di due ingressi; per fare un esempio una porta NAND potrebbe avere in teoria un numero di ingressi piuttosto elevato, anche se in pratica difficilmente ce ne sono più di otto.

Dato che ci si riferirà frequentemente alle principali tecnologie di costruzione delle porte logiche ne citiamo ora le principali, anche se questo argomento fa parte del livello dei dispositivi. Le due tecnologie principali sono quella **bipolare** e quella MOS (*Metal Oxide Semiconductor*, "semiconduttore metallo ossido"). Fra le bipolari, i tipi più importanti sono la **TTL** (*Transistor-Transistor Logic*), per anni il motore dell'elettronica digitale, e la **ECL** (*Emitter-Coupled Logic*), utilizzata quando è richiesto un funzionamento a velocità molto elevate. Attualmente nei circuiti dei calcolatori si impiega largamente la tecnologia MOS.

Le porte logiche MOS sono più lente delle TTL e delle ECL, ma richiedono molta meno potenza e hanno una dimensione decisamente inferiore, permettendo così di metterne insieme un numero elevato in uno spazio limitato. Esistono diverse varietà di porte MOS, tra cui le PMOS, le NMOS e le CMOS. I transistor MOS, pur essendo costruiti in modo differente rispetto ai transistor bipolari, sono tuttavia in grado di funzionare come degli interruttori elettronici. La maggior parte delle CPU e delle memorie moderne utilizza la tecnologia CMOS che funziona a una tensione di circa +1,5 volt. Questo è tutto ciò che occorre conoscere a proposito del livello dei dispositivi; i lettori interessati ad approfondire lo studio di questo livello possono consultare i riferimenti bibliografici suggeriti sul sito web del libro.



A	X
0	1
1	0

(a)

A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

(b)

A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

(c)

A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

(d)

A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

(e)

Figura 3.2 Simboli e comportamenti funzionali di cinque porte logiche elementari.

3.1.2 Algebra di Boole

Per poter descrivere i circuiti costruiti combinando le porte logiche occorre definire un nuovo tipo di algebra in cui variabili e funzioni possono assumere soltanto i valori 0 e 1. Questa struttura viene chiamata **algebra di Boole**, in quanto è stata sviluppata dal matematico inglese George Boole (1815-1864). Per essere precisi, ci riferiamo in realtà a un particolare tipo di algebra booleana, chiamata **algebra booleana minimale**; ciononostante il termine "algebra booleana" è utilizzato in modo talmente esteso che non faremo alcuna distinzione. Esattamente come esistono le funzioni nell'algebra ordinaria (cioè quella del liceo), esistono anche le funzioni nell'algebra booleana. Una funzione booleana ha una o più variabili di input e genera un valore di output che dipende soltanto dai valori di queste variabili. Una semplice funzione f può essere definita assumendo che $f(A)$ valga 1 se A vale 0 e $f(A)$ valga 0 se A vale 1; essa corrisponde alla funzione NOT della Figura 3.2(a).

Una funzione booleana di n variabili può essere completamente descritta mediante una tabella di 2^n righe; dato che esistono 2^n possibili combinazioni dei valori di input, ciascuna riga può quindi indicare il valore della funzione per una data combinazione degli input. Una tabella di questo tipo è chiamata **tabella di verità**; tutte le tabelle della Figura 3.2 ne sono un esempio. Se si decide di elencare sempre le righe di una tabella di verità in ordine numerico, cioè seguendo la sequenza 00, 01, 10 e 11 nel caso di due variabili, la funzione può essere completamente descritta dal numero binario a 2^n bit che si ottiene leggendo in verticale la colonna del risultato. Adottando questa convenzione, NAND è 1110, NOR è 1000, AND è 0001 e OR è 0111. Con due variabili esistono ovviamente soltanto 16 diverse funzioni booleane, corrispondenti alle 16 combinazioni possibili della stringa a 4 bit che rappresenta i risultati. Nell'algebra ordinaria esiste al contrario un numero infinito di funzioni a due variabili, nessuna delle quali può essere descritta dando una tabella che specifica i risultati per tutti i possibili valori di input. Nell'algebra ordinaria ogni variabile può infatti assumere un infinito numero di valori.

La Figura 3.3(a) mostra la tabella di verità di una funzione booleana a tre variabili: $M = f(A, B, C)$. Essa corrisponde alla funzione logica di maggioranza che vale 0 se la maggioranza dei suoi valori di input vale 0, mentre vale 1 se la maggioranza degli input vale 1. Anche se è possibile specificare in modo completo qualsiasi funzione booleana dandone la sua tabella di verità, all'aumentare del numero di variabili questa notazione diventa di dimensioni eccessivamente grandi. Per questo motivo si preferisce spesso adottare una notazione differente.

Per capire da dove nasce questa notazione alternativa si noti che ogni funzione booleana può essere descritta specificando quali combinazioni delle variabili di input producono, come risultato, 1. Nel caso della funzione mostrata nella Figura 3.3(a) ci sono quattro combinazioni di variabili di input che rendono M pari a 1. Per convenzione aggiungeremo una linea sopra una variabile di input per indicare che il suo valore è invertito (e non la apporremo in caso contrario). Utilizzeremo inoltre la notazione implicita della moltiplicazione, oppure un puntino, per denotare la funzione booleana AND (prodotto logico) e il simbolo + per indicare la funzione booleana OR (somma logica). Per esempio, seguendo queste convenzioni, $A\overline{B}C$ assume valore 1 solo quando A = 1, B = 0 e C = 1. Analogamente $A\overline{B} + B\overline{C}$ assume valore 1 solo quando (A = 1 e B = 0) oppure (B = 1 e C = 0). Le quattro righe della Figura 3.3(a) che producono in output il bit 1 sono: $\overline{A}BC$, $A\overline{B}C$, $AB\overline{C}$ e ABC; dato che la funzione M è vera (cioè vale 1) se una qualsiasi di queste quattro combinazioni è vera, è possibile riscrivere in modo più compatto la tabella di verità usando la seguente notazione

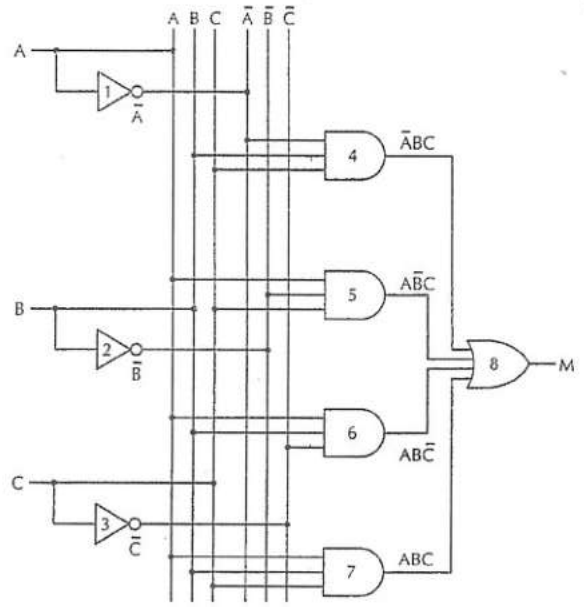
$$M = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$$

Una funzione di n variabili può quindi essere descritta mediante una somma di un massimo di 2^n termini, ciascuno dei quali è costituito dal prodotto logico di n variabili.

Come vedremo tra poco questa formulazione è particolarmente importante in quanto porta direttamente a un'implementazione della funzione per mezzo di porte logiche di tipo standard.

A	B	C	M
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

(a)



(b)

Figura 3.3 (a) Tabella di verità della funzione di maggioranza di tre variabili. (b) Un circuito per (a).

È importante tenere a mente la differenza tra una funzione booleana astratta e la sua implementazione circuitale. Una funzione booleana consiste di variabili, come A, B e C, e operatori booleani come AND, OR e NOT, ed è descritta per mezzo di una tabella di verità o una funzione booleana, per esempio

$$F = A\bar{B}C + AB\bar{C}$$

Una funzione booleana può essere implementata mediante un circuito elettronico (che, in generale, può essere realizzato in vari modi), che utilizza segnali per rappresentare le variabili di input e di output oltre ad alcune porte logiche come AND, OR e NOT. In seguito, anche se talvolta potrà essere ambiguo, utilizzeremo la notazione AND, OR e NOT per riferirci agli operatori booleani e AND, OR e NOT per riferirci alle porte logiche.

3.1.3 Implementazione delle funzioni booleane

Come detto precedentemente è possibile implementare facilmente una funzione booleana se la si formula come una somma di prodotti. Utilizzando la Figura 3.3 come esempio possiamo vedere come si realizza questa implementazione. Nella Figura 3.3(b) gli input *A*, *B* e *C* sono rappresentati sul lato sinistro, mentre l'output della funzione è mostrato a destra. Dato che è necessario disporre dei complementi (inversi) delle variabili di input,

essi sono generati intercettando gli input e facendoli passare attraverso gli invertitori indicati con i numeri 1, 2 e 3. Per rendere la figura più leggibile sono state disegnate sei linee verticali, tre delle quali connesse alle variabili di input e le restanti tre ai loro complementi. Queste linee forniscono un modo pratico per originare gli input delle porte logiche successive. Per esempio le porte 5, 6 e 7 usano *A* come input; in un circuito reale queste porte logiche potrebbero essere collegate direttamente ad *A* senza utilizzare alcun filo "verticale" intermedio.

Il circuito contiene quattro porte AND, una per ciascun termine che compone l'equazione di *M* (cioè una per ogni riga della tabella di verità in cui il bit della colonna risultato vale 1). Ciascuna porta AND calcola una riga della tabella di verità, com'è indicato nella figura. Il risultato finale viene poi calcolato effettuando l'OR fra tutti i prodotti.

Nel circuito mostrato nella Figura 3.3(b) è stata utilizzata una convenzione che verrà impiegata frequentemente nel corso del testo: l'incrocio fra due linee non implica alcuna connessione a meno che non sia presente un pallino nero nel punto di intersezione. Per esempio l'output della porta 3 incrocia tutte e sei le linee verticali, ma è connesso soltanto con $\bar{C}$. Dato che alcuni autori utilizzano convenzioni diverse, occorre prestare attenzione alla notazione impiegata.

Seguendo l'esempio della Figura 3.3 dovrebbe essere chiaro come si possa ottenere un metodo generale per implementare un circuito che realizzi una qualsiasi funzione booleana.

1. Si scrive la tabella di verità della funzione.
2. Ci si munisce di invertitori per generare la negazione di ciascun input.
3. Si utilizza una porta AND per ciascun termine il cui valore nella colonna risultato vale 1.
4. Si collegano le porte AND agli input appropriati.
5. Si connettono tutti gli output delle porte AND nella porta OR.

Anche se abbiamo dimostrato la possibile implementazione di una qualsiasi funzione booleana mediante le porte NOT, AND e OR, spesso è più vantaggioso implementare circuiti utilizzando un solo tipo di porta logica. Fortunatamente esiste un modo diretto per convertire i circuiti generati dall'algoritmo precedente in uno che utilizzi unicamente la porta NAND, oppure la NOR. L'unica cosa che serve per effettuare una simile conversione è un modo per implementare le porte NOT, AND e OR utilizzando un solo tipo di porta logica. La Figura 3.4 mostra l'implementazione di queste tre porte utilizzando soltanto porte NAND (riga in alto), oppure soltanto porte NOR (riga in basso). Questo è il modo più diretto, ma esistono tuttavia anche altri metodi di conversione.

Un metodo per implementare una funzione booleana utilizzando soltanto le porte NAND o NOR consiste nel realizzarla inizialmente mediante le porte NOT, AND e OR seguendo a precedente procedura. Successivamente occorre sostituire le porte logiche con più ingressi con circuiti equivalenti composti da porte con due soli ingressi. $A + B + C + D$ può per esempio essere calcolata come $(A + B) + (C + D)$ utilizzando tre porte logiche a due ingressi. Infine le porte NOT, AND e OR devono essere sostituite dai circuiti della Figura 3.4.

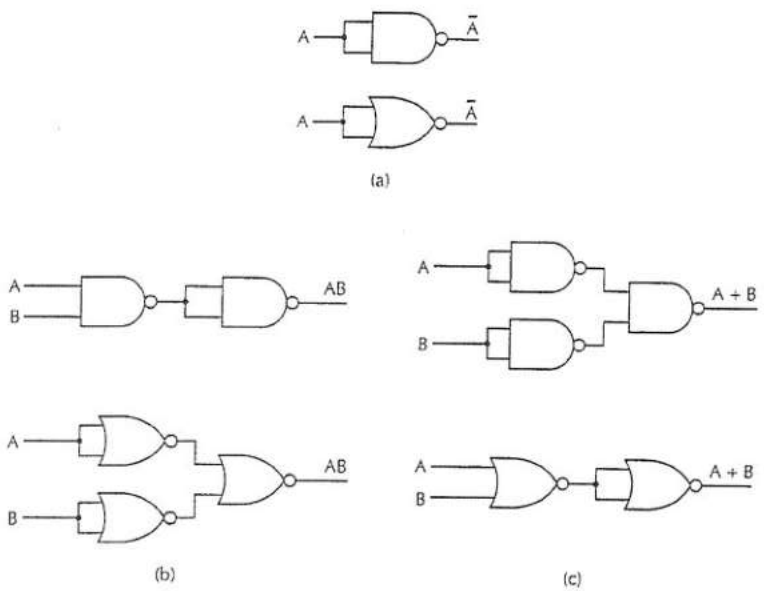


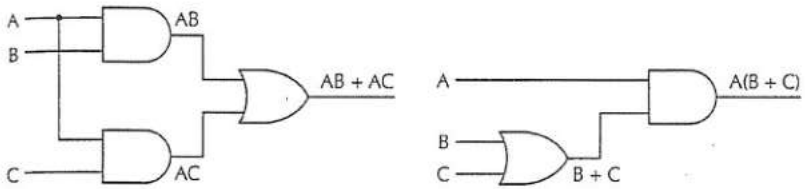
Figura 3.4 Costruzione delle porte logiche (a) NOT, (b) AND e (c) OR utilizzando soltanto porte NOR.

Questa procedura mostra che esiste sempre una possibile implementazione; tuttavia i circuiti ottenuti non sono ottimali, nel senso che non utilizzano il minimo numero possibile di porte logiche. Sia le porte NAND sia le porte NOR costituiscono un insieme di connettivi **funzionalmente completo**, nel senso che una qualsiasi funzione booleana può essere calcolata usando soltanto uno di questi due tipi di porte. Nessun'altra porta logica gode di questa proprietà; questo è un ulteriore motivo per il quale NAND e NOR sono spesso utilizzate come elementi base nella costruzione dei circuiti.

3.1.4 Equivalenza di circuiti

Spesso i progettisti cercano di ridurre il numero di porte logiche utilizzate nei loro prodotti per limitare le dimensioni delle schede con i circuiti stampati, diminuire il consumo energetico e aumentare la velocità. Per ridurre la complessità di un circuito un progettista deve trovare un altro circuito che calcoli la stessa funzione di quello originario, ma che sia composto da un numero inferiore di porte (oppure da porte più semplici, per esempio porte a due ingressi al posto di porte a quattro ingressi).

Come esempio di utilizzo di tecniche dell'algebra booleana per la ricerca di circuiti equivalenti, consideriamo il circuito e la tabella di verità per la funzione $AB + AC$ mostrati nella Figura 3.5(a). Molte delle regole dell'algebra ordinaria valgono anche per l'algebra booleana; in particolare, usando la proprietà distributiva, è possibile fattorizzare $AB + AC$ nella forma $A\ (B + C)$, la cui tabella di verità e il cui circuito sono mostrati nella Figura 3.5(b). Dato che due funzioni sono equivalenti se e solo se hanno lo stesso valore di output per tutti i possibili input, è facile verificare dalle tabelle di verità della Figura 3.5 che $A\ (B + C)$ e $AB + AC$ sono effettivamente equivalenti. Pur essendo equivalenti il circuito della Figura 3.5(b) è chiaramente migliore di quello della Figura 3.5(a), in quanto è composto da un numero minore di porte logiche.



A	B	C	AB	AC	AB + AC
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	0	0
0	1	1	0	0	0
1	0	0	0	0	0
1	0	1	0	1	1
1	1	0	1	0	1
1	1	1	1	1	1

(a)

A	B	C	A	B + C	A(B + C)
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	0	1	0
1	0	0	1	0	0
1	0	1	1	1	1
1	1	0	1	1	1
1	1	1	1	1	1

(b)

Figura 3.5 Due funzioni equivalenti. (a) $AB + AC$. (b) $A\ (B + C)$.

Di solito un progettista di circuiti parte da una formula e in seguito, applicando le leggi dell'algebra di Boole, cerca di semplificarla. Dalla funzione finale può infine essere costruito il circuito.

Per utilizzare questo approccio occorre conoscere alcune identità: le principali sono elencate nella Figura 3.6. È interessante notare che ciascuna legge compare in due forme, **duali** l'una rispetto all'altra. Scambiando AND con OR e anche 1 con 0 è possibile creare una delle due forme a partire dall'altra. È possibile verificare tutte le leggi costruendo le loro tabelle di verità; i risultati sono ragionevolmente intuitivi per tutte le leggi tranne che per la legge di De Morgan, la proprietà di assorbimento e la legge distributiva della somma rispetto al prodotto. La legge di De Morgan può inoltre essere estesa a più di due variabili, per esempio $\overline{ABC} = \overline{A} + \overline{B} + \overline{C}$.

La legge di De Morgan suggerisce una notazione alternativa. La Figura 3.7(a) mostra la forma AND in cui la negazione è indicata, sia per gli input sia per gli output, median-

te i cerchietti di inversione. Una porta logica OR con gli input invertiti è quindi equivalente a una porta NAND. Dalla Figura 3.7(b), che rappresenta la forma duale della legge di De Morgan, dovrebbe essere chiaro che una porta NOR può essere disegnata come una porta AND con gli input invertiti. Negando entrambe le forme della legge di De Morgan si ottengono le Figure 3.7(c) e (d), che mostrano due rappresentazioni equivalenti delle porte logiche AND e OR. Esistono simboli analoghi per le forme a più variabili della legge di De Morgan (per esempio, una porta NAND a n input diventa una porta OR con n input invertiti).

Nome	Forma AND	Forma OR
Elemento neutro	$1A = A$	$0 + A = A$
Assorbimento	$0A = 0$	$1 + A = 1$
Idempotenza	$AA = A$	$A + A = A$
Complementazione	$A\overline{A} = 0$	$A + \overline{A} = 1$
Proprietà commutativa	$AB = BA$	$A + B = B + A$
Proprietà associativa	$(AB)C = A(BC)$	$(A + B) + C = A + (B + C)$
Proprietà distributiva	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Legge di assorbimento	$A(A + B) = A$	$A + AB = A$
Legge di De Morgan	$\overline{AB} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A}\,\overline{B}$

Figura 3.6 Identità dell'algebra booleana.

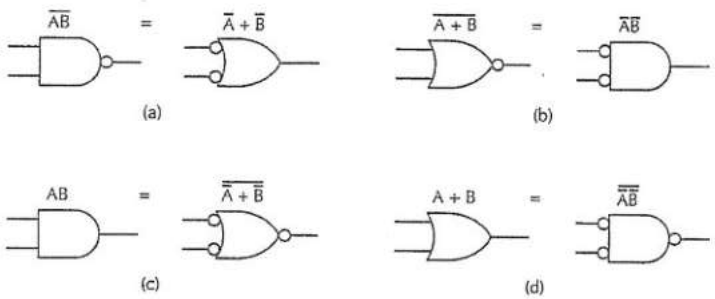


Figura 3.7 Simboli equivalenti per alcune porte logiche: (a) NAND. (b) NOR. (c) AND. (d) OR.

La conversione della tabella di verità dalla rappresentazione somma-di-prodotti alla forma pura NAND o NOR risulta agevolata dalle identità della Figura 3.7 e dalle corrispondenti versioni per le porte a più ingressi. Come esempio consideriamo la funzione XOR OR esclusivo) della Figura 3.8(a), il cui circuito nella forma standard somma-di-prodotti è rappresentato nella Figura 3.8(b). Per convertirlo nella forma NAND bisognerebbe ridisegnare le linee che collegano l'output delle porte AND all'input della porta OR utilizzando due cerchietti d'inversione, come illustra la Figura 3.8(c); infine, utilizzando

l'equivalenza mostrata nella Figura 3.7(a) si ottiene il circuito della Figura 3.8(d). Le variabili $\overline{A}$ e $\overline{B}$ possono essere generate da A e B utilizzando le porte NAND o NOR con i loro input legati insieme. Si noti che i cerchietti d'inversione possono essere spostati a piacere lungo una linea, muovendoli per esempio dagli output delle porte di input della Figura 3.8(d) agli input della porta di output.

Come ultima osservazione sull'equivalenza di circuiti dimostreremo che, in modo sorprendente, una stessa porta logica reale può calcolare diverse funzioni a seconda delle convenzioni adottate. La Figura 3.9(a) mostra l'output di una certa porta, F, al variare della combinazione degli input (input e output sono indicati in volt). Se adottiamo la convenzione, chiamata **logica positiva**, secondo la quale 0 volt è il valore logico 0 e 1,5 volt è il valore logico 1, otteniamo la tabella di verità della Figura 3.9(b), corrispondente alla funzione AND. Se al contrario adottiamo la **logica negativa**, per la quale 0 volt rappresenta il valore logico 1 e 1,5 volt il valore logico 0, otteniamo la tabella di verità mostrata nella Figura 3.9(c), corrispondente alla funzione OR.

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

(a)

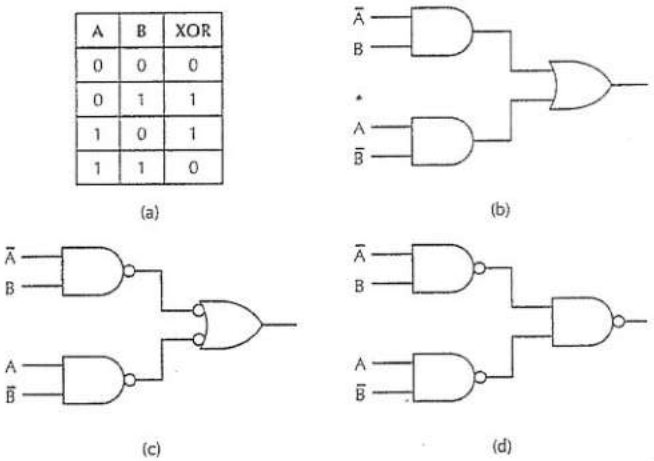


Figura 3.8 (a) Tabella di verità della funzione XOR. (b)-(d) Tre circuiti per calcolarla.

A	B	F
0	0	0
0	5	0
5	0	0
5	5	5

(a)

A	B	F
0	0	0
0	1	0
1	0	0
1	1	1

(b)

A	B	F
1	1	1
1	0	1
0	1	1
0	0	0

(c)

Figura 3.9 (a) Caratteristiche elettriche di un dispositivo. (b) Logica positiva. (c) Logica negativa.

La convenzione scelta per assegnare le tensioni ai valori logici riveste quindi un'importanza cruciale. Se non specificato diversamente, d'ora in poi utilizzeremo la logica positiva; i termini valore logico 1, vero e alto saranno quindi fra loro sinonimi, così come lo saranno i termini valore logico 0, falso e basso.

3.2 Circuiti logici digitali elementari

Nei paragrafi precedenti abbiamo visto come implementare tabelle di verità e altri semplici circuiti utilizzando singole porte logiche. Anche se una volta era comune farlo, attualmente solo pochi circuiti sono realmente costruiti "porta per porta". Al giorno d'oggi nella costruzione di circuiti i componenti elementari sono generalmente rappresentati da moduli contenenti un certo numero di porte. Nei paragrafi successivi analizzeremo più da vicino questi blocchi elementari e vedremo come vengono utilizzati e come sono costruiti a partire da singole porte logiche.

3.2.1 Circuiti integrati

Le porte logiche non sono prodotte o vendute individualmente, ma in unità chiamate **circuiti integrati**, alle quali spesso ci si riferisce con i termini **IC** o **chip**.

Un circuito integrato è un pezzo rettangolare di silicio di varie dimensioni a seconda di quante porte sono necessarie per implementare i componenti del chip. I circuiti piccoli misurano circa 2 mm × 2 mm, mentre i grandi stampi possono arrivare a 18 mm × 18 mm. I circuiti integrati sono montati in supporti di plastica o di ceramica che possono essere molto più grandi dei circuiti ospitati nei casi in cui siano necessari molti piedini (o pin "contatto") per collegare il chip al mondo esterno. Ogni piedino si collega all'ingresso o all'uscita di una porta del chip, oppure alla potenza o alla terra.

La Figura 3.10 mostra alcuni comuni supporti per circuiti integrati utilizzati per i chip attuali. I chip più piccoli, come per esempio quelli utilizzati per microcontrollori domestici o per le RAM, utilizzano un supporto di tipo **DIP** (*Dual Inline Package*, "contenitore a due file di piedini"). Il DIP è un supporto con due file di pin che viene posizionato in un alloggiamento (detto anche *socket*, "zoccolo, attacco o presa") corrispondente sulla scheda madre. I supporti DIP più comuni hanno 14, 16, 18, 20, 22, 24, 28, 40, 64 o 68 pin. Per i grandi circuiti integrati sono spesso utilizzati supporti quadrati con perni sui quattro lati o sul fondo. Due comuni supporti per i circuiti integrati più grandi sono **PGA** (*Pin Grid Arrays*) e **LGA** (*Land Grid Arrays*). I PGA hanno i pin sul fondo del supporto che si inseriscono in un alloggiamento corrispondente sulla scheda madre. I socket PGA utilizzano spesso un meccanismo che permette l'inserimento del circuito senza sforzo; viene in seguito azionata una levetta per applicare una pressione laterale a tutti i pin del PGA, in modo che il circuito sia tenuto saldamente ancorato al suo alloggiamento. I supporti LGA hanno invece piccoli pad piatti sul fondo del chip e il socket LGA ha una copertura che si inserisce sul supporto applicando una forza rivolta verso il basso, assicurando così che tutti i pad LGA siano in contatto con i pad del suo alloggiamento.

Poiché molti supporti per circuiti integrati hanno una forma simmetrica, può essere un problema capire l'orientamento per una corretta installazione del chip. I DIP hanno tipicamente una marcatura in un angolo che va fatta coincidere con un segno corrispondente sul socket DIP. I PGA hanno in genere un pin mancante. Se si tenta quindi di inserire il PGA nel suo alloggiamento in modo non corretto, il PGA non entra. Poiché i chip LGA non hanno piedini, per la corretta installazione si applica una tacca su uno o due lati del chip, che corrisponde a una tacca sul socket LGA. Il chip LGA non entrerà nel suo alloggiamento se le due tacche non corrispondono.

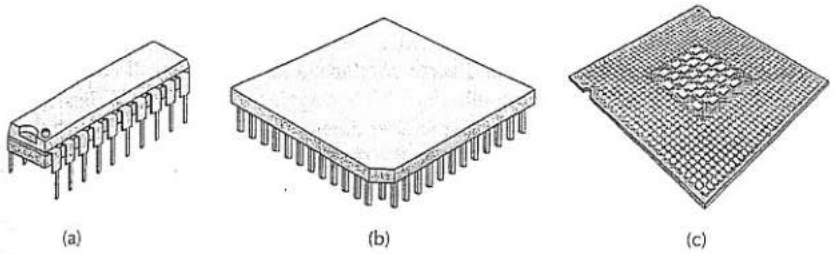


Figura 3.10 Tipologie di supporti per circuiti integrati: DIP (a), PGA (b), LGA (c).

Per i nostri scopi tutte le porte logiche considerate sono ideali, nel senso che l'output appare non appena viene applicato il voltaggio in input. In realtà i chip hanno un ritardo temporale finito, chiamato **ritardo della porta**, che comprende sia il tempo dovuto alla propagazione del segnale attraverso il chip, sia il tempo di commutazione. Generalmente questi ritardi sono compresi tra centinaia di picosecondi e pochi nanosecondi.

Lo stato dell'arte attuale della tecnologia permette di inserire più di un miliardo di transistor all'interno di un chip. Dato che ogni circuito può essere costruito mediante porte NAND, si potrebbe pensare che un produttore potrebbe realizzare un chip estremamente generalizzato contenente 500 milioni di porte NAND. Sfortunatamente però un chip di questo tipo richiederebbe 1.500.000.000 pin; considerando che la distanza standard tra i pin è di 1 mm, un chip LGA dovrebbe avere un lato di 38 m per far posto a tutti i pin, il che potrebbe porre dei limiti alla sua commercializzazione. È chiaro che l'unico modo per trarre vantaggio dallo stato attuale della tecnologia consiste nel progettare circuiti con un elevato rapporto porte/pin. Nei paragrafi successivi esamineremo alcuni semplici MSI che, per implementare una determinata funzione, combinano un buon numero di porte interne pur richiedendo un limitato numero di connessioni esterne.

3.2.2 Reti combinatorie

Molte applicazioni della logica digitale richiedono un circuito, chiamato **rete combinatoria**, con più input e più output, in cui gli output sono unicamente determinati dagli input. Non tutti i circuiti hanno questa proprietà; un circuito contenente elementi di memoria può per esempio generare valori che dipendono non solo dalle variabili d'ingresso, ma anche dai valori memorizzati. Un circuito che implementa una tabella di verità, come quella della Figura 3.3(a), è un tipico esempio di rete combinatoria. In questo paragrafo esamineremo le reti combinatorie utilizzate più di frequente.

Multiplexer

In logica digitale un ***multiplexer*** è un circuito con 2^n dati di input, un valore di output e n input di controllo; gli input di controllo permettono di selezionare uno dei dati di input, che viene "instradato" verso l'output. La Figura 3.11 mostra un diagramma di un multiplexer con otto input. Le tre linee di controllo, A, B e C, codificano un numero a 3 bit che specifica quale delle otto linee di input deve essere instradata verso la porta OR e quindi verso l'output. Indipendentemente dal valore definito dalle linee di controllo, sette delle porte AND genereranno sempre il valore 0, mentre quella rimanente produrrà in output 0 oppure 1, a seconda del valore della linea d'ingresso selezionata. Ciascuna porta AND può essere abilitata da una diversa combinazione degli input di controllo. Il circuito del multiplexer è mostrato nella Figura 3.11; aggiungendo anche la tensione e la terra può essere confezionato in un contenitore dotato di 14 pin.

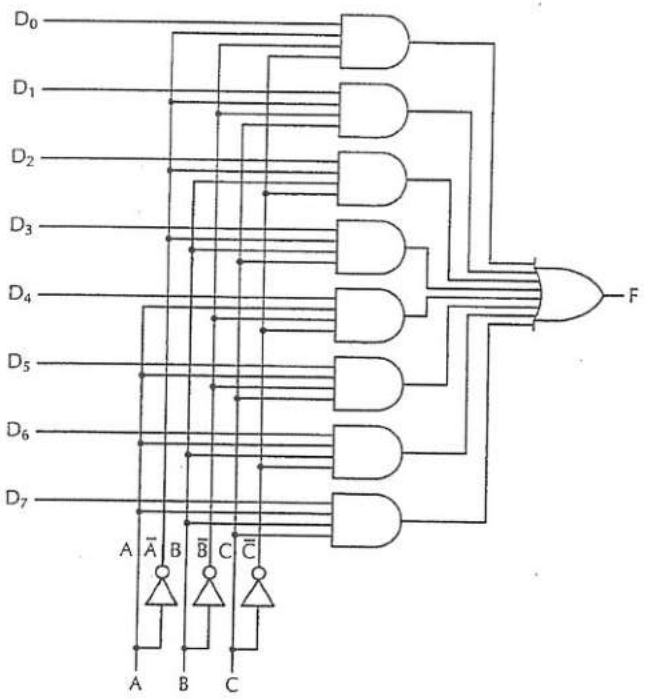


Figura 3.11 Multiplexer a otto vie.

La Figura 3.12(b) mostra come possiamo utilizzare il multiplexer per implementare la funzione di maggioranza della Figura 3.3(a). In corrispondenza di ciascuna combinazione di A, B e C viene selezionata una delle linee dei dati di input e ciascuna di loro è

collegata a V_{cc} (valore logico 1) oppure alla terra (valore logico 0). L'algoritmo che permette di collegare gli input in modo corretto è semplice: l'input D_i è lo stesso valore presente nella riga i della tabella di verità. Nella Figura 3.3(a) le righe 0, 1, 2 e 4 valgono 0 e quindi gli input corrispondenti sono collegati alla terra; le righe rimanenti valgono invece 1 e sono quindi collegate al valore logico 1. Seguendo questa strategia è possibile implementare una qualsiasi tabella di verità mediante il chip della Figura 3.12(a).

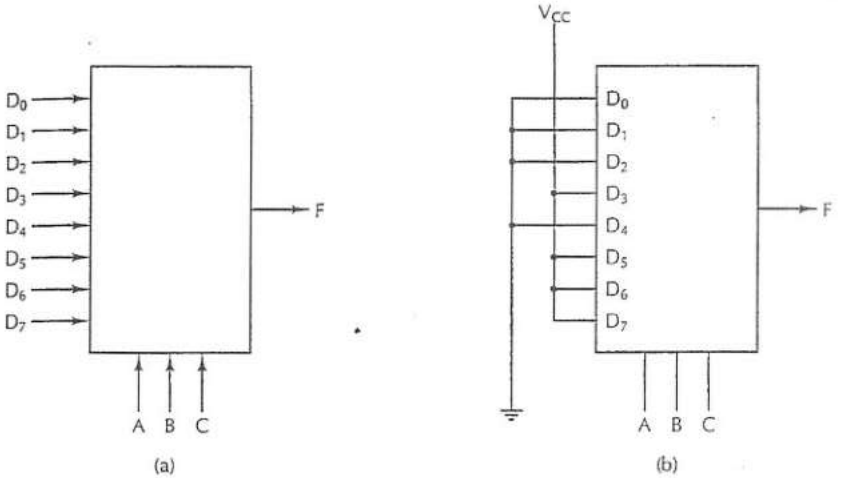


Figura 3.12 (a) Multiplexer MSI. (b) Lo stesso multiplexer per calcolare la funzione di maggioranza.

Finora abbiamo visto come utilizzare un multiplexer per selezionare un input fra alcuni valori d'ingresso e per implementare una tabella di verità. Un'altra delle possibili applicazioni è la conversione di dati da parallelo a seriale. Se si immettono 8 bit di dati nelle linee di input e se si fa variare il valore (binario) delle linee di controllo da 000 a 111, si ottengono sulla linea di output gli 8 bit in serie. Un utilizzo molto comune della conversione da parallelo a seriale lo si riscontra nelle tastiere: la pressione di ogni tasto definisce implicitamente un numero a 7 o 8 bit che deve essere inviato su un collegamento seriale, per esempio USB.

L'inverso del multiplexer è il **demultiplexer**, che redirige il suo segnale di input verso uno dei 2^n output in base ai valori delle linee di controllo; se il valore binario definito dalle linee di controllo è k, viene selezionato l'output k.

Decodificatori

Come secondo esempio analizzeremo ora un circuito, chiamato **decodificatore** (*decoder*), che accetta come input un numero a n bit e lo utilizza per impostare a 1 una sola delle 2^n linee di output. La Figura 3.13 mostra questo circuito nel caso $n = 3$.

Per capire in quali situazioni può essere utile questo circuito immaginiamo una piccola memoria di otto chip, da 256 MB ciascuno. Gli indirizzi del chip 0 variano da 0 a 256 MB, quelli del chip 1 da 256 MB a 512 MB e così via. Quando si fornisce alla memoria un indirizzo, si utilizzano i suoi 3 bit più significativi per selezionare uno degli otto chip. In riferimento al circuito della Figura 3.13 questi 3 bit corrispondono ai tre input, A, B e C; a seconda del loro valore solo una delle linee di output, assume il valore 1, mentre le altre rimangono a 0. Ciascuna linea di output permette poi di abilitare uno degli otto chip della memoria; dato che solo una linea di output viene impostata al valore 1, solo uno dei chip viene abilitato.

Il funzionamento del circuito della Figura 3.13 è semplice. Ciascuna porta AND ha tre input, il primo dei quali è A o $\overline{A}$, il secondo B o $\overline{B}$ e il terzo C o $\overline{C}$, e viene abilitata da una diversa combinazione dei valori di input: D_0 da $\overline{A}\,\overline{B}\,\overline{C}$, D_1 da $\overline{A}\,\overline{B}\,C$ e così via.

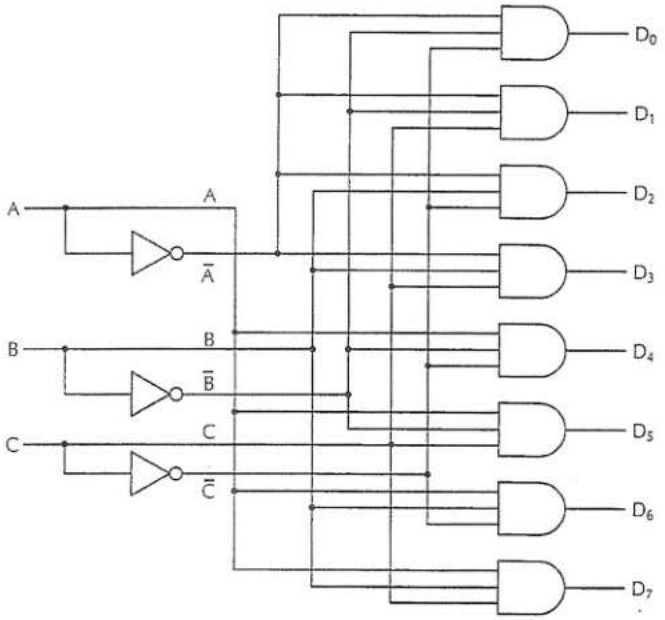


Figura 3.13 Decodificatore da 3 a 8.

Comparatori

Un altro circuito particolarmente utile è il **comparatore**, che permette di confrontare due stringhe di bit. Il semplice comparatore mostrato nella Figura 3.14 accetta due input, A e B, ciascuno lungo 4 bit, e genera 1 se sono uguali, mentre 0 se sono diversi. Il circuito è basato sulla porta logica XOR (EXCLUSIVE OR), che produce in output un valore 0 se i suoi input sono uguali e un valore 1 se sono diversi. Se due stringhe in ingresso sono uguali, tutte e quattro le porte XOR devono generare come risultato 0. Questi quattro segnali possono poi essere connessi a una stessa porta logica OR in modo da produrre un valore 0 quando gli input sono uguali e un valore 1 nel caso contrario. Nel nostro esempio abbiamo utilizzato una porta NOR nell'ultimo stadio del circuito in modo da invertire il risultato del test: 1 significa uguale, mentre 0 significa diverso.

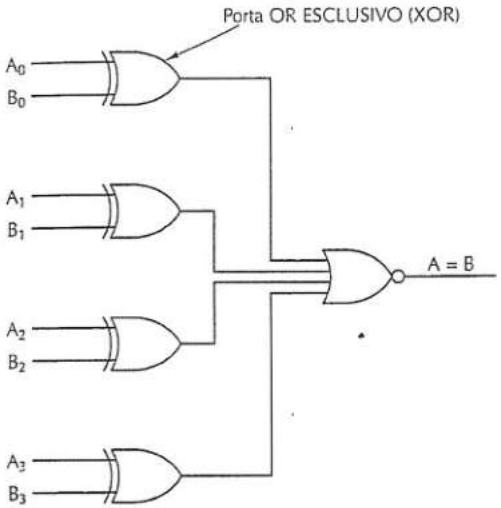


Figura 3.14 Semplice comparatore a 4 bit.

Array logici programmabili

Un chip molto generale che permette di calcolare somme di prodotti è l'**array logico programmabile** o **PLA** (*Programmable Logic Array*), di cui la Figura 3.15 mostra un semplice esempio. Questo chip ha 12 ingressi e al suo interno questi valori vengono invertiti; quindi il numero totale di segnali di input diventa 24. Il cuore del circuito è costituito da una schiera di 50 porte AND che possono avere come input un qualsiasi sottoinsieme dei 24 segnali di input. Una matrice di 24 × 50 bit fornita dall'utente determina le connessioni desiderate tra i segnali di input e le 50 porte AND. Ogni linea di input connessa alle 50 porte AND contiene un fusibile. Al momento della fabbricazione del chip i 1200 fusibili sono intatti; successivamente l'utente può programmare la matrice bruciando i fusibili con l'applicazione di un'alta tensione.

L'uscita del circuito consiste in 6 porte OR, che possono avere fino a 50 input, corrispondenti agli output delle porte AND. Anche in questo caso l'utente deve fornire una matrice (50 × 6) per specificare quali connessioni instaurare. Il chip ha 20 pin in tutto, 12 per gli input, 6 per gli output, e 2 per la tensione e la terra.

Come esempio di utilizzo di un circuito PLA consideriamo nuovamente il circuito mostrato nella Figura 3.3(b), che ha tre input, quattro porte AND, una porta OR e tre invertitori. Il nostro PLA, dopo aver impostato inizialmente le connessioni in modo appropriato, può calcolare la stessa funzione usando tre dei suoi 12 input, quattro delle 50 porte AND e una delle sue 6 porte OR. Le quattro porte AND dovrebbero calcolare rispettivamente ABC, ABC, ABC e ABC, e la porta OR dovrebbe accettare in input questi quattro prodotti. In realtà lo stesso PLA potrebbe essere utilizzato per calcolare quattro funzioni di analoga complessità.

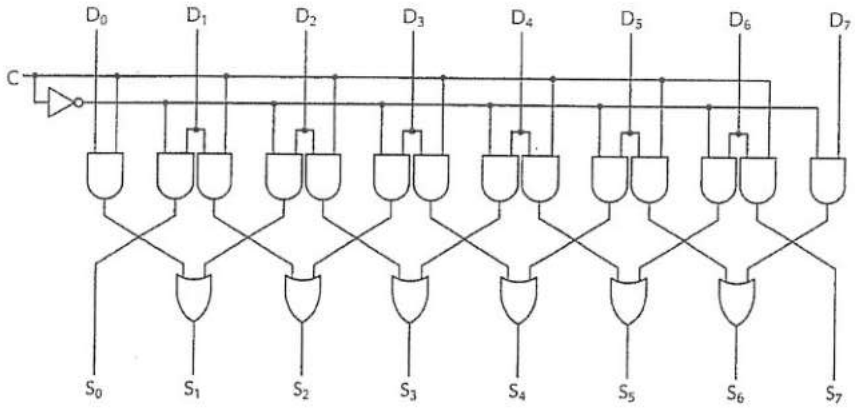


Figura 3.15 Array logico programmabile a 12 input e 6 output. I quadratini rappresentano i fusibili che possono essere bruciati per determinare la funzione da calcolare. I fusibili sono disposti in due matrici: quella superiore per le porte AND, quella inferiore per le porte OR.

Nel caso di funzioni semplici il numero di variabili rappresenta un fattore vincolante, mentre per funzioni più complesse il limite potrebbe essere rappresentato dal numero di porte AND o OR.

Anche se i PLA *programmabili sul campo*, come quello appena descritto, vengono ancora utilizzati, oggi, in molte applicazioni, si preferisce impiegare dei PLA costruiti ad hoc. Questi ultimi sono più economici di quelli programmabili e, in caso di grandi volumi, vengono progettati sulla base delle specifiche del cliente.

Alla luce della presentazione dei tre modi diversi per implementare la tabella di verità della Figura 3.3(a) possiamo passare a un confronto. Utilizzando componenti SSI sono necessari 4 chip; una soluzione alternativa, mostrata nella Figura 3.12(b), prevede invece di utilizzare un unico multiplexer di tipo MSI. Infine l'ultima possibilità consiste nell'usare un quarto di un chip PLA. Se occorre implementare più funzioni il chip PLA è ovviamente più efficiente rispetto agli altri due metodi; nel caso invece di circuiti semplici, i chip SSI e MSI sono preferibili in quanto più economici.

3.2.3 Circuiti per l'aritmetica

È arrivato ora il momento di passare dai circuiti generici alle reti combinatorie. Ricordiamo che le reti combinatorie hanno uscite che sono funzioni dei loro ingressi, ma i circuiti utilizzati per fare calcoli aritmetici non hanno questa proprietà. Inizieremo con un semplice registro a scorrimento (*shifter*) a 8 bit, continueremo guardando com'è costruito un sommatore e infine esamineremo le unità aritmetico-logiche che svolgono un ruolo centrale all'interno di tutti i calcolatori.

Registri a scorrimento

Il primo circuito aritmetico MSI che analizziamo ha otto input e otto output (vedi la Figura 3.16). Gli input sono collegati alle linee D_0, ..., D_7, mentre l'output, corrispondente all'input traslato di un bit, risulta disponibile sulle linee S_0, ..., S_7. La linea di controllo, C, ha valore 0 se lo spostamento deve avvenire verso sinistra, e 1 in caso contrario. Nel caso di uno spostamento a sinistra si inserisce uno 0 nel bit 7, e nel caso di uno shift a destra si inserisce uno 0 nel bit 0.

A	B	Somma	Riporto
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

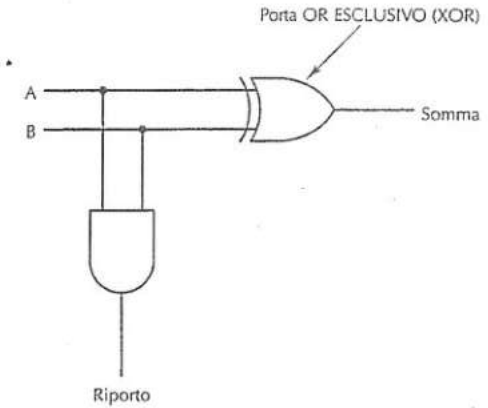


Figura 3.16 Registro a scorrimento a sinistra di 1 bit.

Per comprendere il funzionamento del circuito si noti che per tutti i bit c'è una coppia di porte AND, fatta eccezione per le porte che si trovano alle estremità. Quando $C = 1$ la porta che si trova a destra in ciascuna coppia viene abilitata, lasciando passare il bit corrispondente verso l'output. Dato che la porta AND è collegata all'input della porta OR alla sua destra, si ottiene uno scorrimento verso destra. Quando $C = 0$ è la porta AND di sinistra in ciascuna coppia a essere abilitata, producendo uno spostamento verso sinistra.

Sommatori

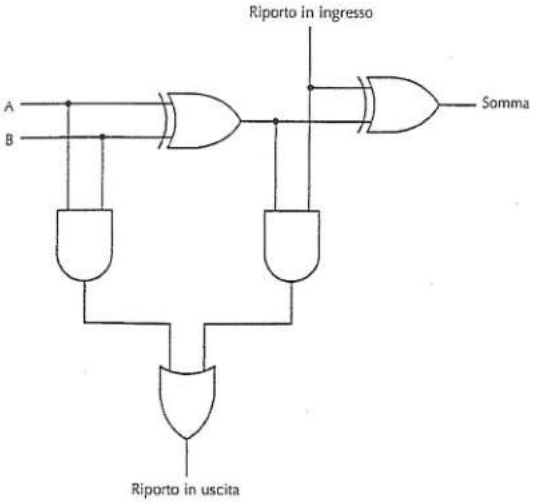
È quasi impossibile immaginare un calcolatore che non sia in grado di eseguire le somme. Per questo motivo un circuito che effettui l'addizione è una parte di essenziale

importanza per qualsiasi CPU. La Figura 3.17(a) mostra la tabella di verità per la somma di interi a 1 bit, in cui sono presenti due output: la somma dei due input, *A* e *B*, e il riporto da sommare alla successiva posizione (a sinistra). La Figura 3.17(b) rappresenta un circuito, chiamato **half adder** (cioè *semisommatore*, non tenendo conto del riporto in ingresso), capace di calcolare il bit della somma e il bit del riporto.

Questo circuito è in grado di sommare correttamente i bit meno significativi di due stringhe binarie, ma non è in grado di eseguire correttamente la somma degli altri bit, dato che non riesce a gestire il riporto che arriva dalle posizioni precedenti. Per far ciò è necessario un **sommatore** come quello mostrato nella Figura 3.18. Osservando attentamente il circuito dovrebbe apparire chiaro che un sommatore è costruito a partire da due semisommatori. La linea di output *Sum* vale 1 se una o tre delle linee *A*, *B* e *Carry in* valgono 1. *Carry out* vale 1 se sia *A* sia *B* valgono 1 (input di sinistra della porta OR) oppure se uno dei due vale 1 e anche *Carry in* vale 1. L'unione dei due semisommatori permette di generare in output sia il bit della somma sia il bit del riporto.

A	B	Riporto in ingresso	Somma	Riporto in uscita
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

(a)



(b)

Figura 3.17 (a) Tabella di verità di un sommatore. (b) Circuito di un sommatore.

Per esempio, per generare un sommatore di due parole a 16 bit occorre replicare il circuito della Figura 3.18(b) per 16 volte. Per un dato bit il riporto in uscita viene utilizzato come riporto in entrata per il bit alla sua sinistra. Il riporto in entrata del bit all'estremità destra è collegato al valore 0. Questo tipo di sommatore è chiamato **sommatore a propagazione di riporto**, dato che nel caso peggiore, sommando 1 a 111...111 (binario), la somma non può essere completata finché il riporto non si sia propagato lungo tutta la parola, dal bit all'estremità destra fino a quello all'estremità sinistra. Esistono anche altri tipi di sommatori che non hanno questo ritardo e risultano dunque più veloci; per questo motivo vengono generalmente preferiti rispetto a quelli a propagazione di riporto.

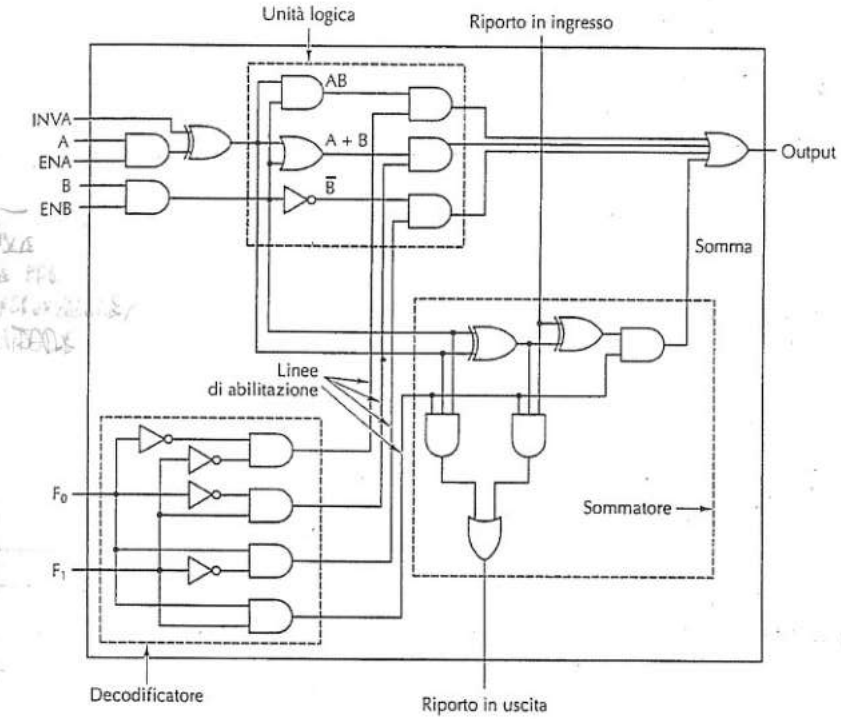


Figura 3.18 ALU a 1 bit.

Per fare un esempio di sommatore più veloce immaginiamo di dividere un sommatore a 32 bit in uno per i 16 bit meno significativi e in un altro per i 16 bit più significativi. Nel momento in cui comincia l'addizione quest'ultimo non può ancora iniziare a lavorare poiché è obbligato ad attendere il valore del riporto per un tempo pari a 16 somme di singoli bit. Consideriamo però la seguente modifica. Invece di avere un singolo sommatore per i bit più significativi ne creiamo due identici in parallelo duplicando l'hardware corrispondente. Il circuito così modificato consiste in tre sommatori a 16 bit: uno per i bit meno significativi e due per quelli più significativi, *U0* e *U1*, funzionanti in parallelo. Nel sommatore *U0* viene collegato il valore 0 come riporto, mentre nel sommatore

U1 viene collegato il valore 1. Tutti e tre i sommatori possono ora cominciare nello stesso momento, anche se soltanto uno dei due sommatori *U0* e *U1* sarà corretto. Dopo aver atteso il completamento delle prime 16 somme di singoli bit si conoscerà il valore del riporto da aggiungere al sommatore dei 16 bit più significativi. Grazie a questo valore è possibile selezionare il sommatore corretto fra *U0* e *U1*. Questo trucco permette di dimezzare il tempo richiesto dall'addizione; un simile sommatore viene chiamato a **selezione del riporto**. Lo stesso trucco può essere ripetuto per costruire i sommatori a 16 bit come più sommatori a 8 bit e così via.

Unità aritmetico logiche

La maggior parte dei calcolatori contiene un unico circuito in grado di effettuare le operazioni AND, OR e somma di due parole. Di solito un circuito di questo tipo funzionante su parole a n bit è costruito a partire da n identici circuiti per le singole posizioni dei bit. La Figura 3.19 è un esempio di tale circuito, chiamato **unità aritmetico logica** o **ALU** (*Arithmetic Logic Unit*); esso può calcolare una qualsiasi delle quattro funzioni, A AND B, A OR B, $\overline{B}$ oppure $A + B$, in base al valore binario (00, 01, 10 oppure 11) definito dalle linee F_0 e F_1 preposte alla selezione della funzione aritmetica. Si noti che in questo caso con $A + B$ si intende la somma aritmetica di A e B e non l'operazione booleana OR.

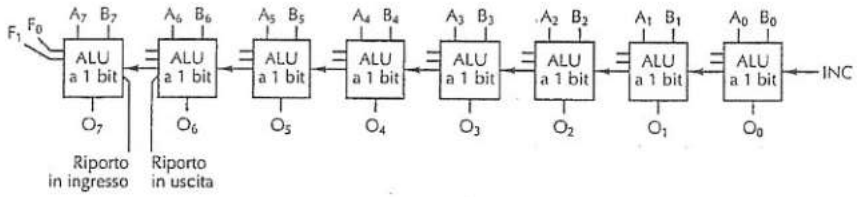


Figura 3.19 Otto ALU a 1 bit connesse per comporre una ALU a 8 bit. Per semplificare non sono mostrati segnali di abilitazione e segnali di inversione.

Il settore in basso a sinistra della nostra ALU contiene un decodificatore a 2 bit che permette di generare, in base ai segnali di controllo F_0 ed F_1, i segnali di attivazione delle quattro operazioni. In base ai valori di F_0 e F_1 viene selezionata una sola delle quattro linee di attivazione, permettendo all'output della funzione selezionata di passare attraverso la porta logica OR che genera l'output finale.

Il settore in alto a sinistra contiene le porte logiche necessarie per calcolare A AND B, A OR B e $\overline{B}$; in base alle linee di attivazione che escono dal decodificatore uno solo di questi risultati viene passato alla porta logica finale OR. Dato che solo uno degli output del decodificatore varrà 1, soltanto una delle quattro porte AND che forniscono i valori d'ingresso alla porta OR verrà attivata; l'output delle altre sarà invece 0, indipendentemente dai valori di A e B. Oltre a poter usare A e B come input per le operazioni logiche o aritmetiche, è anche possibile forzare uno dei due valori a 0 negando rispettivamente ENA oppure ENB. È inoltre possibile ottenere il valore di $\overline{A}$ abilitando il segnale INVA. Nel

Capitolo 4 vedremo alcuni utilizzi di INVA, ENA e ENB. In condizioni normali entrambi gli input sono abilitati; ENA e ENB valgono quindi 1, mentre INVA è impostato a 0. In questa situazione A e B sono immessi all'interno dell'unità logica senza alcuna modifica.

Il settore in basso a destra della ALU contiene un sommatore per calcolare la somma di A e B, e gestire allo stesso tempo i riporti; la gestione dei riporti è necessaria dato che con ogni probabilità vari circuiti dello stesso tipo potrebbero essere collegati fra loro per permettere operazioni su intere parole. Circuiti come quello mostrato nella Figura 3.19 sono attualmente disponibili e vengono chiamati **bit slices** ("suddivisioni di un bit"), che permettono ai progettisti di calcolatori di costruire ALU di larghezza arbitraria. La Figura 3.20 mostra una ALU a 8 bit costruita unendo otto ALU a 1 bit. Il segnale INC è utilizzato solo per le operazioni di addizione; quando è attivo permette di incrementare il risultato (cioè sommargli 1), rendendo così possibile il calcolo di somme come $A + 1$ e $A + B + 1$.

Anni fa, un bit slice era un vero e proprio chip che si poteva comprare. Oggi un bit slice è una libreria che un progettista può replicare in un programma CAD per produrre un file per le macchine di produzione di chip. L'idea resta comunque essenzialmente la stessa.

3.2.4 Clock

In molti circuiti digitali l'ordine secondo il quale si verificano gli eventi è cruciale. In alcuni casi un evento deve precederne un altro, mentre in altre situazioni due eventi devono avvenire simultaneamente. Molti circuiti digitali utilizzano dei clock per gestire la sincronizzazione e permettere ai progettisti di ottenere le relazioni temporali desiderate. In questo contesto un **clock** è un circuito che emette una serie di impulsi di larghezza definita e a intervalli temporali costanti. L'intervallo temporale compreso tra le estremità di due impulsi consecutivi è detto **ciclo di clock**. La frequenza degli impulsi è generalmente compresa tra 100 MHz e 4 GHz, corrispondenti a cicli di clock compresi tra 10 ns e 250 ps. Per ottenere un'accuratezza elevata di solito la frequenza di clock è controllata da un oscillatore a cristalli.

In un calcolatore possono verificarsi più eventi durante uno stesso ciclo di clock. Se però è necessario che si verifichino in uno specifico ordine occorre dividere il ciclo di clock in sottocicli. Una tecnica che viene spesso utilizzata per ottenere una risoluzione più fine rispetto a quella del clock consiste nell'intercettare la linea del clock principale e inserirla in un circuito di cui si conosce il ritardo; in questo modo è possibile generare un secondo segnale di clock la cui fase è traslata rispetto a quella del clock principale, come mostra la Figura 3.20(a). Il diagramma di temporizzazione della Figura 3.20(b) fornisce quattro diversi riferimenti temporali utilizzabili per sincronizzare eventi discreti.

1. Fronte di salita di C1.
2. Fronte di discesa di C1.
3. Fronte di salita di C2.
4. Fronte di discesa di C2.

Associando diversi eventi ai quattro fronti è possibile stabilire per loro una desiderata sequenza. Se, all'interno di ogni ciclo di clock, sono necessari più di quattro riferimen-

ti temporali occorre collegare al clock principale altre linee secondarie e utilizzare circuiti con ritardi diversi.

In alcuni circuiti si è interessati maggiormente agli intervalli temporali piuttosto che agli istanti di tempo. A un evento potrebbe per esempio essere consentito di verificarsi in qualsiasi istante durante il quale C1 è alto e non per forza esattamente in corrispondenza del fronte di salita. Un altro evento potrebbe verificarsi solo quando C2 è alto. Se sono necessari più di due diversi intervalli temporali si possono utilizzare più linee di clock oppure si può fare in modo che gli intervalli in cui i segnali dei due clock sono alti si sovrappongano parzialmente. In quest'ultimo caso si possono distinguere quattro diversi intervalli: $\overline{C1}$ AND $\overline{C2}$, $\overline{C1}$ AND C2, C1 AND $\overline{C2}$ e C1 AND C2.

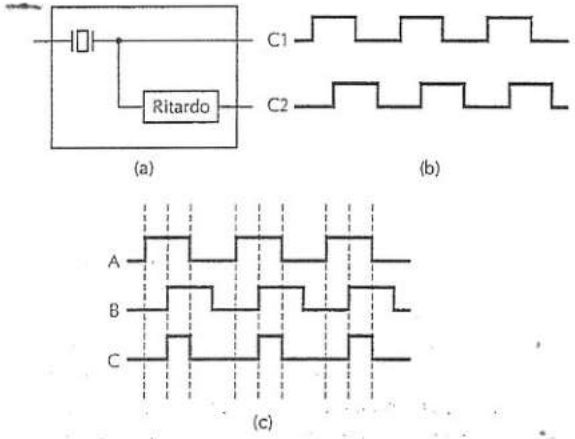


Figura 3.20 (a) Clock. (b) Diagramma di temporizzazione del clock. (c) Generazione di un clock asimmetrico.

Per essere più precisi i clock sono simmetrici nel senso che il tempo speso nello stato in cui il segnale è alto è uguale al tempo speso quando il segnale è basso, come mostra la Figura 3.20(b). Per generare una sequenza di impulsi asimmetrici il clock principale viene sfasato utilizzando un circuito ritardante e ne viene calcolato l'AND logico rispetto al segnale originale, come mostra il segnale C della Figura 3.20(c).

3.3 Memoria

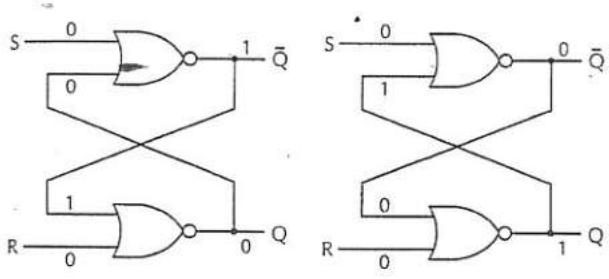
Una componente essenziale di ogni calcolatore è la memoria; se non ci fosse non potrebbe esistere nessun calcolatore, almeno nella forma in cui lo conosciamo. La memoria è utilizzata per conservare sia le istruzioni da eseguire sia i dati. Nei paragrafi successivi esamineremo i componenti base di un sistema di memoria; partiremo dal livello delle porte logiche per comprenderne il funzionamento e il modo in cui sono combinate per formare memorie più capaci.

3.3.1 Latch

Per creare una memoria a 1 bit è necessario disporre di un circuito che in qualche modo "ricordi" i precedenti valori di input. La Figura 3.21(a) mostra come sia possibile costruire un circuito di questo tipo utilizzando due porte NOR. Anche se è possibile costruire circuiti analoghi con porte NAND, nel seguito non li menzioneremo più, dato che dal punto di vista concettuale sono identici alle versioni basate su porte NOR.

Il circuito della Figura 3.21(a) è chiamato **latch SR** e ha due input: *S*, per impostare (*Setting*) il valore del latch e *R* per azzerarlo (*Resetting*). Il circuito ha anche due output, *Q* e $\overline{Q}$, che, come vedremo tra poco, sono complementari l'uno rispetto all'altro. Diversamente dalle reti combinatorie l'output di un latch non è determinato unicamente dai valori di input correnti.

Per vedere come ciò avviene assumiamo, come si verifica nella maggior parte dei casi, che sia *S* sia *R* valgano 0. Ai fini della spiegazione assumiamo inoltre che $Q = 0$. Dato che *Q* viene reinserito come input della porta NOR visualizzata in alto, entrambi gli input di questa porta valgono 0 generando come output il valore 1. Il valore 1 è riutilizzato come input della porta visualizzata in basso, i cui input sono 1 e 0 e producono come output $Q = 0$. Questo stato, mostrato nella Figura 3.21(a), è stabile.



A	B	NOR
0	0	1
0	1	0
1	0	0
1	1	0

(c)

Figura 3.21 (a) Latch di tipo NOR nello stato 0. (b) Latch di tipo NOR nello stato 1. (c) Tabella di verità del NOR.

Immaginiamo ora che *Q* non sia 0, ma 1, mentre *R* e *S* continuano a valere 0. La porta logica visualizzata in alto ha come input 0 e 1; il suo output $\overline{Q}$ vale 0 ed è riutilizzato come input della porta visualizzata in basso. Anche questo stato è stabile ed è mostrato nella Figura 3.21(b). Uno stato in cui entrambi gli output valgono 0 è invece instabile, in quanto obbliga entrambe le porte ad avere due 0 come input, il che, se fosse vero, produrrebbe come risultato 1 e non 0. Analogamente è impossibile che entrambi gli output valgano 1, dato che ciò forzerebbe gli input a 0 e 1, il cui risultato dovrebbe essere 0 e non 1. La conclusione è semplice: per $R = S = 0$ il latch ha due stati stabili che identificheremo con 0 e 1 in base al valore di *Q*.

Esaminiamo ora quali effetti producono sullo stato del latch i valori dell'input. Supponiamo che *S* diventi 1, mentre $Q = 0$. Gli input del NOR superiore sono quindi 1 e 0 e

forzano l'output $\overline{Q}$ a valere 0. Questo cambiamento porta entrambi gli input della porta inferiore a valere 0, producendo in output il valore 1. Settando quindi S (cioè impostandone il valore a 1) si può far passare lo stato da 0 a 1. Settando R a 1 quando il latch si trova nello stato 0 non si produce invece alcun effetto, dato che l'output della porta NOR in basso è 0 sia che i suoi input valgano 10 sia che valgano 11.

Applicando un ragionamento simile si può verificare facilmente che impostare S a 1 quando il latch è nello stato $Q = 1$ non produce alcun effetto, mentre settando R si modifica lo stato del latch portandolo nello stato $Q = 0$. Riassumendo si può dire che quando S è impostato temporaneamente a 1 lo stato del latch diventa $Q = 1$, indipendentemente dallo stato in cui si trovava precedentemente. Allo stesso modo quando si imposta temporaneamente R a 1 si forza il latch a passare nello stato $Q = 0$. Il circuito "ricorda" quindi quale valore, se S oppure R, è stato settato per ultimo; utilizzando questa proprietà è possibile costruire le memorie dei calcolatori.

Latch SR temporizzato

Spesso è preferibile impedire che un latch cambi di stato se non in specifici momenti. Un circuito che gode di questa caratteristica è detto **latch SR temporizzato**; per costruirlo occorre modificare leggermente il circuito visto precedentemente. Il circuito della Figura 3.22 ha un input aggiuntivo, il clock, il cui valore è generalmente 0. Quando il clock vale 0 entrambe le porte AND generano in output il valore 0, indipendentemente dai valori di S e R, impedendo quindi al latch di cambiare di stato. Quando il clock vale 1 le porte AND non bloccano più i segnali S e R che possono dunque tornare a pilotare lo stato del latch. Nonostante il suo nome non è obbligatorio che il segnale collegato all'input aggiuntivo debba per forza essere comandato da un clock. I termini ***enable*** e ***strobe*** sono largamente utilizzati per indicare che l'input clock vale 1, ovvero che il circuito è dipendente dallo stato di S e da quello di R.

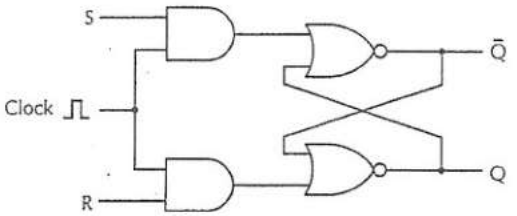


Figura 3.22 Latch SR temporizzato.

Finora abbiamo intenzionalmente evitato di spiegare che cosa succede se sia S sia R valgono 1: il circuito diventa non deterministico finché sia R sia S non tornino ad assumere il valore 0. L'unico stato consistente per $S = R = 1$ è quando $Q = \overline{Q} = 0$; non appena entrambi gli input ritornano al valore 0 il latch deve tuttavia passare istantaneamente in uno dei suoi due stati stabili. Se uno dei due input torna a 0 prima dell'altro, prevale quello che rimane al valore 1 più a lungo, dato che quando uno solo degli input vale 1

esso determina lo stato del latch. Se invece entrambi gli input ritornano a 0 nello stesso istante (cosa che può verificarsi molto raramente) il latch passa in uno dei due stati stabili in modo del tutto casuale.

Latch D temporizzato

Un buon modo per risolvere l'ambiguità dei latch SR (causata dalla situazione $S = R = 1$) è... evitare che si verifichi. La Figura 3.23 mostra un circuito che ha un solo input, D. Dato che l'input della porta AND rappresentata in basso è sempre il complemento dell'input di quella superiore, non può mai accadere che entrambi gli input valgono 1. Quando $D = 1$ e il clock vale 1, il latch viene portato nello stato $Q = 1$, mentre, quando $D = 0$ e il clock vale 1, il latch passa nello stato $Q = 0$. In altre parole quando il clock vale 1 il valore corrente di D viene campionato e memorizzato nel latch. Questo circuito, chiamato **latch D temporizzato**, è una vera e propria memoria a 1 bit, in cui il valore memorizzato è sempre disponibile sulla linea Q. Per caricare in memoria il valore corrente di D occorre spedire un impulso positivo sulla linea del clock.

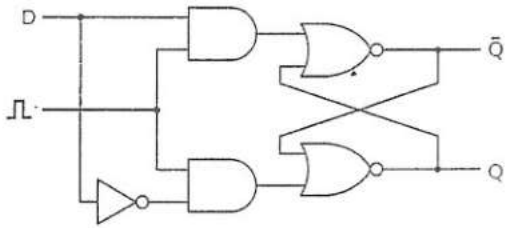


Figura 3.23 Latch D temporizzato.

Il circuito appena descritto richiede 11 transistor; esistono tuttavia circuiti più sofisticati (ma con un'implementazione meno ovvia) che possono memorizzare 1 bit utilizzando solo sei transistor. Nelle implementazioni reali si adottano queste soluzioni più sofisticate, che impiegano meno transistor. Il circuito descritto resta sempre stabile se vi è corrente (la corrente non è mostrata). Più avanti vedremo circuiti di memoria che dimenticano rapidamente il loro stato se non sono in qualche modo "richiamati".

3.3.2 Flip-flop

In molti circuiti è necessario campionare il valore di una certa linea in un particolare istante e memorizzarlo. In questi circuiti, chiamati **flip-flop**, la transizione di stato non si verifica quando il clock vale 1, ma durante la transizione del clock da 0 a 1 (fronte di salita) oppure da 1 a 0 (fronte di discesa). In questa situazione la lunghezza dell'impulso del clock non ha alcuna importanza, purché le transizioni si verifichino con sufficiente velocità.

Per sottolinearlo ulteriormente ripetiamo la differenza che c'è tra un flip-flop e un latch: un flip-flop è a **commutazione sul fronte**, mentre un latch è a **commutazione a**

livello. Occorre però prestare attenzione al fatto che in letteratura spesso questi termini vengono confusi; molti autori utilizzano il termine "flip-flop" per indicare un latch e viceversa.

Esistono vari approcci per progettare un flip-flop. Se per esempio esistesse un metodo per generare un impulso di lunghezza estremamente breve sul fronte di salita, si potrebbe immettere tale impulso in un latch D. Il circuito che implementa questa soluzione è mostrato nella Figura 3.24(a).

A prima vista potrebbe sembrare che l'output della porta AND debba essere sempre zero, dato che l'AND tra un qualsiasi segnale e il suo inverso vale sempre zero; in realtà la situazione, anche se in modo sottile, è diversa. L'invertitore induce un piccolo, ma non nullo, ritardo di propagazione che permette al circuito di funzionare in modo corretto. Supponiamo di misurare la tensione in quattro punti diversi: *a*, *b*, *c* e *d*. Il segnale di input, misurato in *a*, è un lungo impulso di clock, come mostra, in basso, la Figura 3.24(b); il segnale in *b* è invece rappresentato dal grafico che nella figura si trova sopra quello precedente. Si noti che questo segnale, oltre a essere stato invertito, risulta anche sfasato di un ritardo che è di pochi nanosecondi (a seconda del tipo di invertitore utilizzato).

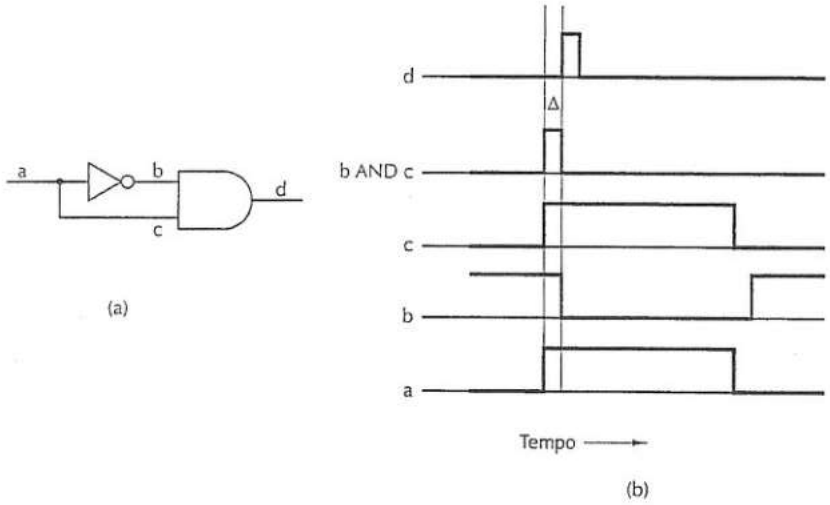


Figura 3.24 (a) Generatore d'impulsi. (b) Diagrammi temporali.

Anche il segnale *c* subisce un ritardo, che però è determinato soltanto dal tempo di propagazione del segnale (a due terzi della velocità della luce). Se la distanza tra *a* e *c* è, per esempio, di 20 μm, il ritardo di propagazione è di un decimo di picosecondo. Tale valore è certamente trascurabile rispetto al tempo di commutazione dell'invertitore. Di conseguenza per qualsiasi intento o scopo del circuito, il segnale *c* può essere considerato identico al segnale *a*.

Quando gli input *b* e *c* vengono processati dalla porta AND si ottiene come risultato un impulso di breve durata, come mostra la Figura 3.25(b); la larghezza Δ dell'impulso è uguale al ritardo dell'invertitore, generalmente inferiore a 5 ns. Com'è illustrato nella Figura 3.24(b) l'output della porta AND corrisponde semplicemente a questo impulso sfasato del ritardo interno alla porta logica AND. Questo sfasamento temporale significa che il latch D verrà attivato con un ritardo fisso rispetto al fronte di salita del clock; tuttavia ciò non ha alcun effetto sulla durata dell'impulso. Un impulso di 1 ns che segnala quando campionare la linea D può essere considerato sufficientemente corto per una memoria con un ciclo della durata di 10 ns; in un caso simile il circuito completo potrebbe essere quello della Figura 3.25. Occorre sottolineare che il vantaggio dell'architettura di questo flip-flop è di essere di facile comprensione; tuttavia nella realtà si ricorre di solito ad architetture più sofisticate.

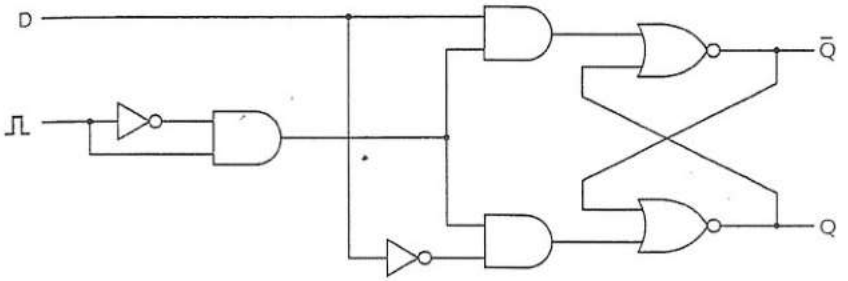


Figura 3.25 Flip-flop D.

La Figura 3.26 mostra i simboli comunemente usati per i latch e i flip-flop. La Figura 3.26(a) indica un latch il cui stato viene caricato quando il clock, *CK*, vale 1; al contrario lo stato del latch della Figura 3.26(b) è generalmente 1, ma passa temporaneamente al valore 0 per assumere lo stato dalla linea *D*. Le Figure 3.26(c) e (d) rappresentano due flip-flop invece che due latch; ciò è indicato dal simbolo triangolare vicino all'input del clock. La Figura 3.26(c) cambia stato in corrispondenza del fronte di salita dell'impulso del clock (transizione da 0 a 1), mentre la Figura 3.26(d) cambia stato in corrispondenza del fronte di discesa (transizione da 1 a 0). Molti latch e flip-flop, seppur non tutti, hanno anche l'output $\overline{Q}$ e alcuni sono dotati di due input aggiuntivi, *Set* o *Preset* (che forzano lo stato a $Q = 1$) e *Reset* o *Clear* (che forzano lo stato a $Q = 0$).

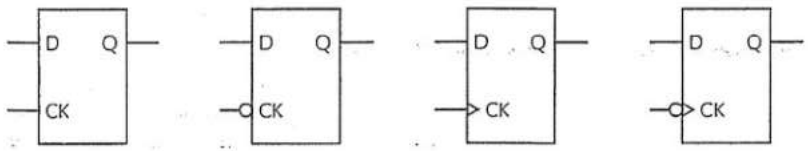


Figura 3.26 Flip-flop D temporizzati.

3.3.3 Registri

I Flip-Flop possono essere combinati per creare dei registri che memorizzano dati composti da più bit. La figura 3.27 mostra come otto flip-flop possono essere raggruppati per formare un registro da 8 bit. Il registro riceve in ingresso un valore di 8 bit (da *I0* a *I7*) quando vi è una transizione del clock CK. Per implementare un registro tutte le linee di clock sono collegate allo stesso segnale in ingresso CK, in modo che quando si ha una transizione di clock il registro riceve dal canale di ingresso il nuovo dato di 8 bit.

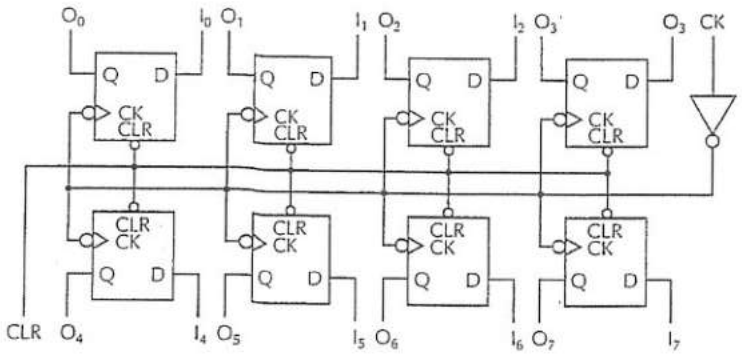


Figura 3.27 T Un registro a 8 bit costruito a partire da flip-flop a 1 bit.

I singoli flip-flop sono del tipo mostrato nella Figura 3.25(d), anche se il cerchietto d'inversione non è presente per via dell'invertitore collegato al segnale di clock CK; per questa differenza i flip-flop vengono caricati in corrispondenza della transizione di salita. Anche tutti e otto i canali di cancellazione sono collegati fra loro, di modo che quando CLR va a 0 tutti i flip-flop siano forzati ad assumere lo stato 0. Ci si potrebbe chiedere per quale motivo il clock CK è invertito nella linea di input per poi essere invertito nuovamente in corrispondenza di ciascun flip-flop. Il motivo è che un segnale potrebbe non avere sufficiente corrente per pilotare tutti i flip-flop; l'invertitore svolge in realtà il ruolo di amplificatore.

Una volta progettato un registro a 8 bit, lo si può utilizzare per costruire registri più grandi. Per esempio, un registro a 32 bit può essere costruito combinando due registri a 16 bit, unendo i loro segnali di clock CK e di cancellazione CLR. Nel corso del Capitolo 4 analizzeremo più da vicino i registri e il loro impiego.

3.3.4 Organizzazione della memoria

Nei paragrafi precedenti siamo partiti dalla semplice memoria a 1 bit della Figura 3.23 per arrivare alla memoria a 8 bit della Figura 3.27. Per realizzare memorie di dimensione maggiore è però necessaria un'organizzazione di tipo diverso, nella quale sia possibile indirizzare singole parole. La Figura 3.28 mostra un'organizzazione della memoria molto

comune che soddisfa questo requisito. L'esempio illustra una memoria con quattro parole a 3 bit in cui ciascuna operazione legge o scrive un'intera parola. Anche se la memoria ha una capacità totale (12 bit) decisamente maggiore rispetto a quella del flip-flop ottale visto precedentemente, essa richiede un numero inferiore di pin. Una caratteristica ancora più rilevante è che questa organizzazione è facilmente estendibile a memorie di dimensione maggiore. Si noti che il numero di parole è sempre una potenza di 2.

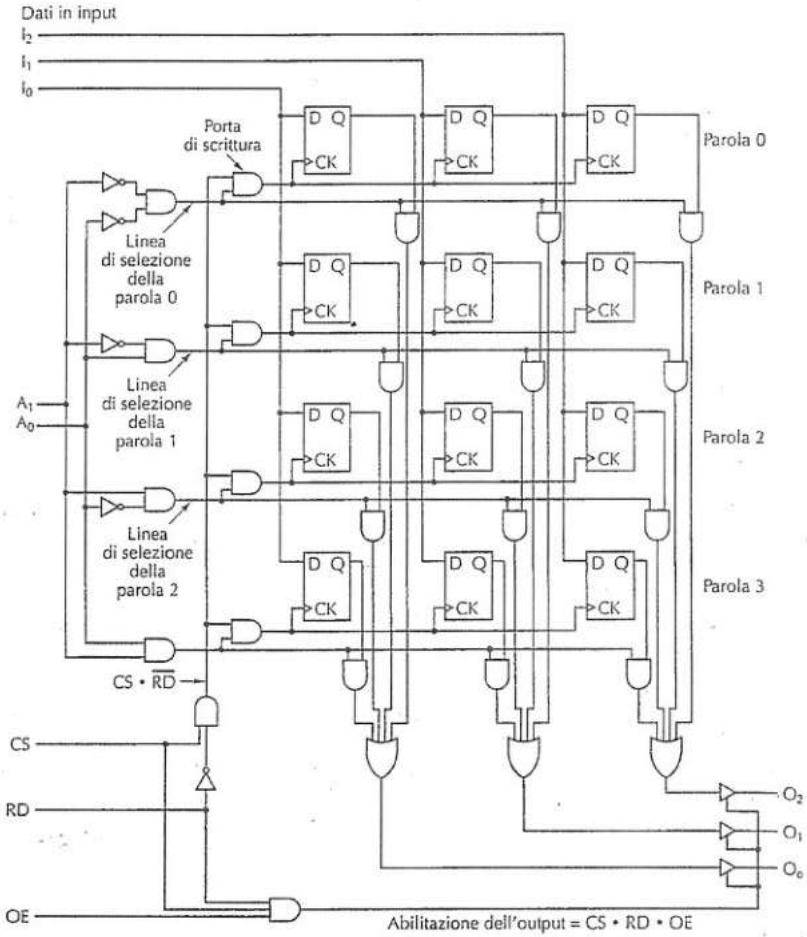


Figura 3.28 Diagramma logico di una memoria 4 × 3. Ogni riga è una della quattro parole a 3 bit. Lettura e scrittura riguardano sempre parole complete.

A prima vista la memoria della Figura 3.28 potrebbe sembrare complessa, ma in realtà, se si considera la regolarità della struttura, si nota che è piuttosto semplice. Delle otto linee di input tre sono per i dati, I_0, I_1 e I_2, due per l'indirizzo, A_0 e A_1, e tre per i controlli, CS per la selezione del chip, RD per distinguere tra lettura e scrittura e OE per l'abilitazione dell'output. Le tre linee di output, O_0, O_1 e O_2, sono invece dedicate ai dati. È interessante osservare che questa memoria da 12 bit necessita di meno segnali rispetto ai registri a 8 bit visti in precedenza. I registri a 8 bit utilizzano 20 pin, includendo corrente e terra, mentre le memorie da 12 bit ne richiedono 13, perché, a differenza dei registri, condividono un segnale di output. Nel nostro esempio, 4 bit di memoria condividono lo stesso segnale di output. Il valore delle linee di indirizzo determina a quale dei 4 bit di memoria è permesso di ricevere o restituire un valore.

Per utilizzare questo chip è necessario che una componente logica esterna imposti, in caso di lettura, sia CS sia RD al valore alto (valore logico 1), mentre, in caso di scrittura, li imposti al valore basso (valore logico 0). Le due linee dell'indirizzo devono essere impostate in modo da indicare quale delle quattro parole a 3 bit deve essere letta o scritta. In lettura le linee di input non vengono utilizzate e la parola selezionata viene resa disponibile sulle linee di output. In scrittura i bit presenti sulle linee di input dei dati vengono caricati nella parola di memoria selezionata, mentre le linee di output non vengono utilizzate.

Per comprendere il funzionamento della memoria osserviamo ora in modo più dettagliato la Figura 3.28. Le quattro porte AND situate a sinistra della memoria formano un decodificatore e servono a selezionare la parola. I quattro invertitori in input sono stati collocati per far sì che ciascuna porta logica sia abilitata (l'output è alto) da un indirizzo diverso. Ciascuna porta è collegata a una linea per la selezione di una delle parole, indicate dall'alto verso il basso con i numeri 0, 1, 2 e 3. Quando ilchip viene selezionato per un'operazione di scrittura la linea verticale etichettata con $CS \cdot \overline{RD}$ assume valore alto, abilitando quindi una della quattro porte di scrittura; la porta abilitata dipende da quale linea per la selezione della parola ha assunto valore alto. L'output della porta di scrittura guida tutti i segnali CK relativi alla parola selezionata, caricando i dati di input nei flip-flop che costituiscono la parola stessa. La scrittura è possibile soltanto quando CS è alto e RD è basso; l'unica parola a essere scritta è quella selezionata dai segnali A_0 e A_1, mentre le altre non vengono modificate.

L'operazione di lettura è simile: la decodifica dell'indirizzo, per esempio, si svolge esattamente nello stesso modo, la linea $CS \cdot \overline{RD}$ assume però valore basso, disabilitando di conseguenza tutte le porte di scrittura e impedendo la modifica dei flip-flop. La linea per la selezione della parola scelta abilita le porte AND cui sono collegati i bit Q della parola selezionata. La parola selezionata spedisce quindi i propri dati alle porte OR mostrate nella parte bassa della figura. Dato che le altre parole generano in output valori 0, il risultato delle porte OR è identico al valore memorizzato nella parola selezionata. Le tre parole non selezionate non forniscono quindi alcun contributo all'output finale.

Anche se si potrebbe progettare un circuito nel quale le tre porte OR siano collegate direttamente alle tre linee di output dei dati, in alcuni casi ciò potrebbe causare dei problemi. Nell'illustrazione abbiamo distinto le linee di input da quelle di output, ma nelle memorie reali si utilizzano le stesse linee. Se avessimo collegato le porte OR alle linee

di output il chip avrebbe cercato di spedire in output i dati anche durante un'operazione di scrittura, forzando ciascuna linea ad assumere un particolare valore e interferendo quindi con i dati di input. Per questo motivo è preferibile avere la possibilità di connettere le porte OR alle linee di output durante le letture e di disconnetterle completamente durante le scritture. Quello di cui abbiamo bisogno è un interruttore elettronico che possa instaurare o interrompere una connessione in una frazione di nanosecondo.

Un interruttore del genere è chiamato **buffer non invertente** e la Figura 3.29(a) mostra il simbolo che lo rappresenta. Esso ha un dato di input, un dato di output e un input di controllo. Quando l'input di controllo è alto il buffer funge da collegamento, come mostra la Figura 3.29(b). Quando invece l'input di controllo è basso il buffer si comporta come un circuito aperto, come mostra la Figura 3.29(c): è come se qualcuno avesse scollegato il dato di output dal resto del circuito con una pinza tagliafili. Tuttavia, diversamente dall'utilizzo di una pinza tagliafili, la connessione può essere ripristinata in una frazione di nanosecondo semplicemente reimpostando il segnale di controllo al valore alto.

La Figura 3.29(d) mostra un **buffer invertente** che si comporta come un normale invertitore quando il controllo ha valore alto e disconnette l'output dal circuito quando il controllo ha valore basso. I due buffer sono **dispositivi a tre stati**, in quanto possono generare in output i valori 0 e 1, oppure nessuno dei due (circuito aperto). I buffer inoltre amplificano il segnale e possono quindi guidare più input allo stesso tempo; talvolta vengono impiegati all'interno dei circuiti proprio per questa ragione, anche quando non sono richieste le loro proprietà d'inversione.

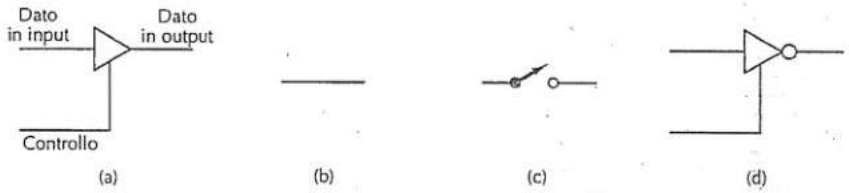


Figura 3.29 (a) Buffer non invertente. (b) Risultato di (a) quando il controllo è alto. (c) Risultato di (a) quando il controllo è basso. (d) Buffer invertente.

Tornando al circuito della memoria dovrebbe essere chiaro quale sia il ruolo dei buffer non invertenti che si trovano sulle linee di output dei dati. Quando CS, RD e OE hanno tutti il valore alto anche il segnale per l'abilitazione dell'output è alto; ciò permette di attivare i buffer e di spedire una parola sulle linee di output. Al contrario, quando uno qualsiasi dei segnali CS, RD e OE è basso, l'output è scollegato dal resto del circuito.

3.3.5 Chip di memoria

Una proprietà interessante della memoria della Figura 3.28 è che può essere facilmente ampliata. Quella che abbiamo descritto è una memoria 4 × 3, costituita cioè da quattro

parole di 3 bit ciascuna. Per estenderla alla dimensione 4 × 8 occorre semplicemente aggiungere cinque colonne composte da quattro flip-flop ciascuna, così come cinque linee di input aggiuntive e altrettante linee di output. Per passare dalla dimensione 4 × 3 a quella 8 × 3 si devono aggiungere quattro nuove righe di tre flip-flop ciascuna, oltre a una linea aggiuntiva, A_2, per l'indirizzo. Una memoria con questa struttura dovrebbe avere un numero di parole esprimibile come una potenza di 2 al fine di massimizzare l'efficienza; il numero di bit di una parola può invece assumere qualsiasi valore.

Dato che la tecnologia dei circuiti integrati è particolarmente adatta a realizzare chip la cui struttura interna abbia uno schema bidimensionale ripetuto, i chip di memoria ne sono un'applicazione ideale. Con l'avanzamento della tecnologia continua ad aumentare il numero di bit che si può inserire in un chip. Tale valore raddoppia all'incirca ogni 18 mesi (legge di Moore). I chip di dimensioni maggiori non sempre rendono obsoleti quelli più piccoli, dato che spesso occorre trovare dei compromessi tra diversi fattori, quali la capacità, la velocità, l'alimentazione, il prezzo e la comodità di interfacciamento. Di solito i chip di dimensioni maggiori sono venduti come prodotti di qualità superiore e quindi a un prezzo per bit più elevato rispetto a quelli più vecchi e più piccoli.

Fissata una dimensione della memoria esistono diversi modi per organizzare il chip. La Figura 3.30 mostra due possibili organizzazioni per un vecchio chip di memoria della dimensione di 4 Mbit: 512K × 8 e 4096K × 1. Per inciso, le dimensioni dei chip di memoria sono generalmente indicate in bit, e non in byte, e anche noi ci atterremo a questa convenzione. Nella Figura 3.30(a) sono necessarie 19 linee per indirizzare uno dei 2^{19} byte e otto linee di output per caricare e memorizzare i byte selezionati.

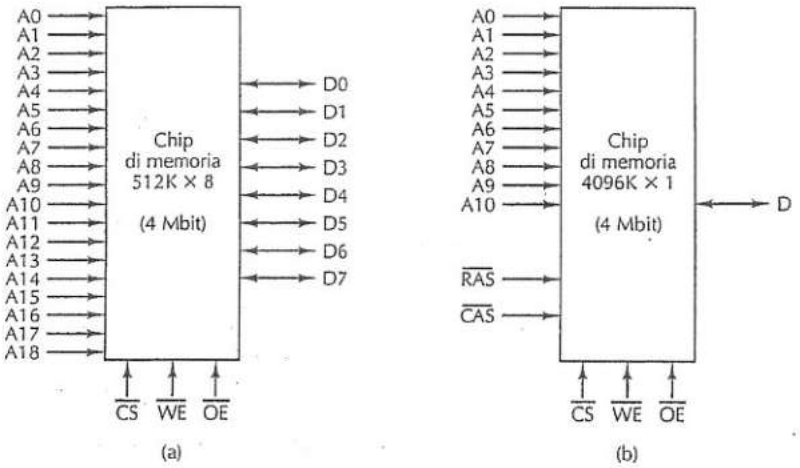


Figura 3.30 Due modi di organizzare un chip di memoria a 4 Mbit.

Occorre sottolineare un aspetto della terminologia utilizzata. Su alcuni pin l'applicazione di un'alta tensione genera una qualche azione, mentre su altri le azioni sono innescate da bassi valori di tensione. Per evitare confusione diremo che un segnale è **asserito** (piuttosto che dire che assume valore alto o basso) per indicare che è impostato in modo da generare una qualche azione. Alcuni pin sono asseriti con valore alto, mentre altri con valore basso. I pin asseriti con valore basso sono identificati da una linea sopra il loro nome. Un segnale chiamato CS (*Chip Select*) viene quindi asserito con il valore alto, mentre uno chiamato $\overline{\text{CS}}$ è asserito con il valore basso. L'opposto di asserito è **negato**; quando non succede nulla di particolare i pin sono negati.

Torniamo ora al nostro chip di memoria. Dato che un calcolatore ha di solito vari chip di memoria è necessario disporre di un segnale che selezioni il chip richiesto in un dato momento, di modo che solo esso risponda al segnale, mentre tutti gli altri lo ignorino. Per questo scopo si utilizza il segnale $\overline{\text{CS}}$, asserito quando si intende abilitare il chip. Inoltre è necessario un metodo per distinguere tra le operazioni di lettura e quelle di scrittura; ciò viene fatto dal segnale $\overline{\text{WE}}$ (*Write Enable*), utilizzato per indicare che i dati devono essere scritti piuttosto che letti. Infine il segnale $\overline{\text{OE}}$ (*Output Enable*) viene asserito per guidare i segnali di output; quando non è asserito, l'output del chip è sconnesso dal circuito.

Nella Figura 3.30(b) si utilizza un diverso schema per l'indirizzamento. Al suo interno questo chip è organizzato come una matrice di 2048 × 2048 celle a 1 bit, che forniscono una capacità totale di 4 Mbit. Per indirizzare il chip si seleziona inizialmente una riga immettendo un numero a 11 bit sui pin dell'indirizzo e asserendo il segnale $\overline{\text{RAS}}$ (*Row Address Strobe*, "strobe dell'indirizzo di riga"). Successivamente si immette sui pin dell'indirizzo un numero di colonna e si asserisce il segnale $\overline{\text{CAS}}$ (*Column Address Strobe*, "strobe dell'indirizzo di colonna"). Il chip risponde accettando, oppure generando in output, un dato da 1 bit.

I chip di memoria di grandi dimensioni sono spesso costruiti come matrici di $n \times n$ indirizzate da numeri di riga e colonna. Questo tipo di architettura riduce il numero di pin necessari, ma rende allo stesso tempo più lento il chip, in quanto sono necessari due cicli di indirizzamento, uno per la riga e uno per la colonna. Per riguadagnare parte della velocità persa a causa dell'architettura, in alcuni chip è possibile specificare un indirizzo di riga seguito da una sequenza di indirizzi di colonna in modo da poter accedere a bit consecutivi all'interno di una stessa riga.

Anni fa i chip di memoria più grandi erano spesso organizzati come nella Figura 3.30(b). Quando però la dimensione delle parole di memoria è passata da 8 bit a 32 bit, i chip larghi 1 bit sono diventati poco convenienti. Per costruire una memoria con parole a 32 bit mediante chip 4096K × 1 è infatti necessario utilizzarne 32 in parallelo. Questi 32 chip forniscono una capacità totale di 16 MB, mentre l'utilizzo di chip 512K × 8 richiede solo quattro chip in parallelo e fornisce memorie della dimensione di 2 MB. Per evitare di avere 32 chip per la memoria, quasi tutti i produttori offrono attualmente famiglie di chip con larghezze da 4, 8 e 16 bit. Ovviamente se si considerano parole a 64 bit la situazione è ancora più critica.

La Figura 3.31 mostra due esempi di moderni chip a 512 Mbit. I chip hanno quattro banchi di memoria di 128 Mbit ciascuno e richiedono quindi due linee per la selezione

Dopo la SDRAM il passaggio successivo è stata la SDRAM **DDR** (*Double Data Rate*, "memoria a duplice tasso di trasferimento"). In queste memorie il chip produce un output sul fronte di salita del segnale di clock e uno sul fronte di discesa, raddoppiando così il tasso di trasferimento dati. Un chip DDR largo 8 bit e funzionante a 200 MHz genera in output, per 200 milioni di volte al secondo (ovviamente per un breve intervallo di tempo), due valori a 8 bit; questi dati corrispondono a un *burst rate* (cioè una frequenza di picco) teorico di 3,2 Gbps. Le memorie DDR2 e DDR3 offrono prestazioni migliori delle DDR grazie all'incremento delle velocità del bus a 533 MHz e 1067 MHz rispettivamente. Quando questo libro è andato in stampa la prima volta, il chip DDR3 più veloce poteva trasferire dati alla velocità di 17.067 GB/s

Chip di memoria non volatile

Le RAM non sono l'unico tipo di chip di memoria. In molte applicazioni, come i giocattoli, gli elettrodomestici e le automobili, il programma e alcuni dati devono rimanere memorizzati anche quando viene tolta l'alimentazione. Inoltre non si richiede mai la modifica del programma né dei dati installati. Questi requisiti hanno portato allo sviluppo delle **ROM** (*Read-Only Memory*, "memoria di sola lettura") che non possono essere modificate o cancellate, né intenzionalmente né accidentalmente. I dati sono inseriti durante la sua fabbricazione. Ciò avviene esponendo alla luce un materiale fotosensibile attraverso una maschera contenente il pattern di bit desiderato e incidendo quindi la superficie esposta (o non esposta). L'unico modo per cambiare il programma consiste nella sostituzione dell'intero chip.

In grandi volumi le ROM sono molto più economiche delle RAM dato che il costo per la realizzazione della maschera viene ammortizzato dal gran numero di esemplari. Le memorie ROM non sono però flessibili, dato che non possono essere modificate dopo la fabbricazione; inoltre il tempo che passa tra l'ordine e la ricezione delle ROM può essere di settimane. Per far sì che le aziende potessero sviluppare più agevolmente nuovi prodotti basati su ROM sono state inventate le **PROM** (*Programmable ROM*, ROM programmabile). Una PROM differisce da una ROM per il fatto che può essere programmata (una volta) sul posto, evitando quindi i tempi per il completamento dell'ordine da parte del produttore. Molte PROM contengono un array di piccoli fusibili, che possono essere bruciati: ciò viene fatto selezionando righe e colonne e applicando un'alta tensione a un particolare pin del chip.

Il passo successivo in questa linea di prodotti è stata la **EPROM** (*Erasable PROM*, PROM cancellabile), i cui campi non solo possono essere programmati, ma anche cancellati. Quando si espone la piccola lente al quarzo che si trova nella EPROM a un'intensa luce ultravioletta per 15 minuti, tutti i bit assumono il valore 1. Se ci si aspetta che la produzione di un chip richiederà un gran numero di fasi di test le EPROM risultano molto più economiche delle PROM, dato che possono essere riutilizzate. In genere le EPROM hanno la stessa organizzazione delle RAM statiche. La EPROM 27C040 a 4 Mbit utilizza per esempio la stessa organizzazione della Figura 3.31(a), generalmente impiegata per una RAM statica. È interessante notare che chip vecchi come questo non sono completamente morti, ma sono diventati più economici e hanno trovato un nuovo utilizzo in prodotti di bassa fascia particolarmente sensibili ai costi. Un chip 27C040 oggi costa meno di 3$ e si può spendere ancora meno se lo si compra in grandi quantità.

Meglio ancora delle EPROM sono la **EEPROM** (la prima E sta per *elettricamente*) che possono essere cancellate applicando impulsi elettrici invece di dover inserire il chip in una camera speciale per l'esposizione alla luce ultravioletta. Inoltre queste memorie sono riprogrammabili senza doverle rimuovere dal circuito, mentre una EPROM deve essere inserita nello speciale dispositivo che ne permette la programmazione. Le EEPROM però non possono essere più grandi di 1/64 delle comuni EPROM e la loro velocità è solo la metà. Le EEPROM non possono competere con le DRAM e le SRAM dato che sono 10 volte più lente, hanno una capacità 100 volte minore e sono molto più costose; esse sono utilizzate solo quando la loro proprietà di non volatilità è cruciale.

Un tipo più recente di EEPROM è la **memoria flash**. Diversamente dalla EPROM, che è cancellabile per esposizione alla luce ultravioletta, e dalla EEPROM, nella quale ogni singolo byte è cancellabile, la memoria flash è cancellabile a blocchi e riscrivibile. Come la EEPROM anche la memoria flash può essere cancellata senza doverla rimuovere dal circuito. Varie aziende producono piccole schede con circuiti stampati che contengono anche 64 GB di memoria flash utilizzate, oltre che per altri scopi, come "pellicole" per memorizzare le foto delle macchine fotografiche digitali. Come detto nel Capitolo 2, la memoria flash sta iniziando a sostituire i dischi magnetici. Utilizzata come disco, la memoria flash offre minori tempi d'accesso e minori consumi, ma con un costo per bit molto più alto. La Figura 3.32 mostra un riepilogo dei vari tipi di memorie.

Tipo	Categoria	Cancellazione	Byte modificabili	Volatile	Tipico utilizzo
SRAM	Read/write	Elettrica	Sì	Sì	Cache di secondo livello
DRAM	Read/write	Elettrica	Sì	Sì	Memoria centrale (vecchia)
SDRAM	Read/write	Elettrica	Sì	Sì	Memoria centrale (recente)
ROM	Read-only	Impossibile	No	No	Elettrodomestici (prodotti in grandi volumi)
PROM	Read-only	Impossibile	No	No	Dispositivi (prodotti in piccoli volumi)
EPROM	Read-mostly	Raggi UV	No	No	Prototipazione di dispositivi
EEPROM	Read-mostly	Elettrica	Sì	No	Prototipazione di dispositivi
Flash	Read/write	Elettrica	No	No	"Pellicola" per macchine fotografiche digitali

Figura 3.32 Confronto tra vari tipi di memoria.

FPGA

Come abbiamo visto nel Capitolo 1, gli FPGA (*field-programmable gate array*) sono circuiti integrati che contengono una logica programmabile e permettono di formare un circuito logico arbitrario semplicemente caricando l'FPGA con i dati di configurazione appropriati. Il vantaggio principale di questi circuiti è la possibilità di costruire nuovi circuiti hardware in poche ore, piuttosto che attendere i mesi necessari per fabbricare circuiti integrati. Nonostante ciò, questi ultimi non sono sul viale del tramonto, essendo ancora in possesso di un significativo vantaggio di costo rispetto agli FPGA sui grandi volumi; sono inoltre più veloci e consumano meno energia. A causa dei loro vantaggi in fase di progettazione, tuttavia, gli FPGA vengono spesso utilizzati per la prototipazione e la progettazione di applicazioni su piccola scala.

Guardiamo ora all'interno di un FPGA e cerchiamo di capire come può essere utilizzato per implementare una vasta gamma di circuiti logici. Il chip FPGA contiene due componenti principali che vengono replicati più volte: le LUT (*Look-Up Table*, tabelle di ricerca) e le **interconnessioni programmabili**. Esaminiamo il modo in cui questi componenti vengono utilizzati.

Una LUT, mostrata nella Figura 3.33(a), è una piccola memoria programmabile che produce un segnale in uscita che viene poi trasmesso alla interconnessione programmabile ed eventualmente a un registro. La memoria programmabile viene usata per creare una funzione logica arbitraria. La LUT nella figura ha una memoria 16 x 4 che può emulare qualsiasi circuito logico con 4 bit di ingresso e 4 bit di uscita. La programmazione della LUT richiede il caricamento nella memoria delle risposte appropriate della logica combinatoria che deve essere emulata. In altre parole, se la logica combinatoria produce il valore di Y per un dato ingresso X, il valore Y va scritto nella LUT all'indice X.

L'esempio nella Figura 3.33(b) mostra come una singola LUT a 4 ingressi potrebbe implementare un contatore a 3 bit con reset. Il contatore dell'esempio continua a contare aggiungendo uno (modulo 4) al valore corrente del contatore, a meno che il segnale di reset CLR venga asserito, nel qual caso il contatore riporta il suo valore a zero.

Per implementare il contatore dell'esempio, le quattro entry superiori della LUT sono poste a zero. Queste voci restituiscono il valore zero quando il contatore viene resettato. Quindi, il bit più significativo della LUT (I_3) rappresenta l'ingresso di reset (CLR), che è asserito con un valore logico 1. Nelle rimanenti voci della LUT il valore di indice I_0..3 contiene (I + 1) modulo 4. Per completare il progetto, i segnali in uscita O_0..3 devono essere collegati, tramite l'interconnessione programmabile, ai segnali interni di ingresso I_0..3.

Per capire meglio il contatore con reset realizzato con un FPGA, consideriamo le operazioni che esegue. Se, per esempio, lo stato corrente del contatore è 2 e il segnale di reset non è asserito, l'indirizzo di input della LUT sarà 2 e l'output verso i flip-flop sarà di conseguenza 3. Se, nello stesso stato, il segnale di reset (CLR) viene asserito, l'input della LUT sarà 6 e quindi lo stato successivo sarà 0.

Può sembrare, tutto sommato, un modo arcano per costruire un contatore con reset e in effetti un progetto completamente personalizzato con un circuito contatore e segnali di ripristino al flip-flop sarebbe più piccolo, più veloce, e consumerebbe meno energia.

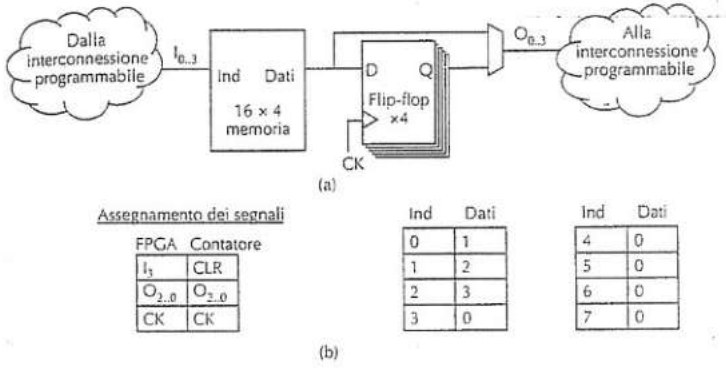


Figura 3.33 (a) La LUT di un FPGA. (b) La configurazione della LUT per la realizzazione di un contatore a 3 bit con reset.

Il vantaggio principale del progetto basato su FPGA è che si può costruire artigianalmente a casa in un'ora, mentre il più efficiente design completamente personalizzato deve essere fabbricato a partire dal silicio e l'operazione potrebbe richiedere anche più di un mese.

Per utilizzare un FPGA il progetto deve essere descritto con una rappresentazione del circuito o con un linguaggio di descrizione hardware (cioè un linguaggio di programmazione usato per descrivere strutture hardware). Il progetto viene poi elaborato da un sintetizzatore, che mappa il circuito su un'architettura FPGA specifica. Succede spesso che il progetto che si desidera realizzare non possa essere mappato sull'FGPA. Gli FPGA sono realizzati con un numero variabile di LUT e quelli che ne hanno di più costano di più. In generale, se il vostro progetto non si adatta all'FPGA, è necessario semplificarlo o rinunciare ad alcune funzionalità, oppure acquistare un FPGA più grande (e più costoso). Progetti molto grandi potrebbero non trovare spazio nemmeno nel più grande FPGA. In questo caso il progettista dovrà mappare il progetto in più FPGA; questo compito è sicuramente più difficile, ma è ancora una passeggiata rispetto alla progettazione di un circuito integrato personalizzato.

3.4 Chip della CPU e bus

Forti delle conoscenze acquisite sui circuiti integrati, sui clock e sui circuiti di memoria possiamo ora cominciare a mettere insieme i vari pezzi per affrontare il sistema nel suo complesso. In questo paragrafo analizzeremo inizialmente alcuni aspetti generali delle CPU dal punto di vista del livello logico digitale, **contatti** compresi (significato dei segnali sui singoli pin), e forniremo anche un'introduzione all'architettura dei bus, dato che le CPU dipendono strettamente dal modo in cui questi sono progettati. Nei paragrafi successivi daremo esempi più dettagliati di CPU, bus, e loro interfacce.

FPGA

Come abbiamo visto nel Capitolo 1, gli **FPGA** (*field-programmable gate array*) sono circuiti integrati che contengono una logica programmabile e permettono di formare un circuito logico arbitrario semplicemente caricando l'FPGA con i dati di configurazione appropriati. Il vantaggio principale di questi circuiti è la possibilità di costruire nuovi circuiti hardware in poche ore, piuttosto che attendere i mesi necessari per fabbricare circuiti integrati. Nonostante ciò, questi ultimi non sono sul viale del tramonto, essendo ancora in possesso di un significativo vantaggio di costo rispetto agli FPGA sui grandi volumi; sono inoltre più veloci e consumano meno energia. A causa dei loro vantaggi in fase di progettazione, tuttavia, gli FPGA vengono spesso utilizzati per la prototipazione e la progettazione di applicazioni su piccola scala.

Guardiamo ora all'interno di un FPGA e cerchiamo di capire come può essere utilizzato per implementare una vasta gamma di circuiti logici. Il chip FPGA contiene due componenti principali che vengono replicati più volte: le LUT (*Look-Up Table*, tabelle di ricerca) e le **interconnessioni programmabili**. Esaminiamo il modo in cui questi componenti vengono utilizzati.

Una LUT, mostrata nella Figura 3.33(a), è una piccola memoria programmabile che produce un segnale in uscita che viene poi trasmesso alla interconnessione programmabile ed eventualmente a un registro. La memoria programmabile viene usata per creare una funzione logica arbitraria. La LUT nella figura ha una memoria 16 x 4 che può emulare qualsiasi circuito logico con 4 bit di ingresso e 4 bit di uscita. La programmazione della LUT richiede il caricamento nella memoria delle risposte appropriate della logica combinatoria che deve essere emulata. In altre parole, se la logica combinatoria produce il valore di Y per un dato ingresso X, il valore Y va scritto nella LUT all'indice X.

L'esempio nella Figura 3.33(b) mostra come una singola LUT a 4 ingressi potrebbe implementare un contatore a 3 bit con reset. Il contatore dell'esempio continua a contare aggiungendo uno (modulo 4) al valore corrente del contatore, a meno che il segnale di reset CLR venga asserito, nel qual caso il contatore riporta il suo valore a zero.

Per implementare il contatore dell'esempio, le quattro entry superiori della LUT sono poste a zero. Queste voci restituiscono il valore zero quando il contatore viene resettato. Quindi, il bit più significativo della LUT (I_3) rappresenta l'ingresso di reset (CLR), che è asserito con un valore logico 1. Nelle rimanenti voci della LUT il valore di indice I_0..3 contiene (I + 1) modulo 4. Per completare il progetto, i segnali in uscita O_0..3 devono essere collegati, tramite l'interconnessione programmabile, ai segnali interni di ingresso I_0..3.

Per capire meglio il contatore con reset realizzato con un FPGA, consideriamo le operazioni che esegue. Se, per esempio, lo stato corrente del contatore è 2 e il segnale di reset non è asserito, l'indirizzo di input della LUT sarà 2 e l'output verso i flip-flop sarà di conseguenza 3. Se, nello stesso stato, il segnale di reset (CLR) viene asserito, l'input della LUT sarà 6 e quindi lo stato successivo sarà 0.

Può sembrare, tutto sommato, un modo arcano per costruire un contatore con reset e in effetti un progetto completamente personalizzato con un circuito contatore e segnali di ripristino al flip-flop sarebbe più piccolo, più veloce, e consumerebbe meno energia.

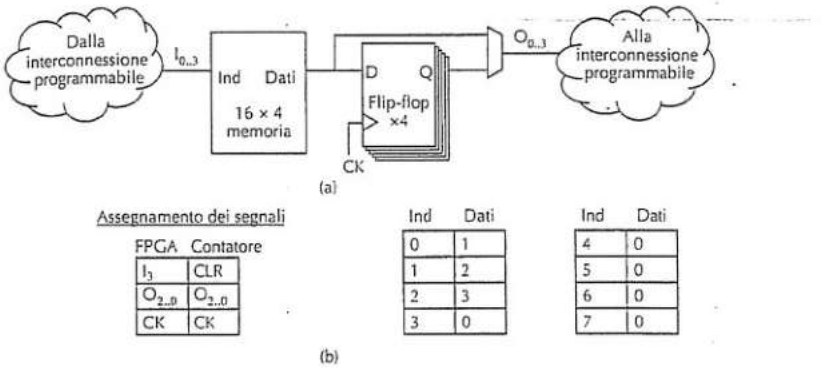


Figura 3.33 (a) La LUT di un FPGA. (b) La configurazione della LUT per la realizzazione di un contatore a 3 bit con reset.

Il vantaggio principale del progetto basato su FPGA è che si può costruire artigianalmente a casa in un'ora, mentre il più efficiente design completamente personalizzato deve essere fabbricato a partire dal silicio e l'operazione potrebbe richiedere anche più di un mese.

Per utilizzare un FPGA il progetto deve essere descritto con una rappresentazione del circuito o con un linguaggio di descrizione hardware (cioè un linguaggio di programmazione usato per descrivere strutture hardware). Il progetto viene poi elaborato da un sintetizzatore, che mappa il circuito su un'architettura FPGA specifica. Succede spesso che il progetto che si desidera realizzare non possa essere mappato sull'FGPA. Gli FPGA sono realizzati con un numero variabile di LUT e quelli che ne hanno di più costano di più. In generale, se il vostro progetto non si adatta all'FPGA, è necessario semplificarlo o rinunciare ad alcune funzionalità, oppure acquistare un FPGA più grande (e più costoso). Progetti molto grandi potrebbero non trovare spazio nemmeno nel più grande FPGA. In questo caso il progettista dovrà mappare il progetto in più FPGA; questo compito è sicuramente più difficile, ma è ancora una passeggiata rispetto alla progettazione di un circuito integrato personalizzato.

3.4 Chip della CPU e bus

Forti delle conoscenze acquisite sui circuiti integrati, sui clock e sui circuiti di memoria possiamo ora cominciare a mettere insieme i vari pezzi per affrontare il sistema nel suo complesso. In questo paragrafo analizzeremo inizialmente alcuni aspetti generali delle CPU dal punto di vista del livello logico digitale, **contatti** compresi (significato dei segnali sui singoli pin), e forniremo anche un'introduzione all'architettura dei bus, dato che le CPU dipendono strettamente dal modo in cui questi sono progettati. Nei paragrafi successivi daremo esempi più dettagliati di CPU, bus, e loro interfacce.

3.4.1 Chip della CPU

Tutte le CPU moderne sono contenute in un unico chip, rendendo in questo modo ben definita l'interazione con il resto del sistema. Ogni chip di CPU ha un insieme di pin attraverso il quale passano tutte le relative comunicazioni verso il mondo esterno. Alcuni pin spediscono i segnali della CPU, altri ricevono quelli del mondo esterno e altri ancora possono fare entrambe le cose. Conoscendo la funzione di tutti i pin possiamo capire in quale modo la CPU interagisce con la memoria e i dispositivi di I/O nel livello logico digitale.

I pin di una CPU possono essere divisi in tre tipi: indirizzi, dati e controlli. Questi pin sono collegati ad analoghi pin presenti sulla memoria, o sui chip di I/O, mediante un insieme di cavi paralleli chiamato bus. La CPU, per prelevare un'istruzione, ne imposta l'indirizzo sui suoi pin di indirizzamento, e poi asserisce una o più linee di controllo per informare la memoria che vuole leggere una parola. La memoria risponde spedendo la parola richiesta sui pin della CPU dedicati ai dati e asserendo un segnale che notifica che l'operazione è stata completata. Quando la CPU vede questo segnale accetta la parola ed esegue l'istruzione.

Se l'istruzione richiede la lettura o la scrittura di certi dati, l'intero processo viene ripetuto per ogni parola di dati. In seguito entreremo nei dettagli, ma per il momento è importante capire che la CPU comunica con la memoria e i dispositivi di I/O inviando e accettando segnali sui propri pin. Non esiste altro tipo di comunicazione.

Due dei principali parametri che determinano le prestazioni di una CPU sono il numero di pin d'indirizzo e il numero di pin di dati. Un chip con m pin d'indirizzo può indirizzare fino a 2^m locazioni di memoria; valori comuni di m sono 16, 20, 32 e 64. Analogamente un chip con n pin di dati può leggere o scrivere in un'unica operazione una parola a n bit (valori comuni di n sono 8, 32 e 64). Una CPU con 8 pin di dati dovrà eseguire quattro operazioni per leggere una parola a 32 bit, mentre una CPU con 32 pin di dati può compiere lo stesso lavoro in una sola operazione. Il chip con 32 pin di dati è quindi molto più veloce, ma molto più costoso.

Oltre ai pin d'indirizzo e dei dati, le CPU sono dotate anche di alcuni pin di controllo. Questi regolano il flusso e la temporizzazione dei dati da e verso la CPU, e possono essere utilizzati anche in vari altri modi. Tutte le CPU hanno pin per l'alimentazione (generalmente da +1,2 a +1,5 volt) e per la terra, oltre a un segnale di clock (un'onda quadra con una particolare frequenza); gli altri pin variano invece in modo considerevole da chip a chip. Ciononostante i pin di controllo possono essere approssimativamente raggruppati nelle seguenti categorie principali:

1. controllo del bus;
2. interrupt;
3. arbitraggio del bus;
4. comunicazione con il coprocessore;
5. stato;
6. altro.

In seguito descriveremo brevemente queste categorie e forniremo maggiori dettagli quando studieremo i chip Intel Core i7, TI OMAP4430 e Atmel ATmega168. La Figura 3.34 mostra un generico chip di una CPU che usa questi gruppi di segnali.

I pin per il controllo del bus mandano principalmente sul bus dei segnali dalla CPU (destinati alla memoria e ai chip di I/O) per notificare quando la CPU vuole leggere dalla o scrivere in memoria oppure compiere altre azioni. La CPU utilizza questi pin per controllare il resto del sistema e comunicargli quali operazioni intende compiere.

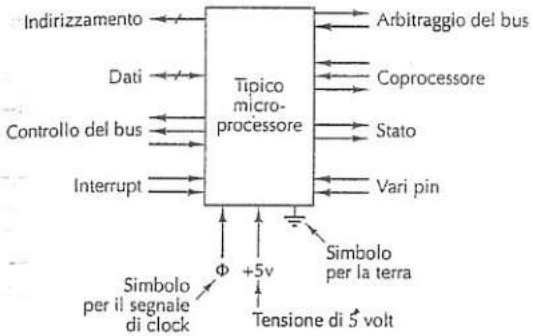


Figura 3.34 Una generica CPU. Le frecce indicano i segnali di input e di output. I trattini diagonali indicano pin multipli; per una specifica CPU un valore ne indica la quantità.

I pin di interrupt sono degli input che giungono alla CPU dalle periferiche. Nella maggior parte dei sistemi la CPU può comunicare a un dispositivo di I/O l'inizio di un'operazione e, mentre questo porta avanti il proprio lavoro, passa a eseguire qualche altra operazione. Non appena il dispositivo ha completato la propria operazione asserisce un segnale su uno di questi pin in modo da interrompere la CPU e permetterle di utilizzare il dispositivo, per esempio per controllare se si siano verificati degli errori di I/O. Alcune CPU hanno un pin di output per spedire una conferma in risposta a un segnale di interrupt.

I pin per l'arbitraggio del bus sono necessari per regolare il traffico sul bus, allo scopo di evitare che due dispositivi cerchino di usarlo nello stesso momento. Ai fini dell'arbitraggio la CPU conta quanto un altro dispositivo e anch'essa deve fare esplicita richiesta di utilizzo del bus.

Alcune CPU sono progettate per funzionare con coprocessori come i chip in virgola mobile oppure, in alcuni casi, i chip grafici o di altro tipo. Per facilitare la comunicazione tra CPU e coprocessore sono disponibili pin speciali per ricevere e soddisfare vari tipi di richieste.

Oltre a questi segnali alcune CPU possono o per resettare il calcolatore o effettuare operazioni di debugging, avere altri pin per altri scopi, per esempio per fornire o ricevere informazioni di stato, o ancora per garantire compatibilità con chip di I/O più vecchi.

3.4.2 Bus del calcolatore

Un **bus** è un collegamento elettrico che unisce diversi dispositivi. I bus possono essere classificati in base alla loro funzione; alcuni di loro sono impiegati internamente alla CPU per trasferire dati da e verso la ALU, mentre altri sono esterni alla CPU e servono a connetterla con la memoria o con altri dispositivi di I/O. Ciascun tipo di bus soddisfa certi requisiti e gode di proprietà specifiche. In questo paragrafo e nei successivi ci concentreremo sui bus che connettono la CPU alla memoria e ai dispositivi di I/O, mentre nel prossimo capitolo esamineremo in modo più dettagliato i bus interni alla CPU.

I primi PC avevano un unico bus esterno chiamato anche **bus di sistema**. Esso era composto da 50 a 100 fili paralleli di rame che si inserivano nella scheda madre e i cui connettori erano distanziati a intervalli regolari per permettere l'inserimento di memorie e schede di I/O. Generalmente i personal computer moderni hanno invece un bus specifico tra la CPU e la memoria e (almeno) un altro bus per le periferiche. La Figura 3.35 mostra un sistema minimale, composto da un bus di memoria e un bus di I/O.

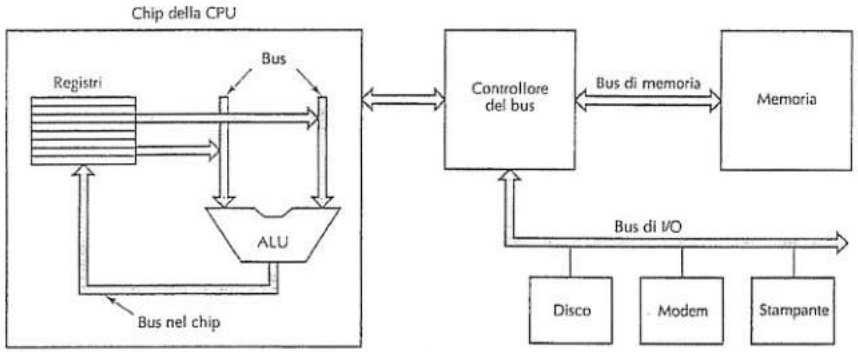


Figura 3.35 Sistema di un calcolatore con più bus.

Come spesso avviene in letteratura i bus sono stati disegnati nella figura come frecce "larghe". Vi è una sottile differenza tra una linea spessa e una linea intersecata da un trattino diagonale vicino al quale si legge un numero (di bit). Quando tutti i bit sono dello stesso tipo, cioè sono, per esempio, bit d'indirizzo oppure bit di dati, si usa comunemente il trattino diagonale. Quando invece sono coinvolte linee d'indirizzo, di dati e di controllo si utilizza con più frequenza la freccia spessa.

Mentre i progettisti della CPU sono liberi di utilizzare all'interno del chip il tipo di bus che preferiscono, per i bus esterni bisogna invece definire delle precise regole di funzionamento, che devono essere rispettate dai dispositivi a loro collegati. La cosa consente che anche schede progettate da altri produttori possano collegarsi correttamente al bus. L'insieme di queste regole è detto **protocollo del bus**. Inoltre devono essere definite le specifiche meccaniche ed elettriche in modo che le schede prodotte da terze parti abbiano le dimensioni corrette e i loro connettori siano meccanicamente compatibili con quelli della scheda madre in termini di tensione, temporizzazione e così via. Altri bus non dispongono di specifiche meccaniche, perché sono progettati per essere utilizzati esclusivamente all'interno di un circuito integrato, per esempio per connettere tra loro le componenti all'interno di un SoC (*system-on-a-chip*).

Nel mondo dei calcolatori esiste un gran numero di bus ampiamente utilizzati. Alcuni fra i più conosciuti, sia attuali sia d'importanza storica, sono i seguenti (tra parentesi sono indicati i calcolatori che li hanno adottati): l'Omnibus (PDP-8), l'Unibus (PDP-11), il Multibus (8086), il bus VME (apparecchiature per i laboratori di fisica), il bus PC IBM (PC/XT), il bus ISA (PC/AT), il bus EISA (80386), il Microchannel (PS/2), il Nubus (Macintosh), il bus PCI (molti PC), il bus SCSI (molti PC e workstation), l'Universal Serial Bus (PC moderni) e il FireWire (elettronica di consumo). Il mondo sarebbe probabilmente migliore se tutti, tranne uno, sparissero di colpo dalla faccia della terra (d'accordo, è un po' troppo, ma che cosa dire se ne scomparissero tutti tranne due?). Sfortunatamente sembra molto difficile che avvenga una standardizzazione in quest'area, in quanto sono stati già fatti troppi investimenti in tutti questi sistemi, fra loro incompatibili.

Master	Slave	Esempio
CPU	Memoria	Prelievo delle istruzioni e dei dati
CPU	Dispositivo di I/O	Inizio del trasferimento dei dati
CPU	Coprocessore	Passaggio dell'istruzione al coprocessore da parte della CPU
I/O	Memoria	DMA (Direct Memory Access)
Coprocessore	CPU	Prelievo degli operandi dalla CPU da parte del coprocessore

Figura 3.36 Esempi di master e slave del bus.

Cominciamo il nostro studio partendo dal funzionamento dei bus. Alcune periferiche che si collegano al bus sono attive e possono iniziare un trasferimento dati, mentre altre sono passive e restano in attesa di una richiesta. Quelle attive sono chiamate ***master***, e quelle passive ***slave***. Quando la CPU ordina al controllore di un disco di leggere o scrivere un blocco, svolge il ruolo di master, e il controllore del disco quello di slave. Successivamente però il controllore del disco fungerà da master nel momento in cui ordina alla memoria di accettare le parole che sta leggendo dal disco. La Figura 3.36 elenca alcune tipiche combinazioni di master e slave. La memoria non può mai fungere da master.

Molto spesso i segnali digitali generati dalle periferiche sono troppo deboli per alimentare un bus, soprattutto se è relativamente lungo o se è collegato a molti dispositivi. Per questo motivo molti master sono connessi al bus mediante un chip chiamato **driver del bus**, che funge essenzialmente da amplificatore digitale; in modo analogo la maggior parte degli slave sono connessi al bus attraverso un **ricevitore del bus**. Per le periferiche che possono svolgere sia il ruolo di master sia quello di slave si utilizza un chip

chiamato **trasmettitore-ricevitore del bus**. Spesso questi chip d'interfaccia sono dei dispositivi a tre stati per permettere loro di essere liberi (sconnessi) quando non sono necessari; un'altra possibilità, che permette di ottenere lo stesso effetto, consiste nell'agganciare la periferica al bus tramite un **collettore aperto**. Quando due o più dispositivi su una linea a collettore aperto asseriscono la linea nello stesso istante, il risultato è un OR di tutti i segnali; spesso questa organizzazione è chiamata **OR-cablata**. Nella maggior parte dei bus alcune linee sono a tre stati mentre altre, che richiedono la proprietà di essere OR-cablate, sono a collettore aperto.

Allo stesso modo della CPU anche il bus ha i propri indirizzi, i propri dati e le proprie linee di controllo. Ciononostante non è necessario che vi sia una corrispondenza uno-a-uno tra i pin della CPU e i segnali del bus. Alcune CPU hanno per esempio tre pin che codificano se si sta effettuando una lettura o una scrittura da memoria, una lettura o una scrittura di I/O oppure un'altra operazione. Un normale bus potrebbe avere una linea per la lettura da memoria, una seconda per la scrittura in memoria, una terza per la lettura da periferica, una quarta per la scrittura su periferica e così via. In questo caso si renderebbe necessario un decodificatore tra la CPU e il bus per far corrispondere le due estremità, cioè per convertire il segnale codificato in 3 bit in segnali distinti che possono essere spediti sulle varie linee del bus.

Il progetto e il funzionamento dei bus sono argomenti piuttosto complessi, sono trattati in vari libri (Anderson et al., 2004; Solari e Willse, 2004). Le principali decisioni da prendere nella progettazione di un bus riguardano l'ampiezza, la temporizzazione, l'arbitraggio e le operazioni che consente. Ciascuna di queste scelte ha un impatto significativo sulla velocità e sulla larghezza di banda del bus.

3.4.3 Ampiezza del bus

Nella progettazione dei bus il parametro più scontato da considerare è la sua ampiezza. Maggiore è il numero di linee d'indirizzo di un bus, maggiore sarà la quantità di memoria che la CPU potrà indirizzare direttamente. Se un bus ha n linee d'indirizzi, una CPU può indirizzare 2^n diverse locazioni di memoria. La cosa sembra piuttosto semplice.

Il problema è che bus più larghi richiedono un numero maggiore di fili rispetto a quelli più stretti. Questo significa anche una maggiore occupazione di spazio (per esempio sulla scheda madre) e la necessità di utilizzare connettori più grandi. Dato che tutti questi fattori rendono il bus più costoso occorre trovare un compromesso tra la dimensione massima di memoria e il costo del sistema. Un sistema con 64 linee d'indirizzi e 2^{32} byte di memoria costerà più di uno con 32 linee d'indirizzi e 2^{32} byte di memoria, ma nel secondo caso non è possibile espandere la memoria in un secondo momento.

Il risultato di questa osservazione è che molti progettisti di sistemi tendono a essere poco lungimiranti, con conseguenze negative. Il PC IBM originario conteneva una CPU 8088 e un bus con indirizzi a 20 bit, come mostrato nella Figura 3.37(a); con 20 bit a disposizione il PC poteva indirizzare soltanto 1 MB di memoria.

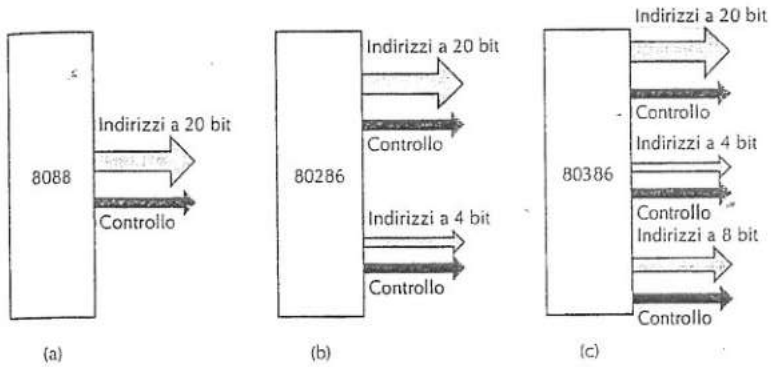


Figura 3.37 Crescita nel tempo degli indirizzi del bus.

Con la CPU successiva, l'80286, Intel decise di aumentare lo spazio degli indirizzi a 16 MB; questo aumento richiese l'aggiunta di quattro linee di bus (senza modificare le 20 precedenti per ragioni di retrocompatibilità), come mostra la Figura 3.37(b). Purtroppo fu necessario aggiungere anche alcune nuove linee di controllo per gestire le nuove linee d'indirizzo. Quando apparve l'80386 furono aggiunte altre otto linee d'indirizzo oltre alle linee di controllo aggiuntive, come mostra la Figura 3.37(c). Il progetto risultante (il bus EISA) è molto meno ordinato di come sarebbe potuto essere se si fosse partiti da zero avendo a disposizione fin dall'inizio tutte e 32 le linee.

Con il tempo, non solo tende a crescere il numero delle linee d'indirizzo, ma anche il numero delle linee di dati, seppur per una ragion diversa. Esistono due modi per aumentare la larghezza di banda dei dati su un bus: diminuire il periodo di clock del bus (più trasferimenti/s) oppure aumentare la larghezza dei dati del bus. Un'altra possibilità è quella di aumentare la velocità del bus, anche se ciò pone alcune difficoltà; i segnali su linee distinte viaggiano infatti a velocità leggermente diverse. Questo problema è conosciuto come **disallineamento del bus**, e più il bus è veloce più questo problema diventa marcato.

Un altro problema che sorge nel caso in cui si intenda velocizzare il bus è la perdita della retrocompatibilità. Schede progettate per bus più lenti non funzioneranno con il nuovo bus. Rendere inutilizzabili le vecchie schede non rende di certo felici né i proprietari di queste schede né i loro produttori. Per questa ragione l'approccio più utilizzato è quello di aggiungere nuove linee di dati, come mostrato nella Figura 3.37. In ogni caso, come ci si potrebbe aspettare, questa crescita incrementale non genera come risultato un'architettura semplice e ordinata. I PC IBM e i loro successori, per fare un esempio, sono passati da otto linee di dati a 16 e, in seguito, a 32 su un bus che è rimasto essenzialmente dello stesso tipo.

Per aggirare il problema di bus troppo ampi a volte i progettisti optano per un **bus multiplexato**. In questa architettura invece di tenere separate le linee d'indirizzo e quelle dei dati, si utilizza un certo numero di linee, per esempio 32, per entrambi. All'inizio

Questa azione inserisce alcuni **stati di attesa** (cicli di bus addizionali) finché la memoria non completi l'operazione e neghi $\overline{\text{WAIT}}$. Nell'esempio è stato inserito uno stato di attesa (T_2), dato che la memoria è troppo lenta. All'inizio di T_3 la memoria nega WAIT, in quanto è sicuro che i dati saranno disponibili nel corso del ciclo corrente.

Durante la prima metà di T_3, la memoria mette i dati sulle linee apposite. Nel fronte di discesa di T_3 la CPU consulta (cioè legge) le linee dei dati, memorizzando il valore in un suo registro. Dovendo leggere i dati la CPU nega $\overline{\text{MREQ}}$ e $\overline{\text{RD}}$. Se necessario può cominciare un altro ciclo di memoria nel successivo fronte di salita del clock. Questa sequenza può essere ripetuta indefinitamente.

Chiariamo ora il significato degli otto simboli presenti nel diagramma della Figura 3.38(b) che elenca le specifiche di temporizzazione. Per esempio, T_{AD} è l'intervallo di tempo tra il fronte di salita del ciclo di clock T_1 e il momento in cui vengono impostate le linee d'indirizzo. Secondo le specifiche di temporizzazione, $T_{AD} \leq 4$ ns. Questo significa che il produttore della CPU garantisce che durante ogni ciclo di lettura la CPU manderà in output l'indirizzo da leggere entro 4 ns dal punto medio del fronte di salita di T_1.

Le specifiche di temporizzazione richiedono inoltre che i dati siano disponibili sulle linee almeno T_{DS} ns (cioè 2 ns) prima del fronte di discesa di T_3. La cosa è necessaria per dare il tempo alla linea di stabilizzarsi prima che la CPU legga. La combinazione dei vincoli su T_{AD} e T_{DS} fa sì che, nel caso peggiore, la memoria abbia a disposizione soltanto $25 - 4 - 2 = 19$ ns tra il momento in cui appare l'indirizzo e quello in cui deve fornire i dati. Dato che sono sufficienti 15 ns, anche nel caso peggiore una memoria a 15 ns è sempre in grado di fornire la risposta durante il ciclo T_3. Al contrario una memoria a 20 ns potrebbe non farcela in tempo e in tal caso dovrebbe inserire un secondo stato di attesa e rispondere durante T_4.

Le specifiche di temporizzazione garantiscono inoltre che l'indirizzo venga impostato almeno 2 ns prima che $\overline{\text{MREQ}}$ sia asserito. Questo intervallo temporale può essere significativo nel caso in cui $\overline{\text{MREQ}}$ piloti la selezione del chip sulla memoria; questo poiché alcune memorie richiedono un certo tempo per l'impostazione dell'indirizzo prima di selezionare il chip. È chiaro che un progettista di CPU non dovrebbe scegliere un chip di memoria che richiede un tempo di setup di 3 ns.

I vincoli su T_M e T_{RL} fanno sì che $\overline{\text{MREQ}}$ e $\overline{\text{RD}}$ vengano asseriti entro 3 ns dalla discesa del clock T_1. Nel caso peggiore il chip di memoria, per mettere i propri dati sul bus, avrà a disposizione solo $10 + 10 - 3 - 2 = 15$ ns dopo che $\overline{\text{MREQ}}$ e $\overline{\text{RD}}$ sono stati asseriti. Questo vincolo si aggiunge (e ne è indipendente) all'attesa di 15 ns necessaria dopo che l'indirizzo è stabile.

T_{MH} e T_{RH} indicano quanto tempo impiegano $\overline{\text{MREQ}}$ e $\overline{\text{RD}}$ per essere negati dopo che i dati sono stati generati. Infine T_{DH} specifica per quanto tempo la memoria deve mantenere i dati sul bus dopo che $\overline{\text{RD}}$ è stato negato. Per quanto riguarda il nostro esempio di CPU, la memoria può rimuovere i dati dal bus non appena viene negato $\overline{\text{RD}}$; tuttavia su alcune CPU reali i dati devono essere mantenuti stabili per un tempo leggermente più lungo.

È importante sottolineare che la Figura 3.38 è una versione decisamente semplificata dei reali vincoli di temporizzazione. Anche se nella realtà vengono specificati molti altri tempi critici, l'esempio riesce tuttavia a dare l'idea del funzionamento di un bus sincrono.

Un altro punto che vale la pena precisare è che i segnali di controllo possono essere asseriti con valore alto o con valore basso. Tocca al progettista del bus determinare quale dei due sia più conveniente, e la scelta è essenzialmente arbitraria. Questa decisione potrebbe essere vista come l'equivalente, in hardware, della scelta che un programmatore compie quando deve decidere se rappresentare un settore vuoto del disco mediante valori 0 o valori 1.

Bus asincroni

Lavorare con i bus sincroni è facile per via dei loro intervalli temporali discreti; ciononostante presentano alcuni problemi. Per fare un esempio, qualsiasi operazione sul bus si svolge in tempi multipli del clock del bus; se una CPU e una memoria sono in grado di completare un trasferimento in 3,1 cicli, sono tuttavia obbligate ad allungare il tempo necessario a 4 cicli, dato che non sono consentite frazioni di cicli.

Inoltre, una volta scelto un ciclo di bus e appositamente costruite le memorie e le schede di I/O, è difficile trarre vantaggio dai successivi sviluppi della tecnologia. Supponiamo per esempio che alcuni anni dopo la realizzazione del sistema della Figura 3.38 si rendano disponibili nuove memorie con tempi di accesso di 8 ns invece che di 15 ns. Con la nuova memoria si eliminerebbe il tempo di attesa, accelerando di conseguenza la macchina. Supponiamo ora che diventino disponibili memorie a 4 ns; ciò non porterebbe ad alcun ulteriore guadagno in termini di prestazioni, poiché con questa architettura il tempo minimo per una lettura è di due cicli. In altri termini, se un bus sincrono ha un insieme eterogeneo di dispositivi, alcuni veloci, altri più lenti, il bus deve essere regolato alla velocità della periferica più lenta, e le altre, pur essendo più veloci, non possono sfruttare tutto il loro potenziale.

È possibile gestire tecnologie caratterizzate da prestazioni diverse mediante l'utilizzo di un bus asincrono, cioè di un bus che non possiede un clock principale, come mostra la Figura 3.39. Il master del bus, invece di legare ogni operazione al clock, dopo aver asserito l'indirizzo, $\overline{\text{MREQ}}$, $\overline{\text{RD}}$ e tutto ciò che gli necessita, asserisce uno speciale segnale che chiameremo $\overline{\text{MSYN}}$ (*Master SYNchronization*). Quando lo slave vede questo segnale esegue il lavoro richiesto alla massima velocità possibile; una volta terminato il proprio compito asserisce $\overline{\text{SSYN}}$ (*Slave SYNchronization*).

Quando il master vede che $\overline{\text{SSYN}}$ è stato asserito, sa che i dati sono disponibili e può quindi memorizzarli; successivamente nega le linee d'indirizzo, $\overline{\text{MREQ}}$, $\overline{\text{RD}}$ e $\overline{\text{MSYN}}$. Quando lo slave vede la negazione di $\overline{\text{MSYN}}$, sa che il ciclo è stato completato e quindi nega SSYN; a questo punto si è tornati alla situazione iniziale, nella quale tutti i segnali sono negati e si attende il master successivo.

Come mostra la Figura 3.39 i diagrammi di temporizzazione dei bus asincroni (e talvolta anche dei bus sincroni) usano frecce per indicare cause ed effetti. L'asserzione di MSYN ha come effetto l'asserzione delle linee dei dati e il fatto che lo slave asserisce $\overline{\text{SSYN}}$. L'asserzione di $\overline{\text{SSYN}}$ causa, a sua volta, l'asserzione delle linee d'indirizzo, di $\overline{\text{MREQ}}$, di $\overline{\text{RD}}$ e di $\overline{\text{MSYN}}$. La negazione di MSYN provoca infine la negazione di $\overline{\text{SSYN}}$, che termina la lettura e riporta il sistema al suo stato originario.

Questa azione inserisce alcuni **stati di attesa** (cicli di bus addizionali) finché la memoria non completi l'operazione e neghi $\overline{\text{WAIT}}$. Nell'esempio è stato inserito uno stato di attesa (T_2), dato che la memoria è troppo lenta. All'inizio di T_3 la memoria nega WAIT, in quanto è sicuro che i dati saranno disponibili nel corso del ciclo corrente.

Durante la prima metà di T_3, la memoria mette i dati sulle linee apposite. Nel fronte di discesa di T_3 la CPU consulta (cioè legge) le linee dei dati, memorizzando il valore in un suo registro. Dovendo leggere i dati la CPU nega $\overline{\text{MREQ}}$ e $\overline{\text{RD}}$. Se necessario può cominciare un altro ciclo di memoria nel successivo fronte di salita del clock. Questa sequenza può essere ripetuta indefinitamente.

Chiariamo ora il significato degli otto simboli presenti nel diagramma della Figura 3.38(b) che elenca le specifiche di temporizzazione. Per esempio, T_{AD} è l'intervallo di tempo tra il fronte di salita del ciclo di clock T_1 e il momento in cui vengono impostate le linee d'indirizzo. Secondo le specifiche di temporizzazione, $T_{AD} \leq 4$ ns. Questo significa che il produttore della CPU garantisce che durante ogni ciclo di lettura la CPU manderà in output l'indirizzo da leggere entro 4 ns dal punto medio del fronte di salita di T_1.

Le specifiche di temporizzazione richiedono inoltre che i dati siano disponibili sulle linee almeno T_{DS} ns (cioè 2 ns) prima del fronte di discesa di T_3. La cosa è necessaria per dare il tempo alla linea di stabilizzarsi prima che la CPU legga. La combinazione dei vincoli su T_{AD} e T_{DS} fa sì che, nel caso peggiore, la memoria abbia a disposizione soltanto $25 - 4 - 2 = 19$ ns tra il momento in cui appare l'indirizzo e quello in cui deve fornire i dati. Dato che sono sufficienti 15 ns, anche nel caso peggiore una memoria a 15 ns è sempre in grado di fornire la risposta durante il ciclo T_3. Al contrario una memoria a 20 ns potrebbe non farcela in tempo e in tal caso dovrebbe inserire un secondo stato di attesa e rispondere durante T_4.

Le specifiche di temporizzazione garantiscono inoltre che l'indirizzo venga impostato almeno 2 ns prima che $\overline{\text{MREQ}}$ sia asserito. Questo intervallo temporale può essere significativo nel caso in cui $\overline{\text{MREQ}}$ piloti la selezione del chip sulla memoria; questo poiché alcune memorie richiedono un certo tempo per l'impostazione dell'indirizzo prima di selezionare il chip. È chiaro che un progettista di CPU non dovrebbe scegliere un chip di memoria che richiede un tempo di setup di 3 ns.

I vincoli su T_M e T_{RL} fanno sì che $\overline{\text{MREQ}}$ e $\overline{\text{RD}}$ vengano asseriti entro 3 ns dalla discesa del clock T_1. Nel caso peggiore il chip di memoria, per mettere i propri dati sul bus, avrà a disposizione solo $10 + 10 - 3 - 2 = 15$ ns dopo che $\overline{\text{MREQ}}$ e $\overline{\text{RD}}$ sono stati asseriti. Questo vincolo si aggiunge (e ne è indipendente) all'attesa di 15 ns necessaria dopo che l'indirizzo è stabile.

T_{MH} e T_{RH} indicano quanto tempo impiegano $\overline{\text{MREQ}}$ e $\overline{\text{RD}}$ per essere negati dopo che i dati sono stati generati. Infine T_{DH} specifica per quanto tempo la memoria deve mantenere i dati sul bus dopo che $\overline{\text{RD}}$ è stato negato. Per quanto riguarda il nostro esempio di CPU, la memoria può rimuovere i dati dal bus non appena viene negato $\overline{\text{RD}}$; tuttavia su alcune CPU reali i dati devono essere mantenuti stabili per un tempo leggermente più lungo.

È importante sottolineare che la Figura 3.38 è una versione decisamente semplificata dei reali vincoli di temporizzazione. Anche se nella realtà vengono specificati molti altri tempi critici, l'esempio riesce tuttavia a dare l'idea del funzionamento di un bus sincrono.

Un altro punto che vale la pena precisare è che i segnali di controllo possono essere asseriti con valore alto o con valore basso. Tocca al progettista del bus determinare quale dei due sia più conveniente, e la scelta è essenzialmente arbitraria. Questa decisione potrebbe essere vista come l'equivalente, in hardware, della scelta che un programmatore compie quando deve decidere se rappresentare un settore vuoto del disco mediante valori 0 o valori 1.

Bus asincroni

Lavorare con i bus sincroni è facile per via dei loro intervalli temporali discreti; ciononostante presentano alcuni problemi. Per fare un esempio, qualsiasi operazione sul bus si svolge in tempi multipli del clock del bus; se una CPU e una memoria sono in grado di completare un trasferimento in 3,1 cicli, sono tuttavia obbligate ad allungare il tempo necessario a 4 cicli, dato che non sono consentite frazioni di cicli.

Inoltre, una volta scelto un ciclo di bus e appositamente costruite le memorie e le schede di I/O, è difficile trarre vantaggio dai successivi sviluppi della tecnologia. Supponiamo per esempio che alcuni anni dopo la realizzazione del sistema della Figura 3.38 si rendano disponibili nuove memorie con tempi di accesso di 8 ns invece che di 15 ns. Con la nuova memoria si eliminerebbe il tempo di attesa, accelerando di conseguenza la macchina. Supponiamo ora che diventino disponibili memorie a 4 ns; ciò non porterebbe ad alcun ulteriore guadagno in termini di prestazioni, poiché con questa architettura il tempo minimo per una lettura è di due cicli. In altri termini, se un bus sincrono ha un insieme eterogeneo di dispositivi, alcuni veloci, altri più lenti, il bus deve essere regolato alla velocità della periferica più lenta, e le altre, pur essendo più veloci, non possono sfruttare tutto il loro potenziale.

È possibile gestire tecnologie caratterizzate da prestazioni diverse mediante l'utilizzo di un bus asincrono, cioè di un bus che non possiede un clock principale, come mostra la Figura 3.39. Il master del bus, invece di legare ogni operazione al clock, dopo aver asserito l'indirizzo, $\overline{\text{MREQ}}$, $\overline{\text{RD}}$ e tutto ciò che gli necessita, asserisce uno speciale segnale che chiameremo $\overline{\text{MSYN}}$ (*Master SYNchronization*). Quando lo slave vede questo segnale esegue il lavoro richiesto alla massima velocità possibile; una volta terminato il proprio compito asserisce $\overline{\text{SSYN}}$ (*Slave SYNchronization*).

Quando il master vede che $\overline{\text{SSYN}}$ è stato asserito, sa che i dati sono disponibili e può quindi memorizzarli; successivamente nega le linee d'indirizzo, $\overline{\text{MREQ}}$, $\overline{\text{RD}}$ e $\overline{\text{MSYN}}$. Quando lo slave vede la negazione di $\overline{\text{MSYN}}$, sa che il ciclo è stato completato e quindi nega SSYN; a questo punto si è tornati alla situazione iniziale, nella quale tutti i segnali sono negati e si attende il master successivo.

Come mostra la Figura 3.39 i diagrammi di temporizzazione dei bus asincroni (e talvolta anche dei bus sincroni) usano frecce per indicare cause ed effetti. L'asserzione di MSYN ha come effetto l'asserzione delle linee dei dati e il fatto che lo slave asserisce $\overline{\text{SSYN}}$. L'asserzione di $\overline{\text{SSYN}}$ causa, a sua volta, l'asserzione delle linee d'indirizzo, di $\overline{\text{MREQ}}$, di $\overline{\text{RD}}$ e di $\overline{\text{MSYN}}$. La negazione di MSYN provoca infine la negazione di $\overline{\text{SSYN}}$, che termina la lettura e riporta il sistema al suo stato originario.

una linea per effettuare la richiesta e una per segnalare la concessione. Il bus della Figura 3.40(b) ha due livelli, 1 e 2 (i bus reali possono averne 4, 8 o 16). A ciascun dispositivo viene assegnato un livello per la richiesta del bus; a quelli per cui la temporizzazione è più critica vengono assegnate priorità più elevate. Nella Figura 3.40(b) i dispositivi 1, 2 e 4 usano la priorità 1, mentre i dispositivi 3 e 5 usano la priorità 2.

Se in uno stesso momento ci sono richieste da livelli di priorità diversi l'arbitro concede il bus a un dispositivo di priorità più elevata; fra dispositivi della stessa priorità si utilizza invece la strategia del collegamento a festone. Nella Figura 3.40(b) in caso di conflitti il dispositivo 2 ha priorità sul dispositivo 4, che ce l'ha a sua volta sul 3. Il dispositivo 5 ha invece la priorità più bassa fra tutti, in quanto si trova alla fine del collegamento a festone a priorità inferiore.

Aprendo una parentesi occorre precisare che non è tecnicamente necessario collegare serialmente i dispositivi 1 e 2 alla linea di concessione del bus di livello 2, dato che non possono effettuare delle richieste su tale linea. Tuttavia per ragioni d'implementazione è più facile collegare tutte le linee di concessione a tutti i dispositivi, piuttosto che instaurare dei collegamenti speciali dipendenti dalla priorità associata a ciascun dispositivo.

Alcuni arbitri hanno una terza linea che viene asserita da un dispositivo nel momento in cui accetta una concessione e si impadronisce del bus. Subito dopo aver asserito questa linea di conferma, il dispositivo può negare le linee di richiesta e di concessione. Il risultato è che altri dispositivi possono richiedere il bus mentre il primo lo sta ancora utilizzando. Nel momento in cui termina il trasferimento in corso, il successivo master del bus sarà già stato selezionato. Esso può cominciare subito dopo che la linea di conferma è stata negata, cioè nel momento in cui può iniziare il nuovo turno determinato dall'arbitraggio. Questo schema richiede una linea aggiuntiva nel bus e una maggior quantità di componenti logici in ciascun dispositivo, ma permette un miglior utilizzo dei cicli del bus.

Nei sistemi in cui la memoria è collegata al bus principale, la CPU deve competere con tutti i dispositivi di I/O praticamente a ogni ciclo. Una soluzione molto comune per questa situazione consiste nell'assegnare alla CPU la priorità più bassa in modo che possa usare il bus soltanto quando nessun altro lo vuole. L'idea sottostante è che la CPU può sempre aspettare, mentre molto spesso i dispositivi di I/O sono obbligati ad acquisire il bus molto velocemente, pena la perdita dei dati in arrivo. I dischi che ruotano ad alte velocità, per esempio, non possono aspettare. In molti calcolatori questo problema viene risolto mettendo la memoria su un bus separato rispetto ai dispositivi di I/O, in modo che la CPU non debba competere con questi per l'accesso al bus.

Un'altra soluzione possibile è la decentralizzazione dell'arbitraggio del bus. Un calcolatore potrebbe per esempio avere 16 linee di richiesta, ciascuna con la propria priorità. Quando un dispositivo vuole utilizzare il bus asserisce la linea di richiesta. Tutti i dispositivi monitorano tutte le linee di richiesta in modo che alla fine di ciascun ciclo del bus ognuno di loro può sapere se era il richiedente con priorità più elevata e se quindi ha diritto a utilizzare il bus durante il ciclo successivo. Rispetto al metodo centralizzato questo schema di arbitraggio richiede un maggior numero di linee di bus, ma evita

il potenziale costo dell'arbitro. Un altro limite è che il numero di dispositivi non può superare il numero delle linee di richiesta.

Un altro tipo di arbitraggio decentralizzato del bus, mostrato nella Figura 3.41, utilizza solamente tre linee, indipendentemente da quanti dispositivi sono presenti. La prima linea del bus è una linea OR-cablata per effettuare le richieste al bus. La seconda linea del bus è chiamata BUSY ed è asserita dal master corrente. La terza è invece utilizzata per arbitrare il bus ed è un collegamento a festone fra tutti i dispositivi; la testa di questa catena viene mantenuta asserita collegandola a un'alimentazione di 5 volt.

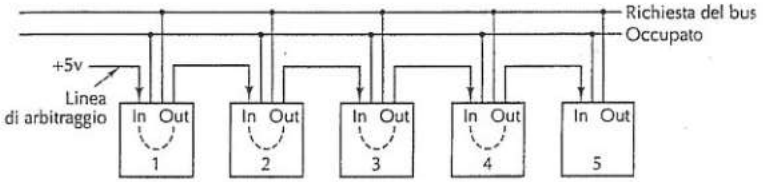


Figura 3.41 Arbitraggio decentralizzato del bus.

Quando nessun dispositivo richiede il bus la linea di arbitraggio, che è asserita, viene propagata attraverso tutti i dispositivi. Per acquisire il bus un dispositivo deve prima controllare se il bus è inattivo e se il segnale dell'arbitraggio che sta ricevendo, IN, è asserito. Se IN è negato, non può diventare il master e nega OUT. Se invece IN è asserito e il dispositivo vuole il bus, allora il dispositivo nega OUT in modo che il dispositivo successivo nel collegamento a festone veda IN negato e neghi anch'esso il proprio OUT. Tutti i dispositivi successivi vedranno pertanto IN negato e negheranno a loro volta OUT. Una volta tornata la calma solo un dispositivo rimarrà con IN asserito e OUT negato; esso diventerà il master del bus, asserirà BUSY e OUT e inizierà il proprio trasferimento.

Con un semplice ragionamento si deduce che fra i dispositivi che richiedono il bus lo ottiene quello che si trova più a sinistra. Questo schema è quindi simile all'originario arbitraggio con collegamento a festone, tranne il fatto che non c'è l'arbitro; questa differenza lo rende più economico, più veloce e non soggetto ai potenziali guasti dell'arbitro.

3.4.6 Operazioni del bus

Fino a questo momento abbiamo trattato solamente bus ordinari con un master (generalmente la CPU) che legge da uno slave (di solito la memoria) o scrive su di esso. Nei casi reali sono possibili anche altri tipi di bus che ora analizzeremo.

Di solito viene trasferita una parola alla volta. Tuttavia quando si utilizza la cache è preferibile prelevare in una volta sola un'intera linea di cache (per esempio 8 parole consecutive di 64 bit). Spesso i trasferimenti di blocchi possono essere effettuati in modo più efficiente rispetto a una sequenza di trasferimenti individuali. All'inizio della lettura di un blocco il master del bus comunica allo slave quante parole devono essere

trasferite inserendo, per esempio durante T_1, il conteggio delle parole sulle linee dei dati. Lo slave, invece di restituire un'unica parola, genera in output a ogni ciclo una nuova parola finché il contatore non si sia esaurito. La Figura 3.42 mostra una versione modificata della Figura 3.38(a) in cui un segnale aggiuntivo BLOCK viene asserito per indicare che è stato richiesto il trasferimento di un blocco. In questo esempio una lettura di un blocco di 4 parole richiede 6 cicli invece che 12.

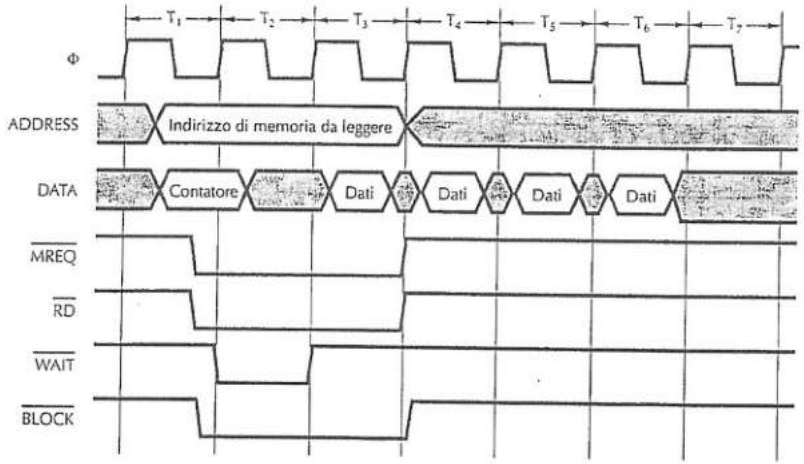


Figura 3.42 Trasferimento di un blocco.

Esistono anche altri tipi di cicli di bus. Per esempio su un sistema multiprocessore con due o più CPU sullo stesso bus è spesso necessario assicurarsi che, in un dato momento, soltanto una CPU utilizzi delle strutture dati critiche della memoria. Una tipica soluzione del problema consiste nell'avere in memoria una variabile che vale 0 quando nessuna CPU sta utilizzando la struttura dati e 1 quando invece è in uso. Se una CPU vuole acquisire l'accesso alla struttura dati deve leggere il valore della variabile e impostarla a 1, se vale 0. Il problema è che, con una buona dose di sfortuna, due CPU potrebbero leggere la variabile durante cicli di bus consecutivi. Se entrambe vedono che la variabile è 0 allora entrambe la imposteranno a 1 e penseranno di essere l'unica CPU che sta utilizzando la struttura dati. Questa sequenza di eventi porta al caos.

Per evitare questa situazione i sistemi multiprocessore hanno spesso uno speciale ciclo di bus leggi-modifica-scrivi che consente a una qualsiasi CPU di leggere una parola dalla memoria, analizzarla e riscriverla in memoria, tutto senza rilasciare il bus. Questo tipo di ciclo evita che altre CPU che intendono utilizzare il bus riescano a impadronirsene interferendo con l'operazione della prima CPU.

Un altro importante tipo di ciclo di bus serve a gestire gli interrupt. Quando la CPU ordina a un dispositivo di I/O di effettuare qualche operazione si aspetta di solito un interrupt alla fine del lavoro; la segnalazione di questi interrupt richiede l'utilizzo del bus.

Dato che più dispositivi potrebbero voler generare un interrupt simultaneamente, si verificano gli stessi tipi di problemi di arbitraggio visti nel caso dei cicli di bus ordinari. La soluzione più usuale prevede di assegnare priorità ai dispositivi e di utilizzare un arbitro centralizzato per dare priorità al dispositivo per cui la temporizzazione è più critica. Esistono chip controllori di interrupt di tipo standard che sono utilizzati in modo molto diffuso. Nei PC basati su processori Intel il chipset incorpora il controllore di interrupt 8259A, illustrato nella Figura 3.43.

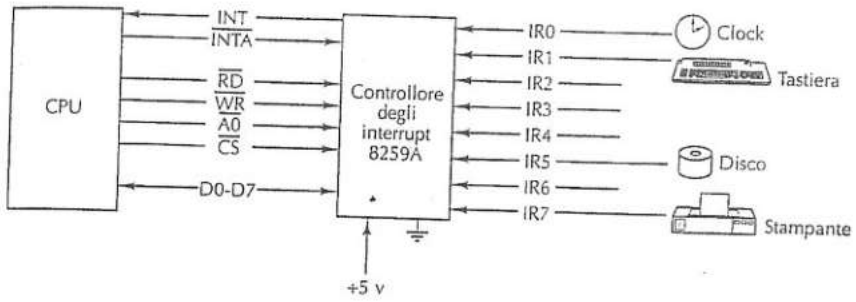


Figura 3.43 Utilizzo del controllore degli interrupt 8259A.

Il chip 8259A ha otto input, chiamati IRx (*Interrupt Request*, "richiesta di interrupt"), con i quali si possono collegare direttamente fino a otto controllori di I/O. Quando uno di questi dispositivi vuole causare un interrupt asserisce la propria linea di input. Quando uno o più input sono asseriti il chip 8259A asserisce INT (*INTerrupt*), che pilota direttamente il pin di interrupt sulla CPU. Quando la CPU è in grado di gestire l'interrupt rispedisce all'8259A un impulso su INTA (*INTerrupt Acknowledge*, "conferma dell'interrupt"); a quel punto l'8259A deve specificare quale input ha causato l'interrupt generando in output il suo numero sul bus dei dati. Questa operazione richiede un ciclo di bus speciale. L'hardware della CPU utilizza quindi quel numero come indice all'interno di una tabella di puntatori, chiamata **vettore di interrupt**, per cercare l'indirizzo della procedura da eseguire per poter gestire l'interrupt.

L'8259A ha al suo interno vari registri che la CPU può leggere e scrivere utilizzando i cicli di bus ordinari e i pin RD (*ReaD*), WR (*WRite*), CS (*Chip Select*) e A0. Quando il software ha gestito l'interrupt ed è pronto per accettarne uno nuovo scrive uno speciale codice all'interno di uno dei registri; ciò fa sì che l'8259A neghi INT, a meno che non vi sia un altro interrupt pendente. In questi registri si può anche scrivere per portare l'8259A in uno dei vari modi disponibili, mascherando per esempio un insieme di interrupt oppure abilitando altre funzionalità.

porta **QPI** (*Quick Path Interconnect*) che connette il processore a un'interconnessione esterna al multiprocessore e permette di costruire sistemi con più di sei processori. La porta QPI invia e riceve richieste di coerenza della cache, oltre a vari altri messaggi per la gestione del multiprocessore, come gli interrupt interprocessore.

Un problema del Core i7, come di tutti gli altri processori moderni desktop, è il consumo energetico e il calore generato. Per prevenire eventuali danni al silicio il calore deve essere rimosso dal chip non appena prodotto. Il Core i7 consuma tra i 17 e i 150 watt, a seconda della frequenza e del modello. Pertanto Intel è costantemente alla ricerca di soluzioni per dissipare il calore prodotto dai suoi chip. Le tecnologie di raffreddamento e la conduzione termica del supporto sono vitali per proteggere il silicio.

Il Core i7 è realizzato in un supporto LGA quadrato di 37,5 mm di lato. Contiene 1.155 pad posti sul fondo, di cui 286 sono per l'alimentazione e 360 sono messi a terra per ridurre il rumore. I pad sono disposti più o meno come un quadrato 40 × 40 mm con la parte centrale di 17 × 25 mm mancante. Inoltre, mancano 20 pad sul perimetro, con una distribuzione asimmetrica, per evitare che il chip venga inserito in maniera scorretta nel suo zoccolo. La Figura 3.44 mostra la disposizione fisica dei pin.

Il chip è attrezzato in modo che sopra di esso sia possibile montare un dissipatore di calore e una ventola per raffreddarlo. Per avere un'idea dell'entità del problema energetico basta accendere una lampadina da 150 Watt, aspettare qualche minuto e sfiorarla (senza però toccarla). La stessa quantità di calore deve essere continuamente dissipata da un processore Core i7 di fascia alta. Quando il Core i7 smette di essere utile come CPU, lo si può quindi sempre riciclare come fornello da campeggio!

Secondo le leggi della fisica ogni fenomeno che produce una gran quantità di calore deve consumare molta energia. In un calcolatore portatile non è desiderabile utilizzare una gran quantità di energia in quanto la carica della batteria è limitata e tenderebbe a consumarsi molto rapidamente. Per risolvere questo problema Intel ha dotato le CPU di un modo per "riposare" quando sono inattive e di entrare in un "sonno profondo" quando è probabile che resteranno inattive per un certo periodo di tempo. Ci sono cinque stati possibili, che variano dalla piena esecuzione al sonno profondo. Negli stati intermedi alcune funzionalità (come lo snooping della cache e la gestione degli interrupt) sono disabilitate, mentre altre continuano a essere utilizzabili. Quando la CPU si trova nello stato di sonno profondo i registri sono preservati, mentre le cache vengono scaricate e spente. Per risvegliarle la CPU è necessario un segnale hardware. Non si sa se il Core i7 sia in grado di sognare quando dorme profondamente.

Disposizione logica dei contatti del Core i7

Dei 1155 pin del Core i7, ne vengono utilizzati 447 per i segnali, 286 per l'alimentazione (a diverse tensioni) e 360 per la terra, mentre gli altri 62 sono riservati a utilizzi futuri. Dato che alcuni segnali logici utilizzano due o più pin (come gli indirizzi di memoria richiesti), il numero di segnali distinti è solamente 131. La Figura 3.45 mostra una versione piuttosto semplificata della disposizione logica dei contatti. Sul lato sinistro della figura ci sono cinque gruppi principali di segnali per il bus di memoria, sul lato destro altri segnali di varia natura.

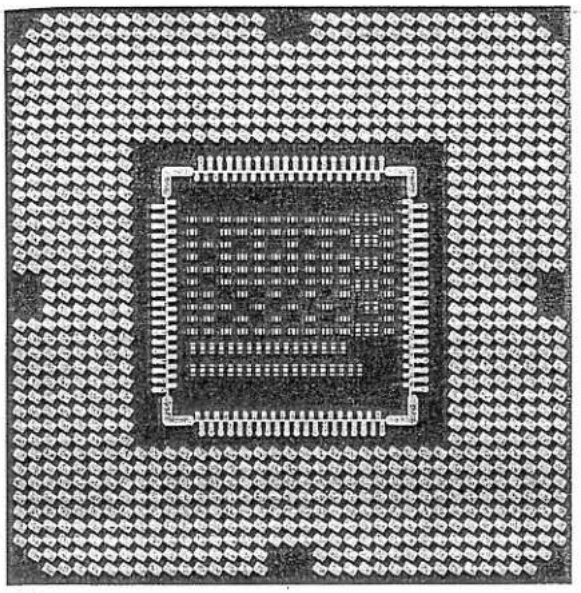


Figura 3.44 Disposizione fisica dei contatti del Core i7.

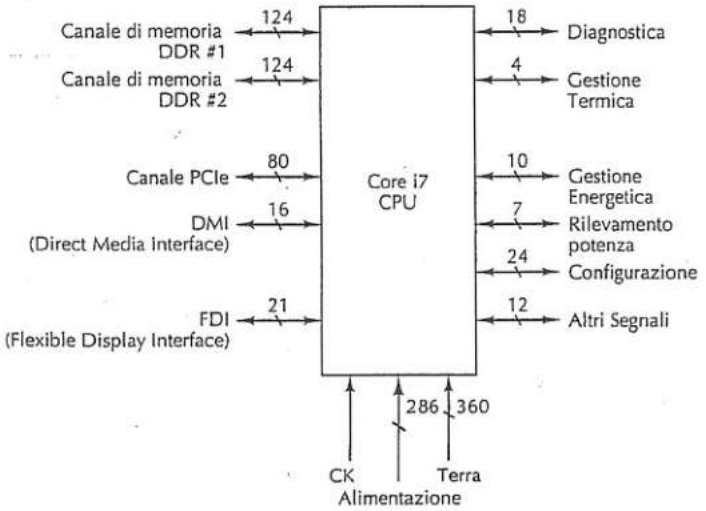


Figura 3.45 Disposizione logica dei contatti del Core i7.

Quando sono presenti più di otto dispositivi di I/O è possibile utilizzare in cascata più chip 8259A. Nel caso più estremo tutti gli otto input possono essere connessi agli output di ulteriori otto 8259A; è possibile così gestire fino a 64 dispositivi di I/O in una rete di interrupt a due livelli. Il chip Intel ICH10, uno dei chip del chipset Core i7, incorpora due controllori di interrupt 8259A. ICH10 dispone dunque di 15 interrupt esterni, uno in meno rispetto ai 16 sui due chip 8259A, perchè un interrupt viene utilizzato per connettere in cascata il secondo 8259A al primo. L'8259A possiede alcuni pin dedicati alla gestione di questa cascata, ma qui ci è sembrato opportuno ometterli. Oggi "8259A" è in realtà una parte di un altro chip.

Non abbiamo affatto esaurito il tema della progettazione dei bus; tuttavia il materiale presentato finora dovrebbe fornire sufficienti conoscenze di base per capire qual è in sostanza il funzionamento di un bus e come le CPU e i bus possono interagire. Ci spostiamo ora dal generale allo specifico e diamo uno sguardo ad alcuni esempi di CPU commerciali e ai loro bus.

3.5 Esempi di chip della CPU

In questo paragrafo esamineremo a livello hardware, in modo piuttosto dettagliato, i chip Intel Core i7, TI OMAP4430 e Atmel ATmega168.

3.5.1 Intel Core i7

Il Core i7 è un diretto discendente della CPU 8088 utilizzata nel PC IBM originario. Il primo Core i7 fu introdotto nel novembre 2008, come una CPU a 4 processori, con 731 milioni di transistor, una frequenza di 3,2 GHz e una *larghezza di linea* di 45 nanometri. La larghezza di linea corrisponde alla grandezza dei collegamenti tra i transistor (ed è anche una misura della dimensione dei transistor stessi). Più ridotta è la larghezza di linea e più transistor possono essere inseriti sul chip. La legge di Moore riguarda fondamentalmente la capacità di continuare a ridurre le larghezze di linea. Larghezze di linea più piccole permettono velocità di clock più elevate. Per fare un confronto con le misure in gioco si pensi che i capelli hanno un diametro che varia tra 20.000 e i 100.000 nanometri, di cui i più fini sono quelli biondi, mentre quelli con diametro maggiore sono quelli neri.

La versione iniziale del Core i7 era basata sull'architettura "Nahalem", mentre le versioni successive si basano sull'architettura "Sandy Bridge". In questo contesto l'architettura rappresenta l'organizzazione interna della CPU, alla quale spesso è assegnato un nome in codice. I progettisti delle architetture di tanto in tanto si inventano qualche nome stravagante per il loro progetto. Un caso degno di nota è quello delle architetture della serie K di AMD, progettate per rompere l'apparente invulnerabilità di Intel nel mercato dei desktop. Il nome in codice della serie K era "Kryptonite", con evidente riferimento all'unica sostanza in grado di colpire Superman: un colpo intelligente alla dominante Intel.

I Core i7 basati su Sandy Bridge hanno portato il numero di transistor a 1,16 miliardi e lavorano alla frequenza di 3,5 GHz, con larghezza di linea di 32 nanometri. Il

Core i7, pur essendo decisamente più avanzato rispetto all'8088 con 29.000 transistor, è completamente compatibile con esso e può eseguire i suoi programmi senza alcuna modifica (lo stesso vale, ovviamente, anche per i programmi di tutti i processori intermedi).

Dal punto di vista del software il Core i7 è una macchina interamente a 64 bit. A livello utente ha tutte le funzionalità ISA dei modelli 80386, 80486, Pentium, Pentium II, Pentium Pro, Pentium III e Pentium 4, compresi gli stessi registri, le stesse istruzioni e la stessa implementazione dello standard in virgola mobile IEEE 754 interamente sul chip, inoltre ha alcune nuove istruzioni dedicate alla crittografia.

Il processore Core i7 è una CPU multicore, ovvero lo stampo di silicio contiene più processori. La CPU è venduta con un numero variabile di processori, per ora dai 2 ai 6, ma nei piani futuri questo numero crescerà. Quando un programmatore scrive un programma parallelo, utilizzando thread e lock, riesce ad avere significativi guadagni nella velocità d'esecuzione sfruttando il parallelismo dei processori multipli. In aggiunta, le singole CPU sono "hyperthreaded", cioè possono essere attivi simultaneamente diversi thread hardware sulla stessa CPU. L'*hyperthreading* (anche chiamato "multithreading simultaneo" dai progettisti) permette di tollerare latenze molto brevi, per esempio un fallimento d'accesso alla cache, con l'attivazione dei thread hardware. Al contrario il threading software può gestire soltanto latenze molto lunghe, per esempio un page fault, a causa dei numerosi cicli necessari per implementare il passaggio di thread via software.

Internamente, a livello di microarchitettura, il Core i7 è un progetto dalle molte potenzialità. Basato sulle architetture dei suoi predecessori, il Core 2 e il Core 2 Duo, il Core i7 può eseguire fino a 4 istruzioni per volta, il che fa di questo processore una macchina superscalare a 4 livelli. Esamineremo questa architettura nel Capitolo 4.

Il Core i7 ha 3 livelli di cache. Ogni processore di un Core i7 ha una cache dati di livello 1 (L1) da 32 KB, una cache istruzioni di livello 1 da 32 KB e una cache di livello 2 (L2) da 256 KB. La cache di secondo livello è unificata, ovvero contiene un misto di dati e istruzioni. Tutti i core condividono una cache unificata di livello 3 (L3) la cui dimensione varia dai 4 ai 15 MB a seconda del modello di processore. La presenza di 3 livelli di cache migliora significativamente le prestazioni del processore, ma ha un grande costo in termini di silicio, visto che il Core i7 può arrivare ad avere un totale di 17 MB di cache su un singolo stampo di silicio.

Siccome ogni chip Core i7 contiene più processori con una cache dati privata, sorgono dei problemi quando un processore modifica nella sua cache un dato presente anche nella cache di un altro processore. Se quest'ultimo prova a leggere quel dato dalla (sua) memoria otterrà un valore non aggiornato. Per garantire la coerenza della memoria in un multiprocessore ciascuna CPU effettua uno **snooping** ("ficcanaso") sul bus della memoria per cercare eventuali riferimenti alle parole che ha memorizzato nella propria cache. Quando vede un tale riferimento si intromette nel bus fornendo il dato richiesto prima che lo possa fare la memoria. Nel Capitolo 8 studieremo lo snooping.

Nei sistemi Core i7 si utilizzano due principali bus esterni, entrambi sincroni: un bus di memoria DDR3 per l'accesso alla memoria centrale DRAM e un bus PCI Express per connettere il processore ai dispositivi di I/O. Le versioni di alta gamma del Core i7 hanno diversi bus di memoria e diversi bus PCI Express. Dispongono inoltre di una

Esaminiamo i segnali a partire da quelli del bus. I primi due segnali del bus sono utilizzati come interfacce verso le DRAM compatibili DDR3. Questo gruppo di segnali fornisce ai banchi DRAM l'indirizzo, i dati, il controllo e il clock. Il Core i7 supporta due canali DRAM DDR3 indipendenti che lavorano su un bus con frequenza di clock di 666 MHz in maniera bidirezionale e permettono 1333 transazioni al secondo. L'interfaccia DDR3 ha un'ampiezza di 64 bit, dunque le due interfacce DDR3 lavorando in tandem possono offrire ai programmi assetati di memoria fino a 20 GB di dati al secondo.

Il terzo gruppo di bus è costituito dall'interfaccia PCI Express ed è utilizzata per connettere le periferiche direttamente alla CPU. PCI Express è un'interfaccia seriale ad alta velocità, in cui ogni singolo collegamento seriale forma una corsia (lane) verso le periferiche. Il collegamento del Core i7 è un'interfaccia x16, il che significa che può gestire 16 corsie simultaneamente per arrivare così a una larghezza di banda aggregata di 16 GB/sec. Nonostante PCI Express sia un canale seriale, sui suoi collegamenti viaggia un ricco insieme di comandi, tra cui i read, i write e gli interrupt dei dispositivi e i comandi di configurazione.

Un'altra categoria di bus è formata dal **DMI** (*Direct Media Interface*), utilizzato per il collegamento della CPU al suo *chipset*. L'interfaccia DMI è simile a PCI Express, anche se lavora a una velocità dimezzata, dato che le sue 4 corsie possono offrire solamente una velocità di trasferimento dati di 2,5 GB/sec. Il chipset di una CPU contiene un ricco insieme di supporti per le interfacce di periferiche aggiuntive, richiesti tipicamente da sistemi di alto livello dotati di molte periferiche di I/O. Il chipset Core i7 utilizza i chip P67 e ICH10. Il P67 è il coltellino svizzero dei chip: offre infatti le interfacce SATA, USB, Audio, PCIe e Flash. Il chip ICH10 fornisce il supporto per interfacce più datate, tra cui l'interfaccia PCI e le funzionalità di controllo degli interrupt del chip 8259A. In aggiunta, ICH10 contiene vari altri circuiti, come controllori di clock real time, timer di eventi e **DMA** (*Direct Memory Access*). Grazie a chip come questi la costruzione di un PC completo risulta notevolmente semplificata.

È possibile configurare il Core i7 in modo che utilizzi gli interrupt allo stesso modo dell'8088 (per la retrocompatibilità) oppure secondo una nuova modalità che impiega un dispositivo chiamato **APIC** (*Advanced Programmable Interrupt Controller*).

Il Core i7 può funzionare a varie tensioni, ma deve sapere quale di queste viene utilizzata. I segnali per la gestione dell'alimentazione sono utilizzati per la selezione automatica della tensione, per comunicare alla CPU che la tensione è stabile e per altri aspetti riguardanti l'alimentazione. Sempre mediante questi segnali vengono gestiti i vari stati di riposo, dato che la loro scelta è fatta allo scopo di ridurre il consumo energetico.

Il Core i7, nonostante una gestione sofisticata dell'alimentazione, si riscalda notevolmente. Ogni chip Core i7 contiene diversi sensori di temperatura per la protezione dei circuiti in silicio, in grado di rilevare situazioni di surriscaldamento. Il gruppo monitoraggio termico si occupa della gestione termica e permette alla CPU di indicare alle sue componenti le situazioni di rischio di surriscaldamento. La CPU asserisce uno dei suoi pin quando la sua temperatura interna supera i 130° C (266° F). Se la CPU raggiunge questa temperatura, probabilmente inizia a sognare il pensionamento e una sua riconversione in fornello da campeggio!

Anche a queste temperature l'utente non si deve preoccupare della sicurezza del Core i7. Quando i sensori interni rilevano una situazione di possibile surriscaldamento viene avviata la limitazione termica (*thermal throttling*), una tecnica in grado di ridurre rapidamente il calore generato facendo in modo che la CPU lavori soltanto ogni N cicli di clock. Più grande è il valore di N, più il funzionamento del processore verrà limitato e più velocemente si raffredderà. Senza l'ideazione della limitazione termica le CPU, in caso di guasti al sistema di raffreddamento, sarebbero bruciate. Le prove di questi tempi bui della gestione termica delle CPU si possono recuperare cercando «exploding CPU» su YouTube. Questi video sono spesso fasulli, ma il problema è reale.

Il segnale Clock fornisce al processore il clock di sistema, usato internamente per generare una varietà di clock basati su suoi multipli o sue frazioni. È proprio così! È possibile generare anche dei multipli della frequenza di clock di sistema grazie a un dispositivo molto intelligente chiamato **DLL** (*Delay-Locked Loop*).

Il gruppo per la diagnostica contiene segnali per sistemi di test e debugging secondo lo standard IEEE 1149.1 dei test **JTAG** (*Joint Test Action Group*). L'ultimo gruppo è un calderone di vari altri segnali dedicati a utilizzi particolari.

Pipeline sul bus di memoria DDR3 del Core i7

Le moderne CPU come il Core i7 pretendono molto dalle memorie DRAM. I singoli processori possono produrre richieste di accesso più velocemente di quanto una lenta DRAM possa fornire i valori, e questo problema viene amplificato quando diversi processori effettuano richieste simultanee. Per evitare che la CPU rimanga in attesa a causa della mancanza di dati è fondamentale ottenere dalla memoria il massimo throughput possibile. Per questa ragione il bus di memoria DDR3 del Core i7 è strutturato a pipeline, in modo che su di esso possano avvenire fino a quattro transazioni contemporanee. Il concetto di pipeline è stato visto nel contesto delle CPU (Figura 2.4) nel corso del Capitolo 2, e anche le memorie possono avere un'architettura a pipeline.

Per consentire l'uso della pipeline, le richieste di memoria del Core i7 sono composte da tre fasi.

- Fase di ACTIVATE della memoria, che "apre" una riga della DRAM per prepararla a successivi accessi.
- Fase di READ o WRITE della memoria, in cui si possono effettuare accessi multipli a singole parole appartenenti alla riga correntemente aperta della DRAM, oppure accessi multipli a una sequenza di parole appartenenti alla riga corrente della DRAM con l'utilizzo del burst mode.
- La fase di PRECHARGE, che "chiude" la riga corrente della DRAM e prepara la memoria per il prossimo comando ACTIVATE.

Il segreto dell'accesso a memoria del Core i7 sta nell'organizzazione strutturata in più *banchi* della DRAM DDR3 sul suo chip. Un banco è un blocco di memoria DRAM a cui si può accedere in parallelo con altri banchi di memoria, anche se questi sono contenuti sullo stesso chip. Una DRAM DDR3 è tipicamente formata da 8 banchi di DRAM. Tuttavia, le specifiche dell'interfaccia DDR3 permettono fino a quattro accessi concorrenti soltanto su un singolo canale DDR3. Nel diagramma temporale della Figura

Esaminiamo i segnali a partire da quelli del bus. I primi due segnali del bus sono utilizzati come interfacce verso le DRAM compatibili DDR3. Questo gruppo di segnali fornisce ai banchi DRAM l'indirizzo, i dati, il controllo e il clock. Il Core i7 supporta due canali DRAM DDR3 indipendenti che lavorano su un bus con frequenza di clock di 666 MHz in maniera bidirezionale e permettono 1333 transazioni al secondo. L'interfaccia DDR3 ha un'ampiezza di 64 bit, dunque le due interfacce DDR3 lavorando in tandem possono offrire ai programmi assetati di memoria fino a 20 GB di dati al secondo.

Il terzo gruppo di bus è costituito dall'interfaccia PCI Express ed è utilizzata per connettere le periferiche direttamente alla CPU. PCI Express è un'interfaccia seriale ad alta velocità, in cui ogni singolo collegamento seriale forma una corsia (lane) verso le periferiche. Il collegamento del Core i7 è un'interfaccia x16, il che significa che può gestire 16 corsie simultaneamente per arrivare così a una larghezza di banda aggregata di 16 GB/sec. Nonostante PCI Express sia un canale seriale, sui suoi collegamenti viaggia un ricco insieme di comandi, tra cui i read, i write e gli interrupt dei dispositivi e i comandi di configurazione.

Un'altra categoria di bus è formata dal **DMI** (*Direct Media Interface*), utilizzato per il collegamento della CPU al suo *chipset*. L'interfaccia DMI è simile a PCI Express, anche se lavora a una velocità dimezzata, dato che le sue 4 corsie possono offrire solamente una velocità di trasferimento dati di 2,5 GB/sec. Il chipset di una CPU contiene un ricco insieme di supporti per le interfacce di periferiche aggiuntive, richiesti tipicamente da sistemi di alto livello dotati di molte periferiche di I/O. Il chipset Core i7 utilizza i chip P67 e ICH10. Il P67 è il coltellino svizzero dei chip: offre infatti le interfacce SATA, USB, Audio, PCIe e Flash. Il chip ICH10 fornisce il supporto per interfacce più datate, tra cui l'interfaccia PCI e le funzionalità di controllo degli interrupt del chip 8259A. In aggiunta, ICH10 contiene vari altri circuiti, come controllori di clock real time, timer di eventi e **DMA** (*Direct Memory Access*). Grazie a chip come questi la costruzione di un PC completo risulta notevolmente semplificata.

È possibile configurare il Core i7 in modo che utilizzi gli interrupt allo stesso modo dell'8088 (per la retrocompatibilità) oppure secondo una nuova modalità che impiega un dispositivo chiamato **APIC** (*Advanced Programmable Interrupt Controller*).

Il Core i7 può funzionare a varie tensioni, ma deve sapere quale di queste viene utilizzata. I segnali per la gestione dell'alimentazione sono utilizzati per la selezione automatica della tensione, per comunicare alla CPU che la tensione è stabile e per altri aspetti riguardanti l'alimentazione. Sempre mediante questi segnali vengono gestiti i vari stati di riposo, dato che la loro scelta è fatta allo scopo di ridurre il consumo energetico.

Il Core i7, nonostante una gestione sofisticata dell'alimentazione, si riscalda notevolmente. Ogni chip Core i7 contiene diversi sensori di temperatura per la protezione dei circuiti in silicio, in grado di rilevare situazioni di surriscaldamento. Il gruppo monitoraggio termico si occupa della gestione termica e permette alla CPU di indicare alle sue componenti le situazioni di rischio di surriscaldamento. La CPU asserisce uno dei suoi pin quando la sua temperatura interna supera i 130° C (266° F). Se la CPU raggiunge questa temperatura, probabilmente inizia a sognare il pensionamento e una sua riconversione in fornello da campeggio!

Anche a queste temperature l'utente non si deve preoccupare della sicurezza del Core i7. Quando i sensori interni rilevano una situazione di possibile surriscaldamento viene avviata la limitazione termica (*thermal throttling*), una tecnica in grado di ridurre rapidamente il calore generato facendo in modo che la CPU lavori soltanto ogni N cicli di clock. Più grande è il valore di N, più il funzionamento del processore verrà limitato e più velocemente si raffredderà. Senza l'ideazione della limitazione termica le CPU, in caso di guasti al sistema di raffreddamento, sarebbero bruciate. Le prove di questi tempi bui della gestione termica delle CPU si possono recuperare cercando «exploding CPU» su YouTube. Questi video sono spesso fasulli, ma il problema è reale.

Il segnale Clock fornisce al processore il clock di sistema, usato internamente per generare una varietà di clock basati su suoi multipli o sue frazioni. È proprio così! È possibile generare anche dei multipli della frequenza di clock di sistema grazie a un dispositivo molto intelligente chiamato **DLL** (*Delay-Locked Loop*).

Il gruppo per la diagnostica contiene segnali per sistemi di test e debugging secondo lo standard IEEE 1149.1 dei test **JTAG** (*Joint Test Action Group*). L'ultimo gruppo è un calderone di vari altri segnali dedicati a utilizzi particolari.

Pipeline sul bus di memoria DDR3 del Core i7

Le moderne CPU come il Core i7 pretendono molto dalle memorie DRAM. I singoli processori possono produrre richieste di accesso più velocemente di quanto una lenta DRAM possa fornire i valori, e questo problema viene amplificato quando diversi processori effettuano richieste simultanee. Per evitare che la CPU rimanga in attesa a causa della mancanza di dati è fondamentale ottenere dalla memoria il massimo throughput possibile. Per questa ragione il bus di memoria DDR3 del Core i7 è strutturato a pipeline, in modo che su di esso possano avvenire fino a quattro transazioni contemporanee. Il concetto di pipeline è stato visto nel contesto delle CPU (Figura 2.4) nel corso del Capitolo 2, e anche le memorie possono avere un'architettura a pipeline.

Per consentire l'uso della pipeline, le richieste di memoria del Core i7 sono composte da tre fasi.

- Fase di ACTIVATE della memoria, che "apre" una riga della DRAM per prepararla a successivi accessi.
- Fase di READ o WRITE della memoria, in cui si possono effettuare accessi multipli a singole parole appartenenti alla riga correntemente aperta della DRAM, oppure accessi multipli a una sequenza di parole appartenenti alla riga corrente della DRAM con l'utilizzo del burst mode.
- La fase di PRECHARGE, che "chiude" la riga corrente della DRAM e prepara la memoria per il prossimo comando ACTIVATE.

Il segreto dell'accesso a memoria del Core i7 sta nell'organizzazione strutturata in più *banchi* della DRAM DDR3 sul suo chip. Un banco è un blocco di memoria DRAM a cui si può accedere in parallelo con altri banchi di memoria, anche se questi sono contenuti sullo stesso chip. Una DRAM DDR3 è tipicamente formata da 8 banchi di DRAM. Tuttavia, le specifiche dell'interfaccia DDR3 permettono fino a quattro accessi concorrenti soltanto su un singolo canale DDR3. Nel diagramma temporale della Figura

porta **QPI** (*Quick Path Interconnect*) che connette il processore a un'interconnessione esterna al multiprocessore e permette di costruire sistemi con più di sei processori. La porta QPI invia e riceve richieste di coerenza della cache, oltre a vari altri messaggi per la gestione del multiprocessore, come gli interrupt interprocessore.

Un problema del Core i7, come di tutti gli altri processori moderni desktop, è il consumo energetico e il calore generato. Per prevenire eventuali danni al silicio il calore deve essere rimosso dal chip non appena prodotto. Il Core i7 consuma tra i 17 e i 150 watt, a seconda della frequenza e del modello. Pertanto Intel è costantemente alla ricerca di soluzioni per dissipare il calore prodotto dai suoi chip. Le tecnologie di raffreddamento e la conduzione termica del supporto sono vitali per proteggere il silicio.

Il Core i7 è realizzato in un supporto LGA quadrato di 37,5 mm di lato. Contiene 1.155 pad posti sul fondo, di cui 286 sono per l'alimentazione e 360 sono messi a terra per ridurre il rumore. I pad sono disposti più o meno come un quadrato 40 × 40 mm con la parte centrale di 17 × 25 mm mancante. Inoltre, mancano 20 pad sul perimetro, con una distribuzione asimmetrica, per evitare che il chip venga inserito in maniera scorretta nel suo zoccolo. La Figura 3.44 mostra la disposizione fisica dei pin.

Il chip è attrezzato in modo che sopra di esso sia possibile montare un dissipatore di calore e una ventola per raffreddarlo. Per avere un'idea dell'entità del problema energetico basta accendere una lampadina da 150 Watt, aspettare qualche minuto e sfiorarla (senza però toccarla). La stessa quantità di calore deve essere continuamente dissipata da un processore Core i7 di fascia alta. Quando il Core i7 smette di essere utile come CPU, lo si può quindi sempre riciclare come fornello da campeggio!

Secondo le leggi della fisica ogni fenomeno che produce una gran quantità di calore deve consumare molta energia. In un calcolatore portatile non è desiderabile utilizzare una gran quantità di energia in quanto la carica della batteria è limitata e tenderebbe a consumarsi molto rapidamente. Per risolvere questo problema Intel ha dotato le CPU di un modo per "riposare" quando sono inattive e di entrare in un "sonno profondo" quando è probabile che resteranno inattive per un certo periodo di tempo. Ci sono cinque stati possibili, che variano dalla piena esecuzione al sonno profondo. Negli stati intermedi alcune funzionalità (come lo snooping della cache e la gestione degli interrupt) sono disabilitate, mentre altre continuano a essere utilizzabili. Quando la CPU si trova nello stato di sonno profondo i registri sono preservati, mentre le cache vengono scaricate e spente. Per risvegliarle la CPU è necessario un segnale hardware. Non si sa se il Core i7 sia in grado di sognare quando dorme profondamente.

Disposizione logica dei contatti del Core i7

Dei 1155 pin del Core i7, ne vengono utilizzati 447 per i segnali, 286 per l'alimentazione (a diverse tensioni) e 360 per la terra, mentre gli altri 62 sono riservati a utilizzi futuri. Dato che alcuni segnali logici utilizzano due o più pin (come gli indirizzi di memoria richiesti), il numero di segnali distinti è solamente 131. La Figura 3.45 mostra una versione piuttosto semplificata della disposizione logica dei contatti. Sul lato sinistro della figura ci sono cinque gruppi principali di segnali per il bus di memoria, sul lato destro altri segnali di varia natura.

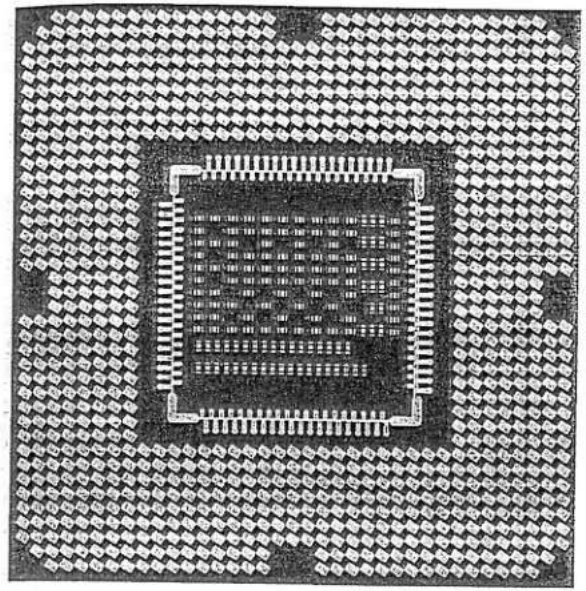


Figura 3.44 Disposizione fisica dei contatti del Core i7.

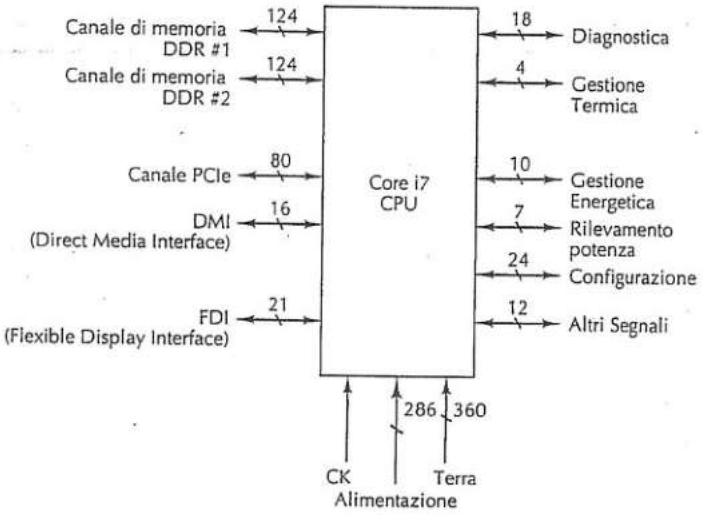


Figura 3.45 Disposizione logica dei contatti del Core i7.

3.46 è mostrato come il Core i7 effettua 4 accessi di memoria a 3 distinti banchi DRAM. Gli accessi sono completamente sovrapposti, in modo che le letture sulla DRAM avvengono in parallelo sullo stesso chip. La figura mostra quali comandi portano a operazioni successive mediante l'uso di frecce nel diagramma temporale.

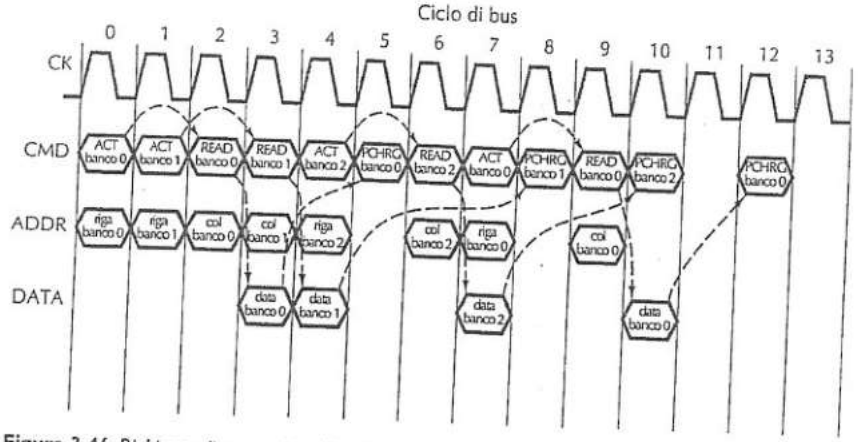


Figura 3.46 Richieste di memoria sull'interfaccia DDR3 del Core i7 gestite con pipeline.

Come mostrato nella figura 3.46, l'interfaccia di memoria DDR3 dispone di quattro segnali primari: il *clock* del bus (CK), il segnale di *comando* del bus (CMD), il segnale di *indirizzo* (ADDR) e quello di *dato* (DATA). Il segnale CK dirige tutte le attività del bus. Il segnale CMD indica quale attività si richiede alla DRAM connessa. Il comando ACTIVATE specifica l'indirizzo della riga DRAM da aprire per mezzo del segnale ADDR. Per eseguire una READ, viene fornito l'indirizzo della colonna DRAM per mezzo del segnale ADDR e la DRAM, dopo un intervallo di tempo fissato, mette il valore letto sul segnale DATA. Alla fine il comando PRECHARGE indica il banco da precaricare per mezzo del segnale ADDR. Ai fini di questo esempio, il comando ACTIVATE deve precedere il primo READ allo stesso banco di due cicli di bus DDR3 e i dati sono prodotti un ciclo di bus dopo il comando READ. Inoltre, l'operazione di PRECHARGE deve avvenire almeno due cicli di bus dopo l'ultima operazione di read allo stesso banco DRAM.

Il parallelismo nelle richieste alla memoria può essere osservato dalle richieste READ a distinti banchi di DRAM. I primi due accessi READ ai banchi 0 e 1 sono completamente sovrapposti e producono i loro risultati nei cicli di clock 3 e 4 rispettivamente. L'accesso al banco 2 si sovrappone parzialmente al primo accesso al banco 1 e, infine, la seconda READ al banco 0 si sovrappone parzialmente all'accesso al banco 2.

Ci si potrà chiedere come fa il Core i7 a sapere quando riceverà una risposta alle sue richieste di READ, e quando potrà effettuare nuove richieste. La risposta è che sa quando riceve e quando iniziare le richieste perché possiede un modello completo delle attività interne di ogni DRAM DDR3 collegata. È quindi in grado di anticipare la restituzione dei dati nel ciclo corretto e saprà di dover evitare l'inizio di un'operazione PRECHARGE prima che siano passati due cicli dall'ultima READ. Il Core i7 può anticipare tutte queste attività perché l'interfaccia di memoria DDR3 è **un'interfaccia sincrona di memoria**. Ogni attività impiega quindi un ben noto numero di cicli di bus DDR3. Anche con tutte queste conoscenze, la costruzione di un'interfaccia DDR3 di alte prestazioni e completamente gestita a pipeline non è un compito banale, perché è richiesto l'utilizzo di diversi temporizzatori interni e di rilevatori di conflitti per implementare in maniera efficiente la gestione delle richieste di memoria.

3.5.2 Texas Instruments OMAP4430

Come secondo esempio di CPU esaminiamo adesso il **SoC** (*System-on-a-Chip*) Texas Instruments (TI) OMAP4430. Questa CPU implementa il set di istruzioni ARM ed è rivolto ad applicazioni mobili ed embedded come smartphone, tablet e apparati Internet. Giustamente chiamato SoC (system-on-a-chip), incorpora una vasta gamma di dispositivi in modo tale da implementare uno strumento completo di elaborazione in combinazione con le periferiche fisiche (touchscreen, memoria flash e così via).

Il SoC OMAP4430 include due Core ARM A9, degli acceleratori aggiuntivi e una vasta gamma di interfacce per periferiche. L'organizzazione interna dell'OMAP4430 è mostrata nella Figura 3.47. I Core ARM A9 sono microarchitetture superscalari a 2 livelli. In aggiunta, sul chip OMAP4430, ci sono tre acceleratori: un processore grafico PowerVR SGX540, un processore di segnale di immagine (ISP) e un processore video IVA3. Il processore SGX540 fornisce un efficiente rendering 3D programmabile, come le GPU che si trovano nei PC desktop, anche se è più piccolo e più lento. L'ISP è un processore programmabile per la manipolazione efficiente di immagini, destinato alla tipologia di operazioni che necessarie in una fotocamera digitale di fascia alta. L'IVA3 implementa un'efficiente codifica e decodifica video, con prestazioni sufficienti per supportare applicazioni 3D come quelle delle console di gioco portatili. Inoltre, nel SoC OMAP4430 si trova una vasta gamma di interfacce per periferiche, tra cui un controllore di touchscreen, un controllore di tastiera e le interfacce DRAM, Flash, USB e HDMI. Texas Instruments ha dettagliato la tabella di marcia per la serie di CPU OMAP. I progetti futuri avranno qualcosa in più di tutto: più core ARM, più GPU e più periferiche.

Il SoC OMAP4430 è stato introdotto nei primi mesi del 2011 con due core ARM A9 a 1 GHz e con l'utilizzo di una tecnologia a 45 nanometri. Un aspetto fondamentale della sua progettazione è che esegue gran quantità di calcoli con un consumo molto basso, proprio perché è rivolto a dispositivi mobili alimentati a batteria nei quali più efficiente è il progetto, più a lungo l'utente può aspettare prima di ricaricare la batteria.

I molti processori incorporati nell'OMAP4430 servono in maniera specifica per sostenere la sua missione di funzionamento a bassa potenza. Il processore grafico, ISP, e IVA3 sono acceleratori programmabili che forniscono buone capacità di calcolo con un minor consumo di energia rispetto a quella richiesta dalle stesse attività in esecuzione sulla CPU ARM A9. Completamente alimentato, il SoC OMAP4430 consuma solo 600 mW di potenza. Rispetto a un Core i7 di fascia alta, i chip OMAP4430 utilizzano

al massimo 1/250 della potenza. L'OMAP4430 implementa anche una modalità di sospensione molto efficace; quando tutti i componenti sono inattivi, il chip consuma solamente 100 μW. Una sospensione efficiente è fondamentale per applicazioni mobili con lunghi periodi di standby, come per esempio i telefoni cellulari. Minore è l'energia utilizzata in modalità di sospensione, più a lungo il cellulare può rimanere in standby.

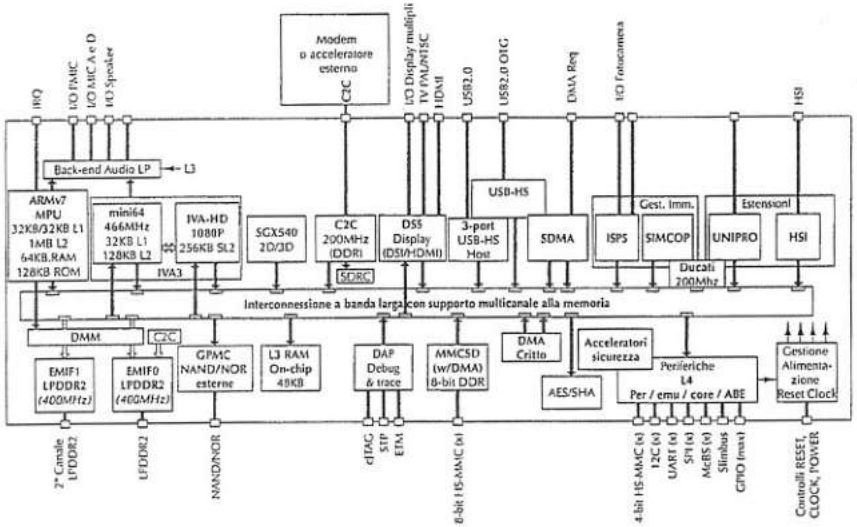


Figura 3.47 Organizzazione interna del SoC OMAP4430.

Per ridurre ulteriormente le richieste energetiche dell'OMAP4430, il progetto incorpora varie funzionalità di gestione dell'energia, tra cui una *scala dinamica di tensione* e il *power gating*. La scala dinamica di tensione consente ai componenti di essere in esecuzione più lentamente a una tensione inferiore, con conseguente riduzione significativa dei requisiti di potenza. Se per i calcoli non è necessaria la massima velocità della CPU, la tensione può essere abbassata per rendere più lenta la CPU e risparmiare così più energia. Il *power gating* è uno strumento di gestione energetica ancora più aggressivo che permette di togliere completamente l'alimentazione ai componenti che non sono in uso, eliminandone in tal modo l'assorbimento di potenza. Per esempio, in un'applicazione per tablet, se l'utente non sta guardando un film il processore video IVA3 viene completamente spento. Al contrario, quando l'utente guarda un film, l'IVA3 funziona al massimo delle sue potenzialità per effettuare le operazioni di decodifica video, mentre le due CPU ARM A9 possono rimanere in sospensione.

Nonostante la loro vocazione al lavoro con misere quantità di joule, i core ARM A9 godono di una microarchitettura molto capace. Sono in grado di decodificare ed eseguire fino a 2 istruzioni per ogni ciclo. Come avremo modo di imparare nel Capitolo 4,

questo valore rappresenta il massimo throughput della microarchitettura. Non ci si aspetti però di poter eseguire questo numero di istruzioni in ogni ciclo. Piuttosto, si pensi a questo valore come al massimo delle prestazioni garantite dal produttore, un livello che il processore non supererà mai. In molti cicli verranno eseguite meno di due istruzioni a causa della miriade di "ostacoli" che possono mettere le istruzioni in una condizione di stallo, con conseguente riduzione del throughput. Per far fronte a molte di queste limitazioni del throughput, il core ARM A9 incorpora un potente predittore di salti (*branch predictor*), un'esecuzione fuori sequenza delle istruzioni e un sistema di memoria altamente ottimizzato.

Il sistema di memoria OMAP4430 ha due cache interne principali L1 per ogni processore ARM A9: una di 32 KB per le istruzioni e una di 32 KB per i dati. Come il Core i7, OMAP4430 dispone anche di una cache di livello 2 sul chip, ma a differenza del Core i7 questa cache ha dimensioni relativamente piccole (1 MB) ed è condivisa da entrambi i core ARM A9. Le cache sono alimentate attraverso due canali DRAM LPDDR2 a bassa potenza. L'interfaccia di memoria LPDDR2 è derivata dall'interfaccia standard DDR2, modificata per richiedere un minor numero di fili e operare a tensioni che offrano maggiore efficienza energetica. Inoltre, il controllore della memoria incorpora varie ottimizzazioni per l'accesso alla memoria, come il *tiled prefetching* e il supporto alla *in-memory rotation*.

Anche se nel corso del Capitolo 4 tratteremo le cache in maggior dettaglio è utile spendere ora alcune parole al riguardo. Tutta la memoria centrale è suddivisa in linee di cache (blocchi) di 32 byte. All'interno della cache di primo livello sono mantenute le 1024 linee d'istruzioni e le 1024 linee di dati maggiormente utilizzate. Le linee di cache usate frequentemente, ma che non trovano spazio nella cache di primo livello, vengono memorizzate in quella di secondo livello. Questa cache contiene dati e istruzioni provenienti dalle due CPU ARM A9, mischiati fra loro senza alcun ordine particolare. La cache di livello 2 contiene le 32.768 linee della memoria principale utilizzate più recentemente.

Quando si verifica un fallimento (miss) nella cache di primo livello la CPU spedisce l'identificatore della linea che sta cercando (tag dell'indirizzo) alla cache di secondo livello. La risposta (tag dei dati) comunica alla CPU se la linea si trova all'interno della cache di secondo livello e, in tal caso, in quale stato si trova. Se la linea è memorizzata nella cache la CPU può ottenerla. Per estrarre un valore dalla cache di secondo livello sono necessari 19 cicli. Un simile tempo di attesa è piuttosto lungo, per questo motivo i programmatori intelligenti cercano di ottimizzare i loro programmi perché utilizzino meno dati, rendendo più probabile la presenza dei dati nella più veloce cache di livello 1.

Se la linea di cache non si trova neanche nella cache di secondo livello, occorre prelevarla dalla memoria centrale attraverso l'interfaccia LPDDR2. Questa interfaccia dell'OMAP4430 è incorporata sul chip per permettere la connessione diretta tra la DRAM e l'OMAP4430. Per accedere alla memoria, la CPU deve prima di tutto inviare la parte alta dell'indirizzo di memoria al chip DRAM utilizzando le 13 linee di indirizzo. Questa operazione, chiamata ACTIVATE, carica un'intera riga di memoria della DRAM in un buffer di riga. In seguito la CPU può emettere più comandi `READ` o `WRITE` inviando la parte restante dell'indirizzo sulle stesse 13 linee e inviando (o ricevendo) il dato sulle 32 linee di dati.

Mentre attende i risultati, la CPU può continuare a svolgere altri compiti. Per esempio, un fallimento di accesso alla cache per effettuare il prefetching di un'istruzione non impedisce alla CPU di proseguire l'esecuzione di una o più istruzioni già prelevate, ognuna delle quali può far riferimento a dati non presenti in nessuna cache. Sulle due interfacce LPDDR2 possono dunque essere effettuate più operazioni alla volta, anche da parte dello stesso processore. Tocca al controllore della memoria tener traccia di tutte queste operazioni e soddisfare le richieste di memoria nell'ordine più efficiente.

Quando finalmente i dati sono pronti, dalla memoria arrivano 4 byte alla volta. Un'operazione di memoria può utilizzare la lettura o la scrittura in una modalità detta *burst mode* che permette di leggere o scrivere diversi indirizzi contigui all'interno della stessa riga DRAM. Questa modalità è particolarmente efficiente per la lettura o scrittura di blocchi di cache. Notiamo qui che la descrizione del chip OMAP4430 appena fornita, come quella del Core i7 data in precedenza, è stata notevolmente semplificata.

Il chip OMAP4430 è distribuito in un supporto **PBGA** (*ball grid array*) con 547 pin (Figura 3.48). I PBGA sono simili agli LGA, salvo avere i collegamenti a forma di piccole sfere di metallo piuttosto che di pad quadrati. I due supporti non sono compatibili: ecco un'altra prova del fatto che non si può infilare un piolo quadrato in un buco tondo. Il supporto dell'OMAP4430 consiste in una matrice rettangolare di 28x26 sfere, con due anelli di sfere mancanti all'interno e due mezze righe e mezze colonne vuote disposte in maniera asimmetrica, utili a evitare l'inserimento non corretto del chip sul socket BGA.

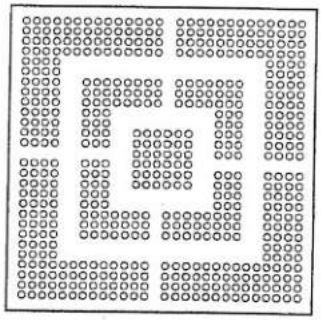


Figura 3.48 Disposizione fisica dei contatti del SoC OMAP4430.

È difficile confrontare chip CISC (come il Core i7) e chip RISC (come l'OMAP4430) basandosi soltanto sulla velocità di clock. Per esempio, i due core AMD A9 hanno una velocità di esecuzione di picco di quattro istruzioni per ciclo di clock, più o meno la stessa velocità di esecuzione del processore superscalare a 4 livelli Core i7. Il Core i7 raggiunge tuttavia una velocità di esecuzione più alta, perché dispone di fino a 6 processori che operano a una frequenza di clock di 3,5 volte più alta (3,5 GHz) rispetto all'OMAP4430. L'OMAP4430 può sembrare una tartaruga in confronto alla lepre Core i7, ma la tartaruga consuma molta meno energia e in qualche caso può arrivare prima, in particolare quando la capacità della batteria della lepre è ridotta.

3.5.3 Il microcontrollore Atmel ATmega168

Il Core i7 e l'OMAP4430 sono piattaforme ad alte prestazioni progettate per dispositivi con grandi capacità di calcolo. Il primo è destinato principalmente alle applicazioni desktop, e il secondo alle applicazioni mobili. Quando si pensa ai calcolatori si ha in mente questo tipo di sistemi. C'è però un altro intero mondo che è effettivamente ancora più vasto: quello dei sistemi integrati. In questo paragrafo daremo un rapido sguardo a questa realtà.

Forse è un po' esagerato affermare che ogni dispositivo elettrico dal costo superiore ai 100 $ contiene al suo interno un calcolatore. Tuttavia al giorno d'oggi televisori, telefoni cellulari, agende elettroniche, forni a microonde, videocamere digitali, videoregistratori, stampanti laser, allarmi antifurto, protesi acustiche, giochi elettronici e tanti altri dispositivi sono controllati mediante calcolatori. Di solito si tende a rendere più economici i calcolatori inseriti all'interno di questi oggetti piuttosto che dotarli di elevate prestazioni; ciò porta a compromessi diversi rispetto a quelli che si compiono per le CPU ad alte prestazioni studiate finora.

Come abbiamo detto nel Capitolo 1, il microcontrollore Atmel ATmega168 è molto diffuso principalmente grazie al suo basso costo (circa 1$). In più, come vedremo tra poco, è un chip particolarmente versatile che rende il collegamento agevole e poco costoso. Esaminiamo allora questo chip, la cui disposizione fisica dei contatti è mostrata nella Figura 3.49.

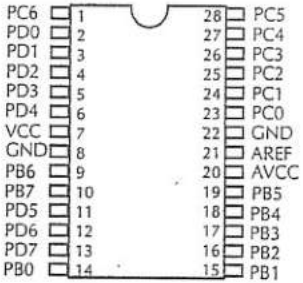


Figura 3.49 Disposizione fisica dei contatti di Atmel ATmega168.

L'usuale supporto dell'ATmega168 ha 28 pin (ma ne esistono altre varianti). Si nota probabilmente a prima vista che la disposizione dei contatti del chip è piuttosto strana rispetto agli esempi analizzati in precedenza. In particolare, il chip non è dotato di linee indirizzo né di linee dati. Ciò è dovuto al fatto che il chip non è stato progettato per essere connesso a una memoria, ma soltanto a dispositivi. Tutta la memoria (SRAM o Flash) è contenuta all'interno del processore, eliminando così il bisogno di avere pin per i dati e gli indirizzi, come mostrato nella figura 3.50.

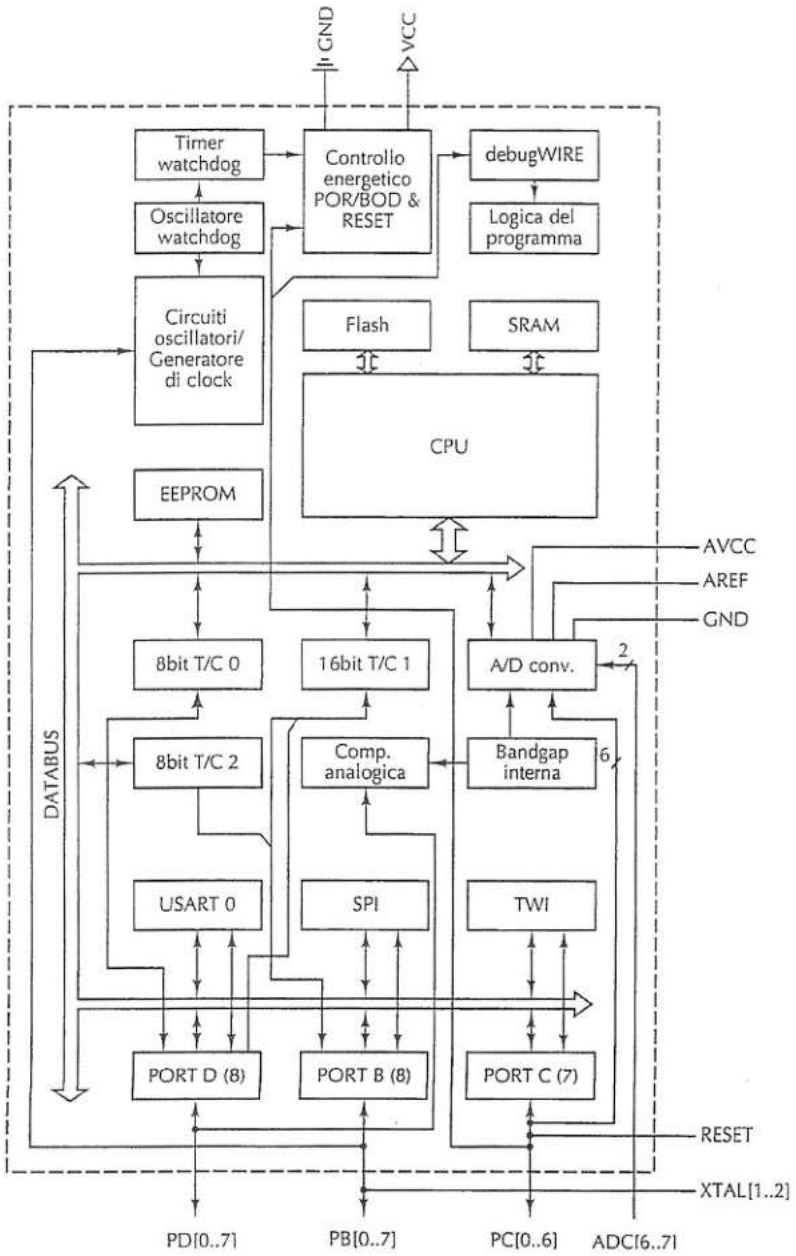


Figura 3.50 Architettura interna e disposizione logica dei contatti dell'ATmega168.

Al posto di tali pin, l'ATmega168 ha 27 porte di I/O, 8 linee nelle porte B e D e 7 linee nella porta C. Queste linee sono progettate per essere collegate alle periferiche di I/O e ognuna può essere configurata internamente dal software di avvio per essere utilizzata come input o come output. Per esempio, in un forno a microonde una linea digitale configurata in ingresso può essere il sensore di "forno aperto" e una linea in uscita può controllare l'accensione e spegnimento del generatore di microonde. Il software dell'ATmega168 controlla la chiusura del forno prima di accendere il generatore. Se il forno viene aperto all'improvviso, il software toglie l'alimentazione. Nella pratica c'è anche un sistema di blocco hardware.

Eventualmente, sei segnali in ingresso alla porta C possono essere configurati come I/O analogici. I pin analogici possono leggere il livello di tensione in ingresso o impostare il livello di tensione in uscita. Estendiamo il nostro esempio considerando un forno dotato di sensore di temperatura per riscaldare i cibi alla temperatura desiderata. Il sensore di temperatura viene connesso in ingresso alla porta C, così che il software possa leggere la tensione del sensore e trasformare questo valore in una temperatura mediante una funzione di traduzione specifica per il sensore. Gli altri pin dell'ATmega168 sono l'alimentazione (VCC), due pin messi a terra (GND) e due pin per configurare la circuiteria analogica di I/O (AREF, AVCC).

L'architettura interna dell'ATmega168 è, come nel caso di OMAP4430, un SoC con memoria e una vasta gamma di dispositivi interni. L'ATmega168 è distribuito con 16KB di memoria flash interna per la memorizzazione di informazioni che variano raramente (come le istruzioni del programma), 1KB di EEPROM e una memoria non volatile che può essere scritta dal software e nella quale vengono memorizzate le informazioni di configurazione di sistema. Nell'esempio del microonde, la EEPROM potrebbe memorizzare un bit per indicare se l'ora deve essere visualizzata in formato 12 o 24 ore. L'ATmega168 può incorporare anche 1KB di SRAM, dove il software può salvare temporaneamente le variabili.

Il processore interno esegue le 131 istruzioni AVR, ognuna lunga 16 bit. Si tratta di un processore a 8 bit, che opera su dati di 8 bit e ha registri interni da 8 bit. L'insieme di istruzioni include istruzioni speciali per permettere al processore di operare efficientemente su dati più grandi di 8 bit. Per esempio, per eseguire un'addizione su dati da 16 bit, o più grandi, il processore offre l'istruzione "somma con riporto" che somma due valori e aggiunge il riporto dell'addizione precedente. Tra i restanti componenti interni c'è un orologio in tempo reale e altre interfacce, tra cui il supporto per collegamenti seriali, i collegamenti PWM (*pulse-widht-modulated*), il collegamento I2C (*Inter-IC bus*) e i controllori di I/O digitale e analogico.

3.6 Esempi di bus

I bus sono il collante che tiene insieme i sistemi di calcolo. In questo paragrafo daremo uno sguardo dettagliato ad alcuni modelli molto diffusi: il bus PCI e l'Universal Serial Bus. Il bus PCI è il bus di I/O delle periferiche principalmente utilizzato oggi sui PC. Ne esistono di due tipi: il vecchio bus PCI e il nuovo e più veloce PCI Express (PCIe). L'Universal Serial Bus è un bus per periferiche a bassa velocità, come mouse e tastiere,

che sta guadagnando sempre più popolarità. La seconda e la terza versione del bus USB funzionano a velocità molto più elevate. Nei paragrafi successivi analizzeremo questi bus per scoprire come funzionano.

3.6.1 Bus PCI

Sul PC IBM originario la maggior parte delle applicazioni era di tipo testuale. Gradualmente, con l'introduzione di Windows, presero piede le interfacce grafiche per l'utente. Nessuna di queste applicazioni richiedeva a vecchi bus di sistema come il bus ISA un eccessivo sforzo. Tuttavia la situazione cambiò radicalmente nel momento in cui molte applicazioni, in particolare i giochi multimediali, cominciarono a utilizzare il calcolatore per visualizzare immagini a pieno schermo e in movimento.

Facciamo un semplice calcolo. Consideriamo uno schermo con la risoluzione di 1024 × 768 con 3 byte per pixel: ogni schermata contiene 2,25 MB di dati. Per ottenere un movimento senza scatti sono necessarie almeno 30 schermate al secondo per un tasso di trasferimenti dati complessivo di 67,5 MB/s. La situazione in realtà è ancora peggiore, dato che per visualizzare un video da un hard disk, da un CD-ROM o da un DVD i dati devono passare dal lettore del disco alla memoria lungo il bus. Successivamente, per essere visualizzati, i dati devono viaggiare nuovamente sul bus per raggiungere l'adattatore grafico. È necessaria quindi una larghezza di banda di 135 MB/s solo per il video, senza considerare la larghezza di banda per la CPU e altri dispositivi.

Il bus ISA, predecessore del bus PCI aveva una larghezza di banda massima di 16,7 MB/s, determinata da una velocità di 8,33 MHz e dalla capacità di trasferire 2 byte per ciclo; il bus EISA poteva invece spostare 4 byte per ciclo, raggiungendo così 33,3 MB/s. Nessuno di questi due bus si avvicinava alla larghezza di banda necessaria per visualizzare video a schermo intero.

Con i moderni schermi Full HD la situazione è ancora peggiore. Con una risoluzione di 1920x1080 e 30 frame/sec la larghezza di banda è di 155 MB/s (310 se i dati devono attraversare due volte il bus). Ovviamente il bus EISA non può nemmeno pensare di poter gestire questa situazione.

Nel 1990 Intel colse questa necessità e progettò un nuovo bus con una larghezza di banda molto più ampia rispetto allo stesso bus **EISA** (*Extended ISA*) e lo chiamò **PCI** (*Peripheral Component Interconnect*). Per incoraggiarne l'adozione, Intel brevettò il bus PCI e poi rese di pubblico dominio i brevetti, in modo che qualunque azienda potesse costruire delle periferiche per il bus senza pagare alcun diritto. Intel fondò inoltre un consorzio, chiamato *PCI Special Interest Group* per la gestione del futuro del bus. La conseguenza di queste azioni fu l'incremento della sua diffusione. A partire dal Pentium, praticamente ogni calcolatore basato su Intel, così come molti altri computer, aveva un bus PCI. Anche Sun ha una versione di UltraSPARC, l'UltraSPARC IIIi, che utilizza il bus PCI. Shanley e Anderson (1999) e Solari e Willse (2004) trattano il bus PCI molto approfonditamente.

Il bus PCI originario trasferiva 32 bit per ciclo e funzionava a 33 MHz (tempo di ciclo di 30 ns) per una larghezza di banda totale di 133 MB/s. Nel 1993 fu introdotto il PCI 2.0 e nel 1995 il PCI 2.1. La versione PCI 2.2 ha invece delle funzionalità specifiche per i calcolatori portatili (la maggior parte delle quali serve a risparmiare energia). Il bus

PCI può funzionare fino a 66 MHz e può gestire trasferimenti di 64 bit, per una larghezza di banda totale di 528 MB/s; con questa banda è possibile (assumendo che il disco e il resto del sistema siano pronti per il compito) visualizzare in modo fluido i video a tutto schermo.

Anche se 528 MB/s suonano come una velocità apprezzabile, al bus PCI restavano ancora due problemi. Primo, la larghezza di banda non era sufficiente per essere usata come bus di memoria. Secondo, non era compatibile con tutte le vecchie schede ISA ancora in circolazione. La decisione presa da Intel fu di progettare calcolatori con tre o più bus. Com'è possibile vedere nella Figura 3.51 la CPU può comunicare con la memoria centrale attraverso uno speciale bus di memoria; inoltre è presente un bus ISA collegato al bus PCI. Questa soluzione soddisfaceva tutti i requisiti e per questo motivo nel corso degli anni '90 fu adottata in modo molto diffuso.

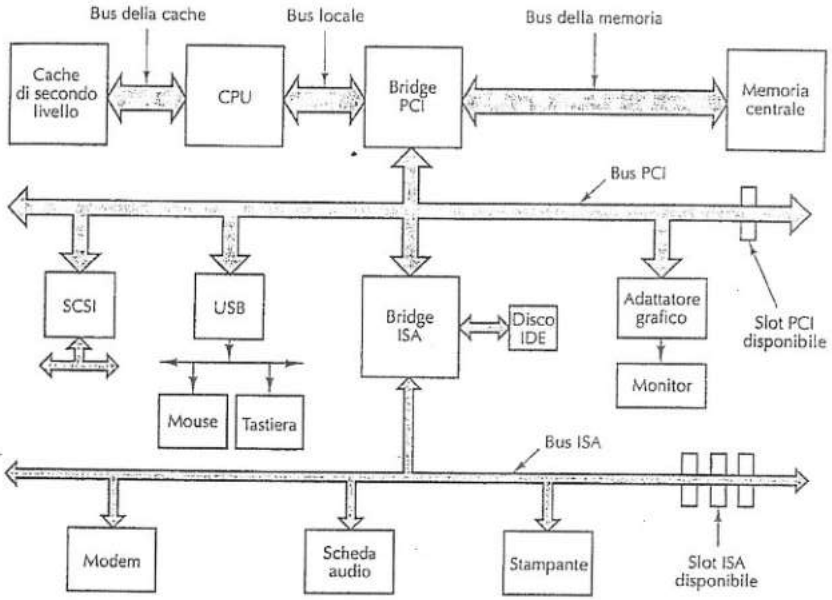


Figura 3.51 Architettura di uno dei primi Pentium. I bus più spessi hanno una maggiore larghezza di banda rispetto a quelli più sottili (la figura non è in scala).

In questa architettura le due componenti chiave sono i due chip bridge prodotti da Intel (che aveva quindi un grande interesse nell'intero progetto). Il bridge PCI connette la CPU, la memoria e il bus PCI, mentre il bridge ISA connette il bus PCI al bus ISA e supporta inoltre uno o due dischi IDE. Quasi tutti i sistemi con questa architettura hanno uno o più slot PCI liberi per aggiungere nuove periferiche ad alta velocità, e uno o più slot ISA per periferiche a bassa velocità.

Il vantaggio principale dell'architettura mostrata nella Figura 3.51 è che la CPU, usando un bus di memoria proprietario, ha una larghezza di banda estremamente alta verso la memoria. Il bus PCI offre invece un'ampia larghezza di banda per periferiche veloci come i dischi SCSI, gli adattatori grafici e così via; inoltre, grazie al bus ISA, è ancora possibile utilizzare le vecchie schede. Nella figura è presente un altro riquadro, definito USB, che si riferisce all'Universal Serial Bus e sarà trattato nel capitolo successivo.

Sarebbe stato più pratico avere un solo tipo di schede PCI. Sfortunatamente non è così, dato che vi sono varie possibilità per tensione, ampiezza e temporizzazione.

Il bus PCI supporta due diverse tensioni, dato che i calcolatori più vecchi funzionavano spesso a 5 volt, mentre con quelli più recenti si tende a usare 3,3 volt. I connettori sono gli stessi, tranne per due frammenti di plastica che impediscono all'utente di inserire una scheda a 5 volt in un bus PCI a 3,3 volt e viceversa. Fortunatamente esistono schede universali che supportano entrambe le tensioni e che possono essere inserite nei due tipi di slot. Le schede esistono anche in versione 32 bit oppure 64 bit. Quelle a 32 bit hanno 120 pin, e le altre hanno ulteriori 64 pin; l'aggiunta di pin è analoga al modo in cui fu esteso il bus del PC IBM per portarlo a 16 bit (vedi Figura 3.51). Un sistema con bus PCI che supporta schede a 64 bit può accettare anche schede a 32 bit, mentre non è vero il contrario. Infine i bus PCI e le loro schede possono funzionare a 33 oppure a 66 MHz e la scelta viene effettuata collegando un pin all'alimentazione oppure collegandolo alla terra. I connettori sono dello stesso tipo per entrambe le velocità.

Alla fine degli anni '90 quasi tutti furono d'accordo nel considerare il bus ISA ormai superato e decisero di escluderlo dalle nuove architetture. In quel periodo però le risoluzioni dei monitor aumentarono, raggiungendo in alcuni casi 1600 × 1200; inoltre crebbe la richiesta di video fluido e a tutto schermo, in particolar modo nell'area dei giochi interattivi. Intel decise quindi di aggiungere un altro bus per pilotare esclusivamente le schede grafiche. Questo bus prese il nome di **AGP** (*Accelerated Graphics Port*). La versione iniziale, AGP 1.0, funzionava a 264 MB/s. Questo bus, al quale ci si riferisce con 1x, era più lento del PCI, ma in compenso era dedicato solamente alla scheda grafica. Nel corso degli anni apparvero nuove versioni, come l'AGP 3.0 che raggiunse una larghezza di banda di 2,1 GB/s (8x). Al giorno d'oggi anche questo bus è stato soppiantato da tecnologie ancora più veloci, in particolare il bus PCI Express che può pompare la bellezza di 16 GB/sec di dati su collegamenti seriali ad alta velocità. La Figura 3.52 mostra un moderno sistema basato sul Core i7.

In un moderno sistema basato su Core i7 numerose interfacce sono integrate direttamente sul chip della CPU. I due canali di memoria DDR3 da 1333 transazioni al secondo connettono alla memoria principale e offrono una larghezza di banda aggregata di 10 GB/sec. All'interno della CPU c'è anche un canale PCI Express a 16 corsie che può essere configurato in maniera ottimale come singolo bus PCI Express a 16 bit o come due canali PCI Espress indipendenti a 8 bit. L'insieme delle 16 corsie fornisce nel complesso una larghezza di banda di 16 GB/sec verso le periferiche di I/O.

La CPU si collega al bridge principale, il P67, per mezzo di un'interfaccia seriale DMI a 20 Gbit/sec (2,5 GB/sec). Il P67 dispone di numerose interfacce I/O ad alte prestazioni, offre 8 corsie PCI Express aggiuntive e l'interfaccia per dischi SATA e implementa 14 interfacce USB 2.0, una Ethernet 10G e l'interfaccia audio.

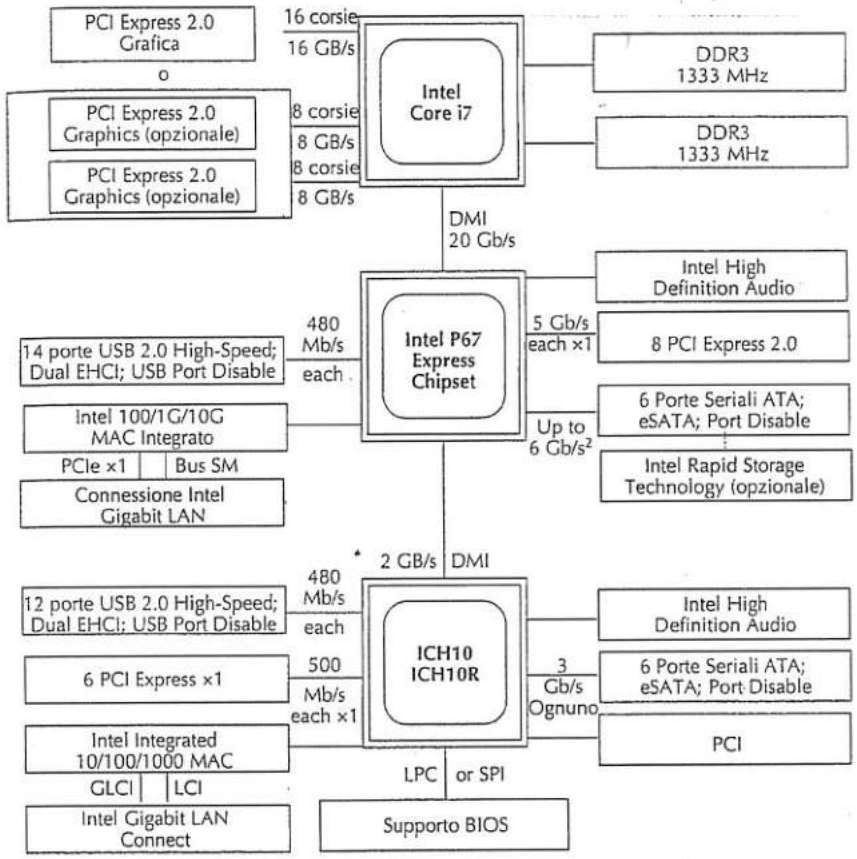


Figura 3.52 La struttura del bus di un moderno sistema Core i7.

Il chip ICH10 fornisce il supporto per le interfacce più datate e le periferiche più vecchie ed è connesso al P67 attraverso un'interfaccia DMI più lenta. ICH10 implementa il bus PCI, Ethernet 1G, porte USB, una PCI Express vecchio stile e le interfacce SATA. I nuovi sistemi potrebbero non incorporare il chip ICH10, richiesto solamente per la compatibilità con interfacce più vecchie.

Anche il bus PCI, come tutti quelli successivi al bus del PC IBM originario, è sincrono. Tutte le transazioni sul bus PCI avvengono tra un master, detto convenzionalmente **iniziatore**, e uno slave, chiamato convenzionalmente **destinatario**. Inoltre, per ridurre il numero totale di pin, le linee degli indirizzi e dei dati sono multiplexate. In questo modo sulle schede PCI sono necessari soltanto 64 pin per i segnali d'indirizzo e di dati, anche se PCI supporta sia indirizzi sia dati a 64 bit.

I pin multiplexati per gli indirizzi e per i dati funzionano nel modo seguente. Nel ciclo 1 di un'operazione di lettura il master immette l'indirizzo sul bus. Nel ciclo 2 il master rimuove l'indirizzo e il bus è invertito in modo che lo slave lo possa utilizzare; infine, nel ciclo 3, lo slave genera in output i dati richiesti. Nelle operazioni di scrittura il bus non deve essere invertito, dato che è il master a fornire sia l'indirizzo sia i dati. Ciononostante la transazione minima rimane tuttavia della durata di 3 cicli. Se non è in grado di rispondere nei tre cicli, lo slave può inserire degli stati di attesa. Sono inoltre consentiti trasferimenti di blocchi di dimensione illimitata, e diversi tipi di cicli di bus.

Arbitraggio del bus PCI

Prima di poter utilizzare il bus PCI, i dispositivi devono acquisirlo. Come mostrato nella Figura 3.53 si utilizza un arbitro centralizzato. Nella maggior parte delle architetture l'arbitro del bus è integrato in uno dei chip bridge. Ogni dispositivo PCI ha due linee dedicate che lo collegano all'arbitro: una, REQ#, per richiedere il bus, e l'altra, GNT#, per riceverne la concessione. Una nota: REQ# è la notazione PCI per $\overline{\text{REQ}}$.

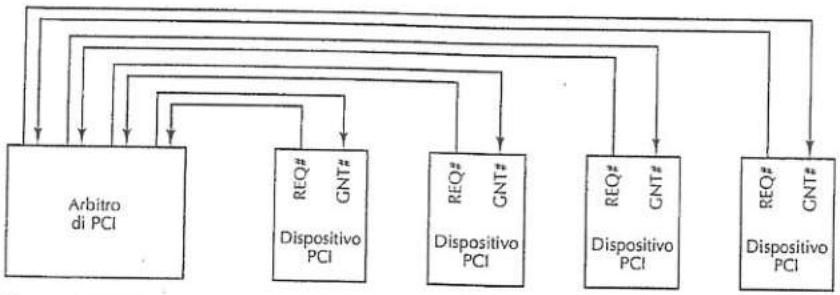


Figura 3.53 Il bus PCI utilizza un arbitro del bus centralizzato.

Per richiedere il bus un dispositivo PCI (compresa la CPU) deve asserire REQ# e attendere finché non veda che la linea GNT# è stata asserita dall'arbitro. Quando ciò si verifica il dispositivo può utilizzare il bus durante il ciclo successivo. Le specifiche PCI non definiscono l'algoritmo che l'arbitro deve utilizzare; sono consentiti arbitraggi round-robin, arbitraggi basati su priorità e altre politiche. Ovviamente un arbitro dovrebbe essere imparziale in modo che nessun dispositivo sia obbligato ad attendere all'infinito la concessione del bus.

Il bus viene concesso per una transazione alla volta, anche se può avere lunghezza teoricamente illimitata. Se un dispositivo vuole effettuare una seconda transazione e nessun altro dispositivo sta richiedendo il bus, allora può continuare a utilizzarlo; tuttavia tra la prima e la seconda transazione deve spesso essere inserito un ciclo di inattività. In particolari circostanze e in assenza di contesa per l'utilizzo del bus, un dispositivo può tuttavia effettuare transazioni in sequenza senza dover inserire un ciclo di inattività. L'arbitro può negare la linea GNT# nel caso in cui un master stia effettuando un trasferimento molto lungo e contemporaneamente qualche altro dispositivo richieda il bus. Dato che il master del bus deve monitorare la linea GNT#, esso deve rilasciare il bus al ciclo successivo non appena vede che la linea è stata negata. Questo schema permette trasferimenti molto lunghi (che sono efficienti) quando c'è un solo candidato master, ma allo stesso tempo continua a garantire una risposta veloce quando più dispositivi competono per l'utilizzo del bus.

Segnali del bus PCI

Nel bus PCI ci sono dei segnali obbligatori, mostrati nella Figura 3.54(a), e dei segnali opzionali, mostrati nella Figura 3.54(b). I pin rimanenti dei 120 (o 184) totali sono utilizzati per la tensione, la terra e altre funzioni che non elencheremo. Le colonne *Master* (iniziatore) e *Slave* (destinatario) indicano chi asserisce il segnale su una normale transazione; nel caso in cui il segnale sia asserito da un altro dispositivo (per esempio, CLK) entrambe le colonne sono lasciate vuote.

Analizziamo ora, seppur brevemente, i segnali del bus PCI, iniziando con quelli obbligatori. Il segnale CLK pilota il bus ed è in sincronia con la maggior parte degli altri segnali. Diversamente dal bus ISA, le transazioni del bus PCI iniziano in corrispondenza del fronte di discesa di CLK, che si trova nel mezzo del ciclo invece che all'inizio.

I 32 segnali AD sono destinati agli indirizzi e ai dati (per le transazioni a 32 bit). In genere l'indirizzo viene asserito durante il ciclo 1, mentre i dati durante il ciclo 3. Il segnale PAR è un segnale di parità per AD. Il segnale C/BE# è utilizzato per due scopi distinti; nel ciclo 1 contiene il comando del bus (lettura di una parola, blocco e così via), mentre nel ciclo 2 contiene una stringa di 4 bit che indica quali byte di una parola a 32 bit sono validi. Usando C/BE# è possibile leggere oppure scrivere arbitrariamente 1, 2, 3 byte oppure una parola intera.

Il segnale FRAME# viene asserito dal master per iniziare una transazione sul bus comunicando allo slave che l'indirizzo e i comandi del bus sono validi. Di solito, nelle letture, IRDY# viene asserito nello stesso momento di FRAME#; esso comunica che il master è pronto ad accettare dati in ingresso. Nel caso di una scrittura, IRDY# viene invece asserito in un secondo momento, quando i dati si trovano sul bus.

Il segnale IDSEL è legato al fatto che ogni dispositivo PCI deve avere uno spazio di configurazione di 256 byte per le informazioni sulle sue proprietà e che ogni altro dispositivo può leggere (asserendo IDSEL). La proprietà Plug-and-Play di alcuni sistemi operativi utilizza questo spazio di configurazione per determinare quali dispositivi sono presenti sul bus.

Passiamo ora ai segnali asseriti dallo slave. Il primo di questi, DEVSEL#, annuncia che lo slave ha rilevato il proprio indirizzo sulle linee AD ed è pronto a instaurare una transazione. Se DEVSEL# non viene asserito entro un certo intervallo di tempo il master considera scaduto il tempo e assume che il dispositivo indirizzato è assente oppure è guasto.

Il secondo segnale per lo slave, TRDY#, viene asserito nelle letture per annunciare che i dati sono disponibili sulle linee AD, mentre nelle scritture per comunicare che è pronto ad accettare i dati.

Segnale	Linee	Master	Slave	Descrizione
CLK	1			Clock (33 MHz oppure 66 MHz)
AD	32	×	×	Linee d'indirizzo e dei dati multiplexate
PAR	1	×		Bit di parità dell'indirizzo e dei dati
C/BE	4	×		Comando del bus/bit map per i byte abilitati
FRAME#	1	×		Indica che AD e C/BE sono asserite
IRDY#	1	×		Lettura: il master accetterà i dati; scrittura: i dati sono presenti
IDSEL	1	×		Seleziona lo spazio di configurazione invece che quello di memoria
DEVSEL#	1		×	Lo slave ha decodificato il proprio indirizzo ed è in ascolto
TRDY#	1		×	Lettura: i dati sono presenti; scrittura: lo slave accetterà i dati
STOP#	1		×	Lo slave vuole interrompere immediatamente la transazione
PERR#	1			Il destinatario ha rilevato un errore sulla parità dei dati
SERR#	1			Errore sulla parità dell'indirizzo ed errore di sistema
REQ#	1			Arbitraggio del bus: richiesta di appropriazione del bus
GNT#	1			Arbitraggio del bus: concessione del bus
RST#	1			Reinizializza il sistema e tutti i dispositivi

(a)

Segnale	Linee	Master	Slave	Descrizione
REQ64#	1	×		Richiesta di eseguire una transazione a 64 bit
ACK64#	1		×	Concessione per eseguire una transazione a 64 bit
AD	32	×		32 bit aggiuntivi per l'indirizzo o per i dati
PAR64	1	×		Parità per i 32 bit aggiuntivi (indirizzo/dati)
C/BE#	4	×		4 bit aggiuntivi per l'abilitazione dei byte
LOCK	1	×		Blocco del bus per permettere transazioni multiple
SBO#	1			Successo di una cache remota (per un multiprocessore)
SDONE	1			È stato effettuato lo snooping (per un multiprocessore)
INTx	4			Richiesta di un interrupt
JTAG	5			Segnali per i test JTAG IEEE 1149.1
M66EN	1			Collegato all'alimentazione o terra (66 o 33 MHz)

(b)

Figura 3.54 Segnali (a) obbligatori e (b) opzionali del bus PCI.

I tre segnali successivi servono a notificare un errore. Il primo è STOP#: lo slave lo asserisce se si verifica qualcosa di disastroso e se desidera cancellare la transazione corrente. Il successivo, PERR#, è utilizzato per notificare che nel ciclo precedente si è verificato un errore sulla parità dei dati. In lettura viene asserito dal master, e in scrittura dallo slave. Infine, SERR# serve a notificare errori di sistema o riguardanti gli indirizzi.

I segnali REQ# e GNT# sono utilizzati per l'arbitraggio del bus e non sono asseriti dal master corrente, ma dal dispositivo che intende diventare master del bus. L'ultimo segnale obbligatorio è RST#, utilizzato per resettare il sistema quando l'utente preme il pulsante RESET oppure quando un dispositivo notifica un errore irrimediabile. Quando questo segnale viene asserito tutti i dispositivi vengono resettati e il calcolatore viene riavviato.

Passiamo ora ai segnali opzionali, la maggior parte dei quali riguarda l'espansione da 32 a 64 bit. I segnali REQ64# e ACK64# consentono al master di richiedere l'autorizzazione a effettuare una transazione a 64 bit, e allo slave di accettarla. I segnali AD, PAR64 e C/BE# sono semplicemente un'estensione dei corrispondenti segnali a 32 bit.

I tre segnali successivi non riguardano il passaggio da 32 a 64 bit, ma i sistemi multiprocessore, una funzionalità che le schede PCI non sono obbligate a dover supportare. Il segnale LOCK permette di bloccare il bus per transazioni multiple, mentre i due successivi riguardano lo snooping necessario per mantenere la coerenza della cache.

I segnali INTx sono utilizzati per richiedere interrupt. Sulle schede PCI ci possono essere fino a quattro dispositivi logici distinti e ciascuno di loro può avere la propria linea per la richiesta di interrupt. Il segnale JTAG serve per la procedura di test JTAG IEEE 1149.1. Infine il segnale M66EN è collegato a un valore alto o basso per impostare la velocità di clock e non deve essere modificato mentre il sistema è in funzione.

Transazioni del bus PCI

Il bus PCI (come in genere tutti i bus) è in realtà molto semplice. Per entrare ancora più in sintonia con il suo funzionamento consideriamo il diagramma di temporizzazione della Figura 3.55 che mostra una transazione di lettura, seguita da un ciclo di inattività, seguito a sua volta da una transazione di scrittura, effettuate tutte dallo stesso master del bus.

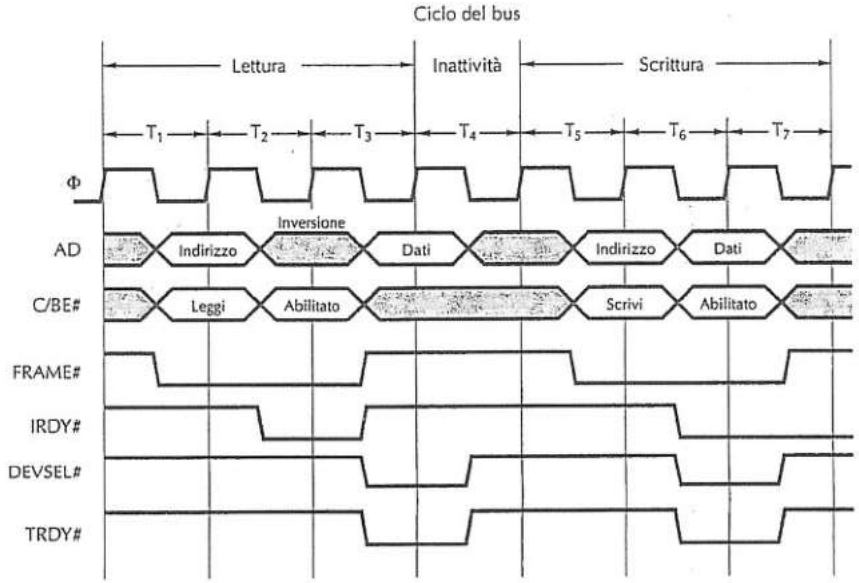


Figura 3.55 Esempi di transazioni di un bus PCI a 32 bit. I primi tre cicli sono utilizzati per un'operazione di lettura. Segue un ciclo d'inattività e infine tre cicli per un'operazione di scrittura.

Quando durante T_1 si verifica il fronte di discesa del clock, il master inserisce l'indirizzo di memoria su AD e il comando del bus su C/BE#; asserisce inoltre FRAME# per iniziare la transazione sul bus.

Durante T_2 il master rilascia l'indirizzo del bus per permettere che venga invertito in modo che lo slave possa pilotarlo durante T_3. Il master modifica inoltre C/BE# per indicare quali byte della parola indirizzata vuole abilitare (cioè leggere).

Durante T_3 lo slave asserisce DEVSEL# in modo che il master sappia che ha rilevato il proprio indirizzo e che si sta adoperando per rispondere. Inoltre scrive i dati sulle linee AD e asserisce TRDY# per notificare il fatto al master. Se lo slave non è in grado di rispondere velocemente deve tuttavia asserire DEVSEL# per annunciare la propria presenza, ma deve lasciare TRDY# negato finché non possa fornire i dati richiesti. Questa procedura potrebbe inserire uno più stati di attesa. In questo esempio (e spesso nelle situazioni reali) il ciclo successivo è inattivo. All'inizio di T_5 vediamo lo stesso master iniziare una scrittura. Come al solito inizia scrivendo sul bus l'indirizzo e il comando del bus. In questo caso però asserisce i dati già nel secondo ciclo; infatti, dato che è lo stesso dispositivo a pilotare le linee AD, non c'è bisogno di un ciclo per l'inversione del bus. In T_7 la memoria accetta i dati.

3.6.2 PCI Express

Anche se il bus PCI funziona in modo adeguato per la maggior parte delle applicazioni attuali, la richiesta di una maggior larghezza di banda per l'I/O sta creando confusione nell'architettura dei PC, un tempo più ordinata. Dalla Figura 3.53 risulta chiaro che il bus PCI non è più l'elemento centrale che tiene unite le varie parti del PC. Parte di quel ruolo è stato preso dal chip bridge.

Il cuore del problema è che ci sono sempre più dispositivi di I/O la cui velocità è eccessiva per il bus PCI. Spingere ulteriormente la frequenza di clock non è una buona idea, perché i problemi di disallineamento del bus, interferenze tra i cavi ed effetti di capacitanza renderebbero la situazione ancora peggiore. Ogni volta che un dispositivo di I/O è diventato troppo veloce per il bus PCI (come le schede grafiche, gli hard disk, le reti e così via), Intel ha aggiunto al chip bridge una nuova e speciale porta per permettere a quel dispositivo di scavalcare il bus PCI. Ovviamente questa non può essere una soluzione a lungo termine.

Un ulteriore problema del bus PCI è che le schede sono piuttosto grandi. In genere le schede PCI standard misurano 17,5 x 10,7 cm e quelle più piccole misurano 12 x 3,6 cm. Nessuna di queste schede può trovare comodamente spazio in un computer portatile, né tantomeno in un dispositivo mobile, quando invece i produttori cercano di produrre dispositivi ancora più piccoli. Alcune compagnie vorrebbero inoltre separare i componenti del PC, inserendo per esempio la CPU e la memoria in un piccolo contenitore sigillato e l'hard disk all'interno del monitor. Con le schede PCI questo non è possibile.

Sono state proposte varie soluzioni, ma quella che ha vinto un posto nei computer attuali (aveva possibilità rilevanti, dato che c'era dietro Intel) è chiamata **PCI Express**. Questo prodotto ha poco a che fare con il bus PCI, e tra l'altro non è nemmeno un vero e proprio bus, ma gli addetti al marketing non volevano lasciarsi sfuggire un nome così

conosciuto come PCI. I PC con un bus PCI Express sono oggi standard. Scopriamo ora come funziona questo bus.

Architettura di PCI Express

Il cuore della soluzione PCI Express (spesso abbreviata in PCIe) è la rinuncia al bus parallelo con i suoi numerosi master e slave e la scelta di un'architettura basata su connessioni seriali punto-a-punto ad alta velocità. Questa soluzione rappresenta un radicale elemento di rottura con la tradizione ISA/EISA/PCI e adotta molte idee provenienti dal mondo delle reti locali e in particolare dalla commutazione Ethernet. L'idea base si riduce a questo: in fondo in fondo un PC è un insieme di CPU, memoria e chip controllori di I/O, che necessitano di essere connessi fra loro. Quello che fa PCI Express è fornire un commutatore di uso generale per connettere i chip mediante collegamenti seriali. La Figura 3.56 illustra una tipica configurazione.

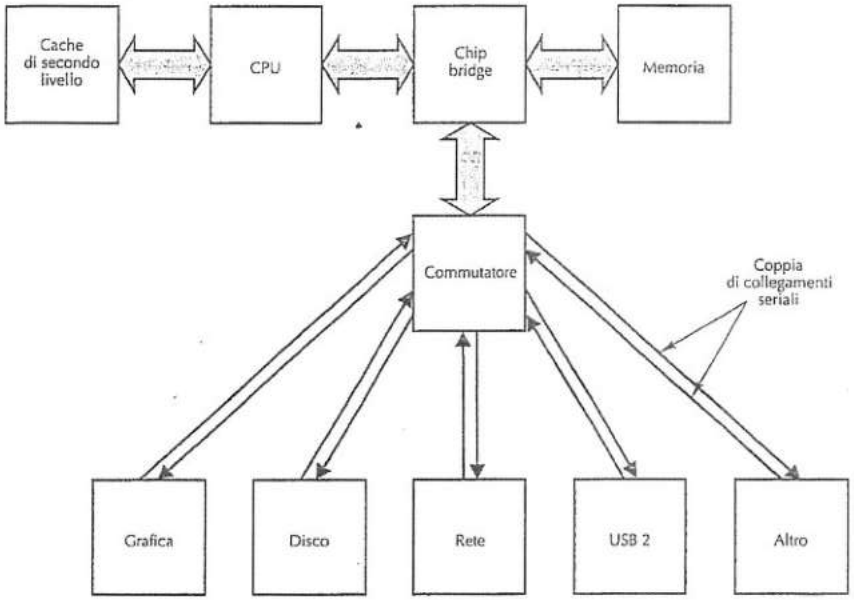


Figura 3.56 Tipico sistema PCI Express.

Come mostra la Figura 3.56, CPU, memoria e cache sono connesse al chip bridge in modo tradizionale. La novità è rappresentata da un commutatore connesso al bridge (parte integrante del bridge stesso, oppure integrato direttamente all'interno del processore). Ogni chip di I/O ha una connessione dedicata punto-a-punto con il commutatore, che consiste in una coppia di canali unidirezionali, uno che giunge al commutatore e uno

che parte da esso. Ciascun canale è composto da due fili, uno per trasportare il segnale e uno per la terra, in modo da fornire un'elevata immunità al rumore durante le connessioni ad alta velocità. Questa architettura ha sostituito le precedenti con uno schema molto più uniforme, nel quale tutti i dispositivi sono trattati in modo uguale.

L'architettura PCI Express differisce da quella vecchia basata sul bus PCI in tre aspetti principali. Due di loro sono già stati esaminati: un commutatore centralizzato al posto di un bus con più connessioni e l'uso di strette connessioni seriali punto-a-punto al posto di un ampio bus parallelo. La terza differenza è più sottile. Il modello concettuale del bus PCI è quello di un master che lancia allo slave un comando per leggere una parola oppure un blocco di parole. Il modello di PCI Express è invece quello di un dispositivo che spedisce un pacchetto di dati a un altro dispositivo. Il concetto di **pacchetto**, che consiste di un'intestazione e di un campo dati, deriva dal mondo delle reti. L'**intestazione** contiene le informazioni di controllo, eliminando quindi la necessità dei vari segnali di controllo presenti sul bus PCI; il **campo dati** contiene invece i dati che devono essere trasferiti. Un PC con PCI Express è in realtà una rete a commutazione di pacchetto in miniatura.

Oltre a queste tre principali rotture con il passato vi sono anche altre differenze. Per esempio, l'utilizzo di un codice per la rilevazione degli errori nei pacchetti fornisce un grado di affidabilità più elevato rispetto a quello di PCI. Inoltre, la connessione tra un chip e il commutatore è diventata più lunga, arrivando fino a 50 cm e permettendo il partizionamento del sistema; si introduce l'espandibilità, dato che un dispositivo può essere in realtà un altro commutatore (fino a un massimo di tre). E ancora: i dispositivi sono inseribili "a caldo", cioè possono essere aggiunti o rimossi dal sistema mentre è in funzione. Infine, dato che le connessioni seriali sono molto più piccole dei vecchi connettori PCI, i dispositivi e i calcolatori possono essere realizzati in dimensioni inferiori. Insomma, PCI Express rappresenta una deviazione molto significativa rispetto alla strada del bus PCI.

Pila di protocolli di PCI Express

Il sistema PCI Express, rimanendo aderente al modello delle reti a commutazione di pacchetto, possiede una pila stratificata di protocolli. Un **protocollo** è un insieme di regole che governano la comunicazione tra due parti. Una pila di protocolli è una gerarchia di protocolli, che gestisce diversi problemi in livelli distinti. Consideriamo per esempio una lettera commerciale. Esistono precise convenzioni riguardo il posizionamento e il contenuto dell'intestazione, l'indirizzo del destinatario, la data, la formula di saluto, il formato, la firma e così via. Tutto ciò potrebbe essere visto analogamente. Inoltre esiste un altro gruppo di convenzioni riguardo la busta, come la sua dimensione, dove e in che formato indicare gli indirizzi del mittente e del destinatario, dove affrancare il francobollo e così via. Questi due livelli e i loro protocolli sono indipendenti l'uno dall'altro. È possibile per esempio cambiare completamente l'impaginazione della lettera, pur utilizzando la stessa busta o viceversa. La stratificazione dei protocolli consente una progettazione modulare e flessibile e per decenni è stata usata ampiamente nel mondo del software delle reti. La novità è che in questo caso viene adottata nell'hardware del "bus". La pila di protocolli di PCI Express è mostrata nella Figura 3.57(a), e discussa di seguito.

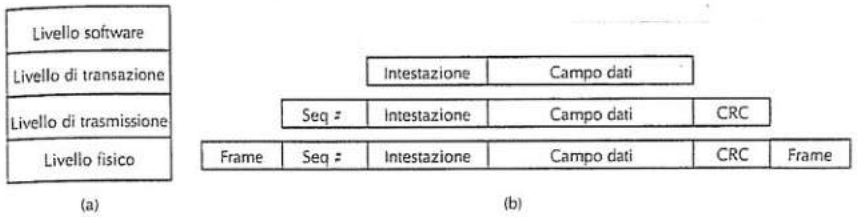


Figura 3.57 (a) Pila di protocolli di PCI Express. (b) Formato di un pacchetto.

Esaminiamo i livelli a partire dal basso. Il livello inferiore è il **livello fisico** che tratta lo spostamento dei bit da un mittente a un destinatario lungo una connessione punto-a-punto. Ciascuna connessione punto-a-punto consiste in una o più coppie di collegamenti simplex (cioè unidirezionali). Nel caso più semplice vi è soltanto una coppia in ciascuna direzione, ma se ne possono avere anche 2, 4, 8, 16 o 32. Ciascun collegamento è chiamato **corsia** e il numero di corsie in ciascuna direzione deve essere lo stesso. La prima generazione di prodotti deve supportare un tasso di trasferimento dati in ciascuna direzione di almeno 2,5 Gbps, ma ci si aspetta che la velocità in ciascuna direzione raggiungerà fra non molto i 10 Gbps.

Diversamente dai bus ISA/EISA/PCI, PCI Express non ha un clock principale. I dispositivi sono liberi di trasmettere non appena hanno dei dati da spedire; questa libertà rende più veloce il sistema, ma solleva un problema. Supponiamo che il bit 1 sia codificato con +3 volt e il bit 0 con 0 volt. Se i primi byte sono tutti 0 come può il destinatario sapere che ci sono dei dati in fase di trasmissione? Dopo tutto una sequenza di bit 0 ha lo stesso identico aspetto di un collegamento inattivo. Questo problema è risolto utilizzando quella che viene chiamata **codifica 8b/10b**, in cui si utilizza un simbolo a 10 bit per codificare 1 byte di dati effettivi. Fra tutte le 1024 possibili stringhe di 10 bit sono state opportunamente scelte quelle che hanno sufficienti transizioni del clock per tenere sincronizzati il mittente e il destinatario sul limite di un bit, anche senza l'utilizzo di un clock principale. La conseguenza della codifica 8b/10b è che un link con la capacità grezza di 2,5 Gbps può trasportare soltanto 2 Gbps di dati (netti) dell'utente.

Se il livello fisico gestisce la trasmissione di bit, il **livello di trasmissione** si occupa della trasmissione dei pacchetti. Esso prende l'intestazione e il campo dati fornitigli dal livello di transazione e vi aggiunge un numero di sequenza e un codice a correzione di errore chiamato **CRC** (*Cyclic Redundancy Check*, "controllo a ridondanza ciclica"). Il CRC viene generato eseguendo un particolare algoritmo sull'intestazione e sul campo dati. Quando il destinatario riceve un pacchetto esegue la stessa computazione sull'intestazione e sul campo dati e confronta il proprio risultato con il CRC trasportato nel pacchetto. Se corrispondono, restituisce un breve **pacchetto di ackowledgment** ("conferma") per confermare la corretta ricezione del pacchetto. Se invece i due valori non corrispondono, il destinatario richiede una ritrasmissione. In questo modo l'integrità dei dati risulta maggiore rispetto ai sistemi basati sul bus PCI, che non si preoccupano di verificare ed eventualmente ritrasmettere i dati lungo il bus.

Si utilizza inoltre un meccanismo di controllo di flusso per evitare che un mittente veloce sommerga di pacchetti un destinatario più lento. Nel meccanismo utilizzato un destinatario dà al mittente un certo numero di crediti, corrispondenti alla quantità di spazio di buffer che ha a disposizione per memorizzare i pacchetti in entrata. Quando terminano i crediti il mittente deve interrompere la spedizione finché non ottenga nuovi crediti. Questo schema, utilizzato diffusamente in tutte le reti, evita che i dati vadano persi a causa di una differenza di velocità tra il mittente e il destinatario.

Il **livello di transazione** gestisce le azioni sul bus. La lettura di una parola dalla memoria richiede due transazioni: una iniziata dalla CPU o dal canale DMA per richiedere alcuni dati e una iniziata dal mittente per fornire i dati. I compiti del livello di transazione vanno però oltre la semplice gestione di letture e scritture. Esso aggiunge un valore al pacchetto di trasmissione fornitogli dal livello di trasmissione. Per iniziare, la trasmissione può dividere ciascuna corsia in più **circuiti virtuali**, fino a un massimo di otto, ciascuno dei quali gestisce una diversa classe del traffico. Il livello di transazione può etichettare i pacchetti in base alla loro classe di traffico, che può comprendere attributi che indicano per esempio: alta priorità, bassa priorità, non effettuare lo snooping, ammettere la consegna in ordine arbitrario, e altro ancora. Il commutatore può utilizzare questi tag quando deve scegliere il nuovo pacchetto da gestire.

Ciascuna transazione utilizza uno dei quattro seguenti spazi d'indirizzi.

1. Spazio di memoria (per letture e scritture ordinarie).
2. Spazio di I/O (per indirizzare i registri del dispositivo).
3. Spazio di configurazione (per l'inizializzazione del sistema e così via).
4. Spazio dei messaggi (per la segnalazione, gli interrupt e così via).

Lo spazio di memoria e lo spazio di I/O sono simili a quelli dei sistemi attuali. Lo spazio di configurazione può essere utilizzato per implementare funzionalità come il plug-and-play. Lo spazio dei messaggi prende il ruolo di molti dei segnali di controllo esistenti, ed è necessario in quanto nessuna delle linee di controllo del bus PCI è presente in PCI Express.

Il **livello software** che interfaccia il sistema PCI Express al sistema operativo può emulare il bus PCI in modo che sia possibile utilizzare sistemi operativi non modificati. Operando in questo modo ovviamente non si trae pieno vantaggio dalla potenza di PCI Express; la retrocompatibilità è tuttavia necessaria finché i sistemi operativi non saranno modificati per utilizzare appieno PCI Express. L'esperienza mostra che ciò può richiedere un certo periodo di tempo.

La Figura 3.58(b) mostra il flusso d'informazioni. Quando viene passato un comando a livello software, esso lo passa a livello di transazione, che lo riformula in termini di intestazione e campo dati. Queste due parti sono poi passate a livello di trasmissione, che aggiunge un numero di sequenza in testa e il CRC in coda. Questo pacchetto, allungato, viene successivamente passato a livello fisico, che aggiunge a entrambe le estremità informazioni per formare un pacchetto fisico che infine può essere effettivamente trasmesso. Dal lato del destinatario si svolge il processo inverso, durante il quale vengono eliminate l'intestazione e la coda e il risultato viene passato a livello di transazione.

Il concetto secondo il quale ogni livello aggiunge nuova informazione ai dati mentre lo passa al protocollo sottostante è stato utilizzato per decenni e con grande successo nel mondo delle reti. La differenza principale tra una rete e PCI Express è che nel campo delle reti il codice dei vari livelli è quasi sempre una parte software del sistema operativo, mentre con PCI Express fa interamente parte dell'hardware del dispositivo.

PCI Express è un argomento complesso e, per maggiori informazioni, si consultino (Mayhew e Krishnan, 2003; Solari e Congdon, 2005). Si tratta anche di una tecnologia in evoluzione. Nel 2007 è stato rilasciato PCIe 2.0, con velocità di 500 MB/s per corsia e supporto per 32 corsie, per una larghezza di banda totale di 16 GB/s. Nel 2011 è quindi arrivato PCIe 3.0, che ha cambiato la codifica da 8b/10b a 128b/130b e che può eseguire 8 miliardi di transazioni al secondo, il doppio di PCIe 2.0.

3.6.3 Universal Serial Bus

I bus PCI e PCI Express sono adatti per collegare periferiche ad alta velocità al calcolatore, ma sono troppo costosi per dispositivi a bassa velocità come tastiere e mouse. Tradizionalmente i dispositivi standard di I/O erano connessi al calcolatore in un modo specifico, mentre restavano liberi alcuni slot ISA e PCI per aggiungere nuovi dispositivi. Purtroppo questo schema è stato fonte di problemi fin dall'inizio.

Per esempio, ciascun nuovo dispositivo di I/O dispone sia della scheda ISA sia di quella PCI e spesso è compito dell'utente impostare interruttori e ponticelli per assicurarsi che non vadano in conflitto con le altre schede. L'utente deve quindi aprire il contenitore, inserire attentamente la scheda, chiudere il contenitore e riavviare il sistema. Per molti utenti queste operazioni appaiono complesse e si prestano a errori; inoltre il numero di slot ISA e PCI è molto limitato (in genere due o tre). Anche con le schede plug-and-play l'utente deve aprire il calcolatore per inserire la scheda; inoltre non si risolve il problema dei pochi slot a disposizione.

Nel 1993, per risolvere questo problema, i rappresentanti di sette società (Compaq, DEC, IBM, Intel, Microsoft, NEC, e Northern Telecom) si riunirono per progettare un modo migliore per collegare a un calcolatore periferiche a bassa velocità. In seguito, a queste società se ne aggiunsero altre centinaia e, nel 1998 venne rilasciato uno standard detto **USB** (*Universal Serial Bus*, "bus seriale universale") che oggi è implementato diffusamente nei personal computer. Lo standard è descritto in modo approfondito in Anderson (1997) e Tan (1997).

In seguito sono elencati alcuni degli obiettivi delle aziende che hanno concepito originariamente USB e che hanno dato via al progetto.

1. Gli utenti non dovrebbero essere obbligati a impostare interruttori e contatti sulle schede dei dispositivi.
2. Gli utenti non dovrebbero essere obbligati ad aprire il computer per installare nuovi dispositivi di I/O.
3. Dovrebbe esserci un solo tipo di cavo per connettere tutti i dispositivi.
4. I dispositivi di I/O dovrebbero trarre la propria alimentazione dal cavo.
5. Dovrebbe essere possibile collegare fino a 127 dispositivi a un calcolatore.

6. Il sistema dovrebbe supportare dispositivi funzionanti in tempo reale (per esempio dispositivi audio, telefoni).
7. I dispositivi dovrebbero essere installabili a calcolatore acceso.
8. Non dovrebbe essere necessario riavviare il sistema dopo aver installato un nuovo dispositivo.
9. Il nuovo bus e i suoi dispositivi di I/O dovrebbero avere bassi costi di produzione.

USB raggiunge tutti questi obiettivi. USB è progettato per dispositivi a bassa velocità come tastiere, mouse, macchine fotografiche digitali, scanner, telefoni digitali e così via La Versione 1.0 ha una larghezza di banda di 1,5 Mbps, che è sufficiente per le tastiere e i mouse, mentre la Versione 1.1 funziona a 12 Mbps, un valore sufficiente per stampanti, macchine fotografiche digitali e molti altri dispositivi. La versione 2.0 offre ai dispositivi una velocità massima di 480 Mbps, sufficiente per il supporto di dischi fissi esterni, webcam ad alta definizione e interfacce di rete. Il più recente USB 3.0 spinge la velocità a 5 Gbps. Solamente il tempo ci dirà quali nuove e super-assetate applicazioni scaturiranno da questa interfaccia con larghezza di banda altissima.

Un sistema USB è composto da un **hub principale** che si collega al bus del sistema (vedi Figura 3.51) e che possiede delle prese per i cavi che connettono i dispositivi di I/O o per hub di espansione, in modo da fornire un maggior numero di prese. La topologia di un sistema USB è quindi un albero la cui radice è l'hub principale. I cavi hanno connettori diversi sull'estremità dell'hub e su quella del dispositivo, in modo da evitare che l'utente possa accidentalmente connettere due prese dell'hub.

Il cavo consiste in quattro collegamenti: due per i dati, uno per l'alimentazione (+5 volt) e uno per la terra. Il sistema di segnalazione trasmette uno 0 come una transizione di tensione e un 1 come l'assenza di transizione; in questo modo una lunga sequenza di 0 genera un impulso regolare.

Quando viene collegato un nuovo dispositivo l'hub principale rileva l'evento e interrompe il sistema operativo; quest'ultimo interroga il dispositivo per sapere di che periferica si tratta e di quanta banda ha bisogno. Se il sistema operativo decide che c'è sufficiente larghezza di banda per il dispositivo, gli assegna un indirizzo univoco (tra 1 e 127) e scarica nel dispositivo, oltre a questo indirizzo, anche altre informazioni necessarie a configurare i registri. In questo modo è possibile connettere nuovi dispositivi "al volo", senza alcuna configurazione da parte dell'utente e senza dover installare una scheda ISA o PCI. Le schede non inizializzate hanno indirizzo 0 e possono dunque essere indirizzate. Per rendere più semplici i collegamenti molti dispositivi USB contengono al loro interno un hub, in modo da poter accettare ulteriori periferiche USB. Un monitor può per esempio avere due porte USB per potervi collegare gli altoparlanti destro e sinistro.

Da un punto di vista logico il sistema USB può essere visto come un insieme di *pipe* (condotti) di bit che partono dall'hub principale e arrivano alle periferiche di I/O. Ciascun dispositivo può dividere la sua pipe in un massimo di 16 condotti per diversi tipi di dati (per esempio, audio e video). All'interno di ogni condotto i dati viaggiano dall'hub principale alla periferica o viceversa, mentre non esiste alcun traffico di dati tra due dispositivi di I/O.

Esattamente ogni 1,00 ± 0,05 ms, l'hub principale spedisce in broadcast un nuovo frame per mantenere sincronizzati tutti i dispositivi. Un frame è associato a un condotto di bit e consiste in pacchetti, il primo dei quali viaggia dall'hub principale verso il dispositivo. I pacchetti successivi di un frame possono viaggiare nello stesso senso oppure dal dispositivo all'hub principale. La Figura 3.58 mostra una sequenza di quattro frame. Come si può vedere, dato che i frame 0 e 2 non implicano alcuna azione è sufficiente un pacchetto SOF (*Start of Frame*, "inizio del pacchetto"). Questo pacchetto viene sempre spedito in broadcast a tutti i dispositivi. Per fare un esempio il frame 1 potrebbe essere un'interrogazione a uno scanner, affinché restituisca i bit che ha rilevato nell'immagine che sta scannerizzando; il frame 3 potrebbe consistere invece nella spedizione di dati a qualche dispositivo, per esempio una stampante.

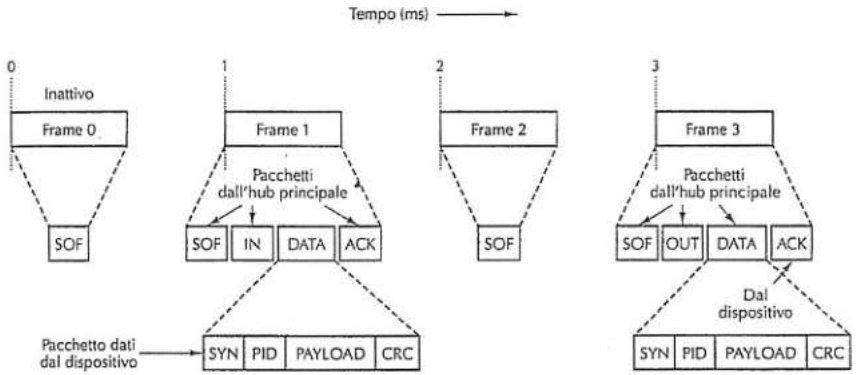


Figura 3.58 L'hub principale USB spedisce frame ogni millisecondo.

USB supporta quattro tipi di frame: controllo, isocrono, *bulk* e interrupt. I frame di controllo sono usati per configurare i dispositivi, assegnare loro dei comandi e interrogarli sul loro stato. I frame isocroni servono per i dispositivi funzionanti in tempo reale (come microfoni, altoparlanti o telefoni) che devono spedire o accettare dati a precisi intervalli temporali. Hanno un ritardo altamente prevedibile, ma non forniscono alcuna ritrasmissione in caso di errore.

I frame bulk sono utilizzati per trasferimenti di grandi dimensioni per dispositivi che non richiedono un funzionamento in tempo reale, come le stampanti. Infine i frame di interrupt sono necessari in quanto USB non supporta gli interrupt. Per esempio, invece di avere una tastiera che provoca un interrupt ogni volta che viene premuto un tasto, il sistema operativo la interroga ogni 50 ms per raccogliere i dati pendenti relativi alla pressione dei tasti.

Un frame è composto da uno o più pacchetti, alcuni dei quali possono spostarsi in entrambe le direzioni. Esistono quattro tipi di pacchetti: *token*, dati, *handshake* e pacchetti speciali. I pacchetti token (come SOF, IN e OUT della Figura 3.59) viaggiano

dall'hub principale fino alla periferica e servono per il controllo del sistema. Il pacchetto SOF (*Start of Frame*) è il primo di ogni frame e ne marca l'inizio: se non c'è alcuna operazione da compiere esso è l'unico pacchetto del frame. Il pacchetto token IN è un'interrogazione (*poll*) utilizzata per richiedere al dispositivo la resistuzione di alcuni dati. I suoi campi indicano quale flusso di bit viene interrogato in quel momento, in modo che il dispositivo possa sapere quale dato deve restituire (nel caso in cui abbia più flussi). Il pacchetto token OUT segnala che seguiranno dei dati per il dispositivo. Un quarto tipo di pacchetto token, SETUP (non mostrato nella figura), è utilizzato invece per la configurazione.

Il formato dei pacchetto dati (mostrato nella Figura 3.59) consiste in un campo di 8 bit per la sincronizzazione, in un identificatore a 8 bit del tipo di pacchetto (PID), nel campo dati e in un CRC a 16 bit per rilevare gli errori. Esistono tre tipi di pacchetti handshake: ACK (il pacchetto precedente è stato ricevuto correttamente), NAK (è stato rilevato un errore CRC) e STALL (attendere prego, in questo momento sono occupato).

Relativamente alla Figura 3.59, osserviamo che ogni millisecondo l'hub principale deve spedire un frame, anche se non deve essere effettuata alcuna operazione. I frame 0 e 2 consistono semplicemente di un pacchetto SOF, che indica che non vi è nulla da fare. Il frame 1 è un'interrogazione e inizia quindi con i pacchetti SOF e IN che viaggiano dal calcolatore alla periferica, seguiti poi da un pacchetto DATA dalla periferica al calcolatore. Il pacchetto ACK comunica alla periferica che i dati sono stati ricevuti correttamente; in caso di errore viene invece restituito un pacchetto NAK e viene rispedito il pacchetto bulk di dati (ma ciò non avviene per i dati isocroni). La struttura del frame 3 è simile a quella del frame 1, tranne per il fatto che in questo caso il flusso di dati va dal calcolatore al dispositivo.

Dopo aver terminato, nel 1998, la definizione dello standard USB, i tecnici coinvolti nel progetto, non avendo nient'altro da fare, cominciarono a lavorare a una nuova versione ad alta velocità di USB: **USB 2.0**. Questo standard è simile al vecchio USB 1.1 ed è retrocompatibile con esso, tranne per il fatto che aggiunge una terza velocità, pari a 480 Mbps, alle due già esistenti. Sono state inoltre introdotte alcune differenze di minor entità, che riguardano per esempio l'interfaccia tra hub principale e controllore. USB 1.1 ammetteva due tipi d'interfaccia. Il primo, **UHCI** (*Universal Host Controller Interface*), fu progettato da Intel lasciando gran parte dell'onere agli ingegneri software (leggi: Microsoft). Il secondo, **OHCI** (*Open Host Controller Interface*), fu progettato da Microsoft scaricando gran parte del lavoro agli ingegneri hardware (leggi: Intel). In USB 2.0 tutti si sono trovati d'accordo nell'utilizzare una nuova e unica interfaccia chiamata **EHCI** (*Enhanced Host Controller Interface*).

USB, che ora funziona a 480 Mbps, è diventato un chiaro concorrente del bus seriale IEEE 1394, indicato di solito con il nome FireWire, che raggiunge la velocità di 400 o 800 Mbps. Visto che virtualmente ogni computer basato su Intel ha il supporto USB 2.0 o USB 3.0 è probabile che, a tempo debito, 1394 sparisca. Questa perdita non sarebbe portata tanto da questioni di tecnologia, quanto piuttosto da una guerra per il territorio. USB è un prodotto dell'industria dei computer, mentre 1394 è un prodotto dell'industria dell'elettronica di consumo. Quando si deve collegare una fotocamera a un computer ogni produttore vorrebbe si usasse il suo cavo, e questa volta pare abbiano

vinto quelli dei computer. Lo standard USB 3.0 è stato annunciato otto anni dopo l'introduzione di USB 2.0.

USB 3.0 supporta sul suo cavo l'enorme larghezza di banda di 5 Gbps, sebbene la modulazione del collegamento sia adattativa e si riesca dunque a raggiungere la velocità massima solo con un cablaggio di livello professionale. I dispositivi USB 3.0 sono strutturalmente identici ai precedenti dispositivi USB e implementano totalmente lo standard USB 2.0: se collegati con una presa USB 2.0, funzionano correttamente.

3.7 Interfacce

Un classico calcolatore di piccole e medie dimensioni è composto da un chip della CPU, da un chipset, da alcuni chip di memoria e da alcuni dispositivi di I/O, tutti connessi fra loro mediante un bus. A volte tutti questi componenti vengono integrati in un unico SoC, come nel caso del TI OMAP4430. Abbiamo già esaminato alcuni dettagli delle memorie, delle CPU e dei bus. È giunto ora il momento di analizzare l'ultima tessera del puzzle, le interfacce di I/O; è attraverso queste porte che il calcolatore comunica con il mondo esterno.

3.7.1 Interfacce di I/O

Numerose interfacce di I/O sono già disponibili e con il passare del tempo ne vengono introdotte sempre di nuove. Chip molto comuni sono gli UART, gli USART, i controllori dei CRT, i controllori dei dischi e i PIO. Una **UART** (*Universal Asynchronous Receiver Transmitter*) è un'interfaccia che può leggere un byte da un bus di dati e generarlo in output, un bit alla volta, su una linea seriale per un terminale, oppure può ricevere input da un terminale. Generalmente le interfacce UART consentono velocità comprese tra 50 e 19.200 bps, dimensione dei caratteri variabili tra 5 e 8 bit, uno, uno e mezzo o due bit di stop e possono eventualmente fornire un controllo sulla parità pari o dispari; tutto questo avviene sotto il controllo del software. Le interfacce **USART** (*Universal Synchronous Asynchronous Receiver Transmitters*) possono gestire trasmissioni sincrone utilizzando vari protocolli, oltre a supportare tutte le funzionalità UART. Dato che le interfacce UART hanno perso importanza con la scomparsa dei modem telefonici, analizzeremo ora l'interfaccia parallela come esempio di chip di I/O.

Interfacce PIO

Un esempio tipico di interfaccia **PIO** (*Parallel Input/Output*), basata sul progetto originale Intel 8255A, è mostrato nella Figura 3.59. L'interfaccia è dotata di una collezione di linee di I/O (per esempio, nella figura ce ne sono 24) che possono collegare qualsiasi interfaccia di dispositivo digitale, per esempio tastiere, interruttori, luci, stampanti. In poche parole il programma della CPU può scrivere 0 o 1 su ogni linea, oppure può leggere in input lo stato di ogni linea, garantendo così un'elevata flessibilità. Un piccolo sistema basato su CPU che usa PIO può controllare diversi dispositivi fisici, come robot, tostapane o forni a microonde.

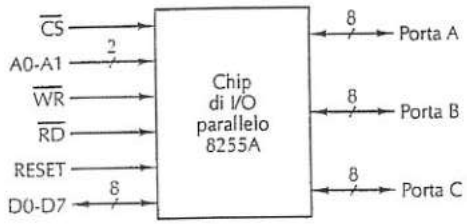


Figura 3.59 Interfaccia PIO a 24 bit.

L'interfaccia PIO viene tipicamente utilizzata nei sistemi integrati. L'interfaccia PIO viene configurata grazie a un registro di configurazione di 3 bit, che specifica se le tre porte indipendenti a 8 bit devono essere utilizzate per un segnale digitale in ingresso (0) o in uscita (1). Scrivendo il valore appropriato nel registro di configurazione è possibile impostare ogni configurazione di ingresso e uscita per le tre porte. A ciascuna porta è associato un registro latch a 8 bit. Per impostare le linee su una porta, la CPU scrive semplicemente un numero a 8 bit nel registro corrispondente e tale numero rimane sulla linea di output finché il registro non venga riscritto. Per utilizzare una porta in input, la CPU non fa che leggere il registro corrispondente.

È possibile costruire interfacce PIO più sofisticate. Per esempio, un utilizzo diffuso è per l'handshaking con dispositivi esterni. Per spedire un output verso un dispositivo che non è sempre pronto ad accettare i dati, l'interfaccia PIO può renderli disponibili su una porta di output e attendere che il dispositivo restituisca un segnale per indicare che ha accettato i dati e che ne desidera degli altri. La logica necessaria per memorizzare questi impulsi e renderli disponibili alla CPU include un segnale di ready e una coda di registri a 8 bit per ogni porta.

Dal diagramma funzionale del PIO possiamo vedere che oltre ai 24 pin necessari per le tre porte esso dispone di otto linee che lo connettono direttamente al bus dei dati, una linea per la selezione del chip, le linee per la lettura e la scrittura, due linee d'indirizzo e una linea per resettare il chip. Le due linee d'indirizzo selezionano uno dei quattro registri interni, corrispondenti alle porte A, B, C e al registro di configurazione. Di solito le due linee d'indirizzo sono connesse ai bit meno significativi del bus degli indirizzi. La linea CS permette di combinare più interfacce PIO a 24 bit per formare un'interfaccia PIO più grande, aggiungendo linee di indirizzo e utilizzandole per la selezione dell'interfaccia appropriata, individuata asserendo la corretta linea CS.

3.7.2 Decodifica dell'indirizzo

Finora siamo stati intenzionalmente vaghi circa il modo in cui la selezione del chip è asserita sulla memoria e sui chip di I/O. Per vedere con maggior attenzione come ciò si svolge, consideriamo un semplice calcolatore integrato a 16 bit composto da una CPU, una EPROM per il programma da 2KB × 8 byte, una RAM per i dati da 2KB × 8 byte e un'interfaccia PIO.

Questo piccolo sistema potrebbe essere utilizzato come prototipo del cervello di un giocattolo economico o di un semplice elettrodomestico. Una volta entrata in fase produttiva la EPROM dovrebbe essere sostituita da una ROM.

L'interfaccia PIO può essere selezionata come un vero dispositivo di I/O o come parte della memoria. Se scegliamo di usarla come un dispositivo di I/O, dobbiamo selezionarla usando una linea di bus che indica esplicitamente che ci si sta riferendo a un dispositivo di I/O e non a una memoria. Se scegliamo il secondo approccio, quello dell'**I/O mappato in memoria**, dobbiamo assegnargli 4 byte dello spazio della memoria, necessari per indirizzare le tre porte e il registro di controllo. La scelta è praticamente arbitraria; noi sceglieremo l'I/O mappato in memoria in quanto mette in evidenza alcuni interessanti problemi riguardanti l'interfaccia di I/O.

Sia la EPROM sia la RAM richiedono uno spazio degli indirizzi di 2 KB, mentre il chip PIO richiede soltanto 4 byte. Dato che lo spazio degli indirizzi del nostro esempio è di 64 KB dobbiamo semplicemente scegliere dove collocare i tre dispositivi. La Figura 3.61 mostra una scelta possibile. La EPROM occupa gli indirizzi fino a 2 KB, la RAM occupa gli indirizzi compresi tra 32 e 34 KB, e il chip PIO occupa gli ultimi 4 byte dello spazio degli indirizzi, da 65532 a 65535. Dal punto di vista del programmatore lo spazio degli indirizzi scelto non fa alcuna differenza; dal punto di vista dell'interfaccia invece sì. Se avessimo scelto d'indirizzare il chip PIO mediante lo spazio di I/O non ci sarebbe stato bisogno di alcun indirizzo di memoria (ma sarebbero stati necessari quattro indirizzi dello spazio di I/O).

Usando l'assegnamento degli indirizzi della Figura 3.60 la EPROM potrebbe essere selezionata da uno qualsiasi degli indirizzi di memoria a 16 bit della forma 00000xxxxxxxxxx (binario). In altre parole ogni indirizzo in cui i 5 bit più significativi sono 0 cade nei 2 KB bassi della memoria, cioè nella EPROM. La selezione del chip della EPROM potrebbe quindi essere collegata a un comparatore a 5 bit con input impostato al valore 00000.

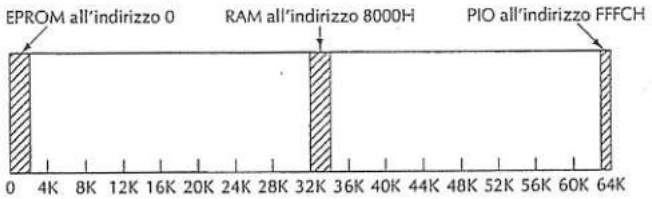


Figura 3.60 Posizione della EPROM, della RAM e del chip PIO nel nostro spazio d'indirizzamento di 64 KB.

Un modo migliore per ottenere lo stesso risultato consiste nell'usare una porta OR a cinque ingressi collegati alle linee d'indirizzo che vanno da A11 a A15. Se e solo se tutte le linee valgono 0 l'output varrà 0, asserendo di conseguenza il segnale $\overline{CS}$ (asserito con il valore basso). Questo approccio di indirizzamento è illustrato nella Figura 3.60(a) ed è chiamato decodifica piena degli indirizzi (*full-address decoding*).

Si può utilizzare lo stesso principio anche per la RAM, che però dovrebbe rispondere agli indirizzi binari della forma 10000xxxxxxxxxx. Quindi, come mostra la figura, è necessario un invertitore aggiuntivo. La decodifica degli indirizzi del chip PIO è invece leggermente più complicata, dato che il chip viene selezionato dai quattro indirizzi della forma 1111111111111xx. Nella figura è mostrato un circuito che asserisce $\overline{CS}$ soltanto quando sul bus degli indirizzi appare l'indirizzo corretto. Il circuito utilizza due porte NAND per alimentare una porta OR. Utilizzando SSI per costruire la logica di decodifica degli indirizzi della Figura 3.61(a) sono necessari sei chip: i quattro chip a otto input, una porta OR e un chip con tre invertitori.

Tuttavia, se il calcolatore è realmente composto soltanto dalla CPU, da due chip di memoria e dal chip PIO, possiamo ricorrere a un trucco che semplifica di gran lunga la decodifica degli indirizzi. Il trucco è basato sul fatto che tutti e soli gli indirizzi della EPROM hanno uno 0 nel bit più significativo, A15. Quindi, come mostra la Figura 3.61(b), possiamo semplicemente collegare $\overline{CS}$ direttamente ad A15.

A questo punto la scelta di collocare la RAM a 8000H appare meno arbitraria. La decodifica della RAM può basarsi sull'osservazione che gli unici indirizzi validi della forma10xxxxxxxxxxxxx ricadono nella RAM; è quindi sufficiente decodificare 2 bit. In modo analogo ogni indirizzo che inizia con 11 deve per forza essere un indirizzo del chip PIO. La logica per la decodifica completa può quindi essere costituita solamente da due porte NAND e da un invertitore. Dato che un invertitore è realizzabile collegando i due ingressi di una porta NAND, tutto ciò di cui abbiamo bisogno è un chip con quattro porte NAND.

La logica di decodifica degli indirizzi della Figura 3.61(b) è chiamata **decodifica parziale dell'indirizzo**, dato che non utilizza l'indirizzo completo. Una proprietà di questo metodo è che la lettura degli indirizzi 0001000000000000, 0001100000000000 o 0010000000000000 fornirà lo stesso risultato. In realtà qualsiasi indirizzo della metà inferiore della memoria selezionerà la EPROM; ciò non provocherà alcun danno dato che gli indirizzi extra non saranno utilizzati. Se però si sta progettando un calcolatore che potrà in futuro essere espanso (cosa poco probabile nel caso di un giocattolo) bisognerebbe evitare la decodifica parziale, poiché vincola eccessivamente lo spazio degli indirizzi.

Un'altra comune tecnica d'indirizzamento consiste nell'usare un decodificatore, come quello mostrato nella Figura 3.13. Connettendo i tre input alle tre linee dell'indirizzo più significative otteniamo otto output, corrispondenti agli indirizzi nei primi 8K, a quelli nei secondi 8K e così via. Per un calcolatore con otto RAM di 8K x 8, un simile chip fornisce una decodifica completa. Anche per un calcolatore con otto chip di memoria 2K x 8 è sufficiente un singolo decodificatore, fermo restando che i chip di memoria devono essere collocati in diversi settori da 8 KB dello spazio degli indirizzi. Ricordiamoci della precedente osservazione su quanto sia importante la posizione dei chip della memoria e dei chip di I/O all'interno dello spazio degli indirizzi.

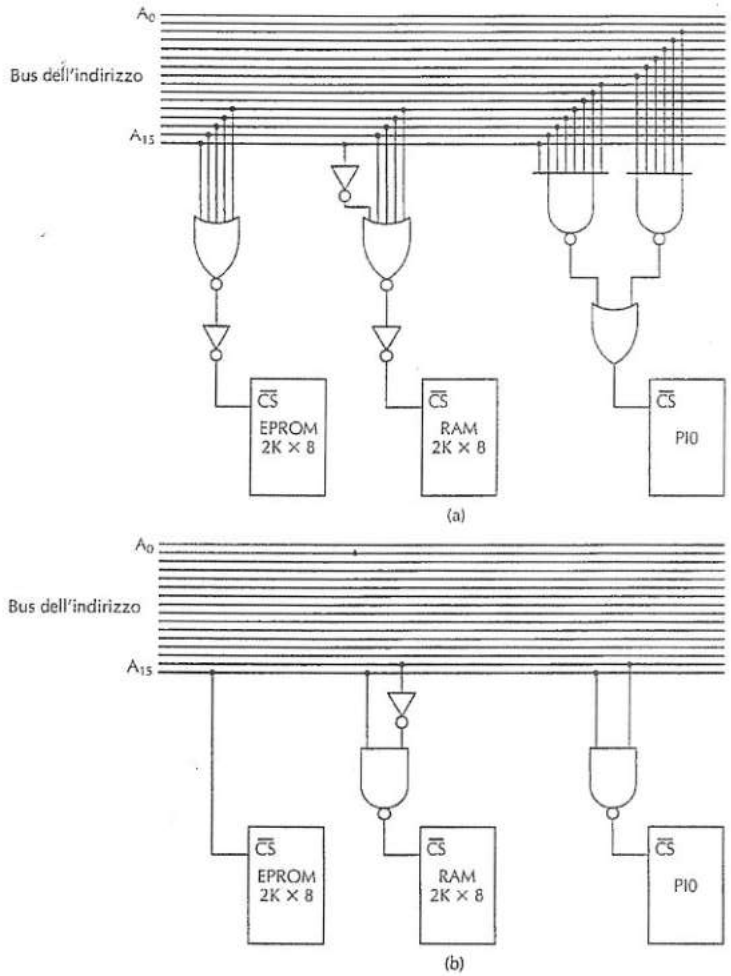


Figura 3.61 Decodifica dell'indirizzo: (a) completa; (b) parziale.

3.8 Riepilogo

I calcolatori sono costruiti a partire da chip di circuiti integrati contenenti piccoli elementi di commutazione chiamati porte logiche, tra cui le più comuni sono AND, OR, NAND, NOR e NOT. È possibile costruire semplici circuiti combinando direttamente le singole porte.

Alcuni esempi di circuiti sono i multiplexer, i demultiplexer, i codificatori, i decodificatori, gli shifter e le ALU. Utilizzando un PLA è possibile programmare funzioni booleane arbitrarie; se sono necessarie molte funzioni booleane spesso i PLA risultano ancora più efficienti. Utilizzando le leggi dell'algebra di Boole è possibile trasformare un circuito in un circuito equivalente dalla forma diversa; in molti casi usando queste tecniche è possibile produrre circuiti più economici.

L'aritmetica dei calcolatori è realizzata mediante sommatori. Per sommare una coppia di bit si possono usare due semisommatori. Un sommatore per una parola composta da più bit può essere realizzato connettendo più sommatori in cascata.

I componenti delle memorie (statiche) sono i latch e i flip-flop, ciascuno dei quali può memorizzare un bit d'informazione. Possono essere combinati linearmente in latch e flip-flop aventi una qualsiasi dimensione di parola. Le memorie possono essere di vario tipo: RAM, ROM, PROM, EPROM, EEPROM e flash. Le memorie statiche non devono essere riaggiornate, dato che mantengono i valori memorizzati fintanto che ricevono alimentazione. Le memorie dinamiche, al contrario, devono essere aggiornate periodicamente per compensare la scarica dei piccoli condensatori che si trovano sul chip.

I componenti di un sistema di un calcolatore sono connessi fra loro mediante bus. Molti pin di un classico chip della CPU guidano direttamente una linea del bus. Le linee del bus possono essere divise in linee d'indirizzo, di dati e di controllo. I bus sincroni sono guidati da un clock principale, mentre i bus asincroni usano una strategia di handshaking per sincronizzare lo slave con il master.

Il Core i7 è un esempio di moderna CPU. I sistemi che lo utilizzano hanno un bus di memoria, un bus PCIe e un bus USB. La connessione PCIe è lo strumento principale di collegamento ad alta velocità tra le parti interne di un computer. Anche l'ARM è una moderna CPU dalle elevate prestazioni, ma è pensata per i sistemi integrati e i dispositivi mobili, nei quali è importante il basso consumo di potenza. L'Atmel ATmega168 è un esempio di chip a basso costo adatto ad apparecchi piccoli ed economici e a molte altre applicazioni particolarmente sensibili ai costi.

Interruttori, luci, stampanti e molti altri dispositivi di I/O possono essere interfacciati utilizzando interfacce parallele di I/O. Secondo le necessità questi chip possono essere configurati per essere parte dello spazio di I/O o parte dello spazio di memoria. A seconda dell'applicazione possono essere decodificati parzialmente oppure in modo completo.

PROBLEMI

1. I circuiti analogici sono soggetti a rumori che provocano distorsioni alle loro uscite. I circuiti digitali sono immuni al rumore? Si discuta la risposta.

2. Un professore di logica matematica entra in un bar e dice: "Voglio un cappuccino oppure un toast e una birra". Sfortunatamente il barista, essendo stato bocciato in quinta elementare, non sa (o non se ne preoccupa) se "e" abbia oppure no la precedenza su "o" ma, per quanto gliene possa interessare, un'interpretazione vale l'altra. Quali dei seguenti casi sono un'interpretazione corretta dell'ordine (si noti che qui la parola "oppure" significa "or esclusivo").
   a. Solo un cappuccino.
   b. Solo un toast.
   c. Solo una birra.
   d. Un toast e una birra.
   e. Un cappuccino e una birra.
   f. Un toast e un cappuccino.
   g. Tutti e tre.
   h. Nulla – il professore non verrà servito visto che si è comportato da sapientone.

3. Un missionario che si è perso nella California del Sud si ferma a un bivio della strada. Sa che nella zona ci sono due bande di motociclisti, una delle quali dice sempre la verità, mentre l'altra mente sempre. Il missionario vuole sapere quale strada porta a Disneyland; quale domanda dovrebbe porre?

4. Si utilizzi una tabella di verità per mostrare la funzione $X = (X \text{ AND } Y) \text{ OR } (X \text{ AND NOT } Y)$.

5. Esistono quattro funzioni booleane di una singola variabile e 16 di due variabili. Quante funzioni di tre variabili esistono? E di n variabili?

6. Si mostri com'è possibile costruire la funzione AND mediante due porte NAND.

7. Utilizzando il multiplexer a tre vie della Figura 3.12 si implementi una funzione il cui output sia la parità degli input; l'output deve quindi essere 1 se e solo se un numero pari degli input vale 1.

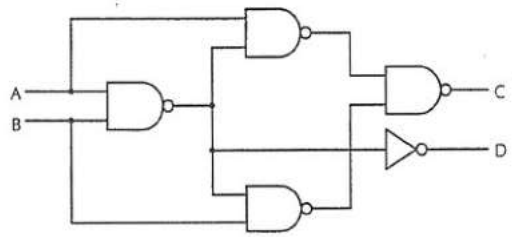
8. Proviamo a ragionare: il multiplexer della Figura 3.12 è in grado di calcolare effettivamente un'arbitraria funzione booleana a *quattro* variabili. Si dimostri in che modo, si fornisca un esempio e si disegni il diagramma logico per la funzione che vale 0 se il numero della riga della tabella di verità si scrive con un numero pari di lettere, mentre vale 1 se le lettere sono dispari. Per esempio, 0000 = zero = 4 lettere → 0; 0111 = sette = 5 lettere → 1; 1101 = tredici = 7 lettere → 1. *Suggerimento*: se chiamiamo D la quarta variabile di input possiamo collegare le otto linee di input a V_{cc}, alla terra, a D oppure a $\overline{D}$.

9. Si disegni il diagramma logico di un codificatore a 2 bit; il circuito ha quattro linee di input di cui una soltanto ha valore alto in un dato istante e due linee di output il cui valore binario a 2 bit indica quale input è alto.

10. Si disegni il diagramma logico di un demultiplexer a 2 bit; il circuito ha una singola linea di input che è indirizzata verso una delle quattro linee di output in base allo stato delle due linee di controllo.

11. Si ridisegni, con sufficiente dettaglio, il PLA delle Figura 3.15, per mostrare com'è possibile implementare la maggior parte delle funzioni logiche della Figura 3.3. In particolare si presti attenzione a mostrare quali connessioni sono presenti in entrambe le matrici.

12. Che cosa fa questo circuito?



13. Un chip MSI molto comune è il sommatore a 4 bit. È possibile agganciare quattro di questi chip per formare un sommatore a 16 bit; quanti pin ci si aspetta che avrà il sommatore? Perché?

14. Un sommatore a n bit può essere costruito collegando in cascata n sommatori in serie, in cui il riporto C_i dello stadio i giunge dall'output dello stadio $i-1$. Il riporto C_0 nello stadio 0 vale 0. Dato che ciascuno stadio richiede T ns per produrre la propria somma e il proprio riporto, il riporto allo stadio i sarà valido solo iT ns dopo l'inizio della somma. Per valori di n elevati il tempo necessario affinché il riporto giunga fino all'ultimo stadio po-

trebbe essere eccessivamente lungo. Si progetti un sommatore che funzioni più velocemente. *Consiglio*: ciascun C_i può essere espresso in termini dei bit dell'operando A_{i-1} e B_{i-1} e del riporto C_{i-1}. Utilizzando questa relazione è possibile esprimere C_i come una funzione degli input agli stadi compresi tra 0 e $i-1$; è quindi possibile generare simultaneamente tutti i riporti.

15. Si assuma che nella Figura 3.19 tutte le porte logiche abbiano un ritardo di propagazione di 1 ns e che sia possibile ignorare tutti gli altri ritardi. Qual è il tempo minimo al quale si ha la certezza che un circuito progettato in questo modo genererà un output valido?

16. La ALU della Figura 3.20 è in grado di eseguire somme a 8 bit in complemento a due. È in grado di eseguire anche le sottrazioni in complemento a due? In caso affermativo si spieghi in che modo. In caso negativo si modifichi il circuito in modo che sia in grado di compiere le sottrazioni.

17. Una ALU a 16 bit è costruita mediante 16 ALU a 1 bit, ciascuna delle quali richiede 10 ns per compiere una somma. Se è presente un ulteriore ritardo di 1 ns dovuto alla propagazione tra una ALU e la successiva, dopo quanto tempo appare il risultato di una somma a 16 bit?

18. Talvolta per una ALU a 8 bit come quella della Figura 3.20 è utile generare in output il valore costante –1. Si forniscano due modi diversi per raggiungere questo scopo. Per ciascun modo si specifichino i valori dei sei segnali di controllo.

19. Che cos'è lo stato di riposo degli input *R* e *S* di un latch SR costruito mediante due porte NAND?

20. Il circuito della Figura 3.26 è un flip-flop pilotato dal fronte di salita del clock. Si modifichi questo circuito per ottenere un flip-flop che sia pilotato dal fronte di discesa del clock.

21. La memoria 4 × 3 della Figura 3.29 utilizza 22 porte AND e tre porte OR. Quante di queste porte sarebbero necessarie se si espandesse il circuito strutturandolo come 256 × 8?

22. Per poter pagare il tuo nuovo personal computer hai deciso di dare consulenze a produttori di chip SSI con poca esperienza. Su richiesta di un'importante cliente uno di questi produttori sta pensando di costruire un chip composto da quattro flip-flop D, ciascuno contenente sia *Q* sia $\overline{Q}$. Il progetto proposto prevede inoltre che tutti e quattro i segnali di clock siano legati fra loro, mentre non sono presenti né i segnali di preset né di clear. Si dia un parere professionale sul progetto.

23. Man mano che in un singolo chip la dimensione della memoria viene ridotta, aumenta il numero di pin richiesti per indirizzarla. Spesso non è conveniente avere un grande numero di pin dell'indirizzo su un singolo chip. Si pensi a un metodo per indirizzare 2^n parole di memoria utilizzando meno di *n* pin.

24. Un calcolatore con un bus di dati largo 32 bit utilizza una RAM dinamica implementata con un chip 1Mbit × 1. Qual è la più piccola memoria (in byte) che questo calcolatore può avere?

25. Facendo riferimento al diagramma di temporizzazione della Figura 3.38 si supponga di rallentare il clock da 10 ns (com'è mostrato nella figura) a 20 ns, senza modificare i vincoli di temporizzazione. Nel caso peggiore quanto tempo impiega la memoria per prelevare i dati dal bus durante T_3 dopo che $\overline{\text{MREQ}}$ è stato asserito?

26. Facendo nuovamente riferimento alla Figura 3.38 si supponga di lasciare il clock a 100 MHz, ma di aumentare T_{DS} fino a 4 ns. Sarebbe possibile utilizzare chip di memoria da 10 ns?

27. Nella Figura 3.38(b) si specifica che TML deve essere di almeno 3 ns. È possibile immaginare un chip in cui ciò è negativo? In particolare, la CPU potrebbe asserire $\overline{\text{MREQ}}$ prima che l'indirizzo diventi stabile? Se sì, perché? Se no, perché?

28. Si assuma che il trasferimento a blocchi della Figura 3.42 venga effettuato sul bus della Figura 3.38. Nel caso di lunghi blocchi di dati quanta larghezza di banda si guadagna usando un trasferimento a blocchi invece che più trasferimenti singoli? Si assuma successivamente che il bus abbia una larghezza di 32 bit invece che di 8 bit e si risponda nuovamente alla domanda.

29. Si indichino con T_{A1} e T_{A2} i tempi di transizione delle linee d'indirizzo della Figura 3.39, con T_{MREQ1} e T_{MREQ2} i tempi di transizione di $\overline{\text{MREQ}}$ e così via. Si scrivano tutte le disuguaglianze determinate dal protocollo di full handshaking.

30. I chip multicore, con più CPU sullo stesso chip, sono diventati molto diffusi. Che vantaggi hanno rispetto a un sistema costituito da diversi PC collegati via Ethernet?

31. Perché sono improvvisamente apparsi sul mercato i chip multicore? Ci sono fattori tecnologici che hanno tracciato il cammino? La legge di Moore ha giocato qualche ruolo?

32. Qual è la differenza tra un bus di memoria e un bus PCI?

33. La maggior parte dei bus a 32 bit permette letture e scritture a 16 bit. Esiste qualche ambiguità sulla posizione in cui vengono posti i dati? Si argomenti la risposta.

34. Molte CPU hanno un tipo speciale di ciclo di bus per la conferma degli interrupt. Perché?

35. Un calcolatore a 64 bit con un bus a 400 MHz richiede quattro cicli per leggere una parola a 64 bit. Quanta larghezza di banda consuma la memoria nel caso peggiore, ovvero assumendo che ogni volta vi siano riletture e riscritture consecutive?

36. Un calcolatore a 64 bit con un bus a 400 MHz richiede quattro cicli per leggere una parola a 64 bit. Quanta larghezza di banda consuma la memoria nel caso peggiore, ovvero assumendo ogni volta continue riletture e riscritture consecutive?

37. Una CPU a 32 bit con linee d'indirizzo A2–A31 richiede che tutti i riferimenti alla memoria siano allineati; le parole devono quindi essere indirizzate a multipli di 4 byte e le mezze-parole devono essere indirizzate in corrispondenza dei byte pari. I byte possono essere ovunque. Quante sono le combinazioni legali per le letture della memoria e quanti pin sono necessari per esprimerle? Si diano due risposte e si presenti un esempio per ciascuna.

38. Le CPU moderne hanno uno, due o anche tre livelli di cache sul chip. Perché sono necessari più livelli di cache?

39. Si supponga che una CPU abbia una cache di primo livello e una di secondo livello, con tempi di accesso rispettivamente di 1 ns e 2 ns. Il tempo di accesso alla memoria centrale è di 10 ns. Se hanno successo il 20% degli accessi alla cache di primo livello e il 60% degli accessi alla cache di secondo livello, qual è il tempo medio di accesso?

40. Si calcoli la larghezza di banda necessaria per visualizzare un filmato 1280 × 960 a colori a 30 fotogrammi al secondo e in true-color. Si assuma che i dati debbano passare sul bus due volte, la prima dal CD-ROM alla memoria e la seconda dalla memoria allo schermo.

41. Quali dei segnali della Figura 3.56 non sono strettamente necessari affinché il protocollo del bus funzioni?

42. Un sistema basato su PCI Express ha collegamenti a 10 Mbps (capacità lorda). Per funzionare a 16x quanti collegamenti sono necessari in ciascuna direzione? Qual è la capacità lorda in ciascuna direzione? Quale la capacità netta?

43. Tutte le istruzioni di un calcolatore richiedono due cicli di bus, uno per prelevare l'istruzione e uno per prelevare i dati. Ciascun ciclo di bus richiede 10 ns e ciascuna istruzione 20 ns (cioè l'elaborazione interna è trascurabile). Il calcolatore ha un disco con 2048 settori, 512 byte per traccia e una rotazione di 5 ms. Rispetto alla normalità a quale percentuale si riduce la velocità del calcolatore durante un trasferimento DMA a 32 bit, considerando che ciascuno richiede un ciclo di bus?

44. Su un bus USB la massima dimensione del campo dati di un pacchetto dati isocrono è di 1023 byte. Assumendo che un dispositivo possa spedire solo un pacchetto dati per frame, qual è la massima larghezza di banda per un singolo dispositivo isocrono?

45. Nella Figura 3.61(b) si immagini di aggiungere una terza linea di input alla porta NAND che seleziona il chip PIO. Che cosa si otterrebbe se questa linea venisse connessa ad A13?

46. Si scriva un programma per simulare il comportamento di un array $m \times n$ di porte NAND a due input. Questo circuito, contenuto in un solo chip, ha j pin di input e k pin di output. I valori di j, k, m e n sono parametri della simulazione determinati a tempo di compilazione. Il programma dovrebbe iniziare leggendo una "lista dei collegamenti", in cui per ciascun collegamento sono specificati l'input e l'output. Un input può essere uno dei j pin di input oppure l'output di una porta NAND. Un output è uno dei k pin di output oppure un input di una porta NAND. Gli input non utilizzati hanno valore logico 1. Il programma, dopo aver letto la "lista dei collegamenti", dovrebbe stampare l'output di ciascuno dei 2^j possibili input. Chip come questo, composti da un array di porte, sono usati in modo diffuso per definire circuiti personalizzati su un chip; il motivo è che la maggior parte del lavoro (il collocamento dell'array di porte sul chip) è indipendente dal circuito da implementare. Da progetto a progetto cambiano soltanto i collegamenti.

47. Si scriva un programma nel linguaggio di programmazione preferito per leggere due espressioni booleane arbitrarie e vedere se rappresentano la stessa funzione. Il linguaggio di input comprende singole lettere per le variabili booleane, gli operandi AND, OR e NOT e le parentesi. Ciascuna espressione deve essere fornita su una sola linea di input. Il programma deve calcolare le tabelle di verità di entrambe le funzioni e confrontarle.

CAPITOLO 4

Livello di microarchitettura

Al di sopra del livello logico digitale si trova il livello di microarchitettura. Com'è illustrato nella Figura 1.2 il suo compito consiste nell'implementare il livello ISA (*Instruction Set Architecture*, "architettura dell'insieme d'istruzioni"). Il modo in cui viene progettato il livello di microarchitettura non dipende solamente dall'ISA che si intende implementare, ma anche dagli obiettivi di costi e prestazioni del calcolatore. Molti ISA moderni, in particolare nelle architetture RISC, sono costituiti da istruzioni semplici che generalmente è possibile eseguire in un unico ciclo di clock. Nel caso di ISA più complessi, come l'insieme di istruzioni del Core i7, l'esecuzione di una singola istruzione può invece richiedere più cicli. Per eseguire un'istruzione può essere necessario localizzare gli operandi all'interno della memoria, leggerli e infine memorizzare il risultato nuovamente in memoria. Spesso l'ordinamento di queste operazioni all'interno di una singola istruzione porta a una strategia di controllo diversa rispetto a quella degli ISA più semplici.

4.1 Esempio di microarchitettura

Sarebbe preferibile introdurre gli argomenti del capitolo spiegando i principi generali su cui si basa la progettazione della microarchitettura; sfortunatamente però non esistono principi generali: ogni ISA rappresenta infatti un caso particolare. Per questo motivo analizzeremo in modo dettagliato un esempio pratico. Come avevamo promesso nel Capitolo 1 abbiamo scelto come ISA di esempio un sottoinsieme della Java Virtual Machine e, dato che questo sottoinsieme contiene solo istruzioni su interi, abbiamo deciso di chiamarlo IJVM. Nel capitolo successivo tratteremo l'intera JVM.

Inizieremo descrivendo la microarchitettura sopra la quale implementeremo IJVM. Come abbiamo già visto nel corso del Capitolo 1, molte delle architetture che, come IJVM, contengono istruzioni complesse sono implementate mediante la microprogrammazione. IJVM, pur essendo un insieme di piccole dimensioni, rappresenta tuttavia un buon punto di partenza per descrivere il controllo e l'ordinamento delle istruzioni.

La nostra microarchitettura conterrà un microprogramma (registrato in una ROM) il cui compito sarà quello di prelevare, decodificare ed eseguire le istruzioni IJVM. Dato che abbiamo bisogno di un piccolo microprogramma che guidi in modo efficiente le singole porte logiche non possiamo utilizzare l'interprete Oracle JVM; questo interprete è stato infatti scritto in C per essere portabile e non può controllare l'hardware. Dato che l'hardware realmente utilizzato consiste solamente nei componenti elementari descritti nel Capitolo 3, in teoria il lettore, dopo aver compreso interamente il capitolo, potrebbe uscire di casa, farsi una bella scorta di transistor ed essere in grado di costruire da solo questo sottoinsieme della macchina JVM. Agli studenti che riusciranno a portare a termine il compito verrà assegnato un credito in più (e verrà offerta loro anche una seduta psichiatrica gratuita!).

Un modello convenzionale per progettare una microarchitettura consiste nel concepirla come un problema di programmazione, in cui ogni istruzione del livello ISA è una funzione che deve essere richiamata dal programma principale. In questo modello il programma principale è un semplice ciclo senza fine che determina la funzione da invocare, la richiama e poi ricomincia la propria esecuzione, come suggerito dalla Figura 2.3.

Il microprogramma ha delle variabili che costituiscono lo **stato** del calcolatore. Ogni funzione cambia il valore di almeno una delle variabili, modificando di conseguenza lo stato del calcolatore. Il *Program Counter* (PC, "contatore d'istruzioni") è una delle variabili che fanno parte dello stato e indica la locazione di memoria contenente la successiva funzione (cioè la successiva istruzione ISA) da eseguire. Durante l'esecuzione di un'istruzione il PC viene fatto avanzare in modo da farlo puntare all'istruzione successiva.

Le istruzioni IJVM sono corte e dall'aspetto semplice. Ogni istruzione è composta da alcuni campi, di solito uno o due, con uno scopo predefinito. Il primo campo di ogni istruzione è il **codice operativo** (*opcode*), che identifica il tipo d'istruzione, indicando se è di tipo ADD, di tipo BRANCH o altro. In molte istruzioni è presente un campo aggiuntivo che specifica l'operando: per esempio le istruzioni che accedono a una variabile locale devono indicare a *quale* variabile si riferiscono.

Questo modello di esecuzione, chiamato talvolta **fetch-decodifica-esecuzione**, è utile a livello astratto e può anche costituire la base per l'implementazione di ISA complessi come IJVM. In seguito descriveremo come funziona questo modello, che aspetto ha la sua microarchitettura e com'è controllato dalle microistruzioni. L'insieme delle microistruzioni compone il microprogramma e ciascuna di loro ha il controllo del percorso dati durante un ciclo. Nel corso del capitolo presenteremo e tratteremo in modo dettagliato il microprogramma.

4.1.1 Percorso dati

Il **percorso dati** è quella parte della CPU che contiene la ALU, i suoi input e i suoi output. Il percorso dati del nostro esempio è mostrato nella Figura 4.1. Anche se è stato attentamente ottimizzato per interpretare i programmi IJVM esso è tuttavia molto simile ai percorsi dati della maggior parte delle macchine. Il nostro percorso dati contiene dei registri a 32 bit, a cui abbiamo assegnato nomi come PC, SP e MDR. Alcuni di questi nomi sono già familiari, ma è importante capire che è possibile accedere a questi registri

solamente a livello di microarchitettura (cioè dal microprogramma). Il motivo per cui sono stati assegnati tali nomi è che generalmente questi registri memorizzano il valore di una variabile omonima appartenente all'architettura del livello ISA. La maggior parte dei registri può inviare il proprio contenuto sul bus B, collegato in input alla ALU. L'output della ALU guida invece lo *shifter*, che a sua volta invia il proprio risultato sul bus C; i valori di quest'ultimo possono essere scritti allo stesso tempo in uno o più registri. Per il momento non è presente il bus A (che verrà aggiunto in una fase successiva).

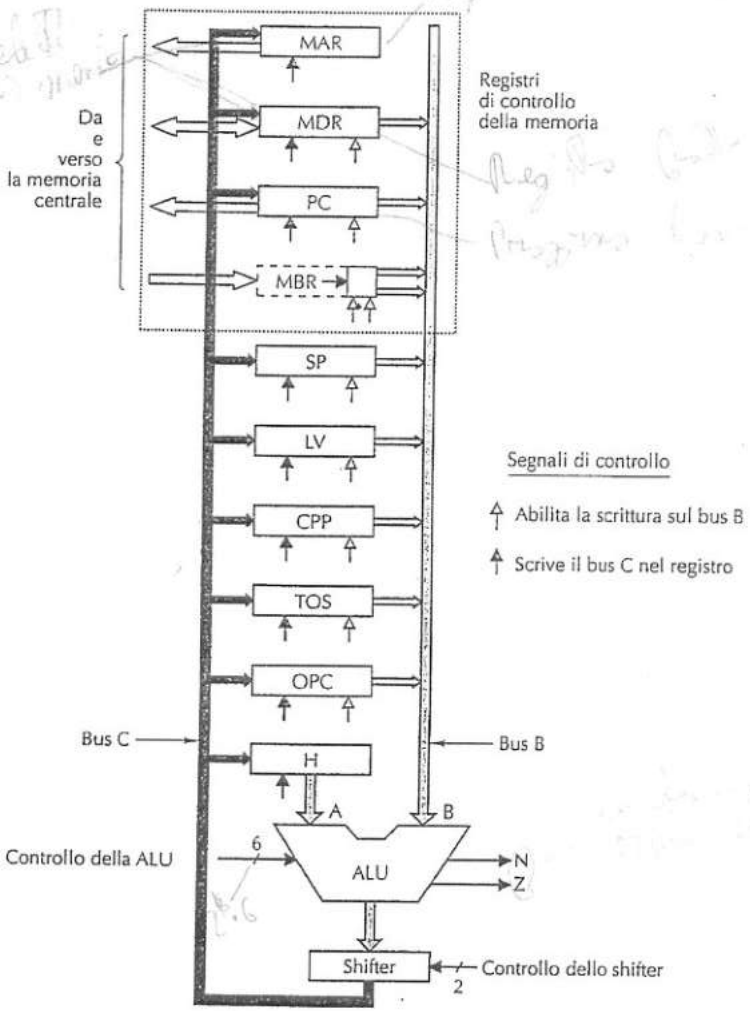


Figura 4.1 Percorso dati della microarchitettura utilizzata per l'esempio di questo capitolo.

La ALU è identica a quella vista nelle Figure 3.18 e 3.19, e la sua funzione è determinata da sei linee di controllo. Nella Figura 4.1 il trattino diagonale con a fianco il numero "6" indica che ci sono sei linee per il controllo della ALU. Fra queste F_0 e F_1 determinano l'operazione della ALU, ENA e ENB abilitano individualmente i due input, INVA inverte l'input di sinistra e INC forza la presenza di un riporto nel bit meno significativo, sommando quindi 1 al risultato. Non tutte le 64 combinazioni delle linee della ALU hanno un ruolo significativo.

La Figura 4.2 mostra alcune delle combinazioni più interessanti. Non tutte queste funzioni sono necessarie per IJVM, ma molte torneranno utili quando tratteremo l'intero insieme JVM. In molti casi esistono inoltre molteplici possibilità per raggiungere lo stesso risultato. Nello schema + e – indicano la somma e la sottrazione aritmetica; $-A$ indica il complemento a due di A.

F_0	F_1	ENA	ENB	INVA	INC	Funzione
0	1	1	0	0	0	A
0	1	0	1	0	0	B
0	1	1	0	1	0	A
1	0	1	1	0	0	B
1	1	1	1	0	0	A + B
1	1	1	1	0	1	A + B + 1
1	1	1	0	0	1	A + 1
1	1	0	1	0	1	B + 1
1	1	1	1	1	1	B – A
1	1	0	1	1	0	B – 1
1	1	1	0	1	1	–A
0	0	1	1	0	0	A AND B
0	1	1	1	0	0	A OR B
0	1	0	0	0	0	0
1	1	0	0	0	1	1
1	1	0	0	1	0	–1

Figura 4.2 Segnali di controllo in ingresso alla ALU e corrispondenti funzioni.

La ALU della Figura 4.1 richiede due dati in ingresso: un input sinistro (A) e uno destro (B). A quello di sinistra è collegato un registro H di mantenimento (*holding*), mentre al registro di destra è collegato il bus B. Quest'ultimo può essere caricato con i valori di una qualsiasi delle nove sorgenti indicate dalle nove frecce grigie la cui punta tocca il bus. Nel corso del capitolo tratteremo un'architettura di tipo differente, basata su due bus, che ci costringerà a effettuare nuove scelte e nuovi bilanciamenti.

È possibile caricare un valore in H scegliendo una funzione della ALU il cui compito sia semplicemente quello di far passare al suo interno l'input di destra (proveniente dal bus B) per poi porlo in output senza alcuna modifica. Una simile funzione può essere ottenuta attraverso la somma di due input della ALU negando però il segnale ENA, di modo che l'input di sinistra sia forzato al valore zero. Sommando zero all'input proveniente dal bus B si ottiene come risultato lo stesso valore presente sul bus B. Per poter memorizzare questo risultato in H lo si può far passare attraverso lo shifter senza fargli subire alcuna modifica.

Oltre alle funzioni appena citate, per gestire l'output della ALU è possibile utilizzare altre due linee di controllo. SLL8 (*Shift Left Logical*, "scorrimento logico a sinistra") trasla il valore a sinistra di un byte, impostando gli 8 bit meno significativi a 0. SRA1 (*Shift Right Arithmetic*, "scorrimento aritmetico a destra") trasla invece il valore di 1 bit a destra, lasciando inalterato il bit più significativo.

È possibile leggere o scrivere esplicitamente lo stesso registro durante un unico ciclo. Per incrementare SP di 1 è possibile portare il valore di SP sul bus B, disabilitare l'input sinistro della ALU, abilitare il segnale INC e memorizzare il risultato nuovamente all'interno di SP. Com'è possibile leggere e scrivere un registro nello stesso ciclo senza generare dei dati incoerenti? La soluzione risiede nel fatto che la lettura e la scrittura sono in realtà eseguite in momenti diversi all'interno del ciclo. Quando si seleziona un registro come input destro della ALU i suoi valori vengono inseriti nel bus B in una fase iniziale del ciclo e vengono poi continuamente mantenuti sul bus per tutta la durata del ciclo. La ALU compie quindi le proprie operazioni generando un risultato che giunge al bus C passando attraverso lo shifter. Verso la fine del ciclo, quando gli output della ALU e dello shifter sono sicuramente stabili, un segnale di clock dà inizio alla memorizzazione del contenuto del bus C all'interno di uno o più registri. Uno di questi registri potrebbe tranquillamente essere lo stesso dal quale proveniva l'input del bus B. Seguendo la modalità appena descritta, che sfrutta la precisa temporizzazione del percorso dati, è possibile leggere e scrivere lo stesso registro durante un solo ciclo.

Temporizzazione del percorso dati

La Figura 4.3 mostra la temporizzazione degli eventi sul percorso dati. All'inizio di ogni ciclo di clock viene generato un breve impulso. Come mostrato nella Figura 3.20(c), questo impulso può essere determinato dal clock principale. In corrispondenza del fronte di discesa dell'impulso vengono impostati i bit che piloteranno tutte le porte logiche. Quest'operazione richiede un intervallo di tempo finito e conosciuto a priori: Δw. Il registro richiesto viene quindi selezionato e il suo contenuto viene portato sul bus B; prima che il suo valore diventi stabile occorre attendere un tempo Δx. A questo punto la ALU e lo shifter, i cui circuiti combinatori hanno funzionato continuamente, hanno finalmente dati validi su cui operare. I loro output diventano stabili dopo un altro intervallo temporale, Δy. Passato un ulteriore tempo Δz i risultati vengono propagati lungo il bus C fino ai registri in cui possono essere caricati in corrispondenza del fronte di salita dell'impulso successivo. Il caricamento all'interno dei registri dovrebbe essere pilotato dal fronte del segnale ed essere veloce; in questo modo, anche se alcuni dei registri di input vengono modificati, gli effetti di queste modifiche giungeranno sul bus C solo dopo un tempo sufficientemente lungo rispetto al momento in cui sono stati caricati i registri. Inoltre, in corrispondenza del fronte di salita dell'impulso, il registro che stava alimentando il bus B smette di farlo, in preparazione del ciclo successivo. Nella figura sono indicati MPC, MIR e la memoria; i loro ruoli saranno presentati a breve.

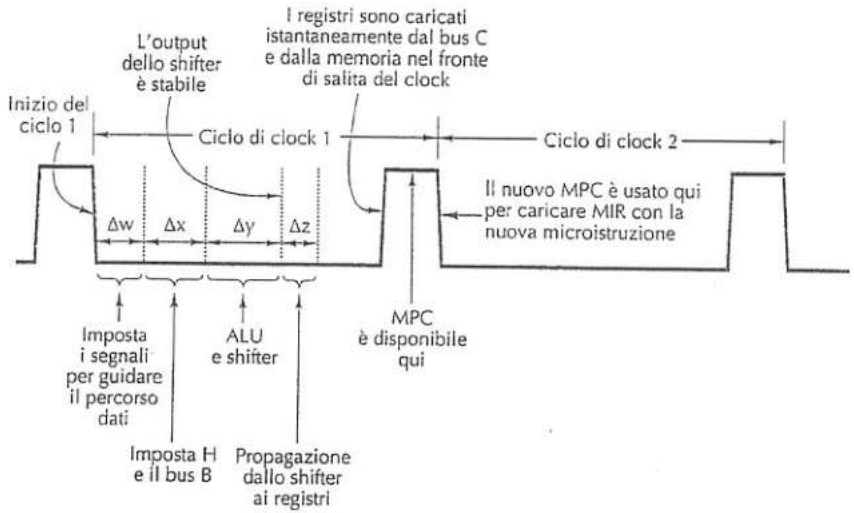


Figura 4.3 Temporizzazione di un ciclo di clock del percorso dati.

È importante rendersi conto che all'interno del percorso dati esiste un tempo di propagazione finito, anche se non sono presenti elementi di memorizzazione. Modificare il valore sul bus B non modifica il bus C se non dopo un intervallo di tempo finito (dovuto ai ritardi che vengono introdotti a ogni passo). Di conseguenza, anche se un'operazione di scrittura modifica uno dei registri di input, il valore sarà reinserito in modo sicuro al suo interno, molto tempo prima che il valore (non più corretto) che si sta inserendo sul bus B (oppure H) possa raggiungere la ALU.

Per far sì che questa architettura funzioni è necessaria una rigida temporizzazione, un lungo ciclo di clock, un ritardo di propagazione attraverso la ALU conosciuto a priori e un veloce caricamento dei registri dal bus C. Pur essendo un compito complesso, grazie a un'attenta opera di ingegneria è possibile progettare il percorso dati in modo che funzioni correttamente in ogni momento. Le macchine reali funzionano proprio in questa maniera.

Un modo diverso per vedere il ciclo del percorso dati consiste nel pensare che esso sia implicitamente diviso in più sottocicli e che l'inizio del sottociclo 1 sia guidato dal fronte di discesa del clock. Di seguito sono elencate le attività che si svolgono durante i sottocicli, accompagnate (tra parentesi) dalla durata del sottociclo corrispondente.

1. Si impostano i segnali di controllo (Δw).
2. I registri vengono caricati nel bus B (Δx).
3. La ALU e lo shifter svolgono le loro operazioni (Δy).
4. I risultati vengono propagati lungo il bus C e ritornano nei registri (Δz).

L'intervallo di tempo che segue Δz ha una certa tolleranza, perché i tempi non sono esatti. In corrispondenza del fronte di salita del ciclo successivo i risultati vengono memorizzati nei registri.

Abbiamo detto che è possibile pensare ai sottocicli come se fossero definiti *implicitamente*. Con questo vogliamo intendere che nessun impulso di clock o altro segnale esplicito comunica alla ALU quando funzionare né dice ai risultati di entrare nel bus C. In realtà la ALU e lo shifter funzionano in continuazione; tuttavia i loro input vanno considerati come inconsistenti fino al tempo $\Delta w + \Delta x$ dopo il fronte di discesa del clock. Analogamente anche i loro output sono inconsistenti finché non sia trascorso un tempo $\Delta w + \Delta x + \Delta y$ dopo il fronte di discesa del clock. Gli unici segnali espliciti che guidano il percorso dati sono il fronte di discesa del clock, che fa partire il ciclo del percorso dati, e quello di salita, che carica i registri dal bus C. I limiti degli altri sottocicli sono determinati implicitamente dai tempi di propagazione insiti nei circuiti utilizzati. È responsabilità dell'ingegnere progettista assicurarsi che il tempo $\Delta w + \Delta x + \Delta y + \Delta z$ giunga sufficientemente in anticipo rispetto al fronte di salita del clock, in modo che il caricamento dei registri possa funzionare in ogni momento.

Operazioni della memoria

La nostra macchina ha due modi diversi per comunicare con la memoria: una porta a 32 bit con indirizzi espressi in parole e una porta a 8 bit con indirizzi espressi in byte. Come mostra la Figura 4.1, la porta a 32 bit è controllata da due registri, MAR (*Memory Address Register*, "registro degli indirizzi di memoria") e MDR (*Memory Data Register*, "registro dei dati di memoria"). La porta a 8 bit è controllata invece da un unico registro, PC, che legge 1 byte negli 8 bit meno significativi di MBR. Questa porta può soltanto leggere i dati dalla memoria.

Ciascuno dei registri (così come tutti gli altri della Figura 4.1) è comandato da uno o due **segnali di controllo**. Una freccia bianca sotto un registro indica un segnale di controllo che abilita l'output del registro verso il bus B. Dato che MAR non ha una connessione al bus B, esso non ha il segnale per l'abilitazione. Neanche H ne è provvisto, dato che è l'unico possibile input sinistro della ALU ed è quindi sempre abilitato.

Una freccia nera sotto un registro indica un segnale di controllo che scrive nel registro (cioè "carica") un valore proveniente dal bus C. Dato che MBR non può essere caricato dal bus C, non ha un segnale di scrittura (anche se dispone di altri due segnali, descritti più avanti). Per iniziare una lettura o una scrittura occorre caricare il registro di memoria appropriato e successivamente inviare alla memoria un segnale di scrittura (non mostrato nella Figura 4.1).

MAR contiene gli indirizzi espressi in parole, in cui i valori 0, 1, 2, e così via, si riferiscono quindi a parole consecutive. PC contiene invece gli indirizzi espressi in byte e quindi i valori 0, 1, 2, e così via, fanno riferimento a byte consecutivi. Se si inserisce in PC il valore 2 e si fa partire un'operazione di lettura, il byte 2 della memoria verrà letto e inserito negli 8 bit meno significativi di MBR. Se invece si inserisce il valore 2 in MAR e si fa partire una lettura, saranno i byte 8–11 (cioè la parola 2) della memoria a essere letti e inseriti in MDR.

Queste due diverse modalità di accesso sono necessarie poiché MAR e PC saranno utilizzati per far riferimento a due parti diverse della memoria. In seguito risulterà più chiaro il motivo di questa distinzione, ma per il momento basta dire che la combinazione MAR/MDR è utilizzata per leggere e scrivere parole di dati del livello ISA, mentre la combinazione PC/MBR è utilizzata per leggere il programma eseguibile del livello ISA, che consiste in un flusso di byte. Tutti gli altri registri contenenti indirizzi, come MAR, utilizzano indirizzi espressi in parole.

Nelle reali implementazioni è presente un'unica memoria, orientata al byte. Attraverso un semplice espediente è possibile consentire a MAR di contare il numero di parole (cosa necessaria per il modo in cui JVM è definita) anche se gli indirizzi della memoria fisica sono espressi in byte. Quando MAR viene portato sul bus degli indirizzi i suoi 32 bit non vengono mappati direttamente sulle 32 linee, da 0 a 31. Al contrario il bit 0 di MAR viene collegato alla linea 2 del bus degli indirizzi, il bit 1 di MAR viene collegato alla linea 3 del bus degli indirizzi e così via. I 2 bit più alti di MAR vengono scartati, dato che sono necessari soltanto per indirizzi superiori a 2^{32}, nessuno dei quali è significativo per la nostra macchina che ha un limite d'indirizzamento di 4 GB. In tal modo, quando MAR vale 1, viene posto sul bus l'indirizzo 4; quando MAR vale 2, viene posto l'indirizzo 8, e così via. Questo espediente è illustrato nella Figura 4.4.

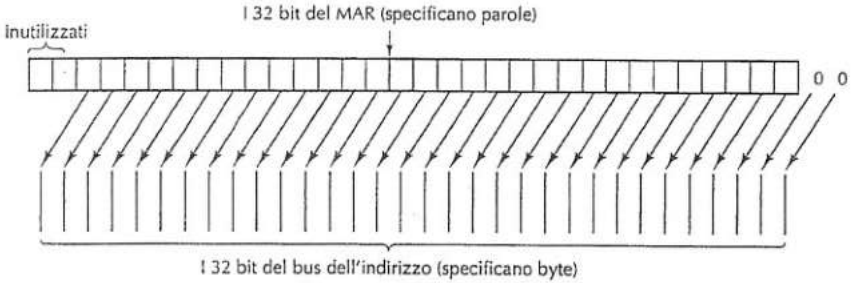


Figura 4.4 Corrispondenza tra i bit di MAR e i bit del bus dell'indirizzo.

Com'è già stato menzionato i dati letti dalla memoria attraverso la porta a 8 bit sono restituiti all'interno di MBR, un registro a 8 bit. MBR può essere copiato nel bus B in due modi distinti: con o senza segno. Quando si richiede un valore senza segno la parola a 32 bit inserita nel bus B contiene il valore di MBR negli 8 bit meno significativi, mentre i restanti 24 bit sono impostati a 0. I valori senza segno sono utili come indici di tabelle oppure quando occorre assemblare un intero a 16 bit a partire da 2 byte consecutivi (e senza segno) del flusso dati dell'istruzione.

L'altra possibilità per convertire il registro MBR a 8 bit in una parola a 32 bit consiste nel trattarlo come un valore con segno compreso tra –128 e +127 e utilizzare questo numero per generare una parola a 32 bit che abbia lo stesso valore numerico. Questa conversione viene effettuata mediante un procedimento chiamato **estensione del segno** che consiste nel duplicare il bit del segno (quello più a sinistra) di MBR nei 24 bit più alti del bus B. Quando si sceglie questa tecnica i 24 bit più alti saranno tutti 0 oppure tutti 1, a seconda che il bit più a sinistra di MBR valga 0 oppure 1.

La scelta tra convertire gli 8 bit di MBR in un valore a 32 bit con o senza segno prima di copiarlo sul bus B è determinata dal segnale di controllo (indicato nella Figura 4.1 dalle frecce bianche sotto MBR) che è asserito. La necessità di distinguere tra queste due opzioni giustifica la presenza delle due frecce. Il rettangolo tratteggiato che nella figura si trova alla sinistra di MBR indica che è possibile far sì che il registro MBR a 8 bit si comporti come una sorgente a 32 bit del bus B.

4.1.2 Microistruzioni

Per controllare il percorso dati della Figura 4.1 abbiamo bisogno di 29 segnali suddivisibili in cinque gruppi funzionali:

- 9 segnali per controllare la scrittura dei dati dal bus C all'interno dei registri;
- 9 segnali per controllare l'abilitazione dei registri sul bus B per l'input della ALU;
- 8 segnali per controllare le funzioni della ALU e dello shifter;
- 2 segnali (non mostrati) per indicare alla memoria di leggere (scrivere) attraverso MAR (MDR);
- 1 segnale (non mostrato) per indicare il prelievo dalla memoria attraverso PC o MBR.

I valori di questi 29 segnali specificano le operazioni da eseguire durante un ciclo del percorso dati. Un ciclo consiste nel portare i valori dei registri sul bus B, propagare i segnali attraverso la ALU e lo shifter, guidarli sul bus C e infine scrivere i risultati nel registro o nei registri appropriati. Inoltre, nel caso in cui sia asserito il segnale per una lettura dalla memoria, l'operazione viene fatta iniziare alla fine del ciclo del percorso dati, dopo che MAR è stato caricato. Alla fine del ciclo *seguente* i dati della memoria sono disponibili in MBR oppure in MDR e sono utilizzabili nel ciclo *ancora successivo*. In altre parole una lettura della memoria (su una delle due porte) iniziata alla fine del ciclo k trasmette dati che non possono essere utilizzati nel ciclo $k + 1$, ma soltanto a partire dal ciclo $k + 2$.

Questo comportamento apparentemente contraddittorio è spiegato nella Figura 4.3. Durante il ciclo 1 i segnali di controllo della memoria vengono generati soltanto verso la fine del ciclo, subito dopo il momento in cui MAR e PC sono caricati in corrispondenza del fronte di salita del clock. Assumeremo che la memoria inserisca i propri risultati nei bus di memoria entro un ciclo, di modo che MBR e/o MDR possano essere caricati, insieme a tutti gli altri registri, nel successivo fronte di salita del clock.

Detto in altre parole, carichiamo MAR alla fine del ciclo del percorso dati e poco dopo facciamo partire l'operazione di memoria. Di conseguenza non possiamo aspettarci che i risultati di un'operazione di lettura siano già disponibili in MDR all'inizio del ciclo successivo, soprattutto se la larghezza dell'impulso di clock è breve. Se la memoria richiede un ciclo di clock non c'è tempo a sufficienza; deve per forza passare un ciclo del percorso dati tra l'inizio di una lettura della memoria e l'utilizzo del risultato. Ovvia-

mente durante questo ciclo è possibile eseguire altre operazioni, a patto che queste non necessitino di parole dalla memoria.

L'ipotesi che la memoria richieda un ciclo per eseguire la propria operazione è equivalente ad assumere che la frequenza di successi della cache di primo livello sia pari al 100%. Questa assunzione non è mai vera, ma, per gli scopi che ci siamo posti, sarebbe troppo complesso considerare un tempo di ciclo della memoria di durata variabile.

Dato che MBR e MDR sono caricati insieme a tutti gli altri registri in corrispondenza del fronte di salita del clock, essi possono essere letti nei cicli in cui si sta svolgendo una nuova lettura della memoria. Essi restituiscono valori vecchi in quanto la lettura della memoria non ha ancora avuto il tempo di sovrascriverli e aggiornarli. Tuttavia questa situazione non presenta ambiguità; finché i nuovi valori non siano caricati in MBR e MDR nel fronte di salita del clock, i valori precedenti sono ancora presenti e utilizzabili. Dato che una lettura richiede solamente un ciclo, è possibile eseguire letture in sequenza durante due cicli consecutivi. È possibile inoltre utilizzare nello stesso momento entrambe le porte di memoria, anche se tentare di leggere e scrivere simultaneamente lo stesso byte genera risultati indefiniti.

In alcuni casi può essere utile scrivere l'output presente nel bus C in più di un registro, mentre in nessun caso ha senso abilitare nello stesso momento più di un registro sul bus B (in alcuni casi reali ciò potrebbe addirittura essere dannoso). Con pochi circuiti aggiuntivi è possibile ridurre il numero di bit necessari per la selezione delle sorgenti che alimentino il bus B. Ci sono soltanto nove possibili registri di input che possono guidare il bus B (considerando separatamente le versioni con e senza segno di MBR). Possiamo quindi codificare in 4 bit l'informazione del bus B e utilizzare un decodificatore per generare 16 segnali di controllo, 7 dei quali non vengono utilizzati. Se si trattasse di un progetto commerciale gli ingegneri subirebbero forti pressioni dai loro capi per costringerli a sbarazzarsi di uno dei registri, in modo da rendere sufficienti soltanto 3 bit. In quanto accademici noi possiamo permetterci l'enorme lusso di sprecare 1 bit semplicemente per ottenere un'architettura più semplice e ordinata.

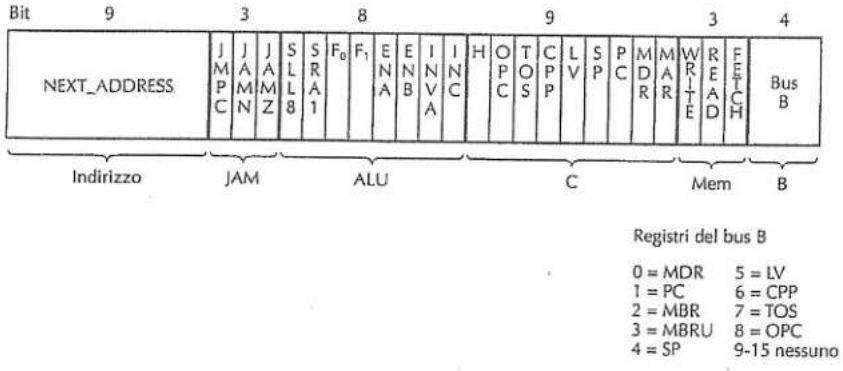


Figura 4.5 Formato delle microistruzioni di Mic-1 (da descrivere brevemente).

A questo punto possiamo controllare il percorso dati con 9 + 4 + 8 + 2 + 1 = 24 segnali, quindi con 24 bit. Tuttavia questi bit controllano il percorso dati soltanto per un ciclo. La seconda parte del controllo consiste invece nel determinare che cosa deve essere effettuato durante il ciclo successivo. Per includere questo aspetto nel progetto del controllore definiremo un formato che ci permetterà di descrivere le operazioni da eseguire utilizzando i 24 bit di controllo più due campi aggiuntivi: NEXT_ADDRESS e JAM. Il loro contenuto sarà descritto a breve. La Figura 4.5 mostra un possibile formato, diviso in sei gruppi (elencati sotto l'istruzione) e contenente i 36 segnali seguenti.

Addr – Contiene l'indirizzo di una potenziale successiva microistruzione.
JAM – Determina come viene selezionata la successiva microistruzione.
ALU – Seleziona le funzioni della ALU e dello shifter.
C – Seleziona quali registri sono scritti dal bus C.
Mem – Seleziona la funzione della memoria.
B – Seleziona la sorgente del bus B; la codifica è mostrata nella figura.

In teoria l'ordinamento dei gruppi è arbitrario, anche se in realtà quello della Figura 4.6 è stato scelto attentamente in modo da minimizzare l'incrocio fra le linee. Nei diagrammi schematici simili alla Figura 4.6 l'incrocio fra linee spesso corrisponde a collegamenti che si incrociano sui chip. Dato che causano difficoltà nei progetti è buona norma cercare di minimizzarli.

4.1.3 Unità di controllo microprogrammata: Mic-1

Finora abbiamo descritto come viene controllato il microprogramma, ma non abbiamo ancora spiegato come si decide quale dei segnali di controllo abilitare durante ciascun ciclo. Ciò è determinato da un **sequenzializzatore** che ha la responsabilità di far avanzare passo passo la sequenza di operazioni necessarie per eseguire una singola istruzione ISA.

Durante ogni ciclo il sequenzializzatore deve produrre due tipi d'informazione:

1. lo stato di ogni segnale di controllo del sistema;
2. l'indirizzo della microistruzione da eseguire subito dopo.

La Figura 4.6 è un diagramma a blocchi dettagliato dell'intera microarchitettura della nostra macchina di esempio, che chiameremo **Mic-1**. A prima vista potrebbe fare un po' paura, ma vale la pena studiarlo attentamente. Quando avremo compreso il significato di tutti i rettangoli e di tutte le linee della figura, saremo sulla giusta strada per una piena comprensione del livello di microarchitettura. Il diagramma a blocchi è composto da due parti principali: il percorso dati, sulla sinistra, già approfonditamente analizzato e la sezione di controllo, sulla destra, che considereremo a partire da questo momento.

L'elemento più grande e più importante della sezione di controllo è una memoria chiamata **memoria di controllo**. Anche se a volte viene implementata come un insieme di porte logiche, può essere utile pensarla come una memoria che contiene l'intero microprogramma. In generale ci riferiremo a essa con il nome di memoria di controllo per non confonderla con la memoria principale, a cui si accede mediante i registri MBR e

MDR. In ogni caso, dal punto di vista funzionale, la memoria di controllo è semplicemente un circuito che memorizza le microistruzioni invece che le istruzioni ISA. Nel nostro caso contiene 512 parole, consistenti in una microistruzione a 36 bit del tipo mostrato nella Figura 4.5. In realtà non sono necessarie tutte queste parole, ma (per ragioni che spiegheremo tra poco) abbiamo bisogno di indirizzi per 512 parole distinte.

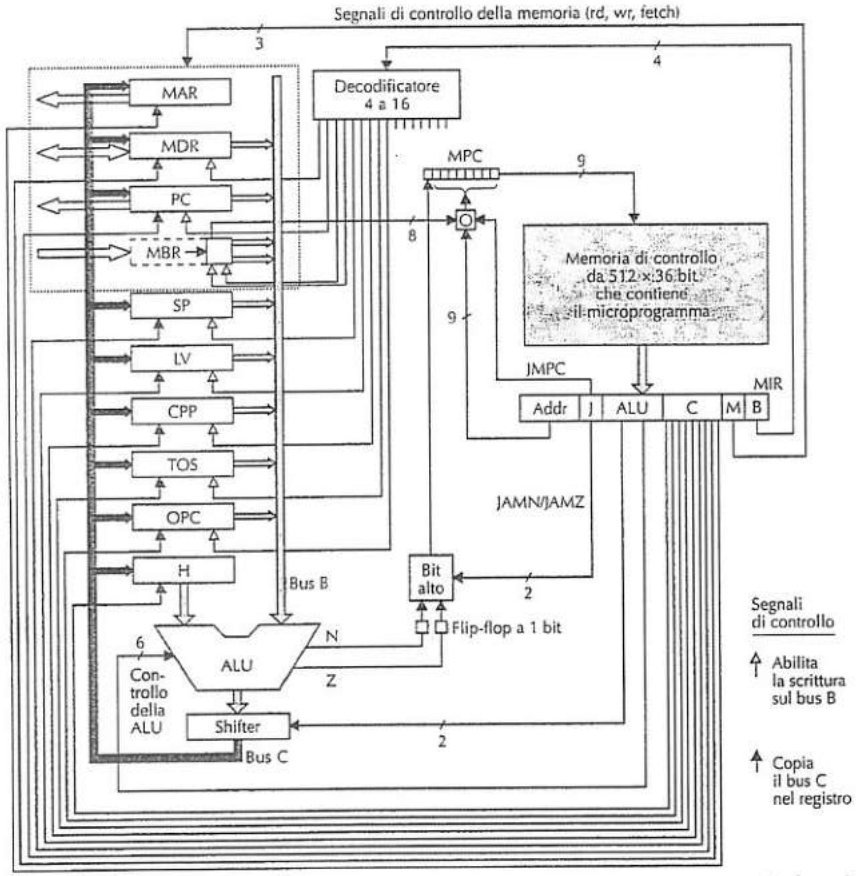


Figura 4.6 Diagramma a blocchi del nostro esempio di microarchitettura di Mic-1.

Vai a 302

La memoria di controllo differisce dalla memoria centrale per un aspetto importante: le istruzioni della memoria centrale sono sempre eseguite nell'ordine determinato dagli indirizzi (tranne nel caso delle diramazioni), mentre così non è nel caso delle microistruzioni. Nella Figura 2.3 incrementare il contatore di programma significa che la successiva istruzione da eseguire corrisponde a quella che segue l'istruzione corrente all'interno della memoria.

I microprogrammi richiedono maggior flessibilità (dato che le sequenze delle microistruzioni tendono a essere brevi) e quindi, solitamente, ogni microistruzione specifica in modo esplicito il proprio successore.

Anche la memoria di controllo necessita di un proprio registro di indirizzo e di un proprio registro dei dati (ma non dei segnali di lettura e scrittura dato che la memoria viene letta in continuazione). Chiameremo **MPC** (*MicroProgram Counter*) il registro degli indirizzi della memoria di controllo. Questo nome è incongruo, dato che le locazioni non seguono un ordine preciso; esse sono specificate in modo esplicito e quindi non c'è bisogno di effettuare alcun conteggio (ma come si fa a mettere in discussione la tradizione?). Il registro dei dati della memoria viene invece chiamato **MIR** (*MicroInstruction Register*, "registro della microistruzione corrente"). Il suo ruolo consiste nel memorizzare la microistruzione in corso di esecuzione, i cui bit determinano i segnali di controllo che guidano il percorso dati.

Nella Figura 4.6 il registro MIR contiene gli stessi sei gruppi mostrati nella Figura 4.5. I gruppi Addr e J (iniziale di JAM) controllano la selezione della microistruzione successiva e verranno illustrati a breve. Il gruppo della ALU contiene 8 bit che selezionano la funzione della ALU e guidano lo shifter. I bit C indicano in quali registri verrà caricato l'output della ALU dal bus C. I bit M controllano le operazioni della memoria.

Infine gli ultimi 4 bit guidano il decodificatore, che determina quale registro deve essere portato sul bus B. Nel nostro esempio abbiamo scelto di utilizzare un decodificatore standard da 4 a 16, anche se servono solamente nove possibilità. In un progetto ottimizzato anche nei minori dettagli si sarebbe utilizzato un decodificatore da 4 a 9. In questo caso si è deciso di utilizzare un circuito standard già disponibile invece di progettarne appositamente uno nuovo. L'utilizzo di circuiti standard è più semplice e meno soggetto a errori.

Il funzionamento (Figura 4.6) è il seguente. All'inizio di un ciclo di clock (il fronte di discesa del clock nella Figura 4.3) la parola contenuta nella memoria di controllo e puntata da MPC viene trasferita in MIR. Il tempo necessario per effettuare questa operazione è indicato nella figura con Δw. In termini di sottocicli, si può pensare che MIR venga caricato durante il primo sottociclo.

Subito dopo, i vari segnali si propagano all'interno del percorso dati. Grazie a questi segnali il contenuto di un registro viene inserito nel bus B e la ALU sa quale operazione deve eseguire. Seguono poi varie azioni, fino ad arrivare al secondo sottociclo. Dopo un intervallo di $\Delta w + \Delta x$ dall'inizio del ciclo, gli input della ALU diventano stabili.

Dopo un altro Δy, tutto nel circuito si è stabilizzato, compresi gli output della ALU, di N, di Z e dello shifter. I valori di N e Z vengono quindi salvati in una coppia di flip-flop. Questi bit, come tutti i registri caricati dal bus C, vengono salvati in corrispondenza del fronte di salita del clock, verso la fine del ciclo del percorso dati. L'output della ALU non viene memorizzato, ma semplicemente inserito nello shifter. Le attività della ALU e dello shifter si svolgono nel corso del sottociclo 3.

Dopo un ulteriore intervallo di tempo Δz, l'output dello shifter raggiunge i registri attraverso il bus C. I registri possono successivamente essere caricati verso la fine del

ciclo (nel fronte di salita dell'impulso del clock della Figura 4.3). Nel sottociclo 4 vengono caricati i registri e i flip-flop N e Z. Il sottociclo termina leggermente dopo il fronte di salita del clock, quando tutti i risultati sono stati salvati, i risultati delle precedenti operazioni di memoria sono disponibili e MPC è stato caricato. Questo processo continua ripetutamente finché qualcuno non si stufi e spenga la macchina.

Il microprogramma, oltre a guidare il percorso dati, deve parallelamente determinare quale sarà la microistruzione successiva, dato che non è necessario che esse vengano eseguite nello stesso ordine in cui appaiono nella memoria di controllo. Il calcolo dell'indirizzo della microistruzione successiva comincia dopo che MIR è stato caricato ed è diventato stabile. Inizialmente i 9 bit del campo NEXT_ADDRESS vengono copiati all'interno di MPC. Mentre si svolge questa copia viene ispezionato il campo JAM; se il suo valore è 000, allora non viene eseguita alcuna azione aggiuntiva. Quando termina la copia di NEXT_ADDRESS, MPC punta alla microistruzione successiva.

Se invece uno o più bit di JAM valgono 1, allora è necessario compiere delle azioni. Se JAMN è abilitato, se ne calcola l'OR logico con il flip-flop N (1 bit) e si memorizza il risultato nel bit più significativo di MPC. Analogamente, se è abilitato JAMZ, si calcola l'OR tra questo segnale e il flip-flop Z. Se sono abilitati entrambi, si calcola invece l'OR rispetto a tutti e due. La ragion per cui sono necessari i flip-flop N e Z è che, dopo il fronte di salita del clock (mentre il clock è ancora alto), il bus B non viene più alimentato e quindi gli output della ALU non possono più essere considerati corretti. Salvando in N e Z i flag di stato della ALU è possibile rendere stabili questi valori, per poterli utilizzare per calcolare correttamente MPC, indipendentemente dalle operazioni che la ALU compie nel frattempo.

Nella Figura 4.6 i componenti logici che effettuano questa elaborazione sono etichettati con la scritta "Bit alto". La funzione booleana che calcola questo bit è la seguente:

F = (JAMZ AND Z) OR (JAMN AND N) OR NEXT_ADDRESS[8]

Si noti che in tutti i casi possibili MPC può assumere soltanto uno di questi due valori:

1. il valore di NEXT_ADDRESS
2. il valore di NEXT_ADDRESS, in cui il bit più significativo è calcolato in OR con 1.

Non esistono altre possibilità. Se il bit più significativo di NEXT_ADDRESS valesse già 1, non avrebbe alcun senso utilizzare JAMN o JAMZ.

Si noti che quando tutti i bit di JAM valgono zero, l'indirizzo della successiva microistruzione da eseguire è semplicemente il numero a 9 bit contenuto nel campo NEXT_ADDRESS. In caso contrario, esistono due potenziali indirizzi per la microistruzione successiva: NEXT_ADDRESS e NEXT_ADDRESS calcolato in OR con 0x100 (assumendo che NEXT_ADDRESS ≤ 0xFF). (Si noti che 0x indica che il numero che lo segue è espresso in forma esadecimale.) Questo punto è illustrato nella Figura 4.7. La microistruzione corrente, che si trova nella locazione 0x75, ha il campo NEXT_ADDRESS = 0x92 e JAMZ impostato a 1. Di conseguenza l'indirizzo della microistruzione successiva dipende dal bit Z memorizzato durante la precedente operazione della ALU. Se il bit Z vale 0, la microistruzione successiva comincerà a partire dall'indirizzo 0x92; altrimenti, l'indirizzo sarà 0x192.

Il terzo bit del campo JAM è JMPC. Se è impostato a 1, gli 8 bit di MBR sono collegati in OR con gli 8 bit meno significativi del campo NEXT_ADDRESS della microistruzione precedente. Il risultato viene inviato a MPC. Il quadratino etichettato con "O" nella Figura 4.6 esegue l'OR di MBR e NEXT_ADDRESS quando JMPC vale 1, mentre lascia semplicemente passare NEXT_ADDRESS in MPC quando JMPC vale 0. In genere quando JMPC vale 1, gli 8 bit meno significativi di NEXT_ADDRESS valgono 0. Il bit più significativo può essere 0 oppure 1 e quindi il valore di NEXT_ADDRESS utilizzato con JMPC è generalmente 0x000 oppure 0x100. Il motivo per cui a volte si usa 0x000 mentre altre volte si usa 0x100 sarà spiegato in seguito.

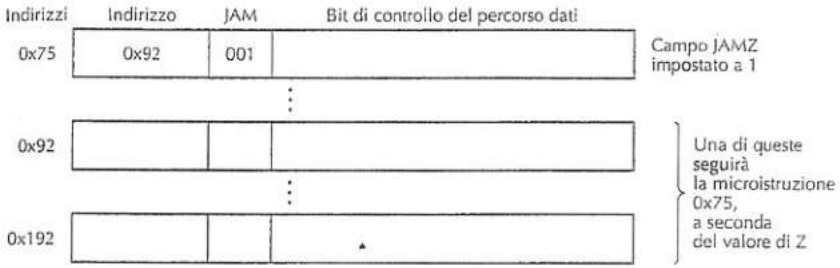


Figura 4.7 Le microistruzioni con JAMZ = 1 hanno due possibili successori.

La possibilità di calcolare l'OR di MBR e NEXT_ADDRESS e di memorizzare il risultato in MPC permette di implementare in modo efficiente una diramazione. Si noti che è possibile specificare uno qualsiasi dei 256 indirizzi unicamente sulla base dei bit presenti in MBR.

In un utilizzo tipico MBR contiene un codice operativo; in questo modo l'uso di JMPC fornisce, per ogni possibile codice operativo, un unico indirizzo in cui trovare la successiva microistruzione da eseguire. Questo metodo è utile per effettuare velocemente un salto verso la funzione associata al codice operativo appena prelevato.

Dato che la comprensione della temporizzazione della macchina è di fondamentale importanza per gli argomenti che seguiranno, vale la pena ripetere nuovamente il concetto. Questa volta lo faremo in termini di sottocicli, per permetterne una più facile visualizzazione; non dimentichiamoci però che gli unici reali eventi del clock sono il fronte di discesa, che fa partire il ciclo, e il fronte di salita, che carica i registri e i flip-flop N e Z. Si guardi per favore, ancora una volta, la Figura 4.3.

Durante il sottociclo 1, inziato in corrispondenza del fronte di discesa del clock, la microistruzione all'indirizzo contenuto in quel momento in MPC viene caricato all'interno di MIR. Durante il sottociclo 2 i segnali si propagano da MIR verso l'esterno e il registro selezionato viene caricato sul bus B. Durante il sottociclo 3 la ALU e lo shifter svolgono le proprie operazioni e generano un risultato stabile. Durante il sottociclo 4 il bus C, i bus di memoria e i valori della ALU diventano stabili. In corrispondenza del

fronte di salita si verificano vari eventi: il contenuto del bus C viene caricato nei registri, i flip-flop N e Z vengono caricati, e MBR e MDR ottengono i loro risultati dall'operazione di memoria iniziata alla fine del precedente ciclo del percorso dati (se presente). Non appena MBR diventa stabile, si carica MPC in previsione della successiva microistruzione. MPC ottiene quindi il proprio valore verso la metà dell'intervallo, quando il clock è alto, ma dopo che MBR/MDR sono diventati disponibili. Il momento esatto in cui caricare MPC potrebbe essere determinato dal livello del segnale (invece che dal fronte) oppure da un ritardo temporale fisso rispetto al fronte di salita del clock. Ciò che veramente conta è che MPC non venga caricato finché non siano pronti i registri da cui dipende (MBR, N e Z). Non appena il clock scende MPC può indirizzare la memoria di controllo dando così inizio a un nuovo ciclo.

Si noti che ogni ciclo è autocontenuto; esso specifica che cosa andrà sul bus B, quali operazioni la ALU e lo shifter dovranno effettuare, dove verrà memorizzato il bus C e, infine, quale dovrà essere il successivo valore di MPC.

Vale la pena fare un'ultima osservazione riguardo alla Figura 4.6. Finora abbiamo considerato MPC come un normale registro, con una capacità di 9 bit, che viene caricato mentre il clock è alto. In realtà non c'è bisogno di utilizzare un registro: tutti i suoi input possono alimentare direttamente la memoria di controllo. È sufficiente che siano inviati alla memoria di controllo in corrispondenza del fronte di discesa del clock, quando MIR viene selezionato e letto; non c'è alcun bisogno di memorizzare effettivamente questi segnali all'interno di MPC. Per questa ragione MPC può essere implementato come un **registro virtuale**, cioè il luogo in cui raccogliere dei segnali e che assomiglia più a un pannello elettronico che a un vero e proprio registro. Il fatto di rendere MPC un registro virtuale semplifica la temporizzazione: in questo caso gli eventi si verificano solamente in corrispondenza dei fronti di salita e discesa del clock e in nessun altro momento. Se qualcuno ritiene che sia più facile interpretare MPC come un vero e proprio registro, può continuare a farlo, in quanto ciò non è altro che un diverso, ma comunque valido, punto di vista.

4.2 Esempio di ISA: IJVM

Continuiamo il nostro esempio introducendo il livello ISA della macchina che sarà interpretato dal microprogramma eseguito nella microarchitettura della Figura 4.6 (IJVM). Per praticità a volte ci riferiremo all'architettura dell'insieme d'istruzioni (ISA) con il termine di **macroarchitettura**, in contrapposizione con la microarchitettura. Prima di descrivere IJVM faremo una breve digressione che ci permetterà di metterne in luce le motivazioni.

4.2.1 Stack

Praticamente tutti i linguaggi di programmazione supportano il concetto di procedure (metodi), dotate di un insieme di variabili locali. È possibile accedere a queste variabili dall'interno della procedura, ma ciò diventa impossibile una volta che la procedura termina. La domanda che sorge è: "In quale parte della memoria bisogna memorizzare queste variabili?".

La soluzione più semplice, che consiste nell'assegnare a ciascuna variabile un indirizzo assoluto della memoria, non funziona. Il problema nasce dal fatto che una procedura potrebbe richiamare se stessa; nel corso del Capitolo 5 studieremo le procedure che si comportano in questo modo e che sono dette ricorsive. Per il momento è sufficiente dire che se una procedura è attiva (cioè è stata invocata) due volte, è impossibile memorizzare le sue variabili in locazioni assolute della memoria in quanto la seconda invocazione interferirebbe con la prima.

Si usa quindi un'altra strategia. Per memorizzare le variabili viene riservata un'area della memoria, chiamata **stack** ("pila"), al cui interno però non si stabiliscono indirizzi assoluti per le singole variabili. Si imposta invece un registro, chiamiamolo LV, in modo che punti alla base delle variabili locali per la procedura corrente. Nella Figura 4.8(a) è stata richiamata una procedura *A*, che possiede tre variabili locali, *a1*, *a2* e *a3*; per memorizzarle è stato quindi riservato uno spazio di memoria a partire dalla locazione puntata da LV. Un altro registro, SP, punta alla parola che si trova nella locazione più alta all'interno dello stack delle variabili locali di *A*. Se LV è 100 e le parole sono di 4 byte, il valore di SP sarà 108. Per far riferimento alle variabili locali si fornisce il loro spiazzamento (*offset*) rispetto a LV. La struttura dati compresa tra LV e SP (considerando anche le parole puntate dai due registri) è chiamata **blocco delle variabili locali** di *A*.

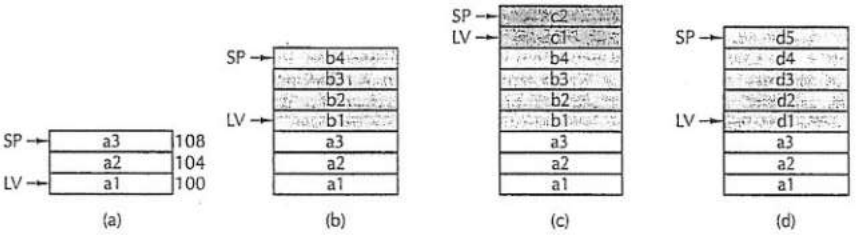


Figura 4.8 Utilizzo dello stack per memorizzare le variabili locali. (a) Mentre *A* è attiva. (b) Dopo che *A* ha richiamato *B*. (c) Dopo che *B* ha richiamato *C*. (d) Dopo che *C* e *B* hanno restituito il controllo e *A* ha richiamato *D*.

Consideriamo ora che cosa succede se *A* richiama un'altra procedura, diciamo *B*. Dove dovrebbero essere memorizzate le quattro variabili locali (*b1, b2, b3, b4*) di *B* ? Risposta: nello stack sopra il blocco di *A*, come mostra la Figura 4.8(b). Si noti che LV è stato modificato dalla chiamata alla procedura in modo da puntare alle variabili locali di *B* invece che a quelle di *A*. È possibile far riferimento alle variabili locali di *B* fornendo il loro spiazzamento rispetto a LV. Analogamente, se *B* richiama *C*, LV e SP vengono nuovamente modificati in modo da allocare lo spazio necessario per memorizzare le due variabili di *C*; questa situazione è mostrata nella Figura 4.8(c).

Quando *C* termina, *B* diventa nuovamente attiva e lo stack viene modificato per tornare allo stato mostrato nella Figura 4.8(b), in modo che LV torni a puntare alle variabili locali di *B*. Analogamente, quando *B* restituisce il controllo, si torna nuovamente

alla situazione mostrata nella Figura 4.8(a). In tutte le situazioni LV punta alla base del blocco dello stack corrispondente alla procedura correntemente attiva, mentre SP punta alla cima dello stesso blocco.

Supponendo ora che *A* richiami *D*, che ha cinque variabili locali, si ottiene la situazione mostrata nella Figura 4.8(d), che illustra come le variabili locali di *D* usino la stessa area di memoria utilizzata precedentemnte da *B*, oltre a una parte di quella che era stata utilizzata da *C*. Organizzando la memoria in questo modo, è possibile allocare solamente la memoria necessaria alle procedure attive. Quando una procedura restituisce il controllo viene rilasciata la memoria utilizzata dalle sue variabili locali.

Oltre a memorizzare le variabili locali gli stack hanno anche un altro utilizzo. Possono essere utilizzati per memorizzare gli operandi durante il calcolo di un'espressione aritmetica. Quando uno stack è utilizzato in questo modo ci si riferisce a esso con il termine di **stack degli operandi**. Supponiamo per esempio che prima di richiamare *B*, *A* debba eseguire il calcolo

```
a1 = a2 + a3;
```

Un modo per effettuare questa somma consiste nel porre *a2* in cima allo stack, come mostra la Figura 4.9(a). Qui SP è stato incrementato del numero di byte che formano una parola, diciamo 4, in modo da puntare all'indirizzo in cui è stato memorizzato il primo operando. In seguito *a3* viene posto in cima allo stack, come mostra la Figura 4.9(b). Aprendo una parentesi riguardo alla notazione utilizzata nel testo, specifichiamo che tutti i frammenti di programmi saranno scritti utilizzando il carattere Courier, com'è già stato fatto in precedenza. Utilizzeremo questo carattere anche per i codici operativi del linguaggio assemblativo e per i registri della macchina, mentre nel testo scriveremo in *corsivo* i nomi delle variabili e delle procedure del programma. La differenza risiede nel fatto che le variabili e i nomi delle procedure sono scelti dall'utente, mentre i codici operativi e i nomi dei registri sono predefiniti.

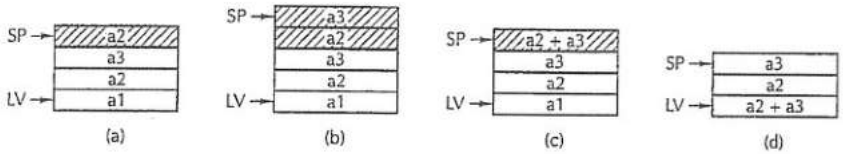


Figura 4.9 Utilizzo dello stack degli operandi per il calcolo di un'espressione aritmetica.

Il calcolo effettivo può a questo punto essere realizzato eseguendo un'istruzione che preleva due parole dallo stack, le somma e inserisce il risultato nuovamente nello stack, come mostra la Figura 4.9(c). Infine, la parola che si trova in cima allo stack può essere rimossa e memorizzata nella variabile locale *a1*, com'è illustrato nella Figura 4.9(d).

Il blocco delle variabili locali e lo stack degli operandi possono essere mischiati tra loro. Per esempio quando si calcola un'espressione come $x^2 + f(x)$, una parte dell'espressione (per esempio, x^2) può trovarsi nello stack degli operandi nel momento in cui viene invocata la funzione *f*. Il risultato della funzione viene lasciato nello stack, al di sopra di x^2, in modo che l'istruzione successiva li possa sommare.

Vale la pena precisare che mentre tutte le macchine utilizzano uno stack per memorizzare le variabili locali, non tutte usano uno stack degli operandi per eseguire le operazioni aritmetiche. In realtà la maggior parte non lo utilizza; JVM e IJVM funzionano però in questo modo e per questo motivo abbiamo deciso di introdurlo.

4.2.2 Modello della memoria di IJVM

Siamo ora pronti ad analizzare l'architettura di IJVM. Fondamentalmente essa consiste in una memoria concepibile in due modi distinti: come un array di 4.294.967.296 byte (4 GB) oppure come un array di 1.073.741.824 parole da 4 byte. Diversamente dalla maggior parte degli ISA, la Java Virtual Machine non rende direttamente visibili a livello ISA gli indirizzi di memoria assoluti, ma utilizza degli indirizzi impliciti che forniscono la base per l'uso di puntatori. L'unico modo che le istruzioni IJVM hanno per accedere alla memoria è quello di indicizzarla utilizzando questi puntatori. In ogni momento sono definite le seguenti aree di memoria.

1. *Porzione costante di memoria.* I programmi IJVM non possono scrivere in quest'area che contiene costanti, stringhe e puntatori ad altre aree di memoria cui è possibile far riferimento. È caricata quando il programma è portato in memoria e in seguito non viene modificata. Esiste un registro implicito, CPP, contenente l'indirizzo della prima parola della porzione costante di memoria.
2. *Blocco delle variabili locali.* Per ogni invocazione di un metodo viene allocata un'area in cui memorizzare le variabili locali durante l'intero ciclo di vita dell'invocazione. Quest'area è chiamata **blocco delle variabili locali**. Nella parte iniziale di questo blocco sono memorizzati i parametri (chiamati anche argomenti) con cui è stato invocato il metodo. Il blocco delle variabili locali non comprende lo stack degli operandi, che è separato. Tuttavia, per motivi di efficienza, qui abbiamo scelto di implementare lo stack degli operandi immediatamente sopra il blocco delle variabili locali. È presente un registro implicito, LV, contenente l'indirizzo della prima locazione all'interno del blocco delle variabili locali. I parametri passati durante l'invocazione del metodo sono memorizzati all'inizio del blocco delle variabili locali.
3. *Stack degli operandi.* Il blocco dello stack non può superare una certa dimensione, stabilita in anticipo dal compilatore Java. Lo spazio per lo stack degli operandi è allocato direttamente sopra il blocco delle variabili locali (Figura 4.10). Nella nostra implementazione conviene pensare che lo stack degli operandi faccia parte del blocco delle variabili locali. In ogni caso c'è un registro implicito contenente l'indirizzo della parola in cima allo stack. Si noti che, diversamente da CPP e LV, questo puntatore, SP, cambia durante l'esecuzione del metodo a seconda che gli operandi siano inseriti nello stack o rimossi da quest'ultimo.
4. *Area dei metodi.* Infine c'è una regione di memoria in cui risiede il programma, simile all'area "testo" di un processo UNIX. È presente un registro implicito che contiene l'indirizzo della successiva istruzione da prelevare. Questo puntatore è

chiamato contatore di programma, oppure PC. Diversamente dalle altre regioni di memoria l'area dei metodi è trattata come un array di byte.

Per quanto riguarda i puntatori occorre precisare un aspetto. I registri CPP, LV e SP sono tutti puntatori a *parole*, e non a *byte*; anche i loro spiazzamenti sono espressi come numero di parole. Per il sottoinsieme d'istruzioni su interi che abbiamo scelto, tutti i riferimenti a elementi che si trovano nella porzione costante di memoria, nel blocco delle variabili locali e nello stack sono definiti come parole; allo stesso modo tutti gli spiazzamenti utilizzati all'interno di questi blocchi sono definiti in termini di parole. Per esempio LV, LV + 1 e LV + 2 fanno riferimento alle prime tre parole del blocco delle variabili locali. LV, LV + 4 e LV + 8 fanno invece riferimento a parole che si trovano a intervalli di quattro parole (16 byte) l'una dall'altra.

Al contrario, PC contiene un indirizzo espresso in byte; un'addizione o una sottrazione effettuata su PC modifica quindi l'indirizzo in base a un certo numero di byte, e non di parole. L'indirizzamento di PC è diverso da quello degli altri registri; la speciale porta di memoria fornita per PC su Mic-1 ne è una dimostrazione. Come abbiamo già visto questa porta è larga soltanto 1 byte. Incrementare PC di uno e dare inizio a una lettura corrisponde a prelevare il *byte* successivo; incrementare SP di uno e dare inzio a una lettura corrisponde invece a prelevare la *parola* successiva.

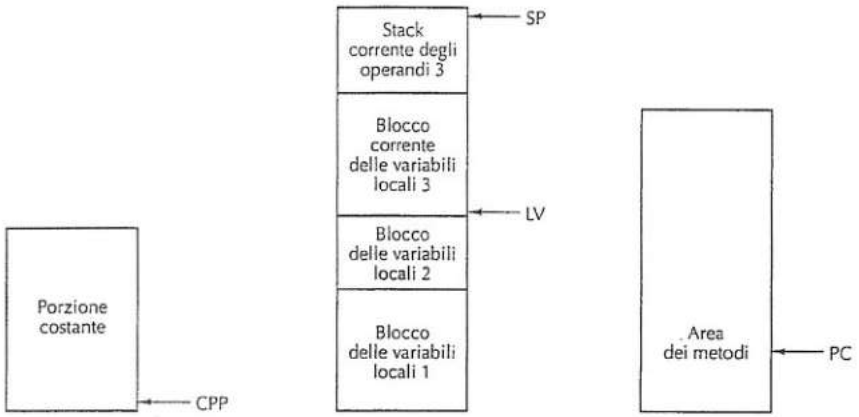


Figura 4.10 Parti della memoria di IJVM.

4.2.3 Insieme d'istruzioni IJVM

La Figura 4.11 mostra l'insieme d'istruzioni IJVM. Ogni istruzione è composta da un codice operativo e in alcuni casi da un operando, che può essere uno spiazzamento o una costante. La prima colonna mostra la codifica esadecimale dell'istruzione, la seconda il nome mnemonico in linguaggio assemblativo e la terza una breve descrizione del significato dell'istruzione.

Esa	Nome mnemonico	Significato
0x10	BIPUSH *byte*	Scrive un byte in cima allo stack
0x59	DUP	Legge la parola in cima allo stack e la inserisce in cima allo stack
0xA7	GOTO *offset*	Diramazione incondizionata
0x60	IADD	Sostituisce le due parole in cima allo stack con la loro somma
0x7E	IAND	Sostituisce le due parole in cima allo stack con il loro AND
0x99	IFEQ *offset*	Estrae una parola dalla cima dello stack ed effettua una diramazione se ha valore zero
0x9B	IFLT *offset*	Estrae una parola dalla cima dello stack ed effettua una diramazione se ha valore negativo
0x9F	IF_ICMPEQ *offset*	Estrae le due parole in cima allo stack ed effettua una diramazione se sono uguali
0x84	IINC *varnum const*	Aggiunge una costante a una variabile locale
0x15	ILOAD *varnum*	Scrive una variabile locale in cima allo stack
0xB6	INVOKEVIRTUAL *disp*	Invoca un metodo
0x80	IOR	Sostituisce le due parole in cima allo stack con il loro OR
0xAC	IRETURN	Termina un metodo restituendo un valore intero
0x36	ISTORE *varnum*	Preleva una parola dalla cima dello stack e la memorizza in una variabile locale
0x64	ISUB	Sostituisce le due parole in cima allo stack con la loro differenza
0x13	LDC_W *index*	Scrive in cima allo stack una costante proveniente dalla porzione costante di memoria
0x00	NOP	Non esegue nulla
0x57	POP	Rimuove la cima allo stack
0x5F	SWAP	Scambia le due parole in cima allo stack
0xC4	WIDE	Istruzione prefisso: l'istruzione successiva ha un indice a 16 bit

Figura 4.11 Insieme d'istruzioni di IJVM. Gli operandi *byte*, *const* e *varnum* sono lunghi 1 byte. Gli operandi *disp*, *index* e *offset* sono lunghi 2 byte.

Alcune istruzioni permettono di inserire nello stack una parola proveniente da varie fonti, come per esempio la porzione costante di memoria (LDC_W), il blocco delle variabili locali (ILOAD) e l'istruzione stessa (BIPUSH). Una variabile può anche essere estratta dallo stack e memorizzata nel blocco delle variabili locali (ISTORE). È possibile eseguire due operazioni aritmetiche (IADD e ISUB) e due operazioni logiche, cioè booleane, (IAND e IOR) utilizzando come operandi le due parole che si trovano in cima allo stack. In tutte le operazioni logiche e aritmetiche vengono estratte due parole dallo stack e il risultato viene inserito sopra di esso. Sono fornite quattro istruzioni per i salti, una non

condizionale (GOTO) e tre condizionali (IFEQ, IFLT e IF_ICMPEQ). Tutte queste istruzioni, se accettate, modificano il valore di PC in base alla grandezza del loro spiazzamento (16 bit con segno), che si trova nell'istruzione, subito dopo il codice operativo. Questo spiazzamento viene aggiunto all'indirizzo dell'istruzione. Ci sono anche istruzioni IJVM che permettono di scambiare le due parole in cima allo stack (SWAP), di duplicare la parola che si trova in cima (DUP) e di rimuoverla (POP).

Alcune istruzioni hanno più formati, per permettere di utilizzare una forma più breve per le versioni utilizzate più frequentemente. In IJVM abbiamo incluso due dei vari meccanismi che JVM mette a disposizione a questo scopo. In un caso abbiamo omesso la versione corta in favore di quella più generale. In un altro caso mostriamo come si può utilizzare l'istruzione prefisso WIDE per modificare l'istruzione successiva.

Infine c'è un'istruzione (INVOKEVIRTUAL) per invocare un altro metodo e un'istruzione (IRETURN) per terminare il metodo e restituire il controllo a quello che l'aveva invocato. A causa della complessità del meccanismo abbiamo semplificato leggermente la definizione, rendendo possibile un semplice metodo per invocare una chiamata e restituire il controllo. Questa restrizione, a differenza di quanto avviene realmente in Java, ci permette solamente di invocare un metodo che esiste all'interno dello stesso oggetto del metodo chiamante. Tale restrizione invalida severamente l'orientamento a oggetti, ma permette di non dover localizzare dinamicamente il metodo. (Se non si ha familiarità con la programmazione orientata agli oggetti si può ignorare senza alcun problema quest'ultima osservazione. Quello che abbiamo fatto è stato trasformare Java in un linguaggio non orientato agli oggetti, come C o Pascal.) Su tutti i calcolatori, fatta eccezione per la JVM, l'indirizzo della procedura da richiamare è determinato direttamente dall'istruzione CALL; il nostro approccio è quindi il caso normale e non rappresenta un'eccezione.

Il meccanismo per l'invocazione dei metodi funziona nel modo seguente. Prima di tutto il chiamante inserisce sullo stack un riferimento (puntatore) all'oggetto da chiamare. (Questo riferimento non è necessario in IJVM dato che non può essere specificato alcun altro oggetto, ma lo abbiamo mantenuto per coerenza con JVM.) Nella Figura 4.12(a) questo riferimento è indicato con OBJREF. Successivamente il chiamante inserisce sullo stack i parametri del metodo che, in questo esempio, sono *Parametro 1*, *Parametro 2* e *Parametro 3*. Infine viene eseguita l'istruzione INVOKEVIRTUAL.

Quest'istruzione include uno spiazzamento che indica una posizione all'interno della porzione costante di memoria; questa locazione contiene l'indirizzo dell'area dei metodi in cui inizia il metodo che si sta invocando. Tuttavia, mentre il codice del metodo risiede nella locazione puntata da questo puntatore, i primi 4 byte nell'area dei metodi contengono dati speciali. I primi 2 byte sono interpretati come un intero a 16 bit che indica il numero di parametri del metodo (i parametri stessi sono stati precedentemente inseriti in cima allo stack). In questo conteggio OBJREF viene considerato come un parametro: il parametro 0. Questo intero a 16 bit fornisce, insieme al valore di SP, la locazione di OBJREF. Si noti che LV punta a OBJREF invece che al primo, vero, parametro; questa scelta è sostanzialmente arbitraria.

Anche i successivi 2 byte dell'area dei metodi sono interpretati come un intero a 16 bit, che però indica la dimensione del blocco delle variabili locali per il metodo invocato.

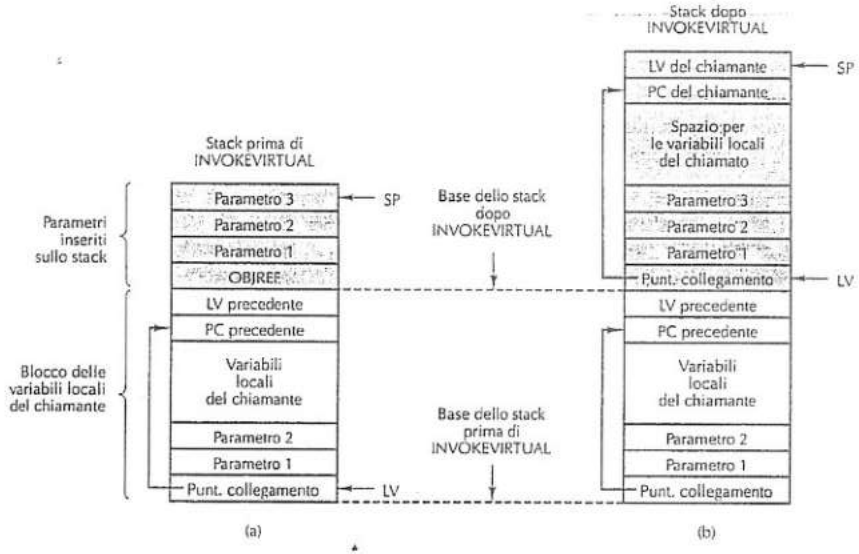


Figura 4.12 (a) Memoria prima di eseguire l'istruzione INVOKEVIRTUAL. (b) Dopo averla eseguita.

Questo valore è necessario, dato che verrà stabilito un nuovo stack per il metodo, immediatamente sopra il blocco delle variabili locali. Infine il quinto byte dell'area dei metodi contiene il primo codice operativo da eseguire.

Descriviamo ora l'effettiva sequenza di operazioni (Figura 4.12) che si verificano quando viene eseguita l'istruzione INVOKEVIRTUAL. I due byte senza segno che seguono il codice operativo sono utilizzati per costruire un indice relativo alla porzione costante di memoria (il primo byte è quello più significativo). L'istruzione calcola l'indirizzo base del nuovo blocco delle variabili locali sottraendo il numero di parametri dal puntatore allo stack e impostando LV in modo che punti a OBJREF. In questa locazione, sovrascrivendo OBJREF, viene memorizzato l'indirizzo della locazione in cui si trova il vecchio PC. Questo indirizzo è calcolato sommando la dimensione del blocco delle variabili locali (parametri + variabili locali) all'indirizzo contenuto in LV. Immediatamente sopra l'indirizzo in cui memorizzare il vecchio PC c'è l'indirizzo in cui deve essere memorizzato il vecchio LV. Al di sopra di questo indirizzo inizia lo stack per la procedura che è stata appena richiamata. SP viene impostato in modo da puntare al vecchio LV, che è l'indirizzo immediatamente sotto la prima locazione vuota dello stack. Ricordiamoci che SP punta sempre alla parola in cima allo stack. Se lo stack è vuoto, allora punta alla prima locazione sotto la fine dello stack, dato che i nostri stack crescono verso l'alto, verso gli indirizzi più grandi. Anche nelle nostre figure gli stack crescono verso l'alto, con gli indirizzi che aumentano in direzione del bordo superiore della pagina.

L'ultima operazione richiesta per terminare l'esecuzione di INVOKEVIRTUAL è impostare PC in modo che punti al quinto byte nell'area del codice del metodo.

L'istruzione IRETURN inverte la sequenza delle operazioni compiute da INVOKEVIRTUAL, come mostra la Figura 4.13. Essa dealloca lo spazio utilizzato dal metodo che sta restituendo il controllo e riporta lo stack nel suo precedente stato, tranne per il fatto che (1) la parola OBJREF (ora sovrascritta) e tutti i parametri sono stati estratti dallo stack e che (2) il valore restituito dal metodo è stato inserito in cima allo stack, nella locazione occupata precedentemente da OBJREF. Per memorizzare il vecchio stato l'istruzione IRETURN deve essere in grado di riportare i puntatori PC e LV ai loro precedenti valori. Per far ciò accede al puntatore di collegamento (*link pointer*), cioè alla parola identificata dal puntatore LV corrente. Non dimentichiamo che in questa locazione, in cui era precedentemente memorizzato OBJREF, l'istruzione INVOKEVIRTUAL ha memorizzato l'indirizzo contenente il vecchio PC. Questa parola, e quella che si trova al di sopra, vengono recuperate e utilizzate per restituire a PC e a LV i loro precedenti valori. Il valore fornito dal metodo, memorizzato in cima allo stack, viene copiato nella locazione in cui era originariamente memorizzato OBJREF, e SP viene reimpostato in modo da puntare a questa locazione. Il controllo viene quindi restituito all'istruzione immediatamente successiva rispetto all'istruzione INVOKEVIRTUAL.

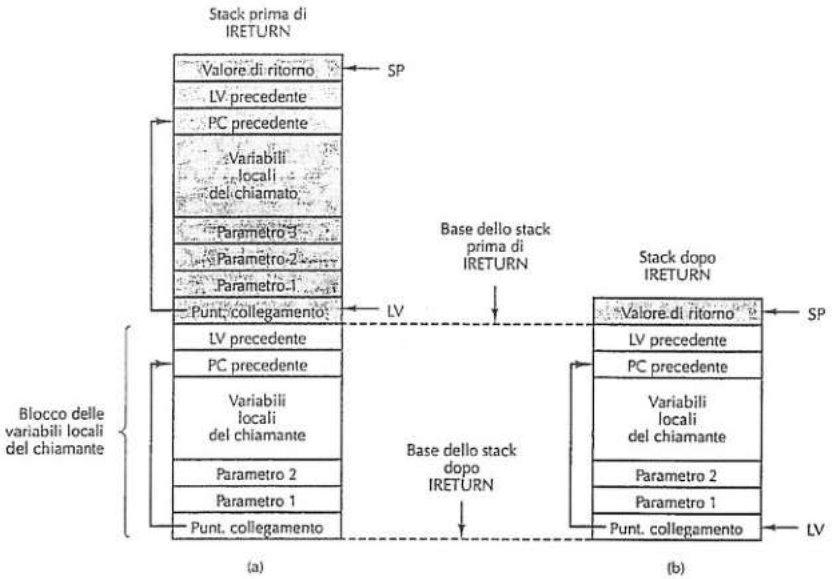


Figura 4.13 (a) Memoria prima di eseguire l'istruzione IRETURN. (b) Dopo averla eseguita.

Finora nella nostra macchina non abbiamo introdotto istruzioni di input/output, e non lo faremo neanche in seguito. Queste istruzioni non sono necessarie, a meno che non le richieda la Java Virtual Machine; tuttavia le specifiche ufficiali di JVM non fanno mai

cenno alle istruzioni di I/O. La teoria è che una macchina che non esegue alcun input o output è "sicura". (In JVM lettura e scrittura sono eseguite per mezzo di chiamate a speciali metodi di I/O.)

4.2.4 Compilazione da Java a IJVM

Vediamo ora le relazioni tra Java e IJVM. Un compilatore Java che riceve in ingresso il semplice frammento di codice Java della Figura 4.14(a) dovrebbe generare qualcosa di simile al codice assemblativo della Figura 4.14(b). I commenti (preceduti dai caratteri //) e i numeri di linea alla sinistra del codice assemblativo non fanno parte dell'output del compilatore. L'assemblatore Java dovrebbe in seguito tradurre il programma assemblativo nel codice binario mostrato nella Figura 4.14(c). (In realtà il compilatore Java crea il proprio programma assemblativo e genera il codice binario direttamente.) In questo esempio abbiamo assunto che *i* sia la variabile locale 1, *j* la variabile locale 2 e *k* la variabile locale 3.

```
i = j + k;                1     ILOAD j       // i = j + k      0x15 0x02
if (i == 3)               2     ILOAD k                      0x15 0x03
   k = 0;                 3     IADD                         0x60
else                      4     ISTORE i                     0x36 0x01
   j = j - 1;             5     ILOAD i       // if (i == 3)   0x15 0x01
                          6     BIPUSH 3                     0x10 0x03
                          7     IF_ICMPEQ L1                 0x9F 0x00 0x0D
                          8     ILOAD j       // j = j - 1     0x15 0x02
                          9     BIPUSH 1                     0x10 0x01
                         10     ISUB                         0x64
                         11     ISTORE j                     0x36 0x02
                         12     GOTO L2                      0xA7 0x00 0x07
                         13  L1: BIPUSH 0      // k = 0        0x10 0x00
                         14     ISTORE k                     0x36 0x03
                         15  L2:
        (a)                          (b)                          (c)
```

Figura 4.14 (a) Frammento di programma Java. (b) Il frammento in linguaggio assemblativo Java. (c) Programma IJVM in esadecimale.

Il codice compilato è semplice. Inizialmente *j* e *k* vengono inseriti in cima allo stack, vengono sommati, e il risultato viene memorizzato in *i*. Successivamente *i* e la costante 3 vengono messi nello stack e confrontati. Se sono uguali, si effettua una diramazione che porta in *L1*, dove *k* è impostato al valore 0. Se invece sono diversi il test dà esito negativo e viene eseguito il codice che segue l'istruzione IF_ICMPEQ. Dopo alcune istruzioni si effettua una diramazione che porta in *L2*, cioè nel punto in cui i rami then e else si riuniscono.

La Figura 4.15 mostra come varia lo stack degli operandi durante l'esecuzione del programma IJVM della Figura 4.14(b). Prima che cominci l'esecuzione del codice lo stack è vuoto, com'è indicato dalla linea orizzontale sopra il numero 0. Dopo la prima istruzione ILOAD, *j* si trova sullo stack, com'è indicato dal rettangolo *j* sopra il numero

1 (questo valore significa che è stata eseguita un'istruzione). Dopo la seconda istruzione ILOAD, nello stack ci sono due parole, come mostrano i due rettangoli sopra il numero 2. Dopo l'istruzione IADD, nello stack c'è soltanto una parola, che contiene la somma $j + k$. Quando la parola in cima allo stack viene estratta e memorizzata in i lo stack è vuoto, come mostra la linea sopra il numero 4.

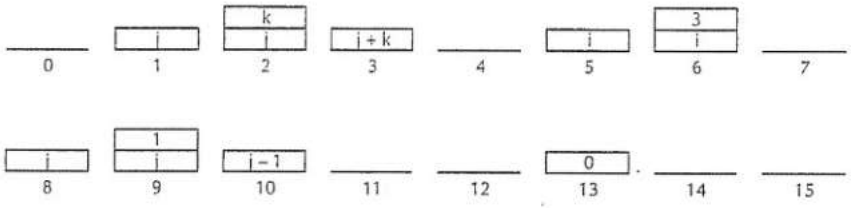


Figura 4.15 Stack dopo ogni istruzione della Figura 4.14(b).

L'istruzione 5 (ILOAD) inzia il costrutto if inserendo i nello stack (in 5). Successivamente (in 6) arriva la costante 3. Dopo il confronto lo stack è nuovamente vuoto (7). L'istruzione 8 rappresenta l'inizio del ramo else del frammento di programma Java. La parte else continua fino all'istruzione 12, punto in cui c'è la diramazione che porta a *L2* e che permette di saltare il codice relativo al ramo then.

4.3 Implementazione di esempio

Dopo aver specificato nel dettaglio la microarchitettura e la macroarchitettura, rimane il problema dell'implementazione. In altre parole, che aspetto ha e come funziona un programma che viene eseguito sulla prima e che interpreta la seconda? Prima di poter rispondere a queste domande dobbiamo considerare in modo preciso la notazione che utilizzeremo per descrivere l'implementazione.

4.3.1 Microistruzioni e notazione

In teoria potremmo descrivere la memoria di controllo in binario, 36 bit per parola. Ma, come avviene normalmente nei linguaggi di programmazione, è molto vantaggioso introdurre una notazione che esprime l'essenza dei problemi da trattare mascherando allo stesso tempo i dettagli che possono essere ignorati o che possono essere meglio gestiti in maniera automatica. È importante capire che il linguaggio da noi scelto ha lo scopo di illustrare i concetti e non di facilitare la progettazione di un'architettura efficiente. Se questo fosse il nostro obiettivo dovremmo utilizzare una differente notazione, in grado di fornire al progettista la massima flessibilità possibile. Un aspetto in cui questo problema è rilevante riguarda la scelta degli indirizzi.

Dato che la memoria non è ordinata dal punto di vista logico, non esiste alcuna naturale "istruzione successiva" da poter impiegare quando specifichiamo una sequenza di

operazioni. Molta della potenza dell'organizzazione del controllo dipende dalla capacità del progettista (o dell'assemblatore) di specificare gli indirizzi in modo efficiente. Cominciamo dunque con l'introdurre un semplice linguaggio simbolico che descrive in modo completo le operazioni senza spiegare in modo esaustivo come sono stati determinati tutti gli indirizzi.

La nostra notazione specifica su una singola linea tutte le attività che si svolgono durante un unico ciclo di clock. In teoria per descrivere le operazioni potremmo utilizzare un linguaggio ad alto livello. Tuttavia un controllo ciclo-dopo-ciclo è molto importante, in quanto dà la possibilità di effettuare più operazioni concorrenti; inoltre è necessario per poter analizzare i cicli in modo da comprenderne lo svolgimento e verificare le operazioni. Se l'obiettivo è quello di definire un'implementazione veloce ed efficiente (a parità di altri fattori, veloce ed efficiente è sempre meglio che lento e inefficiente) ogni ciclo ha la sua importanza. Le implementazioni reali celano molte sottili astuzie, come l'uso di sequenze o operazioni difficilmente comprensibili per cercare di risparmiare anche un solo ciclo. Risparmiare cicli ha un importante impatto sulle prestazioni: se un'istruzione a quattro cicli può essere ridotta a due cicli, allora verrà eseguita al doppio della velocità.

Un possibile approccio per definire la notazione si limita all'elenco dei segnali che dovrebbero essere attivati durante ciascun ciclo di clock. Supponiamo che in un ciclo vogliamo incrementare il valore di SP, oltre a dare inizio a un'operazione di lettura; desideriamo inoltre che l'istruzione successiva sia quella che risiede nella locazione 122 della memoria di controllo. Potremmo scrivere

```
ReadRegister = SP, ALU = INC, WSP, Read, NEXT_ADDRESS = 122
```

dove WSP significa "scrivere il registro SP". Questa notazione è completa, ma di difficile lettura. Al suo posto preferiamo combinare le operazioni in un modo naturale e intuitivo in modo da cogliere la sostanza di quello che sta avvenendo.

```
SP = SP + 1; rd
```

Chiamiamo il nostro linguaggio ad alto livello Micro Assembly Language, "MAL" (iniziale di "malato", quello che uno diventa se è obbligato a scrivere molte linee di codice in questo linguaggio). MAL è definito appositamente per rispecchiare le caratteristiche della microarchitettura. Durante ogni ciclo è possibile scrivere qualsiasi registro, anche se in genere ne viene scritto soltanto uno. Soltanto un registro può essere collegato al lato B della ALU, mentre sul lato A le scelte possibili sono +1, 0, –1 e il registro H. Possiamo inoltre usare un semplice costrutto di assegnamento, come in Java, per indicare l'operazione da eseguire; per copiare un dato da SP a MDR possiamo per esempio dire

```
MDR = SP
```

Per indicare che si intende utilizzare funzioni della ALU diverse dal semplice passaggio verso il bus B possiamo scrivere

```
MDR = H + SP
```

che somma il contenuto del registro H a SP e scrive il risultato in MDR. L'operatore + è commutativo (il che significa che l'ordine degli operandi è ininfluente); quindi l'istruzione può essere riscritta come

```
MDR = SP + H
```

e genera la stessa microistruzione a 36 bit; per essere precisi H deve però essere l'operando sinistro della ALU.

Dobbiamo fare attenzione a usare soltanto le operazioni lecite. Quelle più importanti sono mostrate nella Figura 4.16, in cui SOURCE può essere MDR, PC, MBR, MBRU, SP, LV, CPP, TOS oppure OPC (MBRU indica la versione senza segno di MBR). Tutti questi registri possono fungere da sorgenti per la ALU sul bus B. In modo analogo DEST può essere uno qualsiasi fra MAR, MDR, PC, SP, LV, CPP, TOS, OPC e H; tutti questi registri possono essere la destinazione dell'output della ALU che viene inviato sul bus C. Questo formato è insidioso, in quanto molti costrutti apparentemente corretti sono invece illegali. Per esempio, l'assegnamento

```
MDR = SP + MDR
```

sembra perfettamente ragionevole, ma non può in alcun modo essere eseguito in un unico ciclo sul percorso dati della Figura 4.6. Questa restrizione è dovuta al fatto che in un'addizione (fatta eccezione per un incremento o un decremento) uno degli operandi deve essere il registro H.

DEST = H
DEST = SOURCE
DEST = H
DEST = SOURCE
DEST = H + SOURCE
DEST = H + SOURCE + 1
DEST = H + 1
DEST = SOURCE + 1
DEST = SOURCE – H
DEST = SOURCE – 1
DEST = –H
DEST = H AND SOURCE
DEST = H OR SOURCE
DEST = 0
DEST = 1
DEST = –1

Figura 4.16 Operazioni consentite. Possono tutte essere estese aggiungendo "<< 8" per traslare il risultato a sinistra di 1 byte. Un'operazione comune è H = MBR << 8.

Analogamente, l'assegnamento

```
H = H – MDR
```

potrebbe essere utile, ma è anch'esso ineseguibile dato che l'unica possibile sorgente per il sottraendo è il registro H. È compito dell'assemblatore rifiutare quelle istruzioni che sembrano valide, ma in realtà non lo sono.

Estendiamo ora la notazione per permettere assegnamenti multipli mediante la ripetizione del simbolo di uguaglianza. Per esempio, per sommare 1 a SP, memorizzare il risultato nuovamente in SP e scriverlo anche all'interno di MDR, si può utilizzare la seguente notazione

```
SP = MDR = SP + 1
```

Per indicare letture e scritture di parole di dati da 4 byte aggiungeremo semplicemente rd e wr nella microistruzione. Il prelievo di un byte attraverso la porta a 1 byte si indica con fetch. Gli assegnamenti e le operazioni della memoria possono svolgersi durante lo stesso ciclo, il che è indicato scrivendoli sulla stessa linea.

Per evitare possibili confusioni ripetiamo che Mic-1 ha due modi per accedere alla memoria. Le letture e le scritture di parole a 4 byte usano MAR/MDR e nelle microistruzioni sono indicate rispettivamente da rd e wr. Le letture dei codici operativi a 1 byte dal flusso dell'istruzione utilizzano PC/MBR e sono indicate nelle microistruzioni mediante la parola fetch. Le due operazioni possono procedere in modo simultaneo.

Tuttavia lo stesso registro non può ricevere un valore dalla memoria e dal percorso dati durante lo stesso ciclo. Consideriamo il codice

```
MAR = SP; rd
MDR = H
```

La prima microistruzione assegna alla fine della seconda microistruzione un valore della memoria a MDR. Tuttavia anche la seconda microistruzione assegna nello stesso momento un valore a MDR. Questi due assegnamenti sono in conflitto e non sono permessi, poiché in tal caso il risultato sarebbe indefinito.

Ricordiamoci che ogni microistruzione deve fornire in modo esplicito l'indirizzo della successiva microistruzione da eseguire. Tuttavia spesso succede che una microistruzione sia invocata solamente da un'altra microistruzione, in particolare da quella che si trova nella linea immediatamente sopra di essa. Per facilitare il lavoro del programmatore normalmente il microassemblatore assegna un indirizzo a ciascuna microistruzione (non necessariamente consecutiva all'interno della memoria di controllo) e completa il campo NEXT_ADDRESS in modo che le microistruzioni scritte su linee consecutive vengano eseguite in sequenza.

Tuttavia in alcuni casi il microprogrammatore desidera inserire una diramazione, condizionale oppure no. La notazione per i salti incondizionati è semplice:

```
goto label
```

e può essere inclusa in ogni microistruzione per indicare in modo esplicito il proprio successore. Per esempio, la maggior parte delle sequenze di microistruzioni termina con

un ritorno alla prima istruzione del ciclo principale. Dunque l'ultima istruzione di queste sequenze include generalmente

```
goto Main1
```

Si noti che anche quando una microistruzione contiene goto il percorso dati è disponibile per le normali operazioni. Dopo tutto ogni microistruzione contiene un campo NEXT_ADDRESS e quello che fa goto non è altro che comunicare al microassemblatore di inserire in questo campo un particolare valore, al posto dell'indirizzo in cui esso aveva deciso di mettere la microistruzione della linea successiva. In linea di principio ogni linea dovrebbe avere un'istruzione goto che però, per comodità, viene omessa quando l'indirizzo di destinazione è la linea successiva.

Per i salti condizionali abbiamo invece bisogno di una notazione differente. Ricordiamoci che JAMN e JAMZ usano i bit N e Z, che vengono impostati in base all'output della ALU. A volte è necessario testare un registro per vedere se il suo valore è 0. Un modo per farlo consiste nel far passare il valore del registro attraverso la ALU e nel memorizzare il risultato nuovamente nel registro stesso. Scrivere

```
TOS = TOS
```

può sembrare strano, ma svolge correttamente il compito (impostando il flip-flop Z in base a TOS). Tuttavia per migliorare l'aspetto dei microprogrammi estendiamo MAL, aggiungendo due nuovi registri immaginari, N e Z, a cui è possibile effettuare degli assegnamenti. Per esempio,

```
Z = TOS
```

fa passare TOS attraverso la ALU, impostando quindi i flip-flop Z (e N), senza però effettuare alcuna memorizzazione nei registri. Utilizzare Z oppure N come destinazione non fa altro che indicare al microassemblatore di impostare a 0 tutti i bit del campo C della Figura 4.5. Il percorso dati esegue un ciclo in cui sono consentite tutte le normali operazioni, ma in cui non viene scritto alcun registro. Si noti che è ininfluente il fatto che la destinazione sia N oppure Z, dato che la microistruzione generata dal microassemblatore è identica. I programmatori che scelgono intenzionalmente quella "sbagliata", per punizione dovrebbero essere obbligati a lavorare per una settimana intera con il PC IBM originale a 4,77 MHz!

La sintassi per comunicare al microassemblatore di impostare il bit JAMZ è

```
if (Z) goto L1; else goto L2
```

Dato che l'hardware richiede che gli 8 bit meno significativi di questi indirizzi siano identici, è compito del microassemblatore assegnare gli indirizzi correttamente. D'altra parte, dato che *L2* può trovarsi in un qualsiasi punto delle prime 256 parole della memoria di controllo, il microassemblatore ha molta libertà nel cercare un altro indirizzo che possa essere accoppiato al primo.

In genere queste ultime due istruzioni vengono combinate; per esempio

```
Z = TOS; if (Z) goto L1; else goto L2
```


Come risultato MAL genera una microistruzione in cui TOS viene fatto passare attraverso la ALU (ma senza essere memorizzato in alcun registro) in modo che il suo valore imposti il bit Z. Poco dopo aver caricato in Z il bit di condizione della ALU, viene calcolato in OR all'interno del bit più significativo di MPC, obbligando a prelevare l'indirizzo della successiva microistruzione o da *L2* oppure da *L1* (che deve essere 256 più di *L2*). MPC sarà stabile e pronto per essere utilizzato nella successiva microistruzione.

Infine abbiamo bisogno di una notazione per utilizzare il bit JMPC. Quella che adotteremo è

```
goto (MBR OR valore)
```

Questa sintassi indica al microassemblatore di utilizzare *valore* per NEXT_ADDRESS e di impostare il bit JMPC in modo che in MPC venga calcolato l'OR logico tra MBR e NEXT_ADDRESS. Se *valore* è 0, allora la situazione è quella normale ed è sufficiente scrivere semplicemente

```
goto (MBR)
```

Si noti che in questo caso non si verifica il problema dell'estensione del segno (cioè la differenza tra MBR e MBRU), dato che soltanto gli 8 bit meno significativi di MBR sono collegati a MPC (Figura 4.6). Si osservi inoltre che quello che si utilizza è il valore di MBR disponibile alla fine del ciclo corrente. Un prelievo che inizia in *questa* microistruzione avviene troppo in ritardo per poter modificare la scelta della microistruzione successiva.

4.3.2 Implementazione di IJVM con Mic-1

Siamo finalmente arrivati al momento in cui possiamo mettere insieme tutti i pezzi. La Figura 4.17 è il microprogramma eseguito su Mic-1 che interpreta IJVM. È un programma sorprendentemente corto, con un totale di sole 112 microistruzioni. Per ciascuna microistruzione ci sono tre colonne: l'etichetta simbolica, il microcodice effettivo e un commento. Come abbiamo già sottolineato più volte, si può notare che microistruzioni consecutive non sono necessariamente localizzate in indirizzi consecutivi all'interno della memoria di controllo.

Oramai le scelte dei nomi per la maggior parte dei registri della Figura 4.1 dovrebbero essere ovvie: CPP, LV e SP sono utilizzati per memorizzare, rispettivamente, i puntatori alla porzione costante di memoria, al blocco delle variabili locali e alla cima dello stack, mentre PC mantiene l'indirizzo del successivo byte da prelevare dal flusso dell'istruzione. MBR è un registro a 1 byte che mantiene in modo sequenziale i byte dello stream dell'istruzione provenienti dalla memoria per poterli interpretare. TOS e OPC sono registri aggiuntivi il cui utilizzo viene descritto in seguito.

In qualsiasi momento ognuno di questi registri contiene un particolare valore; se necessario è tuttavia possibile utilizzare ciascuno di loro come un registro temporaneo. All'inizio e alla fine di ogni istruzione, TOS contiene il valore puntato da SP, la parola che si trova in cima allo stack. Questo valore è ridondante, in quanto la parola può essere letta in qualsiasi momento dalla memoria; ciononostante avendola a disposizione in un registro è spesso possibile risparmiare un riferimento alla memoria. Per poche istruzioni il mantenimento di TOS implica un numero maggiore di operazioni di memoria. Per

esempio, l'istruzione POP deve leggere dalla memoria la nuova parola che si trova in cima allo stack per poterla copiare all'interno di TOS.

Etichetta	Operazioni	Commenti
Main1	PC = PC + 1; fetch; goto (MBR)	MBR contiene il codice operativo; prelievo del byte successivo; diramazione
nop1	goto Main1	Non esegue nulla
iadd1 iadd2 iadd3	MAR = SP = SP – 1; rd H = TOS MDR = TOS = MDR + H; wr; goto Main1	Legge la seconda parola in cima allo stack H = cima dello stack Somma le due parole in cima allo stack; scrive in cima allo stack
isub1 isub2 isub3	MAR = SP = SP – 1; rd H = TOS MDR = TOS = MDR – H; wr; goto Main1	Legge la seconda parola in cima allo stack H = cima dello stack Esegue la sottrazione; scrive in cima allo stack
iand1 iand2 iand3	MAR = SP = SP – 1; rd H = TOS MDR = TOS = MDR AND H; wr; goto Main1	Legge la seconda parola in cima allo stack H = cima dello stack Esegue l'AND; scrive nella nuova cima dello stack
ior1 ior2 ior3	MAR = SP = SP – 1; rd H = TOS MDR = TOS = MDR OR H; wr; goto Main1	Legge la seconda parola in cima allo stack H = cima dello stack Esegue l'OR; scrive nella nuova cima dello stack
dup1 dup2	MAR = SP = SP + 1 MDR = TOS; wr; goto Main1	Incrementa SP e lo copia in MAR Scrive la nuova parola dello stack
pop1 pop2 pop3	MAR = SP = SP – 1; rd TOS = MDR; goto Main1	Legge la seconda parola in cima allo stack Attende che il nuovo TOS sia letto dalla memoria Copia la nuova parola in TOS
swap1 swap2 swap3 swap4 swap5 swap6	MAR = SP – 1; rd MAR = SP H = MDR; wr MDR = TOS MAR = SP – 1; wr TOS = H; goto Main1	Imposta MAR a SP – 1; legge la seconda parola dallo stack Imposta MAR con la parola in cima allo stack Salva TOS in H; scrive la seconda parola in cima allo stack Copia il vecchio TOS in MDR Imposta MAR a SP – 1; scrive la seconda parola nello stack Aggiorna TOS
bipush1 bipush2 bipush3	SP = MAR = SP + 1 PC = PC + 1; fetch MDR = TOS = MBR; wr; goto Main1	MBR = byte da inserire nello stack Incrementa PC; preleva il successivo codice operativo Estende il segno della costante e la inserisce nello stack
iload1 iload2 iload3 iload4 iload5	H = LV MAR = MBRU + H; rd MAR = SP = SP + 1 PC = PC + 1; fetch; wr TOS = MDR; goto Main1	MBR contiene l'indice; copia LV in H MAR = indirizzo della variabile locale da inserire nello stack SP punta alla nuova cima dello stack; prepara la scrittura Incrementa PC; ottiene il nuovo codice operativo; scrive la cima dello stack Aggiorna TOS
istore1 istore2 istore3 istore4 istore5 istore6	H = LV MAR = MBRU + H MDR = TOS; wr SP = MAR = SP – 1; rd PC = PC + 1; fetch TOS = MDR; goto Main1	MBR contiene l'indice; copia LV in H MAR = indirizzo della variabile locale da memorizzare Copia TOS in MDR; scrive la parola Legge la nuova parola in cima allo stack Incrementa PC; preleva il successivo codice operativo Aggiorna TOS

Figura 4.17 Microprogramma per Mic-1.

wide1 wide2	PC = PC + 1; fetch; goto (MBR OR 0x100)	Preleva il byte dell'operando o il successivo codice operativo Diramazione a più destinazioni con il bit alto impostato a 1
wide_iload1 wide_iload2 wide_iload3 wide_iload4	PC = PC + 1; fetch H = MBRU << 8 H = MBRU OR H MAR = LV + H; rd; goto iload3	MBR contiene il primo byte dell'indice; preleva il secondo H = primo byte dell'indice traslato a sinistra di 8 bit H = indice a 16 bit della variabile locale MAR = indirizzo della variabile locale da inserire nello stack
wide_istore1 wide_istore2 wide_istore3 wide_istore4	PC = PC + 1; fetch H = MBRU << 8 H = MBRU OR H MAR = LV + H; goto istore3	MBR contiene il primo byte dell'indice; preleva il secondo H = primo byte dell'indice traslato a sinistra di 8 bit H = indice a 16 bit della variabile locale MAR = indirizzo della variabile locale da memorizzare
ldc_w1 ldc_w2 ldc_w3 ldc_w4	PC = PC + 1; fetch H = MBRU << 8 H = MBRU OR H MAR = H + CPP; rd; goto iload3	MBR contiene il primo byte dell'indice; preleva il secondo H = primo byte dell'indice << 8 H = indice a 16 bit relativo alla porzione costante di memoria MAR = indirizzo della costante nella porzione costante
iinc1 iinc2 iinc3 iinc4 iinc5 iinc6	H = LV MAR = MBRU + H; rd PC = PC + 1; fetch H = MDR PC = PC + 1; fetch MDR = MBR + H; wr; goto Main1	MBR contiene l'indice; copia LV in H Copia LV + indice in MAR; legge la variabile Preleva la costante Copia la variabile in H Preleva il successivo codice operativo Inserisce in MDR la somma; aggiorna la variabile
goto1 goto2 goto3 goto4 goto5 goto6	OPC = PC – 1 PC = PC + 1; fetch H = MBR << 8 H = MBRU OR H PC = OPC + H; fetch goto Main1	Salva l'indirizzo del codice operativo MBR contiene il primo byte dell'indice; preleva il secondo Trasla e salva in H il primo byte con segno H = spiazzamento a 16 bit della diramazione Aggiunge lo spiazzamento a OPC Attende il prelievo del successivo codice operativo
iflt1 iflt2 iflt3 iflt4	MAR = SP = SP – 1; rd OPC = TOS TOS = MDR N = OPC; if (N) goto T; else goto F	Legge la seconda parola in cima allo stack Salva temporaneamente TOS in OPC Inserisce in TOS la nuova cima dello stack Diramazione in base al bit N
ifeq1 ifeq2 ifeq3 ifeq4	MAR = SP = SP – 1; rd OPC = TOS TOS = MDR Z = OPC; if (Z) goto T; else goto F	Legge la seconda parola in cima allo stack Salva temporaneamente TOS in OPC Inserisce in TOS la nuova cima dello stack Diramazione in base al bit Z
if_icmpeq1 if_icmpeq2 if_icmpeq3 if_icmpeq4 if_icmpeq5 if_icmpeq6	MAR = SP = SP – 1; rd MAR = SP = SP – 1 H = MDR; rd OPC = TOS TOS = MDR Z = OPC – H; if (Z) goto T; else goto F	Legge la seconda parola in cima allo stack Imposta MAR per leggere la nuova cima dello stack Copia in H la seconda parola dello stack Salva temporaneamente TOS in OPC Inserisce in TOS la nuova cima dello stack Se le due parole in cima allo stack sono uguali, goto T, altrimenti, goto F
T	OPC = PC – 1; goto goto2	Uguale a goto1; necessario per l'indirizzo destinazione
F F2 F3	PC = PC + 1 PC = PC + 1; fetch goto Main1	Salta il primo byte dello spiazzamento PC punta ora al nuovo codice operativo Attende il prelievo del codice operativo

Figura 4.17 Microprogramma per Mic-1 (continua).

invokevirtual1	PC = PC + 1; fetch	MBR = byte 1 dell'indice; incrementa PC; ottiene il secondo byte
invokevirtual2	H = MBRU << 8	Trasla e salva in H il primo byte
invokevirtual3	H = MBRU OR H	H = spiazzamento del puntatore al metodo rispetto a CPP
invokevirtual4	MAR = CPP + H; rd	Ottiene dall'area CPP un puntatore al metodo
invokevirtual5	OPC = PC + 1	Salva temporaneamente il PC di ritorno in OPC
invokevirtual6	PC = MDR; fetch	PC punta ora al nuovo metodo; ottiene il numero di parametri
invokevirtual7	PC = PC + 1; fetch	Preleva il secondo byte del numero di parametri
invokevirtual8	H = MBRU << 8	Trasla e salva in H il primo byte
invokevirtual9	H = MBRU OR H	H = numero di parametri
invokevirtual10	PC = PC + 1; fetch	Preleva il primo byte del numero di variabili locali
invokevirtual11	TOS = SP – H	TOS = indirizzo di OBJREF – 1
invokevirtual12	TOS = MAR = TOS + 1	TOS = indirizzo di OBJREF (nuovo LV)
invokevirtual13	PC = PC + 1; fetch	Preleva il secondo byte del numero di variabili locali
invokevirtual14	H = MBRU << 8	Trasla e salva in H il primo byte
invokevirtual15	H = MBRU OR H	H = numero di variabili locali
invokevirtual16	MDR = SP + H + 1; wr	Sovrascrive OBJREF con il puntatore di collegamento
invokevirtual17	MAR = SP = MDR;	Imposta SP e MAR con la locazione in cui memorizzare il vecchio PC
invokevirtual18	MDR = OPC; wr S	Salva il vecchio PC sopra le variabili locali
invokevirtual19	MAR = SP = SP + 1	SP punta alla locazione in cui salvare il vecchio LV
invokevirtual20	MDR = LV; wr	Salva il vecchio LV sopra il PC salvato
invokevirtual21	PC = PC + 1; fetch	Preleva il primo codice operativo del nuovo metodo
invokevirtual22	LV = TOS; goto Main1	Imposta LV per puntare al blocco LV
ireturn1	MAR = SP = LV; rd	Reimposta SP e MAR per ricevere il puntatore di collegamento
ireturn2	LV = MAR = MDR; rd	Attende che si completi la lettura
ireturn3	MAR = LV + 1	Imposta LV con il puntatore di collegamento; ottiene il vecchio PC
ireturn4	PC = MDR; rd; fetch	Imposta MAR per leggere il vecchio LV
ireturn5	MAR = SP	Ripristina PC; preleva il successivo codice operativo
ireturn6	LV = MDR	Imposta MAR per scrivere TOS
ireturn7	MDR = TOS;	Ripristina LV
ireturn8	wr; goto Main1	Salva il valore di ritorno sulla cima originale dello stack

Figura 4.17 Microprogramma per Mic-1 (continua).

Il registro OPC è un registro temporaneo (cioè di "appunti") e non ha un utilizzo predefinito. È usato per esempio per salvare l'indirizzo del codice operativo di un'istruzione di diramazione, mentre PC viene incrementato per accedere ai parametri. Viene utilizzato come registro temporaneo anche nelle istruzioni di diramazione condizionale di IJVM.

Come tutti gli interpreti, il microprogramma della Figura 4.17 ha un ciclo principale che preleva, decodifica ed esegue le istruzioni del programma interpretato, in questo caso le istruzioni IJVM. Il suo ciclo principale inizia nella linea etichettata con Main1. All'inizio del ciclo deve valere la condizione che PC sia già stato caricato con un indirizzo di una locazione di memoria contenente un codice operativo; inoltre questo codice operativo deve già essere stato memorizzato all'interno di MBR. Ciò implica che prima di iniziare ogni iterazione dobbiamo assicurarci che PC sia stato aggiornato in modo da puntare al successivo codice operativo da interpretare e che il byte del codice operativo stesso sia già stato portato all'interno di MBR.

All'inizio di ogni istruzione viene eseguita la stessa sequenza di operazioni; è importante che la lunghezza di questa sequenza sia la più breve possibile. Attraverso un'attenta progettazione dell'hardware Mic-1 e del software siamo riusciti a ridurre il ciclo principale a una sola istruzione. A partire dal momento in cui viene avviata la macchina, ogni volta che viene eseguita questa microistruzione il codice operativo IJVM da eseguire si trova già all'interno in MBR. Quello che fa la microistruzione è effettuare un salto nel microcodice per eseguire questa istruzione IJVM, oltre a iniziare il prelievo del byte che segue il codice operativo; questo byte potrebbe essere un operando oppure un nuovo codice operativo.

A questo punto possiamo svelare il vero motivo per cui le istruzioni non vengono eseguite sequenzialmente, ma ciascuna di loro è obbligata a indicare esplicitamente il proprio successore. Tutti gli indirizzi della memoria di controllo corrispondenti ai codici operativi devono essere riservati per la prima parola della corrispondente istruzione dell'interprete. Dalla Figura 4.11 vediamo quindi che il codice che interpreta POP inizia in 0x57, mentre il codice che interpreta DUP inizia in 0x59. (Come fa MAL a sapere che POP deve iniziare in 0x57 è uno dei misteri dell'universo, probabilmente esiste da qualche parte un documento segreto che ne spiega il motivo.)

Sfortunatamente il codice corrispondente a POP è lungo tre microistruzioni e quindi, se fosse memorizzato in parole consecutive, interferirebbe con l'inizio di DUP. All'interno di ciascuna sequenza le microistruzioni che seguono quella iniziale devono essere memorizzate nelle locazioni di memoria ancora libere, non riservate per i codici operativi. Per questo motivo occorre effettuare molti salti da una locazione di memoria all'altra; se regolarmente, dopo poche microistruzioni, occorresse utilizzare microdiramazioni esplicite (cioè microistruzioni che effettuano salti) per saltare da un buco a un altro della memoria, il tempo sprecato sarebbe significativo.

Per vedere come funziona l'interprete, assumiamo che MBR contenga il valore 0x60, cioè il codice operativo corrispondente a IADD (Figura 4.11). Nel ciclo principale composto da una sola microistruzione compiamo tre azioni.

1. Incrementiamo PC, in modo che contenga l'indirizzo del primo byte dopo il codice operativo.
2. Iniziamo a prelevare il byte successivo da portare all'interno di MBR. Questo byte, prima o poi, si rivelerà necessario, o come operando per l'istruzione IJVM corrente oppure come codice operativo successivo (come nel caso dell'istruzione IADD, che non ha il byte per l'operando).
3. All'inizio di Main1 effettuiamo una diramazione verso l'indirizzo contenuto in MBR. Questo indirizzo, memorizzato nel registro dalla precedente microistruzione, equivale al valore numerico del codice operativo che in quel momento è in esecuzione. Occorre prestare particolare attenzione al fatto che il valore che si sta iniziando a estrarre in questa microistruzione non gioca alcun ruolo nella diramazione.

Il prelievo del byte successivo comincia in questo punto in modo che sia disponibile all'inizio della terza microistruzione. In seguito potrebbe risultare necessario oppure no, ma conviene comunque far partire il prelievo, dato che la cosa non produce assolutamente alcun danno.

Se il byte contenuto in MBR è composto da soli 0, allora identifica il codice operativo di un'istruzione NOP. In questo caso la successiva microistruzione viene prelevata dalla locazione 0 ed è quella etichettata con nop1. Dato che questa istruzione non esegue alcunché essa effettua semplicemente un salto all'inizio del ciclo principale, dove viene ripetuta la sequenza utilizzando però il nuovo codice operativo memorizzato in MBR.

Sottolineiamo ancora una volta il fatto che nella Figura 4.17 le microistruzioni non sono memorizzate consecutivamente all'interno della memoria e che Main1 non si trova all'indirizzo 0 della memoria di controllo (dato che all'indirizzo 0 deve essere memorizzata nop1). È compito del microassemblatore posizionare le microistruzioni negli indirizzi adatti e collegarle in sequenze di breve lunghezza usando il campo NEXT_ADDRESS. Ogni sequenza inizia all'indirizzo corrispondente al valore numerico del codice operativo IJVM che interpreta (POP comincia per esempio in 0x57), ma il resto della sequenza può trovarsi in qualsiasi punto della memoria di controllo, e non necessariamente negli indirizzi immediatamente successivi.

Consideriamo ora l'istruzione IADD di IJVM. Dopo la diramazione presente nel ciclo principale si raggiunge la microistruzione con l'etichetta iadd1. Questa istruzione dà inizio alle operazioni specifiche di IADD.

1. TOS è già presente, mentre occorre prelevare dalla memoria la seconda parola a partire dalla cima dello stack.
2. TOS deve essere sommata alla parola che è appena stata prelevata dalla memoria.
3. Il risultato, che deve essere inserito in cima allo stack, va memorizzato sia in memoria sia nel registro TOS.

Per poter prelevare l'operando dalla memoria è necessario decrementare il puntatore allo stack e scriverlo all'interno di MAR. Si noti che, per praticità, questo indirizzo è anche l'indirizzo che sarà utilizzato nella successiva scrittura. Inoltre, dato che questa locazione sarà la nuova cima dello stack, occorre assegnare questo valore anche a SP. Una singola operazione può quindi determinare il nuovo valore di SP e MAR, decrementare SP e scriverlo in entrambi i registri.

Queste azioni vengono svolte durante il primo ciclo, iadd1, mentre inizia l'operazione di lettura. Inoltre MPC ottiene l'indirizzo di iadd2 leggendo il valore del campo NEXT_ADDRESS di iadd1. Successivamente iadd2 viene letta dalla memoria di controllo. Durante il secondo ciclo, nell'attesa che l'operando sia letto dalla memoria, la parola in cima allo stack viene copiata da TOS a H, in modo che, una volta completata la lettura, sia disponibile per il calcolo dell'addizione.

All'inizio del terzo ciclo, iadd3, MDR contiene l'addendo prelevato dalla memoria. In questo ciclo l'addendo viene sommato al contenuto di H, e il risultato viene memorizzato sia all'interno di MDR sia all'interno di TOS. Viene fatta partire anche un'operazione di scrittura per memorizzare all'interno della memoria la nuova parola che si trova in cima allo stack. In questo ciclo goto assegna a MPC l'indirizzo di Main1, facendoci tornare al punto d'inizio, in modo da poter eseguire l'istruzione seguente.

Se il successivo codice operativo IJVM contenuto in MBR è 0x64 (ISUB), la sequenza di eventi che si svolge è praticamente identica a quella appena vista. Dopo aver eseguito Main1, il controllo viene trasferito alla microistruzione che si trova all'indirizzo 0x64 (isub1). Questa microistruzione è seguita da isub2 e isub3, e poi nuovamente da Main1. L'unica differenza tra questa sequenza e la precedente è che in isub3 il contenuto di H viene sottratto da MDR invece di sommarlo.

L'interpretazione di IAND è praticamente identica a quella di IADD e di ISUB, tranne per il fatto che si calcola l'AND tra le due parole in cima allo stack invece di sommarle oppure sottrarle. Nel caso dell'istruzione IOR la situazione è del tutto simile.

Se il codice operativo IJVM è DUP, POP oppure SWAP occorre modificare lo stack. L'istruzione DUP replica semplicemente la parola che si trova in cima allo stack. Dato che il valore di questa parola è già contenuto in TOS, l'operazione non fa altro che incrementare SP in modo che punti alla nuova locazione e memorizzare TOS in quella posizione. L'istruzione POP è altrettanto semplice e consiste nel decrementare SP per scartare la parola in cima allo stack. Tuttavia, per mantenere corretto il valore memorizzato in TOS, occorre leggere dalla memoria la nuova parola che si trova in cima allo stack per scriverla all'interno di TOS. Infine l'istruzione SWAP effettua lo scambio dei valori contenuti nelle due locazioni che si trovano in cima allo stack. L'operazione è facilitata dal fatto che TOS contiene già uno di questi valori, e non è necessario leggerlo dalla memoria. In seguito tratteremo in modo più dettagliato questa istruzione.

L'istruzione BIPUSH è un po' più complicata poiché, come mostra la Figura 4.18, il codice operativo è seguito da un solo byte, che deve essere interpretato come un intero con segno. Questo byte, che durante Main1 è già stato portato all'interno di MBR, deve essere esteso con segno fino a 32 bit e copiato all'interno di MDR. Infine SP viene incrementato e copiato in MAR, in modo da permettere la scrittura dell'operando in cima allo stack. Questo operando deve anche essere copiato nel registro TOS. Inoltre è importante notare che, prima di ritornare al programma principale, PC deve essere nuovamente incrementato in modo da rendere disponibile in Main1 il successivo codice operativo.

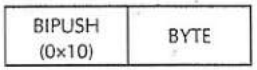


Figura 4.18 Formato dell'istruzione BIPUSH.

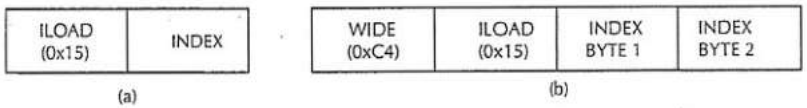


Figura 4.19 (a) ILOAD con un indice a 1 byte. (b) WIDE ILOAD con un indice a 2 byte.

Consideriamo ora l'istruzione ILOAD. Come mostra la Figura 4.19(a) anche in questa istruzione il codice operativo è seguito da un byte. In questo caso si tratta però di un indice (senza segno) usato per identificare nello spazio delle variabili locali la parola da mettere nello stack. Dato che si usa un solo byte è possibile distinguere soltanto $2^8 = 256$

parole, per la precisione le prime 256 parole dello spazio delle variabili locali. L'istruzione ILOAD richiede sia una lettura (per ottenere la parola) sia una scrittura (per porla in cima allo stack). Tuttavia per poter determinare l'indirizzo di lettura occorre sommare lo spiazzamento, contenuto in MBR, al valore di LV. Dato che è possibile accedere sia a MBR sia a LV solamente attraverso il bus B, inizialmente si copia LV in H (in iload1), e poi si somma MBR. Dopo che il risultato è stato copiato in MAR può aver inizio una lettura (in iload2).

Tuttavia l'utilizzo di MBR come indice è leggermente diverso dall'uso che ne è stato fatto in BIPUSH. Nel caso di un indice il suo valore è sempre positivo e quindi il byte che lo rappresenta deve essere considerato come un intero senza segno; nel caso dell'istruzione BIPUSH esso è stato invece interpretato come un intero a 8 bit con segno. L'interfaccia tra MBR e il bus è stata attentamente progettata in modo da consentire entrambi i tipi di operazione. Nel caso di BIPUSH (intero con segno a 8 bit) l'operazione corretta è l'estensione con segno, in cui il bit più a sinistra del primo byte di MBR viene copiato nei 24 bit più alti del bus B. Nel caso di ILOAD (intero senza segno a 8 bit) l'operazione corretta è invece un riempimento con valori 0; in questo caso tutti e 24 i bit più alti del bus B vengono impostati a 0. Usando segnali separati è possibile distinguere le due operazioni e indicare quale deve essere effettuata (Figura 4.6). Nel microcodice ciò è indicato da MBR (estensione con segno, come in bipush3) oppure da MBRU (senza segno, come in iload2).

Mentre si attende che la memoria fornisca l'operando (in iload3), SP viene incrementato in modo che il suo valore indichi la locazione in cui memorizzare il risultato, ovvero la nuova cima dello stack. Questo valore viene copiato anche in MAR per preparare la scrittura dell'operando nello stack. PC deve essere nuovamente incrementato per prelevare il codice operativo successivo (in iload4). Infine MDR viene copiato in TOS in modo da riflettere correttamente la nuova cima dello stack (in iload5).

ISTORE è l'operazione inversa di ILOAD e consiste nel rimuovere una parola dalla cima dello stack e nel memorizzarla nella locazione specificata dalla somma tra LV e l'indice contenuto nell'istruzione. ISTORE ha lo stesso formato di ILOAD, mostrato nella Figura 4.19(a), tranne per il fatto che il codice operativo è 0x36 invece che 0x15. Dato che si conosce già (in TOS) il valore della parola in cima allo stack, si potrebbe pensare che sia sufficiente memorizzare il contenuto di TOS direttamente in memoria; in realtà il comportamento dell'istruzione è leggermente più complesso. Occorre infatti prelevare la nuova parola che si trova in cima allo stack e per far ciò è necessario effettuare una lettura oltre alla scrittura; tuttavia non è importante l'ordine secondo il quale vengono eseguite queste due operazioni di memoria (se possibile possono anche essere eseguite in parallelo).

Sia ILOAD sia ISTORE possono accedere solamente alle prime 256 variabili locali. Ovviamente, anche se nella maggior parte dei programmi questo limite potrebbe essere superiore allo spazio effettivamente necessario, è fondamentale avere la possibilità di accedere a una variabile ovunque essa si trovi (nello spazio delle variabili locali). Per superare questa restrizione IJVM utilizza lo stesso meccanismo impiegato in JVM: si usa uno speciale codice operativo, WIDE, conosciuto come **byte di prefisso**, seguito dal codice operativo di ILOAD oppure da quello di ISTORE. Quando si incontra questa sequenza si modificano le definizioni di ILOAD e ISTORE, in modo che l'indice che segue

il codice operativo sia un valore a 16 bit invece che a 8 bit. L'uso del codice operativo WIDE è mostrato nella Figura 4.19(b).

WIDE viene decodificato nel solito modo, effettuando una diramazione che porta all'istruzione wide1 in cui inizia il microcodice specifico per la gestione di questo codice operativo. Anche se il codice operativo sul quale si vuole effettuare l'"allargamento" è già presente in MBR, wide1 preleva il primo byte che lo segue, dato che la logica del microprogramma si aspetta sempre di trovarlo in quella posizione. In wide2 viene effettuata una seconda diramazione in base al byte che segue WIDE. Tuttavia, dato che sia WIDE ILOAD sia WIDE ISTORE richiedono microcodice diverso rispetto a ILOAD e ISTORE, la seconda diramazione non può utilizzare il codice operativo come indirizzo di destinazione, come invece fa Main1.

In wide2 viene invece calcolato l'OR tra il codice operativo e 0x100, e il risultato viene memorizzato in MPC. Come conseguenza l'interpretazione di WIDE ILOAD comincia in 0x115 (al posto di 0x15) e l'interpretazione di WIDE ISTORE comincia in 0x136 (al posto di 0x36), e così via. Così facendo ogni codice operativo di tipo WIDE inizia a un indirizzo che è 256 (cioè, 0x100) parole più in alto rispetto al corrispondente codice operativo di tipo normale. La Figura 4.20 mostra la sequenza iniziale di microistruzioni sia per ILOAD sia per WIDE ILOAD.

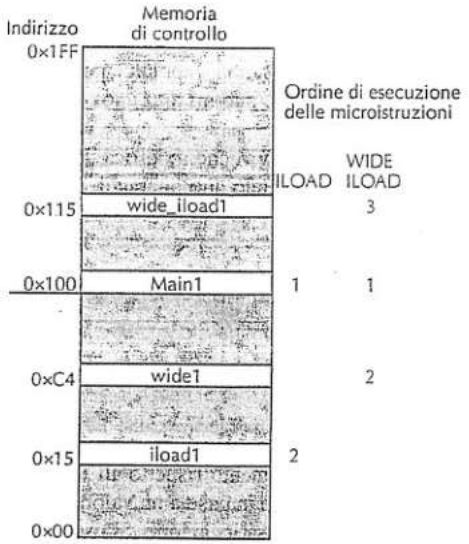


Figura 4.20 La sequenza iniziale di microistruzioni per ILOAD e WIDE ILOAD. Gli indirizzi sono puramente esemplificativi.

Raggiunto il codice che implementa WIDE ILOAD (0x115), esso differisce dal normale codice di ILOAD soltanto per il fatto che l'indice deve essere costruito concatenando 2

byte invece che estendendo con segno un unico byte. La concatenazione e la successiva addizione devono essere compiute in passi distinti, il primo dei quali consiste nel copiare in H il byte 1 dell'indice traslato a sinistra di 8 bit. Dato che l'indice è un intero senza segno, MBR viene poi esteso con valori zero utilizzando MBRU. Successivamente viene sommato il secondo byte (l'operazione di addizione è identica alla concatenazione, dato che ora il byte meno significativo di H vale zero, garantendo quindi che non ci sarà alcun riporto tra i byte) e il risultato viene memorizzato nuovamente in H. A partire da questo punto l'operazione può procedere, esattamente come se fosse il caso normale di ILOAD. Piuttosto che duplicare le ultime istruzioni di ILOAD (da iload3 a iload5), abbiamo semplicemente introdotto un salto da wide_iload4 a iload3. Si noti tuttavia che, durante l'esecuzione dell'istruzione, PC deve essere incrementato due volte affinché punti correttamente al successivo codice operativo. Sia l'istruzione ILOAD, sia la sequenza WIDE_ILOAD lo incrementano una volta.

La stessa situazione si verifica nel caso di WIDE_ISTORE: dopo aver eseguito le prime quattro microistruzioni (da wide_istore1 a wide_istore4), la sequenza è identica a quella di ISTORE a partire dalla terza istruzione. Viene quindi introdotto un salto da wide_istore4 a istore3.

La successiva istruzione d'esempio che prendiamo in considerazione è LDC_W. Questo codice operativo differisce da ILOAD per due aspetti. In primo luogo, l'istruzione ha uno spiazzamento di 16 bit senza segno (come la versione "allargata" di ILOAD). In secondo luogo, l'indice è relativo a CPP e non a LV, dato che la sua funzione consiste nel leggere dalla porzione costante di memoria e non dal blocco delle variabili locali. (In realtà esiste anche una versione corta di LDC_W, chiamata LDC; non l'abbiamo però inclusa in IJVM, dato che la versione lunga comprende già al suo interno tutti i possibili casi di quella corta, anche se richiede 3 byte al posto di soli 2.)

L'istruzione IINC è, oltre a ISTORE, l'unica istruzione IJVM che modifica una variabile locale. Come mostra la Figura 4.21, essa lo fa includendo due operandi, ciascuno dei quali è lungo 1 byte.

Figura 4.21 L'istruzione INC ha due diversi campi per gli operandi.

L'istruzione IINC utilizza INDEX per specificare lo spiazzamento rispetto all'inizio del blocco delle variabili locali, legge la variabile, la incrementa in base a un valore, CONST, contenuto nell'istruzione e memorizza il risultato nella stessa locazione. Si osservi che questa istruzione può incrementare di un valore negativo, nel senso che CONST è una costante a 8 bit con segno, il cui valore è dunque compreso tra –128 e +127. La versione completa di JVM comprende una versione "allargata" di IINC in cui ciascun operando è lungo 2 byte.

Arriviamo ora alla prima istruzione di salto di IJVM: GOTO. L'unica funzione di questa istruzione è quella di modificare il valore di PC, in modo che la successiva istruzione IJVM da eseguire sia quella che si trova nell'indirizzo calcolato sommando lo spiazzamento a 16 bit (con segno) all'indirizzo del codice operativo del salto.Una complicazione è data dal fatto che lo spiazzamento è relativo al valore che PC aveva all'inizio della decodifica dell'istruzione e non a quello prelevato dopo i due byte dello spiazzamento.

Per chiarire questo punto osserviamo nella Figura 4.22(a) la situazione all'inizio di Main1. Il codice operativo è già presente in MBR, ma PC non è ancora stato incrementato. Nella Figura 4.22(b) vediamo la situazione all'inizio di goto1. A questo punto PC è stato incrementato, ma il primo byte dello spiazzamento non è ancora stato prelevato e salvato in MBR. Una microistruzione dopo ci troviamo nel caso della Figura 4.22(c) in cui il vecchio PC, che punta al codice operativo, è stato salvato in OPC e il primo byte dello spiazzamento è stato memorizzato in MBR. Il valore salvato in OPC è necessario, in quanto lo spiazzamento dell'istruzione IJVM GOTO è relativo a esso e non al valore corrente di PC. Questo è in realtà il motivo principale per il quale abbiamo bisogno del registro OPC.

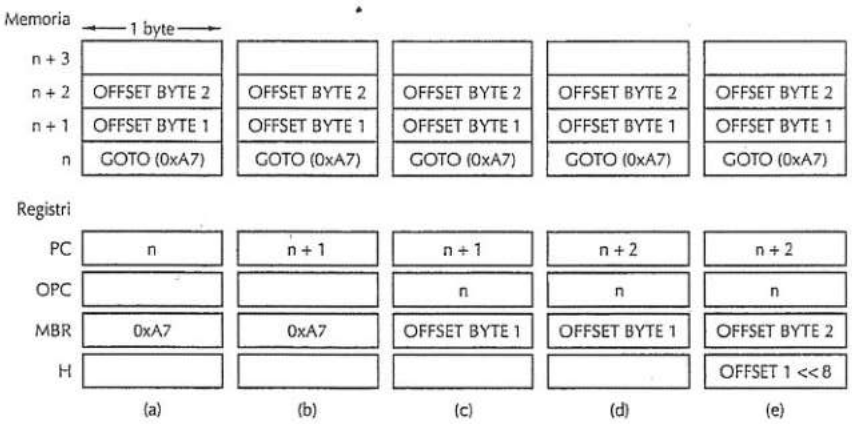


Figura 4.22 Situazione all'inizio di alcune microistruzioni. (a) Main1. (b) goto1. (c) goto2. (d) goto3. (e) goto4.

La microistruzione in goto2 inizia a prelevare il secondo byte dello spiazzamento, portando, all'inizio di goto3, alla situazione mostrata nella Figura 4.22(d). Dopo che il primo byte dello spiazzamento è stato traslato a sinistra di 8 bit e copiato in H, arriviamo a goto4 e allo stato della Figura 4.22(e). Ora il primo byte dello spiazzamento si trova in H traslato a sinistra di 8 bit, il secondo byte dello spiazzamento è in MBR e l'indirizzo base è salvato in OPC. In goto5 otteniamo il nuovo indirizzo da memorizzare in PC

costruendo in H lo spiazzamento completo a 16 bit e sommandolo alla base. Si faccia attenzione al fatto che in goto4 utilizziamo MBRU al posto di MBR, dato che non vogliamo estendere con segno il secondo byte. Inoltre lo spiazzamento a 16 bit viene costruito calcolando in realtà l'OR logico tra le due metà. Infine, prima di tornare all'istruzione Main1, dobbiamo prelevare il successivo codice operativo, dato che all'inizio di ogni iterazione del ciclo principale il microporgramma si aspetta di trovarlo all'interno di MBR.

Nell'istruzione IJVM goto gli spiazzamenti sono valori a 16 bit con segno, compresi tra un minimo di –32768 e un massimo di +32767. Questo significa che non è possibile effettuare salti verso etichette la cui distanza superi questi limiti. Questa proprietà può essere vista come un bug oppure come una delle caratteristiche funzionali di IJVM (e anche di JVM). Chi sostiene che sia un bug ritiene che la definizione di JVM non dovrebbe restringere lo stile di programmazione. Chi invece sostiene che sia una caratteristica di JVM crede che il lavoro di molti programmatori migliorerebbe radicalmente se fossero inseguiti dall'incubo di veder apparire questo temibile messaggio da parte del compilatore

```
Il programma è troppo grande e complesso. Occorre riscriverlo.
La compilazione è annullata.
```

Sfortunatamente (dal nostro punto di vista) questo messaggio appare solamente quando una clausola then o else supera 32 KB, corrispondenti a circa 50 pagine di codice Java.

Consideriamo ora le tre istruzioni di salto condizionale di IJVM: IFLT, IFEQ e IF_ICMPEQ. Le prime due estraggono la parola in cima allo stack ed effettuano una diramazione se e solo se è rispettivamente minore di zero oppure uguale a zero. L'istruzione IF_ICMPEQ estrae invece le due parole in cima allo stack e compie una diramazione se e solo se sono uguali. In tutti e tre i casi è necessario leggere la nuova parola che si trova in cima allo stack e memorizzarla in TOS.

Per tutte e tre queste istruzioni il controllo è simile. Inizialmente l'operando, o gli operandi, vengono salvati nei registri; successivamente viene letto il nuovo valore in cima allo stack e inserito in TOS; infine vengono effettuati il test e la diramazione. Come prima istruzione consideriamo IFLT. La parola da testare è già presente in TOS, ma dato che IFLT estrae una parola dallo stack occorre leggere la nuova parola che si trova in cima allo stack per memorizzarla in TOS. Questa lettura inizia in iflt1. Durante iflt2 viene temporaneamente salvata in OPC la parola da testare, in modo che il nuovo valore possa essere memorizzato in TOS senza perdere il valore corrente. Infine, durante iflt4, la parola che si vuole testare, salvata in OPC, viene fatta passare nella ALU senza memorizzarla in modo da testare il bit N. Questa microistruzione contiene anche una diramazione che sceglie la destinazione T se il test ha successo, mentre la F in caso contrario.

Se il test ha successo il resto dell'operazione è essenzialmente uguale all'istruzione GOTO; la sequenza continua quindi con goto2, in un punto intermedio della sequenza di GOTO. Se invece il test fallisce occorre eseguire una sequenza di breve durata (F, F2 e F3) per scavalcare il resto dell'istruzione (lo spiazzamento) prima di ritornare a Main1 e continuare con l'istruzione successiva.

Il codice in ifeq2 e ifeq3 segue la stessa logica, con l'unica differenza che si usa il bit Z invece del bit N. In entrambi i casi è compito dell'assemblatore di MAL riconoscere che gli indirizzi T e F sono speciali e assicurarsi che siano posizionati nella memoria di controllo in modo che i loro indirizzi differiscano soltanto nel bit più a sinistra.

Anche la logica dell'istruzione IF_ICMPEQ è più o meno simile a quella di IFEQ, tranne per il fatto che in questo caso dobbiamo leggere anche il secondo operando. Il secondo operando viene caricato in H durante if_icmpeq3, quando inizia la lettura della nuova parola in cima allo stack. Anche questa volta la parola che si trova in cima allo stack viene salvata in OPC e quella nuova viene memorizzata in TOS. Infine il test if_icmpeq6 è simile a quello in ifeq4.

Consideriamo ora l'implementazione di INVOKEVIRTUAL e IRETURN; com'è stato descritto nel Paragrafo 4.2.3 sono le istruzioni che permettono di richiamare una procedura e di restituire il controllo al chiamante. INVOKEVIRTUAL è l'istruzione più complessa implementata in IJVM ed è costituita da una sequenza di 22 microistruzioni, mostrate nella Figura 4.12. L'istruzione utilizza uno spiazzamento a 16 bit per determinare l'indirizzo del metodo da invocare. Nella nostra implementazione lo spiazzamento è semplicemente un valore di distanza all'interno della porzione costante di memoria; questa locazione punta al metodo da invocare. Non dimentichiamo però che i primi 4 byte di ciascun metodo *non* sono istruzioni; essi contengono infatti due puntatori a 16 bit. Il primo fornisce il numero di parole usate per i parametri (compreso OBJREF, vedi Figura 4.12). Il secondo fornisce invece la dimensione dell'area delle variabili locali espressa come numero di parole. Questi campi sono prelevati attraverso la porta a 8 bit e vengono assemblati come se fossero due spiazzamenti a 16 bit presenti all'interno di un'istruzione.

Successivamente vengono memorizzate delle informazioni di collegamento necessarie per poter riportare la macchina allo stato precedente. Queste informazioni comprendono l'indirizzo d'inizio di ciascuna area delle variabili locali oltre al precedente valore di PC; esse sono memorizzate immediatamente sopra l'area delle variabili locali appena creata e sotto il nuovo stack. Infine, prima di ritornare a Main1 e poter iniziare l'esecuzione della nuova istruzione, viene prelevato il codice operativo dell'istruzione successiva e PC viene incrementato.

IRETURN è un'istruzione semplice che non contiene alcun operando. Essa utilizza semplicemente l'indirizzo memorizzato nella prima parola dell'area delle variabili locali per recuperare le informazioni di collegamento. In seguito reimposta SP, LV e PC ai loro precedenti valori e, come mostra la Figura 4.13, copia in cima allo stack originale il valore di ritorno presente in cima allo stack corrente.

4.4 Progettazione del livello di microarchitettura

La progettazione del livello di microarchitettura, come tutto ciò che riguarda i calcolatori, è caratterizzata da compromessi e bilanciamenti. Sono molte le caratteristiche di un calcolatore che si vorrebbero ottimizzare: velocità, costi, affidabilità, facilità di utilizzo, consumo energetico e dimensioni fisiche. Fra tutti i possibili compromessi ce n'è uno in particolare che determina le principali scelte che un progettista deve compiere: l'equili-

brio tra la velocità della macchina e i suoi costi. In questo paragrafo analizzeremo il problema nel dettaglio per vedere quali elementi occorre bilanciare, come riuscire a ottenere prestazioni elevate e a quale costo, in termini di hardware e complessità, ciò sia possibile.

4.4.1 Velocità/costi

Ultimamente l'impiego di nuove tecnologie ha prodotto nei calcolatori il maggior incremento di velocità finora registrato; tuttavia questo aspetto esula dagli scopi del libro, dato che siamo maggiormente interessati agli aumenti di velocità dovuti all'architettura. I miglioramenti dovuti a modifiche dell'architettura dei calcolatori, pur non essendo stati stupefacenti quanto quelli dovuti all'impiego di circuiti più rapidi, hanno avuto comunque un impatto notevole. La velocità può essere misurata in più modi; in ogni caso, una volta scelto un ISA e stabilita la tecnologia dei circuiti, esistono principalmente tre approcci tramite i quali è possibile aumentare la velocità di esecuzione.

1. Ridurre il numero di cicli di clock necessari per eseguire un'istruzione.
2. Semplificare l'organizzazione in modo che il ciclo di clock possa essere più breve.
3. Sovrapporre l'esecuzione delle istruzioni.

Mentre i primi due punti sono ovvi, esiste un numero sorprendentemente alto di possibili progettazioni che possono influire in modo drastico sul numero di cicli di clock, sul periodo di clock o, molto spesso, su entrambi i fattori. In questo paragrafo daremo un esempio di come la codifica e la decodifica di un'istruzione siano in grado di modificare il ciclo di clock.

Il numero di cicli di clock necessario per eseguire un insieme di operazioni è conosciuto con il nome di **lunghezza del percorso**. In alcuni casi la lunghezza del percorso può essere accorciata aggiungendo dei componenti hardware specializzati. Se per esempio si aggiunge al PC un incrementatore (ovvero un sommatore in cui un addendo è sempre impostato a 1) è possibile eliminare alcuni cicli, dato che non è più necessario utilizzare la ALU per far avanzare il suo valore; in questo caso il prezzo da pagare è rappresentato dall'hardware aggiuntivo. Tuttavia questa possibilità non aiuta tanto quanto ci si potrebbe aspettare. Per la maggior parte delle istruzioni, durante gli stessi cicli spesi per incrementare PC, viene infatti svolta un'operazione di lettura; di conseguenza l'esecuzione dell'istruzione successiva non può essere anticipata, dato che dipende dal dato che deve giungere dalla memoria.

Se si intende ridurre il numero di cicli necessari per prelevare le istruzioni occorre qualcosa di più rispetto al solo circuito che incrementa il PC. La tecnica più efficace per una significativa accelerazione del prelievo delle istruzioni è la terza, ovvero la sovrapposizione dell'esecuzione di più istruzioni. Separare i componenti del circuito relativi al prelievo dell'istruzione (la porta a 8 bit della memoria e i registri MBR e PC) è più efficace se si rende quest'unità indipendente, dal punto di vista funzionale, dal percorso dati. In questo modo l'unità può prelevare autonomamente il successivo codice operativo o operando, anche in modo asincrono rispetto al resto della CPU. In tal modo è possibile prelevare in anticipo una o più istruzioni.

Nell'esecuzione di molte istruzioni una delle operazioni più costose in termini di tempo consiste nel prelevare uno spiazzamento a 2 byte, nell'estenderlo in modo appropriato e nel depositarlo nel registro H in preparazione di un'addizione (ciò serve, per esempio, per calcolare l'indirizzo PC ± n a cui porta una diramazione). Una possibile soluzione potrebbe essere quella di rendere la porta della memoria larga 16 bit. Purtroppo però questa scelta complicherebbe l'operazione; la larghezza della memoria è infatti di 32 bit e, dato che i 16 bit richiesti potrebbero superare i limiti della parola, anche una singola lettura di 32 bit non riuscirebbe necessariamente a prelevare entrambi i byte richiesti.

La sovrapposizione dell'esecuzione delle istruzioni è di gran lunga la soluzione più interessante e offre le maggiori opportunità per un incremento drastico della velocità. La semplice sovrapposizione del prelievo e dell'esecuzione di un'istruzione ha un'efficacia sorprendente. Esistono anche alcune tecniche più sofisticate che si spingono oltre, permettendo per esempio di sovrapporre l'esecuzione di più istruzioni. Questa idea è infatti il cuore delle architetture dei calcolatori moderni. In seguito illustreremo alcune delle tecniche più semplici per sovrapporre l'esecuzione delle istruzioni e spiegheremo quali sono le motivazioni che portano all'adozione di tecniche ancora più sofisticate.

La velocità costituisce soltanto metà del problema; l'altra è rappresentata dai costi. Anche il costo può essere misurato in vari modi, ma è difficile darne una precisa definizione. Alcune misure si basano semplicemente sul numero di componenti (il che aveva un senso quando i processori venivano costruiti a partire da componenti acquistati separatamente e poi assemblati). Oggi l'intero processore è invece realizzato su un unico chip. Resta comunque vero che chip più complessi sono più costosi rispetto a quelli più semplici e piccoli. Anche se è possibile contare i singoli componenti, come i transistor, le porte logiche e le unità funzionali, spesso questo valore non è importante quanto invece lo è l'area occupata sul circuito integrato. Maggiori sono le funzioni incluse nel processore e più grande diventa il chip; all'aumentare delle sue dimensioni i costi di produzione crescono, ma in modo molto più veloce rispetto alla sua area. Per questo motivo spesso i progettisti parlano di costi in termini di "proprietà terriera", riferendosi in questo modo all'area richiesta dal circuito (presumibilmente misurata in pico-acri).

Uno dei circuiti che nella storia è stato studiato in modo più accurato è il sommatore. Sono stati realizzati migliaia di progetti e quelli più veloci, oltre a essere molto complessi, hanno prestazioni decisamente migliori rispetto a quelli più lenti. Il progettista di sistema deve decidere se l'aumento di velocità giustifica l'aumento della "proprietà terriera".

I sommatori non sono però gli unici componenti per i quali esistono più scelte possibili. Praticamente qualsiasi componente del sistema può essere progettato per funzionare più velocemente o più lentamente, con un'influenza sul costo. La sfida che si presenta al progettista è quella di identificare i componenti che possono migliorare il sistema nel modo più marcato, aumentandone la velocità. È interessante sapere che spesso un singolo componente può essere sostituito da uno molto più veloce producendo però soltanto un effetto limitato, se non addirittura nullo, sulla velocità globale del sistema. Nei paragrafi successivi analizzeremo alcuni problemi di progettazione e i relativi compromessi nella scelta dei componenti.

Per determinare a quale velocità può arrivare il clock, uno dei fattori chiave è la quantità di lavoro da eseguire durante ogni ciclo. Ovviamente all'aumentare del lavoro da eseguire il ciclo di clock si allunga. In realtà però la cosa non è così semplice, dato che l'hardware è sufficientemente evoluto per eseguire più compiti in parallelo; ciò che determina effettivamente la lunghezza del ciclo di clock è quindi la sequenza di operazioni che devono essere eseguite *serialmente* durante ogni singolo ciclo.

Un aspetto che può essere controllato è la quantità di operazioni di decodifica da eseguire. Per esempio, nella Figura 4.6 abbiamo visto che, nonostante qualunque dei nove registri potesse essere caricato nella ALU dal bus B, all'interno della parola della microistruzione avevamo bisogno solamente di 4 bit per indicare il registro selezionato. Purtroppo questo risparmio ha i suoi contro: il ritardo introdotto dal circuito di decodifica nel percorso dati. Ciò significa che il registro selezionato per portare i propri dati sul bus B riceverà il comando leggermente in ritardo e solo in seguito i suoi dati verranno effettivamente messi sul bus. Ciò provoca un effetto a catena: la ALU riceverà i propri input, e quindi produrrà il risultato, leggermente in ritardo e, con lo stesso ritardo, il risultato verrà scritto sul bus C. Dato che spesso questo ritardo determina quanto deve essere lungo il ciclo di clock, ciò potrebbe limitare la sua frequenza, costringendo quindi l'intero calcolatore a funzionare a una velocità leggermente inferiore. C'è dunque un bilanciamento tra velocità e costi. La riduzione della memoria di controllo di 5 bit per parola avviene a spese di un rallentamento del clock. Il progettista, quando deve compiere la scelta più appropriata, deve tener ben presente gli obiettivi della progettazione. Se si vuole implementare un sistema ad alte prestazioni, l'utilizzo di un decodificatore non è probabilmente una buona idea, ma lo è per i sistemi a basso costo.

4.4.2 Riduzione della lunghezza del percorso di esecuzione

La macchina Mic-1 è stata progettata per essere allo stesso tempo moderatamente semplice e moderatamente veloce, anche se questi obiettivi sono chiaramente in conflitto l'uno con l'altro. In poche parole, le macchine semplici non sono veloci e le macchine veloci non sono semplici. La CPU di Mic-1 utilizza inoltre una quantità minima di hardware: 10 registri, la semplice ALU della Figura 3.19 replicata 32 volte, uno shifter, un decodificatore, una memoria di controllo e alcuni bit qua e là, necessari per tenere insieme i diversi componenti. L'intero sistema potrebbe essere costruito con meno di 5000 transistor, la ROM di controllo e la memoria centrale.

Dopo aver visto com'è possibile implementare IJVM mediante microcodice e con poco hardware è giunto ora il momento di analizzare delle alternative più veloci. Studieremo ora alcuni modi per ridurre il numero di microistruzioni delle istruzioni ISA (cioè, per ridurre la lunghezza del percorso di esecuzione).

Unione del ciclo d'interpretazione con il microcodice

In Mic-1 il ciclo principale consiste in una microistruzione da eseguire all'inizio di ogni istruzione IJVM. In alcuni casi è possibile sovrapporla all'istruzione precedente e in realtà in parte questo è già stato fatto. Notiamo infatti che quando `Main1` viene eseguito il codice operativo da interpretare è già presente all'interno di `MBR`. Il codice operativo

si trova già nel registro, in quanto è già stato prelevato o dal precedente ciclo principale oppure durante l'esecuzione dell'istruzione precedente.

È possibile spingere ancora più in avanti quest'idea, che consiste nel sovrapporre l'inizio dell'istruzione. In alcuni casi è possibile addirittura ridurre il ciclo principale fino a farlo scomparire. Ciò può verificarsi nel modo seguente. Consideriamo le sequenze di microistruzioni che terminano con un salto a `Main1`. In ognuno di questi punti l'istruzione del ciclo principale può essere spostata alla fine della sequenza (piuttosto che all'inizio della sequenza successiva), replicando così in più punti del microcodice la diramazione a più destinazioni (mantenendo però sempre lo stesso insieme di destinazioni). In alcuni casi la microistruzione `Main1` può essere unita alle microistruzioni precedenti, dato che non sempre queste vengono utilizzate completamente.

La Figura 4.23 mostra la sequenza dinamica di microistruzioni nel caso di un'istruzione `POP`. Il ciclo principale si verifica prima e dopo ciascuna istruzione, anche se la figura mostra solamente l'occorrenza successiva all'istruzione `POP`. Notiamo che l'esecuzione di questa istruzione richiede quattro cicli di clock: tre per le microistruzioni specifiche di `POP` e uno per il ciclo principale.

Etichetta	Operazioni	Commenti
pop1	MAR = SP = SP – 1; rd	Legge la parola sotto la cima dello stack
pop2		Attende che il nuovo TOS sia letto dalla memoria
pop3	TOS = MDR; goto Main1	Copia la nuova parola in TOS
Main1	PC = PC + 1; fetch; goto (MBR)	MBR contiene il codice operativo; prelievo del byte successivo; diramazione

Figura 4.23 Sequenza della versione originale del microprogramma per l'esecuzione di POP.

Nella Figura 4.24 la sequenza è stata ridotta a tre istruzioni, unendo le istruzioni del ciclo principale alle altre; per eseguire alcune delle operazioni di `Main1` è stato infatti sfruttato il ciclo di clock `pop2` in cui la ALU non veniva utilizzata. È importante notare che questa sequenza termina con una diramazione che porta l'esecuzione direttamente nel punto del codice specifico per l'istruzione successiva: complessivamente tre cicli sono sufficienti. Questa piccola modifica riduce di un ciclo il tempo di esecuzione della successiva microistruzione. Ciò equivale ad accelerare il clock da 250 MHz (microistruzioni da 4 ns) a 333 MHz (microistruzioni da 3 ns), senza realmente modificarne la temporizzazione.

L'istruzione `POP` si presta particolarmente bene a questo trattamento, dato che nel mezzo della sua esecuzione è presente un ciclo inattivo in cui la ALU non viene utilizzata.

Dato che il ciclo principale richiede l'uso della ALU, se si vuole ridurre la lunghezza di un'istruzione, è necessario trovare al suo interno un ciclo in cui la ALU è inattiva. Questi cicli "morti" non sono comuni, ma quando si verificano vale la pena unire `Main1` alla fine di ciascuna sequenza di microistruzioni. Il prezzo da pagare è solamente un

piccolo incremento della memoria di controllo. È possibile formulare in questo modo la prima tecnica che ci permette di ridurre la lunghezza del percorso:

unire il ciclo d'interpretazione alla fine delle sequenze di microcodice.

Etichetta	Operazioni	Commenti
pop1	MAR = SP = SP – 1; rd	Legge la parola sotto la cima dello stack
Main1.pop	PC = PC + 1; fetch	MBR contiene il codice operativo; prelievo del byte successivo
pop3	TOS = MDR; goto (MBR)	Copia la nuova parola in TOS; diramazione a seconda del codice operativo

Figura 4.24 Sequenza della versione migliorata del microprogramma per l'esecuzione di POP.

Architettura a tre bus

Cos'altro possiamo fare per ridurre la lunghezza del percorso di esecuzione? Un'altra semplice modifica consiste nell'utilizzo di due bus di input, A e B, per la ALU, per un totale di tre bus. Tutti (almeno la maggior parte) dei registri dovrebbero avere accesso a entrambi i bus di input. Il vantaggio di avere due bus di input è che in questo modo diventa possibile sommare fra loro, in unico ciclo, due registri qualsiasi. Per comprendere il valore di questa funzionalità consideriamo l'implementazione di `ILOAD` per Mic-1 (Figura 4.25).

Etichetta	Operazioni	Commenti
iload1	H = LV	MBR contiene l'indice; copia LV in H
iload2	MAR = MBRU + H; rd	MAR = indirizzo della variabile locale da inserire nello stack
iload3	MAR = SP = SP + 1	SP punta alla nuova cima dello stack; prepara la scrittura
iload4	PC = PC + 1; fetch; wr	Incrementa PC; prelievo del successivo codice operativo; scrive in cima allo stack
iload5	TOS = MDR; goto Main1	Aggiorna TOS
Main1	PC = PC + 1; fetch; goto (MBR)	MBR contiene il codice operativo; prelievo del byte successivo; diramazione

Figura 4.25 Codice di Mic-1 per l'esecuzione di ILOAD.

Possiamo osservare che durante `iload1` `LV` è copiato all'interno di `H`. Ciò avviene per l'unico motivo di poterlo successivamente sommare a `MBRU` durante `iload2`. Nella nostra architettura originale a due bus non esiste alcun modo per sommare direttamente due registri arbitrari; l'unica soluzione consiste nel copiarne inizialmente uno in `H`. Nella nuova architettura a tre bus possiamo invece risparmiare un ciclo, come si può vedere nella Figura 4.26. In questo caso abbiamo sommato il ciclo d'interpretazione a `ILOAD`, ma così facendo non abbiamo né aumentato né diminuito la lunghezza del percorso di esecuzione.

Etichetta	Operazioni	Commenti
iload1	MAR = MBRU + LV; rd	MAR = indirizzo della variabile locale da inserire sullo stack
iload2	MAR = SP = SP + 1	SP punta alla nuova cima dello stack; prepara la scrittura
iload3	PC = PC + 1; fetch; wr	Incrementa PC; prelievo del successivo codice operativo; scrive in cima allo stack
iload4	TOS = MDR	Aggiorna TOS
iload5	PC = PC + 1; fetch; goto (MBR)	MBR contiene già il codice operativo; prelievo del byte dell'indice

Figura 4.26 Codice dell'architettura a tre bus per l'esecuzione di ILOAD.

Tuttavia l'aggiunta del terzo bus ha ridotto il tempo totale di esecuzione di `ILOAD`, portandolo da sei a cinque cicli. A questo punto abbiamo a disposizione una seconda tecnica che ci permette di ridurre la lunghezza del percorso:

passare da un'architettura a due bus a una a tre bus.

Unità di prelievo dell'istruzione

Entrambe le tecniche precedenti sono valide, ma per ottenere un netto miglioramento delle prestazioni abbiamo bisogno di qualcosa di molto più radicale. Facciamo un passo indietro e analizziamo le parti comuni a tutte le istruzioni, ovvero il prelievo e la decodifica dei campi. Per ogni istruzione si possono verificare le seguenti operazioni:

1. il valore di PC viene fatto passare attraverso la ALU per incrementarlo;
2. il valore di PC è utilizzato per prelevare il byte successivo nel flusso delle istruzioni;
3. gli operandi sono letti dalla memoria;
4. gli operandi sono scritti in memoria;
5. la ALU esegue una computazione e i risultati vengono memorizzati.

Se un'istruzione contiene campi aggiuntivi (per gli operandi), questi vanno prelevati esplicitamente, un byte alla volta, e poi assemblati prima di poterli utilizzare. Prelevare e assemblare un operando impegna la ALU per almeno un ciclo per byte, per incrementare PC e in seguito assemblare l'indice, o lo spiazzamento, risultante. Durante praticamente ogni ciclo la ALU viene utilizzata, oltre che per il "lavoro" vero e proprio dell'istruzione, anche per una grande varietà di operazioni relative al prelievo dell'istruzione e l'assemblaggio dei campi al suo interno.

Per poter sovrapporre il ciclo principale è necessario liberare la ALU da alcuni di questi compiti. Ciò potrebbe essere fatto introducendo una seconda ALU, anche se per

molte operazioni non è realmente necessario disporre di tutte le funzionalità che fornisce una ALU completa. Possiamo notare infatti che in molti casi la ALU viene semplicemente usata come un cammino per copiare un valore da un registro a un altro. Questi cicli potrebbero essere eliminati introducendo percorsi dati aggiuntivi che non passino attraverso la ALU. Per esempio potrebbe essere vantaggioso creare un percorso da TOS a MDR, oppure da MDR a TOS, dato che la parola in cima allo stack viene scambiata frequentemente tra questi due registri.

In Mic-1 è possibile eliminare gran parte del carico di lavoro che grava sulla ALU creando un'unità indipendente che si occupa del prelievo e dell'elaborazione delle istruzioni. Questa unità, chiamata **IFU** (*Instruction Fetch Unit*, "unità di fetch delle istruzioni"), può incrementare il PC in modo indipendente e può prelevare i byte prima ancora che siano richiesti. Per implementare questa unità è sufficiente un incrementatore, cioè un circuito molto più semplice di un sommatore. Portando ancora più avanti l'idea di un'unità separata per la gestione delle istruzioni è possibile realizzare una IFU in grado di assemblare gli operandi a 8 e 16 bit, in modo che siano immediatamente pronti in qualsiasi momento vengano richiesti. Per poter far ciò esistono almeno due modi distinti.

1. La IFU può interpretare realmente ciascun codice operativo, determinando quanti campi aggiuntivi devono essere prelevati, e assemblarli in un registro pronto per essere utilizzato dall'unità di esecuzione principale.
2. La IFU può trarre vantaggio dal fatto che le istruzioni sono in realtà un flusso di byte e rendere disponibili, in ogni momento, le successive porzioni di 8 e 16 bit, anche se potrebbero non essere necessarie. Spetta poi all'unità di esecuzione principale richiedere i dati veramente necessari.

La Figura 4.27 mostra i principi fondamentali del secondo schema. Al posto di un solo registro MBR a 8 bit, ora ce ne sono due: MBR1 a 8 bit e MBR2 a 16 bit. La IFU tiene traccia del byte, o dei byte, utilizzati più recentemente dall'unità di esecuzione principale. Inoltre, allo stesso modo di quanto avveniva in Mic-1, rende disponibile all'interno di MBR1 il byte successivo; la differenza è che in questo caso l'unità, non appena rileva che MBR1 è stato letto, preleva il byte successivo e lo memorizza immediatamente all'interno di MBR1. Come in Mic-1 il registro ha due interfacce con il bus B: MBR1 e MBR1U. La prima è estesa col segno a 32 bit, mentre la seconda è estesa con bit zero.

In modo analogo, MBR2 fornisce le stesse funzionalità memorizzando i successivi 2 byte. Anche questo registro ha due interfacce con il bus B, MBR2 e MBR2U, che forniscono rispettivamente l'estensione con il segno a 32 bit e l'estensione con bit zero.

La IFU è responsabile del prelievo di un flusso di byte. Essa compie il proprio lavoro utilizzando una convenzionale porta di memoria a 4 byte, prelevando in anticipo intere parole a 4 byte e caricando i byte consecutivi all'interno di un registro a scorrimento. Questo registro ha la funzione di mantenere una coda di byte provenienti dalla memoria per alimentare MBR1 e MBR2, fornendo loro uno, o due byte, alla volta nell'ordine in cui sono stati prelevati.

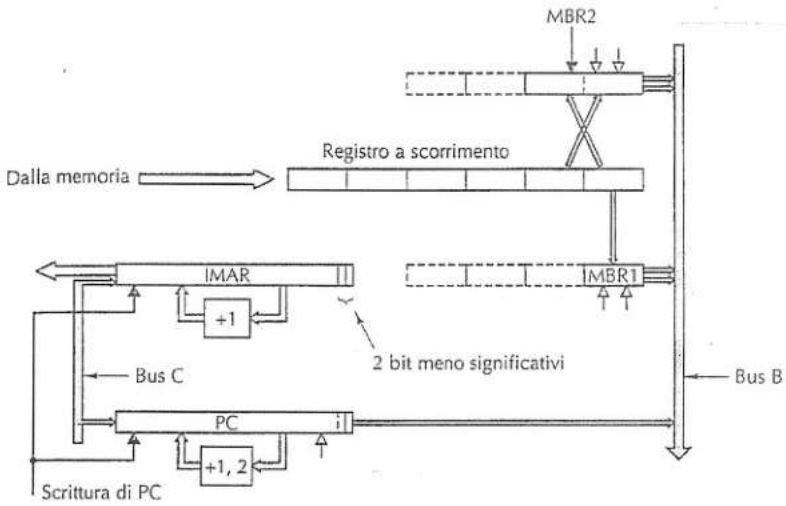


Figura 4.27 Unità di prelievo per Mic-1.

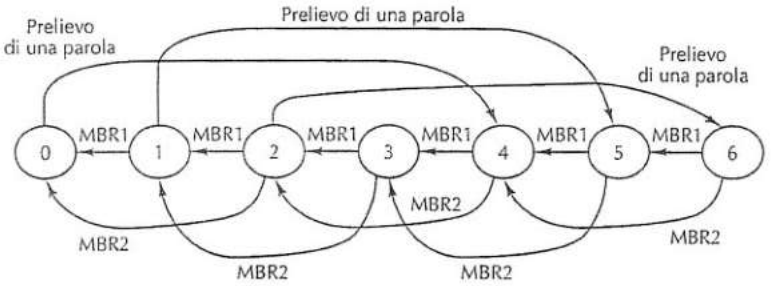
MBR1 contiene sempre il byte più vecchio del registro a scorrimento, mentre MBR2 contiene i 2 byte più vecchi (il più vecchio dei due è a sinistra), che servono a formare un intero a 16 bit, come mostrato nella Figura 4.19(b). I 2 byte di MBR2 possono anche provenire da parole diverse, dato che in memoria le istruzioni IJVM non sono allineate alle parole.

Ogni volta che viene letto MBR1, il registro a scorrimento viene traslato a destra di 1 byte, e di 2 quando viene letto MBR2. In seguito MBR1 e MBR2 vengono ricaricati rispettivamente con il byte o con la coppia di byte più vecchi. Se nella parte sinistra del registro a scorrimento c'è sufficiente spazio per contenere un'intera nuova parola, la IFU inizia un ciclo di memoria per leggerla. Assumiamo che, dopo la lettura di uno dei due registri MBR, questo venga nuovamente riempito all'inizio del ciclo successivo, in modo che sia possibile effettuare più letture durante cicli consecutivi.

Come mostra la Figura 4.28, è possibile modellare l'architettura della IFU mediante un **automa a stati finiti**. Questi automi sono costituiti da **stati** (rappresentati da cerchi) e da **transizioni** (rappresentate come archi che collegano uno stato con un altro). Ogni stato rappresenta una possibile situazione nella quale si può trovare l'automa. Il nostro automa ha sette stati, corrispondenti ai sette stati del registro a scorrimento della Figura 4.27. Gli stati sono identificati dagli interi da 0 a 6, che indicano il numero di byte presenti in un dato momento nel registro.

Un arco rappresenta un evento che si può verificare. Nel nostro caso ci sono tre possibili eventi. Il primo è la lettura di un byte da MBR1, e quando si verifica il registro a scorrimento viene attivato e trasla a destra per estrarre 1 byte, riducendo dunque lo stato di 1. Il secondo evento è una lettura di 2 byte da MBR2, che riduce lo stato di 2. Entram-

be queste transizioni fanno sì che MBR1 e MBR2 vengano ricaricati nuovamente. Quando l'automa si porta negli stati 0, 1 o 2, ha inizio un riferimento alla memoria per prelevare una nuova parola (assumendo che la memoria non sia già occupata nella lettura di una parola). Quando termina la lettura della parola l'automa avanza di 4 stati.



Transizioni
MBR1: avviene quando viene letta MBR1
MBR2: avviene quando viene letta MBR2
Prelievo di una parola: avviene quando viene letta una parola della memoria e vengono inseriti 4 byte nel registro a scorrimento

Figura 4.28 Automa a stati finiti che implementa la IFU.

Per un corretto funzionamento, l'automa deve bloccarsi quando gli viene fatta una richiesta che non è in grado di soddisfare, per esempio fornire il valore di MBR2 quando all'interno del registro c'è un solo byte e la memoria è ancora occupata nel prelievo della nuova parola. Inoltre l'automa può eseguire solamente un'azione alla volta e occorre quindi serializzare gli eventi in entrata. Infine, l'automa deve essere aggiornato ogni volta che PC viene modificato. Questi dettagli rendono l'automa più complicato di quanto abbiamo mostrato precedentemente; tuttavia molti dispositivi hardware sono progettati come automi a stati finiti.

La IFU possiede un proprio registro per l'indirizzo di memoria, chiamato IMAR, utilizzato per puntare all'indirizzo di memoria in cui si trova la parola da prelevare. Questo registro possiede un proprio incrementatore e non occorre quindi utilizzare la ALU per aggiornarlo (affinché si riferisca alla parola successiva). La IFU deve monitorare il bus C per rilevare quando il PC viene caricato e copiare di conseguenza il nuovo valore in IMAR. Dato che il nuovo valore del PC potrebbe non trovarsi sul limite di una parola, la IFU deve prelevare la parola corretta e regolare in modo appropriato il registro di scorrimento.

Utilizzando la IFU, il PC viene modificato dalla ALU solamente nel caso in cui sia necessario cambiare la natura sequenziale del flusso di byte delle istruzioni. Ciò si verifica quando viene soddisfatta la condizione di una diramazione e nel caso delle istruzioni INVOKEVIRTUAL e IRETURN.

È compito della IFU tenere aggiornato il PC, dato che il microprogramma non lo incrementa più in modo esplicito quando vengono prelevati i codici operativi. Per sapere quando occorre incrementare il PC, la IFU rileva quando viene consumato un byte del flusso dell'istruzione, cioè quando viene letto MBR1, MBR2, o le loro versioni senza segno. A seconda di quanti byte sono stati consumati un incrementatore aumenta il valore del PC di 1 oppure di 2. In questo modo il PC contiene sempre l'indirizzo del primo byte non ancora consumato. All'inizio di ogni istruzione, MBR1 contiene il codice operativo dell'istruzione stessa.

Si noti la presenza di due incrementatori, con funzioni distinte. PC conta i *byte* e viene incrementato di 1 o di 2; IMAR conta invece le *parole* e viene incrementato soltanto di 1 (per 4 nuovi byte). Analogamente a MAR, anche IMAR è collegato al bus dell'indirizzo in modo disallineato, con il bit 0 di IMAR connesso alla linea d'indirizzo 2, e così via, in modo da convertire implicitamente gli indirizzi di parole in indirizzi di byte.

Come vedremo tra poco in modo più dettagliato, il fatto di non dover incrementare PC nel ciclo principale rappresenta un grande vantaggio. Spesso infatti la microistruzione nella quale viene incrementato il PC compie poco lavoro aggiuntivo; se si eliminano queste istruzioni si riduce quindi il percorso di esecuzione. In questo caso il compromesso sta nell'aumento del numero di componenti hardware per disporre di una macchina più veloce. La nostra terza tecnica per ridurre la lunghezza del percorso prevede quindi

che le istruzioni siano prelevate dalla memoria da un'unità funzionale specializzata.

4.4.3 Architettura con prefetching: Mic-2

La IFU permette di ridurre in modo considerevole la lunghezza media del percorso di un'istruzione. In primo luogo elimina interamente il ciclo principale, dato che alla fine di ogni istruzione si effettua un semplice salto all'istruzione successiva. Inoltre risparmia l'utilizzo della ALU per l'incremento del PC, e riduce la lunghezza del percorso ogni volta che viene calcolato un indice o uno spiazzamento a 16 bit (dato che assembla il valore a 16 bit e lo fornisce direttamente alla ALU come un valore a 32 bit, evitando di doverlo assemblare all'interno di H). La Figura 4.29 mostra Mic-2, cioè una versione migliorata di Mic-1 in cui è stata aggiunta la IFU della Figura 4.27. La Figura 4.30 mostra il microcodice per la nuova macchina.

Per vedere un esempio del funzionamento di Mic-2 consideriamo IADD. La macchina preleva la seconda parola dello stack ed esegue l'addizione come prima, tranne per il fatto che, una volta terminata l'operazione, non è più necessario tornare a Main1 per incrementare PC e passare alla microistruzione successiva. Quando la IFU vede che durante iadd3 è stato effettuato un riferimento a MBR1, il suo registro a scorrimento trasla tutto il proprio contenuto a destra e ricarica sia MBR1 sia MBR2. L'unità effettua quindi una transizione verso uno stato che è inferiore di uno rispetto a quello corrente e, se il nuovo stato è 2, la IFU inizia a prelevare una nuova parola dalla memoria. Tutto ciò viene effettuato dall'hardware, senza alcun intervento del microcodice. Questo è il motivo che permette di ridurre IADD da quattro a tre microistruzioni.

Mic-2 permette di ottenere miglioramenti più marcati per alcune istruzioni piuttosto che altre. LDC_W passa da nove a tre microistruzioni; SWAP invece passa soltanto da otto

a sei microistruzioni. Per migliorare le prestazioni globali del sistema ciò che conta veramente è il miglioramento che si può ottenere con le istruzioni usate più frequentemente. Le istruzioni più comuni sono `ILOAD` (da 6 a 3), `IADD` (da 4 a 3) e `IF_ICMPEQ` (da 13 a 10 quando la condizione è verificata; da 10 a 8 altrimenti). Anche se per misurare il miglioramento delle prestazioni bisognerebbe eseguire dei test, è tuttavia evidente che il guadagno sia significativo.

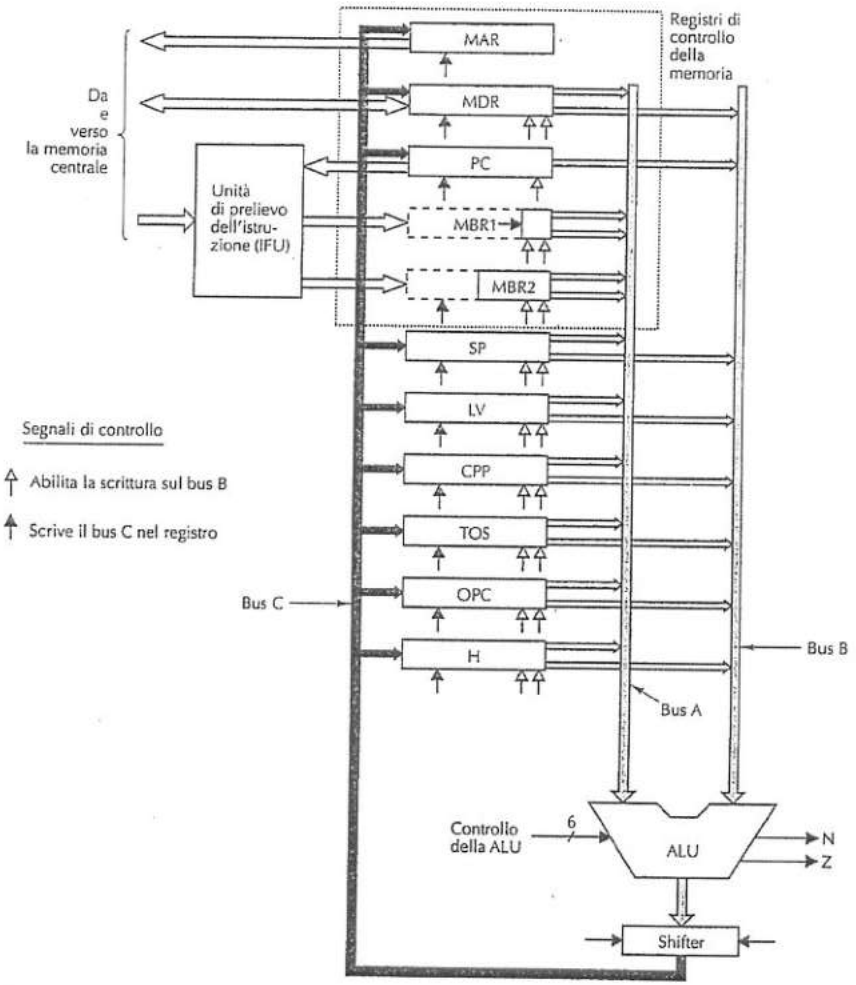


Figura 4.29 Percorso dati di Mic-2.

Etichetta	Operazioni	Commenti
nop1	goto (MBR)	Salto all'istruzione successiva
iadd1 iadd2 iadd3	MAR = SP = SP – 1; rd H = TOS MDR = TOS = MDR + H; wr; goto (MBR1)	Legge la parola sotto la cima dello stack H = cima dello stack Somma le due parole in cima allo stack; scrive in cima allo stack
isub1 isub2 isub3	MAR = SP = SP – 1; rd H = TOS MDR = TOS = MDR – H; wr; goto (MBR1)	Legge la parola sotto la cima dello stack H = cima dello stack Sottrae TOS da TOS Prelevato-1
iand1 iand2 iand3	MAR = SP = SP – 1; rd H = TOS MDR = TOS = MDR AND H; wr; goto (MBR1)	Legge la parola sotto la cima dello stack H = cima dello stack AND tra TOS Prelevato-1 e TOS
ior1 ior2 ior3	MAR = SP = SP – 1; rd H = TOS MDR = TOS = MDR OR H; wr; goto (MBR1)	Legge la parola sotto la cima dello stack H = cima dello stack OR tra TOS Prelevato-1 e TOS
dup1 dup2	MAR = SP = SP + 1 MDR = TOS; wr; goto (MBR1)	Incrementa SP e lo copia in MAR Scrive la nuova parola sullo stack
pop1 pop2 pop3	MAR = SP = SP – 1; rd TOS = MDR; goto (MBR1)	Legge la parola sotto la cima dello stack Attende che si completi la lettura Copia la nuova parola in TOS
swap1 swap2 swap3 swap4 swap5 swap6	MAR = SP – 1; rd MAR = SP H = MDR; wr MDR = TOS MAR = SP – 1; wr TOS = H; goto (MBR1)	Legge la parola sotto la cima dello stack; imposta MAR a SP Prepara la scrittura della nuova seconda parola Salva il nuovo TOS; scrive la seconda parola nello stack Copia il vecchio TOS in MDR Scrive il vecchio TOS nella seconda posizione dello stack Aggiorna TOS
bipush1 bipush2	SP = MAR = SP + 1 MDR = TOS = MBR1; wr; goto (MBR1)	Imposta MAR per scrivere la nuova cima dello stack Aggiorna lo stack in TOS e nella memoria
iload1 iload2 iload3	MAR = LV + MBR1U; rd MAR = SP = SP + 1 TOS = MDR; wr; goto (MBR1)	Imposta MAR a LV + indice per la lettura SP punta alla nuova cima dello stack; prepara la scrittura Aggiorna lo stack in TOS e nella memoria
istore1 istore2 istore3 istore4 istore5	MAR = LV + MBR1U MDR = TOS; wr MAR = SP = SP – 1; rd TOS = MDR; goto (MBR1)	Imposta MAR a LV + indice Copia TOS per la scrittura in memoria Decrementa SP, legge il nuovo TOS Attende che si completi la lettura Aggiorna TOS
wide1	goto (MBR1 OR 0x100)	L'indirizzo successivo è il risultato dell'OR tra 0x100 e il codice operativo
wide_iload1 wide_istore1	MAR = LV + MBR2U; rd; goto iload2 MAR = LV + MBR2U; goto istore2	Identico a iload1, ma con un indice a 2 byte Identico a istore1, ma con un indice a 2 byte
ldc_w1	MAR = CPP + MBR2U; rd; goto iload2	Uguale a wide_iload1, ma indicizza a partire da CPP

Figura 4.30 Microprogramma per Mic-2 (continua).

iinc1	MAR = LV + MBR1U; rd	Imposta MAR a LV + indice per la lettura
iinc2	H = MBR1	Imposta H a una costante
iinc3	MDR = MDR + H; wr; goto (MBR1)	Incrementa di una costante e aggiorna
goto1	H = PC – 1	Copia PC in H
goto2	PC = H + MBR2	Aggiunge uno spiazzamento e aggiorna PC
goto3		Deve attendere che la IFU prelevi il nuovo codice operativo
goto4	goto (MBR1)	Salto alla nuova istruzione
iflt1	MAR = SP = SP – 1; rd	Legge la parola sotto la cima dello stack
iflt2	OPC = TOS	Salva temporaneamente TOS in OPC
iflt3	TOS = MDR	Inserisce in TOS la nuova cima dello stack
iflt4	N = OPC; if (N) goto T; else goto F	Diramazione in base al bit N
ifeq1	MAR = SP = SP – 1; rd	Legge la parola sotto la cima dello stack
ifeq2	OPC = TOS	Salva temporaneamente TOS in OPC
ifeq3	TOS = MDR	Inserisce in TOS la nuova cima dello stack
ifeq4	Z = OPC; if (Z) goto T; else goto F	Diramazione in base al bit Z
if_icmpeq1	MAR = SP = SP – 1; rd	Legge la parola sotto la cima dello stack
if_icmpeq2	MAR = SP = SP – 1	Imposta MAR per leggere la nuova cima dello stack
if_icmpeq3	H = MDR; rd	Copia in H la parola sotto la cima dello stack
if_icmpeq4	OPC = TOS	Salva temporaneamente TOS in OPC
if_icmpeq5	TOS = MDR	Inserisce in TOS la nuova cima dello stack
if_icmpeq6	Z = H – OPC; if (Z) goto T; else goto F	Se le due parole in cima allo stack sono uguali, goto T, altrimenti, goto F
T	H = PC – 1; goto goto2	Uguale a goto1
F	H = MBR2	Aggiorna i byte in MBR2 per scartarne il contenuto
F2	goto (MBR1)	
invokevirtual1	MAR = CPP + MBR2U; rd	Inserisce in MAR l'indirizzo del puntatore al metodo
invokevirtual2	OPC = PC	Salva il PC di ritorno in OPC
invokevirtual3	PC = MDR	Imposta PC al primo byte del codice del metodo
invokevirtual4	TOS = SP – MBR2U	TOS = indirizzo di OBJREF – 1
invokevirtual5	TOS = MAR = H = TOS + 1	TOS = indirizzo di OBJREF
invokevirtual6	MDR = SP + MBR2U + 1; wr	Sovrascrive OBJREF con il puntatore di collegamento
invokevirtual7	MAR = SP = MDR	Imposta SP e MAR alla locazione in cui memorizzare il vecchio PC
invokevirtual8	MDR = OPC; wr	Si prepara a salvare il vecchio PC
invokevirtual9	MAR = SP = SP + 1	Incrementa SP per puntare alla locazione in cui memorizzare il vecchio LV
invokevirtual10	MDR = LV; wr	Salva il vecchio LV
invokevirtual11	LV = TOS; goto (MBR1)	Imposta LV per puntare al parametro Ø
ireturn1	MAR = SP = LV; rd	Reimposta SP e MAR per leggere il puntatore di collegamento
ireturn2	LV = MAR = MDR; rd	Attende il puntatore di collegamento
ireturn3	MAR = LV + 1	Imposta LV e MAR al puntatore di collegamento; legge il vecchio PC
ireturn4	PC = MDR; rd	Imposta MAR per puntare al vecchio LV; legge il vecchio LV
ireturn5	MAR = SP	Ripristina PC
ireturn6	LV = MDR	Ripristina LV
ireturn7	MDR = TOS; wr; goto (MBR1)	Salva il valore di ritorno sulla cima originale dello stack
ireturn8		

Figura 4.30 Microprogramma per Mic-2.

4.4.4 Architettura a pipeline: Mic-3

Mic-2 rappresenta ovviamente un miglioramento rispetto a Mic-1: è più veloce e utilizza una minor quantità di memoria di controllo, anche se in termini di "proprietà terriera" il costo della IFU è senza dubbio maggiore del guadagno ottenuto riducendo la memoria di controllo. Mic-2 è dunque una macchina considerevolmente più veloce con un prezzo leggermente maggiore.

Che cosa si può dire riguardo al tentativo di diminuire il tempo del ciclo? In buona parte il tempo del ciclo è determinato dalla tecnologia impiegata. Più piccoli sono i transistor e minore è la distanza che li separa, maggiore è la velocità alla quale può funzionare il clock. Per una data tecnologia il tempo richiesto per eseguire un'operazione sull'intero percorso dati è fisso (almeno dal nostro punto di vista). Ciononostante esistono alcuni spazi di libertà che tra poco cercheremo di sfruttare pienamente.

L'alternativa che abbiamo è quella di introdurre nella macchina un maggior parallelismo. Ora come ora Mic-2 funziona in modo altamente sequenziale; copia i registri sul bus, aspetta che la ALU e lo shifter li processino e infine scrive i risultati nuovamente all'interno dei registri. Fatta eccezione per la IFU, in questo funzionamento vi è poco parallelismo; vale la pena provare ad aumentarlo.

Com'è già stato detto, il ciclo del clock è limitato dal tempo necessario affinché i segnali si possano propagare lungo il percorso dati. La Figura 4.3 mostra schematicamente quali sono, durante ciascun ciclo, i ritardi attraverso i vari componenti. Il ciclo del percorso dati è composto da tre componenti principali:

1. il tempo necessario per portare i registri selezionati sui bus A e B;
2. il tempo impiegato dalla ALU e dallo shifter per compiere il proprio lavoro;
3. il tempo necessario per riportare i risultati nei registri e memorizzarli al loro interno.

Nella Figura 4.31 si vede una nuova architettura a tre bus che, oltre alla IFU, presenta anche tre *latch* (registri) inseriti a metà di ciascun bus. Questi registri vengono scritti a ogni ciclo. In effetti i latch spezzano il percorso dati in parti distinte che possono funzionare indipendentemente l'una dall'altra. Per riferirci a questa architettura utilizzeremo il nome **Mic-3**, oppure modello **a pipeline**.

Come possono essere d'aiuto questi registri? Con la nuova architettura occorrono ora tre cicli di clock per utilizzare il percorso dati: uno per caricare i latch A e B, uno per far funzionare la ALU e lo shifter e per caricare il latch C, e uno per memorizzare il latch nei registri. Questo è sicuramente peggiore di quello che avevamo già. Siamo pazzi? Ovviamente no! Infatti il vantaggio dato dall'inserimento dei latch è duplice:

1. possiamo accelerare il clock dato che il ritardo massimo è ora più corto;
2. possiamo usare tutte le parti del percorso dati durante ogni ciclo.

Spezzando il percorso dati in tre parti il ritardo massimo si riduce, con il risultato che la frequenza del clock può essere maggiore. Supponendo di dividere il ciclo del percorso dati in tre intervalli di tempo, lunghi circa 1/3 di quello originario, possiamo triplicare la velocità del clock. La cosa non è proprio realistica, dato che abbiamo anche aggiunto due registri all'interno del percorso dati, ma come prima approssimazione può essere accettabile.

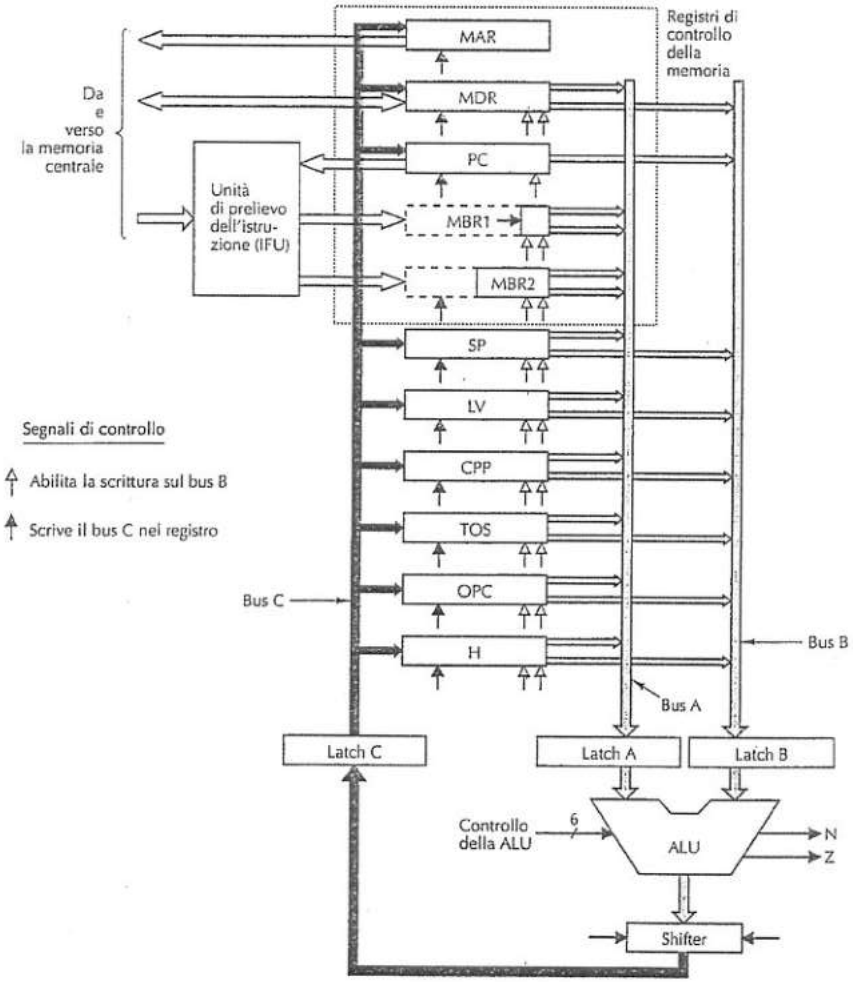


Figura 4.31 Percorso dati a tre bus utilizzato in Mic-3.

Dato che abbiamo assunto che letture e scritture di memoria possono essere soddisfatte dalla cache di primo livello e dato che questa cache è realizzata con lo stesso materiale dei registri, continueremo ad assumere che un'operazione di memoria richieda un solo ciclo (in pratica però ciò potrebbe non essere così semplice).

Il secondo punto riguarda il throughput piuttosto che la velocità delle singole istruzioni. In Mic-2 la ALU resta inattiva durante la prima e la terza parte di ogni ciclo di

clock, ma spezzando il percorso dati in tre parti saremo in grado di utilizzare la ALU in ogni ciclo, ottenendo così una quantità di lavoro tre volte maggiore.

Poniamo ora la nostra attenzione sul percorso dati di Mic-3. Prima di iniziare ci serve però una notazione per i latch. La scelta più ovvia è quella di chiamarli A, B e C e di trattarli come dei registri, tenendo ben presenti i vincoli del percorso dati. La Figura 4.32 mostra la sequenza di codice relativa all'implementazione di SWAP per Mic-2.

Etichetta	Operazioni	Commenti
swap1	MAR = SP – 1; rd	Legge la parola sotto la cima dello stack; imposta MAR a SP
swap2	MAR = SP	Si prepara a scrivere la nuova seconda parola
swap3	H = MDR; wr	Salva il nuovo TOS; scrive la seconda parola nello stack
swap4	MDR = TOS	Copia il vecchio TOS in MDR
swap5	MAR = SP – 1; wr	Scrive il vecchio TOS nella seconda posizione dello stack
swap6	TOS = H; goto (MBR1)	Aggiorna TOS

Figura 4.32 Codice di Mic-2 per SWAP.

Proviamo ora a implementare questa stessa sequenza sulla macchina Mic-3. Ricordiamoci che il percorso dati comprende ora tre cicli: uno per caricare A e B, uno per eseguire l'operazione e caricare C e uno per scrivere i risultati nei registri. Chiameremo passo **elementare** ciascuna di queste parti. L'implementazione di SWAP per Mic-3 è data nella Figura 4.33. Nel ciclo 1 iniziamo con swap1 copiando SP in B. Non importa quale valore viene memorizzato in A, dato che per sottrarre 1 da B bisogna negare ENA (Figura 4.2). Per semplicità non mostreremo gli assegnamenti che non sono utilizzati. Nel ciclo 2 effettuiamo la sottrazione. Nel ciclo 3 il risultato viene memorizzato in MAR e alla fine del ciclo inizia l'operazione di lettura (dopo che MAR è stato memorizzato). La lettura dalla memoria richiede un solo ciclo, e non sarà completata se non alla fine del ciclo 4; ciò è indicato dall'assegnamento a MDR durante il ciclo 4. Il valore in MDR non può essere letto prima del ciclo 5.

Tornando al ciclo 2, osserviamo che si può dapprima copiare SP in B per farlo passare all'interno della ALU nel ciclo 3 e infine memorizzarlo all'interno di MAR nel ciclo 4. Fin qui, nessun problema. Dovrebbe essere chiaro che se riusciamo a mantenere questo ritmo, iniziando una nuova microistruzione a ogni ciclo, potremo triplicare la velocità della macchina. Questo guadagno deriva dal fatto che possiamo lanciare a ogni ciclo una nuova microistruzione e che Mic-3 ha, rispetto a Mic-2, il triplo di cicli di clock per secondo. Quella che in realtà abbiamo creato è una CPU a pipeline.

Purtroppo nel ciclo 3 sorge un intoppo. Vorremmo cominciare a occuparci di swap3, ma la sua prima operazione consiste nel far passare MDR attraverso la ALU e MDR sarà disponibile dalla memoria solamente all'inizio del ciclo 5. La situazione in cui un passo elementare non può iniziare, dato che è in attesa di un risultato che non è ancora stato prodotto da un precedente passo elementare, viene chiamata **dipendenza RAW** (o **dipendenza effettiva**). Spesso ci si riferisce a queste situazioni con il termine *hazard*,

cioè "rischio". RAW è l'acronimo di *Read After Write* ("lettura dopo scrittura") e indica che un passo elementare vuole leggere un registro che non è ancora stato scritto. In questo caso l'unica cosa sensata da fare è ritardare l'inizio di swap3 finché MDR non sia disponibile, nel ciclo 5. Siamo cioè in una situazione di **stallo**. Una volta ottenuto il valore richiesto possiamo ricominciare a far partire una microistruzione per ciclo, dato che non ci sono più dipendenze. La microistruzione swap6 sfiora una dipendenza, in quanto legge H nel ciclo immediatamente successivo a quello in cui swap3 la scrive; se swap5 avesse provato a leggere H, sarebbe rimasta in stallo per un ciclo.

	Swap1	Swap2	Swap3	Swap4	Swap5	Swap6
C	MAR=SP–1; rd	MAR=SP	H=MDR; wr	MDR=TOS	MAR=SP–1; wr	TOS=H; goto (MBR1)
1	B=SP					
2	C=B–1	B=SP				
3	MAR=C; rd	C=B				
4	MDR=Mem	MAR=C				
5			B=MDR			
6			C=B	B=TOS		
7			H=C; wr	C=B	B=SP	
8			Mem=MDR	MDR=C	C=B–1	B=H
9					MAR=C; wr	C=B
10					Mem=MDR	TOS=C
11						goto (MBR1)

Figura 4.33 Implementazione di SWAP su Mic-3.

Il programma di Mic-3 richiede più cicli di quello di Mic-2, ma viene eseguito più velocemente. Se indichiamo con ΔT ns il tempo di ciclo di Mic-3, allora Mic-3 impiega $11\Delta T$ ns per eseguire SWAP. Mic-2 impiega invece 6 cicli, ciascuno dei quali è lungo $3\Delta T$, per un totale di $18\Delta T$. L'uso della pipeline ha reso la macchina più veloce, anche se è necessario rimanere in stallo per risolvere una dipendenza.

Dato che l'uso della pipeline è una tecnica chiave in tutte le CPU moderne è importante comprenderne a fondo il funzionamento. Nella Figura 4.34 vediamo il percorso dati della Figura 4.31 reso graficamente come una pipeline. La prima colonna rappresenta ciò che avviene durante il primo ciclo, la seconda colonna ciò che avviene nel secondo, e così via (assumendo che non si verifichino stalli). La regione in grigio dell'istruzione 1 durante il ciclo 1 indica che la IFU è occupata nel prelievo dell'istruzione 1. Al successivo scoccare del clock, durante il ciclo 2, i registri richiesti dall'istruzione 1 vengono caricati nei latch A e B, mentre la IFU è occupata nel prelievo dell'istruzione 2; anche in questo caso le operazioni sono indicate da rettangoli grigi.

Durante il ciclo 3 l'istruzione 1 utilizza la ALU e lo shifter per eseguire la propria operazione, mentre i latch A e B vengono caricati per l'istruzione 2 e, allo stesso tempo, l'istruzione 3 comincia a essere prelevata. Infine, durante il ciclo 4, ci sono quattro

istruzioni in corso d'esecuzione. I risultati dell'istruzione 1 cominciano a essere memorizzati, il lavoro della ALU per l'istruzione 2 volge al termine, i latch A e B vengono caricati per l'istruzione 3 e si comincia a prelevare l'istruzione 4.

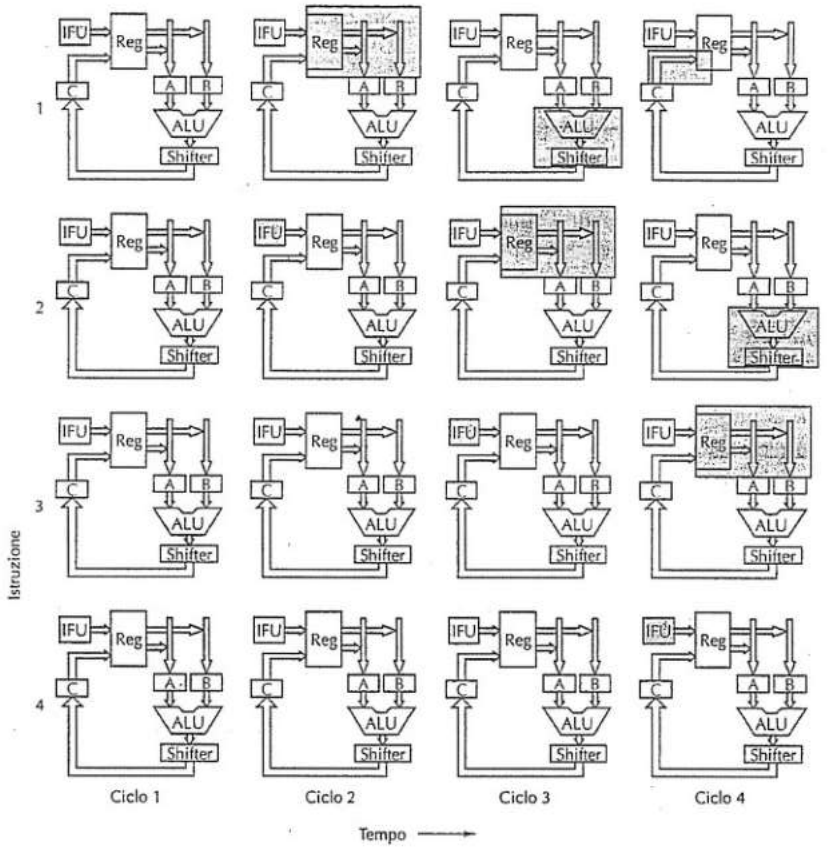


Figura 4.34 Rappresentazione grafica del funzionamento di una pipeline.

Se avessimo mostrato il ciclo 5 e quelli successivi la situazione sarebbe stata la stessa del ciclo 4: tutte e quattro le parti del percorso dati che possono funzionare indipendentemente continuano infatti a lavorare in parallelo. Questa organizzazione rappresenta una pipeline a 4 stadi, in cui uno preleva le istruzioni, uno accede agli operandi, uno esegue le operazioni della ALU e uno scrive i risultati nei registri. La pipeline è simile a quella della Figura 2.4(a), con la differenza che manca lo stadio di decodifica. Nell'uso della pipeline l'aspetto più importante da cogliere è che, anche se una singola istru-

zione richiede quattro cicli di clock per essere eseguita, durante ogni ciclo di clock può iniziare una nuova istruzione e mentre una, iniziata precedente, termina la propria esecuzione.

La Figura 4.34 può essere guardata in un altro modo, seguendo orizzontalmente ciascuna istruzione lungo tutta la larghezza della pagina. Per quanto riguarda l'istruzione 1, durante il ciclo 1 la IFU è in funzione. Nel ciclo 2 i suoi registri cominciano a essere portati sui bus A e B. Nel ciclo 3 la ALU e lo shifter svolgono le operazioni da lei richieste. Infine, nel ciclo 4, i suoi risultati sono memorizzati nuovamente all'interno dei registri. Occorre notare che sono disponibili quattro sezioni dell'hardware e durante ciascun ciclo una data istruzione ne utilizza uno soltanto, lasciando le restanti sezioni a disposizione delle altre istruzioni.

Un'analogia utile a capire come sono organizzate le pipeline è la linea di montaggio di una fabbrica di automobili. Per astrarre gli aspetti essenziali del modello immaginiamo che a ogni minuto venga colpito un grande gong e che in quel momento tutte le macchine si spostino, lungo la linea di montaggio, da un'isola a quella successiva. In ogni stazione i lavoratori eseguono una particolare operazione sull'automobile che hanno di fronte, per esempio aggiungere il volante o installare i freni. A ciascun battito del gong (1 ciclo) viene inserita una nuova automobile nella linea di montaggio e un'altra termina di essere assemblata. Quindi a ogni ciclo viene completata una macchina, anche se l'intero assemblaggio potrebbe constare di centinaia di cicli. La fabbrica può produrre un'automobile al minuto indipendentemente da quanto tempo è effettivamente necessario per assemblare un'automobile. Questa è la potenza della pipeline, e può essere sfruttata tanto per le CPU quanto per le fabbriche di automobili.

4.4.5 Pipeline a sette stadi: Mic-4

Un aspetto sul quale abbiamo intenzionalmente sorvolato riguarda il fatto che ciascuna microistruzione selezioni il proprio successore. La maggior parte di loro seleziona semplicemente quella che la segue nella sequenza corrente, mentre l'ultima, come `swap6`, spesso esegue una diramazione che blocca la pipeline, dato che è impossibile continuare a prelevare nuove microistruzioni. Occorre trovare un modo migliore per gestire questo punto.

La nostra successiva (e ultima) microarchitettura è Mic-4. Le sue parti principali sono mostrate nella Figura 4.35, nella quale sono stati trascurati molti dettagli per rendere più chiara l'illustrazione. Come per Mic-3, anche questa microarchitettura è dotata di una IFU che preleva dalla memoria le parole e mantiene i vari registri `MBR`.

La IFU alimenta inoltre con il flusso di byte una nuova componente, chiamata **unità di decodifica**. Questa unità è dotata di una ROM interna indicizzata attraverso il codice operativo IJVM. Ciascuna riga contiene due parti: la lunghezza dell'istruzione IJVM corrispondente e un indice relativo a un'altra ROM: la ROM delle micro-operazioni. La lunghezza dell'istruzione IJVM è utilizzata per permettere all'unità di decodifica di analizzare il flusso di byte entrante e suddividerlo in istruzioni, sapendo in ogni momento quali byte rappresentano i codici operativi e quali invece gli operandi. Se l'istruzione corrente è lunga 1 byte (per esempio, `POP`), allora l'unità di decodifica sa che il byte successivo è un codice operativo. Se invece l'istruzione corrente è lunga 2 byte, l'unità

di decodifica sa che il byte successivo è un operando, seguito da un altro codice operativo. Quando incontra il prefisso `WIDE`, l'unità trasforma il byte successivo in uno speciale codice operativo più ampio (per esempio `WIDE + ILOAD` diventa `WIDE_ILOAD`).

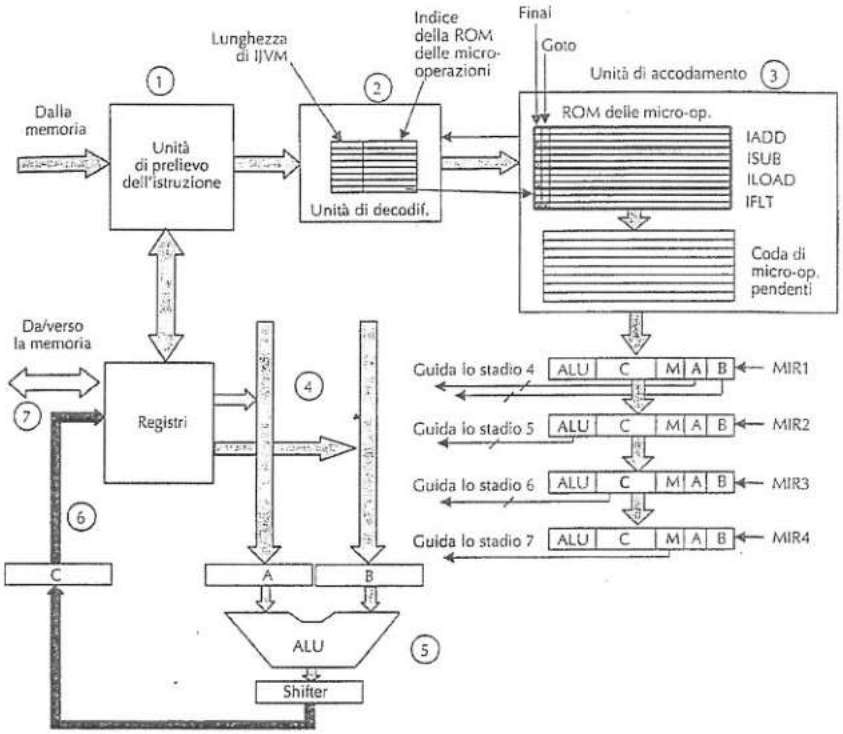


Figura 4.35 Componenti principali di Mic-4.

L'unità di decodifica invia alla componente successiva, chiamata **unità di accodamento**, l'indice relativo alla ROM delle micro-operazioni che ha trovato nella sua tabella. Oltre a vari circuiti, quest'unità contiene due tabelle interne, una in una RAM e l'altra in una ROM. Quest'ultima tabella contiene il microprogramma, in cui ciascuna istruzione IJVM ha un certo numero di elementi consecutivi, chiamati **micro-operazioni**. Queste devono essere in ordine e quindi non è possibile usare espedienti come quello che in Mic-2 permette a `wide_iload2` di effettuare un salto verso `iload2`. Ogni sequenza IJVM deve essere scandita interamente. Per tale motivo, alcune sequenze devono essere duplicate.

Le micro-operazioni sono simili alle microistruzioni della Figura 4.5, tranne per il fatto che sono prive dei campi `NEXT_ADDRESS` e `JAM`, ma hanno un nuovo campo per

specificare l'input del bus A. Sono anche presenti due nuovi bit: Final e Goto. Il bit Final viene impostato a 1 nell'ultima micro-operazione IJVM di ciascuna sequenza per indicarne la fine. Il bit Goto viene invece impostato a 1 per indicare le micro-operazioni che effettuano microdiramazioni condizionali; il loro formato è diverso da quello delle normali micro-operazioni ed è costituito dai bit JAM e da un indice relativo alla ROM delle micro-operazioni. Le microistruzioni, che precedentemente eseguivano delle operazioni con il percorso dati ed effettuavano allo stesso tempo una microdiramazione condizionale (per esempio, `iflt4`), devono ora essere separate in due micro-operazioni distinte.

L'unità di accodamento funziona nel modo seguente. Riceve dall'unità di decodifica un indice della ROM delle micro-operazioni, cerca la micro-operazione corrispondente e la copia in una coda interna. Successivamente copia nella coda anche le due micro-operazioni seguenti. L'unità continua in questo modo finché non incontra una micro-operazione in cui il bit Final vale 1; in questo caso la copia e si arresta. L'unità di accodamento, assumendo che non abbia incontrato una micro-operazione con il bit Goto attivo e che ci sia ancora molto spazio nella coda, rispedisce all'unità di decodifica un segnale di conferma. Quando vede la conferma, l'unità di decodifica spedisce all'unità di accodamento l'indice della successiva istruzione IJVM.

In questo modo la sequenza d'istruzioni IJVM in memoria viene convertita in una sequenza di micro-operazioni in una coda. Queste micro-operazioni alimentano i registri MIR, che spediscono i segnali che permettono di controllare il percorso dati. Esiste tuttavia un altro fattore che dobbiamo considerare: i campi delle micro-operazioni non sono attivi nello stesso momento. I campi A e B sono attivi durante il secondo ciclo, il campo C lo è durante il terzo ciclo e ogni operazione di memoria si svolge nel quarto ciclo.

Per permettere che ciò funzioni correttamente abbiamo introdotto nella Figura 4.35 quattro registri MIR indipendenti. All'inizio di ciascun ciclo di clock (il tempo Δw nella Figura 4.3) MIR3 viene copiato in MIR4, MIR2 in MIR3, MIR1 in MIR2 e MIR1 viene caricato con una nuova micro-operazione proveniente dalla coda delle micro-operazioni. Ogni MIR emette i propri segnali, ma non tutti vengono utilizzati. I campi A e B provenienti da MIR1 selezionano i registri che guidano i latch A e B, mentre il campo ALU in MIR1 non viene utilizzato e non è connesso a nessuna componente del percorso dati.

Dopo un ciclo di clock, questa micro-operazione si è spostata in MIR2 e i registri che ha selezionato sono stati salvati nei latch A e B, in attesa che l'avventura cominci. Il suo campo ALU viene ora utilizzato per guidare la ALU. Nel ciclo successivo il suo campo C riscriverà il risultato nei registri e poi si sposterà in MIR4 e darà inizio all'operazione di memoria richiesta utilizzando MAR (o MDR, nel caso di una scrittura), che sarà già stato caricato.

C'è un ultimo aspetto di Mic-4 che occorre approfondire: le microdiramazioni. Alcune istruzioni IJVM, come IFLT, richiedono di effettuare dei salti condizionati in base, per esempio, al bit N. Quando si verifica una microdiramazione la pipeline non può continuare. Per gestire questo caso abbiamo aggiunto alla micro-operazione il bit Goto. Quando incontra una micro-operazione che possiede questo bit attivo mentre la sta copiando all'interno della coda, l'unità di accodamento capisce in anticipo che ci sarà un problema e non spedisce all'unità di decodifica il segnale di conferma. Il risultato è

che la macchina rimarrà in stallo in questo punto finché la microdiramazione non sarà stata risolta.

Presumibilmente alcune istruzioni IJVM successive alla diramazione saranno già state portate nell'unità di decodifica (ma non nell'unità di accodamento) nel momento in cui si incontra una micro-operazione con il bit Goto attivo e non viene rispedito il segnale di conferma (cioè il permesso per continuare). Per riportare un po' d'ordine nella situazione e ritornare sui propri passi è necessario disporre di hardware speciale; la spiegazione di questi meccanismi esula però dagli scopi del libro. Edsger Dijkstra non ebbe idea di quanto avesse ragione quando scrisse il famoso: "GOTO Statement Considered Harmful" (Dijkstra, 1968a), cioè "L'istruzione GOTO è da considerarsi pericolosa".

Abbiamo percorso un lungo cammino a partire da Mic-1. Mic-1 è un componente hardware molto semplice, in cui quasi tutto il controllo è implementato in software. Mic-4 è organizzata altamente a pipeline, con sette stadi e un hardware molto più complesso. La pipeline è mostrata schematicamente nella Figura 4.36, in cui i numeri cerchiati fanno riferimento ai componenti della Figura 4.35. Mic-4 effettua automaticamente il prefetch di un flusso di byte dalla memoria, li decodifica trasformandoli in istruzioni IJVM che converte in una sequenza di micro-operazioni utilizzando una ROM; accoda poi queste microoperazioni per poterle utilizzare quando sarà necessario. Se lo si desidera è possibile legare i primi tre stadi della pipeline al clock del percorso dati, ma in realtà non c'è sempre del lavoro da eseguire. La IFU, per esempio, può certamente evitare di inviare durante ogni ciclo di clock un nuovo codice operativo IJVM all'unità di decodifica; le istruzioni IJVM richiedono infatti più cicli per essere eseguite e se si spedisse un codice operativo a ogni ciclo si riempirebbe rapidamente la coda.

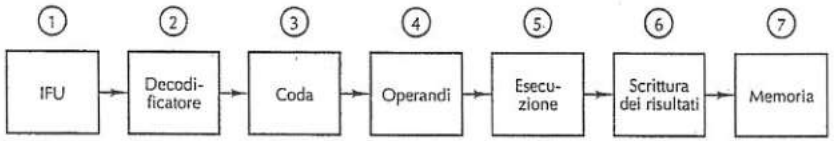


Figura 4.36 Pipeline di Mic-4.

A ogni ciclo di clock vengono traslati in avanti i registri MIR e la micro-operazione che si trova alla fine della coda viene copiata in MIR1 per iniziare la propria esecuzione. I segnali di controllo che partono dai quattro MIR si diffondono dunque lungo il percorso dati generando le azioni desiderate. Ciascun MIR controlla una parte diversa del percorso dati e quindi passi elementari differenti.

In questa architettura abbiamo una CPU organizzata profondamente a pipeline, con passi molto brevi e quindi una frequenza di clock molto elevata. Molte CPU sono progettate in questo modo, specialmente quelle che devono implementare un vecchio (CISC) insieme d'istruzioni. Per esempio, come vedremo più avanti nel corso di questo capitolo, l'implementazione del Core i7 è, per certi aspetti, concettualmente simile a Mic-4.

4.5 Miglioramento delle prestazioni

Tutti i produttori di calcolatori vorrebbero che le loro macchine funzionassero alla massima velocità possibile. In questo paragrafo presenteremo alcune tecniche avanzate sulle quali si sta focalizzando la ricerca attuale al fine di migliorare le prestazioni dei sistemi (principalmente quelle della CPU e della memoria). Per via della natura fortemente competitiva del mercato dei calcolatori l'intervallo di tempo che passa dal momento in cui la ricerca propone nuove idee per rendere più veloce un computer e la loro effettiva implementazione in prodotti commerciali è estremamente breve. Per questo motivo la maggior parte delle idee che tratteremo sono già in uso in un'ampia gamma di prodotti disponibili sul mercato.

È possibile dividere l'esposizione in due categorie principali: i miglioramenti dell'implementazione e quelli dell'architettura. I primi sono i modi di realizzare nuove CPU o nuove memorie che permettono al sistema di essere più veloce senza modificarne l'architettura. Ciò significa che i vecchi programmi potranno essere eseguiti anche sulla nuova macchina, aspetto che rappresenta un grande vantaggio per il successo commerciale del prodotto. Un modo, ma non l'unico, per migliorare l'implementazione è l'utilizzo di un clock più veloce. I guadagni di prestazioni ottenuti dall'80386, passando per l'80486 e il Pentium, fino ad arrivare a progetti più recenti come il Core i7, sono dovuti a una migliore implementazione, mentre l'architettura è rimasta essenzialmente la stessa per tutte le macchine.

Alcuni tipi di miglioramenti possono essere ottenuti solamente cambiando l'architettura. A volte questi cambiamenti sono incrementali e consistono per esempio nell'aggiunta di nuove istruzioni o registri, in modo che i vecchi programmi possano continuare a funzionare anche sui nuovi modelli. In questo caso, per trarre pieno vantaggio dalle prestazioni della macchina, è necessario modificare il software o almeno ricompilarlo con un nuovo compilatore che tenga conto delle nuove funzionalità.

A volte i progettisti si rendono tuttavia conto che la vecchia architettura, essendo sopravvissuta fin troppo a lungo, ha esaurito la propria utilità e che l'unico modo per ottenere dei progressi è quello di ricominciare da capo progettandone una nuova. La rivoluzione RISC negli anni '80 fu uno di questi punti di rottura; un altro sta per arrivare, e nel corso del Capitolo 5 ne vedremo un esempio (l'Intel IA-64).

Nel resto del paragrafo analizzeremo quattro diverse tecniche per migliorare le prestazioni della CPU. Inizieremo presentando tre miglioramenti la cui implementazione si è ormai affermata nelle macchine attuali e successivamente passeremo a un miglioramento che, per funzionare al massimo, necessita di un piccolo supporto da parte dell'architettura. Queste tecniche sono la memoria cache, la predizione dei salti, l'esecuzione fuori sequenza con la rinomina dei registri e l'esecuzione speculativa.

4.5.1 Memoria cache

Nella storia della progettazione dei calcolatori una delle sfide più ardue è stata quella di definire un sistema di memoria capace di fornire al processore gli operandi alla velocità alla quale esso li può elaborare. Il recente ed elevato tasso di crescita della velocità dei processori non è stato accompagnato da un analogo incremento della velocità delle memorie. Negli ultimi decenni le memorie sono diventate più lente rispetto alle CPU. Per via dell'enorme importanza che riveste la memoria primaria questa situazione ha frenato in modo significativo lo sviluppo di sistemi ad alte prestazioni, e allo stesso tempo ha stimolato la ricerca di nuovi modi per aggirare il problema della velocità della memoria, che diventa di anno in anno più lenta rispetto alla CPU.

I processori moderni effettuano una quantità travolgente di richieste al sistema di memoria, sia in termini di latenza (il ritardo nel fornire un operando) sia in termini di banda (la quantità di dati fornita per unità di tempo). Questi due aspetti che caratterizzano un sistema di memoria sono purtroppo in larga parte in conflitto. Molte tecniche che permettono di aumentare la larghezza di banda incrementano allo stesso tempo la latenza. Per esempio, la pipeline utilizzata in Mic-3 può essere applicata a un sistema di memoria in modo da gestire efficientemente la sovrapposizione di più richieste alla memoria. Purtroppo, come per Mic-3, si ottiene una latenza maggiore per le singole operazioni di memoria. Dato che la velocità di clock del processore continua ad aumentare, diventa sempre più difficile realizzare un sistema di memoria in grado di fornire gli operandi in uno o due cicli di clock.

Un modo per combattere questo problema è l'uso delle cache. Come abbiamo visto nel Paragrafo 2.2.5 le cache mantengono all'interno di una piccola e veloce memoria le parole di memoria utilizzate più di recente, in modo da accelerarne l'accesso. Se nella cache è presente una percentuale sufficientemente elevata delle parole di memoria necessarie, è possibile ridurre enormemente l'effettiva latenza della memoria.

Una delle tecniche più efficaci per migliorare sia la larghezza di banda sia la latenza è l'uso di più cache. Una tecnica elementare che funziona in modo efficace consiste nell'introdurre due cache separate, una per le istruzioni e una per i dati. Il sistema con due cache distinte per i dati e per le istruzioni, spesso chiamato **a cache separata**, fornisce vari vantaggi. In primo luogo è possibile far partire le operazioni di memoria indipendentemente per ciascuna cache, raddoppiando effettivamente la larghezza di banda del sistema di memoria. Questo è il motivo per cui ha senso fornire due porte distinte per la memoria, come abbiamo fatto in Mic-1: ogni porta ha la propria cache. Occorre notare che le due cache hanno accessi indipendenti alla memoria centrale.

Attualmente molti sistemi di memoria sono ancora più complicati. Spesso tra la memoria centrale e le cache delle istruzioni e dei dati è presente un'altra cache, chiamata **cache di secondo livello**. In realtà, per disporre di sistemi di memoria ancora più sofisticati, si possono avere anche tre o più livelli di cache. Il chip stesso della CPU contiene una piccola cache per le istruzioni e una piccola cache per i dati, le cui dimensioni sono in genere comprese tra 16 e 64 KB. Dopo questa piccola memoria c'è una cache di secondo livello, che non si trova nel chip della CPU, ma che può essere inclusa all'interno del suo involucro, vicina al chip e a esso collegata tramite circuiti ad alta velocità. Questa cache è di solito unificata e contiene quindi sia dati sia istruzioni. Generalmente la dimensione delle cache L2 è compresa tra 512 KB e 1 MB. La cache di terzo livello si trova sulla scheda del processore ed è costituita da alcuni megabyte di SRAM, un tipo di memoria molto più veloce rispetto alla memoria DRAM centrale. Di solito le cache sono annidate, nel senso che l'intero contenuto della cache di primo livello è compreso nella cache di secondo livello e tutto il contenuto di quest'ultima è compreso in quella di terzo livello.

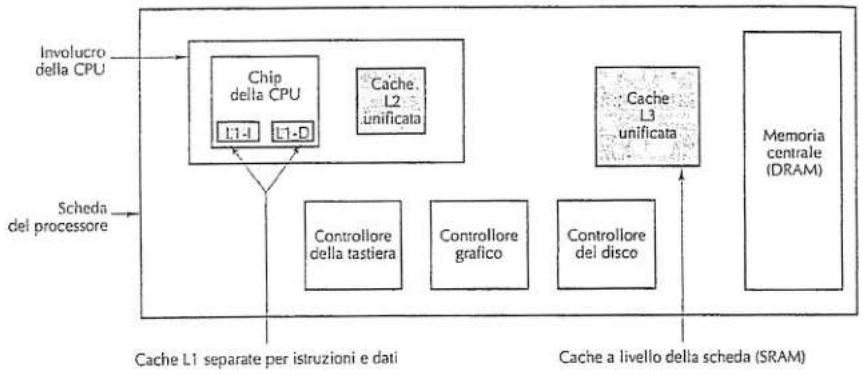


Figura 4.37 Sistema con tre livelli di cache.

Per raggiungere il proprio scopo le cache sfruttano due tipi di località degli indirizzi. La **località spaziale** riflette la considerazione che è molto probabile che in un prossimo futuro si accederà a locazioni di memoria i cui indirizzi sono numericamente simili a quello di una locazione appena utilizzata. La **località temporale** si verifica quando si accede nuovamente a locazioni di memoria già utilizzate di recente. Ciò potrebbe avvenire per esempio nel caso di locazioni di memoria vicine alla cima dello stack oppure nel caso delle istruzioni di un ciclo. Nella progettazione di una cache la località temporale viene principalmente sfruttata scegliendo che cosa scartare nel caso in cui si verifichi un fallimento della cache. Molti algoritmi di sostituzione sfruttano la località temporale scartando gli elementi che non sono stati utilizzati di recente.

Tutte le cache utilizzano lo stesso modello. La memoria centrale è divisa in blocchi di dimensione fissa chiamati **linee di cache**. Una linea di cache è composta generalmente da 4 a 64 byte consecutivi. Le linee sono numerate consecutivamente a partire da 0: se la dimensione della linea è di 32 byte, la linea 0 conterrà i byte compresi tra 0 e 31, la linea 1 quelli compresi tra 32 e 63, e così via. In ogni momento la cache contiene delle linee. Quando si effettua un riferimento alla memoria il circuito che controlla la cache verifica se la parola richiesta si trova nella cache. Se così è, la parola può essere utilizzata, risparmiando un viaggio fino alla memoria centrale. In caso contrario, viene rimossa una linea dalla cache per sostituirla con la linea richiesta, prelevata dalla memoria centrale oppure da un livello di cache più distante. Esistono molte varianti di questo schema, ma in tutte l'idea base è di tenere il più a lungo possibile nella cache le linee usate più frequentemente, in modo da massimizzare il numero di riferimenti alla memoria che la cache riesce a soddisfare.

Cache a corrispondenza diretta

La cache più semplice è conosciuta con il nome di **cache a corrispondenza diretta** (*direct-mapped cache*). La Figura 4.38(a) mostra un esempio di cache di questo tipo che contiene 2048 elementi. Ciascun elemento (riga) della cache può memorizzare esattamente una linea di cache della memoria centrale. Se ipotizziamo che la linea di cache abbia dimensione di 32 byte, la cache può memorizzare 2048 elementi di 32 byte, per un totale di 64 KB. Gli elementi della cache sono composti da tre parti.

1. Il bit `Valid` che indica se il dato nell'elemento è valido oppure no. All'avvio del sistema tutti gli elementi della cache sono marcati come non validi.
2. Il campo `Tag` che è un valore univoco a 16 bit, corrispondente alla linea di memoria da cui provengono i dati.
3. Il campo `Data` che contiene una copia del dato della memoria. Questo campo memorizza una linea di cache di 32 byte.

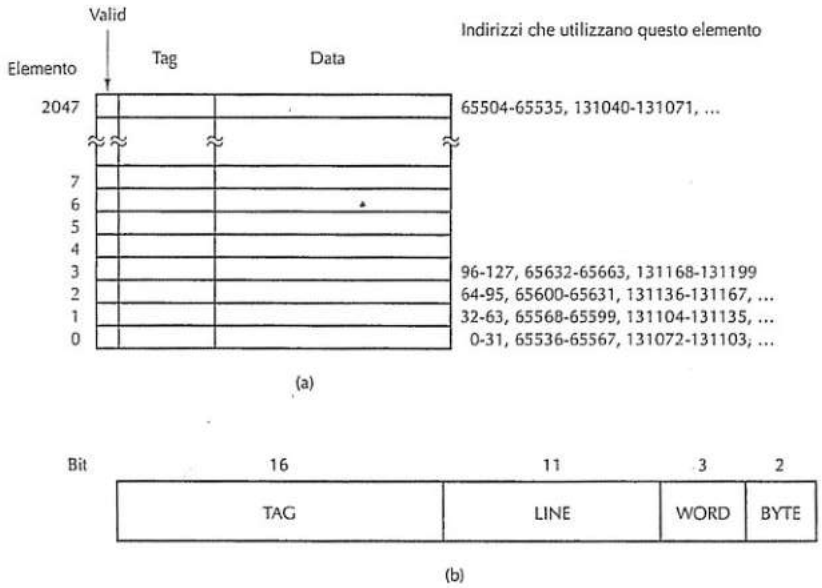


Figura 4.38 (a) Cache a corrispondenza diretta. (b) Indirizzo virtuale a 32 bit.

Nelle cache a corrispondenza diretta una data parola di memoria può essere memorizzata in un'unica posizione della cache. Per ogni indirizzo di memoria esiste un solo posto all'interno della cache in cui cercare il dato. Per memorizzare e prelevare dati dalla cache l'indirizzo è diviso in quattro campi, come mostra la Figura 4.38(b):

1. il campo `TAG` corrisponde ai bit `Tag` memorizzati in un elemento della cache;
2. il campo `LINE` indica l'elemento della cache contenente i dati corrispondenti, se sono presenti;

3. il campo WORD indica a quale parola si fa riferimento all'interno della linea;
4. il campo BYTE non viene generalmente utilizzato, ma se si richiede un solo byte esso indica quale byte è richiesto all'interno della parola. Per una cache che fornisce soltanto parole a 32 bit, questo campo varrà sempre 0.

Quando la CPU genera un indirizzo di memoria, l'hardware estrae dall'indirizzo gli 11 bit del campo LINE e li usa come indice all'interno della cache per cercare uno dei 2048 elementi. Se questo elemento è valido si effettua un confronto tra il campo TAG dell'indirizzo di memoria e il campo Tag dell'elemento della cache. Se sono uguali, l'elemento della cache contiene la parola richiesta; questa situazione è chiamata **cache *hit*** ("successo della cache"). In tal caso la parola che si vuole leggere può essere presa direttamente dalla cache, evitando di dover andare fino alla memoria. Dalla linea di cache viene estratta solamente la parola effettivamente richiesta, mentre non viene utilizzato il resto della linea. Se l'elemento della cache non è valido oppure se i due campi "tag" non corrispondono, il dato richiesto non è presente nella cache: questa situazione è chiamata **cache *miss*** ("fallimento della cache"). In questo caso la linea della cache a 32 byte viene prelevata dalla memoria e memorizzata nell'elemento appropriato della cache, sostituendo quello che era presente precedentemente. In ogni caso se l'elemento della cache era stato modificato dopo averlo caricato, occorre riscriverlo in memoria prima di poterlo sovrascrivere.

Nonostante la complessità della decisione, l'accesso alla parola richiesta può essere notevolmente veloce. Nel momento in cui si conosce l'indirizzo, si conosce anche l'esatta locazione della parola *sempre che si trovi nella cache*. Ciò significa che è possibile leggere la parola e spedirla al processore nello stesso momento in cui esso sta controllando se la parola è quella corretta (confrontando i due "tag"). Il processore riceve quindi dalla cache una parola nello stesso momento (se non addirittura in anticipo) in cui viene a sapere se la parola è effettivamente quella richiesta.

Questo schema di corrispondenza inserisce linee di memoria consecutive in elementi della cache consecutivi. Nella cache è possibile memorizzare effettivamente fino a 64 KB di dati contigui. Tuttavia all'interno della cache non è possibile memorizzare allo stesso tempo due linee i cui indirizzi differiscono esattamente di 64 KB (65.536 byte) o di un multiplo di questo valore (dato che il loro campo LINE è uguale). Per esempio, se un programma accede a dati che si trovano nella locazione X e successivamente esegue un'istruzione che richiede dati presenti alla locazione $X + 65.536$ (o in qualsiasi altra locazione della stessa linea), la seconda istruzione obbligherà a ricaricare l'elemento della cache, sovrascrivendo quello precedente. Se ciò avviene con una certa frequenza, le prestazioni degradano. Il comportamento di una cache può essere addirittura peggiore di quanto non sarebbe in assenza di cache, dato che ogni operazione di memoria comporta la lettura di un'intera linea di cache e non semplicemente di una parola.

Le cache a corrispondenza diretta sono le più comuni e funzionano in modo efficiente, dato che collisioni come quelle descritte precedentemente si verificano molto raramente, se non addirittura mai. Un compilatore molto astuto, nel posizionare i dati e le istruzioni all'interno della memoria, potrebbe per esempio tener conto delle possibili collisioni della cache, cercando di evitarle. Si noti che il caso particolare descritto precedentemente non si potrebbe verificare in un sistema con cache separate per le istruzioni e per i dati, in quanto le richieste che erano in collisione verrebbero in questo caso gestite da cache differenti. Utilizzare due cache al posto di una sola porta un secondo vantaggio: una maggiore flessibilità nella gestione d'indirizzi di memoria in conflitto.

Cache set-associative

Com'è stato detto precedentemente, molte linee della memoria sono in competizione per l'utilizzo di uno stesso elemento della cache. Se un programma che utilizza la cache della Figura 4.38(a) usa intensamente le parole comprese tra l'indirizzo 0 e l'indirizzo 65.536, si verificheranno costantemente dei conflitti e ogni riferimento alla memoria potrebbe potenzialmente espellere un elemento dalla cache. Una soluzione a questo problema è quello di consentire che in ciascun elemento della cache ci possano essere due o più linee. Una cache con n possibili elementi per ciascun indirizzo è chiamata **cache set-associativa a n vie**. La Figura 4.39 mostra una cache associativa a quattro vie.

Queste cache sono più complesse di quelle a corrispondenza diretta. Infatti, anche se l'insieme di elementi della cache da esaminare viene calcolato a partire dall'indirizzo di memoria, è tuttavia necessario controllare un insieme di n elementi per vedere se la linea richiesta è presente nella cache. Questa verifica deve essere effettuata molto rapidamente. L'esperienza e le simulazioni hanno mostrato tuttavia che cache a due o a quattro vie garantiscono prestazioni sufficientemente buone, da rendere accettabile l'aumento dei componenti digitali richiesti.

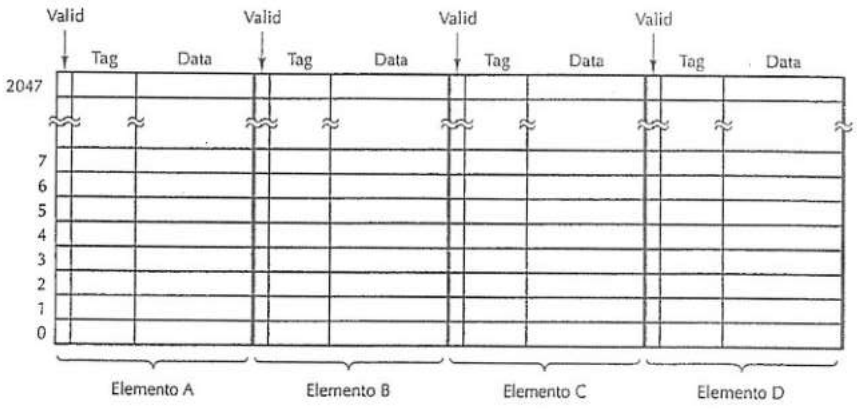


Figura 4.39 Cache associativa a quattro vie.

L'uso di una cache set-associativa pone il progettista di fronte a una scelta; quando viene portata all'interno della cache una nuova linea, quale delle presenti deve essere scartata? Per poter compiere la scelta migliore bisognerebbe poter predire il futuro; un algoritmo che nella maggior parte delle situazioni funziona abbastanza bene è detto **LRU** (acronimo di *Least Recently Used*, "meno recentemente usata"). Questo algoritmo mantiene un

ordinamento temporale di ciascun insieme di elementi ai quali si può far riferimento in base a una certa locazione di memoria. Quando si accede a una delle linee presenti nella cache l'algoritmo aggiorna la lista marcando l'elemento utilizzato più di recente. Quando giunge il momento di sostituire un elemento, l'algoritmo scarta quello che si trova alla fine della lista, cioè quello al quale è stato effettuato un accesso meno di recente.

Portando l'idea all'estremo è possibile immaginare una cache a 2048 vie contenente un unico insieme di 2048 linee. In questo caso tutti gli indirizzi di memoria vengono mappati in un unico insieme e quindi la ricerca all'interno della cache obbliga a confrontare l'indirizzo con 2048 diversi tag. Occorre notare che ciascun elemento deve avere dei componenti logici che permettano di controllare la corrispondenza con il tag. Dato che la lunghezza del campo `LINE` è pari a 0, il campo `TAG` costituisce l'intero indirizzo, fatta eccezione per i campi `WORD` e `BYTE`. Inoltre quando viene rimpiazzata una linea della cache, tutte e 2048 le locazioni sono candidate alla sostituzione. Per mantenere e gestire una lista di 2048 elementi è necessario un lavoro talmente grande da rendere inapplicabile l'algoritmo LRU. (Ricordiamoci che questa lista deve essere aggiornata per ogni operazione di memoria, non soltanto quando si verifica un fallimento della cache.) Nella maggior parte delle circostanze le cache set-associative a molte vie non hanno prestazioni sensibilmente migliori di quelle con un piccolo numero di vie. Il risultato sorprendente è che in alcuni casi le prestazioni sono addirittura peggiori. Per questi motivi è raro incontrare cache con più di quattro vie.

Infine, anche le operazioni di scrittura pongono un particolare problema nell'uso delle cache. Quando un processore scrive una parola, e questa parola è presente nella cache, ovviamente esso deve aggiornare la parola oppure scaricare l'elemento dalla cache. In quasi tutte le progettazioni si sceglie di aggiornare la cache. Ma che cosa si può dire riguardo l'aggiornamento della copia nella memoria centrale? Questa operazione può essere rinviata finché la linea della cache non sia pronta a essere sostituita dall'algoritmo LRU. La scelta in questo caso è difficile e nessuna opzione è nettamente migliore delle altre. L'aggiornamento immediato nella memoria centrale dell'elemento della cache è chiamato **write-through** ("scrivi attraverso"). Questo approccio è generalmente il più semplice e il più affidabile, dato che il contenuto della memoria rimane sempre aggiornato; ciò è utile, per esempio, se si verifica un errore ed è necessario recuperare lo stato della memoria. Purtroppo però questa tecnica richiede molto traffico verso la memoria; per questo motivo si tende a utilizzare un'implementazione più sofisticata, conosciuta con il nome di **write-deferred** ("scrittura ritardata") o **write-back**.

Sempre al riguardo delle operazioni di scrittura occorre affrontare un ulteriore problema: che cosa fare se si verifica una scrittura in una locazione che in quel momento non è memorizzata nella cache? Bisogna portare il dato nella cache o basta scriverlo in memoria? Anche in questo caso nessuna delle due risposte è migliore dell'altra. La maggior parte dei progetti che ritardano la scrittura in memoria tende a portare i dati nella cache quando si verifica un fallimento in scrittura; questa tecnica è conosciuta con il nome di **write allocation** ("allocazione in scrittura"). Al contrario, quando si impiega la scrittura diretta, si tende generalmente a non allocare un elemento della cache a ogni operazione di scrittura, dato che ciò complicherebbe un'organizzazione altrimenti semplice. L'allocazione in scrittura risulta vincente solo nel caso di ripetute scritture della stessa parola o di diverse parole facenti però parte di una stessa linea della cache.

Le prestazioni della cache influenzano largamente le prestazioni generali del sistema, dato che la differenza tra la velocità della CPU e quella della memoria è molto grande. Per questa ragione la ricerca di migliori strategie per utilizzare le cache è un campo di studio ancora molto attivo (Sanchez e Kozyrakis, 2011; Gaur et al., 2011).

4.5.2 Predizione dei salti

I calcolatori moderni sono profondamente organizzati a pipeline. La pipeline della Figura 4.36 ha sette stadi, mentre quelle dei calcolatori ad alte prestazioni ne hanno spesso 10 o più. L'uso delle pipeline funziona in modo ottimale su codice lineare, dato che l'unità di prelievo può semplicemente leggere dalla memoria parole consecutive e spedirle all'unità di decodifica prima ancora che siano effettivamente richieste.

L'unico, piccolo, problema di questo magnifico modello è che non è, neanche in piccola parte, realistico. I programmi non sono sequenze lineari di codice, ma sono pieni d'istruzioni di salto. Consideriamo le semplici istruzioni della Figura 4.40(a). La variabile *i* viene confrontata con 0 (probabilmente il test più comune) e, a seconda del risultato, si assegna un valore diverso alla variabile *k*.

La Figura 4.40(b) mostra una traduzione in linguaggio assemblativo che verrà esaminato più avanti nel corso del libro; per ora i dettagli non sono importanti, in ogni caso a seconda della macchina e del compilatore con ogni probabilità il codice assomiglierà più o meno a quello mostrato nella Figura 4.40(b). La prima istruzione confronta *i* con 0. La seconda effettua una diramazione verso l'etichetta *Else* (l'inizio della clausola `else`) nel caso in cui *i* sia diverso da 0. La terza istruzione assegna a *k* il valore 1. La quarta effettua una diramazione verso il codice dell'istruzione successiva. Il compilatore ha opportunamente inserito in quel punto un'etichetta, *Next*, in modo che ci sia una destinazione verso la quale effettuare il salto. La quinta istruzione assegna a *k* il valore 2.

```
if (i == 0)
    k = 1;
else
    k = 2;
```

(a)

```
        CMP i,0     ; confronta i con 0
        BNE Else    ; salta a Else se diversi
Then:   MOV k,1     ; sposta 1 in k
        BR  Next    ; salta a Next
Else:   MOV k,2     ; sposta 2 in k
Next:
```

(b)

Figura 4.40 (a) Frammento di programma. (b) Il frammento tradotto in un linguaggio assemblativo.

In questo esempio l'aspetto che merita una particolare osservazione è che due delle cinque istruzioni sono salti. Inoltre una di queste, `BNE`, è un salto condizionale, ovvero un salto effettuato se e soltanto se è soddisfatta una particolare condizione: in questo caso, il fatto che i due operandi della precedente istruzione, `CMP`, siano diversi. La più lunga sequenza lineare di codice è composta da due sole istruzioni. Per questi motivi

risulta particolarmente difficile prelevare le istruzioni a una velocità elevata in modo da alimentare la pipeline.

A prima vista potrebbe sembrare che i salti incondizionati, come l'istruzione `BR Next` della Figura 4.40(b), non costituiscano un problema. Dopo tutto non c'è alcuna ambiguità sulla destinazione da raggiungere. Perché l'unità di prelievo non può semplicemente continuare a leggere le istruzioni a partire dall'indirizzo destinazione (il punto verso il quale viene effettuato il salto)?

Il problema risiede nella natura stessa della pipeline. Nella Figura 4.36 possiamo vedere per esempio che la decodifica dell'istruzione si svolge nel secondo stadio. L'unità di prelievo deve quindi decidere dove prelevare le istruzioni successive ancor prima di conoscerne il tipo. Può scoprire di aver prelevato un salto incondizionato soltanto dopo un ciclo, nel momento in cui ha già cominciato a prelevare l'istruzione che segue il salto incondizionato. La conseguenza è che un buon numero di macchine organizzate a pipeline (come l'UltraSPARC III) ha la proprietà di eseguire l'istruzione *che segue* un salto incondizionato, anche se logicamente non dovrebbe farlo. La posizione immediatamente successiva a un salto è chiamata **posizione di ritardo**. Il Core i7 (e la macchina usata nella Figura 4.40(b)) non hanno questa proprietà, ma la complessità interna che permette di evitare questo problema è enorme. Un compilatore ottimizzato cercherà di inserire nelle posizioni di ritardo alcune istruzioni utili; spesso però non ci sono istruzioni disponibili e in tal caso il compilatore è obbligato a inserire in quel punto un'istruzione `NOP`. L'aggiunta di queste istruzioni permette di mantenere corretto il programma, rendendolo però più lento e di dimensioni maggiori.

Se i salti incondizionati provocano delle noie, quelli condizionali ne provocano di più. Non solo occorre considerare anche in questo caso delle posizioni di ritardo, ma oltretutto l'unità di prelievo viene a conoscenza dell'indirizzo in cui deve leggere in un momento della pipeline ancora più ritardato. Le prime macchine a pipeline rimanevano semplicemente in **stallo** finché non venivano a sapere se occorresse o meno effettuare la diramazione. Nel caso in cui i salti condizionali costituiscano il 20% delle istruzioni il fatto di rimanere in stallo per tre o quattro cicli a ogni salto condizionale è letale per le prestazioni.

Per queste ragioni quello che fa la maggior parte delle macchine quando incontra un salto condizionale è predire se la diramazione verrà o meno effettuata. Sarebbe bello se si potesse collegare una sfera di cristallo a uno slot PCIe libero (o meglio, nell'IFU) in modo da aiutare la previsione; finora però nessuno è riuscito a implementare questo approccio.

In mancanza di una simile periferica sono stati escogitati vari metodi per effettuare la predizione. Un metodo molto semplice è il seguente: si assume che debbano essere effettuati tutti i salti condizionali all'indietro, e non tutti quelli in avanti. Il ragionamento alla base della prima parte di questa strategia è che i salti all'indietro sono spesso collocati alla fine di un ciclo. La maggior parte dei cicli viene eseguita più volte e quindi, in genere, ha senso scommettere che occorra compiere un salto all'indietro per tornare all'inizio del ciclo.

La seconda parte del ragionamento si fonda su una base meno solida. Alcuni salti in avanti si verificano quando si rilevano nel software delle condizioni di errore (per esempio, non è possibile aprire un file). Gli errori sono rari, quindi la maggior parte dei salti loro associata non dovrebbe essere effettuata. Ovviamente esiste un gran numero di salti che non sono legati alla gestione degli errori e la frequenza di successi non è altrettanto buona quanto quella dei salti all'indietro. Questa regola, pur non essendo l'ideale, comunque è... meglio di niente.

Se si predice correttamente una diramazione non occorre compiere alcuna operazione particolare. L'esecuzione continua semplicemente a partire dall'indirizzo di destinazione. Il problema sorge quando la predizione si rivela errata. Determinare dove andare, e andarci, non è difficile. La parte complicata è annullare le istruzioni già eseguite che non avrebbero dovuto esserlo.

Esistono due modi di affrontare questa situazione. Il primo consiste nel consentire, dopo aver predetto un salto, l'esecuzione delle istruzioni solo finché queste non si apprestino a modificare lo stato della macchina (per esempio memorizzando un valore in un registro). Invece di sovrascrivere il registro, il valore calcolato viene inserito in un registro temporaneo (nascosto) e copiato in quello reale solo dopo aver verificato che la predizione era corretta. Il secondo metodo consiste nel registrare (per esempio in un registro temporaneo nascosto) il valore di ogni registro in procinto di essere sovrascritto, in modo che la macchina possa essere riportata allo stato in cui era nel momento in cui è stata effettuata la predizione errata. Entrambe le soluzioni sono complesse e per funzionare correttamente devono tener traccia di una quantità industriale d'informazioni. La situazione può diventare veramente complessa se si incontra un secondo salto condizionale ancor prima di scoprire se il primo era stato predetto correttamente o meno.

Predizione dinamica dei salti

Il fatto che le predizioni siano accurate è ovviamente molto importante, dato che in tal caso la CPU ha la possibilità di procedere alla massima velocità. Per questo motivo gran parte della ricerca attuale punta a migliorare gli algoritmi di predizione dei salti (Chen et al., 2003; Falcon et al., 2004; Jimenez, 2003; Parikh et al., 2004). Uno dei possibili approcci prevede che la CPU abbia (in appositi componenti hardware) una tabella della storia dei salti, nella quale tiene traccia dei salti condizionali incontrati in modo da poterli rianalizzare quando si verificano nuovamente. La versione più semplice di questo approccio è mostrata nella Figura 4.41(a). In questo caso la tabella della storia dei salti contiene un elemento per ogni istruzione di salto condizionale; l'elemento contiene l'indirizzo dell'istruzione di salto oltre a un bit che indica se è stata effettuata la diramazione l'ultima volta che l'istruzione è stata eseguita. Usando questo schema la predizione consiste semplicemente nell'assumere che il salto si comporterà allo stesso modo dell'occorrenza precedente. Se la predizione si rivela sbagliata, allora si modifica il bit nella tabella della storia dei salti.

Esistono vari modi per organizzare questa tabella, e corrispondono in sostanza alle possibili organizzazioni delle cache. Consideriamo una macchina con istruzioni a 32 bit, allineate in corrispondenza delle parole, in modo che i 2 bit meno significativi degli indirizzi di memoria siano sempre 00. Con una tabella della storia a corrispondenza diretta che contiene 2^n elementi è possibile estrarre gli $n + 2$ bit meno significativi dell'indirizzo destinazione di un'istruzione di salto e traslarli a destra di 2 bit; questo

numero a n bit può essere utilizzato come indice della tabella della storia. L'indice così ottenuto permette di controllare se l'indirizzo memorizzato corrisponde a quello del salto. Come per le cache, anche in questo caso non è necessario memorizzare gli $n + 2$ bit meno significativi, che possono quindi essere omessi (e vengono memorizzati solamente i bit più significativi, cioè il tag). Se l'elemento cercato è presente nella tabella, la predizione utilizza il bit come previsione del futuro comportamento del salto. Se invece è presente un tag errato oppure l'elemento della tabella non è valido, si verifica un fallimento, allo stesso modo in cui avviene nelle cache. In questo caso è possibile utilizzare la regola dei salti in avanti/all'indietro.

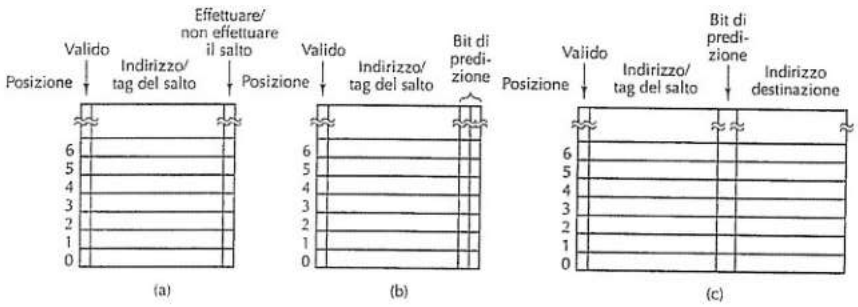


Figura 4.41 (a) Storia dei salti a 1 bit. (b) Storia dei salti a 2 bit. (c) Corrispondenza tra gli indirizzi delle istruzioni di salto e gli indirizzi destinazione.

Se la tabella della storia ha, per esempio, 4096 elementi, i salti che si trovano agli indirizzi 0, 16384, 32768, ... saranno in conflitto, in modo analogo al problema già visto con le cache. È possibile adottare la stessa soluzione: un elemento associativo a 2, a 4 o a n-vie. Come per la cache il fattore limitante è costituito da un singolo insieme a n-vie, in cui occorre confrontare un indirizzo con tutti i tag.

Se la larghezza della tabella è abbastanza grande e se vi è sufficiente associatività, allora questo schema funziona bene nella maggior parte delle situazioni. Ciononostante esiste un problema che può verificarsi in modo sistematico.

Quando si termina l'esecuzione di un ciclo, il salto che si trova alla fine verrà predetto in modo errato; ancor peggio, questo errore di previsione cambierà il bit nella tabella dei salti per indicare che in futuro occorrerà prevedere "di non effettuare il salto". Quando si rientrerà nel ciclo, alla fine della prima iterazione il salto che si trova in fondo verrà predetto in modo errato. Questo problema si può verificare spesso se il ciclo è innestato all'interno di un altro ciclo oppure nel caso in cui faccia parte di una procedura richiamata frequentemente.

Per eliminare questa predizione sbagliata possiamo dare una seconda chance all'elemento della tabella, ovvero modificare la predizione soltanto dopo che due previsioni consecutive si sono rivelate errate. Questo approccio, mostrato nella Figura 4.41(b), richiede che nella tabella della storia siano presenti due bit di predizione, uno per ciò che "si suppone" occorra fare con il salto e uno per tener traccia di ciò che si è verificato l'ultima volta.

Un modo leggermente diverso per interpretare questo algoritmo consiste nel vederlo come un automa a stati finiti con quattro stati (Figura 4.42). Dopo una serie di corrette predizioni di "non effettuare il salto", l'automa si troverà nello stato 00 che predice di "non effettuare il salto" alla volta successiva. Se quella predizione è errata, passerà allo stato 01, ma continuerà a predire di "non effettuare il salto". Solo se anche questa predizione si rivela errata, allora la macchina passerà allo stato 11 suggerendo di effettuare la diramazione. In sostanza il bit a sinistra nello stato rappresenta la predizione, mentre il bit a destra tiene traccia dell'ultimo salto. Questa strategia prevede di utilizzare soltanto 2 bit per memorizzare la storia dei salti, ma se ne possono usare anche 4 o 8.

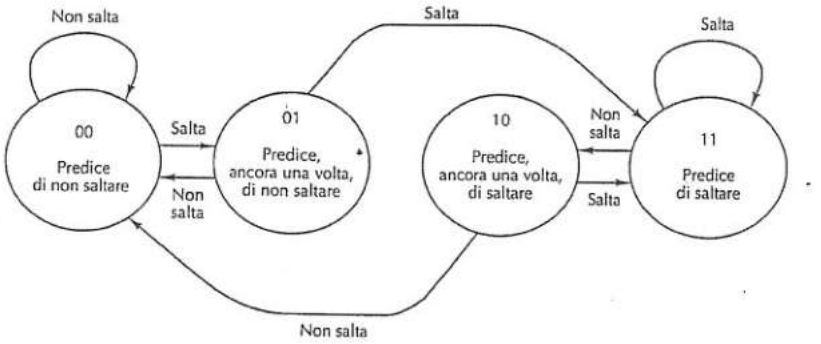


Figura 4.42 Automa a stati finiti a 2 bit per la predizione delle diramazioni.

Già nella Figura 4.28 avevamo visto un automa a stati finiti. In effetti tutti i nostri microprogrammi possono essere descritti da questo modello in cui ogni linea rappresenta un particolare stato nel quale si può trovare l'automa e in cui è possibile compiere delle transizioni verso un insieme finito di stati. Gli automi a stati finiti sono utilizzati diffusamente in varie fasi della progettazione hardware.

Finora abbiamo assunto che la destinazione di un salto condizionale fosse nota, generalmente come indirizzo esplicito al quale andare (contenuto all'interno dell'istruzione stessa) oppure come uno spiazzamento relativo all'istruzione corrente (cioè, un valore con segno da aggiungere al contatore d'istruzioni). Questa assunzione è valida nella maggior parte dei casi, tuttavia esistono alcune istruzioni di salti condizionali che calcolano l'indirizzo di destinazione eseguendo delle operazioni aritmetiche sui registri. L'automa della Figura 4.42 predice in modo accurato se intraprendere oppure no i salti, ma queste previsioni sono inutili se l'indirizzo di destinazione è sconosciuto. Un modo per gestire questa situazione consiste nel memorizzare nella tabella della storia l'effettivo indirizzo verso il quale è stato effettuato il salto l'ultima volta, come mostra la Figu-

ra 4.41(c). Seguendo questo metodo, se la tabella indica che l'ultima volta il salto all'indirizzo 516 è stato effettuato e ha portato all'indirizzo 4000, allora, nel caso in cui la predizione dica di effettuare la diramazione, l'esecuzione continuerà nuovamente a partire dall'indirizzo 4000.

Un approccio differente per la predizione dei salti consiste nel tener traccia del fatto che gli ultimi k salti condizionali siano stati oppure no effettuati, indipendentemente dalle istruzioni alle quali erano associati. Questo numero a k bit, memorizzato in un **registro a scorrimento della storia dei salti**, viene successivamente confrontato in parallelo con tutte le chiavi a k bit degli elementi di una tabella della storia; se il confronto ha esito positivo si utilizza la predizione trovata nella tabella. Anche se può sembrare strano questa tecnica funziona piuttosto bene.

Predizione statica delle diramazioni

Tutte le tecniche per la predizione delle diramazioni viste finora sono di tipo dinamico, cioè vengono effettuate durante l'esecuzione del programma. Uno dei vantaggi delle tecniche dinamiche è che si adattano al comportamento corrente del programma. Lo svantaggio è che richiedono un costoso hardware specializzato e una grande complessità del chip.

Una strategia alternativa consiste nel richiedere l'aiuto del compilatore. Quando il compilatore incontra un'istruzione come

```
for (i = 0; i < 1000000; i++) { ... }
```

sa con certezza che la diramazione alla fine del ciclo sarà effettuata quasi tutte le volte. Se esistesse un modo per comunicarlo all'hardware, si potrebbe risparmiare una gran quantità di lavoro.

Anche se ciò rappresenta una modifica architetturale (e non soltanto un problema d'implementazione), alcune macchine, come l'UltraSPARC III, hanno un secondo insieme d'istruzioni di salto condizionale in aggiunta a quelle ordinarie (necessarie per garantire la retrocompatibilità). Le nuove istruzioni contengono un bit attraverso cui il compilatore può specificare se ritiene che la diramazione verrà oppure no effettuata. Quando si incontra una di queste istruzioni l'unità di fetch esegue semplicemente l'azione specificata dal bit. Per di più queste istruzioni non richiedono spazio prezioso nella tabella delle diramazioni, riducendo di conseguenza i conflitti al suo interno.

L'ultima tecnica che consideriamo per effettuare la predizione dei salti si basa sul *profiling* (Fisher e Freudenberger, 1992). Anche questa tecnica è di tipo statico, ma, invece di delegare al compilatore il compito di prevedere quali diramazioni verranno oppure no effettuate, si manda in esecuzione il programma (di solito su un simulatore) per catturare il comportamento delle diramazioni. Questa informazione viene fornita al compilatore, che la utilizza per impostare le istruzioni speciali di salto condizionale mediante le quali può specificare all'hardware in che modo deve comportarsi.

4.5.3 Esecuzione fuori sequenza e rinomina dei registri

La maggior parte delle CPU moderne funziona sia a pipeline sia in modo superscalare, com'è illustrato nella Figura 2.6. In generale ciò comporta la presenza di un'unità di

fetch che preleva istruzioni dalla memoria prima che siano necessarie, in modo da alimentare un'unità di decodifica. L'unità di decodifica attribuisce l'istruzione decodificata all'unità funzionale appropriata perché sia eseguita. In alcuni casi l'unità di decodifica può scomporre le singole istruzioni in micro-operazioni prima di distribuirle alle unità funzionali, secondo le capacità delle unità stesse.

Ovviamente l'architettura più semplice prevede che tutte le istruzioni siano eseguite nell'ordine in cui sono prelevate dalla memoria (ipotizzando per il momento che l'algoritmo di predizione delle diramazioni non sbagli mai). Tuttavia l'esecuzione in ordine non sempre fornisce prestazioni ottimali a causa delle dipendenze tra istruzioni. Le istruzioni che richiedono un valore calcolato da un'istruzione precedente non possono essere eseguite fintanto che quell'istruzione non abbia prodotto il valore desiderato. In tali situazioni (dipendenze RAW) le istruzioni devono aspettare. Esistono però anche altri tipi di dipendenze, come mostreremo in seguito.

Nel tentativo di aggirare questi problemi e ottenere prestazioni migliori, alcune CPU permettono di saltare le istruzioni dipendenti per raggiungere le istruzioni successive che non lo sono. Naturalmente, l'algoritmo interno di scheduling delle istruzioni impiegato deve garantire che l'esecuzione produca lo stesso risultato che produrrebbe l'esecuzione ordinata. Vediamo come funziona il riordinamento delle istruzioni su di un esempio dettagliato.

Per illustrare la natura del problema, cominciamo con una macchina che lancia le istruzioni sempre secondo l'ordine in cui appaiono nel programma e che, inoltre, richiede vengano eseguite nello stesso ordine. L'importanza di quest'ultima caratteristica risulterà evidente in seguito.

La macchina dell'esempio ha otto registri visibili al programmatore, con nomi che vanno da `R0` a `R7`. Tutte le istruzioni aritmetiche usano tre registri: due per gli operandi e uno per il risultato, così come avviene nel Mic-4. Ipotizziamo che, se un'istruzione è decodificata nel ciclo n, allora la sua esecuzione comincia al ciclo $n + 1$. Per un'istruzione semplice, come l'addizione o la sottrazione, la scrittura del risultato nel registro di destinazione avviene alla fine del ciclo $n + 2$. Per un'istruzione più complessa, come la moltiplicazione, la scrittura avviene alla fine del ciclo $n + 3$. Per rendere l'esempio realistico, consentiamo all'unità di decodifica di lanciare fino a due istruzioni per ciclo di clock. Le CPU superscalari in commercio possono lanciare comunemente dalle quattro alle sei istruzioni per ciclo di clock.

La sequenza di esecuzione dell'esempio è mostrata nella Figura 4.43, dove la prima colonna specifica il ciclo e la seconda il numero dell'istruzione. La terza colonna elenca le istruzioni decodificate, la quarta specifica le istruzioni che vengono lanciate (con un massimo di due per ciclo di clock). L'ultima colonna contiene l'istruzione che è stata ritirata (completata) durante il ciclo corrispondente. Si tenga presente che in questo esempio si richiede che sia il lancio sia l'esecuzione avvengano in ordine, cioè l'istruzione $k + 1$ non può essere lanciata prima che sia stata lanciata l'istruzione k, e l'istruzione $k + 1$ non può essere completata (e la scrittura del risultato nel registro apposito non può avvenire) finché non viene completata l'istruzione k. Le altre sedici colonne saranno presentate tra breve.

					Registri letti								Registri scritti							
C	I	Decodificata	L	R	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
1	1	R3=R0*R1	1		1	1										1				
	2	R4=R0+R2	2		2	1	1									1	1			
2	3	R5=R0+R1	3		3	2	1									1	1	1		
	4	R6=R1+R4	–		3	2	1									1	1	1		
3					3	2	1									1	1	1		
4				1	2	1	1										1	1		
				2	1	1												1		
				3																
5			4			1		1	1										1	
	5	R7=R1*R2	5			2	1		1										1	1
6	6	R1=R0–R2	–			2	1		1										1	1
7				4		1	1													1
8				5																
9			6		1		1							1						
	7	R3=R3*R1	–		1		1							1						
10					1		1							1						
11				6																
12			7			1		1								1				
	8	R1=R4+R4	–			1		1								1				
13						1		1								1				
14						1		1								1				
15				7																
16			8						2					1						
17									2					1						
18				8																

Figura 4.43 CPU superscalare che lancia e ritira le istruzioni in ordine (C = ciclo; L = lanciata; R = ritirata).

Dopo aver decodificato un'istruzione, l'unità di decodifica deve decidere se questa può essere lanciata immediatamente o meno. Per prendere la decisione, l'unità di decodifica deve conoscere lo stato di tutti i registri. Se, per esempio, l'istruzione corrente richiede un registro il cui valore non è stato ancora computato, l'istruzione corrente non può essere lanciata e la CPU è costretta ad attendere.

Manteniamo traccia dell'uso dei registri usando uno strumento chiamato **scoreboard** ("tabella dei punteggi"), utilizzato per la prima volta nel CDC 6600. Questa tabella è provvista di un piccolo contatore per ciascun registro che rappresenta il numero d'istruzioni attualmente in esecuzione che richiedono quel particolare registro come sorgente di un operando. Se, poniamo, sono ammesse al massimo 15 istruzioni in esecuzione simultanea, allora sarà sufficiente un contatore di 4 bit. Quando un'istruzione è lanciata

si incrementano le voci dello scoreboard corrispondenti ai registri dei suoi operandi, e quando un'istruzione è ritirata, le voci vengono decrementate.

Lo scoreboard contiene anche contatori per tener traccia dei registri usati come destinazioni. Poiché è permessa una sola scrittura alla volta, tali contatori possono essere lunghi un solo bit. Le ultime sedici colonne sulla destra della Figura 4.43 mostrano lo scoreboard.

Nelle macchine reali lo scoreboard registra anche l'uso delle unità funzionali, per evitare il lancio di un'istruzione per la quale non vi siano unità funzionali disponibili. Per semplicità, qui consideriamo il caso in cui ci siano sempre unità funzionali appropriate disponibili, così possiamo omettere le unità funzionali dallo scoreboard.

La prima riga della Figura 4.43 mostra I1 (istruzione 1), che moltiplica R0 e R1 e pone il risultato in R3. Dal momento che nessuno di questi registri è già in uso, l'istruzione viene lanciata e lo scoreboard è aggiornato con l'annotazione che R0 e R1 sono usati in lettura e R3 in scrittura. Nessuna delle istruzioni successive può scrivere in alcuno di questi registri o leggere il contenuto di R3 finché I1 non venga ritirata. Trattandosi di una moltiplicazione, l'istruzione verrà completata al termine del ciclo 4. I valori delle righe dello scoreboard riflettono lo stato dei registri al lancio delle diverse istruzioni. Le caselle vuote rappresentano il valore 0 dei contatori.

Poiché la macchina dell'esempio è una macchina superscalare che può lanciare due istruzioni per ciclo, la seconda istruzione viene lanciata anch'essa durante il ciclo 1. Tale istruzione somma R0 e R1, salvando il risultato in R4. Per stabilire se un'istruzione può essere lanciata si applicano le regole seguenti:

1. se un operando è in fase di scrittura, non lanciare (dipendenza RAW);
2. se il registro del risultato è in lettura, non lanciare (dipendenza WAR);
3. se il registro del risultato è in scrittura, non lanciare (dipendenza WAW).

Abbiamo già parlato di dipendenza RAW, che si verifica quando un'istruzione deve usare come operando il risultato di un'altra istruzione non ancora completata. Gli altri due tipi di dipendenze sono meno gravi: si tratta essenzialmente di conflitti di risorse. In una **dipendenza di tipo WAR** (*Write After Read*) un'istruzione cerca di sovrascrivere un registro che un'istruzione precedente potrebbe non aver terminato di leggere, e la **dipendenza WAW** (*Write After Write*) è simile. Entrambe possono essere evitate richiedendo che la seconda istruzione salvi il suo risultato da qualche altra parte (magari temporaneamente). Un'istruzione può essere lanciata, se non si verifica nessuna delle tre dipendenze e l'unità funzionale di cui ha bisogno è disponibile. Nel nostro caso l'istruzione 2 usa il registro R0 che è in lettura da parte di un'istruzione in corso di svolgimento, ma questa sovrapposizione è consentita e così l'istruzione viene lanciata. Analogamente, la terza istruzione può essere lanciata durante il ciclo 2.

Veniamo ora all'istruzione 4, che richiede l'uso di R4. Sfortunatamente notiamo che, alla riga 3, il registro R4 è in fase di scrittura. In questo caso ci troviamo di fronte a una dipendenza RAW, perciò l'unità di decodifica va in stallo finché R4 non torni disponibile. Durante l'attesa, l'unità smette di prelevare istruzioni dall'unità di fetch. Quando il buffer dell'unità di fetch si riempie, l'unità smette di eseguire il prefetch.

È utile notare che l'istruzione seguente secondo l'ordine del programma, cioè la 5, non causa alcun conflitto con le istruzioni in corso di svolgimento. Se l'architettura non richiedesse il lancio delle istruzioni in ordine, sarebbe possibile decodificare e lanciare l'istruzione 5.

Osserviamo quindi che cosa succede durante il ciclo 3. L'istruzione 2 termina alla fine del ciclo 3, trattandosi di un'addizione (due cicli). Sfortunatamente però non può essere ritirata (liberando R4 per l'uso da parte dell'istruzione 4). Perché? La ragione è che questa architettura richiede anche il ritiro ordinato delle istruzioni. Perché? Quale problema potrebbe sorgere dal salvare il risultato in R4 e marcarlo come disponibile?

La risposta è sottile e al contempo importante. Se le istruzioni possono essere completate fuori sequenza, in caso venisse sollevato un interrupt, sarebbe molto difficile salvare lo stato della macchina per poterlo ripristinare successivamente. In particolare, non sarebbe possibile stabilire che siano state eseguite tutte le istruzioni fino a un certo indirizzo e non quelle successive. Questa capacità è detta **interrupt preciso** ed è una caratteristica auspicabile in una CPU (Moudgill e Vassiliadis, 1996). Il ritiro delle istruzioni fuori sequenza rende gli interrupt imprecisi, ed è questo il motivo per cui alcune macchine completano le istruzioni in ordine.

Tornando all'esempio, alla fine del ciclo 4 le tre istruzioni in esecuzione possono essere ritirate, e l'istruzione 4 può essere infine lanciata durante il ciclo 5, insieme alla 5 appena decodificata. Ogniqualvolta un'istruzione viene ritirata, l'unità di decodifica deve controllare se c'è un'istruzione in stallo che può essere lanciata.

Durante il ciclo 6 l'istruzione 6 è in stallo perché deve scrivere nel registro R1 che è occupato, e la sua esecuzione comincia solo al ciclo 9. Il completamento dell'intera sequenza delle otto istruzioni richiede diciotto cicli a causa di molte dipendenze, nonostante l'hardware sia capace di lanciare due istruzioni a ogni ciclo. D'altra parte, si noti che scorrendo la colonna *L* della Figura 4.43 si evince come tutte le istruzioni siano state lanciate in ordine; la colonna *R* mostra come siano tutte state ritirate in ordine.

Prendiamo ora in esame un'altra architettura che ammette l'esecuzione fuori sequenza. Ora le istruzioni possono essere sia lanciate sia ritirate fuori sequenza. La stessa sequenza di otto istruzioni è mostrata nella Figura 4.44, laddove sono ora consentiti il lancio e il ritiro fuori sequenza.

La prima differenza si verifica al ciclo 3. Anche se l'istruzione 4 è in stallo, è permesso decodificare e lanciare la 5 visto che non è in conflitto con nessuna istruzione in esecuzione. A ogni modo, la possibilità di saltare alcune istruzioni causa un nuovo problema.

Si immagini che l'istruzione 5 utilizzi un operando calcolato dall'istruzione 4, l'istruzione saltata; con l'attuale scoreboard non lo avremmo notato. Bisogna quindi estendere la tabella per tenere traccia delle scritture effettuati dalle istruzioni saltate. A tal fine è possibile aggiungere un bit per registro (non mostrati nella figura), per annotare le scritture effettuati dalle istruzioni in stallo. La regola per il lancio delle istruzioni deve ora essere estesa per prevenire il lancio delle istruzioni i cui operandi sono il risultato di un'istruzione precedente che è stata saltata.

Tornando alle istruzioni 6, 7 e 8 nella Figura 4.43, notiamo che la 6 restituisce un valore in R1 che è richiesto dall'istruzione 7. Notiamo anche che tale valore non è mai più richiesto perché l'istruzione 8 sovrascrive R1. Non c'è motivo di usare R1 per contenere il risultato dell'istruzione 6. Per di più R1 è una pessima scelta come registro intermedio, nonostante sia ragionevole per un compilatore o per un programmatore abituato all'idea di esecuzione sequenziale senza sovrapposizione d'istruzioni.

					Registri letti								Registri scritti							
C	I	Decodificata	L	R	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
1	1	R3=R0*R1	1		1	1										1				
	2	R4=R0+R2	2		2	1	1									1	1			
2	3	R5=R0+R1	3		3	2	1									1	1	1		
	4	R6=R1+R4	–		3	2	1									1	1	1		
3	5	R7=R1*R2	5		3	3	2									1	1	1		1
	6	S1=R0–R2	6		4	3	3									1	1	1		1
				2	3	3	2									1		1		1
4			4		3	4	2		1							1		1	1	1
	7	R3=R3*S1	–		3	4	2		1							1		1	1	1
	8	S2=R4+R4	8		3	4	2		3							1		1	1	1
				1	2	3	2		3									1	1	1
				3	1	2	2		3										1	1
5				6		2	1		3					1					1	1
6			7			2	1	1	3					1		1			1	1
				4		1	1	1	2					1		1				1
				5				1	2					1						
				8				1								1				
7								1								1				
8								1								1				
9				7																

Figura 4.44 Operazioni di una CPU superscalare che lancia e ritira le istruzioni fuori sequenza (C = ciclo; L = lanciata; R = ritirata).

Nella Figura 4.44 introduciamo una nuova tecnica per risolvere questo problema: la **rinomina dei registri**. Una saggia unità di decodifica preferirà all'uso di R1 nell'istruzione 6 (ciclo 3) e 7 (ciclo 4) l'uso di un registro segreto, S1, invisibile al programmatore. In questo modo l'istruzione 6 può essere lanciata in modo concorrente con la 5. Le CPU moderne hanno sovente dozzine di registri segreti destinati alla rinomina di registri. Questa tecnica riesce in genere a eliminare le dipendenze WAR e WAW.

In corrispondenza dell'istruzione 8 usiamo ancora la rinomina dei registri. Questa volta R1 è rinominato come S2 di modo che l'addizione possa cominciare prima che R1 sia libero, alla fine del ciclo 6. Nel caso in cui R1 fosse a questo punto la vera destinazione del risultato, il contenuto di S2 potrebbe sempre essere ricopiato in R1 in tempo utile. Oppure, soluzione ancora migliore, è possibile rinominare le sorgenti di tutte le istruzioni future che avranno bisogno di R1 alla locazione dove il suo valore è effettiva-

È utile notare che l'istruzione seguente secondo l'ordine del programma, cioè la 5, non causa alcun conflitto con le istruzioni in corso di svolgimento. Se l'architettura non richiedesse il lancio delle istruzioni in ordine, sarebbe possibile decodificare e lanciare l'istruzione 5.

Osserviamo quindi che cosa succede durante il ciclo 3. L'istruzione 2 termina alla fine del ciclo 3, trattandosi di un'addizione (due cicli). Sfortunatamente però non può essere ritirata (liberando R4 per l'uso da parte dell'istruzione 4). Perché? La ragione è che questa architettura richiede anche il ritiro ordinato delle istruzioni. Perché? Quale problema potrebbe sorgere dal salvare il risultato in R4 e marcarlo come disponibile?

La risposta è sottile e al contempo importante. Se le istruzioni possono essere completate fuori sequenza, in caso venisse sollevato un interrupt, sarebbe molto difficile salvare lo stato della macchina per poterlo ripristinare successivamente. In particolare, non sarebbe possibile stabilire che siano state eseguite tutte le istruzioni fino a un certo indirizzo e non quelle successive. Questa capacità è detta **interrupt preciso** ed è una caratteristica auspicabile in una CPU (Moudgill e Vassiliadis, 1996). Il ritiro delle istruzioni fuori sequenza rende gli interrupt imprecisi, ed è questo il motivo per cui alcune macchine completano le istruzioni in ordine.

Tornando all'esempio, alla fine del ciclo 4 le tre istruzioni in esecuzione possono essere ritirate, e l'istruzione 4 può essere infine lanciata durante il ciclo 5, insieme alla 5 appena decodificata. Ogniqualvolta un'istruzione viene ritirata, l'unità di decodifica deve controllare se c'è un'istruzione in stallo che può essere lanciata.

Durante il ciclo 6 l'istruzione 6 è in stallo perché deve scrivere nel registro R1 che è occupato, e la sua esecuzione comincia solo al ciclo 9. Il completamento dell'intera sequenza delle otto istruzioni richiede diciotto cicli a causa di molte dipendenze, nonostante l'hardware sia capace di lanciare due istruzioni a ogni ciclo. D'altra parte, si noti che scorrendo la colonna *L* della Figura 4.43 si evince come tutte le istruzioni siano state lanciate in ordine; la colonna *R* mostra come siano tutte state ritirate in ordine.

Prendiamo ora in esame un'altra architettura che ammette l'esecuzione fuori sequenza. Ora le istruzioni possono essere sia lanciate sia ritirate fuori sequenza. La stessa sequenza di otto istruzioni è mostrata nella Figura 4.44, laddove sono ora consentiti il lancio e il ritiro fuori sequenza.

La prima differenza si verifica al ciclo 3. Anche se l'istruzione 4 è in stallo, è permesso decodificare e lanciare la 5 visto che non è in conflitto con nessuna istruzione in esecuzione. A ogni modo, la possibilità di saltare alcune istruzioni causa un nuovo problema.

Si immagini che l'istruzione 5 utilizzi un operando calcolato dall'istruzione 4, l'istruzione saltata; con l'attuale scoreboard non lo avremmo notato. Bisogna quindi estendere la tabella per tenere traccia delle scritture effettuati dalle istruzioni saltate. A tal fine è possibile aggiungere un bit per registro (non mostrati nella figura), per annotare le scritture effettuati dalle istruzioni in stallo. La regola per il lancio delle istruzioni deve ora essere estesa per prevenire il lancio delle istruzioni i cui operandi sono il risultato di un'istruzione precedente che è stata saltata.

Tornando alle istruzioni 6, 7 e 8 nella Figura 4.43, notiamo che la 6 restituisce un valore in R1 che è richiesto dall'istruzione 7. Notiamo anche che tale valore non è mai

più richiesto perché l'istruzione 8 sovrascrive R1. Non c'è motivo di usare R1 per contenere il risultato dell'istruzione 6. Per di più R1 è una pessima scelta come registro intermedio, nonostante sia ragionevole per un compilatore o per un programmatore abituato all'idea di esecuzione sequenziale senza sovrapposizione d'istruzioni.

					Registri letti								Registri scritti							
C	I	Decodificata	L	R	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
1	1	R3=R0*R1	1		1	1										1				
	2	R4=R0+R2	2		2	1	1									1	1			
2	3	R5=R0+R1	3		3	2	1									1	1	1		
	4	R6=R1+R4	–		3	2	1									1	1	1		
3	5	R7=R1*R2	5		3	3	2									1	1	1		1
	6	S1=R0–R2	6		4	3	3									1	1	1		1
				2	3	3	2									1		1		1
4			4		3	4	2		1							1		1	1	1
	7	R3=R3*S1	–		3	4	2		1							1		1	1	1
	8	S2=R4+R4	8		3	4	2		3							1		1	1	1
				1	2	3	2		3									1	1	1
				3	1	2	2		3										1	1
5				6		2	1		3					1					1	1
6			7			2	1	1	3					1		1			1	1
				4		1	1	1	2					1		1				1
				5				1	2					1		1				
				8				1								1				
7								1								1				
8								1								1				
9				7																

Figura 4.44 Operazioni di una CPU superscalare che lancia e ritira le istruzioni fuori sequenza (C = ciclo; L = lanciata; R = ritirata).

Nella Figura 4.44 introduciamo una nuova tecnica per risolvere questo problema: la **rinomina dei registri**. Una saggia unità di decodifica preferirà all'uso di R1 nell'istruzione 6 (ciclo 3) e 7 (ciclo 4) l'uso di un registro segreto, S1, invisibile al programmatore. In questo modo l'istruzione 6 può essere lanciata in modo concorrente con la 5. Le CPU moderne hanno sovente dozzine di registri segreti destinati alla rinomina di registri. Questa tecnica riesce in genere a eliminare le dipendenze WAR e WAW.

In corrispondenza dell'istruzione 8 usiamo ancora la rinomina dei registri. Questa volta R1 è rinominato come S2 di modo che l'addizione possa cominciare prima che R1 sia libero, alla fine del ciclo 6. Nel caso in cui R1 fosse a questo punto la vera destinazione del risultato, il contenuto di S2 potrebbe sempre essere ricopiato in R1 in tempo utile. Oppure, soluzione ancora migliore, è possibile rinominare le sorgenti di tutte le istruzioni future che avranno bisogno di R1 alla locazione dove il suo valore è effettiva-

mente memorizzato. In ogni caso, con questa tecnica l'addizione dell'istruzione 8 può essere lanciata più prontamente.

Su molte macchine reali il meccanismo di rinomina è profondamente connaturato con il modo in cui sono organizzati i registri: ci sono molti registri segreti e una tabella che mette in corrispondenza i registri visibili al programmatore con quelli segreti. Così il registro usato effettivamente al posto di R0 viene individuato, per esempio, guardando alla voce di posto 0 nella tabella delle corrispondenze. In questo modo non c'è un vero registro R0, ma solo un legame tra il nome R0 e uno dei registri segreti. Al fine di evitare le dipendenze tale legame cambia frequentemente nel corso dell'esecuzione.

Dalla quarta colonna della Figura 4.44 si evince che le istruzioni non sono state lanciate né ritirate in ordine. La conclusione di questo esempio è semplice: attraverso l'utilizzo dell'esecuzione fuori sequenza e della rinomina di registri è stato possibile abbreviare il calcolo di un fattore due.

4.5.4 Esecuzione speculativa

Nelle sezioni precedenti abbiamo introdotto il concetto di riordino delle istruzioni, allo scopo di migliorare le prestazioni. Sebbene non sia stato dichiarato in modo esplicito, finora l'attenzione è stata rivolta al riordinamento che avviene all'interno di un singolo **blocco elementare**. Passiamo adesso a un'analisi più approfondita di questo aspetto.

I programmi sono suddivisibili in blocchi elementari, ciascuno dei quali è costituito da una sequenza lineare d'istruzioni con un punto d'ingresso all'inizio e un punto di uscita alla fine. Un blocco elementare non contiene al suo interno alcuna struttura di controllo (come i costrutti `if` o `while`) di modo che la sua traduzione in linguaggio macchina non presenti nessuna diramazione. Il collegamento fra diversi blocchi elementari è costituito dalle strutture di controllo.

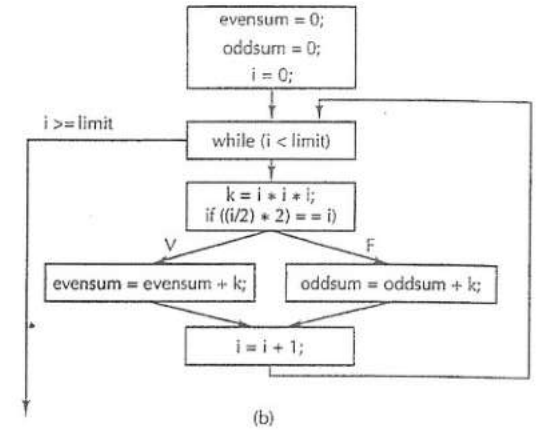
Un programma in questa forma può essere rappresentato attraverso un grafo orientato (Figura 4.45). In questo esempio le somme dei cubi dei numeri pari e dei numeri dispari vengono calcolate separatamente fino a un certo limite e memorizzate rispettivamente in *evensum* e *oddsum*. All'interno di ciascun blocco elementare il riordino delle istruzioni descritto nella sezione precedente funziona in maniera corretta.

Il problema è che la maggior parte dei blocchi elementari è di breve lunghezza e quindi non vi è al loro interno sufficiente parallelismo da poter sfruttare in modo efficace. Di conseguenza, il passo successivo per cercare di utilizzare tutti i cicli disponibili consiste nel permettere al riordinamento di superare i limiti che separano i blocchi elementari. Il guadagno maggiore si ottiene quando un'istruzione potenzialmente lenta può essere spostata più in alto nel grafo di modo che la sua esecuzione possa iniziare in modo anticipato. Questa potrebbe essere un'istruzione LOAD, un'operazione in virgola mobile, oppure l'inizio di una lunga catena di dipendenze. La tecnica di anticipare l'esecuzione del codice prima di una diramazione è nota come **slittamento**.

Immaginiamo che nella Figura 4.45 tutte le variabili siano memorizzate nei registri tranne *evensum* e *oddsum* (per mancanza di registri). Potrebbe aver senso spostare le loro istruzioni di LOAD all'inizio del ciclo, prima ancora di calcolare *k*, per farle cominciare anticipatamente di modo che i loro valori siano già disponibili nel momento in cui vengano richiesti. Ovviamente, durante ogni iterazione, solo uno di loro sarà effettivamente necessario, mentre l'altra operazione di LOAD verrà sprecata; se però sia la cache sia la memoria sono strutturate a pipeline e se vi sono cicli disponibili, potrebbe valerne la pena. L'esecuzione di parti del codice prima ancora di sapere se saranno effettivamente richieste è chiamata **esecuzione speculativa**. Per poter usare questa tecnica sono necessari sia il supporto da parte del compilatore e dell'hardware sia alcune estensioni dell'architettura.


```
evensum = 0;
oddsum = 0;
i = 0;
while (i < limit) {
    k = i * i * i;
    if (((i/2) * 2) == i)
        evensum = evensum + k;
    else
        oddsum = oddsum + k;
    i = i + 1;
}
```

(a)



(b)

Figura 4.45 (a) Frammento di programma. (b) Grafico a blocchi corrispondente.

È compito del compilatore spostare esplicitamente le istruzioni, dato che un loro riordinamento che scavalchi i limiti dei blocchi elementari è generalmente al di là delle possibilità dell'hardware.

L'esecuzione speculativa introduce alcuni problemi interessanti. Fra questi, è essenziale che nessuna delle istruzioni speculative produca risultati irrevocabili, dato che in un secondo momento potrebbe risultare che non dovevano essere eseguite. Nell'esempio della Figura 4.45 è corretto effettuare il fetch di *evensum* e *oddsum*, e anche calcolare la somma non appena *k* è disponibile (prima del costrutto `if`), ma non è corretto trasferire il risultato in memoria. In sequenze d'istruzioni più complesse, un modo comune per evitare che una parte del codice speculativo scriva nei registri prima ancora di sapere se ciò è voluto, consiste nel rinominare tutti i registri di destinazione utilizzati dal codice speculativo. In questo modo vengono modificati solo i registri di lavoro, così che non sorgano problemi se in ultima istanza il codice risulta non richiesto. Se al contrario il codice è necessario, allora i registri di lavoro vengono copiati nei registri di destinazione corretti. Come si può immaginare, non è semplice utilizzare la tecnica dello scoreboard per tener traccia di tutto ciò, ma può tuttavia essere realizzato disponendo di hardware sufficiente.

In ogni caso esiste un altro problema introdotto dal codice speculativo che non può essere risolto attraverso la semplice rinomina dei registri. Che cosa succede se l'esecu-

zione di un'istruzione speculativa causa un'eccezione? Una situazione fastidiosa, anche se non irrimediabile, si verifica quando un'istruzione LOAD causa un fallimento in una cache con blocchi di grandi dimensioni (per esempio 256 byte) e di una memoria molto più lenta sia della CPU sia della cache. Se un'istruzione di LOAD realmente necessaria pone la macchina in attesa per vari cicli, durante la fase di caricamento del blocco della cache, ciò deve essere considerato normale, dato che la parola è stata effettivamente richiesta. Al contrario, mettere installo una macchina durante il fetch di una parola che successivamente non risulterà necessaria è controproducente, al punto che un numero eccessivo di queste "ottimizzazioni" potrebbe rendere la CPU più lenta di quanto non lo sarebbe se non si facesse ricorso alle stesse. Se la macchina è dotata di memoria virtuale (trattata nel Capitolo 6) allora un'istruzione di LOAD speculativa potrebbe addirittura causare un errore di pagina e quindi richiedere un accesso al disco per ottenere la pagina richiesta. Dato che i falsi errori di pagina possono avere un effetto devastante sulle prestazioni è importante evitare che si manifestino.

Una soluzione utilizzata in alcune macchine moderne consiste nell'avere una speciale istruzione speculativa di LOAD, SPECULATIVE-LOAD, che cerchi di prelevare la parola dalla cache, ma che non compia alcuna azione se non la trova. Se, nel momento in cui viene richiesto, il valore è presente, allora può essere utilizzato; altrimenti l'hardware deve andare a recuperarlo sul momento. In questo modo il fallimento della cache non produce alcuna penalità se alla fine il valore risulta essere non necessario.

Una situazione decisamente peggiore è rappresentata dalla seguente istruzione

```
if (x > 0) z = y/x;
```

dove x, y e z sono variabili in virgola mobile. Si supponga che tutte le variabili siano prelevate e memorizzate in anticipo nei registri e che venga usata la tecnica dello slittamento del codice per spostare l'esecuzione della lunga operazione in virgola mobile prima del test if. Sfortunatamente x vale 0 e l'eccezione generata dalla divisione per zero fa terminare il programma. Il risultato è che l'uso dell'esecuzione speculativa ha fatto fallire un programma corretto. La cosa peggiore è che ciò è avvenuto nonostante il programmatore avesse previsto il problema e programmato esplicitamente il codice in modo da evitarlo. Difficilmente una simile situazione può rendere felici i programmatori.

Una possibile soluzione consiste nell'avere delle versioni speciali delle istruzioni che possano sollevare delle eccezioni, e nell'aggiungere a ogni registro un bit chiamato **poison bit** ("bit avvelenato"). Quando un'istruzione speculativa speciale fallisce, essa, al posto di causare un'eccezione, assegna il valore 1 al poison bit del registro risultato. Se in un secondo momento un'istruzione regolare utilizza questo registro allora viene sollevata un'eccezione (com'è giusto che sia). In ogni caso, se il risultato non è mai utilizzato, allora il poison bit può essere cancellato senza provocare alcun danno.

4.6 Esempi del livello di microarchitettura

In questo paragrafo mostreremo alcuni brevi esempi di tre processori all'avanguardia e vedremo come vengano impiegati al loro interno i concetti analizzati nel corso di questo capitolo. Dato che le macchine reali sono enormemente complesse (contengono milioni di porte logiche) la loro descrizione sarà necessariamente limitata agli aspetti essenziali. Le macchine in esame sono le solite usate finora: Core i7, OMAP4430 e ATmega168.

4.6.1 Microarchitettura della CPU Core i7

Dall'esterno il Core i7 appare come una macchina CISC tradizionale, dotata di processori che supportano un insieme d'istruzioni molto esteso e poco maneggevole, con operazioni su interi a 8, 16 e 32 bit e operazioni in virgola mobile a 32 e 64 bit. I registri visibili sono solo otto e sono tutti diversi tra loro. La lunghezza delle istruzioni varia da 1 a 17 byte. In breve, si tratta di un'architettura ereditata dal passato che sembra fare ogni cosa in modo errato.

Ciononostante il Core i7 contiene al suo interno un nucleo RISC moderno, semplice, efficace e altamente strutturato a pipeline; la velocità di clock è estremamente elevata ed è presumibile che essa continuerà ad aumentare anche nei prossimi anni. È sorprendente come gli ingegneri della Intel siano riusciti a costruire un processore moderno per implementare un'architettura di vecchia concezione. In questo paragrafo studieremo il funzionamento dell'architettura Core i7.

Descrizione della microarchitettura Sandy Bridge del Core i7

La microarchitettura Core i7, chiamata microarchitettura **Sandy Bridge**, rappresenta un notevole perfezionamento delle precedenti microarchitetture Intel, tra cui P6 e P4. La Figura 4.46 mostra una visione semplificata di questa architettura.

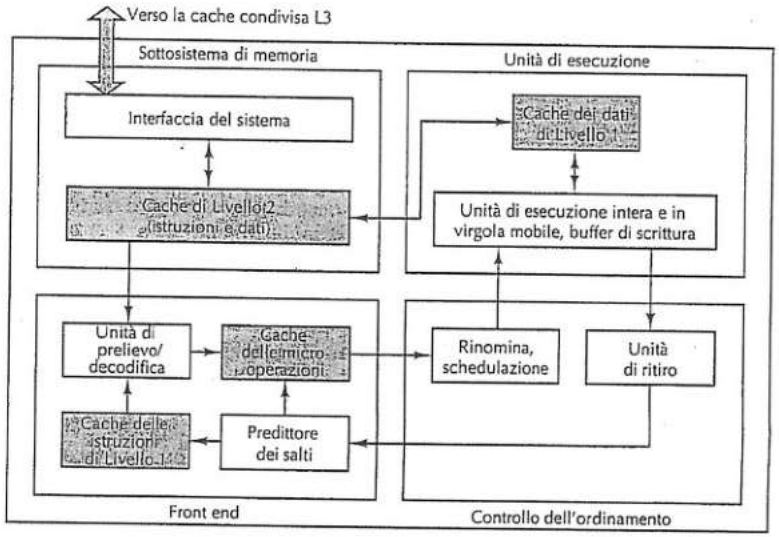


Figura 4.46 Diagramma a blocchi della microarchitettura Sandy Bridge del Core i7.

Il Core i7 è costituito da quattro parti principali: il sottosistema di memoria, il *front end*, il controllo dell'esecuzione fuori sequenza e le unità esecutive. Passiamo ad analizzare questi componenti in senso antiorario a partire da quello situato in alto a sinistra nella Figura 4.46.

Ogni processore del Core i7 contiene un sottosistema di memoria con una cache L2 (di livello 2) unificata e la logica per l'accesso alla cache L3 (di livello 3). Tutti i processori condividono un'unica grande cache L3, l'ultima prima di lasciare il chip della CPU e intraprendere il lungo viaggio verso la RAM esterna attraverso il bus di memoria. Le cache L2 del Core i7 sono di 256 KB e sono organizzate come cache associative a 8 vie con linee di cache da 64 byte. La dimensione della cache L3 varia da 1 a 20 MB: pagando di più si ottiene una cache più grande. Indipendentemente dalla dimensione, la L3 è organizzata come cache associativa a 12 vie con linee da 64 byte. Se un accesso alla L3 fallisce, la richiesta viene inviata alla RAM esterna attraverso il bus DDR3.

Alla cache L1 sono associate due unità di prefetch (non mostrate nella figura) che tentano di prelevare i dati da livelli di memoria più bassi e copiarli nella L1 prima che siano richiesti. Una di queste unità rileva i casi in cui il processore sta prelevando dalla memoria un flusso sequenziale di dati. In questi casi, l'unità preleva il blocco di memoria successivo. Una seconda e più sofisticata unità di prefetch tiene traccia della sequenza di indirizzi utilizzati dalle istruzioni di lettura e scrittura dello specifico programma. Se questi indirizzi si succedono con regolarità (per esempio, 0x1000... 0x1020... 0x1040...) l'unità preleva in anticipo il prossimo elemento a cui con una certa probabilità il programma farà accesso. Questa tecnica funziona benissimo con programmi che operano su vettori.

Il sottosistema di memoria nella Figura 4.46 è connesso sia al front end che alla cache dati L1. Il front end è il responsabile del prelievo delle istruzioni dal sottosistema di memoria, della loro decodifica in micro operazioni di tipo RISC e della loro memorizzazione in due cache istruzioni. Tutte le istruzioni prelevate vengono poste nella cache istruzioni di livello 1, delle dimensioni di 32 KB, organizzata come cache associativa con blocchi da 64 byte. Le istruzioni prelevate dalla cache L1 vengono passate ai decodificatori che determinano la sequenza di micro-operazioni che la pipeline deve eseguire per implementarle. Questo meccanismo di decodifica colma il divario tra un vecchio insieme di istruzioni CISC e un moderno percorso dati RISC.

Le micro-operazioni ottenute dalla decodifica alimentano la cache delle micro-operazioni, chiamata da Intel cache delle istruzioni di livello 0 (L0). La cache delle micro-operazioni è simile a una tradizionale cache istruzioni, ma è anche dotata di molto spazio "di respiro" per memorizzare la sequenza di micro-operazioni prodotte da ogni singola istruzione. Quando si memorizzano micro-operazioni decodificate al posto delle istruzioni originali non è più necessaria la decodifica delle istruzioni in ordine d'esecuzione. A prima vista si può pensare che questo sistema sia stato ideato per velocizzare la pipeline (e in effetti velocizza il processo di produzione di un'istruzione), ma Intel sostiene che la cache delle micro operazioni è stata aggiunta per ridurre il consumo energetico del front end. Con la cache delle istruzioni in ordine il resto del front end resta per l'80% del tempo sospeso in modalità di risparmio energetico.

Il front end realizza anche la predizione dei salti. Il predittore dei salti deve indovinare quando il flusso d'esecuzione di un programma interrompe un andamento puramente sequenziale e deve farlo molto prima che l'istruzione di salto sia eseguita. Il predittore dei salti del Core i7 è degno di nota. Purtroppo per noi, le specifiche dei predittori di salti sono, nella maggioranza dei casi, mantenute gelosamente segrete, perché le prestazioni di un predittore sono spesso la componente più critica nella velocità complessiva di un progetto. Maggiore è la precisione della predizione che i progettisti riescono a ottenere da ogni micrometro quadrato di silicio, migliori saranno le prestazioni dell'intero sistema. Alla luce di ciò, le società mantengono sotto chiave questi segreti e minacciano di perseguire penalmente qualunque dipendente che decidesse di rendere noti questi gioielli di conoscenza. A noi basta dire che tutti i predittori tengono traccia dell'effetto dei precedenti salti e utilizzano questa informazione per fare predizioni. I dettagli su quali informazioni vengono registrate, come sono memorizzate e come vengono utilizzate costituiscono proprio la parte top secret del progetto. Dopo tutto, se si avesse un modo fantastico per predire il futuro, probabilmente non lo si pubblicherebbe sul Web per farlo conoscere al mondo intero.

Le istruzioni sono trasferite dalla cache delle micro-operazioni allo schedulatore fuori-sequenza secondo l'ordine del programma, ma non sono necessariamente lanciate in quell'ordine. Quando lo schedulatore incontra una micro-operazione che non può essere eseguita, la trattiene, pur continuando a processare il flusso d'istruzioni per lanciare le istruzioni successive per cui sono disponibili tutte le risorse (registri, unità funzionali, e così via). Anche la rinomina dei registri viene effettuata in questa fase per permettere alle istruzioni con dipendenza WAR o WAW di continuare senza alcun ritardo.

Sebbene le istruzioni possano essere lanciate fuori sequenza, i requisiti dell'architettura Core i7 per gestire le interruzioni precise fanno sì che le istruzioni ISA debbano essere ritirate (cioè aver reso visibili i loro risultati) in ordine. L'unità di ritiro gestisce questo compito.

Nel back end del processore si trovano le unità di esecuzione che processano istruzioni su interi, in virgola mobile e specializzate. Sono presenti varie unità di esecuzione che funzionano in parallelo. Esse ottengono i loro dati dall'insieme dei registri (*register file*) e dalla cache di dati L1.

Pipeline Sandy Bridge del Core i7

Nella Figura 4.47 è mostrata una versione semplificata della microarchitettura Sandy Bridge in cui è rappresentata anche la pipeline. Nella parte alta dell'immagine si trova il front end, il cui compito è quello di prelevare istruzioni dalla memoria e prepararle per l'esecuzione. Il front end è alimentato dalle nuove istruzioni x86 che provengono dalla cache delle istruzioni L1 e decodifica queste istruzioni in micro-operazioni per memorizzarle nella cache delle micro-operazioni, che ne può contenere circa 1,5K. Le prestazioni fornite da una cache di queste dimensioni sono paragonabili a quelle di una cache convenzionale L0 della dimensione di 6 KB. La cache delle micro-operazioni mantiene, all'interno di ogni linea della traccia, gruppi di sei micro-operazioni. Se la sequenza di micro-operazioni è più lunga, è possibile collegare fra loro più linee della traccia.

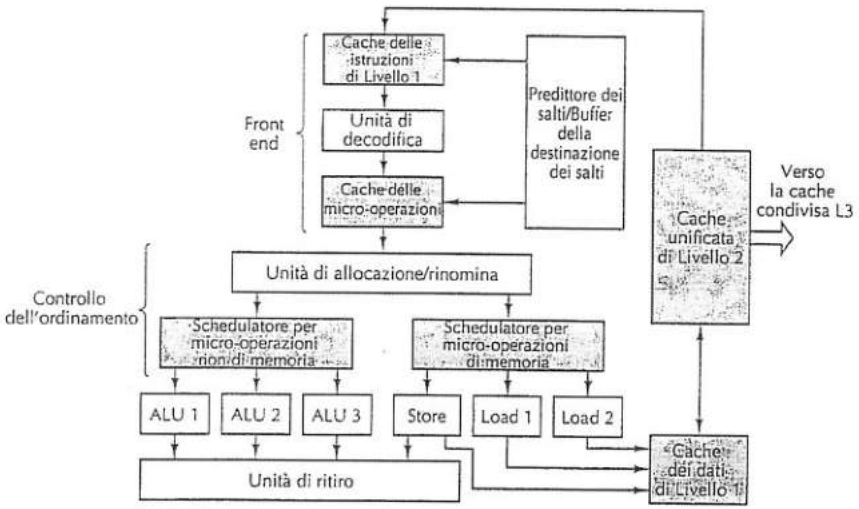


Figura 4.47 Vista semplificata del percorso dati del Corei7.

Se l'unità di decodifica incontra un salto condizionale, cerca nel predittore dei salti la destinazione predetta. Il predittore dei salti contiene la storia di tutte le diramazioni incontrate in passato e la utilizza per indovinare se un nuovo salto condizionale dovrà essere eseguito oppure no. È proprio per queste operazioni che si usa l'algoritmo top secret.

Se l'istruzione di diramazione non è presente nella tabella, si utilizza la predizione statica. Quando si incontra un salto all'indietro si assume che faccia parte di un ciclo e viene quindi effettuato; l'accuratezza di questo tipo di predizione è estremamente elevata. Quando invece si incontra un salto in avanti si assume che faccia parte di un'istruzione `if` e si sceglie di non effettuare la diramazione. In questo caso l'accuratezza della predizione statica è molto più bassa rispetto a quella dei salti all'indietro.

In caso di salto si consulta il **BTB** (*Branch Target Buffer*, "buffer della destinazione dei salti") per determinare l'indirizzo di destinazione. Il BTB mantiene l'indirizzo di destinazione dell'ultima occorrenza del salto. Nella maggior parte dei casi questo indirizzo è corretto (di fatto è sempre corretto in caso di salti con spiazzamento costante). I salti indiretti, come quelli utilizzati dalle chiamate di funzioni virtuali o dal costrutto `switch` del linguaggio C++, hanno molteplici destinazioni. Ecco che in questi casi le predizioni del BTB possono essere errate.

La seconda parte della pipeline, l'unità di controllo dell'esecuzione fuori sequenza, è alimentata dalla cache delle micro-operazioni. Man mano che le micro-operazioni giungono dal front end, fino a 4 per ogni ciclo, l'unità di **allocazione** e **rinomina** ne tiene traccia in una tabella, chiamata **ROB** (*ReOrder Buffer*, "buffer di riordinamento"), di 168 elementi. Ognuno di questi memorizza lo stato della micro-operazione prima che

essa venga ritirata. L'unità di allocazione e rinomina effettua quindi un controllo per verificare se le risorse richieste dalla micro-operazione sono disponibili. Se lo sono, la micro-operazione viene inserita in una delle due code dello schedulatore: una per le micro-operazioni di memoria e una per tutte le altre. In caso contrario la micro-operazione viene ritardata, ma quelle successive vengono in ogni caso elaborate, portando dunque a un'esecuzione fuori sequenza delle micro-operazioni. L'obiettivo di questa strategia è quello di tenere il più possibile occupate le unità funzionali. In un qualsiasi momento è possibile gestire fino a 154 istruzioni, di cui fino a 64 possono essere di lettura dalla memoria e 36 di scrittura in memoria.

A volte una micro-operazione può rimanere in stallo, poiché deve scrivere in un registro che in quel momento viene letto o scritto da una micro-operazione precedente. Come abbiamo già visto questi conflitti sono chiamati dipendenze WAR e WAV. Attraverso la rinomina della sua destinazione è possibile effettuare lo scheduling della micro-operazione affinché venga immediatamente eseguita; con questa tecnica la scrittura avviene in uno dei 160 registri di lavoro invece che all'interno della destinazione prevista, che risulta essere occupata. Se non ci sono registri disponibili o se la micro-operazione ha una dipendenza RAW (che non può mai essere risolta), l'allocatore annota la natura del problema all'interno dell'elemento della tabella ROB. La micro-operazione viene poi inserita in una delle code di esecuzione nel momento in cui tutte le risorse richieste si rendono disponibili.

Quando le micro-operazioni sono pronte per essere eseguite, l'unità di allocazione e rinomina le inserisce nelle due code dello schedulatore. Queste code inviano le micro-operazioni pronte per l'esecuzione alle sei unità funzionali che seguono:

1. ALU 1 e unità per moltiplicazioni in virgola mobile;
2. ALU 2 e unità per addizioni e sottrazioni in virgola mobile;
3. ALU3, trattamento dei salti e unità per confronti in virgola mobile;
4. istruzioni di memorizzazione;
5. istruzioni di caricamento 1;
6. istruzioni di caricamento 2.

Dato che gli schedulatori e le ALU possono elaborare un'operazione per ciclo, lo schedulatore di un Core i7 a 3 GHz è in grado di emettere 18 miliardi di operazioni al secondo, ma questo livello di throughput non verrà in pratica mai raggiunto. Dal momento che il front end fornisce un massimo di quattro micro-operazioni per ciclo, blocchi da sei micro-operazioni possono essere rilasciati solo per brevi periodi, perché le code dello schedulatore si svuotano rapidamente. Inoltre, ogni unità di memoria impiega quattro cicli per elaborare le operazioni e non è dunque in grado di contribuire costantemente alla velocità di esecuzione di picco. Pur non essendo in grado di saturare completamente le risorse di esecuzione, le unità funzionali forniscono una notevole capacità di esecuzione e per questo motivo l'unità di controllo fuori-sequenza va incontro a non pochi problemi per riuscire a tenerle occupate.

Le tre ALU non sono identiche. La ALU 1 può eseguire tutte le operazioni aritmetiche e logiche, oltre alle moltiplicazioni e alle divisioni. La ALU 2 può invece eseguire

solamente le istruzioni aritmetiche e logiche. La ALU3 può eseguire operazioni aritmetiche e logiche e risolvere i salti. Esistono delle differenze anche tra le due unità in virgola mobile.

La prima può eseguire operazioni aritmetiche in virgola mobile, tra cui le moltiplicazioni, mentre la seconda può eseguire solamente addizioni, sottrazioni e operazioni di spostamento (sempre in virgola mobile).

Le ALU e le unità in virgola mobile sono alimentate da una coppia di banchi di registri (*register file*) di 128 elementi; uno per i valori interi, e l'altro per quelli in virgola mobile. Questi registri forniscono tutti gli operandi per le istruzioni da eseguire e servono anche da deposito per i risultati. Al livello ISA sono visibili otto registri (EAX, EBX, ECX, EDX, e così via), ma, per via della rinomina dei registri, gli otto che contengono effettivamente i "veri" valori variano nel tempo a seconda di come cambia la corrispondenza durante l'esecuzione.

L'architettura Sandy Bridge ha introdotto le istruzioni **AVX** (*Advanced Vector Extension*) che supportano operazioni parallele a 128 bit sui vettori. Queste operazioni riguardano sia vettori di interi sia vettori di numeri in virgola mobile. Questa nuova estensione ISA raddoppia la dimensione possibile dei vettori rispetto alle precedenti versioni SSE e SSE2. Come può questa architettura implementare operazioni a 256 bit con percorsi dati e unità funzionali di 128 bit? Coordinando abilmente due porte dello schedulatore a 128 bit per creare un'unità funzionale a 256 bit.

La cache dati L1 è strettamente accoppiata al back-end della pipeline Sandy Bridge. Si tratta di una cache di 32 KB che contiene numeri interi, numeri in virgola mobile e altri tipi di dati e, a differenza della cache delle micro-operazioni, non viene decodificata in alcun modo, ma mantiene semplicemente una copia dei byte in memoria. La cache dati L1 è una cache associativa a 8 vie con 64 byte per linea di cache. Si tratta di una cache write-back, il che significa che quando una linea di cache viene modificata, il dirty bit della linea viene asserito, mentre i dati vengono ricopiati sulla cache L2 quando vengono eliminati dalla cache dati L1. La cache è in grado di gestire due operazioni di lettura e una di scrittura per ogni ciclo di clock. Questi accessi multipli vengono implementati utilizzando il *banking*, che divide la cache in sottocache separate (8 nel caso di Sandy Bridge). Se i tre accessi avvengono su banchi distinti possono procedere in simultanea, altrimenti gli accessi conflittuali al banco, tranne uno, dovranno essere in stallo. Se una parola non è presente nella cache L1, viene inviata una richiesta alla cache L2, che risponde immediatamente oppure recupera la linea di cache dalla cache condivisa L3 e quindi risponde. Nello stesso istante possono aver luogo fino a dieci richieste dalla cache L1 per la cache L2.

Dato che le micro-operazioni vengono eseguite fuori sequenza, le operazioni di scrittura nella cache L1 non sono permesse finché non siano state ritirate tutte le istruzioni che precedono una particolare scrittura. **L'unità di ritiro** ("completamento") ha il compito di ritirare in ordine le istruzioni e di tener traccia del punto in cui ci si trova. Se si verifica un interrupt le istruzioni non ancora ritirate vengono annullate; in questo modo il Core i7 gode della proprietà che, quando si verifica un interrupt, esiste un punto in cui tutte le istruzioni precedenti sono state completate e nessuna di quelle successive produce alcun effetto.

Se viene ritirata un'istruzione di memorizzazione mentre qualche istruzione che la precede è ancora in esecuzione, non è possibile aggiornare la cache L1. I risultati vengono quindi inseriti in un buffer per le memorizzazioni pendenti: questo buffer è composto da 36 elementi che corrispondono a 36 possibili memorizzazioni contemporaneamente in esecuzione. In seguito, se un'istruzione di caricamento prova a leggere i dati memorizzati, può esserle restituito il valore contenuto nel buffer per le memorizzazioni pendenti, anche se non è ancora stato aggiornato all'interno della cache L1. Questo processo è chiamato avanzamento **store-to-load**. Anche se questo meccanismo di avanzamento può sembrare semplice, la sua implementazione è piuttosto complicata perché le scritture che intervengono potrebbero non aver ancora calcolato i loro indirizzi. In questo caso, la microarchitettura non può sapere quale operazione presente nel buffer produrrà il valore necessario. Il processo di determinazione dell'operazione di scrittura che fornisce il valore per la lettura viene chiamato disambiguazione.

A questo punto dovrebbe essere evidente che il Core i7 ha una microarchitettura molto complessa, progettata per poter eseguire il vecchio insieme d'istruzioni Pentium su un moderno nucleo di tipo RISC con una profonda struttura a pipeline. Per raggiungere questo scopo suddivide le istruzioni Pentium in micro-operazioni, le memorizza all'interno di una cache e ne inserisce tre alla volta nella pipeline, per essere eseguite da un insieme di ALU; in condizioni ottimali queste unità sono in grado di eseguire fino a sei microoperazioni per ciclo. Le micro-operazioni sono eseguite fuori sequenza, ma vengono ritirate in ordine e anche la memorizzazione dei risultati all'interno delle cache L1 e L2 avviene in ordine.

4.6.2 Microarchitettura della CPU OMAP4430

Il cuore del processore OMAP4430 è costituito da due processori ARM Cortex A9, microarchitetture ad alte prestazioni che implementano le istruzioni ARM (versione 7). Il Cortex A9, progettato da ARM ltd., è utilizzato in diverse varianti in una vasta gamma di dispositivi integrati. ARM non produce il processore, ma fornisce il progetto ai produttori che vogliono includerlo nei loro SoC (nel nostro caso alla Texas Instruments).

Il processore Cortex A9 è a 32 bit, ha registri e un percorso dati a 32 bit. Come l'architettura interna, anche il bus di memoria è a 32 bit. A differenza del Core i7, il Cortex A9 è una vera e propria architettura RISC; ciò significa che non è necessario adottare per l'esecuzione un complicato meccanismo di conversione di vecchie istruzioni CISC in sequenze di micro-operazioni, perchè le istruzioni ARM sono già micro-operazioni. Tuttavia nel corso degli anni sono state aggiunte istruzioni per la grafica e la multimedialità la cui esecuzione richiede la presenza di hardware specializzato.

Descrizione della microarchitettura Cortex A9 dell'OMAP4430

La Figura 4.48 mostra il diagramma a blocchi del Cortex A9. Nel complesso è molto meno complicato della microarchitettura Sandy Bridge del Core i7, perché deve implementare un'architettura ISA decisamente più semplice. Ciononostante alcuni dei componenti chiave sono simili a quelli del Core i7. Le somiglianze sono determinate principalmente dalla tecnologia disponibile, dai limiti imposti dal consumo energetico e da fattori economici. Per esempio, entrambi i progetti adottano gerarchie di cache multili-

vello per andare incontro agli stretti vincoli di costo delle tipiche applicazioni integrate; tuttavia, l'ultimo livello del sistema di memoria cache del Cortex A9 (L2) è solamente di 1MB, significativamente più piccolo rispetto all'ultimo livello di cache (L3) del Core i7, che può essere 20 volte più grande. Le differenze risiedono invece principalmente nella necessità, o meno, di dover realizzare un ponte tra un vecchio sistema d'istruzioni CISC e un moderno nucleo di tipo RISC.

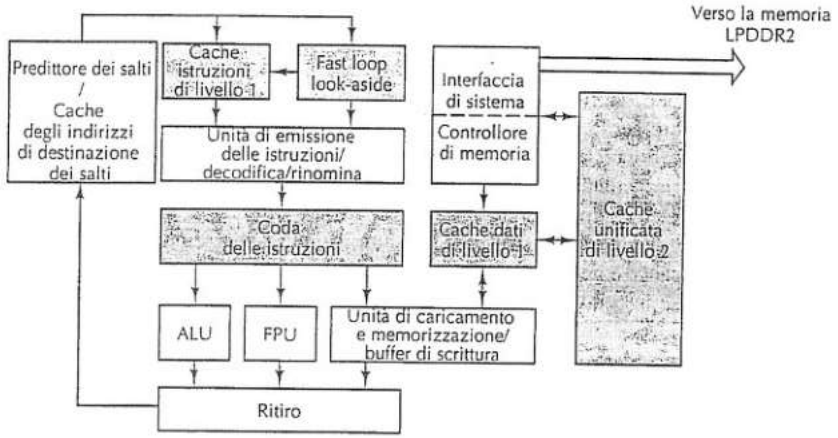


Figura 4.48 Diagramma a blocchi della microarchitettura Cortex A9 dell'OMAP 4430.

Nella parte alta della Figura 4.48 si trova la cache delle istruzioni a 32 KB, di tipo associativo a 4 vie, che utilizza linee di cache da 32 byte. Dato che la maggior parte delle istruzioni ARM sono di 4 byte, questa cache può mantenere circa 8K istruzioni, leggermente più della cache delle micro-operazioni del Core i7.

L'unità di **rifornimento delle istruzioni** prepara fino a quattro istruzioni per ciclo per l'esecuzione. Nel caso in cui si verifichi un fallimento della cache L1, viene emesso un numero minore d'istruzioni. Quando si incontra un salto condizionale si consulta un predittore di salti con 4K voci per predire se il salto verrà effettuato o meno.

Se la predizione dice che avverrà il salto, viene consultata la cache degli indirizzi di destinazione dei salti, con 1K elementi, per predire l'indirizzo di destinazione. Inoltre, se il front-end rileva che il programma sta eseguendo un *ciclo stretto* (ovvero un piccolo ciclo non innestato), carica il ciclo nella cache *fast-loop look-aside*. Questa ottimizzazione velocizza il prelievo delle istruzioni e riduce i consumi, perché le cache e i predittori possono restare in modalità di sospensione durante l'esecuzione del ciclo stretto.

L'output dell'unità di emissione delle istruzioni confluisce nei decodificatori, che determinano di quali risorse e di quali input ha bisogno un'istruzione. Come nel caso del Core i7, le istruzioni vengono rinominate dopo la decodifica per eliminare i rischi di WAR che possono rallentare l'esecuzione fuori sequenza. Dopo la rinomina, le istruzioni vengono poste nella coda di consegna delle istruzioni, che le rilascerà, potenzialmente fuori sequenza, quando i loro input saranno pronti per le unità funzionali.

La coda di emissione delle istruzioni invia le istruzioni alle unità funzionali, come mostra la Figura 4.48. L'unità di esecuzione intera contiene due ALU, una piccola pipeline per le istruzioni di salto e il banco dei registri che mantiene i registri ISA e alcuni registri di lavoro. La pipeline del Cortex A9 può eventualmente contenere circuiti di calcolo aggiuntivi che agiscono come unità funzionali. ARM supporta un motore per il calcolo in virgola mobile chiamato VFP e una elaborazione vettoriale SIMD su interi chiamata NEON.

L'unità di caricamento e memorizzazione gestisce varie istruzioni che si riferiscono alla memoria, e ha percorsi dati verso la cache dei dati e il buffer di scrittura. La **cache dei dati** è una tradizionale cache L1 da 32 KB di tipo associativo a 4 vie e con linee di 32 byte. Il **buffer di scrittura** mantiene le istruzioni di memorizzazione che non hanno ancora scritto il loro valore nella cache dei dati. Un'istruzione di load in esecuzione proverà prima a prelevare il valore dal buffer di scrittura utilizzando l'avanzamento store-to-load, come nel Core i7, e poi, se il valore non è disponibile nel buffer di scrittura, lo preleverà dalla cache dei dati. Un possibile esito dell'esecuzione di una load è l'indicazione, proveniente dal buffer di scrittura, di una necessaria attesa, dovuta al fatto che una precedente istruzione di scrittura con indirizzo sconosciuto sta bloccando l'esecuzione. Nel caso in cui l'accesso alla cache dei dati L1 fallisca, il blocco di memoria viene prelevato dalla cache L2 unificata. In alcune circostanze il Cortex A9 è in grado di eseguire il prefetching hardware dalla cache L2 nella cache L1, per migliorare le operazioni di lettura e scrittura.

Il chip OMAP4430 contiene anche la logica per il controllo degli accessi a memoria. Tale logica è divisa in due parti: l'interfaccia di sistema e il controllore di memoria. L'interfaccia permette il collegamento alla memoria su un bus LPDDR2 a 32 bit e tutte le richieste di memoria all'ambiente esterno passano attraverso questa interfaccia. Il bus LPDDR2 supporta un indirizzamento a 26 bit verso 8 banchi che restituiscono parole di 32 bit. La memoria centrale può arrivare, teoricamente, alla dimensione di 4GB per ogni canale LPDDR2. L'OMAP ha due di questi canali e può così indirizzare fino a 8 GB di memoria esterna.

Il controllore della memoria mappa indirizzi virtuali a 32 bit in indirizzi fisici a 32 bit. Il Cortex A9 supporta la memoria virtuale (trattata nel Capitolo 6) con pagine di 4 KB. Per accelerare la corrispondenza ci sono delle tabelle speciali, chiamate **TLB** (*Translation Lookaside Buffers*), per confrontare l'indirizzo virtuale al quale si sta facendo riferimento con quelli referenziati nel passato più recente. Sono presenti due di queste tabelle per gestire la mappatura degli indirizzi di istruzioni e dati.

Pipeline Cortex A9 dell'OMAP4430

Il Cortex A9 dispone di una pipeline a 11 stadi. La Figura 4.49 ne mostra una rappresentazione semplificata, in cui gli 11 stadi sono indicati, sul lato sinistro, da abbreviazioni. Esaminiamo brevemente ogni stadio. Lo stadio *Fe1* (Fetch #1) si trova all'inizio della pipeline ed è il punto in cui l'indirizzo della successiva istruzione da prelevare viene utilizzato per indicizzare la cache delle istruzioni e dare il via a una predizione dei salti. Di solito questo indirizzo corrisponde a quello che segue l'istruzione corrente.

Tuttavia questo ordinamento sequenziale può essere alterato per varie ragioni; per esempio quando l'istruzione precedente è una diramazione che secondo la predizione deve essere eseguita, oppure quando occorre gestire un interrupt o un'eccezione. Dato che non è possibile effettuare il prelievo dell'istruzione e la predizione dei salti in un solo ciclo, lo stadio *Fe2* (Fetch #2) offre ulteriore tempo per portare a termine queste operazioni. Nello stadio Fe3 (Fetch #3) le istruzioni prelevate (al massimo quattro) vengono inserite nella coda delle istruzioni.

Negli stadi *De1* e *De2* (*Decode*, "decodifica") vengono decodificate le istruzioni. In questo passo vanno determinati gli input di cui le istruzioni hanno bisogno (registri e memoria) e quali risorse richiederanno per la loro esecuzione (unità funzionali). Una volta che la decodifica è completata, le istruzioni entrano nello stadio *Re* (*Rename*, "rinomina"), in cui i registri utilizzati vengono rinominati al fine di eliminare dipendenze WAR e WAW durante l'esecuzione fuori sequenza. Questo stadio contiene la tabella di rinomina, che cataloga quali registri fisici mantengono al momento tutti i registri dell'architettura. Con l'utilizzo di questa tabella ogni registro di input può essere facilmente rinominato. Al registro di output deve essere assegnato un nuovo registro fisico, scelto all'interno di una collezione di registri inutilizzati, che sarà utilizzato dall'istruzione fino al suo completamento.

Successivamente le istruzioni entrano nello stadio Iss (*Instruction Issue*, "rilascio dell'istruzione"), in cui vengono inserite nella coda di rilascio delle istruzioni. Questa coda controlla le istruzioni per le quali gli input sono tutti pronti. Al verificarsi di questa condizione, gli input delle istruzioni vengono acquisiti (dal banco dei registri fisici o dal bus bypass) e le istruzioni vengono inviate allo stadio di esecuzione. Come il Core i7, anche il Cortex A9 può emettere le istruzioni in ordine diverso rispetto a quello del programma. In ogni ciclo possono essere rilasciate fino a 4 istruzioni. La scelta delle istruzioni è vincolata dalla disponibilità delle unità funzionali.

Lo stadio *Ex* (*Execute*, "esecuzione") è quello in cui le istruzioni vengono realmente eseguite. La maggior parte delle istruzioni aritmetiche, booleane e di shift utilizzano le ALU intere e vengono completate in un ciclo, i caricamenti e le memorizzazioni impiegano due cicli (se trovano i valori nella cache L1) e le moltiplicazioni richiedono tre cicli. Lo stadio Ex contiene diverse unità funzionali:

1. ALU 1 intera;
2. ALU 2 intera;
3. unità di moltiplicazione;
4. ALU in virgola mobile e vettoriale SIMD (opzionalmente con supporto VFP e NEON);
5. unità di caricamento e memorizzazione.

Nel primo stadio *Ex* vengono anche processate le istruzioni di salto condizionato e ne viene determinata la direzione (salto oppure nessun salto). In caso di errata predizione viene inviato un segnale allo stadio *Fe1* e la pipeline viene svuotata.

Al completamento della loro esecuzione le istruzioni entrano nello stadio **WB** (*WriteBack*, "riscrittura"), in cui ogni istruzione aggiorna immediatamente il banco dei registri fisici. Quando un'istruzione diventa la più vecchia può scrivere i registri risultato nel

banco registri architetturale. In caso di eccezione o interrupt, sono questi ultimi valori, e non quelli nei registri fisici, a essere resi visibili. Questa azione di memorizzare i registri nel banco architetturale corrispondente al ritiro nel Core i7. In aggunta, nello stadio WB, ogni istruzione di memorizzazione completa la scrittura dei suoi risultati nella cache L1.

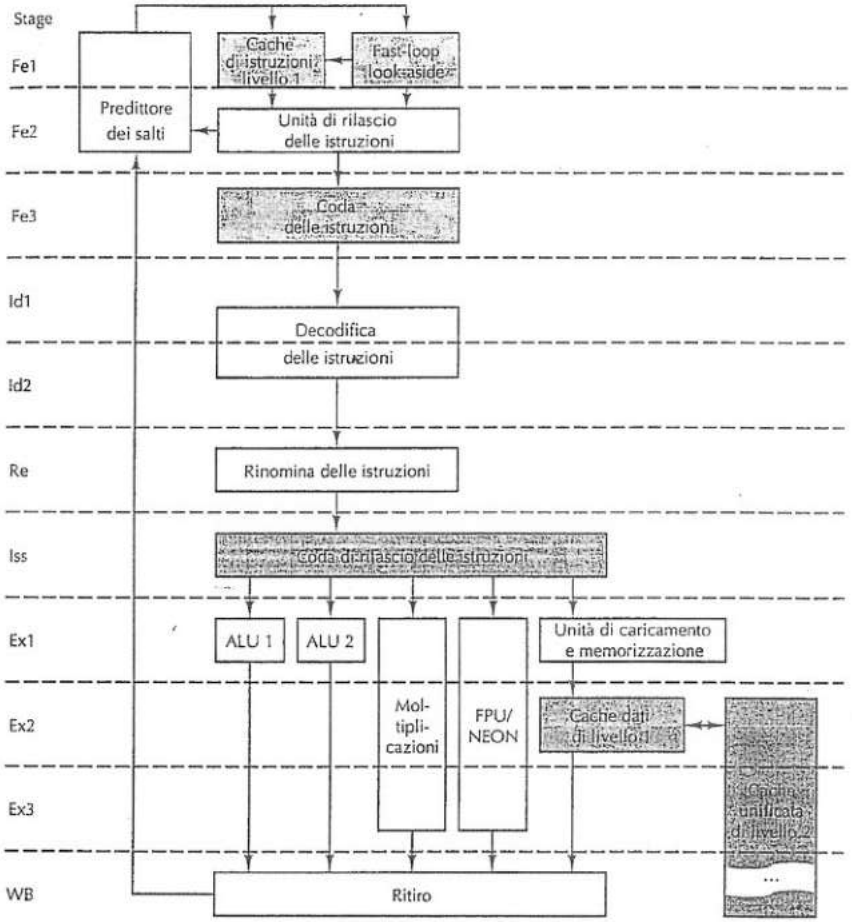


Figura 4.49 Rappresentazione semplificata della pipeline Cortex A9 dell'OMAP4430.

Questa descrizione del Cortex A9 è lontana dall'essere completa, ma permette di avere un'idea sostanzialmente corretta del suo funzionamento e delle differenze che la contraddistinguono rispetto alla microarchitettura del Core i7.

4.6.3 Microarchitettura del microcontrollore ATmega168

Il nostro ultimo esempio è la microarchitettura dell'Atmel ATmega168 (Figura 4.50) che è considerevolmente più semplice delle due precedenti. Il motivo risiede nelle piccole dimensioni del chip e nella sua economicità, vincoli dovuti al suo utilizzo nei sistemi integrati. Uno dei principali obiettivi della progettazione di questo chip era di renderlo economico, non veloce. Economico e Semplice sono sempre dei buoni amici. Economico e Veloce invece non sempre lo sono.

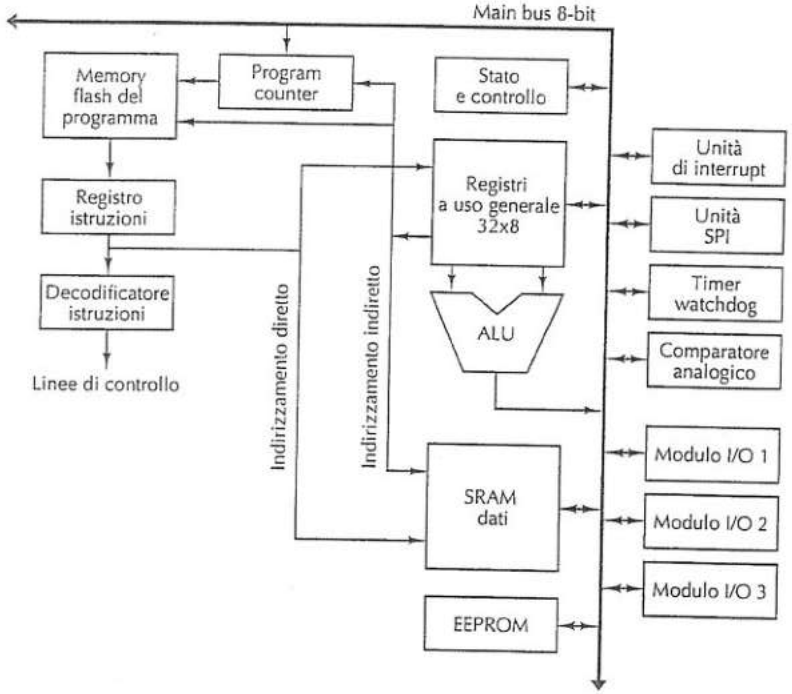


Figura 4.50 Microarchitettura dell'ATmega168.

Il cuore dell'ATmega168 è rappresentato dal bus principale a 8 bit, cui sono collegati i bit dei registri e di stato, le ALU, la memoria e i dispositivi di input/output. Descriviamoli brevemente. Il banco dei registri contiene 32 registri a 8 bit, utilizzati per memorizzare valori temporanei del programma. Il registro di stato e controllo mantiene i codici condizione dell'ultima operazione della ALU (per esempio segno, overflow, negativo, zero, riporto) più un bit che indica se vi è un interrupt pendente. Il program counter contiene l'indirizzo dell'istruzione in esecuzione. Per realizzare una operazione

ALU gli operandi vengono letti dai registri e inviati alla ALU. L'output della ALU può essere scritto in qualsiasi registro scrivibile attraverso il bus principale.

L'ATmega168 ha memorie per dati e istruzioni. La SRAM dei dati è da 1KB, troppo grande per essere interamente indirizzata con un indirizzo a 8 bit sul bus principale. L'architettura AVR permette quindi di costruire indirizzi con coppie consecutive di registri da 8 bit, producendo così indirizzi di 16 bit che permettono di supportare fino a 64 KB di memoria dati. La EEPROM offre fino a 1KB di spazio di memorizzazione non volatile in cui i programmi possono scrivere le variabili che devono essere mantenute anche in assenza di corrente.

Un meccanismo similare viene utilizzato per indirizzare la memoria del programma. In questo caso però 64 KB di spazio non sono sufficienti, nemmeno per sistemi integrati a basso costo. Per permettere una memoria delle istruzioni più grande l'architettura AVR utilizza tre registri di pagina RAM (RAMPX, RAMPY, RAMPZ), ognuno da 8 bit. Il registro di pagina RAM viene concatenato con una coppia di registri per formare un indirizzo di 24 bit, permettendo così uno spazio di indirizzamento per le istruzioni di 16 MB.

Interrompiamoci e riflettiamo un attimo. 64 KB di codice sono troppo pochi per un microcontrollore che deve essere utilizzato per un gioco o per piccole applicazioni. Nel 1964, IBM ha rilasciato il System 360 model 30, che aveva in totale 64 KB di memoria (senza meccanismi per poterla aumentare). Veniva venduto per $ 250.000, corrispondenti al valore a oggi di circa 2 milioni di dollari. L'ATmega168 costa $ 1 circa, anche meno se per ordini di grandi quantità. Se si guarda, per esempio, il listino prezzi della Boeing si scopre che il prezzo degli aeroplani non è sceso di 250.000 volte negli ultimi 50 anni; nemmeno quello delle macchine, né il prezzo dei televisori: solo il prezzo dei computer è sceso di così tanto.

In aggiunta, l'ATmega168 ha integrati sul chip un controllore di interrupt, un'interfaccia SPI (*serial port interface*) e dei timer, essenziali per le applicazioni in tempo reale. Ci sono anche tre porte digitali di I/O a 8 bit, che permettono all'ATmega168 di controllare fino a 24 collegamenti esterni verso pulsanti, sensori, luci, attuatori, ecc. È proprio la presenza di queste porte e dei timer che rende possibile l'utilizzo dell'ATmega168 per applicazioni integrate senza bisogno di chip supplementari.

L'ATmega168 è un processore sincrono in cui la maggior parte delle istruzioni, anche se non tutte, viene eseguita in un ciclo di clock. Il processore è dotato di una pipeline a due stadi, stadio di prelievo e stadio di esecuzione, in modo che mentre un'istruzione viene prelevata, l'istruzione precedente viene eseguita. Per eseguire le istruzioni in un ciclo, il ciclo di clock deve far spazio alla lettura dei registri dal banco dei registri, seguita dall'esecuzione dell'istruzione nella ALU, seguita poi dalla riscrittura dei registri nel banco dei registri. Visto che tutte queste operazioni avvengono in un unico ciclo, non vi è alcuna necessità di una logica di bypass o del rilevamento degli stalli. Le istruzioni del programma sono eseguite in ordine, in un unico ciclo e senza sovrapposizioni con altre istruzioni.

Anche se potremmo fornire maggiori dettagli su questo processore, la descrizione data in questo paragrafo e la Figura 4.50 sono sufficienti per avere un'idea di base della sua microarchitettura. L'ATmega168 ha un unico bus principale (per ridurre l'area del

chip), al quale sono collegati un insieme eterogeneo di registri e una varietà di memorie e dispositivi di I/O. Durante ogni ciclo del percorso due operandi vengono letti dal banco dei registri, fatti passare attraverso la ALU e il risultato viene memorizzato all'interno di un registro, proprio come nei calcolatori moderni.

4.7 Confronto tra i7, OMAP4430 e ATmega168

I tre esempi considerati sono molto differenti tra loro, pur avendo dei punti in comune. Il Core i7 è un insieme d'istruzioni CISC datato che gli ingegneri di Intel amerebbero fervidamente poter gettare nella Baia di San Francisco, se ciò non violasse la legge della California sull'inquinamento delle acque. L'OMAP4430 è una vera e propria architettura RISC, con un insieme d'istruzioni semplice e ordinato. L'ATmega168 è invece un semplice processore a 8 bit per le applicazioni integrate. Nel cuore di ognuno ci sono dei registri e una o più ALU che eseguono semplici operazioni aritmetiche e booleane.

Nonostante le ovvie differenze esterne il Core i7 e l'OMAP4430 hanno delle unità di esecuzione abbastanza simili. Le unità di esecuzione di entrambi accettano micro-operazioni che specificano un codice operativo, due registri sorgenti e un registro destinazione. Esse sono in grado di eseguire in ciascun ciclo una micro-operazione; hanno inoltre pipeline con un elevato numero di stadi, effettuano la predizione dei salti e sono dotate di cache separate per i dati e per le istruzioni.

Queste analogie tra i componenti interni non sono casuali e neanche dovuti all'interminabile passaggio degli ingegneri della Silicon Valley da una società a un'altra. Come abbiamo visto nelle architetture di Mic-3 e Mic-4, è facile e naturale costruire un percorso dati a pipeline che prende due registri sorgente, li fa passare attraverso la ALU e ne memorizza il risultato nuovamente in un registro. Nella Figura 4.34 è possibile vedere una rappresentazione grafica di una pipeline di questo tipo. Con la tecnologia attualmente disponibile, questa architettura è la più efficiente.

La differenza principale tra il Core i7 e l'OMAP4430 è nel modo in cui prendono le istruzioni del loro livello ISA e le assegnano alle unità funzionali. Il Core i7 deve suddividere le sue istruzioni CISC per portarle nel formato a tre registri richiesto dall'unità funzionale. Questo è ciò che fa il front end della Figura 4.47: trasforma istruzioni grandi in micro-operazioni dal formato semplice e regolare. L'OMAP4430 non deve effettuare questa conversione, dato che le sue istruzioni native ARM hanno già un formato semplice e regolare. Questo è il motivo per cui la maggior parte dei nuovi ISA sono di tipo RISC: permettono una miglior corrispondenza tra le istruzioni ISA e l'unità di esecuzione interna.

Può essere istruttivo confrontare la nostra architettura finale, il Mic-4, con i due esempi tratti dal mondo reale. Il Mic-4 assomiglia al Core i7, dato che entrambi devono interpretare un insieme d'istruzioni ISA di tipo non-RISC. Per far ciò entrambi suddividono le istruzioni ISA in micro-operazioni composte da un codice operativo, due registri sorgenti e un registro destinazione. In entrambi i casi le micro-operazioni vengono accodate prima di essere eseguite. Mic-4 emette le istruzioni, le esegue e le ritira sempre in ordine, mentre il Core i7 utilizza una politica di emissione in ordine, esecuzione fuori sequenza e ritiro in ordine.

Mic-4 e OMAP4430 non sono effettivamente confrontabili, dato che l'insieme d'istruzioni ISA dell'OMAP4430 è composto da istruzioni RISC (cioè micro-operazioni a tre registri) che non devono essere suddivise. Ognuna di queste istruzioni può essere eseguita, senza alcuna modifica, in un unico ciclo del percorso dati.

Tuttavia, diversamente dal Core i7 e dall'OMAP4430, l'ATmega168 è una macchina molto semplice. È più di tipo RISC che di tipo CISC, dato che la maggior parte delle sue istruzioni può essere eseguita in un singolo ciclo di clock e non deve essere suddivisa. Non sono presenti né pipeline né cache; inoltre l'emissione, l'esecuzione e il ritiro avvengono tutte in ordine. Nella sua semplicità assomiglia a Mic-1.

4.8 Riepilogo

Il cuore di ogni calcolatore è rappresentato dal percorso dati, che contiene dei registri, uno, due o tre bus, e una o più unità funzionali come le ALU e gli shifter. Il ciclo di esecuzione principale consiste nel prelevare operandi dai registri e nello spedirli, utilizzando i bus, alle ALU e alle altre unità funzionali per l'esecuzione. I risultati vengono poi memorizzati nuovamente nei registri.

Il percorso dati può essere controllato da un sequenzializzatore che preleva le microistruzioni da una memoria di controllo. Ogni microistruzione contiene dei bit che controllano il percorso dati durante un ciclo e specificano quali operandi selezionare, quale operazione eseguire e che cosa occorre fare con il risultato. Inoltre, ogni microistruzione specifica il proprio successore, di solito in modo esplicito, indicandone l'indirizzo. Alcune microistruzioni modificano questo indirizzo di base calcolandone l'OR con alcuni bit prima di utilizzarlo.

La macchina IJVM è una macchina a stack con codici operativi da 1 byte che permettono di usare lo stack per inserire, estrarre e combinare (cioè addizionare) parole. Nella microarchitettura Mic-1 è stata presentata un'implementazione microprogrammata. Aggiungendo un'unità per il prelievo delle istruzioni, in grado di caricare in anticipo i byte del flusso delle istruzioni, è possibile eliminare un gran numero di riferimenti al contatore d'istruzioni, accelerando notevolmente la macchina.

Esistono diversi modi per progettare il livello di microarchitettura. Occorre compiere molte scelte, come quella tra un'architettura a due bus e una a tre bus, tra i campi codificati oppure decodificati delle microistruzioni, tra l'utilizzo o meno del prefetching, sulla profondità delle pipeline, e molte altre ancora. Mic-1 è una semplice macchina controllata via software, in cui l'esecuzione è sequenziale e non vi è parallelismo. Al contrario Mic-4 è una microarchitettura altamente parallela con una pipeline a sette stadi.

Le prestazioni possono essere migliorate agendo sotto diversi aspetti della microarchitettura. Il modo principale per migliorare le prestazioni consiste nell'uso di memorie cache. Per accelerare i riferimenti alla memoria si usano comunemente cache a corrispondenza diretta e cache set-associative. Anche la predizione dei salti, sia statica sia dinamica, è importante, così come l'esecuzione fuori sequenza e l'esecuzione speculativa.

Le tre macchine di esempio, Core i7, OMAP4430 e ATmega168, hanno una microarchitettura invisibile ai programmatori del linguaggio assemblativo ISA. Il Core i7 utilizza uno schema complesso per convertire le istruzioni ISA in micro-operazioni, memorizzarle in una cache e alimentare con loro un nucleo RISC superscalare. Questo schema permette inoltre l'esecuzione fuori sequenza, la rinomina dei registri e tutti gli altri meccanismi descritti nel libro che consentono di ottenere anche un ultimo, minimo, miglioramento della velocità dell'hardware. L'OMAP4430 ha una pipeline profonda, ma a parte ciò è relativamente semplice, emettendo, eseguendo e ritirando le istruzioni in ordine. L'ATmega168 è decisamente semplice: è composto da un singolo bus principale al quale sono collegate una manciata di registri e una ALU.

PROBLEMI

1. Quali sono i quattro passi utilizzati dalle CPU per eseguire le istruzioni?
2. Nella Figura 4.6 il registro del bus B è codificato in un campo a 4 bit, mentre il bus C è rappresentato come una mappa di bit. Perché?
3. Nella Figura 4.6 è rappresentato un rettangolo etichettato come "Bit alto". Si disegni il circuito che lo realizza.
4. Quando viene abilitato il campo JMPC di una microistruzione, viene calcolato l'OR logico tra MBR e NEXT_ADDRESS, per formare l'indirizzo della microistruzione successiva. Esistono situazioni in cui ha senso che il campo NEXT_ADDRESS valga 0x1FF e che si utilizzi JMPC?
5. Supponiamo che nell'esempio della Figura 4.14(a) si aggiunga l'istruzione

   k = 5;

   dopo l'istruzione if. Quale dovrebbe essere il nuovo codice assemblativo? Si assuma che il compilatore sia ottimizzato.
6. Si forniscano due differenti traduzioni IJVM della seguente istruzione Java:

   i = k + n + 5;

7. Si scriva l'istruzione Java che produce il seguente codice IJVM:

   ```
   ILOAD j
   ILOAD n
   ISUB
   BIPUSH 7
   ISUB
   DUP
   IADD
   ISTORE i
   ```

8. Nel testo abbiamo affermato che quando si traduce in binario l'istruzione

   if (Z) goto L1; else goto L2

   L2 deve trovarsi nelle prime 256 parole della memoria di controllo. Non sarebbe stato ugualmente possibile avere, per esempio, *L1* all'indirizzo 0x40 e *L2* all'indirizzo 0x140? Si motivi la risposta.
9. Nel microprogramma Mic-1, MDR viene copiato all'interno di H, durante `if_icmpeq3`. Dopo qualche linea viene sottratto da TOS per effettuare un test di uguaglianza. Sicuramente sarebbe stato meglio avere in questo punto la seguente istruzione:


   ```
   If_cmpeq3 Z = TOS – MDR; rd
   ```

   Perché ciò non è stato fatto?
10. Si calcoli quanti nanosecondi impiega Mic-1 a 2.5 GHz per eseguire la seguente istruzione Java

    i = j + k;

11. Si risponda nuovamente alla domanda precedente considerando questa volta Mic-2 a 2,5 GHz. In base al calcolo effettuato quanto dovrebbe impiegare su Mic-2 un programma che viene eseguito da Mic-1 in 100 s?
12. Si scriva il microcodice che implementa per Mic-1 l'istruzione JVM POPTWO (che rimuove due parole dalla cima dello stack).
13. Su una macchina JVM completa esistono codici operativi speciali per caricare nello stack, senza utilizzare l'istruzione generale ILOAD, le variabili locali da 0 a 3. Come dovrebbe essere modificata IJVM per sfruttare nel modo migliore queste istruzioni?
14. L'istruzione ISHR (scorrimento aritmetico a destra su interi) esiste in JVM, ma non in IJVM. Essa utilizza i due valori in cima allo stack, sostituendoli con un unico valore, il risultato. La seconda parola a partire dalla cima dello stack è l'operando da traslare. Il suo contenuto è spostato a destra di un valore compreso tra 0 e 31; questo valore è specificato nei 5 bit meno significativi della parola in cima allo stack (gli altri 27 bit sono invece ignorati). Il bit del segno viene replicato a destra per tanti bit quanti sono quelli da spostare. Il codice operativo di ISHR è 122 (0x7A).
    a. Qual è l'operazione aritmetica che equivale a traslare a destra di un valore 2?
    b. Si estenda il microcodice per includere questa istruzione in IJVM.
15. L'istruzione ISHL (scorrimento a sinistra su interi) esiste in JVM, ma non in IJVM. Essa utilizza i due valori in cima allo stack, sostituendoli con un unico valore, il risultato. La seconda parola a partire dalla cima dello stack è l'operando da traslare. Il suo contenuto è spostato a destra di un valore compreso tra 0 e 31; questo valore è specificato nei 5 bit meno significativi della parola in cima allo stack (gli altri 27 bit sono invece ignorati). Sulla destra vengono inseriti tanti valori zero quanti sono i bit da spostare. Il codice operativo di ISHL è 120 (0x78).
    a. Qual è l'operazione aritmetica che equivale a traslare a sinistra di un valore 2?
    b. Si estenda il microcodice per includere questa istruzione in IJVM.
16. L'istruzione JVM INVOKEVIRTUAL deve conoscere il numero dei suoi parametri. Perché?
17. Si implementi per Mic-2 l'istruzione JVM DLOAD. Essa ha un indice a 1 byte e inserisce nello stack la variabile locale che si trova in questa posizione. Successivamente inserisce nello stack anche la parola che si trova nell'indirizzo successivo, più in alto.
18. Si disegni un automa a stati finiti per gestire il punteggio di una partita di tennis. Le regole del tennis sono le seguenti. Per vincere occorrono almeno quattro punti e occorre avere almeno due punti più dell'avversario. Si inizia nello stato (0, 0), che indica che non è stato ancora attribuito alcun punteggio. Si aggiunga quindi uno stato (1, 0) che indica che A ha segnato un punto; si etichetti con *A* l'arco che collega (0, 0) con (1, 0). Si aggiunga poi uno stato (0, 2) che indica che *B* ha segnato un punto e si etichetti con *B* l'arco proveniente da (0, 0). Si continui ad aggiungere stati e archi finché non siano stati inclusi tutti i possibili stati.
19. Si consideri nuovamente il problema precedente. Ci sono degli stati che potrebbero essere condensati in un unico stato senza cambiare il risultato delle partite? Se sì, quali sono?
20. Si disegni un automa a stati finiti per la predizione dei salti che sia più tenace di quello della Figura 4.42 nel senso di modificare le predizioni soltanto dopo tre previsioni errate consecutive.
21. Il registro di scorrimento della Figura 4.27 ha una capacità massima di 6 byte. Si potrebbe costruire una versione più economica della IFU con un registro di scorrimento a 5 byte? E che cosa si può dire di una versione con registro a 4 byte?

22. Dopo aver esaminato, nell'esercizio precedente, versioni più economiche della IFU, consideriamo ora quelle più costose. Ci potrebbe essere una qualche ragione per avere registri di scorrimento più grandi, per esempio di 12 byte? Si motivi la risposta.

23. Nel microprogramma Mic-2, il codice dell'istruzione `if_icmpeq6` porta a T quando Z è impostato a 1. Il codice in T è tuttavia identico a quello in `goto1`. Sarebbe stato possibile andare direttamente in `goto1`? Così facendo la macchina sarebbe risultata più veloce?

24. In Mic-4 l'unità di decodifica fa corrispondere al codice operativo IJVM un indice della ROM dove sono memorizzate le micro-operazioni corrispondenti. Potrebbe sembrare più semplice omettere semplicemente la fase di decodifica e inserire direttamente il codice operativo IJVM nella coda. Si potrebbe usare il codice operativo di IJVM come un indice all'interno della ROM, allo stesso modo di Mic-1. Che cosa c'è di errato in questa strategia?

25. Perché i computer sono equipaggiati con più livelli di cache? Non sarebbe meglio avere semplicemente un unico livello di cache più grande?

26. Un calcolatore ha due livelli di cache. Supponiamo che il 60% dei riferimenti alla memoria sia soddisfatto dalla cache di primo livello, il 35% da quella di secondo livello e il 5% fallisca con entrambe le cache. I tempi di accesso sono rispettivamente di 5 ns, 15 ns e 60 ns. I tempi relativi alla cache di secondo livello e alla memoria iniziano a partire dal momento in cui si sa che è necessario accedere a queste memorie (per esempio, un accesso alla cache di secondo livello non comincia prima di un fallimento della cache di primo livello). Qual è il tempo medio di accesso?

27. Alla fine del Paragrafo 4.5.1 abbiamo detto che la tecnica della *write allocation* è vantaggiosa solo se è probabile che in una riga ci siano più scritture nella stessa linea. Che cosa si può dire riguardo il caso in cui una scrittura sia seguita da più letture?

28. Nella prima stesura di questo libro la Figura 4.39 mostrava una cache associativa a tre vie invece di una a quattro vie. Uno dei revisori del testo ha avuto un improvviso scatto d'ira, affermando che quella figura avrebbe confuso orribilmente gli studenti, dato che tre non è una potenza di due e i calcolatori eseguono ogni operazione in binario. Poiché il cliente ha sempre ragione la figura è stata modificata in una cache associativa a quattro vie. Questa persona aveva effettivamente ragione? Si argomenti la risposta.

29. Molti architetti dei computer passano molto tempo cercando di rendere le pipeline più profonde. Perché?

30. Un calcolatore con una pipeline a cinque stadi tratta i salti condizionali ponendo la pipeline in stallo per tre cicli dopo aver rilevato una di queste diramazioni. Di quanto degrada le prestazioni questa strategia di mettere in stallo l'esecuzione, nel caso in cui il 20% di tutte le istruzioni siano salti condizionali? Si ignorino tutti gli altri motivi che possono mettere in stallo l'esecuzione, eccezion fatta per i salti condizionali.

31. Supponiamo che un calcolatore sia in grado di effettuare il prefetch di un massimo di 20 istruzioni. Tuttavia, in media, quattro di queste istruzioni sono salti condizionali, ciascuno dei quali può essere predetto correttamente nel 90% dei casi. Qual è la probabilità che il prefetching sia sulla strada giusta?

32. Supponiamo di voler cambiare la progettazione della macchina usata nella Figura 4.43, in modo da avere 16 registri al posto di 8. Modifichiamo quindi l'istruzione 6 in modo che usi R8 come propria destinazione. Che cosa succede nei cicli a partire dal ciclo 6?

33. Generalmente le dipendenze provocano dei problemi alle CPU con pipeline. Ci sono delle ottimizzazioni che possono essere fatte per gestire le dipendenze di tipo WAR e che possono migliorare realmente la situazione? Quali?

34. Si riscriva l'interprete di Mic-1, facendo però in modo che LV punti alla prima variabile locale invece che al puntatore di collegamento.

35. Si scriva un simulatore per una cache a corrispondenza diretta a 1 via. Il numero degli elementi e la dimensione della linea devono essere parametri della simulazione. Si facciano degli esperimenti con il simulatore e si commentino i dati ricavati.

CAPITOLO 5

Livello di architettura dell'insieme d'istruzioni

Questo capitolo tratta in dettaglio il livello di architettura dell'insieme d'istruzioni (ISA, *Instruction Set Architecture*). Tale livello si trova tra quello della microarchitettura e il sistema operativo, come raffigurato nella Figura 1.2. Storicamente, il livello ISA fu il primo a essere sviluppato, e costituiva infatti l'unico livello presente. Ancor oggi si usa riferirsi a questo livello come all'"architettura" di una macchina o altre volte (in modo improprio) come al "linguaggio assemblativo".

Il livello ISA ha una rilevanza particolare che lo rende importante per i progettisti di sistemi: costituisce l'interfaccia tra il software e l'hardware. Anche se, in linea di principio, sarebbe possibile disporre di un hardware che esegue direttamente programmi scritti in C, C++, Java o in altri linguaggi d'alto livello, non si tratta di una buona idea. Così facendo si perderebbe l'incremento di prestazioni garantito dalla compilazione rispetto all'interpretazione. Inoltre, per ragioni di praticità, è auspicabile che i computer siano capaci di eseguire codice scritto in più linguaggi, invece che in uno solo.

L'approccio prediletto dalla quasi totalità dei progettisti di sistemi è di partire da vari linguaggi d'alto livello per poi tradurli in una forma intermedia comune, il livello ISA, e quindi costruire l'hardware in grado di eseguire direttamente i programmi di livello ISA. Tale livello definisce l'interfaccia tra i compilatori e l'hardware ed è il linguaggio che entrambi possono comprendere. Le relazioni intercorrenti tra compilatori, livello ISA e hardware sono mostrate nella Figura 5.1.

In linea di principio, la fase di progettazione di una nuova macchina richiede la consultazione sia dei progettisti del compilatore, sia dei progettisti dell'hardware, al fine di individuare le caratteristiche desiderate per il livello ISA.

Se gli autori del compilatore richiedono alcune caratteristiche che gli ingegneri non possono implementare con costi contenuti (per esempio un'istruzione *branch-and-do-payroll*, "salta e contabilizza"), allora queste vengono escluse dal progetto. Allo stesso modo se i responsabili dell'hardware ideano una qualche nuova caratteristica ingegnosa che pretendono di implementare (per esempio una memoria in cui è velocissimo acce-

dere alle locazioni il cui indirizzo è un numero primo), ma gli addetti al software non riescono a capire come scrivere del codice che la possa usare, la proposta rimarrà sulla carta. Dopo molte trattative e simulazioni emergerà e verrà infine implementato un livello ISA ottimizzato alla perfezione per il linguaggio di programmazione richiesto.

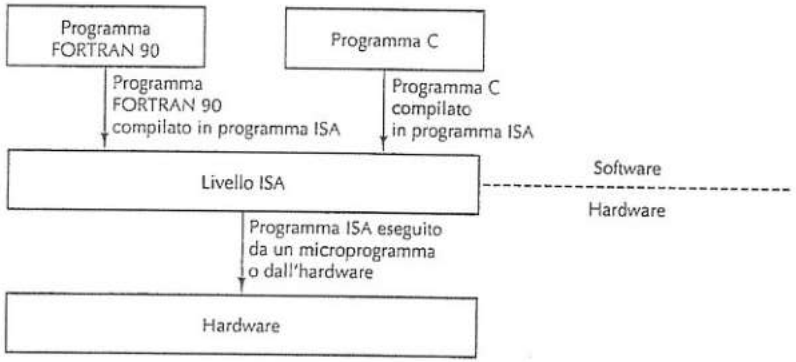


Figura 5.1 Il livello ISA è l'interfaccia tra compilatori e hardware.

Almeno in teoria. Nella bieca realtà, quando si tratta di sviluppare una nuova macchina la prima domanda posta dai potenziali utenti è: "il progetto è compatibile con il suo predecessore?". La seconda è: "posso farci girare il mio vecchio sistema operativo?". La terza è: "riuscirà a eseguire tutte le mie applicazioni senza bisogno di modificarle?". Qualora la risposta a una qualsiasi di queste domande fosse "no", i progettisti avrebbero dei seri problemi a giustificare il proprio lavoro. I clienti sono poco inclini a gettar via il loro vecchio software e ricominciare a scriverlo da capo.

Questo atteggiamento costringe gli architetti di computer a mantenere l'ISA costante di modello in modello, o a renderlo almeno **retrocompatibile**. Con ciò intendiamo che la nuova macchina debba essere in grado di eseguire i vecchi programmi senza modifiche. D'altra parte è del tutto comprensibile che una nuova macchina metta a disposizione alcune istruzioni e caratteristiche innovative che possono essere sfruttate solo dal software nuovo. Con riferimento alla Figura 5.1, i progettisti sono liberi di fare ciò che vogliono a livello hardware a patto che garantiscano un ISA retrocompatibile con i modelli precedenti; il livello hardware non interessa quasi a nessuno (e quasi nessuno sa come funziona). Così è possibile optare per un progetto di microprogrammazione o per l'esecuzione diretta, o aggiungere pipeline, funzionalità superscalari o qualunque altra cosa si desideri, ammesso che sia mantenuta la retrocompatibilità con l'ISA precedente. L'obiettivo è assicurare che i vecchi programmi girino sul nuovo processore e la sfida diventa la progettazione di macchine migliori soggette ai vincoli di retrocompatibilità.

Questo non vuol dire che la progettazione ISA conti poco; un buon ISA presenta vantaggi significativi rispetto a un cattivo progetto, specie in termini di potenza grezza di calcolo a parità di costi. Progetti ISA altrimenti equivalenti possono differire anche

del 25% in termini di prestazioni. Intendiamo affermare semplicemente che il mercato rende difficile (se non impossibile) l'abbandono di un ISA datato per introdurne uno nuovo. Ciononostante, di tanto in tanto emerge un nuovo ISA per uso generale, il che capita molto più spesso nei settori di mercato specializzati (per esempio nella progettazione di sistemi integrati o di processori multimediali). Per queste ragioni è importante comprendere la progettazione ISA.

Che cosa rende un ISA un "buon ISA"? Ci sono due fattori principali. Innanzitutto un buon ISA dovrebbe definire un insieme d'istruzioni che può essere implementato efficientemente da tecnologie presenti e future, il che risulta in progetti duraturi e dunque economicamente vantaggiosi. Una cattiva progettazione è più difficile da implementare e potrebbe richiedere molte più porte logiche per realizzare il processore, nonché più memoria per eseguirne i programmi. Potrebbe inoltre rallentare l'esecuzione perché offuscherebbe la possibilità di sovrapporre l'esecuzione di alcune istruzioni, richiedendo espedienti molto sofisticati per raggiungere prestazioni equivalenti. Un progetto che trae vantaggio dalle peculiarità di una particolare tecnologia può rivelarsi un fuoco di paglia e fornire un sola generazione d'implementazioni economicamente vantaggiose, per poi essere sorpassato da ISA più lungimiranti.

In secondo luogo, un buon ISA dovrebbe favorire una compilazione del codice "pulita". La regolarità e la completezza del ventaglio di scelte disponibili per il compilatore costituiscono un tratto importante chè non sempre gli ISA rispettano. Si tratta di proprietà importanti per il compilatore, che altrimenti potrebbe trovarsi in difficoltà nell'operare la scelta migliore tra alternative limitate, in special modo quando alcune scelte apparentemente ovvie non sono permesse dall'ISA. In breve, dato che l'ISA è l'interfaccia tra hardware e software, dovrebbe soddisfare sia i progettisti hardware (essere facile da implementare efficientemente), sia i progettisti software (essere favorevole alla produzione di codice di qualità).

5.1 Panoramica del livello ISA

Per cominciare lo studio del livello ISA chiediamoci che cos'è. Potrebbe sembrare una domanda semplice a cui rispondere, ma solleva più complicazioni di quanto si potrebbe immaginare a prima vista. Qui di seguito trattiamo alcune questioni controverse, dopodiché daremo uno sguardo a modelli di memoria, registri e istruzioni.

5.1.1 Proprietà del livello ISA

In linea di principio, il livello ISA si può definire come l'aspetto che la macchina assume agli occhi di un programmatore in linguaggio macchina. Siccome oramai nessun programmatore (sensato) programma più in linguaggio macchina, ridefiniamo il concetto dicendo che il codice di livello ISA è l'output di un compilatore (senza tener conto, per il momento, delle chiamate di sistema operativo e del linguaggio assemblativo simbolico). Al fine di produrre codice di livello ISA, il progettista del compilatore deve conoscere il modello di memoria, quali registri ci sono, quali sono i tipi di dati e d'istruzioni disponibili, e così via. L'insieme di tutte queste informazioni definisce il livello ISA.

Secondo questa definizione, il fatto che la microarchitettura sia o meno microprogrammata (o che disponga di pipeline, o che sia superscalare, e così via) non fa parte del livello ISA, perché non è visibile al progettista del compilatore. In realtà questa osservazione non è del tutto corretta, poiché alcune di queste proprietà influenzano le prestazioni, il che è visibile a chi scrive il compilatore. Si consideri, per esempio, un progetto superscalare che prevede l'emissione di due istruzioni in successione immediata all'interno dello stesso ciclo se un'istruzione è di tipo intero e l'altra in virgola mobile. Se il compilatore alternasse istruzioni intere a istruzioni in virgola mobile otterrebbe prestazioni visibilmente migliori che non in caso contrario. Così i dettagli dell'operazione superscalare *sono* visibili a livello ISA, e quindi la separazione tra i livelli non è così chiara come potrebbe apparire inizialmente.

Per alcune architetture, ma non sempre, il livello ISA è specificato attraverso un documento formale di definizione, spesso redatto da un consorzio di aziende. Per esempio ARM v7 (la versione 7 di ARM) ha una definizione ufficiale pubblicata da ARM Ltd. Lo scopo di un documento di definizione è di mettere i diversi produttori in grado di costruire macchine capaci di eseguire lo stesso codice, la cui esecuzione dia esattamente gli stessi risultati.

Nel caso di ARM, l'idea è di consentire ai diversi fornitori di chip di produrre chip ARM che funzionino tutti in modo identico, differendo solo per costi e prestazioni. Perché l'idea funzioni, è necessario che i produttori di chip conoscano il funzionamento di un chip ARM (a livello ISA). Di conseguenza il documento di definizione specifica il modello di memoria, i registri presenti, il comportamento delle istruzioni e così via, ma non la microarchitettura.

Tali documenti di definizione contengono **sezioni normative**, che impongono alcuni requisiti, e **sezioni informative**, concepite per aiutare il lettore nella comprensione, ma che non fanno parte della definizione formale. Le sezioni normative usano frequentemente locuzioni quali *deve*, *non deve* e *dovrebbe* rispettivamente per richiedere, proibire e suggerire aspetti dell'architettura. Per esempio, una frase tipo

L'esecuzione di un codice operativo riservato deve causare una trap

asserisce che se il programma esegue un codice operativo che non è definito, deve provocare il sollevamento di una trap e non limitarsi a ignorare l'avvenimento. Un approccio alternativo potrebbe lasciare la scelta libera, nel qual caso la frase diventerebbe

L'effetto dell'esecuzione di un codice operativo riservato è definito dall'implementazione.

Ciò significa che chi scrive il compilatore non può contare su nessun comportamento prestabilito; si lascia a ciascun produttore la libertà di fare la propria scelta. La maggior parte delle specifiche architetturali è corredata da risultati di test sperimentali che verificano la reale conformità di un'implementazione alla specifica corrispondente.

È evidente che la ragione per cui l'ARM v7 ha un documento che ne definisce il livello ISA è fare in modo che tutti i chip ARM possano eseguire lo stesso codice. Per diversi anni non è esistito nessun documento formale di definizione per l'ISA IA-32 (chiamato anche ISA x86), perché Intel non aveva interesse ad agevolare altri produttori a costruire chip compatibili ai suoi. Anzi, Intel è ricorsa alla giustizia nel tentativo di

impedire agli altri produttori di clonare i suoi chip, ma ha perso la causa. Tuttavia, verso la fine degli anni '90, Intel rilasciò le specifiche complete del set di istruzioni IA-32, forse perché sentì di aver sbagliato strada e volle aiutare progettisti e programmatori, oppure perché gli Stati Uniti, il Giappone e l'Europa stavano accusandola di possibili violazioni delle leggi antitrust. Il riferimento per l'ISA, ben scritto, viene continuamente aggiornato ed è disponibile sul sito web per gli sviluppatori Intel (http://developer.intel.com). La versione rilasciata con il Core i7 conta 4161 pagine, a ricordarci ancora una volta che il Core i7 è un computer con un insieme delle istruzioni *complesso*.

Un'altra proprietà importante del livello ISA è che la maggior parte dei processori è dotata di almeno due modalità d'esecuzione. La **modalità kernel** serve a eseguire il sistema operativo e permette l'esecuzione di tutte le istruzioni. La **modalità utente** ha lo scopo di eseguire i programmi applicativi e non consente l'esecuzione di certe istruzioni "delicate" (come quelle che manipolano direttamente la cache). All'interno di questo capitolo focalizziamo l'attenzione in prevalenza sulle istruzioni in modalità utente e sulle loro proprietà.

5.1.2 Modelli di memoria

Tutti i computer suddividono la memoria in celle indirizzate in modo consecutivo. Al momento la dimensione più comune delle celle è di 8 bit, ma in passato sono state usate celle di dimensioni diverse, da 1 a 60 bit (Figura 2.10). Una cella di 8 bit si chiama **byte** (o **ottetto**). La ragione per prediligere i byte è che i caratteri nella tabella ASCII occupano 7 bit, così che un carattere ASCII più un bit di parità (raramente usato) riempiono esattamente un byte. Altre codifiche, come Unicode e UTF-8, usano multipli di 8 bit per rappresentare i caratteri.

In genere i byte vengono raggruppati in parole di 4 byte (32 bit) o di 8 byte (64 bit) ed esistono istruzioni apposite per manipolare intere parole. Molte architetture esigono che le parole siano allineate lungo le loro estremità e così, per esempio, una parola di 4 byte può cominciare agli indirizzi 0, 4, 8, e così via, ma non agli indirizzi 1 o 2. Allo stesso modo una parola di 8 byte può cominciare all'indirizzo 0, 8 o 16, ma non all'indirizzo 4 o 6. L'allineamento delle parole di 8 byte è illustrato nella Figura 5.2.

Figura 5.2 Una parola di 8 byte in una memoria little-endian (a) allineata e (b) non allineata. Alcune macchine richiedono che le parole siano allineate.

Spesso l'allineamento è richiesto perché in tal modo la memoria riesce a funzionare in modo più efficiente. Per esempio il Core i7 preleva dalla memoria 8 byte alla volta per mezzo di un'interfaccia DDR3 che supporta solamente accessi allineati ai 64 bit. Il Core i7 non potrebbe quindi, neanche volendo, referenziare indirizzi di memoria non allineati, perché l'interfaccia di memoria richiede indirizzi multipli di 8.

D'altra parte il requisito dell'allineamento può anche causare difficoltà. Nel Core i7 i programmi possono referenziare parole che cominciano a qualsiasi indirizzo, una proprietà ereditata dall'8088 che aveva un bus dati largo 1 byte (e perciò nessun bisogno di allineare i riferimenti alla memoria lungo multipli di 8 byte). Se un programma del Core i7 legge una parola di 4 byte dall'indirizzo 7, l'hardware deve effettuare un accesso a memoria per recuperare i byte da 0 a 7, più un secondo accesso per recuperare i byte da 8 a 15. Infine la CPU deve estrarre i 4 byte richiesti dai 16 byte letti dalla memoria e assemblarli nel giusto ordine affinché formino una parola di 4 byte. Eseguire regolarmente queste operazioni non porta a velocità fulminanti.

La capacità di leggere parole che cominciano a indirizzi arbitrari richiede nel chip funzionalità logiche supplementari, il che lo rende più grande e più costoso. Gli ingegneri di progetto farebbero volentieri a meno di ciò, e imporrebbero che ogni programma effettuasse riferimenti alla memoria allineati. Il problema è che, ogniqualvolta gli ingegneri chiedono "a chi interessa poter eseguire codice 8088 antiquato che effettua riferimenti sbagliati in memoria?", gli addetti al marketing rispondono lapidari: "ai nostri clienti".

Gran parte dei processori a livello ISA dispone di un solo spazio lineare degli indirizzi, che si estende dall'indirizzo 0 fino a un certo massimo, generalmente 2^{32} o 2^{64} byte. Esistono tuttavia macchine che dispongono di spazi degli indirizzi separati per le istruzioni e per i dati, di modo che il fetch di un'istruzione all'indirizzo 8 proviene da un diverso spazio degli indirizzi rispetto al fetch di un dato all'indirizzo 8. Questo schema è sì più complesso di quello con spazio degli indirizzi unitario, ma presenta due vantaggi. Per prima cosa è possibile referenziare 2^{32} byte di programma e 2^{32} byte di dati usando indirizzi di soli 32 bit. In secondo luogo, poiché le scritture avvengono sempre nello spazio dei dati diviene impossibile sovrascrivere il programma accidentalmente, il che elimina una possibile sorgente di bachi di programma. La separazione dello spazio delle istruzioni da quello dei dati rende inoltre più difficili gli attacchi dei malware, perché il software "maligno" non può accedere al programma (non può nemmeno indirizzarlo).

Si noti che disporre di spazi degli indirizzi separati per dati e istruzioni non è lo stesso che disporre di una cache separata di primo livello. Nel primo caso il numero totale d'indirizzi viene raddoppiato e gli accessi agli indirizzi portano a risultati differenti, a seconda che avvengano nello spazio dei dati o delle istruzioni. Nel caso della cache separata c'è un solo spazio degli indirizzi, soltanto che cache differenti ne contengono porzioni differenti.

Un ulteriore aspetto del modello di memoria a livello ISA è la semantica della memoria. È ragionevole aspettarsi che un'istruzione LOAD, eseguita dopo un'istruzione STORE e sullo stesso indirizzo, restituisca il valore appena memorizzato. Invece sappiamo dal Capitolo 4 che in molte architetture le microistruzioni sono riordinate, e questo

genera il pericolo concreto che la memoria esibisca comportamenti inattesi. La faccenda si complica ulteriormente nel caso di un multiprocessore, dove ogni CPU invia un flusso di richieste di accessi in lettura e scrittura (eventualmente riordinate) alla stessa memoria condivisa.

I progettisti di sistema possono scegliere tra molti approcci risolutivi del problema. A un estremo c'è la possibilità di serializzare tutte le richieste d'accesso a memoria, così che ciascuna viene completata prima che venga emessa la successiva. Questa strategia degrada le prestazioni, ma dà luogo alla semantica di memoria più semplice in assoluto (tutte le operazioni sono eseguite esattamente nell'ordine specificato dal programma).

All'altro estremo c'è il caso in cui non si dà nessun tipo di garanzia. Per forzare un ordine sulla memoria il programma deve eseguire un'istruzione SYNC, che blocca l'emissione di nuove operazioni sulla memoria finché le precedenti non risultino completate. Questa scelta genera un grosso carico di lavoro supplementare per il compilatore, che deve conoscere il funzionamento della microarchitettura in dettaglio, ma assicura ai progettisti hardware la massima libertà nell'ottimizzazione dell'utilizzo della memoria.

Si danno anche modelli di memoria intermedi, in cui l'hardware blocca automaticamente l'emissione di certi accessi a memoria (per esempio quelli che coinvolgono dipendenze RAW o WAR), ma non di altri. Nonostante l'esposizione della microarchitettura a livello ISA generi tutte queste anomalie abbastanza fastidiose (quanto meno per i programmatori che scrivono il compilatore e il linguaggio assemblativo), l'abitudine è molto diffusa. Questa tendenza è dovuta alle implementazioni sottostanti, quali il riordinamento delle microistruzioni, le pipeline profonde, i livelli multipli di cache, e così via. Nel corso di questo capitolo incontreremo altri esempi di effetti così poco naturali.

5.1.3 Registri

Tutti i computer dispongono di qualche registro visibile a livello ISA. Il loro compito è il controllo dell'esecuzione del programma, il contenimento dei risultati temporanei o altro. In genere i registri visibili a livello microarchitetturale, quali il TOS e il MAR nella Figura 4.1, non sono visibili a livello ISA. Tuttavia alcuni di loro, come il program counter e il puntatore allo stack, sono visibili a entrambi i livelli. D'altro canto i registri visibili a livello ISA sono sempre visibili a livello della microarchitettura, perché è lì che sono implementati.

I registri del livello ISA possono essere divisi grosso modo in due categorie: registri specializzati e registri d'uso generale. I primi comprendono il program counter, il puntatore allo stack e altri registri dedicati a funzioni specifiche. I registri d'uso generale sono destinati invece a contenere le variabili locali più importanti e i risultati parziali del calcolo. La loro funzione principale è di consentire un accesso rapido a dati usati ricorrentemente (in pratica per evitare accessi in memoria). Le macchine RISC, dotate di CPU veloci e memorie relativamente lente, hanno in genere almeno 32 registri d'uso generale, ma nei nuovi progetti la tendenza è di incrementarne il numero.

I registri d'uso generale di alcune macchine sono del tutto simmetrici e intercambiabili. Ognuno può fare esattemente le cose che gli altri possono fare. Se i registri sono tutti equivalenti il compilatore può scegliere indifferentemente se usare R1 o R25 per mantenere un risultato temporaneo: la scelta del registro non ha importanza.

I registri d'uso generale di altre macchine possono invece essere in qualche modo specializzati. Per esempio, il Core i7 ha un registro chiamato EDX utilizzabile come registro d'uso generale, ma che è anche destinato a ricevere la metà del prodotto in una moltiplicazione e la metà del dividendo in una divisione.

Anche quando i registri d'uso generale sono completamente intercambiabili, è usuale che i sistemi operativi o i compilatori adottino convenzioni nel modo di utilizzarli. Per esempio alcuni registri possono contenere i parametri di chiamata a una procedura e altri essere usati come registri di lavoro. Se un compilatore memorizza una variabile locale importante in R1 e poi richiama una procedura di libreria che considera R1 un registro di lavoro, al termine dell'esecuzione della procedura R1 potrebbe contenere della spazzatura. Laddove esistono convenzioni su come vadano usati i registri di un sistema, è consigliabile che i compilatori e i programmatori del linguaggio assemblativo le rispettino... per evitare problemi.

Oltre ai registri del livello ISA visibili ai programmi dell'utente, esiste un numero sostanziale di registri specializzati visibile solo in modalità kernel e che controlla cache, memoria, dispositivi di I/O e altre funzionalità hardware della macchina. Possono essere impiegati solo dal sistema operativo, perciò i compilatori e gli utenti non hanno bisogno di riconoscerli.

Il **registro di flag**, detto anche PSW (*Program Status Word*), è una specie di ibrido tra la modalità kernel e quella utente. Questo registro di controllo contiene vari bit di natura eterogenea che sono necessari alla CPU, tra cui i più importanti sono i codici di condizione, che vengono impostati a ogni ciclo dell'ALU e riflettono lo stato del risultato dell'operazione più recente. Alcuni bit tipici che rappresentano codici di condizione sono:

N – posto a 1 dopo risultato negativo;
Z – posto a 1 dopo risultato uguale a zero;
V – posto a 1 se il risultato ha causato un overflow;
C – posto a 1 se il risultato ha causato un riporto oltre l'ultimo bit più significativo;
A – posto a 1 se si è verificato un riporto oltre il terzo bit (riporto ausiliario, si veda di seguito);
P – posto a 1 se il risultato è pari (parità nulla).

I codici di condizione sono importanti perché sono utilizzati dalle istruzioni di confronto e di salto condizionato. Per esempio, l'istruzione CMP sottrae due operandi e imposta il codice di condizione in base alla differenza. Se gli operandi sono uguali, la differenza è zero e il bit di codice di condizione Z viene posto a 1. Una successiva istruzione BEQ (*branch on equal*, "salta se uguali") controlla il bit Z ed effettua il salto se questo ha valore 1.

Il PSW non contiene soltanto codici di condizione, ma il resto del contenuto varia da macchina a macchina. Alcuni campi addizionali molto comuni sono la modalità di macchina (cioè, utente o kernel), i bit di traccia (usati nel debugging), il livello di priorità della CPU e lo stato di attivazione degli interrupt. Spesso il PSW è leggibile in modalità utente, ma alcuni dei suoi campi possono essere scritti solo in modalità kernel (per esempio il bit di modalità kernel/utente).

5.1.4 Istruzioni

La caratteristica principale del livello ISA è l'insieme d'istruzioni macchina che definisce, che specifica ciò che la macchina è in grado di fare. Comprende sempre le istruzioni STORE e LOAD (in forme diverse) finalizzate al trasferimento di dati dai registri alla memoria e viceversa, nonché l'istruzione MOVE per la copia di dati tra registri. Sono sempre presenti le istruzioni aritmetiche, le istruzioni booleane e quelle di confronto dei dati con eventuale salto condizionato dal risultato del confronto. Abbiamo già incontrato alcune istruzioni ISA comuni (vedi la Figura 4.11) e ne incontreremo molte di più in questo capitolo.

5.1.5 Panoramica del livello ISA del Core i7

In questo capitolo analizziamo estesamente tre ISA molto differenti tra loro: IA-32 di Intel, incorporato nel Core i7, la versione 7 dell'architettura ARM, implementata nel SoC OMAP4430, e l'architettura AVR a 8 bit, utilizzata dal microcontrollore ATmega168. L'intento non è quello di fornire una descrizione esaustiva di questi ISA, ma piuttosto di mostrare gli aspetti importanti dell'ISA in generale e la loro variabilità da un particolare ISA all'altro. Cominciamo con il Core i7.

Il Core i7 è frutto dell'evoluzione di molte generazioni, e risalendo il suo albero genealogico possiamo raggiungere alcuni tra i primi processori mai costruiti, come già trattato nel Capitolo 1. Il suo ISA, oltre a mantenere completo supporto per l'esecuzione di programmi scritti per l'8086 e per l'8088 (che avevano lo stesso ISA), contiene addirittura rimembranze dell'8080, un processore a 8 bit diffusissimo negli anni '70. L'8080 fu a sua volta influenzato da vincoli di compatibilità dell'ancora precedente 8008, basato sul 4004, un chip a 4 bit in uso ai tempi in cui i dinosauri vagavano sulla Terra.

Dal punto di vista del software, sia l'8086 sia l'8088 erano macchine a 16 bit effettivi (anche se l'8088 aveva un bus dati a 8 bit). Anche il loro successore, l'80286, era una macchina a 16 bit, e presentava come principale miglioria un maggiore spazio degli indirizzi, sebbene pochi programmi se ne siano mai avvantaggiati, visto che era composto da 16.384 segmenti di 64 KB, invece che da una memoria lineare di 2^{30} byte.

L'80386 fu la prima macchina a 32 bit della famiglia Intel. Tutti i processori successivi (80486, Pentium, Pentium Pro, Pentium II, Pentium III, Pentium 4, Celeron, Xeon, Pentium M, Centrino, Core 2 duo, Core i7 e così via) hanno essenzialmente la stessa architettura dell'80386, nota come **IA-32**, perciò focalizziamo la nostra attenzione su questa architettura. Gli unici cambiamenti architetturali significativi introdotti dopo l'80386 sono stati le istruzioni MMX, SSE e SSE2 nelle versioni più recenti. Si tratta d'istruzioni altamente specializzate e progettate per incrementare le prestazioni di applicazioni multimediali. Un'altra importante estensione è stata la x-86 a 64 bit (nota come x86-64), che porta la dimensione dei calcoli interi e degli indirizzi virtuali a 64 bit. Mentre molte estensioni sono state introdotte da Intel e poi implementate dalla concorrenza, in questo caso il primato spetta ad AMD.

Il Core i7 è dotato di tre modalità operative, due delle quali lo fanno funzionare come un 8088. Nella **modalità reale** tutte le caratteristiche introdotte dopo l'8088 sono disattivate e il Core i7 si comporta come un semplice 8088. Se un programma fa qualcosa di

sbagliato, la macchina si blocca. Se l'Intel avesse progettato gli esseri umani, li avrebbe fatti con un bit per ritornare alla modalità scimpanzé (con gran parte del cervello disattivata, privi di parola, con dimora sugli alberi, predilezione per le banane, e così via).

A un gradino più in alto c'è la **modalità virtuale 8086**, che consente di eseguire vecchi programmi dell'8088 in modo protetto. In questa modalità la macchina è controllata da un vero e proprio sistema operativo. Per eseguire un programma 8088, il sistema operativo crea un ambiente isolato speciale che si comporta come un 8088, con la sola differenza che, se il programma si blocca, la macchina non si arresta, ma passa il controllo al sistema operativo. Quando un utente di Windows apre una finestra MS-DOS, il programma che la esegue gira in modalità virtuale 8086 per proteggere lo stesso Windows da eventuali comportamenti errati dei programmi MS-DOS.

Infine abbiamo la **modalità protetta**, in cui il Core i7 si comporta come un Pentium 4 e non come un 8088 molto costoso. Sono previsti quattro livelli di privilegi, controllati da bit di PSW. Il livello 0, usato dal sistema operativo, corrisponde alla modalità kernel degli altri computer e ha accesso completo alla macchina. Il livello 3, usato per i programmi utente, blocca l'accesso a certe istruzioni critiche e controlla i registri per impedire a un programma utente malintenzionato di mandare in panne l'intera macchina. I livelli 1 e 2 sono usati raramente.

Il Core i7 dispone di uno spazio degli indirizzi enorme: la memoria è divisa in 16.384 segmenti, ciascuno dei quali va dall'indirizzo 0 all'indirizzo $2^{32} - 1$. Tuttavia la maggior parte dei sistemi operativi (compreso UNIX e tutte le versioni di Windows) supporta un solo segmento, così che la maggior parte delle applicazioni vede a tutti gli effetti solo uno spazio degli indirizzi lineare di 2^{32} byte, e talvolta parte di questo spazio è occupata dal sistema operativo. Ogni byte dello spazio degli indirizzi ha un proprio indirizzo, e le parole sono lunghe 32 bit. Le parole sono memorizzate in formato little-endian (il byte meno significativo ha indirizzo più basso).

I registri del Core i7 sono mostrati nella Figura 5.3. I primi quattro registri, EAX, EBX, ECX ed EDX, sono registri a 32 bit di uso più o meno generale, anche se ciascuno ha le sue peculiarità. EAX è il registro aritmetico principale; EBX è usato per contenere puntatori (a indirizzi di memoria); ECX è usato nei cicli; EDX è necessario per le moltiplicazioni e per le divisioni durante le quali, insieme a EAX, contiene prodotti e dividendi di 64 bit. Tutti questi registri sono utilizzabili come registri di 16 bit (nei 16 bit meno significativi) o di 8 bit (negli 8 bit meno significativi) e quindi agevolano la manipolazione di quantità di 16 e 8 bit rispettivamente. I registri a 32 bit furono introdotti con l'80386, come anche il prefisso E, che sta per Esteso.

Anche i tre registri successivi sono in un certo senso generali, ma con maggiori peculiarità. I registri ESI ed EDI servono a contenere puntatori alla memoria, in special modo per le istruzioni hardware di manipolazione di stringhe, dove ESI punta alla stringa sorgente ed EDI alla stringa destinazione. Anche il registro EBP è un puntatore ed è usato generalmente per referenziare l'indirizzo base del record d'attivazione corrente, analogamente a LV nella IJVM. Quando un registro (come EBP) è usato per puntare all'indirizzo base del record d'attivazione locale viene detto per l'appunto **puntatore al record d'attivazione** (*frame pointer*). Infine ESP è il puntatore allo stack.

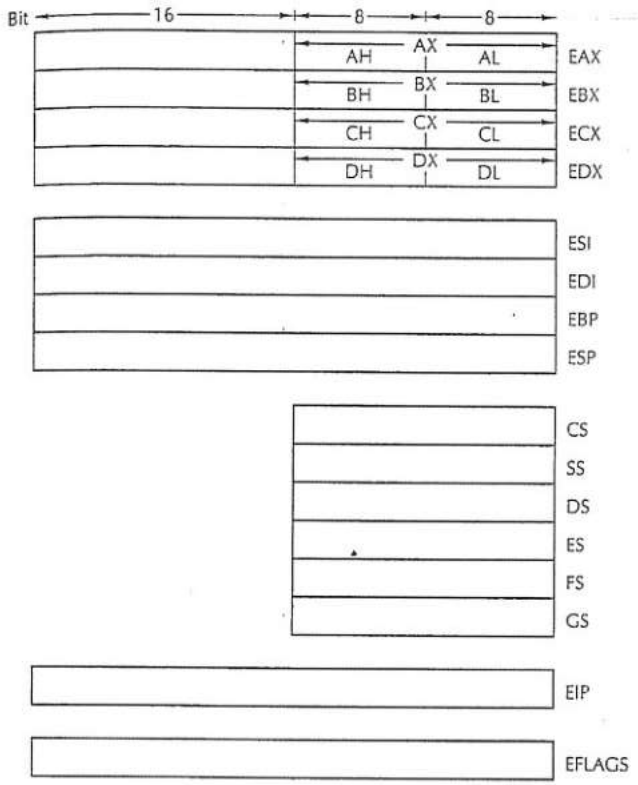


Figura 5.3 Registri principali di Core i7.

Il gruppo successivo di registri, che vanno da CS a GS, è costituito da registri di segmento. Si può dire che sono trilobiti elettronici, antichi fossili giunti fino a noi dall'epoca in cui l'8088 cercava d'indirizzare 2^{20} byte di memoria usando indirizzi da 16 bit. Basti sapere che possono essere tranquillamente ignorati quando il Core i7 è impostato per usare un solo spazio degli indirizzi lineare a 32 bit. Poi c'è l'EIP (*Extended Instruction Pointer*), che è il program counter. Infine troviamo EFLAGS, che è analogo a PSW.

5.1.6 Panoramica del livello ISA dell'OMAP4430 ARM

L'architettura ARM venne introdotta per la prima volta da Acorn Computer nel 1985 ed era ispirata dall'attività di ricerca svolta a Berkeley negli anni '80 (Patterson, 1985; Patterson e Séquin, 1982). L'architettura ARM originale (chiamata ARM2) era a 32 bit e supportava uno spazio degli indirizzi a 26 bit. L'OMAP4430 utilizza la microarchitettura ARM Cortex A9 che implementa la versione 7 dell'architettura ARM, oggetto di

studio in questo capitolo. Per coerenza con il resto del libro parleremo dell'OMAP4430, anche se a livello ISA tutti i progetti basati su ARM Cortex A9 si corrispondono.

La struttura della memoria dell'OMAP4430 è chiara e semplice: la memoria indirizzabile è un vettore di 2^{32} byte. I processori ARM sono bi-endian, in modo da poter accedere alla memoria nei due ordini big-endian e little-endian. L'ordine è specificato in un blocco di memoria di sistema che viene letto immediatamente dopo il reset del processore. Per assicurare una lettura corretta questo blocco deve essere in formato little-endian anche se la macchina va configurata per operare in big-endian.

È importante che l'ISA preveda una limitazione dello spazio degli indirizzi più grande delle necessità implementative, perché in futuro quasi certamente sarà necessario incrementare la dimensione della memoria accessibile dal processore. Lo spazio di indirizzamento a 32 bit dell'ISA ARM sta dando diverse preoccupazioni ai progettisti, perché molti sistemi basati su ARM, come gli smartphone, hanno già più di 2^{32} byte di memoria. Fino a oggi i progettisti hanno aggirato l'ostacolo utilizzando unità di memoria flash a cui si accede tramite un'interfaccia disco che supporta uno spazio di indirizzamento di dimensioni maggiori orientato ai blocchi. Per dare una soluzione a questo problema che potrebbe ridurre le vendite, la società ARM ha recentemente pubblicato la definizione dell'ISA ARM versione 8, con il supporto di uno spazio di indirizzamento a 64 bit.

Uno dei problemi più seri che hanno dovuto affrontare le architetture di successo è stata la limitazione alla quantità di memoria imposta dal livello ISA. Nell'informatica l'unico errore che non si può aggirare è la mancanza di bit. Un giorno i vostri nipoti vi chiederanno come potevano funzionare i computer dei vecchi tempi con indirizzi di soli 32 bit e con soli 4 GB di memoria effettiva, quando un qualsiasi videogioco avrà bisogno di almeno 1 TB solo per partire.

L'ISA ARM è elegante, anche se l'organizzazione dei registri è complicata dal tentativo di semplificare la codifica di alcune istruzioni. L'architettura mappa il *program counter* nel banco dei registri come registro R15, per permettere la creazione di salti con operazioni dell'ALU aventi R15 come registro destinazione. L'esperienza dimostra che l'organizzazione dei registri causa più problemi di quanti ne risolva, ma la vecchia regola della retrocompatibilità la rende imprescindibile.

L'ISA ARM ha due gruppi di registri: 16 d'uso generale da 32 bit e 32 in virgola mobile da 32 bit (se è supportato il coprocessore VFP). I registri d'uso generale hanno nomi che vanno da R0 fino a R15, anche se possono assumere nomi diversi a seconda del contesto. La Figura 5.4 mostra i nomi alternativi e la funzione dei vari registri.

Tutti i registri d'uso generale sono di 32 bit e possono essere letti e scritti da una varietà d'istruzioni di caricamento e memorizzazione. Gli utilizzi indicati nella Figura 5.4 sono in parte dovuti a convenzioni, ma anche giustificati dal modo in cui l'hardware tratta i registri. In generale è poco saggio deviare dalle modalità d'utilizzo elencate nella figura, a meno che non siate "cintura nera" di ARM e sappiate veramente ciò che state facendo. È responsabilità del compilatore o del programmatore assicurarsi che un programma acceda ai registri in maniera corretta e che effettui l'aritmetica appropriata al caso. Per esempio, è molto facile caricare numeri in virgola mobile nei registri d'uso generale e poi effettuare su di loro l'addizione intera, il che produce un risultato che non ha alcun senso, ma che la CPU calcolerà senza batter ciglio se così istruita.

Registri	Nomi alternativi	Funzione
R0 – R3	A1 – A4	Contengono i parametri della procedura che viene invocata
R4 – R11	V1 – V8	Contengono le variabili locali della procedura corrente
R12	IP	Registro chiamata intraprocedura (per chiamate a 32 bit)
R13	SP	Puntatore allo stack
R14	LR	Contiene l'indirizzo di ritorno della funzione corrente
R15	PC	Program counter

Figura 5.4 Registri d'uso generale di ARM v7.

I registri Vx sono usati per memorizzare costanti, variabili e puntatori necessari alle procedure e devono essere memorizzati e ricaricati all'ingresso e all'uscita di ogni procedura, se necessario. I registri Ax sono usati per il passaggio di parametri a procedura per evitare accessi in memoria. Più avanti analizzeremo questo meccanismo in dettaglio.

Quattro registri dedicati sono usati per scopi specifici. Il registro IP serve per aggirare le limitazioni dell'istruzione ARM per le chiamate di funzioni (BL) che non può indirizzare tutti i 2^{32} byte dello spazio degli indirizzi. Se la destinazione di una chiamata è troppo lontana per poter essere espressa, l'istruzione chiamerà un frammento di codice che utilizza l'indirizzo contenuto nel registro IP come destinazione della chiamata alla funzione. Il registro SP indica la posizione corrente della cima dello stack; il suo valore viene aggiornato ogni volta che si effettuano operazioni di push e pop. Il terzo registro a uso speciale è LP, utilizzato dalle chiamate a procedura per mantenere l'indirizzo di ritorno. Come menzionato in precedenza il quarto registro a uso speciale è il PC. La scrittura di un valore in questo registro redirige il prelievo delle istruzioni al nuovo indirizzo depositato nel program counter. Un altro importante registro dell'architettura ARM è il registro di stato del programma, **PSR** (*Program Status Register*), che mantiene lo stato delle precedenti operazioni dell'ALU, tra cui i bit Zero, Negative e Overflow.

L'ISA ARM (nella configurazione che include il coprocessore VFP), dispone anche di 32 registri in virgola mobile di 32 bit. A questi registri si può accedere direttamente, trattandoli come 32 valori in virgola mobile a precisione singola, oppure li si può trattare come 16 valori in virgola mobile da 64 bit, a precisione doppia. La dimensione del registro in virgola mobile è determinata dall'istruzione; in generale, di tutte le istruzioni in virgola mobile sono disponibili le due varianti per lavorare in precisione singola o doppia.

L'architettura ARM è un'**architettura load/store**, ossia le sole operazioni che accedono direttamente alla memoria sono quelle di load e store, utili al trasferimento di dati tra i registri e la memoria. Tutti gli operandi delle istruzioni logiche e aritmetiche devono essere contenuti nei registri o nell'istruzione stessa (non in memoria), e così tutti i risultati devono essere salvati in un registro (non in memoria).

5.1.7 Panoramica del livello ISA dell'ATmega168 AVR

Il nostro terzo esempio è l'ATmega168. A differenza del Core i7 (usato prevalentemente nei desktop e nelle *server farm*) e dell'OMAP4430 (impiegato soprattutto nei telefoni, tablet e altri dispositivi mobili), l'ATmega168 è usato nei sistemi integrati, quali i semafori e le radiosveglie, per il loro controllo, la gestione dei pulsanti, delle luci e delle altre parti che costituiscono l'interfaccia utente. In questo paragrafo forniamo una breve introduzione tecnica dell'ISA AVR dell'ATmega168.

L'ATmega168 ha una sola modalità e nessuna protezione hardware, poiché non si dà mai il caso che esegua programmi di utenti potenzialmente ostili. Anche il modello della memoria è estremamente semplice: c'è uno spazio di memoria di 16 KB per i programmi e uno, distinto, di 1 KB per i dati. Ogni indirizzo fa quindi riferimento a una diversa memoria a seconda se si sta accedendo al programma o ai dati. Gli spazi di programma e dati sono separati al fine di implementare lo spazio dei programmi in una memoria flash e lo spazio dei dati in una SRAM.

L'ATmega168 fa uso di un'organizzazione di memoria a due livelli per offrire una maggior sicurezza. La memoria flash del programma è divisa nella sezione del boot loader e nella sezione delle applicazioni; le dimensioni delle sezioni sono determinate da bit che vengono programmati una volta sola al primo avvio del microcontrollore. Per ragioni di sicurezza solo il codice della sezione del boot loader può aggiornare la memoria flash. Grazie a questa funzionalità un qualsiasi codice (comprese le applicazioni scaricate di terze parti) può essere posizionato nella sezione delle applicazioni con la certezza che non possa intaccare altro codice presente nel sistema (perché il codice sarà eseguito dallo spazio delle applicazioni dal quale non è possibile scrivere sulla memoria flash). Per vincolare maggiomente il sistema, un distributore può firmare digitalmente il codice. Quando il codice è firmato, il boot loader lo caricherà nella memoria flash solo se la firma digitale proviene da un distributore di software approvato. In tal modo, il sistema eseguirà solamente il codice che è stato "benedetto" da un distributore fidato. Questo approccio è piuttosto flessibile e permette di sostituire anche il boot loader, a patto che il nuovo codice sia correttamente firmato. La stessa tecnica viene utilizzata da Apple e TiVo per assicurare che il codice in esecuzione sui propri dispositivi sia al riparo da possibili danneggiamenti.

L'ATmega168 contiene 32 registri a uso generale da 8 bit, chiamati R0 – R31, a cui le istruzioni accedono tramite un campo di 5 bit che specifica il numero del registro. Una peculiarità dei registri dell'ATmega168 è che essi sono anche presenti nello spazio di memoria. Il byte 0 dello spazio dati è equivalente al registro R0 e quando un'istruzione modifica il registro R0 e poi legge il byte 0 di memoria, trova in questa posizione il nuovo valore scritto in R0. Analogamente, il byte 1 nella memoria è R1, e così via fino al byte 31. Questa organizzazione è rappresentata nella figura 5.5.

Agli indirizzi di memoria da 32 a 95, immediatamente sopra ai 32 registri a uso generale, ci sono 64 byte riservati per l'accesso ai registri dei dispositivi di I/O, inclusi i dispositivi interni al SoC.

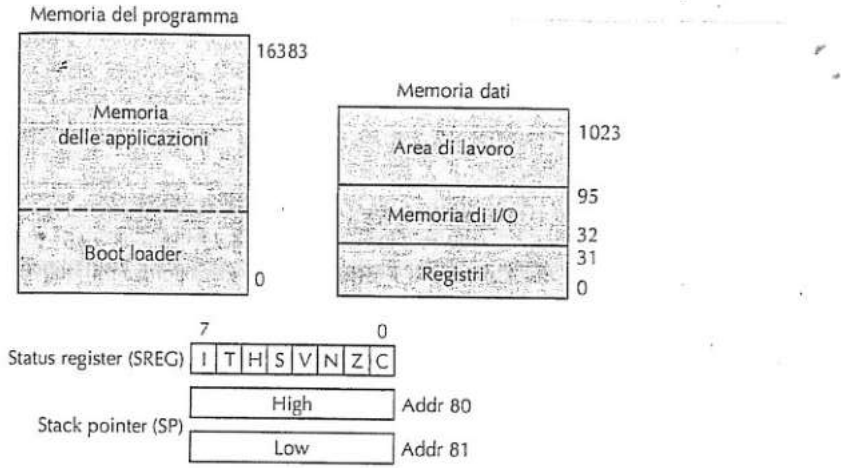


Figura 5.5 Organizzazione dei registri e della memoria sul chip dell'ATmega168.

Oltre ai quattro insiemi di otto registri ciascuno, l'ATmega168 dispone di un piccolo numero di registri specializzati, tra cui quelli più importanti sono indicati nella Figura 5.5. A partire da sinistra, il *registro di stato* contiene nell'ordine: bit di abilitazione degli interrupt, bit ausiliario di riporto, bit di segno, bit di overflow, flag negativo, flag zero, bit di riporto. Tutti questi bit di stato, a eccezione del bit di abilitazione degli interrupt, sono impostati a seguito di un'istruzione aritmetica.

Il bit I del registro di stato consente di attivare e disattivare gli interrupt globalmente. Se il bit I è posto a 0 tutti gli interrupt sono disabilitati, perciò con una sola istruzione di azzeramento di questo bit è possibile disabilitare tutti gli interrupt. Asserendo il bit si permettono eventuali interrupt pendenti e interrupt futuri. A ogni dispositivo è associato un bit di abilitazione degli interrupt. Se il dispositivo è abilitato e il bit di abilitazione globale degli interrupt è asserito, il dispositivo può interrompere il processore.

Lo stack pointer, SP, mantiene l'indirizzo corrente dello spazio dei dati a cui le istruzioni di PUSH e POP accederanno, in maniera simile alle analoghe istruzioni della JVM Java del Capitolo 4. Lo stack pointer si trova all'indirizzo 80 della memoria di I/O. Visto che un singolo byte non è sufficiente per indirizzare i 1024 byte della memoria dati, lo stack pointer è composto da due locazioni di memoria consecutive che formano un indirizzo di 16 bit.

5.2 Tipi di dati

Tutti i computer hanno bisogno di dati. In effetti lo scopo di molti sistemi di calcolo è proprio quello di elaborare dati finanziari, commerciali, scientifici, ingegneristici o di altro tipo. All'interno del computer, i dati devono essere rappresentati in una forma

specifica. A livello ISA sono disponibili una varietà di tipi di dati diversi che esamineremo in seguito.

Una delle questioni chiave è la presenza o meno di supporto hardware per un particolare tipo di dati. Supporto hardware vuol dire che una o più istruzioni si aspettano i dati in un formato particolare e l'utente non è libero di scegliere un formato differente. Per esempio i contabili hanno la singolare abitudine di scrivere i numeri negativi con il segno meno alla destra del numero invece che alla sua sinistra, dove lo mettono gli informatici. Immaginate che il responsabile del centro di calcolo di uno studio di contabilità decida di fare una buona impressione sul suo direttore modificando i numeri di tutti i computer di modo che usino il bit meno significativo (invece del più significativo) come bit di segno. Ciò produrrebbe sicuramente un grande effetto sul direttore, infatti tutto il software inizierebbe a fare degli errori assurdi. L'hardware si aspetta un certo formato d'interi e non funziona correttamente se gli si passa altro.

Considerate ora un altro studio di contabilità, questa volta incaricato dallo Stato di valutare il debito pubblico. Non basterebbe certo l'aritmetica a 32 bit perché i numeri coinvolti sono molto più grandi di 2^{32} (pari a 4 milioni circa). Una possibile soluzione potrebbe essere l'utilizzo di due interi di 32 bit per rappresentare ciascun numero, il che fa 64 bit in tutto. Se la macchina non supporta questo tipo di numeri in **precisione doppia**, si rende necessaria la gestione software di tutta la loro aritmetica, e in tal caso l'ordine delle due parti costituenti il numero non conta, visto che l'hardware non ne viene coinvolto. È questo un esempio di tipo di dati non supportato dall'hardware, per cui quindi non è richiesta una rappresentazione hardware. Nei prossimi paragrafi ci concentreremo sui tipi di dati supportati dall'hardware, per i quali sono quindi richiesti formati specifici.

5.2.1 Tipi di dati numerici

I tipi di dati possono essere divisi in due categorie: numerici e non. Il primo tipo di dati numerici è costituito dagli interi. Ci sono interi di lunghezze diverse, in genere di 8, 16, 32 e 64 bit. Gli interi servono a contare oggetti (per esempio il numero di cacciaviti nel magazzino di un ferramenta), per identificare oggetti (per esempio il numero di conto corrente bancario) e per molto altro. La maggior parte dei computer moderni memorizza gli interi nella notazione binaria in complemento a due, pur se in passato sono stati usati anche altri sistemi. I numeri binari sono analizzati in dettaglio nell'Appendice A.

Alcuni computer supportano sia gli interi senza segno sia quelli con segno. Nel primo caso non c'è alcun bit di segno e tutti i bit rappresentano dati. Questo tipo di dati presenta il vantaggio di disporre di un bit in più e così una parola di 32 bit può contenere un intero senza segno di valore compreso tra 0 e $2^{32} - 1$, estremi inclusi. D'altra parte un intero con segno di 32 bit rappresentato in complemento a due può gestire numeri minori o uguali a $2^{31} - 1$ ma, naturalmente, può gestire anche numeri negativi.

Per rappresentare i numeri che non possono essere espressi con gli interi (per esempio 3,5) si usano i numeri in virgola mobile, trattati nell'Appendice B. Questi sono lunghi 32, 64 o a volte 128 bit. Quasi tutti i computer dispongono d'istruzioni per svolgere aritmetica in virgola mobile, e la maggioranza di questi ha registri separati per contenere operandi interi e operandi in virgola mobile.

Alcuni linguaggi di programmazione, in particolare il COBOL, mettono a disposizione un tipo di dati per i numeri decimali. Le macchine che vogliono favorire l'uso del COBOL spesso supportano i numeri decimali nell'hardware, codificando ogni cifra decimale con 4 bit e quindi impacchettando due cifre decimali in un byte (*Binary Coded Decimal*, "decimali in codifica binaria" o BCD). Purtroppo l'aritmetica dei numeri decimali impacchettati non funziona molto bene, perciò si rendono necessarie delle istruzioni di correzione-decimale-aritmetica. Queste istruzioni hanno bisogno di conoscere il riporto oltre il terzo bit, ed è per questo che i codici di condizione spesso contengono anche un bit di riporto ausiliario. A margine di ciò, l'infame baco del millennio, detto anche *Y2K* (da *Year 2000*), è stato causato dalla decisione dei programmatori COBOL di usare due sole cifre decimali (rappresentate con 8 bit) per rappresentare l'anno, perché più economico rispetto all'uso di quattro cifre decimali. Questione di ottimizzazione.

5.2.2 Tipi di dati non numerici

Sebbene i primi computer si siano guadagnati da vivere macinando numeri, i computer moderni sono spesso impiegati per svolgere applicazioni non numeriche, quali spedizione di email, navigazione su Internet, fotografia digitale, creazione di contenuti multimediali e loro riproduzione. Queste applicazioni richiedono altri tipi di dati spesso supportati dalle istruzioni del livello ISA. Ovviamente i caratteri sono un tipo di dati importante in questo contesto, ma non tutti i computer ne forniscono il supporto hardware. I codici carattere più comuni sono ASCII e UNICODE, che definiscono rispettivamente caratteri di 7 e di 16 bit. Per una loro definizione dettagliata si veda il Capitolo 2.

Non è raro che il livello ISA comprenda istruzioni speciali per la gestione di stringhe di caratteri, ovvero sequenze di caratteri. Le stringhe sono spesso delimitate da un carattere speciale di fine stringa, oppure dotate di un campo lunghezza che può essere usato per ricavare la terminazione della stringa.

Anche i valori booleani sono importanti. Un valore booleano può assumere solo uno dei seguenti valori: vero o falso. In teoria un bit solo basta a rappresentare un dato booleano, associando 0 a falso e 1 a vero (o viceversa). In pratica vengono usati comunque interi byte per rappresentare questi dati, dal momento che i bit individuali non sono indirizzabili direttamente e sono perciò difficili da accedere. Comunemente si adotta la convenzione secondo cui 0 vuol dire falso e tutto il resto rappresenta il valore vero.

L'unica situazione in cui un valore booleano è rappresentato da un solo bit è all'interno di array di booleani, laddove una parola di 32 bit può contenere 32 valori binari. Una struttura di dati siffatta è detta **bit map** ("mappa di bit") e la si ritrova in molti contesti. Per esempio si può usare una bit map per tener traccia dei blocchi liberi di un disco. Se un disco ha n blocchi, la bit map è lunga n bit.

L'ultimo tipo di dati che consideriamo è il tipo puntatore, nient'altro che un indirizzo di macchina. Abbiamo già incontrato i puntatori più volte. Nelle macchine Mic-x i registri `SP`, `PC`, `LV` e `CPP` sono tutti esempi di puntatori. Un'operazione molto comune su tutte le macchine consiste nell'usare i puntatori per accedere a variabili che si trovano a una distanza prefissata da loro, che è precisamente il modo in cui funziona `ILOAD`. Anche se utili, i puntatori sono causa di un gran numero di errori di programmazione che spesso portano a gravi conseguenze. Per questa ragione devono essere usati con estrema cura.

5.2.3 Tipi di dati del Core i7

Il Core i7 supporta gli interi con segno in complemento a due, gli interi senza segno, i numeri decimali in codifica binaria e i numeri in virgola mobile nel formato IEEE 754 (come indicato nella Figura 5.6). A causa delle sue umili origini di macchina a 8/16 bit, il Core i7 gestisce, oltre agli interi a 32 bit, anche gli interi di quelle lunghezze, e mette a disposizione numerose istruzioni per gestire la loro aritmetica, le operazioni booleane e i confronti su di loro. Il processore può opzionalmente essere utilizzato nella modalità a 64 bit che supporta registri e operazioni a 64 bit. Gli operandi non devono essere necessariamente allineati in memoria, ma se lo sono, cioè se gli indirizzi delle parole sono multipli di 4 byte, le prestazioni ne risultano incrementate.

Tipo	8 bit	16 bit	32 bit	64 bit
Interi con segno	×	×	×	× (64 bit)
Interi senza segno	×	×	×	× (64 bit)
Interi decimali in codifica binaria	×			
Numeri in virgola mobile			×	×

Figura 5.6 Tipi di dati numerici del Core i7. I tipi supportati sono indicati dalla crocetta ×. I tipi contrassegnati con "64 bit" sono supportati soltanto nella modalità a 64 bit.

Il Core i7 è ben fornito anche nella manipolazione di caratteri ASCII di 8 bit: ci sono istruzioni speciali per la copia e la ricerca di stringhe di caratteri. Queste istruzioni si applicano sia alle stringhe di lunghezza nota, sia alle stringhe con carattere di fine stringa e sono usate molto di frequente dalle librerie che manipolano le stringhe.

5.2.4 Tipi di dati dell'OMAP4430 ARM

La CPU OMAP4430 supporta un ampio spettro di formati di dati, come evidenziato nella Figura 5.7. Per quanto riguarda i soli interi, può gestire operandi da 8, 16 e 32 bit, con o senza segno. La gestione dei tipi di dato piccoli dell'OMAP4430 è un po' più intelligente di quella del Core i7. Internamente, l'OMAP4430 è una macchina a 32 bit con percorso dati e istruzioni da 32 bit. Un programma può specificare dimensione e segno di un valore da caricare (per esempio, per caricare un byte con segno si usa LDRSB) e il valore viene convertito dalle istruzioni di caricamento nel corrispondente valore a 32 bit. Analogamente, anche le istruzioni di memorizzazione specificano dimensione e segno del valore da scrivere in memoria e fanno accesso solamente alla porzione specificata del registro di input.

Gli interi con segno usano il formato in complemento a due. Sono disponibili operandi in virgola mobile di 32 e 64 che rispettano lo standard IEEE 754. Tutti gli operandi devono essere allineati in memoria.

Non ci sono istruzioni hardware speciali per il supporto di tipi di dati carattere e stringa. La loro manipolazione avviene totalmente via software.

Tipo	8 bit	16 bit	32 bit	64 bit
Interi con segno	×	×	×	
Interi senza segno	×	×	×	
Interi decimali in codifica binaria				
Numeri in virgola mobile			×	×

Figura 5.7 Tipi di dati numerici della CPU ARM OMAP4430. I tipi supportati sono indicati dalla crocetta ×.

5.2.5 Tipi di dati dell'ATmega168

L'ATmega168 dispone di una varietà di tipi di dati molto limitata. Tutti i registri sono lunghi 8 bit, con una sola eccezione, e così anche gli interi sono di 8 bit, così come i caratteri. In pratica il solo tipo di dati le cui operazioni aritmetiche sono realmente supportate dall'hardware è il byte di 8 bit, come si evince dalla Figura 5.8.

Per facilitare l'accesso alla memoria, l'ATmega168 include anche un supporto limitato a puntatori senza segno di 16 bit. I puntatori da 16 bit X, Y e Z sono formati dalla concatenazione delle coppie di registri di 8 bit R26/R27, R28/R29 e R30/R31 rispettivamente. Quando un'istruzione di caricamento utilizza X, Y o Z come operando per l'indirizzo, il processore può incrementare o decrementare il valore a seconda della necessità.

Tipo	8 bit	16 bit	32 bit	64 bit
Interi con segno	×			
Interi senza segno	×	×		
Interi decimali in codifica binaria				
Numeri in virgola mobile				

Figura 5.8 Tipi di dati numerici dell'ATmega168. I tipi supportati sono indicati dalla crocetta ×.

5.3 Formati d'istruzione

Un'istruzione consiste in un opcode (codice operativo), di solito corredato da altre informazioni quali la provenienza degli operandi e la destinazione dei risultati. L'argomento generale che tratta della provenienza degli operandi (cioè il loro indirizzo) è detto **indirizzamento** e verrà illustrato in dettaglio nel prosieguo di questo paragrafo.

La Figura 5.9 mostra i diversi formati delle istruzioni di livello 2. Un'istruzione è dotata di un opcode che ne specifica il comportamento, e può contenere nessuno, uno, due o tre indirizzi.

Alcune macchine hanno tutte le istruzioni della stessa lunghezza, altre dispongono d'istruzioni di lunghezza diversa. Le istruzioni possono essere più corte, altrettanto lunghe o più lunghe della dimensione di parola. La scelta di avere tutte le istruzioni della stessa lunghezza semplifica la loro decodifica, ma spesso implica uno spreco di spazio,

visto che tutte le istruzioni devono essere lunghe quanto la più lunga. Sono anche possibili altri compromessi. La Figura 5.10 illustra alcune delle relazioni che possono intercorrere tra la lunghezza delle istruzioni e la dimensione di parola.

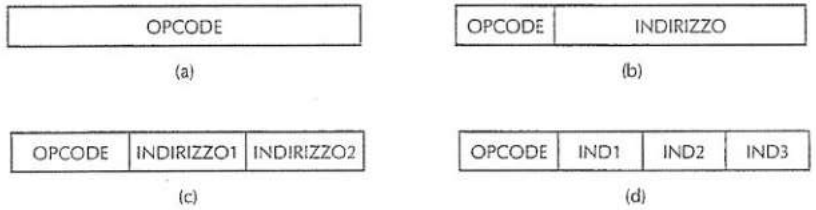


Figura 5.9 Quattro formati d'istruzioni: (a) Istruzione senza indirizzi. (b) Istruzione a un indirizzo. (c) Istruzione a due indirizzi. (d) Istruzione a tre indirizzi.

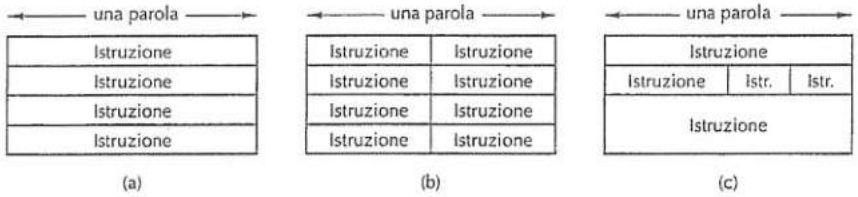


Figura 5.10 Possibili relazioni tra lunghezza delle istruzioni e dimensione di parola.

5.3.1 Criteri progettuali per i formati d'istruzioni

La scelta dei formati d'istruzione prevede che i progettisti di computer tengano conto di molti fattori. Si tratta di una decisione difficile, da non sottovalutare, e che va presa molto presto nel processo di progettazione. Se poi il computer ha successo commerciale è possibile che il suo insieme d'istruzioni sopravviva per altri quarant'anni o più. Una capacità molto preziosa è quella di saper aggiungere nuove istruzioni all'insieme per sfruttare le opportunità che si possono presentare nel corso degli anni, sempre che l'architettura, e l'azienda che l'ha progettata, sopravvivano abbastanza a lungo perché il progetto si affermi.

L'efficienza di un determinato ISA dipende fortemente dalla tecnologia impiegata nell'implementazione del computer. La tecnologia cambia molto velocemente con il passare degli anni e alcune scelte fatte per l'ISA possono rivelarsi (con il senno di vent'anni dopo) infelici. Per esempio un progetto basato sullo stack (come quello dell'IJVM) è molto valido se gli accessi in memoria sono veloci, altrimenti la strada da percorrere è quella di avere molti registri (come fa l'OMAP4430). Se pensate che sia questa una scelta facile da prendere vi invito a procurarvi un foglio e a scriverci le vostre previsioni riguardo (1) la velocità di una CPU tipica e (2) il tempo di accesso a una comune memoria RAM reperibili nei computer tra vent'anni. Piegate il foglio, conservatelo e rileggetelo al momento opportuno. Altrimenti pubblicate oggi stesso le vostre previsioni direttamente su Internet.

Naturalmente anche i progettisti più lungimiranti a volte sbagliano. E anche se non sbagliassero, devono spesso tener in considerazione il breve termine: se il loro progetto elegante di ISA costasse anche solo poco più dello scadente progetto concorrente, l'azienda potrebbe non sopravvivere abbastanza perché il mondo riesca ad apprezzare l'eleganza del loro progetto

A parità di progetto, le istruzioni corte sono preferibili a quelle lunghe. Un programma fatto di n istruzioni di 16 bit occupa metà spazio di memoria rispetto a un programma di n istruzioni di 32 bit. Questo fattore potrebbe risultare sempre meno importante in futuro, a giudicare dalla continua diminuzione del prezzo della memoria, se non fosse che le dimensioni del software metastatizzano con una velocità superiore rispetto alla riduzione del prezzo della stessa.

Minimizzare la dimensione delle istruzioni potrebbe renderle per giunta più difficili da decodificare o da sovrapporre. In conseguenza di ciò, il criterio di ottenere istruzioni di dimensione minima deve essere valutato in base al tempo richiesto per la decodifica e l'esecuzione delle istruzioni.

Un'altra ragione importante per minimizzare la dimensione delle istruzioni, e che assumerà maggior rilievo al crescere delle prestazioni dei processori, è legata all'ampiezza (o larghezza) di banda della memoria (il numero di bit che può trasferire in un secondo). Nell'ultimo decennio abbiamo assistito a una crescita impressionante della velocità dei processori, che non è stata affiancata da un incremento altrettanto significativo dell'ampiezza di banda della memoria. E così le deficienze dei sistemi di memoria, che non sono in grado di trasferire istruzioni e operandi con la stessa celerità con cui vengono processati dalla CPU, diventano sempre più un freno per i processori. La larghezza di banda delle memorie dipende dal loro contenuto tecnologico e dalla qualità della ingegnerizzazione del progetto.

Non solo la memoria principale costituisce un collo di bottiglia, ma anche le cache: se la larghezza di banda di un'istruzione in cache è di t bps e se la lunghezza media di un'istruzione è di r bit, la cache può distribuire al massimo t/r istruzioni al secondo. Si noti che questa quantità è un *limite superiore* al tasso di prestazione del processore, anche se si stanno facendo molti sforzi di ricerca per oltrepassare questa barriera apparentemente invalicabile. Ovviamente il ritmo con cui vengono eseguite le istruzioni (cioè la velocità del processore) è legato alla lunghezza delle istruzioni: istruzioni più corte significano un processore più veloce. Dal momento che molti processori moderni sono in grado di eseguire più istruzioni nello stesso ciclo di clock, si impone la necessità di effettuare fetch multipli a ogni ciclo. Questo aspetto della cache delle istruzioni rende la dimensione delle istruzioni un criterio progettuale importante, le cui conseguenze hanno una forte influenza sulle prestazioni.

Il secondo criterio progettuale è prevedere nel formato delle istruzioni spazio sufficiente a esprimere tutte le operazioni volute. Progettare una macchina con 2^n operazioni, ognuna delle quali esprimibile con meno di n bit, è impossibile. Non ci sarebbe neanche lo spazio sufficiente nel codice operativo per indicare di che istruzione si tratti. La storia

è piena di esempi di progetti folli in cui non sono stati previsti abbastanza codici operativi in eccedenza per fronteggiare aggiunte successive all'insieme d'istruzioni.

Un terzo criterio concerne il numero di bit in un campo degli indirizzi. Si consideri il progetto di una macchina con caratteri di 8 bit e con una memoria principale sufficiente a contenere 2^{32} caratteri. I progettisti potrebbero scegliere d'indirizzare unità di 8, 16, 24 o 32 bit, ma anche di altre dimensioni.

Immaginate che cosa succederebbe se il gruppo di progettazione si dividesse in due fazioni agguerrite, una a sostegno del byte come unità base della memoria, l'altra favorevole ad avere parole di 32 bit. I primi proporrebbero una memoria di 2^{32} byte, numerati da 0 a 4.294.967.295, mentre i secondi proporrebbero una memoria di 2^{30} parole, numerate da 0 a 1.073.741.823.

Il primo gruppo porrebbe l'accento sul fatto che, nell'organizzazione con parole di 32 bit, per confrontare due caratteri un programma non solo dovrebbe fare il fetch delle intere parole contenenti i caratteri, ma dovrebbe poi estrarre i caratteri dalle parole al fine di confrontarli. Così facendo si richiederebbero istruzioni aggiuntive e si sprecherebbe quindi dello spazio. L'organizzazione a 8 bit, d'altro canto, fornirebbe un indirizzo per ogni carattere, rendendo il confronto molto più semplice.

I sostenitori della parola di 32 bit replicherebbero indicando il fatto che la loro proposta comporterebbe 2^{30} indirizzi diversi, raggiungibili con soli 30 bit invece che con i 32 richiesti dall'altra proposta per indirizzare la stessa memoria. Indirizzi più corti consentono istruzioni più corte, che occuperebbero perciò meno spazio e richiederebbero meno tempo in fase di fetch. In alternativa si potrebbe mantenere in uso gli indirizzi di 32 bit e referenziare così una memoria di 16 GB invece di miseri 4.

Questo esempio dimostra che una maggiore risoluzione nell'accesso alla memoria si paga al prezzo d'indirizzi più lunghi e conseguentemente d'istruzioni più lunghe. Il massimo di risoluzione si ha quando l'organizzazione di memoria è tale per cui ogni bit è indirizzabile direttamente (come, per esempio, nel Burroughs B1700). All'altro estremo ci sono le memorie costituite da parole molto lunghe (per esempio, la serie Cyber della CDC aveva parole di 60 bit).

I computer moderni hanno raggiunto un compromesso che, in un certo senso, racchiude il peggio di entrambe le scelte. Da un lato si usano tanti bit quanti sono necessari per indirizzare individualmente tutti i byte, dall'altra spesso si accede alla memoria per leggere una, due, o addirittura quattro parole alla volta. Tanto per fare un esempio, la lettura di un byte di memoria comporta nel Core i7 la lettura di almeno 8 byte e probabilmente di un'intera linea di cache di 64 byte.

5.3.2 Codice operativo espandibile

Nel paragrafo precedente abbiamo visto come una lunghezza degli indirizzi contenuta e una buona risoluzione di memoria si ottengano l'una a discapito dell'altra. In questo paragrafo esaminiamo altri compromessi che coinvolgono i codici operativi e gli indirizzi. Si consideri un'istruzione lunga $(n + k)$ bit, composta da un opcode di k bit e da un solo indirizzo di n bit. Un'istruzione siffatta consente 2^k operazioni diverse e permette d'indirizzare 2^n celle di memoria. In alternativa gli stessi $(n + k)$ bit potrebbero essere

spezzati in $(k - 1)$ bit di opcode e $(n + 1)$ bit d'indirizzo, dimezzando il numero d'istruzioni, ma al contempo raddoppiando la dimensione della memoria raggiungibile, oppure raggiungendo la stessa quantità di memoria ma con risoluzione doppia. Viceversa la scelta di avere opcode di $(k + 1)$ bit e indirizzi di $(n - 1)$ bit rende di più in termini di operazioni, ma al prezzo di un minor numero di celle indirizzabili, o di una peggiore risoluzione nell'accesso alla stessa quantità di memoria. Sulla falsariga degli esempi appena esposti, si possono costruire compromessi molto più sofisticati tra lunghezza degli opcode e lunghezza degli indirizzi.

Introduciamo ora il concetto di **codice operativo espandibile**, che descriviamo mediante un semplice esempio. Si consideri una macchina in cui le istruzioni sono lunghe 16 bit e gli indirizzi 4 bit, come nella Figura 5.11. Questa situazione potrebbe essere ragionevole per una macchina che ha 16 registri (dunque indirizzabili con 4 bit), sufficienti per lo svolgimento di tutte le operazioni aritmetiche. Un'architettura possibile sarebbe quella di comporre ogni istruzione con 4 bit di opcode e tre indirizzi, per un totale di 16 bit.

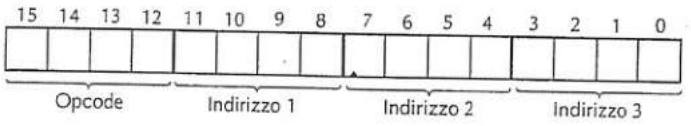


Figura 5.11 Un'istruzione composta da un opcode di 4 bit e da tre campi d'indirizzo di 4 bit.

Tuttavia, se i progettisti avessero bisogno di 15 istruzioni con tre indirizzi, 14 con due indirizzi, 31 con un indirizzo e 16 senza alcun indirizzo, potrebbero usare gli opcode da 0 a 14 per le istruzioni da tre indirizzi e interpretare l'opcode 15 diversamente (Figura 5.12).

L'opcode 15 indica che il codice operativo è contenuto nei bit da 8 a 15 invece che nei bit da 12 a 15. I bit da 0 a 3 e quelli da 4 a 7 formano ancora due indirizzi, come di consueto. Le 14 istruzioni con due indirizzi hanno tutte i 4 bit più significativi posti a 1111, mentre i bit di posto da 8 a 11 progrediscono da 0000 a 1101. Le istruzioni con bit più significativi pari a 1111 e bit da 8 a 11 pari a 1110 oppure a 1111 vengono trattate diversamente, come se il loro opcode fosse contenuto nei bit da 4 a 15. Si dispone così di 32 nuovi opcode. Poiché ne sono richiesti solo 31, l'opcode 111111111111 è interpretato come se il vero opcode fosse costituito dai bit da 0 a 15, il che fornisce altre 16 istruzioni senza indirizzi.

Nel corso della spiegazione abbiamo visto crescere la dimensione dell'opcode sempre più: da 4 bit per le istruzioni con tre indirizzi, a 8 bit per le istruzioni con due indirizzi, a 12 bit per le istruzioni con un indirizzo, per finire a 16 bit per le istruzioni senza indirizzi.

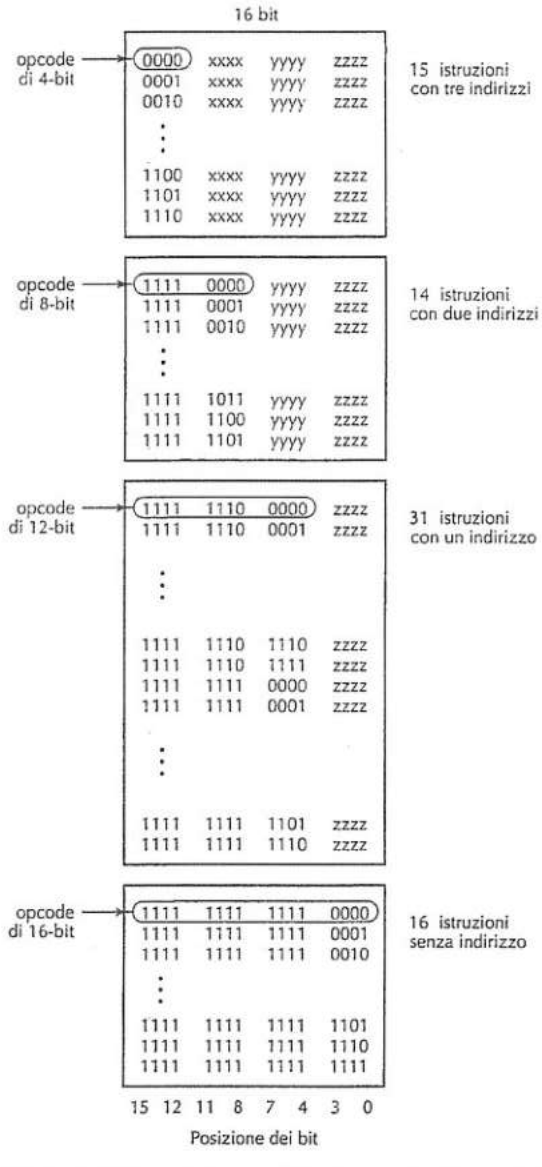


Figura 5.12 Un opcode espandibile che consente di avere 15 istruzioni con tre indirizzi, 14 con due indirizzi, 31 con un indirizzo e 16 senza indirizzo. I campi contrassegnati con *xxxx*, *yyyy* e zzzz sono di 4 bit.

L'idea di codice operativo espandibile evidenzia il compromesso che sussiste tra lo spazio riservato all'opcode e lo spazio per le altre informazioni. Nella pratica i codici operativi espandibili non sono così regolari come nel nostro esempio. La capacità di usare dimensioni di opcode variabili può venire sfruttata in due modi diversi: in primo luogo si può mantenere costante la dimensione delle istruzioni, assegnando gli opcode più corti alle istruzioni che hanno bisogno di più bit per specificare informazioni di altra natura; in secondo luogo, è possibile minimizzare la lunghezza media delle istruzioni, scegliendo gli opcode più corti possibile per le istruzioni comuni e lasciando i più lunghi per le istruzioni d'uso più raro.

Se si spinge l'idea di opcode di lunghezza variabile fino alle sue estreme conseguenze, è possibile minimizzare la lunghezza media delle istruzioni codificandone ciascuna con il minimo numero di bit necessari. Sfortunatamente ciò comporterebbe istruzioni di lunghezze diverse e non allineate nemmeno al byte. Sebbene siano esistiti ISA con questa proprietà (per esempio lo sventurato Intel 432), l'importanza dell'allineamento è così rilevante per la decodifica rapida delle istruzioni che un tale grado di ottimizzazione è quasi certamente controproducente. Nondimeno viene spesso impiegato a livello dei byte.

5.3.3 Formati delle istruzioni del Core i7

I formati delle istruzioni del Core i7 sono molto complessi e irregolari, con fino a sei campi di lunghezza variabile, cinque dei quali sono opzionali. Il loro schema generale è illustrato nella Figura 5.13. Lo stato delle cose è dovuto al fatto che l'architettura si è evoluta per molte generazioni e fin dagli inizi ha incluso alcune scelte inadeguate che non sono più state messe in discussione, nel nome della retrocompatibilità. Come regola generale per le istruzioni con due operandi, se un operando si trova in memoria, l'altro non vi si può trovare, perciò esistono istruzioni per fare la somma di due registri, di un registro e una locazione di memoria, ma non di due locazioni di memoria.

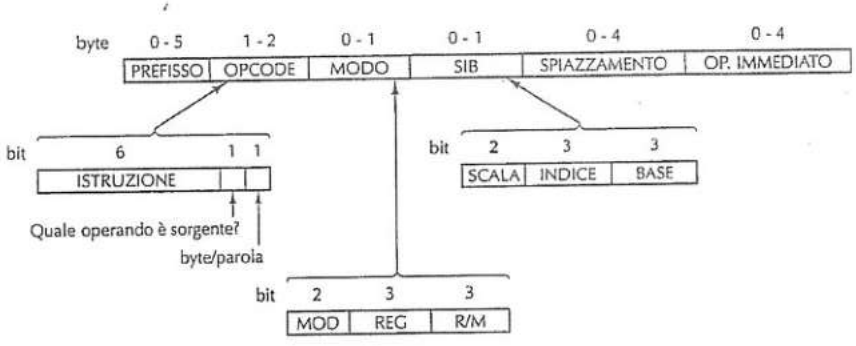


Figura 5.13 Formati d'istruzione di Core i7.

Nelle prime architetture Intel tutti i codici operativi erano lunghi un byte, sebbene venisse usato largamente il concetto di byte prefisso per modificare alcune istruzioni. Un **byte prefisso** è un codice operativo supplementare preposto all'inizio di un'istruzione per cambiarne il comportamento. L'istruzione WIDE dell'IJVM è un esempio di byte prefisso. A un certo stadio dell'evoluzione dei suoi processori, Intel esaurì malauguratamente gli opcode disponibili, così l'opcode 0xFF fu scelto per designare un **codice di escape**, a indicare la presenza di un secondo byte per la specifica dell'istruzione.

I singoli bit degli opcode del Core i7 danno ben poche informazioni riguardo le istruzioni. L'unica struttura riconoscibile nel campo opcode di alcune istruzioni è l'uso del bit meno significativo per distinguere tra istruzioni di un byte o di una parola, e l'uso del bit adiacente per stabilire se l'eventuale indirizzo di memoria è da considerarsi come sorgente o come destinazione dell'operazione. Perciò l'opcode va interamente decodificato per determinare quale classe di operazioni eseguire e, di conseguenza, quanto è lunga l'istruzione corrispondente. Questo si traduce nella difficoltà di ottenere implementazioni con prestazioni elevate, dal momento che si richiede una decodifica impegnativa prima di poter stabilire dove comincia l'istruzione successiva.

In gran parte delle istruzioni che prevedono un operando in memoria, dopo il byte di opcode si trova un byte (di modo) contenente tutte le informazioni relative all'operando; è composto da un campo MOD da 2 bit e da due campi di 3 bit, REG e R/M. Talvolta i primi 3 bit di questo byte sono usati come estensione dell'opcode, dando vita a un opcode di 11 bit in totale. A ogni modo, il campo MOD di due bit consente solo quattro modalità d'indirizzamento degli operandi e almeno un operando è vincolato a trovarsi nei registri; secondo logica ciascuno dei registri EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP dovrebbe poter fungere da operando, ma le regole di codifica proibiscono alcune combinazioni e le riservano a casi particolari. Alcune modalità richiedono un bit supplementare, detto **SIB** (da *Scala*, *Indice* e *Base*), che fornisce un'ulteriore specificazione. Nel complesso questo schema non è certo ideale, bensì un compromesso tra esigenze contrastanti, da un lato la retrocompatibilità e dall'altro la voglia di aggiungere nuove funzionalità originariamente non previste.

Inoltre, alcune istruzioni sono dotate di 1, 2 o 4 byte addizionali che specificano l'indirizzo di memoria (lo spiazzamento) e addirittura di altri 1, 2 o 4 byte atti a contenere una costante (operando immediato).

5.3.4 Formati delle istruzioni dell'OMAP4430 ARM

L'ISA dell'OMAP4430 comprende istruzioni da 16 e 32 bit allineate in memoria. In genere le istruzioni sono molto semplici e ciascuna specifica una singola azione. Una tipica istruzione aritmetica specifica due indirizzi che forniscono gli operandi sorgente e un solo registro come destinazione. Le istruzioni a 16 bit sono versioni più snelle di quelle a 32 bit: fanno le stesse operazioni, ma accettano come operandi solo due registri (il registro di destinazione deve essere uno dei due registri in input) e possono utilizzare soltanto i primi otto registri. I progettisti hanno chiamato "Thumb ISA" questa versione ridotta dell'ISA ARM.

Alcune varianti aggiuntive delle istruzioni permettono di utilizzare, al posto di uno dei registri, delle costanti senza segno di 3, 8, 12, 16 o 24 bit. Nelle istruzioni di caricamento vengono sommati due registri (o un registro e una costante con segno di 8 bit) per specificare l'indirizzo di memoria da leggere. Il dato viene poi scritto in un altro registro specificato.

Il formato delle istruzioni ARM a 32 bit è mostrato nella figura 5.14. Il lettore attento noterà che alcuni formati hanno gli stessi campi (per esempio, LONG MULTIPLY e SWAP). Nel caso dell'istruzione SWAP, il decodificatore riconosce l'istruzione solo quando si accorge che la combinazione dei valori dei campi non è ammessa dall'istruzione MUL. Alcuni formati sono stati aggiunti per le estensioni delle istruzioni e per il Thumb ISA. Quando questo libro è stato scritto il numero di formati delle istruzioni era 21, ma questo numero continua a crescere (fra non molto qualche azienda la pubblicizzerà come "la macchina RISC più complessa del mondo"!). Tuttavia la maggior parte delle istruzioni utilizza ancora i formati mostrati nella figura.

Figura 5.14 Formati delle istruzioni a 32 bit di ARM.

I bit 26 e 27 di ogni istruzione sono quelli inizialmente utilizzati per determinare il formato dell'istruzione e dicono all'hardware dove trovare il resto del codice operativo, se c'è. Per esempio, se i bit 26 e 27 sono entrambi 0, se il bit 25 è 0 (l'operando non è un valore immediato) e se lo shift degli operandi in ingresso non è illegale (a indicare che l'istruzione è una moltiplicazione o una branch exchange), allora entrambe le sorgenti sono registri. Se invece il bit 25 è 1, allora una sorgente è un registro e l'altra una costante compresa tra 0 e 4095. In entrambi i casi la destinazione è un registro. E' disponibile uno spazio di codifica sufficiente per un massimo di 16 istruzioni, tutte correntemente utilizzate.

Nelle istruzioni a 32 bit non è possibile includere una costante di 32 bit. L'istruzione MOVT imposta i 16 bit più significativi di un registro a 32 bit, lasciando che un'altra istruzione imposti i 16 bit rimanenti. La MOVT è l'unica istruzione nel suo formato.

Ogni istruzione da 32 bit ha lo stesso campo di 4 bit nei bit più significativi (da 28 a 31). Questi bit costituiscono il campo condizione e rendono ogni istruzione una *istruzio-*

ne predicato (predicated instruction). Le istruzioni predicato vengono normalmente eseguite nel processore, ma prima di scrivere il risultato in un registro (o in memoria), si controlla la condizione dell'istruzione. Per le istruzioni ARM la condizione è basata sul registro di stato del processore (PSR). Questo registro mantiene le proprietà aritmetiche dell'ultima operazione aritmetica (per esempio, zero, negativo, owerflow, ...). Se la condizione non è verificata il risultato dell'istruzione condizionale viene eliminato.

Il formato delle istruzioni di salto incorpora il valore immediato più grande, utilizzato per calcolare l'indirizzo destinazione per i salti o le chiamate a procedura. Si tratta di un formato speciale, perché è l'unico in cui sono necessari 24 bit di dati per specificare un indirizzo. Per questa istruzione vi è un unico opcode di 3 bit. L'indirizzo corrisponde all'indirizzo destinazione diviso per 4: ciò permette un'estensione dei salti di circa 2^{25}, relativamente all'istruzione corrente.

I progettisti dell'ISA ARM volevano utilizzare completamente ogni combinazione di bit, incluse combinazioni illegali di operandi, per specificare le istruzioni. Questo approccio rende la logica di decodifica delle istruzioni estremamente complicata, ma allo stesso tempo consente al numero massimo di operazioni di essere codificate in istruzioni della lunghezza fissa di 16 o 32 bit.

5.3.5 Formati delle istruzioni dell'ATmega168 AVR

L'ATmega168 AVR dispone di sei semplici formati d'istruzioni, come illustrato nella Figura 5.15. Le istruzioni sono lunghe 2 o 4 byte. Il primo formato consiste in un opcode e due registri operandi, di cui uno è utilizzato solo in input e l'altro sia in input che in output. L'istruzione ADD per i registri, per esempio, utilizza questo formato.

Anche il secondo formato è di 16 bit e comprende un opcode aggiuntivo e un numero di registro di 5 bit. Questo formato permette di aumentare il numero di operazioni codificate dall'ISA, al prezzo di ridurre il numero di operandi a uno solo. Le istruzioni che utilizzano questo formato eseguono un'operazione unaria, ricevendo in ingresso un solo registro e scrivendo l'output sullo stesso registro. Tra gli esempi di queste operazioni vi sono la negazione e l'incremento.

Il terzo formato ha un operando immediato senza segno di 8 bit. Per far spazio a un valore immediato così grande per soli 16 bit, le istruzioni che utilizzano questa codifica possono avere un solo registro operando (utilizzato sia come input che come output) e il registro può appartenere soltanto all'insieme dei registri da R16 a R31 (in modo da poter limitare la codifica del numero di registro a 4 bit). Anche il numero dei bit per l'opcode è dimezzato, permettendo così a solo 4 istruzioni (SUBCI, SUBI, ORI, ANDI) di utilizzare questo formato.

Il quarto formato codifica le istruzioni di load e store, che utilizzano un operando immediato senza segno di 6 bit. Il registro base è un registro fissato che non viene specificato nella codifica dell'istruzione, perché è implicito nell'opcode.

Il quinto e il sesto formato sono utilizzati per i salti e le chiamate di procedura. Il primo utilizza un valore immediato con segno di 12 bit che viene aggiunto al valore del registro PC per calcolare la destinazione del salto. L'ultimo formato espande lo spiazzamento a 22 bit, portando la dimensione dell'istruzione AVR a 32 bit.

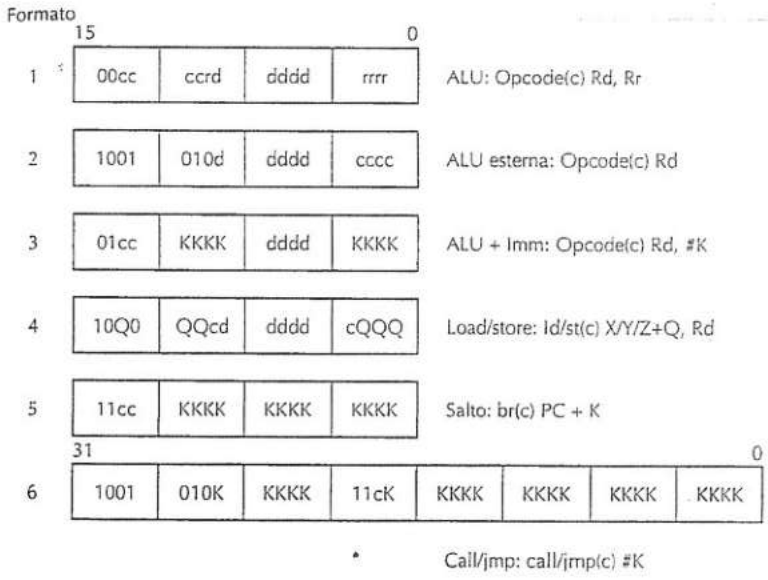


Figura 5.15 Il formato delle istruzioni dell'ATmega168 AVR.

5.4 Indirizzamento

Molte istruzioni contengono operandi e si pone il problema di come specificarne la posizione. L'**indirizzamento** è l'argomento che tratta di queste problematiche che ora affrontiamo.

5.4.1 Modalità d'indirizzamento

Finora ci siamo curati molto poco di come vengono interpretati i bit dei campi d'indirizzo per trovare gli operandi. È giunto ora il momento di approfondire l'argomento della modalità d'indirizzamento. Come vedremo, ci sono diversi modi per implementarla.

5.4.2 Indirizzamento immediato

Il modo più semplice con cui un'istruzione può specificare un operando è di contenere, nel campo riservato al suo indirizzo, l'operando stesso invece che un indirizzo o qualunque altra informazione che ne descriva la posizione. Un operando così specificato si dice **immediato**, poiché viene recuperato automaticamente dalla memoria nello stesso momento in cui viene effettuato il fetch dell'istruzione; dunque è immediatamente disponibile all'uso. La Figura 5.16 mostra una possibile istruzione immediata per il caricamento della costante 4 nel registro R1.

MOV	R1	4

Figura 5.16 Istruzione per il caricamento della costante 4 nel registro R1.

L'indirizzamento immediato ha la virtù di non richiedere un riferimento supplementare in memoria per effettuare il fetch dell'operando. Presenta però lo svantaggio di poter fornire un solo operando per volta; inoltre, l'entità del valore è limitata dalla dimensione del campo. Ciononostante è una tecnica in uso presso molte architetture per la specifica di piccole costanti intere.

5.4.3 Indirizzamento diretto

Un metodo per specificare un operando in memoria è darne l'indirizzo completo. Questa modalità si chiama **indirizzamento diretto**. Al pari di quello immediato, l'indirizzamento diretto presenta alcune limitazioni: l'istruzione accederà sempre alla stessa locazione di memoria. Se da una parte il valore contenuto può cambiare, la locazione non può. Per questa ragione l'indirizzamento diretto serve solo ad accedere a variabili globali il cui indirizzo è noto in fase di compilazione. Nonostante ciò, tale modalità è molto usata, perché molti programmi definiscono variabili globali. Nel seguito considereremo il modo in cui il computer stabilisce quali indirizzi sono immediati e quali diretti.

5.4.4 Indirizzamento a registro

L'indirizzamento a registro è concettualmente analogo all'indirizzamento diretto, ma specifica un registro invece di una locazione di memoria. Si tratta della modalità d'indirizzamento di gran lunga più utilizzata nella quasi totalità dei computer, dato che i registri sono veloci in accesso e hanno indirizzi brevi. Molti compilatori si sforzano di prevedere quali variabili saranno richiamate più spesso (per esempio gli indici di ciclo) e le destinano ai registri.

Questa modalità d'indirizzamento è nota semplicemente come **modalità a registro**. Nelle architetture load/store quali l'OMAP4430 ARM, quasi tutte le istruzioni usano esclusivamente questa modalità d'indirizzamento. L'unico caso in cui non viene usata è quando un operando è trasferito dalla memoria in un registro (istruzione `LOAD`) o da un registro in memoria (istruzione `STORE`). Ma anche in questi casi uno degli operandi è un registro contenente l'indirizzo della parola di memoria in lettura o scrittura.

5.4.5 Indirizzamento a registro indiretto

In questa modalità l'operando in esame proviene o è destinato alla memoria, ma il suo indirizzo non è incorporato nell'istruzione, come nel caso dell'indirizzamento diretto: l'indirizzo è contenuto in un registro. Un indirizzo usato in questa maniera prende il nome di **puntatore**. Un grande vantaggio dell'indirizzamento a registro indiretto è che può referenziare la memoria senza dover necessariamente incorporare un intero indirizzo di memoria all'interno dell'istruzione. Per di più è anche possibile usare diverse parole di memoria in occasione di esecuzioni diverse della stessa istruzione.

Per capire perché può essere utile indirizzare una parola diversa a ogni esecuzione, immaginate un ciclo che passi in rassegna i 1024 elementi di un vettore d'interi e calcoli la loro somma nel registro R1. Altri due registri, poniamo R2 e R3, sono usati fuori dal ciclo per puntare rispettivamente al primo elemento dell'array e all'indirizzo immediatamente successivo all'ultimo elemento dell'array. Se l'array comincia all'indirizzo *A* ed è composto da 1024 interi di 4 byte ciascuno, il primo indirizzo dopo l'array sarà *A* + 4096. Un codice assemblativo tipico per un calcolo del genere su una macchina a due indirizzi è riportato nella Figura 5.17.

```
        MOV R1,#0          ; aggiorna la somma in R1, posto inizialmente a zero
        MOV R2,#A          ; R2 = indirizzo dell'array A
        MOV R3,#A+4096     ; R3 = indirizzo della prima parola dopo A
CICLO:  ADD R1,(R2)        ; recupera l'operando attraverso R2, registro indiretto
        ADD R2,#4          ; incrementa R2 di una parola (4 byte)
        CMP R2,R3          ; abbiamo già finito?
        BLT CICLO          ; se R2 < R3 non abbiamo finito, quindi si continua
```

Figura 5.17 Generico programma assemblativo per il calcolo della somma degli elementi di un array.

In questo semplice programma ci avvaliamo di parecchie modalità d'indirizzamento. Le prime tre istruzioni usano la modalità a registro per il primo operando (la destinazione) e la modalità immediata per il secondo operando (una costante denotata dal simbolo #). La seconda istruzione copia l'*indirizzo* di *A* in R2, non il suo contenuto, il che è specificato all'assembler tramite il simbolo #. Allo stesso modo la terza istruzione copia in R2 l'indirizzo della prima parola oltre l'array.

È interessante notare che il ciclo vero e proprio non contiene nessun indirizzo di memoria. La quarta istruzione usa le modalità a registro e quella a registro indiretto; la quinta usa la modalità a registro e quella immediata; la sesta usa due volte la modalità a registro. L'istruzione `BLT` potrebbe usare un indirizzo di memoria, ma è più probabile che specifichi l'indirizzo del salto con uno spiazzamento di 8 bit relativo a se stessa. Il rifiuto totale di utilizzare indirizzi di memoria ha prodotto un ciclo conciso e veloce. Detto per inciso, questo è un vero programma per il Core i7, laddove abbiamo rinominato le istruzioni e i registri e modificato la notazione per renderlo di più facile lettura. Infatti la sintassi standard del linguaggio assemblativo del Core i7 (MASM) rasenta l'assurdo a causa del retaggio lasciatogli dall'8088, suo antenato diretto.

Facciamo notare che, in teoria, esiste un'altra soluzione per svolgere questa computazione, che non usa l'indirizzamento a registro indiretto: il ciclo avrebbe dovuto contenere un'istruzione per sommare *A* a R1, quale

```
ADD R1, A
```

dopodiché, a ogni iterazione del ciclo, l'istruzione stessa potrebbe venire incrementata di 4, di modo che dopo una sola iterazione diventi

```
ADD R1, A+4
```

e così via fino alla fine del ciclo.

Un programma che modifica se stesso si dice un programma **auto-modificante**. L'idea non fu di altri se non di John Von Neumann e poteva essere sensata sui primi computer, che non disponevano d'indirizzamento a registro indiretto. Oggigiorno i programmi automodificanti sono considerati di pessimo stile e difficili da capire. Inoltre non sono condivisibili da processi diversi e non funzionerebbero nemmeno su macchine con una cache di primo livello separata, laddove la I-cache (cache delle istruzioni) non disponesse di circuiti per i write-back (proprio perché i progettisti dell'hardware avrebbero supposto che i programmi non si automodifichino). Infine, i programmi automodificanti non possono funzionare su macchine che hanno spazi separati di memoria per dati e indirizzi. L'idea del codice auto-modificante è comunque, fortunatamente, scomparsa.

5.4.6 Indirizzamento indicizzato

Spesso è auspicabile poter referenziare una parola di memoria che si trova a un dato spiazzamento dal contenuto di un registro. Abbiamo visto qualche esempio con l'IJVM in cui le variabili locali sono referenziate specificando il loro spiazzamento rispetto a LV. L'indirizzamento alla memoria che si ottiene specificando un registro (in via esplicita o implicita), più uno spiazzamento costante, si dice **indirizzamento indicizzato**.

Gli accessi alle variabili locali dell'IJVM usano un puntatore alla memoria (LV) contenuto in un registro, più un piccolo spiazzamento contenuto nell'istruzione stessa, come mostrato dalla Figura 4.19(a). Tuttavia è possibile anche la soluzione opposta: tenere il puntatore alla memoria nell'istruzione e il piccolo offset in un registro. Per mostrare il funzionamento di questa modalità consideriamo un esempio. Supponiamo di avere due vettori, A e B, di 1024 parole ciascuno e di voler calcolare A_i AND B_i per tutti i componenti e poi fare l'OR di questi 1024 prodotti logici per verificare se c'è almeno una coppia di componenti che non sia nulla. Sarebbe possibile salvare gli indirizzi di A e B in due registri e poi visitare gli array in stretta successione, un elemento per volta, in modo analogo a quanto esposto nella Figura 5.17. Benché sia una soluzione corretta, possiamo fare di meglio e usare uno schema più generale, come illustrato nella Figura 5.18.

Il funzionamento di questo programma è semplice. C'è bisogno di quattro registri:

1. R1 – contiene l'OR cumulativo dei prodotti logici;
2. R2 – l'indice i usato per la visita degli array;
3. R3 – la costante 4096, che è il primo valore di i da non considerare;
4. R4 – un registro di lavoro per mantenere il calcolo di ogni prodotto.


```
        MOV R1,#0       ; aggiorna gli OR in R1, posto inizialmente a zero
        MOV R2,#0       ; R2 = indice i del prodotto corrente: A[i] AND B[i]
        MOV R3,#4096    ; R3 = indice del primo valore da non considerare
CICLO:  MOV R4,A(R2)    ; R4 = A[i]
        AND R4,B(R2)    ; R4 = A[i] AND B[i]
        OR R1,R4        ; OR di tutti i prodotti booleani in R1
        ADD R2,#4       ; i = i + 4 (passi di 1 parola = 4 byte alla volta)
        CMP R2,R3       ; abbiamo già finito?
        BLT CICLO       ; se R2 < R3 non abbiamo finito, quindi si continua
```

Figura 5.18 Generico programma assemblativo per il calcolo dell'OR di A_i AND B_i (A e B sono array di 1024 elementi).

Dopo l'inizializzazione dei registri comincia il ciclo di sei istruzioni. La prima, etichettata come CICLO, effettua il fetch di A_i in R4 con modalità indicizzata: il registro R2 viene sommato all'indirizzo A e la somma è usata come riferimento in memoria, sebbene non venga memorizzata in nessun registro visibile all'utente. La notazione

```
MOV R4, A(R2)
```

indica che la destinazione, R4, è usata in modalità registro, mentre la sorgente usa la modalità indicizzata con offset A e registro R2. Se A vale, poniamo, 124300, l'aspetto reale di questa istruzione macchina è qualcosa di simile a quanto mostrato nella Figura 5.19.

MOV	R4	R2	124300

Figura 5.19 Rappresentazione di MOV R4, A(R2).

Alla prima iterazione del ciclo, R2 vale 0 (è stato così inizializzato), perciò la parola di memoria indicizzata è A_0, all'indirizzo 124300, e questa viene salvata in R4. All'iterazione successiva R2 vale 4, perciò la parola di memoria indicizzata è A_1 all'indirizzo 124304, e così via.

Come avevamo anticipato, in questo esempio l'offset che si trova nell'istruzione è in realtà il puntatore alla memoria e il valore contenuto nel registro è un piccolo intero che viene incrementato durante il calcolo. Questa forma ovviamente richiede che nell'istruzione ci sia un campo offset abbastanza grande da contenere un indirizzo, perciò è meno efficiente dell'altra alternativa. Nondimeno si rivela spesso la scelta migliore.

5.4.7 Indirizzamento indicizzato esteso

Alcune macchine dispongono della cosiddetta modalità d'**indirizzamento indicizzato esteso**, in cui l'indirizzo di memoria è calcolato sommando tra loro il contenuto di due

registri più un offset (opzionale). Un registro funge da base e l'altro da indice. Questa modalità ci sarebbe stata utile nell'esempio precedente, perché avremmo potuto inizializzare R5 con *A* e R6 con *B* fuori dal ciclo. Così facendo avremmo rimpiazzato l'istruzione all'etichetta CICLO e quella successiva con

```
CICLO:    MOV R4,(R2+R5)
          AND R4,(R2+R6)
```

Poter disporre di una modalità d'indirizzamento indiretto tramite la somma di due registri e senza offset sarebbe l'ideale. Alternativamente, anche un'istruzione con un offset di 8 bit costituirebbe un miglioramento rispetto al codice originario, perché potremmo sempre porre entrambi gli offset a 0. D'altra parte, se gli offset fossero di 32 bit, non avremmo guadagnato nulla usando questa modalità. Nella pratica, le macchine che dispongono di questa modalità sono corredate della forma con offset di 8 o 16 bit.

5.4.8 Indirizzamento a stack

Abbiamo già sottolineato che è molto consigliabile rendere le istruzioni macchina quanto più corte possibile. Il limite alla riduzione della lunghezza degli indirizzi equivale a non averne per nulla. Come già visto nel Capitolo 4 le istruzioni senza indirizzi, come l'istruzione IADD, sono possibili in associazione con uno stack. In questo paragrafo analizziamo più da vicino l'indirizzamento a stack.

Notazione polacca inversa

È tradizione in matematica indicare l'operatore in mezzo agli operandi, come in $x + y$, invece che dopo gli operandi, come in $x\,y\,+$. La forma con l'operatore "in" mezzo è detta notazione **infissa**, mentre la forma con l'operatore dopo gli operandi si chiama **postfissa** o anche **notazione polacca inversa**, dal logico polacco J. Lukasiewicz (1958) che ne studiò le proprietà.

La notazione polacca inversa presenta un certo numero di vantaggi rispetto all'infissa in merito alla scrittura di formule algebriche. Per prima cosa qualsiasi formula può essere espressa correttamente senza parentesi. Inoltre la valutazione delle formule in tale notazione si addice particolarmente ai compilatori con stack. Infine, per gli operatori infissi si definisce una precedenza, che è arbitraria e poco gradita. Per esempio sappiamo che $a \times b + c$ vuol dire $(a \times b) + c$ e non $a \times (b + c)$ perché alla moltiplicazione è stata assegnata, in modo arbitrario, una precedenza maggiore rispetto alla somma. Ma lo scorrimento verso sinistra ha precedenza sull'AND booleano? Chi può dirlo? La notazione polacca inversa elimina questa seccatura.

Esistono molteplici algoritmi per convertire formule infisse in notazione polacca inversa. Quella fornita qui è un riadattamento di un'idea di E. W. Dijkstra. Supponete che una formula sia composta dai seguenti simboli: le variabili, gli operatori binari (a due operandi) $+ - \times /$ e le parentesi destra e sinistra. Il simbolo $\perp$ delimita la formula: è anteposto al suo primo simbolo e segue l'ultimo.

La Figura 5.20 mostra un tracciato ferroviario che collega New York alla California, con al centro una diramazione che si diparte in direzione del Texas. Si pensi alla linea da New York alla California come a una linea principale con una ramificazione in basso,

verso il Texas, intesa come una soluzione per lo stoccaggio temporaneo. I nomi e le direzioni non sono importanti: ciò che importa è la distinzione tra la linea principale e il sito alternativo utilizzato come magazzino. A ogni simbolo della formula corrisponde un vagone ferroviario. Il treno procede verso ovest (verso sinistra) e ogni volta che un vagone raggiunge lo scambio deve fermarsi e stabilire se procedere direttamente verso la California o passare prima per il Texas. I vagoni contenenti le variabili procedono sempre diretti per la California, mentre quelli contenenti altri simboli, prima di entrare nello scambio, devono ispezionare il contenuto del vagone più vicino che si trova lungo la diramazione per il Texas.

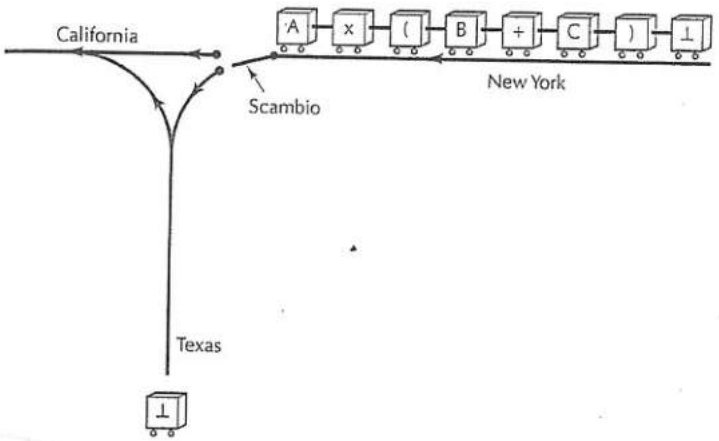


Figura 5.20 Ogni vagone ferroviario rappresenta un simbolo della formula infissa da convertire in notazione polacca inversa.

La tabella nella Figura 5.21 elenca le situazioni che si possono verificare, a seconda del vagone fermo allo scambio e del vagone a lui più vicino lungo il tracciato che porta in Texas. Il primo $\perp$ va sempre in Texas. I numeri della figura rimandano ai seguenti eventi:

1. il vagone allo scambio procede verso il Texas;
2. il vagone che per ultimo ha intrapreso la linea per il Texas inverte il percorso e prosegue per la California;
3. il vagone allo scambio e quello che è entrato più di recente nella diramazione per il Texas vengono entrambi dirottati altrove e scompaiono (cioè si elidono);
4. stop: i simboli presenti al momento in California rappresentano la notazione polacca inversa della formula se letti da sinistra verso destra;
5. stop: è stato rilevato un errore sintattico nella formula originaria.

Vagone entrato più di recente nella linea per il Texas \ Vagone fermo allo scambio	⊥	+	−	×	/	(	)
⊥	4	1	1	1	1	1	5
+	2	2	2	1	1	1	2
−	2	2	2	1	1	1	2
×	2	2	2	2	2	1	2
/	2	2	2	2	2	1	2
(	5	1	1	1	1	1	3

Figura 5.21 Tavola di decisione utilizzata dall'algoritmo per la conversione da notazione infissa a polacca inversa.

Al termine di ogni azione intrapresa viene effettuato un nuovo confronto tra il vagone attualmente fermo allo scambio, che potrebbe essere lo stesso del confronto precedente o il successivo, e il vagone che per ultimo è proseguito per il Texas. Il processo continua finché non si raggiunge l'evento 4. Notate che la linea per il Texas è usata come uno stack, dove l'instradamento di un vagone verso il Texas corrisponde a un'operazione di push, mentre l'inversione di marcia di un vagone che si trova già in Texas e che prosegue per la California corrisponde a un'operazione di pop.

L'ordine delle variabili è lo stesso nelle due notazioni, invece l'ordine degli operatori può cambiare. Nella notazione polacca inversa gli operatori compaiono nell'ordine in cui verranno effettivamente eseguiti durante la valutazione dell'espressione. La Figura 5.22 fornisce diversi esempi di formule infisse e delle stesse in notazione polacca inversa.

Notazione infissa	Notazione polacca inversa
A + B × C	A B C × +
A × B + C	A B × C +
A × B + C × D	A B × C D × +
(A + B) / (C − D)	A B + C D − /
A × B / C	A B × C /
((A + B) × C + D) / (E + F + G)	A B + C × D + E F + G + /

Figura 5.22 Esempi di espressioni in notazione infissa e le stesse in notazione polacca inversa.

Valutazione delle formule in notazione polacca inversa

La notazione polacca inversa è la notazione ideale per la valutazione di una formula da parte di un computer dotato di stack. Una formula è costituita da *n* simboli, ciascuno dei quali è un operando o un operatore. L'algoritmo che si avvale di uno stack per la valutazione di una formula in notazione polacca inversa è molto semplice: scorre la stringa da sinistra verso destra e, quando incontra un operando, lo impila sullo stack. Invece quando incontra un operatore ne esegue l'istruzione corrispondente.

La Figura 5.23 illustra la valutazione della formula

```
(8 + 2 × 5) / (1 + 3 × 2 − 4)
```

nell'IJVM. La stessa formula espressa in notazione polacca inversa è

```
8 2 5 × + 1 3 2 × + 4 − /
```

Passo	Stringa rimanente	Istruzione	Stack
1	8 2 5 × + 1 3 2 × + 4 − /	BIPUSH 8	8
2	2 5 × + 1 3 2 × + 4 − /	BIPUSH 2	8, 2
3	5 × + 1 3 2 × + 4 − /	BIPUSH 5	8, 2, 5
4	× + 1 3 2 × + 4 − /	IMUL	8, 10
5	+ 1 3 2 × + 4 − /	IADD	18
6	1 3 2 × + 4 − /	BIPUSH 1	18, 1
7	3 2 × + 4 − /	BIPUSH 3	18, 1, 3
8	2 × + 4 − /	BIPUSH 2	18, 1, 3, 2
9	× + 4 − /	IMUL	18, 1, 6
10	+ 4 − /	IADD	18, 7
11	4 − /	BIPUSH 4	18, 7, 4
12	− /	ISUB	18, 3
13	/	IDIV	6

Figura 5.23 Uso dello stack per la valutazione di una formula in notazione polacca inversa.

Nella figura abbiamo introdotto le istruzioni `IMUL` e `IDIV`, rispettivamente di moltiplicazione e di divisione. Il numero sulla cima dello stack è l'operando destro, non quello sinistro; questa precisazione è importante perché nella divisione e nella sottrazione l'ordine degli operandi conta (a differenza dell'addizione e della moltiplicazione). In altre parole, `IDIV` è stata definita appositamente in modo tale che, dopo aver fatto il push del numeratore e poi quello del denominatore, la sua esecuzione produca come risultato la divisione corretta. Si noti la semplicità di generazione del codice per l'IJVM: si scorre la notazione polacca inversa della formula e si restituisce un'istruzione per ciascun simbolo. Se il simbolo è una costante o una variabile, si restituisce un'istruzione di push sullo stack; se il simbolo è un operatore, si restituisce l'istruzione che esegue l'operazione.

5.4.9 Modalità d'indirizzamento per istruzioni di salto

Fin qui abbiamo analizzato le istruzioni che operano sui dati. Anche le istruzioni di salto (e le chiamate di procedura) necessitano di modalità d'indirizzamento per specificare

l'indirizzo di destinazione. Le modalità riscontrate finora funzionano grosso modo anche per i salti. L'indirizzamento diretto è di certo un'opzione possibile, secondo cui l'indirizzo di destinazione viene semplicemente riportato per intero all'interno dell'istruzione.

Esistono anche altre modalità d'indirizzamento sensate. L'indirizzamento a registro indiretto consente al programma di calcolare l'indirizzo di destinazione, scriverlo in un registro e quindi effettuare il salto. È la modalità più flessibile perché l'indirizzo di destinazione può essere calcolato durante l'esecuzione, ma è anche la più incline alla generazione di bachi quasi impossibili da scovare.

Un'altra modalità ragionevole è la modalità indicizzata, che specifica un certo offset rispetto all'indirizzo contenuto in un registro. Manifesta le stesse proprietà della modalità a registro indiretto.

Un'ulteriore opzione è l'indirizzamento relativo al PC: un offset (con segno) contenuto nell'istruzione stessa viene sommato al program counter per ottenere l'indirizzo di destinazione. In realtà non è altro che una modalità indicizzata che usa il PC come registro base.

5.4.10 Ortogonalità dei codici operativi e delle modalità d'indirizzamento

Dal punto di vista software, le istruzioni e l'indirizzamento dovrebbero manifestare una struttura regolare, con un numero minimo di formati d'istruzioni. Tale struttura agevolerebbe molto il compilatore a produrre codice di qualità. Gli opcode dovrebbero consentire tutte le modalità d'indirizzamento, laddove sensato, e ogni registro (anche FP, SP e PC) dovrebbe essere utilizzabile da tutte le modalità a registro.

Un esempio di progetto elegante per una macchina a tre indirizzi prevede i formati d'istruzioni di 32 bit della Figura 5.24 e supporta fino a 256 codici operativi. Il formato 1 prevede due indirizzi sorgente e un indirizzo destinazione ed è usato da tutte le istruzioni logico-aritmetiche.

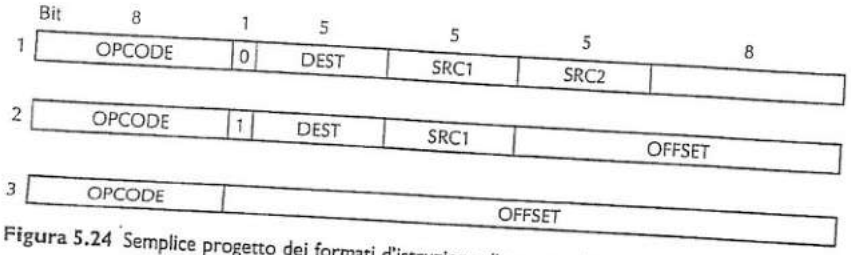


Figura 5.24 Semplice progetto dei formati d'istruzione di una macchina a tre indirizzi.

Il campo finale di 8 bit è inutilizzato e può essere destinato per differenziare ulteriormente le istruzioni. Per esempio, un solo opcode potrebbe specificare l'insieme di operazioni in virgola mobile, e il campo supplementare potrebbe distinguerle l'una dall'altra. Proseguendo, se il bit 23 è asserito, l'istruzione usa il formato 2, in cui il secondo operando non è più un registro, bensì una costante immediata con segno di 13 bit. È questo un formato confacente alle istruzioni LOAD e STORE che referenziano la memoria in modo indicizzato.

Occorrono poche altre istruzioni, come i salti condizionati, facilmente adattabili al formato 3. Per esempio tutte le istruzioni di salto (condizionato), di chiamata di procedura, e così via, potrebbero avere un proprio opcode, il che lascia 24 bit a disposizione per l'offset relativo al PC. Se l'offset è interpretato come numero di parole, allora abbraccia un intervallo di ± 32 MB. Si potrebbe anche riservare un certo numero di opcode per istruzioni LOAD e STORE che necessitano dell'offset lungo del formato 3. Si tratterebbe d'istruzioni non del tutto generali (che per esempio potrebbero agire per definizione solo sul registro R0), ma il loro uso sarebbe alquanto limitato.

Si consideri ora il progetto di una macchina a due indirizzi, mostrata nella Figura 5.25, che può specificare parole di memoria per entrambi gli operandi. Una tale macchina è in grado di sommare una parola di memoria a un registro, oppure sommare tra loro due parole di memoria. Allo stato attuale gli accessi in memoria sono relativamente gravosi, perciò questo progetto non è quasi mai preso in considerazione. Se i progressi tecnologici renderanno in futuro gli accessi in memoria e alla cache più convenienti, risulterà un progetto particolarmente semplice ed efficiente. Le macchine PDP-11 e VAX ottennero un grande successo e dominarono la scena informatica per vent'anni proprio grazie a un progetto di questo tipo.

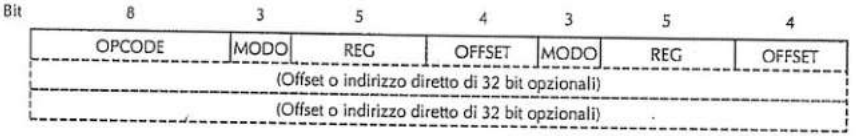


Figura 5.25 Semplice progetto dei formati d'istruzione di una macchina a due indirizzi.

Anche in questo schema gli opcode sono di 8 bit, ma questa volta disponiamo di 12 bit per specificare la sorgente e di altri 12 per la destinazione. Ciascun operando è corredato di 3 bit per la modalità, 5 bit per il registro e 4 per l'offset. Con 3 bit potremmo gestire tutte le modalità, immediata, diretta, a registro, a registro indiretto, indicizzata, a stack, e avremmo ancora spazio per altre due modalità. È questo un progetto semplice ed elegante, assicura una facile compilazione ed è abbastanza flessibile, soprattutto se il program counter, il puntatore allo stack e il puntatore alle variabili locali sono tutti accessibili come registri d'uso generale.

L'unico problema che si presenta è che per l'indirizzamento diretto abbiamo bisogno di un numero di bit maggiore per gli indirizzi. La soluzione del PDP-11 e del VAX era di aggiungere all'istruzione una parola supplementare per ogni indirizzo di operando indirizzato direttamente. Potremmo anche usare una delle due modalità d'indirizzamento libere per una modalità indicizzata con un offset di 32 bit posposto all'istruzione. Così facendo, una somma di due operandi in memoria, entrambi indirizzati direttamente o

con la lunga forma indicizzata, prenderebbe nel caso peggiore 96 bit e userebbe tre cicli di bus (uno per l'istruzione, due per gli indirizzi). Sarebbero inoltre necessari tre cicli aggiuntivi per prelevare i due operandi e per scrivere il risultato. D'altra parte quasi tutte le architetture RISC richiederebbero almeno 96 bit, se non di più, per sommare due parole di memoria qualsiasi, e userebbero almeno quattro cicli di bus, a seconda della modalità di indirizzamento degli operandi.

Le alternative alla Figura 5.26 sono molte. Questo progetto rende possibile l'esecuzione dell'istruzione

```
i= j;
```

con una sola istruzione di 32 bit, ammesso che *i* e *j* si trovino entrambe tra le prime 16 variabili locali. Per variabili oltre la sedicesima bisogna optare però per gli offset di 32 bit. Un'alternativa sarebbe quella di avere un altro formato con un solo offset di 8 bit invece di due offset di 4 bit, più una regola per stabilire se si riferisce alla sorgente o alla destinazione, ma non a entrambe. Le possibilità sono illimitate così come i compromessi, e i progettisti di una macchina devono giocare su molti fattori per ottenere un buon risultato.

5.4.11 Modalità d'indirizzamento del Core i7

Le modalità d'indirizzamento del Core i7 sono assai irregolari e cambiano a seconda che una certa istruzione sia in modalità di 16, 32 o 64 bit. Non affronteremo le modalità di 16 e 64 bit, dato che quella di 32 è già abbastanza intricata. Il Pentium prevede le modalità immediata, diretta, a registro, a registro indiretto, più una speciale per l'indirizzamento di elementi di un array. Il problema è che non tutte le modalità si applicano a tutte le istruzioni, e non tutti i registri sono utilizzabili da tutte le modalità, il che accresce la difficoltà delle mansioni del compilatore e porta a un codice più scadente.

Il byte MODE della Figura 5.13 controlla le modalità d'indirizzamento. Uno degli operandi è specificato in combinazione dai campi MOD e R/M, mentre l'altro è sempre un registro, specificato dal valore del campo REG. La Figura 5.26 elenca le 32 combinazioni che possono essere specificate congiuntamente dai 2 bit di MOD e dai 3 bit di R/M. Per esempio quando entrambi i campi valgono zero l'operando viene letto all'indirizzo di memoria contenuto nel registro EAX.

Le colonne 01 e 10 hanno a che fare con modalità in cui il registro è sommato a un offset di 8 o 32 bit posposto all'istruzione. Se l'offset è di 8 bit, viene portato a 32 (estensione del segno) prima della somma. Per fare un esempio, un'istruzione ADD, con R/M = 011, MOD = 01 e offset pari a 6, calcola la somma di EBX più 6 e usa il risultato come indirizzo della parola di memoria contenente un operando. EBX non ne risulta modificato.

La colonna MOD = 11 consente di specificare due registri alla volta: il primo è usato per le istruzioni di una parola, il secondo per le istruzioni di un byte. Osservate la parziale irregolarità della tabella: per esempio non c'è modo di usare EBP in modalità indiretta, né di usare ESP come base di un offset.

	MOD			
R/M	00	01	10	11
000	M[EAX]	M[EAX + OFFSET8]	M[EAX + OFFSET32]	EAX o AL
001	M[ECX]	M[ECX + OFFSET8]	M[ECX + OFFSET32]	ECX o CL
010	M[EDX]	M[EDX + OFFSET8]	M[EDX + OFFSET32]	EDX o DL
011	M[EBX]	M[EBX + OFFSET8]	M[EBX + OFFSET32]	EBX o BL
100	SIB	SIB con OFFSET8	SIB con OFFSET32	ESP o AH
101	Diretto	M[EBP + OFFSET8]	M[EBP + OFFSET32]	EBP o CH
110	M[ESI]	M[ESI + OFFSET8]	M[ESI + OFFSET32]	ESI o DH
111	M[EDI]	M[EDI + OFFSET8]	M[EDI + OFFSET32]	EDI o BH

Figura 5.26 Modalità d'indirizzamento a 32 bit del Core i7. M[*x*] è la parola di memoria all'indirizzo *x*.

In alcune modalità c'è un byte aggiuntivo, detto SIB, che segue il byte MODE (Figura 5.13). Il byte SIB specifica due registri e un fattore di scala. Quando il byte SIB è presente, l'indirizzo dell'operando si calcola moltiplicando l'indirizzo indice per 1, 2, 4 o 8 (a seconda della scala), sommando il risultato al registro base; infine, c'è la possibilità che il tutto vada sommato a uno spiazzamento di 8 o 32 bit, secondo quanto specidicato da MOD. Pressoché tutti i registri possono essere usati come indice o come base.

Le modalità SIB sono utili per accedere a elementi di un array; considerate per esempio l'istruzione Java

```
for (i = 0; i < n; i++) a[i] = 0;
```

dove a è un array d'interi di 4 byte locale alla procedura corrente. In genere EBP è usato per puntare alla base del record d'attivazione dello stack contenente le variabili e gli array locali, come mostrato nella Figura 5.27. Il compilatore potrebbe mantenere *i* in EAX e, per accedere ad a[i], userebbe una modalità SIB in cui l'indirizzo dell'operando si otterrebbe sommando 4 × EAX a EBP e a 8. L'istruzione Java potrebbe così assegnare un valore ad a[i] con una sola istruzione di STORE.

Il gioco vale la candela? Difficile a dirsi. Sicuramente questa istruzione, se usata propriamente, fa risparmiare qualche ciclo. La frequenza del suo utilizzo dipende dal compilatore e dall'applicazione. Il problema è che questa istruzione occupa dello spazio sul chip che si sarebbe potuto impiegare in modo diverso se non fosse esistita. Per esempio la cache di primo livello avrebbe potuto essere più grande, o il chip più piccolo, consentendo forse una velocità di clock leggermente superiore.

Sono questi i compromessi sui quali i progettisti sono spesso chiamati a pronunciarsi. Di norma vengono eseguite numerose simulazioni prima dell'implementazione sul silicio, ma le simulazioni sono indicative solo se si ha già una buona idea di quel che sarà il carico di lavoro. Si può scommettere sul sicuro che i progettisti dell'8088 non abbiano incluso i browser di Internet nel loro insieme di prova. Ciononostante, alcuni discendenti di quel prodotto sono oggi usati principalmente per la navigazione del Web

e così le scelte fatte vent'anni fa potrebbero rivelarsi completamente inadeguate per le applicazioni correnti. Ancora una volta in nome della retrocompatibilità, quando una caratteristica entra in un progetto è impossibile estrometterla.

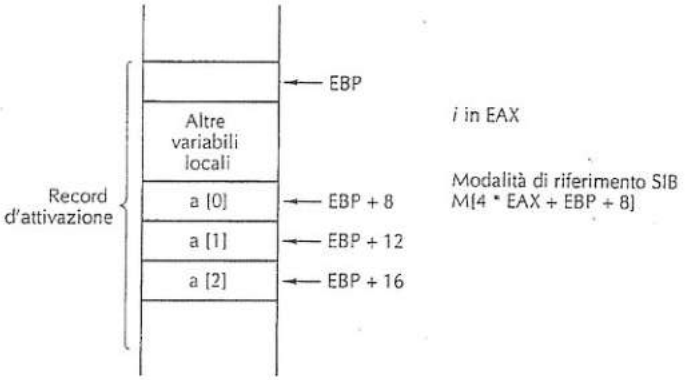


Figura 5.27 Accesso ad a[i].

5.4.12 Modalità d'indirizzamento dell'OMAP4430

Nell'ISA dell'OMAP4430 tutte le istruzioni si avvalgono dell'indirizzamento immediato o a registro, eccezion fatta per quelle che indirizzano la memoria. Nella modalità a registro sono i 5 bit a specificare quale indirizzo usare. Nella modalità immediata, i dati sono contenuti in una costante (con segno) di 12 bit. Non ci sono altre modalità disponibili per le istruzioni logico-aritmetiche o simili.

Ci sono due tipi d'istruzioni che indirizzano la memoria: le istruzioni di load (LDR) e quelle di store (STR). Le istruzioni LDR e STR possono indirizzare la memoria in tre modalità diverse: nella prima si usa la somma di due registri come valore d'indirizzamento indiretto; la seconda calcola l'indirizzo come somma di un registro base e di uno spiazzamento con segno di 13 bit; la terza calcola l'indirizzo come somma del program counter e di uno spiazzamento con segno di 13 bit. Quest'ultima modalità, chiamata indirizzamento relativo al program counter, è utile per caricare costanti memorizzate con il codice del programma.

5.4.13 Modalità d'indirizzamento dell'ATmega168 AVR

L'ATmega168 presenta una struttura d'indirizzamento abbastanza regolare. Ci sono quattro modalità basilari. La prima è la modalità di indirizzamento a registro, in cui l'operando è un registro. I registri sono utilizzati sia come sorgente che come destinazione. La seconda è la modalità di indirizzamento immediato, in cui un valore immediato senza segno di 8 bit viene codificato nell'istruzione. Le restanti modalità possono essere utilizzate soltanto dalle istruzioni di load e store. La terza modalità è l'indirizzamento diretto. In questa modalità l'operando è in memoria a un indirizzo contenuto nell'istruzione stessa. Nelle istruzioni a 16 bit l'indirizzamento diretto è limitato a 7 bit (possono quindi essere specificati solo gli indirizzi da 0 a 127). L'architettura AVR definisce anche un'istruzione a 32 bit, in modo da poter ospitare un indirizzo di 16 bit, garantendo così il supporto per 64 KB di memoria. La quarta modalità è l'indirizzamento indiretto basato su un registro. In questo caso il registro contiene un puntatore all'operando. Visto che i registri normali hanno una dimensione di 8 bit, le istruzioni di load e store utilizzano coppie di registri per specificare indirizzi di memoria a 16 bit, così da poter indirizzare fino a 64 KB di memoria. L'architettura supporta l'utilizzo di tre coppie di registri, chiamate X, Y e Z e formate rispettivamente dai registri R26/R27, R28/R29 e R30/R31. Per caricare un indirizzo nel registro X, per esempio, il programma dovrà caricare un valore di 8 bit nei registri R26 e R27, utilizzando due istruzioni.

5.4.14 Analisi delle modalità d'indirizzamento

Abbiamo dunque introdotto un certo numero di modalità d'indirizzamento. Quelle utilizzate da Core i7, OMAP4430 e ATmega168 sono riassunte nella Figura 5.28. Si tenga conto però, come già sottolineato, che non tutte le istruzioni possono avvalersi della gamma completa delle modalità d'indirizzamento.

Modalità d'indirizzamento	Core i7	OMAP4430 ARM	ATmega168 AVR
Immediata	×	×	×
Diretta	×		×
A registro	×	×	×
A registro indiretta	×	×	×
Indicizzata	×	×	
Indicizzata estesa		×	

Figura 5.28 Modalità d'indirizzamento a confronto.

Nella pratica un ISA, per essere efficace, non ha bisogno di molte modalità d'indirizzamento. Oggigiorno la maggior parte del codice scritto a questo livello viene generato dai compilatori (a parte forse nel caso dell'ATmega168), perciò gli aspetti più importanti delle modalità d'indirizzamento di un'architettura sono la possibilità di scegliere tra poche alternative chiare, il cui costo (in termini di tempo di calcolo e dimensioni del codice) sia valutabile prontamente. Ciò si traduce in una presa di posizione drastica: una macchina dovrebbe offrire ogni scelta possibile, oppure una sola. Ogni via di mezzo costringe il compilatore a fare delle scelte per cui non è abbastanza preparato o sofisticato.

Le architetture più eleganti dispongono in genere di un numero estremamente limitato di modalità d'indirizzamento e pongono vincoli molto stringenti al loro uso. Nella pratica, alla maggior parte delle applicazioni basta poter disporre delle modalità immediata, diretta, a registro e indicizzata. Inoltre, ogni registro dovrebbe essere utilizzabile

al pari di ogni altro, compresi il puntatore al record d'attivazione, il puntatore allo stack e il program counter. Modalità d'indirizzamento più complicate possono sì ridurre il numero d'istruzioni, ma al costo dell'introduzione di sequenze di operazioni non facilmente eseguibili in modo parallelo ad altre operazioni sequenziali.

Abbiamo così completato lo studio dei compromessi possibili tra opcode e indirizzi da una parte, e forme d'indirizzamento dall'altra. Ogniqualvolta vi avvicinate a un nuovo computer dovreste esaminarne le istruzioni e le modalità d'indirizzamento, non solo per scoprire quali sono disponibili, ma anche per capire le scelte fatte e le conseguenze che avrebbero avuto le scelte alternative.

5.5 Tipi d'istruzioni

Anche se possono differire nei dettagli, le istruzioni del livello ISA possono essere suddivise approssimativamente in una mezza dozzina di gruppi facilmente rintracciabili in tutte le macchine. Oltre a questi, ciascun computer dispone di una manciata d'istruzioni insolite, aggiunte per motivi di compatibilità con modelli precedenti o a seguito di un'idea brillante di un progettista, o perché un'agenzia governativa ha pagato il produttore perché la includesse nel progetto. Proveremo a dar conto brevemente di tutte le categorie più comuni, senza pretendere di essere esaustivi.

5.5.1 Istruzioni di trasferimento dati

Poter copiare dati da una locazione all'altra è fondamentale per tutte le operazioni. Per copia intendiamo la creazione di un nuovo oggetto costituito da una sequenza di bit identica all'originale. L'uso tecnico della parola "trasferimento" è diverso da quello del linguaggio comune. Quando diciamo che Mario Cervi si è trasferito da New York alla California, non intendiamo dire che è stata creata una copia identica di Mario Cervi in California e che l'originale si trova ancora a New York. Quando diciamo che il contenuto della locazione di memoria 2000 è stato trasferito in un qualche registro, intendiamo sempre che vi è stata creata una copia fedele e che l'originale si trova ancora indisturbato nella sua locazione 2000. Le istruzioni di trasferimento dati dovrebbero chiamarsi piuttosto istruzioni di "duplicazione dei dati", ma oramai la locuzione "trasferimento di dati" è entrata nell'uso.

Ci sono due ragioni per copiare i dati da una locazione a un'altra, una delle quali è fondamentale: l'assegnamento di valori a variabili. L'assegnamento

```
A = B
```

si implementa con la copia del valore all'indirizzo di memoria *B* nella locazione di memoria *A*, come specificato dal programmatore. La seconda motivazione per la copia dei dati è prepararli a un accesso e a un uso efficienti. Si è già visto come molte istruzioni abbiano accesso solo alle variabili contenute nei registri. Poiché i dati possono provenire da due sorgenti (la memoria o i registri) ed essere destinati a due locazioni (in memoria o nei registri), ci sono quattro tipi diversi di trasferimenti possibili. Alcuni computer dispongono di quattro istruzioni diverse, una per ogni situazione, altri hanno una sola istruzione per tutte le situazioni. Altri ancora usano LOAD per trasferire dalla memoria verso i registri, STORE dai registri alla memoria, MOVE per i trasferimenti tra registri e non dispongono di alcuna istruzione per la copia tra locazioni di memoria.

Le istruzioni per il trasferimento dati devono specificare in qualche maniera la quantità di dati da trasferire. In alcuni ISA esistono istruzioni per trasferire quantità di dati variabili da un byte fino all'intera memoria. Sulle macchine con parole di lunghezza fissa l'unità di trasferimento consueta è proprio la parola. Il trasferimento di quantità maggiori o minori di una parola va svolto via software per mezzo di operazioni di scorrimento e fusione. Alcuni ISA prevedono funzionalità aggiuntive per la copia sia di quantità più piccole di una parola (solitamente per multipli di byte), sia di più parole per volta. La copia di più parole è tanto più delicata quanto più grande è il massimo numero di parole trasferibili, perché un'operazione di questo genere può impiegare molto tempo e c'è il rischio che venga interrotta nel bel mezzo dell'esecuzione. Alcune macchine con lunghezza di parola variabile dispongono d'istruzioni che specificano solo gli indirizzi sorgente e destinazione, senza indicare la quantità da trasferire, così la copia dei dati continua fino al raggiungimento di un marcatore di fine dati.

5.5.2 Operazioni binarie

Le operazioni binarie sono quelle che producono un risultato dalla combinazione di due operandi. Tutti gli ISA hanno istruzioni per l'addizione e la sottrazione d'interi. Anche la moltiplicazione e la divisione d'interi è pressoché standard. È abbastanza ovvio che un computer sia dotato d'istruzioni aritmetiche e non ci dilunghiamo in merito.

Un altro insieme di operazioni binarie comprende le istruzioni booleane. Benché esistano 16 funzioni booleane in due variabili, ben poche macchine (forse nessuna) dispongono d'istruzioni per tutte e 16. In genere sono disponibili AND, OR e NOT, qualche volta anche XOR (OR ESCLUSIVO), NOR e NAND.

Un uso importante di AND è l'estrazione di bit da una parola. Considerate per esempio una macchina con parole di 32 bit che possono ospitare quattro caratteri di 8 bit. Se si vuole separare il secondo carattere dagli altri tre al fine di visualizzarlo sullo schermo, è necessario creare una parola che lo contenga negli 8 bit più a destra e che contenga tutti zero nei 24 bit rimanenti, perciò detta parola **giustificata a destra**.

L'estrazione del carattere avviene facendo l'AND della parola con una costante, detta **maschera**. Il risultato di questa operazione è che tutti i bit indesiderati vengono posti a zero, vale a dire mascherati, come mostrato nello schema seguente

```
10110111 10111100 11011011 10001011   A
00000000 11111111 00000000 00000000   B (maschera)
00000000 10111100 00000000 00000000   A AND B
```

Quindi il risultato viene fatto scorrere di 16 bit verso destra in modo da isolare il carattere da estrarre all'estremità destra della parola.

Un uso importante di OR è quello di impacchettare bit in una parola, che è l'operazione inversa dell'estrazione. Per cambiare gli 8 bit meno significativi di una parola da 32 bit senza modificare gli altri, per prima cosa mascheriamo gli 8 bit indesiderati, dopodiché il nuovo carattere è inserito facendone l'OR, come mostrato di seguito.

```
10110111 10111100 11011011 10001011 A
11111111 11111111 11111111 00000000 B (maschera)
10110111 10111100 11011011 00000000 A AND B
00000000 00000000 00000000 01010111 C
10110111 10111100 11011011 01010111 (A AND B) OR C
```

L'operazione AND tende a rimuovere i bit 1, perché il suo risultato non contiene mai più bit 1 di quanti ce ne siano in ciascun operando. L'operazione OR tende a inserire bit 1 perché il risultato contiene sempre almeno tanti bit 1 quanti ce ne sono nell'operando con numero maggiore. L'operazione XOR, d'altra parte, ha un comportamento simmetrico, perché tende a inserire in media tanti bit 1 quanti ne rimuove. Si tratta di una proprietà utile in alcune situazioni, come nella generazione di numeri casuali.

Molti dei computer odierni supportano anche un insieme d'istruzioni in virgola mobile, più o meno corrispondenti alle operazioni aritmetiche sugli interi. Gran parte delle macchine mette a disposizione almeno due formati di numeri in virgola mobile, il più corto per esigenza di velocità, l'altro per quelle situazioni particolari in cui si richiede una precisione maggiore. Ci sono molte varianti possibili per i formati in virgola mobile, eppure c'è uno standard che è oggi quasi universalmente accettato: IEEE 754. I numeri in virgola mobile, e il formato IEEE 754, sono presentati nell'Appendice B.

5.5.3 Operazioni unarie

Le operazioni unarie prendono in ingresso un operando e restituiscono un risultato. Queste istruzioni sono in genere più corte di quelle binarie, visto che contengono un indirizzo in meno, sebbene spesso richiedano la specifica di altre informazioni.

Le istruzioni per lo scorrimento (*shift*) o la rotazione del contenuto di una parola si rivelano molto utili, perciò sono spesso presenti in più varianti. Le operazioni di scorrimento possono spostare i bit verso destra o verso sinistra, e i bit che fuoriescono dalla parola si intendono perduti. Le rotazioni sono scorrimenti in cui i bit che escono da un'estremità della parola riappaiono all'altra estremità. Illustriamo con un esempio la differenza tra le due operazioni.

```
00000000 00000000 00000000 01110011  A
00000000 00000000 00000000 00011100  A scorso verso destra di 2 bit
11000000 00000000 00000000 00011100  A ruotato verso destra di 2 bit
```

Scorrimenti e rotazioni sono utili in entrambe le direzioni. Se una parola di n bit è ruotata verso sinistra di k bit, si ottiene lo stesso risultato che ruotandola verso destra di $n - k$ bit.

Gli scorrimenti verso destra sono spesso usati in associazione con l'estensione del segno, ovvero le posizioni all'estremità di sinistra lasciate vacanti dallo scorrimento vengono riempite con il bit di segno originario, 0 o 1. È come se il bit di segno venisse trascinato verso destra dallo scorrimento. Tra le altre cose, ciò implica che un numero negativo resta negativo. È quanto succede nel seguente esempio di scorrimento di 2 bit verso destra.


```
11111111 11111111 11111111 11110000 A
00111111 11111111 11111111 11111100 A scorso senza estensione del segno
11111111 11111111 11111111 11111100 A scorso con estensione del segno
```

Un'applicazione importante dello scorrimento è la moltiplicazione e la divisione per potenze di 2. Se un intero positivo viene fatto scorrere di k bit verso sinistra allora, a meno di overflow, il risultato rappresenta il numero iniziale moltiplicato per 2^k. Se un numero positivo viene fatto scorrere verso destra di k bit, si ottiene il numero iniziale diviso per 2^k.

Lo scorrimento può essere usato per accelerare certe operazioni aritmetiche. Si consideri, per esempio, il calcolo di $18 \times n$ con n intero positivo. Poiché $18 \times n = 16 \times n + 2 \times n$, è possibile ottenere $16 \times n$ facendo scorrere n verso sinistra di 4 bit, $2 \times n$ facendo scorrere n verso sinistra di 1 bit. La somma di questi due numeri è proprio $18 \times n$, così la moltiplicazione è stata realizzata tramite un trasferimento, due scorrimenti e un'addizione, il che è spesso più veloce di una moltiplicazione vera e propria. Ovviamente è uno stratagemma che il compilatore può usare solo se almeno uno dei fattori è una costante.

Tuttavia lo scorrimento di numeri negativi, anche se fatto con estensione del segno, dà risultati molto diversi. Si consideri per esempio il numero –1, in complemento a uno, che, se fatto scorrere di una posizione verso sinistra, produce –3. Un altro scorrimento di un bit verso sinistra porta a –7:

```
11111111 11111111 11111111 11111110  –1 in complemento a uno
11111111 11111111 11111111 11111100  –1 scorso verso sinistra di 1 bit = –3
11111111 11111111 11111111 11111000  –1 scorso verso sinistra di 2 bit = –7
```

Lo scorrimento verso sinistra dei numeri negativi in complemento a uno non equivale a moltiplicare per 2. Invece il loro scorrimento a destra simula la divisione correttamente. Prendete ora la rappresentazione di –1 in complemento a due. Facendola scorrere di 6 bit verso destra si ottiene ancora –1, il che è sbagliato visto che la parte intera di –1/64 è 0:

```
11111111 11111111 11111111 11111111  –1 in complemento a due
11111111 11111111 11111111 11111111  –1 scorso di 6 bit verso destra = –1
```

In generale lo scorrimento verso destra introduce errori perché tronca il risultato. Invece lo scorrimento a sinistra simula davvero la moltiplicazione per 2.

Le operazioni di rotazione sono utili per l'impacchettamento di sequenze di bit in parole e per il loro spacchettamento. Se si vuole esaminare una parola bit per bit, la si può ruotare di 1 bit alla volta (non importa in che direzione) ed esaminare a ogni passo il contenuto del bit di segno; dopo aver esaminato tutti i bit la parola risulta ripristinata nella sua forma originale. Le operazioni di rotazione sono più genuine delle operazioni di scorrimento, perché non comportano perdita d'informazione: gli effetti di un'operazione di rotazione arbitraria possono essere annullati da un'altra operazione di rotazione.

Alcune operazioni binarie coinvolgono certi operandi così di frequente che alle volte gli ISA dispongono d'istruzioni unarie per svolgerle più velocemente. La copia del

valore zero in una certa parola di memoria o in un registro è estremamente frequente nella fase di inizializzazione di una computazione. La copia di zero è naturalmente un caso speciale d'istruzione di trasferimento dati. Per ragioni di efficienza, si usa spesso l'operazione CLR che contiene un solo indirizzo, quello della locazione da azzerare (*clear*).

La somma di una parola con il valore 1 è usata comunemente per contare. Una forma unaria dell'istruzione ADD è l'operazione INC che incrementa di 1. L'operazione di negazione è un altro esempio: negare X corrisponde a calcolare $0 - X$, una sottrazione binaria, e poiché è un'operazione di uso frequente alle volte si fornisce un'apposita istruzione unaria NEG. Sottolineiamo qui la differenza tra l'operazione aritmetica NEG e l'operazione logica NOT. L'operazione NEG produce l'**opposto** di un numero, quello cioè che dà 0 se sommato al numero originale. L'operazione NOT si limita a invertire i bit della parola. Le due operazioni sono molto simili, e infatti sono identiche nella rappresentazione in complemento a uno (nell'aritmetica in complemento a due, l'istruzione NEG si ottiene invertendo i singoli bit e poi aggiungendo 1).

Le istruzioni a uno o due operandi sono spesso raggruppate in base al loro uso invece che secondo il numero di operandi richiesto. Un insieme contiene le operazioni aritmetiche, compresa la negazione. Un altro insieme comprende le operazioni logiche e lo scorrimento, dal momento che vengono spesso usate congiuntamente per realizzare l'estrazione di dati.

5.5.4 Confronti e salti condizionati

Quasi tutti i programmi hanno bisogno della capacità di esaminare il contenuto dei dati e alterare la sequenza di esecuzione delle istruzioni in base al risultato dell'ispezione. Un semplice esempio di ciò è la funzione radice quadrata, $\sqrt{x}$: se x è negativo la procedura restituisce un messaggio d'errore, altrimenti calcola la radice quadrata. La funzione sqrt (da *square root*) deve perciò essere in grado di esaminare x e di scegliere come proseguire nell'esecuzione a seconda che x sia o meno negativa.

Un metodo comune per farlo è fornire istruzioni di salto condizionato che verificano una certa condizione e saltano a un particolare indirizzo di memoria se la condizione è soddisfatta. Alle volte c'è un bit nell'istruzione che indica se il salto deve avvenire a condizione soddisfatta o non soddisfatta. Spesso l'indirizzo di destinazione del salto non è assoluto, ma relativo all'istruzione corrente.

La condizione che viene testata più comunemente è se un determinato bit della macchina è posto o meno a 0. Se un'istruzione esamina il bit di segno di un numero e salta all'etichetta ETICHETTA qualora il bit valga 1, allora il comando che si trova alla posizione ETICHETTA sarà eseguito solo se il numero della condizione è negativo, altrimenti (se il numero è nullo o positivo) verrà eseguito il comando che segue il salto condizionato.

Molte macchine hanno bit usati per specificare certe condizioni. Per esempio ci potrebbe essere un bit di overflow asserito ogniqualvolta un'operazione aritmetica produce un risultato errato. Esaminare questo bit vuol dire verificare se l'istruzione precedente abbia o meno causato un overflow, e in caso positivo è possibile saltare a una routine di errore in grado di intraprendere le dovute azioni correttive.

Analogamente, alcuni processori hanno un bit di riporto che viene asserito quando si verifica un riporto all'ultimo bit di sinistra, per esempio quando vengono sommati due numeri negativi. Un riporto al bit più significativo è un evento normale, da non confondere con un overflow. Il test del bit di riporto è necessario all'aritmetica in precisione multipla (per esempio quando un intero è rappresentato in due o più parole).

Verificare se un parola vale zero è importante per i cicli e per molti altri scopi. Se tutte le istruzioni di salto condizionato esaminassero un bit alla volta, ci vorrebbero tanti confronti quanti sono i bit per verificare che tutti i bit di una parola valgono zero. Per evitare questo scenario, molte macchine mettono a disposizione istruzioni di confronto per esaminare una parola intera e, se nulla, effettuare il salto. Naturalmente questa soluzione non fa che passare la "patata bollente" alla microarchitettura. Di solito l'hardware contiene un registro i cui bit sono tutti messi in OR al fine di disporre di un solo bit che stabilisce immediatamente se la parola contiene uno o più bit asseriti. Il bit Z della Figura 4.1 viene calcolato di norma invertendo il risultato dell'OR di tutti i bit in uscita dalla ALU.

I confronti tra parole o tra caratteri sono importanti per verificare se sono uguali e, se non lo sono, per stabilire qual è maggiore, il che è indispensabile per poterli ordinare. Per effettuare questo test sono necessari tre indirizzi: due per i dati e uno verso cui saltare se la condizione è vera. I computer con formati d'istruzioni a tre indirizzi non incontrano difficoltà in tal senso, gli altri devono impiegare qualche espediente per aggirare il problema.

Una soluzione comune prevede un'istruzione che effettua il confronto e imposta uno o più bit di condizione per memorizzare il risultato. Un'istruzione successiva può esaminare i bit di condizione e saltare se i due valori messi a confronto si sono rivelati uguali, o diversi, o se il primo era maggiore, e così via. Il Core i7, l'OMAP4430 ARM e l'ATmega168 AVR usano questo approccio.

Il confronto di due numeri implica la comprensione di alcune sottigliezze. Infatti il confronto non è facile quanto una sottrazione: la sottrazione tra un numero positivo molto grande e uno negativo molto grande provoca un overflow, poiché il risultato della sottrazione non può essere rappresentato. L'istruzione di confronto invece deve stabilire se la condizione in esame è soddisfatta o meno e restituire sempre il risultato corretto; un confronto non può provocare un overflow.

Un'altra questione delicata nel confronto tra numeri è stabilire se un numero è da considerarsi con o senza segno. I numeri binari di tre bit possono essere ordinati in uno dei due modi. Dal più piccolo al più grande:

senza segno	con segno
000	100 (più piccolo)
001	101
010	110
011	111
100	000
101	001
110	010
111	011 (più grande)

La colonna di sinistra mostra gli interi positivi da 0 a 7 in ordine crescente. La colonna di destra mostra gli interi con segno in complemento a due che vanno da –4 a +3. Rispondere alla domanda "se 011 sia maggiore di 100" dipende dal modo in cui sono letti i numeri, se con segno o senza segno. La maggior parte degli ISA dispone d'istruzioni per gestire entrambi gli ordinamenti.

5.5.5 Invocazione di procedura

Una procedura è un insieme d'istruzioni che svolge un certo compito e che può essere invocata (chiamata) da diversi punti di un programma. Il termine **subroutine** è usato spesso invece di procedura, soprattutto in riferimento a programmi in linguaggio assemblativo. In C le procedure si chiamano funzioni, anche se non sono propriamente funzioni in senso matematico. Il termine usato in Java è **metodo**. Quando una procedura ha terminato il proprio compito l'esecuzione deve riprendere dall'istruzione successiva alla chiamata. A tal fine l'indirizzo di ritorno deve essere passato alla procedura o salvato da qualche parte dove possa essere recuperato al momento del ritorno.

L'indirizzo di ritorno può essere salvato in tre posti diversi: in memoria, in un registro o nello stack. La soluzione di gran lunga peggiore è metterlo in una locazione di memoria fissa. In questo modo se la procedura chiamasse un'altra procedura, la seconda chiamata causerebbe la perdita dell'indirizzo di ritorno della prima.

Un lieve miglioramento consiste nel far sì che l'istruzione di chiamata di procedura salvi l'indirizzo di ritorno nella prima parola della procedura, cosicché la prima istruzione del codice eseguibile si trovi nella seconda parola. La procedura può quindi rientrare saltando indirettamente tramite la prima parola o anche direttamente, se l'hardware consente l'inserimento dell'opcode per il salto direttamente nella prima parola, insieme all'indirizzo di ritorno. La procedura potrebbe richiamare altre procedure, visto che ciascuna alloca lo spazio per il proprio indirizzo di ritorno. Questo schema fallisce però se la procedura chiama se stessa, perché il primo indirizzo di ritorno verrebbe cancellato dalla seconda chiamata. La capacità di una procedura di chiamare se stessa, detta **ricorsione**, è di estrema importanza tanto per gli studiosi di programmazione quanto per i programmatori. Questo schema fallisce anche se la procedura *A* chiama la procedura *B*, *B* chiama la procedura *C* e *C* chiama *A* (ricorsione indiretta o "a festone", *daisy-chain*). Questo schema di memorizzazione dell'indirizzo di ritorno nella prima parola di una procedura era utilizzato sul CDC 6600, il computer più veloce al mondo per gran parte degli anni '60. Il principale linguaggio utilizzato sul 6600 era il FORTRAN, che proibiva la ricorsione e poteva dunque funzionare. Ma era, ed è tuttora, una pessima idea.

Un miglioramento sostanziale si ottiene se l'istruzione di chiamata di procedura pone l'indirizzo di ritorno in un registro, affidando alla procedura la responsabilità di salvarlo in un posto sicuro. Se la procedura è ricorsiva dovrà preoccuparsi di salvare l'indirizzo di ritorno in un posto differente ogni volta che viene invocata.

La cosa migliore che può fare l'istruzione di chiamata di procedura è porre l'indirizzo di ritorno in cima a uno stack. Quando la procedura termina fa un pop dell'indirizzo di ritorno e lo scrive nel program counter. Con questa forma di chiamata la ricorsione non causa alcun problema particolare; l'indirizzo di ritorno viene salvato automaticamente senza sovrascrivere indirizzi di ritorno precedenti. In queste condizioni la ricorsione funziona perfettamente. Abbiamo già visto questa modalità di salvataggio dell'indirizzo di ritorno in IJVM (Figura 4.12).

5.5.6 Istruzioni di ciclo

Poiché capita spesso di dover eseguire un gruppo d'istruzioni un numero prefissato di volte, molte macchine dispongono d'istruzioni per facilitare questo compito. Tutti gli schemi prevedono un contatore che viene incrementato o decrementato di un certo valore costante a ogni iterazione del ciclo. Il contatore viene anche esaminato a ogni iterazione; il ciclo termina quando si verifica una certa condizione.

Un metodo possibile è inizializzare il contatore al di fuori del ciclo e quindi cominciarne immediatamente l'esecuzione. L'ultima istruzione del ciclo aggiorna il contatore e, se la condizione di terminazione non è stata ancora soddisfatta, torna alla prima istruzione del ciclo. In caso contrario il ciclo termina e si prosegue dalla prima istruzione dopo il ciclo. Questa tipologia di ciclo è detta con valutazione in coda. Nella Figura 5.29(a) ne diamo un esempio in C (non avremmo potuto scriverlo in Java perché non fornisce istruzioni di `goto`).

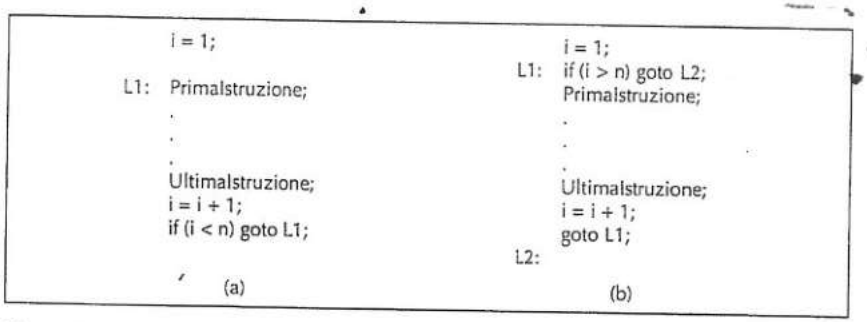


Figura 5.29 (a) Ciclo con verifica della condizione in coda. (b) Ciclo con verifica della condizione in testa.

Il ciclo con valutazione in coda ha la proprietà di essere eseguito sempre almeno una volta, anche se *n* fosse minore a uguale a 0. Considerate come esempio un programma gestore delle schede del personale di una ditta. A un certo punto della sua esecuzione il programma legge i dati di un particolare impiegato e salva in *n* il numero di figli che ha. Quindi esegue *n* volte un ciclo in cui, per ogni figlio, legge il suo nome, il sesso e la data di nascita affinché la ditta possa spedirgli un regalo di compleanno. Se l'impiegato non ha figli, *n* vale 0, eppure il ciclo verrà eseguito una volta e potrebbe spedire erroneamente un regalo di compleanno.

La Figura 5.29(b) mostra un'altra modalità di valutazione che funziona correttamente anche per valori di *n* minori o uguali a 0. Notate come il confronto cambi nelle due

modalità, perciò se l'incremento e la valutazione sono effettuati da un'unica istruzione ISA, i progettisti sono costretti a scegliere una modalità o l'altra.

Considerate il codice che dovrebbe essere generato per l'istruzione

```
for (i = 0; i < n; i++) { istruzioni }
```

Se il compilatore non dispone d'informazioni su n, allora deve usare l'approccio della Figura 5.29(b) per poter gestire correttamente anche i casi in cui $n \leq 0$. Se invece può stabilire che $n > 0$, magari osservando la locazione cui n è stata assegnata, allora può usare il codice migliore della Figura 5.29(a). Il vecchio FORTRAN stabiliva che tutti i cicli dovevano essere eseguiti almeno una volta, proprio per consentire sempre la generazione del codice più efficiente della Figura 5.29(a). Nel 1977 anche la comunità FORTRAN capì che non era una buona idea avere un'istruzione di ciclo dalla semantica stravagante, e che ogni tanto restituiva una risposta errata, solo per risparmiare un'istruzione di salto per ciclo, e corresse il tiro. C e Java non hanno mai avuto questo problema.

5.5.7 Input/Output

Nessun raggruppamento d'istruzioni manifesta la stessa variabilità tra macchine diverse come le istruzioni di I/O. Attualmente i personal computer usano tre schemi diversi di I/O:

1. I/O programmato con attesa attiva;
2. I/O *interrupt driven* (cioè innescato dagli interrupt);
3. I/O con DMA.

Vediamoli in dettaglio. Il metodo di I/O più semplice possibile è quello **programmato** impiegato di solito nei microprocessori di fascia bassa come, per esempio, i sistemi integrati o i sistemi che devono rispondere rapidamente agli stimoli esterni (sistemi in tempo reale). Queste CPU hanno in genere una sola istruzione di input e una sola di output, che trasferisce un carattere per volta da un prefissato registro al dispositivo di I/O prescelto. Il processore deve eseguire una sequenza prestabilita d'istruzioni per ciascun carattere letto o scritto.

Come semplice esempio di questo metodo, si consideri il terminale con quattro registri di 1 byte mostrato nella Figura 5.30. Due registri sono utilizzati per lo stato e i dati in input, mentre gli altri due sono usati per lo stato e i dati in output. Ogni registro ha un indirizzo unico. Se l'I/O è mappato in memoria, i quattro registri fanno parte dello spazio degli indirizzi della memoria del computer e possono essere letti e scritti mediante le istruzioni ordinarie. In caso contrario ci sono istruzioni speciali di I/O, che chiamiamo `IN` e `OUT`, per la lettura e la scrittura dei registri. In entrambi i casi l'I/O avviene tramite il trasferimento dati e del loro stato tra la CPU e questi registri.

Il registro dello stato della tastiera utilizza 2 soli bit degli 8 disponibili: il bit più significativo (il 7) è posto a 1 via hardware ogniqualvolta arriva un carattere; questo evento solleva un interrupt se e solo se il bit 6 era stato asserito precedentemente via software (tratteremo gli interrupt più avanti). Nel caso dell'I/O programmato la CPU aspetta dati in ingresso effettuando un ciclo serrato e ripetuto di letture sul registro di

stato della tastiera, aspettando che il bit 7 assuma il valore 1. Non appena ciò si verifica, il software legge il carattere dal registro buffer della tastiera. La lettura del registro dati della tastiera causa l'azzeramento del bit `CARATTERE DISPONIBILE`.

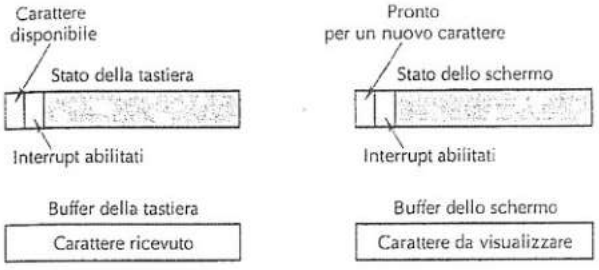


Figura 5.30 Registri dei dispositivi di un semplice terminale.

L'output funziona in modo analogo. Per visualizzare un carattere sullo schermo il software per prima cosa legge il registro di stato dello schermo per assicurarsi che il bit `PRONTO` valga 1 e in caso negativo si ripete finché il bit non viene asserito a indicare che il dispositivo è pronto ad accettare un carattere. Non appena il terminale è pronto, il software scrive un carattere nel registro di buffer dello schermo, il che provoca la sua trasmissione allo schermo e anche l'azzeramento del bit `PRONTO` (nel registro di stato dello schermo) da parte del dispositivo. Dopo la visualizzazione del carattere, il controllore imposta automaticamente a 1 il bit `PRONTO` non appena il dispositivo è preparato a trattare il carattere successivo.

Un esempio di I/O programmato è dato dalla procedura Java nella Figura 5.31. Questa procedura presenta due parametri: un array di caratteri per riporre il risultato e il numero di caratteri (*count*) presenti nell'array (al massimo 1024). Il corpo della procedura è costituito da un ciclo che restituisce un carattere alla volta. Per ogni carattere la CPU deve attendere innanzitutto che il dispositivo sia pronto prima di poter ottenerlo. È ragionevole pensare che le procedure *in* e *out* siano delle routine scritte in linguaggio assemblativo per la lettura e la scrittura dei registri di dispositivo specificati dai rispettivi parametri (il registro di stato per *in*, quello di buffer per *out*). L'implicita divisione per 128, ottenuta per mezzo di uno shift, permette di sbarazzarsi dei 7 bit meno significativi, riportando il bit `PRONTO` in posizione 0.

Lo svantaggio principale dell'I/O programmato è che la CPU passa gran parte del suo tempo in un ciclo serrato in cui attende che il dispositivo risulti pronto. Questo approccio si chiama **attesa attiva** (*busy waiting*) ed è accettabile se la CPU non ha null'altro da eseguire (sebbene anche un semplice controllore debba spesso monitorare molteplici eventi concorrenti). Tuttavia questa strategia è uno spreco in tutti quei casi in cui ci siano da svolgere altri compiti, quali l'esecuzione di ulteriori programmi, e richiede perciò l'individuazione di un altro metodo di I/O.

```
public static void output_buffer(char buf[ ],int count) {
  // Spedisce un blocco di dati al dispositivo
  int status, i, ready;
  for (i = 0; i < count; i++) {
    do {
      status = in(display_status_reg);          // ottiene lo stato
      ready = (status >> 7) & 0x01;             // isola il bit PRONTO
    } while (ready != 1);
    out(display_buffer_reg, buf[i]);
  }
}
```

Figura 5.31 Esempio di I/O programmato.

Un modo per evitare l'attesa attiva è far sì che la CPU faccia partire il dispositivo di I/O e gli impartisca l'ordine di generare un interrupt quando ha finito. Possiamo capire come funziona guardando la Figura 5.30. Il software può richiedere all'hardware di segnalargli quando un'operazione di I/O è conclusa tramite l'asserzione del bit INTERRUPT ABILITATI nel registro di dispositivo. Riprenderemo l'analisi degli interrupt più avanti in questo capitolo quando tratteremo il controllo del flusso.

Val la pena far presente che in molti computer il segnale di interrupt viene generato mettendo in AND i bit INTERRUPT ABILITATI e PRONTO. Se il software abilitasse gli interrupt prima di cominciare le operazioni di I/O, verrebbe sollevato immediatamente un interrupt, poiché il bit PRONTO varrebbe 1. Dunque potrebbe essere indispensabile prima far partire il dispositivo, per poi abilitare immediatamente gli interrupt. La scrittura di un byte nel registro di stato non modifica il bit PRONTO, che è di sola lettura.

Benché l'I/O guidato dagli interrupt costituisca un grande progresso rispetto all'I/O programmato, è ben lungi dall'essere perfetto. Il problema è che ci vuole un interrupt per ogni carattere trasferito, e l'elaborazione di un interrupt è gravosa. Occorre un modo per ridurre drasticamente il numero di interrupt.

La soluzione si ottiene tornando all'I/O programmato, ma affidandolo a qualcun altro che non sia la CPU (la soluzione di molti problemi consiste nell'affidare il lavoro a qualcun altro). La Figura 5.32 mostra come fare: aggiungiamo al sistema un nuovo chip, il controllore **DMA** (*Direct Memory Access*), con accesso diretto al bus.

Il chip DMA ha al suo interno almeno quattro registri accessibili dal software in esecuzione nella CPU: il primo contiene l'indirizzo di memoria di partenza per la lettura o scrittura; il secondo conta il numero di byte (o parole) da trasferire; il terzo specifica il numero di dispositivo o lo spazio d'indirizzamento di I/O da usare, il che determina il dispositivo di I/O desiderato; il quarto stabilisce se i dati vanno letti dal dispositivo di I/O o se vanno scritti su di esso.

Per trasferire un blocco di 32 byte dall'indirizzo di memoria 100 al terminale (sia questo il dispositivo 4) la CPU scrive i numeri 32, 100 e 4 nei primi tre registri DMA, più il codice per la scrittura (poniamo sia 1) nel quarto registro, come illustrato nella Figura 5.32. A questo punto il DMA effettua una richiesta di bus per leggere il byte 100 dalla memoria, analogamente a come farebbe la CPU. Una volta ottenuto il byte, il controllore DMA effettuerebbe una richiesta di I/O al dispositivo 4 finalizzata alla scrittura del byte. Dopo il completamento di queste due operazioni, il controllore DMA incrementa di 1 il suo registro d'indirizzo e decrementa di 1 il suo registro contatore. Se il registro contatore è ancora positivo, si prosegue con la lettura da memoria di un altro byte e con la relativa scrittura nel dispositivo.

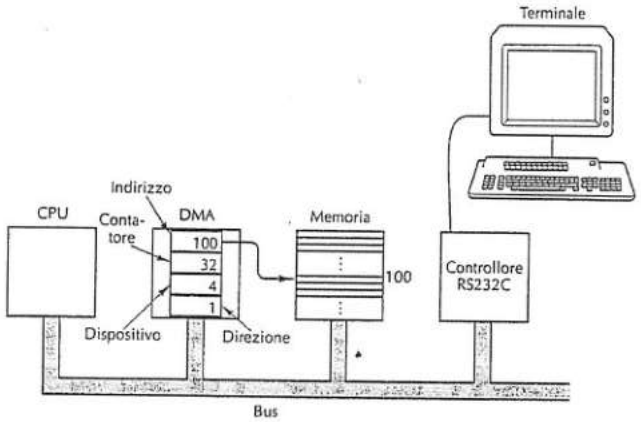


Figura 5.32 Controllore DMA.

Infine, quando il contatore si azzera, il controllore DMA smette di trasferire dati e manda un impulso sulla linea di interrupt collegata al chip della CPU. In presenza di DMA, la CPU deve solo inizializzare pochi registri, dopo di che è libera di svolgere altri compiti fino al completamento del trasferimento, segnalato da un interrupt proveniente dal controllore DMA. Alcuni controllori DMA dispongono di due, tre o più insiemi di registri per controllare trasferimenti simultanei.

Anche se con il DMA la CPU viene sollevata dal carico pesante dell'I/O, il procedimento non è del tutto gratuito. Se un dispositivo ad alta velocità, per esempio un disco, è in fase di trasferimento controllato dal DMA, ci vorranno molti cicli di bus per gli accessi alla memoria e al dispositivo. Durante questi cicli la CPU deve restare in attesa (il DMA ha una priorità di bus sempre maggiore di quella della CPU, perché i dispositivi di I/O difficilmente tollerano i ritardi). Il fenomeno che si verifica quando il controllore DMA sottrae cicli di bus alla CPU si dice **appropriazione di cicli** (*cycle stealing*, "furto di cicli"). Nondimeno il guadagno che si ottiene nel non dover gestire un interrupt per byte (o per parola) ripaga largamente del danno derivato dall'appropriazione di cicli.

5.5.8 Istruzioni del Core i7

In questo paragrafo e nei due successivi analizzeremo gli insiemi d'istruzioni delle nostre tre architetture esemplificative: il Core i7, l'OMAP4430, e l'ATmega168. Cia-

scuna di loro dispone di un nucleo d'istruzioni generate normalmente dai compilatori, più un insieme d'istruzioni usate raramente o destinate esclusivamente al sistema operativo. Nella nostra analisi focalizziamo l'attenzione sulle istruzioni comuni. Cominciamo dal Core i7 che è il più complicato. Dopo averlo affrontato sarà tutto in discesa.

L'insieme d'istruzioni del Core i7 è una miscellanea di vere istruzioni di 32 bit e resti dell'8088, suo antenato diretto. Nella Figura 5.33 mostriamo una piccola selezione di comuni istruzioni utilizzabili facilmente dai compilatori e dai programmatori odierni. È una lista tutt'altro che completa, visto che non include le istruzioni in virgola mobile, le istruzioni di controllo, né altre di uso meno frequente (tipo l'uso del byte AL come chiave di ricerca). In ogni caso restituisce una rappresentazione abbastanza fedele delle potenzialità del Core i7.

Molte delle istruzioni del Core i7 referenziano uno o due operandi, che possono trovarsi nei registri o in memoria. Per esempio l'istruzione ADD somma la sorgente alla destinazione, mentre la INC incrementa il suo operando di 1. Alcune istruzioni presentano varianti strettamente connesse tra loro. Per esempio l'istruzione di scorrimento può spostare i bit verso sinistra o verso destra, e può trattare il bit di segno in modo speciale o meno. Gran parte delle istruzioni presenta una varietà di codifiche, che cambia a seconda della natura degli operandi.

I campi SRC della Figura 5.33 sono sorgenti d'informazioni e non vengono modificati. Invece i campi DST sono destinazioni e di norma vengono modificati dall'istruzione. Ci sono alcune regole che stabiliscono che cosa può fungere da sorgente e che cosa da destinazione, che cambiano da istruzione a istruzione senza nessun criterio particolare, ma non ce ne curiamo in questa sede. Molte istruzioni presentano tre varianti, con operandi di 8, 16 e 32 bit. La distinzione tra i tipi avviene tramite l'opcode e/o un bit dell'istruzione. La lista della figura si sofferma soprattutto sulle istruzioni di 32 bit.

Per semplicità abbiamo suddiviso le istruzioni in vari gruppi: il primo contiene le istruzioni per il trasferimento dati all'interno della macchina, tra registri, e locazioni di memoria o di stack. Il secondo gruppo si occupa dell'aritmetica, con o senza segno. Nel caso della moltiplicazione e della divisione, il prodotto o il dividendo di 64 bit sono memorizzati in EAX (la parte meno significativa) e EDX (la parte più significativa).

Il terzo gruppo si occupa dell'aritmetica dei decimali in codifica binaria, detti anche BCD, che tratta ogni byte come due **nibble** (mezzi byte) di 4 bit. Ogni nibble contiene una cifra decimale (da 0 a 9); le configurazioni da 1010 a 1111 sono inutilizzate. Così un intero di 16 bit può contenere un numero decimale compreso tra 0 e 9999. Questa codifica è poco efficiente, ma non richiede la conversione dell'input da decimale in binario e quindi dell'output in decimale. Queste istruzioni sono usate per l'aritmetica dei numeri BCD, molto utilizzati dai programmi COBOL.

Le istruzioni booleane e quelle di scorrimento/rotazione manipolano in modi diversi i bit all'interno dei byte o delle parole. Ne presentiamo diverse varianti.

I due gruppi successivi trattano test, confronti, e salti basati sui loro risultati. I risultati dei test e dei confronti sono salvati in vari bit del registro EFLAGS. Jxx individua un insieme d'istruzioni di salto condizionato dal risultato del confronto precedente (cioè dai bit di EFLAGS).

Trasferimenti

MOV DST,SRC	Trasferisce SRC in DST
PUSH SRC	Impila SRC sullo stack
POP DST	Pop di una parola dallo stack e la pone in DST
XCHG DS1, DS2	Scambia DS1 e DS2
LEA DST,SRC	Carica in DST l'indirizzo effettivo di SRC
CMOVcc DST,SRC	Trasferimento condizionato

Aritmetica

ADD DST, SRC	Somma SRC a DST
SUB DST, SRC	Sottrae SRC da DST
MUL SRC	Moltiplica EAX con SRC (senza segno)
IMUL SRC	Moltiplica EAX con SRC (con segno)
DIV SRC	Divide EDX:EAX per SRC (senza segno)
IDIV SRC	Divide EDX:EAX per SRC (con segno)
ADC DST,SRC	Somma SRC a DST e somma il bit di riporto
SBB DST,SRC	Sottrae SRC e il riporto da DST
INC DST	Incrementa DST di 1
DEC DST	Decrementa DST di 1
NEG DST	Nega DST (lo sottrae a 0)

Decimali in codifica binaria

DAA	Codifica decimale
DAS	Codifica decimale per la sottrazione
AAA	Codifica ASCII per l'addizione
AAS	Codifica ASCII per la sottrazione
AAM	Codifica ASCII per la moltiplicazione
AAD	Codifica ASCII per la divisione

Booleane

AND DST, SRC	AND di SRC e DST, risultato in DST
OR DST, SRC	OR di SRC e DST, risultato in DST
XOR DST, SRC	OR esclusivo di SRC e DST, risultato in DST
NOT DST	Rimpiazza DST con il suo complemento a uno

Scorrimento/rotazione

SAL/ SAR DST,#	Scorrimento di DST verso s/d di # bit
SHL/ SHR DST,#	Scorrimento logico di DST verso s/d di # bit
ROL/ROR DST,#	Rotazione di DST verso s/d di # bit
RCL/ RCR DST,#	Rotazione di DST attraverso il riporto di # bit

Test/confronto

TEST SRC1,SRC2	AND degli operandi, modifica EFLAGS
CMP SRC1,SRC2	Modifica EFLAGS secondo SRC1- SRC2

Trasferimento del controllo

JMP ADDR	Salta all'indirizzo ADDR
Jxx ADDR	Salti condizionati da EFLAGS
CALL ADDR	Chiama la procedura all'indirizzo ADDR
RET	Ritorno da procedura
IRET	Ritorno da interrupt
LOOPxx	Cicla finché la condizione non sia soddisfatta
INT n	Innesca un interrupt software
INTO	Innesca un interrupt se il bit di overflow = 1

Stringhe

LODS	Carica una stringa
STOS	Memorizza una stringa
MOVS	Muove una stringa
CMPS	Copia una stringa
SCAS	Scandisce una stringa

Codici di condizione

STC	Asserisce il bit di riporto in EFLAGS
CLC	Azzera il bit di riporto in EFLAGS
CMC	Complemento del bit di riporto in EFLAGS
STD	Asserisce il bit di direzione in EFLAGS
CLD	Azzera il bit di direzione in EFLAGS
STI	Asserisce il bit di interrupt in EFLAGS
CLI	Azzera il bit di interrupt in EFLAGS
PUSHFD	Impila EFLAGS sullo stack
POPFD	Fa la pop di EFLAGS dallo stack
LAHF	Carica AH da EFLAGS
SAHF	Memorizza AH in EFLAGS

Miscellanea

SWAP DST	Cambia ordinamento dei byte di DST
CWQ	Estende EAX a EDX: EAX per la divisione
CWDE	Estende il numero di 16 bit in AX a EAX
ENTER SIZE, LV	Crea record d'attivazione di SIZE byte
LEAVE	Disfa il record d'attivazione creato da ENTER
NOP	Operazione nulla
HLT	Terminazione
IN AL, PORT	Un byte in input da PORT ad AL
OUT PORT, AL	Un byte in output da AL a PORT
WAIT	Attende un interrupt

s/d = sinistra/destra
SRC = sorgente
DST = destinazione
# = numero di scorrimenti/rotazioni
LV = puntatore alle variabili locali

Figura 5.33 Alcune istruzioni intere del Core i7.

Il Core i7 dispone di parecchie istruzioni per caricamento, memorizzazione, trasferimento, confronto e scansione di stringhe di caratteri o di parole. Queste istruzioni possono essere precedute da un byte speciale chiamato REP che ne causa l'esecuzione ripetuta finché non venga soddisfatta la condizione che ECX, decrementato a ogni iterazione, abbia raggiunto il valore 0. Così facendo è possibile trasferire, confrontare, e così via, blocchi arbitrari di dati. Il gruppo successivo gestisce i codici di condizione.

L'ultimo gruppo è una miscellanea d'istruzioni che non rientrano in nessuna delle categorie precedenti e comprende le conversioni, la gestione dei record d'attivazione dello stack, la terminazione della CPU e l'I/O.

Il Core i7 prevede molti **prefissi**, tra cui il già menzionato REP. Ognuno di questi prefissi è un byte speciale che può precedere la maggior parte delle istruzioni, analogamente a WIDE nell'IJVM. REP provoca la ripetizione dell'istruzione successiva finché ECX non raggiunga 0, come già detto. REPZ e REPNZ causano la ripetizione dell'istruzione che precedono fintanto che il codice di condizione Z non viene, rispettivamente, asserito o azzerato. LOCK riserva il bus per tutta la durata dell'esecuzione di un'istruzione, per consentire la sincronizzazione multiprocessore. Altri prefissi vengono usati per forzare un'istruzione in modalità di 16 bit o di 32 bit, il che non solo cambia la lunghezza degli operandi, ma ridefinisce completamente le modalità d'indirizzamento. Infine il Core i7 dispone di un complesso schema di segmentazione del codice, dei dati, dello stack e altro, un altro retaggio dell'8088. Sono previsti alcuni prefissi per forzare l'uso d'indirizzi di memoria in determinati segmenti, ma (fortunatamente) non sono oggetto della nostra analisi.

5.5.9 Istruzioni della CPU ARM OMAP4430

Quasi tutte le istruzioni intere ARM del modo utente che un compilatore può generare sono elencate nella figura 5.34. Non sono incluse nella lista le istruzioni in virgola mobile, le istruzioni di controllo (come quelle per la gestione della cache o il riavvio di sistema), le istruzioni che coinvolgono spazi degli indirizzi diversi da quello dell'utente e le estensioni, come per esempio Thumb. L'insieme è sorprendentemente piccolo: l'ISA ARM OMAP4430 è davvero un RISC.

Le istruzioni LDR e STR sono ovvie, con versioni per 1, 2 e 4 byte. Quando si carica un numero costituito da meno di 32 bit in un registro (di 32 bit), si può decidere se estenderlo con segno o con zero (ci sono istruzioni disponibili a entrambi gli scopi).

Il gruppo successivo riguarda le istruzioni aritmetiche, che possono opzionalmente impostare i bit dei codici di condizione del registro di stato del processore. Sulle macchine CISC sono molte le istruzioni che modificano i codici di condizione, ma su quelle RISC ciò non è auspicabile perché riduce la libertà del compilatore di spostare le istruzioni nel tentativo di riempire gli intervalli di ritardo. Se l'ordine originale delle istruzioni è A ... B ... C, dove A imposta alcuni codici di condizione poi esaminati da B, il compilatore non può inserire C tra A e B nel caso C modificasse i codici di condizione. Per questa ragione molte istruzioni vengono fornite in duplice versione, di modo che il compilatore usi preferenzialmente la versione che non modifica i codici di condizione, a meno che abbia intenzione di esaminarne il contenuto in un momento successivo. Il programmatore specifica l'impostazione dei codici di condizione aggiungendo una

"S" in coda al nome dell'istruzione (per esempio ADDS). Un bit dell'istruzione indica al processore che i codici di condizione devono essere impostati. Sono anche supportate le moltiplicazioni e le moltiplicazioni con accumulo.

Il gruppo di scorrimento/rotazione contiene uno scorrimento verso sinistra e due verso destra che operano su un registro di 32 bit. L'istruzione di rotazione a destra implementa una rotazione circolare dei bit del registro, in modo tale che il valore del bit meno significativo diventa il valore del bit più significativo.

Caricamento

LDRSB DST,ADDR	Carica byte con segno (8 bit)
LDRB DST,ADDR	Carica byte senza segno (8 bit)
LDRSH DST,ADDR	Carica mezza parola con segno (16 bit)
LDRH DST,ADDR	Carica mezza parola senza segno (16 bit)
LDR DST,ADDR	Carica parola (32 bit)
LDM S1,REGLIST	Carica parole multiple

Memorizzazione

STRB DST,ADDR	Memorizza byte (8 bit)
STRH DST,ADDR	Memorizza mezza parola (16 bit)
STR DST,ADDR	Memorizza parola (32 bit)
STM SRC,REGLIST	Memorizza parole multiple

Aritmetica

ADD DST,S1,S2IMM	Somma
ADD DST,S1,S2IMM	Somma con riporto
SUB DST,S1,S2IMM	Sottrazione
SUB DST,S1,S2IMM	Sottrazione con riporto
RSB DST,S1,S2IMM	Sottrazione inversa
RSC DST,S1,S2IMM	Sottrazione inversa con riporto
MUL DST,S1,S2	Moltiplicazione
MLA DST,S1,S2,S3	Moltiplicazione e accumulo
UMULL D1,D2,S1,S2	Moltiplicazione long senza segno
SMULL D1,D2,S1,S2	Moltiplicazione long con segno
UMLAL D1,D2,S1,S2	MLA long senza segno
SMLAL D1,D2,S1,S2	MLA long con segno
CMP S1,S2IMM	Confronta e imposta PSR

Scorrimento/rotazione

LSL DST,S1,S2IMM	Scorrimento logico a sinistra
LSR DST,S1,S2IMM	Scorrimento logico a destra
ASR DST,S1,S2IMM	Scorrimento aritmetico a destra
ROR DSR,S1,S2IMM	Rotazione a destra

Booleane

TST DST,S1,S2IMM	Test dei bit
TEQ DST,S1,S2IMM	Test equivalenza
AND DST,S1,S2IMM	AND
EOR DST,S1,S2IMM	OR esclusivo
ORR DST,S1,S2IMM	OR
BIC DST,S1,S2IMM	Pulizia dei bit

Trasferimento di controllo

Bcc IMM	Salto a PC+IMM
BLcc IMM	Salto con link a PC+IMM
BLcc S1	Salto con link a registro

Miscellanea

MOV DST,S1	Trasferimento registro
MOVT DST,IMM	Trasferimento di imm nei bit più significativi
MVN DST,S1	NOT di un registro
MRS DST,PSR	Lettura PSR
MSR PSR,S1	Scrittura PSR
SWP DST,S1,ADDR	Scambio di parola registro/memoria
SWPB DST,S1,ADDR	Scambio di byte registro/memoria
SWI IMM	Interruzione via software

S1 = registro sorgente
S2IMM = registro sorgente o immediato
S3 = registro sorgente (quando sono necessari 3 registri)
DST = registro destinazione
D1 = registro destinazione (1 di 2)
D2 = registro destinazione (2 di 2)

ADDR = indirizzo di memoria
IMM = valore immediato
REGLIST = lista dei registri
PSR = registro dello stato del processore
cc = condizione di salto

Figura 5.34 Le principali istruzioni intere della CPU ARM OMAP4430.

Gli scorrimenti sono usati soprattutto per la manipolazione di bit. Le rotazioni sono utili per operazioni di crittografia e di elaborazione delle immagini. Gran parte delle macchine CISC dispone di un numero ingente d'istruzioni di scorrimento e rotazione, quasi

tutte completamente inutili. Pochi scrittori di compilatori passeranno le notti a piangerne la scomparsa!

Il gruppo d'istruzioni booleane è analogo al gruppo delle aritmetiche. Comprende le operazioni AND, EOR, TST, TEQ e BIC. Le ultime tre sono di dubbia utilità, però possono essere svolte in un solo ciclo e richiedono poco hardware supplementare; per questo motivo sono state incluse nel progetto. Anche i progettisti delle macchine RISC di tanto in tanto soccombono alle tentazioni.

Il gruppo successivo d'istruzioni contiene i trasferimenti di controllo. Bcc rappresenta l'insieme delle istruzioni che saltano al verificarsi di varie condizioni L'istruzione BLcc è simile, nel senso che effettua il salto sotto opportune condizioni, ma in aggiunta deposita l'indirizzo della successiva istruzione nel registro link (R14). Questa istruzione è utile per implementare le chiamate di procedura. A differenza di tutte le altre architetture RISC non esiste una esplicità istruzione di salto a registro. Questa operazione può essere realizzata mediante un'istruzione MOV, impostando come destinazione il program counter (R15).

Sono previste due modalità di chiamata di procedura. La prima istruzione BLcc utilizza il formato di salto della figura 5.14 con uno spiazzamento di 24 bit relativo al program counter. Questo valore è sufficiente a raggiungere un'istruzione entro 32 MB dal chiamante, in entrambe le direzioni. La seconda istruzione BLcc salta all'indirizzo contenuto in un registro specificato e può essere utilizzata per implementare chiamate di procedura con binding dinamico (per esempio, le funzioni virtuali di C++) oppure chiamate oltre il limite dei 32 MB.

L'ultimo gruppo contiene alcune istruzioni di vario genere. MOVT serve per ovviare all'impossibilità di mettere un operando immediato di 32 bit in un registro. Il modo di procedere è usare MOVT per impostare i bit da 16 a 31 e affidare quindi i bit rimanenti all'istruzione successiva grazie al formato immediato. Le istruzioni MRS e MSR permettono lettura e scrittura della parola di stato del processore (PSR). Le istruzioni SWP realizzano scambi atomici tra registri e locazioni di memoria. Queste istruzioni implementano le primitive di sincronizzazione multiprocessore che impareremo nel Capitolo 8. Infine, l'istruzione SWI permette gli interrupt via software (un modo eccessivamente elegante per dire che effettua una chiamata di sistema).

5.5.10 Istruzioni dell'ATmega168 AVR

L'ATmega168 ha un repertorio d'istruzioni semplice, mostrato nella Figura 5.35. Ogni riga specifica il nome abbreviato dell'istruzione, ne dà una descrizione concisa e fornisce un segmento di pseudocodice che chiarisce il funzionamento dell'istruzione. Come ci si poteva attendere, ci sono molte istruzioni MOV per il trasferimento di dati tra i registri. Ci sono istruzioni per le operazioni di push e di pop su di uno stack indirizzato da uno stack pointer (SP) di 16 bit presente in memoria. Si può accedere alla memoria mediante indirizzamento immediato, indiretto basato su registro o indiretto basato su un registro più uno spiazzamento. Per permettere fino a 64 KB di memoria, l'istruzione di caricamento con un indirizzo immediato è un'istruzione di 32 bit. La modalità di indirizzamento indiretto utilizza le coppie di registir X, Y e Z, che combinano due registri di 8 bit per formare un singolo puntatore di 16 bit.

Istruzione	Descrizione	Semantica
ADD DST,SRC	Somma	DST ← DST + SRC
ADC DST,SRC	Somma con riporto	DST ← DST + SRC + C
ADIW DST,IMM	Somma di immediato a una parola	DST+1:DST ← DST+1:DST + IMM
SUB DST,SRC	Sottrazione	DST ← DST - SRC
SUBI DST,IMM	Sottrazione di immediato	DST ← DST - IMM
SBC DST,SRC	Sottrazione con riporto	DST ← DST - SRC - C
SBCI DST,IMM	Somma di immediato con riporto	DST ← DST - IMM - C
SBIW DST,IMM	Sottrazione di immediato a una parola	DST+1:DST ← DST+1:DST - IMM
AND DST,SRC	AND logico	DST ← DST AND SRC
ANDI DST,IMM	AND logico con immediato	DST ← DST AND IMM
OR DST,SRC	OR logico	DST ← DST OR SRC
ORI DST,IMM	OR logico con immediato	DST ← DST OR IMM
EOR DST,SRC	OR esclusivo	DST ← DST XOR SRC
COM DST	Complemento a uno	DST ← 0xFF - DST
NEG DST	Complemento a due	DST ← 0x00 - DST
SBR DST,IMM	Imposta i bit in un registro	DST ← DST OR IMM
CBR DST,IMM	Cancella i bit in un registro	DST ← DST AND (0xFF - IMM)
ING DST	Incremento	DST ← DST + 1
DEC DST	Decremento	DST ← DST - 1
TST DST	Verifica per minore o uguale a zero	DST ← DST AND DST
CLR DST	Cancella il contenuto di un registro	DST ← DST XOR DST
SER DST	Imposta registro	DST ← 0xFF
MUL DST,SRC	Moltiplicazione senza segno	R1:R0 ← DST * SRC
MULS DST,SRC	Moltiplicazione con segno	R1:R0 ← DST * SRC
MULSU DST,SRC	Moltiplicazione con segno con operando senza segno	R1:R0 ← DST * SRC
RJMP IMM	Salto relativo a PC	PC ← PC + IMM + 1
IJMP	Salto indiretto a Z	PC ← Z (R30:R31)
JMP IMM	Salto	PC ← IMM
RCALL IMM	Chiamata relativa	STACK ← PC+2, PC ← PC + IMM + 1
ICALL	Chiamata indiretta (Z)	STACK ← PC+2, PC ← Z (R30:R31)
CALL	Chiamata	STACK ← PC+2, PC ← IMM
RET	Ritorno	PC ← STACK
CP DST,SRC	Confronto	DST – SRC
CPC DST,SRC	Confronto con riporto	DST – SRC – C
CPI DST,IMM	Confronto con immediato	DST – IMM
BRcc IMM	Salto condizionato	If cc(true) PC ← PC + IMM + 1
MOV DST,SRC	Copia di registro	DST ← SRC
MOVW DST,SRC	Copia di coppia di registri	DST+1:DST ← SRC+1:SRC
LDI DST,IMM	Caricamento immediato	DST ← IMM
LDS DST,IMM	Caricamento diretto	DST ← MEM[IMM]
LD DST,XYZ	Caricamento indiretto	DST ← MEM[XYZ]
LDD DST,XYZ+IMM	Caricamento indiretto con spiazzamento	DST ← MEM[XYZ+IMM]
STS IMM,SRC	Memorizzazione diretta	MEM[IMM] ← SRC
ST XYZ,SRC	Memorizzazione indiretta	MEM[XYZ] ← SRC
STD XYZ+IMM,SRC	Memorizzazione indiretta con spiazzamento	MEM[XYZ+IMM] ← SRC
PUSH REGLIST	Push di un registro sullo stack	STACK ← REGLIST
POP REGLIST	Pop di un registro dallo stack	REGLIST ← STACK
LSL DST	Scorrimento logico a sinistra di una posizione	DST ← DST LSL 1
LSR DST	Scorrimento logico a destra di una posizione	DST ← DST LSR 1
ROL DST	Rotazione a sinistra di una posizione	DST ← DST ROL 1
ROR DST	Rotazione a destra di una posizione	DST ← DST ROR 1
ASR DST	Scorrimento aritmetico a destra di una posizione	DST ← DST ASR 1

SRS = registro sorgente
DST = registro destinazione
IMM = IMM = valore immediato
XYZ = X, Y o Z, coppie di registri
Accesso alla memoria di tipo A

Figura 5.35 Le istruzioni dell'ATmega168 AVR.

L'ATmega168 dispone di semplici istruzioni aritmetiche per somma, sottrazione e moltiplicazione, dove le ultime due usano due registri. Sono disponibili anche istruzioni d'incremento e decremento, usate di frequente. Ci sono anche le istruzioni booleane, di scorrimento e di rotazione. Le istruzioni di salto e di chiamata possono avere come destinazione un indirizzo immediato, un indirizzo relativo al program counter o un indirizzo contenuto nella coppia di registri Z.

5.5.11 Insiemi d'istruzioni a confronto

I tre esempi di insiemi d'istruzioni che abbiamo visto sono molto diversi tra loro. Il Core i7 è una classica macchina CISC a 32 bit e a due indirizzi, con una lunga storia, modalità d'indirizzamento peculiari e molto irregolari, nonché con molte istruzioni per l'accesso in memoria. La CPU ARM OMAP4430 è una moderna macchina RISC a 32 bit e a tre indirizzi, con architettura load/store, a malapena qualche modalità d'indirizzamento e un insieme d'istruzioni efficiente e compatto. L'ATmega168 è un minuscolo processore integrato, progettato per essere realizzato su un solo chip.

Se ogni macchina è fatta a modo proprio ci sono delle buone ragioni. Il progetto del Core i7 risponde a tre esigenze principali:

1. retrocompatibilità;
2. retrocompatibilità;
3. retrocompatibilità.

Considerando lo stato dell'arte attuale, a nessuno verrebbe in mente di progettare una macchina così irregolare e con così pochi registri, tutti diversi tra loro. Queste caratteristiche complicano la scrittura dei compilatori. La mancanza di registri inoltre costringe i compilatori a riversare continuamente le variabili in memoria per poi ricaricarle, un lavoro gravoso anche se si hanno due o tre livelli di cache. La velocità del Core i7 è una riprova delle capacità degli ingegneri dell'Intel, a dispetto dei vincoli posti dall'ISA. Eppure abbiamo visto nel capitolo precedente quanto sia complessa la sua implementazione.

L'OMAP4430 rappresenta un progetto di ISA allo stato dell'arte. Ha un ISA di 32 bit, ha molti registri e un insieme d'istruzioni che predilige le operazioni a tre registri, più un piccolo gruppo d'istruzioni LOAD e STORE. Tutte le istruzioni sono della stessa lunghezza, anche se il numero di formati è sfuggito un po' di mano. Ciononostante l'implementazione resta semplice ed efficiente. Gran parte dei nuovi progetti tende a somigliare all'architettura ARM OMAP4430, ma con meno formati d'istruzione.

L'ATmega168 AVR offre un insieme d'istruzioni semplice e abbastanza regolare, relativamente con poche istruzioni e poche modalità d'indirizzamento. Si distingue per la presenza di 32 registri di 8 bit, per l'accesso rapido ai dati, per la capacità di rendere i registri accessibili nello spazio di memoria e per le istruzioni di manipolazione dei bit sorprendentemente potenti. Il suo vanto principale è di richiedere un numero molto limitato di transitor per l'implementazione, il che consente di ospitarne un gran numero su di una sola piastra, dando luogo a un costo unitario di CPU molto contenuto.

5.6 Controllo del flusso

Il controllo del flusso riguarda la sequenza con cui le istruzioni vengono eseguite dinamicamente, ovvero durante l'esecuzione del programma. In mancanza d'istruzioni di salto o di chiamate di procedura, le istruzioni vengono eseguite di norma nello stesso ordine con cui si susseguono in memoria. Le chiamate di procedura alterano il controllo del flusso, perché arrestano la procedura correntemente in esecuzione e cominciano l'esecuzione della procedura chiamata. Le coroutine sono legate alle procedure e causano un'alterazione simile (sono utili nella simulazione di processi paralleli). Anche le trap e gli interrupt provocano un'alterazione del flusso esecutivo quando si verificano certe condizioni speciali. Sono tutti argomenti trattati nei paragrafi successivi.

5.6.1 Flusso sequenziale e diramazioni

Molte istruzioni non alterano il controllo del flusso; dopo l'esecuzione di un'istruzione, viene recuperata ed eseguita l'istruzione che la segue in memoria. Al termine di ciascuna istruzione, il program counter viene incrementato della lunghezza dell'istruzione elaborata. Se lo si osserva per un tempo abbastanza lungo, rispetto al tempo di esecuzione di una singola istruzione, il program counter è una funzione lineare nella variabile tempo e che aumenta, in ogni unità di tempo, di una quantità pari alla lunghezza media delle istruzioni diviso il loro tempo medio di esecuzione. In altre parole, l'ordine dinamico secondo cui il processore esegue le istruzioni è quello con cui appaiono nel listato del programma, come mostrato dalla Figura 5.36(a). Se un programma contiene salti, questa semplice relazione tra ordinamento delle istruzioni in memoria e ordine di esecuzione non vale più. In presenza di salti il program counter non è più una funzione monotòna crescente nel tempo, come si evince dalla Figura 5.36(b). In ragione di ciò, diventa difficile visualizzare la sequenza di esecuzione delle istruzioni a partire dal listato.

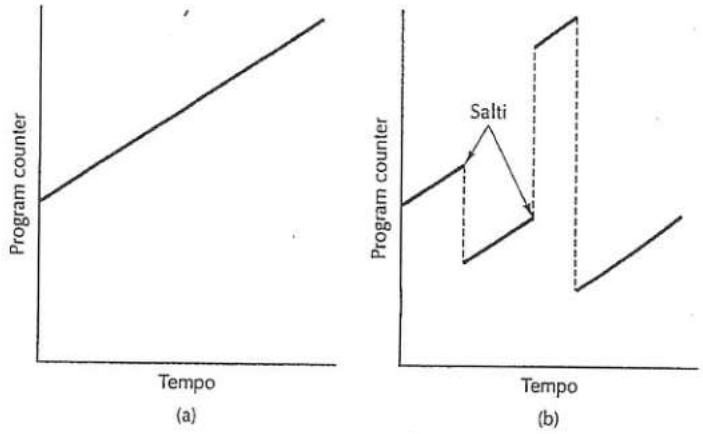


Figura 5.36 Valore del program counter in funzione del tempo. (a) Senza salti. (b) Con salti.

Quando i programmatori incontrano difficoltà nel tener traccia della sequenza di esecuzione delle istruzioni da parte del processore, commettono errori con più facilità. Questa osservazione ha portato Dijkstra a scrivere una nota (1968a), allora controversa, dal titolo "GO TO Statement Considered Harmful" ("L'istruzione GOTO è da considerarsi dannosa"), in cui suggeriva di evitare le istruzioni di `goto`.

Da quel testo è scaturita la rivoluzione della programmazione strutturata, tra i cui dogmi c'è la sostituzione di tutti i comandi `goto` con forme di controllo più strutturate, quali i cicli. Naturalmente quando questi programmi vengono compilati in programmi di livello 2, questi ultimi possono contenere molti salti, dal momento che l'implementazione di `if`, `while` e di altre strutture di controllo di alto livello richiede la capacità di saltare da una parte all'altra.

5.6.2 Procedure

Le procedure costituiscono la tecnica più importante per la strutturazione dei programmi. Da un certo punto di vista una chiamata di procedura altera il flusso esecutivo tanto quanto un salto, ma a differenza di un salto alla terminazione del suo compito la procedura ripassa il controllo al comando o all'istruzione che segue la sua chiamata.

Tuttavia, da un altro punto di vista, il corpo di una procedura può essere visto come la definizione di una nuova istruzione di alto livello. Così facendo, la chiamata di procedura è concepibile come un'istruzione singola, senza riguardo alla sua effettiva complessità. Per la comprensione di una porzione di codice contenente una chiamata di procedura è sufficiente sapere *che cosa* fa, non *come* lo fa.

Una tipologia di procedure particolarmente interessanti è quella delle **procedure ricorsive**, ovvero quelle procedure che richiamano se stesse direttamente o indirettamente attraverso una successione di altre procedure. Lo studio delle procedure ricorsive favorisce una profonda comprensione del modo in cui sono implementate le chiamate di procedura, e della natura delle variabili locali. Diamo ora un esempio di procedura ricorsiva.

Quello della "torre di Hanoi" è un vecchio problema che ammette una semplice soluzione basata sulla ricorsione. In un monastero di Hanoi ci sono tre pioli d'oro: sul primo sono impilati, attraverso un foro apposito, 64 dischi d'oro. I dischi sono di dimensioni diverse e ciascuno è poggiato su di un disco di diametro maggiore del proprio. Gli altri due pioli sono inizialmente vuoti. I monaci del monastero hanno il compito di trasferire tutti i dischi (uno alla volta) sul terzo piolo, ma senza mai poggiare un disco di dimensioni maggiori su uno più piccolo. Si dice che quando avranno evaso il loro compito, il mondo avrà fine. Se volete sperimentare direttamente potete provare a usare dei dischi qualunque (e meno di 64), ma una volta che avrete risolto il problema non aspettatevi che succeda alcunché. Per ottenere l'effetto "fine del mondo" dovete usarne 64 e tutti d'oro. La Figura 5.37 mostra la configurazione iniziale con 5 dischi.

La soluzione consiste nel muovere gli n dischi dal piolo 1 al piolo 3 spostandone prima $n-1$ dal piolo 1 al 2, quindi impilando il disco rimanente sul piolo 3 e infine spostando gli $n-1$ dischi del piolo 2 sul piolo 3. La Figura 5.38 illustra questa soluzione (per $n = 3$).

Per risolvere il problema abbiamo bisogno di una procedura per spostare n dischi dal piolo i al piolo j. La sua invocazione

```
towers(n, i, j)
```

produce la visualizzazione della soluzione. Per prima cosa la procedura effettua il test $n = 1$. Se così fosse la soluzione sarebbe banale, basterebbe spostare il disco da i a j. Se $n = 1$ allora la soluzione consiste nelle tre fasi già illustrate, ciascuna delle quali comporta una chiamata di procedura ricorsiva.

La soluzione completa è data dalla Figura 5.39. La chiamata

```
towers(3, 1, 3)
```

che risolve il problema della Figura 5.38, provoca altre tre chiamate, e cioè

```
towers(2, 1, 2)
towers(1, 1, 3)
towers(2, 2, 3)
```

La prima e la terza causano a loro volta tre chiamate, per un totale di sette.

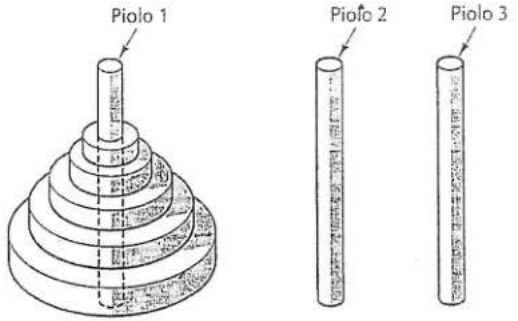


Figura 5.37 Configurazione iniziale del problema della torre di Hanoi con cinque dischi.

Al fine di poter gestire le procedure ricorsive abbiamo bisogno di uno stack per memorizzare i parametri e le variabili locali a ogni invocazione, analogamente a quanto mostrato nel caso dell'IJVM. Ogni volta che una procedura viene chiamata, in cima allo stack viene allocato un record d'attivazione per la procedura stessa. Il record d'attivazione di creazione più recente è quello in uso corrente. Nei nostri esempi lo stack cresce verso l'alto, dagli indirizzi di memoria più piccoli ai più grandi, proprio come nell'IJVM. Perciò il record d'attivazione più recente è caratterizzato da un indirizzo maggiore di tutti gli altri. Oltre al puntatore allo stack, che punta alla cima della pila, risulta spesso conveniente avere un puntatore al record d'attivazione, `FP` (*frame pointer*), che punta a una locazione nota del record d'attivazione (per esempio il puntatore di collegamento, come nell'IJVM, o la prima variabile locale).

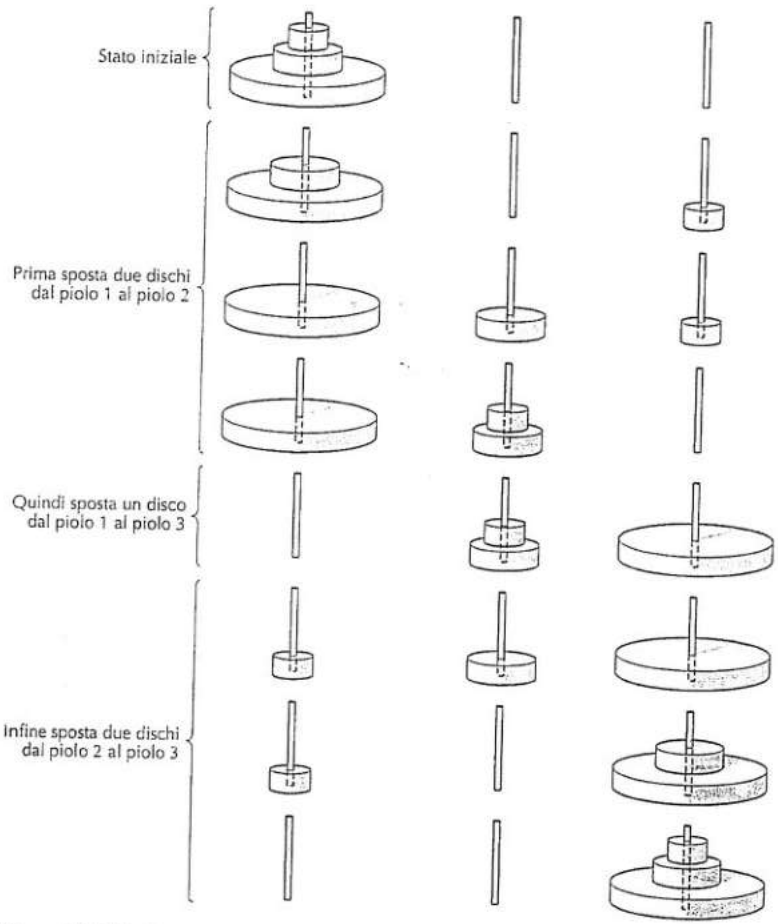


Figura 5.38 Passi necessari per la soluzione del problema della torre di Hanoi con tre dischi.

La Figura 5.40 mostra il record d'attivazione di una macchina con parole di 32 bit. La chiamata originale `towers` impila n, i e j in cima allo stack ed esegue quindi un'istruzione `CALL` che impila l'indirizzo di ritorno sullo stack, all'indirizzo 1012. Al momento dell'ingresso, la procedura memorizza nello stack il vecchio valore di `FP` alla locazione 1016 e incrementa il puntatore allo stack per allocare spazio sufficiente alle variabili locali. Poiché c'è una sola variabile locale di 32 bit (k), `SP` è incrementato di 4 in 4 fino a 1020. La Figura 5.40(a) mostra la situazione dello stack al termine di queste operazioni.


```
public void towers(int n, int i, int j) {
   int k;

   if (n == 1)
      System.out.println( "Sposta un disco da " + i + " a " + j);
   else {
      k = 6 – i – j;
      towers(n – 1, i, k);
      towers(1, i, j);
      towers(n – 1, k, j);
   }
}
```

Figura 5.39 Procedura per la soluzione del problema della torre di Hanoi.

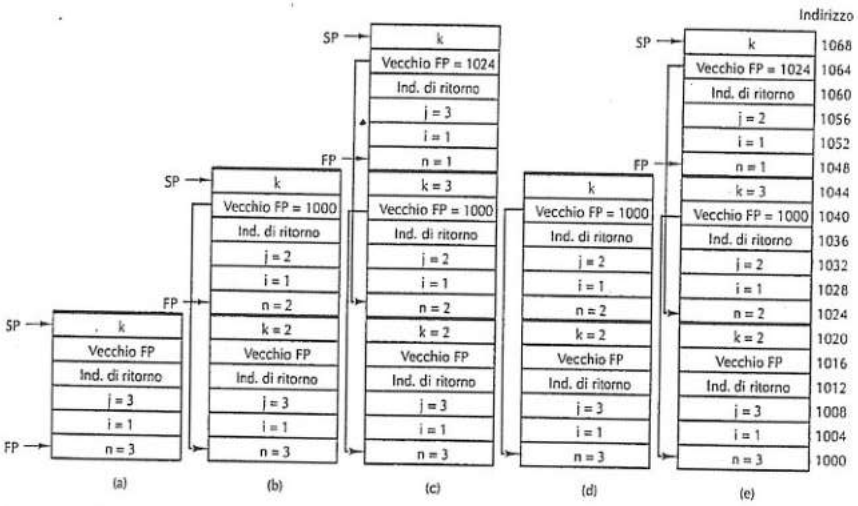


Figura 5.40 Lo stack in alcuni momenti dell'esecuzione del codice della Figura 5.39.

Le prime cose che una procedura invocata deve fare sono il salvataggio del vecchio `FP` (così che possa essere ripristinato all'uscita della procedura), la copia di `SP` in `FP` e l'eventuale incremento di `FP` di una parola, a seconda della locazione cui puntare nel nuovo record d'attivazione. In questo esempio `FP` punta alla prima variabile locale, ma nell'IJVM `LV` indirizzava il puntatore di collegamento. La gestione del puntatore al record d'attivazione può essere leggermente diversa su macchine distinte, a seconda che abbia l'indirizzo della base del record d'attivazione, come nella Figura 5.40, o della sua cima oppure di altro. A tal fine è utile confrontare le Figure 5.40 e 4.12 per comprendere due modi alternativi di gestire il puntatore di collegamento. Ci sono anche altri modi

possibili. Comunque sia, il punto chiave è poter effettuare un ritorno da procedura e ripristinare lo stato dello stack alla situazione in cui si trovava appena prima dell'invocazione della procedura corrente.

Il codice che si preoccupa di salvare il vecchio FP, che stabilisce il nuovo FP e accresce il puntatore allo stack per riservare spazio alle variabili locali si chiama **prologo della procedura** (*procedure prolog*). All'uscita dalla procedura si rende invece necessario ripulire lo stack, che rappresenta la fase di **epilogo della procedura**. Due delle caratteristiche più importanti di ogni computer sono la brevità e la velocità dei meccanismi di prologo ed epilogo delle procedure. Se sono lunghi e lenti, le chiamate di procedura ne soffriranno e i programmatori devoti all'efficienza impareranno a evitare di scrivere molte procedure brevi, cui preferiranno programmi lunghi, monolitici e poco strutturati. Le istruzioni ENTER e LEAVE del Core i7 sono state progettate proprio per far funzionare i prologhi e gli epiloghi delle procedure in modo efficiente. Naturalmente queste istruzioni hanno un loro modello di gestione del puntatore al record d'attivazione, per cui se il compilatore ha un modello diverso deve fare a meno di usarle.

Torniamo ora al problema della torre di Hanoi. Ogni chiamata di procedura aggiunge un nuovo record d'attivazione allo stack che verrà rimosso al suo ritorno. Per illustrare l'uso dello stack nell'implementazione delle procedure ricorsive, tracciamo le chiamate a partire da

```
towers(3, 1, 3)
```

La Figura 5.40(a) mostra lo stack appena prima della chiamata di questa procedura. Per prima cosa la procedura controlla il valore di n, e quando scopre che $n = 3$, assegna k e opera la chiamata

```
towers(2, 1, 2)
```

Al completamento di questa chiamata lo stack è quello della Figura 5.40(b) e la procedura ricomincia dall'inizio (a ogni chiamata di procedura l'esecuzione riparte dall'inizio). Anche questa volta il confronto n è diverso da 1, perciò la procedura inizializza il proprio k e chiama

```
towers(1, 1, 3)
```

Ora lo stack è nella situazione della Figura 5.40(c) e il program counter punta all'inizio della procedura. Questa volta però $n = 1$ e viene visualizzata una riga di risultato. Dopodiché la procedura rientra e rimuove un record d'attivazione, ripristinando FP e SP come indicato nella Figura 5.43(d). Successivamente l'esecuzione continua dall'indirizzo di ritorno, che è la seconda chiamata:

```
towers(1, 1, 2)
```

Questa provoca l'aggiunta di un nuovo record allo stack come mostrato nella Figura 5.40(e). Quindi viene visualizzata una nuova riga di risultato e il record d'attivazione è rimosso dallo stack. Le chiamate di procedura proseguono con questo andamento, finché non viene completata l'esecuzione della chiamata originaria, al che viene rimosso

dallo stack anche il record d'attivazione della Figura 5.40(a). Per capire davvero come funziona la ricorsione consigliamo di simulare con carta e penna l'intera esecuzione di

```
towers(3, 1, 3)
```

5.6.3 Coroutine

Nell'usuale sequenza di chiamata c'è una chiara distinzione tra procedura chiamante e procedura chiamata. Si consideri la Figura 5.41, dove la procedura A chiama la procedura B.

La procedura B svolge dei calcoli, e restituisce il controllo all'altra. A prima vista sembrerebbe una situazione simmetrica, perché né A né B costituiscono il programma principale, ma due procedure (la procedura A potrebbe essere stata invocata dal programma principale, ma ciò è irrilevante). Il controllo passa da A a B, durante la chiamata, mentre durante il rientro è trasferito da B ad A.

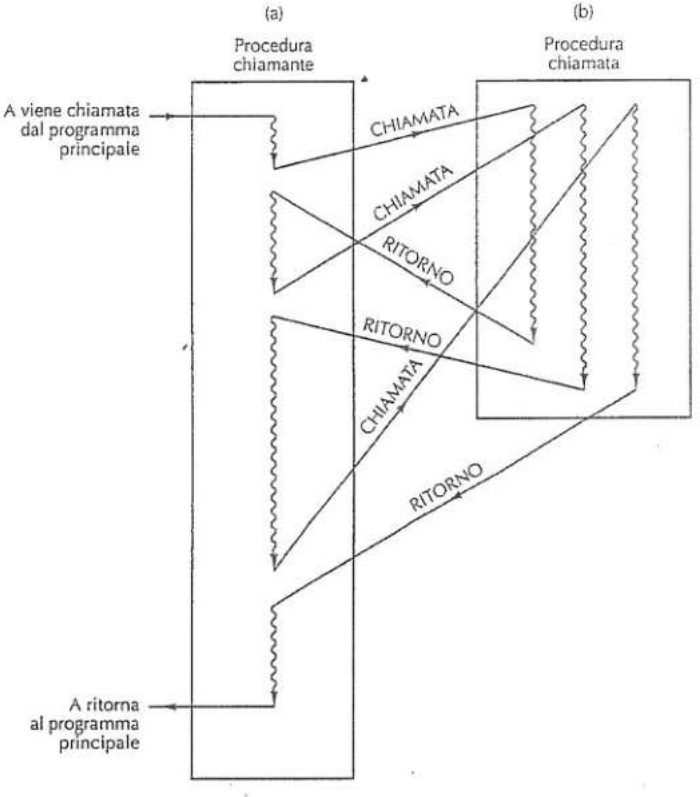


Figura 5.41 L'esecuzione di una procedura invocata inizia sempre dalla sua prima istruzione.

L'asimmetria nasce dal fatto che, nel primo caso l'esecuzione della procedura *B* comincia dal suo inizio, mentre nel secondo caso l'esecuzione non comincia dall'inizio di *A*, ma dall'istruzione immediatamente successiva alla chiamata. Se poi *A* chiama ancora *B*, l'esecuzione riparte dall'inizio di *B*, non dall'istruzione che segue il ritorno precedente. Ogniqualvolta *A* chiama *B* quest'ultima ricomincia dall'inizio, mentre *A* non ricomincia mai da capo, ma continua semplicemente ad andare avanti.

Questa differenza si riflette nel modo in cui il controllo è passato tra *A* e *B*. Per chiamare *B*, la procedura *A* si avvale dell'istruzione di chiamata di procedura, che memorizza l'indirizzo di ritorno (cioè l'indirizzo dell'istruzione che segue la chiamata) in un qualche luogo adatto allo scopo, per esempio in cima allo stack. Quindi pone l'indirizzo di *B* nel program counter, completando così la chiamata. Quando *B* ritorna, non usa l'istruzione di chiamata ma si avvale dell'istruzione di ritorno, che semplicemente fa il pop dell'indirizzo di ritorno dallo stack e lo copia nel program counter.

Alle volte è utile avere due procedure che si chiamano a vicenda nel modo illustrato nella Figura 5.42. Come in precedenza, quando *B* ritorna ad *A*, *B* salta all'istruzione appena successiva alla sua chiamata. Quando è invece *A* a passare il controllo a *B*, non salta all'inizio di *B* (tranne la prima volta) bensì all'istruzione successiva al "ritorno" più recente, vale a dire successiva alla chiamata di *A* più recente. Due procedure che si comportano in questo modo prendono il nome di **coroutine**.

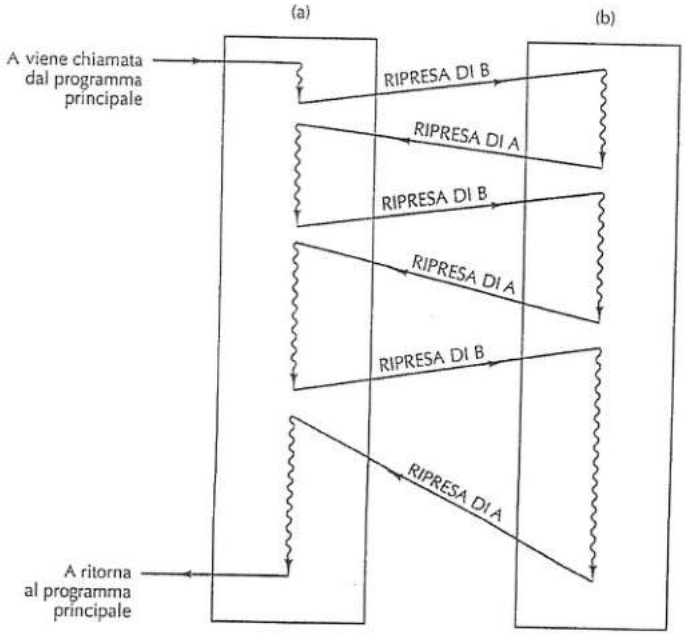


Figura 5.42 Alla ripresa di una coroutine, la sua esecuzione comincia da dove era stata interrotta, non dall'inizio.

Le coroutine sono usate comunemente per la simulazione dell'elaborazione parallela su singola CPU. Ogni coroutine gira in maniera pseudo-parallela alle altre, come se disponesse di una propria CPU. Questo stile di programmazione semplifica la scrittura di alcune applicazioni; inoltre è utile per la validazione del software destinato a girare su sistemi multiprocessore.

Le istruzioni CALL e RETURN non funzionano per le chiamate di coroutine perché, anche se l'indirizzo del salto viene recuperato dallo stack come per un'istruzione di ritorno, in questo caso è la stessa chiamata di coroutine a salvare l'indirizzo di ritorno per un successivo ritorno a essa. Sarebbe bello se esistesse un'istruzione per scambiare la cima dello stack con il program counter. In particolare questa istruzione effettuerebbe per prima cosa il pop del vecchio indirizzo di ritorno dallo stack verso un registro interno, quindi eseguirebbe una push del program counter sullo stack e infine copierebbe il contenuto del registro interno nel program counter. Dal momento che verrebbero effettuate sullo stack una push e un pop di una parola ciascuna, il puntatore allo stack non si sposterebbe. Questa istruzione si riscontra di rado, perciò viene spesso simulata per mezzo di numerose istruzioni.

5.6.4 Trap

Una **trap** ("trappola") è una specie di chiamata di procedura automatica effettuata quando si verificano certe condizioni causate da un programma e che sono in genere eventi rilevanti, ma di rara occorrenza. Ne è un buon esempio l'overflow: in molti computer, se il risultato di un'operazione aritmetica eccede il più grande numero rappresentabile, si verifica una trap, ovvero il controllo del flusso viene interrotto e riprende da una locazione di memoria prefissata, invece di proseguire in sequenza. In tale locazione di memoria si trova l'indirizzo per un salto a una procedura detta **gestore di trap**, che svolge le azioni appropriate al caso, come la stampa di un messaggio di errore. Viceversa non si verifica nessuna trap se il risultato di un'operazione è nell'intervallo dei numeri rappresentati dalla macchina.

Il concetto chiave delle trap è che sono fatte scattare da condizioni eccezionali causate dal programma stesso e rilevate dall'hardware o dal microprogramma. Un metodo alternativo per la gestione dell'overflow è di avere un registro di 1 bit asserito ogniqualvolta si verifica un overflow. Se un programmatore vuole controllare la presenza di un overflow deve includere un'istruzione esplicita di "salto se overflow asserito" dopo ogni istruzione aritmetica. Si tratta di una soluzione dispendiosa sia in tempo sia in spazio. Le trap fanno risparmiare tempo di esecuzione e memoria rispetto alla verifica affidata al controllo esplicito del programmatore.

Le trap potrebbero essere implementate tramite un test esplicito effettuato dal microprogramma (o dall'hardware); se viene rilevato un overflow, l'indirizzo di trap è caricato nel program counter. Ciò che fa scattare una trap a un certo livello potrebbe essere tenuto sotto controllo da un programma a un livello più basso. Il test effettuato dal microprogramma è ancora conveniente in termini di tempo rispetto al test svolto dal programmatore, perché può essere sovrapposto a qualche altra operazione. Inoltre consente un risparmio di memoria, perché basta che sia effettuato una sola volta, per esem-

pio nel ciclo principale del microprogramma, indipendentemente dal numero d'istruzioni aritmetiche che ci sono nel programma principale.

Alcune delle condizioni che causano comunemente trap sono gli overflow e gli underflow (interi o in virgola mobile), le violazioni di protezione, gli opcode non definiti, gli overflow di stack, i tentativi di utilizzare dispositivi di I/O inesistenti, i tentativi di fetch di una parola da un indirizzo dispari, la divisione per zero.

5.6.5 Interrupt

Gli **interrupt** ("interruzioni") sono cambiamenti nel flusso esecutivo causati non dal programma in esecuzione, ma da qualche altro problema, in genere determinato dall'I/O. Per esempio un programma potrebbe ordinare al disco di iniziare il trasferimento delle informazioni e di inviare un interrupt non appena il trasferimento termini. Come le trap, gli interrupt interrompono il programma in esecuzione e trasferiscono il controllo a un gestore deputato a svolgere le azioni appropriate. Al loro compimento, il gestore di interrupt restituisce il controllo al programma interrotto. È suo compito far riprendere il processo interrotto esattamente dallo stesso stato in cui si trovava al momento dell'interruzione, il che implica il ripristino di tutti i registri interni allo stato precedente all'interrupt.

La differenza essenziale tra le trap e gli interrupt è che le *trap* sono sincrone al programma e gli *interrupt* sono asincroni. Se un programma viene rieseguito un milione di volte con lo stesso input, le trap si verificheranno sempre nello stesso punto, mentre gli interrupt possono variare a seconda, per esempio, del momento in cui viene premuto il tasto d'invio al terminale. La ragione alla base della riproducibilità delle trap e all'irriproducibilità degli interrupt è che le trap sono causate direttamente dal programma, gli interrupt, invece, sono causati dal programma per lo più indirettamente.

Per comprenderne meglio il funzionamento consideriamo un tipico esempio di interrupt: un calcolatore deve scrivere sullo schermo una riga di caratteri. Per prima cosa il software di sistema raccoglie in un buffer tutti i caratteri da scrivere, inizializza una variabile globale `ptr` in modo che punti all'inizio del buffer e quindi assegna a una seconda variabile globale `count` il numero di caratteri da visualizzare. Successivamente verifica che il terminale sia pronto e, in caso affermativo, invia il primo carattere (per mezzo di registri come quelli della Figura 5.30). Dopo aver fatto partire l'attività di I/O, la CPU è libera di eseguire un altro programma o di svolgere lavoro di altro genere.

Nel volgere di poco tempo, il carattere viene visualizzato e l'interrupt può essere lanciato. In forma semplificata, i passi si succedono nell'ordine seguente.

AZIONI HARDWARE

1. Il controllore del dispositivo attiva una linea di interrupt sul bus di sistema per dar via alla sequenza di interrupt.
2. Non appena pronta a gestire l'interrupt, la CPU attiva sul bus un segnale di conferma (acknowledgment) dell'interrupt.
3. Quando il controllore del dispositivo vede confermata la ricezione del proprio segnale di interrupt, invia sulla linea dati un piccolo intero che lo identifica. Questo numero si chiama **vettore di interrupt**.

4. La CPU preleva il vettore di interrupt dal bus e lo salva temporaneamente.
5. La CPU impila il program counter e il registro `PSW` sullo stack.
6. Quindi la CPU usa il vettore di interrupt come indice per individuare il nuovo program counter all'interno di una tabella posta all'inizio della memoria. Per esempio, se il program counter è di 4 byte, il vettore di interrupt *n* corrisponde all'indirizzo 4 *n*. Questo nuovo program counter punta all'inizio della routine di servizio dell'interrupt per il controllore che l'ha lanciato. Spesso anche `PSW` viene caricato o modificato (per esempio per disabilitare ulteriori interrupt).

AZIONI SOFTWARE

1. La routine di servizio dell'interrupt comincia con il salvare (sullo stack o in una tabella di sistema) tutti i registri che utilizza per poterli ripristinare successivamente.
2. In genere ogni vettore di interrupt è condiviso da tutti i dispositivi dello stesso tipo, perciò non identifica univocamente il terminale che ha causato l'interrupt. Si può risalire al numero del terminale attraverso la lettura di alcuni registri di dispositivo.
3. A questo punto è possibile leggere ogni altra informazione sull'interrupt, come il codice di stato.
4. Nel caso si fosse verificato un errore di I/O, lo si può gestire da questo momento.
5. Le variabili globali `ptr` e `count` vengono aggiornate; la prima è incrementata perché punti al byte successivo, la seconda è decrementata a indicare che manca un byte in meno da visualizzare. Se `count` è ancora maggiore di 0, ci sono altri caratteri da visualizzare, e quello ora puntato da `ptr` viene copiato nel registro buffer di output.
6. Se richiesto, viene inviato un codice speciale per specificare al dispositivo o al controllore dell'interrupt che l'interrupt è stato trattato.
7. Ripristino di tutti i registri salvati.
8. Esecuzione dell'istruzione `RITORNO DA INTERRUPT` che ripristina la modalità e lo stato della CPU a prima del sollevamento dell'interrupt. Infine il computer riprende la sua attività dal punto in cui era stato interrotto.

Un concetto chiave legato agli interrupt è la **trasparenza**. Al verificarsi di un interrupt si intraprendono alcune azioni e si procede con l'esecuzione di un certo codice, ma quando tutto è finito il computer deve ripartire esattamente dallo stato che aveva prima dell'interrupt. Una routine di interrupt che manifesta questa proprietà si dice *trasparente*. Avere interrupt trasparenti ne semplifica la comprensione.

Se un computer ha un solo dispositivo di I/O, gli interrupt funzionano sempre nel modo appena descritto, e non resta da aggiungere nulla a quanto detto. D'altro canto un grosso calcolatore può disporre di numerosi dispositivi di I/O, molti dei quali attivi nello stesso momento e magari per conto di utenti diversi. Esiste una probabilità non nulla che un secondo dispositivo di I/O voglia generare il *proprio* interrupt mentre una routine di interrupt è già in esecuzione.

In questa evenienza si possono prevedere due strategie. La prima è far sì che ogni routine di interrupt per prima cosa disabiliti eventuali interrupt successivi, ancor prima di salvare i registri. Questo approccio mantiene lo schema semplice, visto che gli interrupt sono trattati in modo strettamente sequenziale, ma può generare problemi ai dispositivi che non tollerano attese oltre una certa durata. Per esempio, su una linea di comunicazione a 9600 bps arrivano caratteri ogni 1042 μs, pronti o meno a riceverli. Se il primo carattere non è stato ancora elaborato, quando arriva il secondo è possibile che alcuni dati vadano persi.

Se un calcolatore possiede dispositivi di I/O per i quali la gestione del tempo è critica, una migliore strategia consiste nell'assegnare una priorità a ogni dispositivo di I/O: alta per i dispositivi molto critici, bassa per quelli meno critici. Allo stesso modo anche la CPU dovrebbe definire alcune priorità, in genere determinate da un campo in PSW. Quando un dispositivo di priorità n causa un'interruzione, anche la routine di interrupt dovrebbe girare a livello di priorità n.

Durante l'esecuzione di una routine di interrupt di priorità n, ogni tentativo di interruzione di dispositivi di priorità inferiore viene ignorato fintanto che la routine non sia stata completata e che la CPU non sia tornata a eseguire codice di priorità più bassa. Viceversa gli interrupt provenienti da dispositivi a priorità maggiore dovrebbero essere trattati senza attesa.

Essendo le stesse routine di interrupt soggette a interruzioni, il modo migliore per mantenerne una gestione corretta è far sì che tutti gli interrupt siano trasparenti. Si consideri un semplice esempio di interrupt multipli. Un computer ha tre dispositivi di I/O, una stampante, un disco e una linea (seriale) RS232, rispettivamente con priorità 2, 4 e 5. Al tempo $t = 0$ è in esecuzione un programma dell'utente, ma all'improvviso, al tempo $t = 10$, si verifica un interrupt dalla stampante. Viene allora fatta partire la routine di servizio dell'interrupt (*Interrupt Service Routine*, ISR) della stampante, come mostrato nella Figura 5.43.

Al tempo $t = 15$, la linea RS232 invoca attenzione e genera un interrupt. Dal momento che la linea RS232 ha priorità maggiore (5) della stampante (2), il suo interrupt si verifica. Lo stato della macchina, impegnata al momento nell'esecuzione della routine di servizio dell'interrupt della stampante, viene impilato sullo stack e parte l'esecuzione della routine di servizio dell'interrupt da RS232.

Poco dopo, all'istante $t = 20$, il disco richiede a sua volta di essere servito. Tuttavia la sua priorità (4) è minore di quella della routine di interrupt correntemente in esecuzione (5), perciò l'hardware della CPU non conferma l'interrupt, e questo resta sospeso in attesa. All'istante $t = 25$ la routine della linea RS232 termina e lo stato viene ripristinato al momento precedente l'interruzione da parte di RS232, ossia ritorna all'esecuzione della routine di servizio della stampante di priorità 2. Non appena la CPU passa alla priorità 2 e prima che ne venga eseguita una sola istruzione, il controllo passa all'interrupt del disco di priorità 4 e viene eseguita la sua routine di servizio. Al termine di essa, la routine della stampante ottiene di poter proseguire. Infine, al tempo $t = 40$, tutte le routine di servizio degli interrupt sono state completate e il programma dell'utente riprende da dove era stato interrotto.

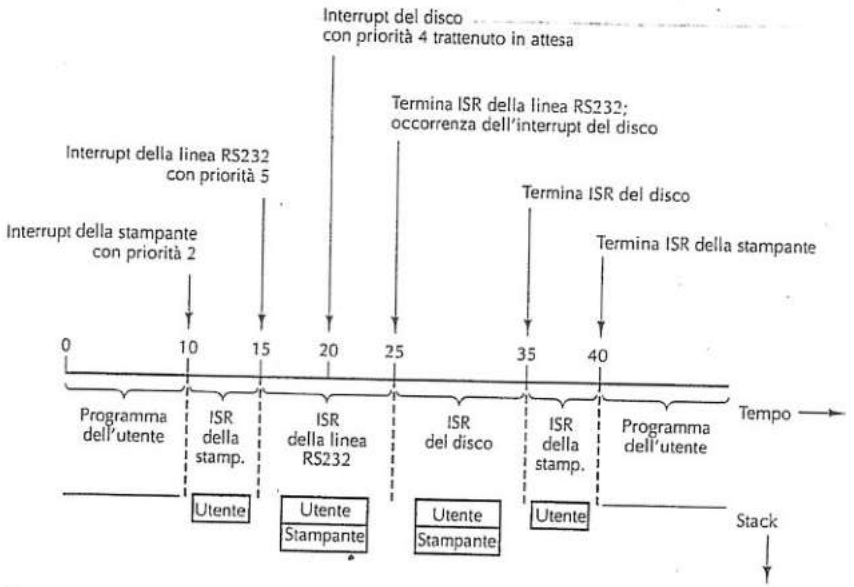


Figura 5.43 Sequenza di interrupt.

A partire dall'8088, tutte le CPU Intel sono state dotate di due livelli (priorità) di interrupt: mascherabile e non mascherabile. Gli interrupt non mascherabili sono usati in genere solo per segnalare eventi quasi catastrofici, come gli errori di parità della memoria. Tutti i dispositivi di I/O usano interrupt mascherabili.

Quando un dispositivo di I/O lancia un interrupt, la CPU usa il vettore di interrupt per indicizzare una tabella di 256 elementi al fine di trovare l'indirizzo della sua routine di servizio. Gli elementi della tabella sono descrittori di segmenti di 8 byte e la tabella può trovarsi ovunque in memoria. Un registro globale punta al suo inizio.

Essendoci un solo livello di interrupt utilizzabile, la CPU non ha modo di far sì che un dispositivo di priorità maggiore interrompa una routine di servizio di priorità media, e di garantire al contempo che ciò sia vietato a un dispositivo di priorità bassa. Per risolvere questo problema, le CPU Intel sono usate spesso in congiunzione con un controllore esterno di interrupt (per esempio un 8259A). Quando sopraggiunge il primo interrupt, diciamo di priorità n, la CPU viene interrotta. Se poi segue un interrupt di priorità maggiore, il controllore di interrupt causa una seconda interruzione. Se invece il secondo interrupt è di livello inferiore, viene trattenuto fino al completamento del primo. Affinché questo schema funzioni, il controllore di interrupt deve sapere quando termina la routine di servizio corrente, così la CPU deve inviare un comando quando il trattamento dell'interrupt corrente viene completato.

5.7 Un esempio: le torri di Hanoi

Dopo aver studiato l'ISA di tre macchine diverse possiamo ora trarre le somme presentando l'esempio di programma delle torri di Hanoi nel caso delle due macchine più complesse. Abbiamo già offerto una versione Java di questo esempio nella Figura 5.39. Nei paragrafi seguenti proponiamo due programmi in codice assemblativo per le torri di Hanoi.

Ricorreremo a un piccolo espediente: invece di fornire la traduzione della versione Java per il Core i7 e per l'OMAP4430, ne daremo una traduzione dal C per ovviare ad alcuni problemi dell'I/O di Java. La sola differenza è la sostituzione della chiamata Java a `println` con l'istruzione C standard

```
printf("Move a disk from %d to %d\n", i, j)
```

A questo proposito non è rilevante la sintassi delle stringhe di formattazione della `printf` (fondamentalmente una stringa è interpretata letteralmente a eccezione della sequenza %d, che specifica il formato di visualizzazione decimale per l'intero successivo). Ciò che qui conta è che la procedura viene chiamata con tre parametri: una stringa di formattazione e due interi. La ragione dell'utilizzo della versione C per il Core i7 e per l'OMAP4430 è che la libreria Java di I/O non è disponibile in forma nativa per queste macchine, mentre la libreria C lo è. La differenza è minima e coinvolge la sola istruzione di stampa sullo schermo.

5.7.1 Le torri di Hanoi nel linguaggio assemblativo del Core i7

La Figura 5.44 fornisce una possibile traduzione della versione C delle torri di Hanoi per il Core i7; buona parte del codice è di comprensione immediata. Il registro `EBP` è usato come puntatore al record d'attivazione. Le prime due parole sono direttive per il linker, così il primo vero parametro, n (N in questo caso, non essendo MASM case sensitive[1]) si trova alla posizione `EBP` + 8, seguito da i e j alle posizioni `EBP` + 12 e `EBP` + 16, rispettivamente. La variabile locale k si trova in `EBP` + 20.

La procedura comincia con l'attivare un nuovo record alla fine del precedente. Per far ciò copia `ESP` nel puntatore al record d'attivazione, `EBP`. Quindi confronta n a 1 e salta alla clausola `else` se $n > 1$. Il ramo `then` impila sullo stack tre valori: l'indirizzo della stringa di formattazione, i e j, chiama `printf`. I parametri sono impilati in ordine inverso, come richiesto dai programmi C. Ciò si rende necessario perché il puntatore alla stringa di formattazione si trovi in cima allo stack. Poiché `printf` ha un numero variabile di parametri, se i parametri fossero stati impilati in ordine, `printf` non avrebbe avuto modo di stabilire quante locazioni scorrere lungo lo stack per trovare la stringa di formattazione. Dopo la chiamata si somma 12 a `ESP` per rimuovere i parametri dallo stack. Ovviamente essi non vengono davvero cancellati dalla memoria, ma la modifica di `ESP` li rende inaccessibili alle normali operazioni sullo stack.

La clausola `else`, che comincia a `L1`, è di facile comprensione. Per prima cosa calcola $6 - i - j$ e memorizza il valore in k. Qualsiasi sia il valore di i e j, il terzo piolo è

[1] Un linguaggio si definisce *case sensitive* se distingue tra caratteri minuscoli e maiuscoli, ovvero se considera a e A due identificativi diversi (*N.d.T.*).

sempre $6 - i - j$. Memorizzandone il valore in k ci si risparmia di doverlo ricalcolare una seconda volta. Successivamente la procedura richiama se stessa tre volte, con parametri ogni volta differenti. Dopo ogni chiamata lo stack viene ripulito.

Le procedure ricorsive spesso confondono chi vi si avvicina per la prima volta, ma risultano facili da capire quando osservate a questo livello. Tutto ciò che succede è l'aggiunta dei parametri in cima allo stack e la chiamata stessa di procedura.

```
        .586                        ; compila per il Core i7
.MODEL FLAT
PUBLIC_towers                       ; esporta 'towers'
EXTERN_printf:NEAR                  ; importa printf
.CODE
_towers:  PUSH EBP                  ; salva EBP (puntatore al record d'attivazione) e decrementa ESP
          MOV EBP, ESP              ; imposta nuovo puntatore al record al di sopra di ESP
          CMP [EBP+8] ,1            ; if (n == 1)
          JNE L1                    ; salta se n è diverso da 1
          MOV EAX, [EBP+16]         ; printf(" ...", i, j);
          PUSH EAX                  ; si noti che i parametri i, j e la stringa
          MOV EAX, [EBP+12]         ; di formattazione sono impilati sullo stack
          PUSH EAX                  ; in ordine inverso. È la convenzione per le chiamate C
          PUSH OFFSET FLAT:format   ; è l'indirizzo della stringa formato
          CALL_printf               ; chiama printf
          ADD ESP, 12               ; rimuove i  parametri dallo stack
          JMP Done                  ; abbiamo concluso
L1:       MOV EAX, 6                ; comincia k = 6 – i – j
          SUB EAX, [EBP+12]         ; EAX = 6 – i
          SUB EAX, [EBP+16]         ; EAX = 6 – i – j
          MOV [EBP+20], EAX         ; k = EAX
          PUSH EAX                  ; comincia towers(n – 1, i, k)
          MOV EAX, [EBP+12]         ; EAX = i
          PUSH EAX                  ; push di i
          MOV EAX, [EBP+8]          ; EAX = n
          DEC EAX                   ; EAX = n – 1
          PUSH EAX                  ; push di n – 1
          CALL_towers               ; chiama  towers(n – 1, i, 6 – i – j)
          ADD ESP, 12               ; rimuove i  parametri dallo stack
          MOV EAX, [EBP+16]         ; comincia towers(1, i, j)
          PUSH EAX                  ; push di j
          MOV EAX, [EBP+12]         ; EAX = i
          PUSH EAX                  ; push di i
          PUSH 1                    ; push di 1
          CALL_towers               ; chiama towers(1, i, j)
          ADD ESP, 12               ; rimuove i  parametri dallo stack
          MOV EAX, [EBP+12]         ; comincia towers(n – 1, 6 – i – j, j)
          PUSH EAX                  ; push di j
          MOV EAX, [EBP+20]         ; EAX = k
          PUSH EAX                  ; push di k
          MOV EAX, [EBP+8]          ; EAX = n
          DEC EAX                   ; EAX = n – 1
          PUSH EAX                  ; push di n – 1
          CALL_towers               ; chiama towers(n – 1, 6 – i – j, j)
          ADD ESP, 12               ; modifica il puntatore allo stack
Done:     LEAVE                     ; preparazione all'uscita
          RET 0                     ; ritorno al chiamante
.DATA
format    DB "Move disk from %d to %d\n"  ; stringa di formattazione
END
```

Figura 5.44 Le torri di Hanoi nel linguaggio del Core i7.

5.7.2 Le torri di Hanoi nel linguaggio assemblativo dell'OMAP4430 ARM

Ora riproviamo la traduzione, questa volta per l'OMAP4430 ARM. Il codice è elencato nella Figura 5.45. Dal momento che il codice dell'OMAP4430 è particolarmente illeggibile, anche a livello del codice assemblativo e pur dopo molta pratica, ci siamo presi la libertà di cominciare con il definire alcuni simboli per renderlo più chiaro.

```
#define Param0                          r0
#define Param1                          r1
#define Param2                          r2
#define FormatPtr                       r0
#define k                               r7
#define n_minus_1                       r5
        .text
towers: push {r3, r4, r5, r6, r7, lr}   @ Salva l'indirizzo di ritorno e i registri usati
        mov r4, Param1
        mov r6, Param2
        cmp Param0, #1                  @ n = 1 ?
        bne else                        @ Se non vale n=1 salto alla sequenza di codice "else"
        movw FormatPtr, #:lower16:format  @ Carica il puntatore alla stringa "format"
        movt FormatPtr, #:upper16:format
        bl printf                       @ Stampa la stringa
        pop {r3, r4, r5, r6, r7, pc}
else:   rsb k, r1, #6                   @ k = 6 – i – j
        subs k, k, r2
        add n_minus_1, r0, #-1          @ Calcola (n-1) per la chiamata ricorsiva
        mov r0, n_minus_1
        mov r2, k
        bl towers                       @ Chiama towers(n–1, i, k)
        mov r0, #1
        mov r1, r4
        mov r2, r6
        bl towers                       @ Chiama towers(1, k, j)
        mov r0, n_minus_1
        mov r1, k
        mov r2, r6
        bl towers                       @ Chiama towers(n–1, k, j)
        pop {r3, r4, r5, r6, r7, pc}    @ Ripristina i registri usati e ritorna al chiamante
        .global main
main:   push {lr}                       @ Salva l'indirizzo di ritorno
        mov Param0, #3
        mov Param1, #1
        mov Param2, Param0
        bl towers                       @ Chiama towers(3, 1, 3)
        pop {pc}                        @ Preleva l'indirizzo di ritorno e ritorna al chiamante
format: .ascii "Move a disk from %d to %d\n\0"
```

Figura 5.45 Le torri di Hanoi nel linguaggio della CPU ARM OMAP4430.

Affinché possa funzionare, il programma deve essere fatto passare prima del suo assemblaggio attraverso un programma che si chiama *cpp*, un preprocessore C. Inoltre, in questo caso abbiamo usato lettere minuscole, perché l'assemblatore dell'OMAP4430 ARM ne richiede l'uso (casomai il lettore volesse provare a immettere il programma in una macchina OMAP4430).

Dal punto di vista algoritmico la versione per l'OMAP4430 è identica a quella per il Core i7. Entrambe cominciano con l'esaminare n e saltano alla clausola `else` se $n > 1$. La difficoltà principale nella versione per ARM è dovuta ad alcune proprietà dell'ISA.

Per cominciare, l'OMAP4430 deve passare l'indirizzo della stringa di formattazione alla *printf*, ma la macchina non può limitarsi a copiare l'indirizzo nel registro contenente il parametro in uscita, perché non c'è modo, con una sola istruzione, di mettere una costante di 32 bit in un registro, ma ce ne vogliono due: una `MOVW` e una `MOVT`.

Bisogna poi osservare che le operazioni sullo stack sono gestite automaticamente dalle istruzioni di PUSH e POP all'inizio e alla fine delle funzioni. Queste istruzioni gestiscono anche il salvataggio e il ripristino dell'indirizzo di ritorno, salvando LP quando si entra nella funzione e ripristinandolo prima dell'uscita.

5.8 Architettura IA-64 e Itanium 2

Intorno all'anno 2000 alcuni ingegneri di Intel iniziarono a pensare che la società stava arrivando al punto di aver spremuto del tutto la linea di processori IA-32. I nuovi modelli godevano ancora dei vantaggi portati dagli avanzamenti tecnologici, ovvero della più piccola dimensione dei transistor (che porta a una maggiore velocità di clock). Tuttavia diventava sempre più difficile escogitare nuovi stratagemmi per velocizzare ulteriormente le implementazioni, dato che i vincoli imposti dall'ISA IA-32 si rivelavano sempre più costrittivi.

Secondo alcuni ingegneri l'unica soluzione realistica era abbandonare IA-32 come linea principale di sviluppo e spostarsi verso un ISA completamente nuovo. È proprio ciò che Intel ha iniziato a fare: vi erano infatti progetti per due nuove linee. EMT-64 è un rifacimento ampliato del tradizionale ISA Pentium, con registri di 64 bit e spazio degli indirizzi a 64 bit. Questo nuovo ISA risolve il problema dello spazio degli indirizzi, ma presenta ancora le complicazioni implementative dei suoi predecessori. In pratica può essere visto come un Pentium esteso.

L'altra nuova architettura, sviluppata congiuntamente da Intel e Hewlett Packard, venne chiamata **IA-64**. Si tratta di una macchina completamente a 64 bit, non di un'estensione di una macchina a 32 bit. Inoltre si differenzia radicalmente dell'architettura IA-32 per molti aspetti. Inizialmente si rivolge al mercato dei server di fascia alta, ma Intel sperava di poter raggiungere anche il mercato dei desktop. Ciò non avvenne: nonostante i suoi difetti, gli acquirenti rifiutarono di abbandonare l'architettura IA-32. A ogni modo, la sua architettura è così diversa da tutto quanto fin qui studiato che merita una trattazione a parte. La prima implementazione dell'architettura IA-64 è la serie Itanium. Nel resto del paragrafo analizzeremo l'architettura IA-64 e la CPU Itanium 2 che l'implementa.

5.8.1 Il problema dell'ISA IA-32

Prima di addentrarci nei dettagli di IA-64 e dell'Itanium 2 è utile ricordare gli aspetti negativi dell'ISA IA-32 per capire quali sono i problemi che Intel ha inteso risolvere con la nuova architettura. La questione fondamentale è che IA-32 è un ISA datato, le cui proprietà sono inadatte alla tecnologia corrente. È un ISA CISC con istruzioni di lunghezza variabile e una miriade di formati differenti difficili da decodificare velocemente. La tecnologia attuale lavora meglio con ISA RISC che hanno una sola lunghezza per le istruzioni e opcode di lunghezza fissa facili da decodificare. Le istruzioni IA-32 possono essere suddivise in microistruzioni di tipo RISC in fase di esecuzione, ma ciò richiede hardware (cioè superficie del chip), porta via tempo e contribuisce alla complessità del progetto. Questi sono i primi punti a sfavore.

Inoltre IA-32 è un ISA orientato alla memoria ed è a due indirizzi. La maggior parte delle istruzioni referenzia la memoria e la maggior parte dei programmatori e dei compilatori non si cura di far riferimento continuo alla memoria. La tecnologia attuale favorisce gli ISA load/store che accedono alla memoria solo per recuperare gli operandi e trasferirli nei registri, altrimenti effettuano tutta la computazione usando istruzioni a tre indirizzi di registri. Per di più questa deficienza peggiorerà con il passare del tempo, visto che la frequenza di clock delle CPU cresce molto più velocemente rispetto alla velocità della memoria. Questo è il secondo punto a sfavore.

Inoltre IA-32 dispone di un insieme di registri piccolo e irregolare. Non solo questo imbriglia i compilatori, ma il numero esiguo di registri d'uso generale (quattro o sei, a seconda che si contino anche `EDI` ed `ESI`) esige che i risultati intermedi siano riversati continuamente in memoria, risultando in un sovrannumero di accessi anche laddove non sarebbero necessari. E con questo terzo punto a sfavore IA-32 perde il primo tempo della partita.

Cominciamo ora il secondo tempo. Il numero limitato di registri causa molte dipendenze, in special modo dipendenze WAR non necessarie, perché i risultati devono essere messi da qualche parte e non c'è più spazio nei registri. Per aggirare la mancanza di registri bisogna che l'implementazione gestisca la rinomina internamente, un artificio tra i peggiori, per nascondere i registri all'interno del buffer di riordinamento. Per evitare di bloccarsi frequentemente a seguito di miss della cache, le istruzioni devono essere eseguite fuori sequenza. Tuttavia la semantica di IA-32 specifica precise interruzioni di modo tale che le istruzioni fuori sequenza siano ritirate in ordine. Tutto ciò richiede però altro hardware molto complesso. Quarto punto a sfavore.

Per svolgere velocemente tutte queste operazioni occorre una pipeline con molti stadi. A sua volta, l'utilizzo di queste pipeline comporta che le istruzioni impieghino molti cicli per venir completate. Di conseguenza per garantire che le istruzioni introdotte nelle pipeline siano quelle giuste è essenziale una predizione dei salti molto precisa. Una predizione sbagliata richiede lo svuotamento della pipeline, un'operazione molto costosa, perciò anche un tasso di predizioni erronee abbastanza basso può causare un degrado sostanziale delle prestazioni. Quinto punto a sfavore.

Per alleviare il problema delle predizioni sbagliate il processore deve effettuare l'esecuzione speculativa, con tutti i problemi che comporta, specie quando i riferimenti alla memoria causano un'eccezione. Sesto punto a sfavore.

Non occorre continuare per capire che ci troviamo davanti a un vero problema. Non abbiamo neanche menzionato il fatto che gli indirizzi di 32 bit di IA-32 limitano i singoli programmi a 4 GB di memoria, un limite problematico su server di fascia alta. EMT-64 risolve questo problema, ma non tutti gli altri.

Dopo tutto la situazione di IA-32 può essere paragonata con ragione allo stato della meccanica celeste appena prima di Copernico. La teoria astronomica allora dominante stabiliva che la Terra fosse immobile e che i pianeti ruotassero con epicicli attorno a essa. D'altra parte, al migliorare delle osservazioni e al crescere delle discrepanze tra i dati osservati e il modello, furono aggiunti epicicli a epicicli finché l'intero modello collassò a causa della sua complessità interna.

Intel si trova ora nello stesso guaio. Una porzione enorme dei transistor del Pentium 4 è destinata a scomporre le istruzioni CISC, stabilire ciò che può essere svolto in parallelo, risolvere conflitti, fare predizioni, rimediare alle conseguenze di predizioni scorrette e ad altra ordinaria amministrazione, lasciando una frazione sorprendentemente piccola per svolgere i compiti che l'utente ha effettivamente richiesto. La conclusione cui è giunta Intel è l'unica sensata: buttare via tutto (IA-32) e ricominciare da capo dopo aver fatto tabula rasa (IA-64). EMT-64 consente un certo margine di respiro, ma si tratta davvero di nascondere il problema della complessità sotto un tappeto di novità.

5.8.2 Modello IA-64 e calcolo che utilizza il parallelismo esplicito

L'idea chiave alla base di IA-64 è di spostare il carico di lavoro dalla fase di esecuzione alla fase di compilazione. Nel Core i7, durante l'esecuzione la CPU riordina le istruzioni, rinomina i registri, fa lo scheduling delle unità funzionali e svolge molto altro lavoro per stabilire come occupare pienamente tutte le risorse hardware. Nel modello IA-64 il compilatore prevede tutto ciò e produce un programma che può essere eseguito così com'è, senza che l'hardware debba destreggiarsi tra tutti quei dettagli durante l'esecuzione. Per esempio, invece di dire al compilatore che la macchina ha otto registri quando in realtà ne ha 128, per poi cercare di evitare le dipendenze durante l'esecuzione, il modello IA-64 comunica al compilatore il numero effettivo di registri della macchina, così da poter produrre programmi senza conflitti tra registri. Analogamente, in questo modello il compilatore tiene traccia delle unità funzionali occupate e non emette istruzioni che usano unità funzionali non disponibili. Il modello che rende il parallelismo sottostante nell'hardware visibile al compilatore si chiama **EPIC** (*Explicitly Parallel Instruction Computing*, "calcolo che utilizza il parallelismo esplicito"). EPIC può essere considerato in un certo senso il successore di RISC.

Il modello IA-64 presenta numerose funzionalità per incrementare le prestazioni. Tra queste ce ne sono alcune per ridurre i riferimenti in memoria, per lo scheduling delle istruzioni, per ridurre i salti condizionati e altro. Vediamole ora una per una, riferendo come sono implementate nell'Itanium 2.

5.8.3 Riduzione degli accessi in memoria

L'Itanium 2 ha un modello di memoria semplice. La memoria consiste di 2^{64} byte di memoria lineare. Ci sono istruzioni per accedere a unità di memoria di 1, 2, 4, 16 e 10

byte, quest'ultima per i numeri in virgola mobile IEEE 745 di 80 bit. I riferimenti in memoria non devono essere allineati alle loro estremità naturali, ma se non lo sono ciò comporta un degrado delle prestazioni. L'ordinamento della memoria può essere sia little-endian sia big-endian, specificato tramite il valore di un bit appartenente a un registro a disposizione del sistema operativo.

Nei computer moderni l'accesso alla memoria costituisce un collo di bottiglia molto stretto, perché le CPU sono molto più veloci della memoria. Un modo per ridurre gli accessi alla memoria è disporre di una grande cache di primo livello sul chip e di una cache di secondo livello ancora più grande, vicina al chip. Oltre all'uso delle cache ci sono altri modi per ridurre gli accessi in memoria, alcuni dei quali sono sfruttati da IA-64.

Il modo migliore per velocizzare i riferimenti in memoria è... non usarli per niente. L'implementazione Itanium 2 del modello IA-64 ha 128 registri d'uso generale, di 64 bit ciascuno. I primi 32 registri sono statici, mentre i rimanenti 96 sono usati come registri di stack, in un modo molto simile allo schema a finestra di registri di altri processori RISC come l'UltraSPARC. A differenza dell'UltraSPARC il numero di registri visibili al programma è però variabile e può cambiare da procedura a procedura. Così ogni procedura ha accesso a 32 registri statici più un numero (variabile) di registri allocati dinamicamente.

All'atto della chiamata di una procedura il puntatore allo stack è incrementato, affinché i parametri d'ingresso siano visibili nei registri, ma non viene allocato alcun registro per le variabili locali. È la procedura stessa a decidere quanti registri necessita, e per allocarli incrementa lo stack pointer. Questi registri non devono essere salvati all'ingresso né ripristinati all'uscita, anche se la procedura deve aver cura di salvare e poi ripristinare i registri statici da modificare. Grazie al numero variabile di registri disponibili, legato alle reali esigenze di ogni procedura, si evita lo spreco dei già pochi registri; le chiamate di procedura possono così innestarsi a maggior profondità prima che si debbano riversare i registri in memoria.

L'Itanium 2 dispone anche di 128 registri in virgola mobile che rispettano il formato IEEE 745. Non vengono usati come registri di stack, ma il loro numero abbondante consente di memorizzare i risultati intermedi di molte operazioni in virgola mobile, senza bisogno di salvarli temporaneamente in memoria.

Ci sono anche 64 registri predicativi di 1 bit, otto registri di salto e 128 registri per applicazioni specifiche usati per vari propositi, come il passaggio di parametri tra programmi applicativi e il sistema operativo. La Figura 5.46 dà una panoramica dei registri dell'Itanium 2.

5.8.4 Scheduling delle istruzioni

Uno dei problemi principali del Core i7 è la difficoltà insita nel trovare uno scheduling delle varie istruzioni, a favore delle diverse unità funzionali, che eviti le dipendenze. Ci vogliono meccanismi esageratamente complicati per gestire tutti questi aspetti in fase d'esecuzione, e così un'ingente porzione dell'area del chip è dedicata al loro trattamento. IA-64 e Itanium 2 evitano questi problemi scaricando il lavoro sul compilatore. L'idea chiave è che un programma consista in una sequenza di **gruppi d'istruzioni**.

Entro un certo limite le istruzioni di un gruppo non sono mai in conflitto tra di loro, non usano più unità funzionali e risorse di quelle a disposizione della macchina, non contengono dipendenze RAW e WAW, ma solo dipendenze WAR limitate. I gruppi consecutivi d'istruzioni danno l'impressione di un'esecuzione strettamente sequenziale, laddove l'esecuzione del secondo gruppo non comincia finché non sia stato completato il primo. Tuttavia la CPU può iniziare a elaborare (in parte) il secondo gruppo non appena lo reputi sicuro.

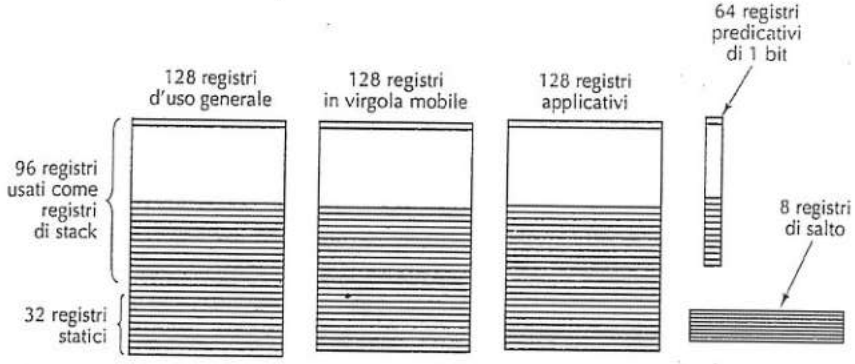


Figura 5.46 Registri di Itanium 2.

Una conseguenza di queste regole è che la CPU è libera di scegliere l'ordine di esecuzione delle istruzioni (inclusa l'esecuzione parallela), senza timore di conflitti. Il comportamento di un programma con un gruppo d'istruzioni che viola le regole non è definito. Spetta al compilatore riordinare il codice assemblativo generato dal programma sorgente al fine di soddisfare tutti i requisiti. In fase di sviluppo dell'applicazione il compilatore può mettere ciascuna istruzione in un gruppo per ottenere una compilazione rapida; è una soluzione facile, ma che porta a prestazioni molto scarse. Al momento di produrre il codice definitivo il compilatore può impiegare molto tempo per ottimizzarlo.

Le istruzioni sono organizzate in **pacchetti d'istruzioni** (*bundle*) di 128 bit, come mostrato nella parte alta della Figura 5.47. Ogni pacchetto contiene tre istruzioni di 41 bit e un campo template di 5 bit. Un gruppo d'istruzioni non deve prendere necessariamente un numero intero di pacchetti; può cominciare e terminare nel mezzo di un pacchetto.

Esistono più di cento formati d'istruzioni. La Figura 5.47 ne mostra uno tipico per le operazioni della ALU, come la `ADD` che somma due registri in un terzo. Il primo campo `GRUPPO DI OPERAZIONI` è il gruppo principale e specifica in sostanza la classe generale di appartenenza dell'istruzione, per esempio le operazioni ALU intere. Il campo successivo, `TIPO DI OPERAZIONE`, restituisce la particolare operazione richiesta, per esempio `ADD` o `SUB`. Quindi vengono i tre campi registro e infine c'è il `REGISTRO PREDICATIVO`, trattato a breve.

Figura 5.47 Pacchetto d'istruzioni IA-64 contenente tre istruzioni.

Il template del pacchetto specifica le unità funzionali necessarie al pacchetto e anche l'eventuale presenza e posizione di un'estremità del pacchetto d'istruzioni. Le unità funzionali principali sono le ALU (intera e non), le operazioni di memoria, le operazioni floating point e i salti. Ovviamente l'ortogonalità completa con sei unità e tre istruzioni richiederebbe 216 combinazioni, più altre 216 per indicare un marcatore di gruppo d'istruzioni dopo l'istruzione 0, altre 216 per indicare un marcatore di gruppo d'istruzioni dopo l'istruzione 1, e ancora 216 per indicare un marcatore di gruppo d'istruzioni dopo l'istruzione 2. Con soli 5 bit disponibili è consentito solo un numero molto limitato di queste combinazioni. C'è da dire che, se anche ci fosse un modo di specificare tre istruzioni in virgola mobile in un pacchetto d'istruzioni, non servirebbe a nulla, perché la CPU non può avviare simultaneamente tre istruzioni in virgola mobile. Le combinazioni previste sono quelle effettivamente attuabili.

5.8.5 Riduzione dei salti condizionati: attribuzione di predicati

Un'altra caratteristica importante di IA-64 è il modo nuovo di trattare i salti condizionati. Se ci fosse un modo di liberarsene quasi del tutto, le CPU ne beneficierebbero grandemente in semplicità e velocità. Di primo acchito si direbbe impossibile sbarazzarsi dei salti condizionati, perché i programmi sono pieni d'istruzioni `if`, e invece IA-64 usa una tecnica, detta **attribuzione di predicati** (*predication*), che può ridurre di molto il loro numero (August et al., 1998; Hwu, 1998). Descriviamola brevemente.

Nelle architetture tradizionali tutte le istruzioni sono incondizionate, nel senso che quando la CPU raggiunge un'istruzione non fa altro che svolgerla. Non c'è alcun dubbio del tipo "fare o non fare, questo è il dilemma". Al contrario, in un'architettura predicativa le istruzioni contengono condizioni (predicati) che stabiliscono se devono essere eseguite o meno. Questo cambio di paradigma da istruzioni incondizionate a istruzioni predicative rende possibile la riduzione di (molti) salti condizionati. Invece di scegliere tra due sequenze d'istruzioni incondizionate, si fondono tutte le istruzioni in una sola sequenza d'istruzioni predicative, usando predicati diversi per istruzioni diverse.

Per vedere come funziona l'attribuzione di predicati consideriamo la Figura 5.48, che mostra un semplice esempio di **esecuzione condizionata**, concetto precursore dell'attribuzione di predicati. La Figura 5.48(a) contiene un'istruzione `if`, mentre la Figura 5.48(b) ne mostra la traduzione in tre istruzioni: un confronto, un salto condizionato e un trasferimento.

Nella Figura 5.48(c) ci liberiamo del salto condizionato grazie a una nuova istruzione, `CMOVZ`, che è un trasferimento condizionato. Il suo funzionamento è controllare se il terzo registro, `R1`, vale 0; se è così copia `R3` in `R2`, altrimenti non esegue nulla.

```
if (R1 == 0)          CMP R1,0          CMOVZ R2,R3,R1
    R2 = R3;          BNE L1
                      MOV R2,R3
                  L1:
```

(a) (b) (c)

Figura 5.48 (a) Un'istruzione if. (b) Codice assemblativo per (a). (c) Un'istruzione condizionata.

Una volta resa disponibile un'istruzione che può copiare dati quando un certo registro vale 0, ci vuole poco per avere un'istruzione che trasferisce dati quando un certo registro non è 0: chiamiamola `CMOVN`. Con queste due istruzioni siamo sulla buona strada per l'esecuzione pienamente condizionata. Si immagini un'istruzione `if` con molti assegnamenti nel ramo `then` e molti altri nel ramo `else`. L'intera istruzione può essere tradotta in codice che imposta un certo registro a 0 se la condizione è falsa e a un altro valore se è vera. A seguire, il ramo `then` può essere compilato in una sequenza d'istruzioni `CMOVN` e il ramo `else` in una sequenza d'istruzioni `CMOVZ`.

Tutte queste istruzioni, l'impostazione del registro, le `CMOVN` e le `CMOVZ`, formano un singolo blocco elementare senza salti condizionati. L'ordine delle istruzioni può addirittura essere modificato, sia dal compilatore (che può anticipare gli assegnamenti prima del test) sia durante l'esecuzione. L'unico limite è che il valore della condizione deve essere noto prima che le istruzioni condizionate siano ritirate (verso la fine della pipeline). La Figura 5.49 illustra un semplice esempio con un ramo `then` e un ramo `else`.

```
if (R1 == 0) {        CMP R1,0          CMOVZ R2,R3,R1
    R2 = R3;          BNE L1            CMOVZ R4,R5,R1
    R4 = R5;          MOV R2,R3         CMOVN R6,R7,R1
} else {              MOV R4,R5         CMOVN R8,R9,R1
    R6 = R7;          BR L2
    R8 = R9;      L1: MOV R6,R7
}                     MOV R8,R9
                  L2:
```

(a) (b) (c)

Figura 5.49 (a) Un'istruzione if. (b) Codice assemblativo per (a). (c) Esecuzione condizionata.

Fin qui abbiamo presentato istruzioni condizionate molto semplici (appartenenti in realtà all'ISA IA-32), ma si tenga presente che tutte le istruzioni di IA-64 sono predicative. Ciò significa che l'esecuzione di ogni istruzione può essere resa condizionata. I 6 bit addizionali cui si è fatto riferimento prima servono a selezionare uno dei 64 registri predicativi di 1 bit. Perciò un'istruzione `if` sarà compilata in codice che imposta a 1 uno dei registri predicativi se la condizione è vera, e a 0 viceversa. Allo stesso tempo imposta automaticamente un altro registro predicativo al valore opposto. Grazie all'attribuzione di predicati le istruzioni macchina che costituiscono le clausole `then` ed `else` possono essere fuse in un solo flusso d'istruzioni, dove le prime si avvalgono del predicato originale, e le seconde del suo opposto. Nel momento del controllo verrà eseguito un solo un insieme di istruzioni.

Per quanto semplice, l'esempio della Figura 5.50 mostra l'idea fondamentale di come l'attribuzione di predicati sia utilizzabile per l'eliminazione dei salti. L'istruzione `CMPEQ` confronta due registri e asserisce il registro predicativo `P4` se sono uguali, lo azzera se sono diversi. Inoltre imposta un registro accoppiato, diciamo `P5`, secondo la condizione complementare, dopo di che è possibile elencare di seguito le istruzioni delle parti `then` ed `else`, ciascuna condizionata da un certo registro predicativo (indicato tra parentesi angolari). Lo schema si può applicare a qualsiasi tipo d'istruzione, purché opportunamente predicata.

IA-64 conduce questa idea alle sue estreme conseguenze, disponendo d'istruzioni di confronto che impostano registri predicativi e d'istruzioni aritmetiche e non, la cui esecuzione dipende da un certo registro predicativo. Le istruzioni predicative possono essere inserite nella pipeline in sequenza, senza stalli né altri problemi, ragion per cui sono così utili.

L'attribuzione di predicati su IA-64 avviene facendo sì che tutte le istruzioni siano eseguite. In fondo alla pipeline, al momento di ritirare un'istruzione si verifica se il suo predicato è vero. In caso affermativo l'istruzione è ritirata normalmente e il suo risultato è salvato nel registro destinazione. Se invece il predicato è falso, non si effettua alcun write-back e l'istruzione non ha effetto. L'attribuzione di predicati è esaminata in maggior dettaglio in Dulong (1998).

```
if (R1 == R2)                CMP R1,R2          CMPEQ R1,R2,P4
    R3 = R4 + R5;            BNE L1             <P4> ADD R3,R4,R5
else                         MOV R3,R4          <P5> SUB R6,R4,R5
    R6 = R4 – R5             ADD R3,R5
                             BR L2
                         L1: MOV R6,R4
                             SUB R6,R5
                         L2:
        (a)                      (b)                    (c)
```

Figura 5.50 (a) Un'istruzione if. (b) Codice assemblativo per (a). (c) Esecuzione predicativa.

5.8.6 Caricamenti speculativi

Un'altra caratteristica di IA-64 che ne velocizza l'esecuzione è la presenza di `LOAD` speculative. Se un'istruzione `LOAD` è speculativa e non va a buon fine, invece di causare un'eccezione, semplicemente si ferma e viene asserito un bit associato al registro da caricare che lo segnala come invalido. Si tratta proprio del poison bit introdotto nel Capitolo 4. Se il registro associato al poison bit viene usato successivamente, allora viene sollevata un'eccezione, che altrimenti non si verifica.

Il modo usuale di impiegare la speculazione è di permettere al compilatore di anticipare istruzioni `LOAD` in posizioni antecedenti al loro effettivo bisogno. Se cominciano prima è probabile che il loro risultato sia disponibile al momento opportuno. Il compilatore inserisce un'istruzione `CHECK` laddove necessita di un registro appena caricato. Se il valore è disponibile la `CHECK` si comporta come una `NOP` e l'esecuzione prosegue immediatamente. Se invece il valore non è ancora disponibile, l'istruzione successiva deve andare in stallo. Se si è verificata un'eccezione e il poison bit è asserito, anche l'eccezione sospesa viene sollevata.

In conclusione, le macchine che implementano l'architettura IA-64 devono la loro velocità a diversi fattori. Al centro c'è il motore RISC a tre indirizzi, con pipeline, di tipo load/store e che è allo stato dell'arte della materia. È già un miglioramento notevole rispetto alla complessa architettura IA-32.

In più IA-64 è provvista di un modello di parallelismo esplicito che richiede un compilatore in grado di stabilire quali istruzioni eseguire contemporaneamente senza conflitti e quindi capace di raggrupparle in pacchetti d'istruzioni. Così facendo la CPU può limitarsi alla pura gestione dello scheduling di un pacchetto senza inutili ripensamenti. Spostare il lavoro dall'esecuzione al momento della compilazione porta sempre a un successo.

Inoltre, l'attribuzione di predicati consente di fondere le istruzioni di entrambi i rami di un'istruzione `if` in un solo flusso, eliminando i salti condizionati, e quindi la necessità di predire quale ramo verrà percorso. Infine le `LOAD` speculative rendono possibile il fetch anticipato degli operandi, senza penalità in caso risultassero inutili.

Nel complesso l'architettura di Itanium è un progetto impressionante che sembra servire al meglio gli architetti e gli utenti, ma... avete un Itanium sul vostro computer? Ne abbiamo uno noi sul nostro? Conoscete qualcuno che ne possiede uno? Le risposte sono: no, no, no e (probabilmente) no. Più di due lustri dopo la sua introduzione, la sua adozione può essere descritta, in maniera politicamente corretta, come "mancata". Ciononostante, Intel continua a commissionare la produzione di sistemi basati su Itanium, anche se il loro utilizzo è limitato a server di alta fascia.

Torniamo alle motivazioni che in origine hanno portato alla creazione dell'architettura IA-64. Itanium fu progettato per sopperire alle numerose mancanze dell'architettura IA-32. Dato che la nuova architettura non è stata adottata, come ha potuto Intel affrontare queste mancanze? Come vedremo nel capitolo 8, la chiave di volta che ha permesso di portare avanti l'architettura IA-32 non è stata la riprogettazione dell'ISA, ma piuttosto il passaggio al calcolo parallelo, attraverso il progetto di chip multiprocessore.

Per maggiori informazioni sull'Itanium 2 e sulla sua micro-architettura si veda McNairy e Soltis, 2003; Rusu *et al.*, 200).

5.9 Riepilogo

L'architettura dell'insieme d'istruzioni è ciò a cui molte persone pensano in temini di "linguaggio macchina", anche se nelle macchine CISC è generalmente costruita su un livello più basso di microcodice. A questo livello la macchina ha una memoria orientata al byte o alla parola, costituita da decine di megabyte, e dispone d'istruzioni quali MOVE, ADD e BEQ.

Gran parte dei calcolatori moderni dispone di una memoria organizzata come una sequenza di byte, con gruppi di 4 o 8 byte a formare le parole. In genere ci sono tra gli 8 e i 32 registri, ciascuno lungo una parola. Su alcune macchine (come il Core i7) i riferimenti in memoria non devono essere allineati alle estremità naturali delle parole di memoria, mentre su altri (come l'OMAP4430 ARM) ciò è richiesto. Anche se non è richiesto l'allineamento delle parole le prestazioni sono migliori se le parole sono allineate.

Le istruzioni hanno in genere uno, due o tre operandi, indirizzati in modo immediato, diretto, a registro, indicizzato o in altri modi. Alcune macchine dispongono di un gran numero di complesse modalità d'indirizzamento. In molti casi, i compilatori non sono in grado di usarle in maniera efficace, e alcune modalità restano inutilizzate. Ci sono in genere istruzioni per trasferire dati, operazioni unarie e binarie, comprese le operazioni aritmetiche e quelle booleane, salti, chiamate di procedura, cicli e alle volte istruzioni per l'I/O. Le istruzioni in genere trasferiscono una parola dalla memoria in un registro (o viceversa), sommano, sottraggono, moltiplicano o dividono due registri o un registro e una parola di memoria, oppure confrontano due elementi nei registri o in memoria. Non è insolito trovare computer con più di 200 istruzioni nel loro repertorio. Le macchine CISC ne hanno spesso molte di più.

Il controllo del flusso a livello 2 si realizza tramite una varietà di primitive, inclusi i salti, le chiamate di procedura, le chiamate di coroutine, le trap e gli interrupt. I salti sono usati per concludere una sequenza d'istruzioni e cominciarne una nuova a una diversa locazione di memoria (potenzialmente distante). Le procedure sono un'astrazione di questo meccanismo e consentono di isolare una parte di un programma come un'unità che può essere richiamata da molti punti diversi. L'astrazione ottenuta con l'uso delle procedura, in una qualsiasi forma, è la base della programmazione moderna. Senza le procedure (o le loro equivalenti) sarebbe impossibile scrivere qualsiasi software moderno. Le coroutine permettono l'esecuzione simultanea di due thread di controllo. Le trap sono utilizzate per segnalare situazioni eccezionali, come l'overflow aritmetico. Gli interrupt permettono all'I/O di interagire concorrentemente con la computazione principale: la CPU riceve un segnale non appena l'I/O ha completato la sua attività.

Le torri di Hanoi costituiscono un problema semplice e divertente che ammette l'elegante soluzione ricorsiva che abbiamo esaminato. Sono anche state trovate soluzioni iterative al problema, ma queste sono molto più complicate e meno eleganti di quella ricorsiva che abbiamo visto.

Infine, l'architettura IA-64 usa il modello di calcolo EPIC per facilitare l'uso del parallelismo da parte dei programmi. Per migliorare le prestazioni si avvale di gruppi d'istruzioni, dell'attribuzione di predicati e di LOAD speculative. In definitiva potrebbe rappresentare un significativo miglioramento del Core i7, anche se scarica gran parte del peso della parallelizzazione sul compilatore. A ogni modo, è sempre meglio che il lavoro sia eseguito in fase di compilazione piuttosto che in fase di esecuzione.

PROBLEMI

1. Una parola di un computer little-endian con parole di 32 bit contiene il valore numerico 3. Se viene trasmessa a un computer big-endian byte per byte e ivi memorizzata, con il byte 0 al posto del byte 0, il byte 1 al posto del byte 1, e così via, quale sarà il suo valore numerico nella macchina big-endian quando viene letta come un intero di 32 bit?

2. In passato diversi computer e sistemi operativi hanno utilizzato spazi separati per dati e istruzioni, permettendo fino a 2^k indirizzi per il programma e lo stesso per i dati, con l'utilizzo di indirizzi di k bit. Per esempio, per k=32 un programma poteva accedere a 4GB di istruzioni e 4GB di dati, per un totale di 8 GB di spazio di indirizzamento. Visto che quando si utilizza questo schema un programma non può sovrascrivere se stesso, come può il sistema operativo caricare i programmi in memoria?

3. Si progetti un opcode espandibile, in cui le seguenti istruzioni possano essere tutte codificate in istruzioni di 36 bit:
   15 istruzioni con due indirizzi di 12 bit e un identificativo di registro di 4 bit
   650 istruzioni con un indirizzo di 12 bit e un identificativo di registro di 4 bit
   80 istruzioni senza indirizzi né registri.

4. Una certa macchina dispone d'istruzioni di 16 bit e d'indirizzi di 6 bit. Alcune istruzioni contengono un indirizzo, altre due. Se ci sono n istruzioni a due indirizzi, qual è il numero massimo d'istruzioni a un indirizzo?

5. È possibile progettare un opcode espandibile che consenta di codificare le seguenti istruzioni in istruzioni di 12 bit? Un registro è identificato da 3 bit.
   4 istruzioni con tre registri
   255 istruzioni con un registro
   16 istruzioni senza registri

6. Dati i seguenti valori di memoria e una macchina a un indirizzo dotata di accumulatore, quali valori sono caricati all'interno dell'accumulatore dalle istruzioni specificate?
   la parola 20 contiene 40
   la parola 30 contiene 50
   la parola 40 contiene 60
   la parola 50 contiene 70

   a. LOAD IMMEDIATE 20
   b. LOAD DIRECT 20
   c. LOAD INDIRECT 20
   d. LOAD IMMEDIATE 30
   e. LOAD DIRECT 30
   f. LOAD INDIRECT 30

7. Si confrontino le macchine a 0, 1, 2 e 3 indirizzi scrivendo un programma per il calcolo di
   $X = (A + B \times C) / (D - E \times F)$
   per ciascuna delle quattro macchine. Le istruzioni disponibili sono:

0 Indirizzi	1 Indirizzo	2 Indirizzi	3 Indirizzi
PUSH M	LOAD M	MOV (X = Y)	MOV (X = Y)
POP M	STORE M	ADD (X = X + Y)	ADD (X = X + Y)
ADD	ADD M	SUB (X = X – Y)	SUB (X = X – Y)

```
SUB          SUB M        MUL (X = X * Y)     MUL (X = X * Y)
MUL          MUL M        DIV (X = X / Y)     DIV (X = X / Y)
DIV          DIV M
```

M è un indirizzo di memoria di 16 bit. *X*, *Y* e *Z* sono o indirizzi di 16 bit o registri di 4 bit. La macchina a 0 indirizzi usa uno stack, la macchina a 1 indirizzo usa un accumulatore, mentre le altre due hanno 16 registri e istruzioni per operare su tutte le possibili combinazioni di locazioni di memoria e registri. SUB X, Y sottrae Y da *X* e SUB X, Y, Z sottrae *Z* da *Y* e pone il risultato in *X*. Con opcode di 8 bit e lunghezze d'istruzioni che sono multiple di 4 bit, di quanti bit ha bisogno ciascuna macchina per calcolare *X*?

8. Si escogiti un meccanismo d'indirizzamento che consenta di specificare, in uno spazio d'indirizzi molto grande, un insieme arbitrario di 64 indirizzi non necessariamente contigui, mediante un campo di 6 bit.

9. Si indichi uno svantaggio del codice auto-modificante non menzionato nel testo.

10. Si convertano le formule seguenti da notazione infissa a polacca inversa.
    a. A + B + C + D – E
    b. (A – B) × (C + D) + E
    c. (A × B) + (C × D) + E
    d. (A – B) × (((C – D × E) / F) / G) × H

11. Quali delle seguenti coppie di formule in notazione polacca inversa sono matematicamente equivalenti?
    a. A B + C + e A B C ++
    b. A B – C – e A B C – –
    c. A B × C + e A B C + ×

12. Si convertano le seguenti formule da notazione polacca inversa a notazione infissa.
    a. A B – C + D ×
    b. A B / C D / +
    c. A B C D E + × × /
    d. A B C D E × F / + G – H / × +

13. Si scrivano tre formule in notazione polacca inversa che non possono essere convertire in notazione infissa.

14. Si convertano le seguenti formule booleane infisse in notazione polacca inversa.
    a. (A AND B) OR C
    b. (A OR B) AND (A OR C)
    c. (A AND B) OR (C AND D)

15. Si converta la seguente formula infissa in notazione polacca inversa e si scriva del codice IJVM per valutarla.
    (5 × 2 + 7) – (4 / 2 + 1)

16. Quanti registri ci sono nella macchina che ha i formati d'istruzione della Figura 5.24?

17. Nella Figura 5.24 il bit 23 è usato per distinguere il formato 1 dal formato 2. Non c'è alcun bit per distinguere il formato 3. Come fa l'hardware a sapere quando usarlo?

18. Capita spesso nella programmazione di dover determinare dove si posiziona una variabile *X* rispetto a un intervallo [*A*, *B*]. Se si dispone di un'istruzione a tre indirizzi con operandi *A*, *B* e *X*, quanti bit codici di condizione si dovranno impostare per rappresentare il risultato del confronto?

19. Si descrivano un vantaggio e uno svantaggio dell'indirizzamento relativo al program counter.

20. Il Core i7 ha un bit codice di condizione che tiene traccia del riporto oltre il terzo bit a seguito di un'operazione aritmetica. A che cosa serve?

21. Un tuo amico ti ha appena svegliato telefonandoti nel cuore della notte e ti ha messo a parte delle sua nuova idea brillante: un'istruzione con due opcode. Gli consigli di correre all'ufficio brevetti o di tornare sui banchi di scuola?

22. Confronti del tipo
    `if (k == 0) ...`
    `if (a > b) ...`
    `if (k < 5) ...`
    sono comuni in programmazione. Si progetti un'istruzione che effettui questi test in modo efficiente: quali sono i campi dell'istruzione?

23. Dato il numero binario di 16 bit 1001 0101 1100 0011, si mostrino gli effetti delle seguenti operazioni:
    a. uno scorrimento verso destra di 4 bit con riempimento di zeri
    b. uno scorrimento verso destra di 4 bit con estensione del segno
    c. uno scorrimento verso sinistra di 4 bit
    d. una rotazione verso sinistra di 4 bit
    e. una rotazione verso destra di 4 bit.

24. Come si può azzerare una parola di memoria su una macchina priva d'istruzione CLR?

25. Si calcoli il valore dell'espressione booleana (*A* AND *B*) OR *C* per
    A = 1101 0000 1010 0011
    B = 1111 1111 0000 1111
    C = 0000 0000 0010 0000

26. Si escogiti un modo per scambiare il valore di due variabili *A* e *B* senza usare una terza variabile o registro. *Suggerimento*: si pensi all'operazione di OR ESCLUSIVO.

27. Su un certo computer è possibile svolgere le seguenti operazioni in meno tempo di quanto impiegato da una moltiplicazione: trasferire un numero da un registro all'altro, far scorrere entrambi di un certo quantitativo di bit e sommare i risultati. Sotto quali condizioni questa sequenza d'istruzioni è utile per calcolare "costante × variabile"?

28. Macchine diverse hanno diverse densità d'istruzioni (numero di byte richiesti per svolgere un certo calcolo). Tradurre i seguenti frammenti di codice Java nel linguaggio assemblativo del Core i7 e in linguaggio IJVM. Calcolare quindi il numero di byte richiesti da ciascuna macchina per calcolare le espressioni. Si ipotizzi che *i* e *j* sono variabili locali in memoria, per il resto si facciano sempre le assunzioni più ottimistiche
    a. `i= 3;`
    b. `i= j;`
    c. `i=j –1;`

29. Le istruzioni di ciclo illustrate nel testo servono per la gestioni di cicli `for`. Progettare invece un'istruzione che potrebbe essere utile per gestire comuni cicli `while`.

30. Se i monaci di Hanoi possono spostare un disco al minuto (non hanno fretta di portare a termine il compito perché le loro professionalità sono poco apprezzate sul mercato del lavoro di Hanoi), quanto impiegheranno a risolvere il problema con 64 dischi? Esprimere il risultato in anni.

31. Per quale motivo i dispositivi di I/O inviano il vettore di interrupt sul bus? Non potrebbero invece memorizzare questa informazione in una tabella di memoria?

32. Un certo computer usa il meccanismo DMA per leggere dal disco. Il disco ha 64 settori di 512 byte per ogni traccia. Il tempo di rotazione del disco è 16 ms. Il bus è largo 16 bit e ogni trasferimento sul bus impiega 500 ns. Un'istruzione di CPU richiede in media due cicli di bus. In che misura la CPU viene rallentata dal DMA?

33. Il trasferimento con DMA descritto nella figura 5.32 richiede 2 trasferimenti su bus per spostare i dati tra un dispositivo di I/O e la memoria. Si descriva come si possono migliorare le prestazioni del DMA utilizzando l'architettura del bus mostrata nella Figura 3.35.

34. Perché le routine di servizio dell'interrupt sono associate a priorità mentre le procedure normali non hanno priorità?

35. L'architettura IA-64 contiene un numero di registri insolitamente elevato, 64. La scelta di averne così tanti è legata in qualche modo all'uso dell'attribuzione di predicati? Se sì, in che modo? Se no, allora perché ce n'è così tanti?

36. Nel testo analizziamo il concetto delle LOAD speculative, però non si fa nessuna menzione a istruzioni di STORE speculative. Perché no? Perché sono essenzialmente analoghe alle LOAD speculative o c'è un'altra ragione per non parlarne?

37. Quando si vogliono collegare due reti locali vi si inserisce in mezzo un computer chiamato *bridge*. Ogni pacchetto trasmesso in ciascuna delle due reti causa un interrupt sul bridge, affinché possa stabilire se deve inoltrare il pacchetto da una parte all'altra. Si supponga che ci vogliono 250 μs per gestire l'interrupt e l'ispezione di ogni pacchetto, mentre il suo inoltro è a carico dell'hardware DMA e non aggrava il carico della CPU. Se tutti i pacchetti sono di 1 KB, qual è il massimo tasso di trasferimento dati che il bridge può tollerare su ciascuna delle due reti prima di cominciare a perdere pacchetti?

38. Nella Figura 5.40 il puntatore al record d'attivazione punta alla prima variabile locale. Di che informazione ha bisogno il programma per effettuare il ritorno da una procedura?

39. Si scriva una subroutine in linguaggio assemblativo per convertire in ASCII un intero binario con segno.

40. Si scriva una subroutine in linguaggio assemblativo per convertire una formula infissa in notazione polacca inversa.

41. Le torri di Hanoi non sono il solo problema ricorsivo popolare tra gli informatici. Un altro problema tra i più amati è il calcolo di $n!$. Si scriva una procedura in un linguaggio assemblativo per il calcolo di $n!$.

42. Se non si è convinti che la ricorsione è alle volte indispensabile si provi a programmare le torri di Hanoi senza usare la ricorsione e senza simulare la soluzione ricorsiva mantenendo lo stack?

CAPITOLO 6

Livello macchina del sistema operativo

Il "motivo conduttore" di questo libro è un calcolatore moderno visto come una serie di livelli, ciascuno responsabile di nuove funzionalità rispetto a quelli sottostanti. Abbiamo già incontrato il livello logico digitale, il livello di microarchitettura e quello di architettura dell'insieme d'istruzioni. È ora tempo di salire di un altro livello, nel dominio del sistema operativo.

Un **sistema operativo** è un programma che, dal punto di vista del programmatore, aggiunge una moltitudine di nuove istruzioni e caratteristiche di più alto livello rispetto a quelle fornite dal livello ISA. In genere il sistema operativo è implementato per la maggior parte via software, ma non esiste alcuna argomentazione teorica che ne vieti l'implementazione in hardware, come succede normalmente per i microprogrammi (quando presenti). Per brevità, ci riferiremo al livello che implementa il sistema operativo con il termine di livello **OSM** (**Operating System Machine**). Si veda al proposito la Figura 6.1.

Nonostante entrambi i livelli OSM e ISA siano astratti (nel senso che non sono veri livelli hardware), sono sostanzialmente differenti. L'insieme d'istruzioni del livello OSM contiene tutte le istruzioni disponibili ai programmatori di applicazioni, pressoché tutte le istruzioni del livello ISA, così come l'insieme delle nuove istruzioni aggiunte dal sistema operativo, dette **chiamate di sistema** (*system call*): una chiamata di sistema invoca un predeterminato servizio del sistema operativo, a tutti gli effetti una sua istruzione. Una chiamata di sistema tipica è la lettura di dati da un file. Nel prosieguo del testo indichiamo i nomi delle chiamate di sistema con un carattere che si distingue da quello del testo.

Il livello OSM è sempre interpretato. Quando un utente esegue un'istruzione OSM, come la lettura di dati da file, il sistema operativo svolge l'istruzione passo dopo passo, proprio come un microprogramma svolgerebbe un'istruzione ADD. D'altra parte, quando un programma esegue un'istruzione di livello ISA, questa viene calcolata direttamente

dal sottostante livello microarchitetturale, senza alcun intervento da parte del sistema operativo.

Figura 6.1 Livello macchina del sistema operativo.

In questo libro non possiamo che fornire un'introduzione molto succinta alla materia dei sistemi operativi, ma ci soffermeremo su tre argomenti importanti: il primo è la memoria virtuale, una tecnica messa a disposizione da molti sistemi operativi moderni per far sembrare che la macchina abbia più memoria di quanta ne ha davvero; il secondo è l'I/O su file, un concetto più ad alto livello delle istruzioni di I/O, analizzato nel capitolo precedente; il terzo argomento è il calcolo parallelo, ovvero il modo in cui i processi vengono eseguiti, come comunicano e si sincronizzano. Il concetto di processo è decisamente importante, e ne daremo una descrizione dettagliata nel corso del capitolo. Per il momento potete pensare a un processo come a un programma in esecuzione unito alle sue informazioni di stato (memoria, registri, program counter, stato dell'I/O e così via). Dopo aver analizzato questi principi in generale, vedremo come si applicano ai sistemi operativi di due delle nostre macchine paradigmatiche, cioè il Core i7 (Windows 7) e l'OMAP4430 ARM (Linux). L'ATmega168 non ha sistema operativo poiché è usato di norma all'interno di sistemi integrati.

6.1 Memoria virtuale

All'inizio dell'era informatica, le memorie dei computer erano costose e poco capienti. L'IBM 650, il calcolatore scientifico più avanzato del suo tempo (fine anni '50), aveva solo 2000 parole di memoria. Uno dei primi compilatori ALGOL 60 venne scritto per un computer con soltanto 1024 parole di memoria. Uno dei primi sistemi multiutente a condivisione di tempo girava abbastanza bene su di un PDP-1 con una memoria di 4096 parole di 18 bit, usata sia dal sistema operativo sia dai programmi degli utenti. In quel periodo, i programmatori impiegavano molto tempo nel tentativo di "assottigliare" i programmi perché entrassero in memorie minuscole. Alle volte si rendeva indispensabile usare un algoritmo molto più lento rispetto a un altro per il solo motivo che quello

più veloce era troppo "grande", ovvero nessun programma che lo usasse avrebbe potuto essere contenuto nella memoria del calcolatore.

La soluzione tradizionale a questo problema era l'uso di una memoria secondaria, per esempio di un disco. Il programmatore divideva il programma in un certo numero di pezzi, detti **overlay** ("ricoprimento"), ciascuno dei quali poteva entrare in memoria. Al fine di eseguire il programma si recuperava il primo overlay e lo si eseguiva per un po'. Al termine della sua esecuzione il primo pezzo leggeva l'overlay successivo e lo mandava in esecuzione e così via. Stava al programmatore ripartire il programma in parti, decidere dove memorizzarle in memoria secondaria, organizzare il trasferimento degli overlay tra la memoria principale e la secondaria, e quindi gestire l'intero procedimento di overlay senza ricevere alcun aiuto dal calcolatore.

Pur se usata per molti anni, questa tecnica richiedeva troppo lavoro da dedicare alla gestione degli overlay. Nel 1961 un gruppo di ricercatori di Manchester, in Inghilterra, propose un metodo per eseguire il processo di overlay automaticamente, in modo tale che il programmatore non avrebbe dovuto neanche accorgersi della sua presenza (Fotheringham, 1961).

Questo metodo, oggi chiamato **memoria virtuale**, presentava il vantaggio evidente di liberare il programmatore da un grosso lavoro di noiosa preparazione. Inizialmente utilizzata nell'ambito di progetti di ricerca sulla progettazione di sistemi di calcolo, a partire dagli anni '70 la memoria virtuale era divenuta disponibile su quasi tutti i calcolatori. Oggi anche i computer su chip singolo, tra cui il Core i7 e la CPU ARM OMAP4430, dispongono di sistemi di memoria virtuale molto sofisticati, come avremo modo di osservare nel corso di questo capitolo.

6.1.1 Paginazione

L'idea proposta dal gruppo di Manchester era quella di separare i concetti di spazio degli indirizzi e di locazioni di memoria. Si consideri, per esempio, un computer tipico di quel momento storico, dotato plausibilmente d'istruzioni con campi d'indirizzo di 16 bit e di 4096 parole di memoria. Un programma per questo computer avrebbe potuto indirizzare 65536 parole di memoria, poiché esistono 65536 (2^{16}) stringhe di 16 bit, ciascuna corrispondente a un diverso indirizzo di memoria. Si tenga presente che il numero di parole indirizzabili dipende esclusivamente dal numero di bit nell'indirizzo e non ha nulla a che fare con il numero di parole di memoria effettivamente disponibili. Lo **spazio degli indirizzi** di questo computer è fatto dai numeri 0, 1, 2, ..., 65535 perché è quello l'insieme degli indirizzi possibili. Il computer può avere comunque meno di 65536 parole di memoria.

Prima dell'invenzione della memoria virtuale, si guardava diversamente agli indirizzi minori di 4096 e a quelli uguali o maggiori di 4096. Anche se non venivano chiamati esplicitamente così, il primo gruppo d'indirizzi era considerato spazio utile degli indirizzi, il secondo spazio inutile degli indirizzi (inutile perché non aveva corrispondenza con gli effettivi indirizzi di memoria). Non si distingueva tra spazio degli indirizzi e locazioni di memoria, perché l'hardware imponeva una corrispondenza uno a uno tra di loro.

L'idea di separare lo spazio degli indirizzi e gli indirizzi di memoria è la seguente: in ogni istante sono direttamente accessibili 4096 parole di memoria, ma non c'è bisogno che corrispondano agli indirizzi di memoria tra 0 e 4095. Per esempio, potremmo "comunicare" al computer che d'ora in poi, ogniqualvolta ci si riferisce all'indirizzo 4096, occorre usare la parola di memoria all'indirizzo 0. Quando si referenzia l'indirizzo 4097, si deve usare la parola di memoria all'indirizzo 1; se si referenzia l'indirizzo 8191, bisogna usare la parola di memoria all'indirizzo 4095 e così via. In altri termini, abbiamo definito una corrispondenza tra lo spazio degli indirizzi e le effettive locazioni di memoria, illustrata nella Figura 6.2.

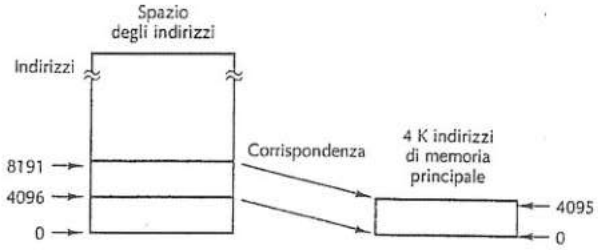


Figura 6.2 Corrispondenza tra gli indirizzi virtuali da 4096 a 8191 e gli indirizzi di memoria principale da 0 a 4095.

Secondo questa corrispondenza tra lo spazio degli indirizzi e le reali locazioni di memoria, una macchina con 4 KB di memoria, ma senza memoria virtuale, presenta una semplice corrispondenza fissa tra gli indirizzi compresi tra 0 e 4095 e le 4096 parole di memoria. Una domanda interessante è: "che cosa succede se un programma salta a un indirizzo tra 8192 e 12287?". Su di una macchina priva di memoria virtuale il programma causerebbe una trap che visualizzerebbe sullo schermo un messaggio adeguatamente severo, per esempio "memoria referenziata inesistente", e terminerebbe il programma. Su di una macchina con memoria virtuale si verificherebbero i seguenti avvenimenti:

1. il contenuto della memoria principale verrebbe salvato su disco;
2. verrebbero localizzate su disco le parole dalla 8192 alla 12287;
3. le parole dalla 8192 alla 12287 verrebbero caricate in memoria;
4. si modificherebbe la mappa degli indirizzi per mappare gli indirizzi da 8192 a 12287 nelle locazioni di memoria da 0 a 4095;
5. l'esecuzione continuerebbe come se non fosse successo nulla di insolito.

Questa tecnica di overlay automatico si chiama **paginazione** e le parti di programma lette dal disco si chiamano **pagine**.

È possibile anche una modalità più sofisticata di corrispondenza tra lo spazio degli indirizzi e le reali locazioni di memoria. Per evitare una possibile confusione, ci riferiremo agli indirizzi utilizzabili dal programma con il termine di **spazio degli indirizzi virtuali**, mentre ci riferiremo alle locazioni effettive, fisicamente cablate in memoria come allo **spazio degli indirizzi fisici**. Una **mappa di memoria** o **tabella delle pagine** specifica qual è l'indirizzo fisico corrispondente a ogni indirizzo virtuale. Facciamo qui l'ipotesi che ci sia spazio sufficiente sul disco per memorizzare l'intero spazio degli indirizzi virtuali (o quanto meno la sua porzione in uso).

I programmi vengono scritti come se ci fosse memoria sufficiente per l'intero spazio degli indirizzi virtuali, anche se non è così. I programmi possono caricare o memorizzare ogni parola nello spazio degli indirizzi virtuali, o saltare a qualsiasi istruzione situata ovunque nello spazio degli indirizzi virtuali, senza curarsi del fatto che in realtà non c'è sufficiente memoria fisica. In effetti, il programmatore può scrivere programmi senza neanche sapere che esiste la memoria virtuale, semplicemente pensando che il calcolatore abbia una grande memoria.

Questo concetto è di cruciale importanza e sarà poi messo a confronto con la segmentazione, che invece richiede che il programmatore sia conscio dell'esistenza dei segmenti. Lo sottolineiamo ancora una volta: la paginazione dà l'illusione al programmatore di una memoria grande, continua e lineare, delle stesse dimensioni dello spazio degli indirizzi virtuali. In realtà la memoria principale disponibile potrebbe essere più piccola (o più grande) dello spazio degli indirizzi virtuali. La simulazione di questa grande memoria principale mediante la paginazione non può essere individuata dal programma (se non eseguendo test sui tempi di esecuzione). Ogniqualvolta viene referenziato un indirizzo, l'istruzione o la parola di dati corrispondente sono percepite come presenti. Dal momento che il programmatore può lavorare come se la paginazione non esistesse, il meccanismo di paginazione si dice **trasparente**.

L'idea che un programmatore possa usare alcune caratteristiche inesistenti senza doversi preoccupare di come funzionano non è, dopo tutto, un'assoluta novità. L'insieme d'istruzioni a livello ISA include spesso un'istruzione `MUL`, anche se la microarchitettura sottostante manca di un moltiplicatore hardware. L'illusione che la macchina possa moltiplicare è sorretta dal microcodice. Allo stesso modo, la memoria virtuale messa a disposizione dal sistema operativo può fornire l'illusione che gli indirizzi virtuali si riflettano nella memoria reale, anche se non è così. Solo i programmatori del sistema operativo (e gli studenti della materia) devono sapere su che cosa si regge quest'illusione.

6.1.2 Implementazione della paginazione

Un requisito fondamentale per la memoria virtuale è disporre di un disco che contenga l'intero programma e tutti i dati. Il disco può essere meccanico a rotazione oppure allo stato solido. Per il resto del libro parleremo per semplicità di "disco" o "disco fisso", lasciando sottointeso che sono inclusi anche le unità allo stato solido. Concettualmente è più semplice pensare alla copia del programma su disco come all'originale, e ai pezzi trasferiti di quando in quando in memoria centrale come alle copie, piuttosto che il contrario. Naturalmente è importante mantenere l'originale aggiornato: quando vengono effettuati cambiamenti sulla copia in memoria centrale questi dovrebbero essere propagati all'originale (prima o poi).

Lo spazio degli indirizzi virtuali è suddiviso in un certo numero di pagine della stessa dimensione. Le dimensioni di pagina più in voga vanno dai 512 ai 64 KB, anche se occasionalmente vengono usate pagine di 4 MB. La dimensione di pagina è sempre una potenza di 2, per esempio 2^k, di modo che tutti gli indirizzi possano essere rappresentati con k bit. Allo stesso modo, anche lo spazio degli indirizzi fisici è suddiviso in porzioni che hanno le dimensioni di una pagina, di modo che ogni pezzo di memoria principale possa contenere esattamente una pagina. Questi pezzi di memoria principale che servono a contenere le pagine si dicono **blocchi di memoria** (*page frame*). La memoria principale della Figura 6.2 contiene un solo blocco di memoria, ma una memoria reale ne contiene in genere alcune migliaia.

La Figura 6.3(a) illustra un modo di suddividere in pagine di 4 KB i primi 64 KB di spazio degli indirizzi virtuali. In realtà qui stiamo parlando di 64 KB di memoria e di pagine di 4 K d'indirizzi; un indirizzo potrebbe essere lungo un byte, ma anche una parola (nei computer in cui indirizzi consecutivi individuano parole consecutive). La memoria virtuale della Figura 6.3 potrebbe essere implementata per mezzo di una tabella delle pagine grande quanto il numero di pagine presenti nello spazio degli indirizzi virtuali. Per semplicità, nella figura mostriamo solo i primi 16 elementi della tabella. Quando un programma cerca di accedere a una parola di memoria nei primi 64 KB del suo spazio degli indirizzi virtuali (o per fare il fetch di un'istruzione o di un dato, o per salvare alcuni dati) per prima cosa genera un indirizzo virtuale compreso tra 0 e 65532 (nell'ipotesi che gli indirizzi di parola siano divisibili per 4). La generazione di questo indirizzo può avvenire tramite una qualsiasi tecnica usuale d'indirizzamento: indicizzato, indiretto e così via.

La Figura 6.3(b) mostra una memoria fisica che consiste in otto blocchi di memoria di 4 KB. Questa memoria potrebbe essere limitata a 32 KB perché (1) è tutto ciò di cui dispone la macchina (potrebbe bastare a un processore integrato in una lavatrice o in un forno a microonde) oppure (2) perché il resto della memoria è stato allocato ad altri programmi.

Si consideri ora il meccanismo di corrispondenza tra un indirizzo virtuale di 32 bit e un indirizzo fisico della memoria principale. Dopo tutto la sola cosa comprensibile alla memoria sono gli indirizzi di memoria principale, non gli indirizzi virtuali, perciò bisogna essere in grado di fornirglieli. Ogni computer dotato di memoria virtuale è provvisto di un dispositivo atto alla corrispondenza tra indirizzi virtuali e fisici: si chiama MMU (*Memory Management Unit*, "unità di gestione della memoria") e può risiedere nel chip della CPU oppure in un chip separato che lavori a stretto contatto con la CPU. La MMU del nostro esempio mappa indirizzi virtuali di 32 bit in indirizzi fisici di 15 bit, perciò ha bisogno di registri d'ingresso di 32 bit e di registri d'uscita di 15 bit.

Si veda la Figura 6.4 per capire come funziona la MMU. Quando viene inviato un indirizzo virtuale di 32 bit alla MMU, questa divide l'indirizzo in un numero di pagina virtuale di 20 bit e in un offset di 12 bit all'interno della pagina (si ricordi che le pagine del nostro esempio sono di 4 KB). Il numero di pagina virtuale è usato per indicizzare la tabella delle pagine al fine di trovare l'elemento corrispondente alla pagina referenziata. Nel caso della Figura 6.4 il numero di pagina virtuale è 3 e così viene selezionato il terzo elemento della tabella delle pagine.

Pagina	Indirizzi virtuali
15	61440 – 65535
14	57344 – 61439
13	53248 – 57343
12	49152 – 53247
11	45056 – 49151
10	40960 – 45055
9	36864 – 40959
8	32768 – 36863
7	28672 – 32767
6	24576 – 28671
5	20480 – 24575
4	16384 – 20479
3	12288 – 16383
2	8192 – 12287
1	4096 – 8191
0	0 – 4095

(a)

Blocco di memoria	Primi 32 K di memoria principale: Indirizzi fisici
7	28672 – 32767
6	24576 – 28671
5	20480 – 24575
4	16384 – 20479
3	12288 – 16383
2	8192 – 12287
1	4096 – 8191
0	0 – 4095

(b)

Figura 6.3 (a) Primi 64 KB degli indirizzi virtuali suddivisi in 16 pagine da 4 KB. (b) Memoria principale di 32 KB suddivisa in otto blocchi di memoria di 4 KB.

Per prima cosa la MMU verifica nella tabella delle pagine se la pagina referenziata è presente in memoria centrale. Ovviamente, con 2^{20} pagine virtuali e soli otto blocchi di memoria, non tutte le pagine virtuali possono trovarsi contemporaneamente in memoria. Nello svolgere questa verifica la MMU esamina il **bit presente/assente** nell'elemento della tabella delle pagine. Nel nostro esempio il bit è a 1, a significare cha la pagina si trova correntemente in memoria.

Il passo successivo è il recupero del valore del blocco di memoria contenuto nell'elemento selezionato (6 in questo caso) e la sua copia nei 3 bit più significativi del registro d'uscita di 15 bit. Ci vogliono esattamente tre bit perché ci sono otto blocchi nella memoria fisica. Contemporaneamente a questa operazione si effettua la copia dei 12 bit meno significativi dell'indirizzo virtuale (il campo offset di pagina) all'interno dei 12 bit meno significativi del registro d'uscita, come mostrato nella figura. Questo indirizzo di 15 bit è ora pronto per essere inviato alla cache o alla memoria per la sua ricerca.

La Figura 6.5 mostra una possibile corrispondenza tra le pagine virtuali e i blocchi di memoria fisici. La pagina virtuale 0 si trova nel blocco di memoria 1 e la pagina virtuale 1 nel blocco di memoria 0. La pagina virtuale 2 non si trova in memoria. La pagina virtuale 3 si trova nel blocco di memoria 2. La pagina virtuale 4 non si trova in memoria. La pagina virtuale 5 si trova nel blocco di memoria 6 e così via.

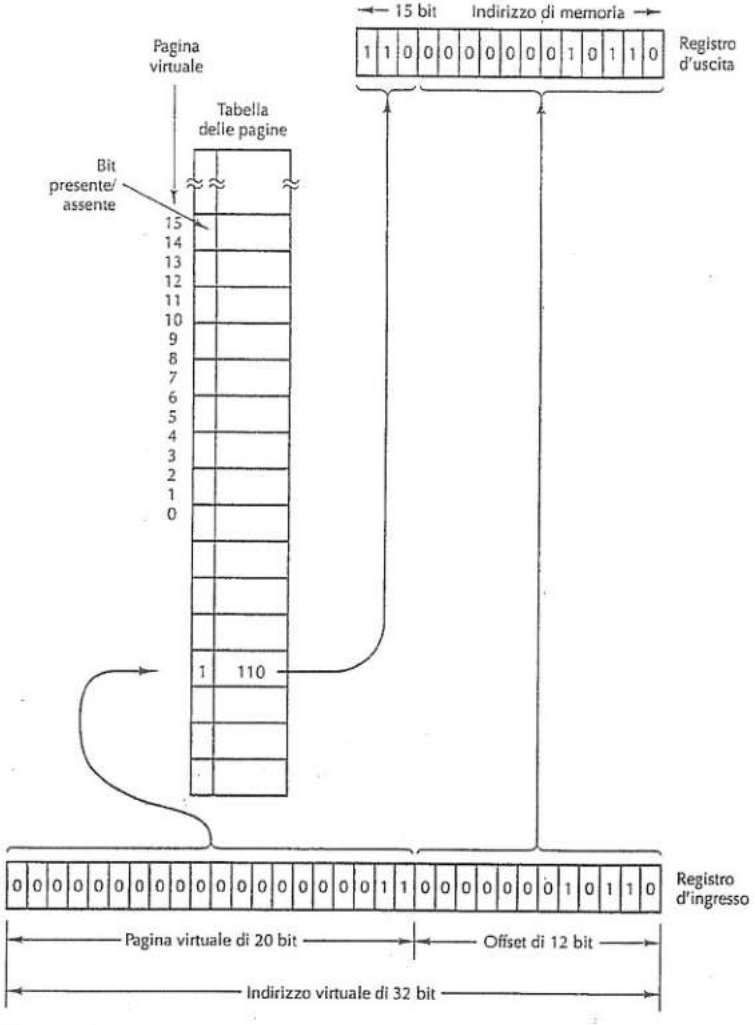


Figura 6.4 Formazione di un indirizzo fisico a partire da un indirizzo virtuale.

6.1.3 Paginazione a richiesta e working set

Nella precedente trattazione abbiamo supposto che la pagina virtuale referenziata si trovasse in memoria principale, ma questa assunzione non è sempre vera, perché non c'è spazio sufficiente in memoria principale per ospitare tutte le pagine virtuali. Quando si fa riferimento a un indirizzo contenuto in una pagina che non è presente in memoria si

solleva un **errore di pagina** (*page fault*). Dopo l'occorrenza di un errore di pagina è necessario che il sistema operativo legga dal disco la pagina richiesta, inserisca la sua nuova posizione all'interno della memoria fisica nella tabella delle pagine e ripeta l'istruzione che ha causato l'errore.

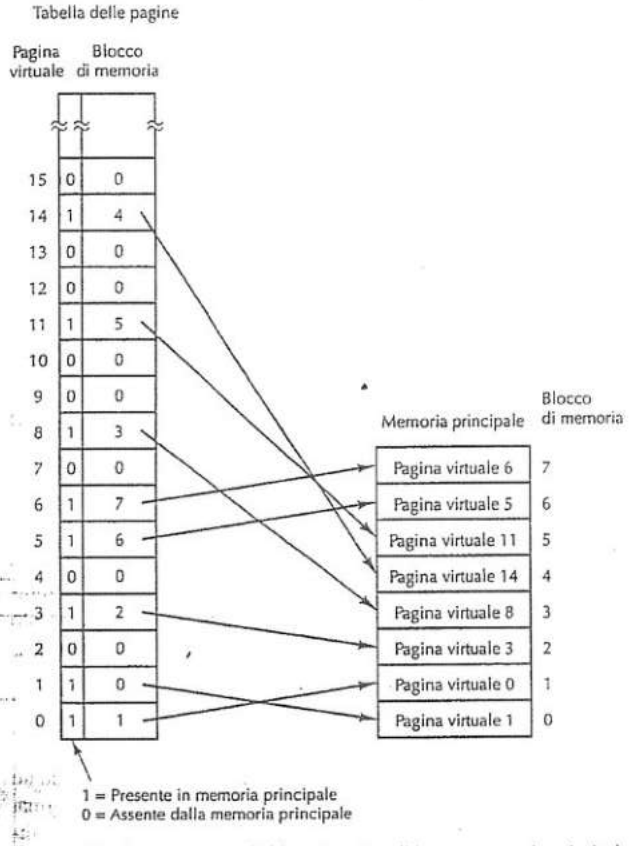


Figura 6.5 Assegnamento di 16 pagine virtuali in una memoria principale composta di otto blocchi.

Su una macchina con memoria virtuale è possibile lanciare l'esecuzione di un programma anche se la memoria principale non contiene alcuna sua istruzione. Bisogna semplicemente indicare nella tabella delle pagine che ogni pagina virtuale del programma si trova in memoria secondaria e non in memoria principale. Quando la CPU prova a effettuare il fetch della prima istruzione si verifica immediatamente un errore di pagina, che causa il caricamento in memoria della pagina contenente la prima istruzione e il suo

ingresso nella tabella delle pagine. Dopodiché può cominciare l'esecuzione della prima istruzione; se questa contiene due indirizzi su pagine differenti, e diverse dalla pagina dell'istruzione, allora si verificheranno altri due errori di pagina e dovranno essere recuperate altre due pagine prima che l'istruzione possa essere finalmente eseguita. L'istruzione successiva può a sua volta causare altri errori di pagina e così via.

Questo funzionamento della memoria virtuale si chiama **paginazione a richiesta**, in analogia al ben noto algoritmo di allattamento a richiesta dei bambini: il neonato viene allattato quando piange (piuttosto che in base a una precisa tabella oraria). Nella paginazione a richiesta una pagina è portata in memoria principale solo quando viene effettuata una richiesta, non prima.

L'interrogativo circa l'utilità della paginazione a richiesta è rilevante solo nella fase iniziale dell'esecuzione di un programma. Dopo un certo tempo d'esecuzione, le pagine necessarie saranno già state radunate in memoria. Se, tuttavia, il computer è usato a condivisione di tempo e i processi si alternano ogni 100 ms, ciascun programma verrà fatto ripartire molte volte nel corso della sua esecuzione. Poiché c'è una mappa di memoria per ogni programma e questa deve essere sostituita ogniqualvolta due programmi si danno il cambio, come succede per esempio in un computer a condivisione di tempo, il problema se reiterato diventa critico.

L'approccio alternativo si basa sull'osservazione che molti programmi non referenziano il loro spazio degli indirizzi in modo uniforme, bensì i riferimenti tendono a raggrupparsi su di un piccolo numero di pagine. Questo concetto è noto come **principio di località**. Un riferimento alla memoria può servire al fetch di un'istruzione o di dati, o alla memorizzazione di dati; per ogni istante di tempo t si può definire l'insieme delle pagine usate dai k riferimenti a memoria più recenti. Denning (1968) ha chiamato questo insieme **working set** ("insieme di lavoro").

Visto che il working set in genere cambia lentamente nel tempo, è possibile indovinare con un buon margine di certezza quali saranno le pagine necessarie a un programma nel momento in cui viene fatto ripartire, basandosi sul suo working set al momento in cui è stato fermato. Così le pagine possono essere caricate in anticipo prima di far partire il programma (se ci stanno in memoria).

6.1.4 Politica di sostituzione delle pagine

In teoria il working set, l'insieme delle pagine usate attivamente e frequentemente da un programma, può essere tenuto in memoria per ridurre il numero di errori di pagina. Tuttavia, i programmatori raramente conoscono le pagine del working set, perciò l'insieme deve essere costruito dinamicamente dal sistema operativo. Quando un programma referenzia una pagina che non si trova in memoria principale questa deve essere recuperata dal disco. D'altra parte, per farle spazio sarà opportuno in genere spostare un'altra pagina sul disco e si rende quindi necessario un algoritmo per decidere quale pagina rimuovere.

Effettuare questa scelta a caso è quasi sicuramente una cattiva idea. Se capitasse di scegliere la pagina appena caricata a seguito dell'errore, un altro errore di pagina si verificherebbe non appena venisse richiesto il fetch dell'istruzione successiva. Gran parte dei sistemi operativi cerca di predire quale sia la pagina di memoria meno utile,

ossia la cui mancanza produrrebbe meno effetti deleteri sull'esecuzione del programma. Un modo di agire è predire quando avverrà il successivo riferimento a ciascuna pagina e rimuovere quella per cui la predizione è più lontana nel futuro. In altre parole, si cerca di scegliere una pagina che non verrà usata per molto tempo invece di rimuoverne una che sarà richiesta a breve.

Un algoritmo molto diffuso, detto **LRU** (*Least Recently Used*), estromette la pagina usata meno recentemente perché a questa corrisponde la massima probabilità di non far parte del working set corrente. Nonostante LRU funzioni mediamente bene, ci sono situazioni patologiche, come quella che ora descriviamo, in cui fallisce miseramente.

Si immagini un programma che esegue un grosso ciclo che si estende per nove pagine virtuali e che gira su di una macchina con spazio in memoria fisica per sole otto pagine. La Figura 6.6(a) mostra la memoria principale nel momento in cui il programma raggiunge la pagina 7. Quando si tenta di fare il fetch di un'istruzione della pagina virtuale 8 si verifica un errore di pagina e si pone il problema di quale pagina estromettere. L'algoritmo LRU sceglie la pagina virtuale 0 perché è quella usata meno recentemente. La pagina virtuale 0 viene rimossa e la pagina virtuale 8 la rimpiazza, come illustrato nella Figura 6.6(b).

Dopo aver eseguito le istruzioni della pagina virtuale 8, il programma salta all'inizio del ciclo, alla pagina virtuale 0. Anche questo passo causa un errore di pagina. La pagina virtuale 0, quella appena estromessa, viene riportata in memoria. L'algoritmo LRU sceglie di rimuovere ora la pagina 1, conducendo alla situazione della Figura 6.6(c). Il programma si intrattiene per un po' nella pagina 0, quindi cerca di recuperare un'istruzione della pagina virtuale 1, causando un errore di pagina. Bisogna riportare indietro la pagina 1 ed estromettere la pagina 2.

Dovrebbe essere chiaro che in questo caso l'algoritmo LRU opera sempre la scelta peggiore (esistono anche altri algoritmi che falliscono in condizioni analoghe). Se però la memoria centrale disponibile eccede la dimensione del working set, allora LRU tende a minimizzare il numero di errori di pagina.

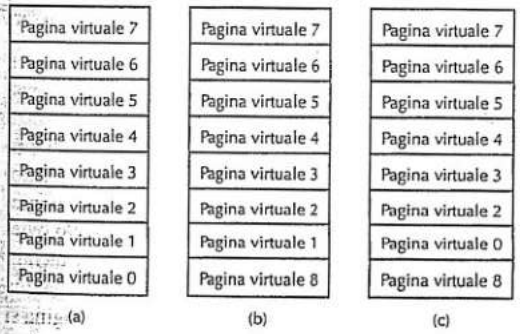


Figura 6.6 Fallimento dell'algoritmo LRU.

Un altro algoritmo di sostituzione di pagina è **FIFO** (*First-In First-Out*), che rimuove la pagina caricata meno recentemente, indipendentemente dal momento in cui sia stata referenziata. Ogni pagina è associata a un contatore, inizialmente posto a 0. Dopo la gestione di un errore di pagina, il contatore di ciascuna pagina presente in memoria viene incrementato di un'unità e il contatore della pagina appena immessa è posto a 0. Quando si rende necessario scegliere un pagina da estromettere, si seleziona quella con contatore più alto, cioè la pagina che ha assistito a più errori di pagina e che quindi è stata caricata prima di qualsiasi altra pagina presente in memoria, il che la rende (si spera) probabilmente meno utile delle altre.

Se il working set è più grande del numero di blocchi di memoria disponibili, non esiste alcun algoritmo, a meno che non abbia doti divinatorie, che possa dare buoni risultati e si verificheranno frequenti errori di pagina. Se un programma genera frequentemente e continuamente errori di pagina, allora è detto **thrashing** ("sconfitto, battuto"). Viceversa, se un programma usa una grande porzione dello spazio degli indirizzi virtuali ma ha un working set piccolo, che cambia lentamente nel tempo e che sta tutto in memoria principale, allora darà ben pochi problemi. Questa osservazione è vera anche per quei programmi che, nel corso della loro vita, usano un numero di parole della memoria virtuale centinaia di volte maggiore del numero di parole disponibili in memoria principale.

Se la pagina che sta per essere estromessa non è stata mai modificata dal momento in cui è stata caricata (cosa molto probabile se la pagina contiene soltanto istruzioni), non è necessario riscriverla su disco, perché questo ne contiene una copia fedele. Se invece è stata modificata, la copia su disco non è più accurata e la pagina deve essere riscritta.

Se esistesse un modo per stabilire quando una pagina è rimasta invariata dal suo caricamento (e in tal caso la pagina si dice "pulita", "intatta") o quando ha subito dei cambiamenti (e allora si dice "sporca"), si eviterebbero tutte le riscritture di pagine pulite, risparmiando molto tempo. Molti calcolatori dispongono nella MMU di 1 bit per pagina, detto appunto *dirty bit*, posto a 0 quando una pagina viene caricata e asserito dal microprogramma o dall'hardware quando è fatta oggetto di salvataggi (cioè quando è stata "sporcata"). Il sistema operativo può capire se una pagina è pulita o sporca osservando il suo dirty bit, e così decidere se va riscritta o meno.

6.1.5 Dimensione di pagina e frammentazione

Nell'eventualità in cui il programma e i dati dell'utente riempiano esattamente un numero intero di pagine, quando si trovano in memoria non causeranno alcuno spreco di spazio, mentre in caso contrario l'ultima pagina conterrà dello spazio inutilizzato. Per esempio, se il programma e i dati richiedono 26.000 byte su una macchina con pagine di 4096 byte, le prime sei pagine verranno riempite per un totale di $6 \times 4096 = 24.576$ byte, mentre l'ultima pagina conterrà $26.000 - 24.576 = 1424$ byte. Dal momento che in ogni pagina c'è spazio per 4096 byte, 2672 byte verranno sprecati. Ogni volta che la settima pagina si trova in memoria questi byte "vuoti" ne occuperanno una parte senza però svolgere alcuna funzione. Il problema legato a questo tipo di spreco di byte si chiama **frammentazione interna** (perché lo spazio inutilizzato si trova all'interno di una pagina).

Se la dimensione di pagina è di n byte, la quantità media di spazio sprecato nell'ultima pagina per frammentazione interna sarà di $n/2$ byte, il che sembrerebbe suggerire l'adozione di dimensioni di pagina più piccole per minimizzare lo spreco. D'altro canto a una dimensione di pagina piccola corrisponde un gran numero di pagine e quindi un tabella delle pagine più grande. Se la tabella delle pagine è implementata in hardware, le sue dimensioni richiedono un maggior numero di registri per immagazzinarla, il che eleva i costi di produzione del calcolatore. Inoltre, anche i tempi di caricamento e salvataggio dei registri aumentano ogni volta che un programma viene fatto partire o viene fermato.

D'altra parte, le pagine piccole fanno un uso poco efficiente della larghezza di banda del disco: visto che bisogna aspettare sempre almeno 10 ms prima che un trasferimento possa cominciare (tempo di ricerca più ritardo rotazionale), è evidente che i trasferimenti sono tanto più efficienti quanto più byte trasferiscono. Trasferire 8 KB alla velocità di 100 MB/s aggiunge solo 70 µs rispetto al trasferimento di 1 KB.

Tuttavia, le pagine piccole presentano il vantaggio di causare meno thrashing, essendo il working set costituito da un gran numero di piccole regioni separate dello spazio degli indirizzi virtuali. Si consideri per esempio una matrice A di 10.000×10.000 elementi, memorizzata come A [1,1], A [2,1], A [3,1] e così via, in parole consecutive di 8 byte. L'ordinamento per colonna implica che gli elementi della riga 1, A [1,1], A [1,2], A [1,3] e così via, si trovano distanti 80.000 byte. Se un programma svolgesse i propri calcoli su tutti gli elementi di questa riga, dovrebbe utilizzare 10.000 regioni, ciascuna separata dalla successiva da 79.992 byte. Se la dimensione di pagina è di 8 KB ciò richiede una disponibilità totale di 80 MB per contenere tutte le pagine in uso.

La dimensione di pagina di 1 KB richiederebbe invece soli 10 MB di RAM per contenere tutte le pagine. Se la memoria a disposizione è di 32 MB, il programma andrebbe in thrashing con pagine di 8 KB, ma non con pagine di 1 KB. Tutto considerato, la tendenza attuale si orienta verso dimensioni di pagina grandi. Nella pratica, il minimo al giorno d'oggi è 4 KB.

6.1.6 Segmentazione

La memoria virtuale vista fin qui è unidimensionale perché gli indirizzi virtuali vanno da 0 a un certo indirizzo massimo, in ordine progressivo. In molte situazioni può essere decisamente preferibile disporre di due o più spazi degli indirizzi virtuali separati piuttosto che di uno solo. Per fare un esempio, un compilatore potrebbe avere molte tabelle che si riempiono man mano che la compilazione procede, tra cui:

1. la tabella dei simboli per contenere i nomi e gli attributi delle variabili;
2. il codice sorgente, memorizzato per la visualizzazione del listato;
3. una tabella per contenere tutte le costanti intere e in virgola mobile utilizzate;
4. l'albero sintattico del programma;
5. lo stack, utilizzato per le chiamate di procedura del compilatore.

Ciascuna di queste tabelle cresce continuamente durante la compilazione. L'ultima addirittura cresce e decresce in modi imprevedibili. In una memoria unidimensionale

queste cinque tabelle sarebbero state allocate in unità contigue dello spazio degli indirizzi virtuali, come illustrato nella Figura 6.7.

Figura 6.7 In uno spazio degli indirizzi unidimensionale due tabelle in espansione possono collidere.

Se un programma ha un numero eccezionalmente elevato di variabili, la parte d'indirizzi allocata per la tabella dei simboli si potrebbe riempire, nonostante ci sia molto spazio libero nelle altre tabelle. Naturalmente il compilatore potrebbe cavarsela con un messaggio di errore comunicando l'impossibilità di proseguire la compilazione per la presenza di troppe variabili, ma non è un comportamento molto leale, specie considerando tutto lo spazio libero delle altre tabelle.

Un'altra possibilità prevede che il compilatore si comporti come Robin Hood, sottraendo spazio alle tabelle che ne hanno troppo per ridistribuirlo a quelle che ne sono povere. Questo rimescolamento è possibile, ma è un po' come gestire manualmente gli overlay: se va bene è una seccatura, se va male diventa un grosso carico di lavoro tedioso e poco remunerativo.

C'è bisogno di un modo per liberare il programmatore dalla gestione dell'espansione e della contrattura delle tabelle, analogamente al modo in cui la memoria virtuale elimina la preoccupazione di dover organizzare i programmi in overlay.

Una soluzione semplice è prevedere molti spazi degli indirizzi completamente indipendenti, detti **segmenti**. Ogni segmento consiste in una sequenza lineare d'indirizzi, che va da 0 a un valore massimo. La lunghezza di ciascun segmento può variare a sua volta da 0 a un massimo consentito: segmenti diversi potrebbero avere, e in genere hanno, lunghezze diverse. Per di più la lunghezza dei segmenti potrebbe cambiare durante l'esecuzione: la lunghezza del segmento dello stack potrebbe essere incrementata dopo ogni push e decrementata dopo ogni pop.

Poiché ciascun segmento costituisce uno spazio separato d'indirizzi, i diversi segmenti possono crescere o decrescere indipendentemente, senza influenzarsi a vicenda. Se uno stack ha bisogno di più spazio per crescere all'interno del proprio segmento, allora può prenderselo perché non c'è nient'altro nel suo spazio degli indirizzi con cui possa collidere.

Ovviamente può sempre succedere che un segmento venga riempito completamente, ma accade molto raramente, perché la sua dimensione è di solito molto grande. Per specificare un indirizzo di una memoria segmentata o bidimensionale, il programma deve fornire un indirizzo composto da due parti: un numero di segmento e un indirizzo all'interno del segmento. La Figura 6.8 illustra una memoria segmentata usata per le tabelle del compilatore illustrate in precedenza.

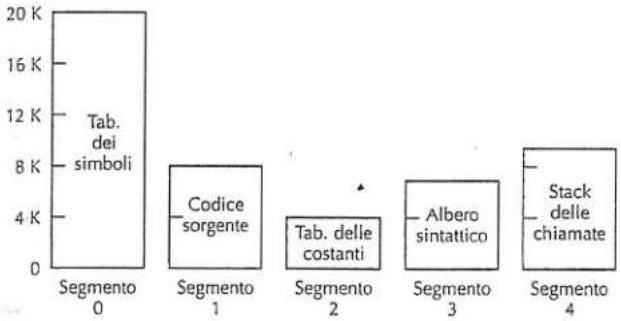


Figura 6.8 Una memoria segmentata consente a ciascuna tabella di crescere o decrescere indipendentemente dalle altre.

Sottolineiamo il fatto che un segmento è un'entità *logica* della cui esistenza il programmatore è conscio e quindi la usa come una singola entità logica. Un segmento potrebbe contenere una procedura, un array, uno stack o una raccolta di variabili e in genere contiene oggetti di tipo omogeneo.

Una memoria segmentata presenta anche altri vantaggi oltre a semplificare la gestione delle strutture dati con dimensione variabile. Se ogni procedura occupa un segmento separato e inizia all'indirizzo 0, il collegamento delle procedure compilate separatamente risulta grandemente agevolato. Quando tutte le procedure di un programma sono state compilate e collegate, la chiamata di una procedura del segmento n userà l'indirizzo (n, 0) per indirizzare la parola 0 (il punto d'ingresso).

Se poi la procedura del segmento n viene modificata e ricompilata, non c'è bisogno di modificare le altre procedure (perché il loro indirizzo iniziale non è stato modificato), neanche nel caso in cui la nuova versione sia più grande della vecchia. In una memoria unidimensionale le procedure sono di norma impacchettate una di seguito all'altra, senza spazio tra di loro, perciò la modifica delle dimensioni di una procedura può causare la variazione dell'indirizzo iniziale di altre procedure indipendenti. La cosa implica

la modifica di tutte le procedure che richiamano una di quelle spostate, per memorizzare il loro nuovo indirizzo iniziale. Questo procedimento diventa costoso su programmi composti da centinaia di procedure.

La segmentazione facilita anche la condivisione di dati o procedure tra più programmi. Se un computer ha molti programmi in esecuzione parallela (vera o simulata) che usano tutti certe procedure di libreria, fornire a ciascuno una propria copia privata delle procedure risulterebbe un gran spreco di memoria principale. Se invece ogni procedura occupa un segmento separato, possono essere condivise facilmente, risparmiando così lo spazio di memoria richiesto da copie multiple.

Si è detto che ogni segmento forma un'entità logica visibile al programmatore, come una procedura, un array o uno stack, dunque è possibile definire modalità di protezione diverse su segmenti diversi. Un segmento di procedura potrebbe essere specificato come solo eseguibile, proibendone accessi in lettura o scrittura. Un array di numeri in virgola mobile potrebbe essere impostato come accessibile in lettura/scrittura, ma non in esecuzione, così che verrebbero rilevati e negati eventuali tentativi di salto verso di esso. Questi tipi di protezione sono di grande aiuto nello scovare errori di programmazione.

È importante rendersi conto del perché la protezione è concepibile in una memoria segmentata, ma non in una memoria paginata (cioè lineare). In una memoria segmentata l'utente conosce il contenuto di ogni segmento e di norma nessun segmento contiene sia una procedura sia uno stack, ma l'una o l'altro. Il fatto che ciascun segmento contenga un solo tipo di oggetto rende possibile l'attribuzione di una protezione appropriata al tipo in questione. La Figura 6.9 mette a confronto paginazione e segmentazione.

Domanda	Paginazione	Segmentazione
Il programmatore deve esserne conscio?	No	Sì
Quanti spazi lineari d'indirizzi ci sono?	1	Molti
Lo spazio degli indirizzi virtuali può eccedere la dimensione della memoria?	Sì	Sì
È facile gestire le tabelle a dimensione variabile?	No	Sì
Perché è stata inventata questa tecnica?	Per simulare memorie grandi	Per fornire molteplici spazi d'indirizzi

Figura 6.9 Confronto tra paginazione e segmentazione.

Il contenuto di una pagina è, in un certo senso, accidentale. Il programmatore non ha alcun sentore della paginazione. Anche se in linea di principio sarebbe possibile usare una manciata di bit per ogni elemento della tabella delle pagine al fine di specificare il tipo di accesso consentito, in tal caso il programmatore dovrebbe tener traccia, nel proprio spazio degli indirizzi, dei punti di transizione tra pagine. Questo non va bene perché è proprio il genere di amministrazione che la paginazione vuole eliminare. L'utente di una memoria segmentata ha l'illusione che tutti i segmenti si trovino in memoria allo stesso tempo, perciò può referenziarli senza curarsi della loro gestione.

6.1.7 Implementazione della segmentazione

La segmentazione può essere implementata in due modi diversi: con lo swapping ("scambio") o con la paginazione. Nel primo schema, in ogni istante la memoria contiene un certo numero di segmenti. Se viene fatto un riferimento a un segmento che non si trova correntemente in memoria, allora viene caricato. Se non c'è abbastanza spazio per accoglierlo, bisogna prima scrivere uno o più segmenti su disco (a meno che esista già una copia pulita su disco, nel qual caso la copia in memoria viene semplicemente abbandonata). In un certo qual modo lo swapping di segmenti non è dissimile dalla paginazione su richiesta: i segmenti sono caricati una volta richiesti.

Tuttavia, l'implementazione della segmentazione differisce dalla paginazione in modo essenziale: le pagine hanno dimensione fissa, i segmenti no. La Figura 6.10(a) mostra un esempio di memoria fisica che contiene inizialmente cinque segmenti. Si consideri ora che cosa succede quando il segmento 1 viene estromesso e viene caricato al suo posto il segmento 7, che è più piccolo. Ci troviamo nella configurazione di memoria della Figura 6.10(b). Lo spazio tra il segmento 7 e 2 è area non utilizzata, cioè una lacuna. Poi il segmento 4 è rimpiazzato dal segmento 5 (Figura 6.10(c)) e il segmento 3 dal 6 (Figura 6.10(d)). Dopo un po' di tempo la memoria si troverà divisa in segmenti e lacune. Questo fenomeno si chiama **frammentazione esterna** (perché lo spazio sprecato è al di fuori dei segmenti). Alle volte ci si riferisce alla frammentazione esterna con il termine di **checkerboarding**, cioè "comportamento a scacchiera".

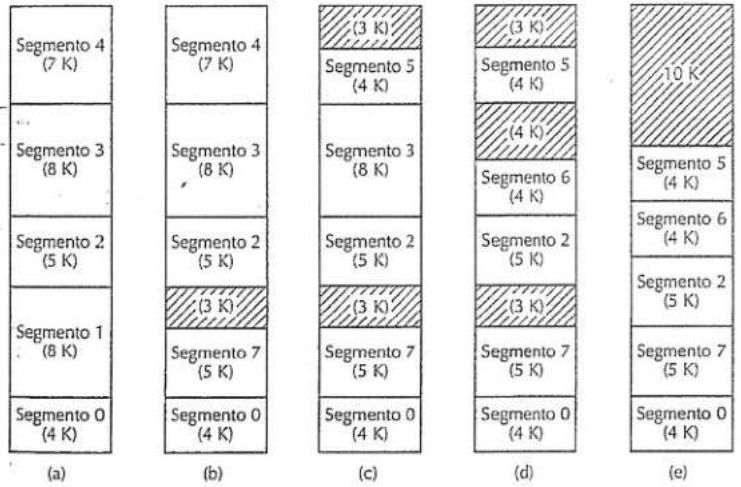


Figura 6.10 (a)-(d) Incremento della frammentazione esterna. (e) Rimozione della frammentazione esterna mediante compattamento.

Si consideri ciò che si verificherebbe se il programma accedesse al segmento 3 nel momento in cui la memoria è affetta da frammentazione esterna, come nella Figura 6.10(d). Lo spazio globalmente libero è 10 K, più di quanto serva al segmento 3, ma poiché è distribuito in frammenti piccoli e inutili, il segmento 3 non può essere caricato. Bisognerebbe rimuovere un altro segmento.

C'è un modo per evitare la frammentazione esterna: ogni volta che appare una lacuna, si può spostare il segmento che segue la lacuna in direzione della locazione di memoria 0, eliminando così quella lacuna, ma creandone una grande alla fine del segmento. In alternativa si potrebbe aspettare fino al momento in cui la frammentazione esterna diventi grave (per esempio quando eccede una certa percentuale della memoria totale) prima di eseguire un compattamento (spostando le lacune lontano dai segmenti). La Figura 6.10(e) mostra la memoria della Figura 6.10(d) dopo compattamento. Lo scopo del compattamento della memoria (a volte chiamato *garbage collection*, cioè "raccolta della spazzatura") è di compattare tutte le piccole e inutilizzabili lacune in una sola, in cui poter caricare uno o più segmenti. Il compattamento presenta l'inconveniente ovvio di portar via del tempo ed eseguirlo dopo la creazione di ogni singola lacuna, consumando troppe risorse.

Se il tempo di compattamento è così lungo da risultare inaccettabile, si rende necessario trovare un algoritmo per determinare quale lacuna usare per caricare un particolare segmento. Tale algoritmo richiede il mantenimento di una lista degli indirizzi e della dimensione delle lacune. Un algoritmo diffuso, chiamato **best fit** ("miglior corrispondenza"), sceglie la lacuna più piccola che può contenere il segmento, nel tentativo di far corrispondere un segmento a ogni lacuna per evitare di frammentarne una che potrebbe servire per ospitare un segmento grande.

Un altro algoritmo molto usato, **first fit** ("prima corrispondenza"), scandisce circolarmente la lista delle lacune e sceglie la prima in grado di ospitare il segmento. Naturalmente così facendo impiega meno tempo di quanto sia necessario alla scansione dell'intera lista operata da best fit. Sorprende invece sapere che first fit è un algoritmo migliore di best fit anche in termini di prestazioni complessive, dal momento che best fit tende a generare moltissime lacune totalmente inservibili (Knuth, 1997).

First fit e best fit tendono a diminuire la dimensione media delle lacune. Ogni volta che un segmento viene caricato in una lacuna più grande di lui, il che capita praticamente sempre (le corrispondenze esatte sono rare), la lacuna viene divisa in due parti: una parte è occupata dal segmento, e l'altra costituisce una nuova lacuna, più piccola di quella originaria. A meno di un processo di compensazione che ricrei lacune più grandi a partire da quelle piccole, alla lunga sia first fit sia best fit tendono a creare in memoria molte lacune inutilizzabili.

Un meccanismo di compensazione è il seguente. Quando viene rimosso un segmento dalla memoria e uno o entrambi i suoi vicini sono lacune e non segmenti, le lacune adiacenti possono saldarsi in una sola. Se venisse rimosso il segmento 5 dalla Figura 6.10(d), le lacune circostanti e i 4 KB usati dal segmento verrebbero fusi in una sola lacuna di 11 KB.

All'inizio di questo paragrafo si è detto che ci sono due modi di implementare la segmentazione: con lo swapping o con la paginazione. La trattazione ha fin qui riguardato lo swapping. Secondo questo schema i segmenti vengono spostati dalla memoria al disco su richiesta. L'altro modo di implementare la segmentazione è di suddividere ogni segmento in un numero prefissato di pagine e svolgere la loro paginazione su richiesta. Secondo questo schema alcune delle pagine del segmento potranno trovarsi in memoria, altre sul disco. La paginazione dei segmenti richiede una tabella delle pagine per ogni segmento. Non essendo un segmento altro che uno spazio lineare degli indirizzi, tutte le tecniche viste fin qui per la paginazione si applicano direttamente a ogni segmento. L'unica novità è la presenza di una tabella delle pagine per ciascun segmento.

Uno dei primi sistemi operativi a combinare la segmentazione con la paginazione fu **MULTICS** (*MULTiplexed Information and Computing Service*), nato come sforzo congiunto di M.I.T., dei Laboratori Bell e della General Electric (Corbato e Vyssotsky, 1965; Organick, 1972). Gli indirizzi di MULTICS erano formati da due parti: un numero di segmento e un indirizzo all'interno del segmento. C'era un segmento descrittore per ogni processo, contenente a sua volta un descrittore per ciascun segmento. Quando l'hardware riceveva un indirizzo virtuale, il numero di segmento era usato come indice nel segmento descrittore per localizzare il descrittore del segmento cui accedere, come mostrato nella Figura 6.11. Il descrittore puntava alla tabella delle pagine, rendendo possibile la paginazione di ogni segmento nelle modalità usuali. Al fine di incrementare le prestazioni, una **memoria associativa** hardware di 16 bit era usata per contenere le combinazioni segmento/pagina usate più di recente, in modo da potervi accedere velocemente. Anche se MULTICS è oggi estinto, ha avuto una vita molto lunga, dal 1965 al 30 ottobre 2000, data in cui l'ultimo sistema MULTICS è stato spento. Pochi altri sistemi operativi hanno resistito per 35 anni. Inoltre, il suo spirito gli è sopravvissuto e le memorie virtuali di tutte le CPU Intel a partire dal 386 sono state modellate sul suo esempio. La storia di MULTICS, insieme ad altri aspetti del sistema, è descritta sul sito www.multicians.org.

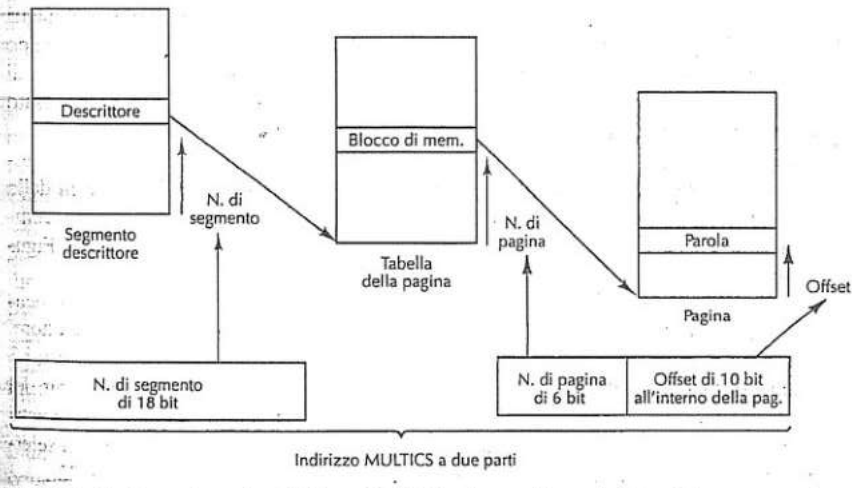


Figura 6.11 Conversione di un indirizzo MULTICS a due parti in un indirizzo fisico.

6.1.8 Memoria virtuale del Core i7

Il Core i7 è fornito di un sistema sofisticato di memoria virtuale che supporta la paginazione a richiesta, la segmentazione pura e la segmentazione paginata. Nel nucleo della memoria virtuale del Core i7 ci sono due tabelle: LDT (*Local Descriptor Table*, "tabella dei descrittori locali") e GDT (*Global Descriptor Table*, "tabella dei descrittori globali"). Ciascun programma ha la propria LDT, ma l'unica GDT è condivisa da tutti i programmi. LDT descrive i segmenti locali a ogni programma, tra cui il suo codice, i dati, lo stack e così via, mentre GDT descrive i segmenti di sistema, compreso il sistema operativo stesso.

Come già descritto nel Capitolo 5, per accedere a un segmento un programma Core i7 deve caricare un selettore per quel segmento in uno dei suoi registri di segmento. Durante l'esecuzione, CS contiene il selettore per il segmento di codice, DS quello per il segmento di dati e così via. Ogni selettore è un numero di 16 bit, come illustrato nella Figura 6.12.

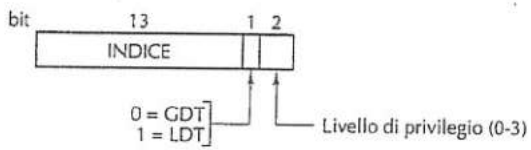


Figura 6.12 Selettore del Core i7.

Un bit del selettore specifica se il segmento è locale oppure globale (cioè se si trova in LDT o in GDT). Altri 13 bit specificano il numero dell'elemento di LDT o di GDT, perciò entrambe le tabelle possono contenere 8 KB (2^{13}) descrittori di segmenti. Gli altri 2 bit riguardano la protezione e verranno trattati in seguito. Il descrittore 0 non è valido e causa una trap se viene usato. È possibile caricarlo senza problemi in un registro di segmento per indicare che il registro non è al momento disponibile, ma se viene usato provoca comunque una trap.

Nell'istante in cui un selettore viene caricato in un registro di segmento, il descrittore corrispondente è recuperato da LDT o GDT e memorizzato nei registri interni della MMU, dove può essere referenziato velocemente. Un descrittore è fatto di 8 byte, compreso l'indirizzo della base del segmento, la sua dimensione e altre informazioni (Figura 6.13).

Il formato del selettore è stato concepito in modo intelligente per facilitare la localizzazione del descrittore. Prima si sceglie tra LDT e GDT, in base al bit 2 del selettore, quindi si copia il selettore in un registro di lavoro della MMU e ne vengono azzerati i 3 bit meno significativi, il che equivale a moltiplicare per otto il numero di 13 bit del selettore. Infine gli si somma l'indirizzo della tabella LDT o GDT (tenuto nei registri interni della MMU) per ottenere un puntatore diretto al descrittore. Per esempio, il selettore 72 fa riferimento all'elemento 9 di GDT, che si trova all'indirizzo GDT + 72.

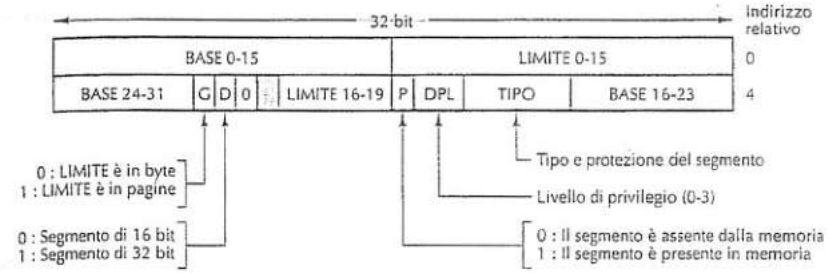


Figura 6.13 Descrittore di un segmento di codice nel Core i7. I segmenti di dati sono leggermente diversi.

Seguiamo passo dopo passo il meccanismo di conversione di una coppia (selettore, offset) in un indirizzo fisico.

Non appena stabilito il registro di segmento da utilizzare, l'hardware può trovare il descrittore completo corrispondente al selettore memorizzato nel registro interno. Se il segmento non esiste (selettore 0), oppure se non si trova correntemente in memoria (P vale 0), si verifica una trap. La prima eventualità è un errore di programmazione, la seconda richiede l'intervento del sistema operativo per recuperare il segmento.

Dopo di ciò, l'hardware verifica se l'offset eccede la fine del segmento e, in caso affermativo, si ha un'altra trap. Secondo logica, il descrittore dovrebbe contenere semplicemente un campo di 32 bit per specificare la dimensione del segmento, ma essendoci soltanto 20 bit disponibili, si usa uno schema differente. Se il bit G di granularità vale 0, allora il campo LIMITE rappresenta la dimensione esatta del segmento, fino a un massimo di 1 MB. Se invece G vale 1, il campo LIMITE restituisce la dimensione del segmento in pagine invece che in byte. La dimensione di pagina nel Core i7 non è mai minore di 4 KB, perciò bastano 20 bit per segmenti grandi fino a 2^{32} byte.

Ipotizziamo che il segmento si trovi in memoria e che l'offset sia corretto. In tal caso il Core i7 somma il campo BASE di 32 bit del descrittore all'offset, per formare quello che è detto un **indirizzo lineare**, mostrato nella Figura 6.14. Il campo BASE è suddiviso in tre parti, sparpagliate all'interno del descrittore per motivi di compatibilità con l'80286, il cui campo BASE è di soli 24 bit. La lunghezza del campo BASE permette a un segmento di cominciare in una locazione qualsiasi dello spazio degli indirizzi lineari di 32 bit. Se la paginazione è disabilitata (da un bit del registro globale di controllo) l'indirizzo lineare è interpretato come indirizzo fisico e viene quindi inviato alla memoria per la sua lettura o scrittura. Senza la paginazione abbiamo dunque uno schema di segmentazione pura, dove il descrittore di un segmento ne individua l'indirizzo base. È consentito ai segmenti di sovrapporsi accidentalmente, forse perché la verifica che siano tutti disgiunti richiederebbe troppo tempo e comporterebbe eccessive complicazioni.

D'altra parte, se la paginazione è abilitata, l'indirizzo lineare è interpretato come un indirizzo virtuale e viene trasformato nell'indirizzo fisico tramite le tabelle delle pagine, come già visto. La sola complicazione qui è che, con indirizzi virtuali di 32 bit e pagine

di 4 KB, è possibile avere segmenti da un milione di pagine, perciò viene usata una corrispondenza a due livelli per ridurre la dimensione della tabella delle pagine nel caso di segmenti piccoli.

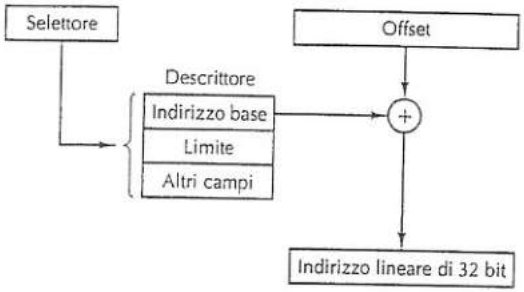


Figura 6.14 Conversione di una coppia (selettore, offset) in un indirizzo lineare.

Ogni programma in esecuzione ha una **directory delle pagine** costituita da 1024 elementi di 32 bit e localizzata presso un indirizzo puntato da un registro globale. Ogni elemento di questa directory punta a una tabella delle pagine costituita anch'essa di 1024 elementi di 32 bit. Gli elementi della tabella delle pagine puntano ai blocchi di memoria. Lo schema è quello della Figura 6.15.

La Figura 6.15(a) raffigura un indirizzo lineare suddiviso in tre campi: DIR, PAGINA e OFF. Il campo DIR è usato per primo per indicizzare la directory delle pagine al fine di trovare il puntatore alla tabella delle pagine appropriata. Poi viene usato il campo PAGINA come indice nella tabella delle pagine per trovare l'indirizzo del blocco di memoria. Infine OFF è sommato all'indirizzo del blocco di memoria per formare l'indirizzo fisico del byte o della parola cui accedere.

Gli elementi della tabella delle pagine sono tutti di 32 bit, di cui 20 per il numero del blocco di memoria. I bit restanti contengono bit di accesso, un dirty bit (asserito dall'hardware a beneficio del sistema operativo se il blocco è stato modificato dopo il caricamento), bit di protezione e altri bit.

Ogni tabella delle pagine contiene elementi per 1024 blocchi di memoria di 4 KB, perciò una sola tabella delle pagine raggiunge 4 MB di memoria. Ai segmenti più piccoli di 4 MB basta una directory delle pagine con un solo elemento, cioè il puntatore alla sua unica tabella delle pagine. Così facendo l'informazione accessoria per i segmenti piccoli è di sole due pagine, invece dei milioni di pagine che sarebbero richieste in una tabella delle pagine a un solo livello.

Per evitare di accedere ripetutamente alla memoria, la MMU del Core i7 ha un hardware speciale per la ricerca veloce delle combinazioni DIR– PAGINA usate più di recente, e le mappa negli indirizzi fisici dei blocchi di memoria corrispondenti. I passi illustrati nella Figura 6.15 vengono svolti effettivamente solo quando la combinazione corrente non sia stata usata di recente.

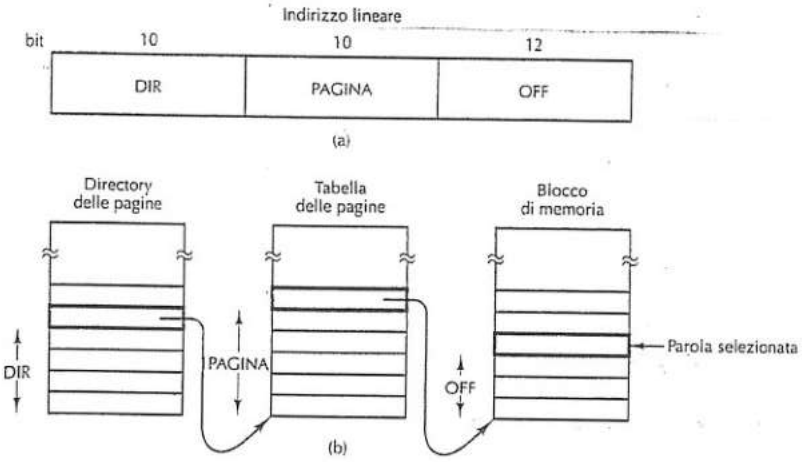


Figura 6.15 Corrispondenza tra un indirizzo lineare e un indirizzo fisico.

Non c'è alcuna ragione per avere un valore diverso da 0 nel campo BASE del descrittore quando si usa la paginazione. Ciò è dovuto alla semplice osservazione che l'effetto del campo BASE è di accedere a un elemento nel mezzo della directory delle pagine che si trova a un certo offset dall'elemento iniziale, invece che all'elemento iniziale stesso. Le uniche ragioni che giustificano l'esistenza del campo BASE sono consentire la segmentazione pura (non paginata) e garantire la retrocompatibilità con il vecchio 80286, privo di paginazione.

Vale la pena menzionare il fatto che, se un'applicazione non ha bisogno della segmentazione e richiede un singolo spazio degli indirizzi di 32 bit paginato, è facile soddisfare la sua richiesta. Basta impostare tutti i registri di segmento allo stesso selettore, il cui descrittore ha BASE = 0 e LIMITE = massimo. Se si usa un solo spazio degli indirizzi, l'offset dell'istruzione diviene l'indirizzo lineare e si ottiene in effetti la paginazione tradizionale.

Abbiamo così concluso la trattazione della memoria virtuale del Core i7. Abbiamo visto solo una piccola parte (usata comunemente) del sistema di memoria virtuale del Core i7. Il lettore interessato può approfondire le sue conoscenze con la documentazione del Core i7 e scoprire anche le estensioni a 64 bit degli indirizzi virtuali e il supporto per gli spazi degli indirizzi fisici virtualizzati. Prima di abbandonare l'argomento, resta da aggiungere qualche osservazione a proposito della protezione, essendo questa una materia intimamente legata alla memoria virtuale. Il Core i7 supporta quattro livelli di protezione, da 0 (massimi privilegi) a 3, illustrati nella Figura 6.16. In ogni momento un programma in esecuzione si trova a un certo livello di protezione, indicato da un campo di 2 bit nel registro hardware **PSW** (*Program Status Word*), contenente i codici di condizione e svariati altri bit di stato. Inoltre, ogni segmento del sistema si trova a un certo livello di protezione.

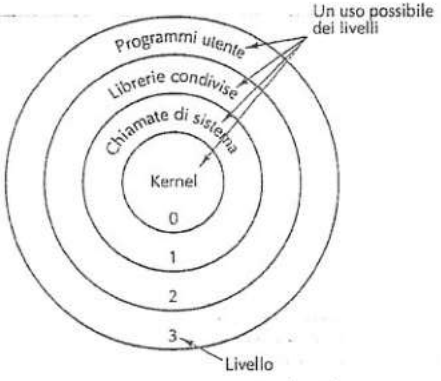


Figura 6.16 Protezione nel Core i7.

Fintantoché un programma si limita a usare segmenti al suo stesso livello, non ci sono problemi. Sono consentiti i tentativi di accesso a dati che si trovano a un livello più alto, mentre i tentativi di accesso a dati a livello più basso sono vietati e causano una trap. È consentito richiamare procedure a livelli diversi (più alti o più bassi), benché in modo severamente controllato. Un'istruzione `CALL` che voglia effettuare una chiamata tra livelli deve contenere un selettore invece di un indirizzo. Questo selettore designa un descrittore che si chiama **call gate** ("accesso alle chiamate") e che restituisce l'indirizzo della procedura da invocare. Così facendo si nega la possibilità di saltare nel mezzo di un segmento di codice di livello diverso; è possibile accedere solo ai punti d'ingresso ufficiali.

Un uso possibile di questo meccanismo è suggerito dalla Figura 6.16. A livello 0 troviamo il kernel del sistema operativo, che gestisce l'I/O, la memoria e altre questioni critiche. A livello 1 troviamo il gestore delle chiamate di sistema. I programmi utente possono richiamare procedure a questo livello perché svolgano chiamate di sistema, ma solo quelle appartenenti a una lista specifica di procedure protette. Il livello 2 contiene le procedure di libreria, eventualmente condivise tra molti programmi in esecuzione. I programmi utente le possono richiamare, ma non modificare. Infine i programmi utente girano a livello 3, cui è associata la protezione minima. Al pari dello schema di gestione della memoria, anche il sistema di protezione del Core i7 è ispirato fortemente a quello di MULTICS.

Le trap e gli interrupt usano un procedimento simile a quello dei call gate: anch'essi referenziano descrittori, invece d'indirizzi assoluti, e questi descrittori puntano alle specifiche procedure da eseguire. Il campo `TIPO` della Figura 6.13 opera la distinzione tra segmenti di codice e segmenti di dati, oltre che tra vari tipi di call gate.

6.1.9 Memoria virtuale della CPU ARM OMAP4430

La CPU ARM OMAP4430 è una macchina a 32 bit che supporta una memoria virtuale paginata basata su indirizzi virtuali di 32 bit che vengono tradotti in indirizzi fisici

di 32 bit. Una CPU ARM può dunque supportare fino a 2^{32} byte (4 GB) di memoria fisica. Sono supportate quattro dimensioni di pagina: 4KB, 64 KB, 1MB e 16 MB. Le corrispondenze indotte da queste quattro dimensioni di pagina sono illustrate nella Figura 6.17.

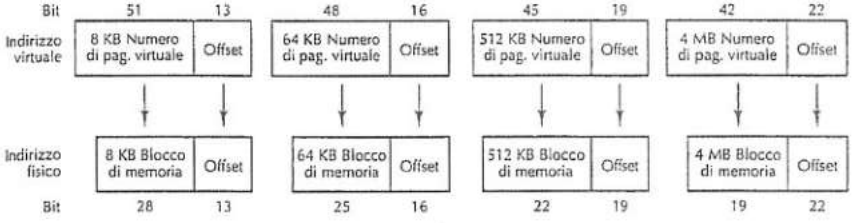


Figura 6.17 Corrispondenze tra indirizzi virtuali e fisici nella CPU ARM OMAP4430.

La CPU ARM OMAP4430 utilizza una struttura di tabella delle pagine simile a quella del Core i7. La mappatura della tabella delle pagine per una pagina degli indirizzi virtuali di 4 KB è mostrata nella Figura 6.18(a). La tabella dei descrittori di primo livello è indicizzata dai 12 bit più significativi dell'indirizzo virtuale. Una sua entry indica l'indirizzo fisico della tabella dei descrittori di secondo livello. Questo indirizzo, combinato con i successivi 8 bit dell'indirizzo virtuale, fornisce l'indirizzo del descrittore di pagina. Il descrittore di pagina contiene l'indirizzo del blocco di pagina fisica e altre informazioni sui permessi di accesso alla pagina.

La mappatura della memoria virtuale della CPU ARM OMAP4430 contempla quattro dimensioni di pagina. Le dimensioni di 1 e di 16 MB vengono mappate con un descrittore di pagina collocato nella tabella dei descrittori di primo livello. Infatti, non c'è bisogno delle tabelle di secondo livello, perché, in ognuna di queste, tutte le entry punterebbero alla stessa grande pagina fisica. I descrittori delle pagine di 64 KB sono posti nella tabella dei descrittori di secondo livello. Visto che ogni voce della tabella dei descrittori di secondo livello mappa 4 KB di pagina di indirizzi virtuali in una pagina degli indirizzi fisici di 4 KB, per le pagine da 64 KB sono richiesti 16 descrittori identici nella pagina dei descrittori di secondo livello. Perché dunque un programmatore di sistemi dovrebbe dichiarare una dimensione di pagina di 64 KB quando questo richiederebbe lo stesso spazio che si utilizza per mappare una più flessibile pagina di 4 KB? Perché, come vedremo tra poco, le pagine di 64 KB richiedono meno voci nella tabella TLB, che è una risorsa critica per ottenere buone prestazioni.

Niente rallenta un programma più del collo di bottiglia dovuto alla memoria. Osservando attentamente la Figura 6.18, noterete probabilmente che per ogni accesso del programma alla memoria sono richiesti due accessi aggiuntivi per la traduzione degli indirizzi. Questo sovraccarico del 200% nel numero di accessi per la traduzione degli indirizzi virtuali renderebbe un programma eccessivamente lento. Per evitare questo collo di bottiglia la CPU ARM OMAP4430 incorpora una tabella hardware chiamata

TLB ("Translation Lookaside Buffer") che permette di stabilire rapidamente la corrispondenza tra numeri di pagina virtuale e numeri di blocco di pagina fisica. Per una dimensione di pagina di 4KB ci sono 2^{20} numeri di pagina virtuale, ovvero più di un milione. Ovviamente non possono essere tutti mappati nella tabella e la TLB conterrà solo i numeri di pagina virtuale più recentemente usati.

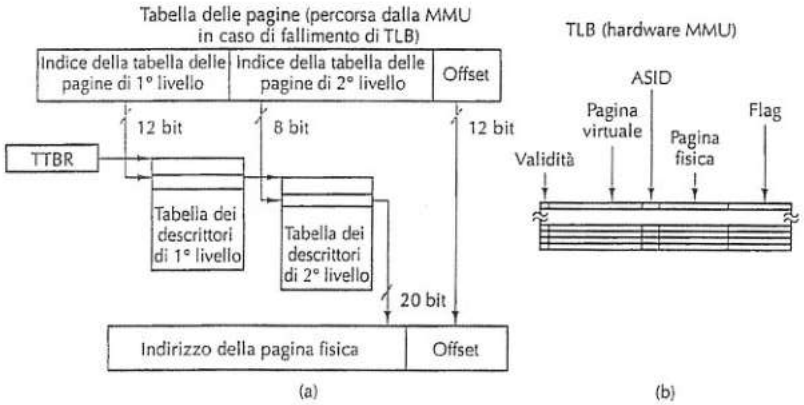


Figura 6.18 Strutture dati utilizzata dalla CPU ARM OMAP4430 per la traduzione degli indirizzi virtuali. (a) Tabella di traduzione degli indirizzi. (b) TLB.

La tracciatura delle istruzioni e dei dati avviene separatamente: la TLB contiene per ciascuna delle due categorie i 128 numeri di pagina virtuale usati più di recente. Ogni elemento di TLB contiene un numero di pagina virtuale e il numero del blocco di memoria fisico corrispondente. Alla ricezione di un numero di processo, detto **identificatore dello spazio degli indirizzi (ASID)**, e di un indirizzo virtuale all'interno di quello spazio di indirizzi, la MMU usa una circuiteria speciale per confrontare il numero di pagina virtuale in esso contenuto con tutti gli elementi di TLB simultaneamente, al fine di trovare quel contesto. Se l'esito del confronto è positivo, si forma un indirizzo fisico di 32 bit, componendo il numero del blocco di memoria estratto da TLB con l'offset preso dall'indirizzo virtuale; in questa fase si impostano anche alcuni flag, compresi i bit di protezione. La TLB è illustrata nella Figura 6.18(b).

Se invece non si trova alcuna corrispondenza, si verifica un **fallimento di TLB**, che dà il via a un "cammino" hardware attraverso le tabelle delle pagine. Quando si trova il nuovo descrittore della pagina fisica nella tabella delle pagine si verifica se la pagina è in memoria e, se lo è, viene caricata nella TLB la corrispondente traduzione dell'indirizzo. Se la pagina non è in memoria viene avviata la gestione del page fault. Poiché la TLB è formata da un numero ridotto di voci, è probabile che si debba rimuovere una voce esistente per far spazio a una nuova voce. I futuri accessi alla pagina rimossa dovranno ancora percorrere le tabelle delle pagine per ottenere un indirizzo. Se si accede a molte pagine troppo velocemente, la TLB non riuscirà a svolgere il suo compito e la maggior parte degli accessi alla memoria richiederanno un overhead del 200% per la traduzione degli indirizzi.

È interessante confrontare i sistemi di memoria virtuale di Core i7 e OMAP4430. Il Core i7 supporta la segmentazione pura, la paginazione pura e la segmentazione paginata. La CPU ARM OMAP4430 dispone solo della paginazione. Sia il Core i7 che l'OMAP4430 utilizzano l'hardware per percorrere la tabella delle pagine al fine di ricaricare la TLB in caso di fallimento di accesso. Altre architetture, come SPARC e MIPS, in caso di fallimento di TLB cedono il controllo al sistema operativo. Queste ultime architetture definiscono istruzioni con privilegi speciali per la manipolazione della TLB, in modo che il sistema operativo possa percorrere la tabella delle pagine e caricare nella TLB i valori necessari per tradurre gli indirizzi.

6.1.10 Memoria virtuale e caching

La memoria virtuale con paginazione su richiesta e il caching possono sembrare a un primo sguardo completamente diversi, tuttavia sono molto simili dal punto di vista concettuale. Con la memoria virtuale il programma è mantenuto su disco e suddiviso in pagine di dimensione fissa; solo alcune di queste pagine si trovano in memoria principale. Se il programma usa prevalentemente le pagine in memoria si verificano pochi errori di pagina e il programma viene eseguito celermente. Con il caching, il programma è contenuto tutto in memoria e suddiviso in blocchi di cache di dimensione prefissata, alcuni dei quali si trovano nella cache. Se il programma usa prevalentemente i blocchi di cache, allora ci saranno pochi fallimenti di cache e l'esecuzione procederà velocemente. Dal punto di vista concettuale i due meccanismi sono la stessa cosa, ma operano a due diversi livelli gerarchici.

Naturalmente la memoria virtuale e il caching presentano anche alcune differenze. Una è che i fallimenti di cache sono gestiti dall'hardware, mentre gli errori di pagina sono gestiti dal sistema operativo. Inoltre i blocchi di cache sono in genere molto più piccoli delle pagine (per esempio 64 byte contro 8 KB). Si aggiunga che la corrispondenza tra pagine virtuali e blocchi di memoria è diversa: le tabelle delle pagine sono organizzate per essere indicizzate dai bit più significativi dell'indirizzo virtuale, mentre le cache sono indicizzate dai bit meno significativi degli indirizzi di memoria. Nonostante ciò, è importante capire che queste sono differenze di carattere implementativo, il concetto sottostante è molto simile.

6.2 Virtualizzazione hardware

Tradizionalmente le architetture hardware sono state progettate con la prospettiva di dover eseguire un solo sistema operativo alla volta. La proliferazione di risorse condivise, come i servizi cloud, trae beneficio dalla possibilità di eseguire più sistemi operativi contemporaneamente. Per esempio, i servizi di hosting di Internet forniscono ai clienti paganti un sistema completo sul quale possono essere installati servizi web. Installare un nuovo computer nella sala macchine ogni volta che arriva un nuovo clien-

te avrebbe costi proibitivi. I servizi di hosting utilizzano piuttosto la **virtualizzazione**, per supportare l'esecuzione di diversi sistemi completi, compresi i sistemi operativi, sullo stesso server. Solo quando i server esistenti raggiungono un carico eccessivo il servizio di hosting deve installare una nuova macchina fisica.

Anche se esistono approcci software alla virtualizzazione, questi di solito rallentano il sistema virtuale e richiedono modifiche al sistema operativo o l'utilizzo di complessi analizzatori di codice per riscrivere i programmi in tempo reale. Questi svantaggi hanno suggerito ai progettisti l'idea di dover supportare direttamente la **virtualizzazione hardware**.

Questa tecnica, come mostrato nella Figura 6.19, è una combinazione di supporti hardware e software che permette la simultanea esecuzione di diversi sistemi operativi su una singola macchina fisica. Ogni **macchina virtuale** eseguita sulla macchina fisica appare all'utente come un sistema completo e indipendente. L'**hypervisor** è un componente software, simile al kernel di un sistema operativo, che crea e gestisce le istanze di macchine virtuali. L'hardware fornisce gli eventi visibili al software che sono necessari all'hypervisor per implementare le politiche di gestione (policy) per la CPU, la memoria di massa e i dispositivi di I/O.

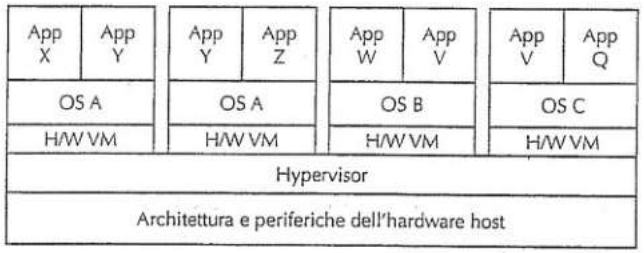


Figura 6.19 La virtualizzazione hardware permette diversi sistemi operativi in esecuzione simultanea sulla stesso hardware. L'hypervisor implementa la condivisione della memoria e dei dispositivi di I/O della macchina host.

La coesistenza di diverse macchine virtuali sulla stessa macchina fisica (il computer host), ognuna delle quali esegue un sistema operativo potenzialmente diverso, offre numerosi benefici. Nei sistemi server la virtualizzazione dà agli amministratori di sistema la capacità di installare più macchine virtuali sulla stessa macchina fisica e di spostare le macchine virtuali tra diversi server per distribuire al meglio il carico totale. Le macchine virtuali danno inoltre all'amministratore di sistema la possibilità di un controllo capillare sull'accesso all'I/O. Per esempio, la larghezza di banda di un'interfaccia di rete virtualizzata può essere suddivisa basandosi sul livello dei servizi utenti. Ai singoli utenti la virtualizzazione è in grado di offrire la possibilità di eseguire diversi sistemi operativi in contemporanea.

Per l'implementazione della virtualizzazione hardware tutte le istruzioni dell'architettura devono accedere soltanto alle risorse della macchina virtuale corrente. Per la

maggior parte delle istruzioni si tratta di una richiesta banale. Per esempio, le istruzioni aritmetiche hanno soltanto bisogno di accedere al banco dei registri che può essere virtualizzato copiando i registri della macchina virtuale nel banco dei registri del processore host al cambio di contesto della macchina virtuale.

La virtualizzazione delle istruzioni per l'accesso alla memoria (per esempio delle istruzioni di caricamento e memorizzazione) è leggermente più complicata, perché queste istruzioni devono effettivamente accedere alla memoria fisica allocata alla macchina virtuale correntemente in esecuzione. Di solito un processore che supporta la virtualizzazione hardware offre funzionalità aggiuntive per la mappatura delle pagine che permettono la mappatura di pagine di memoria fisica della macchina virtuale in pagine di memoria fisica della macchina fisica.

Infine, le istruzioni di I/O (incluso l'I/O memory/mapped) non accedono direttamente ai dispositivi fisici di I/O, perché le politiche di virtualizzazione prevedono una suddivisione dell'accesso ai dispositivi. Questo controllo capillare dell'I/O viene tipicamente implementato mediante interrupt che vengono inviati all'hypervisor ogni volta che una macchina virtuale tenta l'accesso a un dispositivo di I/O. A questo punto l'hypervisor può implementare le politiche di accesso alle risorse a sua discrezione. Di solito, viene supportato un sottoinsieme dei dispositivi di I/O e ci si aspetta che il sistema operativo in esecuzione sulla macchina virtuale, chiamato sistema operativo ospite (guest), utilizzi questi dispositivi supportati.

6.2.1 Virtualizzazione hardware nel Core i7

La virtualizzazione hardware nel Core i7 è supportata grazie alle estensioni VMX (*virtual machine extensions*), una combinazione di estensioni di istruzioni, memoria e interrupt che permette una gestione efficace delle macchine virtuali. Con VMX, la virtualizzazione della memoria è implementata per mezzo del sistema EPT (*Extended Page Table*) che viene abilitato con la virtualizzazione hardware. La tabella EPT traduce gli indirizzi fisici delle pagine della macchina virtuale negli indirizzi fisici delle pagine della macchina host. Questa mappatura è realizzata con una struttura multilivello di tabelle di pagina che viene attraversata in caso di fallimento di accesso alla TLB. L'hypervisor è il manutentore di questa tabella e può quindi implementare la politica di condivisione della memoria fisica desiderata.

La virtualizzazione delle operazioni di I/O, sia per l'I/O memory/mapped che per le istruzioni di I/O, è implementata per mezzo del supporto esteso agli interrupt definito dalla VMCS (*Virtual-Machine Control Structure*). Ogni volta che una macchina virtuale accede a un dispositivo di I/O viene inviato un interrupt all'hypervisor. Quando l'hypervisor riceve l'interrupt può realizzare l'operazione di I/O via software, adottando le politiche necessarie che permettono la condivisione dei dispositivi tra le macchine virtuali.

6.3 Istruzioni di I/O a livello OSM

L'insieme d'istruzioni del livello ISA è completamente diverso dall'insieme d'istruzioni della microarchitettura. Differiscono sia le operazioni disponibili ai due livelli, sia i

formati delle istruzioni. L'esistenza di poche istruzioni sostanzialmente invariate da un livello all'altro va considerata come puramente accidentale.

L'insieme d'istruzioni del livello OSM contiene invece molte delle istruzioni del livello ISA, più un numero limitato d'istruzioni importanti, meno una manciata d'istruzioni potenzialmente nocive. L'input/output è uno dei campi in cui i due livelli differiscono maggiormente e la ragione è semplice: un utente che fosse in grado di eseguire le vere istruzioni di I/O del livello ISA potrebbe leggere dati confidenziali memorizzati in qualsiasi locazione del sistema, potrebbe scrivere sui terminali degli altri utenti e, in generale, costituirebbe una minaccia per la sicurezza del sistema. In secondo luogo, nessun programmatore dotato di senno vorrebbe usare l'I/O del livello ISA, perché è estremamente noioso e complicato. Si tratta di impostare campi e bit in vari registri dei dispositivi, attendere il completamento delle operazioni e quindi verificarne l'esito. Un esempio di ciò sono i bit del registro di dispositivo dei dischi, usati in genere per rilevare gli errori seguenti (tra i tanti possibili):

1. il braccio del disco non è riuscito a portare a termine la ricerca;
2. il buffer specifica locazioni di memoria inesistenti;
3. l'I/O del disco è cominciato prima che venisse completata un'altra attività di I/O già iniziata;
4. errore di temporizzazione della lettura;
5. riferimento a un disco inesistente;
6. riferimento a un cilindro inesistente;
7. riferimento a un settore inesistente;
8. errore di checksum del dato letto;
9. errore di verifica dei dati dopo scrittura.

Quando si verifica uno di questi errori viene asserito il bit corrispondente nel registro di dispositivo. Sono pochi gli utenti che vogliono preoccuparsi di tener traccia di tutti questi bit di errore e delle altre informazioni di stato.

6.3.1 File

Un modo di organizzare l'I/O virtuale è di usare l'astrazione chiamata **file**. Nella sua forma più semplice, un file consiste in una sequenza di byte scritti su di un dispositivo di I/O. Se il dispositivo di I/O è una memoria di massa, quale un disco, il file può essere letto successivamente; se non si tratta di una memoria di massa (ma per esempio di una stampante) chiaramente il file non può più essere letto. Un disco può contenere molti file e ogni file può essere di natura diversa: un'immagine, un foglio di calcolo oppure il testo del capitolo di un libro. File diversi hanno lunghezze e proprietà distinte. L'astrazione del file permette un'organizzazione semplice dell'I/O.

Dal punto di vista del sistema operativo un file è semplicemente una sequenza di byte. Ogni ulteriore strutturazione riguarda i programmi applicativi. L'I/O dei file è svolto tramite chiamate di sistema per l'apertura, lettura, scrittura e chiusura di file.

Prima di poter essere letti i file devono essere aperti. Il procedimento di apertura di un file consente al sistema operativo di localizzarlo su disco e di caricare in memoria le informazioni necessarie per accedervi.

La chiamata di sistema per la lettura deve avere almeno i seguenti parametri:

1. un'indicazione di quale file leggere tra quelli aperti;
2. un puntatore a un buffer in memoria per immagazzinare i dati letti;
3. il numero di byte da leggere.

La chiamata `read` pone i dati richiesti nel buffer e in genere restituisce il numero di byte letti effettivamente (tale numero potrebbe essere inferiore al numero di byte richiesti: non è possibile leggere 2000 byte da un file di 1000 byte).

A ogni file aperto è associato un puntatore che indica il successivo byte da leggere. Dopo una lettura il puntatore è incrementato secondo il numero di byte letti, perciò letture consecutive leggono blocchi di dati consecutivi del file. In genere è possibile assegnare a questo puntatore un valore specifico, di modo da accedere direttamente a ogni locazione del file. Quando un programma ha finito di leggere un file può chiuderlo per informare il sistema operativo che non gli serve più, consentendogli così di liberare lo spazio occupato nella tabella dalle informazioni sul file.

I mainframe vengono ancora utilizzati (specialmente per gestire siti di commercio elettronico di grandi dimensioni) e alcuni di questi eseguono sistemi operativi tradizionali (anche se la maggior parte utilizza Linux). I sistemi operativi dei mainframe hanno una nozione più sofisticata di file e vale la pena dare un veloce sguardo a questo modello per mostrare che il metodo UNIX non è l'unico possibile. Per questi sistemi tradizionali un file è una sequenza di **record logici**, ciascuno dotato di una struttura ben definita. Un record logico, per esempio, potrebbe essere una struttura di cinque oggetti: due stringhe di caratteri, "nome" e "supervisore", due interi, "dipartimento" e "ufficio" e un valore booleano, "diSessoFemminile". Alcuni sistemi operativi fanno una distinzione tra file contenenti record con la stessa struttura e file contenenti record di tipo diverso.

L'istruzione elementare di input legge il record successivo del file specificato e lo mette in memoria principale all'indirizzo indicato, come illustrato nella Figura 6.20. Per svolgere questa operazione, l'istruzione virtuale deve sapere quale file leggere e dove riporre il record in memoria. Spesso si danno opzioni per leggere un determinato record, specificato o per posizione o tramite una chiave.

L'istruzione elementare di output scrive un record logico dalla memoria in un file. Scritture sequenziali consecutive producono nel file record logici consecutivi.

6.3.2 Implementazione delle istruzioni di I/O a livello OSM

Per comprendere l'implementazione delle istruzioni virtuali di I/O è necessario esaminare le modalità di organizzazione e memorizzazione dei file. Una questione basilare che devono affrontare tutti i file system (la parte del sistema operativo dedicata alla gestione dei file) è l'allocazione di memoria di massa. L'unità di allocazione (a volte chiamata *blocco*) può essere un singolo settore del disco, ma spesso consiste in un blocco di settori consecutivi.

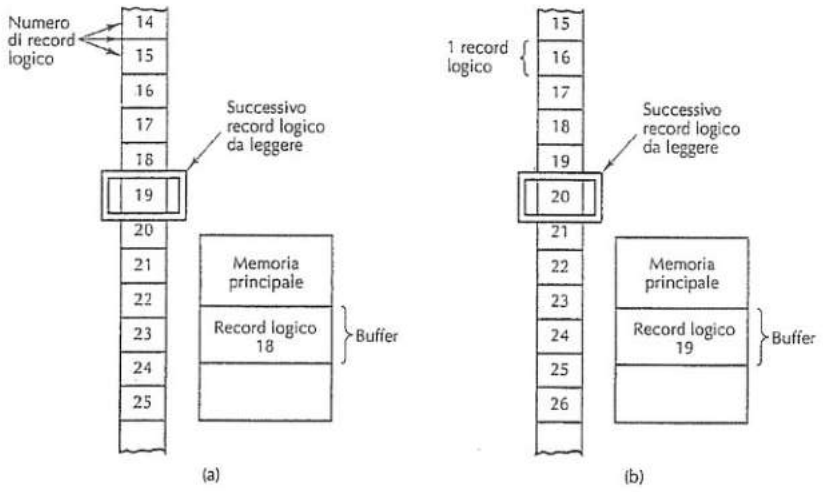


Figura 6.20 Lettura di un file costituito di record logici. (a) Prima della lettura del record 19. (b) Dopo la lettura del record 19.

Un'altra proprietà fondamentale dell'implementazione di un file system è la memorizzazione dei file in unità di allocazione consecutive o meno. La Figura 6.21 raffigura un semplice disco la cui superficie contiene 5 tracce di 12 settori ciascuna. La Figura 6.21(a) mostra uno schema di allocazione in cui i file occupano settori consecutivi, lo schema comune dei CD-ROM. La Figura 6.21(b) mostra invece uno schema in cui il settore è ancora l'unità di allocazione, ma dove un file non è vincolato a occupare settori consecutivi. Questo è lo schema utilizzato di norma nei dischi.

Esiste una differenza importante tra la nozione di file dei programmatori di applicazioni e quella del sistema operativo. Il programmatore vede il file come una sequenza di byte o di record logici. Il sistema operativo vede il file come una collezione ordinata, non necessariamente consecutiva, di unità di allocazione su disco.

Per poter soddisfare la richiesta del byte o del record logico alla locazione *n* di un file, il sistema operativo deve sapere come localizzare i dati. Se il file è allocato in maniera consecutiva, al sistema operativo basta riconoscere l'inizio del file per calcolare la posizione del byte o del record logico richiesto.

In caso contrario, non è possibile calcolare la posizione di un byte o di un record logico arbitrario all'interno del file, a partire dalla sua sola posizione iniziale. Ci sarà dunque bisogno di una tabella contenente le unità di allocazione e i loro indirizzi su disco: la **tabella degli indici**. Quest'ultima può essere organizzata come una lista d'indirizzi di blocchi del disco (come in UNIX), come una lista di sequenze di blocchi consecutivi (utilizzata in Windows 7) o come una lista di record logici, specificandone l'indirizzo e l'offset su disco. Alle volte i record logici sono associati a una **chiave** e i programmi possono fare riferimento a un record attraverso la sua chiave, piuttosto che

attraverso il suo numero. In questo caso è necessario il secondo tipo di organizzazione, che aggiunge a ogni elemento della lista la chiave del record oltre alla sua posizione su disco. Questa organizzazione è comune nei mainframe.

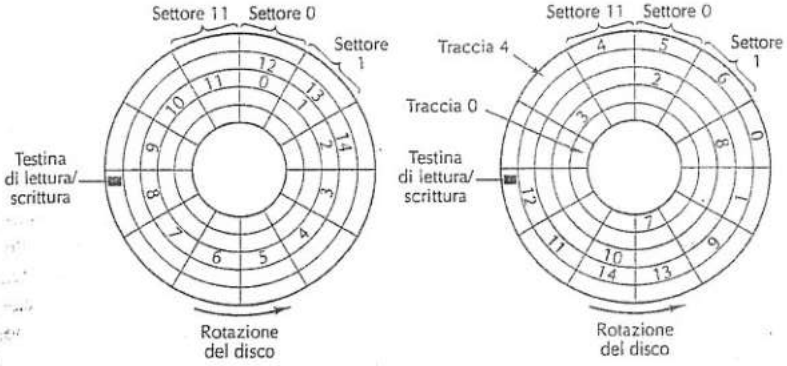


Figura 6.21 Strategie di allocazione su disco. (a) Un file in settori consecutivi. (b) Un file in settori non consecutivi.

Un altro metodo per localizzare le unità di allocazione di un file è l'organizzazione del file come una lista concatenata. Ogni unità di allocazione contiene l'indirizzo dell'unità che le succede. Per implementare in modo efficiente questo schema è necessario mantenere in memoria principale una tabella con gli indirizzi di tutte le unità successive. Per esempio, se il disco contiene 64 KB di unità d'allocazione, il sistema operativo dovrebbe gestire in memoria una tabella di 64 KB di elementi, ciascuno contenente l'indice del suo successore. Se un file occupa le unità 4, 52 e 19, l'elemento 4 della tabella dovrebbe contenere un 52, l'elemento 52 un 19, e l'elemento 19 conterrebbe un codice speciale (per esempio 0 o –1) a indicare la fine del file. Era questo il funzionamento del file system di MS-DOS e di Windows 95/98. Le più recenti versioni di Windows (2000, XP, Vista e 7) supportano questo schema, ma possiedono anche un proprio file system nativo più simile a quello di UNIX.

Fin qui abbiamo considerato file allocati consecutivamente o meno, ma non abbiamo chiarito perché si utilizzano entrambe le soluzioni. La prima consente un'amministrazione semplice dei blocchi, ma è di difficile implementazione quando non è nota a priori la dimensione massima di file. Se un file comincia al settore *j* e cresce nei settori consecutivi può succedere che, se non ha abbastanza spazio per espandersi, finisca per collidere con un altro file nel settore *k*. Questa evenienza non crea problemi se il file non è allocato consecutivamente, poiché i blocchi successivi possono essere messi altrove sul disco. Se un disco contiene svariati file in espansione, senza che sia nota la loro

dimensione finale, è pressoché impossibile memorizzarli in modo consecutivo. Il trasferimento di un file esistente è spesso possibile, ma tuttavia molto costoso.

D'altro canto, se la dimensione massima dei file è nota a priori, la situazione cambia. Per esempio il programma di masterizzazione di un CD-ROM può preallocare per ogni file una sequenza consecutiva di settori della sua lunghezza. Così i file di lunghezza 1200, 700, 200 e 900 possono essere salvati sul CD-ROM rispettivamente a partire dai settori 0, 1200, 1900 e 3900 (non prendiamo in considerazione qui la tabella dei contenuti). È facile accedere a qualsiasi locazione di un file perché il suo primo settore è noto.

Per poter allocare su disco spazio sufficiente per memorizzare i file nuovi, il sistema operativo deve tener traccia dei blocchi liberi e di quelli occupati da file. Nel caso del CDROM il calcolo viene effettuato all'inizio una volta per tutte, ma nel caso di un disco i file vengono salvati e cancellati di continuo. È possibile mantenere una lista di tutte le lacune, ovvero le successioni di unità di allocazione libere e contigue. Questa lista prende il nome di **lista delle locazioni libere** (*free list*). La Figura 6.22(a) illustra la lista delle locazioni libere per il disco della Figura 6.21(b), con un settore per unità di allocazione.

In alternativa è possibile mantenere una bit map con un bit per ogni unità di allocazione, come mostrato nella Figura 6.22(b). I bit sono posti a 1 se l'unità di allocazione corrispondente è già occupata e azzerati in caso contrario.

Traccia	Settore	Numero di settori nella lacuna
0	0	5
0	6	6
1	0	10
1	11	1
2	1	1
2	3	3
2	7	5
3	0	3
3	9	3
4	3	8

(a)

	Settore											
Traccia	0	1	2	3	4	5	6	7	8	9	10	11
0	0	0	0	0	0	1	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1	0
2	1	0	1	0	0	0	1	0	0	0	0	0
3	0	0	0	1	1	1	1	1	1	0	0	0
4	1	1	1	0	0	0	0	0	0	0	0	1

(b)

Figura 6.22 Due modi di tener traccia dei settori disponibili. (a) Con una lista delle locazioni libere. (b) Con una bit map.

La prima soluzione presenta il vantaggio di semplificare la ricerca di uno spazio libero di una data dimensione, anche se presenta lo svantaggio di richiedere una struttura dati di lunghezza variabile: la lunghezza della lista varia alla creazione o alla cancellazione di file, una caratteristica poco gradita. La bit map ha il vantaggio di avere una dimensione costante e in più richiede la sola modifica di un bit per cambiare lo stato di allocazione di un'unità, da libera a occupata o viceversa. Risulta invece difficile la ricerca di un blocco di una data lunghezza. Entrambe le tecniche richiedono l'aggiornamento della lista o della tabella all'atto di allocazione o di rimozione di un file.

Prima di terminare la trattazione dell'implementazione del file system è bene analizzare la dimensione dell'unità di allocazione, che chiama in gioco molti fattori diversi. Innanzitutto va detto che il tempo di ricerca e il ritardo rotazionale del disco predominano sui tempi di accesso. Dopo aver investito 5-10 ms per raggiungere l'inizio di un'unità di allocazione è decisamente preferibile leggere 8 KB (circa 80 μs) che 1 KB (circa 10 μs), poiché leggere 8 KB in tranche di 1 KB richiede otto ricerche su disco. L'efficienza del trasferimento gioca a favore delle unità di allocazione grandi. Con i dischi allo stato solido che diventano più comuni e meno costosi, questo argomento cesserà di valere, perchè per questi dispositivi non esiste un tempo di accesso.

A favore di delle unità di allocazione grande c'è anche il fatto che avere unità di allocazione piccole vuol dire averne molte, il che a sua volta implica indici di file lunghi o grandi tabelle di liste concatenate in memoria. Come nota storica, ricordiamo che MS-DOS iniziò con unità di allocazione che erano di un settore da 512 byte e con numeri di 16 bit utilizzati per identificare i settori. Quando i dischi hanno superato i 65.536 settori, l'unico modo per utilizzare tutto lo spazio del disco e continuare a usare numeri da 16 bit per identificare le unità di allocazione era utilizzare unità si allocazione sempre più grandi. La prima versione di Windows 95 presentava lo stesso problema, poi risolto in una versione successiva con indirizzi di 32 bit. Windows 98 supportava entrambe le dimensioni.

A favore di unità di allocazione piccole è invece la considerazione che ben pochi file occupano esattamente un numero intero di unità di allocazione. Di conseguenza quasi tutti i file sprecano dello spazio nell'ultima unità che occupano e, se un file è molto più grande dell'unità di allocazione, in media il suo spreco è dell'ordine della metà dell'unità di allocazione stessa. Più è grande l'unità di allocazione, maggiore è lo spreco: se i file sono in media molto più piccoli dell'unità di allocazione, verrà sprecato gran parte dello spazio su disco.

Per fare un esempio, in una partizione di 2 GB MS-DOS o della prima versione di Windows 95, in cui le unità di allocazione erano quindi di 32 KB, un file di 100 caratteri sprecava 32.668 byte di spazio del disco. L'efficienza della memorizzazione gioca a favore delle unità di allocazione piccole. A causa del prezzo sempre più basso dei dischi di grandi dimensioni, l'efficienza in termini di tempo tende oggi a essere il fattore più importante. Le unità di allocazione tendono di conseguenza a crescere e lo spreco di spazio sul disco viene semplicemente accettato.

6.3.3 Istruzioni per la gestione di directory

Ai primordi dell'era informatica dati e programmi venivano conservati su schede perforate archiviate negli schedari dei propri uffici. Questa soluzione risultò sempre meno agevole al crescere del numero e della dimensione dei dati e dei programmi e condusse infine all'idea di usare una memoria secondaria (per esempio un disco) come alternativa agli schedari per la memorizzazione dei dati e dei programmi. Le informazioni accessibili direttamente dal computer senza bisogno dell'intervento umano si dicono **in linea** (*online*) in contrapposizione alle informazioni **non in linea** (*offline*), le quali richiedono la mediazione di un operatore perché il computer possa accedervi (per esempio tramite l'inserimento di un CD-ROM, di una chiavetta USB o di una scheda SD).

L'informazione in linea è memorizzata nei file ed è resa accessibile ai programmi per mezzo delle istruzioni di I/O. In realtà l'informazione in linea ha bisogno d'istruzioni ulteriori per il suo mantenimento, per il raggruppamento in unità agevoli e per la protezione da tentativi di utilizzo non autorizzati.

I sistemi operativi sono soliti organizzare i file raggruppandoli in **directory** (cartelle). La Figura 6.23 raffigura un esempio di organizzazione per directory. Tutti i sistemi mettono a disposizione almeno le chiamate di sistema per assicurare le seguenti funzionalità:

1. creare un file e inserirlo in una directory;
2. cancellare un file da una directory;
3. rinominare un file;
4. cambiare lo stato di protezione di un file.

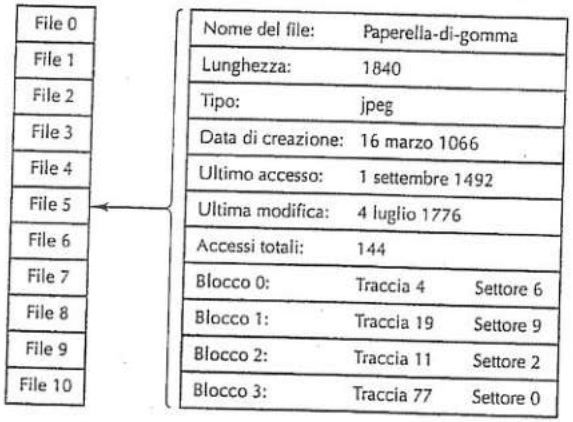


Figura 6.23 Esempio di directory di file utente e un suo tipico elemento.

Sono possibili diversi schemi di protezione. Uno è far sì che l'utente specifichi una password per ciascuno dei propri file: quando un programma desidera accedere al file deve fornire la password e attendere la sua convalida da parte del sistema operativo. Un altro metodo di protezione richiede a ciascun utente di specificare esplicitamente, per ogni file, una lista di utenti: solo i programmi di quegli utenti potranno accedere al file.

Negli sistemi operativi moderni gli utenti possono gestire più di una directory. Ogni directory è a sua volta un file e può far parte così di un'altra directory, dando origine a un albero di directory. Questa molteplicità è particolarmente utile quando si lavora su più progetti allo stesso tempo: ogni directory può raggruppare tutti i file che afferiscono a un dato progetto, evitando le interferenze tra file di progetti diversi. Inoltre le directory agevolano la condivisione di file tra utenti membri di uno stesso gruppo.

6.4 Istruzioni per il calcolo parallelo a livello OSM

Alcuni tipi di calcolo possono essere programmati per essere svolti con maggior profitto da due o più processi cooperanti che girano in parallelo (cioè simultaneamente, su processori diversi) piuttosto che da un solo processo. Altri calcoli possono essere divisi in parti da far eseguire parallelamente, per ridurre il tempo dell'intera computazione. Nei paragrafi successivi illustreremo alcune istruzioni virtuali necessarie per consentire a più processi di collaborare in parallelo.

Le leggi della fisica forniscono un'altra ragione a favore dell'interesse attualmente rivolto al calcolo parallelo. Secondo la teoria della relatività speciale di Einstein è impossibile trasmettere un segnale elettrico a una velocità superiore a quella della luce, che è di circa 30 cm/ns (300.000 km/s) nel vuoto, e un po' inferiore nei fili di rame e nelle fibre ottiche. Questo limite ha implicazioni importanti sulla strutturazione dei computer. Per esempio se la CPU richiede dati alla memoria principale e questa si trova a 30 cm, ci vorrà almeno 1 ns perché la richiesta giunga alla memoria e ancora 1 ns perché la risposta giunga alla CPU. Perciò i computer che vorranno lavorare in tempi sotto il nanosecondo dovranno essere molto miniaturizzati o, in alternativa, essere dotati di molte CPU. Un computer con mille CPU da 1 ns può raggiungere (in linea teorica) la stessa potenza di calcolo di una CPU con cicli di 0,001 ns, ed è molto più facile e più economico da costruire. Il calcolo parallelo sarà discusso in dettaglio nel Capitolo 8.

Su un calcolatore con più di una CPU è possibile assegnare ciascun processo cooperativo a una CPU, facendo sì che progrediscano tutti simultaneamente. Se invece c'è una sola CPU è possibile simulare l'effetto dell'elaborazione parallela alternando l'esecuzione dei diversi processi per brevi lassi di tempo. In altre parole, il processore è condiviso dai diversi processi.

La Figura 6.24 mostra la differenza tra il vero calcolo parallelo, possibile con più processori, e quello simulato, attuato su di un solo processore fisico. Anche quando il parallelismo è solo simulato risulta utile guardare a ogni processo come se disponesse di un processore virtuale dedicato. Nella simulazione si verificano gli stessi problemi di comunicazione riscontrabili nel vero calcolo parallelo. In entrambi i casi, effettuare il debug dei problemi è molto difficile.

6.4.1 Creazione dei processi

Ogni programma in esecuzione fa parte di un processo. Un processo è caratterizzato da uno stato e da uno spazio degli indirizzi tramite cui accedere ai dati e al programma. Lo stato comprende almeno il program counter, una PSW, un puntatore allo stack e dei registri d'uso generale.

Quasi tutti i sistemi operativi moderni consentono la creazione e la terminazione dinamica dei processi. Esiste perciò una chiamata di sistema per la creazione di un processo, fondamentale per realizzare il calcolo parallelo. Questa chiamata di sistema può limitarsi a clonare il processo chiamante oppure gli può consentire di specificare stato iniziale, programma, dati e indirizzo iniziale del nuovo processo.

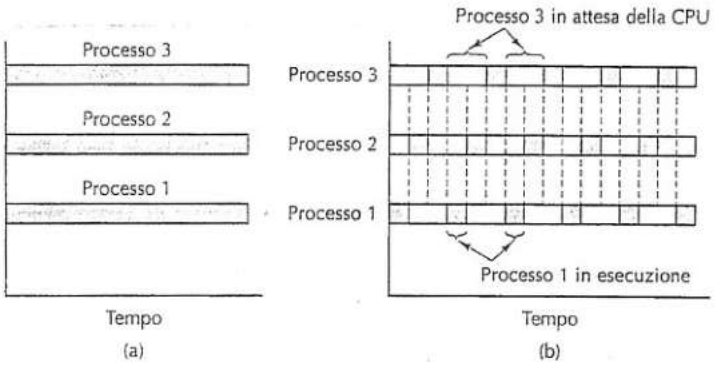


Figura 6.24 (a) Calcolo genuinamente parallelo su CPU multiple. (b) Calcolo parallelo simulato tramite l'alternanza della singola CPU all'interno di un insieme di tre processi.

In alcuni casi il processo creatore (genitore) mantiene controllo parziale o anche totale del processo creato (figlio). A tal fine esistono istruzioni virtuali perché il processo genitore possa fermare, far ripartire, esaminare e porre fine al processo figlio. In altri casi il genitore ha meno controllo sul figlio: una volta creato il processo figlio, il genitore non ha modo di forzarlo a fermarsi, ripartire, farsi esaminare o terminare la propria esecuzione. I due processi procedono nelle rispettive attività in modo indipendente l'uno dall'altro.

6.4.2 Corsa critica

In molti casi i processi paralleli hanno bisogno di comunicare e di sincronizzarsi al fine di poter svolgere il proprio compito. Qui ci avvaliamo di un esempio dettagliato per trattare la sincronizzazione tra processi ed esaminare alcune delle difficoltà che comporta; nel paragrafo successivo forniremo le soluzioni a queste problematiche.

Si consideri la situazione di due processi indipendenti, il processo 1 e il processo 2, che comunicano attraverso un buffer condiviso in memoria principale; per comodità chiamiamo il primo **produttore** e il secondo **consumatore**. Il produttore calcola i numeri primi e li ripone nel buffer uno per volta. Il consumatore li rimuove dal buffer uno alla volta e li visualizza sullo schermo.

Questi due processi girano in parallelo con velocità diverse. Se il produttore si rende conto che il buffer è pieno allora *va a dormire*, ovvero sospende temporaneamente le sue operazioni in attesa di un segnale dal consumatore. Successivamente il consumatore, dopo aver rimosso un numero dal buffer, invia il segnale al produttore per *svegliarlo*, cioè per farlo ripartire. Analogamente, il consumatore va a dormire quando il buffer è vuoto. Il produttore lo sveglierà non appena avrà inserito un numero nel buffer vuoto.

In questo esempio usiamo un buffer circolare per la comunicazione tra processi. I puntatori *in* e *out* sono usati nel modo seguente: *in* punta alla parola libera successiva (dove il produttore inserirà il prossimo numero primo), *out* punta al numero che sarà

rimosso per primo dal consumatore. Il buffer è vuoto quando *in* = *out*, come nella Figura 6.25(a). La Figura 6.25(b) mostra la situazione dopo che il produttore ha generato alcuni numeri primi. La Figura 6.25(c) mostra la situazione dopo che il consumatore ha rimosso e stampato alcuni di quei numeri primi. Le Figure 6.25(d)-(f) illustrano una possibile evoluzione del buffer a seguito dell'attività dei due processi. La cima del buffer è logicamente contigua con la sua base, ossia il buffer si riavvolge in modo circolare. Se si verifica una raffica di inserimenti tali che *in* punti alla parola precedente di quella puntata da *out* (per esempio *in* = 52 e *out* = 53) allora il buffer è pieno. L'ultima parola libera non è utilizzata, perché altrimenti non ci sarebbe alcun modo per capire se *in* = *out* indica il buffer vuoto o il buffer pieno.

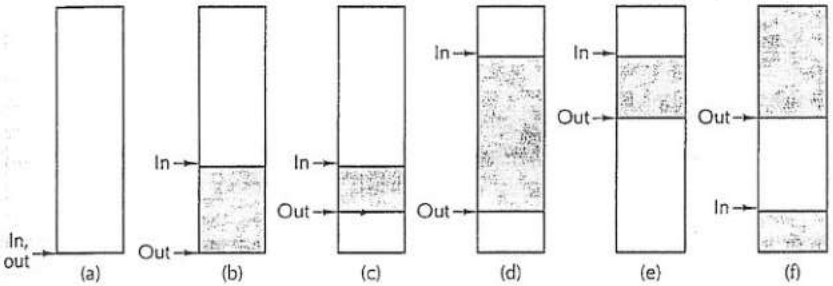


Figura 6.25 Uso di un buffer circolare.

La Figura 6.26 mostra una semplice implementazione in Java del problema produttore-consumatore. Questa soluzione usa tre classi: `m`, `producer` e `consumer`. La classe principale `m` contiene alcune definizioni di costanti, i puntatori al buffer *in* e *out* e il buffer stesso, che in questo esempio può contenere 100 primi da `buffer[0]` a `buffer[99]`. Le classi producer e consumer svolgono il lavoro effettivo.

Il codice si avvale dei thread Java per simulare il calcolo parallelo. Le classi `producer` e `consumer` sono istanziate rispettivamente nelle variabili `p` e `c`; sono entrambe classi derivate dalla classe base `Thread` provvista del metodo `run`, che serve a contenere il codice del thread. L'invocazione del metodo `start` di un oggetto derivato da `Thread` provoca l'avvio di un nuovo thread (l'esecuzione del suo metodo `run`).

Un thread somiglia a un processo, con la differenza che tutti i thread di un programma Java accedono allo stesso spazio degli indirizzi; questa caratteristica permette loro di condividere un buffer. Se un calcolatore ha due o più CPU è possibile assegnare ciascun thread a una diversa CPU e ottenere quindi il vero parallelismo. Se c'è una sola CPU, i thread la condividono secondo lo schema di condivisione del tempo. Continueremo a far riferimento ai processi produttore e consumatore (visto che il nostro interesse è per i processi paralleli) anche se Java supporta solo i thread paralleli e non i processi veramente paralleli.

```
public class m {
   final public static int BUF_SIZE = 100;                    // il buffer va da 0 a 99
   final public static long MAX_PRIME = 100000000000L;        // valore d'arresto
   public static int in = 0, out = 0;                         // puntatori ai dati
   public static long buffer[ ] = new long[BUF_SIZE];         // memorizza i numeri primi
   public static producer p;                                  // nome del produttore
   public static consumer c;                                  // nome del consumatore

   public static void main(String args[ ]) {                  // classe main
      p = new producer( );                                    // crea il produttore
      c = new consumer( );                                    // crea il consumatore
      p.start( );                                             // avvia il produttore
      c.start( );                                             // avvia il consumatore
   }
   // funzione di servizio per l'incremento circolare di in e out
   public static int next(int k) {if (k < BUF_SIZE – 1) return(k+1); else return(0);}
}

class producer extends Thread {                               // classe produttore
   public void run( ) {                                       // codice del produttore
      long prime = 2;                                         // variabile di lavoro

   while (prime < m.MAX_PRIME) {
         prime = next_prime(prime);                           // istruzione P1
         if (m.next(m.in) == m.out) suspend( );               // istruzione P2
         m.buffer [m.in] = prime;                             // istruzione P3
         m.in = m.next(m.in);                                 // istruzione P4
         if (m.next(m.out) == m.in) m.c.resume( );            // istruzione P5
      }
   }

   private long next_prime(long prime){ ... }                 // funzione che calcola il numero primo successivo
}


class consumer extends Thread {                               // classe consumatore
   public void run( ) {                                       // codice consumatore
      long emirp = 2;                                         // variabile di lavoro

      while (emirp < m.MAX_PRIME) {
         if (m.in == m.out) suspend( );                       // istruzione C1
         emirp = m.buffer [m.out];                            // istruzione C2
         m.out = m.next(m.out);                               // istruzione C3
         if (m.out == m.next(m.next(m.in))) m.p.resume( );    // istruzione C4
         System.out.println(emirp);                           // istruzione C5
      }
   }
}
```

Figura 6.26 Calcolo parallelo in situazione fatale di corsa critica.

La funzione `next` serve a incrementare *in* e *out* in modo agevole, senza dover controllare ogni volta il verificarsi di un riavvolgimento circolare. Se il parametro di `next` è minore o uguale a 98 la funzione restituisce l'intero successivo. Se invece il parametro vale 99 abbiamo raggiunto la fine del buffer e la funzione restituisce 0.

C'è bisogno poi di un modo per consentire ai processi di dormire quando non possono continuare l'esecuzione. I progettisti di Java hanno previsto questa necessità e hanno incluso i metodi `suspend` (sonno) e `resume` (risveglio) nella classe `Thread` sin dalla prima versione di Java. Se ne fa uso nella Figura 6.26.

Veniamo quindi al codice del produttore e del consumatore vero e proprio. All'inizio il produttore genera un nuovo numero primo nell'istruzione P1. Si noti qui l'uso di `m.MAX_PRIME`: il prefisso `m.` indica che intendiamo il valore `MAX_PRIME` definito nella classe `m`. Per le stesse ragioni `m.` è anche il prefisso di `in`, `out`, `buffer` e `next`.

In seguito il produttore controlla (in P2) se *in* è arretrato di una posizione rispetto a *out*. In tal caso (per esempio *in* = 62, *out* = 63) il buffer è pieno e il produttore va a dormire invocando `suspend` in P2. Se il buffer non è pieno il nuovo primo può essere inserito nel buffer (P3) e *in* incrementato (P4). Se in P5 il nuovo valore di *in* eccede di 1 *out* (per esempio *in* = 17 e *out* = 16), dovevano essere necessariamente uguali prima dell'incremento di *in*. Il produttore conclude che il buffer era precedentemente vuoto e che il consumatore sta ancora dormendo, dunque chiama la `resume` per svegliarlo (P5). Infine il produttore comincia a preoccuparsi del successivo valore da produrre.

Il programma del consumatore è strutturato in modo simile. All'inizio effettua un test (C1) per verificare se il buffer è vuoto; in caso affermativo non c'è lavoro da svolgere e il consumatore può andare a dormire. In caso negativo, il consumatore rimuove il successivo numero da stampare (C2) e incrementa *out* (C3). A questo punto, se *out* è due posizioni oltre *in* (C4) vuol dire che precedentemente distavano l'un l'altro una sola posizione; poiché *in* = *out* –1 è la condizione di "buffer pieno", il produttore deve essere ancora in fase di sonno; quindi il consumatore lo risveglia chiamando la `resume`. Infine stampa il numero (C5) e ricomincia lo stesso ciclo.

Sfortunatamente questa soluzione contiene un difetto fatale, illustrato nella Figura 6.27. Si ricordi che i due processi girano in modo asincrono ed, eventualmente, a diverse velocità. Si consideri il caso della Figura 6.27(a), dove il buffer contiene un solo elemento nel posto 21, con *in* = 22 e *out* = 21. Se il produttore si trova all'istruzione P1 in cerca di un numero primo e il consumatore è occupato in C5 a stampare l'elemento di posizione 20, allora il consumatore termina la stampa del numero, effettua il test di C1 e in C2, preleva dal buffer l'ultimo numero presente, quindi incrementa *out*. In questo istante *in* e *out* valgono entrambi 22. Poi il consumatore stampa il numero e torna a C1, dove legge *in* e *out* dalla memoria per effettuarne il confronto, come mostrato nella Figura 6.27(b).

In questo preciso istante, dopo che il consumatore ha prelevato *in* e *out* dalla memoria e prima di averli confrontati, il produttore trova il numero primo successivo, lo inserisce nel buffer con l'istruzione P3 e incrementa *in* in P4. In altre parole, *in* vale ora *out* + 1 a indicare la presenza di un elemento nel buffer e il produttore ne deduce (erroneamente) che il consumatore è sospeso, perciò gli invia un segnale di risveglio (cioè chiama `resume`), come raffigurato dalla Figura 6.27(c). Invece il consumatore è ancora sveglio e così il segnale di risveglio va perso. Il produttore inizia la ricerca di un altro numero primo.

Adesso il consumatore continua la propria attività: ha già recuperato *in* e *out* dalla memoria prima dell'inserimento nel buffer dell'ultimo numero trovato dal produttore. Poiché entrambi i puntatori valgono 22 il consumatore va a dormire. Successivamente il produttore trova un nuovo numero primo, osserva che *in* = 24 e *out* = 22, e conclude che ci sono due numeri nel buffer (vero) e che il consumatore è sveglio (falso). Il produttore prosegue la sua esecuzione e arriva il momento in cui riempie il buffer e va a dormire. Ora entrambi i processi sono dormienti e lo rimarranno per sempre.

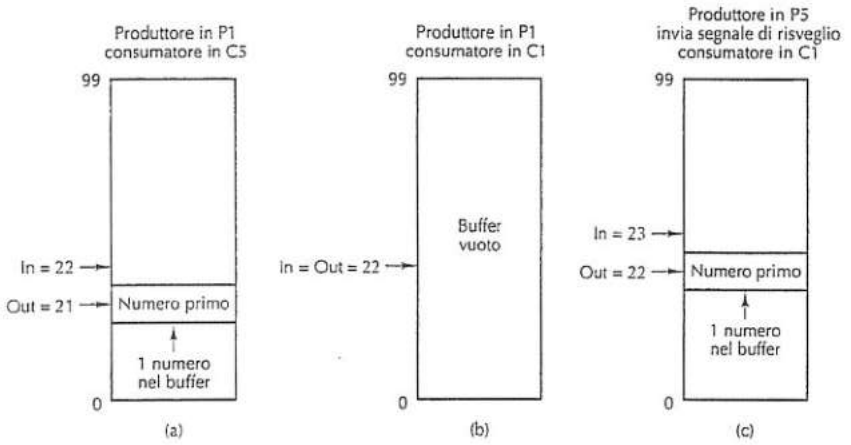


Figura 6.27 Insuccesso del meccanismo di comunicazione tra produttore e consumatore.

Il problema qui è stato originato dal fatto che, nel lasso di tempo tra la lettura di *in* e *out* da parte del consumatore e il suo assopimento, il produttore è intervenuto di soppiatto, ha scoperto che *in* = *out* + 1, ha pensato che il consumatore stesse dormendo (non ancora vero) e gli ha spedito un segnale di sveglia che è andato perso proprio perché il consumatore era ancora sveglio. Questa problematica è nota con il nome di **corsa critica**, perché il successo del metodo dipende dal vincitore della corsa al test di *in* e *out* dopo l'incremento di *out*.

Il problema delle corse critiche è ben noto ed è così serio che, svariati anni dopo l'introduzione di Java, Sun ha cambiato l'interfaccia della classe `Thread` rinnegando l'uso di `suspend` e `resume` perché conducevano spesso a condizioni di corsa critica. La soluzione alternativa proposta si basa sul linguaggio, ma visto che ci interessiamo qui ai sistemi operativi, prendiamo in considerazione un'ulteriore soluzione fornita da molti di loro, compresi UNIX e Windows 7.

6.4.3 Sincronizzazione dei processi tramite semafori

È possibile risolvere le situazioni di corsa critica in almeno due modi. Una soluzione consiste nel dotare ciascun processo di un "bit risveglio pendente". Ogni volta che viene spedito un segnale di risveglio a un processo ancora attivo il suo bit risveglio pendente viene asserito. Se un processo va a dormire con questo bit asserito, la sua esecuzione riprende immediatamente e il bit viene azzerato. Il bit risveglio pendente serve a conservare un segnale di risveglio superfluo per un uso futuro.

Anche se questo metodo risolve la corsa critica tra due processi, fallisce però nel caso generale di comunicazione tra n processi, perché bisognerebbe salvare $n - 1$ bit di risvegli pendenti per contare fino a $n - 1$, ma si tratta evidentemente di una soluzione astrusa.

Dijkstra (1968b) ha proposto una soluzione più generale al problema della sincronizzazione tra processi paralleli che prevede l'uso di variabili intere (non negative) chiamate **semafori**. Il sistema operativo fornisce due chiamate di sistema, `up` e `down`, per agire sui semafori: la prima per incrementare un semaforo di 1, la seconda per decrementarlo di 1.

Se si effettua un'operazione `down` di un semaforo che ha valore positivo, questo viene decrementato di 1 e il processo corrispondente continua. Invece se il semaforo vale 0 la `down` non può andare a buon fine; il processo corrispondente è messo a dormire e resta in questo stato finché un altro processo esegue una `up` su quel semaforo. In genere i processi dormienti sono posti in una coda al fine di poterne tener traccia.

L'istruzione `up` su un semaforo controlla se il semaforo è a 0, nel qual caso lo incrementa di un'unità e consente al processo dormiente di completare l'operazione `down` che ha causato la sua sospensione; il risultato di queste operazioni è che il semaforo vale ancora 0, ma entrambi i processi possono ora continuare la loro esecuzione. Un'istruzione `up` su di un semaforo non nullo, lo incrementa di 1. Un semaforo serve essenzialmente a contare i segnali di risveglio per uso futuro, così che non vadano persi. Una proprietà fondamentale delle istruzioni sui semafori è che, una volta che un processo ha intrapreso un'operazione su di un semaforo, nessun altro può accedervi finché il primo non ha completato l'istruzione o finché non resta sospeso nel tentativo di effettuare una `down` su 0. La Figura 6.28 riassume le proprietà essenziali delle chiamate di sistema `up` e `down`.

Come già menzionato, Java possiede una soluzione basata sul linguaggio per trattare le corse critiche: occupandoci qui di sistemi operativi, cerchiamo un modo di rappresentare in Java i semafori anche se non fanno parte del linguaggio o delle classi standard. Lo facciamo ipotizzando l'esistenza di due metodi nativi, `up` e `down`, che svolgono i compiti delle chiamate di sistema `up` e `down` (rispettivamente) e che accettano in ingresso parametri interi. Questi metodi ci consentono di esprimere l'uso dei semafori in Java.

Istruzione	Semaforo = 0	Semaforo > 0
Up	Semaforo = semaforo + 1; se un altro processo era stato arrestato nel tentativo di completare un'istruzione down su questo semaforo, adesso può completare la down e proseguire la propria esecuzione	Semaforo = semaforo + 1
Down	Il processo resta sospeso finché un altro processo non esegue una up su questo semaforo	Semaforo = semaforo – 1

Figura 6.28 Risultato delle operazioni su semafori.

La Figura 6.29 mostra come eliminare la corsa critica tramite l'uso dei semafori. Aggiungiamo alla classe `m` i due semafori `available` (disponibile), che inizialmente vale 100, e `filled` (riempito), posto inizialmente a 0. Come prima, il produttore comincia l'esecuzione da P1 e il consumatore parte da C1. L'esecuzione di `down` su `filled` interrompe immediatamente il processo consumatore. Quando il produttore ha trovato il numero primo invoca `down` con parametro `available`, assegnandogli il valore 99. All'istruzione P5 chiama `up` con parametro `filled`, assegnandogli il valore 1 e liberando così

il consumatore dall'attesa; questi può ora completare la sua chiamata down sospesa. In questo istante `filled` vale 0 ed entrambi i processi sono in esecuzione.

```
public class m {
   final public static int BUF_SIZE = 100;                    // il buffer va da 0 a 99
   final public static long MAX_PRIME = 1000000000000L;       // valore d'arresto
   public static int in = 0,out = 0;                          // puntatori ai dati
   public static long buffer[ ] = new long[BUF_SIZE];         // memorizza i numeri primi
   public static producer p;                                  // nome del produttore
   public static consumer c;                                  // nome del consumatore
   public static int filled = 0, available = 100;             // semafori

   public static void main(String args[ ]) {                  // classe main
      p = new producer( );                                    // crea il produttore
      c = new consumer( );                                    // crea il consumatore
      p.start( );                                             // avvia il produttore
      c.start( );                                             // avvia il consumatore
   }
   // funzione di servizio per l'aggiornamento circolare di in e out
   public static int next(int k) {if (k < BUF_SIZE – 1) return(k+1); else return(0);}
}

class producer extends Thread {                               // classe produttore
   native void up(int s); native void down(int s);            // metodi sui semafori
   public void run( ) {                                       // codice del produttore
      long prime = 2;                                         // variabile di lavoro

      while (prime < m.MAX_PRIME) {
         prime = next_prime(prime);                           // istruzione P1
         down(m.available);                                   // istruzione P2
         m.buffer[m.in] = prime;                              // istruzione P3
         m.in = m.next(m.in);                                 // istruzione P4
         up(m.filled);                                        // istruzione P5
      }
   }

   private long next_prime(long prime){ ... }                 // funzione che calcola il numero primo successivo
}

class consumer extends Thread {                               // classe consumatore
   native void up(int s); native void down(int s);            // metodi sui semafori
   public void run( ) {                                       // codice del consumatore
      long emirp = 2;                                         // variabile di lavoro

      while (emirp < m.MAX_PRIME) {
         down(m.filled);                                      // istruzione C1
         emirp = m.buffer[m.out];                             // istruzione C2
         m.out = m.next(m.out);                               // istruzione C3
         up(m.available);                                     // istruzione C4
         System.out.println(emirp);                           // istruzione C5
      }
   }
}
```

Figura 6.29 Calcolo parallelo con i semafori.

Torniamo alla precedente situazione di corsa critica in una situazione in cui *in* = 22, *out* = 21, il produttore si trova in P1 e il consumatore in C5. Dunque il consumatore completa l'istruzione e torna a C1, dove invoca down su `filled`, che passa così da 1 a 0. Poi

il consumatore preleva l'ultimo numero disponibile nel buffer, chiama up su `available`, che arriva a 100, stampa a schermo il valore e torna nuovamente in C1. Un attimo prima che il consumatore chiami down, il produttore trova il successivo numero primo ed esegue in rapida successione le istruzioni P2, P3 e P4.

Adesso `filled` vale 0. Il produttore è sul punto di eseguire una up su `filled` e il consumatore una down sullo stesso parametro. Se il consumatore esegue per primo l'istruzione rimarrà sospeso finché il produttore non lo libererà (chiamando la up). Se invece parte per primo il produttore, questi porrà il semaforo a 1 e il consumatore non verrà sospeso. In entrambi i casi non viene perso alcun segnale di risveglio; era proprio l'obiettivo che ci eravamo proposti introducendo i semafori.

La proprietà essenziale delle operazioni sui semafori è che sono indivisibili: una volta iniziata un'operazione su di un semaforo, nessun altro processo può usare il semaforo finché il primo non abbia completato l'operazione o non rimanga sospeso. Per di più, l'uso dei semafori scongiura la perdita dei segnali di risveglio. L'istruzione `if` della Figura 6.26 non è indivisibile: è possibile che un processo spedisca un segnale di risveglio tra la valutazione della condizione e l'esecuzione dell'istruzione selezionata.

Il problema della sincronizzazione di processi è stato risolto dichiarando indivisibili le chiamate di sistema up e down, richiamate dai metodi up e down. Perché ciò accada, il sistema operativo deve proibire che un semaforo venga usato da più di un processo per volta, o può almeno limitarsi a interrompere l'esecuzione di ogni altro processo finché quello impiegato nella chiamata di sistema non la porti a compimento. Sui sistemi monoprocessore i semafori sono implementati, qualche volta, mediante la disattivazione degli interrupt durante le operazioni sui semafori. Sui sistemi multiprocessore questo stratagemma non funziona.

La sincronizzazione mediante semafori funziona per un numero arbitrario di processi. Ci possono essere allo stesso tempo molti processi dormienti, impegnati nel tentativo di completare una chiamata di sistema down sullo stesso semaforo. Quando un altro processo effettua una up su quel semaforo, uno dei processi in attesa può completare finalmente la down e riprendere l'esecuzione. Il valore del semaforo resta 0 e gli altri processi restano in attesa.

Usiamo un'analogia per chiarire la natura dei semafori. Immaginate un torneo con 20 squadre di pallavolo suddiviso in 10 partite (i processi), ciascuna disputata sul proprio campo, e una cassa (il semaforo) contenente i palloni. Sfortunatamente ci sono solo 7 palloni disponibili (il semaforo assume valori tra 0 e 7). L'inserimento di un pallone nella cassa equivale a una up perché incrementa il valore del semaforo; l'estrazione di un pallone corrisponde a una down poiché ne decrementa il valore.

All'inizio del torneo ogni campo invia un giocatore al canestro per appropriarsi di un pallone. Sette di loro riusciranno nell'impresa (completano la down), tre sono costretti ad aspettare un pallone (cioè non riescono a completare la down). Le loro partite sono temporaneamente sospese; prima o poi una delle altre partite finirà e il rispettivo pallone verrà riportato nel canestro (esecuzione di una up). Grazie a questa operazione uno dei tre giocatori in attesa potrà accaparrarsi la palla (completando la sua down incompiuta) e dar via alla propria partita. Gli altri due giocatori restano in attesa che vengano riportati nel cesto altri palloni; solo allora le ultime due partite potranno essere disputate.

6.5 Sistemi operativi di esempio

In questo paragrafo procediamo con la trattazione dei nostri sistemi d'esempio, il Core i7 e l'OMAP4430. Per ciascuno analizzeremo uno dei sistemi operativi utilizzati sul processore. Per il Core i7 faremo riferimento a Windows e per OMAP4430 a UNIX. Cominciamo da quest'ultimo perché è più semplice e per molti versi più elegante. Inoltre questo ordine è più sensato e naturale dal momento che UNIX è stato progettato e implementato molto tempo prima di Windows 7 e lo ha assai influenzato.

6.5.1 Introduzione

In questo paragrafo introdurremo brevemente i due sistemi di esempio, UNIX e Windows 7, focalizzandoci sulla loro storia, la loro struttura e le chiamate di sistema.

UNIX

UNIX venne sviluppato presso i laboratori Bell nei primi anni '70. La prima versione, scritta da Ken Thompson in linguaggio assemblativo del minicomputer PDP-7, venne presto seguita da una versione per il PDP-11 (scritta nell'allora nuovo linguaggio C, ideato e sviluppato da Dennis Ritchie). Nel 1974 Ritchie e Thompson pubblicarono un articolo su UNIX che è rimasto una pietra miliare del settore (Ritchie e Thompson, 1974). In ragione del lavoro descritto in quell'articolo furono successivamente insigniti del prestigioso ACM Turing Award (Ritchie, 1984; Thompson, 1984). La pubblicazione dell'articolo stimolò molte università a richiedere una copia di UNIX ai laboratori Bell. Essendo questi di proprietà della AT&T, ai tempi un'azienda monopolista cui non era concesso fare affari nel mondo dell'informatica, la richiesta venne accolta senza difficoltà e la licenza di UNIX venne ceduta alle università per una modica cifra.

Per una di quelle coincidenze che spesso fanno la storia, il PDP-11 era il computer prediletto dai dipartimenti di informatica di quasi tutte le università e i sistemi operativi di cui era equipaggiato erano considerati terribili tanto dai professori quanto dagli studenti. UNIX colmò velocemente questa lacuna, specialmente perché era distribuito con tutti i codici sorgenti e così chiunque poteva armeggiarci liberamente, cosa che fecero in molti.

Una delle prime università a procurarsi UNIX fu la University of California, a Berkeley. Avendo a disposizione tutto il codice sorgente, i ricercatori di Berkeley poterono modificare il sistema in modo sostanziale. Tra i risultati più importanti ci fu l'implementazione sul minicomputer VAX, l'aggiunta della memoria virtuale paginata, l'estensione dei nomi di file da 14 a 255 caratteri e l'inclusione del protocollo di rete TCP/IP, usato ancor oggi in Internet (soprattutto perché si trovava nello UNIX di Berkeley all'avvento delle reti).

Mentre Berkeley procedeva nelle sue modifiche, la stessa AT&T continuava a sviluppare UNIX, giungendo al System III nel 1982 e al System V nel 1984. Alla fine degli anni '80 circolavano due versioni di UNIX diverse e abbastanza incompatibili: UNIX Berkeley e System V. Questa scissione del mondo di UNIX, unita al fatto che mancavano gli standard per i formati dei programmi binari, inibì fortemente il successo commerciale di UNIX perché nessun fornitore di software avrebbe potuto scrivere e confezionare programmi UNIX che funzionassero su ogni sistema UNIX (a differenza di MS-DOS). Dopo molti litigi, il comitato per gli standard IEEE creò uno standard chiamato **POSIX** (*Portable Operating System-IX*, "sistema operativo portabile-IX"), noto anche con il nome di P1003 (il suo numero di standard IEEE). Successivamente POSIX è divenuto uno standard internazionale.

Lo standard è diviso in molte sezioni, ciascuna dedicata a una parte differente di UNIX. La prima sezione, P1003.1, definisce le chiamate di sistema; la seconda sezione, P1003.2, definisce le funzionalità basilari e così via; P1003.1 definisce circa 60 chiamate di sistema che devono essere fornite da tutti i sistemi conformi allo standard. Si tratta delle chiamate elementari per la lettura e scrittura di file, per la creazione di nuovi processi e così via. Praticamente tutti i sistemi UNIX oggi in commercio supportano le chiamate di sistema P1003.1 e in genere molti ne supportano anche altre, in special modo quelle definite da System V e/o da UNIX Berkeley. Queste chiamate di sistema addizionali possono raggiungere facilmente le 200 unità.

Nel 1987 uno degli autori di questo libro (Tanenbaum) distribuì il codice sorgente di MINIX, una piccola versione di UNIX, per uso accademico (Tanenbaum, 1987). Linus Torvald, dopo aver studiato MINIX presso la sua università di Helsinki e dopo averlo installato sul suo PC di casa, decise di scrivere un proprio clone di MINIX, battezzato Linux e divenuto da allora molto popolare.

Molti sistemi operativi in esecuzione su piattaforme ARM sono basati su Linux. MINIX e Linux sono entrambi conformi allo standard POSIX e tutto quanto verrà detto in questo capitolo su UNIX si applica anche a loro, se non specificato diversamente.

La Figura 6.30 fornisce una prima suddivisione in categorie delle chiamate di sistema di Linux. Le categorie più numerose e importanti comprendono le chiamate di sistema per la gestione dei file e delle directory. Linux è prevalentemente conforme a POSIX P1003.1, anche se in alcuni casi gli sviluppatori si sono allontanati dalle specifiche. In generale, non è comunque difficile fare in modo che programmi conformi a POSIX girino su sistemi Linux.

Categoria	Alcuni esempi
Gestione dei file	Apertura, lettura, scrittura, chiusura e lock di file
Gestione delle directory	Creazione e cancellazione delle directory; trasferimento di file
Gestione dei processi	Generazione, terminazione, tracing dei processi e invio di segnali tra loro
Gestione della memoria	Condivisione di memoria tra processi; protezione di pagine
Lettura e scrittura di parametri	Consultazione dell'ID di un utente, di un gruppo, di un processo; assegnamento della priorità
Data e ora	Impostazione dell'ora di accesso a file; timer d'intervallo; profilo esecutivo
Rete	Imposta/accetta connessioni; invia/riceve messaggi
Miscellanea	Creazione di utenti; manipolazione delle quote su disco; riavvio del sistema

Figura 6.30 Suddivisione approssimativa delle chiamate di sistema UNIX.

La categoria delle chiamate di rete è invece dovuta soprattutto a UNIX Berkeley, che ha introdotto il concetto di **socket**, ossia il punto terminale di una connessione di rete, modellata sull'esempio delle prese a muro per le connessioni telefoniche. Un processo UNIX può creare una socket, collegarsi a essa e stabilire una connessione a una socket di un'altra macchina, grazie alla quale può scambiare dati in modo bidirezionale, avvalendosi in genere del protocollo TCP/IP. Una frazione rilevante dei server di Internet funzionano con UNIX proprio perché la sua tecnologia di rete esiste da decenni ed è ormai stabile e matura.

È difficile fare delle asserzioni generali sulla struttura del sistema operativo UNIX perché ne esistono molte implementazioni, ciascuna a suo modo diversa dalle altre. Ciò nondimeno, la Figura 6.31 vale per quasi tutte le implementazioni. Alla sua base si trova il livello dei driver di dispositivo che mette il sistema operativo al riparo dal contatto con l'hardware vero e proprio. In un primo momento i driver di dispositivo venivano scritti come entità indipendenti e separate dagli altri driver, causando un'inutile reiterazione degli sforzi, visto che molti driver devono affrontare problematiche analoghe, quali il controllo del flusso, la gestione degli errori, le priorità, la separazione dei dati dal controllo e così via. Questa osservazione suggerì a Dennis Ritchie lo sviluppo del modello **stream** ("flusso") per la scrittura di driver modulari. Uno stream permette di stabilire una connessione bidirezionale tra un processo dell'utente e un dispositivo hardware e di inserire uno o più moduli lungo il percorso. Il processo dell'utente invia dati lungo lo stream e questi vengono elaborati o trasformati da ciascun modulo finché non giungono all'hardware. I dati provenienti dai dispositivi subiscono l'elaborazione inversa.

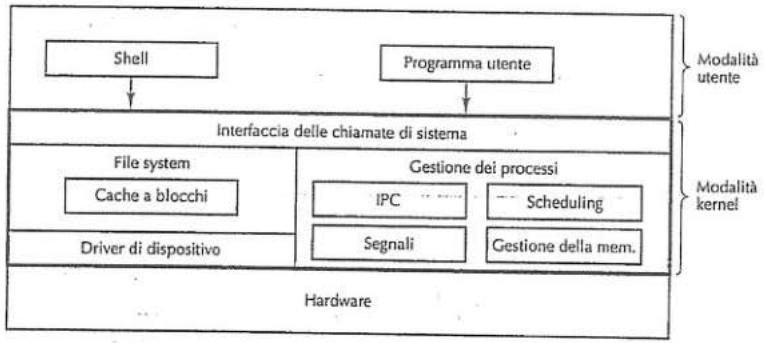


Figura 6.31 Struttura di un comune sistema UNIX.

Al di sopra dei driver di dispositivo c'è il file system che gestisce i nomi dei file, le directory, l'allocazione dei blocchi del disco, la protezione e altro ancora. Parte del file system è la **cache a blocchi** che memorizza i blocchi letti dal disco più di recente in caso dovessero tornare utili a breve. Nel corso degli anni sono stati utilizzati molti file system diversi, tra cui il fast file system di Berkeley (McKusick et al., 1984) e file system a struttura logaritmica (Rosenblum e Ousterhout, 1991; Seltzer et al., 1993).

Un'altra parte del kernel UNIX riguarda la gestione dei processi che, tra le altre cose, si occupa della **IPC** (*InterProcess Communication*, "comunicazione tra processi"). La IPC mette a disposizione svariati meccanismi che consentono ai processi di comunicare tra loro e di sincronizzarsi per evitare le corse critiche. Il codice per la gestione dei processi gestisce anche lo scheduling dei processi in base alle loro priorità. Anche i segnali, che costituiscono una forma di interrupt software (asincrono), sono gestiti a questo livello, così come la memoria. La gran parte dei sistemi UNIX supporta la memoria virtuale con paginazione su richiesta e alle volte fornisce anche delle caratteristiche in più, come la possibilità di condividere regioni dello spazio degli indirizzi tra più processi.

Sin dalla sua nascita, UNIX fu pensato come un sistema dalle dimensioni contenute al fine di incrementarne affidabilità e prestazioni. La prima versione di UNIX era tutta testuale e usava terminali che potevano visualizzare 24 o 25 righe di 80 caratteri ASCII. L'interfaccia utente era gestita dalla **shell** ("conchiglia, involucro"), un programma che girava a livello utente e offriva un'interfaccia a linea di comando. Poiché la shell non faceva parte del kernel era facile aggiungere a UNIX nuove shell, e con il passare del tempo ne sono state realizzate molte altre, sempre più sofisticate.

In seguito all'introduzione dei primi terminali grafici, venne sviluppato presso il M.I.T. un sistema a finestre per UNIX chiamato **X Windows**, poi raffinato mediante l'aggiunta di **Motif**, una **GUI** (*Graphical User Interface*, "interfaccia grafica per l'utente") più elaborata. Queste interfacce si sono con il tempo evolute in veri e propri ambienti desktop con un'accattivante gestione a finestre, dotati di strumenti per aumentare la produttività e di programmi di utilità. Esempi di questi ambienti desktop sono GNOME e KDE. Quasi tutto il codice di X Windows e delle GUI che lo accompagnano girano a livello utente, fuori dal kernel, perché si attengono alla filosofia di UNIX che vuole il kernel di dimensioni contenute.

Windows 7

Al suo lancio nel 1981, il primo PC IBM era equipaggiato con MS-DOS 1.0, un sistema operativo a 16 bit, a singolo utente e orientato alla linea di comando. Questo sistema operativo era costituito da 8 KB di codice residente in memoria. Due anni dopo fece la sua comparsa MS-DOS 2.0, un sistema molto più potente da 24 KB; conteneva un elaboratore di linea di comando (la shell) più un certo numero di caratteristiche prese in prestito da UNIX. Nel 1984 IBM lanciò sul mercato i PC/AT basati sul 286 ed equipaggiati con MS-DOS 3.0, che aveva ormai raggiunto i 36 KB. Con il passare degli anni MS-DOS continuò ad acquisire nuove funzionalità, ma restava pur sempre un sistema orientato alla linea di comando.

Ispirata dal successo dei Macintosh di Apple, Microsoft decise di dotare MS-DOS di un'interfaccia grafica per l'utente che chiamò **Windows**. Le prime tre versioni di Windows, culminate nella versione 3.x, non erano veri sistemi operativi, bensì interfacce grafiche per l'utente montate su MS-DOS, che manteneva ancora il controllo della macchina. Tutti i programmi giravano nello stesso spazio degli indirizzi e così un baco in uno di loro poteva arrestare malamente l'intero sistema.

La distribuzione di Windows 95 nel 1995 mantenne MS-DOS, anche se introdusse la nuova versione 7.0. Windows 95 e MS-DOS 7.0 assommavano insieme quasi tutte le

caratteristiche di un sistema operativo completo, compresa la memoria virtuale, la gestione dei processi e la multiprogrammazione. Tuttavia Windows 95 non era un programma a 32 bit effettivi, ma conteneva grosse porzioni di codice a 16 bit (affiancate al codice a 32 bit) e si appoggiava ancora sul file system di MS-DOS, ereditandone quasi tutti i limiti. I soli cambiamenti di una certa importanza furono l'aggiunta del supporto dei nomi di file lunghi, a estendere gli 8 + 3 caratteri consentiti in MS-DOS, e la possibilità di avere più di 65.536 blocchi su un disco.

MS-DOS sopravvisse anche al lancio di Windows 98 nel 1998 (anche se era arrivato nel frattempo alla versione 7.1) e continuò a eseguire codice a 16 bit. Nonostante la migrazione di qualche altra funzionalità dalla parte MS-DOS verso Windows e l'organizzazione dei dischi che rendeva standard il supporto dei dischi grandi, al di là delle apparenze, Windows 98 non era poi così diverso da Windows 95. La differenza principale stava nell'interfaccia per l'utente che integrava in modo più stretto il desktop, Internet e la riproduzione video. Fu questa integrazione ad attirare l'attenzione del Dipartimento di Giustizia degli Stati Uniti che citò Microsoft in giudizio per aver assunto una posizione illegale di monopolio. A Windows 98 succedette per breve tempo Windows Millennium Edition (ME) che ne costituiva una versione leggermente migliorata.

Parallelamente a questi sviluppi, Microsoft stava lavorando alacremente al completamento di un nuovo sistema operativo a 32 bit, per la cui scrittura era partita da zero, e che chiamò **Windows New Technology** o **Windows NT**. In un primo momento venne presentato come il sistema operativo che avrebbe rimpiazzato tutti gli altri nei PC basati su Intel (così come i PowerPC MIPS), ma per qualche ragione ebbe una diffusione lenta e venne più tardi ricollocato nelle fasce alte di mercato, dove conquistò una propria nicchia. La seconda versione di NT prese il nome di Windows 2000 e divenne la versione di punta, anche nel mercato dei desktop. XP è il successore di Windows 2000, ma si tratta in questo caso di cambiamenti secondari (una migliore compatibilità con i sistemi precedenti e alcune nuove funzioni). Nel 2007 venne rilasciato Windows Vista, in cui vennero implementate numerose migliorie grafiche e aggiunte diverse applicazioni per gli utenti, come Media Center. L'adozione di Vista venne rallentata dalle sue scarse prestazioni e dalle alte richieste in termini di risorse. Solo due anni dopo vide la luce Windows 7, che per molti aspetti non è altro che una versione di Vista risistemata. Windows 7 funziona meglio sull'hardware più datato e richiede molte meno risorse.

Windows 7 è commercializzato in sei diverse versioni. Tre di queste sono destinate agli utenti home dei diversi paesi, due agli utenti business e l'ultima combina tutte le funzioni delle altre. Queste versioni sono pressoché identiche: le differenze risiedono nella loro destinazione d'uso, nelle funzionalità più avanzate e nelle ottimizzazioni che sono incluse. Noi ci concentreremo sugli aspetti centrali del sistema, senza fare ulteriori distinzioni tra le varie versioni.

Prima di addentrarci nell'interfaccia che Windows 7 presenta ai programmatori diamo un breve sguardo alla sua struttura interna, illustrata nella Figura 6.32. La struttura di Windows 7 è costituita da un certo numero di moduli organizzati a livelli e che contribuiscono a implementare il sistema operativo. A ogni modulo corrisponde una funzione particolare e un'interfaccia ben definita con gli altri moduli. Quasi tutti i moduli sono scritti in C, anche se una parte dell'interfaccia del dispositivo grafico è scritta in C++ e una piccola porzione dei livelli più bassi è scritta in linguaggio assemblativo.

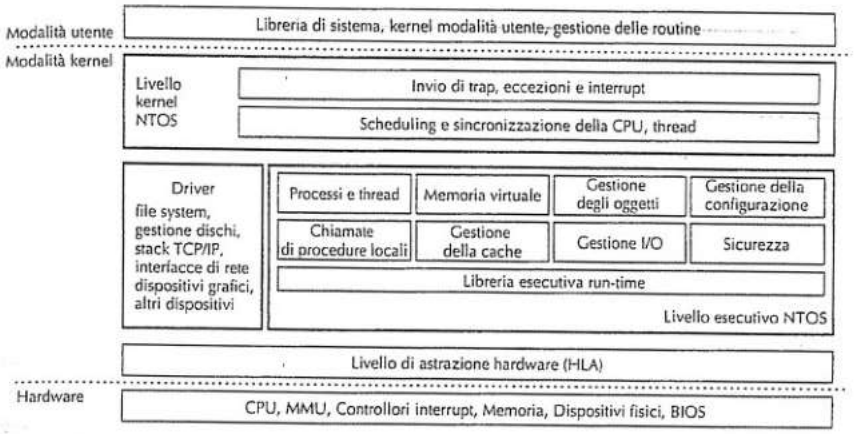


Figura 6.32 La struttura di Windows 7.

Alla base della struttura c'è un sottile **livello di astrazione hardware** (*hardware abstraction layer*) che ha lo scopo di presentare al resto del sistema operativo dispositivi hardware astratti e privi degli spigoli e delle idiosincrasie di cui abbonda l'hardware reale. Tra i dispositivi modellati ci sono le cache non residenti nel chip, i timer, i bus di I/O, i controllori di interrupt e i DMA. La loro presentazione in forma idealizzata al resto del sistema operativo facilita la portabilità di Windows 7 su altre piattaforme hardware, dal momento che tutte le modifiche da effettuare sono concentrate in un unico posto.

Sopra il livello HAL il codice è diviso in due parti principali: il livello esecutivo NTOS e i driver, che includono il file system, il networking e la grafica. Sopra questi si trova il livello kernel. Tutto questo codice viene eseguito in modalità kernel protetta.

L'esecutivo gestisce le astrazioni fondamentali di Windows 7, tra cui i thread, i processi, la memoria virtuale, gli oggetti kernel e le configurazioni. Nello stesso livello ci sono anche i gestori per le chiamate di procedure locali, la cache dei file, l'I/O e il software per la sicurezza.

Il livello kernel gestisce trap, eccezioni, scheduling e sincronizzazione. Fuori dal kernel troviamo i programmi utente e la libreria di sistema usata per intefacciarsi al sistema operativo. Al contrario di UNIX, Microsoft non vuole incoraggiare i programmatori a effettuare chiamate di sistema direttamente, ma preferisce che questi chiamino procedure della libreria. Per garantire conformità tra diverse versioni di Windows (per esempio XP, Vista e 7), Microsoft ha definito un insieme di chiamate detto **API di Win32** (*Application Programming Interface*, "interfaccia per la programmazione delle applicazioni"). Si tratta di procedure di libreria che alle volte si avvalgono delle chiamate di sistema per eseguire il compito loro richiesto, alle altre operano direttamente nella procedura di libreria all'interno dello spazio utente. Anche se dalla definizione di Win32 sono state aggiunte diverse chiamate, noi ci concentreremo sul nucleo centrale delle chiamate di libreria. In seguito all'adattamento di Windows alle macchine a 64 bit,

Microsoft ha cambiato il nome di Win32 per includere la versione a 64 bit, ma ai nostri fini basterà l'analisi della prima versione.

La filosofia delle API di Win32 è completamente diversa da quella di UNIX. Nel caso di UNIX le chiamate di sistema sono tutte pubbliche e formano un'interfaccia minimale: l'eliminazione di una sola di loro ridurrebbe la funzionalità del sistema operativo. Al contrario Win32 fornisce un'interfaccia ridondante (ci sono spesso tre o quattro modi diversi di fare la stessa cosa) che comprende anche alcune funzioni che non dovrebbero essere (e che infatti non sono) chiamate di sistema, per esempio una chiamata API per la copia di un file intero.

Molte chiamate API di Win32 servono a creare oggetti del kernel, come file, processi, thread, pipe e così via, e come risultato restituiscono al chiamante l'**handle** ("impugnatura") dell'oggetto. L'handle può poi essere usato per svolgere le operazioni definite sull'oggetto. L'handle appartiene al processo che ha creato l'oggetto corrispondente e non può essere passato direttamente a un altro processo perché ne faccia uso (nello stesso modo in cui i descrittori di file UNIX non possono essere passati ad altri processi perché ne facciano uso). Tuttavia, in determinate circostanze è possibile duplicare un handle e passarne la copia a un altro processo in modo protetto, consentendogli l'accesso a un oggetto che non gli appartiene. Un handle può essere accompagnato da un descrittore di sicurezza che specifica in dettaglio quali processi possono operare su di esso, e con quali operazioni.

Qualche volta si sente parlare di Windows 7 come di un sistema orientato agli oggetti, perché l'unico modo di manipolare gli oggetti del kernel è di invocare metodi (funzioni API) sui loro handle, che vengono restituiti quando gli oggetti sono creati. D'altra parte, mancano a Windows 7 alcune delle proprietà fondamentali dei sistemi orientati agli oggetti quali l'ereditarietà e il polimorfismo.

6.5.2 Esempi di memoria virtuale

In questo paragrafo affrontiamo la memoria virtuale di UNIX e di Windows 7 che, dal punto di vista del programmatore, sono molto simili.

Memoria virtuale di UNIX

Tutti i processi UNIX hanno tre segmenti: codice, dati e stack (Figura 6.33). Nelle macchine dotate di un solo spazio lineare d'indirizzi, in genere si pone il codice alla base della memoria, lo si fa seguire dai dati e si alloca lo stack in cima alla memoria stessa. La dimensione del codice è fissa, mentre i dati e lo stack sono liberi di crescere in direzioni opposte. Questo modello è molto semplice da implementare su qualsiasi macchina ed è quello usato dalle varianti di Linux utilizzate sulle CPU ARM OMAP4430.

Inoltre, se la macchina consente la paginazione, l'intero spazio degli indirizzi può essere paginato senza che il programmatore neanche se ne accorga. L'unica cosa che il programmatore sa è che può prendersi più spazio per i suoi programmi di quanto ce ne sia in memoria fisica. Per consentire comunque la condivisione di tempo tra un numero arbitrario di processi, i sistemi UNIX che non dispongono della paginazione, in genere fanno lo swap tra memoria e disco di interi processi per volta.

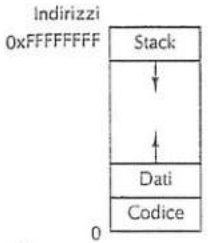


Figura 6.33 Spazio degli indirizzi di un processo UNIX.

Per quanto riguarda la memoria virtuale (con paginazione a richiesta) dello UNIX Berkeley non resta da aggiungere altro. Per il System V (e quindi Linux) invece va aggiunto che esso comprende diverse funzionalità che consentono agli utenti una gestione molto più sofisticata della loro memoria virtuale. Di particolare importanza è la possibilità che un processo mappi un file (o una sua parte) nel proprio spazio degli indirizzi. Per esempio, se un file di 12 KB è mappato all'indirizzo virtuale 144 KB, la lettura della parola all'indirizzo 144 KB equivale alla lettura della prima parola del file. Grazie a questo meccanismo è possibile leggere e scrivere file senza ricorrere alle chiamate di sistema. Dato che la dimensione di un file può eccedere quella dello spazio degli indirizzi virtuali, è possibile mapparne in memoria anche solo una parte. L'operazione comincia con l'apertura del file che restituisce il suo descrittore *fd*, usato successivamente per identificare il file corrispondente da mappare. Quindi il processo effettua la chiamata

```
paddr = mmap(indirizzo_virtuale, lunghezza, protezione, flag, fd, file_offset)
```

che mappa `lunghezza` byte, a partire dalla posizione `offset` all'interno del file, negli indirizzi virtuali che cominciano a `indirizzo_virtuale`. In alternativa è possibile impostare il parametro `flag` in modo che sia il sistema a scegliere un indirizzo virtuale, restituito nella variabile `paddr`. La regione mappata deve comprendere un numero intero di pagine e deve essere allineata alle loro estremità. Il parametro `protezione` può specificare una combinazione qualsiasi di permessi in lettura, scrittura o esecuzione. La corrispondenza può essere poi rimossa con la chiamata `unmap`.

Più processi possono mappare lo stesso file contemporaneamente. La condivisione può avvenire secondo due opzioni. La prima prevede che tutte le pagine siano condivise, perciò le scritture effettuate da un processo sono visibili a tutti gli altri. Questa opzione mette a disposizione un percorso per la comunicazione tra processi con larghezza di banda elevata. L'altra opzione prevede la condivisione delle pagine fintanto che non siano state modificate; non appena un processo tenta di scrivere in una pagina si verifica un errore di protezione, a seguito del quale il sistema operativo fornisce al processo una copia privata della pagina su cui scrivere. Questo schema, noto con il nome di **copia dopo scrittura** (*copy on write*), torna utile quando si vuol dare a ogni processo l'illusione di essere l'unico a mappare un certo file. In questo modello la condivisione è un'ottimizzazione, e non parte della semantica.

Memoria virtuale di Windows 7

In Windows 7 ogni processo utente ha il proprio spazio degli indirizzi virtuali. Nella versione a 32 bit di Windows 7, gli indirizzi virtuali sono lunghi 32 bit, così ogni processo ha uno spazio degli indirizzi virtuali di 4 GB. I primi 2 GB sono disponibili per il codice e i dati, i successivi 2 GB permettono l'accesso (limitato) alla memoria del kernel, fatta eccezione per le versioni Server di Windows, in cui la suddivisione è di 3 GB per l'utente e 1 GB per il kernel. Lo spazio degli indirizzi virtuali è paginato a richiesta, con una dimensione di pagina fissa (4 KB nel Core i7). Lo spazio degli indirizzi nella versione a 64 bit di Windows 7 è simile. Tuttavia, lo spazio di codice e dati è formato dagli 8 terabyte più bassi dello spazio degli indirizzi virtuali.

Le pagine virtuali si possono trovare in uno dei tre stati seguenti: libero, riservato o impegnato. Una **pagina libera** è una pagina non attualmente in uso; un riferimento a essa causa un errore di pagina. All'avvio di un processo tutte le sue pagine sono libere finché non comincia la corrispondenza tra programma e dati nel suo spazio degli indirizzi. Una volta che una parte del codice o dei dati è mappata in una pagina, questa si dice **impegnata** (*committed*). Un riferimento a una pagina impegnata viene mappato per mezzo dell'hardware della memoria virtuale e ha successo solo se la pagina si trova in memoria principale. In caso contrario, si verifica un errore di pagina e il sistema operativo si preoccupa di caricare la pagina dal disco. Una pagina virtuale può anche essere **riservata** per indicare che non è disponibile alla corrispondenza, a meno che la riserva venga rimossa esplicitamente. Le pagine riservate vengono usate quando una sequenza di pagine consecutive può essere necessaria in futuro, come nel caso dello stack. Oltre a questi attributi, una pagina può anche essere leggibile, scrivibile o eseguibile. I primi e gli ultimi 64 KB di memoria sono sempre liberi al fine di intercettare gli errori di puntamento (i puntatori non inizializzati valgono in genere 0 o –1).

A ciascuna pagina impegnata corrisponde su disco una pagina ombra (*shadow page*) dove viene conservata quando non è presente in memoria principale. Le pagine libere o riservate non hanno una pagina ombra, perciò i riferimenti a esse causano errori di pagina (il sistema non può caricare una pagina dal disco se questa non esiste). Le pagine ombra sono organizzate su disco in uno o più file di paginazione. Il sistema operativo si preoccupa di tener traccia della corrispondenza tra le pagine virtuali e le parti dei file di paginazione. Nel caso del testo del programma (con permesso di sola esecuzione) è il file binario eseguibile a contenere le pagine ombra; le pagine di dati sono invece contenute in file speciali.

Windows 7, al pari di System V, permette la corrispondenza diretta tra file e regioni dello spazio degli indirizzi virtuali (cioè in sequenze di pagine consecutive). Una volta mappato nello spazio degli indirizzi, un file può essere letto o scritto tramite le comuni operazioni di riferimento alla memoria.

I file mappati in memoria sono implementati allo stesso modo delle pagine impegnate, con la sola differenza che le pagine ombra si trovano in questo caso nei file stessi invece che nel file di paginazione. Di conseguenza, è possibile che la versione di un file mappato in memoria sia diversa da quella su disco (a causa di scritture recenti nello spazio degli indirizzi virtuali). Tuttavia, quando il file viene estromesso dalla memoria o a seguito di una richiesta esplicita di *flush*, la versione su disco viene aggiornata dalla memoria.

Windows 7 permette espressamente la corrispondenza contemporanea di un file da parte di due o più processi, anche a indirizzi virtuali differenti. Grazie alla lettura e scrittura di parole di memoria, i processi possono comunicare tra loro e scambiarsi dati con una larghezza di banda molto elevata, poiché non è richiesta alcuna operazione di copia. Processi diversi possono avere permessi d'accesso diversi. Poiché tutti i processi che mappano lo stesso file condividono le stesse pagine, i cambiamenti effettuati da uno di loro sono visibili immediatamente a tutti gli altri, anche se non è ancora stata aggiornata la copia su disco.

Le API di Win32 contengono diverse funzioni per consentire a un processo la gestione esplicita della propria memoria virtuale; le più importanti sono elencate nella Figura 6.34. Tutte queste funzioni operano su regioni costituite da una singola pagina o da una sequenza di due o più pagine consecutive nello spazio degli indirizzi virtuali.

Funzione API	Significato
VirtualAlloc	Riserva o impegna una pagina
VirtualFree	Libera o disimpegna una pagina
VirtualProtect	Cambia la protezione in lettura/scrittura/esecuzione di una regione
VirtualQuery	Esamina lo stato di una regione
VirtualLock	Rende una regione residente in memoria (cioè ne disabilita la paginazione)
VirtualUnlock	Rende una regione paginabile nel modo consueto
CreateFileMapping	Crea un oggetto di mappatura file e gli assegna un nome (se specificato)
MapViewOfFile	Colloca (una parte di) un file nello spazio degli indirizzi
UnmapViewOfFile	Rimuove un file mappato dallo spazio degli indirizzi
OpenFileMapping	Apre un oggetto di mappatura file precedentemente creato

Figura 6.34 Le più importanti chiamate API di Windows XP per la gestione della memoria virtuale.

Le prime quattro funzioni API sono autoesplicative. Con le altre due, un processo può costringere la paginazione a tenere un certo numero di pagine in memoria o può rimuovere questo vincolo. La cosa può essere utile alle applicazioni in tempo reale, ma è consentita solo ai programmi dell'amministratore di sistema. Inoltre, il sistema operativo stabilisce dei vincoli cui devono attenersi i processi perché non diventino troppo avidi. Seppur non elencate nella Figura 6.34, Windows 7 dispone di funzioni API per consentire a un processo di accedere alla memoria virtuale di un altro processo per il quale ha ricevuto il controllo (cioè di cui ha un handle).

Le ultime quattro funzioni API elencate servono alla gestione dei file mappati in memoria. La mappatura di un file comincia con la creazione di un oggetto di mappatura file mediante `CreateFileMapping`. La funzione restituisce un handle dell'oggetto di mappatura file e, se richiesto, lo associa a un nome nel file system perché possa essere usato da un altro processo. Le due funzioni seguenti servono rispettivamente alla mappatura e alla rimozione di file dalla memoria. Un file mappato è un file su disco (o una sua parte) che può essere letto o scritto accedendo solamente allo spazio degli indirizzi

virtuali, senza esplicite operazioni di I/O. L'ultima funzione può essere invocata da un processo per mappare un file già mappato da un altro processo. In questo modo, due o più processi possono condividere regioni dei rispettivi spazi degli indirizzi.

Queste sono le funzioni API basilari attorno alle quali è costruito il sistema di gestione della memoria. Per fare un esempio, esistono funzioni API per l'allocazione o la rimozione di strutture dati in uno o più *heap* ("mucchio, cumulo"). Gli heap sono usati per la memorizzazione di quelle strutture dati che vengono create e distrutte dinamicamente e che non sono oggetto di *garbage collection* (l'operazione di rimozione automatica delle strutture dati inutilizzate a cura del sistema), perciò sta al software dell'utente liberare i blocchi di memoria virtuale che non sono più in uso. La gestione dello heap in Windows 7 è simile alla funzione `malloc` dei sistemi UNIX, con la differenza che in Windows 7 ci possono essere più heap gestiti indipendentemente.

6.5.3 Esempi di I/O a livello OS

L'obiettivo principale del sistema operativo è quello di fornire servizi ai programmi utente, soprattutto servizi di I/O come la scrittura e la lettura di file. Sia UNIX sia Windows 7 offrono un ampio ventaglio di servizi di I/O ai programmi utente. Se è vero che esiste una chiamata Windows 7 equivalente a ogni chiamata UNIX, non vale l'inverso, perché Windows 7 possiede molte più chiamate e ogni sua chiamata è molto più complicata della controparte UNIX.

I/O virtuale di UNIX

Buona parte del successo di UNIX risiede nella sua semplicità che, a sua volta, è una conseguenza diretta dell'organizzazione del file system. Un file ordinario è una sequenza lineare di byte che comincia alla posizione 0 e prosegue fino a un massimo di $2^{64} - 1$ byte. Il sistema operativo non impone alcuna struttura ai file, anche se gli utenti interpretano i file di testo ASCII come una successione di righe, ciascuna completata da un carattere di fine linea.

A ogni file aperto è associato un puntatore al byte successivo da leggere o in cui scrivere. Le chiamate di sistema `read` e `write` leggono o scrivono dati a partire dalla posizione nel file indicata dal puntatore. Entrambe le chiamate implicano l'avanzamento del puntatore di un numero di posizioni uguale al numero di byte trasferiti. È tuttavia possibile l'accesso diretto a una posizione del file fornendone il valore al puntatore.

Oltre ai file ordinari, UNIX supporta anche file speciali, usati per accedere ai dispositivi di I/O. In genere ogni dispositivo di I/O è associato a uno o più file speciali. Un programma può utilizzare il dispositivo di I/O attraverso la lettura o scrittura su questi file. È questo il modo in cui vengono gestiti dischi, stampanti, terminali e molti altri dispositivi.

La Figura 6.35 elenca le chiamate di UNIX più importanti relative al file system. La chiamata `creat` (CREATE sarebbe sbagliato!) è utilizzabile per creare un nuovo file, anche se non è più necessaria perché nelle ultime versioni anche la funzione `open` è in grado di farlo. `unlink` cancella un file, nell'ipotesi che questo si trovi in una sola directory.

Chiamata di sistema	Significato
creat(name, mode)	Crea un file (mode specifica la modalità di protezione)
unlink(name)	Cancella un file (assumendo che ci sia un solo link a esso)
open(name, mode)	Apre o crea un file e restituisce il suo descrittore
close(fd)	Chiude un file
read(fd, buffer, count)	Legge count byte in un buffer
write(fd, buffer, count)	Scrive count byte da un buffer
lseek(fd, offset, w)	Sposta il puntatore del file secondo offset e w
stat(name, buffer)	Restituisce le informazioni sul file
chmode(name, mode)	Cambia la modalità di protezione del file
fcntl(fd, cmd, ...)	Esegue varie operazioni di controllo come il lock di (parte di) un file

Figura 6.35 Chiamate principali per il file system di UNIX.

La chiamata `open` si usa per aprire file esistenti (e per crearne di nuovi). L'indicatore `mode` specifica la modalità d'apertura (in lettura, scrittura e così via). La chiamata restituisce come risultato il **descrittore di file**, un piccolo intero che serve a identificare il file nelle chiamate successive.

L'I/O propriamente detto si svolge con le chiamate `read` e `write`, le quali dispongono di un descrittore del file da usare, un buffer per i dati da leggere o contenente i dati da scrivere, più un valore `count` che stabilisce la quantità di dati da trasmettere. `lseek` serve a spostare il puntatore al file, in modo da realizzare l'accesso diretto alle sue posizioni.

La chiamata `stat` serve per ottenere informazioni sui file, quali la dimensione, la data dell'ultimo accesso, il proprietario e altro ancora. `chmod` modifica la modalità di protezione del file, per esempio consentendo o negando la lettura del file agli utenti che non ne sono proprietari. Infine `fcntl` svolge svariate operazioni, tra cui l'aggiunta o la rimozione di un blocco che indica se il file è stato riservato o meno per l'accesso esclusivo da parte di un processo.

La Figura 6.36 illustra il funzionamento delle principali chiamate di I/O; il codice raffigurato è ridotto, in quanto non comprende il pur necessario controllo degli errori. Prima di entrare nel ciclo, il programma apre un file esistente, `data`, e ne crea uno nuovo, `newf`; le due chiamate prendono in ingresso come secondo parametro i bit di protezione che specificano la modalità dei due file, rispettivamente in lettura e in scrittura, e restituiscono a loro volta due descrittori di file, `infd` e `outfd`. Se una delle due chiamate non dovesse andare a buon fine, restituirebbe un descrittore di file negativo, a indicare l'insuccesso.

La procedura `read` ha tre parametri: un descrittore di file, un buffer e un numero di byte. La `read` prova a leggere dal file il numero di byte specificato e a porlo nel buffer, ma il numero di byte effettivamente letto è restituito in `count` e sarà minore di `bytes` se il file è troppo corto. La chiamata `write` ripone i byte appena letti nel file di output. Il ciclo prosegue finché non termina la lettura di tutto il file di input, al che il ciclo termina e i file vengono chiusi.

```
/* Apertura dei descrittori di file. */
infd = open("data", 0);
outfd = creat("newf", ProtectionBits);

/* Ciclo di copia. */
do {
        count = read(infd, buffer, bytes);
        if (count > 0) write(outfd, buffer, count);
} while (count > 0);

/* Chiusura dei file. */
close(infd);
close(outfd);
```

Figura 6.36 Frammento di programma per la copia di un file con chiamate di sistema UNIX. Usiamo il C perché mette bene in evidenza le chiamate di sistema di livello più basso (a differenza di Java, che le nasconde).

I descrittori di file di UNIX sono dei numeri interi (in genere minori di 20): i descrittori 0, 1 e 2 sono speciali e riservati rispettivamente a **standard input, standard output** e **standard error,** che puntano di norma alla tastiera, allo schermo e ancora allo schermo, ma che possono essere rediretti[1] su file specificati dall'utente. Molti programmi di UNIX prendono dati in ingresso dallo standard input e scrivono in uscita sullo standard output; spesso sono chiamati **filtri.**

Intimamente legato al file system è il directory system. Ogni utente può possedere molte directory, che a loro volta possono contenere file o altre sottodirectory. I sistemi UNIX sono configurati spesso come mostrato dalla Figura 6.37, con una directory principale, la **directory radice** (*root directory*), che contiene le sottodirectory *bin* (per i programmi di esecuzione frequente), *dev* (per i file speciali di dispositivi di I/O), *lib* (per le librerie) e *usr* (per le directory degli utenti). Nell'esempio la directory *usr* contiene le sottodirectory *ast* e *jim*; la prima contiene a sua volta due file, *data* e *foo.c*, e una sottodirectory, *bin*, che contiene quattro giochi.

Quando sono presenti diversi dischi (o diverse partizioni dello stesso disco), questi possono essere montati (*mount*) sull'albero dei nomi in modo che tutti i file di tutti i dischi appaiano nella stessa gerarchia di directory e che siano tutti raggiungibili dalla directory radice.

Un file è rintracciabile tramite il suo **percorso** (*path*) a partire dalla radice. Un percorso contiene la lista di tutte le directory attraversate dalla radice fino al file, separate da un carattere barra. Per esempio, il percorso assoluto del file *gioco2* è */usr/ast/bin/gioco2*. Si dice **percorso assoluto** un percorso di file che comincia dalla radice.

A ciascun programma in esecuzione è associata una **directory di lavoro** (*working directory*). I percorsi dei file possono essere specificati relativamente alla directory di lavoro, nel qual caso cominciano con una barra, perché siano distinguibili dai percorsi

1 Per redirezione di un puntatore (descrittore di file) si intende l'attribuzione a esso di una nuova locazione (direzione) a cui puntare (*N.d.T.*).

assoluti. Parliamo in tal caso di **percorsi relativi**: se la directory di lavoro è */usr/ast*, *gioco3*, il file è raggiungibile usando il percorso relativo *bin/gioco3*. Un utente può creare un **collegamento** (*link*) a un file di un altro utente grazie alla chiamata di sistema `link`. Nell'esempio precedente */usr/ast/bin/gioco3* e */usr/jim/jotto* puntano allo stesso file. Non sono permessi collegamenti a directory per evitare la formazione di cicli nel directory system. Le chiamate `open` e `creat` accettano come parametro sia percorsi relativi sia assoluti.

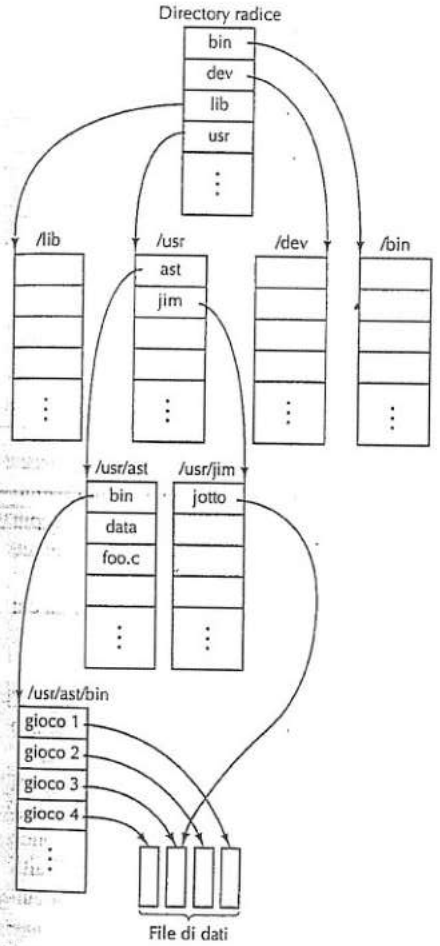


Figura 6.37 Parte del directory system di un tipico sistema UNIX.

Le principali chiamate di sistema per la gestione delle directory UNIX sono elencate nella Figura 6.38. `mkdir` crea una nuova directory e `rmdir` rimuove una directory (vuota) già esistente. Le successive tre chiamate si usano per leggere gli elementi di una directory: la prima apre la directory, la seconda legge i suoi elementi, la terza chiude la directory. `chdir` cambia la directory di lavoro.

Chiamata di sistema	Significato
mkdir(name, mode)	Crea una directory nuova
rmdir(name)	Elimina una directory vuota
opendir(name)	Apre una directory per la lettura
readdir(dirpointer)	Legge l'elemento successivo di una directory
closedir(dirpointer)	Chiude una directory
chdir(dirname)	Cambia la directory di lavoro in *dirname*
link(name1, name2)	Crea un elemento di directory *name2* che punta a *name1*
unlink(name)	Rimuove *name* dalla sua directory

Figura 6.38 Chiamate principali di UNIX per la gestione delle directory.

`link` crea in una directory un nuovo elemento che punta a un file esistente. Per esempio, l'elemento */usr/jim/jotto* potrebbe essere stato creato dalla chiamata

```
link("/usr/ast/bin/gioco3", "/usr/jim/jotto")
```

o da una chiamata equivalente con, in ingresso, percorsi relativi dipendenti dalla directory di lavoro del programma che effettua la chiamata. `unlink` rimuove un elemento dalla directory: se il file ha un solo collegamento viene cancellato, ma se ne ha di più viene mantenuto. Non ha alcuna importanza se il collegamento rimosso era l'originale o una copia.

Una volta creato, un collegamento ha tutti i diritti e le proprietà dell'originale, da cui è indistinguibile. La chiamata

```
unlink("/usr/ast/bin/gioco3")
```

rende *gioco3* accessibile d'ora in poi solo attraverso il percorso */usr/jim/jotto*. In questo modo è possibile usare `link` e `unlink` per "trasferire" file da una directory all'altra.

A ogni file (e alle directory, che sono esse stesse dei file) è associata una stringa di bit che specifica chi può accedervi, ed è costituita da tre campi RWX: il primo controlla i permessi si lettura (R), scrittura (W) ed esecuzione (X) del proprietario, il secondo controlla i permessi degli altri utenti dello stesso gruppo, il terzo riguarda tutti gli altri utenti. Così RWX R-X --X vuol dire che il proprietario può leggere, scrivere ed eseguire il file (se si tratta di un programma eseguibile, altrimenti la X sarebbe assente), mentre gli altri utenti del suo gruppo possono solo leggerlo o eseguirlo e gli estranei possono solo eseguirlo. Con questi permessi gli estranei possono usare il programma, ma non rubarlo (copiarlo), poiché non hanno il permesso in lettura. L'attribuzione di un gruppo

agli utenti è fatta dall'amministratore di sistema, chiamato in genere **superuser**, che ha anche il potere di scavalcare il meccanismo di protezione e può leggere, scrivere o eseguire qualunque file.

Veniamo ora brevemente all'implementazione dei file e delle directory UNIX (per una trattazione completa si veda Vahalia, 1996). A ogni file (e quindi a ogni directory) è associato un blocco d'informazioni di 64 byte detto **i-node**, che contiene le informazioni sul proprietario e sui permessi del file, sulla locazione dei dati e altre cose simili. Gli i-node dei file sono conservati in sequenza all'inizio del disco o all'inizio di un gruppo di cilindri, se il disco è così suddiviso. UNIX è in grado di calcolare l'indirizzo su disco di un i-node a partire dal suo numero nella sequenza.

Gli elementi delle directory consistono in due parti: un nome di file e un numero di i-node. Quando un programma esegue

```
open("foo.c", 0)
```

il sistema cerca il file di nome "foo.c" nella directory di lavoro al fine di localizzarne il numero di i-node. Una volta trovatone il numero può accedere all'i-node e a tutte le informazioni sul file.

Se viene specificato un percorso più lungo, i passi precedenti vengono ripetuti finché non sia stato attraversato l'intero percorso. Per esempio, per localizzare il numero di i-node di */usr/ast/data*, il sistema cerca prima l'elemento *usr* nella directory radice. Quando ha trovato l'i-node di *usr* può leggere il file corrispondente (ripetiamo: in UNIX le directory sono file) e cercare al suo interno l'elemento *ast*, arrivando a localizzare il numero di i-node del file */usr/ast*. La lettura di */usr/ast* consente al sistema di trovare l'elemento *data* e quindi il numero di i-node di */usr/ast/data*. Grazie a quel numero è possibile trovare tutto quanto riguarda il file suddetto.

Il formato, i contenuti e l'organizzazione degli i-node variano leggermente da sistema a sistema (specie se il sistema svolge attività di rete), ma almeno le voci seguenti si trovano in tutte le versioni di i-node:

1. il tipo del file, i 9 bit di protezione RWX e qualche altro bit;
2. il numero di collegamenti al file (numero di elementi di directory che lo punta);
3. l'identità del proprietario;
4. il gruppo del proprietario;
5. la lunghezza del file in byte;
6. tredici indirizzi su disco;
7. la data dell'ultimo accesso in lettura;
8. la data dell'ultimo accesso in scrittura;
9. la data dell'ultima modifica all'i-node.

Riguardo il tipo, un file può essere un file ordinario, una directory o uno dei due tipi di file speciali che si usano per distinguere i dispositivi di I/O strutturati a blocchi da quelli non strutturati. Abbiamo già analizzato il numero di collegamenti e l'identificativo del proprietario. La lunghezza del file è data da un intero di 32 bit che specifica l'ultima

locazione del file con un contenuto informativo: è del tutto lecito creare un file, fare una `lseek` alla posizione 1.000.000 e scrivere un byte, nel qual caso la lunghezza del file è di 1.000.001 byte. Va detto però che l'eventuale memorizzazione del file non richiederebbe alcuno spazio per i byte "mancanti".

I primi 10 indirizzi del disco puntano a blocchi di dati. Se la dimensione di un blocco è 1024 byte, in questo modo è possibile gestire file fino alla dimensione di 10.240 byte. L'indirizzo 11 punta a un blocco del disco che si chiama **blocco indiretto**, contenente diversi indirizzi del disco. Con un blocco di 1024 byte e indirizzi del disco di 32 bit, il blocco indiretto conterrebbe 256 indirizzi del disco. In questo modo è possibile gestire file fino alla dimensione di $10.240 + 256 \times 1024 = 272.384$ byte. Nel caso dei file ancora più grandi si usa l'indirizzo 12 che punta a un blocco contenente 256 blocchi indiretti, riuscendo a gestire così file di $272.384 + 256 \times 256 \times 1024 = 67.381.248$ byte. Se anche questo schema con **blocco indiretto doppio** non è sufficiente per contenere il file, si usa l'indirizzo 13 per puntare a un **blocco indiretto triplo** contenente 256 indirizzi di blocchi indiretti doppi. Usando congiuntamente gli indirizzi dei blocchi diretti, indiretti, indiretti doppi e tripli si possono indirizzare 16.843.018 blocchi, raggiungendo il limite teorico di dimensione per i file di 17.247.250.432 byte. Se gli indirizzi del disco sono di 64 bit invece che di 32 bit e i blocchi sono di 4 KB, i file possono essere davvero molto, molto grandi. In realtà, essendo i puntatori di file di 32 bit, il limite pratico è di 4.294.967.295 byte. I blocchi liberi del disco sono mantenuti in una lista concatenata da cui vengono rimossi uno alla volta quando necessari. Di conseguenza, i blocchi di ciascun file si trovano sparpagliati alla rinfusa per tutto il disco.

Al fine di rendere le operazioni di I/O più efficienti, all'apertura di un file il suo inode viene copiato (per ragioni di comodità) in una tabella in memoria principale e lì mantenuto finché il file non viene chiuso. Inoltre viene tenuta in memoria anche una raccolta dei blocchi referenziati di recente; poiché i file vengono letti in modo sequenziale nella stragrande maggioranza dei casi, succede spesso che un riferimento alla locazione di un file si trovi all'interno dello stesso blocco del riferimento precedente. Motivato da questa considerazione, il sistema cerca di leggere anche il blocco *successivo* a quello correntemente in lettura prima che venga effettivamente referenziato, in modo da accelerare l'esecuzione. Tutti questi meccanismi di ottimizzazione sono nascosti all'utente: quando un utente effettua una `read`, il suo programma resta sospeso finché non siano disponibili nel buffer i dati da lui richiesti.

Alla luce di queste nozioni, possiamo ora capire come funziona l'I/O su file. La `open` induce il sistema a cercare le directory nel percorso specificato: se la ricerca ha successo, l'i-node viene caricato nella tabella interna. Le chiamate `read` e `write` richiedono al sistema il calcolo del numero di blocco a partire dalla posizione corrente nel file. Gli indirizzi su disco dei primi 10 blocchi si trovano sempre in memoria centrale (nell'i-node); i blocchi di numero maggiore richiedono la lettura preventiva di uno o più blocchi indiretti. `lseek` si limita a spostare il puntatore alla posizione corrente senza effettuare alcun I/O.

È ora semplice comprendere anche `link` e `unlink`: `link` cerca all'interno del primo argomento il numero di i-node, quindi crea un elemento di directory nel secondo argomento, assegnandogli lo stesso numero di i-node. Infine incrementa di un'unità il numero di collegamenti nell'i-node. `unlink` rimuove un elemento da una directory e decrementa il numero di collegamenti nell'i-node. Se il numero arriva a zero, il file viene cancellato e i suoi blocchi resi disponibili tramite la loro inserzione nella lista dei blocchi liberi.

I/O di Windows 7

Windows 7 supporta molti file system, tra cui i più importanti sono NTFS (*NT File System*) e FAT (*File Allocation Table*, "tabella di allocazione dei file"). NTFS è stato sviluppato specificamente per NT; FAT è il vecchio file system di MS-DOS, usato anche da Windows 95/98 (anche se con l'introduzione dei nomi di file lunghi). Poiché FAT è ormai obsoleto, eccezion fatta per le chiavette USB e le schede di memoria per fotocamere, ci limitiamo allo studio del primo.

I nomi di file in NTFS possono essere lunghi un massimo di 255 caratteri e utilizzano Unicode per consentire agli utenti di quei paesi la cui lingua non deriva dal latino (per esempio i giapponesi, gli indiani o gli israeliani), di attribuire ai file nomi nella loro lingua madre. Per la precisione, Windows 7 usa Unicode estesamente al proprio interno; fin da Windows 2000 il sistema viene distribuito in un unico codice binario e le differenze linguistiche, che si manifestano nei menu, nei messaggi di errore e così via, sono gestite tramite file di configurazione differenziati per nazione. NTFS è un sistema completamente *case sensitive* (cioè, per esempio, *pippo* è diverso da *PIPPO*) ma sfortunatamente le API di Win32 lo sono solo parzialmente per i nomi di file e non lo sono affatto per i nomi di directory, così questa proprietà viene a mancare ai programmi che usano Win32.

Come per UNIX, un file Windows 7 è semplicemente una sequenza lineare di byte, che però può raggiungere i $2^{64} - 1$ byte. I puntatori di file esistono, come in UNIX, ma sono di 64 bit invece che di 32, in modo da poter gestire file della massima lunghezza possibile. Le chiamate di funzioni API di Win32 per la manipolazione dei file e delle directory sono abbastanza simili alla loro controparte UNIX, a parte il fatto che hanno spesso più parametri e che si avvalgono di un modello di sicurezza diverso. L'apertura di un file restituisce un handle, poi usato per la lettura o la scrittura del file. A differenza di UNIX, però, gli handle non sono piccoli interi, perché devono servire a identificare potenzialmente milioni di oggetti kernel.

Le funzioni principali delle API di Win32 per la gestione dei file sono elencate nella Figura 6.39.

Veniamo ora alle chiamate di sistema. `CreateFile` genera un file nuovo e restituisce il suo handle; si usa anche per aprire i file già esistenti, dato che non esiste una funzione *open* specifica. Nella figura non abbiamo incluso i parametri delle funzioni API di Windows 7 perché sono molto voluminosi. Solo per fare un esempio, i parametri della `CreateFile` sono sette:

1. un puntatore al nome del file da creare o da aprire;
2. dei flag per indicare se il file può essere letto, scritto, o entrambe le cose;
3. dei flag che specificano se il file può essere aperto contemporaneamente da più processi;

4. un puntatore al descrittore di sicurezza che stabilisce chi può accedere al file;
5. dei flag che indicano il comportamento da assumere se il file esiste/non esiste;
6. dei flag che riguardano gli attributi del file, come il suo essere un file di archivio, compresso e così via;
7. l'handle di un file da cui clonare gli attributi per l'attribuzione al nuovo file.

Funzione API	UNIX	Significato
CreateFile	open	Crea un file o apre un file esistente; restituisce l'handle
DeleteFile	unlink	Cancella da una directory la voce relativa a un file esistente
CloseHandle	close	Chiude un file
ReadFile	read	Legge dati da un file
WriteFile	write	Scrive dati in un file
SetFilePointer	lseek	Assegna al puntatore del file una determinata posizione
GetFileAttributes	stat	Restituisce le informazioni sul file
LockFile	fcntl	Mette in lock una regione del file per garantire la mutua esclusione
UnlockFile	fcntl	Toglie il lock da una regione di un file precedentemente riservata

Figura 6.39 Funzioni principali delle API di Win32 per l'I/O di file. La seconda colonna riporta la funzione di UNIX più simile.

Le successive sei funzioni API della Figura 6.39 sono abbastanza simili alle corrispondenti chiamate di UNIX. Si noti tuttavia che l'I/O di Windows 7 è in linea di principio asincrono, anche se un processo può aspettare il completamento. Le ultime due consentono di porre o di togliere il lock su una regione di un file per garantirne l'accesso esclusivo da parte di un processo (ovvero la mutua esclusione tra processi).

A partire da queste funzioni API è possibile scrivere una procedura per la copia di un file analoga a quella della Figura 6.36. La Figura 6.40 mostra una siffatta procedura (senza controllo degli errori), concepita sul modello della versione UNIX. Nella pratica non c'è alcun bisogno di usare il codice della figura, perché esiste la funzione API `CopyFile` che chiama una procedura di libreria il cui codice è simile a quello indicato nell'esempio.

Windows 7 supporta un file system gerarchico simile a quello di UNIX, anche se il carattere separatore dei componenti di un nome è \ invece di /, un retaggio di MS-DOS. Esiste il concetto di directory di lavoro e i nomi di percorso possono essere assoluti o relativi. C'è comunque una differenza sostanziale rispetto a UNIX: UNIX permette di "montare" (*mount*) insieme i file system di dischi o di macchine diversi in un unico albero dei nomi a partire da una radice comune, nascondendo così la struttura dei dischi al software. Le versioni più vecchie di Windows (pre Windows 2000) non avevano questa proprietà, perciò i nomi di file assoluti devono cominciare con la lettera del drive (l'unità) che indica il disco logico in cui si trova il file, per esempio in *C:\windows\system\foo.dll*. A partire da Windows 2000 è stata introdotta la possibilità di montare un file system nello stile di UNIX.


```
/* Apertura dei file per input e output. */
inhandle = CreateFile("data", GENERIC_READ, 0, NULL, OPEN_EXISTING, 0, NULL);
outhandle = CreateFile("newf", GENERIC_WRITE, 0, NULL, CREATE_ALWAYS,
        FILE_ATTRIBUTE_NORMAL, NULL);

/* Copia del file. */
do {
        s = ReadFile(inhandle, buffer, BUF_SIZE, &count, NULL);
        if (s > 0 && count > 0) WriteFile(outhandle, buffer, count, &ocnt, NULL);
} while (s > 0 && count > 0);

/* Chiusura dei file. */
CloseHandle(inhandle);
CloseHandle(outhandle);
```

Figura 6.40 Frammento di programma per la copia di un file con le funzioni API di Windows 7. Usiamo il C perché mette bene in evidenza le chiamate di sistema di livello più basso (a differenza di Java, che le nasconde).

La Figura 6.41 elenca le funzioni API più importanti per la gestione delle directory insieme alle corrispettive funzioni UNIX. I loro nomi sono sufficientemente autoesplicativi.

Funzione API	UNIX	Significato
CreateDirectory	mkdir	Crea una directory nuova
RemoveDirectory	rmdir	Elimina una directory vuota
FindFirstFile	opendir	Prepara alla lettura degli elementi di una directory
FindNextFile	readdir	Legge l'elemento di directory successivo
MoveFile		Sposta un file da una directory in un'altra
SetCurrentDirectory	chdir	Cambia la directory di lavoro corrente

Figura 6.41 Principali funzioni API di Win32 per la gestione di directory. La seconda colonna indica la funzione di UNIX più simile, se ne esiste una.

Windows 7 è dotato di un meccanismo di sicurezza molto più elaborato di quello di molti sistemi UNIX. Benché esistano centinaia di funzioni API relative alla sicurezza, cerchiamo di fornire una breve descrizione dell'idea generale. Quando un utente effettua il login, il sistema operativo affida al suo processo iniziale un **gettone di accesso** (*access token*) che contiene il **SID** (*Security ID*) dell'utente, una lista dei gruppi di sicurezza cui egli appartiene, i privilegi speciali di cui dispone, il livello di integrità del processo, e poche altre informazioni. L'idea del gettone di accesso è di concentrare tutte le informa-

zioni di sicurezza in un solo posto facile da reperire. Tutti i processi creati a partire da quello iniziale ereditano lo stesso gettone di accesso.

Uno dei parametri d'ingresso che può essere passato all'atto della creazione di un oggetto è il suo **descrittore di sicurezza**, contenente una lista di controllo degli accessi, ACL (*Access Control List*). Ogni elemento della lista consente oppure vieta un certo insieme di operazioni sull'oggetto da parte di un certo SID o di un determinato gruppo. Per esempio un file potrebbe avere un descrittore di sicurezza secondo cui Marta non ha accesso al file, Marco può leggerlo soltanto, Linda può leggerlo o scrivervi e tutti i membri del gruppo XYZ possono soltanto leggere la lunghezza del file, nient'altro. Si può anche impostare per default il divieto di accesso a tutti quelli che non sono esplicitamente elencati.

Quando un processo cerca di eseguire un'operazione su un oggetto tramite un handle, il gestore della sicurezza prende il token di accesso del processo e controlla prima di tutto il livello di integrità nel token. Un processo non può mai ottenere un handle con permessi di scrittura per oggetti con un livello di integrità più alto. I livelli di integrità sono usati principalmente per restringere il campo d'azione del codice caricato da un browser Web nelle modifiche al sistema. Dopo il controllo del livello di integrità, il gestore della sicurezza scorre ordinatamente la lista delle voci di ACL. Non appena incontra un elemento corrispondente al SID o a gruppo del chiamante, il tipo di accesso ivi consentito è considerato come definitivo. Per questo motivo si usa elencare nella ACL prima gli elementi che negano l'accesso, poi quelli che lo consentono, così che un utente cui è stato negato espressamente l'accesso non riesca a rientrare dalla porta di servizio perché appartenente a un gruppo autorizzato. Il descrittore di sicurezza contiene anche informazioni per la certificazione degli accessi all'oggetto.

Diamo ora uno sguardo all'implementazione di file e directory in Windows 7. Ogni disco è suddiviso in volumi statici indipendenti che hanno la stessa funzione delle partizioni UNIX. Ogni volume contiene una bit map, file e directory, più altre strutture dati per la gestione delle informazioni. Ciascun volume è organizzato come una sequenza lineare di **cluster** ("raggruppamenti") di dimensione prefissata (compresa tra i 512 byte e i 64 KB) che dipende dalla dimensione del volume stesso. I riferimenti a un cluster avvengono tramite il suo offset dall'inizio del volume, per il quale si usa un numero di 64 bit.

La struttura dati principale di un volume è la **MFT** (*Master File Table*, "tabella principale di allocazione dei file") che contiene un elemento per ogni file o directory del volume, analogo all'i-node di UNIX. La stessa MFT è un file e perciò può essere posizionata ovunque nel volume. Questa proprietà è utile in caso di blocchi di disco difettosi all'inizio del volume, dove solitamente si trova la struttura MTF. I sistemi UNIX memorizzano di solito alcune informazioni chiave all'inizio di ogni volume e nel caso (alquanto improbabile) in cui uno di questi blocchi sia irrimediabilmente danneggiato l'intero volume deve essere riposizionato.

La MFT (illustrata nella Figura 6.42) comincia con un'intestazione contenente le informazioni relative al volume, come il puntatore alla directory di radice, il file di boot, il file dei blocchi corrotti, l'amministrazione della lista delle locazioni libere e così via. Di seguito si trovano gli elementi relativi ai file o alle directory che occupano 1 KB

ciascuno, tranne quando la dimensione del cluster è di 2 KB o più. Ogni elemento contiene i metadati (informazioni amministrative) che riguardano il file o la directory. Sono consentiti diversi formati, uno dei quali è presentato nella Figura 6.42.

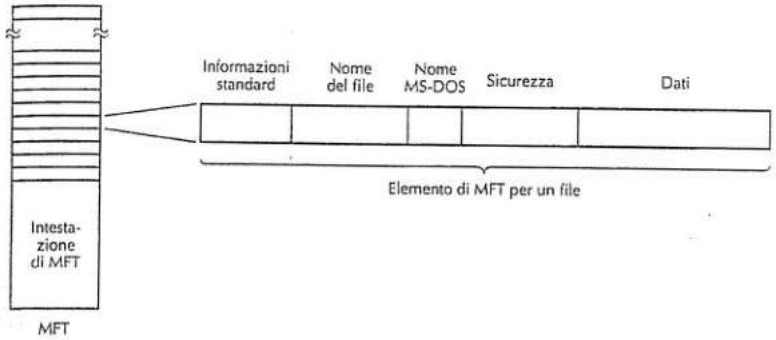


Figura 6.42 Tabella MFT di Windows XP.

Il campo delle informazioni standard contiene alcuni dati come le etichette temporali (*time stamp*) necessarie a POSIX, il computo del numero di collegamenti, i bit di sola lettura o di archivio e così via. Questo campo ha lunghezza fissa ed è sempre presente tra i metadati. Il campo del nome del file contiene un massimo di 255 caratteri Unicode. Affinché i file siano ancora accessibili dai programmi a 16 bit, viene fornito anche un campo per il nome MS-DOS costituito da otto caratteri alfanumerici eventualmente seguiti da un punto e da non più di tre caratteri alfanumerici per l'estensione. Se il vero nome del file rispetta già lo standard MS-DOS 8+3, non viene fornito un secondo nome MS-DOS.

Il campo successivo contiene le informazioni di sicurezza. Fino alla versione di Windows NT 4.0 questo campo conteneva il descrittore di file vero e proprio. A partire da Windows 2000 si è deciso di accentrare tutte le informazioni di sicurezza in un file unico e lasciare nel campo sicurezza un puntatore a una locazione del file.

Il campo dei dati nell'elemento di MFT contiene il file vero e proprio se questo è abbastanza piccolo, risparmiando così un accesso a disco. Questo stratagemma si chiama **file immediato** (Mullender e Tanenbaum, 1984). Per quanto riguarda i file più grandi, il campo dati contiene i puntatori ai cluster contenenti i dati oppure, più comunemente, a intere sequenze di cluster consecutivi; così con un solo identificativo di cluster e un parametro di lunghezza è possibile specificare una quantità di dati arbitraria. Se un elemento di MFT dovesse rivelarsi insufficiente a contenere tutte le informazioni richieste da un file è sempre possibile accorparlo a uno o più elementi successivi della tabella.

La dimensione massima dei file è di 2^{64} byte. Per capire quanto grande sia 2^{64} byte (cioè 2^{67} bit), si immagini una sequenza di zeri e di uno che occupino ciascuno un millimetro: una sequenza di 2^{67} bit siffatti (2^{64} byte) sarebbe lunga 15 anni luce, abbastanza

per oltrepassare i confini del sistema solare, raggiungere Alpha Centauri e tornare indietro.

Il file system NTFS presenta molte altre proprietà interessanti, tra cui il supporto a flussi multipli di dati per ogni file, la crittografia, la compressione dei dati e la tolleranza agli errori ottenuta per mezzo delle transazioni atomiche. Per informazioni ulteriori riguardo NTFS si consulti (Russinovich e Solomon 2005).

6.5.4 Esempi di gestione dei processi

UNIX e Windows 7 prevedono entrambi la possibilità di suddividere un compito (*job*) tra più processi che possono girare in modo (pseudo)parallelo e comunicare con lo schema produttore-consumatore già illustrato. In questo paragrafo analizziamo la gestione dei processi dei due sistemi. Vedremo anche che entrambi supportano il parallelismo all'interno di un singolo processo mediante i thread.

Gestione dei processi in UNIX

Un processo UNIX può in ogni istante usare la chiamata di sistema `fork` per creare un sottoprocesso che è una sua esatta replica. Il processo originale è detto **genitore** e quello nuovo si chiama processo **figlio**. Appena dopo la `fork`, i due processi sono identici e condividono addirittura gli stessi descrittori di file. Di lì in poi ciascuno prosegue per la propria strada e svolge il proprio compito indipendentemente dall'altro.

Spesso il processo figlio manipola i descrittori di file ed esegue una chiamata di sistema `exec` che rimpiazza il suo programma e i suoi dati con il programma e i dati contenuti in un certo file eseguibile specificato come parametro della chiamata `exec`. Per esempio, quando un utente immette il comando *xyz* in un terminale, l'interprete a linea di comando (la shell) esegue una `fork` per creare un processo figlio che svolga il programma *xyz* a seguito di una `exec`.

I due processi girano in parallelo (con o senza `exec`) a meno che il genitore voglia attendere la terminazione del figlio prima di continuare. In questa evenienza, il genitore esegue una chiamata di sistema `wait` o `waitpid` che causa la sua sospensione fino al termine dell'esecuzione del figlio. Solo allora il genitore riprende l'esecuzione.

I processi possono eseguire quante `fork` vogliono, dando vita così a un albero di processi. La Figura 6.43 mostra il processo *A* che ha eseguito due `fork` e ha così generato i due figli *B* e *C*. Anche *B* ha eseguito due volte una `fork`, mentre *C* l'ha eseguita una volta sola, per un totale di sei processi.

In UNIX i processi possono comunicare tra loro attraverso una struttura chiamata **pipe** ("conduttura"). Una pipe è una specie di buffer in cui un processo può scrivere un flusso di dati disponibili al prelievo da parte di un altro processo. I byte sono prelevati da una pipe sempre nell'ordine in cui sono stati inseriti, non è mai consentito l'accesso diretto a una certa posizione. Le pipe non preservano le estremità dei messaggi, perciò se un processo effettua quattro scritture da 128 byte e un altro processo effettua una lettura da 512 byte, il secondo riceverà tutti i dati in una volta sola, senza alcuna traccia del fatto che sono stati scritti in più operazioni.

System V e Solaris mettono a disposizione un'altra modalità di comunicazione tra processi che usa le **code di messaggi** (*message queue*). Un processo può usare la chiamata `msgget` per creare una nuova coda di messaggi o aprirne una esistente, può invocare `msgsnd` o `msgrecv` per spedire o ricevere un messaggio su una coda. I messaggi che passano in una coda differiscono per molti versi da quelli che attraversano le pipe. Innanzitutto vengono preservati gli estremi dei messaggi, mentre una pipe è solo un flusso di dati. Poi i messaggi possono avere priorità, così quelli urgenti possono sopravanzare quelli meno importanti. Infine, i messaggi hanno un tipo, che può essere specificato tramite la `msgrecv`.

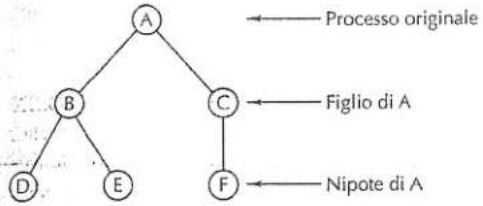


Figura 6.43 Albero di processi in UNIX.

Un altro meccanismo per la comunicazione tra processi è la condivisione di una o più regioni dei loro spazi degli indirizzi. UNIX gestisce la condivisione di memoria attraverso la corrispondenza simultanea tra alcune pagine e gli spazi degli indirizzi di tutti i processi. L'effetto di questa soluzione è che una scrittura operata in una regione condivisa è immediatamente visibile agli altri processi. Questo meccanismo garantisce una comunicazione con larghezza di banda molto elevata. Le chiamate di sistema coinvolte nella condivisione di memoria si chiamano `shmat` e `shmop`.

Un'altra caratteristica di System V e di Linux è il supporto dei semafori, il cui funzionamento ricalca in buona sostanza quanto visto nell'esempio del produttore-consumatore.

Un'ulteriore funzionalità offerta dai tutti i sistemi UNIX conformi a POSIX è la gestione di thread multipli all'interno di un singolo processo. I **thread** sono processi leggeri che condividono lo stesso spazio degli indirizzi e tutto ciò che gli è associato: i descrittori di file, le variabili d'ambiente e i timer esterni. Tuttavia, ogni thread ha il proprio program counter, i propri registri e il proprio stack. Quando un thread si blocca (per esempio perché si trova in attesa del completamento di un'operazione di I/O o di un altro evento) l'esecuzione può passare agli altri thread dello stesso processo. Due thread produttore-consumatore che operano all'interno dello stesso processo sono simili, ma non identici, a due processi costituiti ciascuno da un solo thread e che condividono un segmento di memoria come buffer. Le differenze stanno nel fatto che in quest'ultimo caso ogni processo ha i propri descrittori di file, variabili d'ambiente e così via, mentre nel primo caso questi enti sono condivisi. Abbiamo incontrato i thread Java nell'esempio del produtttore-consumatore. Spesso il sistema esecutivo di Java si avvale dei thread del sistema operativo per i propri thread, ma può anche scegliere di non farlo.

I server web sono un buon esempio per illustrare l'utilità dei thread. Questi server possono contenere una cache della pagine web usate più comunemente, così se arriva la richiesta di una pagina presente nella cache, questa può essere inviata immediatamente. In caso contrario, la pagina deve essere caricata dal disco e ciò comporta tempi maggiori (20 ms circa), durante i quali il processo resta bloccato e non può servire nuove richieste in ingresso, anche se si trattasse di richieste di pagine nella cache.

La soluzione consiste nel generare più thread all'interno del processo server, dove tutti condividono la stessa cache delle pagine web. Quando si blocca un thread, gli altri possono continuare a gestire le richieste in ingresso. In alternativa, sarebbe possibile moltiplicare il numero dei processi invece di quello dei thread, ma ciò richiederebbe probabilmente una replica della cache per ogni processo, con conseguente spreco di memoria.

Lo standard di UNIX per i thread si chiama **pthread** ed è definito da POSIX (P1003.1C). Sono previste chiamate per la gestione e per la sincronizzazione di thread, ma non è specificato se questi devono risiedere nello spazio del kernel o in quello dell'utente. Le chiamate più comuni per la gestione dei thread sono elencate nella Figura 6.44.

La prima chiamata, `pthread_create`, crea un nuovo thread, e dopo il suo completamento lo spazio degli indirizzi del chiamante contiene un thread in più in esecuzione. Quando un thread ha completato il proprio compito chiama `pthread_exit`, che ne provoca la terminazione. Un thread può attendere la terminazione di un altro thread tramite la chiamata `pthread_join`: se il thread che si vuole attendere è già terminato la chiamata si interrompe immediatamente, altrimenti il chiamante resta bloccato in attesa.

Chiamata di thread	Significato
pthread_create	Crea un nuovo thread nello spazio degli indirizzi del chiamante
pthread_exit	Provoca la terminazione del thread chiamante
pthread_join	Aspetta la terminazione di un thread
pthread_mutex_init	Crea un nuovo mutex
pthread_mutex_destroy	Distrugge un mutex
pthread_mutex_lock	Riserva un mutex
pthread_mutex_unlock	Rilascia un mutex
pthread_cond_init	Crea una variabile di condizione
pthread_cond_destroy	Distrugge una variabile di condizione
pthread_cond_wait	Attende su una variabile di condizione
pthread_cond_signal	Libera un thread in attesa su di una variabile di condizione

Figura 6.44 Principali chiamate POSIX per i thread.

I thread si sincronizzano grazie a particolari lock chiamati **mutex** e usati generalmente per sorvegliare l'uso di buffer condivisi da due thread. Per avere la certezza di essere l'unico ad accedere a una risorsa condivisa, un thread deve assicurarsi il mutex prima di modificare la risorsa e deve rimuoverlo non appena ha concluso il proprio accesso. Se

tutti i thread seguissero questo protocollo, evitare le corse critiche sarebbe possibile. I mutex somigliano a semafori binari, cioè a semafori che possono assumere solo i valori 0 e 1. Il nome "mutex" deriva dal fatto che sono usati per assicurare la mutua esclusione (*mutual exclusion*) da certe risorse.

Le chiamate `pthread_mutex_init` e `pthread_mutex_destroy` servono rispettivamente a creare e a distruggere un mutex. Un mutex può trovarsi in uno di due stati: riservato o non riservato. Per riservare un mutex non riservato, un thread usa `pthread_mutex_lock` che ha effetto immediato; viceversa, se il mutex è già riservato la chiamata diventa bloccante. Spetta al thread che aveva riservato il mutex precedentemente chiamare `pthread_mutex_unlock` per rilasciarlo non appena abbia completato le proprie operazioni sulla risorsa condivisa.

I mutex servono a salvaguardare gli accessi a breve termine alle variabili condivise, mentre per la sincronizzazione a lungo termine (come nell'attesa per un'unità a nastro) si usano le **variabili di condizione**. La loro creazione e distruzione avviene tramite le chiamate `pthread_cond_init` e `pthread_cond_destroy`.

L'uso di una variabile di condizione coinvolge due thread, uno in attesa sulla variabile e un altro che effettua la segnalazione per sopraggiunta disponibilità. Per esempio, se un thread realizza che l'unità a nastro che gli serve è occupata, chiama `pthread_cond_wait` su una variabile di condizione associata all'unità e precedentemente concordata tra tutti i thread. Quando il thread che ha in uso l'unità completa la propria attività su di essa (non importa quanto tempo dopo), chiama `pthread_cond_signal` per liberare esattamente un thread in attesa su quella variabile di condizione (se ce n'è almeno uno). Se non c'è alcun thread in attesa il segnale va perso; le variabili di condizione non sono contatori come i semafori. Esistono poche altre operazioni definite sui thread, sui mutex e sulle variabili di condizione che però non tratteremo.

Gestione dei processi in Windows 7

Windows 7 supporta esistenza, comunicazione e sincronizzazione di più processi. Ogni processo contiene almeno un thread. Processi e thread (che possono essere schedulati dal processo stesso) costituiscono un insieme di strumenti molto generale per la gestione del parallelismo sia su sistemi uniprocessore (macchine con CPU singola) sia multiprocessore (macchine con più CPU).

I processi sono creati mediante la funzione API `CreateProcess` che accetta 10 parametri, ciascuno dotato di molte opzioni. Si tratta evidentemente di un progetto molto più complesso dello schema di UNIX, la cui `fork` non ha parametri, mentre `exec` ne ha solo tre: il puntatore al nome del file da eseguire, l'array dei parametri individuati nella linea di comando e le stringhe d'ambiente. Con buona approssimazione, i 10 parametri di `CreateProcess` servono a contenere:

1. un puntatore al nome del file eseguibile;
2. la stessa linea di comando (prima dell'analisi di parsing);
3. un puntatore al descrittore di sicurezza del processo;
4. un puntatore al descrittore di sicurezza del thread iniziale;
5. un bit che determina se il nuovo processo eredita gli handle del processo creatore;

6. vari flag (per esempio la modalità d'errore, la priorità, il debugging, le console);
7. un puntatore alle stringhe d'ambiente;
8. un puntatore al nome della directory di lavoro del nuovo processo;
9. un puntatore alla struttura che descrive la finestra iniziale a schermo;
10. un puntatore alla struttura che serve per restituire 18 valori al chiamante.

Windows 7 non impone alcun rapporto di parentela o gerarchia tra processi: tutti i processi creati sono uguali. D'altra parte, poiché uno dei 18 parametri restituiti al chiamante è l'handle del nuovo processo (che gli consente un certo controllo sul processo neonato) risulta definita implicitamente una gerarchia in base alla relazione "il processo *x* possiede l'handle del processo *y*". Nonostante sia vietato il passaggio diretto di handle tra processi, c'è comunque un modo con cui un processo può adattare un handle per passarlo a un altro e così la gerarchia definita implicitamente può avere vita breve.

Ogni processo di Windows 7 all'atto della creazione contiene un solo thread, ma è possibile aggiungervene successivamente di nuovi. La creazione dei thread è più semplice di quella dei processi; `CreateThread` ha soli 6 parametri invece di 10: descrittore di sicurezza, dimensione dello stack, indirizzo iniziale, un parametro definito dall'utente, stato iniziale (pronto o bloccato) e ID. La creazione dei thread è svolta dal kernel che è dunque al corrente della loro esistenza (i thread non sono implementati interamente nello spazio dell'utente come avviene in altri sistemi).

Durante la propria attività di scheduling, il kernel guarda soltanto i thread eseguibili, senza prestare attenzione al processo a cui appartengono. Ciò significa che il kernel è sempre informato sullo stato (pronto o bloccato) di ogni thread. I thread sono oggetti del kernel e in quanto tali hanno un handle e un descrittore di sicurezza. Poiché è possibile il passaggio di handle tra processi, è altresì possibile per un processo controllare (o creare) i thread di un altro processo. È questa una caratteristica utile, per esempio, alla scrittura di debugger.

I processi possono comunicare in un numero svariato di modi; attraverso pipe, pipe con nome, *mailslot*, socket, chiamate di procedura a distanza e file condivisi. Alla creazione di una pipe si può scegliere tra due modalità di funzionamento: a byte e a messaggi. Le pipe a byte funzionano come quelle di UNIX. Le pipe a messaggi sono abbastanza simili, ma preservano le estremità dei messaggi: così quattro scritture di 128 byte ciascuna verranno ricevute come quattro messaggi di 128 byte e non come un solo messaggio di 512 byte, come accadrebbe con una pipe a byte. Esistono anche le pipe con nome, che mettono a disposizione entrambe le modalità. A differenza delle pipe ordinarie, le pipe con nome possono essere usate anche via rete.

Le socket sono come le pipe, se non che di norma collegano processi che risiedono su macchine diverse, ma possono essere usate anche per la comunicazione tra processi della stessa macchina. In genere non assicurano vantaggi particolari rispetto alla connessione di due macchine tramite una pipe, con o senza nome.

Le chiamate di procedura a distanza (*remote procedure call*) consentono al processo *A* di richiedere al processo *B* di chiamare per proprio conto una terza procedura nello spazio degli indirizzi di *B*, e di farsi restituire il risultato. Esistono diverse limitazioni sui parametri; per esempio non ha alcun senso passare un puntatore a un altro processo. Piuttosto, gli oggetti puntati devono essere inpacchettati e inviati al processo destinazione.

Infine i processi possono condividere la memoria attraverso la sovrapposizione di uno stesso file. Tutte le scritture effettuate da un processo compaiono anche nello spazio degli indirizzi dell'altro processo. Con questo meccanismo l'implementazione del buffer condiviso nell'esempio del produttore-consumatore è molto semplice.

Così come Windows 7 fornisce molti meccanismi di comunicazione tra processi, mette anche a disposizione un gran numero di tecniche di sincronizzazione, tra cui i semafori, i mutex, le sezioni critiche e gli eventi. Sono tutti meccanismi che operano a livello dei thread, non dei processi, perciò se un thread si blocca su un semaforo ciò non ha alcun effetto sugli (eventuali) altri thread dello stesso processo, che possono così continuare la propria esecuzione.

La funzione API `CreateSemaphore` crea un nuovo semaforo, con la possibilità di assegnargli un certo valore e un valore massimo. I semafori sono oggetti del kernel e hanno quindi un handle e un descrittore di sicurezza. È possibile duplicare l'handle di un semaforo attraverso la chiamata `DuplicateHandle` e passarlo a un altro processo affinché diversi processi si sincronizzino sullo stesso semaforo. Ai semafori può anche essere assegnato un nome nel momento della loro creazione, così che altri processi li possano aprire usandone il nome. Sono presenti le chiamate per le operazioni di up e down, anche se hanno nomi un po' peculiari: `ReleaseSemaphore` e `WaitForSingleObject` rispettivamente. Inoltre è possibile passare a `WaitForSingleObject` un timeout dopo il quale il thread chiamante viene liberato in ogni caso, anche se il semaforo resta a 0 (e sebbene l'uso del timer reintroduca le corse critiche).

Anche i mutex sono oggetti del kernel usati per la sincronizzazione, ma sono più semplici dei semafori perché non sono associati a contatori. Sono fondamentalmente dei lock cui corrispondono le funzioni API `WaitForSingleObject` (per riservarli) e `ReleaseMutex` (per rilasciarli). Al pari degli handle dei semafori, anche gli handle dei mutex possono essere duplicati e passati tra processi di modo che processi diversi accedano agli stessi mutex.

Il terzo meccanismo di sincronizzazione si basa sulle **sezioni critiche**, simili ai mutex. Diversamente da questi però, le sezioni critiche non sono oggetti del kernel, ma sono locali allo spazio degli indirizzi del thread che le ha generate. Di conseguenza non hanno handle né descrittori di sicurezza e non possono essere passate da un processo all'altro. Vengono riservate e rilasciate per mezzo delle chiamate `EnterCriticalSection` e `LeaveCriticalSection`. Sono funzioni molto più veloci delle analoghe definite sui mutex, perché vengono svolte interamente nello spazio dell'utente. Windows 7 offre anche variabili di condizione, lock leggeri di lettura e scrittura, operazioni libere da lock e altri meccanismi di sincronizzazione che operano soltanto tra thread dello stesso processo.

L'ultima forma di sincronizzazione usa ancora oggetti del kernel, che prendono il nome di **eventi**. Un thread può attendere che si verifichi un evento richiamando `WaitForSingleObject`, può liberare un thread in attesa su un evento per mezzo di `SetEvent`, o addirittura invocare `PulseEvent` per risvegliare tutti i thread in attesa su quell'evento. Anche gli eventi presentano molte varianti e consentono di scegliere tra diverse

opzioni. Windows utilizza gli eventi per la sincronizzazione quando un I/O asincrono viene completato, e per altri scopi.

È possibile attribuire un nome agli eventi, ai mutex e ai semafori e memorizzarli nel file system, proprio come avviene per le pipe con nome. Due o più processi possono poi sincronizzarsi tramite la semplice apertura dello stesso evento, mutex o semaforo, senza il bisogno che un processo crei l'oggetto e ne duplichi l'handle per passarlo a tutti gli altri.

6.6 Riepilogo

Si può guardare al sistema operativo come al garante di alcune funzionalità architetturali non presenti nel livello ISA. Prime fra tutte ci sono la memoria virtuale, le istruzioni virtuali di I/O e il supporto del calcolo parallelo.

La memoria virtuale è una funzionalità architetturale che ha lo scopo di fornire ai programmi spazi d'indirizzi più grandi della memoria fisica del sistema, insieme a un meccanismo flessibile e corretto per la protezione e la condivisione della memoria. Può essere implementata con la paginazione pura, con la segmentazione pura o con una combinazione delle due. Nel primo caso lo spazio degli indirizzi è suddiviso in pagine virtuali di uguale dimensione, alcune delle quali sono contenute in blocchi fisici di memoria. Un accesso a una pagina in memoria è tradotto dalla MMU nell'indirizzo fisico corretto, altrimenti il tentativo di accesso causa un errore di pagina. Il Core i7 e la CPU ARM OMAP4430 dispongono entrambi di MMU che gestiscono la memoria virtuale e la paginazione.

L'astrazione di I/O più importante presente a questo livello è il file. Un file consiste in una sequenza di byte o di record logici che può essere letta o scritta senza alcuna nozione del funzionamento dei dischi, delle unità a nastro o di altri dispositivi di I/O. L'accesso ai file può essere sequenziale o diretto (per chiave o per numero di record). Le directory sono utilizzate per raggruppare file. Questi possono essere memorizzati in settori consecutivi del disco o sparpagliati al suo interno. Nel secondo caso, che è il più frequente nei dischi normali, si fa uso di strutture dati per la localizzazione dei blocchi di un file. È possibile mantenere traccia delle locazioni libere del disco tramite una lista o una bit map.

Il calcolo parallelo viene spesso gestito e implementato tramite la simulazione di processi multipli che condividono a tempo la singola CPU. Le interazioni tra processi, se non controllate opportunamente, possono portare a corse critiche. Per scongiurare questo problema si introducono primitive di sincronizzazione, di cui un esempio è dato dai semafori. Grazie ai semafori è possibile risolvere problemi analoghi a quello del produttore-consumatore in maniera semplice ed elegante.

Windows 7e UNIX sono due esempi di sistemi operativi molto sofisticati. Entrambi supportano paginazione, corrispondenza tra file in memoria e file system gerarchico, ed entrambi intendono i file come semplici sequenze di byte. Infine tutti e due i sistemi operativi forniscono la gestione dei processi e dei thread, nonché diverse modalità per la loro sincronizzazione.

PROBLEMI

1. Perché un sistema operativo interpreta solo alcune delle istruzioni del livello 3, mentre un microprogramma interpreta tutte le istruzioni del livello ISA?
2. Una macchina ha uno spazio degli indirizzi virtuali di 32 bit indirizzabile al byte. La dimensione di pagina è di 4 KB. Quante pagine d'indirizzi virtuali esistono?
3. È necessario che le pagine abbiano dimensioni pari a potenze di 2? Sarebbe possibile, in teoria, implementare pagine di 4000 byte? Se sì, si tratterebbe di una soluzione pratica?
4. Una memoria virtuale ha dimensioni di pagina di 1024 parole, otto pagine virtuali e quattro blocchi fisici di memoria. La tabella delle pagine è la seguente:

Pagina virtuale	Blocco di memoria
0	3
1	1
2	assente dalla memoria
3	assente dalla memoria
4	2
5	assente dalla memoria
6	0
7	assente dalla memoria

   a. Elencare tutti gli indirizzi virtuali che causerebbero un errore di pagina.
   b. Quali sono gli indirizzi fisici corrispondenti a 0, 3728, 1023, 1024, 1025, 7800 e 4096?
5. Un computer ha 16 pagine d'indirizzi virtuali e solo quattro blocchi di memoria. La memoria è inizialmente vuota. Se un programma accede alle pagine virtuali secondo l'ordine
   0, 7, 2, 7, 5, 8, 9, 2, 4
   a. Quali riferimenti causerebbero un errore di pagina se si usa LRU?
   b. Quali riferimenti causerebbero un errore di pagina se si usa FIFO?
6. Nel Paragrafo 6.1.4 è stato introdotto un algoritmo per l'implementazione della sostituzione delle pagine secondo la strategia FIFO. Se ne progetti una versione più efficiente. *Suggerimento*: è possibile limitarsi ad aggiornare il contatore della sola pagina appena caricata, lasciando invariato quello delle altre pagine.
7. Nei sistemi paginati trattati nel capitolo, il gestore degli errori di pagina faceva parte del livello ISA e non era contenuto perciò nello spazio degli indirizzi di alcun programma del livello OSM. In realtà lo stesso gestore degli errori di pagina occupa alcune pagine e potrebbe venire esso stesso rimosso in determinate circostanze (per esempio se si adottasse la politica di sostituzione delle pagine FIFO). Che cosa accadrebbe se il gestore degli errori di pagina non si trovasse in memoria quando capita un errore di pagina? Come si potrebbe risolvere il problema?
8. Non tutti i computer dispongono di un meccanismo che asserisce automaticamente un bit hardware quando viene effettuata una scrittura in una pagina. Nondimeno è utile tener traccia delle pagine modificate al fine di evitare la scrittura su disco di tutte le pagine caricate (quando non servono più). Se in ogni pagina esistono alcuni bit hardware per l'abilitazione separata dei permessi in lettura, in scrittura e in esecuzione, come può il sistema operativo usare questi bit per stabilire se una pagina è "pulita" o se è stata modificata?
9. Una memoria segmentata è a paginazione dei segmenti. Ogni indirizzo virtuale ha 2 bit per il numero di segmento, 2 bit per il numero di pagina e 11 bit per l'offset all'interno della pagina. La memoria principale contiene 32 KB, suddivisi in pagine di 2 KB. Per ogni segmento sono possibili solo le configurazioni seguenti: in sola lettura, in lettura/esecuzione, in lettura/scrittura o in lettura/scrittura/esecuzione. La tabella delle pagine e le modalità di protezione sono le seguenti:

Segmento 0		Segmento 1		Segmento 2	Segmento 3	
Sola lettura		Lettura/esecuzione		Lettura/ scrittura/esecuzione	Lettura/scrittura	
Pagina virtuale	Blocco di memoria	Pagina virtuale	Blocco di memoria		Pagina virtuale	Blocco di memoria
0	9	0	su disco	Tabella delle pagine non presente in memoria principale	0	14
1	3	1	0		1	1
2	su disco	2	15		2	6
3	12	3	8		3	su disco

Per ognuno dei seguenti accessi alla memoria virtuale si indichi l'indirizzo fisico che viene calcolato. Se si verifica un errore di pagina, specificarne il tipo.

Accesso	*Segmento*	*Pagina*	*Offset nella pagina*
1. fetch di dati	0	1	1
2. fetch di dati	1	1	10
3. fetch di dati	3	3	2047
4. memorizzazione di dati	0	1	4
5. memorizzazione di dati	3	1	2
6. memorizzazione di dati	3	0	14
7. salto a	1	3	100
8. fetch di dati	0	2	50
9. fetch di dati	2	0	5
10. salto a	3	0	60

10. Alcuni calcolatori consentono l'I/O direttamente nello spazio dell'utente. Per esempio, un programma potrebbe avviare un trasferimento dal disco verso un buffer di un processo dell'utente. Questa possibilità può causare problemi se la memoria virtuale è implementata con compattamento? Si argomenti la risposta.

11. I sistemi operativi che permettono la corrispondenza tra file in memoria richiedono che essa avvenga su multipli di pagine (e che rispetti le estremità delle pagine). Per esempio, se le pagine sono di 4 KB, un file può essere mappato a partire dall'indirizzo virtuale 4096, ma non dall'indirizzo 5000. Perché?

12. Quando nel Core i7 viene caricato un registro di segmento, il descrittore corrispondente viene rintracciato e caricato all'interno di una porzione invisibile del registro di segmento. Si dia una giustificazione per questa scelta dei progettisti Intel.

13. Un programma del Core i7 effettua un accesso al segmento locale 10 con offset 8000. Il campo BASE dell'elemento di LDT corrispondente al segmento 10 contiene il numero 10000. Quale elemento della directory delle pagine usa il Pentium 4? Qual è il numero di pagina? Qual è l'offset?

14. Si confrontino alcuni possibili algoritmi per l'estromissione dei segmenti in una memoria segmentata, ma non paginata.

15. Si confrontino la frammentazione interna e quella esterna. Che cosa si può fare per ridurle?

16. I supermercati sono abituati ad affrontare un problema molto simile a quello della sostituzione di pagine nei sistemi con memoria virtuale. I loro scaffali hanno una capienza fissa a fronte di un numero di prodotti sempre crescente. Se viene commercializzato un nuovo prodotto rivoluzionario, diciamo un cibo per cani molto proteico, sarà necessario fargli spazio estromettendo alcuni prodotti dall'inventario. Gli algoritmi disponibili sono i soliti LRU e FIFO. Quale sarebbe preferibile in questo contesto?

17. La cache e la paginazione sono per molti versi simili. In entrambi i casi ci sono due livelli di memoria (la cache e la memoria principale nel primo caso, la memoria e il disco nel secondo). In questo capitolo abbiamo enunciato alcune argomentazioni a favore delle pagine di disco grandi e altre a favore di quelle piccole. Valgono le stesse argomentazioni per la dimensione delle linee di cache?

18. Perché molti file system richiedono l'apertura preventiva di un file con una chiamata di sistema open prima della lettura?

19. Si confrontino il metodo che usa la bit map e quello che usa la lista delle lacune per la gestione dello spazio libero su di un disco con 800 cilindri, ciascuno costituito di 5 tracce di 32 settori. Quante lacune ci vorrebbero perché la lista relativa ecceda la dimensione della bit map? Si ipotizzi che l'unità di allocazione sia il settore e che una lacuna richieda un elemento di tabella di 32 bit.

20. Un terzo schema di allocazione delle lacune, che si aggiunge a best fit e first fit, è *worst fit*. Questo schema prevede che a un processo sia allocato lo spazio dato dalla più grande lacuna rimanente. Quale vantaggio si può ottenere dall'utilizzo di questo algoritmo?

21. Si descriva uno scopo della chiamata di sistema file open che non sia stato menzionato nel testo.

22. Al fine di prevedere le prestazioni del disco è utile avere un modello dell'allocazione dei dati. Si supponga che il disco sia uno spazio lineare degli indirizzi con svariati settori, costituito da una sequenza di blocchi di dati, quindi da una lacuna, da un'altra sequenza di blocchi di dati e così via. Se alcune misure empiriche dimostrano che le distribuzioni di probabilità dei dati e delle lacune sono uguali e attribuiscono probabilità 2^{-i} alle sequenze di lunghezza i, qual è il valore atteso delle lacune in un disco?

23. In un certo calcolatore, un programma può creare tanti file quanti sono necessari, e tutti i suoi file possono crescere dinamicamente durante l'esecuzione, senza dare alcuna indicazione preventiva al sistema operativo circa la loro dimensione finale. Su un sistema siffatto i file verranno memorizzati in settori consecutivi? Si motivi la risposta.

24. Le statistiche hanno dimostrato che più della metà dei file ha dimensioni di pochi KB e che la stragrande maggioranza dei file è al di sotto di un certo numero di KB (attorno agli 8 KB). D'altro canto, il 10% dei file più voluminosi è responsabile del 95% dell'intera occupazione di spazio su disco. A partire da questi dati, che conclusione si può trarre circa la dimensione dei blocchi del disco?

25. Si consideri una possibile implementazione delle istruzioni sui semafori da parte di un sistema operativo: quando la CPU sta per effettuare una up o down su un semaforo (una variabile intera in memoria), il sistema imposta o maschera i bit di priorità della CPU in modo tale da disabilitare tutti gli interrupt. Quindi effettua il fetch del semaforo, lo modifica e salta in base al suo valore. Solo a questo punto riabilita gli interrupt. Il metodo funziona se:
    a. c'è una sola CPU che effettua la commutazione tra processi ogni 100 ms
    b. due CPU condividono una memoria comune in cui si trova il semaforo.

26. Nella descrizione dei semafori del paragrafo 6.3.3 viene affermato: "In genere i processi dormienti sono posti in una coda al fine di poterne tenere traccia." Quale vantaggio si ottiene utilizzando una coda per i processi in attesa al posto di svegliare un processo dormiente scelto a caso al momento dell'esecuzione di una up?

27. I produttori del Sistema Operativo Indistruttibile hanno ricevuto delle lamentele da parte di clienti a proposito dell'ultima distribuzione del sistema, in cui sono state introdotte le operazioni sui semafori. Secondo i clienti è immorale che un processo si blocchi (lo definiscono "poltrire"). Dato che l'azienda crede nel motto secondo cui il cliente ha sempre ragione, si è pensato di introdurre una terza operazione a supporto di up e down: l'operazione peek ("sbirciare"), che si limita a esaminare il valore di un semaforo senza modificarlo e senza sospendere l'esecuzione. In tal modo, se un programma crede sia immorale fermare la propria esecuzione, può cominciare con l'ispezionare il semaforo e stabilire se è sicuro effettuare una down. Questa soluzione funziona se il semaforo è usato da tre o più processi? E se è usato da due processi?

28. Redigere una tabella che indichi, in funzione dei tempi sotto indicati (da 0 a 1000 ms), quale dei processi P1, P2 e P3 sia in esecuzione e quale sia bloccato. Tutti e tre i processi effettuano istruzioni up e down sullo stesso semaforo. Quando due processi sono bloccati e viene eseguita una up, viene risvegliato il processo con nominativo più basso, ovvero P1 ha precedenza su P2 e P3 e così via. All'inizio tutti e tre i processi sono in esecuzione e il semaforo vale 1.
    Al tempo t = 100 P1 effettua una down
    Al tempo t = 200 P1 effettua una down
    Al tempo t = 300 P2 effettua una up
    Al tempo t = 400 P3 effettua una down
    Al tempo t = 500 P1 effettua una down
    Al tempo t = 600 P2 effettua una up
    Al tempo t = 700 P2 effettua una down
    Al tempo t = 800 P1 effettua una up
    Al tempo t = 900 P1 effettua una up

29. Nei sistemi di prenotazioni aeree è necessario assicurare che durante l'accesso a un file da parte di un processo, nessun altro possa accedervi. In caso contrario due processi diversi, creati per conto di due compagnie aeree diverse, potrebbero prenotare inavvertitamente lo stesso posto dello stesso volo. Si progetti un metodo di sincronizzazione che usi i semafori per garantire l'accesso esclusivo dei processi ai file (ipotizzando che i processi obbediscano alle regole proposte).

30. Gli architetti degli elaboratori spesso forniscono un'istruzione TSL (*Test and Set Lock*, "esamina e riserva") per rendere possibile l'implementazione dei semafori sui computer con più CPU che condividono la stessa memoria. TSL X esamina la locazione X: se contiene il valore 0, viene posta a 1 all'interno di un solo, indivisibile ciclo di memoria, e l'istruzione successiva viene saltata. Se invece non vale 0, la TSL si comporta come una NOP. Grazie a TSL è possibile scrivere procedure *lock* e *unlock* con le seguenti proprietà: *lock(x)* verifica se x è riservata, in caso contrario la riserva e restituisce il controllo. Se x è riservata invece aspetta finché non venga rilasciata, dopo di che la riserva e restituisce il controllo. *unlock* libera una variabile riservata. Se tutti i processi riservano la tabella dei semafori prima di usarla, solo un processo per volta può manipolare le variabili e i puntatori, evitando così le corse critiche. Si scrivano le procedure *lock* e *unlock* in linguaggio assemblativo (dopo aver fatto le ipotesi ritenute ragionevoli).

31. Si mostrino i valori di *in* e *out* per un buffer circolare lungo 65 parole dopo ciascuna delle seguenti operazioni. Entrambi partono da 0.
    a. Inserimento di 22 parole.
    b. Estrazione di 9 parole.
    c. Inserimento di 40 parole.
    d. Estrazione di 17 parole.
    e. Inserimento di 12 parole.
    f. Estrazione di 45 parole.
    g. Inserimento di 8 parole.
    h. Estrazione di 11 parole.

32. Se una versione di UNIX usa blocchi del disco di 2 KB e memorizza in ogni blocco indiretto (singolo, doppio o triplo) 512 indirizzi del disco, qual è la dimensione massima di file? (si considerino puntatori a file di 64 bit).

33. Si supponga che la chiamata di sistema di UNIX
    unlink("/usr/ast/bin/gioco3")
    sia eseguita nel contesto della Figura 6.37. Si descrivano con cura i cambiamenti risultanti nel directory system.

34. Si immagini di dover implementare UNIX su di un microcomputer con pochissima memoria. Se, nonostante tutti gli sforzi adoperati, il sistema risulta ancora troppo grande, si rende necessario il sacrificio di una chiamata di sistema. La scelta ricade su `pipe`, usata per la creazione delle pipe che permettono la comunicazione di flussi di byte tra processi. È ancora possibile implementare la redirezione di I/O in qualche modo? E le pipeline? Si analizzino il problema e le sue possibili soluzioni.

35. I membri della commissione per l'Uguaglianza dei Descrittori di File stanno organizzando una protesta contro UNIX perché le sue chiamate che restituiscono descrittori di file, restituiscono sempre il numero più piccolo non ancora in uso. Di conseguenza i descrittori di file corrispondenti ai numeri grandi non vengono quasi mai usati. La loro proposta è di far sì che venga restituito il numero più piccolo non ancora usato dal programma, invece che quello più piccolo attualmente inutilizzato. Sostengono che si tratta di una richiesta banale da implementare, che non comporterà la modifica dei programmi esistenti e che renderà il sistema più equo. Che cosa ne pensate?

36. In Windows 7 è possibile generare una lista di controllo degli accessi affinché Roberta non abbia alcun accesso a un certo file e tutti gli altri utenti ne abbiano pieno accesso. Quale può essere l'implementazione di una lista siffatta?

37. Si descrivano due modi diversi di programmare il problema del produttore-consumatore usando i buffer condivisi e i semafori di Windows 7. Si rifletta in particolar modo sull'implementazione del buffer condiviso nelle due soluzioni proposte.

38. Capita frequentemente di usare la simulazione per valutare il comportamento degli algoritmi di sostituzione delle pagine. A tal fine, si scriva un simulatore per una memoria virtuale paginata, con 64 pagine di 1 KB. Il simulatore dovrebbe mantenere in memoria una tabella con 64 elementi, uno per pagina, per indicare il numero della pagina fisica corrispondente a ogni pagina virtuale. Il simulatore dovrebbe leggere da un file gli indirizzi virtuali scritti in decimale, uno per riga. Se la pagina corrispondente è in memoria, è sufficiente annotare un successo di pagina (un page hit). Se non è in memoria, il simulatore chiama una procedura per la scelta della pagina da estromettere dalla memoria (cioè per la scelta dell'elemento della tabella da sovrascrivere) e registra un fallimento di pagina (un page miss). Nella realtà non avviene alcuno spostamento di pagina, ma solo la sua simulazione. Si generi un file d'indirizzi casuali e si valutino le prestazioni di LRU e di FIFO. Quindi si generi un file d'indirizzi in cui una frazione costante x degli indirizzi sia ottenuta sommando quattro byte agli indirizzi appena precedenti (per simulare il principio di località). Si eseguano dei test per valori diversi di x e si riferiscano i risultati ottenuti.

39. Il programma della Figura 6.26 ha una corsa critica fatale perché due thread accedono alle stesse variabili condivise in modo non controllato, senza usare semafori o altre tecniche di mutua esclusione. Si esegua il programma e si osservi quanto tempo passa prima del suo bloccaggio. Se non si osserva alcun bloccaggio, si incrementino le dimensioni della finestra di vulnerabilità inserendo qualche istruzione tra la modifica di `m.in` e `m.out` e la loro valutazione. Quante istruzioni bisogna aggiungere perché l'esecuzione si fermi, per esempio, una volta all'ora?

40. Si scriva un programma per UNIX o per Windows 7 che prende in input il nome di una directory e che stampa la lista dei file in essa contenuti, uno per riga. Il nome di ogni file va fatto seguire dalla stampa della sua dimensione. Si stampino i nomi dei file nell'ordine in cui si trovano nella directory. Gli elementi inutilizzati di una directory devono essere stampati con la dicitura (inutilizzato).

CAPITOLO 7

Livello del linguaggio assemblativo

Mentre nei Capitoli 4, 5 e 6 abbiamo illustrato tre diversi livelli che si trovano nella maggior parte degli odierni calcolatori, qui affronteremo principalmente un ulteriore livello, presente anch'esso su praticamente tutti i calcolatori moderni, quello del linguaggio assemblativo. La differenza principale che contraddistingue questo livello rispetto ai livelli di microarchitettura, ISA e macchina del sistema operativo è che il livello del linguaggio assemblativo è implementato mediante traduzione invece che interpretazione.

I programmi che si occupano di tradurre in un particolare linguaggio un programma scritto dall'utente in un altro linguaggio sono chiamati **traduttori**. Il linguaggio nel quale è scritto il programma originale viene chiamato **linguaggio sorgente**, e **linguaggio destinazione** quello nel quale viene convertito. Sia il linguaggio sorgente sia il linguaggio destinazione definiscono dei livelli. Se esistesse un processore in grado di eseguire direttamente i programmi scritti nel linguaggio sorgente, tradurre il programma originale nel linguaggio destinazione non sarebbe necessario.

Si usa la traduzione quando si ha a disposizione un processore (un processore hardware oppure un interprete) per il linguaggio destinazione, ma non per il linguaggio sorgente. Se la traduzione viene effettuata correttamente, l'esecuzione del programma tradotto fornisce esattamente lo stesso risultato che si otterrebbe dal programma sorgente se si avesse a disposizione un processore in grado di eseguirlo. Di conseguenza è possibile implementare un nuovo livello per il quale non esiste un processore, traducendo inizialmente i suoi programmi in un livello destinazione ed eseguendo poi i programmi del livello destinazione ottenuti dalla traduzione.

È importante notare la differenza che esiste fra traduzione, da una parte, e interpretazione, dall'altra. Nella traduzione il programma originale non viene eseguito direttamente, ma viene invece convertito in un programma equivalente chiamato **programma oggetto** o **programma eseguibile binario** la cui esecuzione è portata avanti solo dopo che la traduzione è stata completata. La traduzione viene realizzata in due passi distinti:

1. generazione di un programma equivalente nel linguaggio destinazione;
2. esecuzione del programma appena generato.

Questi due passi non sono simultanei: il secondo non inizia finché il primo non sia stato completato. Al contrario, nell'interpretazione esiste un unico passo: l'esecuzione del programma sorgente originale. Non occorre generare, in una fase iniziale, alcun programma equivalente, anche se a volte il programma sorgente può essere convertito in una forma intermedia (per esempio il bytecode di Java) per facilitarne l'interpretazione.

Mentre si esegue il programma oggetto sono coinvolti solamente tre livelli: il livello di microarchitettura, il livello ISA e il livello macchina del sistema operativo. Di conseguenza, durante l'esecuzione è possibile trovare nella memoria del calcolatore tre diversi programmi: il programma oggetto dell'utente, il sistema operativo e il microprogramma (se esiste), mentre è svanita qualsiasi traccia del programma sorgente originale. Il numero di livelli presenti al momento dell'esecuzione può dunque essere diverso dal numero di livelli presenti prima di effettuare la traduzione. Occorre tuttavia tener presente che, mentre noi definiamo un livello in base alle istruzioni e ai costrutti linguistici resi disponibili ai suoi programmatori (e non solo in base alle tecniche implementative), altri autori distinguono nettamente i livelli implementati dagli interpreti che lavorano a tempo di esecuzione dai livelli implementati mediante la traduzione.

7.1 Introduzione al linguaggio assemblativo

A seconda della relazione che intercorre tra il linguaggio sorgente e il linguaggio destinazione è possibile dividere sommariamente i traduttori in due gruppi. Quando il linguaggio sorgente è essenzialmente una rappresentazione simbolica di un linguaggio macchina numerico, il traduttore è chiamato **assemblatore** e il linguaggio sorgente è chiamato **linguaggio assemblativo**. Quando il linguaggio sorgente è un linguaggio ad alto livello come Java oppure C e il linguaggio destinazione è un linguaggio macchina numerico oppure una sua rappresentazione simbolica: in questo caso il traduttore viene chiamato **compilatore**.

7.1.1 Che cos'è un linguaggio assemblativo

Un puro linguaggio assemblativo è un linguaggio nel quale ciascuna istruzione produce esattamente un'istruzione macchina. In altre parole esiste una corrispondenza uno-a-uno tra le istruzioni macchina e le istruzioni del programma assemblativo. Se ciascuna linea del linguaggio assemblativo contiene esattamente un'istruzione macchina, un programma assemblativo di n linee genererà un programma in linguaggio macchina di n parole.

La ragione dell'utilizzo del linguaggio assemblativo al posto del linguaggio macchina (in binario o esadecimale) è che è molto più facile programmare utilizzando il linguaggio assemblativo. L'uso di una forma simbolica per i nomi e gli indirizzi, al posto della rappresentazione binaria oppure ottale, costituisce un'enorme differenza. La maggior parte dei programmatori è in grado di ricordare che `ADD`, `SUB`, `MUL` e `DIV` sono le abbreviazioni di aggiungi, sottrai, moltiplica e dividi, mentre pochi riescono a ricordarsi i corrispondenti valori numerici utilizzati dalla macchina. Il programmatore in linguaggio assemblativo deve ricordarsi solamente i nomi simbolici, dato che sarà compito dell'assemblatore tradurli in istruzioni macchina.

La stessa osservazione vale per gli indirizzi. Un programmatore può assegnare nomi simbolici alle locazioni di memoria e lasciare all'assemblatore il compito di determinare i corrispondenti valori numerici. Programmando direttamente in linguaggio macchina è invece obbligatorio lavorare sempre con i valori numerici degli indirizzi. Per questi motivi al giorno d'oggi nessuno programma più in linguaggio macchina, anche se alcuni decenni fa, prima che venissero inventati gli assemblatori, era necessario farlo.

Il linguaggio assemblativo, oltre alla corrispondenza uno-a-uno tra le sue istruzioni e le istruzioni macchina, gode di un'altra proprietà che lo contraddistingue dai linguaggi ad alto livello. Il programmatore in linguaggio assemblativo ha accesso a tutte le funzionalità e a tutte le istruzioni disponibili nella macchina di destinazione, mentre il programmatore in linguaggio ad alto livello non ha la stessa libertà di accesso. Se per esempio la macchina di destinazione ha un bit di overflow, un programma assemblativo può testarlo, mentre un programma Java non lo può fare direttamente. Un programma assemblativo, diversamente da uno ad alto livello, può eseguire qualsiasi istruzione dell'insieme delle istruzioni della macchina di destinazione. In poche parole, tutto ciò che può essere fatto in linguaggio macchina, può anche essere fatto in linguaggio assemblativo. Al contrario, un programmatore in linguaggio ad alto livello non ha a disposizione tutte le istruzioni, i registri e le funzionalità della macchina. I linguaggi per la programmazione di sistemi, come C, sono spesso trasversali rispetto a questi due tipi, dato che la loro sintassi è ad alto livello, ma forniscono allo stesso tempo alcune delle possibilità di accesso alla macchina tipiche dei linguaggi assemblativi.

Un'ultima differenza che vale la pena rendere più esplicita è che i programmi in linguaggio assemblativo vengono scritti per una specifica famiglia di macchine, mentre un programma scritto in un linguaggio ad alto livello può essere eseguito su molte macchine diverse. La possibilità di portare il software da una macchina a un'altra rappresenta un grande vantaggio per molte applicazioni.

7.1.2 Perché usare il linguaggio assemblativo

Programmare in linguaggio assemblativo è difficile. Non ci si faccia illusioni. Non è un lavoro per programmatori pavidi e debolucci. Inoltre scrivere un programma in linguaggio assemblativo richiede molto più tempo che scriverlo in un linguaggio ad alto livello, e anche la correzione degli errori e l'aggiornamento del codice sono operazioni molto più complesse.

Ma allora, perché mai si dovrebbe programmare in linguaggio assemblativo? Esistono due motivi: le prestazioni e le possibilità di accesso alla macchina. Prima di tutto un esperto programmatore in linguaggio assemblativo, lavorando molto duramente, può in alcuni casi creare codice più piccolo e veloce di quanto possa fare un programmatore in linguaggio ad alto livello. Per alcune applicazioni la velocità e la dimensione rappresentano due fattori critici; in questa categoria ricadono per esempio il codice di una smart card o di una RFID card, i driver dei dispositivi, le librerie per la manipolazione di stringhe, le routine del BIOS, i cicli interni di applicazioni in tempo reale le cui prestazioni hanno un'importanza fondamentale.

In secondo luogo, alcune procedure devono accedere in modo completo all'hardware (cosa che in genere è impossibile con i linguaggi ad alto livello). In questa categoria ritroviamo per esempio i gestori a basso livello degli interrupt e delle eccezioni dei sistemi operativi e i controllori dei dispositivi di molti sistemi integrati che funzionano in tempo reale.

Un compilatore deve produrre un risultato utilizzabile da un assemblatore oppure eseguire esso stesso il processo di assemblaggio. Comprendere il linguaggio assemblativo è quindi essenziale per capire come lavorano i compilatori.

In secondo, lo studio del linguaggio assemblativo permette di avere una visione più chiara del funzionamento delle macchine reali. Per gli studenti di architettura degli elaboratori scrivere anche poche righe di codice assemblativo è l'unico modo per avere consapevolezza di che cosa sono realmente le macchine a livello di microarchitettura.

7.1.3 Formato delle istruzioni del linguaggio assemblativo

La struttura delle istruzioni di un linguaggio assemblativo rispecchia la struttura delle istruzioni macchina, delle quali le prime forniscono una rappresentazione simbolica; ciononostante i linguaggi assemblativi di macchine diverse e di livelli differenti hanno un numero sufficiente di somiglianze da permettere di parlare di linguaggio assemblativo in termini generali. La Figura 7.1 mostra alcuni frammenti del linguaggio assemblativo x86 che esegue l'istruzione $N = I + J$. Nell'esempio le istruzioni sopra la linea vuota eseguono la computazione e quelle al di sotto sono invece i comandi che indicano all'assemblatore di riservare spazio di memoria per le variabili *I*, *J* e *N* e non rappresentano alcuna istruzione macchina.

Etichetta	Codice operativo	Operandi	Commenti
FORMULA:	MOV ADD MOV	EAX,I EAX,J N,EAX	; registro EAX = I ; registro EAX = I + J ; N = I + J
I	DD	3	; riserva 4 byte inizializzati a 3
J	DD	4	; riserva 4 byte inizializzati a 4
N	DD	0	; riserva 4 byte inizializzati a 0

Figura 7.1 Esecuzione di $N = I + J$ su x86.

Esistono differenti assemblatori per la famiglia Intel (per esempio x86), ciascuno dei quali ha una propria sintassi. Nel corso di questo capitolo utilizzeremo per i nostri esempi il linguaggio assemblativo Microsoft MASM. Ci sono diversi assemblatori anche per ARM, ma la sintassi è simile e quella di x86 e per questa ragione ci limiteremo a un solo esempio.

Le istruzioni del linguaggio assemblativo sono composte da quattro parti: un campo etichetta, un campo per l'operazione (codice operativo o *opcode*), un campo per gli operandi e un campo per i commenti. Le etichette sono utilizzate per fornire nomi simbolici agli indirizzi, e sono necessarie per definire le destinazioni alle quali portano le istruzioni che effettuano salti e per poter accedere alle parole di dati memorizzate mediante nomi simbolici. Se un'istruzione è etichettata, l'etichetta inizia (di solito) nella colonna 1.

L'esempio della Figura 7.1 ha quattro etichette: *FORMULA*, *I*, *J* e *N*. Si noti che MASM richiede che l'etichetta termini con il carattere "due punti" quando è utilizzata per il codice, ma non quando è utilizzata per i dati. Non è un aspetto fondamentale, comunque, e altri assemblatori si comportano in maniera differente. Non c'è alcun aspetto dell'architettura sottostante che suggerisca una scelta piuttosto che un'altra. Un vantaggio offerto dalla notazione che usa i "due punti" è che un'etichetta può comparire, da sola, su una linea, mentre il codice operativo viene scritto nella prima colonna della linea successiva. Questo stile si rivela in un certo modo utile per i compilatori. Senza i "due punti" sarebbe impossibile distinguere un'etichetta da un codice operativo, nel caso in cui entrambi occupino, da soli, una linea del codice assemblativo. Usando i "due punti" si elimina questa potenziale ambiguità.

Una caratteristica infelice di alcuni assemblatori limita a sei o a otto il numero di caratteri di un'etichetta, mentre la maggior parte dei linguaggi ad alto livello permette di utilizzare nomi di lunghezza arbitraria. Nomi lunghi e scelti in modo appropriato rendono i programmi molto più leggibili e facili da capire da chi non ne è l'autore.

Ogni macchina è dotata di registri che hanno bisogno di un nome. I nomi assegnati a registri dell'x86 sono `EAX`, `EBX`, `ECX` e così via.

Il campo Codice operativo contiene un'abbreviazione simbolica del codice operativo oppure un comando per l'assemblatore stesso. La scelta di un nome appropriato è solo una questione di gusti e spesso i progettisti di differenti linguaggi assemblativi compiono scelte diverse. I progettisti dell'assemblatore MASM hanno deciso di usare `MOV` sia per caricare un registro dalla memoria sia per memorizzare un registro, ma avrebbero potuto scegliere di utilizzare `MOVE` oppure `LOAD` e `STORE`.

I programmi in linguaggio assemblativo hanno spesso bisogno di riservare spazio per le variabili. I progettisti del linguaggio MASM hanno scelto di usare `DD` (Define Double), perché nell'8088 la parola era di 16 bit.

Nelle istruzioni del linguaggio assemblativo il campo Operandi viene utilizzato per specificare gli indirizzi e i registri utilizzati come operandi dall'istruzione macchina. Nel caso di un'istruzione per la somma di interi il campo operandi indica quali elementi devono essere sommati fra loro. Nel caso di un'istruzione di salto indica invece a quale indirizzo deve portare la diramazione. Gli operandi possono essere registri, costanti, locazioni di memoria, e così via.

Il campo Commenti è il luogo in cui i programmatori possono inserire utili spiegazioni sul funzionamento del programma a beneficio di altri programmatori che in seguito lo potrebbero utilizzare o modificare (o a beneficio del programmatore stesso alcuni anni dopo aver scritto il codice). Un programma in linguaggio assemblativo senza una tale documentazione è praticamente incomprensibile, spesso anche per il suo stesso autore. Il campo dei commenti è destinato esclusivamente all'uso da parte dell'uomo e non ha alcun effetto sul processo di assemblaggio né tantomeno sul programma che verrà generato.

7.1.4 Pseudoistruzioni

Un linguaggio assemblativo, oltre a specificare quali istruzioni macchina devono essere eseguite, può anche contenere dei comandi indirizzati all'assemblatore stesso, per richiedere, per esempio, di allocare una certa quantità di memoria oppure per passare a una nuova pagina del listato. I comandi diretti all'assemblatore sono chiamati **pseudoistruzioni** o anche **direttive dell'assemblatore**. Nella Figura 7.1 abbiamo già visto una pseudoistruzione molto comune: DD. Nella Figura 7.2 ne sono elencate altre, tratte dall'assemblatore Microsoft MASM per gli x86.

Pseudoistruzione	Significato
SEGMENT	Inizia un nuovo segmento (testo, dati e così via) con certi attributi
ENDS	Termina il segmento corrente
ALIGN	Controlla l'allineamento della successiva istruzione o dei successivi dati
EQU	Definisce un nuovo simbolo uguale a una data espressione
DB	Alloca spazio per memorizzare uno o più byte (inizializzati)
DW	Alloca spazio per memorizzare uno o più elementi di dati (inizializzati) a 16 bit (parola)
DD	Alloca spazio per memorizzare uno o più elementi di dati (inizializzati) a 32 bit (parola doppia)
DQ	Alloca spazio per memorizzare uno o più elementi di dati (inizializzati) a 64 bit (parola quadrupla)
PROC	Inizia una procedura
ENDP	Termina una procedura
MACRO	Inizia una definizione di macro
ENDM	Termina una definizione di macro
PUBLIC	Esporta un nome definito nel modulo
EXTERN	Importa un nome da un altro modulo
INCLUDE	Preleva e include un altro file
IF	Inizia un assemblaggio condizionale in base a una data espressione
ELSE	Inizia un assemblaggio condizionale se la condizione IF precedente è falsa
ENDIF	Termina un assemblaggio condizionale
COMMENT	Definisce un nuovo carattere di inizio commento
PAGE	Genera un'interruzione di pagina nel listato
END	Termina il programma assemblativo

Figura 7.2 Alcune pseudoistruzioni dell'assemblatore MASM.

La pseudoistruzione SEGMENT inizia un nuovo segmento, mentre ENDS lo termina. È consentito iniziare un segmento di testo, contenente codice, poi un segmento dati e tornare successivamente al segmento testo, e così via.

ALIGN forza l'indirizzo della linea successiva, contenente in genere dati, a essere un multiplo dell'argomento della pseudoistruzione. Per esempio se il segmento contiene già 61 byte di dati, l'indirizzo allocato subito dopo la pseudoistruzione ALIGN 4 sarà 64.

EQU assegna un nome simbolico a un'espressione. Per esempio, dopo la pseudoistruzione

```
BASE EQU 1000
```

il simbolo BASE può essere usato in qualsiasi punto al posto di 1000. L'espressione che segue EQU può comprendere alcuni simboli definiti dal programmatore oltre a operatori aritmetici e di altro tipo, come mostra il seguente esempio

```
LIMIT EQU 4 * BASE + 2000
```

La maggior parte degli assemblatori, compreso MASM, richiede che un simbolo sia definito prima di essere utilizzato in un'espressione come quella appena vista.

Le successive quattro pseudoistruzioni, DB, DW, DD e DQ, allocano memoria rispettivamente per una o più variabili della dimensione di 1, 2, 4 oppure 8 byte. Per esempio,

```
TABLE DB 11, 23, 49
```

alloca spazio per 3 byte e li inizializza rispettivamente ai valori 11, 23 e 49. Essa definisce inoltre il simbolo *TABLE* e lo associa all'indirizzo in cui è memorizzato il valore 11.

Le pseudoistruzioni PROC e ENDP definiscono rispettivamente l'inizio e la fine delle procedure del linguaggio assemblativo, che hanno la stessa funzione delle procedure definite in altri linguaggi di programmazione. Analogamente le direttive MACRO e ENDM delimitano l'area in cui compare la definizione di una macroistruzione (argomento trattato più avanti nel corso del capitolo).

Le due successive pseudoistruzioni, PUBLIC e EXTERN, controllano la visibilità dei simboli. È una pratica comune scrivere i programmi in più file e spesso capita che una procedura contenuta in un file richiami una procedura o acceda ai dati definiti in un altro file. La pseudoistruzione PUBLIC permette di esportare un simbolo che deve essere reso disponibile anche ad altri file, rendendo così possibili i riferimenti tra file diversi. Analogamente, per impedire che l'assemblatore segnali un errore quando incontra un simbolo che non è ancora stato definito, si può dichiarare un simbolo come EXTERN. Questa pseudoistruzione indica all'assemblatore che la definizione del simbolo compare in un altro file. I simboli che non sono dichiarati mediante alcuna delle due pseudoistruzioni precedenti sono visibili soltanto localmente, all'interno dello stesso file. Ciò significa che l'utilizzo, per esempio, di *FOO* in più file non genererà alcun conflitto, dato che ciascuna definizione è locale al file in cui si trova.

La direttiva INCLUDE fa sì che l'assemblatore prelevi un altro file e lo includa interamente all'interno di quello corrente. Spesso tali file comprendono definizioni, macroistruzioni e altri elementi che sono richiesti in più file.

Molti assemblatori, compreso MASM, supportano l'assemblaggio condizionale. Per esempio,

```
WORDSIZE EQU 32
IF WORDSIZE GT 32
WSIZE:  DD 64
ELSE
WSIZE:  DD 32
ENDIF
```

alloca una singola parola a 32 bit e chiama *WSIZE* il suo indirizzo. La parola è inizializzata a 64 oppure a 32, in base al valore di *WORDSIZE* (in questo caso 32). In genere si utilizza questo metodo per allocare una variabile quando occorre scrivere un programma che possa essere eseguito da macchine a 32 bit o a 64 bit. Inserendo all'interno di `IF` e `ENDIF` tutte le porzioni del codice dipendenti dalla macchina, e cambiando successivamente una singola definizione, *WORDSIZE*, il programma può essere automaticamente assemblato per entrambe le dimensioni di parola. Grazie a questo approccio è possibile mantenere un solo programma sorgente per più macchine destinazione (di tipo diverso), rendendo di conseguenza più facile lo sviluppo e il mantenimento del software. In molti casi tutte le definizioni dipendenti dalla macchina, come *WORDSIZE*, sono raggruppate in un singolo file, di cui vengono create versioni diverse per ogni tipo di macchina. In questo modo, includendo semplicemente il corretto file delle definizioni, è possibile utilizzare facilmente il programma per macchine diverse.

La pseudoistruzione `COMMENT` permette all'utente di modificare il delimitatore dei commenti, che inizialmente è impostato al carattere "punto e virgola". `PAGE` è utilizzato per controllare il listato che l'assemblatore può generare su richiesta. Infine `END` segna la fine del programma. Oltre a quelle appena elencate, in MASM esistono molte altre pseudoistruzioni. Altri assemblatori per x86 hanno invece un insieme diverso di pseudoistruzioni; difatti queste non sono dettate dall'architettura della macchina, ma dai gusti di chi ha scritto l'assemblatore.

7.2 Macroistruzioni

Spesso i programmatori di linguaggi assemblativi hanno bisogno di ripetere più volte all'interno di un programma alcune sequenze d'istruzioni. Il modo più ovvio per farlo consiste nel riscriverle ogni volta che sia necessario. Tuttavia se una sequenza è lunga, o se deve essere utilizzata molte volte, scriverla ripetutamente è un'operazione tediosa.

Un approccio alternativo consiste nel trasformare la sequenza in una procedura e nel richiamarla quando è richiesta. Questa strategia ha lo svantaggio di dover eseguire, ogni volta che occorre utilizzare la sequenza, un'istruzione di chiamata di procedura e una per restituirne il controllo. Se le sequenze sono corte, per esempio di due istruzioni, ma sono usate frequentemente, il lavoro aggiuntivo determinato dalla chiamata alla procedura può rallentare notevolmente il programma. Le *macro*, abbreviazione usuale di macroistruzioni, forniscono una soluzione facile ed efficiente al problema di ripetere un'identica, o quasi, sequenza d'istruzioni.

7.2.1 Definizione, chiamata ed espansione di macro

Una **definizione di macro** è un modo per assegnare un nome a una porzione di testo. Dopo che una macro è stata definita, il programmatore può scriverne il nome al posto della porzione di programma corrispondente. Una macro in realtà non è altro che un'abbreviazione per una porzione di testo. La Figura 7.3(a) mostra un programma in linguaggio assemblativo per x86 che scambia due volte il contenuto delle variabili *p* e *q*. Queste sequenze possono essere definite come macro, come mostra la Figura 7.3(b). Dopo aver definito la macro, ogni occorrenza di *SWAP* viene sostituita dalle quattro linee:

```
MOV EAX,P
MOV EBX,Q
MOV Q,EAX
MOV P,EBX
```

Il programmatore ha definito *SWAP* come un'abbreviazione delle quattro istruzioni.

```
MOV  EAX,P
MOV  EBX,Q
MOV  Q,EAX
MOV  P,EBX

MOV  EAX,P
MOV  EBX,Q
MOV  Q,EAX
MOV  P,EBX
```

(a)

```
SWAP  MACRO
      MOV EAX,P
      MOV EBX,Q
      MOV Q,EAX
      MOV P,EBX
      ENDM

      SWAP

      SWAP
```

(b)

Figura 7.3 Codice in linguaggio assemblativo per scambiare due volte P e Q. Senza (a) e con (b) macro.

Anche se assemblatori diversi utilizzano notazioni leggermente differenti, in tutti i casi una definizione di macro è composta dalle seguenti parti principali:

1. un'intestazione che indica il nome della macro che si sta definendo;
2. il testo che costituisce il corpo della macro;
3. una pseudoistruzione che segna la fine della definizione (per esempio, ENDM).

Quando l'assemblatore incontra una definizione di macro, la salva nella tabella delle definizioni di macro per poterla utilizzare successivamente. Da quel momento in poi ogni volta che il nome della macro appare come codice operativo, l'assemblatore la sostituisce con il corpo della macro. L'uso del nome di una macro come codice operati-

vo viene detto **chiamata di macro**, mentre la sua sostituzione con il corpo della macro è chiamata **espansione di macro**.

L'espansione della macro viene effettuata durante il processo assemblativo e non durante l'esecuzione del programma. Questo è un punto importante. Il programma della Figura 7.3(a) e quello della Figura 7.3(b) produrranno esattamente lo stesso codice in linguaggio macchina. Guardando solamente il programma tradotto in linguaggio macchina, è impossibile stabilire se, durante la generazione, siano state utilizzate o meno delle macro. Il motivo è che, una volta completata l'espansione delle macro, l'assemblatore elimina le loro definizioni, di cui non rimane alcuna traccia nel programma generato.

Le chiamate di macro non devono essere confuse con le chiamate di procedure. La differenza principale è che una chiamata di macro è un'istruzione diretta all'assemblatore, affinché sostituisca il nome della macro con il suo corpo. Una chiamata di procedura è invece un'istruzione macchina inserita nel programma oggetto e che sarà eseguita in un secondo momento per chiamare la procedura. La Figura 7.4 confronta le chiamate di macro con le chiamate di procedura.

Domanda	Macro	Procedura
Quando viene effettuata la chiamata?	Durante l'assemblaggio	Durante l'esecuzionne
Il corpo è inserito nel programma oggetto in ogni punto in cui avviene una chiamata?	Sì	No
L'istruzione per la chiamata di procedura è inserita nel programma oggetto ed eseguita successivamente?	No	Sì
Occorre usare un'istruzione di ritorno dopo aver effettuato la chiamata?	No	Sì
Quante copie del corpo appaiono nel programma oggetto?	Una per chiamata di macro	Una

Figura 7.4 Confronto tra le chiamate di macro e le chiamate di procedure

Concettualmente conviene pensare che il processo assemblativo avvenga in due fasi. Nella prima vengono salvate tutte le definizioni delle macro e vengono espanse le loro chiamate. Nella seconda il testo risultante viene elaborato come se fosse il programma originale. Vedendolo in questo modo, il programma sorgente viene letto e poi trasformato in un altro programma nel quale sono state rimosse tutte le definizioni delle macro e le chiamate sono state sostituite dai loro corpi. L'output risultante è un programma in linguaggio assemblativo che non contiene alcuna macro e che può essere passato all'assemblatore.

È importante tenere a mente che un programma è semplicemente una stringa di caratteri che comprende lettere, cifre, spazi, segni di punteggiatura e "ritorni a capo" (passaggio a una nuova linea). L'espansione di una macro consiste nel sostituire una parte di questa stringa con una nuova stringa di caratteri. L'uso delle macro è una tecnica per manipolare stringhe di caratteri, senza preoccuparsi del loro significato.

7.2.2 Macro con parametri

Le macro descritte precedentemente sono utilizzabili in sorgenti di breve lunghezza in cui si ripete più volte una sequenza d'istruzioni perfettamente identica. Tuttavia è frequente il caso in cui un programma contiene varie sequenze d'istruzioni che non sono del tutto identiche. Nell'esempio della Figura 7.5(a) la prima sequenza scambia *P* con *Q*, mentre la seconda *R* con *S*.

```
MOV   EAX,P
MOV   EBX,Q
MOV   Q,EAX
MOV   P,EBX

MOV   EAX,R
MOV   EBX,S
MOV   S,EAX
MOV   R,EBX
```

(a)

```
CHANGE  MACRO P1, P2
        MOV EAX,P1
        MOV EBX,P2
        MOV P2,EAX
        MOV P1,EBX
        ENDM

        CHANGE P, Q

        CHANGE R, S
```

(b)

Figura 7.5 Sequenze d'istruzioni quasi identiche. Senza (a) e con (b) macro.

Gli assemblatori di macro gestiscono queste situazioni consentendo che le definizioni delle macro specifichino dei **parametri formali** e che le chiamate forniscano dei **parametri attuali**. Quando una macro viene espansa i parametri formali che appaiono nel suo corpo vengono sostituiti dai corrispondenti parametri attuali. I parametri attuali sono specificati nel campo operandi della chiamata di macro. La Figura 7.5(b) mostra il programma della Figura 7.5(a) riscritto utilizzando una macro con due parametri. *P1* e *P2* sono i parametri formali. Nel momento in cui la macro viene espansa, ogni occorrenza di *P1* nel corpo della macro viene sostituita dal secondo parametro attuale. Nella chiamata di macro

```
CHANGE P, Q
```

P è il primo parametro attuale e *Q* è il secondo. I programmi prodotti dalle due parti della Figura 7.5 sono quindi identici: contengono esattamente le stesse istruzioni con gli stessi operandi.

7.2.3 Caratteristiche avanzate

La maggior parte dei processori di macro mette a disposizione un gran numero di caratteristiche avanzate per rendere più facile la vita dei programmatori in linguaggio assemblativo. In questo paragrafo daremo un'occhiata ad alcune caratteristiche avanzate di

MASM. Un problema che sorge usando gli assemblatori che supportano le macro è la duplicazione delle etichette. Supponiamo che una macro contenga un'istruzione di diramazione condizionale e un'etichetta che rappresenta la destinazione del salto. Se la macro viene chiamata due o più volte, l'etichetta verrà duplicata causando un errore del linguaggio assemblativo. Una soluzione consiste nell'obbligare il programmatore a fornire tramite parametro una diversa etichetta ogni volta che chiama la macro. La soluzione utilizzata da MASM è invece quella di permettere la dichiarazione di etichetta LOCAL, e lasciare che l'assemblatore generi un'etichetta diversa per ogni espansione della macro. Altri assemblatori utilizzano la regola secondo la quale le etichette numeriche sono automaticamente definite come locali.

MASM e la maggior parte degli assemblatori permettono di definire macro all'interno di altre macro. Questa funzionalità risulta particolarmente utile se usata insieme all'assemblaggio condizionale. In genere la stessa macro viene definita in entrambi i rami di un costrutto IF, come il seguente:

```
M1 MACRO
   IF WORDSIZE GT 16
M2      MACRO
        ...
        ENDM
   ELSE
M2      MACRO
   ...
        ENDM
   ENDIF
        ENDM
```

In entrambi i casi la macro *M2* viene definita, ma la sua definizione dipende dal fatto che il programma venga assemblato su una macchina a 16 bit oppure su una a 32 bit. Se *M1* non viene chiamata, *M2* non viene definita.

Infine, le macro possono chiamare altre macro, comprese loro stesse. Se una macro è ricorsiva, cioè se richiama se stessa, deve passare un parametro modificato a ogni espansione e che fa terminare la ricorsione quando raggiunge un determinato valore. Se così non fosse, l'assemblatore potrebbe entrare in un ciclo infinito, e in tal caso l'utente dovrebbe interrompere esplicitamente l'assemblatore.

7.2.4 Implementazione delle funzionalità macro negli assemblatori

Per implementare una funzionalità macro un assemblatore deve essere in grado di eseguire due funzioni: salvare le definizioni delle macro ed espandere le loro chiamate. Esaminiamo una alla volta queste due operazioni.

L'assemblatore deve mantenere una tabella con tutti i nomi delle macro e, associato a ogni nome, un puntatore alla definizione precedentemente memorizzata, in modo che sia possibile recuperarla quando è necessario. Alcuni assemblatori hanno una tabella separata per i nomi delle macro, mentre altri sono dotati di un'unica tabella in cui sono mantenute, oltre alle macro, anche tutte le istruzioni macchina e le pseudoistruzioni.

Quando s'incontra una definizione di macro si aggiunge nella tabella un elemento che indica il nome della macro, il numero di parametri formali e un puntatore a un'altra tabella, la tabella delle definizioni delle macro, che conterrà il suo corpo. In questa fase si costruisce anche una lista dei parametri formali da utilizzare nell'elaborazione della definizione. Viene quindi letto il corpo della macro e memorizzato all'interno della tabella delle definizioni. I parametri formali che s'incontrano all'interno del corpo sono indicati mediante simboli speciali. Come esempio mostriamo la rappresentazione interna della definizione della macro *CHANGE*, utilizzando il punto e virgola come "ritorno a capo" e la "e commerciale" (&) come simbolo dei parametri formali:

```
MOV EAX,&P1; MOV EBX,&P2; MOV &P2,EAX; MOV &P1,EBX;
```

Nella tabella delle definizioni delle macro il corpo di ciascuna macro è semplicemente una stringa di caratteri.

Durante il primo passo del processo di assemblaggio, vengono cercati i codici operativi e le macro vengono espanse. Ogni volta che s'incontra una definizione di macro la si memorizza nella tabella delle macro. Quando viene chiamata una macro l'assemblatore interrompe temporaneamente la lettura dal dispositivo di input e inizia a leggere il corpo della macro precedentemente memorizzato. I parametri formali presenti al suo interno vengono poi sostituiti dai parametri attuali specificati dalla chiamata. La presenza del simbolo & prima dei parametri formali permette all'assemblatore di riconoscerli più facilmente.

7.3 Processo di assemblaggio

Nei paragrafi successivi descriveremo brevemente come funziona un assemblatore. Anche se ogni macchina ha il proprio linguaggio assemblativo, i diversi processi di assemblaggio sono tra loro sufficientemente simili da poterci permettere una descrizione generale.

7.3.1 Assemblatori a due passate

Dato che ogni programma in linguaggio assemblativo consiste in una serie d'istruzioni scritte su un'unica riga, di primo acchito potrebbe sembrare naturale avere un assemblatore che legge un'istruzione, la traduce in linguaggio macchina e scrive su file il codice generato, oltre a produrre, se richiesto, un altro file contenente il listato. Questo procedimento andrebbe ripetuto fino alla traduzione dell'intero programma. Purtroppo però questa strategia non funziona.

Consideriamo il caso in cui la prima istruzione sia una diramazione verso *L*. L'assemblatore non può assemblare questa istruzione finché non venga a conoscenza dell'indirizzo dell'istruzione *L*. Dato che questa istruzione potrebbe trovarsi verso la fine del programma l'assemblatore è obbligato a leggere praticamente tutto il programma per trovare il suo indirizzo. Il fatto di utilizzare un simbolo, *L*, prima ancora che sia stato definito, costituisce il **problema dei riferimenti in avanti**; in altre parole si effettua un riferimento a un simbolo la cui definizione ha luogo in un punto successivo del programma.

È possibile gestire i riferimenti in avanti in due modi. Primo, l'assemblatore può leggere il programma sorgente due volte. Ciascuna lettura del programma sorgente è chiamata **passata** e i traduttori che leggono due volte il programma di input sono detti **a due passate**. Durante la prima lettura questi assemblatori raccolgono in una tabella tutte le definizioni dei simboli, comprese le etichette delle istruzioni. Prima di iniziare la seconda passata si conoscono quindi i valori di tutti i simboli. In tal modo non rimane alcun riferimento in avanti ed è possibile leggere ogni istruzione, assemblarla e generare il codice corrispondente. Questo approccio richiede una passata aggiuntiva per elaborare il programma di input, ma è concettualmente semplice.

Un secondo approccio consiste nel leggere il file in ingresso una sola volta, convertirlo in un formato intermedio e memorizzare in una tabella della memoria questa forma intermedia. In seguito viene effettuata una seconda passata sulla tabella, invece che sul codice sorgente come avveniva nel primo approccio. Se è disponibile una sufficiente quantità di memoria (fisica o virtuale) questo approccio risparmia il tempo dovuto alle operazioni di I/O. Se non viene richiesta la generazione del listato, è possibile ridurre al minimo indispensabile questa forma intermedia.

In entrambi gli approcci, la prima passata ha anche il compito di salvare tutte le definizioni di macro e di espanderle quando si incontrano le loro chiamate. Generalmente quindi la definizione dei simboli e l'espansione delle macro avvengono in un'unica passata.

7.3.2 Prima passata

La principale funzione della prima passata è quella di costruire la cosiddetta **tabella dei simboli**, contenente i valori di tutti i simboli. Un simbolo è un'etichetta, oppure un valore, al quale è stato assegnato un nome simbolico attraverso una pseudoistruzione, come

```
BUFSIZE EQU 8192
```

Per assegnare nel campo etichetta di un'istruzione un valore a un simbolo, l'assemblatore deve conoscere quale indirizzo avrà tale istruzione durante l'esecuzione del programma. Durante il processo di assemblaggio l'assemblatore mantiene una variabile, chiamata **ILC** (*Instruction Location Counter*, "contatore di locazioni delle istruzioni"), per tener traccia dell'indirizzo che l'istruzione che sta assemblando avrà a tempo di esecuzione. All'inizio della prima passata la variabile ha valore 0 e, come mostra la Figura 7.6, ogni volta che viene elaborata un'istruzione la variabile viene incrementata della sua lunghezza. L'esempio mostrato nella figura è relativo a x86.

Nella maggior parte degli assemblatori la prima passata utilizza tre tabelle interne per i simboli, le pseudoistruzioni e i codici operativi. Se necessario viene mantenuta anche la tabella dei letterali (si veda in seguito la definizione). Come mostra la Figura 7.7, la tabella dei simboli ha una linea per ciascun identificatore (simbolo). I simboli vengono definiti quando sono utilizzati come etichette oppure mediante una definizione esplicita (per esempio, EQU). Ogni elemento della tabella dei simboli contiene il simbolo stesso (o un puntatore a esso), il suo valore numerico e, in alcuni casi, altre informazioni.

Etichetta	Codice operativo	Operandi	Commenti	Lunghezza	ILC
MARIA:	MOV	EAX, I	EAX = I	5	100
	MOV	EBX, J	EBX = J	6	105
ROBERTA:	MOV	ECX, K	ECX = K	6	111
	IMUL	EAX, EAX	EAX = I x I	2	117
	IMUL	EBX, EBX	EBX = J x J	3	119
	IMUL	ECX, ECX	ECX = K x K	3	122
MARIANNA:	ADD	EAX, EBX	EAX = I x I + J x J	2	125
	ADD	EAX, ECX	EAX = I x I + J x J + K x K	2	127
STEFANIA:	JMP	DONE	salto a DONE	5	129

Figura 7.6 Il contatore di locazioni delle istruzioni (ILC) tiene traccia degli indirizzi delle locazioni di memoria in cui le istruzioni saranno caricate. In questo esempio le istruzioni prima di MARIA occupano 100 byte.

Identificatore	Valore	Altre informazioni
MARIA	100	
ROBERTA	111	
MARIANNA	125	
STEFANIA	129	

Figura 7.7 Tabella dei simboli per il programma della Figura 7.6.

Queste informazioni aggiuntive possono comprendere:

1. la lunghezza del campo dati associato al simbolo;
2. i bit di rilocazione (il simbolo cambia valore se il programma è caricato in un indirizzo diverso rispetto a quello che ha assunto l'assemblatore?);
3. se il simbolo sia o meno accessibile dall'esterno della procedura.

La tabella dei codici operativi (la Figura 7.8 ne mostra una parte) contiene un elemento per ciascun codice simbolico (mnemonico) del linguaggio. Ciascun elemento contiene il codice simbolico, due operandi, il valore numerico del codice operativo, la lunghezza dell'istruzione e un numero di tipo (che permette di raggruppare i codici operativi a seconda del numero e tipo di operandi).

Se per esempio un'istruzione ADD contiene come primo operando EAX e come secondo una costante a 32 bit (immed32), si utilizza il codice operativo 0x05 e la lunghezza dell'istruzione è di 5 byte. In realtà le costanti che possono essere espresse con 8 o 16 bit utilizzano codici operativi diversi, ma non sono mostrati nella figura. Se ADD viene utilizzata con due registri come operandi, allora la sua lunghezza risulta di 2 byte e il suo codice operativo è 0x01. La classe dell'istruzione 19 (il valore è arbitrario) dovrebbe essere assegnata a tutte le combinazioni codice operativo-operando che seguono le stesse regole. Le istruzioni facenti parte di questa classe dovrebbero quindi essere ela-

borate allo stesso modo dell'istruzione ADD che usa due registri come operandi. La classe dell'istruzione designa in realtà una procedura dell'assemblatore, richiamata per elaborare tutte le istruzioni di un certo tipo.

Codice operativo	Primo operando	Secondo operando	Codice esadecimale	Lunghezza dell'istruzione	Classe dell'istruzione
AAA	—	—	37	1	6
ADD	EAX	immed32	05	5	4
ADD	reg	reg	01	2	19
AND	EAX	immed32	25	5	4
AND	reg	reg	21	2	19

Figura 7.8 Parti della tabella dei codici operativi per un assemblatore x86.

Alcuni assemblatori permettono ai programmatori di scrivere istruzioni che usano l'indirizzamento immediato anche se nel linguaggio destinazione non esiste alcuna istruzione corrispondente. Per gestire queste istruzioni, chiamiamole *pseudoimmediate*, l'assemblatore alloca alla fine del programma la memoria per l'operando immediato e genera un'istruzione che fa riferimento a esso. Per esempio il mainframe IBM 360 e i suoi successori non hanno istruzioni immediate, ma i programmatori possono comunque scrivere

```
L 14,=F'5'
```

per caricare il registro 14 con una parola costante il cui valore è 5. In questo modo il programmatore non è obbligato a scrivere esplicitamente una pseudoistruzione per allocare una parola inizializzata a 5, indicando la sua etichetta e poi utilizzandola nell'istruzione L. Le costanti per le quali l'assemblatore riserva automaticamente la memoria sono chiamate **letterali**. I letterali, oltre a risparmiare un po' di scrittura di codice al programmatore, migliorano anche la leggibilità del programma rendendo visibile il valore delle costanti all'interno delle istruzioni. La prima passata dell'assemblatore dovrebbe costruire una tabella di tutti i letterali utilizzati nel programma. Le nostre tre macchine di esempio sono dotate d'istruzioni immediate e quindi i loro assemblatori non utilizzano i letterali. Oggigiorno è abbastanza comune che un assemblatore sia dotato d'istruzioni letterali, ma una volta non era così. È probabile che l'uso molto diffuso dei letterali abbia reso evidente agli occhi dei progettisti delle macchine che l'indirizzamento immediato sia una buona idea. Se però è necessario utilizzare i letterali, allora durante l'assemblaggio occorre mantenere una tabella in cui viene aggiunto un elemento ogni volta che si incontra un letterale. Dopo la prima passata questa tabella viene ordinata e gli elementi duplicati vengono rimossi.

La Figura 7.9 mostra una procedura che potrebbe servire come base per la prima passata di un assemblatore. Lo stile di programmazione è autoesplicativo: i nomi delle procedure sono stati scelti per fornire una buona indicazione della loro funzione. L'aspetto più importante è che la Figura 7.9 rappresenta abbastanza bene la struttura della

prima passata. La procedura è sintetica, può essere compresa facilmente, e rende evidente quale deve essere il passo successivo, ovvero la scrittura del corpo delle procedure richiamate al suo interno.

```
public static void pass_one( ) {
  // Questa procedura è uno schema della prima passata di un semplice assemblatore.
  boolean more_input = true;                    // indicatore che interrompe la prima passata
  String line, symbol, literal, opcode;         // campi dell'istruzione
  int location_counter, length, value, type;    // variabili generiche
  final int END_STATEMENT = -2;                 // la fine dell'input

  location_counter = 0;                         // assembla la prima istruzione in 0
  initialize_tables( );                         // inizializzazione generale

  while(more_input) {                           // more_input viene impostato a falso da END
    line = read_next_line( );                   // preleva una linea di input
    length = 0;                                 // # byte nell'istruzione
    type = 0;                                   // di quale tipo (formato) è l'istruzione

    if (line_is_not_comment(line)) {
      symbol = check_for_symbol(line);          // alla linea è associata un'etichetta?
      if (symbol != null)                       // se sì, si registra il simbolo e il valore
        enter_new_symbol(symbol, location_counter);
      literal = check_for_literal(line);        // la linea contiene un letterale?
      if (literal != null)                      // se sì, lo si inserisce nella tabella
        enter_new_literal(literal);

      // Si determina ora il tipo del codice operativo. _1 indica un codice operativo inesistente
      opcode = extract_opcode(line);            // localizza il nome mnemonico del codice operativo
      type = search_opcode_table(opcode);       // trova il formato, p.es. OP REG1,REG2
      if (type < 0)                             // se non è un codice operativo, è una pseudoistruzione?
        type = search_pseudo_table(opcode);
      switch(type) {                            // determina la lunghezza di questa istruzione
        case 1: length = get_length_of_type1(line); break;
        case 2: length = get_length_of_type2(line); break;
        // altri casi
      }
    }
    write_temp_file(type, opcode, length, line);  // informazioni utili per la seconda passata
    location_counter = location_counter + length; // aggiorna loc_ctr
    if (type == END_STATEMENT) {                  // l'input è terminato?
      more_input = false;                         // se sì, si eseguono delle operazioni finali
      rewind_temp_for_pass_two( );                // come riportare al punto di inizio il file temporaneo
      sort_literal_table( );                      // e ordinare la tabella dei letterali
      remove_redundant_literals( );               // e rimuovere da questa i duplicati
    }
  }
```

Figura 7.9 Prima passata di un semplice assemblatore.

Alcune di queste procedure saranno relativamente corte, come *check_for_symbol* che restituisce una stringa di caratteri rappresentante un simbolo, se è presente, oppure null

in caso contrario. Altre procedure, come *get_length_of_type1* e *get_length_of_type2* potrebbero invece essere più lunghe e richiamare altre procedure. In generale il numero di tipi non sarà ovviamente limitato a due, ma dipenderà dal linguaggio che si sta assemblando e da quanti tipi d'istruzioni è composto.

Oltre a facilitarne la stesura, strutturare i programmi in questo modo presenta altri vantaggi. Se più persone lavorano alla scrittura dell'assemblatore le varie procedure possono essere distribuite fra tutti i programmatori. Inoltre tutti i dettagli (più difficoltosi) riguardanti la lettura dell'input rimangono nascosti nella procedura *read_next_line*. Se dovessero cambiare, per esempio a causa di una modifica del sistema operativo, sarebbe necessario modificare solamente una delle procedure ausiliarie, mentre non occorrerebbe effettuare alcuna modifica alla procedura *pass_one*.

Durante la prima passata viene analizzata ogni linea per trovare il codice operativo, determinare il suo tipo e calcolare la lunghezza dell'istruzione. Queste informazioni sono necessarie anche per la seconda passata e quindi può essere utile trascriverle esplicitamente in modo da evitare, nella passata successiva, di analizzare nuovamente la linea da zero. Tuttavia, riscrivere il file di input genera un maggior numero di operazioni di I/O. La scelta tra effettuare più operazioni di I/O per eliminare una seconda analisi delle istruzioni oppure effettuare meno operazioni di I/O, ma più analisi delle linee di codice, è condizionata da vari fattori, tra cui la velocità della CPU rispetto a quella del disco e l'efficienza del file system. In questo esempio scriveremo un file temporaneo contenente il tipo, il codice operativo, la lunghezza e l'effettiva linea di input. La seconda passata legge questa linea al posto di quella presente nel file di input.

La prima passata termina quando viene letta la pseudoistruzione END. Se necessario, a questo punto possono essere ordinate le tabelle dei simboli e dei letterali. Quest'ultimi, dopo essere stati ordinati, possono essere controllati al fine di individuare, ed eventualmente rimuovere, gli elementi doppi.

7.3.3 Seconda passata

La funzione della seconda passata è la generazione del programma oggetto e l'eventuale stampa del listato. Oltre a ciò la seconda passata deve generare delle informazioni richieste dal linker per collegare in un unico file eseguibile le procedure assemblate in momenti distinti. La Figura 7.10 mostra una bozza della procedura che implementa la seconda passata.

Le operazioni compiute dalla seconda passata sono più o meno simili a quelle della prima: le linee vengono lette ed elaborate una alla volta. Dato che all'inizio di ogni linea abbiamo scritto (sul file temporaneo) il tipo, il codice operativo e la lunghezza, queste informazioni vengono lette in modo da risparmiare parte della fase di analisi dell'input. Il lavoro principale della generazione del codice è realizzato dalle procedure *eval_type1*, *eval_type2*, e così via. Ciascuna di queste funzioni gestisce un particolare schema, per esempio un codice operativo e due registri come operandi, genera il codice binario corrispondente all'istruzione e infine lo restituisce all'interno della variabile *code*. Questo codice viene poi scritto su file. Con ogni probabilità la funzione *write_output* accumula in memoria il codice binario generato e lo scrive su file in grosse porzioni per ridurre il traffico dati verso il disco.


```
public static void pass_two( ) {
  // Questa procedura è uno schema della seconda passata di un semplice assemblatore.
  booleanmore_input = true;                  // indicatore che interrompe la seconda passata
  String line, opcode;                       // campi dell'istruzione
  int location_counter, length, type;        // variabili varie
  final int END_STATEMENT  = -2;             // segnala la fine dell'input
  final int MAX_CODE = 16;                   // numero massimo di byte di codice per istruzione
  byte code [ ] = new byte[MAX_CODE ];       // contiene il codice generato da un'istruzione

  location_counter = 0;                      // assembla la prima istruzione in 0

  while (more_input) {                       // more_input viene impostato a falso da END
    type = read_type( );                     // preleva il campo tipo della linea successiva
    opcode = read_opcode( );                 // preleva il campo codice operativo della linea successiva
    length = read_length( );                 // preleva il campo della lunghezza della linea successiva
    line = read_line( );                     // preleva la linea attuale dell'input

    if (type! = 0) {                         // il tipo 0 è per le linee di commento
      switch(type) {                         // genera il codice di output
        case 1: eval_type1(opcode, length, line, code); break;
        case 2: eval_type2(opcode, length, line, code); break;
      // altri casi
      }
    }
    write_output(code);                      // scrive il codice binario
    write_listing(code, line);               // stampa una linea del listato
    location_counter = location_counter + length;   // aggiorna loc_ctr
    if (type == END_STATEMENT) {             // l'input è terminato?
      more_input = false;                    // se sì, si eseguono alcune operazioni finali
      finish_up( );                          // varie e conclusione
    }
  }
}
```

Figura 7.10 Seconda passata di un semplice assemblatore.

L'istruzione sorgente originale e il codice oggetto generato da questa (in formato esadecimale) possono quindi essere stampati oppure memorizzati in un buffer per rimandarne la stampa. Dopo aver modificato ILC la seconda passata preleva l'istruzione successiva.

Finora si è assunto che il programma sorgente non contenga alcun errore. Chiunque abbia mai scritto un programma, in qualsiasi linguaggio, sa bene quanto sia realistica una simile affermazione. Alcuni fra gli errori più comuni sono:

1. un simbolo è stato utilizzato senza essere stato definito;
2. un simbolo è stato definito più di una volta;
3. il nome nel campo del codice operativo non è un codice operativo lecito;
4. un codice operativo non è seguito da un numero sufficiente di operandi;
5. un codice operativo è seguito da un numero eccessivo di operandi;
6. un numero contiene un carattere non valido, per esempio 143G6;

7. un registro è stato utilizzato in modo scorretto (per esempio, una diramazione verso un registro);
8. manca la pseudoistruzione END.

I programmatori sono piuttosto ingegnosi nel trovare sempre nuovi tipi di errori da commettere. Quelli dovuti ai simboli non definiti spesso sono originati da errori di battitura e quindi un assemblatore astuto potrebbe provare a sostituire il simbolo non definito con quello che, fra quelli che sono stati correttamente definiti, più gli assomiglia. Per la maggior parte degli altri errori però c'è ben poco da fare. Quando un assemblatore incontra un'istruzione errata si limita a stampare un messaggio di errore e cerca di continuare l'assemblaggio.

7.3.4 Tabella dei simboli

Durante la prima passata l'assemblatore accumula informazioni sui simboli e i loro valori. Tali informazioni devono essere memorizzate nella tabella dei simboli per essere utilizzate durante la seconda passata. Ora prenderemo brevemente in esame alcuni tra i vari modi di organizzare la tabella dei simboli. In tutti i casi si cerca di simulare una **memoria associativa**, che concettualmente non è altro che un insieme di coppie (simbolo, valore). Dato un simbolo, la memoria associativa deve fornire il valore corrispondente.

La tecnica più semplice consiste nell'implementare la tabella dei simboli come un vettore di coppie, in cui il primo elemento è il simbolo (oppure un puntatore a esso) e il secondo è il valore (oppure un puntatore a esso). Quando si vuole recuperare un simbolo, la routine della tabella dei simboli effettua semplicemente una ricerca lineare all'interno dell'array finché non trova l'elemento desiderato. Questo metodo è facile da programmare, ma allo stesso tempo è lento, dato che per ciascuna ricerca occorre esaminare, in media, metà della tabella.

Un altro modo per organizzare la tabella dei simboli è quello di ordinarla rispetto ai simboli e di utilizzare un algoritmo di **ricerca dicotomica** per cercare il simbolo desiderato. Questo algoritmo funziona confrontando il simbolo con l'elemento centrale della tabella. Se il simbolo precede alfabeticamente l'elemento centrale della tabella, occorrerà continuare la ricerca nella prima metà della tabella. Se invece il simbolo segue l'elemento centrale, occorrerà cercarlo nella seconda metà della tabella. Se l'elemento centrale della tabella è uguale al simbolo cercato, la procedura termina.

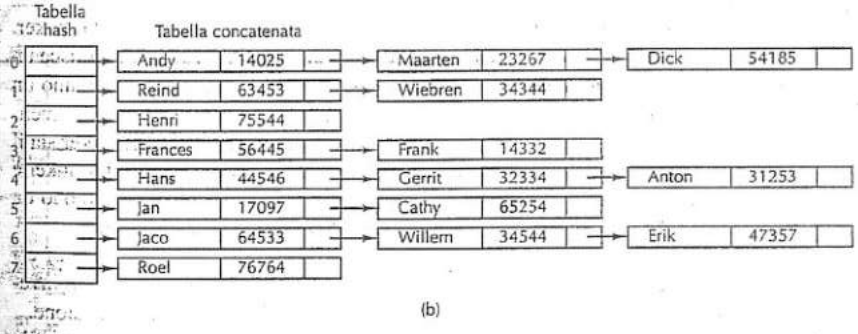
Assumendo che l'elemento centrale non sia uguale al simbolo cercato, possiamo comunque sapere in quale metà della tabella individuarlo. Si può quindi applicare nuovamente la ricerca dicotomica nella metà corretta della tabella; ciò permetterà di trovare il simbolo oppure di conoscere in quale quarto della tabella si trovi. Applicando ricorsivamente l'algoritmo è possibile completare una ricerca all'interno di una tabella di n elementi in circa $\log_2 n$ tentativi. Questa tecnica è ovviamente molto più veloce rispetto alla ricerca lineare, ma richiede che sia conservato l'ordine degli elementi della tabella.

Un modo completamente differente per simulare una memoria associativa è la tecnica conosciuta con il nome di **codifica hash** o **hashing**. In questo approccio occorre definire una funzione "hash" che mappa i simboli nell'intervallo di interi compreso tra 0 e $k - 1$. Una possibile funzione consiste nel moltiplicare tra loro i codici ASCII dei

caratteri del simbolo, ignorando un eventuale overflow, e considerando come risultato il resto della divisione per k di questo valore oppure il risultato della divisione rispetto a un numero primo. In realtà può andar bene qualsiasi funzione dell'input che fornisca una distribuzione uniforme dei valori hash. I simboli possono essere memorizzati in una tabella composta da k **secchielli** (*bucket*) numerati da 0 a $k - 1$. Ogni posizione i della tabella hash punta a una lista concatenata in cui sono memorizzate le coppie (simbolo, valore) il cui valore hash del simbolo vale i. Con n simboli e una tabella hash con k posizioni, una lista avrà una lunghezza media pari a n/k. Se si sceglie k approssimativamente uguale a n, allora in media è possibile localizzare i simboli con un unico passo di ricerca. Modificando k è possibile ridurre la dimensione della tabella a spese però della velocità dell'operazione di ricerca. La codifica hash è illustrata nella Figura 7.11.

Andy	14025	0
Anton	31253	4
Cathy	65254	5
Dick	54185	0
Erik	47357	6
Frances	56445	3
Frank	14332	3
Gerrit	32334	4
Hans	44546	4
Henri	75544	2
Jan	17097	5
Jaco	64533	6
Maarten	23267	0
Reind	63453	1
Roel	76764	7
Willem	34544	6
Wiebren	34344	1

(a)



(b)

Figura 7.11 Codifica hash. (a) Simboli, valori e codici hash derivati dai simboli. (b) Tabella hash a otto elementi con liste concatenate di simboli e valori.

7.4 Collegamento e caricamento

La maggior parte dei programmi è composta da più di una procedura. Generalmente i compilatori e gli assemblatori traducono una procedura alla volta e memorizzano su disco il risultato della traduzione. Prima che il programma possa essere eseguito è necessario recuperare tutte le procedure e collegarle fra loro in modo appropriato. Inoltre, in assenza di memoria virtuale, occorre caricare in memoria centrale il programma ottenuto dal collegamento delle procedure. I programmi che eseguono questi passi sono chiamati in vari modi, tra cui ***linker***, ***linking loader*** e ***linkage editor***. La traduzione completa di un programma sorgente richiede due passi distinti, com'è mostrato nella Figura 7.12:

1. compilazione o assemblaggio dei file sorgente;
2. collegamento dei moduli oggetto.

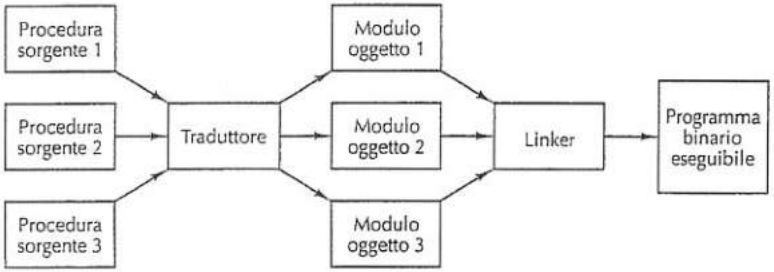


Figura 7.12 Generazione tramite un linker di un programma eseguibile binario a partire da un gruppo di file sorgente tradotti indipendentemente.

Il primo passo viene eseguito dal compilatore o dall'assemblatore, e il secondo dal linker. La traduzione che trasforma una procedura sorgente in un modulo oggetto costituisce un cambiamento di livello, dato che il linguaggio sorgente e quello destinazione hanno un diverso insieme d'istruzioni e usano una notazione differente. Il processo di collegamento invece non rappresenta un cambiamento di livello, dato che sia l'input del linker sia il suo output sono due programmi per la stessa macchina virtuale. La funzione del linker è quella di unire le procedure tradotte separatamente e di collegarle tra loro in modo da poterle eseguire come un'unica unità chiamata **programma eseguibile binario**.

Nei sistemi Windows i moduli oggetto sono caratterizzati dall'estensione *.obj* e i programmi eseguibili binari dall'estensione *.exe*. In UNIX i moduli oggetto hanno invece l'estensione *.o*, mentre i programmi eseguibili binari non hanno alcuna estensione.

Esiste un buon motivo per cui i compilatori e gli assemblatori traducono ciascun file sorgente come un'entità separata. Se un compilatore, o un assemblatore, dovesse leggere una serie di procedure sorgente e produrre direttamente un programma in linguaggio macchina pronto per essere eseguito, allora la modifica anche solo di un'istruzione in una delle procedure sorgente richiederebbe una nuova traduzione di tutte le altre procedure.

Se si usa la tecnica dei moduli oggetto separati (mostrata nella Figura 7.12) è necessario ritradurre soltanto la procedura modificata e non quelle rimaste invariate, anche se occorre ricollegare nuovamente tra loro i moduli oggetto. Di solito la fase di collegamento è molto più veloce della traduzione e permette quindi di risparmiare una grande quantità di tempo durante lo sviluppo di un programma. Questo guadagno risulta particolarmente importante quando si hanno centinaia o migliaia di moduli.

7.4.1 Compiti del linker

Nel processo assemblativo il contatore di locazioni delle istruzioni viene impostato a 0 all'inizio della prima passata. Questo passo corrisponde ad assumere che il modulo oggetto sarà collocato, durante l'esecuzione, all'indirizzo (virtuale) 0. La Figura 7.13 mostra quattro moduli oggetto per una macchina generica. In questo esempio ciascun modulo inizia con un'istruzione `BRANCH` che porta, all'interno del modulo, all'istruzione `MOVE`.

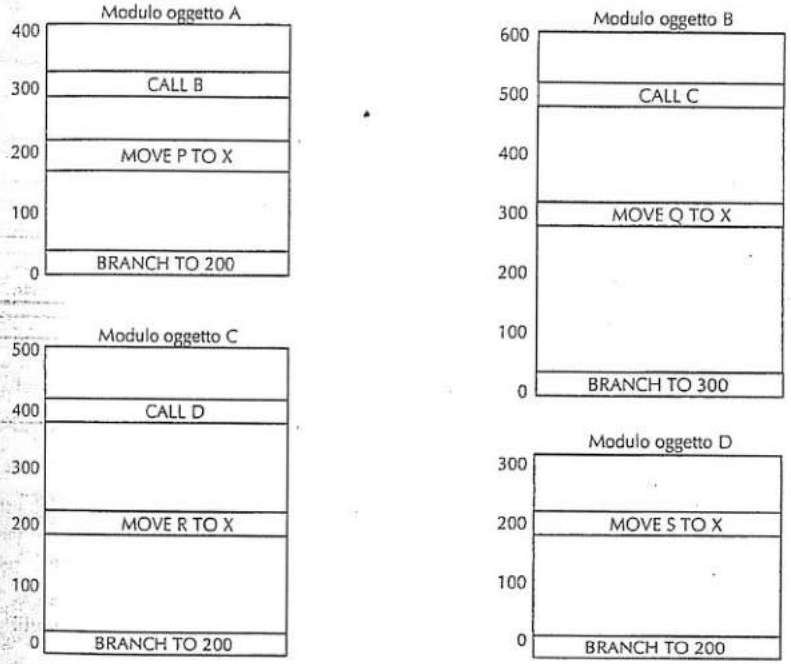


Figura 7.13 Ogni modulo ha il proprio spazio d'indirizzi che inizia da 0.

Per poter eseguire il programma il linker porta in memoria centrale i moduli oggetto al fine di formare l'immagine del programma eseguibile binario, come mostra la Figura 7.14(a).

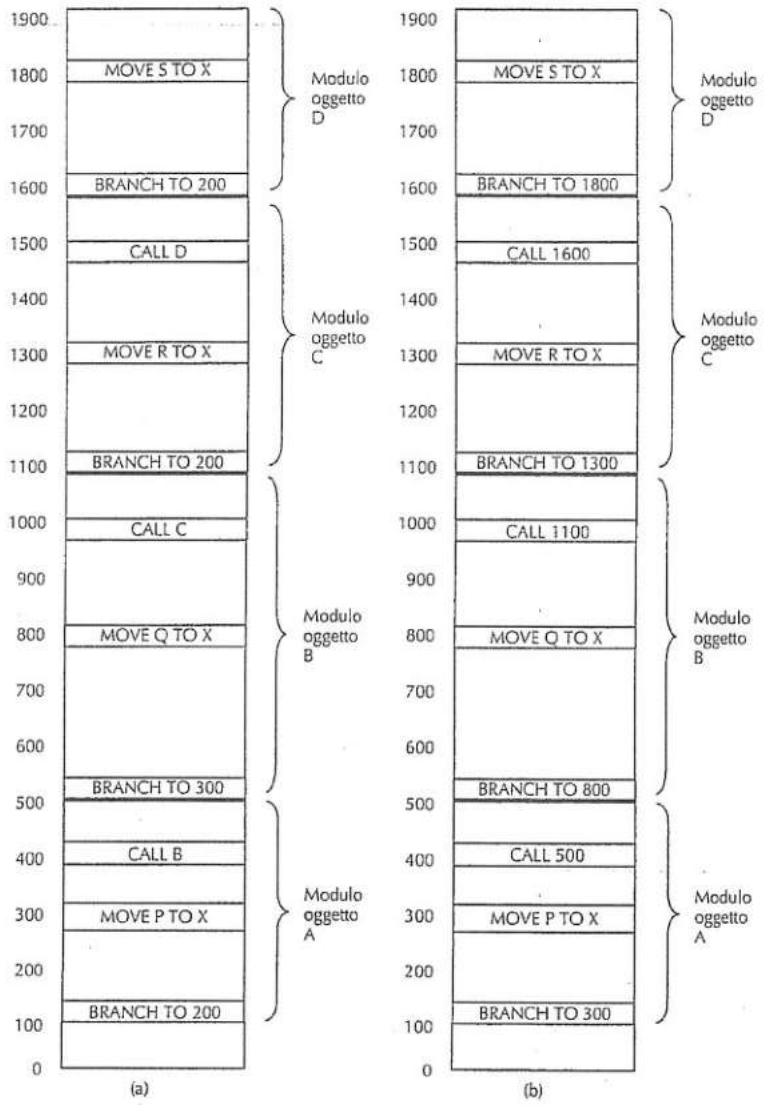


Figura 7.14 (a) I moduli oggetto della Figura 7.13 dopo essere stati posizionati nell'immagine binaria, ma prima di essere rilocati e collegati. (b) Gli stessi moduli oggetto dopo aver effettuato il collegamento e la rilocazione.

L'idea è di creare all'interno del linker un'immagine esatta dello spazio d'indirizzamento virtuale del programma eseguibile e di posizionare tutti i moduli oggetto nelle locazioni corrette. Se non c'è sufficiente memoria per formare l'immagine è possibile utilizzare un file del disco. In genere una piccola sezione della memoria a partire dall'indirizzo zero viene utilizzata per i vettori di interrupt, la comunicazione con il sistema operativo, la cattura dei puntatori non inizializzati e altri scopi. Per questo motivo i programmi di solito iniziano a partire da una locazione maggiore di 0. Nella figura abbiamo (arbitrariamente) fatto iniziare i programmi all'indirizzo 100.

Il programma della Figura 7.14(a) non è ancora pronto per essere eseguito, anche se è caricato all'interno dell'immagine del file eseguibile binario. Consideriamo che cosa succederebbe se l'esecuzione cominciasse con l'istruzione che si trova all'inizio del modulo *A*. Il programma non effettuerebbe correttamente il salto all'istruzione `MOVE`, dato che quella istruzione si trova ora all'indirizzo 300. In realtà tutte le istruzioni che fanno riferimento alla memoria fallirebbero per la stessa ragione. Occorre chiaramente trovare una soluzione al problema.

Il **problema della rilocazione** si verifica perché i moduli della Figura 7.13 hanno spazi degli indirizzi separati. Su una macchina con uno spazio degli indirizzi segmentato, come x86, ogni modulo potrebbe teoricamente avere il proprio spazio degli indirizzi all'interno del proprio segmento. In ogni caso l'unico sistema operativo per x86 che supporta questa possibilità è OS/2; tutte le versioni di Windows e di Unix supportano solamente uno spazio degli indirizzi lineare e quindi tutti i moduli oggetto devono essere uniti fra loro al suo interno. Inoltre neanche le istruzioni per la chiamata di procedure della Figura 7.14(a) funzionano correttamente. Nella figura si vede che all'indirizzo 400 il programmatore prova a chiamare il modulo oggetto *B*; l'assemblatore però non ha modo di sapere quale indirizzo inserire nell'istruzione `CALL B`, dato che ciascuna procedura è stata tradotta indipendentemente. L'indirizzo del modulo oggetto *B* non è infatti conosciuto fino al momento del collegamento. Questo è il **problema dei riferimenti esterni**. I due problemi possono essere risolti in modo semplice dal linker.

Il linker unisce gli spazi degli indirizzi separati dei diversi moduli oggetto all'interno di un unico spazio lineare degli indirizzi seguendo questi quattro passi:

1. costruisce una tabella di tutti i moduli oggetto e delle loro lunghezze;
2. in base a questa tabella assegna un indirizzo di partenza per ciascun modulo;
3. cerca tutte le istruzioni che fanno riferimento alla memoria e aggiunge a ciascuna di loro una **costante di rilocazione** uguale all'indirizzo di partenza del suo modulo;
4. cerca tutte le istruzioni che fanno riferimento ad altre procedure e inserisce in quei punti gli indirizzi delle procedure corrispondenti.

La tabella seguente fornisce nome, lunghezza e indirizzo iniziale di tutti i moduli della Figura 7.14.

Modulo	Lunghezza	Indirizzo iniziale
A	400	100
B	600	500
C	500	1100
D	300	1600

7.4.2 Struttura di un modulo oggetto

Spesso i moduli oggetto sono composti da sei parti, come mostra la Figura 7.15. La prima parte contiene il nome del modulo, informazioni necessarie al linker (come le lunghezze delle singole parti del modulo), e in alcuni casi la data di assemblaggio.

Figura 7.15 Struttura interna di un modulo oggetto prodotto da un traduttore. Il campo *identificativo* è il primo.

La seconda parte è una lista di simboli definiti all'interno del modulo al quale possono fare riferimento altri moduli; i simboli sono associati ai valori corrispondenti. Se per esempio il modulo consiste di una procedura chiamata *bigbug*, il primo elemento della tabella conterrà la stringa di caratteri "bigbug" seguita dall'indirizzo nel quale si trova la procedura. Il programmatore in linguaggio assemblativo utilizza la pseudoistruzione `PUBLIC` della Figura 7.2 per indicare quali simboli devono essere dichiarati come **punti d'ingresso**.

La terza parte del modulo oggetto comprende una lista di simboli utilizzati nel modulo, ma definiti in un altro, e anche una lista che indica quali simboli sono utilizzati dalle varie istruzioni macchina. Questa lista permette al linker di inserire gli indirizzi corretti nelle istruzioni che usano simboli esterni. Una procedura può chiamare altre procedure, tradotte indipendentemente, dichiarando come esterni i loro nomi. Il programmatore in linguaggio assemblativo indica quali simboli devono essere dichiarati come **simboli esterni** utilizzando una pseudoistruzione (per esempio la `EXTERN` della Figura 7.2). In alcuni calcolatori i punti d'ingresso e i riferimenti esterni sono combinati in un'unica tabella.

La quarta parte del modulo oggetto è costituita dal codice assemblato e dalle costanti. Questa è l'unica parte del modulo oggetto che verrà caricata in memoria per essere eseguita. Le altre parti saranno invece utilizzate dal linker ed eliminate prima che inizi l'esecuzione.

La quinta parte è il dizionario di rilocazione. Com'è possibile vedere nella Figura 7.14 occorre aggiungere una costante di rilocazione alle istruzioni contenenti dei riferimenti a indirizzi di memoria. Il linker, ispezionando la quarta parte del modulo, non ha modo di sapere quali parole di dati contengano istruzioni macchina e quali contengano costanti; questa tabella gli fornisce le informazioni sugli indirizzi che devono essere rilocati. Queste informazioni potrebbero formare una tabella di bit, con un bit per ogni indirizzo potenzialmente da rilocare, oppure una lista esplicita degli indirizzi da rilocare.

La sesta parte è composta da un identificatore della fine del modulo, dall'indirizzo a partire dal quale bisogna iniziare l'esecuzione e, in alcuni casi, da una checksum per rilevare gli errori che possono essere avvenuti durante la lettura del modulo.

La maggior parte dei linker richiede due passate. Durante la prima il linker legge tutti i moduli oggetto e costruisce una tabella con i nomi e le lunghezze dei moduli, oltre a una tabella globale di simboli contenenti tutti i punti d'ingresso e i riferimenti esterni. Durante la seconda passata i moduli oggetto vengono letti, rilocati e collegati uno alla volta.

7.4.3 Rilocazione a tempo del binding e dinamica

In un sistema multiprogrammato un programma può essere letto e portato nella memoria centrale, eseguito per un breve periodo di tempo, riportato su disco e poi riletto per essere eseguito un'altra volta. In un sistema di grandi dimensioni, quando è attivo un numero elevato di programmi, è difficile avere la certezza che il programma venga letto e caricato ogni volta nelle stesse locazioni.

La Figura 7.16 mostra che cosa succederebbe se il programma già rilocato della Figura 7.14(b) venisse caricato all'indirizzo 400 invece che all'indirizzo 100 in cui il linker l'aveva inserito originariamente. Tutti gli indirizzi di memoria risulterebbero errati e, per di più, non si potrebbero riutilizzare le informazioni di rilocazione, cancellate molto tempo prima. Anche se queste fossero ancora disponibili, il costo per rilocare tutti gli indirizzi ogni volta che un programma viene riportato in memoria centrale sarebbe troppo alto.

Il problema di spostare programmi che sono già stati collegati e rilocati è profondamente connesso al momento in cui viene completato il collegamento finale tra i nomi simbolici e gli indirizzi assoluti della memoria fisica. Un programma appena scritto contiene dei nomi simbolici per gli indirizzi di memoria, per esempio `BR L`. Il momento nel quale si determina l'effettivo indirizzo della memoria centrale corrispondente a *L* è chiamato **tempo del binding** ("tempo del collegamento").

Esistono almeno sei possibilità:

1. quando il programma viene scritto;
2. quando il programma viene tradotto;
3. quando il programma viene collegato, ma prima che sia caricato;
4. quando il programma viene caricato;
5. quando viene caricato un registro base utilizzato per l'indirizzamento;
6. quando viene eseguita l'istruzione contenente l'indirizzo.

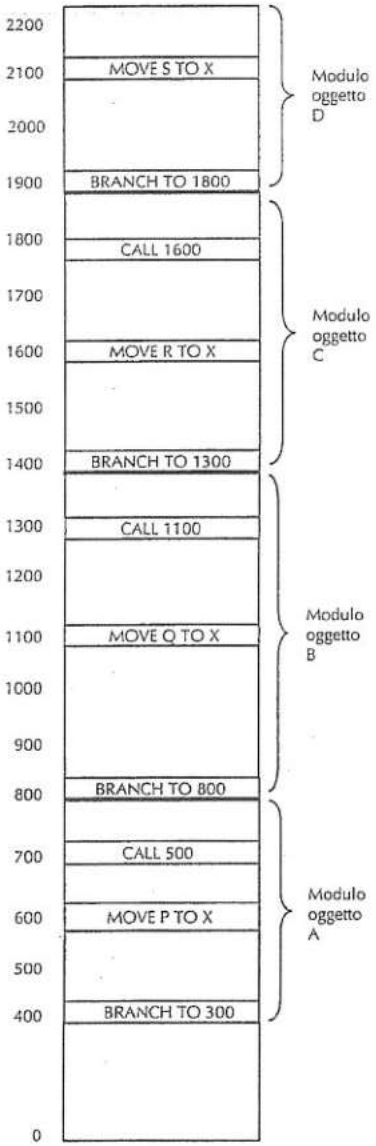


Figura 7.16 Programma binario rilocato della Figura 7.14(b) spostato di 300 posizioni. Molte istruzioni si riferiscono ora a indirizzi di memoria errati.

Se un'istruzione contenente un indirizzo di memoria viene spostata dopo il collegamento, il suo riferimento alla memoria risulta errato (assumendo che anche l'oggetto al quale fa riferimento sia stato spostato). Se il traduttore produce come output un eseguibile binario, il momento del collegamento coincide con quello della traduzione e in questo caso il programma deve essere eseguito all'indirizzo assunto dal traduttore. Il metodo descritto nel paragrafo precedente lega i nomi simbolici agli indirizzi assoluti durante la fase di collegamento; questa è la ragione per cui gli spostamenti dei programmi effettuati dopo il collegamento falliscono (come mostra l'esempio della Figura 7.16).

La questione solleva due tipi di problemi. Il primo riguarda il momento in cui i nomi simbolici vengono legati agli indirizzi virtuali, mentre il secondo riguarda il momento in cui gli indirizzi virtuali vengono legati agli indirizzi fisici. Il collegamento si può dire completato solo dopo che queste due operazioni sono state terminate. Quando il linker riunisce i diversi spazi degli indirizzi crea in realtà uno spazio virtuale di indirizzi. La rilocazione e il collegamento hanno l'effetto di collegare i nomi simbolici con specifici indirizzi virtuali. Questa osservazione è vera a prescindere dal fatto che venga o meno utilizzata la memoria virtuale.

Assumiamo per il momento che lo spazio degli indirizzi della Figura 7.14(b) sia paginato. È chiaro che gli indirizzi virtuali corrispondenti ai nomi simbolici *A*, *B*, *C* e *D* sono già stati determinati, anche se i loro indirizzi fisici dipenderanno, in ogni momento in cui verranno utilizzati, dal contenuto della tabella delle pagine. Un programma eseguibile binario è in realtà un collegamento tra nomi simbolici e indirizzi virtuali.

Qualsiasi meccanismo che permetta di modificare facilmente la corrispondenza degli indirizzi virtuali negli indirizzi fisici della memoria faciliterà lo spostamento dei programmi all'interno della memoria principale, anche dopo che essi sono stati legati a uno spazio degli indirizzi virtuali. Uno di questi meccanismi è la paginazione. Dopo che un programma è stato spostato all'interno della memoria centrale occorre modificare solamente la sua tabella delle pagine e non il programma stesso.

Una seconda tecnica consiste nell'uso di un registro di rilocazione durante l'esecuzione (come avveniva per il CDC 6600 e i suoi successori). Sulle macchine che utilizzano questa tecnica il registro punta sempre all'indirizzo fisico della memoria in cui inizia il programma corrente. Il registro di rilocazione viene sommato via hardware a tutti gli indirizzi, prima che essi siano inviati alla memoria. L'intero processo di rilocazione risulta così trasparente ai programmi utente. Quando viene spostato un programma il sistema operativo deve aggiornare il registro di rilocazione. Questo meccanismo è però meno generale rispetto alla paginazione, dato che l'intero programma deve essere spostato come una singola unità (oppure come due, nel caso in cui vi siano registri di rilocazione separati per il codice e per i dati, come nel processore Intel 8088).

Sulle macchine in grado di effettuare riferimenti alla memoria relativi al contatore d'istruzioni è possibile ricorrere a un terzo meccanismo. Grazie a questa funzionalità è possibile sfruttare il fatto che molte istruzioni di salto sono relative al valore di PC. Ogni volta che si sposta un programma nella memoria centrale occorre modificare solamente il PC. Un programma in cui tutti i riferimenti alla memoria sono relativi al PC oppure sono assoluti (per esempio, gli indirizzi assoluti dei registri dei dispositivi di I/O) è detto **indipendente dalla posizione**. Una procedura indipendente dalla posizione può essere

collocata in qualsiasi punto dello spazio degli indirizzi virtuali senza bisogno di effettuare la rilocazione.

7.4.4 Collegamento dinamico

Nella strategia di collegamento descritta nel Paragrafo 7.4.1 tutte le procedure che un programma potrebbe richiamare sono collegate prima che abbia inizio l'esecuzione. Se si completano tutti i collegamenti prima dell'esecuzione, non si trae pieno vantaggio dalla memoria virtuale, di cui un calcolatore può essere dotato. Molti programmi contengono procedure, chiamate soltanto in particolari, e infrequenti, circostanze. I compilatori, per esempio, hanno alcune procedure per tradurre istruzioni utilizzate raramente, e altre il cui compito è quello di gestire condizioni di errore che si verificano di rado.

Un modo più flessibile per collegare procedure compilate separatamente è quello di collegarle nel momento in cui ciascuna procedura viene caricata. Questo processo è conosciuto con il nome di **collegamento dinamico**. Il primo calcolatore che ha impiegato questa tecnica è stato MULTICS, la cui pionieristica implementazione è, sotto alcuni aspetti, ancora ineguagliata. Nei paragrafi successivi analizzeremo come avviene il collegamento dinamico in alcuni sistemi.

Collegamento dinamico in MULTICS

L'implementazione di MULTICS del collegamento dinamico associa a ciascun programma un **segmento di collegamento** contenente un blocco d'informazioni per ciascuna procedura del programma. Questo blocco d'informazioni inizia con una parola riservata per l'indirizzo virtuale della procedura, seguita dal nome della procedura, memorizzato come stringa di caratteri.

Quando si utilizza il collegamento dinamico le chiamate di procedura nel linguaggio sorgente vengono tradotte in istruzioni che indirizzano, in modo indiretto, la prima parola del blocco di collegamento (la Figura 7.17(a) ne mostra un esempio). Il compilatore riempie questa parola o con un indirizzo non valido oppure con una speciale stringa di bit che determina un'eccezione.

Quando si richiama una procedura che fa parte di un altro segmento, il tentativo di indirizzare la parola non valida solleva nel linker dinamico un'eccezione. Il linker legge quindi la stringa di caratteri che si trova nella parola che segue l'indirizzo non valido e cerca una procedura con questo nome all'interno della directory dell'utente. Viene quindi assegnato un indirizzo virtuale a questa procedura, di solito all'interno del proprio segmento privato; inoltre, come mostra la Figura 7.17(b), questo indirizzo virtuale viene scritto nel segmento di collegamento, al posto dell'indirizzo non valido. Successivamente viene rieseguita l'istruzione che ha provocato l'errore di collegamento e ciò permette al programma di continuare la propria esecuzione dal punto in cui si trovava prima che venisse sollevata l'eccezione.

Tutti i successivi riferimenti alla procedura avverranno senza provocare errori di collegamento, dato che la prima parola del segmento di collegamento contiene ora un indirizzo virtuale valido. Da ciò ne consegue che il linker dinamico viene invocato solamente la prima volta in cui viene richiamata una procedura e non durante le chiamate successive.

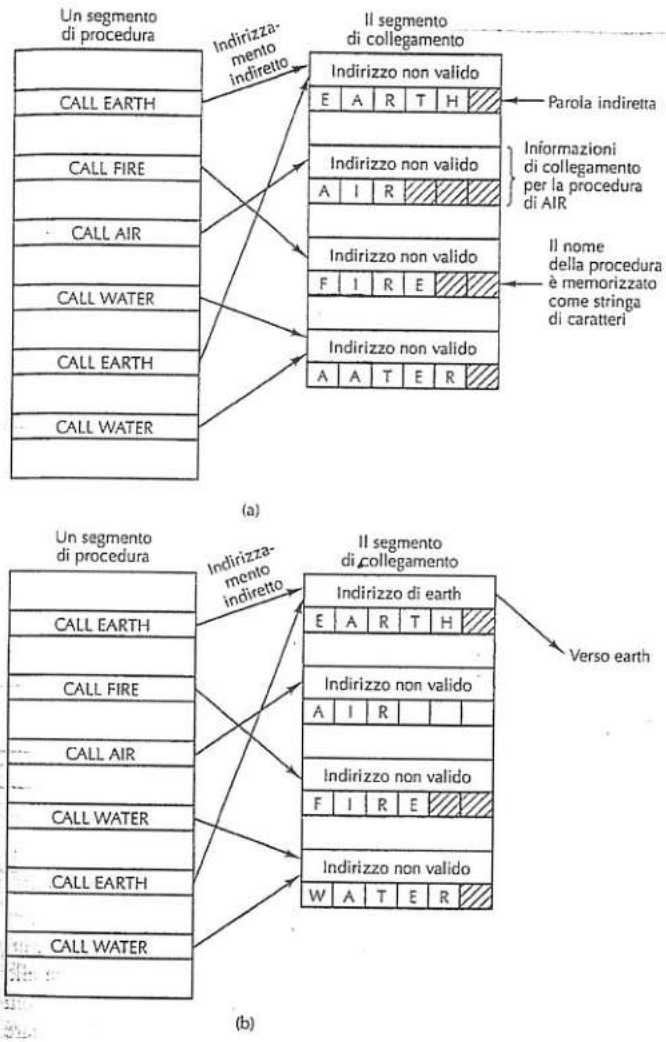


Figura 7.17 Collegamento dinamico. (a) Prima che *EARTH* sia chiamata. (b) Dopo che *EARTH* è stata chiamata e collegata.

Collegamento dinamico in Windows

Tutte le versioni del sistema operativo Windows, compreso NT, supportano e sfruttano pienamente il collegamento dinamico. Allo scopo viene utilizzato un formato di file chiamato **DLL** (*Dynamic Link Library*, "libreria a collegamento dinamico"). Le DLL

possono contenere procedure, dati oppure entrambi, e sono comunemente utilizzate per permettere a due o più processi di condividere librerie di dati e procedure. Molte DLL hanno l'estensione *.dll*, ma vengono utilizzate anche estensioni diverse, come *.drv* (per le librerie di driver) e *.fon* (per le librerie di caratteri).

Nella sua forma più comune una DLL è una libreria composta da un gruppo di procedure che possono essere caricate in memoria e alle quali possono accedere più processi nello stesso istante. La Figura 7.18 rappresenta due programmi che condividono un file DLL contenente quattro procedure, *A*, *B*, *C* e *D*. Nell'illustrazione il programma 1 usa la procedura *A* e il programma 2 usa la *C*, anche se i due programmi avrebbero comunque potuto utilizzare la stessa procedura.

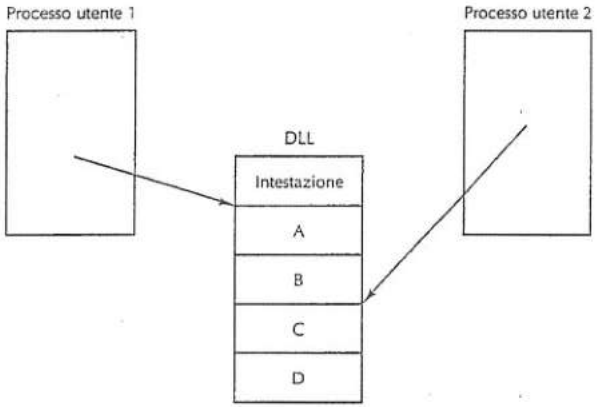


Figura 7.18 Un file DLL usato da due processi.

Il linker costruisce una DLL a partire da un insieme di file di input. La costruzione di una DLL è in pratica molto simile alla creazione di un programma eseguibile binario, tranne che per uno speciale indicatore che viene specificato al linker in modo che crei una DLL. Di solito le DLL vengono costruite a partire da gruppi di librerie di procedure che ci si aspetta verranno utilizzate da più processi. Le procedure d'interfaccia alla libreria delle chiamate di sistema di Windows e le librerie grafiche di grandi dimensioni sono due esempi di DLL molto comuni. L'uso delle DLL ha il vantaggio di risparmiare spazio in memoria e sul disco. Se una libreria usata molto frequentemente venisse collegata staticamente a ciascun programma che ne fa uso, essa comparirebbe all'interno di molti programmi binari eseguibili sia in memoria sia sul disco, sprecando una gran quantità di spazio. Facendo ricorso alle DLL ciascuna libreria è presente una sola volta in memoria e una sola sul disco.

Oltre a risparmiare spazio questo approccio agevola l'aggiornamento delle procedure, anche dopo che i programmi che le utilizzano sono stati compilati e collegati. Per pacchetti software commerciali, di cui raramente gli utenti hanno a disposizione il codice, l'uso delle DLL permette al venditore di correggere i bug presenti nelle librerie semplicemente ridistribuendo via Internet nuovi file DLL, senza dover cambiare in alcun modo il programma binario principale.

La differenza principale tra una DLL e un eseguibile binario è che la DLL non può essere eseguita da sola (non ha al suo interno un programma principale). Inoltre essa contiene nella sua intestazione informazioni di tipo diverso e possiede delle procedure aggiuntive che non sono legate alle procedure della libreria. È presente per esempio una procedura che viene automaticamente chiamata ogni volta che un nuovo processo è collegato alla DLL e un'altra che viene automaticamente chiamata quando un processo viene invece scollegato. Queste procedure possono allocare o deallocare aree di memoria o gestire altri tipi di risorse richieste dalla DLL.

Ci sono due modi per collegare un programma a una DLL. Nel primo, chiamato **collegamento implicito**, il programma dell'utente viene collegato a un file speciale chiamato **libreria importata**, generato da un programma di utilità che estrae alcune informazioni dalla DLL. La libreria importata costituisce il collante che permette al programma utente di accedere alla DLL. Un programma utente può essere collegato a più librerie importate. Quando viene caricato un programma che utilizza il collegamento implicito, Windows lo esamina per vedere quali DLL utilizza e controlla che tutte siano presenti in memoria. Quelle che non lo sono vengono caricate immediatamente (ma non necessariamente nella loro interezza, dato che sono paginate). Vengono apportati alcuni cambiamenti alle strutture dati delle librerie importate in modo da poter localizzare le procedure richiamate (queste modifiche sono analoghe a quelle viste nella Figura 7.17). Inoltre esse devono essere mappate nello spazio degli indirizzi virtuali del programma. A questo punto il programma utente è pronto per essere eseguito e può chiamare le procedure presenti nella DLL come se fossero staticamente legate al programma stesso.

L'alternativa a questo tipo di collegamento è, ovviamente, il **collegamento esplicito**. Questo approccio non richiede l'uso di librerie importate e non obbliga a caricare le DLL nello stesso momento in cui viene caricato il programma dell'utente. Al contrario il programma dell'utente effettua a tempo di esecuzione una chiamata esplicita per legare a sé una DLL e successivamente effettua altre chiamate per ottenere gli indirizzi delle procedure di cui ha bisogno. Dopo aver recuperato questi indirizzi il programma può chiamare le procedure. Quando ha finito di utilizzarle effettua una chiamata finale per scollegarsi dalla DLL. Quando anche l'ultimo processo si è scollegato da una DLL questa può essere rimossa dalla memoria.

È importante comprendere che, a differenza dei thread e dei processi, in una DLL le procedure non hanno una propria identità. Queste procedure vengono eseguite nel thread del chiamante e utilizzano lo stack del chiamante per le proprie variabili locali. Possono avere alcuni dati statici specifici di un processo (così come dati condivisi), ma si comportano esattamente come le procedure collegate staticamente. L'unica vera differenza riguarda il modo in cui viene effettuato il collegamento.

Collegamento dinamico in UNIX

Il sistema UNIX ha un meccanismo che è essenzialmente simile alle DLL di Windows e che si basa su quella che viene chiamata **libreria condivisa**. Analogamente ai file DLL una libreria condivisa è un file archivio contenente procedure o moduli di dati presenti

in memoria a tempo di esecuzione e può essere collegata contemporaneamente a più processi. La libreria standard di C e gran parte del codice relativo alla comunicazione via rete sono esempi di librerie condivise.

UNIX supporta esclusivamente il collegamento implicito e quindi una libreria condivisa consiste in due parti: **statica** (collegata staticamente al file eseguibile) e **destinazione** (richiamata a tempo di esecuzione). Anche se alcuni dettagli presentano delle differenze, i concetti sono essenzialmente uguali a quelli delle DLL.

7.5 Riepilogo

Anche se la maggior parte dei programmi può, e dovrebbe, essere scritta in un linguaggio ad alto livello, in alcune situazioni è necessario utilizzare, almeno in parte, il linguaggio assemblativo. Candidati alla scrittura in linguaggio assemblativo sono i programmi per calcolatori dotati di poche risorse, come le smart card e i processori integrati in dispositivi come le radiosveglie. Un programma in linguaggio assemblativo è una rappresentazione simbolica di un programma scritto nel linguaggio macchina di un livello inferiore. Esso viene tradotto nel linguaggio macchina mediante un programma chiamato assemblatore.

Per le applicazioni che richiedono un'alta velocità di esecuzione si sceglie di scrivere inizialmente l'intero programma in un linguaggio ad alto livello, e poi si determinano i punti in cui viene spesa la maggior parte del tempo; quindi si riscrivono in linguaggio assemblativo solo le porzioni di codice usate in modo più intenso. Questo approccio ha dei vantaggi rispetto a scrivere l'intero programma in codice assemblativo, dato che in genere solo una piccola frazione del codice è in realtà responsabile della maggior parte del tempo di esecuzione.

Molti assemblatori permettono l'uso delle macro, le quali consentono al programmatore di attribuire nomi simbolici a sequenze di codice usate frequentemente. In un secondo momento questi nomi vengono automaticamente sostituiti con il codice corrispondente. Di solito le macro possono anche essere parametrizzate in modo semplice. Le macro sono implementate mediante algoritmi di elaborazione di stringhe di caratteri.

La maggior parte degli assemblatori prevede due passate. La prima si occupa di costruire una tabella per le etichette, i letterali e gli identificatori dichiarati esplicitamente. I simboli possono essere memorizzati in modo disordinato e reperiti mediante ricerche lineari, oppure possono essere tenuti in ordine e poi individuati con una ricerca dicotomica. Un'altra possibilità per memorizzare e cercare i simboli consiste nell'uso di una tabella hash. Se i simboli non devono essere cancellati durante la prima passata, questo metodo solitamente risulta il migliore. La seconda passata genera il codice. Alcune pseudoistruzioni vengono eseguite durante la prima passata, altre durante la seconda.

I programmi assemblati indipendentemente possono essere collegati fra loro per formare un codice binario che può essere eseguito. Questo lavoro è realizzato dal linker. I suoi compiti principali sono la rilocazione e il collegamento dei nomi. Il collegamento dinamico è una tecnica grazie alla quale alcune procedure non sono collegate fino al momento in cui non vengono effettivamente richiamate. Le DLL di Windows e le librerie condivise di UNIX usano il collegamento dinamico.

PROBLEMI

1. In un dato programma il 2% del codice è responsabile del 50% del tempo di esecuzione. Si confrontino le seguenti strategie relative ai tempi di programmazione e di esecuzione. Si assuma che il programma richieda 100 mesi/uomo di lavoro per essere programmato in C; la scrittura del codice assemblativo è invece 10 volte più lenta, ma il programma ottenuto risulta quattro volte più efficiente.
   a. Intero programma in C.
   b. Intero programma in linguaggio assemblativo.
   c. Prima tutto in C, poi il 2% più critico riscritto in linguaggio assemblativo.

2. Le considerazioni che valgono per gli assemblatori a due passate, valgono anche per i compilatori?
   a. Si assuma che i compilatori producano moduli oggetto, non codice assemblativo.
   b. Si assuma che i compilatori producano linguaggio assemblativo simbolico.

3. La maggior parte degli assemblatori per x86 ha l'indirizzo destinazione come primo operando e l'indirizzo sorgente come secondo operando. Quali problemi si sarebbero dovuti risolvere per utilizzare gli operandi in posizione invertita?

4. È possibile assemblare in due passate il seguente programma? EQU è una pseudoistruzione che mette in relazione l'etichetta con l'espressione che si trova nel campo dell'operando.

```
P EQU Q
Q EQU R
R EQU S
S EQU 4
```

5. La Società di Calcolatori DaDueSoldi sta pianificando di produrre un assemblatore per un calcolatore con parole a 48 bit. Per contenere i costi il responsabile del progetto, il Dott. Paperone, ha deciso di limitare la lunghezza degli identificatori in modo che possano essere memorizzati in una sola parola. Paperone ha dichiarato che gli identificatori possono consistere solamente in lettere, tranne la Q che è vietata (per superstizione: infatti è la 17ª lettera dell'alfabeto inglese). Qual è la massima lunghezza dei simboli? Si descriva un proprio schema di codifica.

6. Qual è la differenza tra istruzioni e pseudoistruzioni?

7. Qual è la differenza (se c'è) tra il contatore di locazioni delle istruzioni (ILC) e il contatore d'istruzioni (PC)? Dopo tutto entrambi tengono traccia della successiva istruzione in un programma.

8. Si mostri la tabella dei simboli dopo che si sono incontrate le seguenti istruzioni per x86. La prima istruzione è assegnata all'indirizzo 1000.

```
EVEREST:    POP BX          (1 BYTE)
K2:         PUSH BP         (1 BYTE)
WHITNEY:    MOV BP,SP       (2 BYTES)
MCKINLEY:   PUSHX           (3 BYTES)
FUJI:       PUSH SI         (1 BYTE)
KIBO:       SUB SI,300      (3 BYTES)
```

9. È possibile immaginare delle circostanze nelle quali il linguaggio assemblativo permetta che un'etichetta sia identica a un codice operativo (per esempio, MOV come etichetta)? Si motivi la risposta.

10. Si mostrino i passi necessari per cercare, utilizzando la ricerca dicotomica, la parola Berkeley all'interno della seguente lista: Ann Arbor, Berkeley, Cambridge, Eugene, Madison, New Haven, Palo Alto, Pasadena, Santa Cruz, Stony Brook, Westwood e Yellow Springs. Quando si calcola l'elemento centrale di una lista con un numero pari di elementi si utilizzi l'elemento che si trova subito dopo il punto medio.

11. È possibile utilizzare la ricerca dicotomica su una tabella la cui dimensione è un numero primo?

12. Si calcoli il codice hash dei seguenti simboli sommando le lettere (A = 1, B = 2 e così via) e calcolando il modulo tra il risultato e la dimensione della tabella hash. La tabella hash ha 19 posizioni, numerate da 0 a 18.
    els, jan, jelle, maaike
    Ciascuno di questi simboli genera un unico valore hash? In caso negativo, com'è possibile gestire il problema della collisione?

13. Il metodo della codifica hash descritto nel testo inserisce in una lista concatenata tutti gli elementi che hanno lo stesso codice hash. Un metodo alternativo è quello di avere un'unica tabella a *n* posizioni, in cui ciascuna ha lo spazio per memorizzare una chiave e il suo valore (o un puntatore a quest'ultimo). Se l'algoritmo di hash indica una posizione già occupata, si riprovi con un secondo algoritmo di hash. Se anche questa posizione è già occupata, se ne utilizzi un altro, continuando allo stesso modo finché non se ne trovi una vuota. Se la frazione di posizioni che sono occupate è *R*, quanti tentativi saranno necessari, in media, per poter inserire un nuovo simbolo?

14. Grazie ai progressi tecnologici un giorno potrebbe essere possibile collocare su un chip migliaia di CPU identiche, ciascuna delle quali con poche parole di memoria locale. Se tutte le CPU possono leggere e scrivere tre registri condivisi com'è possibile implementare una memoria associativa?

15. x86 ha un'architettura segmentata, composta da segmenti indipendenti. Un assemblatore per questa macchina potrebbe avere una pseudoistruzione SEG N, per indicare all'assemblatore di collocare il codice successivo nel segmento N. Questo schema avrebbe qualche influenza su ILC?

16. I programmi spesso si collegano a più DLL. Non sarebbe più efficiente inserire tutte le procedure in un'unica grande DLL e collegarla al programma?

17. È possibile mappare una DLL negli spazi degli indirizzi virtuali di due processi in due indirizzi virtuali diversi? Se sì, quali problemi sorgono? Possono essere risolti? Altrimenti, che cosa si può fare per eliminarli?

18. Un modo per effettuare il collegamento (statico) è il seguente. Prima di scandire la libreria il linker costruisce una lista delle procedure richieste, cioè una lista composta dai nomi definiti come EXTERN nei moduli che sta collegando. In seguito il linker analizza linearmente tutta la libreria, estraendo ogni procedura che si trova nella lista di quelle necessarie. Questo schema può funzionare? Altrimenti, perché no, e che rimedi si possono apportare?

19. È possibile utilizzare un registro come parametro attuale in una chiamata di macro? E una costante? Perché sì o perché no?

20. Si sta implementando un assemblatore con macro. Per ragioni estetiche il capo progetto ha deciso che le definizioni di macro non devono precedere le loro chiamate. Quali conseguenze ha questa decisione sull'implementazione?

21. Si pensi a un modo per far entrare un assemblatore con macro in un ciclo infinito.

22. Un linker legge cinque moduli, le cui lunghezze sono rispettivamente 200, 800, 600, 500 e 700. Se i moduli vengono caricati in quest'ordine quanto valgono le costanti di rilocazione?

23. Si scrivano due routine per gestire una tabella di simboli: *enter(symbol, value)* e *lookup(symbol, value)*. La prima inserisce i nuovi simboli nella tabella, mentre la seconda permette di cercare un simbolo al suo interno. Si utilizzi una codifica hash.

24. Si ripeta l'esercizio precedente senza utilizzare una tabella hash: dopo che l'ultimo simbolo viene inserito, si ordini la tabella e si utilizzi un algoritmo di ricerca dicotomica per trovare i simboli.

25. Si scriva un semplice assemblatore per il calcolatore Mic-1 del Capitolo 4. Si fornisca la possibilità di assegnare durante la fase di assemblaggio valori costanti ai simboli e un modo per assemblare una costante in una parola della macchina (queste dovrebbero essere pseudoistruzioni).

26. Si aggiunga la funzionalità delle macro all'assemblatore dell'esercizio precedente.

CAPITOLO 8

Architetture per il calcolo parallelo

Benché i calcolatori diventino sempre più veloci, le aspettative dei loro utenti crescono almeno altrettanto rapidamente. L'ambizione degli astronomi è la simulazione dell'intera storia dell'universo, dal big bang alla fine dei tempi. L'industria farmaceutica gradirebbe progettare sui propri computer medicine per la cura mirata di determinate malattie, invece di sacrificare intere legioni di ratti. I progettisti aeronautici sarebbero in grado di scoprire prodotti dotati di maggiore efficienza energetica se solo i calcolatori potessero farsi carico di tutto il lavoro, senza bisogno di costruire prototipi reali da valutare nella galleria del vento. È chiaro che, anche se è ormai disponibile molta potenza di calcolo, per un gran numero di utenti, e in special modo per gli scienziati e per l'industria, questa potenza non è mai abbastanza.

Nonostante il costante incremento delle frequenze di clock, la velocità dei circuiti non può aumentare indefinitamente. La velocità della luce costituisce già oggi un limite pratico per i progettisti dei calcolatori di fascia alta e le prospettive di riuscire a far muovere gli elettroni e i fotoni ancora più velocemente sono scarse. I problemi di dissipazione del calore fanno sì che i supercalcolatori diventino sempre più sofisticati condizionatori d'aria. Infine, le dimensioni dei transistor continueranno a diminuire e raggiungeranno presto il punto in cui conterranno così pochi atomi che gli effetti della meccanica quantistica (per esempio il principio di indeterminazione di Heisenberg) porranno seri problemi.

In conseguenza di tutto ciò, gli architetti degli elaboratori si rivolgono sempre più spesso al calcolo parallelo per trattare problemi di dimensioni sempre crescenti. Anche se non è possibile realizzare un calcolatore con una sola CPU e un ciclo di 0,001 ns, sarebbe sicuramente possibile costruirne uno con 1000 CPU, ciascuna con ciclo di clock di 1 ns. Sebbene il secondo progetto si avvalga di CPU più lente, in teoria la sua capacità totale di calcolo è la stessa del primo. Questa è l'idea cui sono affidate le speranze per il futuro.

Il parallelismo può essere introdotto a vari livelli. A quello più basso, può essere incorporato nel chip della CPU attraverso un progetto superscalare a pipeline con molteplici unità funzionali; oppure può essere introdotto con l'adozione d'istruzioni basate su parole molto lunghe e dotate di parallelismo implicito. La CPU può essere equipaggiata con caratteristiche speciali che le consentano di gestire il controllo di più thread alla volta, ed è anche possibile integrare più CPU sullo stesso chip. Tutti questi stratagemmi, se usati congiuntamente, possono produrre un incremento delle prestazioni per un fattore 10, al più, rispetto a un progetto puramente sequenziale.

A livello successivo, è possibile aggiungere al sistema delle schede dotate di CPU che arricchiscano le sue capacità di calcolo. In genere queste CPU *plug in* (cioè a innesto) svolgono funzioni specializzate, come l'elaborazione di pacchetti di rete, l'elaborazione multimediale o la crittografia. Consentono un fattore d'incremento delle prestazioni compreso tra 5 e 10, ma solo nel caso di applicazioni specializzate.

D'altra parte, l'unico modo per ottenere un incremento dell'ordine delle centinaia, delle migliaia o dei milioni, è di replicare il numero delle CPU, facendole cooperare in modo efficiente. Questa è l'idea alla base dei multiprocessori e dei multicomputer (*cluster di computer*).

È evidente che il montaggio di migliaia di processori in un unico grande sistema costituisce di per sé un problema che va risolto.

Va detto infine che oggi è possibile strutturare su Internet intere organizzazioni di computer come reti di calcolo (griglie) con connessioni lasche. Sono sistemi alle prime armi, ma manifestano un grosso potenziale per il futuro.

Due CPU o elementi di calcolo contigui che svolgono calcoli in modo molto interattivo e la cui connessione ha un'elevata larghezza di banda e un ritardo trascurabile, si dicono **legati strettamente** (*tightly coupled*). Viceversa si dicono **legati debolmente** (*loosely coupled*) se si trovano molto distanti, hanno una banda ridotta, un ritardo elevato e svolgono i loro calcoli in modo poco interattivo. In questo capitolo ci interessiamo ai principi progettuali delle varie forme di parallelismo e ne analizziamo una varietà di esempi. Cominciamo dai sistemi a legame più stretto, cioè quelli che usano il parallelismo nel chip, spostandoci gradualmente verso sistemi con legame più debole, per finire con lo spendere qualche parola sul *grid computing* ("calcolo su griglie di computer"). La Figura 8.1 mostra lo spettro delle possibilità (da legame più stretto a più debole).

La questione del parallelismo è oggetto di intensa ricerca a tutti i livelli, perciò nel corso del capitolo segnaleremo un gran numero di riferimenti bibliografici, rivolgendo particolare attenzione agli articoli più recenti. Diverse conferenze e riviste pubblicano articoli sull'argomento e la letteratura cresce rapidamente.

8.1 Parallelismo nel chip

Un modo per incrementare la produttività di un chip è di fargli svolgere più compiti alla volta: in altre parole, sfruttare il parallelismo. In questo paragrafo ci interessiamo ai modi in cui si può ottenere un incremento delle prestazioni tramite il parallelismo a livello del chip, compreso il parallelismo del livello delle istruzioni, il multithreading e la coabitazione di più CPU sullo stesso chip. Sono tecniche abbastanza diverse e ciascuna dà un proprio contributo. Il loro comune obiettivo è quello di svolgere più attività nello stesso tempo.

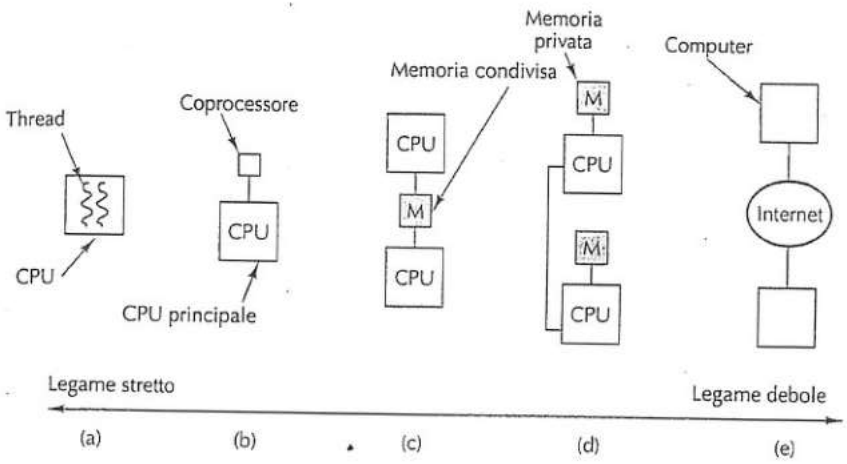


Figura 8.1 (a) Parallelismo nel chip. (b) Coprocessore. (c) Multiprocessore. (d) Multicomputer. (e) Griglia.

8.1.1 Parallelismo a livello delle istruzioni

A livello delle istruzioni, un modo per ottenere il parallelismo è di emettere più istruzioni per ciclo di clock. Ci sono due tipi di CPU a emissione multipla: processori superscalari e processori VLIW. Vi abbiamo già accennato precedentemente, ma può essere utile ritornare sull'argomento.

Abbiamo già visto le CPU superscalari (Figura 2.6). Nel caso più generale, c'è un momento lungo la pipeline in cui un'istruzione è pronta per essere eseguita. Le CPU superscalari sono in grado di emettere, verso le unità di esecuzione, più istruzioni in un solo ciclo. Il numero d'istruzioni emesse dipende dal progetto del processore e dalle circostanze contingenti. L'hardware determina il numero massimo d'istruzioni che può essere emesso, in genere da due a sei. In ogni caso un'istruzione non verrà emessa se attende un calcolo che non è stato ancora completato o se necessita di un'unità funzionale non disponibile.

L'altra forma di parallelismo a livello delle istruzioni è quello dei processori **VLIW** (*Very Long Instruction Word*, "con parola d'istruzione molto lunga"). Nella loro forma originaria, le macchine VLIW avevano davvero parole molto lunghe per contenere istruzioni che usavano diverse unità funzionali. Si consideri per esempio la Figura 8.2(a), in cui la macchina dispone di cinque unità funzionali e può eseguire simultaneamente due operazioni intere, una in virgola mobile, un caricamento e una memorizzazione. Un'istruzione VLIW per questa macchina conterrebbe cinque codici operativi e cinque

coppie di operandi: un codice e una coppia di operandi per ogni unità funzionale. Se ci sono 6 bit per codice operativo, 5 bit per registro e 32 bit per gli indirizzi di memoria, le istruzioni possono raggiungere facilmente i 134 bit, il che fa di loro istruzioni abbastanza lunghe.

Questo progetto si rivelò troppo rigido perché non tutte le istruzioni riuscivano a sfruttare tutte le unità funzionali, il che conduceva all'uso di molte inutili NO-OP, come illustrato nella Figura 8.2(b). Perciò le macchine VLIW moderne dispongono di una modalità di costruzione di pacchetti (*bundle*) d'istruzioni, conclusi per esempio dal bit di "fine pacchetto", come mostrato nella Figura 8.2(c). Il processore è in grado di effettuare il fetch e l'emissione di un intero pacchetto per volta. Spetta al compilatore preparare pacchetti d'istruzioni compatibili.

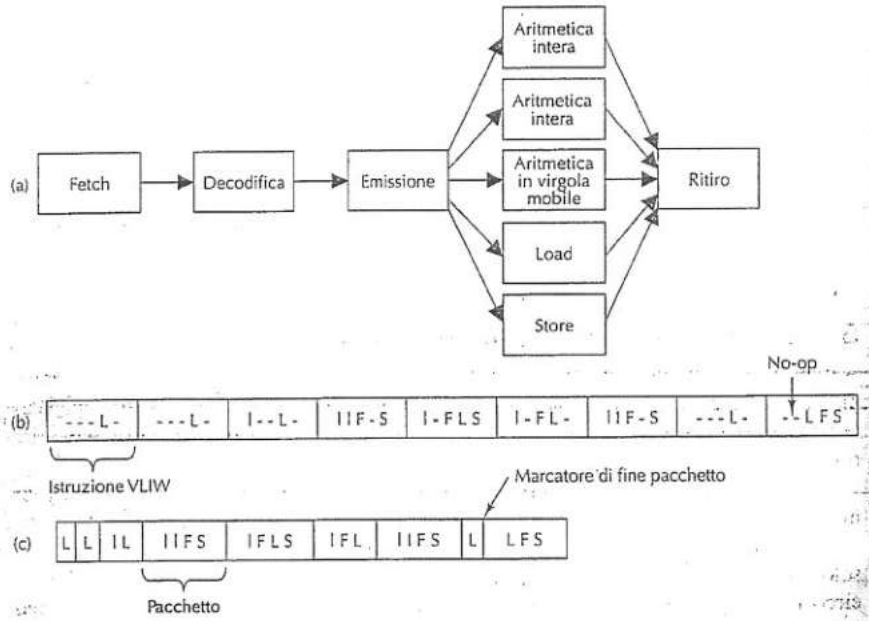


Figura 8.2 (a) Pipeline della CPU. (b) Sequenza d'istruzioni VLIW. (c) Flusso d'istruzioni con marcatori di pacchetto.

A tutti gli effetti, questa tecnica sposta dall'esecuzione alla compilazione il difficile compito di stabilire quali istruzioni possono essere avviate alla esecuzione contemporanea. Questa scelta consente di avere un hardware più semplice e veloce e, poiché un compilatore può far durare l'ottimizzazione per tutto il tempo necessario, permette di assemblare pacchetti d'istruzioni migliori di quanto potrebbe fare l'hardware durante l'esecuzione. Per contro, questo progetto comporta un cambio radicale nelle architetture

delle CPU che sarà difficile da far accettare, come dimostrato dalla scarsa accoglienza suscitata da Itanium, eccezione fatta per applicazioni di nicchia.

Val la pena notare qui che il parallelismo a livello delle istruzioni non è la sola forma di parallelismo di basso livello. C'è infatti il parallelismo a livello della memoria, secondo cui diverse operazioni di memoria vengono svolte contemporaneamente (Chou et al., 2004).

CPU VLIW TriMedia

Nel Capitolo 5 abbiamo studiato l'Itanium-2, un esempio di CPU VLIW. Analizziamo ora **TriMedia**, un processore VLIW molto diverso e progettato da Philips, la compagnia olandese che ha anche inventato i CD audio e i CD-ROM. TriMedia è stato pensato come processore integrato per applicazioni intensive di tipo audio, video o su immagini, nei lettori CD, DVD o Mp3, nei registratori CD o DVD, nelle console TV interattive, nelle macchine fotografiche e videocamere digitali, e così via. Proprio perché destinato a questo tipo di applicazioni, non sorprende che TriMedia differisca molto da Itanium-2, che è invece una CPU per uso generale concepita per i server di fascia alta.

TriMedia è un vero processore VLIW in cui ogni istruzione può contenere cinque **operazioni**. In condizioni ideali, a ogni ciclo di clock viene avviata l'esecuzione di un'istruzione e l'emissione delle cinque operazioni. Il clock ha una frequenza di 266 o 300 MHz, che però equivale a una frequenza cinque volte maggiore, visto che può eseguire cinque operazioni per ciclo. In seguito ci concentreremo sull'implementazione TM3260 di TriMedia (esistono anche altre versioni, le cui differenze sono però trascurabili).

La Figura 8.3 illustra un'istruzione tipica. Si va dalle istruzioni intere standard di 8, 16 e 32 bit, alle istruzioni in virgola mobile IEEE 754, fino alle istruzioni multimediali parallele. Grazie all'emissione di cinque istruzioni multimediali per ciclo, TriMedia è sufficientemente veloce da decodificare un flusso di dati proveniente da una videocamera senza ridurre la dimensione delle immagini, né la frequenza di acquisizione.

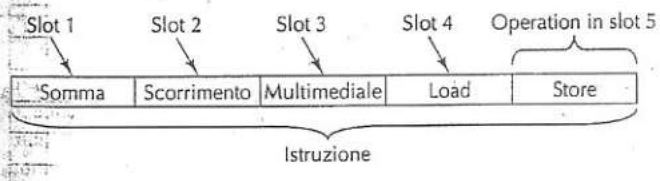


Figura 8.3 Tipica istruzione TriMedia, con cinque operazioni possibili.

TriMedia ha una memoria orientata al byte, con i registri di I/O mappati nello spazio di memoria. Le mezze parole (16 bit) e le parole intere (32 bit) devono essere allineate lungo le loro estremità (ovvero devono iniziare in byte di indirizzo pari o – per le parole intere – multiplo di 4). L'ordinamento dei byte può essere big-endian o little-endian, a seconda di un bit di PSW che può essere impostato dal sistema operativo. Questo bit ha

effetto solo sul modo in cui operano i trasferimenti di caricamento e memorizzazione tra i registri e la memoria. La CPU contiene una cache specializzata associativa a 8 vie, con linee di 64 byte sia per la cache delle istruzioni, sia per quella dei dati. La cache delle istruzioni è di 64 KB, quella dei dati è di 16 KB. Ci sono 128 registri di 32 bit per uso generale. Il registro `R0` è cablato al valore 0, `R1` a 1. I tentativi di modificare uno di questi registri hanno sulla CPU l'effetto di un infarto. I restanti 126 registri sono tutti equivalenti dal punto di vista funzionale e possono essere usati per qualsiasi scopo. Inoltre esistono quattro registri di 32 bit specializzati: il program counter, `PSW` e due registri legati agli interrupt. Infine, viene utilizzato un registro di 64 bit per contare i cicli dall'ultimo reset della CPU. A 300 MHz, ci vogliono circa 2000 anni perché il contatore vada in overflow. Il TriMedia TM3260 ha 11 diverse unità funzionali per svolgere le operazioni aritmetiche, logiche e di controllo del flusso, oltre a una per il controllo di cache che non prenderemo in considerazione. Le prime due colonne della Figura 8.4 indicano il nome dell'unità e ne danno una breve descrizione, la terza specifica il numero di copie hardware dell'unità e la quarta dà la latenza dell'unità, cioè il numero di cicli di clock che impiega per completare l'operazione. Notiamo qui che tutte le unità funzionali sono a pipeline, eccetto la unità VM (in virgola mobile) per la divisione/radice quadrata. La latenza indicata nella tabella stabilisce il tempo necessario alla disponibilità del risultato di un'operazione, ma a ogni ciclo è possibile iniziare una nuova operazione. Per esempio, tre istruzioni consecutive possono contenere ciascuna due caricamenti, così in un certo istante saranno in esecuzione sei operazioni di caricamento, ma a stadi differenti.

Infine, le ultime cinque colonne mostrano le possibili corrispondenze tra posizione delle operazioni nell'istruzione e le unità funzionali. Per esempio, l'operazione di confronto in virgola mobile deve occupare necessariamente il terzo posto di un'istruzione.

Unità	Descrizione	#	Latenza	1	2	3	4	5
Costante	Operazioni immediate	5	1	x	x	x	x	x
ALU intera	Aritmetica a 32 bit, operazioni booleane	5	1	x	x	x	x	x
Scorrimento	Scorrimento di più bit	2	1	x	x	x	x	x
Load/Store	Operazioni di memoria	2	3	x	x			
MUL Int/VM	Moltiplicazione intera e VM a 32 bit	2	3		x	x		
ALU VM	Aritmetica VM	2	3	x			x	
Confronto VM	Confronti VM	1	1			x		
sqrt/div VM	Divisione e radice quadrata VM	1	17		x			
Salti	Controllo del flusso	3	3		x	x	x	
ALU DSP	Aritmetica multimediale a due o quattro operandi (16 o 8 bit)	2	3	x		x		x
MUL DSP	Moltiplicazioni multimediali a due o quattro operandi (16 o 8 bit)	2	3		x	x		

Figura 8.4 Unità funzionali di TM3260; numero, latenza e posizioni che possono avere nelle istruzioni.

L'unità costante si usa per le operazioni immediate, come il caricamento in un registro di un numero contenuto nell'operazione stessa. L'ALU intera svolge la somma, la sottrazione e le usuali operazioni booleane, e inoltre impacchetta/spacchetta le operazioni. L'unità di scorrimento può traslare il contenuto di un registro di un certo numero di bit in entrambe le direzioni.

L'unità load/store effettua il fetch delle parole di memoria verso i registri o viceversa. TriMedia è essenzialmente una CPU RISC potenziata, e le operazioni normali lavorano sui registri, mentre soltanto l'unità load/store è usata per gli accessi alla memoria, con trasferimenti di 8, 16 o 32 bit.

L'unità per la moltiplicazione gestisce sia i prodotti tra interi, sia tra numeri in virgola mobile. Le tre unità successive gestiscono la somma/sottrazione in virgola mobile, i confronti, la radice quadrata e la divisione.

Le operazioni di salto sono eseguite dall'unità di salto. Dopo un salto c'è un ritardo di 3 cicli, perciò le tre istruzioni successive a un salto (fino a 15 operazioni) vengono eseguite sempre, anche nel caso di salti incondizionati.

Veniamo infine alle unità multimediali per la gestione delle speciali operazioni multimediali. Il termine **DSP** sta per *Digital Signal Processor* ("processore dei segnali digitali"), le cui funzioni sono svolte in questo caso proprio dalle operazioni multimediali. Prima di entrare nei dettagli di queste operazioni, segnaliamo che usano tutte l'**aritmetica satura** (*saturated arithmetic*) invece dell'aritmetica in complemento a due usata dalle operazioni intere. Quando un'operazione produce un risultato che non può essere espresso a causa di un overflow, non viene sollevata un'eccezione, né viene restituito un risultato inservibile, bensì si ottiene il numero valido più vicino a quello corretto. Per esempio, se si usano i numeri di 8 bit senza segno, la somma di 130 + 130 dà 255. A causa del fatto che non tutte le operazioni possono occupare tutti i posti, accade frequentemente che un'istruzione non contenga tutte e cinque le operazioni che può ospitare

Se un posto non è utilizzato, la sua dimensione viene ridotta per minimizzare lo spreco di spazio. Le operazioni presenti occupano 26, 34 o 42 bit. A seconda delle operazioni presenti, un'istruzione TriMedia può essere lunga dai 2 ai 28 byte, compresa l'intestazione di dimensione costante.

TriMedia non effettua controlli in fase d'esecuzione per verificare che le operazioni in un'istruzione siano compatibili. Se non lo sono, vengono eseguite comunque e producono un risultato errato. L'assenza del controllo è stata una scelta progettuale deliberata e intesa a risparmiare tempo e transistor. Il Core i7, durante l'esecuzione, svolge i controlli necessari per sincerarsi della compatibilità di tutte le operazioni superscalari, ma paga questa certezza in complessità, tempo e transistor. TriMedia si risparmia questa spesa affidando lo scheduling al compilatore, che dispone di tutto il tempo che vuole per ottimizzare con cura il posizionamento delle operazioni nelle istruzioni. Il rovescio della medaglia è che, se un'operazione necessita di un'unità funzionale non disponibile, l'intera istruzione va in stallo finché l'unità non torna disponibile.

Come nell'Itanium-2, le operazioni TriMedia sono predicative. Ogni operazione (a parte due eccezioni trascurabili) specifica un registro da esaminare prima dell'esecuzione dell'operazione; se il bit meno significativo del registro è asserito, l'istruzione viene

eseguita, viceversa viene saltata. A ognuna delle (al massimo) cinque operazioni viene attribuito un predicato individuale. Un esempio di operazione predicativa è

```
IF R2 IADD R4, R5 -> R8
```

che effettua il test su R2 e, se il suo bit meno significativo vale 1, somma R4 a R5 e salva il risultato in R8. Un'operazione diventa incondizionata se usa R1 (che vale sempre 1) come registro predicativo. Se usa R0 (che vale sempre 0) diventa una no-op.

Le operazioni multimediali di TriMedia possono essere ripartite nei 15 gruppi della Figura 8.5. Molte operazioni coinvolgono tagli (*clipping*) mediante la specifica di un operando e di un intervallo; il taglio forza l'operando all'interno dell'intervallo, ovvero se il suo valore cade fuori dall'intervallo gli attribuisce il valore di uno degli estremi. Possono essere tagliati operandi di 6, 16 o 32 bit. Per fare un esempio, i tagli di 40 e di 340 in un intervallo da 0 a 255 restituiscono rispettivamente 40 e 255. Il gruppo dei tagli effettua appunto operazioni di taglio.

Gruppo	Descrizione
Tagli	Tagli di 4 byte o di 2 mezze parole
Valore assoluto DSP	Valore assoluto saturato con segno
Somma DSP	Somma saturata con segno
Sottrazione DSP	Sottrazione saturata con segno
Moltiplicazione DSP	Moltiplicazione saturata con segno
Min, max	Minimo o massimo di quattro coppie di byte
Confronti	Confronti byte per byte di due registri
Scorrimento	Scorrimento di una coppia di operandi di 16 bit
Somma dei prodotti	Somma con segno di prodotti di 8 o 16 bit
Fusione, impacchettamento, scambio	Manipolazione di byte o mezze parole
Media di operandi di quattro byte	Media senza segno di operandi di quattro byte, byte per byte
Media di byte	Media senza segno di quattro elementi, byte per byte
Moltiplicazione di byte	Moltiplicazione senza segno a 8 bit
Stima del movimento	Somma senza segno di valori assoluti di differenze di 8 bit con segno
Miscellanea	Altre operazioni aritmetiche

Figura 8.5 Principali gruppi di operazioni di TriMedia.

I quattro gruppi successivi della Figura 8.5 eseguono le operazioni indicate su operandi di varie dimensioni, tagliando il risultato in un certo intervallo. Il gruppo min, max esamina due registri e, per ogni byte, trova il valore minimo o massimo. Allo stesso modo, il gruppo dei confronti considera due registri come quattro coppie di byte ed effettua i confronti su ogni coppia.

Le operazioni multimediali non sono eseguite quasi mai su interi di 32 bit perché le immagini sono fatte di pixel RGB, definiti da 8 bit per ciascuno dei colori rosso, verde e blu. Durante l'elaborazione di un'immagine (per esempio la sua compressione) questa viene rappresentata in genere da tre componenti, uno per ogni colore (spazio RGB) o in una forma logicamente equivalente (spazio YUV, descritto nel seguito del capitolo). Comunque sia, gran parte dell'elaborazione si svolge su array rettangolari contenenti interi di 8 bit senza segno.

TriMedia è dotato di molte operazioni ideate specificatamente per l'elaborazione efficiente di interi senza segno di 8 bit. Consideriamo il semplice esempio del vertice superiore sinistro di un array di valori di 8 bit, collocati in memoria (*big-endian*) della Figura 8.6(a). I 4 × 4 blocchi del vertice contengono 16 valori di 8 bit, etichettati da *A* a *P*. Se si vuole trasporre l'immagine, per produrre il risultato della Figura 8.6(b), come si può fare?

Una possibilità prevede l'utilizzo di 12 operazioni per caricare i byte nei registri, seguite da 12 operazioni per memorizzare i byte nella loro posizione corretta (i quattro byte lungo la diagonale non si spostano durante la trasposizione). Il problema è che questo approccio richiede 24 operazioni (lunghe e lente) che accedono alla memoria.

In alternativa si può cominciare con quattro operazioni di caricamento di una parola per volta nei quattro registri da R2 a R5, come mostrato nella Figura 8.6(c). Poi si ottengono le quattro parole di output della Figura 8.6(d) tramite operazioni di mascheratura e scorrimento. Infine, le parole sono pronte per essere salvate in memoria. Sebbene questo metodo riduca il numero di accessi in memoria da 24 a 8, le mascherature e gli scorrimenti sono costosi a causa del gran numero di operazioni richieste per estrarre e inserire ogni byte nella sua posizione corretta.

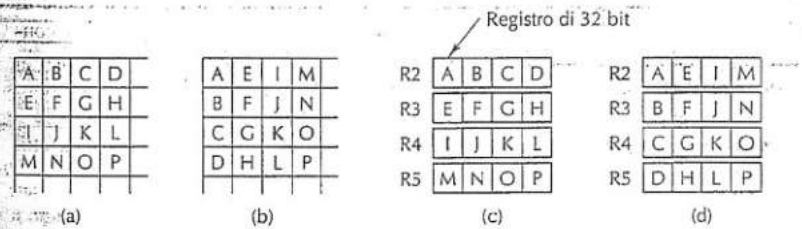


Figura 8.6 (a) Un array di elementi di 8 bit. (b) L'array trasposto. (c) L'array originale memorizzato in quattro registri. (d) L'array trasposto nei quattro registri.

TriMedia consente di raggiungere una soluzione migliore delle precedenti. Si comincia con il fetch delle quattro parole nei registri, però, per costruire l'output, invece delle mascherature e degli scorrimenti, si usano operazioni speciali per l'estrazione e l'inserzione di byte nei registri. In definitiva, la trasposizione può essere ottenuta con 8 accessi alla memoria e 8 operazioni multimediali speciali. Il codice comincia con un'istruzione contenente due `load` in posizione 4 e 5, per il caricamento nei registri R2 e R3,

seguita da un'altra istruzione per caricare R4 e R5. Queste due istruzioni possono usare le altre posizioni per altri scopi. Al termine di tutti i caricamenti è possibile impacchettare le otto operazioni speciali per la costruzione dell'output in due istruzioni, seguite da altre due istruzioni per la memorizzazione. In totale servono solo sei istruzioni, in cui 14 delle 30 posizioni complessive sono ancora disponibili per ospitare altre operazioni. In effetti, il compito può essere svolto complessivamente con l'equivalente di circa tre istruzioni. Le altre operazioni multimediali sono altrettanto efficienti. Grazie alle sue potenti operazioni e alla capienza delle sue istruzioni, TriMedia è molto efficiente per la tipologia di calcolo richiesta dall'elaborazione multimediale.

8.1.2 Multithreading nel chip

Tutte le moderne CPU a pipeline presentano un problema: quando un riferimento in memoria fallisce nelle cache di primo e di secondo livello, bisogna aspettare molto tempo prima che la parola richiesta (e la corrispondente linea di cache) sia caricata nella cache, e così nel frattempo la pipeline è in stallo. Il **multithreading nel chip** costituisce un modo per trattare questa situazione perché, in una certa misura, si riescono a mascherare questi stalli consentendo alla CPU di gestire contemporaneamente più thread di controllo. Infatti, se il thread 1 è bloccato, la CPU ha ancora la possibilità di eseguire il thread 2 e di mantenere l'hardware impegnato.

Per quanto si tratti di un'idea molto semplice, ne esistono diverse varianti che ci accingiamo a esaminare. Il primo approccio si chiama **multithreading a grana fine** *(fine-grained multithreading)* ed è illustrato nella Figura 8.7, che mostra una CPU capace di emettere un'istruzione per ciclo di clock. Nelle Figure 8.7(a)-(c) osserviamo l'attività dei thread *A*, *B* e *C*, lungo 12 cicli di macchina. Durante il primo ciclo, *A* esegue l'istruzione *A1*, che viene completata in un ciclo, quindi nel secondo comincia l'esecuzione di *A2*. Sfortunatamente questa istruzione provoca un fallimento nella cache di primo livello e così si perdono due cicli per recuperare l'istruzione dalla cache di secondo livello. Il thread riprende dal ciclo 5. Anche i thread *B* e *C* di tanto in tanto vanno in stallo, come illustrato nella figura. Secondo questo modello, quando un'istruzione va in stallo, le istruzioni successive non possono essere emesse. Naturalmente, alle volte è tuttavia possibile emettere altre istruzioni se si dispone di uno scoreboard più sofisticato, ma qui ignoriamo questa evenienza.

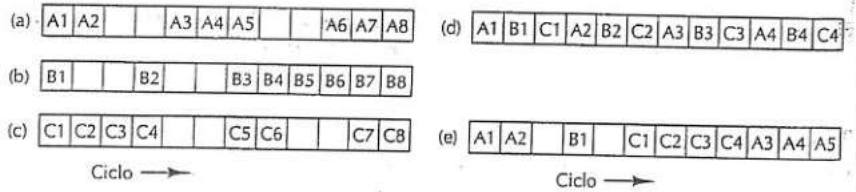


Figura 8.7 (a)-(c) Tre thread: le caselle vuote indicano che il thread è in stallo perché in attesa di dati dalla memoria. (d) Multithreading a grana fine. (e) Multithreading a grana grossa.

Il multithreading a grana fine nasconde gli stalli grazie all'esecuzione a turno dei thread, con una commutazione a ogni ciclo, come mostra la Figura 8.7(d). Quando arriva il quarto ciclo l'operazione di memoria avviata da *A1* è ormai stata completata e l'istruzione *A2* può essere eseguita, anche se richiede il risultato di *A1*. Nell'esempio ogni stallo dura al massimo due cicli, perciò le operazioni vengono sempre completate in tempo visto che ci sono tre thread. Se ci fossero stalli di tre cicli, avremmo bisogno di quattro thread per assicurare la continuità dell'attività, e così via.

Dal momento che non vi è alcuna relazione tra i thread, ciascuno ha bisogno del proprio insieme di registri. All'emissione di un'istruzione è necessario accludere all'istruzione stessa un puntatore al suo insieme di registri così che, se viene referenziato un registro, l'hardware possa sapere quale registro usare. Per questa ragione, il numero massimo di thread che può essere eseguito in parallelo è stabilito a priori in fase di progettazione del chip.

Le operazioni di memoria non sono l'unica causa di stallo. A volte un'istruzione aspetta il risultato di un'istruzione precedente non ancora completata, altre volte un'istruzione non può essere avviata perché segue un salto condizionato la cui destinazione non è ancora nota. Come regola generale, se la pipeline ha k stadi e ci sono almeno k thread in esecuzione a turno, nella pipeline non ci sarà mai più di un'istruzione per thread in esecuzione e perciò non si possono verificare conflitti. In queste condizioni la CPU può girare a pieno ritmo senza stalli.

Ovviamente potrebbero non esserci tanti thread quanti sono gli stadi della pipeline, perciò alcuni progettisti preferiscono un approccio differente, detto **multithreading a grana grossa** *(coarse-grained multithreading)*, illustrato nella Figura 8.7(e). In questo caso *A* si avvia e continua a emettere istruzioni finché non va in stallo, causando lo spreco di un ciclo. A quel punto l'esecuzione viene commutata su *B1* ma, poiché la prima istruzione di *B* va subito in stallo, si verifica un'altra commutazione di thread e al ciclo 6 va in esecuzione *C1*. Visto che si perde un ciclo a ogni stallo, il multithreading a grana grossa è potenzialmente meno efficiente di quello a grana fine, ma presenta il vantaggio di richiedere meno thread per mantenere la CPU occupata. Il multithreading a grana grossa è preferibile in tutte quelle situazioni in cui c'è un numero insufficiente di thread attivi, perché garantisce di trovarne almeno uno da mandare in esecuzione.

Anche se abbiamo descritto il multithreading a grana grossa come una commutazione sugli stalli, non è questa l'unica alternativa. Un'altra possibilità è di effettuare la commutazione immediatamente tutte le volte in cui un'istruzione potrebbe causare uno stallo: si pensi ai caricamenti, alle memorizzazioni e ai salti, ancor prima di scoprire se lo stallo si verificherà davvero. Secondo questa strategia la commutazione occorre in anticipo (non appena l'istruzione viene decodificata) e potrebbe così far evitare i cicli morti. È un po' come dire: "andiamo avanti finché potremmo avere un problema, al che commutiamo giusto in tempo". Così facendo, il multithreading a grana grossa, con le sue frequenti commutazioni, somiglia un po' di più a quello a grana fine.

Indipendentemente dal tipo di multithreading usato, è necessario mantenere traccia dell'appartenenza delle operazioni ai thread. Nel caso del multithreading a grana fine l'unica da poter rintracciare la sua identità mentre attraversa i diversi stadi della pipeline. Nel caso del multithreading a grana grossa si può far di meglio: svuotare la pipeline

a ogni commutazione di thread. In tal modo c'è sempre un solo thread nella pipeline e così la sua identità non è mai messa in dubbio. È evidente che questa soluzione ha senso solo se il tempo che intercorre tra due commutazioni è molto più lungo del tempo di svuotamento della pipeline.

Fin qui abbiamo presupposto che la CPU possa emettere una sola istruzione per ciclo, ma abbiamo già visto che le CPU moderne ne possono emettere di più. Nella Figura 8.8 supponiamo che la CPU possa emettere due istruzioni per ciclo, pur conservando la regola che, se un'istruzione va in stallo, le successive non possono essere emesse. La Figura 8.8(a) mostra il funzionamento di una CPU superscalare a doppia emissione e con multithreading a grana fine. Le prime due istruzioni del thread *A* possono essere emesse nel primo ciclo, ma nel caso di *B* incontriamo subito un problema nel ciclo successivo, e così può essere emessa una sola istruzione, e così via.

La Figura 8.8(b) mostra il funzionamento di una CPU a doppia emissione con multithreading a grana grossa, ma questa volta con uno schedulatore statico che non introduce un ciclo morto dopo lo stallo di un'istruzione. Praticamente, i thread si succedono a turno e la CPU emette due istruzioni per ogni thread finché non incontra uno stallo, nel qual caso commuta al thread successivo all'inizio del ciclo seguente.

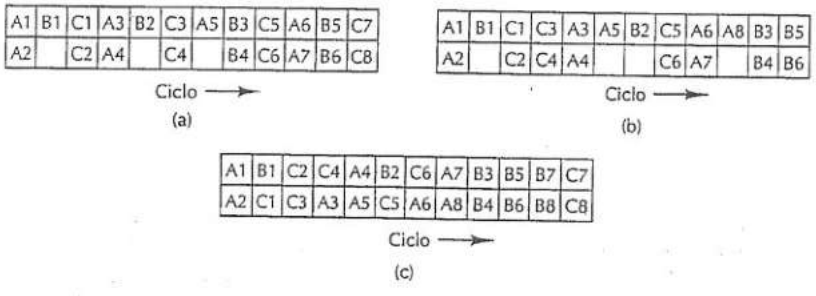


Figura 8.8 Multithreading in una CPU superscalare a doppia emissione. (a) Multithreading a grana fine. (b) Multithreading a grana grossa. (c) Multithreading simultaneo.

C'è una terza possibilità di multithreading con le CPU superscalari, il **multithreading simultaneo**, illustrato nella Figura 8.8(c). Lo si può considerare un raffinamento del multithreading a grana grossa, in cui ciascun thread emette due istruzioni per ciclo fintanto che può, altrimenti, non appena raggiunge uno stallo, viene emessa immediatamente un'istruzione del thread che segue affinché la CPU resti pienamente impegnata. Il multithreading simultaneo aiuta anche a mantenere occupate le unità funzionali. Quando un'istruzione non può essere avviata perché necessita di un'unità funzionale occupata, si può scegliere al suo posto un'istruzione di un altro thread. Nella figura supponiamo che *B8* vada in stallo al ciclo 11, così *C7* viene avviata al ciclo 12.

Per maggiori informazioni sul multithreading si faccia riferimento a (Gebhart et al., 2011) e (Wing-kei et al., 2011).

Hyperthreading nel Core i7

Lo studio astratto del multithreading ci consente ora di affrontare a un esempio pratico: quello del Core i7. Nei primi anni 2000 processori come il Pentium 4 non erano in grado di offrire gli incrementi di prestazioni di cui Intel aveva bisogno per mantenere il livello di vendite. Quando il Pentium 4 era già in produzione gli architetti di Intel cominciarono a riflettere sul modo di velocizzare il processore senza modificare l'interfaccia con il programmatore, la qual cosa sarebbe stata inaccettabile. Furono individuate subito cinque ipotesi.

1. Incrementare la velocità di clock.
2. Posizionare due CPU nel chip.
3. Aggiungere unità funzionali.
4. Allungare la pipeline.
5. Usare il multithreading.

Un modo ovvio per migliorare le prestazioni consiste nell'aumento della velocità del clock senza cambiare nient'altro. È una soluzione relativamente semplice e ben fondata, perciò ogni nuovo chip in genere supera di poco la velocità del suo predecessore. Sfortunatamente, l'incremento delle velocità di clock porta con sé due svantaggi che ne limitano la tollerabilità. Un clock più veloce assorbe più energia, un problema molto grande per i computer portatili e in genere per tutti quelli alimentati a batteria. In secondo luogo, più energia in ingresso vuol dire più calore da dissipare.

La sistemazione di due CPU in un chip è abbastanza semplice, ma equivale circa a raddoppiare l'area del chip se ciascun processore è dotato di cache propria; la riduzione del numero di chip per unità di superficie di un fattore due non fa che raddoppiare i costi unitari di produzione. Se i due chip condividono una cache comune grande quanto quella originale, la superficie del chip non viene raddoppiata, ma la quantità di cache per CPU è dimezzata e ciò riduce le prestazioni. Inoltre, se le applicazioni dei server di fascia alta sono spesso in grado di trarre il massimo giovamento dalla presenza di più CPU, le applicazioni dei desktop non hanno abbastanza parallelismo da giustificarne due.

Anche l'aggiunta di unità funzionali è relativamente semplice, ma bisogna raggiungere il giusto equilibrio. È inutile disporre di 10 ALU se il chip non è in grado di passare loro le istruzioni abbastanza velocemente da tenerle occupate.

L'allungamento della pipeline tramite aggiunta di nuovi stadi, ciascuno destinato allo svolgimento di un compito più piccolo in un tempo più breve, può incrementare le prestazioni, ma aumenta anche gli effetti negativi delle predizioni errate sui salti, dei fallimenti di cache, degli interrupt e in genere di tutti quei fattori che interrompono il normale flusso esecutivo. Inoltre, per poter sfruttare pienamente la maggiore lunghezza della pipeline, è necessario un clock più veloce, il che implica un più elevato consumo di energia e un incremento di produzione di calore.

Resta infine l'introduzione del multithreading, il cui valore aggiunto sta nel permettere che un secondo thread utilizzi l'hardware che sarebbe rimasto altrimenti inattivo. A seguito di alcuni esperimenti, si è stabilito che il supporto del multithreading, a fronte

di un incremento del 5% della superficie del chip, produce un miglioramento delle prestazioni del 25% per molte applicazioni ed è perciò una buona scelta. La prima CPU Intel a usare il multithreading è stata Xeon nel 2002; il multithreading è stato aggiunto in seguito al Pentium 4, dalla versione a 3,06 GHz fino alle versioni più veloci di processori Pentium, tra cui il Core i7. Intel ha dato il nome di **hyperthreading** all'implementazione del multithreading nei suoi processori.

Alla sua base c'è l'idea di consentire a due thread (o anche a due processi, dal momento che la CPU non può distinguere tra thread e processi) di essere eseguiti contemporaneamente. Dal punto di vista del sistema operativo, un Core i7 con hyperthreading somiglia a un biprocessore in cui le CPU condividono cache e memoria principale. Il sistema operativo seleziona i thread per l'esecuzione in modo indipendente; se ci sono due applicazioni in esecuzione nello stesso istante, il sistema operativo può farle eseguire in parallelo. Per esempio, se un demone di posta riceve o spedisce email in background[1] mentre un utente interagisce con un altro programma, i due programmi possono essere eseguiti contemporaneamente proprio come se ci fossero due CPU disponibili.

Il software applicativo progettato per l'esecuzione con thread multipli può avvalersi di entrambe le CPU virtuali. Per esempio, i programmi di montaggio video in genere permettono all'utente di specificare alcuni filtri da applicare ai fotogrammi di un certo intervallo. Questi filtri possono agire su luminosità, contrasto, bilanciamento del bianco e altre proprietà di ciascun fotogramma. Il programma può decidere di affidare i fotogrammi pari a una CPU e quelli dispari all'altra, e le due CPU possono lavorare in parallelo.

Poiché i thread condividono tutte le risorse hardware, si rende necessaria una strategia per la gestione della condivisione. Intel ha identificato quattro strategie utili per la condivisione di risorse associata all'hyperthreading: duplicazione e ripartizione di risorse, condivisione a soglia oppure totale. Vediamole una per volta.

Cominciamo dalle risorse che vengono duplicate per i thread. Per esempio, poiché ogni thread ha il proprio controllo del flusso, si è resa necessaria l'aggiunta di un secondo program counter. È stata duplicata anche la tabella che mappa i registri architetturali (`EAX`, `EBX` e così via) nei registri fisici, nonché il controllore di interrupt, visto che i thread possono essere interrotti in modo indipendente.

Poi c'è la **condivisione ripartita delle risorse** (*partitioned resource sharing*), secondo cui le risorse hardware sono divise rigidamente tra i thread. Per esempio, se la CPU ha una coda tra due stadi funzionali della pipeline, metà delle posizioni della coda potrebbero essere dedicate al thread 1, l'altra metà al thread 2. La ripartizione delle risorse è facile da ottenere, non richiede informazioni addizionali e fa sì che i thread non vadano in conflitto. Se tutte le risorse sono ripartite è come avere due CPU separate. La ripartizione ha anche un aspetto negativo: può succedere facilmente che un thread non usi alcune delle sue risorse che farebbero invece comodo a un altro thread, che però non

1 Per elaborazione in *background* ("sullo sfondo") si intende l'elaborazione a bassa priorità di programmi che non hanno bisogno di interazioni con l'utente, e che quindi sono spesso a lui invisibili (*N.d.T.*).

ha diritto di usarle. Di conseguenza, alcune risorse che sarebbero potute servire a una qualche attività restano invece inoperose.

La **condivisione totale delle risorse** (*full resource sharing*) è il contrario della condivisione ripartita. Secondo questo schema, ogni thread può acquisire tutte le risorse di cui ha bisogno: il primo che arriva si serve da sé. Immaginiamo però due thread, uno veloce che fa solo somme e sottrazioni, uno più lento che svolge moltiplicazioni e divisioni. Se le istruzioni sono caricate dalla memoria con un ritmo maggiore rispetto alla velocità di esecuzione delle moltiplicazioni e delle divisioni, il numero d'istruzioni arretrate del thread lento (che si trova nella coda d'attesa per l'inserimento nella pipeline) crescerà di continuo. A un certo punto le istruzioni arretrate riempiranno completamente la coda d'attesa, costringendo il thread veloce ad arrestarsi per mancanza di spazio nella coda delle istruzioni. La condivisione totale delle risorse risolve il problema di quelle inattive richieste da un thread, ma crea un problema nuovo in cui un thread può impossessarsi di così tante risorse da rallentare l'altro o addirittura da farlo fermare.

Uno schema intermedio è la **condivisione a soglia** (*threshold sharing*), secondo cui un thread può acquisire le risorse dinamicamente (non c'è ripartizione fissa), ma solo fino a un certo valore massimo. Questo approccio permette una certa flessibilità ed esclude il pericolo che un thread vada in *starvation*[2] a causa della sua incapacità di procurarsi le risorse. Per esempio, se nessun thread può acquisire più dei 3/4 della coda delle istruzioni, il thread più veloce potrà sempre proseguire la propria esecuzione, qualsiasi cosa faccia il thread più lento.

L'hyperthreading del Core i7 usa strategie di condivisione differenti a seconda delle risorse, nel tentativo di far fronte ai problemi cui si è accennato. La duplicazione è utilizzata per le risorse di cui i thread hanno sempre bisogno, come il program counter, la mappa dei registri e il controllore di interrupt. La duplicazione di queste risorse richiede un incremento della superficie del chip del 5%, un prezzo accettabile per il multithreading. Le risorse che sono così abbondanti per cui non si corre il rischio di acquisizione da parte di un solo thread, come le linee di cache, sono totalmente condivise in maniera dinamica. Invece le risorse che controllano il funzionamento della pipeline, come le varie code presenti al suo interno, sono ripartite esattamente tra i due thread. La Figura 8.9 illustra la pipeline principale della microarchitettura Sandy Bridge usata nel Core i7, laddove le caselle grigie e quelle bianche indicano l'allocazione delle risorse tra il thread grigio e quello bianco.

Nella figura notiamo che tutte le code sono ripartite a metà tra i thread, perciò nessuno dei due thread può soffocare l'altro. Anche i registri di allocazione e di rinomina sono ripartiti. Lo scheduler è condiviso dinamicamente con una soglia, per evitare che un thread reclami per sé l'intera coda. Gli altri stadi della pipeline sono totalmente condivisi.

2 *To starve* vuol dire "morire di fame"; in questo contesto indica l'inattività involontaria di un thread dovuta all'esaurimento delle risorse (*N.d.T.*).

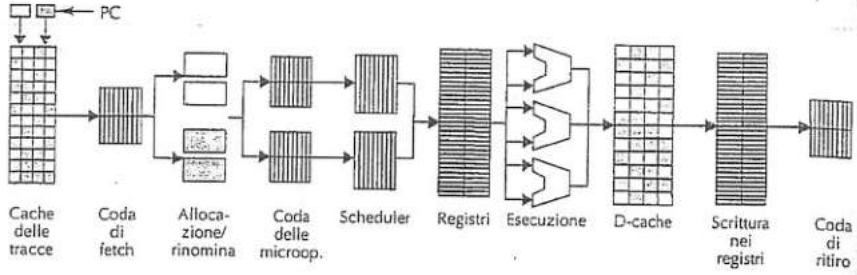


Figura 8.9 Condivisione delle risorse tra i thread della microarchitettura del Core i7.

C'è da dire che il multithreading non è la panacea, ma che esiste anche un suo aspetto negativo. Se la ripartizione delle risorse è facile da implementare, la loro condivisione dinamica, specie se basata su soglia, richiede lavoro di amministrazione per monitorarne l'uso in fase d'esecuzione. Oltre a ciò, esistono delle situazioni in cui i programmi funzionano molto peggio con il multithreading che non senza. Per esempio, si immaginino due thread che, per funzionare bene, hanno bisogno entrambi di 3/4 della cache. Se eseguiti separatamente, girano senza problemi con pochi (costosi) fallimenti di cache. Se eseguiti insieme, ciascuno di loro provoca molti fallimenti di cache e il risultato complessivo potrebbe essere di gran lunga peggiore.

Si possono trovare altre informazioni sul multithreading e sulle sue implementazioni nei processori Intel in (Gerber e Binstock, 2004) e (Gepner et al., 2011).

8.1.3 Multiprocessori in un solo chip

Anche se il multithreading consente un incremento significativo delle prestazioni a un costo contenuto, per certi tipi di applicazioni si richiedono prestazioni molto superiori di quelle che può assicurare il multithreading. A tal fine è diretto lo sviluppo dei multiprocessori (chip con due o più CPU), che suscita particolare interesse nel mercato dei server di fascia alta e dell'elettronica di consumo. Diamo una rapida scorsa a entrambi i settori.

Multiprocessori omogenei in un solo chip

Grazie ai progressi della tecnologia VLSI è oggi possibile inserire in un chip due o più CPU potenti. Queste CPU si definiscono multiprocessori perché condividono le stesse cache, di primo e secondo livello, e la memoria principale (come illustrato nel Capitolo 2). Comunemente trovano applicazione nelle server farm che raggruppano molti server web. La capacità di far coabitare due CPU in un unico sistema, condividendo non solo la memoria, ma anche il disco e le interfacce di rete, permette spesso di raddoppiare le prestazioni del server senza raddoppiarne i costi (anche se il costo della CPU raddoppia, il suo prezzo è solo una frazione di quello dell'intero sistema).

Esistono due tipologie predominanti per il progetto di multiprocessori in un solo chip di piccole dimensioni. La prima è mostrata nella Figura 8.10(a): c'è davvero un solo chip, ma è dotato di una duplice pipeline che gli permette di raddoppiare potenzialmente il throughput. La Figura 8.10(b) mostra il secondo tipo di progetto: nel chip ci sono due *core* separati, ciascuno contenente un'intera CPU. Un **core** ("nucleo, cuore") è un grosso circuito, quale una CPU, un controllore di I/O o una cache, che può essere inserito su di un chip in maniera modulare, spesso uno di fianco all'altro.

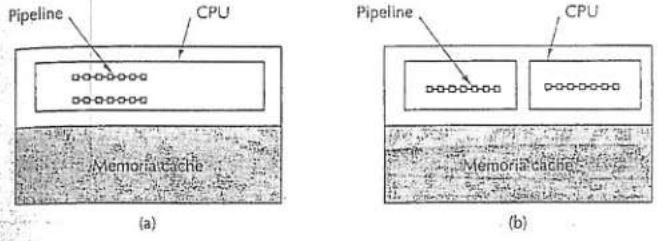


Figura 8.10 Multiprocessori a singolo chip. (a) Un chip con pipeline duplice. (b) Un chip con due core.

Il primo progetto permette la condivisione di alcune risorse, come le unità funzionali, così che una CPU possa sfruttare le risorse inutilizzate da parte dell'altra CPU. Questo approccio però richiede la riprogettazione del chip e non si applica altrettanto bene al crescere del numero di CPU. Per contro, l'inserzione di due o più core CPU sullo stesso chip è relativamente facile.

L'argomento multiprocessori sarà trattato ancora in questo capitolo. Anche se l'analisi sarà focalizzata sui multiprocessori costruiti a partire da chip con CPU singola, si può estendere in larga misura anche ai chip con più CPU.

Il multiprocessore a singolo chip Core i7

La CPU Core i7 è un multiprocessore a singolo chip costruito con quattro (o più) core su un singolo stampo di silicio. L'organizzazione ad alto livello del Core i7 è mostrata nella Figura 8.11.

Ogni processore del Core i7 ha tre cache private. Due di primo livello per dati e istruzioni e una di secondo, unificata. I processori sono connessi alle cache private mediante connessioni dedicate punto a punto. Il livello successivo della gerarchia di memoria è la cache dati L3 unificata e condivisa.

Le cache L2 sono connesse alla cache L3 tramite una **rete ad anello**. Una richiesta di comunicazione che entra nella rete viene girata al nodo successivo della rete, dove si verifica se la richiesta ha raggiunto il suo nodo destinazione. Il processo si ripete fino a quando la richiesta arriva al nodo destinazione oppure ritorna al nodo origine (caso in cui il nodo destinazione non esiste). Il vantaggio della rete ad anello è che si tratta di un modo economico per avere alte larghezze di banda, al costo però di una maggior latenza per il passaggio della richiesta di nodo in nodo. La rete ad anello del Core i7 svolge due compiti principali. Prima di tutto offre un modo per spostare richieste di memoria e di

I/O tra le cache e il processore. Inoltre, implementa i controlli necessari ad assicurare che ogni processore abbia sempre una visione coerente della memoria. Impareremo di più su questi controlli di coerenza più avanti nel capitolo.

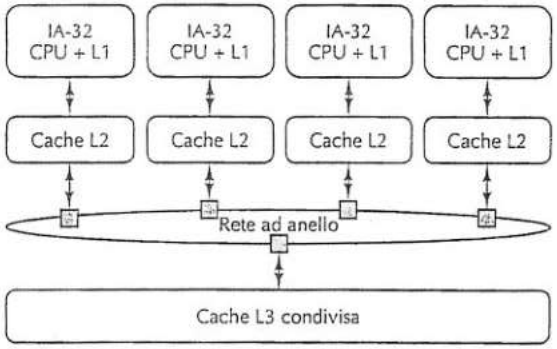


Figura 8.11 Architettura multiprocessore a singolo chip del Core i7.

Multiprocessori eterogenei in un solo chip

Una gamma completamente diversa di multiprocessori in un solo chip è costituita dai sistemi integrati e in special modo dall'elettronica di consumo per contenuti audiovisivi, come apparecchiature TV, lettori DVD, videocamere, console per i videogiochi, telefoni cellulari e così via. Si tratta di sistemi che devono garantire prestazioni impegnative nel rispetto di forti vincoli. Anche se questi dispositivi sembrano a prima vista diversi, in realtà molti di loro non sono altro che piccoli calcolatori dotati di una o più CPU, di memorie, di controllori di I/O e di dispositivi di I/O specifici. Un cellulare, per esempio, contiene una CPU, una memoria, una piccola tastiera, un microfono, un altoparlante e una connessione senza fili alla rete, il tutto in un piccolo involucro.

Si consideri un lettore DVD portatile. Il computer al suo interno deve gestire le funzioni seguenti.

1. Controllo di un servomeccanismo economico e poco affidabile per il puntamento della testina.
2. Conversione da analogico a digitale.
3. Correzione degli errori.
4. Decodifica crittografica e gestione dei diritti digitali.
5. Decompressione video MPEG-2.
6. Decompressione audio.
7. Codifica dell'output per i sistemi televisivi NTSC, PAL o SECAM.

Tutte queste azioni vanno svolte rigorosamente in tempo reale, con vincoli circa la qualità del servizio, l'assorbimento di energia, la dissipazione del calore, le dimensioni, il peso e il prezzo del dispositivo.

I CD, i DVD e i Blu-Ray contengono una lunga spirale d'informazioni (analogamente alla Figura 2.25 relativa ai CD). In questa sezione parleremo dei DVD, perché sono ancora più comuni dei Blu-Ray. Questi sono comunque molto simili ai DVD, eccetto per l'uso di MPEG-4 al posto di MPEG-2 per la codifica. In tutti i dispositivi ottici la testina di lettura deve puntare alla spirale con cura mentre il disco ruota. Si riesce a contenere il prezzo del dispositivo grazie alla progettazione di un meccanismo relativamente semplice e al controllo software della posizione della testina. Il segnale analogico proveniente dalla testina deve essere digitalizzato prima della sua elaborazione. Dopo la conversione è necessaria una pesante correzione degli errori perché i DVD ne contengono molti che devono essere corretti via software. La compressione video usa lo standard MPEG-2 che richiede un calcolo abbastanza complesso (simile alla trasformata di Fourier) per la decompressione. La compressione audio si avvale di un modello psicologico dell'udito e anch'esso richiede calcoli sofisticati in fase di decompressione. Infine, audio e video devono essere trasformati in un formato adatto al televisore e che varia da paese a paese: NTSC, PAL o SECAM. Non sorprende quindi come sia semplicemente impossibile svolgere tutte queste attività in tempo reale, via software e su CPU per uso generale. Ciò di cui c'è bisogno è un multiprocessore eterogeneo contenente più core, ciascuno specializzato per un certo compito. La Figura 8.12 presenta un esempio di lettore DVD.

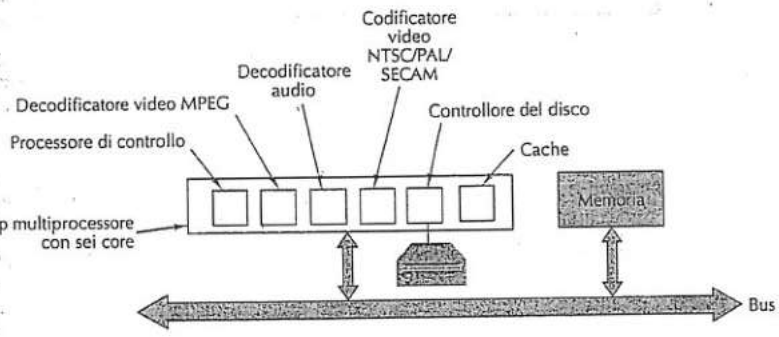


Figura 8.12 Struttura logica dei lettori DVD contenente un multiprocessore eterogeneo con vari core dedicati.

Le funzioni svolte dai core della Figura 8.12 sono tutte diverse; ciascun core è stato progettato con cura per svolgere il proprio compito al meglio e costare il meno possibile. Per esempio, il video dei DVD è compresso con **MPEG-2** (da *Motion Picture Experts Group*, il gruppo di esperti che lo ha inventato). Si procede scomponendo ogni fotogramma (*frame*) in blocchi di pixel e applicando una complicata trasformazione

matematica a ogni blocco. Un fotogramma può essere fatto interamente di blocchi trasformati, oppure si può specificare che un determinato blocco sia uguale a un blocco del fotogramma precedente, localizzato a un offset (Δx, Δy) rispetto alla sua posizione attuale, fatta eccezione per una manciata di pixel cambiati. Un calcolo di questo tipo sarebbe molto lento via software, ma è possibile costruire dei circuiti di decodifica MPEG-2 che possono svolgerlo in modo abbastanza rapido. Analogamente, anche la decodifica audio e la ricodifica composita audio-video del segnale (per rispettare gli standard televisivi mondiali) possono essere svolte con maggior successo da processori dedicati. Queste considerazioni giustificano largamente la progettazione di chip multiprocessori contenenti core specializzati per applicazioni audiovisive. Il processore di controllo è una CPU programmabile di uso generale, perciò il chip multiprocessore può essere usato anche per altre applicazioni simili.

Il telefono cellulare è un altro dispositivo che necessita di un multiprocessore eterogeneo per il suo funzionamento. I modelli attuali possono includere macchine fotografiche, videocamere, giochi, navigatori web, lettori di posta, ricevitori radio di segnali satellitari digitali; essi si basano sulla tecnologia cellulare (CDMA o GSM, a seconda del paese) o su Internet senza fili (IEEE 802.11, detta anche tecnologia WiFi). I modelli futuri saranno dotati di tutti questi dispositivi. Gli apparecchi acquistano sempre più funzionalità, gli orologi diventano mappe basate su GPS, gli occhiali diventano radio e la richiesta di multiprocessori eterogenei continuerà a crescere.

Verrà presto il giorno in cui i chip più grandi avranno decine di miliardi di transistor. Si tratterebbe di una dimensione troppo grande perché si possa progettare a partire dai circuiti elementari. Lo sforzo umano richiesto sarebbe tale da rendere un chip obsoleto ancor prima di essere ultimato. L'unico modo di procedere è usare i core (di fatto delle librerie), ciascuno contenente un sottoassemblato abbastanza grande, e collegarli tramite il montaggio su un chip. Ai progettisti resta solo da decidere quale core CPU usare per il processore di controllo e quali processori specializzati associargli per assisterlo. Spostare il carico di lavoro sul software che gira sul processore di controllo rallenta il sistema, ma consente chip più piccoli (e più economici). La presenza di processori diversi per l'elaborazione audio e video aumenta la superficie del chip e fa lievitare i costi, ma produce prestazioni migliori a parità di frequenza di clock, il che implica un minor consumo energetico e una minore dissipazione del calore. Di conseguenza, i progettisti si trovano sempre più spesso di fronte a questo tipo di compromessi macroscopici, piuttosto che alle prese con i dettagli realizzativi dei componenti hardware.

Le applicazioni audiovisive lavorano su grosse quantità di dati che devono essere elaborate velocemente, perciò la memoria (sotto varie forme) occupa dal 50% al 75% della superficie dei chip e questa percentuale è destinata a salire. Si pongono molti interrogativi di progetto: quanti livelli di cache prevedere? Usare una cache specializzata o una unificata? Quanto dovrebbe essere grande ciascuna cache? Quanto veloce? Vale la pena inserire parte della memoria nel chip? Meglio memoria SRAM o SDRAM? Le risposte a queste domande hanno forti implicazioni sulle prestazioni, sul consumo energetico e sulla dissipazione del calore del chip.

Al di là del progetto dei processori e dei sistemi di memoria, un altro tassello importante per le sue conseguenze è il sistema di comunicazione: come far comunicare tutti i

core tra di loro? Nel caso di piccoli sistemi basterà un solo bus, ma nei sistemi grandi diverrebbe subito un collo di bottiglia. In genere il problema può essere risolto optando per più bus o per un anello che attraversa tutti i core. Nell'ultimo caso, l'arbitraggio avviene tramite il passaggio di un piccolo pacchetto chiamato **token** (gettone) lungo l'anello. Per poter trasmettere, un core deve per prima cosa catturare il token; quando ha finito di comunicare lo può reintrodurre sull'anello perché riprenda a circolare. Questo protocollo previene le collisioni sull'anello.

La Figura 8.13 presenta **CoreConnect** di IBM, un esempio di interconnessione nel chip. È un'architettura per la connessione di core su multiprocessori eterogenei su un chip, usata soprattutto per la progettazione di chip contenenti interi sistemi. In un certo senso, CoreConnect sta ai multiprocessori nel singolo chip come il bus PCI sta al Pentium, ovvero è la colla che tiene uniti i vari pezzi. (Nei moderni sistemi Core i7, la colla è PCIe che è una rete punto a punto priva di un bus condiviso come il PCI). Però, a differenza del bus PCI, CoreConnect è stato progettato senza vincoli di retrocompatibilità dovuti a componenti o protocolli precedenti, e senza i vincoli propri dei bus che risiedono sulle schede, come il limite sul numero di piedini presenti sul connettore.

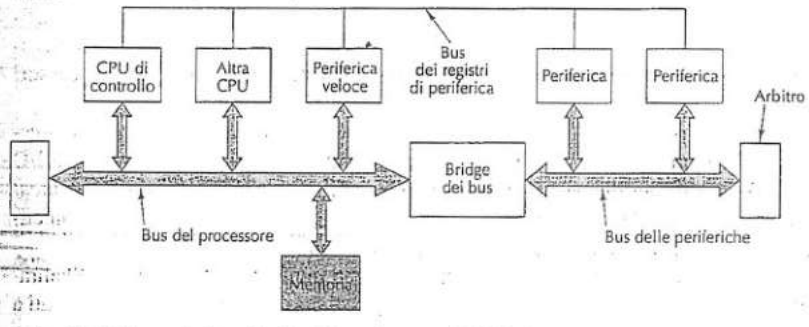


Figura 8.13 Esempio di architettura CoreConnect di IBM.

CoreConnect è composto da tre bus. Quello del **processore** è un bus veloce, sincrono e a pipeline con linee dati di 32, 64 o 128 bit, temporizzate a 66, 133, o 183 MHz. La velocità massima di trasferimento è 23,4 Gbps (contro 4,2 Gbps per il bus PCI). Il funzionamento a pipeline permette ai core di richiedere il bus mentre è già impegnato in un trasferimento e consente a core diversi di usare contemporaneamente linee dati distinte, come per il bus PCI. Il bus del processore è ottimizzato per il trasferimento di blocchi corti ed è concepito per connettere core veloci, come la CPU, il decodificatore MPEG-2, le reti ad alta velocità o componenti similari.

Se il bus del processore fosse esteso a tutto il chip le sue prestazioni degraderebbero, perciò esiste un secondo bus per i dispositivi di I/O più lenti, come gli UART, i timer, i controllori USB e le porte seriali. Questo bus delle periferiche è stato progettato per avere un'interfaccia semplice con le periferiche a 8, 16 o 32 bit e per contenere non più

di qualche centinaio di porte logiche. Anch'esso è sincrono, ma la velocità massima di trasferimento è di 300 Mbps. I due bus sono collegati attraverso un *bridge* simile a quello usato qualche anno fa per collegare i bus PCI e ISA nei PC, prima che gli ISA divenissero obsoleti.

Il terzo bus dei **registri di periferica** è molto lento, asincrono e usa un *handshaking* per permettere a ogni processore di accedere ai registri di tutte le periferiche per il controllo dei rispettivi dispositivi. È concepito per trasferimenti poco frequenti e di pochi byte alla volta.

IBM CoreConnect ha definito uno standard per i bus nel chip, per la loro interfaccia e per l'intelaiatura generale del chip con l'ambizione di creare una versione in miniatura del mondo PCI, in cui ogni produttore possa costruire processori e controllori che si interconnettano con facilità. Esiste però una differenza: nel mondo PCI i produttori costruiscono e vendono schede acquistabili dai distributori e dagli utenti finali. Nel mondo CoreConnect i core sono progettati da terze parti che però non realizzano i prodotti, ma li brevettano come proprietà intellettuale e vendono le licenze alle aziende di elettronica di consumo, che progettano i chip multiprocessori eterogenei basandosi sui core propri e su quelli sottoposti a licenza di terzi. Poiché la produzione di chip così grandi e complessi richiede investimenti massicci in impianti produttivi, nella maggior parte dei casi le aziende di elettronica di consumo si limitano alla progettazione e affidano la realizzazione del chip a produttori di semiconduttori. Esistono core per numerose CPU (ARM, MIPS, PowerPC, e così via), per decodificatori MPEG, per processori di segnali digitali e per tutti i controllori di I/O standard.

Oltre a CoreConnect IBM, anche **AMBA** (*Advanced Microcontroller Bus Architecture*) è usato largamente per connettere CPU ARM ad altre CPU o a dispositivi di I/O (Flynn, 1997). Altri bus di questo tipo, però meno diffusi, sono **VCI** (*Virtual Component Interconnect*) e **OCP-IP** (*Open Core Protocol-International Partnership*), in competizione tra loro per una quota di mercato (Bhakthavatchalu et al., 2010).

E questo è soltanto l'inizio: c'è chi mette intere reti in un chip (Ahmadinia e Shahrabi, 2011). A causa delle difficoltà legate ai problemi di dissipazione del calore (dovuti a loro volta a velocità di clock sempre maggiori) i multiprocessori in un singolo chip sono oggi un tema attuale; rimandiamo a (Gupta et al., 2010; Herrero et al., 2010; Mishra et al., 2011) per ulteriori informazioni.

8.2 Coprocessori

Dopo aver esaminato i modi del parallelismo nel chip, facciamo un passo in avanti e ci interessiamo al modo di velocizzare un calcolatore mediante l'aggiunta di un secondo processore specializzato. Esistono molte varianti di **coprocessori**, di dimensioni molto diverse. Sui mainframe IBM 360, e sui loro successori, esistono canali di I/O indipendenti per l'input e l'output. Allo stesso modo, il CDC 6600 disponeva di 10 processori indipendenti per le operazioni di I/O. Anche la grafica e l'aritmetica in virgola mobile si prestano all'uso di coprocessori, e lo stesso DMA può essere visto come un coprocessore. In alcuni casi la CPU assegna al coprocessore un'istruzione o un insieme d'istruzioni e gli ordina di eseguirle; in altri casi il coprocessore è più indipendente e lavora per conto proprio.

Dal punto di vista fisico, i coprocessori possono costituire apparati separati (i canali di I/O del 360), trovarsi su schede a innesto (i processori di rete) o occupare parte del chip principale (coprocessore in virgola mobile). In tutti i casi, ciò che li caratterizza è il fatto di assistere nell'esecuzione un altro processore, che resta il processore principale. Esaminiamo ora tre settori in cui è possibile velocizzare le prestazioni: elaborazione di rete, multimedia e crittografia.

8.2.1 Processori di rete

Oggigiorno molti calcolatori sono connessi a una rete o a Internet. A seguito del progresso tecnologico dell'hardware di rete, le reti sono oggi così veloci che è sempre più difficile elaborare via software i dati in ingresso e in uscita. A questo riguardo sono stati sviluppati speciali processori di rete per gestire il traffico; molti calcolatori di fascia alta sono dotati di processori di questo tipo. In questo paragrafo cominciamo con l'introdurre le reti, per poi illustrare il funzionamento dei processori di rete.

Introduzione alle reti

Le reti di calcolatori sono di due tipi: **LAN** (*Local-Area Network*, "rete locale"), che connettono più computer all'interno di un edificio o di un campus universitario, e **WAN** (*WideArea Network*, "rete geografica"), per la connessione di calcolatori che si trovano a grande distanza. **Ethernet** è la LAN più diffusa: inizialmente era costituita da cavi spessi contenenti un filo per ogni computer, collegato tramite una spina che veniva chiamata eufemisticamente **spina a vampiro** (*vampire tap*). L'odierna Ethernet prevede che i calcolatori siano collegati a un commutatore (*switch*) centrale, come schematizzato nella Figura 8.14. In origine, Ethernet *si trascinava* a 3 Mbps, ma la prima versione commerciale raggiungeva già i 10 Mbps.

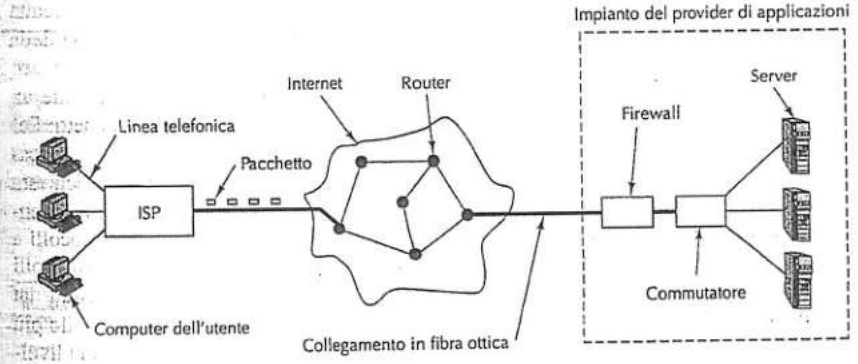


Figura 8.14 Connessione degli utenti ai server di Internet.

È stata poi sostituita da Ethernet veloce (*fast Ethernet*) a 100 Mbps e successivamente da *gigabit Ethernet* a 1 Gbps. Esiste già sul mercato una versione di Ethernet a 10 Gbps e la prossima sarà a 40 Gbps.

Le WAN sono organizzate in modo diverso. Sono costituite da computer per l'instradamento, detti **router**, collegati tramite cavi o fibre ottiche, come illustrato al centro della Figura 8.14. I dati sono suddivisi in frammenti chiamati **pacchetti** (*packet*) che vanno in genere dai 64 ai 1500 byte e che vengono trasferiti dalla macchina mittente alla destinataria passando attraverso uno o più router. A ogni passaggio (*hop*, "salto"), il pacchetto è memorizzato in un router e quindi inoltrato al successivo lungo il cammino non appena la linea di trasmissione lo consenta. Questa tecnica si chiama **commutazione di pacchetto store-and-forward**.

Anche se molte persone pensano a Internet nei termini di un'unica WAN, dal punto di vista tecnico è invece un insieme di molte WAN interconnesse. In ogni caso, questa distinzione non è rilevante ai nostri scopi. La Figura 8.14 fornisce una visione panoramica di Internet dal punto di vista dell'utente domestico. Il suo calcolatore è collegato a un server web tramite una linea telefonica, per mezzo di modem a 56 Kbps o ADSL, come illustrato nel Capitolo 2 (in alternativa è possibile una connessione che usa la linea della TV via cavo, nel qual caso la parte sinistra della figura cambia leggermente e il *provider* diventa la compagnia che fornisce la TV via cavo). Il calcolatore dell'utente spezzetta i dati da inviare al server in pacchetti e li spedisce all'**ISP** (*Internet Service Provider*, "fornitore di servizi Internet") dell'utente, una società che garantisce ai propri utenti l'accesso a Internet. L'ISP ha in genere una connessione ad alta velocità (di solito una fibra ottica) verso una delle reti regionali o una *backbone* (dorsale) di Internet. I pacchetti dell'utente sono inoltrati nella rete un salto alla volta finché non raggiungono il server web.

Molte società che forniscono servizi web dispongono di un **firewall** ("muro tagliafuoco"), un computer specializzato che filtra tutto il traffico in ingresso e cerca di rimuovere i pacchetti indesiderati (per esempio quelli che provengono da hacker che cercano di introdursi nel server). Il firewall è collegato alla LAN, in genere a un commutatore Ethernet, che instrada i pacchetti al server desiderato. Naturalmente, le cose nella realtà sono molto più complesse di come le abbiamo presentate, ma l'idea sostanziale della Figura 8.14 resta valida.

Il software di rete è organizzato per **protocolli**, ciascuno dei quali comprende un insieme di formati, di sequenze di scambio e di regole sul significato dei pacchetti. Per esempio, quando un utente vuole scaricare una pagina web da un server, il suo applicativo di navigazione (*browser*) spedisce al server un pacchetto contenente la richiesta *GET PAGE* e che usa **HTTP** (*HyperText Transfer Protocol*, "protocollo di trasferimento di ipertesti"); il server recepisce la richiesta e la elabora. Esistono molti protocolli e spesso vengono usati in combinazione. Nella maggior parte delle situazioni i protocolli sono strutturati come una serie di livelli (*layer*), in cui i pacchetti vengono passati dai livelli superiori a quelli inferiori perché li elaborino e, una volta raggiunto il livello più basso, perché siano trasmessi. Dal lato destinatario, i pacchetti si fanno strada tra i livelli in ordine inverso, dal basso verso l'alto.

Dal momento che l'elaborazione dei protocolli è l'attività principale dei processori di rete, dobbiamo dilungarci ancora un po' sull'argomento prima di passare ai processori veri e propri. Torniamo per un momento alla richiesta *GET PAGE*. Come avviene la sua spedizione al server web? In pratica il software di navigazione per prima cosa stabilisce una connessione con il server web basata sul protocollo TCP (*Transmission Control Protocol*, "protocollo di controllo della trasmissione"). Il software che implementa questo protocollo verifica che i pacchetti siano stati ricevuti tutti correttamente e nell'ordine giusto. Se un pacchetto viene perso, TCP garantisce che verrà ritrasmesso finché non raggiunga il destinatario.

Il navigatore web compila la richiesta *GET PAGE* come un messaggio HTTP e lo passa al software TCP per la trasmissione in rete. Il software TCP aggiunge all'inizio del messaggio un'intestazione contenente una serie di numeri e altre informazioni, che ovviamente prende il nome di **intestazione TCP** (*TCP header*).

Quando ha finito la sua attività, il software TCP passa l'intestazione e la parte di messaggio vero e proprio (il *payload*, che in questo caso contiene la richiesta *GET PAGE*) a un altro componente software che implementa il **protocollo IP** (*Internet Protocol*). Questo software antepone al messaggio un'**intestazione IP** contenente l'indirizzo del mittente (la macchina da cui proviene il pacchetto) e quello del destinatario (la macchina cui il pacchetto è diretto), il massimo numero di passaggi cui può sopravvivere (per evitare che i pacchetti sopravvivano indefinitamente), una checksum (per rilevare errori di trasmissione o di memoria), e altri campi.

Il pacchetto risultante (dall'intestazione IP, intestazione TCP e richiesta *GET PAGE*) viene passato in basso a livello di trasmissione dei dati (*data link layer*), in cui gli viene allegata un'ulteriore intestazione che serve alla trasmissione vera e propria. Il livello di trasmissione dei dati allega in fondo al messaggio anche una checksum che si chiama **CRC** (*Cyclic Redundancy Check*, "controllo a ridondanza ciclica") e che serve per rilevare errori di trasmissione. Potrebbe sembrare ridondante la presenza di due checksum a livello di trasmissione dei dati e a livello IP, ma serve a migliorare l'affidabilità. A ogni salto lungo la rete viene controllato il valore di CRC, quindi viene rimosso e rigenerato (insieme all'intestazione) secondo un formato adatto al collegamento in uscita. La Figura 8.15 mostra le sembianze del pacchetto quando si trova in una Ethernet; il pacchetto in una linea telefonica (nel caso dell'ADSL) è simile, eccetto per il fatto che viene preceduto da una "intestazione di linea telefonica" invece che da un'intestazione Ethernet. La gestione delle intestazioni è importante ed è una delle mansioni dei processori di rete. È evidente che abbiamo appena sfiorato la superficie di una materia sconfinata quali sono le reti di computer. Per una trattazione più dettagliata si faccia riferimento a (Tanenbaum e Wetherall, 2011).

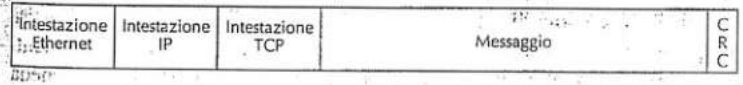


Figura 8.15 Schema dei pacchetti Ethernet.

Introduzione ai processori di rete

Le reti collegano molti tipi di dispositivi. Gli utenti domestici si collegano tramite PC (desktop o portatili), ma anche (e sempre più) tramite console di videogiochi, PDA (computer palmari) e smartphone; le aziende sono collegate tramite PC o server. D'altra parte, esistono anche molti dispositivi che svolgono nella rete una funzione d'intermediazione, tra cui router, commutatori, firewall, proxy web e bilanciatori di carico. È interessante notare che questi sistemi intermediari richiedono una quantità di risorse preponderante nella rete, dal momento che trasferiscono il maggior numero di pacchetti al secondo. Anche i server sono abbastanza esigenti, a differenza dei calcolatori degli utenti domestici.

Un pacchetto ricevuto può richiedere vari tipi di elaborazione, a seconda della rete o del pacchetto stesso, prima di essere inoltrato lungo la linea in uscita o prima di essere consegnato a un programma applicativo. Alcune elaborazioni possibili sono: decidere dove spedire il pacchetto, scomporlo o riassemblarne i pezzi, gestire la sua qualità di servizio (specialmente per i flussi audio o video) o la sua sicurezza (per esempio cifrandolo o decifrandolo), comprimerlo o espanderlo, e così via.

In una LAN a 40 Gbps e con pacchetti di 1 KB, un computer di rete potrebbe dover elaborare quasi 5 milioni di pacchetti al secondo. Con pacchetti di 64 byte si arriva a circa 80 milioni di pacchetti al secondo. Lo svolgimento delle attività di cui sopra nel giro di 12-200 ns è semplicemente impossibile per il software (per non parlare del fatto che di ogni pacchetto servono più copie): è fondamentale l'assistenza dell'hardware.

Una possibile soluzione per elaborare velocemente i pacchetti prevede l'utilizzo di un **ASIC** (*Application-Specific Integrated Circuit*, "circuito integrato per applicazione specifica") progettato su misura. Un chip siffatto è una specie di programma cablato in grado di calcolare l'insieme di funzioni per cui è stato progettato. Attualmente molti router usano circuiti ASIC. Tuttavia i chip ASIC hanno numerosi problemi: innanzitutto richiedono molto tempo sia in fase di progettazione, sia in fase di produzione. Inoltre sono rigidi, perciò se gli si vuole aggiungere una funzionalità bisogna progettare e costruire un chip ex novo. La gestione dei bachi poi è un incubo, visto che l'unico modo per riparare un ASIC è progettare, costruire, spedire e installare un chip nuovo. Infine, si tratta di chip molto costosi, a meno che se ne produca un quantitativo così grande da ammortizzare i costi di sviluppo.

Un'altra possibile soluzione usa i circuiti **FPGA** (*Field Programmable Gate Array*, "circuito a matrice programmabile"), costituiti da un certo numero di porte che possono essere organizzate in un circuito mediante la modifica dei collegamenti tra di loro. Lo sviluppo di questi chip impiega molto meno tempo rispetto agli ASIC e in più gli FPGA possono essere ricablati sul campo: basta rimuoverli dal sistema e inserirli in un dispositivo speciale per la loro riprogrammazione. D'altra parte sono circuiti complessi, lenti e costosi, il che fa di loro circuiti poco richiesti, se non per certe applicazioni di nicchia.

Infine veniamo ai **processori di rete**, dispositivi programmabili che possono gestire i pacchetti in ingresso e in uscita alla stessa velocità con cui viaggiano sui collegamenti (cioè in tempo reale). Un progetto tipico vede il processore di rete posto su una scheda a innesto, insieme a una memoria e ai circuiti logici di supporto. La scheda è collegata a una o più linee di rete che sono dirette al processore di rete, che estrae i pacchetti, li elabora e li spedisce su di una linea diversa (se è un router) o lungo il bus principale di sistema (per esempio lungo il bus PCI) nel caso si tratti di un dispositivo che si trova sul PC di un utente finale. La Figura 8.16 illustra la scheda e il chip di un comune processore di rete.

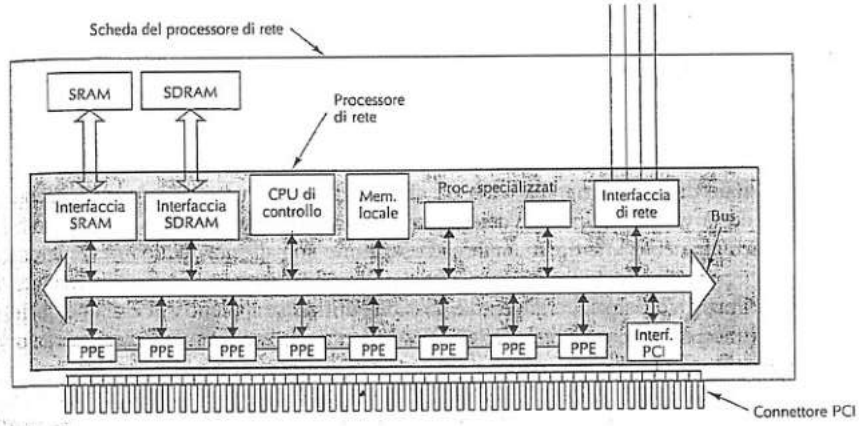


Figura 8.16 Scheda e chip di un comune processore di rete.

La scheda è provvista di memorie SRAM e SDRAM, usate in genere in modi diversi. La SRAM è più veloce e più costosa della SDRAM, perciò la sua disponibilità è molto limitata. La SRAM si usa per contenere le tabelle d'instradamento (*routing table*) e altre strutture dati importanti, mentre la SDRAM memorizza i pacchetti in elaborazione. Grazie alla separazione tra il chip e le memorie, i progettisti possono decidere liberamente le quantità di SRAM e di SDRAM da includere nella scheda. Così facendo, è possibile dotare di poca memoria le schede di fascia bassa con una sola linea in ingresso (destinate per esempio a un PC o a un server), mentre le schede di fascia alta, progettate per i grandi router, possono essere provviste di molta più memoria.

I chip dei processori di rete sono ottimizzati per elaborare velocemente un gran numero di pacchetti in ingresso e in uscita; in particolare, un router con una mezza dozzina di linee potrebbe trovarsi a elaborare milioni di pacchetti al secondo su ogni linea. L'unico modo per raggiungere queste velocità è di costruire processori di rete con molto parallelismo; infatti, ogni processore di rete contiene svariati **PPE** (acronimo di nomi diversi: *Protocol/Programmable/Packet Processing Engine*, "motore di elaborazione del protocollo/programmabile/di pacchetto"). Ogni PPE è un core RISC (eventualmente modificato) con un piccolo quantitativo di memoria per il programma e le variabili.

I motori PPE possono essere organizzati in due modi diversi. Il caso più semplice consiste nell'impiego di PPE identici: quando il processore di rete riceve un pacchetto (dalla rete o dal bus) lo smista a un PPE inattivo perché lo elabori. Se tutti i PPE sono

impegnati, il pacchetto entra in una coda contenuta nella SDRAM della scheda, in attesa che si liberi un PPE. In un'organizzazione di questo tipo non ci sono i collegamenti orizzontali mostrati nella Figura 8.16 tra un PPE e l'altro, perché questi non hanno alcun bisogno di comunicare.

L'altro tipo di organizzazione dei PPE è a pipeline, e ogni motore esegue un passo di elaborazione e inserisce nel pacchetto in uscita un puntatore al PPE successivo nella pipeline. In questo modo, la pipeline dei PPE somiglia molto alle pipeline delle CPU che abbiamo studiato nel Capitolo 2. In entrambe le organizzazioni, i motori PPE sono interamente programmabili.

I progetti più avanzati prevedono PPE con multithreading, dotati perciò di diversi insiemi di registri e di un particolare registro che determina l'insieme correntemente in uso. Grazie a questa caratteristica è possibile eseguire più programmi alla volta e la commutazione da un programma (cioè da un thread) all'altro avviene semplicemente modificando la variabile "insieme dei registri corrente". Comunemente, un PPE commuta immediatamente a favore di un thread eseguibile non appena quello in esecuzione va in stallo, per esempio a seguito di un accesso alla SDRAM (che richiede diversi cicli di clock). Grazie a questo accorgimento è possibile utilizzare appieno i PPE anche quando si bloccano per frequenti accessi alla SDRAM o quando eseguono un'altra operazione esterna piuttosto lenta.

Oltre ai PPE, i processori di rete contengono sempre un processore di controllo (in genere una CPU RISC) impiegato per svolgere tutto quel lavoro che non ha a che fare con l'elaborazione dei pacchetti, come l'aggiornamento delle tabelle d'instradamento. Il suo programma e i suoi dati si trovano in memoria locale del chip. Inoltre, molti chip di rete contengono uno o più processori specializzati per l'esecuzione di operazioni di corrispondenza tra forme (*pattern matching*) o di altre operazioni critiche. Si tratta per lo più di piccoli ASIC in grado di effettuare una sola, semplice operazione, come la ricerca di un indirizzo di destinazione all'interno della tabella d'instradamento. Tutti i componenti del processore di rete comunicano per mezzo di uno o più bus paralleli del chip, con velocità di svariati Gigabit al secondo.

Elaborazione dei pacchetti

Alla sua ricezione, un pacchetto passa attraverso un certo numero di fasi di elaborazione, indipendentemente dal fatto che il processore di rete sia o meno organizzato a pipeline. Alcuni processori di rete dividono queste fasi in due tipi di operazioni, quelle da effettuare sui pacchetti in ingresso (dalla linea di rete o dal bus di sistema), dette perciò **elaborazione in entrata dei pacchetti di rete**, e quelle da effettuare sui pacchetti in uscita, che costituiscono l'**elaborazione in uscita dei pacchetti di rete**. Quando si utilizza questa distinzione, un pacchetto subisce prima l'elaborazione in entrata, poi quella in uscita. La demarcazione tra operazioni in entrata e in uscita è flessibile, perché alcune fasi possono essere svolte indifferentemente in uno dei due momenti (per esempio la raccolta delle statistiche sul traffico).

In seguito analizziamo un possibile ordinamento delle diverse fasi, ma si tenga presente che non tutti i pacchetti necessitano di tutte le fasi e che sono possibili altri ordinamenti ugualmente validi.

1. **Verifica della checksum.** Se il pacchetto in entrata proviene da Ethernet, viene ricalcolato il valore CRC e confrontato con quello nel pacchetto per verificare che non ci siano stati errori di trasmissione. Se il CRC di Ethernet è corretto o assente, viene ricalcolata la checksum IP e confrontata con quella del pacchetto IP per assicurarsi che non sia stato danneggiato da un bit difettoso della memoria del mittente e che avrebbe lasciato traccia nella checksum ivi calcolata. Se le checksum sono corrette, il pacchetto viene accettato per essere ulteriormente elaborato; in caso contrario, viene semplicemente scartato.
2. **Estrazione di campi.** Viene analizzata la parte significativa dell'intestazione e vengono estratti i campi più importanti. All'interno di un commutatore Ethernet viene esaminata solo l'intestazione Ethernet, mentre in un router IP viene ispezionata l'intestazione IP. I campi chiave sono memorizzati nei registri (organizzazione con PPE paralleli) o nella SRAM (organizzazione a pipeline).
3. **Classificazione dei pacchetti.** Il pacchetto è classificato in base a una serie di regole programmabili. La classificazione più semplice distingue tra pacchetti di dati e di controllo, ma di solito si effettuano distinzioni molto più accurate.
4. **Selezione del percorso.** Molti processori di rete predispongono un percorso veloce e ottimizzato per la gestione dei pacchetti più comuni, mentre gli altri pacchetti sono trattati diversamente, spesso a opera del processore di controllo. Si effettua quindi una selezione tra il percorso veloce e quello lento.
5. **Determinazione del destinatario di rete.** I pacchetti IP contengono l'indirizzo del destinatario di 32 bit. Non è possibile, né tanto meno auspicabile, effettuare la ricerca della destinazione di ogni pacchetto IP all'interno di una tabella di 2^{32} elementi, perciò la parte più significativa dell'indirizzo IP viene usata per identificare il numero di rete, il resto per specificare una macchina in quella rete. I numeri di rete possono avere una lunghezza arbitraria, perciò la determinazione del numero della rete destinataria non è banale ed è complicata dal fatto che ci sono più corrispondenze possibili e che quella giusta è la più lunga. In genere questa fase è affidata a un circuito ASIC ad hoc.
6. **Instradamento.** Una volta determinata la rete del destinatario, si sceglie la linea d'uscita lungo cui instradare il pacchetto cercandola in una tabella della SRAM. Anche per questa fase si può usare un circuito ASIC.
7. **Scomposizione e riassemblaggio.** I programmi cercano di passare a livello TCP ingenti quantitativi di dati per ridurre il numero di chiamate di sistema, ma TCP, IP e Ethernet definiscono tutti una dimensione massima per i pacchetti che possono gestire. In conseguenza di queste limitazioni, i dati utili e i pacchetti vengono scomposti dal mittente e i pezzi risultanti vengono riassemblati dal destinatario. È uno dei compiti che possono essere svolti dal processore di rete.
8. **Elaborazione.** Alle volte sono necessari calcoli molto pesanti sui dati che costituiscono il messaggio (come compressione/decompressione o cifratura/decifratura). È un altro compito affidabile al processore di rete.

9. **Gestione dell'intestazione.** Può essere necessario aggiungere, rimuovere o modificare alcuni campi dell'intestazione. Per esempio, l'intestazione IP ha un campo che conta il numero di salti che il pacchetto può effettuare prima di venire scartato. Questo campo va decrementato dopo ogni ritrasmissione: un altro compito adatto al processore di rete.
10. **Gestione della coda.** I pacchetti in entrata o in uscita vengono spesso messi in coda mentre attendono il loro turno di elaborazione. Le applicazioni multimediali potrebbero aver bisogno di un certo intervallo tra un pacchetto e l'altro per evitare il fenomeno del *jitter*. Un firewall o un router potrebbero distribuire il carico di pacchetti in entrata su diverse linee in uscita in base ad alcune regole. Tutte queste attività possono essere svolte dal processore di rete.
11. **Generazione della checksum.** I pacchetti in uscita devono essere provvisti di checksum. La checksum IP può essere generata dal processore di rete, ma il valore di CRC Ethernet di solito viene calcolato dall'hardware.
12. **Contabilità.** In alcuni casi si richiede un po' di contabilità sul traffico di pacchetti, soprattutto quando una rete instrada traffico per conto di altre reti come servizio commerciale. Il processore di rete può occuparsi della contabilità.
13. **Raccolta di statistiche.** Infine, molte organizzazioni intendono accumulare statistiche circa il proprio traffico, per esempio quanti pacchetti sono stati ricevuti e quanti pacchetti sono stati inviati e a che ora del giorno, e altro ancora. Il processore di rete si presta bene alla raccolta di questi dati.

Incremento delle prestazioni

Le prestazioni sono fondamentali per i processori di rete: che cosa si può fare per migliorarle? Prima di rispondere, dobbiamo chiarire cosa si intende per prestazioni. Una metrica possibile è il numero di pacchetti inoltrati ogni secondo, un'altra è il numero di byte inoltrati al secondo. Sono due misure differenti e non è detto che uno schema che funziona bene con pacchetti piccoli funzioni altrettanto bene con pacchetti più grandi. In particolare, nel primo caso potrebbe giovare una riduzione del numero di destinazioni ricercate in ogni secondo, mentre ciò potrebbe non essere vero nel caso di pacchetti grandi.

Il modo più diretto per migliorare le prestazioni è aumentare la velocità di clock del processore di rete. Ovviamente il miglioramento non è lineare, perché può essere influenzato dal tempo di ciclo della memoria e da altri fattori. E poi, un clock più veloce vuol dire più calore da dissipare.

Una soluzione che molto spesso ripaga è l'aggiunta di più PPE e l'incremento del parallelismo, soprattutto quando si usa l'organizzazione parallela dei PPE. Anche l'allungamento della pipeline può aiutare, ma solo se l'intera elaborazione di un pacchetto sia suddivisibile in fasi più corte.

Un'altra tecnica possibile prevede l'adozione di un processore specializzato o di un ASIC per gestire specificatamente le operazioni che richiedono molto tempo, che vengono eseguite ripetutamente e la cui l'esecuzione via hardware risulta molto più veloce di quella via software. Tra i candidati ci sono sicuramente le ricerche nelle tabelle, il calcolo della checksum e la cifratura.

L'aggiunta di un maggior numero di bus interni e l'ampliamento dei bus esistenti può aiutare ad aumentare la velocità contribuendo ad accelerare lo spostamento dei pacchetti nel sistema. Infine, tra i miglioramenti si può considerare anche la sostituzione della SDRAM con memoria SRAM, ma che comporta costi aggiuntivi.

Naturalmente resta molto da dire sui processori di rete. Alcuni riferimenti in tal senso sono (Freitas et al., 2009; Lin et al., 2010; Yamamoto e Nakao, 2011).

8.2.2 Processori grafici

Un altro settore in cui si rendono utili i coprocessori è quello della gestione di elaborazioni grafiche ad alta risoluzione, come il rendering 3D. Le normali CPU non sono molto dotate per elaborare volumi di dati così ingenti come quelli richiesti da queste applicazioni. Perciò la maggior parte dei PC e molti dei processori futuri saranno dotati di GPU (Graphics Processing Units) a cui affidare gran parte del lavoro di elaborazione.

La GPU NVIDIA Fermi

Ci occupiamo del settore dei processori grafici, la cui importanza è in continua ascesa, con un esempio concreto: la **GPU NVIDIA Fermi**, un'architettura utilizzata in una famiglia di chip per l'elaborazione grafica disponibili con diverse velocità e dimensioni. L'architettura della GPU Fermi, mostrata nella Figura 8.17, è organizzata in 16 **SM** (*Streaming Multiprocessor*) ognuno con una sua cache L1 privata a elevata larghezza di banda. Ogni SM contiene 32 core CUDA, per un totale di 512 core CUDA in ogni GPU Fermi. Un core **CUDA** (*Compute Unified Device Architecture*) è un semplice processore che supporta calcolo intero a singola precisione e in virgola mobile. Un SM con 32 core CUDA è mostrato nella Figura 2.7. I 16 SM condividono l'accesso a un'unica cache unificata di secondo livello di 768 KB, che è connessa a un'interfaccia DRAM dotata di più porte. L'interfaccia del processore host offre un percorso di comunicazione tra il sistema host e la GPU per mezzo di un'interfaccia bus DRAM condivisa, di solito utilizzando un'interfaccia PCI-Express.

L'architettura Fermi è progettata per eseguire in maniera efficiente codice per l'elaborazione grafica, video e di immagini, che di solito contiene computazioni ridondanti distribuite su un gran numero di pixel. A causa di questa ridondanza gli SM, in grado di eseguire 16 operazioni alla volta, eseguono in un singolo ciclo operazioni identiche. Questo stile di elaborazione viene chiamato elaborazione **SIMD** (*Single-Instruction Multiple Data*) e ha l'importante vantaggio che ogni SM preleva e decodifica una sola istruzione per ciclo. Per la condivisione dell'elaborazione delle istruzioni tra tutti i core di un SM, NVIDIA dispone di 512 core su un unico stampo di silicio. Se i programmatori fossero in grado di sfruttare tutte le risorse di calcolo (si tratta di un "se" grande e incerto), il sistema offrirebbe notevoli vantaggi computazionali rispetto alle architetture scalari tradizionali, come il Core i7 o l'OMAP4430.

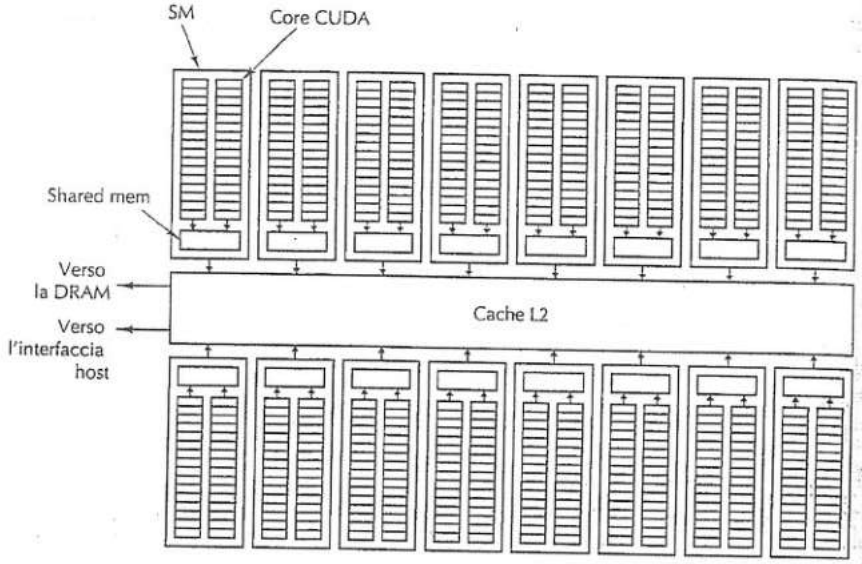


Figura 8.17 L'architettura della GPU Fermi.

I requisiti dell'elaborazione SIMD all'interno degli SM vincolano la tipologia di codice eseguibile su queste unità. Ogni core CUDA, per poter raggiungere le 16 operazioni simultanee, deve infatti eseguire lo stesso codice. Per alleggerire questo onere a carico del programmatore, NVIDIA ha sviluppato il linguaggio di programmazione CUDA. Questo linguaggio specifica il parallelismo del programma utilizzando i thread che vengono poi raggruppati in blocchi, e assegnati agli SM. Se ogni thread in un blocco esegue esattamente la stessa sequenza di codice (cioè, tutte le diramazioni prendono la stessa decisione), verranno eseguite fino a 16 operazioni simultanee (supponendo che vi siano 16 thread pronti per l'esecuzione). Quando i thread su un SM prendono diramazioni diverse, si verifica un effetto dannoso per le prestazioni chiamato divergenza di diramazione. Tale effetto forza i thread con sequenze di codice differenti a essere eseguiti in serie sullo SM. La divergenza di diramazione riduce il parallelismo e rallenta l'elaborazione della GPU. Fortunatamente, vi è una vasta gamma di attività nell'elaborazione grafica e di immagini in grado di evitare divergenze di diramazione e ottenere un buon incremento di velocità. È stato dimostrato che molti altri contesti possono beneficiare dell'architettura SIMD dei processori grafici, come per esempio l'imaging in ambito medico, la dimostrazione automatica, la previsione finanziaria e l'analisi di grafi. Questo allargamento delle potenziali applicazioni delle GPU ha valso loro il nuovo soprannome di **GPGPU** (*General-Purpose Graphics Processing Unit*).

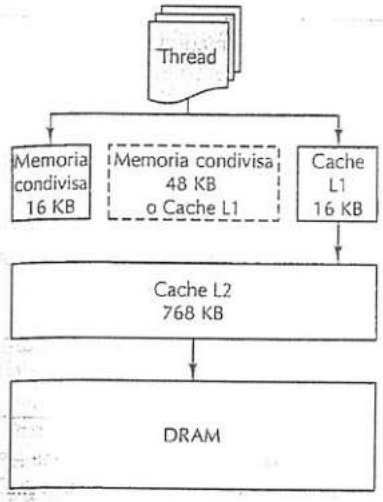


Figura 8.18 La gerarchia di memoria della GPU Fermi.

Senza un'adeguata larghezza di banda la GPU Fermi, con i suoi 512 core CUDA, arriverebbe ad arrestarsi. Per fornire questa larghezza di banda, la GPU Fermi implementa una moderna gerarchia di memoria, come illustrato nella Figura 8.18. Ogni SM ha sia una memoria dedicata condivisa che una cache dati privata di primo livello. La memoria dedicata condivisa viene indirizzata direttamente dai core CUDA e fornisce una condivisione veloce dei dati tra i thread all'interno di un singolo SM. La cache di primo livello velocizza l'accesso ai dati sulla DRAM. Per far spazio alla vasta gamma di dati gli SM possono essere configurati con 16 KB di memoria condivisa e 48 KB di cache di primo livello oppure con 48 KB di memoria condivisa e 16 KB di cache L1. Tutti gli SM condividono una cache unificata di secondo livello di 768 KB. La cache L2 offre un accesso veloce ai dati sulla DRAM che non trovano spazio sulla cache di primo livello e fornisce un meccanismo di condivisioni tra gli SM, anche se questa modalità di condivisione è più lenta della condivisione intra-SM che si ha all'interno della memoria condivisa degli SM. Dopo la cache di secondo livello vi è la DRAM, che mantiene altri dati, immagini e texture utilizzati dai programmi in esecuzione sulla GPU Fermi. I programmi efficienti proveranno in tutti i modi a evitare accessi alla DRAM, perché un solo accesso richiede centinaia di cicli per essere completato.

Per un programmatore esperto, la GPU Fermi rappresenta una delle piattaforme computazionalmente più capaci mai create. Una sola GPU GTX 580 basata su Fermi in esecuzione a 772 MHz con 512 core CUDA può sostenere velocità di calcolo di 1,5 teraflop consumando 250 watt di potenza. Questi valori sono ancora più impressionanti se si considera che il prezzo al dettaglio di una GPU GTX 580 GPU è inferiore ai 600 dollari. A titolo di confronto, nel 1990 il computer più veloce al mondo, il Cray-2, aveva una velocità di 0,002 teraflop e un prezzo (calcolato tenendo conto dell'inflazione) di 30

milioni di dollari. Riempiva inoltre una piccola stanza ed era dotato di un proprio sistema di raffreddamento per dissipare i 150 KW di potenza consumata. La GTX 580 ha una potenza di calcolo 750 volte superiore, costa 1/50000 del prezzo e consuma 1/600 di energia. Non è certo un cattivo affare.

8.2.3 Crittoprocessori

La sicurezza è il terzo settore in cui sono molto diffusi i coprocessori, soprattutto per quanto riguarda le reti. Quando si stabilisce una connessione tra un client e un server, spesso la prima cosa che viene richiesta a entrambi è l'autenticazione reciproca. A tal fine stabiliscono una connessione cifrata così da poter trasferire i dati in modo sicuro, vanificando ogni tentativo di intromissione nella linea.

Il problema legato alla sicurezza è che, per garantirla, bisogna ricorrere alla crittografia, che richiede però molte risorse di calcolo. Esistono due tipologie di metodi crittografici: **a chiave simmetrica** e **a chiave pubblica**. La prima si basa sull'idea di confondere tutti i bit del messaggio, producendo più o meno ciò che si otterrebbe inserendolo in un frullatore elettronico. La seconda si basa sulla moltiplicazione e sull'elevamento a potenza di numeri molto grandi (per esempio di 1024 bit) e impegna molte risorse di tempo.

Molte aziende hanno prodotto *critto-coprocessori*, spesso sotto forma di schede a innesto nel bus PCI, per gestire i calcoli richiesti dalla cifratura sicura dei dati (per la loro trasmissione o memorizzazione) e dalla loro successiva decifratura. Questi coprocessori sono provvisti di hardware speciale che consente loro di svolgere tutte le necessarie operazioni crittografiche molto più velocemente di quanto possa fare una CPU. Sfortunatamente, la descrizione dettagliata del funzionamento dei crittoprocessori richiederebbe innanzitutto una spiegazione abbastanza accurata della stessa crittografia, il che va oltre gli scopi di questo libro. Si possono trovare maggiori informazioni sui critto-coprocessori in (Gaspar et al., 2010; Haghighizadeh et al., 2010; Shoufan et al.,2011).

8.3 Multiprocessori con memoria condivisa

Abbiamo visto come introdurre il parallelismo nel chip o all'interno di singoli sistemi tramite l'aggiunta di un coprocessore. Il passo successivo è la combinazione di più CPU per formare sistemi più grandi. Questi sistemi si dividono in due categorie: multiprocessori e multicomputer. Cominciamo con il definire il significato di questi termini, poi ci inoltreremo nell'analisi dei multiprocessori e dei multicomputer.

8.3.1 Multiprocessori e multicomputer a confronto

In un sistema di calcolo parallelo, le varie CPU che operano su parti diverse di uno stesso compito devono poter comunicare tra loro per scambiarsi informazioni. Le modalità della comunicazione sono oggetto di grande dibattito nella comunità degli architetti degli elaboratori. Sono stati proposti e implementati due progetti diversi, i multiprocessori e i multicomputer, che si distinguono per la presenza o per l'assenza di memoria condivisa. Questa differenza influisce sul loro progetto, sulla loro costruzione e programmazione, nonché sui loro costi e dimensioni.

Multiprocessori

Un **multiprocessore** è un calcolatore in cui tutte le CPU condividono una memoria comune, come schematizzato nella Figura 8.19. Tutti i processi che cooperano in un multiprocessore possono condividere un solo spazio degli indirizzi virtuali mappato nella memoria comune. Ogni processo può leggere o scrivere una parola di memoria per mezzo di semplici istruzioni LOAD e STORE; non serve altro perché del resto si occupa l'hardware. Due processi possono comunicare in modo molto semplice: uno scrive dati in memoria, l'altro è in grado di leggerli.

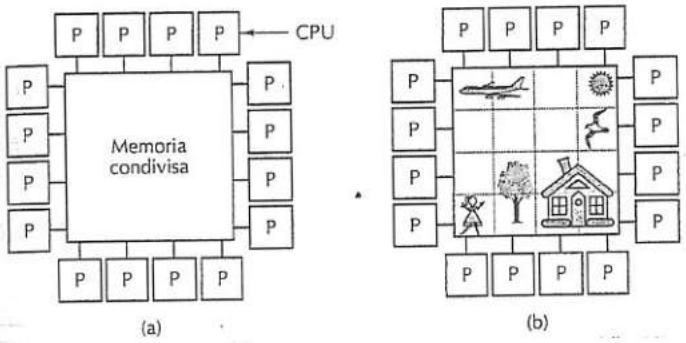


Figura 8.19 (a) Multiprocessore con 16 CPU che condividono una memoria comune. (b) Immagine ripartita in 16 porzioni, ciascuna analizzata da una CPU diversa.

La facilità con cui due (o più) processi riescono a comunicare tramite operazioni sulla memoria è la ragione della diffusione dei multiprocessori. È un modello semplice da capire per i programmatori ed è applicabile con successo a un gran numero di problemi. Si consideri, per esempio, un programma che ispeziona un'immagine e che elenca tutti gli oggetti in essa contenuti. Una copia dell'immagine è contenuta in memoria, come mostrato dalla Figura 8.19(b). Ciascuna delle 16 CPU elabora un processo singolo cui è stata assegnata una delle 16 porzioni dell'immagine da analizzare. Ciononostante, ogni processo ha accesso all'intera immagine, il che è essenziale poiché alcuni oggetti possono occupare più sezioni contigue. Se un processo scopre che un oggetto si estende all'interno di un'altra porzione, può seguire l'oggetto semplicemente leggendo le parole di quella porzione. In questo esempio alcuni oggetti verranno scoperti da più processi, perciò sarà necessario attribuire delle coordinate che permettano di contare a posteriori quante case, alberi e aeroplani ci sono nell'immagine.

Dal momento che tutte le CPU di un multiprocessore vedono la stessa immagine della memoria, c'è una sola copia del sistema operativo e, di conseguenza, c'è una sola

mappa delle pagine e una sola tabella dei processi. Quando un processo si blocca, la sua CPU salva il suo stato nelle tabelle del sistema operativo e cerca al loro interno un altro processo da eseguire.

È proprio questa visione di un'unica memoria che distingue un multiprocessore da un multicomputer, in cui invece ogni calcolatore ha la propria copia del sistema operativo. Un multiprocessore, come tutti i calcolatori, deve avere dispositivi di I/O, come i dischi o gli adattatori di rete, e altre periferiche. In alcuni sistemi multiprocessore solo una certa CPU ha accesso ai dispositivi di I/O ed è perciò dotata di funzioni speciali per l'I/O. Invece si parla di SMP (*Symmetric MultiProcessor*, "multiprocessore simmetrico") quando tutte le CPU hanno uguale accesso a tutti i moduli di memoria e a tutti i dispositivi di I/O, e sono usate in modo intercambiabile dal sistema operativo.

Multicomputer

Il secondo progetto di architettura parallela prevede per ogni CPU una memoria privata, cioè accessibile solo da essa e o non dalle altre. Un progetto di questo tipo prende il nome di **multicomputer** o di **sistema a memoria distribuita** ed è illustrato nella Figura 8.20(a). L'aspetto fondamentale di un multicomputer, che lo distingue da un multiprocessore, è il fatto che ogni CPU è dotata di una memoria privata e locale, cui può accedere tramite semplici istruzioni LOAD e STORE, ma cui nessun altra CPU può accedere. Dunque i multiprocessori hanno un solo spazio degli indirizzi fisici comune a tutte le CPU, mentre i multicomputer hanno uno spazio degli indirizzi fisici per ogni CPU.

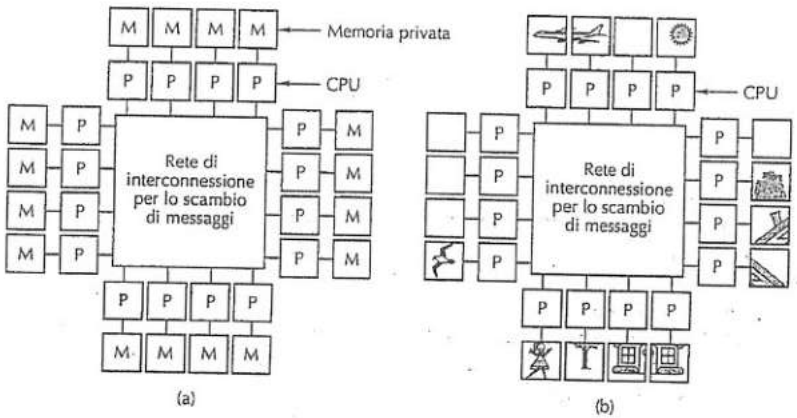


Figura 8.20 (a) Multicomputer con 16 CPU, ciascuna con la propria memoria privata. (b) Immagine bitmap della Figura 8.19 suddivisa tra le 16 memorie.

Visto che le CPU dei multicomputer non possono comunicare attraverso scritture e letture della memoria comune, c'è bisogno di un altro meccanismo di comunicazione: la soluzione adottata è lo scambio di messaggi lungo la rete di interconnessione. Tra i multicomputer annoveriamo BlueGene/L di IBM, Red Storm e i cluster di Google.

L'assenza di una memoria condivisa via hardware ha implicazioni importanti sulla struttura del software di un multicomputer. Non è più possibile avere un solo spazio virtuale degli indirizzi in cui tutti i processi sono in grado di leggere e scrivere per mezzo dell'esecuzione di semplici istruzioni LOAD e STORE. Per esempio, se la CPU 0 della Figura 8.19(b) (quella nell'angolo in alto a sinistra) scopre che parte del proprio oggetto si estende all'interno della porzione assegnata alla CPU 1, può comunque accedere alla coda dell'aeroplano continuando a leggere dati dalla memoria. Viceversa, quando la CPU 0 della Figura 8.20(b) fa la stessa scoperta, non può leggere in alcun modo la memoria della CPU 1. Per accedere a quei dati deve avvalersi di un meccanismo sostanzialmente diverso.

Infatti deve innanzitutto scoprire (in qualche modo) quale CPU possiede i dati che le interessano e spedirle quindi una richiesta di copia dei dati. Di norma questa operazione blocca la CPU finché la richiesta non viene soddisfatta. All'arrivo del messaggio alla CPU 1, il suo software deve analizzarlo e restituire i dati richiesti. Quando la CPU 0 riceve il messaggio di risposta, il suo software si sblocca e continua l'esecuzione.

La comunicazione tra i processi di un multicomputer avviene quindi tramite le primitive software send e receive. Ciò rende la struttura del software molto diversa e di gran lunga più complessa rispetto a un multiprocessore. Inoltre è chiaro come diventi particolarmente problematico in un multicomputer riuscire a suddividere i dati correttamente e assegnarli alle locazioni in modo ottimale; la questione non costituiva una grosso problema nei multiprocessori perché in quel caso la collocazione dei dati non influiva sulla correttezza o sulla programmabilità del codice, tutt'al più sulle sue prestazioni. In poche parole, programmare un multicomputer è molto più difficile che programmare un multiprocessore.

Se le cose stanno così, perché mai qualcuno dovrebbe voler costruire multicomputer, quando i multiprocessori sono più facili da programmare? La risposta è semplice: i multicomputer grandi sono molto più semplici ed economici da costruire rispetto a multiprocessori con lo stesso numero di CPU. Riuscire a implementare una memoria condivisa anche solo da qualche centinaia di CPU è praticamente un'impresa, mentre è semplice costruire un multicomputer con 10.000 CPU o più. Nel seguito del capitolo studieremo un multicomputer con più di 50.000 CPU.

Ci troviamo davanti a un dilemma: i multiprocessori sono difficili da costruire, ma facili da programmare, i multicomputer sono facili da costruire, ma difficili da programmare. Per queste ragioni sono stati compiuti sforzi ingenti per riuscire a costruire sistemi ibridi che fossero relativamente facili da costruire e programmare. Come risultato di questi sforzi si è giunti alla conclusione che la memoria condivisa può essere implementata in modi diversi, ciascuno con i propri vantaggi e svantaggi. Così una fetta considerevole della ricerca attuale nelle architetture parallele si concentra sulla convergenza di architetture multiprocessore e multicomputer che conservi i punti di forza di ciascuna. Il "Santo Graal" di questa disciplina è riuscire a ideare progetti **scalabili**, ovvero che continuino a comportarsi bene anche al crescere delle dimensioni per l'aggiunta di altre CPU.

Un approccio per la costruzione di sistemi ibridi si basa sul fatto che i sistemi di calcolo moderni non sono monolitici, bensì costruiti come una serie di livelli (il tema di questo libro). Questa intuizione apre la strada all'implementazione della memoria condivisa a livelli diversi, come mostra la Figura 8.21. Nella Figura 8.21(a) la memoria condivisa è implementata dall'hardware come in un vero multiprocessore. Secondo questo progetto, c'è una sola copia del sistema operativo con un solo insieme di tabelle e, in particolare, con una sola tabella di allocazione della memoria. Quando un processo ha bisogno di più memoria, invia una trap al sistema operativo e questo cerca una pagina libera nella propria tabella e la mappa nello spazio degli indirizzi del chiamante. Dal punto di vista del sistema operativo c'è una sola memoria ed esso si limita a mantenere traccia via software dell'attribuzione delle pagine ai processi. Come vedremo, sono possibili molte implementazioni hardware di una memoria condivisa.

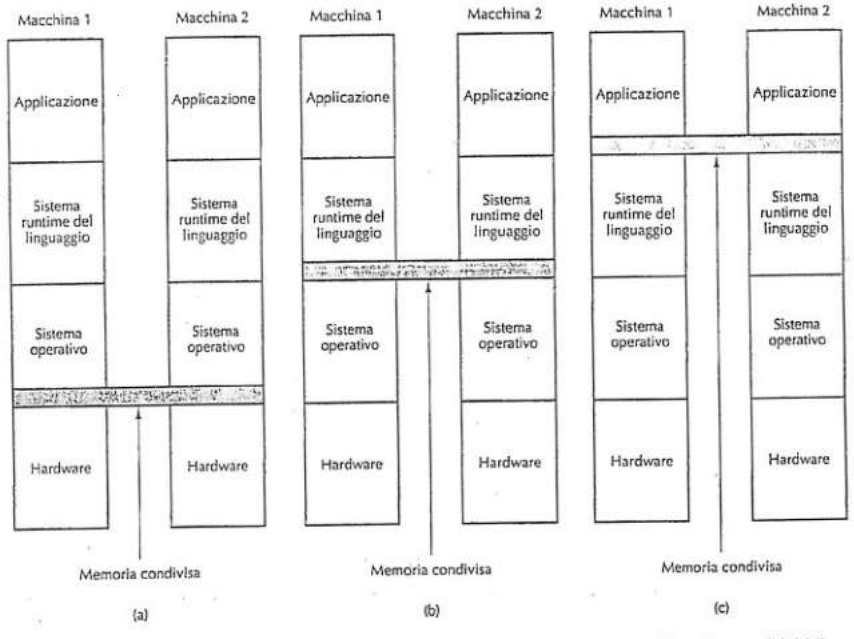


Figura 8.21 Vari livelli d'implementazione di una memoria condivisa. (a) Nell'hardware. (b) Nel sistema operativo. (c) Nel sistema runtime del linguaggio.

Una seconda possibilità è usare l'hardware di un multicomputer e servirsi del sistema operativo per simulare la memoria condivisa come uno spazio paginato degli indirizzi virtuali condiviso in tutto il sistema. Questo approccio si chiama **DSM** (*Distributed Shared Memory*, "memoria condivisa distribuita"; Li e Hudak, 1989) e prevede che ogni

pagina sia localizzata in una delle memorie della Figura 8.20(a). Ogni macchina ha una propria memoria virtuale e una propria tabella delle pagine. Quando una CPU effettua una LOAD o una STORE all'interno di una pagina di cui non dispone, si verifica una trap rivolta al sistema operativo; questo localizza la pagina e richiede alla CPU che la possiede correntemente di estrometterla dalla memoria e di spedirla lungo la rete di interconnessione. Una volta recapitata, la pagina viene mappata in memoria della prima CPU e l'istruzione può riprendere la propria esecuzione. A tutti gli effetti, il sistema operativo non sta facendo altro che gestire gli errori di pagina attingendo a una memoria distante invece che al disco. Dal punto di vista dell'utente, il sistema sembra un sistema a memoria virtuale. Analizzeremo DSM nel prosieguo del capitolo.

Una terza possibilità è implementare la memoria condivisa all'interno del sistema runtime (eventualmente specifico di un linguaggio) nel livello utente. Secondo questo approccio, il linguaggio di programmazione fornisce una specie di astrazione della memoria condivisa che viene poi implementata dal compilatore e dal sistema runtime. Per esempio, il modello Linda si basa sull'astrazione di uno spazio condiviso di tuple (record di dati contenenti un insieme di campi). I processi di tutte le macchine possono prelevare una tupla dallo spazio condiviso delle tuple o inserirvene una. Poiché il controllo degli accessi è effettuato tutto via software (dal sistema runtime di Linda) non c'è bisogno di hardware speciale o di supporto da parte del sistema operativo.

Un altro esempio di memoria condivisa implementata dal sistema runtime di un linguaggio è il modello Orca di condivisione degli oggetti di dati. In Orca, i processi condividono oggetti generici e non tuple, e possono eseguire su di loro alcuni metodi specifici degli oggetti stessi. Quando un metodo invoca una modifica allo stato interno di un oggetto, spetta al sistema runtime assicurarsi che tutte le copie dell'oggetto residenti sulle diverse macchine vengano aggiornate simultaneamente. Sottolineiamo che non c'è bisogno di alcuna assistenza da parte dell'hardware o del sistema operativo perché gli oggetti sono un concetto puramente software. In questo capitolo ci occuperemo ancora di Linda e di Orca.

Tassonomia dei calcolatori paralleli

Torniamo ora al nostro argomento principale: l'architettura dei calcolatori paralleli. Nel corso degli anni sono stati proposti e costruiti molti tipi di calcolatori paralleli, dunque viene naturale chiedersi se è possibile organizzarli in una tassonomia di categorie. Molti ricercatori si sono dedicati a questa attività, con risultati diversi (Flynn, 1972; Treleaven, 1985). Purtroppo non è ancora nato il Linneo[3] del calcolo parallelo. L'unico schema abbastanza usato è quello di Flynn (Figura 8.22), che resta comunque molto approssimativo. La classificazione di Flynn si basa su due concetti: i flussi d'istruzioni e i flussi di dati.

Un flusso d'istruzioni corrisponde a un program counter. Un sistema con n CPU ha n program counter, quindi n flussi d'istruzioni.

3 Carlo Linneo (1707-1778): biologo svedese che inventò il sistema di classificazione delle piante e degli animali ancora in uso e che cataloga gli individui in base alle categorie di regno, phylum (tipo o divisione), classe, ordine, famiglia, genere e specie.

Flussi d'istruzioni	Flussi di dati	Nome	Esempi
1	1	SISD	Modello di Von Neumann
1	Molteplici	SIMD	Supercomputer vettoriali, unità di calcolo vettoriali
Molteplici	1	MISD	Forse nessuno
Molteplici	Molteplici	MIMD	Multiprocessori, multicomputer

Figura 8.22 Tassonomia di Flynn dei calcolatori paralleli.

Un flusso di dati è un insieme di operandi. Per esempio, in un sistema per le previsioni metereologiche dotato di un gran numero di sensori ogni sensore potrebbe emettere un flusso di temperature a intervalli regolari.

I flussi d'istruzioni e di dati sono in buona parte indipendenti, perciò si danno le quattro possibili combinazioni della Figura 8.22. SISD è la macchina sequenziale classica di Von Neumann, caratterizzata da un solo flusso d'istruzioni, un solo flusso di dati e svolge una sola attività alla volta. Le macchine SIMD hanno una sola unità di controllo che emette un'i struzione alla volta, ma dispongono di varie ALU per operare simultaneamente su diversi insiemi di dati. L'ILLIAC IV (Figura 2.7) è il prototipo di macchina SIMD. I mainframe SIMD sono sempre più rari, sebbene i calcolatori convenzionali possano avere alcune istruzioni SIMD per l'elaborazione di materiale audiovisivo (per esempio le istruzioni SSE del Core i7). Comunque sia, c'è un nuovo settore in cui sembrano giocare un ruolo importante le idee del paradigma SIMD: gli *stream processor*. Si tratta di macchine progettate specificatamente per la gestione del rendering multimediale e potrebbero acquisire in futuro una certa importanza (Kapasi et al., 2003).

Le macchine MISD fanno parte di una categoria un po' strana, in cui ci sono molteplici istruzioni che operano sugli stessi dati. Non è ben chiaro se esista una qualche macchina di questo tipo, anche se alcuni ritengono che le macchine a pipeline siano macchine MISD.

Infine vengono le macchine MIMD, semplicemente un insieme di CPU indipendenti che operano come componenti di un sistema più grande. Molti dei calcolatori paralleli fanno parte di questa categoria. I multiprocessori e i multicomputer sono macchine MIMD. La tassonomia di Flynn si ferma qui, ma l'abbiamo arricchita come mostrato nella Figura 8.23. Abbiamo suddiviso le macchine SIMD in due sottrogruppi: il primo comprende i supercomputer numerici e le macchine che operano su vettori, effettuando la stessa operazione su ciascun elemento del vettore; il secondo contiene le macchine di tipo parallelo, come l'ILLIAC IV, in cui un'unità principale di controllo invia le istruzioni a molte ALU indipendenti.

Nella nostra tassonomia abbiamo diviso la categoria MIMD in multiprocessori (macchine a condivisione di memoria) e multicomputer (macchine a scambio di messaggi). Esistono tre tipi di multiprocessori, caratterizzati dal modo in cui implementano la memoria condivisa; sono chiamati **UMA** (*Uniform Memory Access*, "accesso uniforme alla memoria"), **NUMA** (*NonUniform Memory Access*) e **COMA** (*Cache Only Memory Access*). Queste sottocategorie esistono perché in genere la memoria dei multiprocessori molto grandi è divisa in più moduli. Le macchine UMA hanno la proprietà di garantire lo stesso tempo d'accesso a ogni modulo di memoria da parte di ogni CPU, ovvero ogni parola di memoria può essere letta tanto velocemente quanto qualsiasi altra. Se questa proprietà è tecnicamente impossibile da garantire, gli accessi più veloci sono rallentati alla velocità dei più lenti così che i programmatori non si accorgano della differenza. È questa l'accezione di "uniforme" che si intende in questo contesto. L'uniformità permette di fare previsioni sulle prestazioni. il che è importante per la scrittura di codice efficiente.

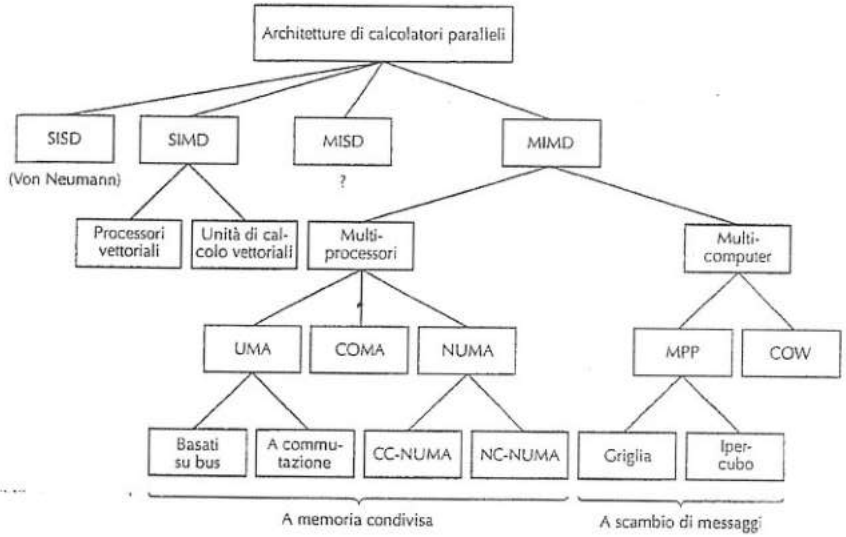


Figura 8.23 Tassonomia dei calcolatori paralleli.

Viceversa, nei multiprocessori NUMA questa proprietà non vale. Spesso c'è un modulo di memoria vicino a ogni CPU e il suo accesso è molto più veloce rispetto ai moduli distanti. Perciò, ai fini delle prestazioni, conta molto la collocazione dei dati e dei programmi. Anche le macchine COMA non sono uniformi, ma in un modo diverso. Ci occuperemo più tardi di queste categorie di macchine.

L'altra categoria principale delle macchine MIMD è costituita dai multicomputer che, a differenza dei multiprocessori, non hanno una memoria principale condivisa a livello di architettura. In altre parole, il sistema operativo di una CPU di un multicomputer non può accedere alla memoria di un'altra CPU tramite la sola esecuzione di un'istruzione `LOAD`. Deve invece spedire un messaggio e aspettare la relativa risposta. La capacità del sistema operativo di leggere parole a distanza effettuando una semplice `LOAD` è ciò che distingue i multiprocessori dai multicomputer. Abbiamo già detto che anche i programmi utente di un multicomputer possono essere resi in grado di accedere alle memorie distanti tramite istruzioni `LOAD` e `STORE`, però si tratta in questo caso di

Flussi d'istruzioni	Flussi di dati	Nome	Esempi
1	1	SISD	Modello di Von Neumann
1	Molteplici	SIMD	Supercomputer vettoriali, unità di calcolo vettoriali
Molteplici	1	MISD	Forse nessuno
Molteplici	Molteplici	MIMD	Multiprocessori, multicomputer

Figura 8.22 Tassonomia di Flynn dei calcolatori paralleli.

Un flusso di dati è un insieme di operandi. Per esempio, in un sistema per le previsioni metereologiche dotato di un gran numero di sensori ogni sensore potrebbe emettere un flusso di temperature a intervalli regolari.

I flussi d'istruzioni e di dati sono in buona parte indipendenti, perciò si danno le quattro possibili combinazioni della Figura 8.22. SISD è la macchina sequenziale classica di Von Neumann, caratterizzata da un solo flusso d'istruzioni, un solo flusso di dati e svolge una sola attività alla volta. Le macchine SIMD hanno una sola unità di controllo che emette un'i struzione alla volta, ma dispongono di varie ALU per operare simultaneamente su diversi insiemi di dati. L'ILLIAC IV (Figura 2.7) è il prototipo di macchina SIMD. I mainframe SIMD sono sempre più rari, sebbene i calcolatori convenzionali possano avere alcune istruzioni SIMD per l'elaborazione di materiale audiovisivo (per esempio le istruzioni SSE del Core i7). Comunque sia, c'è un nuovo settore in cui sembrano giocare un ruolo importante le idee del paradigma SIMD: gli *stream processor*. Si tratta di macchine progettate specificatamente per la gestione del rendering multimediale e potrebbero acquisire in futuro una certa importanza (Kapasi et al., 2003).

Le macchine MISD fanno parte di una categoria un po' strana, in cui ci sono molteplici istruzioni che operano sugli stessi dati. Non è ben chiaro se esista una qualche macchina di questo tipo, anche se alcuni ritengono che le macchine a pipeline siano macchine MISD.

Infine vengono le macchine MIMD, semplicemente un insieme di CPU indipendenti che operano come componenti di un sistema più grande. Molti dei calcolatori paralleli fanno parte di questa categoria. I multiprocessori e i multicomputer sono macchine MIMD. La tassonomia di Flynn si ferma qui, ma l'abbiamo arricchita come mostrato nella Figura 8.23. Abbiamo suddiviso le macchine SIMD in due sottrogruppi: il primo comprende i supercomputer numerici e le macchine che operano su vettori, effettuando la stessa operazione su ciascun elemento del vettore; il secondo contiene le macchine di tipo parallelo, come l'ILLIAC IV, in cui un'unità principale di controllo invia le istruzioni a molte ALU indipendenti.

Nella nostra tassonomia abbiamo diviso la categoria MIMD in multiprocessori (macchine a condivisione di memoria) e multicomputer (macchine a scambio di messaggi). Esistono tre tipi di multiprocessori, caratterizzati dal modo in cui implementano la memoria condivisa; sono chiamati **UMA** (*Uniform Memory Access*, "accesso uniforme alla memoria"), **NUMA** (*NonUniform Memory Access*) e **COMA** (*Cache Only Memory Access*). Queste sottocategorie esistono perché in genere la memoria dei multiprocessori molto grandi è divisa in più moduli. Le macchine UMA hanno la proprietà di garantire lo stesso tempo d'accesso a ogni modulo di memoria da parte di ogni CPU, ovvero ogni parola di memoria può essere letta tanto velocemente quanto qualsiasi altra. Se questa proprietà è tecnicamente impossibile da garantire, gli accessi più veloci sono rallentati alla velocità dei più lenti così che i programmatori non si accorgano della differenza. È questa l'accezione di "uniforme" che si intende in questo contesto. L'uniformità permette di fare previsioni sulle prestazioni, il che è importante per la scrittura di codice efficiente.

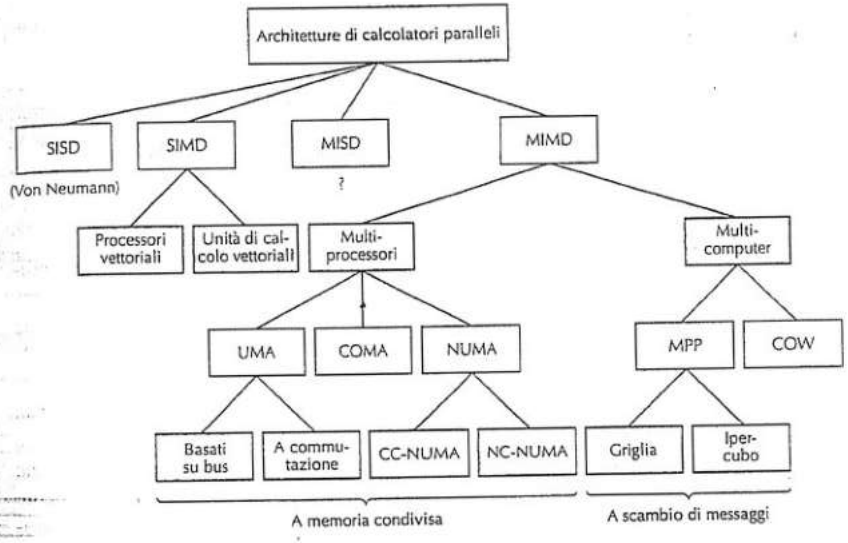


Figura 8.23 Tassonomia dei calcolatori paralleli.

Viceversa, nei multiprocessori NUMA questa proprietà non vale. Spesso c'è un modulo di memoria vicino a ogni CPU e il suo accesso è molto più veloce rispetto ai moduli distanti. Perciò, ai fini delle prestazioni, conta molto la collocazione dei dati e dei programmi. Anche le macchine COMA non sono uniformi, ma in un modo diverso. Ci occuperemo più tardi di queste categorie di macchine.

L'altra categoria principale delle macchine MIMD è costituita dai multicomputer che, a differenza dei multiprocessori, non hanno una memoria principale condivisa a livello di architettura. In altre parole, il sistema operativo di una CPU di un multicomputer non può accedere alla memoria di un'altra CPU tramite la sola esecuzione di un'istruzione `LOAD`. Deve invece spedire un messaggio e aspettare la relativa risposta. La capacità del sistema operativo di leggere parole a distanza effettuando una semplice `LOAD` è ciò che distingue i multiprocessori dai multicomputer. Abbiamo già detto che anche i programmi utente di un multicomputer possono essere resi in grado di accedere alle memorie distanti tramite istruzioni `LOAD` e `STORE`, però si tratta in questo caso di

un'illusione di cui si fa carico il sistema operativo e che non è sorretta dall'hardware. Si tratta di una differenza impercettibile, eppure molto importante. Per queste ragioni i multicomputer sono detti anche **NORMA** (*NO Remote Memory Access*).

È possibile individuare due categorie di multicomputer. La prima contiene processori di tipo **MPP** (*Massively Parallel Processor*), supercomputer molto costosi costituiti da molte CPU strettamente legate da una rete d'interconnessione ad alta velocità brevettata dal costruttore. L'IBM SP/3 ne è un esempio commerciale molto noto.

L'altra categoria raggruppa i PC consueti o le workstation, eventualmente montati in scaffali (*rack*), collegati tramite una tecnologia commerciale d'interconnessione pronta per l'uso. Dal punto di vista teorico non c'è molta differenza, ma i supercomputer giganti sono usati per scopi diversi rispetto alle reti di PC, assemblate dagli utenti a un costo che rappresenta una frazione molto piccola rispetto ai milioni di euro necessari all'acquisto di un MPP. Queste macchine fatte in casa vengono denominate in modi diversi, tra cui **NOW** (*Network of Workstations*), **COW** (*Cluster Of Workstations*) o semplicemente **cluster**.

8.3.2 Semantica della memoria

Anche se i multiprocessori mostrano la memoria alle CPU come un unico spazio condiviso degli indirizzi, spesso ci sono diversi moduli di memoria, ciascuno contenente una parte della memoria fisica. CPU e memorie sono collegate tramite una complessa rete d'interconnessione, come già illustrato nel Paragrafo 8.1.2. È possibile che diverse CPU tentino di leggere contemporaneamente la stessa parola di memoria, mentre altre CPU cercano di sovrascriverne il contenuto; ed è anche possibile che i messaggi di richiesta si sorpassino l'un l'altro durante il tragitto e che vengano quindi consegnati in un ordine diverso rispetto alla loro emissione. Si aggiunga poi il problema dell'esistenza di copie multiple di alcuni blocchi di memoria (per esempio nelle cache) e si capirà facilmente che è necessario adottare delle regole precise per prevenire il caos totale. In questo paragrafo vedremo che cosa voglia dire davvero condividere la memoria e come sia possibile agire nelle circostanze suddette.

Un modo per concepire la semantica della memoria è come contratto tra l'hardware della memoria e il software (Adve e Hill, 1990): se il software accetta di stare ad alcune regole, la memoria prende l'impegno di fornire certi risultati. L'analisi si sposta quindi sulla determinazione di queste regole, chiamate **modelli di consistenza**, per cui esistono molte proposte e altrettante implementazioni (Sorin et al., 2011).

Per chiarire la natura del problema, ipotizziamo che la CPU 0 scriva il valore 1 in una certa parola di memoria e che poco dopo la CPU 1 scriva il valore 2 nella stessa locazione. A questo punto la CPU 2 legge la parola e ottiene il valore 1. Che cosa dovrebbe fare il proprietario di quel calcolatore, riportarlo in negozio per una riparazione?

Consistenza stretta

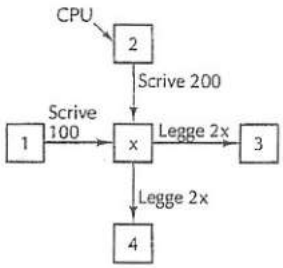
Il modello più semplice è la **consistenza stretta** (*strict consistency*), secondo cui ogni lettura di una locazione x deve sempre restituire il valore della scrittura più recente. I programmatori adorano questo modello, che però è pressoché impossibile da implementare se non con un solo modulo di memoria che serva tutte le richieste nell'ordine in cui

sono recapitate, senza effettuare caching né duplicazione di dati. Sfortunatamente questa implementazione renderebbe la memoria un collo di bottiglia per l'intero sistema e perciò non va presa in seria considerazione.

Consistenza sequenziale

Il secondo modello più promettente è la **consistenza sequenziale** (*sequential consistency*; Lamport, 1979). L'idea è la seguente: se ci sono più richieste di lettura e scrittura per una locazione, l'hardware effettua un rimescolamento (non deterministico) dell'ordine delle richieste e tutte le CPU vedono lo stesso ordine.

Esaminiamone il significato con un esempio. Si ipotizzi che la CPU 1 scriva il valore 100 nella parola x e 1 ns dopo la CPU 2 scriva il valore 200 nella stessa parola x. Quando è passato 1 ns dall'emissione della seconda scrittura (ma non necessariamente dal suo completamento) altre due CPU, la 3 e la 4, effettuano entrambe due letture di x in rapida successione, come mostrato dalla Figura 8.24(a). Le Figure 8.24(b)-(d) mostrano tre possibili ordinamenti dei sei eventi (due scritture e quattro letture): nella prima figura la CPU 3 ottiene (200, 200) e la CPU 4 ottiene (200, 200); nella Figura 8.24(c) ottengono rispettivamente (100, 200) e (200, 200); nella Figura 8.22(d) ottengono rispettivamente (100, 100) e (200, 100). Sono tutte combinazioni lecite, come lo sono le altre che non abbiamo elencato. Si noti che non esiste un singolo valore "corretto".



(a)	(b)	(c)	(d)
	W100	W100	W200
	W200	R3 = 100	R4 = 200
	R3 = 200	W200	W100
	R3 = 200	R4 = 200	R3 = 100
	R4 = 200	R3 = 200	R4 = 100
	R4 = 200	R4 = 200	R3 = 100

Figura 8.24 (a) Due CPU che scrivono una parola di memoria letta da altre due CPU. (b)-(d) Tre permutazioni dell'ordine temporale delle due scritture e delle quattro letture.

Comunque vada, è impossibile che la CPU 3 ottenga (100, 200) mentre la CPU 4 ottiene (200, 100), e questa è l'essenza stessa della consistenza sequenziale. Se ciò potesse capitare, vorrebbe dire che, secondo la CPU 3, la scrittura di 100 da parte della CPU 1 è stata completata prima della scrittura di 200 da parte della CPU 2, e questo va bene. Però vorrebbe dire anche che, secondo la CPU 4, la scrittura di 200 da parte della CPU 2 è stata completata prima della scrittura di 100 da parte della CPU 1. Anche questo risultato è possibile in sé, ma il fatto è che la consistenza sequenziale garantisce l'esistenza di un unico ordinamento globale delle scritture visibile a tutte le CPU. Se la CPU 3 osserva per prima la scrittura di 100, così deve essere anche per la CPU 4.

Nonostante la consistenza sequenziale sia una regola meno potente della consistenza stretta, si rivela comunque molto utile. Infatti stabilisce che, quando più eventi si verificano in modo concorrente, esiste un ordine secondo cui avvengono, determinato forse dalla temporizzazione o dal caso, ma si tratta pur sempre di un ordine che deve essere osservato da tutti i processori. Benché questa osservazione appaia ovvia, passiamo ora a modelli di consistenza che non garantiscono neanche questo.

Consistenza di processore

La **consistenza di processore** (*processor consistency;* Goodman, 1989) è un modello di consistenza meno rigido, ma più facile da implementare su multiprocessori grandi. Manifesta due proprietà:

1. le scritture effettuate da qualsiasi CPU sono percepite da tutte le altre CPU nello stesso ordine in cui sono state emesse;
2. per ogni parola di memoria, tutte le CPU vedono lo stesso ordine di scritture al suo interno.

La prima di queste due importanti proprietà afferma che, se la CPU 1 emette in sequenza le scritture di valori 1A, 1B e 1C in una locazione di memoria, tutti gli altri processori le vedono in quello stesso ordine. Ciò equivale a dire che, ogni processore che osservi il valore assunto da quella locazione di memoria mediante un ciclo di letture, non potrà mai ritenere il valore 1B scritto prima del valore 1A, e così via. La seconda proprietà serve affinché ogni parola di memoria contenga un valore non ambiguo al termine di numerose scritture da parte di più CPU. Tutti devono concordare su chi ha scritto per ultimo.

Pur con questi vincoli, al progettista resta ampio margine di scelta. Si consideri che cosa succede se la CPU 2 emette le scritture 2A, 2B e 2C concorrentemente alle tre scritture della CPU 1. Le altre CPU che stanno osservando la memoria mediante letture ripetute, vedranno un rimescolamento delle sei scritture, per esempio 1A, 1B, 2A, 2B, 1C, 2C oppure 2A, 1A, 2B, 2C, 1B, 1C o altri ancora. La consistenza di processore *non* garantisce che tutte le CPU vedano lo stesso ordinamento (a differenza della consistenza sequenziale). Perciò è del tutto legittimo per l'hardware comportarsi in questo modo e mostrare a ogni CPU un ordinamento potenzialmente diverso. Ciò che è garantito invece è che nessuna CPU veda una sequenza in cui 1B viene prima di 1A, e così via. L'ordine relativo delle scritture emesse da ogni singola CPU viene mantenuto comunque.

Va detto che alcuni autori definiscono la consistenza di processore in modo diverso e che non richiedono la seconda proprietà.

Consistenza debole

Il modello successivo, la **consistenza debole** (*weak consistency*), non garantisce nemmeno l'ordinamento delle scritture effettuate da una sola CPU (Dubois et al., 1986). In una memoria debolmente consistente una CPU potrebbe vedere 1A prima di 1B e un'altra potrebbe vedere l'esatto contrario. Tuttavia, per riportare un po' di ordine nel caos, le memorie debolmente consistenti sono dotate di variabili e di operazioni di sincronizzazione. Quando viene eseguita una sincronizzazione, vengono completate tutte le scritture pendenti e non è possibile effettuarne di nuove finché non termina la sincronizzazione. La sincronizzazione effettua un "flush della pipeline" (scarica le operazioni in coda) e conduce la memoria a uno stato stabile, senza operazioni pendenti. Le operazioni di sincronizzazione sono esse stesse sequenzialmente consistenti, cioè anche se vengono emesse da più CPU, tutti i processori percepiscono lo stesso ordine globale della loro esecuzione.

In questo modello di consistenza, il tempo è diviso in epoche ben definite e delimitate dalle sincronizzazioni (sequenzialmente consistenti), come illustrato dalla Figura 8.25. Non è garantito l'ordine relativo tra 1A e 1B, cioè CPU diverse potrebbero osservare le due scritture in ordine diverso (una CPU potrebbe vedere 1A, 1B, un'altra potrebbe vedere 1B, 1A). Però tutte le CPU devono vedere 1B prima di 1C, perché l'operazione di sincronizzazione forza il completamento di 1A, 1B e 2A prima che 1C, 2B, 3A o 3B possano cominciare. La sincronizzazione è un modo con cui il software può imporre un certo ordinamento nella sequenza di eventi, ma al prezzo di un flush della pipeline di memoria, che richiede un certo tempo e rallenta dunque, in una certa misura, la macchina. Effettuare spesso questa operazione può diventare un problema.

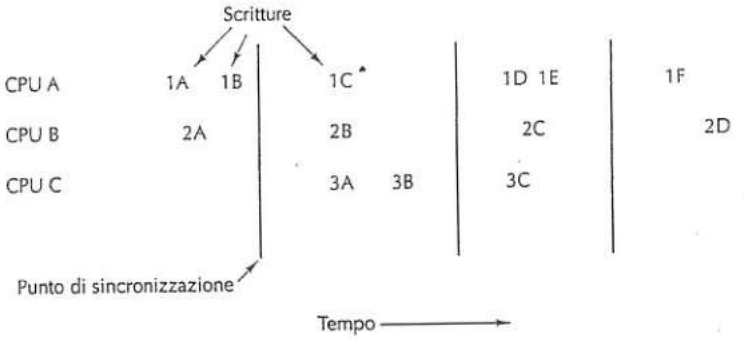


Figura 8.25 La memoria a consistenza debole utilizza la sincronizzazione per suddividere il tempo in una successione di epoche.

Consistenza dopo rilascio

La consistenza debole è abbastanza inefficiente perché comporta il completamento di tutte le operazioni di memoria pendenti e il trattenimento di tutte le operazioni nuove finché non vengano terminate le precedenti. La **consistenza dopo rilascio** (*release consistency*) costituisce un miglioramento e adotta un modello simile alle sezioni critiche (Gharachorloo et al., 1990). L'idea alla base di questo modello è la seguente: quando un processo esce da una regione critica, non è necessario richiedere il completamento immediato di tutte le scritture. Basta solo assicurarsi che vengano effettuate prima del reingresso di un altro processo nella regione critica.

Secondo questo modello, l'operazione di sincronizzazione fornita dalla consistenza debole viene suddivisa in due operazioni differenti. Per leggere o scrivere una variabile condivisa, la CPU (cioè il suo software) deve prima eseguire un'operazione `acquire`

sulla variabile di sincronizzazione per acquisire l'accesso esclusivo ai dati condivisi. Quindi la CPU può usarli come vuole, sia in lettura sia in scrittura. Al termine della sua attività, la CPU invoca l'operazione `release` sulla variabile di sincronizzazione per indicare il suo rilascio. La `release` non forza il completamento di tutte le scritture pendenti, bensì attende per il proprio completamento che siano avvenute tutte le scritture emesse precedentemente. Inoltre non ritarda l'avvio di nuove operazioni di memoria.

All'emissione di una `acquire`, per prima cosa si verifica se tutte le `release` precedenti sono state completate. Se così non è, la `acquire` resta sospesa fino al loro termine (dunque finché non sono state completate tutte le scritture precedenti). Così facendo, se una `acquire` è abbastanza lontana dalla `release` precedente, non deve aspettare alcunché prima di entrare nella regione critica. Se invece avviene troppo presto, dovrà essere ritardata fino al completamento di tutte le `release` pendenti, per garantire il corretto aggiornamento delle variabili della sezione critica. Si tratta di uno schema leggermente più complesso della consistenza debole, ma presenta il vantaggio significativo di non ritardare le istruzioni così spesso, pur mantenendo la consistenza.

La consistenza di memoria non è un capitolo chiuso, la ricerca di nuovi modelli va avanti (Naeem et al., 2011; Sorin et al., 2011; Tu et al., 2010).

8.3.3 Architetture di multiprocessori simmetrici UMA

I multiprocessori più semplici si basano su un solo bus, come illustrato nella Figura 8.26(a). Ci sono due o più CPU, uno o più moduli di memoria, e tutti usano lo stesso bus per la comunicazione. Quando una CPU vuole leggere una parola di memoria, per prima cosa verifica se il bus è occupato; se non lo è, può inserire nel bus l'indirizzo della parola voluta, attivare qualche segnale di controllo e aspettare finché la memoria non spedisce sul bus la parola desiderata.

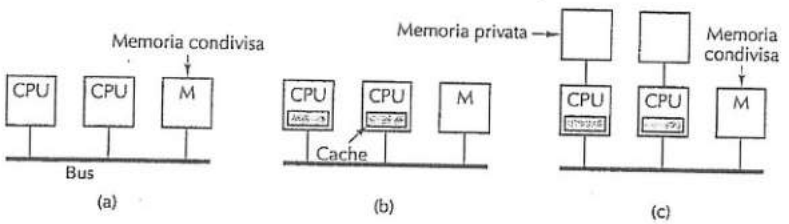


Figura 8.26 Tre multiprocessori basati su bus. (a) Senza caching. (b) Con caching. (c) Con caching e memorie private.

Se invece il bus è occupato, la CPU aspetta che si liberi; sta proprio qui il problema di questo progetto. Se le CPU sono due o tre, la contesa del bus è ancora gestibile; se sono 32 o 64 allora diventa intrattabile perché il sistema sarà limitato completamente dalla larghezza di banda del bus, e la maggior parte delle CPU resterà inattiva per gran parte del tempo.

La soluzione del problema è l'aggiunta di una cache a ogni CPU, come rappresentato nella Figura 8.26(b). La cache può trovarsi all'interno del chip della CPU, vicino alla CPU, sulla scheda del processore o in una posizione intermedia. La cache locale consente di soddisfare molte più letture attingendo al proprio interno, perciò ci sarà molto meno traffico sul bus e il sistema potrà gestire più CPU. Il caching risulta qui una scelta vincente. Tuttavia, come vedremo tra poco, non è semplice mantenere la coerenza delle cache.

Una possibilità alternativa è il progetto della Figura 8.26(c), in cui ogni CPU dispone non solo di una cache, ma anche di una memoria locale privata, cui può accedere grazie a un bus dedicato (privato). Per sfruttare questa configurazione in modo ottimale, il compilatore dovrebbe usare la memoria privata per contenere i sorgenti del programma, le stringhe, le costanti e gli altri dati in sola lettura, gli stack e le variabili locali. La memoria condivisa verrebbe usata solo per la scrittura delle variabili condivise. Questa collocazione riduce in molti casi il traffico sul bus, ma richiede la cooperazione attiva da parte del compilatore.

Cache snooping

Il ragionamento fatto circa le prestazioni è corretto, ma abbiamo liquidato troppo velocemente una questione fondamentale. Se la memoria ha consistenza sequenziale, che cosa succede nel caso in cui la CPU 1 ha nella propria cache una linea e la CPU 2 cerca di leggere una parola dalla stessa linea di cache? In assenza di regole speciali, quest'ultima non farebbe altro che procurarsi una copia della linea da inserire nella propria cache. In teoria è ammissibile che una linea sia presente in due cache diverse. Se poi la CPU 1 modifica la linea e, immediatamente dopo, la CPU 2 legge la propria copia, ma si troverà a manipolare **dati scaduti** (*stale data*), violando così il contratto tra software e memoria. Il programma in esecuzione sulla CPU 2 viene danneggiato da questo comportamento.

A tale proposito la letteratura specializzata parla del **problema della coerenza** o **consistenza della cache**. Si tratta di un problema molto grave che, se non risolto, preclude l'uso del caching e i multiprocessori orientati al bus sarebbero così limitati a due o tre CPU. In ragione dell'importanza della questione, sono state proposte negli anni molte soluzioni (per esempio Goodman, 1983; Papamarcos e Patel, 1984) che definiscono un **protocollo di coerenza delle cache** e, sebbene differiscano nei dettagli, impediscono la presenza simultanea di versioni discordanti di una stessa linea all'interno di due o più cache. L'unità di trasferimento e di memorizzazione di una cache si dice *linea di cache* ed è solitamente di 32 o 64 byte.

Tutte le soluzioni richiedono controllori di cache progettati per essere in grado di "origliare" sul bus, cioè di monitorare tutte le richieste che transitano sul bus e che provengono da altre CPU o cache, al fine di intraprendere le azioni opportune. Questi dispositivi si dicono **snooping cache** o **snoopy cache**, cioè cache che letteralmente *ficcano il naso* nel bus. Il protocollo di coerenza delle cache è composto dall'insieme di regole osservate dalle cache, dalla CPU e dalla memoria per prevenire la presenza di versioni diverse dei dati all'interno di varie cache.

Il più semplice protocollo di coerenza delle cache si chiama **write through** e può essere capito facilmente attraverso le quattro situazioni elencate nella Figura 8.27. Quando una CPU tenta di leggere una parola che non si trova nella sua cache, evento

detto *read miss*, il controllore carica nella cache la linea contenente quella parola. La linea proviene dalla memoria che, secondo questo protocollo, viene mantenuta costantemente aggiornata. Le successive richieste di letture possono essere soddisfatte dalla cache (*read hit*).

In seguito a un fallimento in scrittura, evento detto *write miss*, la parola modificata viene scritta in memoria principale, mentre la linea contenente la parola *non* viene caricata nella cache. In caso di successo in scrittura (*write hit*) viene aggiornata la cache e inoltre la parola viene scritta direttamente in memoria principale. La caratteristica più importante di questo protocollo è che tutte le operazioni di scrittura agiscono sulla memoria, che risulta perciò sempre aggiornata.

Riconsideriamo tali azioni, questa volta dal punto di vista di una *snooping cache* (colonna più a destra nella Figura 8.27). Chiamiamo cache 1 la cache che effettua le azioni e cache 2 la cache "ficcanaso". Dopo un read miss, la cache 1 effettua una richiesta di caricamento di una linea dalla memoria; la cache 2 osserva questa richiesta senza intervenire. Dopo un read hit la richiesta viene soddisfatta localmente, perciò non viene inviata alcuna richiesta sul bus, quindi la cache 2 non sa della lettura della cache 1.

Azione	Richiesta locale	Richiesta a distanza
Read miss	Preleva dati in memoria	Nessuna
Read hit	Usa dati dalla cache locale	"
Write miss	Aggiorna i dati in memoria	"
Write hit	Aggiorna la cache e la memoria	Invalida l'elemento di cache

Figura 8.27 Protocollo write-through di coerenza delle cache.

Le situazioni in scrittura sono più interessanti. Se la CPU 1 effettua una scrittura, la cache 1 invia una richiesta di scrittura sul bus, sia in caso di miss sia di hit. A ogni scrittura, la cache 2 verifica se possiede la parola che deve essere scritta. In caso negativo, dal suo punto di vista si tratta di una semplice richiesta a distanza che provoca un write miss, perciò non intraprende alcuna azione (un fallimento a distanza nella Figura 8.27 significa che la parola non è presente nella snooping cache, indipendentemente dalla sua eventuale presenza nella cache 1; una richiesta può causare localmente un hit nella prima cache e un miss nella snooping cache, o viceversa).

Se invece la cache 1 scrive una parola presente nella cache 2 (richiesta a distanza che ha successo), quest'ultima deve intraprendere qualche azione per evitare di avere dati scaduti, perciò contrassegna come non valido l'elemento di cache contenente la parola modificata. Ciò equivale a rimuovere l'elemento dalla cache. Poiché tutte le cache monitorano tutte le richieste di bus, ogni scrittura di una parola provoca l'aggiornamento della cache del richiedente e della memoria, la rimozione dei dati scaduti dalle altre cache, e così si previene l'insorgenza di versioni incoerenti.

Naturalmente è possibile che la CPU della cache 2 voglia leggere quella stessa parola al ciclo successivo. In tal caso, la cache 2 leggerebbe la parola dalla memoria, che è perfettamente aggiornata. Dopo la lettura la cache 1, la cache 2 e la memoria avrebbero

tutte una copia fedele del dato. Se una delle due CPU effettuasse ora una scrittura, l'elemento di cache dell'altra CPU verrebbe rimosso e la memoria aggiornata.

Sono possibili molte variazioni di questo semplice protocollo. Per esempio, una snooping cache potrebbe evitare di invalidare un proprio elemento a seguito di un write hit, ma anzi, potrebbe aggiornarlo con il nuovo valore. Dal punto di vista concettuale questo comportamento è equivalente a un'invalidazione seguita da un caricamento da memoria principale. In tutti i protocolli di cache bisogna scegliere se adottare una **strategia di aggiornamento** o una **d'invalidazione**.

La scelta produce protocolli che esibiscono prestazioni differenti a seconda dei carichi di lavoro. I messaggi di aggiornamento contengono le informazioni da adeguare e sono perciò più grandi di quelli d'invalidazione, ma prevengono miss futuri della cache.

Un'altra variante prevede di effettuare un caricamento nella snooping cache anche in caso di write miss. La correttezza dell'algoritmo non è compromessa dal caricamento, ma le prestazioni lo sono. La domanda è: "Qual è la probabilità che una parola appena scritta venga riscritta presto?" Se è alta, allora ci sono argomenti a favore del caricamento nella cache dopo un write miss, e si parla di una **politica write-allocate**. Se invece la probabilità è bassa, è preferibile non aggiornare dopo un write miss: se la parola viene letta poco dopo, verrà caricata comunque a seguito di un read miss, perciò si perde poco a non caricarla dopo un write miss.

Così come gran parte delle soluzioni semplici, anche questa è inefficiente. Ogni operazione di scrittura raggiunge la memoria tramite il bus, che quindi risulterà un collo di bottiglia anche con un numero modesto di CPU. Per contenere l'aumento del traffico sul bus sono stati ideati altri protocolli di cache, che hanno in comune la proprietà che non tutte le scritture raggiungono direttamente la memoria. Quando viene modificata una linea di cache, viene asserito al suo interno un bit che annota la sua validità in contrapposizione alla memoria. La linea così modificata dovrà essere scritta prima o poi in memoria, ma ciò si potrebbe verificare dopo molte scritture al suo interno. Questi tipi di protocollo si chiamano **protocolli write-back**.

Protocollo MESI di coerenza delle cache

MESI è un protocollo di coerenza delle cache molto diffuso e di tipo *write-back* (Papamarcos e Patel, 1984). Si chiama così perché si avvale degli stati *Modified, Exclusive, Shared* e *Invalid*, ed è basato su un precedente protocollo detto **write-once** (Goodman, 1983). Il protocollo MESI è utilizzato dal Pentium 4 e da molte altre CPU per lo snooping sul bus. I quattro stati in cui si possono trovare gli elementi della cache sono i seguenti.

1. Non valido: l'elemento di cache non contiene dati validi.
2. Condiviso: la linea potrebbe essere contenuta in più di una cache; la memoria è aggiornata.
3. Esclusivo: nessun'altra cache contiene la linea; la memoria è aggiornata.
4. Modificato: l'elemento è valido; la memoria non lo è; non ci sono altre copie della linea.

Quando una CPU viene avviata, tutti gli elementi di cache sono contrassegnati come non validi. Alla prima lettura la linea referenziata viene prelevata dalla memoria e inserita

nella cache della CPU; il suo stato è esclusivo, perché è la sola copia presente in una cache, come illustrato nella Figura 8.28(a) con riferimento alla CPU 1 dopo la lettura della linea *A*.

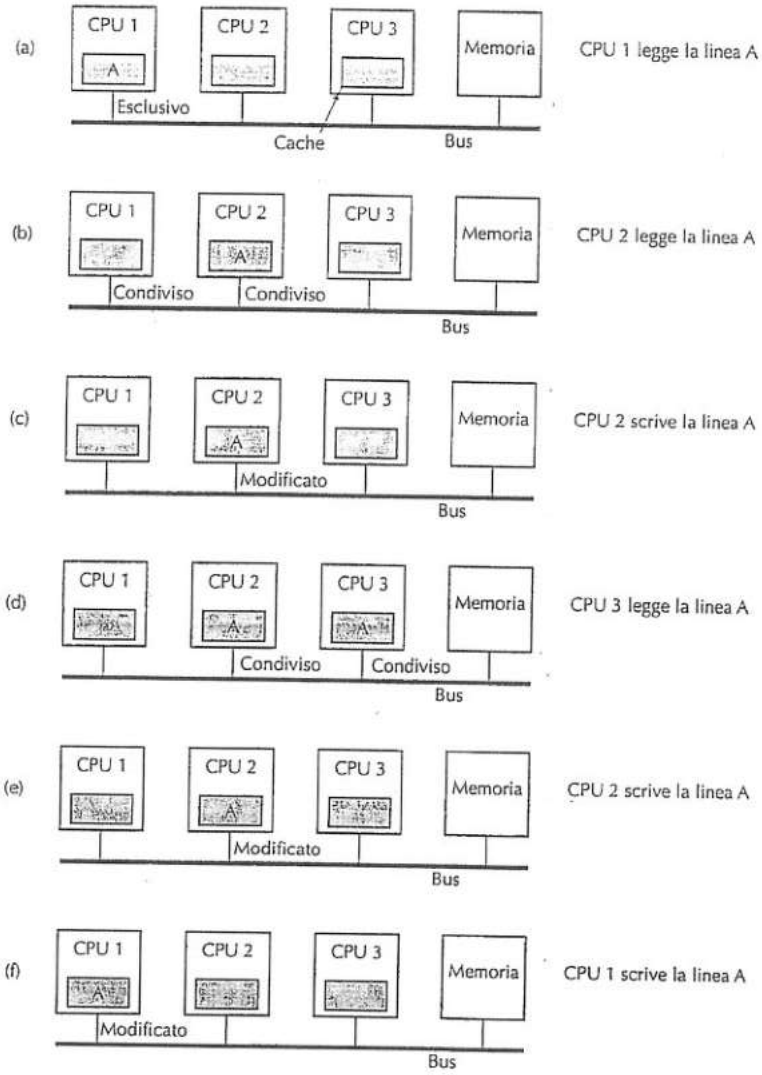


Figura 8.28 Protocollo MESI di coerenza delle cache.

Le letture successive a opera di quella CPU usano l'elemento della cache e non transitano sul bus. È possibile che un'altra CPU prelevi la stessa linea di cache, ma grazie allo snooping la CPU 1 sa di non essere più la sola detentrice e annuncia sul bus di possedere una copia della linea. Entrambe le copie vengono contrassegnate come S (*Shared*), come mostrato nella Figura 8.28(b). Detto altrimenti, lo stato S significa che la linea si trova in una o più cache per la lettura e che la memoria è aggiornata. Le letture di una CPU all'interno di una linea nello stato S non si avvalgono del bus e non ne modificano lo stato.

Si consideri ora che cosa succede se la CPU 2 scrive in una sua linea di cache che si trova nello stato S. La CPU emette un segnale d'invalidazione sul bus, indicando alle altre CPU di eliminare le proprie copie. La copia passa ora allo stato M, come mostrato nella Figura 8.28(c). La linea non viene scritta in memoria. Val la pena notare che se una linea si trova nello stato E al momento di una scrittura, non ci vuole alcun segnale d'invalidazione a beneficio delle altre cache, perché è chiaro che non esistono altre copie.

Si consideri poi che cosa succede quando la CPU 3 legge la linea. La CPU 2 detiene ora la linea e sa che la copia in memoria non è valida, perciò invia sul bus un segnale che indica alla CPU 3 di attendere mentre la linea viene scritta in memoria. Al completamento dell'operazione, la CPU 3 preleva la sua copia e la linea viene contrassegnata come condivisa da entrambe le cache, come mostrato dalla Figura 8.26(d). Dopo di ciò, la CPU 2 scrive ancora nella linea, il che invalida la copia della CPU 3 (Figura 8.26(e)).

Infine anche la CPU 1 scrive una parola nella linea. La CPU 2 osserva il tentativo di scrittura e invia un segnale sul bus che richiede alla CPU 1 di attendere la scrittura della linea in memoria. Al completamento dell'operazione contrassegna la propria linea come non valida, perché sa che un'altra CPU sta per modificarla. A questo punto ci possiamo trovare nella situazione in cui una CPU sta scrivendo all'interno di una linea fuori dalla cache. Se si usa la politica write-allocate, la linea viene caricata nella cache e contrassegnata come modificata, come succede nella Figura 8.28(f). Altrimenti, la scrittura avviene direttamente in memoria e la linea non viene caricata in alcuna cache.

Multiprocessori UMA con commutatori crossbar

Per quanto se ne possa ottimizzare l'uso, la presenza di un solo bus limita il parallelismo dei multiprocessori UMA a circa 16 o 32 CPU. Per superare questo valore ci vuole un diverso tipo di rete d'interconnessione. Il circuito più semplice che collega n CPU a k memorie è il **commutatore crossbar** ("a traversa") mostrato nella Figura 8.28. I commutatori crossbar sono stati usati per decenni all'interno della cabine di commutazione telefoniche per poter collegare in qualsiasi modo un certo numero di linee in ingresso a un insieme di linee in uscita. A ogni intersezione tra una linea orizzontale (in ingresso) e una verticale (in uscita) c'è un **crosspoint** ("punto di incrocio"). Un crosspoint è esso stesso un piccolo commutatore che può essere aperto o chiuso elettronicamente, a seconda che si voglia collegare o meno le linee corrispondenti. La Figura 8.29(a) mostra tre crosspoint chiusi contemporaneamente, consentendo così la comunicazione simultanea tra le seguenti coppie (CPU, memoria): (001, 000), (101, 101) e (110, 010). Sono possibili molte altre combinazioni; infatti il numero di combinazioni è uguale ai diversi modi in cui si possono sistemare otto torri su di una scacchiera senza che si attacchino[4].

4 Si tratta del famoso problema delle Non-Attacking Rooks (*N.d.Rev.*).

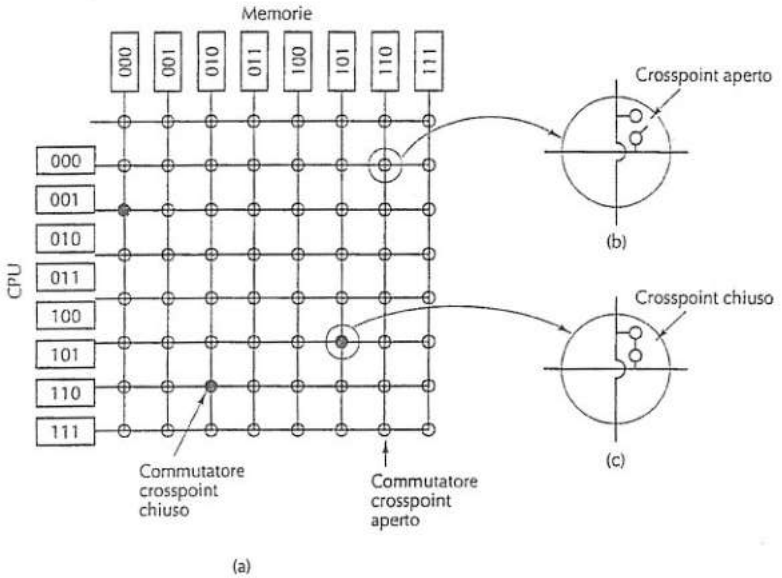


Figura 8.29 (a) Commutatore crossbar 8 × 8. (b) Crosspoint aperto. (c) Crosspoint chiuso.

Una delle proprietà più apprezzabili dei commutatori crossbar è che costituiscono **reti non bloccanti**, ovvero non succede mai che venga negata a una CPU la connessione di cui ha bisogno perché la linea è già occupata (ipotizzando che il modulo di memoria desiderato sia disponibile). Inoltre non c'è bisogno di alcuna pianificazione: anche se sono già state stabilite sette connessioni qualsiasi, è sempre possibile collegare un'ottava CPU a una nuova memoria. Vedremo più avanti alcuni schemi d'interconnessione che non hanno queste proprietà.

Uno dei difetti di questi crossbar è che il numero di crosspoint cresce come n^2. Un progetto può pensare di adottare un commutatore crossbar se il sistema in questione è di medie dimensioni. Più avanti nel capitolo incontreremo un progetto di questo tipo: il Sun Fire E25K. D'altra parte, se ci sono 1000 CPU e 1000 moduli di memoria ci vorrebbero un milione di crosspoint, il che non è fattibile in pratica. In una situazione del genere c'è bisogno di qualcosa di molto diverso.

Multiprocessori UMA con reti a commutazione multilivello

Quel "qualcosa di molto diverso" può essere l'umile commutatore 2 × 2 della Figura 8.30(a). Questo commutatore ha due ingressi e due uscite; i messaggi che provengono da ciascuno dei due input possono essere indirizzati verso ognuna delle due uscite. Ipotizziamo per comodità che i messaggi siano composti di quattro parti, come schematizzato nella Figura 8.30(b). *Modulo* specifica la memoria da usare. *Indirizzo* specifica un indirizzo all'interno di quel modulo. *Opcode* stabilisce il tipo di operazione, per esempio

READ o WRITE. Infine il campo opzionale *Valore* potrebbe contenere un operando, come una parola di 32 bit da scrivere con una WRITE. Il commutatore esamina il campo *Modulo* e lo usa per determinare la linea X o Y lungo cui inviare il massaggio.

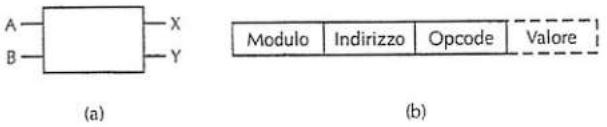


Figura 8.30 (a) Commutatore 2 × 2. (b) Formato di messaggio.

I nostri commutatori 2 × 2 possono essere composti in molti modi per dar vita a una più grande **rete a commutazione multilivello**. Una possibilità economica e senza fronzoli è la **rete omega**, illustrata nella Figura 8.31. In questo caso ci sono otto CPU connesse a otto memorie tramite 12 commutatori. In generale, per collegare n CPU a n memorie ci vogliono $\log_2 n$ livelli di $n/2$ commutatori ciascuno, per un totale di $(n/2) \log_2 n$ commutatori: è un numero molto inferiore a n^2, specie per valori grandi di n.

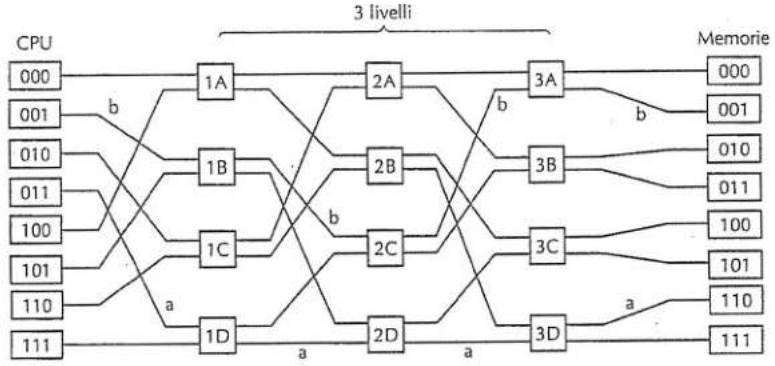


Figura 8.31 Rete di connessione omega.

Spesso ci si riferisce alla struttura dei collegamenti di queste reti con il termine di **mescolamento perfetto** (*perfect shuffle*) perché il mescolamento dei segnali a ogni livello somiglia a un mazzo di carte tagliato in due e rimescolato carta per carta. Per capire il funzionamento di una rete omega, supponiamo che la CPU 011 voglia leggere una parola dal modulo di memoria 110. La CPU spedisce un messaggio READ contenente 110 nel campo *Modulo* al commutatore 1D. Il commutatore preleva il primo bit (quello più a sinistra) da 110 e lo usa per l'instradamento. Lo 0 instrada verso l'uscita

in alto, 1 verso la linea in basso. Poiché il bit vale 1 il messaggio viene instradato a 2D attraverso l'uscita in basso.

Tutti i commutatori di secondo livello, compreso 2D, usano per l'instradamento il secondo bit. In questo caso è ancora un 1 e così il messaggio viene spedito a 3D attraverso l'uscita in basso. Qui viene testato il terzo bit che vale 0. Di conseguenza il messaggio fuoriesce dalla linea in alto e raggiunge la memoria 110, come richiesto. Il cammino percorso da questo messaggio è indicato con una lettera *a* nella Figura 8.31.

Mentre i messaggi attraversano la rete di commutazione, i bit più a sinistra del campo *Modulo* diventano progressivamente inutili. Possono perciò essere riutilizzati registrando al loro interno i numeri delle linee in ingresso, in modo da specificare il percorso di ritorno del messaggio di risposta. Nel caso del cammino *a* le linee in ingresso sono 0 (ingresso in alto di 1D), 1 (ingresso in basso di 2D) e 1 (ingresso in basso di 3D). La risposta viene instradata usando 011, che questa volta va letto da destra a sinistra.

Mentre si svolgono queste operazioni, la CPU 001 vuole scrivere una parola nel modulo di memoria 001. Il procedimento è analogo al precedente: il messaggio viene instradato rispettivamente attraverso le uscite in alto, in alto e poi in basso, come indicato dalla lettera *b*. Alla consegna del messaggio *Modulo* contiene 001, che rappresenta il cammino percorso. Le due richieste possono procedere parallelamente dal momento che usano commutatori, linee e memorie disgiunti.

Si consideri ora che cosa accadrebbe se la CPU 000 volesse accedere al modulo di memoria 000. La sua richiesta entrerebbe in conflitto con quella della CPU 001 in corrispondenza dello switch 3A. Una delle due dovrebbe aspettare; a differenza del commutatore crossbar la rete omega è una **rete bloccante**. Non è possibile elaborare simultaneamente qualsiasi insieme di richieste e i conflitti possono verificarsi per l'uso di un collegamento o di un commutatore sia tra richieste che *vanno* alla memoria, sia tra quelle che *provengono* da essa.

È quindi auspicabile riuscire a distribuire i riferimenti alla memoria in modo uniforme rispetto ai moduli. Una tecnica comune è quella di usare i bit meno significativi come numero di modulo. Si consideri per esempio uno spazio degli indirizzi orientato al byte in un calcolatore che accede prevalentemente a parole di 32 bit. I due bit meno significativi si troveranno spesso a 00, ma i 3 bit successivi saranno distribuiti uniformemente. Se si usano questi 3 bit come numero di modulo, le parole indirizzate in modo consecutivo si troveranno in moduli consecutivi. Un sistema di memoria in cui le parole consecutive si trovano in moduli diversi si dice **interlacciato**. Le memorie interlacciate massimizzano il parallelismo perché la maggior parte dei riferimenti a memoria coinvolge indirizzi consecutivi. È altresì possibile progettare reti di commutazione non bloccanti e che offrono cammini multipli da ogni CPU a ogni modulo di memoria, per meglio distribuire il traffico di rete.

8.3.4 Multiprocessori NUMA

Dovrebbe essere ora chiaro che i multiprocessori UMA a bus singolo sono spesso limitati a non più di qualche dozzina di CPU, e i multiprocessori con commutazione crossbar o multilivello necessitano di molto hardware (costoso) e perciò non possono essere molto più grandi. Per superare le 100 CPU bisogna rinunciare a qualcosa, spesso all'idea

che tutti i moduli di memoria richiedano lo stesso tempo d'accesso. Questa rinuncia conduce ai multiprocessori **NUMA** (*NonUniform Memory Access*) che, come i cugini UMA, gestiscono un solo spazio degli indirizzi per tutte le CPU, ma diversamente da loro, garantiscono un accesso più veloce ai moduli vicini rispetto a quelli lontani. Dunque tutti i programmi UMA continuano a funzionare sulle macchine NUMA, ma le prestazioni degradano molto rispetto alle macchine UMA che hanno la stessa frequenza di clock.

Tutte le macchine NUMA hanno sempre tre caratteristiche che insieme le distinguono dagli altri multiprocessori.

1. C'è un solo spazio degli indirizzi visibile a tutte le CPU.
2. L'accesso alla memoria distante si effettua tramite istruzioni LOAD e STORE.
3. L'accesso alla memoria distante è più lento di quello alla memoria locale.

Quando il tempo di accesso alla memoria distante non è nascosto (perché non c'è caching) si parla di sistemi **NC-NUMA**. Viceversa, quando sono presenti cache coerenti si parla di **CC-NUMA** (almeno tra gli esperti di hardware). Gli esperti di software spesso usano il termine **DSM hardware** perché si tratta fondamentalmente di una memoria condivisa distribuita, ma questa volta implementata in hardware con una dimensione di pagina piccola.

Una macchina NC-NUMA ante litteram fu il Carnegie-Mellon Cm*, illustrato in forma semplificata nella Figura 8.32 (Swan et al., 1977). Cm* conteneva varie CPU LSI-11, ciascuna dotata di una piccola memoria cui accedeva tramite un bus locale (il processore LSI-11 era una versione a singolo chip del DEC PDP-11, un minicomputer molto diffuso negli anni '70). Inoltre, i chip LSI-11 erano collegati attraverso un bus di sistema. Quando la MMU (opportunamente modificata) riceveva una richiesta di accesso a memoria, stabiliva per prima cosa se la parola si trovava in memoria locale. In tal caso la richiesta veniva inoltrata lungo il bus locale per ottenere la parola. In caso contrario, la richiesta veniva instradata lungo il bus di sistema verso il processore che conteneva la parola, che rispondeva di conseguenza. Naturalmente questa seconda evenienza impiegava molto più tempo della prima. Anche se un programma poteva girare senza problemi usando solo la memoria distante, impiegava un tempo circa 10 volte maggiore rispetto alla sua esecuzione basata sulla sola memoria locale.

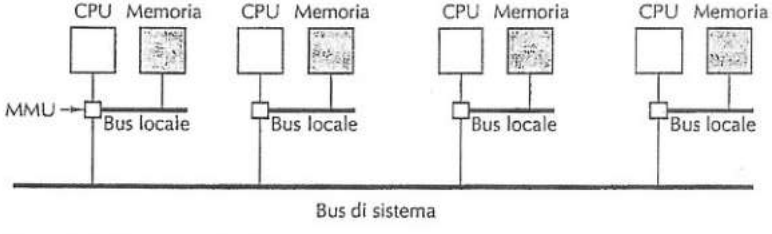


Figura 8.32 Macchina NUMA con due livelli di bus. Il Cm* fu il primo multiprocessore ad adottare questo schema.

La coerenza della memoria in una macchina NC-NUMA è garantita dal fatto che non c'è *caching*. Ogni parola di memoria si può trovare in una sola locazione, perciò non c'è pericolo che ne esista una copia difforme: non ci sono copie. D'altro canto, la collocazione di una pagina in memoria diventa molto importante perché il degrado delle prestazioni dovuto a un cattivo posizionamento dei dati è molto rilevante. In ragione di ciò, le macchine NC-NUMA usano un software molto elaborato per spostare le pagine all'interno del sistema, in modo da garantire le massime prestazioni.

Generalmente esiste un processo demone chiamato **scanner delle pagine**, eseguito a intervalli di pochi secondi. Il suo compito è esaminare le statistiche di utilizzo e spostare le pagine nel tentativo d'incrementare le prestazioni. Se una pagina sembra posizionata male, lo scanner delle pagine la rimuove dalla mappa delle pagine e così l'accesso successivo a essa causerà un errore di pagina. L'errore di pagina è l'occasione per decidere dove "collocare" la pagina, probabilmente in una memoria diversa da quella in cui si trovava prima. Per evitare il *thrashing* si stabilisce la regola secondo cui, una volta caricata, una pagina viene lasciata al proprio posto per almeno un dato numero di secondi. Sono stati studiati molti algoritmi per la paginazione nelle macchine NC-NUMA, ma nessuno di loro si comporta al meglio in ogni circostanza (LaRowe ed Ellis, 1991). Le prestazioni dipendono dall'applicazione.

Multiprocessori NUMA con cache coerente

I progetti di multiprocessori come quello della Figura 8.32 risultano insoddisfacenti al crescere delle dimensioni dei sistemi, perché non prevedono il caching. Dover accedere alla memoria remota ogni volta che si fa riferimento a una parola di memoria non locale vuol dire pagare un alto prezzo in termini di prestazioni. Se però si aggiunge il caching, bisogna assicurare anche la coerenza delle cache. Un modo possibile per garantirla è permettere lo snooping del bus di sistema. Si tratta di una soluzione facile da implementare dal punto di vista tecnico, ma che ben presto si rivela praticamente inattuabile al crescere del numero di CPU. La costruzione di multiprocessori veramente grandi richiede un approccio completamente diverso.

Attualmente il modo più diffuso per costruire grandi **CC-NUMA** (*Cache Coherent NUMA Multiprocessor*, "multiprocessori NUMA con coerenza della cache") è il sistema **multiprocessore basato su directory**. L'idea è quella di mantenere un database per ricordare la collocazione e lo stato di ciascuna linea di cache. Dopo un riferimento a una linea di cache, si interroga il database per scoprire dove si trova la linea e se è intatta o è stata modificata. Poiché il database viene interrogato a ogni istruzione che accede alla memoria, è necessario che sia implementato con hardware specializzato molto veloce, capace di rispondere in una frazione di ciclo di bus.

Per meglio circostanziare l'idea alla base di questi multiprocessori, consideriamo un semplice esempio di un sistema (ipotetico) di 256 nodi, ciascuno costituito da una CPU e da una RAM di 16 MB, collegate tramite un bus locale. La memoria totale è di 2^{32} byte, divisi in 2^{26} linee di cache di 64 byte ciascuna. La memoria è allocata staticamente ai diversi nodi, laddove i primi 16 MB sono attribuiti al nodo 0, le locazioni da 16 a 32 MB al nodo 1, e così via. Si dice che il nodo 0 è il nodo di appartenenza (*home node*) dei blocchi di memoria dei primi 16 MB e delle corrispondenti linee di cache. I nodi

sono collegati per mezzo della rete d'interconnessione, rappresentata nella Figura 8.33(a). La rete d'interconnessione potrebbe essere una griglia, un ipercubo o avere un'altra topologia. Ogni nodo contiene anche gli elementi di directory per le 2^{18} linee di cache da 64 byte che costituiscono i suoi 2^{24} byte di memoria. Ipotizziamo per il momento che ogni linea si possa trovare al massimo in una cache.

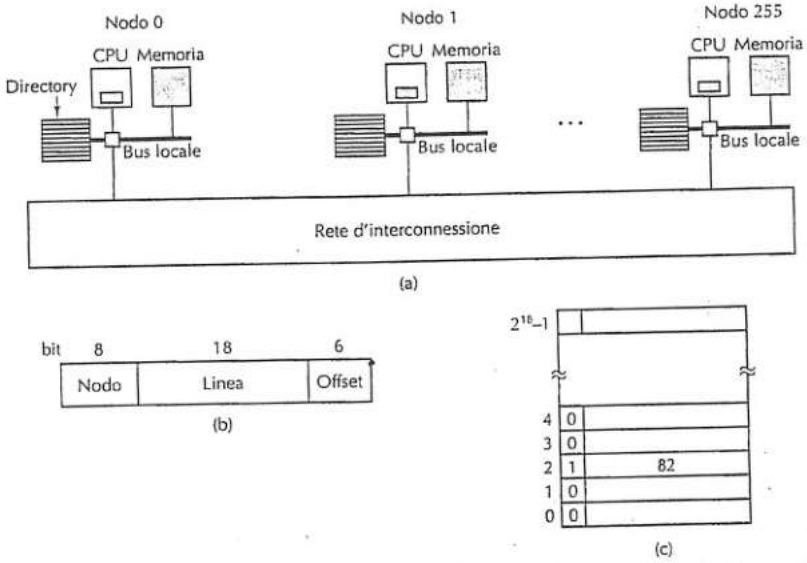


Figura 8.33 (a) Multiprocessore di 256 nodi basato su directory. (b) Divisione in campi di un indirizzo di memoria di 32 bit. (c) Directory in corrispondenza del nodo 36.

Per comprendere il funzionamento della directory seguiamo un'istruzione LOAD della CPU 20, che fa riferimento a una certa linea di cache. All'inizio la CPU emette l'istruzione e la passa alla propria MMU, che la traduce in un indirizzo fisico, diciamo 0x24000108. La MMU divide l'indirizzo nelle tre parti mostrate nella Figura 8.33(b). In decimale, le tre parti valgono: nodo 36, linea 4, offset 8. La MMU si accorge che il riferimento è a una parola di memoria del nodo 36, non del nodo 20, perciò invia un messaggio di richiesta attraverso la rete d'interconnessione verso il nodo che ospita la linea, cioè il 36, e gli richiede se abbia in cache la linea 4 e, in caso affermativo, dove si trovi.

La richiesta, pervenuta al nodo 36 dalla rete d'interconnessione, viene instradata all'hardware della directory, che la usa come indice per la sua tabella di 2^{18} elementi (uno per ogni linea di cache) per estrarre l'elemento 4. Nella Figura 8.33(c) osserviamo che la linea non è presente in cache, perciò l'hardware preleva la linea 4 dalla RAM

mento si ripete nei commutatori successivi. Il ritardo T_d comprende sia il tempo di elaborazione, sia il ritardo dovuto all'attesa nella coda perché si liberi una porta disponibile alla spedizione. Se ci sono n commutatori, allora la latenza totale di sola andata è data dalla formula $T_a + n\ (p/b + T_d) + p/b$, dove l'ultimo termine è richiesto dalla copia tra l'ultimo commutatore e la destinazione.

Nel caso dell'instradamento a cut through virtuale, la latenza di sola andata è nel migliore dei casi $T_a + p/b$ secondi, poiché non ci sono pacchetti sonda da inviare per configurare il circuito, né ritardi dovuti all'operazione di store-and-forward. In sostanza è richiesto solo il tempo di configurazione iniziale per assemblare il pacchetto, più il tempo necessario alla spedizione dei bit. In entrambi i casi va poi aggiunto il ritardo di propagazione, che spesso è trascurabile.

L'altra metrica hardware è la larghezza di banda. Molti programmi paralleli spostano grandi quantità di dati, perciò il numero di byte trasferibili in ogni secondo diventa un parametro cruciale per le loro prestazioni. Esistono molte metriche per la misurazione della larghezza di banda e ne abbiamo già incontrata una: la larghezza di banda di bisezione. Un'altra è la **larghezza di banda complessiva** (*aggregate bandwidth*), definita come la somma delle capacità di tutti i collegamenti. Questa quantità indica il numero massimo di bit che può transitare simultaneamente sulla rete. Un altro parametro importante è la larghezza di banda media in uscita da ogni CPU. Se ciascuna CPU è in grado di spedire dati a 1 MB/s, il sistema non trae alcun giovamento dall'avere una larghezza di banda di bisezione di 100 GB/s. La comunicazione sarà comunque limitata dal quantitativo di dati che può essere emesso da ciascuna CPU.

Nella pratica è pressoché impossibile avvicinarsi ai limiti teorici della larghezza di banda perché bisogna svolgere molte attività accessorie che riducono la capacità. Per esempio, ogni pacchetto richiede sempre lo svolgimento di alcune attività preparatorie come l'assemblaggio, la costruzione della sua intestazione e la spedizione. Spedire 4 pacchetti da 1024 byte non potrà mai raggiungere la stessa larghezza di banda della spedizione di un pacchetto da 4096 byte. Sfortunatamente però, la spedizione di pacchetti piccoli aiuta a diminuire la latenza, dal momento che i pacchetti grandi occupano linee e commutatori per troppo tempo. C'è dunque un'incompatibilità insita nel cercare di raggiungere latenze medie contenute congiuntamente a larghezze di banda elevate. Le prime sono importanti per alcune applicazioni, in altri casi è vero l'esatto contrario. Vale la pena notare che, comunque, è sempre possibile procurarsi maggiore larghezza di banda (aggiungendo cavi su cavi), ma che non è possibile comprare latenze più basse. Perciò è buona regola eccedere nel tentativo di ridurre la latenza il più possibile, per poi preoccuparsi in un secondo momento della larghezza di banda.

Parametri di valutazione software

Le metriche appena illustrate, quali la latenza e la larghezza di banda, si riferiscono alle capacità dell'hardware, ma l'interesse degli utenti è rivolto verso una prospettiva diversa: vogliono sapere quanto si ridurranno i tempi di esecuzione passando da un sistema monoprocessore a uno parallelo. Per quanto li riguarda, la metrica fondamentale è il fattore d'incremento della velocità, cioè di quante volte risulta più veloce l'esecuzione di un programma su un sistema a n processori rispetto a un sistema a monoprocessore.

L'andamento di questo rapporto viene presentato spesso mediante grafici simili a quello della Figura 8.49, in cui sono raffigurati diversi programmi paralleli eseguiti su di un multicomputer costituito da 64 CPU Pentium Pro. Ogni curva mostra l'incremento di velocità di un programma eseguito su k CPU, in funzione di k. La linea punteggiata indica l'incremento massimo possibile che corrisponde alla situazione in cui l'uso di k CPU accelera k volte il programma, per qualsiasi valore di k. Non sono molti i programmi che raggiungono l'incremento ideale, ma alcuni ci vanno vicini. La versione parallela della soluzione al problema degli N-corpi funziona molto bene[7]. La simulazione di awari (un gioco da tavolo di origine africana che si giocava in buchette scavate per terra) si comporta ragionevolmente bene. Al contrario l'inversione di una matrice a banda variabile (*skyline matrix*) non supera mai più di cinque volte la velocità di esecuzione su macchine monoprocessore, indipendentemente dal numero di processori disponibili. Per un excursus su questi programmi e sui relativi risultati si veda (Bal et al., 1998).

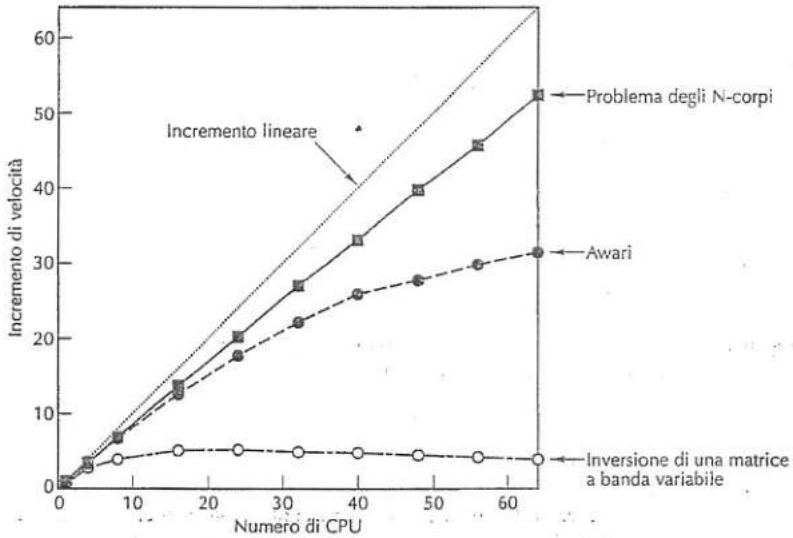


Figura 8.49 I programmi reali hanno incrementi di velocità minori dell'incremento lineare.

L'impossibilità pratica di raggiungere l'incremento ideale è dovuta in parte al fatto che quasi tutti i programmi hanno qualche componente sequenziale, di solito la fase di inizializzazione, la lettura dei dati o la raccolta dei risultati. In queste situazioni la disponibilità di più CPU non aiuta affatto. Nella Figura 8.50(a) mostriamo un programma che gira in T secondi su un monoprocessore, laddove una frazione f di questo tempo è codi-

7 Si tratta del classico problema dell'evoluzione dinamica di un sistema di N masse gravitazionali (*N.d.Rev.*).

ce sequenziale e la rimanente frazione $(1 - f)$ è potenzialmente parallelizzabile. Se questa seconda parte di codice può essere distribuita tra n CPU senza subire rallentamenti, allora il suo tempo di esecuzione può ridursi al meglio da $(1 - f)T$ a $(1 - f)T/n$ secondi, come indicato nella Figura 8.50(b). In ragione di ciò, il tempo totale di esecuzione risulta di $fT + (1 - f)T/n$ secondi. Il fattore d'incremento è esattamente il rapporto tra il tempo di esecuzione del programma originale, T, e il nuovo tempo di esecuzione:

$$\textit{Incremento di velocità} = \frac{n}{1 + (n - 1)f}$$

Quando $f = 0$ otteniamo l'incremento lineare, ma per $f > 0$ l'incremento perfetto diventa impossibile a causa della componente sequenziale. Questa formula è nota come **legge di Amdahl**.

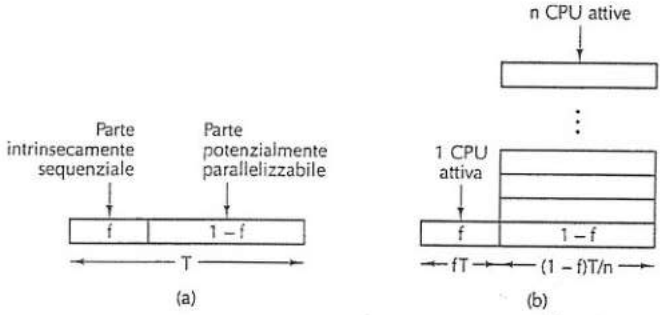


Figura 8.50 (a) I programmi hanno una parte sequenziale e una parallelizzabile. (b) Effetto dell'esecuzione parallela di una parte del programma.

Questa legge non è la sola ragione per l'impossibilità del raggiungimento dell'incremento ideale. Un ruolo in tal senso è giocato anche dalle latenze di comunicazione, dalle limitazioni poste dalla larghezza di banda e dalle inefficienze algoritmiche. Per di più, anche disponendo di 1000 CPU, non tutti i programmi possono essere scritti per utilizzare un numero di processori così elevato, e il tempo necessario per avviarli tutti potrebbe risultare significativo. Va aggiunto che, alle volte, l'algoritmo migliore per un determinato problema non è parallelizzabile efficacemente e quindi bisogna accontentarsi di un algoritmo parallelo subottimale. Al di là di queste osservazioni, è evidente che per molte applicazioni può far comodo che un programma giri n volte più velocemente, anche se ciò comporta l'uso di $2n$ CPU. Dopo tutto le CPU non sono tanto costose e molte aziende proliferano pur raggiungendo percentuali di efficienza considerevolmente inferiori al 100%.

Miglioramento delle prestazioni

Il modo più diretto per migliorare le prestazioni è aggiungere nuove CPU al sistema, benché questa aggiunta vada fatta in modo da non creare colli di bottiglia. I sistemi in cui la potenza di calcolo aumenta proporzionalmente al numero di CPU aggiunte si dicono **scalabili**.

Per meglio comprendere questo concetto, consideriamo il sistema della Figura 8.51(a) composto da quattro CPU collegate da un bus. Scalando il sistema alla dimensione 16, aggiungendo cioè 12 nuove CPU, ci riportiamo alla situazione della Figura 8.51(b). Se il bus ha larghezza di banda pari a b MB/s, la quadruplicazione del numero di CPU riduce la larghezza di banda disponibile per ogni CPU a $b/16$ MB/s. Un sistema siffatto non è scalabile.

Ripetiamo ora la stessa operazione con il sistema a griglia delle Figure 8.51(c)-(d). In questa topologia, l'aggiunta di nuove CPU comporta la creazione di nuovi collegamenti, perciò la crescita delle dimensioni del sistema non provoca la diminuzione della larghezza di banda per CPU che abbiamo osservato nel sistema basato su bus. Infatti, il rapporto tra numero di collegamenti e numero di CPU aumenta da 1 (4 CPU, 4 collegamenti) a 1,5 (16 CPU, 24 collegamenti), dunque l'aggiunta di nuove CPU giova alla larghezza di banda per CPU.

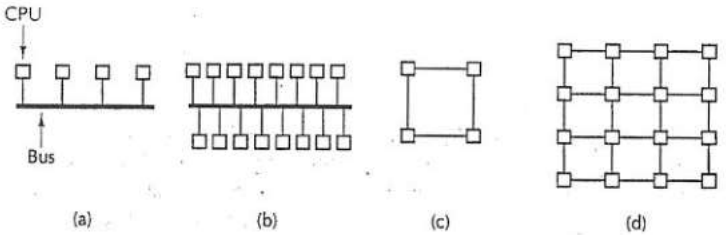


Figura 8.51 (a) Un sistema a bus con 4 CPU. (b) Un sistema a bus con 16 CPU. (c) Un sistema a griglia con 4 CPU. (d) Un sistema a griglia con 16 CPU.

Naturalmente la larghezza di banda non è il solo parametro significativo. Il collegamento al bus di nuove CPU non aumenta il diametro della rete d'interconnessione, né la latenza in assenza di traffico, ma non è così per la griglia. Il diametro di una griglia $n \times n$ è lungo $2(n - 1)$, perciò la latenza nel caso medio e peggiore aumenta grosso modo come la radice quadrata del numero di CPU. Un sistema con 400 CPU ha diametro 38, uno con 1600 CPU ha diametro 78, dunque quadruplicare il numero di CPU equivale approssimativamente a raddoppiare il diametro e quindi la latenza media.

Un sistema scalabile dovrebbe mantenere idealmente la stessa larghezza di banda media per CPU e la stessa latenza media al crescere delle sue dimensioni. Tuttavia nella pratica della progettazione, se si riesce spesso a garantire sufficiente larghezza di banda per CPU, in genere si osserva però un aumento della latenza al crescere delle dimensio-

ni del progetto. Una crescita logaritmica, come quella garantita da un ipercubo, è quanto di meglio si possa fare.

La latenza è un parametro cruciale per le prestazioni di molte applicazioni a granularità fine e media (la granularità di un'applicazione si definisce come il rapporto tra il tempo di calcolo e il tempo di comunicazione). Se un programma ha bisogno di dati che non si trovano nella sua memoria locale, allora sperimenterà un certo ritardo nel procurarseli che sarà tanto maggiore quanto più grande è il sistema, come abbiamo appena argomentato. Il problema si presenta ugualmente nei multiprocessori e nei multicomputer, perché in entrambi i casi la memoria fisica si trova divisa in moduli molto distanti gli uni dagli altri.

In conseguenza di ciò, molti progettisti di sistemi cercano di intervenire su queste distanze per ridurre la latenza o almeno per nasconderla, avvalendosi di diverse tecniche che ci apprestiamo a menzionare. La prima tecnica per mascherare la latenza è la duplicazione dei dati: se presso diverse locazioni esistono delle copie di un blocco di dati, gli accessi provenienti da queste locazioni possono trarne giovamento. Una tecnica di questo tipo è il **caching**, secondo cui una o più copie dei dati sono conservate in prossimità delle locazioni che li utilizzano o cui "appartengono". D'altro canto, un'altra strategia prevede di mantenere copie di pari grado, cioè che hanno lo stesso stato, invece di utilizzare la relazione gerarchica e asimmetrica del caching che distingue tra copia primaria e secondaria. Quando si gestiscono le duplicazioni, implementate sotto qualsiasi forma, bisogna affrontare i problemi relativi alla localizzazione dei blocchi di dati, alla tempistica dei trasferimenti e all'identificazione dei loro detentori. Le risposte a questi problemi vanno dall'allocazione dinamica su richiesta a carico dell'hardware, all'allocazione intenzionale durante il normale caricamento delle variabili mediante le direttive del compilatore. In ogni caso, si pone il problema di garantire la coerenza.

Una seconda tecnica per celare la latenza è il **prefetching**. Se è possibile caricare un dato prima di doverlo utilizzare, la sua lettura può essere sovrapposta alla normale esecuzione e così il dato sarà già disponibile al momento della richiesta. Il prefetching può essere automatico o sotto il controllo del programma. Il caricamento nella cache di un'intera linea, e non solo della parola referenziata, punta sul fatto che probabilmente anche le parole successive verranno richieste molto presto.

Vediamo com'è possibile controllare il prefetching in modo esplicito: quando il compilatore si rende conto che avrà bisogno di certi dati, può inserire un'istruzione esplicita per il loro caricamento e anticiparla in modo tale che i dati siano già disponibili quando necessario. Ciò comporta da parte del compilatore una conoscenza totale della macchina sottostante e della sua temporizzazione, come anche il pieno controllo della localizzazione dei dati. Una siffatta istruzione speculativa di LOAD funziona bene solo se si è certi che i dati saranno richiesti; provocare un errore di pagina a seguito di una LOAD che risulta poi inutile si rivela molto penalizzante.

Una terza tecnica utile per nascondere la latenza è il **multithreading**, come abbiamo già visto. Se la commutazione tra processi può essere resa abbastanza veloce, per esempio dotando ciascun processo di una propria mappa di memoria e di propri registri, allora l'hardware può commutare velocemente da un thread bloccato in attesa di dati distanti verso un thread in grado di proseguire l'esecuzione. Nel caso peggiore la CPU esegue un'istruzione del primo thread, una del secondo, e così via. Così facendo, si è sicuri di mantenere la CPU occupata anche in presenza di latenze di memoria molto lunghe cui andrebbero soggetti i singoli thread.

Una quarta tecnica che aiuta a nascondere la latenza è quella delle **scritture non bloccanti**. In genere l'esecuzione di un'istruzione STORE causa il blocco della CPU fino al completamento dell'operazione. L'avvio di una scrittura non bloccante consente invece la prosecuzione del programma anche se l'operazione di memoria non è stata ancora completata. La prosecuzione dell'esecuzione durante un'istruzione di LOAD è più difficile da realizzare, ma è possibile in caso di esecuzione fuori sequenza.

8.5 Grid computing

Molte delle sfide attuali nelle scienze, nell'ingegneria, nell'industria, nell'ambiente e in altri settori, sono di natura interdisciplinare e di vaste dimensioni. La loro risoluzione richiede esperienza, abilità, conoscenza, disponibilità di mezzi, software e dati provenienti da numerose organizzazioni, spesso ubicate in paesi diversi. Vediamo alcuni esempi.

1. Il progetto di una missione su Marte.
2. Un consorzio di aziende che costruisce un prodotto molto complesso (per esempio, una diga o un aeroplano).
3. Una squadra di soccorso internazionale che coordina gli aiuti dopo una catastrofe naturale.

Alcune di queste cooperazioni sono a lungo termine, altre a breve termine, ma tutte presentano come caratteristica comune il coinvolgimento di organizzazioni separate cui viene richiesto di partecipare con le proprie risorse e procedure di lavoro per raggiungere insieme lo scopo condiviso.

Fino a qualche tempo fa, la condivisione di dati e risorse tra organizzazioni che utilizzavano sistemi operativi, basi di dati e protocolli differenti era molto difficile. Ciononostante, il bisogno crescente di cooperazione su larga scala tra organizzazioni internazionali ha portato allo sviluppo di sistemi per la connessione di calcolatori molto distanti e altrimenti separati, che prendono il nome di tecnologia **grid**, o **grid computing**. In un certo senso tale tecnologia è lo stadio successivo della Figura 8.1 e può essere pensata come un cluster internazionale molto grande, lascamente connesso ed eterogeneo.

Lo scopo della tecnologia grid è fornire un'infrastruttura tecnica che permetta la condivisione di un obiettivo comune tra un gruppo di organizzazioni che formano così un'**organizzazione virtuale**. Tale organizzazione deve essere flessibile, deve poter comprendere un numero di partecipanti molto grande e gestire le loro variazioni, deve consentire ai propri membri di lavorare in accordo quando lo ritengono opportuno, mentre deve garantire loro di conservare il controllo delle proprie risorse con il grado di esclusività desiderato. I ricercatori del settore grid stanno lavorando allo sviluppo di servizi, strumenti e protocolli per rendere queste organizzazioni virtuali in grado di funzionare.

La tecnologia grid è intrinsecamente multilaterale, perché ha molti partecipanti di pari grado e può essere utile metterla in contrapposizione con gli altri modelli di calcolo esistenti. Nel modello client-server una transazione coinvolge due agenti: il server, che offre un certo servizio, e il client, che desidera avvalersene. Un esempio tipico di modello client-server è il Web, in cui gli utenti si rivolgono ai server web per trovare le informazioni. La tecnologia grid differisce anche dalle applicazioni *peer-to-peer* ("da pari a pari"), le quali consentono lo scambio diretto di file tra coppie di utenti (l'e-mail ne è un esempio comune). Poiché la tecnologia grid differisce da tutti questi modelli, richiede nuovi protocolli e tecnologie.

La tecnologia grid ha bisogno di poter accedere a un gran numero di risorse. Ogni risorsa risiede su di un sistema specifico e appartiene a un'organizzazione che decide con quali modalità renderla disponibile, quando e a chi. In astratto si può dire che la tecnologia grid si occupa dell'accesso alle risorse e della loro gestione.

La Figura 8.52 mostra un modo possibile di modellare una tecnologia grid come una gerarchia di livelli. Il **livello struttura** (*fabric layer*) alla base della gerarchia comprende l'insieme dei componenti costitutivi dell'infrastruttura, ovvero CPU, dischi, reti e sensori (per quanto riguarda l'hardware), e i dati e i programmi (per il lato software). Queste sono le risorse messe a disposizione in maniera controllata dalla tecnologia grid.

Livello	Funzione
Applicativo	Applicazioni che condividono le risorse gestite in modo controllato
Di raccolta	Scoperta, mediazione, monitoraggio e controllo di gruppi di risorse
Risorse	Accesso guidato e sicuro alle risorse individuali
Struttura	Risorse fisiche: calcolatori, dispositivi di memorizzazione, reti, sensori, dati e programmi

Figura 8.52 Livelli della tecnologia grid.

Il livello **risorse** (*resource layer*) gestisce le risorse individuali. In molti casi, una risorsa che partecipa a un sistema grid viene gestita da un processo locale che consente agli utenti distanti di accedervi in modo controllato. Questo livello fornisce ai livelli superiori un'interfaccia uniforme per ispezionare le caratteristiche e lo stato delle risorse individuali, per monitorarle e per usarle in modo sicuro.

Il livello successivo è quello **di raccolta** (*collective layer*) che gestisce gruppi di risorse. Una delle sue funzioni è la scoperta di risorse, tramite cui un utente può localizzare cicli di CPU disponibili, spazio su disco o dati specifici. Il livello di raccolta può servirsi di directory o di altri database per gestire queste informazioni, e può anche fornire un servizio di mediazione per trovare le corrispondenze migliori tra fornitori di servizi e loro utenti, eventualmente allocando le risorse poco reperibili tra gli utenti in competizione per accaparrarsele. Il livello di raccolta è responsabile anche della duplicazione dei dati, dell'ammissione di nuovi membri e risorse e del mantenimento del database delle politiche che specifica quali utenti possono usare quali risorse.

Al di sopra dei precedenti si trova il **livello applicativo** (*application layer*), che si avvale dei livelli inferiori per procurarsi le credenziali che provano il suo diritto all'uso di certe risorse, a richiederne l'impiego, a monitorarne i progressi, a trattarne gli errori e a notificare i loro risultati agli utenti.

La caratteristica fondamentale di una buona tecnologia grid è la sicurezza. I proprietari delle risorse sono molto insistenti nel richiedere un controllo ravvicinato delle loro risorse e vogliono sapere chi le usa, per quanto tempo e con quale intensità. Nessuna organizzazione renderebbe disponibili le proprie risorse con una tecnologia grid senza un buon livello di sicurezza. D'altro canto, se un utente dovesse avere un account protetto da password su ogni singolo computer che intende usare, l'uso della tecnologia grid diverrebbe insopportabilmente scomodo. Dunque è necessario sviluppare un modello di sicurezza che risponda a queste richieste adeguatamente.

Una delle caratteristiche fondamentali del modello di sicurezza è richiedere una sola autenticazione per l'ingresso nel sistema (*single sign on*). Il primo passo per l'uso di un tecnologia grid è venire autenticati e acquisire delle credenziali, cioè un documento con firma digitale che individua l'utente per conto del quale va svolto il lavoro. Le credenziali possono essere usate come delega, così, se un'elaborazione ha bisogno di creare sotto-elaborazioni, anche i processi figli possono essere identificati. Quando si presenta una credenziale a una macchina distante, questa deve verificarla con il proprio meccanismo di sicurezza locale. Per esempio, nei sistemi UNIX gli utenti sono identificati tramite un identificatore di 16 bit, ma altri sistemi usano schemi diversi. Infine la tecnologia grid deve prevedere alcuni meccanismi per la specifica, il mantenimento e l'aggiornamento delle politiche di accesso.

L'interoperatività tra organizzazioni e macchine diverse richiede l'adozione di standard, sia in termini di servizi offerti, sia di protocolli usati per il loro accesso. La comunità grid ha creato un'organizzazione, il Global Grid Forum, proprio per gestire il processo di standardizzazione. Un esito del forum è stato la produzione di uno schema chiamato **OGSA** (*Open Grid Services Architecture*) per la sistemazione dei diversi standard in via di sviluppo. Laddove possibile, si utilizzano standard già esistenti: per esempio è stato usato WSDL (*Web Services Definition Language*) per la descrizione dei servizi OGSA. I servizi attualmente in corso di standardizzazione appartengono alle seguenti otto vaste categorie, ma ne verranno certamente create di nuove.

1. Servizi di infrastruttura (per la comunicazione tra risorse).
2. Servizi di gestione delle risorse (prenotazione e messa a disposizione delle risorse).
3. Servizi per i dati (trasferimento e duplicazione dei dati laddove richiesto).
4. Servizi di contesto (descrizione delle risorse richieste e delle politiche di utilizzo).
5. Servizi informativi (ottenimento delle informazioni circa la disponibilità di risorse).
6. Servizi di amministrazione interna (garanzia della qualità di servizio specificata).

7. Servizi di sicurezza (rispetto delle politiche di sicurezza).
8. Esecuzione di servizi amministrativi (gestione dello scheduling dei compiti).

Resterebbe ancora molto da dire su questo argomento, ma per ragioni di spazio non possiamo dilungarci ulteriormente. Per una trattazione approfondita delle tecnologie grid rimandiamo ai riferimenti (Abramson, 2011; Balasangameshwara e Raju, 2012; Celaya e Arronategui, 2011; Foster e Kesselman, 2003; Lee et al., 2011).

8.6 Riepilogo

È sempre più difficile rendere più veloci i computer limitandosi a incrementare la loro frequenza di clock, a causa dei problemi della dissipazione del calore e di altri fattori. I progettisti cercano invece di sfruttare il parallelismo per incrementare le prestazioni. Il parallelismo può essere introdotto a molti livelli, a partire dai più bassi, dove gli elementi dell'elaborazione sono legati molto strettamente, fino a quelli più alti, dove sono legati molto debolmente.

A livello più basso c'è il parallelismo nel chip. In primo luogo esiste il parallelismo a livello delle istruzioni, in cui un'istruzione o una sequenza d'istruzioni emettono diverse operazioni che possono essere svolte in parallelo da unità funzionali distinte. Una seconda forma di parallelismo nel chip è il multithreading, in cui la CPU può commutare continuamente tra un thread e l'altro in base al tipo d'istruzione, creando così un multiprocessore virtuale. Una terza forma di parallelismo nel chip è il multiprocessore a chip singolo, in cui due o più core disposti all'interno dello stesso chip sono in grado di girare in parallelo.

A livello superiore troviamo i coprocessori, che risiedono in genere su schede a innesto e mettono a disposizione potenza di calcolo ausiliaria per certi calcoli specializzati, come l'elaborazione di protocolli di rete o l'elaborazione multimediale. Questi processori aggiuntivi sollevano la CPU da una porzione del lavoro e le permettono così di svolgere altri calcoli mentre si occupano dei loro compiti specializzati.

A livello successivo troviamo i multiprocessori con memoria condivisa. Questi sistemi contengono due o più CPU vere e proprie che condividono una memoria. I multiprocessori UMA comunicano attraverso un bus condiviso (che effettua snooping), un commutatore crossbar o una rete a commutazione multilivello. Sono caratterizzati dal fatto che garantiscono un tempo di accesso uniforme alle diverse locazioni di memoria. Anche i multiprocessori NUMA mostrano a tutti i processi uno spazio degli indirizzi condiviso, ma in questo caso gli accessi a distanza impiegano un tempo considerevolmente maggiore rispetto a quelli locali. Infine, i multiprocessori COMA costituiscono un'ulteriore variante di multiprocessori in cui le linee di cache vengono trasferite all'interno del sistema su richiesta, ma, diversamente da altri tipi di progetto, non appartengono ad alcuna locazione in particolare.

I multicomputer sono sistemi dotati di molte CPU che non condividono memoria. Ciascuna CPU ha la sua memoria privata e comunica con le altre tramite scambio di messaggi. Gli MPP, tra cui ricordiamo BlueGene/L di IBM, sono multicomputer molto grandi dotati di reti di comunicazione specializzate. I cluster sono sistemi più semplici costruiti a partire da componenti comunemente disponibili sul mercato, come le macchine che fanno funzionare Google.

I multicomputer sono spesso programmati avvalendosi di pacchetti software per lo scambio di messaggi quali MPI. Un approccio alternativo consiste nell'usare la condivisione di memoria a livello applicativo; ne sono alcuni esempi un sistema DSM basato su pagine, lo spazio delle tuple di Linda o gli oggetti di Orca o di Globe. DSM simula la memoria condivisa a livello della pagine, rendendo il sistema simile a una macchina NUMA, pur penalizzando molto di più gli accessi a distanza.

Infine, a livello più alto e legato più debolmente, c'è il grid computing. Si tratta di sistemi che raggruppano intere organizzazioni di utenti Internet, disposti a condividere potenza di calcolo, dati e altre risorse.

PROBLEMI

1. Le istruzioni Intel x86 possono raggiungere i 17 byte di lunghezza. Una CPU x86 è VLIW?
2. Non appena le tecnologie hanno permesso agli ingegneri di mettere ancora più transistor in un singolo chip, Intel e AMD hanno scelto di aumentare il numero di core su ogni chip. Sarebbero state possibili scelte differenti?
3. Quali sono i valori saturati di 96, −9, 300 e 256 se l'intervallo di saturazione è 0–255?
4. Si stabilisca se le seguenti istruzioni di TriMedia sono lecite e, in caso negativo, si indichi perché non lo sono.
   a. Somma intera, sottrazione intera, caricamento, somma in virgola mobile, caricamento immediato.
   b. Sottrazione intera, moltiplicazione intera, caricamento immediato, scorrimento, scorrimento.
   c. Caricamento immediato, somma in virgola mobile, moltiplicazione in virgola mobile,salto, caricamento immediato.
5. Le Figure 8.7(d) e 8.7(e) mostrano 12 cicli d'istruzioni. Per ciascuna delle due sequenze, si indichi che cosa succede nei tre cicli successivi.
6. In una certa CPU, un'istruzione che causa un fallimento nella cache di primo livello, ma che ha successo nella cache di secondo livello, richiede in tutto *k* cicli. Se si usa il multithreading per nascondere i fallimenti di cache di primo livello, quanti thread bisogna eseguire contemporaneamente per evitare cicli di inattività con il multithreading a grana fine?
7. La GPU NVIDIA Fermi è simile, come spirito, a una delle architetture studiate nel Capitolo 2. Quale?
8. Una mattina l'ape regina di un certo alveare convoca le api operaie e affida loro, come compito per la giornata, la raccolta di nettare di calendula. Quindi le operaie volano in direzioni diverse in cerca di calendule. Si tratta di un sistema SIMD o MIMD?
9. Quando abbiamo esaminato i modelli di consistenza della memoria, abbiamo detto che un modello di consistenza è una specie di contratto tra il software e la memoria. Perché è necessario un tale contratto?
10. Si consideri un multiprocessore con bus condiviso. Che cosa succede se due processori cercano di accedere alla memoria globale esattamente nello stesso istante?
11. Si consideri un multiprocessore con bus condiviso. Che cosa succede se tre processori cercano di accedere alla memoria globale esattamente nello stesso istante?
12. Si supponga che, per qualche ragione tecnica, una cache possa fare lo snooping solo delle linee degli indirizzi, e non delle linee dati. Questa limitazione avrebbe ripercussioni sul protocollo write through?
13. Si prenda in considerazione un semplice modello di sistema multiprocessore senza caching basato sul bus, in cui ogni quattro istruzioni una accede alla memoria e l'accesso alla memoria occupa il bus per un tempo pari all'e-

secuzione di un'istruzione. Quando il bus è occupato, le CPU richiedenti vengono poste in una coda d'attesa FIFO. Quanto un sistema di 64 CPU risulterà più veloce rispetto a un sistema con una sola CPU?

14. Il protocollo MESI di coerenza delle cache ha quattro stati, mentre altri protocolli write-back di coerenza delle cache ne hanno solo tre. Quale dei quattro stati di MESI può essere sacrificato e con quali conseguenze? Se si dovessero scegliere solo tre stati, quali sarebbero?

15. Il protocollo MESI di coerenza delle cache dà luogo a situazioni in cui una linea di cache richiede una transazione sul bus pur essendo presente nella cache locale? Se sì, si spieghi perché.

16. Ci sono n CPU collegate a un bus comune. La probabilità che una CPU cerchi di usare il bus in un determinato ciclo è p. Qual è la probabilità che durante un ciclo
    a. il bus resti inoperoso (0 richieste);
    b. venga fatta una sola richiesta;
    c. vengano fatte più richieste?

17. Si elenchino i maggiori vantaggi e svantaggi dei commutatori crossbar.

18. Quanti commutatori crossbar ci sono in un E25K di Sun Fire completo?

19. Se si interrompe il collegamento tra i commutatori 2A e 3B della rete omega della figura 8.31, quali elementi ne risultano scollegati?

20. Gli *hot spot* ("punti caldi") sono le locazioni di memoria cui si accede frequentemente e costituiscono un problema grave nelle reti a commutazione multilivello. Sono un problema anche nei sistemi basati su bus?

21. Una rete di commutazione omega collega 4096 CPU RISC, ciascuna con ciclo di 60 ns, a 4096 moduli di memoria infinitamente veloci. Gli elementi che effettuano la commutazione introducono un ritardo di 5 ns. Quanti intervalli di ritardo sono necessari a un'istruzione LOAD?

22. Si consideri una macchina che usa una rete di commutazione omega come quella mostrata nella Figura 8.29. Se il modulo di memoria i contiene il programma e lo stack del processore i, si proponga una piccola modifica della topologia che produca un grosso cambiamento delle prestazioni (RP3 di IBM e Butterfly di BBN usano questa topologia modificata). Quale svantaggio comporta la nuova topologia rispetto a quella originale?

23. In un multiprocessore NUMA, un riferimento alla memoria locale impiega 20 ns, mentre un riferimento a distanza impiega 120 ns. Un certo programma effettua N riferimenti a memoria durante la sua esecuzione, l'1% dei quali avviene verso la pagina P. La pagina P si trova inizialmente a distanza e ci vogliono C secondi per copiarla localmente. In quali condizioni conviene copiare la pagina localmente, in assenza di un suo utilizzo significativo da parte di altri processori?

24. Si consideri un multiprocessore CC-NUMA simile a quello della Figura 8.33, ma con 512 nodi dotati di 8 MB ciascuno. Se le linee di cache sono di 64 byte, qual è la percentuale d'informazioni accessorie necessarie alle directory? L'incremento del numero di nodi aumenta, diminuisce o lascia invariata questa percentuale?

25. Che differenza c'è tra NC-NUMA e CC-NUMA?

26. Si calcoli il diametro di rete delle topologie mostrate nella Figura 8.37.

27. Si calcoli il grado di tolleranza agli errori delle topologie mostrate nella Figura 8.37. Il grado di tolleranza è definito come il massimo numero di collegamenti che è possibile interrompere senza separare la rete in due componenti sconnessi.

28. Si consideri la topologia a doppio toro della Figura 8.37(f) espansa alle dimensioni $k \times k$. Qual è il diametro della rete risultante? Suggerimento: distinguere i casi con k pari e k dispari.

29. Una rete d'interconnessione ha la forma di un cubo $8 \times 8 \times 8$. Ogni collegamento ha una larghezza di banda full duplex di 1 GB/s. Qual è la larghezza di banda di bisezione della rete?

30. La legge di Amdhal limita l'incremento potenziale raggiungibile da un calcolatore parallelo. Si calcoli, in funzione di f, l'incremento massimo raggiungibile per un numero di CPU che tende a infinito. Quali conseguenze ha questo limite per $f = 0,1$?

31. La Figura 8.51 mostra la buona scalabilità di una griglia in contrapposizione alla cattiva scalabilità di un bus. Se ogni bus o collegamento ha una larghezza di banda b, si calcoli la larghezza di banda media per CPU in ciascuno dei quattro casi della figura. Si scalino quindi tutti i sistemi alla dimensione di 64 CPU e si ripeta il calcolo. Qual è il limite se il numero di CPU tende a infinito?

32. Abbiamo trattato nel capitolo tre varianti di send: sincrona, bloccante e non bloccante. Si fornisca un quarto metodo che sia simile alla variante bloccante, ma che abbia proprietà leggermente diverse. Si elenchino vantaggi e svantaggi della versione proposta valutati rispetto alla send bloccante.

33. Si consideri un multicomputer che usa una rete che fa broadcast hardware come Ethernet. Perché è importante il rapporto tra il numero di operazioni di lettura (che non modificano lo stato interno delle variabili) e il numero di operazioni di scrittura (che modificano lo stato interno delle variabili)?

CAPITOLO 9

Bibliografia

Questo capitolo contiene l'elenco alfabetico di tutti i libri e gli articoli citati nel volume.

Abramson, D., "Mixing Cloud and Grid Resources for Many Task Computing", *Atti dell'Int'l Workshop on Many Task Computing on Grids and Supercomputers*, ACM, pp. 1–2, 2011.

Adams, M. e Dulchinos, D., "OpenCable", *IEEE Commun. Magazine*, vol. 39, pp. 98–05, giugno 2001.

Adiga, N.R. *et al.*, "An overview of the BlueGene/L Supercomputer", *Atti di Supercomputing 2002*, ACM, pp. 1–22, 2002.

Adve, S.V. e Hill, M., "Weak Ordering: a new definition", *Atti del 17° Ann. Int'l Symp. On Computer Arch.*, ACM, pp. 2–14, 1990.

Agerwala, T. e Cocke, J., "High performance reduced instruction set processors", *IBM T.J. Watson Research Center Technical Report* RC12434, 1987.

Ahmadinia, A. e Shahrabi, A., "A Highly Adaptive and Efficient Router Architecture for Network-on-Chip", *Computer J.*, vol. 54, pp. 1295–1307, agosto 2011.

Alam, S., Barrett, R., Bast, M., Fahey, M.R., Kuehn, J., McCurdy, Rogers, J., Roth, P., Sankaran, R., Vetter, J.S., Worley, P. e Yu, W., "Early Evaluation of IBM BlueGene/P", *Atti dell'ACM/IEEE Conference on Supercomputing*, ACM/IEEE, 2008.

Alameldeen, A.R. e Wood, D.A., "Adaptive cache compression for high-performance processors", *Atti del 31° Ann. Int'l Sym. on Computer Arch.*, ACM, pp. 212–223, 2004.

Almasi, G.S. *et al.*, "System management in the BlueGene/L Supercomputer", *Atti del 17° Int'l Parallel and Distr. Symp.*, IEEE, 2003a.

Almasi, G.S. *et al.*, "An overview of the Bluegene/L System Software Organization", *Par. Proc. Letters*, vol. 13, pp. 561–574, aprile 2003b.

Amza, C., Cox, A., Dwarkadas, S., Keleher, P., Lu, H., Rajamony, R., Yu, W. e Zwaenepoel, W., "TreadMarks: shared memory computing on a network of workstations", *IEEE Computer Magazine*, vol. 29, pp. 18–28, febbraio 1996.

Anderson, D., *Universal serial bus system architecture*, Reading, MA, Addison-Wesley, 1997.

Anderson, D., Budruk, R., e Shanley, T., *PCI express system architecture*, Reading, MA, Addison-Wesley, 2004.

Anderson, T.E., Culler, D.E. e Patterson D.A., "A case for NOW (Networks of workstations)", *IEEE Micro Magazine*, vol. 15, pp. 54–64, gennaio 1995.

August, D.I., Connors, D.A., Mahlke, S.A., Sias, J.W., Crozier, K.M., Cheng, B.-C., Eaton, P.R., Olaniran, Q.B. e Hwu, W.-M., "Integrated predicated and speculative execution in the IMPACT EPIC Architecture", *Atti del 25° Ann. Int'l Symp. on Computer Arch.*, ACM, pp. 227–237, 1998.

Bal, H.E., *Programming distributed systems*, Hemel Hempstead, England, Prentice Hall Int'l, 1991.

Bal, H.E., Bhoedjang, R., Hofman, R., Jacobs, C., Langendoen, K., Ruhl, T. e Kaashoek, M.F., "Performance evaluation of the Orca Shared Object System", *ACM Trans. on Computer Systems*, vol. 16, pp. 1–40, gennaio-febbraio 1998.

Bal, H.E., Kaashoek, M.F. e Tanenbaum, A.S., "Orca: a language for parallel programming of distributed systems", *IEEE Trans. on Software Engineering*, vol. 18, pp. 190–205, marzo 1992.

Bal, H.E. e Tanenbaum, A.S., "Distributed programming with shared data", *Atti della 1988 Int'l Conf. on Computer Languages*, IEEE, pp. 82–91, 1988.

Balasangameshwara, J. e Raju, N., "A Hybrid Policy for Fault Tolerant Load Balancing in Grid Computing Environments", *J. Network and Computer Applications*, vol. 35, pp. 412–422, gennaio 2012.

Barroso, L.A., Dean, J., e Holzle, U., "Web search for a planet: the Google Cluster Architecture", *IEEE Micro Magazine*, vol. 23, pp. 22–28, marzo-aprile 2003.

Bechini, A., Conte, T.M. e Prete, C.A., "Opportunities and challenges in embedded systems", *IEEE Micro Magazine*, vol. 24, pp. 8–9, luglio-agosto 2004.

Bhakthavatchalu, R., Deepthy, G.R. e Shanooja, S., "Implementation of Reconfigurable Open Core Protocol Compliant Memory System Using VHDL", *Atti dell'Int Conf. on Industrial and Information Systems*, pp. 213–218, 2010.

Bjornson, R.D., "Linda on distributed memory multiprocessors", tesi di dottorato, Yale Univ., 1993.

Blumrich, M., Chen, D., Chiu, G., Coteus, P., Gara, A., Giampapa, M.E., Haring, R.A., Heidelberger, P., Hoenicke, D., Kopcsay, G.V., Ohmcht, M., Steinmacher-Burow, B.D., Takken, T., Vransas, P. e Liebsch, T., "An overview of the BlueGene/L System", *IBM J. Research and Devel.*, vol. 49, marzo-maggio 2005.

Bose, P., "Computer architecture research: shifting priorities and newer challenger", *IEEE Micro Magazine*, vol. 24, p. 5, novembre-dicembre 2004.

Bouknight, W.J., Denenberg, S.A., McIntyre, D.E., Randall, J.M., Sameh, A.H. e Slotnick, D.L., "The Illiac IV System", *Atti IEEE*, pp. 369–388, aprile 1972.

Bradley, D., "A Personal History of the IBM PC", *IEEE Computer*, vol. 44, pp. 19–25, agosto 2011.

Bride, E., "The IBM Personal Computer: A Software-Driven Market", *IEEE Computer*, vol. 44, pp. 34–39, agosto 2011.

Brightwell, R., Camp, W., Cole, B., DeBenedictis, E., Leland, R., Tompkins, H. e MacCabe, A.B., "Architectural specification for massively parallel computers. An experience and measurement-based approach", *Concurrency and computation: practice and experience*, vol. 17, pp. 1–46, 2005.

Brightwell, R., Underwood, K.D., Vaughan, C. e Stevenson, J., "Performance Evaluation of the Red Storm Dual-Core Upgrade", *Concurrency and Computation: Practice and Experience*, vol. 22, pp. 175–190, febbraio 2010.

Burkhardt, H., Frank, S., Knobe B., e Rothnie, J., "Overview of the KSR-1 Computer System", *Technical Report KSR-TR-9202001*, Kendall Square Research Corp, Cambridge, MA, 1992.

Carriero, N. e Gelernter, D., "Linda in context", *Comunicazioni dell'ACM*, vol. 32, pp. 444–458, aprile 1989.

Celaya, J. e Arronategui, U., "A Highly Scalable Decentralized Scheduler of Tasks with Deadlines", *Atti della 12ª Int'l Conf. on Grid Computing*, IEEE/ACM, pp. 58–65, 2011.

Charlesworth, A., "The Sun fireplane interconnect", *IEEE Micro Magazine*, vol. 22, pp. 36–45, gennaio-febbraio 2002.

Charlesworth, A., "The Sun fireplane interconnect", *Atti della Conf. on High Perf. Networking and Computing*, ACM, 2001.

Chen, L., Dropsho, S. e Albonesi, D.H., "Dynamic data dependence tracking and its application to branch prediction", *Atti del 9° Int'l Symp. on High-Performance Computer Arch.*, IEEE, pp. 65–78, 2003.

Cheng, L. e Carter, J.B., "Extending CC-NUMA Systems to Support Write Update Optimizations", *Atti della 2008 ACM/IEEE Conf. on Supercomputing*, ACM/IEEE, 2008.

Chou, Y., Fahs, B. e Abraham, S., "Microarchitecture optimizations for exploiting Memory-level parallelism", *Atti del 31° Ann. Int'l Symp. on Computer Arch.*, ACM, pp. 76–77, 2004.

Cohen, D., "On holy wars and a plea for peace", *IEEE Computer Magazine*, vol. 14, pp. 48–54, otttobre 1981.

Corbato, F.J. e Vyssotsky, V.A., "Introduction and overview of the MULTICS System", *Atti FJCC*, pp. 185–196, 1965.

Denning, P.J., "The working set model for program behavior", *Comunicazioni dell'ACM*, vol. 11, pp. 323–333, maggio 1968.

Dijkstra, E.W., "GOTO statement considered harmful", *Comunicazioni dell'ACM*, vol. 11, pp.147–148, marzo 1968a.

Dijkstra, E.W., "Co-operating sequential processes", *Programming Languages*, F. Genuys (a cura di), New York, Academic Press, 1968b.

Donaldson, G. e Jones, D., "Cable television broadband network architectures", *IEEE Commun. Magazine*, vol. 39, pp. 122–126, giugno 2001.

Dubois, M., Scheurich, C. e Briggs, F.A., "Memory access buffering in multiprocessors", *Atti del 13° Ann. Int'l Symp. on Computer Arch.*, ACM, pp. 434–442, 1986.

Dulong, C., "The IA-64 architecture at work", *IEEE Computer Magazine*, vol. 31, pp. 24–32, luglio 1998.

Dutta-Roy, A., "An overview of cable modem technology and market perspectives", *IEEE Commun. Magazine*, vol. 39, pp. 81–88, giugno 2001.

Faggin, F., Hoff, M.E., Mazor, S., Jr. e Shima, M., "The history of the 4004", *IEEE Micro Magazine*, vol. 16, pp. 10–20, novembre 1996.

Falcon, A., Stark, J., Ramirez, A., Lai, K. e Valero, M., "Prophet/critic hybrid branch prediction", *Atti del 31° Ann. Int'l Symp. on Computer Arch.*, ACM, pp. 250–261, 2004.

Fisher, J.A. e Freudenberger, S.M., "Predicting conditional branch directions from previous runs of a program", *Atti della 5ª Int'l Conf. on Arch. Support for Prog. Lang. and Operating Syst.*, ACM, pp. 85–95, 1992.

Flynn, D., "AMBA: enabling reusable on-chip designs", *IEEE Micro Magazine*, vol. 17, pp. 20–27, luglio 1997.

Flynn, M.J., "Some computer organizations and their effectiveness", *IEEE Trans. On Computers*, vol. C-21, pp. 948–960, settembre 1972.

Foster, I. e Kesselman, C., *The grid 2: blueprint for a new computing infrastructure*, San Francisco, Morgan Kaufman, 2003.

Fotheringham, J., "Dynamic storage allocation in the atlas computer including an automatic use of a backing store", *Comunicazioni dell'ACM*, vol. 4, pp. 435–436, ottobre 1961.

Freitas, H.C., Madruga, F.L., Alves, M. e Navaux, P., "Design of Interleaved Multithreading for Network Processors on Chip", *Atti dell'Int. Symp. on Circuits and Systems*, IEEE, 2009.

Gaspar, L., Fischer, V., Bernard, F., Bossuet, L. e Cotret, P., "HCrypt: A Novel Concept of Crypto-processor with Secured Key Management", *Int. Conf. on Reconfigurable Computing and FPGAs*, 2010.

Gaur, J., Chaudhuri, C. e Subramoney, S., "Bypass and Insertion Algorithms for Exclusive Last-level Caches", *Atti del 38° Int. Symp. on Computer Arch.*, ACM, 2011.

Gebhart, M., Johnson, D.R., Tarjan, D., Keckler, S.W., Dally, W.J., Lindholm, E. e Skadron, K., "Energy-efficient Mechanisms for Managing Thread Context in Throughput Processors", *Atti del 38° Int. Symp. on Computer Arch.*, ACM, 2011.

Geist, A., Beguelin, A., Dongarra, J., Jiang, W., Mancheck, R. e Sunderram, V., *PVM: Parallel Virtual Machine – A user's guide and tutorial for networked parallel computing*, Cambridge, MA, MIT Press, 1994.

Gepner, P., Gamayunov, V. e Fraser, D.L., "The 2nd Generation Intel Core Processor. Architectural Features Supporting HPC", *Atti del 10° Int. Symp. on Parallel and Dist. Computing*, pp. 17–24, 2011.

Gerber, R. e Binstock, A., *Programming with hyper-threading technology*, Santa Clara, CA, Intel Press, 2004.

Gharachorloo, K., Lenoski, D., Laudon, J., Gibbons, P.B., Gupta, A. e Hennessy, J.L., "Memory Consistency and Event Ordering in Scalable Shared-Memory Multiprocessors", *Atti del 17° Ann. Int. Symp. on Comp. Arch.*, ACM, pp. 15–26, 1990.

Ghemawat, S., Gobioff, H. e Leung, S.-T., "The google file system", *Atti del 19° Symp. on Operating Systems Principles*, ACM, pp. 29–43, 2003.

Goodman, J.R., "Using cache memory to reduce processor memory traffic", *Atti del 10° Ann. Int'l Symp. on Computer Arch.*, ACM, pp. 124–131, 1983.

Goodman, J.R., "Cache consistency and sequential consistency", *Tech. Rep.* 61, IEEE Scalable Coherent Interface Working Group, IEEE, 1989.

Goth, G., "IBM PC Retrospective: There Was Enough Right to Make It Work", *IEEE Computer*, vol. 44, pp. 26–33, agosto 2011.

Gropp, W., Lusk, E. e Skjellum, A., *Using MPI: portable parallel programming with the message passing interface*, Cambridge, MA, MIT Press, 1994.

Gupta, N., Mandal, S., Malave, J., Mandal, A. e Mahapatra, R.N., "A Hardware Scheduler for Real Time Multiprocessor System on Chip", *Atti della 23ª Int. Conf. on VLSI Design*, IEEE, 2010.

Gurumurthi, S., Sivasubramaniam, M., Kandemir, M. e Franke H., "Reducing disk power consumption in servers with DRPM", *IEEE Computer Magazine*, vol. 36, pp. 59–66, dicembre 2003.

Hagersten, E., Landin, A. e Haridi, S., "DDM – A cache-only memory architecture", *IEEE Computer Magazine*, vol. 25, pp. 44–54, settembre 1992.

Haghighizadeh, F., Attarzadeh, H. e Sharifkhani, M., "A Compact 8-Bit AES Crypto-processor", *Atti della 2ª Int. Conf. on Computer and Network Tech.*, IEEE, 2010.

Hamming, R.W., "Error detecting and error correcting codes", *Bell Syst. Tech. J.*, vol. 29, pp. 147–160, aprile 1950.

Henkel, J., Hu, X.S. e Bhattacharyya, S.S., "Taking on the embedded system challenge", *IEEE Computer Magazine*, vol. 36, pp. 35–37, aprile 2003.

Hennessy, J.L., "VLSI processor architecture", *IEEE Trans. on Computers*, vol. C-33, pp. 1221–1246, dicembre 1984.

Herrero, E., Gonzalez, J. e Canal, R., "Elastic Cooperative Caching: An Autonomous Dynamically Adaptive Memory Hierarchy for Chip Multiprocessors", *Atti della 23ª Int. Conf. on VLSI Design*, IEEE, 2010.

Hoare, C.A.R., "Monitors, an operating system structuring concept", *Comunicazioni dell'ACM*, vol. 17, pp. 549–557, ottobre 1974 (*erratum* in *Comunicazioni dell'ACM*, vol. 18, p. 95, febbraio 1975).

Hwu, W.-M., "Introduction to predicated execution", *IEEE Computer Magazine*, vol. 31, pp. 49–50, gennaio 1998.

Jimenez, D.A., "Fast path-based neural branch prediction", *Atti del 36° Int. Symp. on Microarchitecture*, IEEE, pp. 243–252, 2003.

Johnson, K.L., Kaashoek, M.F. e Wallach, D.A., "CRL: high-performance all-software distributed shared memory", *Atti del 15° Symp. on Operating Systems Principles*, ACM, pp. 213–228, 1995.

Kapasi, U.J., Rixner, S., Dally, W.J., Khailany, B., Ahn, J.H., Mattson, P. e Owens, J.D., "Programmable stream processors", *IEEE Computer Magazine*, vol. 36, pp. 54–62, agosto 2003.

Kaufman, C., Perlman, R. e Speciner, M., *Network Security*, seconda edizione, Upper Saddle River, NJ, Prentice Hall, 2002.

Kim, N.S., Austin, T., Blaauw, D., Mudge, T., Flautner, K., Hu, J.S., Irwin, M.J., Kandemir, M. e Narayanan, V., "Leakage current: Moore's Law meets static power", *IEEE Computer Magazine*, vol. 36, pp. 68–75, dicembre 2003.

Knuth, D.E., *The art of computer programming: fundamental algorithms*, terza edizione., Reading, MA, Addison-Wesley, 1997.

Kontothanassis, L., Hunt, G., Stets, R., Hardavellas, N., Cierniad, M., Parthasarathy, S., Meira, W., Dwarkadas, S. e Scott, M., "VM-based shared memory on low latency remote memory access networks", *Atti del 24° Ann. Int'l Symp. on Computer Arch.*, ACM, pp. 157–169, 1997.

Lamport, L., "How to make a multiprocessor computer that correctly executes multiprocess programs", *IEEE Trans. on Computers*, vol. C-28, pp. 690–691, settembre 1979.

Larowe, R.P. ed Ellis C.S., "Experimental comparison of memory management policies for NUMA multiprocessors", *ACM Trans. on Computer Systems*, vol. 9, pp. 319–363, novembre 1991.

Lee, J., Keleher, P. e Sussman, A., "Supporting Computing Element Heterogeneity in P2P Grids", *Atti dell'IEEE Int. Conf. on Cluster Computing, IEEE*, pp. 150–158, 2011.

Li, K. e Hudak, P., "Memory coherence in shared virtual memory systems", *ACM Trans. On Computer Systems*, vol. 7, pp. 321–359, novembre 1989.

Lin, Y.-N., Lin, Y.-D. e Lai, Y.-C., "Thread Allocation in CMP-based Multithreaded Network Processors", *Parallel Computing*, vol. 36, pp. 104–116, febbraio 2010.

Lu, H., Cox, A.L., Dwarkadas S., Rajamony, R. e Zwaenwepoel, W., "Software distributed shared memory support for irregular applications", *Atti della 6° Conf. on Prin. and Practice of Parallel Progr.*, pp. 48–56, giugno 1997.

Lukasiewicz, J., *Aristotle's syllogistic*, seconda edizione, Oxford, Oxford University Press, 1958.

Lyytinen, K. e Yoo, Y., "Issues and challenges in ubiquitous computing", *Comunicazioni dell'ACM*, vol. 45, pp. 63–65, dicembre 2002.

Martin, R.P., Vahdat, A.M., Culler, A.M. e Anderson, T.E., "Effects of communication latency, overhead and bandwidth in a cluster architecture", *Atti del 24° Ann. Int'l Symp. on Computer Arch.*, ACM, pp. 85–97, 1997.

Mayhew, D. e Krishnan, V., "PCI express and advanced switching: evolutionary path to building next generation interconnects", *Atti dell'11° Symp. on High Perf. Interconnects IEEE*, pp. 21–29, agosto 2003.

McKusick, M.K., Joy, W.N., Leffler, S.J., e Fabry, R.S., "A fast file system for UNIX", *ACM Trans. on Computer Systems*, vol. 2, pp. 181–197, agosto 1984.

McNairy, C. e Soltis, D., "Itanium 2 processor microarchitecture", *IEEE Micro Magazine*, vol. 23, pp. 44–55, marzo-aprile 2003.

Mishra, A.K., Vijaykrishnan, N. e Das, C.R., "A Case for Heterogeneous On-Chip Interconnects for CMPs", *Atti del 38° Int'l Symp. on Computer Arch.*. ACM, 2011.

Morgan, C., *Portraits in Computing*, New York, ACM Press, 1997.

Moudgill, M. e Vassiliadis, S., "Precise interruptus", *IEEE Micro Magazine*, vol. 16, pp. 58–67, gennaio 1996.

Mullender, S.J. e Tanenbaum, A.S., "Immediate files", *Software-Practice and Experience*, vol. 14, pp. 365–368, 1984.

Naeem, A., Chen, X., Lu, Z. e Jantsch, A., "Realization and Performance Comparison of Sequential and Weak Memory Consistency Models in Network-On-Chip Based Multicore Systems", *Atti della 16ª Design Automation Conf. Asia and South Pacific*, IEEE, pp. 154–159, 2011.

Organick, E., *The MULTICS system*, Cambridge, MA, MIT Press, 1972.

Oskin, M., Chong, F.T. e Chuang, I.L., "A practical architecture for reliable quantum computers", *IEEE Computer Magazine*, vol. 35, pp. 79–87, gennaio 2002.

Papamarcos, M. e Patel, J., "A low overhead coherence solution for multiprocessors with private cache memories", *Atti dell'11° Ann. Int'l Symp. on Computer Arch.*, ACM, pp. 348–354, 1984.

Parikh, D., Skadron, K., Zhang, Y. e Stan, M., "Power-Aware Branch Prediction: characterization and design", *IEEE Trans. on Computers*, vol. 53, pp. 168–186, febbraio 2004.

Patterson, D.A., "Reduced instruction set computers", *Comunicazioni dell'ACM*, vol. 28, pp. 8–21, gennaio 1985.

Patterson, D.A., Gibson, G. e Katz, R., "A case for redundant arrays of inexpensive disks (RAID)", *Atti dell'ACM SIGMOD Int'l Conf. on Management of Data*, ACM, pp. 109–166, 1988.

Patterson, D.A. e Sequin, C.H., "A VLSI RISC", *IEEE Computer Magazine*, vol. 15, pp. 8–22, settembre 1982.

Pountain, D., "Pentium: more RISC than CISC", *Byte*, vol. 18, pp. 195–204, settembre 1993.

Radin, G., "The 801 minicomputer", *Computer Arch. News*, vol. 10, pp. 39–47, marzo 1982.

Raman, S.K., Pentkovski, V. e Keshava, J., "Implementing streaming SIMD extensions on the Pentium III Processor", *IEEE Micro Magazine*, vol. 20, pp. 47–57, luglio-agosto 2000.

Ritchie, D.M., "Reflections on Software Research", *Comunicazioni dell'ACM*, vol. 27, pp. 758–760, agosto 1984.

Ritchie, D.M. e Thompson, K., "The UNIX time-sharing system", *Comunicazioni dell'ACM*, vol. 17, pp. 365–375, luglio 1974.

Robinson, G.S., "Toward the age of smarter storage", *IEEE Computer Magazine*, vol. 35, pp. 35–41, dicembre 2002.

Rosenblum, M. e Ousterhout, J.K., "The design and implementation of a log-structured file system", *Atti del 13° Symp. on Operating System Principles*, ACM, pp. 1–15, 1991.

Russinovich, M.E. e Solomon, D.A., *Microsoft Windows internals*, quarta edizione, Redmond, WA, Microsoft Press, 2005.

Rusu, S., Muljono, H. e Cherkauer, B., "Itanium 2 Processor 6M", *IEEE Micro Magazine*, vol. 24, pp. 10–18, marzo-aprile 2004.

Saha, D. e Mukherjee, A., "Pervasive computing: a paradigm for the 21st century", *IEEE Computer Magazine*, vol. 36, pp. 25–31, marzo 2003.

Sakamura, K., "Making computers invisibile", *IEEE Micro Magazine*, vol. 22, p. 711, 2002.

Sanchez, D. e Kozyrakis, C., "Vantage: Scalable and Efficient Fine-Grain Cache Partitioning", *Atti del 38° Ann. Int. Symp. on Computer Arch.*, ACM, pp. 57–68, 2011.

Scales, D.J., Gharachorloo, K. e Thekkath, C.A., "Shasta: a low-overhead software-only approach for supporting fine-grain shared memory", *Atti della 7ª Int'l Conf. on Arch. Support for Prog. Lang. and Oper. Syst.*, ACM, pp. 174–185, 1996.

Seltzer, M., Bostic, K., McKusick, M.K. e Staelin, C., "An implementation of a log-structured file system for UNIX", *Atti inverno 1993 USENIX Technical Conf.*, pp. 307–326, 1993.

Shanley, T. e Anderson, D., *PCI system architecture*, quarta edizione, Reading, MA, Addison-Wesley, 1999.

Shoufan, A., Huber, N. e Molter, H.G., "A Novel Cryptoprocessor Architecture for Chained Merkle Signature Schemes", *Microprocessors and Microsystems*, vol. 35, pp. 34–47, febbraio 2011.

Singh, G.: "The IBM PC: The Silicon Story", *IEEE Computer*, vol. 44, pp. 40–45, agosto 2011.

Slater, R., *Portraits in silicon*, Cambridge, MA, MIT Press, 1987.

Snir, M., Otto, S.W., Huss-Lederman, S., Walker, D.W. e Dongarra, J., *MPI: the complete reference manual*, Cambridge, MA, MIT Press, 1996.

Solari, E. e Congdon, B., *PCI express design & system architecture*, Research Tech, INc., 2005.

Solari, E. e Willse, G., *PCI and PCI-X hardware and software*, sesta edizione, San Diego, CA, Annabooks, 2004.

Sorin, D.J., Hill, M.D. e Wood, D.A., *A Primer on Memory Consistency and Cache Coherence*, San Francisco, Morgan & Claypool, 2011.

Stets, R., Dwarkadas, S., Hardavellas, N., Hunt, G., Kontothanassis, L., Parthasarathy, S. e Scott, M., "CASHMERE-2L: software coherent shared memory on clustered remote-write networks", *Atti del 16° Symp. on Operating Systems Principles*, ACM, pp. 170–183, 1997.

Summers, C.K., *ADSL: standards, implementation and architecture*, Boca Raton, FL, CRC Press, 1999.

Sunderram, V.B., "PVM: a framework for parallel distributed computing", *Concurrency: practice and experience*, vol. 2, pp. 315–339, dicembre 1990.

Swan, R.J., Fuller, S.H. e Siewiorek, D.P., "Cm*-A modular multiprocessor", *Atti NCC*, pp. 645–655, 1977.

Tan, W.M., *Developing USB PC Peripherals*, San Diego, CA, Annabooks, 1997.

Tanenbaum, A.S. e Wetherall, D.J., *Computer Networks*, quinta edizione, Upper Saddle River, NJ, Prentice Hall, 2011.

Thompson, K., "Reflections on Trusting Trust", *Comunicazioni dell'ACM*, vol. 27, pp. 761–763, agosto 1984.

Thompson, J., Dreisigmeyer, D.W., Jones, T., Kirby, M. e Ladd, J., "Accurate Fault Prediction of BlueGene/P RAS Logs via Geometric Reduction", *IEEE*, pp. 8–14, 2010.

Treleaven, P., "Control-Driven, Data-Driven, and Demand-Driven computer architecture", *Parallel Computing*, vol. 2, 1985.

Tu, X., Fan, X., Jin, H., Zheng, L. e Peng, X., "Transactional Memory Consistency: A New Consistency Model for Distributed Transactional Memory", *Atti della 3ª Int. Joint Conf. on Computational Science and Optimization*, IEEE, 2010.

Vahalia, U., *UNIX Internals*, Upper Saddle River, NJ, Prentice Hall, 1996.

Vahid, F., "The softening of hardware", *IEEE Computer Magazine*, vol. 36, pp. 27–34, aprile 2003.

Vetter, P., Goderis, D., Verpooten L. e Granger, A., "Systems aspects of APON/VDSL deployment", *IEEE Commun. Magazine*, vol. 38, pp. 66–72, maggio 2000.

Vu, T.D., Zhang, L. e Jesshope, C., "The Verification of the On-Chip COMA Cache Coherence Protocol", *Atti della 12ª Int'l Conf. on Algebraic Methodology and Software Technology*, Springer-Verlag, pp. 413–429, 2008.

Weiser, M., "The computer for the 21st Century", *IEEE Pervasive Computing*, vol. 1, pp. 19–25, gennaio-marzo 2002 (pubblicato in originale su *Scientific American*, settembre 1991).

Wilkes, M.V., "Computers then and now", *J. ACM*, vol. 15, pp. 1–7, gennaio 1968.

Wilkes, M.V., "The best way to design an automatic calculating machine", *Atti della Manchester Univ. Computer Inaugural Conf.*, 1951.

Wing-Kei, Y., Huang, R., Xu, S., Wang, S.-E., Kan, E. e Suh, G.E., "SRAM-DRAM Hybrid Memory with Applications to Efficient Register Files in Fine-Grained Multi-Threading Architectures", *Atti del 38° Int. Symp. on Computer Arch., ACM*, 2011.

Yamamoto, S. e Nakao, A., "Fast Path Performance of Packet Cache Router Using Multi-core Network Processor", *Atti del 7° Symp. on Arch. for Network and Comm. Sys.*, ACM/IEEE, 2011.

Zhang, L. e Jesshope, C., "On-Chip COMA Cache-Coherence Protocol for Microgrids of Microthreaded Cores", *Atti della 2007 European Conf. on Parallel Processing*, Springer-Verlag, pp. 38–48, 2008.

APPENDICE A

Aritmetica binaria

L'aritmetica dei calcolatori è un po' diversa da quella che abbiamo studiato alle elementari. La differenza principale sta nel fatto che i calcolatori eseguono operazioni su numeri che hanno una precisione finita e prefissata. Un'altra differenza è che quasi tutti i calcolatori rappresentano i numeri nel sistema binario e non in quello decimale. Questa appendice si occupa precisamente di questi argomenti.

A.1 Numeri a precisione finita

Quando si fanno calcoli aritmetici, in genere non si dà particolare importanza al numero di cifre decimali utilizzate per rappresentare i numeri. I fisici calcolano che nell'universo esistono 10^{78} elettroni, senza preoccuparsi del fatto che ci vogliono 79 cifre decimali per scrivere quel numero per esteso. Una persona che usi carta e penna per calcolare il valore di una funzione e che abbia bisogno di una risposta con sei cifre significative, svolgerà i calcoli mantenendo i risultati intermedi con una precisione di sette, otto cifre, o di quante siano necessarie. Non succede mai che il foglio di carta sia troppo piccolo per contenere i numeri con sette cifre significative.

Le cose sono molto diverse nei calcolatori. Per molti di loro, la quantità di memoria disponibile per la memorizzazione di un numero è fissata al momento della progettazione. Il programmatore può riuscire con qualche sforzo a raddoppiare o a triplicare il numero di cifre che può usare, ma ciò non cambia la natura del problema. La natura intrinsecamente finita di un calcolatore ci costringe a trattare solo numeri rappresentati per mezzo di un numero limitato e costante di cifre, cioè **numeri a precisione finita**.

Per studiare le proprietà dei numeri a precisione finita, cominciamo con l'esaminare l'insieme degli interi positivi rappresentabili con tre cifre decimali, senza virgola e senza segno. Questo insieme ha esattamente 1000 elementi: 000, 001, 002, 003, ..., 999. Con queste limitazioni non possiamo esprimere:

1. i numeri più grandi di 999;
2. i numeri negativi;
3. le frazioni (proprie);
4. i numeri irrazionali;
5. i numeri complessi.

Una proprietà importante dell'aritmetica sull'insieme degli interi è la **chiusura** rispetto alle operazioni di addizione, sottrazione e moltiplicazione. In altre parole, per ogni coppia d'interi i e j, i risultati delle operazioni $i + j$, $i - j$ e $i \times j$ sono tutti interi. L'insieme degli interi non è chiuso rispetto alla divisione, perché esistono valori di i e j per cui i/j non è esprimibile come un intero (per esempio 7/2 e 1/0).

I numeri a precisione finita non sono chiusi rispetto ad alcuna delle quattro operazioni fondamentali, come mostriamo nell'esempio seguente usando numeri decimali con tre cifre decimali:

600 + 600 = 1200 (troppo grande)
003 – 005 = – 2 (negativo)
050 × 050 = 2500 (troppo grande)
007 / 002 = 3,5 (non è intero)

Le violazioni della chiusura possono essere distinte in due classi disgiunte: le operazioni che danno un risultato più grande del massimo dell'insieme (*overflow*) o più piccolo del minimo dell'insieme (*underflow*), e le operazioni il cui risultato non appartiene all'insieme. Le prime tre violazioni dell'esempio sono del primo tipo, la quarta è del secondo.

I calcolatori dispongono di una memoria finita e quindi devono svolgere necessariamente conti aritmetici a precisione finita. Di conseguenza i risultati di alcuni calcoli saranno semplicemente sbagliati dal punto di vista della matematica classica. Un dispositivo di calcolo perfettamente funzionante che fornisca risultati errati può sembrare di primo acchito una stranezza, ma l'errore è una conseguenza logica della sua natura finita. Alcuni calcolatori dispongono di hardware speciale per il rilevamento di errori di overflow.

L'algebra dei numeri a precisione finita è diversa dall'algebra normale. Per esempio consideriamo l'espressione

$$a + (b - c) = (a + b) - c$$

(che è un'identità in base alla proprietà associativa di somma e sottrazione) e valutiamola per $a = 700$, $b = 400$ e $c = 300$. Per calcolare il membro di sinistra cominciamo da $(b - c)$, che dà 100, e aggiungiamo questo valore ad a, per un totale di 800. Per calcolare il secondo membro, calcoliamo prima $(a + b)$, e ciò causa un overflow nell'aritmetica finita degli interi a tre cifre. Il risultato potrebbe dipendere dalla macchina utilizzata per il calcolo, ma non sarà mai 1100. La sottrazione di 300 da un qualunque numero diverso da 1100 non varrà mai 800. L'associatività non vale e l'ordine di esecuzione delle

operazioni è discriminante. Come ulteriore esempio, consideriamo la legge distributiva (del prodotto rispetto alla somma):

$$a \times (b - c) = a \times b - a \times c$$

e valutiamo entrambi i membri per $a = 5$, $b = 210$ e $c = 195$. Il primo membro vale 5×15, cioè 75, ma il secondo membro non dà 75 perché a × b causa un overflow.

A giudicare da questi esempi, si potrebbe concludere che i calcolatori, pur essendo dei dispositivi per uso generale, sono poco portati per i calcoli aritmetici a causa della loro natura intrinsecamente finita. Ovviamente questa conclusione è errata, ma serve a illustrare l'importanza di comprendere il funzionamento e le limitazioni dei calcolatori.

A.2 Sistemi di numerazione in base fissa

Un numero decimale consiste in una sequenza di cifre decimali più, eventualmente, una virgola decimale. La Figura A.1 ne mostra la forma generale e la sua interpretazione posizionale. La scelta di 10 come **base** per l'elevamento a potenza dipende dal fatto che stiamo usando i decimali, ovvero i numeri in base 10. In informatica conviene spesso usare basi diverse da 10. Le basi più importanti sono 2, 8 e 16 e i sistemi di numerazione basati su di loro si chiamano rispettivamente **binari**, **ottali** ed **esadecimali**.

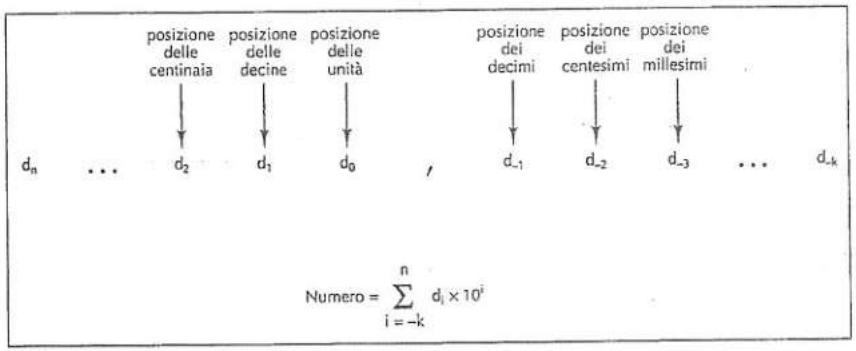


Figura A.1 Forma generale di un numero decimale.

Un sistema di numerazione in base k richiede k simboli diversi per rappresentare le cifre da 0 a $k - 1$. I numeri decimali si scrivono con le 10 cifre decimali

0 1 2 3 4 5 6 7 8 9

Invece i numeri binari non usano dieci cifre, ma si scrivono con le sole due cifre binarie

0 1

I numeri ottali si costruiscono a partire dalle otto cifre ottali

0 1 2 3 4 5 6 7

mentre i numeri esadecimali richiedono 16 cifre, cioè sei nuovi simboli. Per convenzione si usano le lettere maiuscole da A a F per le cifre che seguono 9. Dunque i numeri esadecimali si scrivono con le cifre

0 1 2 3 4 5 6 7 8 9 A B C D E F

Con l'espressione "**bit**" si intende una cifra che può assumere i valori 0 e 1. La Figura A.2 mostra il numero decimale 2001 espresso in binario, in ottale, in decimale e in forma esadecimale. Il numero 7B9 è ovviamente esadecimale, perché il simbolo B si trova solo nei numeri esadecimali. La stringa 111 invece può essere interpretata come un numero in uno qualsiasi dei quattro sistemi di numerazione presentati. Per evitare l'ambiguità, si usa indicare la base con un pedice 2, 8, 10 o 16, tutte le volte che non può essere intuita direttamente dal contesto.

Binario 1 1 1 1 1 0 1 0 0 0 1

$1\times2^{10}+1\times2^{9}+1\times2^{8}+1\times2^{7}+1\times2^{6}+0\times2^{5}+1\times2^{4}+0\times2^{3}+0\times2^{2}+0\times2^{1}+1\times2^{0}=$

$=1024+512+256+128+64+0+16+0+0+0+1=2001$

Ottale 3 7 2 1

$3\times8^{3}+7\times8^{2}+2\times8^{1}+1\times8^{0}=$

$=1536+448+16+1=2001$

Decimale 2 0 0 1

$2\times10^{3}+0\times10^{2}+0\times10^{1}+1\times10^{0}=$

$=2000+0+0+1=2001$

Esadecimale 7 D 1

$7\times16^{2}+13\times16^{1}+1\times16^{0}=$

$=1792+208+1=2001$

Figura A.2 Il numero 2001 in binario, ottale, decimale ed esadecimale.

La Figura A.3 mostra un certo numero d'interi non negativi espressi nelle quattro basi. Forse un giorno, fra migliaia di anni, gli archeologi scopriranno questa tabella e la considereranno la Stele di Rosetta dei sistemi di numerazione usati a cavallo tra il tardo XX secolo e gli inizi del XXI secolo.

A.3 Conversione tra basi

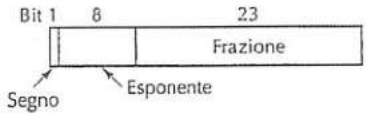
La conversione tra i sistemi ottale o esadecimale e binario è facile. Per convertire un numero binario in ottale basta suddividerlo in gruppi di 3 bit, avendo cura di raggruppare i 3 bit appena a sinistra (o a destra) della virgola e poi tutti gli altri bit. Ogni gruppo di 3 bit può essere convertito direttamente in una cifra ottale da 0 a 7, secondo la conversione indicata dalle prime righe della tabella nella Figura A.3. Potrebbe essere necessario aggiungere uno o più zeri in testa o in coda ai gruppi di bit che non hanno 3 cifre.

Decimale	Binario	Ottale	Esadecimale
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	3	3
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10
20	10100	24	14
30	11110	36	1E
40	101000	50	28
50	110010	62	32
60	111100	74	3C
70	1000110	106	46
80	1010000	120	50
90	1011010	132	5A
100	11001000	144	64
1000	1111101000	1750	3E8
2989	101110101101	5655	BAD

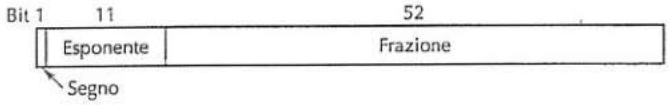
Figura A.3 Alcuni numeri decimali e i loro equivalenti binari, ottali, decimali ed esadecimali.

Anche la conversione da ottale a binario è banale. Ogni cifra ottale viene semplicemente rimpiazzata dall'equivalente numero binario a 3 bit. La conversione da esadecimale a binario è praticamente analoga a quella ottale-binario, con la differenza che le cifre esadecimali corrispondono a gruppi di 4 cifre binarie e non di 3. La Figura A.4 fornisce alcuni esempi di conversioni.

Entrambi i formati iniziano con un bit che rappresenta il segno dell'intero numero, 0 se è positvo e 1 se è negativo. Di seguito vi è l'esponente, che utilizza la notazione in eccesso 127 per la precisione singola e la notazione in eccesso 1023 per la precisione doppia. Gli esponenti minimo (0) e massimo (255 e 2047) non sono utilizzati per i numeri normalizzati; essi hanno un utilizzo speciale che descriveremo in seguito. Infine troviamo le frazioni, rispettivamente di 23 e 52 bit.



(a)



(b)

Figura B.4 Formato IEEE in virgola mobile. (a) Precisione singola. (b) Precisione doppia.

Una frazione normalizzata inizia con una virgola, seguita da un bit 1 e poi dal resto della frazione. Seguendo una pratica iniziata con il PDP-11 gli autori dello standard hanno deciso che il bit 1 che dovrebbe precedere la virgola non doveva essere memorizzato, dato che se ne poteva implicitamente assumere la presenza. Lo standard definisce quindi la frazione in modo leggermente diverso dal solito; consiste in un bit 1 sottointeso, in una virgola binaria sottointesa e poi in 23 (o 52) bit arbitrari. Se tutti e 23 (o 52) i bit della frazione valgono 0, il valore numerico della frazione è 1,0. Se tutti valgono 1, il valore numerico della frazione è leggermente inferiore a 2,0. Per evitare confusione con la forma convenzionale della frazione, la combinazione del valore 1 sottointeso, della virgola binaria sottointesa e dei 23, o 52, bit espliciti è chiamata **significando**, invece di frazione o mantissa. Tutti i numeri normalizzati hanno un significando, *s*, compreso nell'intervallo [1,2].

Le caratteristiche numeriche dello standard sono elencate nella Figura B.5. Come esempio consideriamo i numeri 0,5, 1 e 1,5 nel formato normalizzato in precisione singola. Le loro rappresentazioni in esadecimale sono rispettivamente 3F000000, 3F800000 e 3FC00000.

Uno dei tipici problemi dell'aritmetica in virgola mobile è il modo di gestire l'underflow, l'overflow e i numeri non inizializzati. Lo standard IEEE tratta questi problemi in modo esplicito, traendo parte del proprio approccio da quello utilizzato nel calcolatore CDC 6600. Oltre ai numeri normalizzati lo standard definisce quattro altri tipi numerici, descritti in seguito e mostrati nella Figura B.6.

Caratteristica	Precisione singola	Precisione doppia
Bit nel segno	1	1
Bit nell'esponente	8	11
Bit nella frazione	23	52
Bit totali	32	64
Notazione dell'esponente	in eccesso 127	in eccesso 1023
Intervallo dell'esponente	da –126 a +127	da –1022 a +1023
Numero normalizzato più piccolo	2^{-126}	2^{-1022}
Numero normalizzato più grande	circa 2^{128}	circa 2^{1024}
Intervallo decimale	circa da 10^{-38} a 10^{38}	circa da 10^{-308} a 10^{308}
Numero denormalizzato più grande	circa 10^{-45}	circa 10^{-324}

Figura B.5 Caratteristiche dell'aritmetica IEEE in virgola mobile.

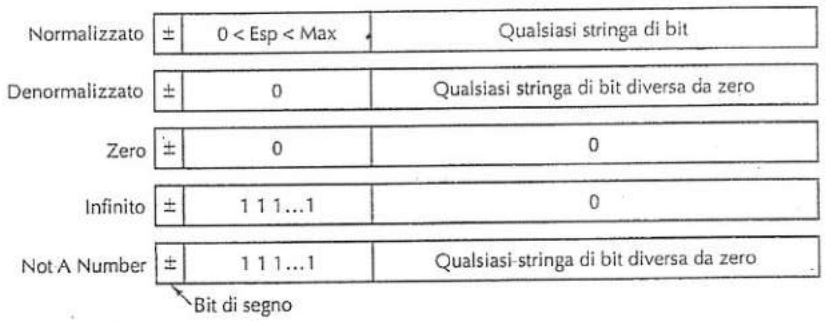


Figura B.6 Tipi di numeri IEEE.

Quando il risultato di un calcolo ha un modulo minore del più piccolo numero in virgola mobile normalizzato sorge un problema, dato che questo sistema non può rappresentarlo.

Prima della definizione dello standard IEEE le scelte più comuni erano: impostare il risultato a zero e continuare oppure generare un'eccezione di underflow. Dato che nessuna di queste soluzioni era veramente soddisfacente, IEEE ha inventato i **numeri denormalizzati**. Questi numeri hanno come esponente 0 e come frazione i successivi 23 (o 52), bit. Il bit 1 implicito alla sinistra della virgola binaria diventa ora 0. È possibile distinguere i numeri normalizzati da quelli denormalizzati, dato che gli ultimi non possono avere un valore 0 come esponente.

Il più piccolo numero normalizzato in precisione singola ha 1 come esponente e 0 come frazione, e rappresenta il numero $1{,}0 \times 2^{-126}$. Il più grande numero denormalizzato

ha 0 come esponente e tutti i bit della frazione e impostati a 1, e rappresenta circa $0{,}9999999 \times 2^{-126}$, che è praticamente uguale al valore precedente. Occorre tuttavia notare che questo numero ha solo 23 bit significativi, contro i 24 dei numeri normalizzati.

Al diminuire del risultato dei calcoli l'esponente rimane fisso a 0, mentre si azzerano progressivamente i primi bit della frazione, riducendo in questo modo sia il valore rappresentato sia il numero di bit significativi della frazione. Il più piccolo numero denormalizzato diverso da zero consiste in un valore 1 nel bit più a destra, con tutti gli altri bit pari a 0. Dato che l'esponente rappresenta 2^{-126} e la frazione rappresenta 2^{-23}, il valore è 2^{-149}. Questo schema rende meno grave il problema dell'underflow, dato che quando il risultato non può essere espresso come un risultato normalizzato non si salta improvvisamente al valore 0, ma il numero di cifre significative diminuisce progressivamente.

In questo schema lo zero ha due rappresentazioni, una positiva e una negativa, determinate dal bit del segno. Entrambe hanno 0 come esponente e come frazione. Anche in questo caso il bit a sinistra della virgola binaria è implicitamente 0 invece di 1.

L'overflow non può essere gestito in modo graduale come l'underflow, dato che non ci sono altre combinazioni di bit. Esiste però una rappresentazione per l'infinito che consiste in un esponente in cui tutti i bit valgono 1 (non permesso per i numeri normalizzati) e di una frazione che vale 0. Questo numero segue le regole matematiche dell'infinito. Per esempio la somma tra infinito e qualsiasi altro valore dà come risultato infinito e qualsiasi numero finito diviso infinito dà come risultato zero. Analogamente ogni numero finito diviso zero dà come risultato infinito.

E se si divide infinito per infinito? Il risultato non è definito. Per gestire questo caso è previsto un ulteriore formato, chiamato **NaN (Not a Number)**. Anch'esso può essere usato come operando con risultati prevedibili.

PROBLEMI

1. Si convertano i seguenti numeri nel formato IEEE in precisione singola. Si scriva il risultato in esadecimale.
   a. 9
   b. 5/32
   c. –5/32
   d. 6,125

2. Si convertano da esadecimali a decimali i seguenti numeri rappresentati nel formato IEEE in virgola mobile in precisione singola:
   a. 42E48000H
   b. 3F880000H
   c. 00800000H
   d. C7F00000H

3. Sul calcolatore 370 il formato dei numeri in virgola mobile in precisione singola è composto da un esponente di 7 bit in notazione in eccesso 64 e da una frazione contenente 24 bit più un bit per il segno; la virgola si trova all'estremità sinistra della frazione. La base dell'elevamento a potenza è 16. L'ordine dei campi è: bit del segno, esponente, frazione. Si esprima, in esadecimale, il numero 7/64 come un numero normalizzato in questo sistema.

4. I seguenti numeri binari in virgola mobile consistono in un bit del segno, un esponente di una base 2 espresso in notazione in eccesso 64 e una frazione a 16 bit. Li si normalizzi.
   a. 0 1000000 0001010100000001
   b. 0 0111111 0000001111111111
   c. 0 1000011 1000000000000000

5. Per sommare due numeri in virgola mobile occorre modificare gli esponenti (traslando i bit della frazione) affinché abbiano lo stesso valore. In seguito è possibile sommare le frazioni e, se necessario, normalizzare il risultato. Si sommino i numeri IEEE in precisione singola 3EE00000H e 3D800000H e si esprima in esadecimale il risultato normalizzato.

6. La Società di Calcolatori BraccioCorto ha deciso di produrre una macchina con numeri in virgola mobile a 16 bit. Il formato in virgola mobile del Modello 0,001 ha un bit per il segno, 7 bit per l'esponente in notazione in eccesso 64 e 8 bit per la frazione. Il formato del Modello 0,002 ha un bit per il segno, 5 bit per l'esponente in notazione in eccesso 16 e 10 bit per la frazione. Entrambi usano la base 2 per l'elevamento a potenza. Quali sono i numeri più piccoli e più grandi rappresentabili sui due modelli? Quante cifre decimali di precisione ha, all'incirca, ciascuno di loro? Comprereste uno di questi modelli?

7. Esiste una situazione nella quale un'operazione su due numeri in virgola mobile possa causare una riduzione drastica del numero di bit significativi nel risultato. Qual è?

8. Alcuni coprocessori in virgola mobile hanno un'istruzione predefinita per la radice quadrata. Un algoritmo possibile è di tipo iterativo (per esempio, Newton-Raphson). Gli algoritmi iterativi hanno bisogno di un'approssimazione iniziale del risultato che poi migliorano costantemente. Come si può ottenere una veloce approssimazione della radice quadrata di un numero in virgola mobile?

9. Si scriva una procedura per sommare due numeri IEEE in virgola mobile in precisione singola. Ciascun numero è rappresentato da un array di 32 elementi booleani.

10. Si scriva una procedura per sommare due numeri in virgola mobile in precisione singola che usano la base 16 per l'esponente e la base 2 per la frazione, ma non hanno un bit 1 sottointeso alla sinistra della virgola. Un numero normalizzato ha, nel quattro bit più a sinistra della frazione, 0001, 0010, ..., 1111, ma non 0000. Un numero viene normalizzato traslando a sinistra di 4 bit la frazione e sottraendo 1 all'esponente.

C.4.5 Chiamate di subroutine

L'8088 dispone di un'istruzione per la chiamata di procedure che, nella terminologia del linguaggio assemblativo, si chiamano **subroutine**. Analogamente al caso delle istruzioni di salto, anche per questo tipo d'istruzioni si distinguono le **chiamate ravvicinate** e quelle **a distanza** (*near* e *far call*). Solo le prime sono state implementate nell'interprete. La destinazione può essere un'etichetta o può trovarsi presso un indirizzo effettivo. I parametri necessari alla subroutine devono essere impilati sullo stack in ordine inverso, come illustrato nella Figura C.6. Nella terminologia del linguaggio assemblativo i parametri si chiamano **argomenti**, ma i due termini sono interscambiabili. Dopo il push degli argomenti si esegue l'istruzione CALL, che comincia con l'impilare sullo stack il valore corrente del program counter per salvare l'indirizzo di ritorno (quello dal quale bisogna riprendere l'esecuzione della routine chiamante al termine della subroutine).

BP+8	...	
BP+6	Argomento 2	
BP+4	Argomento 1	
BP+2	Indirizzo di ritorno	
BP	Vecchio BP	← BP
BP–2	Variabile locale 1	
BP–4	Variabile locale 2	
BP–6	Variabile locale 3	
BP–8	Risultato temporaneo	← SP

Figura C.6 Un esempio di stack durante l'esecuzione di una subroutine.

A questo punto viene caricato il nuovo program counter a partire dall'etichetta o dall'indirizzo effettivo. Se la chiamata è a distanza, il registro del segmento di codice (CS) viene impilato prima del program counter (PC) e si procede al caricamento di entrambi a partire dai dati immediati o dall'indirizzo effettivo. Questo caricamento conclude l'esecuzione dell'istruzione CALL.

L'istruzione di ritorno, RET, non fa altro che recuperare l'indirizzo di ritorno dalla cima dello stack e lo scrive nel program counter, così l'esecuzione del programma riprende immediatamente dall'istruzione successiva alla CALL. Alle volte l'istruzione RET contiene un numero positivo come dato immediato che rappresenta il numero di byte occupati dagli argomenti impilati sullo stack prima della chiamata; basta sommare questo numero a SP per rimuovere gli argomenti dallo stack. La variante a distanza RETF esegue la pop del registro del segmento di codice dopo la pop del program counter, proprio come ci si poteva aspettare.

Gli argomenti devono essere accessibili alla subroutine, perciò in genere la sua esecuzione comincia con il push del puntatore base e una copia del valore corrente di SP in BP. Così facendo, il puntatore base punta ora al suo valore precedente e l'indirizzo di ritorno si trova alla posizione BP + 2, mentre il primo e il secondo argomento si trovano rispettivamente alla posizione BP + 4 e BP + 6. Se la procedura ha delle variabili locali allora è possibile sottrarre il numero di byte richiesto dal valore del puntatore allo stack e le variabili possono essere indirizzate a partire dal puntatore base sommandogli offset negativi. Nell'esempio della Figura C.6 ci sono tre variabili locali di una parola ciascuna, localizzate agli indirizzi BP – 2, BP – 4 e BP – 6. Dunque il registro BP utilizzabile per raggiungere l'intero insieme di argomenti e variabili locali della procedura corrente.

Lo stack viene usato nel modo consueto per contenere i risultati intermedi o per preparare gli argomenti per la chiamata successiva. Al ritorno da una subroutine lo stack può essere ripristinato senza calcolare lo spazio da essa utilizzato, ma basterà copiare il puntatore base nel puntatore allo stack, effettuare la pop del vecchio BP e infine eseguire l'istruzione RET.

Durante una chiamata di subroutine i valori di alcuni registri possono venir modificati. È buona pratica usare qualche tipo di convenzione, di modo che la routine chiamante non debba preoccuparsi dei registri usati dalla routine chiamata. La soluzione più semplice è adottare la stessa convenzione che esiste tra le chiamate di sistema e le subroutine ordinarie: la routine chiamata può modificare solo i registri AX e DX, perciò se uno di questi contiene informazioni importanti per il chiamante è consigliabile che questi ne salvi il contenuto nello stack prima d'impilare gli argomenti della chiamata. Se la subroutine usa anche altri registri allora può impilarli sullo stack immediatamente all'inizio della sua esecuzione e poi ripristinarli prima di chiamare l'istruzione RET. In breve, una buona convenzione richiede che il chiamante salvi AX e DX se li ritiene importanti, mentre il chiamato salva ogni altro registro che intende modificare.

C.4.6 Chiamate di sistema e subroutine di sistema

Per separare le operazioni su file (apertura, chiusura, lettura e scrittura) dalla programmazione in linguaggio assemblativo, i programmi sono eseguiti a livello superiore rispetto a quello del sistema operativo. L'interprete è stato dotato di sette chiamate di sistema e cinque funzioni (elencate nella Figura C.7) che ne permettono l'esecuzione su diverse piattaforme hardware.

Queste routine possono essere attivate con la sequenza di chiamata standard: si comincia con il push degli argomenti sullo stack in ordine inverso, si impila il numero di chiamata e infine si esegue l'istruzione SYS senza operandi per la trap di sistema. La routine di sistema reperisce nello stack tutte le informazioni che le sono necessarie, compreso il numero di chiamata e il servizio di sistema richiesto. I risultati sono restituiti nel registro AX o nella combinazione di registri DX:AX (se il risultato è di tipo long).

L'istruzione SYS non modifica i valori degli altri registri. Al suo completamento, gli argomenti restano impilati sullo stack e vanno rimossi (a cura del chiamante) agendo sul puntatore allo stack, a meno che non servano per una chiamata successiva.

Per comodità, è possibile definire all'inizio del programma assemblativo alcune costanti per i nomi delle chiamate di sistema, in modo da poterle richiamare per nome invece che per numero. Gli esempi che seguono ci permetteranno di descrivere in dettaglio molte chiamate di sistema, perciò in questo paragrafo ne diamo un resoconto abbreviato.

No.	Nome	Argomenti	Valore restituito	Descrizione
5	_OPEN	*name, 0/1/2	descrittore di file	Apre un file
8	CREAT	*name, *mode	descrittore di file	Crea un file
3	READ	fd, buf, nbytes	# di byte	Legge n byte nel buffer buf
4	WRITE	fd, buf, nbytes	# di byte	Scrive n byte nel buffer buf
6	CLOSE	fd	0 se ha successo	Chiude il file con descrittore fd
19	LSEEK	fd, offset(long), 0/1/2	posizione (long)	Sposta il puntatore del file
1	EXIT	status		Interrompe un processo
117	_GETCHAR		carattere letto	Legge un carattere da standard input
122	_PUTCHAR	char	byte scritto	Scrive un carattere sullo standard output
127	_PRINTF	*format, arg		Stampa formattata sullo standard output
121	_SPRINTF	buf, *format, arg		Stampa formattata nel buffer buf
125	_SSCANF	buf, *format, arg		Legge gli argomenti dal buffer buf

Figura C.7 Alcune chiamate di sistema UNIX e alcune subroutine implementate nell'interprete.

L'apertura dei file si effettua con le chiamate OPEN e CREAT, che prendono come primo argomento l'indirizzo della stringa contenente il nome del file. Il secondo argomento di OPEN vale 0 (se il file va aperto in lettura), 1 (se in scrittura) o 2 (lettura/scrittura). Se il file è scrivibile, ma non esiste, allora la chiamata provvede a crearlo. La chiamata CREAT produce un file vuoto con i permessi specificati dal secondo argomento. OPEN e CREAT restituiscono entrambe un piccolo intero nel registro AX; questo numero si chiama **descrittore di file** e può essere usato per lettura, scrittura o chiusura del file. Un descrittore di file negativo indica il fallimento della chiamata. All'avvio di un programma vengono aperti automaticamente tre file con descrittori 0 (standard input), 1 (standard output) e 2 (standard error output).

Le chiamate READ e WRITE hanno tre argomenti: il descrittore di file, un buffer per i dati e il numero di byte da trasferire. Dal momento che gli argomenti sono impilati in ordine inverso, si effettua prima il push del numero di byte, e poi i push dell'indirizzo del buffer, del descrittore di file e infine del numero di chiamata (READ o WRITE). Quest'ordine rispetta la sequenza degli argomenti della chiamata C standard

```
read(fd, buffer, bytes);
```

i cui parametri sono impilati nell'ordine *bytes*, *buffer* e *fd*.

La chiamata CLOSE richiede solo il descrittore di file e restituisce 0 in AX se la chiusura del file ha avuto successo. La chiamata EXIT richiede di impilare sullo stack lo stato di uscita del processo e non restituisce alcun valore.

La chiamata LSEEK modifica il **puntatore di lettura/scrittura** di un file già aperto. Il primo argomento è il descrittore di file, e il secondo è un dato long, perciò va scomposto nelle due parole che lo costituiscono e va impilata per prima la metà più significativa, anche nel caso in cui l'offset fosse contenuto tutto nella metà meno significativa. Il terzo argomento indica se il puntatore di lettura/scrittura va calcolato relativamente all'inizio del file (caso 0), alla posizione corrente (caso 1) o alla fine del file (caso 2). Il valore restituito è la nuova posizione del puntatore relativa all'inizio del file, memorizzato come long. La funzione GETCHAR legge un carattere dall'input standard, lo salva in AL e azzera AH. In caso di fallimento AX è posto invece a –1. La chiamata PUTCHAR scrive un byte sullo standard output e, in caso di successo, restituisce lo stesso byte scritto, mentre restituisce –1 in caso di fallimento.

La chiamata PRINTF stampa informazioni formattate. Il primo argomento della chiamata è l'indirizzo della stringa di formattazione, che specifica il formato dell'output. La stringa "%d" indica che l'argomento successivo è un intero presente nello stack e che quindi va convertito in notazione decimale prima della stampa. Allo stesso modo, "%x" indica il formato esadecimale e "%o" richiede la conversione in ottale. Inoltre, "%s" indica che l'argomento successivo è una stringa che termina con il carattere zero; lo stack contiene l'indirizzo di memoria dove reperirla. Il numero di argomenti che si trova sullo stack dovrebbe corrispondere al numero d'indicazioni di conversioni presente nella stringa di formattazione.

Per esempio, la chiamata C

```
printf("x = %d e y = %d\n", x, y);
```

stampa la stringa in cui le due occorrenze di "%d" sono state sostituite con i valori numerici di x e y. Anche in questo caso, per ragioni di compatibilità con il C, l'ordine dei push è "y", "x" e infine l'indirizzo della stringa di formattazione. Il motivo di questa convenzione è che la *printf* ha un numero variabile di parametri e quindi impilandoli in ordine inverso si è certi di poter individuare facilmente la stringa, che occupa sempre l'ultimo posto. Se i parametri fossero stati impilati da sinistra a destra la stringa di formattazione si troverebbe in qualche locazione precedente dello stack e la procedura *printf* non saprebbe come rintracciarla.

Per quanto riguardo la SPRINTF, il suo primo argomento è il buffer che riceve la stringa da stampare al posto dello standard output. Gli altri argomenti sono gli stessi di PRINTF. La chiamata SSCANF è il contrario di PRINTF nel senso che il suo primo argomento è una stringa contenente gli interi in notazione decimale, ottale o esadecimale, mentre l'argomento successivo è la stringa di formattazione contenente le indicazioni per le conversioni. Gli altri argomenti sono gli indirizzi delle parole di memoria che devono accogliere le informazioni dopo la conversione. Queste subroutine di sistema sono molto versatili e il Paragrafo C.8 ne presenta vari esempi di utilizzo.

C.4.7 Osservazioni sull'insieme d'istruzioni

La definizione ufficiale dell'8088 comprende un prefisso di **deroga di segmento** (*segment override*) che facilita la possibilità di utilizzare indirizzi effettivi che provengono da segmenti differenti; il primo indirizzo di memoria indicato dopo il prefisso viene risolto usando il registro di segmento corrispondente al segmento indicato. Per esempio, l'istruzione

```
ESEG MOV DX,(BX)
```

calcola per prima cosa l'indirizzo di BX usando il segmento extra, poi trasferisce il suo contenuto in DX. Tuttavia non è possibile derogare all'uso del segmento di stack quando l'indirizzo specificato si basa su SP, né all'uso del segmento extra in caso d'istruzione

sulle stringhe che utilizza il registro DI. I registri di segmento SS, DS ed ES istruzioni MOV, ma non è permesso trasferire dati immediati nei registri di segmento né utilizzarli nell'operazione XCHG. La programmazione con le deroghe di segmento e con le modifiche dei segmenti di registro è abbastanza complicata e dovrebbe essere evitata laddove possibile. L'interprete usa segmenti di registro costanti e quindi non incappa in problemi del genere.

La maggior parte dei calcolatori mette a disposizione istruzioni in virgola mobile, disponibili direttamente nel processore, in un coprocessore separato, oppure solo attraverso l'interpretazione via software per mezzo di una specie di trap in virgola mobile. La trattazione di queste funzionalità esula dagli scopi di questa appendice.

C.5 Assemblatore

Conclusa la trattazione dell'architettura dell'8088, ci occupiamo della sua programmazione in linguaggio assemblativo e in particolare degli strumenti che mettiamo a disposizione per il suo apprendimento. Affrontiamo per primo l'assemblatore, poi il tracer e infine forniamo qualche informazione pratica circa il loro utilizzo.

C.5.1 Introduzione

Fin qui abbiamo fatto riferimento alle istruzioni mediante i loro **nomi mnemonici**, cioè quelle abbreviazioni facili da ricordare come ADD e CMP. Lo stesso abbiamo fatto con i registri, indicati per mezzo di nomi simbolici quali AX e BP. Un programma scritto usando nomi simbolici per le istruzioni e per i registri si chiama **programma in linguaggio assemblativo**. L'esecuzione di un tale programma richiede in primo luogo la sua traduzione in stringhe binarie che avviene a cura di un programma, l'**assemblatore** (*assembler*). Il risultato del processo di conversione (l'output dell'assemblatore) si chiama **file oggetto**. Molti programmi effettuano chiamate a subroutine assemblate precedentemente e riposte in librerie. L'esecuzione di tali programmi richiede la combinazione del file oggetto appena assemblato e delle subroutine di libreria (a loro volta file oggetto) in un solo **file binario eseguibile** a opera del **linker** ("redattore di collegamenti"). La traduzione si può dire ultimata solo quando il linker ha terminato di costruire il file binario eseguibile a partire da uno o più file oggetto. A questo punto il sistema operativo può leggere il file binario eseguibile caricato in memoria e gestirne l'esecuzione.

Il primo compito dell'assemblatore è redigere una **tabella dei simboli**, che associa i nomi di variabili e costanti simboliche e le etichette ai numeri binari che rappresentano. Le costanti definite direttamente nel programma possono essere aggiunte alla tabella dei simboli senza ulteriori elaborazioni. Questo lavoro viene svolto nella prima passata.

D'altro canto, le etichette rappresentano indirizzi il cui valore non è immediatamente deducibile e, per calcolarlo, l'assemblatore scandisce il programma riga per riga in quella che è chiamata la **prima passata**. Durante questa fase, l'assemblatore aggiorna un **contatore di locazioni** indicato con il simbolo "." che si pronuncia **dot** ("punto"). Ogni volta che si incontra un'istruzione o un'allocazione di memoria, il contatore di locazioni viene incrementato della dimensione di memoria necessaria a contenere l'elemento incontrato. Perciò, se le prime due istruzioni hanno dimensioni di 2 e 3 byte, un'etichetta che si antepone alla terza istruzione varrà 5.

Per esempio, se un programma inizia con le istruzioni seguenti

```
MOV AX,6
MOV BX,500
L:
```

allora il valore di *L* sarà 5.

All'inizio della **seconda passata** è noto il valore numerico di ogni simbolo. Dato che i nomi mnemonici delle istruzioni hanno valori numerici costanti, è ora possibile cominciare la **generazione del codice**. Le istruzioni sono lette di nuovo una per volta e il loro equivalente binario viene scritto nel file oggetto. Una volta assemblata l'ultima istruzione del programma, il file oggetto è completo.

C.5.2 as88, un assemblatore basato su ACK

Questo paragrafo descrive i dettagli dell'assemblatore/linker *as88*, disponibile nel CD-ROM allegato e presso il nostro sito web. *as88* è in grado di funzionare in associazione con il tracer già presentato, fa parte dell'Amsterdam Compiler Kit (ACK) ed è stato scritto sul modello degli assemblatori UNIX piuttosto che su quello degli assemblatori per MS-DOS o Windows. Questo assemblatore usa il punto esclamativo (!) per i commenti: il contenuto di una parte di riga che segue un punto esclamativo costituisce un commento e non ha alcun effetto sulla produzione del file oggetto. Eventuali righe vuote sono trattate alla stregua dei commenti.

Per la memorizzazione del codice e dei dati dopo la traduzione l'assemblatore usa tre sezioni diverse. Le sezioni sono in relazione con i segmenti di memoria della macchina. La prima è la **sezione di TESTO** per le istruzioni del processore; il segmento dati ospita la **sezione DATI** per l'inizializzazione di memoria necessaria all'avvio del processo; l'ultima sezione si chiama **BSS** (*Block Started by Symbol*), che appartiene ancora al segmento dati ed è destinata all'allocazione della memoria non inizializzata (cioè inizializzata a 0). Ognuna di queste sezioni ha il proprio contatore di locazioni. Lo scopo delle sezioni è permettere all'assemblatore di generare porzioni d'istruzioni e porzioni di dati in modo indipendente, lasciando al linker il compito di riorganizzarle in modo che le istruzioni si trovino tutte nel segmento di testo e i dati nel segmento dati. Ogni riga del codice assemblativo viene tradotta in una sola sezione, ma le righe di codice e le righe di dati possono trovarsi frammiste le une alle altre. Durante l'esecuzione, la sezione di TESTO si trova allocata nel segmento di testo e le sezioni DATI e BSS sono memorizzate (consecutivamente) nel segmento dati.

Le istruzioni o le parole di dati dei programmi in linguaggio assemblativo possono essere precedute da un'etichetta, che può anche occupare un'intera riga (nel qual caso si intende faccia parte dell'istruzione o della parola dati della riga successiva). Per esempio in

```
    CMP AX,ABC
    JE L
    MOV AX,XYZ
L:
```

l'etichetta *L* si riferisce all'istruzione o alla parola di dati che segue. Ci sono due tipi di etichette. Quelle **globali** sono identificatori alfanumerici seguite dai due punti (:), e quelle **locali** possono trovarsi solo nella sezione di TESTO e sono costituite da un'unica cifra seguita dal carattere due punti (:). Le etichette globali devono essere uniche e non possono corrispondere ad alcuna parola chiave del linguaggio, né a un nome mnemonico di un'istruzione. Le etichette locali possono invece occorrere più volte. L'istruzione

```
JE 2f
```

specifica che il salto JE (*Jump Equal*) è in avanti (*f* sta per *forward*) verso la successiva etichetta 2. Analogamente

```
JNE 4b
```

specifica che il salto JNE (*Jump Not Equal*) è all'indietro (*b* sta per *backward*) verso la precedente etichetta *4* più vicina.

L'assemblatore permette l'attribuzione di nomi simbolici alle costanti mediante la sintassi

```
identificatore = espressione
```

dove l'identificatore è una stringa alfanumerica come in

```
BLOCKSIZE = 1024
```

Tutti gli identificatori del linguaggio sono definiti dai loro primi otto caratteri: *BLOCKSIZE* e *BLOCKSIZZ* sono lo stesso simbolo, cioè *BLOCKSIZ*. Le espressioni si costruiscono a partire dalle costanti, dai valori numerici e dagli operatori. Le etichette sono considerate costanti perché il loro valore numerico è stabilito al termine della prima passata.

I valori numerici possono essere **ottali** (cominciano per 0), **decimali** o **esadecimali** (cominciano per 0x o 0X). I numeri esadecimali usano le lettere dalla "a" alla "f" (o dalla "A" alla "F") per rappresentare i valori da 10 a 15. Gli operatori interi sono +, –, *, / e %, rispettivamente per addizione, sottrazione, moltiplicazione, divisione e resto. Gli operatori logici sono &, ^ e ~ per le operazioni bit a bit di AND, OR e NOT. Le espressioni possono contenere parentesi quadre, ma non tonde per non creare confusione con le modalità d'indirizzamento.

L'uso delle etichette nelle espressioni richiede cautela. Le etichette d'istruzioni non possono essere sottratte dalle etichette di dati. La differenza tra etichette confrontabili è un valore numerico, ma né le etichette né le loro differenze possono essere usate come costanti all'interno d'espressioni moltiplicative o logiche. Le espressioni consentite nelle definizioni di costanti possono essere utilizzate anche come costanti nelle istruzioni del processore. Alcuni assemblatori mettono a disposizione il costrutto delle macroistruzioni che raggruppano sotto un unico nome un insieme d'istruzioni, ma *as88* non dispone di questa caratteristica.

Ogni linguaggio assemblativo definisce delle direttive che influiscono sul processo di assemblaggio, ma che non sono tradotte in codice binario e si dicono perciò **pseudoistruzioni**. La Figura C.8 elenca le pseudoistruzioni di *as88*.

Istruzione	Descrizione
.SECT .TEXT	Assembla le linee seguenti nella sezione di TESTO
.SECT .DATA	Assembla le linee seguenti nella sezione DATI
.SECT .BSS	Assembla le linee seguenti nella sezione BSS
.BYTE	Assembla gli argomenti come una sequenza di byte
.WORD	Assembla gli argomenti come una sequenza di parole
.LONG	Assembla gli argomenti come una sequenza di long
.ASCII "str"	Salva la stringa str in ASCII senza farla terminare per zero
.ASCIZ "str"	Salva la stringa str in ASCII facendola terminare per zero
.SPACE n	Incrementa il contatore di locazioni di n posizioni
.ALIGN n	Allinea il contatore di locazioni alla locazione che dista al più n byte
.EXTERN	L'identificatore è un nome esterno

Figura C.8 Le pseudoistruzioni di *as88*.

Il primo gruppo specifica all'assemblatore la sezione in cui vanno elaborate le istruzioni successive. In genere queste indicazioni si scrivono su di una riga separata e possono essere inserite in qualsiasi punto del codice. Per ragioni di carattere implementativo bisogna usare per prima la sezione di TESTO, poi la sezione di DATI e infine la sezione BSS. Una volta specificati questi riferimenti iniziali, le sezioni possono essere usate in un ordine qualsiasi. Inoltre, la prima riga di una sezione dovrebbe cominciare sempre con un'etichetta globale. Non ci sono altri vincoli sull'ordine d'uso delle sezioni.

Il secondo gruppo di pseudoistruzioni contiene le dichiarazioni dei tipi di dati per il segmento dati: `.BYTE`, `.WORD`, `.LONG` e stringa. La parola chiave dei primi tre tipi può essere preceduta da un'etichetta facoltativa, seguita poi da una lista d'espressioni costanti separate da virgole. Le stringhe mettono a disposizione due parole chiave, `ASCII` e `ASCIZ`, la cui unica differenza è che la seconda aggiunge in fondo alla stringa un byte di valore zero. Entrambe sono seguite da una stringa tra virgolette. La definizione delle stringhe consente l'uso di molte sequenze di *escape* tra cui menzioniamo quelle elencate nella Figura C.9. Oltre a ciò, è possibile inserire qualsiasi carattere con un carattere barra seguito dalla sua rappresentazione ottale, come \377 (si usano al massimo tre cifre e in questo caso non c'è bisogno di anteporre lo 0).

La pseudoistruzione `SPACE` causa l'incremento del puntatore alle locazioni del numero di byte specificato come argomento. Viene usata spesso dopo un'etichetta del segmento BSS per riservare lo spazio per l'allocazione di una variabile. La parola chiave `ALIGN` serve a far avanzare il puntatore in corrispondenza della prima locazione di memoria che dista 2, 4 o 8 byte dalla locazione corrente per facilitare l'assemblaggio di parole, long, e altro, allocando una quantità di memoria adeguata. Infine, la parola chiave `EXTERN` serve a dichiarare routine o locazioni di memoria come esterne, perché il linker le possa utilizzare per riferimenti esterni. Non è necessario che la loro definizione si trovi nel file corrente; può trovarsi ovunque, basta che sia raggiungibile dal linker.

Sequenza di escape	Descrizione
\n	Invio a capo (line feed)
\t	Tab
\\	Backslash
\b	Backspace
\f	Avanzamento modulo (form feed)
\r	Ritorno a margine (carriage return)
\"	Virgolette

Figura C.9 Alcune sequenze di *escape* di *as88*.

Benché l'assemblatore sia di per sé uno strumento abbastanza generale, bisogna segnalare alcuni aspetti da tener in conto quando lo si usa con il tracer. L'assemblatore riconosce le parole chiave scritte sia in minuscolo sia in maiuscolo, ma il tracer le visualizza sempre in maiuscolo. L'assemblatore accetta i caratteri "\r" e "\n" come caratteri di invio a capo (fine riga), ma il tracer usa sempre il secondo. Inoltre, anche se l'assemblatore può gestire programmi suddivisi in più file, il tracer richiede invece che il programma sia contenuto in un unico file con estensione ".$". Al suo interno è possibile specificare direttive per includere altri file con il comando

```
#include filename
```

Così facendo il file richiesto viene ricopiato all'interno del file ".$" a partire dalla posizione del comando d'inclusione. L'assemblatore verifica che il file da includere sia già stato elaborato e ne carica una copia, il che è particolarmente utile nei casi in cui diversi file usino uno stesso file d'intestazione. Il comando *#include* deve trovarsi all'inizio di una riga, senza essere preceduto da spazi, e il percorso del file deve essere scritto tra virgolette.

Se c'è un unico file sorgente, *pr.s*, allora il progetto si chiama *pr*, e il file che combina i diversi sorgenti (in questo caso uno solo) sarà *pr.$*. Se c'è più di un file sorgente, allora il nome del progetto è il nome del primo file e funge anche da nome del file ".$", generato dall'assemblatore concatenando il contenuto dei file sorgenti. Questo comportamento può essere modificato da linea di comando indicando l'opzione "–o nomeprogetto" prima del primo file sorgente, nel qual caso il file dei sorgenti combinati si chiamerà *nomeprogetto.$*.

L'inclusione dei file e la presenza di più file sorgenti comporta anche alcuni svantaggi. Innanzitutto è necessario che non ci siano nomi di etichette, di variabili e di costanti presenti in file diversi. Inoltre, dato che il file elaborato dall'assemblatore è in definitiva *nome-progetto.$*, tutti gli eventuali avvisi (*warning*) e messaggi d'errore si riferiscono alle sue righe di codice e non ai singoli file sorgenti. Per progetti molto piccoli, è spesso preferibile inserire tutto il programma in un unico file ed evitare di usare *#include*.

C.5.3 Differenze tra assemblatori dell'8088

L'assemblatore *as88* è modellato sugli assemblatori standard di UNIX e, come tale, differisce dagli altri assemblatori per l'8088: MASM (di Microsoft) e TASM (di Borland), progettati per MS-DOS (e alcuni aspetti di ogni assemblatore sono intimamente legati al sistema operativo). MASM e TASM supportano entrambi tutti i modelli di memoria dell'8088 consentiti in MS-DOS. Per esempio supportano il modello **minuscolo** (*tiny*) di memoria, secondo cui tutto il codice e i tutti dati devono essere contenuti in 64 KB, il modello **piccolo** (*small*), per cui il segmento di codice e il segmento dati possono raggiungere ciascuno i 64 KB, e i modelli **grandi** (*large*), che ammettono molteplici segmenti per il codice e per i dati. La differenza tra questi modelli dipende dall'uso che si fa dei registri di segmento. Il modello grande consente le chiamate a distanza e le modifiche del registro `DS`. Lo stesso processore introduce alcune limitazioni sui registri di segmento (per esempio il registro `CS` non può essere la destinazione di un'istruzione `MOV`). Per semplificare l'attività del tracer, *as88* usa un modello di memoria che somiglia al modello piccolo, anche se, quando è usato senza il tracer, può gestire i registri di segmento senza ulteriori restrizioni.

Questi altri assemblatori non hanno una sezione `.BSS` e inizializzano la memoria solo all'interno delle sezioni DATI. Il file in ingresso all'assemblatore comincia in genere con delle informazioni d'intestazione seguite dalla sezione DATI, indicata con la parola chiave `.data`, seguita a sua volta dal testo del programma dopo la parola chiave `.code`. L'intestazione usa la parola chiave `title` per specificare il nome del programma, la parola chiave `.model` per indicare il modello di memoria e la parola chiave `.stack` per allocare la memoria del segmento di stack. Se il file binario ha estensione *.com*, allora si usa il modello minuscolo di memoria, si considerano i registri come fossero tutti uguali e si riservano i primi 256 byte del segmento condiviso come "Prefisso del Segmento di Programma".

Invece delle direttive `.WORD`, `.BYTE` e `ASCIZ`, questi assemblatori usano la parola chiave `DW` per le parole e `DB` per i byte. Dopo la direttiva `DB` è possibile definire una stringa racchiudendola tra virgolette. Le etichette per la dichiarazione di dati non sono seguite dai due punti. La parola chiave `DUP` serve a inizializzare grandi porzioni di memoria; è preceduta da un numero ed è seguita da un'inizializzazione, come nell'istruzione

```
LABEL DB 1000 DUP (0)
```

che inizializza i 1000 byte di memoria che seguono l'etichetta *LABEL* assegnando loro il byte ASCII zero.

Anche le etichette per le subroutine non sono seguite dai due punti, ma dalla parola chiave `PROC`. L'etichetta è ripetuta anche alla fine della subroutine ed è seguita dalla parola chiave `ENDP`, così l'assemblatore può dedurre l'estensione della subroutine. Non sono supportate le etichette locali.

Le parole chiave delle istruzioni sono identiche per i tre assemblatori MASM, TASM e *as88*. La sorgente segue sempre la destinazione in tutte le istruzioni a due operandi. È pratica comune l'utilizzo dei registri per il passaggio di argomenti alle funzioni invece di impilarli sullo stack. Tuttavia, se le routine assemblate vengono eseguite all'interno di programmi C o C++, è consigliabile usare lo stack per conformarsi al meccanismo di

chiamata delle subroutine C. Questa non è una reale differenza, perché anche in *as88* è possibile usare i registri invece dello stack per il passaggio degli argomenti.

La differenza maggiore tra questi tre assemblatori sta nelle chiamate di sistema. In MASM e in TASM, queste vengono effettuate tramite un interrupt di sistema `INT`. L'interrupt più comune è `INT 21H` che serve alla chiamata di funzioni MS-DOS. Il numero di chiamata è inserito in `AX` e ancora una volta si usa un registro per il passaggio degli argomenti. Esistono vettori di interrupt e numeri di interrupt diversi per i diversi dispositivi; per esempio `INT 16H` serve a chiamare le funzioni del BIOS per la tastiera, mentre `INT 10H` riguarda lo schermo. La programmazione di queste funzioni richiede al programmatore la conoscenza di un gran numero d'informazioni che variano da dispositivo a dispositivo. Le chiamate di sistema di UNIX messe a disposizione da *as88* sono invece molto più facili da usare.

C.6 Tracer

Il tracer-debugger è stato pensato per l'esecuzione su un normale terminale con schermo a 24 × 80 caratteri (VT100) e comandi ANSI standard per i terminali. Se lo si vuole eseguire su macchine UNIX o Linux allora si può usare l'emulatore di terminale messo a disposizione dal sistema X-window. Sulle macchine Windows bisogna caricare il driver *ansi.sys* all'interno dei file d'inizializzazione del sistema nel modo che descriviamo in seguito. La Figura C.10 mostra la suddivisione della schermata del tracer in sette finestre.

Figura C.10 Finestre del tracer.

La finestra in alto a sinistra è la finestra del processore; mostra il contenuto dei registri d'uso generale in notazione decimale, mentre gli altri sono in notazione esadecimale. Poiché il valore numerico del program counter non è di per sé molto informativo, la posizione del sorgente di programma correntemente in esecuzione viene indicata in base alla precedente etichetta globale. Al di sopra del program counter sono elencati i valori di cinque codici di condizione: il simbolo "v" indica un evento di overflow (flag O), mentre le modalità d'incremento o di decremento sono indicate con i simboli ">" e "<" (flag D). Il flag di segno può valere "n" o "p" a seconda che il risultato dell'istruzione sia negativo oppure no. Il flag di zero vale "z" se asserito, mentre il bit di riporto vale "c" quando asserito. Il simbolo "–" indica un flag azzerato.

La finestra in alto al centro visualizza lo stack, rappresentato in esadecimale. La posizione del puntatore allo stack è indicata dalla freccia "=>". L'indirizzo di ritorno delle subroutine è indicato mediante una cifra anteposta al valore esadecimale. La finestra in alto a destra mostra una parte del file sorgente che contiene l'istruzione dell'esecuzione successiva. La freccia "=>" in questa finestra indica la posizione del program counter.

La finestra sotto quella del processore mostra le posizioni del codice contenenti le chiamate di subroutine effettuate più di recente. Appena sotto c'è la finestra dei comandi del tracer in cui viene visualizzata la storia dei comandi impartiti precedentemente, seguita dal cursore. Si noti che ogni comando deve essere seguito dalla pressione del tasto Invio.

La finestra in basso può contenere sei elementi della memoria dei dati globali. Ogni elemento comincia specificando una posizione relativa a un'etichetta, seguita dalla posizione assoluta nel segmento dati, da due punti e quindi da otto byte in esadecimale. Le 11 posizioni successive sono riservate a caratteri, seguiti dalla rappresentazione decimale di quattro parole. I byte, i caratteri e le parole rappresentano tutti lo stesso contenuto di memoria, anche se i caratteri contengono tre byte in più perché all'inizio non è noto se i dati saranno usati come interi con o senza segno, o come stringa.

La prima riga della finestra per l'input/output è riservata all'output per i messaggi d'errore del tracer, la seconda all'input e le righe successive all'output. L'output per gli errori è indicato con la lettera "E", l'input con la lettera "I" e lo standard output con il simbolo ">". La riga di input contiene una freccia "–>" a indicare il puntatore alla posizione di prossima lettura. Se il programma chiama *read* o *getchar*, l'input va immesso nell'area di input e bisogna farlo seguire dalla pressione del tasto Invio. La freccia "–>" indica la parte della riga che non è stata ancora elaborata.

In genere il tracer usa lo standard input per ricevere comandi e dati, ma è possibile anche preparare un file con i comandi per il tracer e uno con le righe di input che si vogliono far leggere prima che il controllo torni allo standard input. I file per i comandi al tracer hanno estensione *.t*, e quelli per l'input di dati hanno estensione *.i*. Il linguaggio assemblativo ammette l'uso indistinto di caratteri minuscoli o maiuscoli per la scrittura delle parole chiave, delle subroutine di sistema e delle pseudoistruzioni. Durante il processo di assemblaggio viene creato il file d'estensione *.$* e, al proprio interno, le parole chiave sono convertite in maiuscolo e vengono eliminati i caratteri di invio a capo. Dunque, per un dato progetto *pr* possono esistere sei file diversi:

1. *pr.s* per il codice del sorgente assemblativo;
2. *pr.$* per il file dei sorgenti combinati;

3. *pr.88* per il file di caricamento;
4. *pr.i* per lo standard input preparato anticipatamente;
5. *pr.t* per i comandi all'interprete preparati anticipatamente;
6. *pr.#* per collegare il codice assemblativo al file di caricamento.

L'ultimo file viene usato dal tracer per riempire la finestra in alto a destra e visualizzare il program counter. Infine, il tracer controlla che il file di caricamento sia stato creato successivamente all'ultima modifica apportata al sorgente di programma, altrimenti emette un avviso al riguardo.

C.6.1 Comandi del tracer

La Figura C.11 elenca i comandi del tracer. Il più importante è indicato nella prima riga ed è il comando che si immette con la semplice pressione del tasto Invio per richiedere l'esecuzione di una sola istruzione da parte del processore. Altrettanto importante è il comando *q* di uscita (*quit*), indicato all'ultima riga della tabella. Se si immette come comando un numero, questo provoca l'esecuzione di un numero corrispondente d'istruzioni: il numero *k* equivale a premere il tasto Invio *k* volte. Si ottiene lo stesso effetto se il numero è seguito da un punto esclamativo, *!*, o da una *X*.

Il comando *g* può essere usato per spostarsi a una certa riga del file sorgente. In particolare, se preceduto da un numero di riga, il tracer esegue tutte le istruzioni che la precedono; se si usa l'etichetta */T*, con o senza *+#*, il numero di riga dove fermarsi, si calcola a partire dall'etichetta d'istruzione *T*; l'immissione di *g* senza alcun'altra indicazione fa sì che il tracer esegua altri comandi finché non raggiunga di nuovo il numero di riga corrente.

Il comando */label* cambia significato a seconda che lo si usi per etichette di dati o d'istruzioni. Nel primo caso viene aggiunta o sostituita una riga della finestra più in basso con l'insieme di dati che si trova presso quell'etichetta. Nel secondo caso, il comando si comporta come il comando *g*. L'etichetta può essere seguita da un segno + o da un numero (indicato dal simbolo # nella Figura C.11) per specificare un offset relativo all'etichetta.

È possibile impostare un **breakpoint** ("punto d'interruzione") in corrispondenza di un'istruzione tramite il comando *b*, che può essere preceduto da un'etichetta d'istruzione ed eventualmente da un offset. Se durante l'esecuzione si raggiunge un'istruzione con breakpoint, il tracer si ferma; per farlo ripartire basta premere il tasto Invio o un comando d'esecuzione. Se non vengono specificati alcun numero e alcuna etichetta, il breakpoint è associato alla riga corrente. Per rimuovere un breakpoint si usa il comando *c*, preceduto eventualmente dagli stessi parametri opzionali di *b* (etichetta e numero). Esiste il comando d'esecuzione *r* che ordina al tracer di continuare l'esecuzione finché non incontri un breakpoint, una chiamata di uscita o la fine del programma.

Il tracer tiene anche traccia del livello di subroutine in cui sta girando il programma. Si può desumere dalla finestra sotto la finestra del processore o anche dai numeri indicati nella finestra dello stack. Ci sono tre comandi che riguardano il livello di subroutine in esecuzione. Il comando – richiede al tracer di proseguire nell'esecuzione finché non raggiunga un livello di subroutine inferiore rispetto a quello attuale. L'effetto di questo comando è l'esecuzione delle istruzioni che mancano al completamento della subroutine correntemente in esecuzione. Al contrario il comando + provoca il proseguimento dell'esecuzione fino al livello di subroutine appena superiore. Il comando = fa proseguire l'esecuzione finché non si incontra lo stesso livello di quello corrente e può essere usato per eseguire una sola subroutine quando ci si trova all'istruzione `CALL`. Quando si usa = la finestra del tracer non mostra i dettagli della subroutine. Si tratta di un comando particolarmente utile in presenza d'istruzioni `LOOP`; infatti l'esecuzione si ferma esattamente alla fine di ogni ciclo.

Indirizzo	Comando	Esempio	Descrizione
			Esegue una istruzione
#	, ! , X	24	Esegue # istruzioni
/T+#	g , ! ,	/start+5g	Continua fino alla riga # dopo etichetta T
/T+#	b	/start+5b	Inserisce breakpoint alla riga # dopo etichetta T
/T+#	c	/start+5c	Rimuove breakpoint alla riga # dopo etichetta T
#	g	108g	Esegue il programma fino alla riga #
	g	g	Esegue il programma finché non torna alla riga corrente
	b	b	Inserisce breakpoint alla riga corrente
	c	c	Rimuove breakpoint dalla riga corrente
	n	n	Esegue programma fino alla riga successiva
	r	r	Esegue programma fino a un breakpoint o alla fine
	\&=	\&=	Esegue programma fino allo stesso livello di subroutine
	-	-	Esegue programma fino al livello di subroutine corrente meno 1
	+	+	Esegue programma fino al livello di subroutine corrente più 1
/D+#		/buf+6	Mostra il segmento dati alla posizione etichetta+#
/D+#	d , !	/buf+6d	Mostra il segmento dati alla posizione etichetta+#
	R , CTRL L	R	Aggiorna le finestre
	q	q	Interrompe il tracing, restituisce il controllo all'interprete di comandi

Figura C.11 Comandi del tracer. Ogni comando deve essere seguito da un invio a capo (tasto Enter o Invio). Una casella vuota indica la necessità di digitare il solo carattere di a capo. I comandi che non devono specificare alcun indirizzo hanno la casella della colonna *Indirizzi* vuota. Il simbolo # indica un offset intero.

C.7 Installazione

In questo paragrafo spieghiamo come usare gli strumenti software disponibili. Per prima cosa è necessario individuare il software corrispondente alla propria piattaforma hardware (qui mettiamo a disposizione versioni già compilate per Solaris, UNIX, Linux e

Windows). Il software è reperibile nel CD-ROM allegato. Le directory principali del CD-ROM sono *Bigendnx*, *LtlendNx* e *MSWindos*, e ciascuna contiene una directory *assembler* con il materiale. Le tre cartelle radice sono destinate rispettivamente ai sistemi UNIX big endian (per esempio le workstation di Sun), ai sistemi UNIX little endian (per esempio Linux per PC) e ai sistemi Windows.

Dopo averla decompressa o copiata, la directory *assembler* dovrebbe contenere i seguenti file e sottodirectory: *READ_ME*, *bin*, *as_src*, *trce_src*, *examples* e *exercise*. I sorgenti precompilati si trovano nella directory *bin*, ma sono presenti anche nella directory *examples* per comodità.

Per farsi un'idea del funzionamento del sistema, basta portarsi nella directory *examples* ed eseguire

```
t88 HlloWrld
```

Questo comando corrisponde al primo esempio del Paragrafo C.8.

La directory *as_src* contiene il codice sorgente dell'assemblatore. I file sorgenti sono scritti in C e per ricompilarli si usa il comando *make*. Nella directory dei sorgenti è presente anche il file *Makefile* che serve per la compilazione sulle piattaforme POSIX compatibili. Sulle piattaforme Windows la compilazione è svolta dal file batch *make.bat*. Dopo la compilazione potrebbe essere necessario copiare i file eseguibili in un'altra directory, o modificare la variabile d'ambiente PATH, affinché l'assemblatore *as88* e il tracer *t88* siano visibili dalle directory contenenti i sorgenti in linguaggio assemblativo. In alternativa, invece di digitare solo *t88* è possibile specificare l'intero percorso dell'eseguibile.

Sui sistemi Windows 2000 e XP è necessario installare il driver di terminale *ansi.sys* inserendo la riga

```
device=%systemRoot%\System32\ansi.sys
```

nel file di configurazione *config.nt*, che si trova al percorso

```
\winnt\system32\config.nt (Windows 2000)
\windows\system32\config.nt (Windows XP)
```

Sui sistemi Windows 95/98/ME il driver va inserito nel file *config.sys*. Nei sistemi UNIX e Linux il driver è in genere già installato.

C.8 Esempi

Nei paragrafi da C.2 a C.4 abbiamo presentato il processore 8088, la sua memoria e le sue istruzioni. Il Paragrafo C.5 ha introdotto il linguaggio assemblativo di *as88* che usiamo in questo testo. Nei Paragrafi C.6 e C.7 abbiamo studiato il tracer e descritto l'installazione degli strumenti software. In teoria queste informazioni dovrebbero già essere sufficienti per la scrittura e il debug di programmi assemblativi con gli strumenti forniti. Ciononostante, a molti lettori potrebbe giovare la presentazione dettagliata di alcuni esempi di programmi assemblativi e il loro debug attraverso il tracer. Perciò qui

presentiamo i programmi d'esempio che abbiamo inserito nella directory *examples* del materiale di supporto. Suggeriamo al lettore di assemblarli tutti e di visualizzare la loro esecuzione con il tracer.

C.8.1 Esempio Hello World

Cominciamo con l'esempio *HlloWrld.s* della Figura C.12. Il programma è mostrato nella finestra di sinistra e i numeri di riga sono indicati dopo il punto esclamativo (!), il simbolo usato dall'assemblatore per i commenti. Le prime tre righe contengono le definizioni di costanti per associare la rappresentazione interna di due chiamate di sistema e di un file di output ai loro nomi convenzionali.

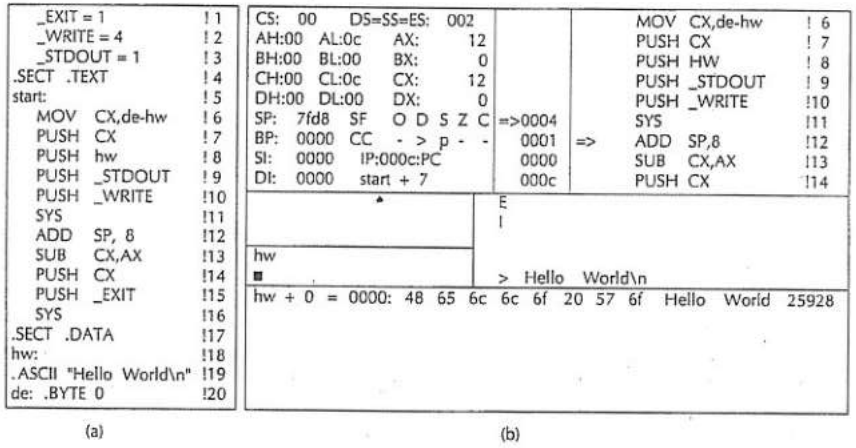


Figura C.12 (a) Programma HlloWrld.s. (b) Finestre del tracer durante la sua esecuzione.

La pseudoistruzione `.SECT` della riga 4 specifica che le righe successive fanno parte della sezione di TESTO, ovvero che si tratta d'istruzioni del processore. Analogamente, la riga 17 indica che le righe sottostanti costituiscono dati. La riga 19 inizializza una stringa di dati di 12 byte, compresi lo spazio e il carattere di fine riga (\n).

Le righe 5, 18 e 20 contengono etichette, individuabili dalla presenza dei due punti. Queste etichette rappresentano valori numerici e in questo sono simili a costanti; tuttavia, il valore numerico non è dato, ma deve essere calcolato dall'assemblatore. L'etichetta *start* si trova all'inizio della sezione di TESTO e quindi vale 0, ma il valore di un'eventuale etichetta successiva nella sezione di TESTO (nell'esempio non ce ne sono) dipenderebbe dal numero di byte che la precedono. Si consideri ora la riga 6 che termina con la differenza di due etichette, cioè con un valore costante. In effetti la riga è equivalente all'istruzione

```
MOV CX,12
```

con la differenza che, nel primo caso, il calcolo della lunghezza della stringa è lasciato all'assemblatore e non al programmatore. Il valore indicato dalla differenza corrisponde alla quantità di spazio riservato alla stringa della riga 19 nella sezione dati. L'istruzione MOV della riga 6 richiede la copia di *de –hw* in CX.

Le righe da 7 a 11 mostrano il funzionamento dell'invocazione di una chiamata di sistema. Le cinque righe sono la traduzione in codice assemblativo della chiamata di funzione C

```
write(1, hw, 12);
```

il cui primo parametro è il descrittore di file per lo standard output (1), il secondo è l'indirizzo della stringa da stampare (*hw*) e il terzo è la lunghezza della stringa (12). Le righe dalla 7 alla 9 impilano questi parametri sullo stack in ordine inverso, secondo la convenzione del C e quella adottata dal tracer. La riga 10 impila sullo stack il numero della chiamata di sistema write (4) e la 11 effettua la chiamata vera e propria. Questa sequenza di chiamata ricalca il funzionamento dei programmi in linguaggio assemblativo scritti per i sistemi UNIX (o Linux), ma per eseguirla direttamente su altri sistemi operativi dovrebbe essere leggermente modificata per aderire alle convenzioni di quei sistemi. Tuttavia, l'assemblatore *as88* e il tracer *t88* usano le convenzioni di chiamata di UNIX anche se sono eseguiti su sistemi Windows.

La chiamata di sistema della riga 11 esegue la scrittura vera e propria. La riga 12 ripristina lo stack alla posizione precedente alla chiamata, spostando il puntatore allo stack indietro di quattro parole di 2 byte ciascuna. Se l'operazione di scrittura ha successo, il numero di byte scritti viene restituito nel registro AX. La riga 13 sottrae il risultato della chiamata di sistema della riga 11 al valore del registro CX, contenente la lunghezza della stringa originale, per stabilire se la chiamata ha avuto successo, ossia per stabilire se sono stati scritti tutti i byte. Di conseguenza, lo stato di uscita del programma sarà 0 soltanto in caso di successo. Le righe 14 e 15 preparano l'esecuzione della chiamata di sistema exit (riga 16) impilando sullo stack lo stato d'uscita e il codice della funzione EXIT.

Le istruzioni MOV e SUBusano il primo operando come destinazione e il secondo come sorgente. Questa è la convenzione adottata dal nostro assemblatore, altri potrebbero usare l'ordine inverso; non c'è alcuna ragione particolare per preferire un ordinamento a un altro.

Proviamo ad assemblare e a eseguire *HlloWrld.s*. Forniamo le indicazioni sui procedimenti da seguire sulle piattaforme UNIX e Windows; le indicazioni per UNIX dovrebbero valere anche per Linux, Solaris, MacOS X e per altre varianti di UNIX. Si comincia con l'aprire una finestra dell'interprete di comandi (la shell). Su Windows bisogna cliccare in sequenza su

```
Start > Programmi > Accessori > Prompt dei comandi
```

Quindi ci si porta nella directory *examples* con il comando *cd* (*Change Directory*). L'argomento di questo comando dipende dalla locazione del sistema in cui è stato copiato il materiale di supporto. Una volta entrati nella directory si può verificare la presenza dei file binari dell'assemblatore e del tracer con il comando UNIX *ls* o con il comando

Windows *dir*. I file binari si chiamano rispettivamente *as88* e *t88*; sui sistemi Windows hanno estensione *.exe*, ma non c'è bisogno di specificare l'estensione per lanciare la loro esecuzione. Se i file suddetti non si trovano nella directory, bisogna trovarli e copiarli al suo interno.

È ora possibile passare all'assemblaggio digitando il comando

```
as88 HlloWrld.s
```

Se il comando produce un errore nonostante l'assemblatore sia presente nella cartella, si può provare a digitare

```
./as88 HlloWrld.s
```

nei sistemi UNIX o

```
.\as88 HlloWrld.s
```

nei sistemi Windows. Se l'assemblaggio termina correttamente, verranno visualizzati i messaggi:

```
Project HlloWrld listfile HlloWrld.S
Project HlloWrld num file HlloWrld.#
Project HlloWrld loadfile HlloWrld.88.
```

che indicano la creazione dei file corrispondenti. Se non ci sono messaggi d'errore, si procede con l'invocazione del tracer:

```
t88 HlloWrld
```

che ne apre le finestre. La freccia nel riquadro in alto a destra punta ora sull'istruzione

```
MOV CX,de-hw
```

della riga 6. A questo punto si può digitare un "a capo" (il tasto Enter o Invio sulle tastiere dei PC) che provoca l'esecuzione di un'istruzione, per cui l'istruzione ora puntata è

```
PUSH CX
```

e CX contiene adesso il valore 12 (nella finestra di sinistra). Dopo un altro "a capo" la finestra centrale contiene il valore 000c, cioè 12 in esadecimale. Questa finestra mostra lo stack, che adesso contiene una parola di valore 12. Se si preme Invio ancora tre volte si eseguono le tre PUSH delle righe 8, 9 e 10, dopodiché lo stack conterrà quattro elementi e il program counter varrà 000b (indicato nella finestra di sinistra).

Dopo un altro "a capo" viene eseguita la chiamata di sistema che produce la stampa della stringa "Hello World\n" nella finestra in basso a destra. Adesso SP vale 0x7ff0, ma dopo un altro "a capo" viene incrementato di 8 e diventa 0x7ff8. Dopo altri quattro invii a capo, la chiamata di sistema viene completata e l'esecuzione del tracer si interrompe.

Per comprendere a fondo il funzionamento dell'assemblaggio può essere utile aprire il file *hlloWrld.s* con un editor. A tal fine conviene evitare di usare programmi come *Word* perché potrebbero formattare il sorgente in modo scorretto. Sulle macchine UNIX

si possono usare per esempio *ex*, *vi*, o *emacs*; sui sistemi Windows si può aprire *Notepad*, un editor molto semplice reperibile cliccando in sequenza su

```
Start > Programmi > Accessori > Notepad
```

Consigliamo di modificare la stringa alla riga 19 per visualizzare un messaggio diverso. Dopo aver salvato il file, si può procedere al suo assemblaggio ed eseguirlo con il tracer. Questa semplice modifica è un primo approccio alla programmazione in linguaggio assemblativo.

C.8.2 Esempio con i registri d'uso generale

L'esempio che presentiamo ora mostra più in dettaglio l'uso dei registri ed evidenzia una delle insidie della moltiplicazione nell'8088. La parte sinistra della Figura C.13 mostra una porzione del programma *genReg.s*, mentre la parte destra raffigura la finestra dei registri del tracer in due momenti diversi dell'esecuzione del programma. La Figura C.13(b) mostra lo stato dei registri dopo l'esecuzione della riga 7. L'istruzione alla riga 4

```
MOV AX,258
```

carica il valore 258 in AX, che equivale a caricare 1 in AH e 2 in AL. Alla riga 5 si effettua la somma di AL e AH, dopodiché AH vale 3. Alla riga 6 viene copiato il contenuto della variabile *times* (10) in CX, alla riga 7 viene caricato l'indirizzo della variabile *muldat* (2) in BX; l'indirizzo di *muldat* è 2, perché questa si trova in corrispondenza del secondo byte della sezione DATI. Questa è la situazione fotografata nella Figura C.13(b): AH vale 3, AL vale 2 e AX vale 770, in quanto 3 × 256 + 2 = 770.

```
start:                  ! 3
    MOV  AX,258         ! 4
    ADDB AH,AL          ! 5
    MOV  CX,(times)     ! 6
    MOV  BX,muldat      ! 7
    MOV  AX,(BX)        ! 8
llp: MUL  2(BX)         ! 9
    LOOP llp            !10
.SECT .DATA             !11
times: .WORD 10         !12
muldat:.WORD 625,2      !13
```

(a)

```
CS: 00   DS=SS=ES: 002
AH:03 AL:02   AX:   770
BH:00 BL:02   BX:     2
CH:00 CL:0a   CX:    10
DH:00 DL:00   DX:     0
SP:  7fe0  SF   O D S Z C
BP:  0000  CC   - > p - -
SI:  0000   IP:0009:PC
DI:  0000   start + 4
```

(b)

```
CS: 00   DS=SS=ES: 002
AH:38 AL:80   AX: 14464
BH:00 BL:02   BX:     2
CH:00 CL:04   CX:     4
DH:00 DL:01   DX:     1
SP:  7fe0  SF   O D S Z C
BP:  0000  CC   v > p - c
SI:  0000   IP:0011:PC
DI:  0000   start + 7
```

(c)

Figura C.13 (a) Porzione del programma. (b) Finestra dei registri del tracer dopo l'esecuzione della riga 7. (c) I registri.

L'istruzione successiva (riga 8) copia il contenuto di *muldat* in AX. Dunque, dopo un ulteriore invio a capo, AX conterrà 625.

Siamo ora pronti per entrare in un ciclo che moltiplica il contenuto di AX alla parola che si trova all'indirizzo 2(BX), cioè all'indirizzo *muldat* + 2, che vale 2. La destinazione implicita dell'istruzione di moltiplicazione è la coppia di registri DX:AX. Dopo la prima iterazione del ciclo il risultato può essere contenuto in una sola parola, perciò AX

contiene il risultato (1250) e DX resta a 0. La Figura C.13(c) mostra il contenuto dei registri dopo 7 moltiplicazioni successive.

All'inizio del ciclo AX valeva 625, perciò il risultato delle sette moltiplicazioni per 2 è 80.000. Questo numero non può più essere contenuto in AX, bensì occupa ora i 32 bit dei registri DX:AX accoppiati. DX vale 1 e AX vale 14.464. L'istruzione LOOP decrementa il registro CX di un'unità a ogni iterazione. Poiché all'inizio dell'esecuzione CX valeva 10, dopo sette operazioni MUL (e dopo sole sei iterazioni dell'istruzione LOOP) vale 4.

Il problema si verifica al momento della moltiplicazione successiva. La moltiplicazione coinvolge AX ma non DX, perciò MUL moltiplica AX (14.464) per 2 producendo 28.928. Il risultato è inserito in AX e DX viene azzerato, il che è algebricamente errato.

C.8.3 Istruzioni di chiamata e puntatori ai registri

L'esempio successivo, *vecprod.s*, è un breve programma per il calcolo del prodotto scalare di due vettori, *vec1* e *vec2*. È mostrato nella Figura C.14.

La prima parte del programma prepara la chiamata di *vecmul* salvando SP in BP e impilando gli indirizzi di *vec2* e *vec1* sullo stack, di modo che *vecmul* possa accedervi. La riga 8 carica in CX la lunghezza in byte del vettore. La riga 9 fa scorrere questo valore di un bit a destra, così CX contiene ora la lunghezza del vettore in parole. Questo valore viene impilato sullo stack alla riga 10. La riga 11 effettua la chiamata a *vecmul*.

Ancora una volta vale la pena sottolineare come gli argomenti delle subroutine vengano impilati in ordine inverso per rispettare la convenzione vigente per le chiamate del linguaggio C. Così facendo, *vecmul* può essere invocata anche all'interno di un programma C con la chiamata

```
vecmul(count, vec1, vec2)
```

Durante l'esecuzione dell'istruzione CALL, l'indirizzo di ritorno viene impilato sullo stack e, usando il tracer, si evince che questo indirizzo è 0x0011.

La prima istruzione della subroutine è PUSH del puntatore base, BP, alla riga 22. BP viene salvato perché servirà a indirizzare gli argomenti e le variabili locali della subroutine. Successivamente viene copiato il puntatore allo stack (riga 23) di modo che il nuovo valore del puntatore base punti al suo vecchio valore.

A questo punto si è pronti per il caricamento nei registri degli argomenti e per l'allocazione dello spazio per le variabili locali. Le tre righe successive prelevano gli argomenti dallo stack e li salvano nei registri. Si ricordi che lo stack è orientato alla parola, perciò i suoi indirizzi dovrebbero essere sempre pari. L'indirizzo di ritorno si trova appena dopo il vecchio puntatore base, perciò viene indirizzato con 2(BP). Segue l'argomento *count*, indirizzato con 4(BP), che viene caricato in CX alla riga 24. Le righe 25 e 26 caricano *vec1* e *vec2* in SI e DI. Questa subroutine utilizza una variabile locale (inizializzata a 0) per salvare il risultato. A tal fine, la riga 27 esegue il push del valore 0.

La Figura C.15 mostra lo stato del processore appena prima del primo ingresso nel ciclo della riga 28. La finestra stretta che si trova in alto e al centro della figura (alla destra dei registri) mostra lo stack. Alla base dello stack c'è l'indirizzo di *vec2* (0x0022), seguito da quello di *vec1* (0x0018) e dal terzo argomento, cioè il numero d'elementi di ciascun vettore (0x0005).

```
_EXIT     = 1                    ! 1   definizione del valore di _EXIT
_PRINTF   = 127                  ! 2   definizione del valore di _PRINTF
.SECT .TEXT                      ! 3   inizio del segmento di TESTO
inpstart:                        ! 4   definizione dell'etichetta inpstart
        MOV  BP,SP               ! 5   salvataggio di SP in BP
        PUSH vec2                ! 6   push dell'indirizzo di vec2
        PUSH vec1                ! 7   push dell'indirizzo di vec1
        MOV CX,vec2-vec1         ! 8   CX = numero di byte nel vettore
        SHR  CX,1                ! 9   CX = numero di parole nel vettore
        PUSH CX                  ! 10  push del numero di parole (count)
        CALL vecmul              ! 11  chiamata di vecmul
        MOV  (inprod),AX         ! 12  copia di AX
        PUSH AX                  ! 13  push del risultato per la stampa
        PUSH pfmt                ! 14  push dell'indirizzo della stringa di formattazione
        PUSH _PRINTF             ! 15  push del codice di funzione di PRINTF
        SYS                      ! 16  chiamata della funzione PRINTF
        ADD  SP,12               ! 17  ripristino dello stack
        PUSH 0                   ! 18  push del codice di stato
        PUSH _EXIT               ! 19  push del codice di funzione di EXIT
        SYS                      ! 20  chiamata della funzione EXIT

vecmul:                          ! 21  inizio di vecmul(count, vec1, vec2)
        PUSH BP                  ! 22  salvataggio di BP nello stack
        MOV  BP,SP               ! 23  copia di SP in BP per l'accesso agli argomenti
        MOV  CX,4(BP)            ! 24  trasf. di count in CX per il controllo del ciclo
        MOV  SI,6(BP)            ! 25  SI = vec1
        MOV  DI,8(BP)            ! 26  DI = vec2
        PUSH 0                   ! 27  push di 0 sullo stack

1:      LODS                     ! 28  copia di (SI) in AX
        MUL  (DI)                ! 29  moltiplicazione di AX per (DI)
        ADD  -2(BP),AX           ! 30  somma di AX con valore accumulato in memoria
        ADD  DI,2                ! 31  incremento di DI per puntare a elemento successivo
        LOOP 1b                  ! 32  se CX > 0, vai all'etichetta 1b
        POP  AX                  ! 33  pop della cima dello stack in AX
        POP  BP                  ! 34  ripristino di BP
        RET                      ! 35  ritorno da subroutine

.SECT .DATA                      ! 36  inizio del segmento DATI
pfmt:. ASCIZ "Inner product is: %d\n" ! 37  definizione di stringa
.ALIGN 2                         ! 38  forza allineamento a indirizzo pari
vec1:.WORD 3,4,7,11,3            ! 39  vettore 1
vec2::WORD 2,6,3,1,0             ! 40  vettore 2
.SECT .BSS                       ! 41  inizio del segmento BSS
inprod:   .SPACE 2               ! 42  allocazione dello spazio per inprod
```

Figura C.14 Programma *vecprod.s*.

Poi c'è l'indirizzo di ritorno (0x0011); il numero 1 alla sua sinistra indica che si tratta di un indirizzo di ritorno di un livello inferiore rispetto al livello del programma principale. La stessa indicazione è presente anche nella finestra sotto i registri, ma questa volta l'indirizzo è indicato in maniera simbolica. Risalendo lo stack, sopra l'indirizzo di ritorno si trova il vecchio valore di BP (0x7fc0) e poi lo zero impilato alla riga 27. La freccia che punta a questo valore indica il valore puntato da SP. La finestra alla destra dello stack mostra un frammento del testo del programma, dove la freccia indica l'istruzione successiva da eseguire.


```
   MOV  BP,SP          ! 5   | CS: 00    DS=SS=ES: 004                |                    |        PUSH BP          ! 22
   PUSH vec2           ! 6   | AH:00  AL:00    AX:       0          |                    |        MOV  BP,SP       ! 23
   PUSH vec1           ! 7   | BH:00  BL:00    BX:       0          |                    |        MOV  CX,4(BP)    ! 24
   MOV  CX,vec2-vec1   ! 8   | CH:00  CL:05    CX:       5          | =>0000             |        MOV  SI,6(BP)    ! 25
   SHR  CX,1           ! 9   | DH:00  DL:00    DX:       0          |   7fc0             |        MOV  DI,8(BP)    ! 26
   PUSH CX             ! 10  | SP:  7fb4  SF  O D S Z C             | 1 0011             |        PUSH 0           ! 27
   CALL vecmul         ! 11  | BP:  7fb6  CC  - > p z -             |   0005     =>1:    | 1:     LODS             ! 28
   ---------                 | SI:  0018    IP:0031:PC              |   0018             |        MUL  (DI)        ! 29
vecmul :               ! 21  | DI:  0022    vecmul+7                |   0022             |        ADD  -2(BP),AX   ! 30
   PUSH BP             ! 22  |                                      |                    |
   MOV  BP,SP          ! 23  | 1 <= inpstart + 7                    |                    |
   MOV  CX,4(BP)       ! 24  |                                      |                    |
   MOV  SI,6(BP)       ! 25  | =                                    |                    |
   MOV  DI,8(BP)       ! 26  | vec1 + 0  =0018:  3  0  4  0  7  0  b  0  ..........  3
   PUSH 0              ! 27  | vec2 + 0  =0022:  2  0  6  0  3  0  1  0  ..........  2
1: LODS                ! 28  | pfmt + 0  =0000: 54 68 65 20 69 6e 20 70  The in prod  26708
   MUL  (DI)           ! 29  | pfmt + 18 =0012: 25 64 21  a  0  0  3  0  %d!.......  25637
   ADD  -2(BP),AX      ! 30  |
   ADD  DI,2           ! 31  |
   LOOP 1b             ! 32  |
```

Figura C.15 Esecuzione di *vecprod.s* al primo ingresso nel ciclo della riga 28.

Esaminiamo ora il ciclo che comincia alla riga 28. L'istruzione LODS carica in AX una parola di memoria dal segmento dati in modalità registro indiretto (tramite il registro SI). Poiché il flag della modalità d'incremento è asserito, LODS si autoincrementa, perciò dopo l'esecuzione della 28, SI punta al successivo elemento di *vec1*.

Per visualizzare graficamente questo meccanismo, basta avviare il tracer con il comando

```
t88 vecprod
```

e attendere il caricamento del programma, quindi digitare

```
/vecmul+7b
```

(seguito da un invio a capo) per inserire un breakpoint alla riga dell'istruzione LODS. Ricordiamo che ogni comando deve essere seguito da un invio a capo e d'ora in avanti eviteremo di ripeterlo. Digitare quindi il comando

```
g
```

affinché il tracer continui a eseguire le istruzioni finché non incontri il breakpoint; si fermerà alla riga contenente LOADS.

La riga 29 moltiplica AX per l'operando sorgente. La parola di memoria richiesta dall'istruzione MUL è prelevata dal segmento dati attraverso il registro DI usato in modalità registro indiretto. La destinazione di MUL è la combinazione di registri DX:AX, che è implicita (cioè non menzionata dall'istruzione).

La riga 30 somma il risultato della moltiplicazione alla variabile locale che si trova all'indirizzo dello stack –2(BP). Dato che MUL non incrementa automaticamente il proprio operando, bisogna effettuare l'incremento esplicitamente (riga 31), dopodiché DI punterà all'elemento successivo del vettore *vec2*.

L'istruzione LOOP conclude questa fase di calcolo: CX viene decrementato e, se è ancora positivo, il programma torna all'etichetta locale *1* della riga 28. L'espressione *1b* sta a indicare proprio la precedente etichetta locale *1* più vicina rispetto alla posizione corrente. Alla terminazione del ciclo, la subroutine esegue la pop del valore da restituire in AX (riga 33), ripristina BP (riga 34) e passa il controllo al programma chiamante (riga 35).

Il programma riprende dalla posizione successiva alla chiamata con l'esecuzione dell'istruzione MOV alla riga 12. Questa è la prima di cinque istruzioni finalizzate alla stampa del risultato. La chiamata di sistema `printf` ricalca il comportamento della funzione *printf* della libreria C standard. I tre argomenti sono impilati sullo stack alle righe 13-15; si tratta del valore intero da stampare, dell'indirizzo della stringa di formattazione (*pfmt*) e del codice di funzione di `printf` (127). La stringa di formattazione *pfmt* contiene la stringa *%d* a indicare che la chiamata `printf` ha un argomento di tipo intero che deve essere inserito nella stampa.

La riga 17 ripristina lo stack. Poiché l'esecuzione del programma è cominciata alla riga 5 con il salvataggio del puntatore allo stack nella locazione del puntatore base, sarebbe stato possibile ripristinare lo stack anche con l'istruzione

```
MOV SP,BP
```

Il vantaggio di questa soluzione è che il programmatore non deve tenere il conto delle dimensioni raggiunte dal record d'attivazione. La questione è di poca importanza per quel che riguarda il programma principale, ma nelle subroutine può aiutare a liberare lo stack dai dati inutili, per esempio le variabili locali obsolete.

La subroutine *vecmul* può essere inclusa in altri programmi. Se il file sorgente *vecprod.s* è inserito nella riga dei comandi dopo un altro file sorgente in ingresso all'assemblatore, la subroutine è utilizzabile per moltiplicare due vettori di lunghezza fissa. Si consiglia in tal caso di rimuovere le definizioni delle costanti `_EXIT` e `_PRINTF` per evitare di definirle due volte. Basta includere in un punto qualsiasi il file *syscalnr.h* per non doversi più preoccupare di definire le costanti delle chiamate di sistema.

C.8.4 Debugging di un programma per la stampa di array

I programmi degli esempi precedenti sono molto semplici e svolgono la propria funzione correttamente. Veniamo ora all'uso del tracer come strumento di supporto per il debugging di programmi con errori. Il programma seguente dovrebbe stampare l'array d'interi che comincia all'etichetta *vec1*, tuttavia la sua versione iniziale contiene ben tre errori. Vedremo come usare l'assemblatore e il tracer per correggere questi errori; cominciamo con il descrivere il codice.

Per ragioni di comodità abbiamo creato il file d'intestazione *syscalnr.h* dove abbiamo inserito tutte le definizioni di costanti che identificano le chiamate di sistema, per non doverle ridefinire all'interno di ogni programma che ne ha bisogno. La riga 1 del programma serve proprio a includere il file d'intestazione. *../syscalnr.h* contiene anche le costanti dei descrittori di file


```
STDIN = 0
STDOUT = 1
STDERR = 2
```

aperti automaticamente all'avvio del processo, così come le etichette d'intestazione per i segmenti del testo e dei dati. Tutte queste definizioni sono molto utilizzate, perciò vale la pena includere *../syscalnr.h* all'inizio di tutti i file sorgente scritti in linguaggio assemblativo. Se il codice sorgente è distribuito tra più file, l'assemblatore provvede a includere il file d'intestazione una sola volta per evitare di ridefinire più volte le stesse costanti.

La Figura C.16 contiene il codice il programma *arrayprt*. Abbiamo omesso il commento delle singole istruzioni, perché dovrebbero risultare ormai familiari, perciò abbiamo utilizzato il formato a due colonne. La riga 4 inserisce l'indirizzo dello stack (ancora vuoto) nel registro puntatore base per consentire il successivo ripristino dello stack alla riga 10; basterà poi copiare il puntatore base nel puntatore allo stack come già descritto nell'esempio precedente. Le righe dalla 5 alla 9 impilano gli argomenti nello stack analogamente a quanto già visto negli esempi precedenti. Le righe dalla 22 alla 25 caricano i registri nella subroutine.

```
#include "../syscalnr.h"          ! 1

.SECT .TEXT                       ! 2
vecpstrt:                         ! 3
   MOV  BP,SP                     ! 4
   PUSH vec1                      ! 5
   MOV  CX,frmatstr-vec1          ! 6
   SHR  CX                        ! 7
   PUSH CX                        ! 8
   CALL vecprint                  ! 9
   MOV  SP,BP                     ! 10
   PUSH 0                         ! 11
   PUSH _EXIT                     ! 12
   SYS                            ! 13

.SECT .DATA                       ! 14
vec1:  .WORD 3,4,7,11,3           ! 15
frmatstr: .ASCIZ "%s"             ! 16

frmatkop:                         ! 17
.ASCIZ "L'array contiene "        ! 18
frmatint: .ASCIZ " %d"            ! 19

.SECT .TEXT                       ! 20
vecprint:                         ! 21
   PUSH BP                        ! 22
   MOV  BP,SP                     ! 23
   MOV  CX,4(BP)                  ! 24
   MOV  BX,6(BP)                  ! 25
   MOV  SI,0                      ! 26
   PUSH frmatkop                  ! 27
   PUSH frmatstr                  ! 28
   PUSH _PRINTF                   ! 29
   SYS                            ! 30
   MOV  -4(BP),frmatint           ! 31
1: MOV  DI,(BX)(SI)               ! 32
   MOV  -2(BP),DI                 ! 33
   SYS                            ! 34
   INC  SI                        ! 35
   LOOP 1b                        ! 36
   PUSH '\n'                      ! 37
   PUSH _PUTCHAR                  ! 38
   SYS                            ! 39
   MOV  SP,BP                     ! 40
   RET                            ! 41
```

Figura C.16 Programma *arrayprt* prima del debugging.

Le righe dalla 27 alla 30 mostrano la stampa di una stringa, quelle dalla 31 alla 34 contengono la chiamata di sistema `printf` per un valore intero. Si noti che l'indirizzo della stringa viene impilato alla riga 27, mentre il valore intero viene copiato nello stack alla riga 33. In entrambi i casi l'indirizzo della stringa di formattazione è il primo argomen-

to della PRINTF. Le righe dalla 37 alla 39 mostrano come stampare un carattere usando la chiamata di sistema putchar.

Dopo aver assemblato il programma, ne lanciamo l'esecuzione con il comando

```
as88 arrayprt.s
```

e riceviamo un errore di operando alla riga 28 del file *arrayprt.$*. Questo file è prodotto dall'assemblatore combinando il file sorgente ai file inclusi ed è il vero input del processo di assemblaggio. Per individuare il contenuto della riga 28 dobbiamo aprire *arrayprt.$* in lettura, mentre non serve a nulla ispezionare *arrayprt.s*, perché i due file non corrispondono, in quanto il primo include il file d'intestazione alla prima riga. La riga 28 di *arrayprt.$* corrisponde alla riga 7 di *arrayprt.s* dal momento che il file d'intestazione, *../syscalnr.h*, contiene 21 righe.

Un modo semplice per visualizzare la riga 28 di *arrayprt.$* su di una macchina UNIX è lanciare il comando

```
head -28 arrayprt.$
```

che mostra le prime 28 righe del file *.$*. L'ultima riga è quella contenente l'errore. Così facendo (o in modo analogo con l'ausilio di un editor) notiamo che l'errore è alla riga 7 del sorgente originale, cioè in corrispondenza dell'istruzione SHR. Se confrontiamo la riga di codice con l'elemento corrispondente della Figura C.4 individuiamo il problema: abbiamo dimenticato d'indicare la lunghezza dello scorrimento. Una volta corretta la riga, diventa

```
SHR CX,1
```

È importante sottolineare che l'errore va corretto nel file sorgente originale *arrayprt.s* e *non* nel sorgente ausiliario *arrayprt.$*, rigenerato automaticamente a ogni chiamata dell'assemblatore.

Il nuovo tentativo di assemblare il codice sorgente dovrebbe andare a buon fine. Si può procedere con il lancio del tracer

```
t88 arrayprt
```

Durante l'esecuzione del tracer notiamo che il risultato del programma non corrisponde al vettore che si trova nel segmento dati. Il vettore contiene: 3, 4, 7, 11, 3 ma i valori visualizzati sono: 3, 1024, ... Chiaramente c'è qualcosa che non va.

Per trovare l'errore facciamo ripartire il tracer, ma questa volta procediamo nella simulazione un'istruzione alla volta, esaminando lo stato della macchina appena prima della stampa del valore errato. Il valore da stampare è memorizzato alle righe 32 e 33; poiché la stampa visualizza un valore errato, è questo il punto dove rivolgere l'attenzione. Durante la seconda iterazione del ciclo, SI contiene un valore dispari, mentre dovrebbe contenere un numero pari, dato che indirizza parole, non byte. L'errore si trova alla riga 35: SI viene incrementato di 1, laddove andrebbe incrementato di 2. Per correggere il baco bisogna modificare la riga in questo modo

```
ADD SI,2
```


Dopo la correzione la lista dei numeri stampati risulta corretta.

Eppure c'è ancora un errore da scovare. Al termine di *vecprint* il tracer visualizza un errore relativo al puntatore allo stack. Il controllo ovvio da effettuare in questo caso è verificare se il valore impilato sullo stack all'atto della chiamata di *vecprint* è uguale al valore che si trova in cima allo stack al momento della chiamata RET della riga 41. Il controllo evidenzia che i due valori sono diversi. La soluzione al problema consiste nel sostituire la riga 40 con le due righe:

```
ADD SP,10
POP BP
```

La prima istruzione rimuove dallo stack le 5 parole impilate durante l'esecuzione di *vecprint*, riportando in cima il valore di BP salvato alla riga 22. La pop di questo valore verso BP ripristina nel registro il valore che aveva prima della chiamata e fa emergere in cima allo stack l'indirizzo di ritorno corretto. Adesso il programma termina correttamente. Il debugging del codice in linguaggio assemblativo somiglia sicuramente più a un'arte che a una scienza, tuttavia risulta molto facilitato dall'uso del tracer rispetto alla "nuda" esecuzione in hardware.

C.8.5 Manipolazione di stringhe e istruzioni su stringhe

Lo scopo principale di questo paragrafo è presentare le istruzioni iterative sulle stringhe. La Figura C.17 mostra due semplici programmi per la manipolazione di stringhe, *strngcpy.s* e *reverspr.s*, contenuti nella directory *examples*. La Figura C.17(a) contiene una subroutine per la copia di una stringa che si avvale di un'altra subroutine, *stringpr*, che non mostriamo qui, ma che è reperibile nel file *stringpr.s*. Per assemblare un programma che comprende più subroutine contenute in file sorgenti diversi, bisogna elencarli tutti dopo il comando *as88*, a cominciare dal file sorgente contenente il programma principale. Quest'ultimo determina il nome del file eseguibile e del file *.$*. Per esempio, nel caso del programma della Figura C.17(a) bisogna digitare

```
as88 strngcpy.s stringpr.s
```

Il programma della Figura C.17(b) stampa le stringhe in ordine inverso. Descriviamo ora i due programmi.

Per sottolineare il fatto che i numeri di riga sono solo commenti, nella Figura C.17(a) abbiamo numerato le righe a partire dalla prima etichetta, omettendo le righe che la precedevano. Il programma principale occupa le righe dalla 2 alla 8: per prima cosa richiama *strngcpy* con due argomenti, la stringa sorgente *mesg2* e la stringa destinazione *mesg1*, perché copi la sorgente nella destinazione.

Diamo uno sguardo a *strngcpy*, cominciando dalla riga 9. La subroutine si aspetta di trovare in cima allo stack gli indirizzi della destinazione e della sorgente. Nelle righe dalla 10 alla 13 si provvede a salvare i registri nello stack di modo che possano essere ripristinati successivamente con le righe dalla 27 alla 30. Alla riga 14 si effettua la copia di SP in BP come di consueto; adesso BP è pronto per essere usato come puntatore base per l'accesso agli argomenti. Ancora una volta il ripristino dello stack avviene mediante la copia di BP in SP, alla riga 26.

```
.SECT. TEXT                                  ! 1
strstart:                                    ! 2
  PUSH mesg1                                 ! 3
  PUSH mesg2                                 ! 4
  CALL strngcpy                              ! 5
  ADD  SP,4                                  ! 6
  PUSH 0                                     ! 7
  PUSH 1                                     ! 8
  SYS                                        ! 9
strngcpy:                                    ! 10
  PUSH CX                                    ! 11
  PUSH SI                                    ! 12
  PUSH DI                                    ! 13
  PUSH BP                                    ! 14
  MOV  BP,SP                                 ! 15
  MOV  AX,0                                  ! 16
  MOV  DI,10(BP)                             ! 17
  MOV  CX,-1                                 ! 18
  REPNZ SCASB                                ! 19
  NEG  CX                                    ! 20
  DEC CX                                     ! 21
  MOV  SI,10(BP)                             ! 22
  MOV  DI,12(BP)                             ! 23
  PUSH DI                                    ! 24
  REP  MOVSB                                 ! 25
  CALL stringpr                              ! 26
  MOV  SP,BP                                 ! 27
  POP  BP                                    ! 28
  POP  DI                                    ! 29
  POP  SI                                    ! 30
  POP  CX                                    ! 31
  RET                                        ! 32
.SECT .DATA                                  ! 33
mesg1: .ASCIZ "Dacci un'occhiata\n"          ! 34
mesg2: .ASCIZ "qrst\n"
.SECT .BSS
```

(a)

```
#include "../syscalnr.h"                     ! 1

start: MOV DI,str                            ! 2
    PUSH AX                                  ! 3
    MOV BP,SP                                ! 4
    PUSH _PUTCHAR                            ! 5
    MOVB AL,'\n '                            ! 6
    MOV CX,-1                                ! 7
    REPNZ SCASB                              ! 8
    NEG CX                                   ! 9
    STD                                      ! 10
    DEC CX                                   ! 11
    SUB DI,2                                 ! 12
MOV SI,DI                                    ! 13
    1:LODSB                                  ! 14
    MOV (BP),AX                              ! 15
    SYS                                      ! 16
    LOOP 1b                                  ! 17
    MOVB (BP),'\n '                          ! 18
    SYS                                      ! 19
    PUSH 0                                   ! 20
    PUSH _EXIT                               ! 21
SYS                                          ! 22
.SECT .DATA                                  ! 23
str: .ASCIZ "reverse\n"                      ! 24
```

(b)

Figura C.17 (a) Copia di una stringa (*strngcpy.s*). (b) Stampa di una stringa in ordine inverso (*reverspr.s*).

Il cuore della subroutine è l'istruzione REP MOVSB della riga 24. L'istruzione MOVSB trasferisce il byte puntato da SI nella locazione di memoria puntata da DI. SI e DI vengono poi incrementati di un'unità. L'istruzione REP crea un ciclo in cui, a ogni iterazione, si esegue MOVSB e si decrementa CX di 1 dopo ogni trasferimento di un byte. Il ciclo termina quando CX raggiunge il valore 0.

Prima di poter eseguire REP MOVSB, bisogna però preparare i registri, come svolto nelle righe dalla 15 alla 22. L'indice sorgente SI viene copiato nella riga 21 dall'argomento che si trova sullo stack; l'indice destinazione DI è assegnato alla riga 22. Il calcolo del valore di CX è più difficile. Si tenga presente che il carattere di terminazione di una stringa è il byte zero. L'istruzione MOVSB non modifica il flag di zero, a differenza dell'istruzione SCASB (*SCAn Byte String*) che confronta il valore puntato da DI al contenuto di AL e incrementa DI al volo. Anche SCASB è un'istruzione iterabile come MOVSB.

Dunque alla riga 15 viene azzerato AX (e anche AL), alla riga 16 viene prelevato dallo stack il valore cui far puntare DI e alla riga 17 CX viene inizializzato a –1. La riga 18 contiene REPNZ SCASB, che effettua il confronto iterato e asserisce la flag zero quando rileva l'uguaglianza. A ogni iterazione del ciclo CX viene decrementato e il ciclo termina quando il flag di zero viene asserito, dal momento che REPNZ controlla sia il valore di CX, sia il contenuto di quel flag. Il numero d'iterazioni di MOVSB si ottiene sottraendo il valore corrente di CX al suo valore iniziale –1, come effettuato alle righe 19 e 20.

Purtroppo si rendono necessarie ben due istruzioni iterative, ma è il prezzo che bisogna pagare per la scelta progettuale secondo cui le istruzioni di trasferimento non possono modificare i codici di condizione. I registri indice devono essere incrementati durante i cicli, e a tal fine è necessario azzerare il flag della modalità d'incremento.

Le righe 23 e 25 stampano la stringa copiata attraverso la chiamata della subroutine *stringpr*, anch'essa contenuta nella directory *examples*. Il suo comportamento è di facile comprensione, perciò ne omettiamo la trattazione.

Il programma della Figura C.17(b) per la stampa invertita comincia con l'usuale riga d'inclusione delle costanti per le chiamate di sistema. La riga 3 impila sullo stack un dato fittizio, mentre la riga 4 fa sì che BP punti alla cima dello stack. Il programma stampa i caratteri ASCII uno per volta, perciò il valore numerico di _PUTCHAR viene posto in cima allo stack. BP punta al carattere da stampare al momento della chiamata di SYS.

Le righe 2, 6 e 7 preparano i registri DI, AL e CX per l'istruzione SCASB. Il registri per la lunghezza e per l'indice di destinazione sono caricati in modo analogo a quanto visto nella routine per la copia di una stringa, ma il valore di AL è ora il carattere di fine riga e non il valore 0. Così facendo, l'istruzione SCASB confronterà i caratteri della stringa *str* con \n e non con 0, e asserirà il flag zero non appena troverà una corrispondenza.

REP SCASB incrementa il registro DI perciò, dopo aver trovato la corrispondenza, l'indice destinazione punta al carattere zero che segue il carattere di fine riga. La riga 12 decrementa DI di 2 di modo che punti all'ultima lettera dell'ultima parola della stringa. Per produrre il risultato desiderato si può procedere percorrendo la stringa in ordine inverso e stampando un carattere alla volta, perciò il flag della modalità d'incremento viene asserito alla riga 10 in modo da invertire la modalità d'incremento dei registri indice adottata dalle istruzioni sulle stringhe. L'istruzione LODSB alla riga 14 copia il carattere in AL e la riga 15 lo inserisce nella locazione dello stack adiacente a _PUTCHAR, così che venga stampato dall'esecuzione di SYS.

Le istruzioni delle righe 18 e 19 stampano un carattere di fine riga e il programma termina con la chiamata _EXIT nel modo usuale.

Questa versione del programma contiene però un baco che può essere individuato eseguendolo con il tracer un'istruzione alla volta.

Il comando */str* inserisce la stringa *str* nell'area dati del tracer. Il valore numerico dell'indirizzo dei dati è visibile, perciò è possibile monitorare la progressione delle posizioni dei registri indice all'interno dei dati rispetto all'indirizzo della stringa.

Il baco si manifesterà solo dopo aver premuto più volte il tasto Invio. Possiamo usare i comandi del tracer per raggiungere il problema più velocemente. Dopo aver avviato il tracer gli si invia il comando *13* per posizionarsi nel mezzo del ciclo. Il comando *b* inserisce un breakpoint in corrispondenza della riga 15. Dopo due invii a capo vediamo

apparire la lettera e (ultimo carattere della stringa "reverse") nell'area di output. Il comando *r* ordina al tracer di continuare l'esecuzione fino al prossimo breakpoint o fino alla fine del processo. In questo modo possiamo scorrere le lettere impartendo ogni volta il comando *r* finché non ci avviciniamo al problema. Solo allora riprendiamo l'esecuzione un'istruzione alla volta, fino a raggiungere l'istruzione critica.

In alternativa è possibile inserire il breakpoint a una riga specifica, ma a tal fine bisogna tenere presente che è stato incluso il file *syscalnr* e che i numeri di riga vanno spostati di 21. Di conseguenza il breakpoint alla riga 15 può essere inserito con il comando *36b*; questa non è una scelta molto elegante, è preferibile usare l'etichetta globale *start* della riga 2 e impartire il comando */start + 13b* che inserisce il breakpoint nello stesso punto e permette di trascurare la dimensione del file d'intestazione.

C.8.6 Tabella di salto

Molti linguaggi di programmazione mettono a disposizione le istruzioni *case* o *switch* per selezionare una destinazione di salto tra numerose alternative in base al valore di una certa variabile. A volte queste diramazioni "a più vie" (*multiway branch*) sono necessari anche nei programmi in linguaggio assemblativo. Si pensi per esempio a un insieme di chiamate di sistema invocabili tramite un'unica trap SYS. Il programma *jumptbl.s* della Figura C.18 mostra un modo per utilizzare questo costrutto nell'assemblatore dell'8088.

Il programma comincia con la stampa della stringa che si trova all'etichetta *strt* e che invita l'utente a immettere una cifra ottale (righe dalla 4 alla 7), quindi legge un carattere dallo standard input (righe 8 e 9). Se il valore di AX è minore di 5 il programma lo interpreta come un marcatore di terminazione di file e salta all'etichetta *8* della riga 22, provocando l'uscita del programma con codice d'uscita 0.

In caso contrario, il carattere inserito in AL viene esaminato nel modo seguente: se è minore del carattere 0 viene interpretato come uno spazio bianco e viene ignorato saltando alla riga 13, per ricevere un altro carattere. Gli input maggiori della cifra 9 sono considerati errati e vengono mappati nel carattere ASCII dei due punti (:) che segue il carattere 9 nella sequenza dei codici della tabella ASCII.

La riga 17 copia in BX il contenuto di AX, che a questo punto contiene un valore compreso tra 0 e i due punti. L'istruzione AND della riga 18 maschera tutti i bit di BX tranne i quattro meno significativi, producendo un risultato che va da 0 a 10 (poiché la cifra 0 corrisponde al codice ASCII 0x30). Il valore di BX viene moltiplicato per 2 alla riga 19 con uno scorrimento a sinistra, dato che ci apprestiamo a indirizzare una tabella di parole e non di byte.

La riga 20 contiene un'istruzione di chiamata. L'indirizzo effettivo, ottenuto sommando BX al valore numerico dell'etichetta *tbl*, viene caricato nel program counter.

Il programma sceglie una delle dieci subroutine a seconda del carattere ricevuto in ingresso dallo standard input. Ciascuna di queste subroutine impila sullo stack l'indirizzo di un messaggio e salta alla chiamata della subroutine di sistema _PRINTF condivisa da tutte e dieci.

Per capire come funziona il programma bisogna sapere che le istruzioni JMP e CALL caricano un certo indirizzo del segmento di testo nel PC. Questo indirizzo non è altro che un numero, poiché il processo di assemblaggio sostituisce tutti gli indirizzi con i loro valori binari. Alla riga 50 questi valori binari vengono utilizzati per inizializzare un array nel segmento dati. L'array che comincia alla locazione *tbl* contiene gli indirizzi *rout0*, *rout1*, *rout2*, e così via, ciascuno lungo due byte. La necessità di usare indirizzi di due byte giustifica lo scorrimento di un bit che abbiamo incontrato alla riga 19. Un array di questo tipo si dice **tabella di salto** (*dispatch table*).


```
#include "../syscalnr.h"    ! 1
.SECT .TEXT                 ! 2
jumpstrt:                   ! 3
    PUSH strt               ! 4
    MOV BP,SP               ! 5
    PUSH _PRINTF            ! 6
    SYS                     ! 7
    PUSH _GETCHAR           ! 8
1:  SYS                     ! 9
    CMP AX,5                ! 10
    JL 8f                   ! 11
    CMPB AL,'0'             ! 12
    JL 1b                   ! 13
    CMPB AL,'9'             ! 14
    JLE 2f                  ! 15
    MOVB AL,'9'+1           ! 16
2:  MOV BX,AX               ! 17
    AND BX,0Xf              ! 18
    SAL BX,1                ! 19
    CALL tbl(BX)            ! 20
    JMP 1b                  ! 21
8:  PUSH 0                  ! 22
    PUSH _EXIT              ! 23
    SYS                     ! 24

rout0:  MOV AX,mes0         ! 25
        JMP 9f              ! 26
rout1:  MOV AX,mes1         ! 27
        JMP 9f              ! 28
rout2:  MOV AX,mes2         ! 29
        JMP 9f              ! 30
rout3:  MOV AX,mes3         ! 31
        JMP 9f              ! 32
rout4:  MOV AX,mes4         ! 33
        JMP 9f              ! 34
rout5:  MOV AX,mes5         ! 35
        JMP 9f              ! 36
rout6:  MOV AX,mes6         ! 37
        JMP 9f              ! 38
rout7:  MOV AX,mes7         ! 39
        JMP 9f              ! 40
rout8:  MOV AX,mes8         ! 41
        JMP 9f              ! 42
erout:  MOV AX,emes         ! 43
9:      PUSH AX             ! 44
        PUSH _PRINTF        ! 45
        SYS                 ! 46
        ADD SP,4            ! 47
        RET                 ! 48

.SECT .DATA                                                          ! 49
tbl: .WORD rout0,rout1,rout2,rout3,rout4,rout5,rout6,rout7,rout8,rout8,erout  ! 50
mes0: .ASCIZ "La cifra è uno zero.\n"                                ! 51
mes1: .ASCIZ "La cifra è un uno.\n"                                  ! 52
mes2: .ASCIZ "La cifra è un due.\n"                                  ! 53
mes3: .ASCIZ "La cifra è un tre.\n"                                  ! 54
mes4: .ASCIZ "La cifra è un quattro.\n"                              ! 55
mes5: .ASCIZ "La cifra è un cinque.\n"                               ! 56
mes6: .ASCIZ "La cifra è un sei.\n"                                  ! 57
mes7: .ASCIZ "La cifra è un sette.\n"                                ! 58
mes8: .ASCIZ "La cifra non è una cifra ottale valida.\n"             ! 59
emes: .ASCIZ "Questa non è una cifra ottale.\n"                      ! 60
strt: .ASCIZ "Inserisci una cifra ottale valida seguita da invio. Per uscire inserisci
          un carattere di fine file.\n"                              ! 61
```

Figura C.18 Programma che implementa una diramazione a più vie usando una tabella di salto.

Osserviamo il funzionamento di queste routine descrivendo la routine *erout* che occupa le righe dalla 43 alla 48 e che gestisce il caso di un carattere esterno all'intervallo delle cifre. Alla riga 43 viene posto in cima allo stack l'indirizzo del messaggio (che si trova

6. Nella Figura C.1 viene usata l'espressione *de* − *hw*, cioè la differenza di due etichette, come operando di un'istruzione. È possibile immaginare una circostanza in cui sarebbe sensato usare come operando la somma di due etichette, per esempio *de* + *hw*? Si motivi la risposta.

7. Si scriva il codice assemblativo per il calcolo dell'espressione:
   x = a + b + 2

8. Una funzione C viene invocata con l'istruzione:
   foobar(x, y);
   Si scriva il codice assemblativo che realizza la stessa chiamata.

9. Si scriva un programma in linguaggio assemblativo che accetti in ingresso espressioni contenenti un intero, un operando e un altro intero, e che stampi in uscita il valore dell'espressione. Il programma deve gestire gli operatori +, −, × e /.

Indice analitico

A

Access Control List (ACL), 510
Accumulatore, 19, 708
ACK (*vedi* Amsterdam Compiler Kit)
ACL (*vedi* Access Control List)
Acorn Archimedes, 47
ADSL (*vedi* Asymmetric DSL)
Advanced Microcontroller Bus Architecture (AMBA), 582
Advanced Programmable Interrupt Controller (APIC), 212
AGP, bus, 224
Aiken, Howard, 16
Albero, 629
Algebra booleana minimale, 154
Algebra di Boole, 151-162
Algoritmi di paginazione, 455-457
  FIFO, 457
  LRU, 456
Algoritmo, 8
Algoritmo di allattamento a richiesta, 454
Algoritmo first fit, 462
Algoritmo First-In First-Out (FIFO), 456
Algoritmo Least Recently Used (LRU), 319, 455
Allocazione in scrittura, 320
Alpha, 14, 27
ALU (*vedi* Arithmetic Logic Unit)
AMBA (*vedi* Advanced Microcontroller Bus Architecture)
American Standard Code for Information Interchange (ASCII), 141
Ampiezza del bus, 196-197
Amsterdam Compiler Kit, 727
Analytical engine, 15, 27
Anello, 577, 629
APIC (*vedi* Advanced Programmable Interrupt Controller)
Apparecchiatura per le telecomunicazioni, 130-138
Apple II, 25
Apple Lisa, 25
Apple Macintosh, 26
Apple Newton, 14, 28, 47
Application Programming Interface (API), 495
Application-Specific Integrated Circuit (ASIC), 586
Arbitraggio del bus, 202-205
Arbitro del bus, 113
Architettura a tre bus, 296
Architettura degli elaboratori, 8, 77
Architettura di un computer, 8
Architettura Harvard, 85
Architettura IA-32, 432
  difetti, 432

Architettura load/store, 365
Architettura superscalare, 67-69
architettura x86, 41
Arduino, 35
Area dei metodi, 267
Argomenti, 722
Arithmetic Logic Unit (ALU), 5, 56-57, 172
Aritmetica binaria, 690
Aritmetica satura, 567
ARM
  bi-endian, 364
  formati d'istruzione, 378-380
  indirizzamento, 394
  introduzione, 46-49
  istruzioni, 410-412
  livello ISA, 363-365
  memoria virtuale, 468-471
  microarchitettura dell'OMAP 4430, 341-343
  tipi di dati, 370
ARM, 46 (*vedi anche* OMAP 4430)
ARM v7, 356, 365
Arrotondamento, 695
As88, 727-732
ASCII (*vedi* American Standard Code for Information Interchange)
ASIC (*vedi* Application Specific Integrated Circuit)
Assemblaggio di memoria, 86
Assemblatore a due passate, 537-545
Assemblatore, 7, 526, 530, 537, 726
  8088, 731
Assemblatore, prima passata, 538-542
Assemblatore, seconda passata, 542-544
Asymmetric DSL, 133
AT attachment disk, 93
ATA packet interface, 93
Atanasoff, John, 16
ATmega168, 219-221, 346, 348
  formati d'istruzione, 380
  istruzioni, 412-413
  livello ISA, 366-368
  microarchitettura, 346-348
  modalità d'indirizzamento, 394-395
  tipi di dati, 371
Atmel ATmega168 (*vedi* ATmega168)
Attesa attiva, 405
Autoincremento, 715
Automa a stati finiti, 299, 325
Auto-modificante, programma, 384
AVR, 49-50

B

Babbage, Charles, 15, 27
Banda larga, 133
Bardeen, John, 19
Base, 152, 683
Basic Input Output System (BIOS), 92
Baud, 132
Best fit, algoritmo, 462
Big Endian, memoria, 77-78
Binario, sistema di numerazione, 683
BIOS (*vedi* Basic Input Output System)
Bit, 74, 684
Bit di parità, 79
Bit map, 369
Bit presente/assente, 451
Blocchi di memoria, 450
Blocco delle variabili locali, 265
Blocco elementare, 332
Blocco indiretto, 506
Blocco indiretto doppio, 506
Blocco indiretto triplo, 498
BlueGene, 631-635
BlueGene/L, 631
BlueGene/P, 631
BlueGene/P e Red Storm, confronto, 631, 639
Blu-ray, 111
Boole, George, 154
Branch Target Buffer (BTB), 338
Brattain, Walter, 19
Breakpoint, 734
BSS (block started by symbol), 727
BTB (*vedi* Branch Target Buffer)
Buffer di prefetch, 65
Buffer di scrittura, OMAP 4430, 343
Buffer invertente, 183
Buffer non invertente, 183
Bundle, Itanium 2, 435
Burroughs B5000, 14, 22
Bus, 20, 55, 111-114, 191-208, 221-239
  asincrono, 201-202
  Core i7, 213-215

  di sistema, 194
  EISA, 113
  ISA, 113
  master, 195, 197
  multiplexato, 197
  PCI, 113-115, 222
  PCIe, 113-115
  sincrono, 198-201
  slave, 195
Bus del processore, 581
Bus delle periferiche, 581
Bus di sistema 184
Bus multiplexato, 197
Byron, Lord, 15
Byte, 76-77, 357
Byte di prefisso, 286, 378

C

Cache, 83, 314-320
  a corrispondenza diretta, 316-318
  di secondo livello, 315
  protocollo MESI, 609-611
  separata, 84, 315, 358
  set-associativa, 319-321
  snooping, 234, 607
  strategia di aggiornamento, 609
  unificata, 85
  write-allocate, 310, 611
  write-back, 309, 320
  write-deferred, 320
  write-through, 320, 608
Cache, miss, 318
Cache, strategia d'invalidazione, 609
Cache, strategia di aggiornamento, 609
Cache, write-back, 320
Cache, write-deferred, 320
Cache, write-through, 320, 599
Cache a blocchi, 492
Cache a corrispondenza diretta, 316
Cache delle micro-operazioni, 325
Cache di secondo livello, 315
Cache hit, 318
Cache Only Memory Access (COMA), 600, 624-625
Cache set-associativa, 319
Cache set-associativa a n vie, 319
Cache specializzata, 85, 315
Cache unificata, 85
Call gate, 468
Capacitore (condensatore), 117
Caricamento. 546-558
Caricamento speculativo, 439
Catamount, 638
CCD (*vedi* Charge-Coupled Device)
CC-NUMA, 615
CD registrabile, 105-108
CD riscrivibile, 108
CDC (*vedi* Control Data Corporation)
CDC 6600, 14, 21
CD-R (*vedi* CD registrabile)
CD-ROM (*vedi* Compact Disc-Read Only Memory)
CD-ROM multisessione, 107
Celeron, 45
Cella di memoria, 75
Central Processing Unit (CPU), 18, 56-72
Centro di calcolo, 24
Charge-Coupled Device (CDC), 139
Checkerboarding, 461
Chiamata al supervisore, 11
Cluster, 38, 510
Chiamata di sistema, 11, 445, 723-725
Chiamata ravvicinata, 722
Chiave di un record, 476
Chip, 162
Chip di memoria, 183-186
Chipset, 207, 212
Chiusura, 682
Cicli di bus, 198
Ciclo del percorso dati, 57
Ciclo del processore, 706
Ciclo di clock, 173
Ciclo di prelievo-decodifica-esecuzione, 58, 250
Ciclo locale, 133
Ciclo virtuoso, 30
Cilindro, disk, 90
Circuiti integrati (IC), 162
Circuiti logici digitali, 162-173
Circuiti per l'aritmetica, 169
Circuito
  aritmetico, 169-172
  reti combinatorie, 163-165

Circuito virtuale, 234
CISC (*vedi* Complex Instruction Set Computer)
Classificazione di pacchetti, 589
Clock, 173-174
Clone, 25
Cloud computing, 39
Cluster computing, 640-645
Cluster di Google, 641-645
Cluster of Workstations, 602
COBOL, 40, 369
Coda di messaggi, 512
Code page, 143
Code point, 143
Codice di escape, 378
Codice operativo (opcode), 250
Codice operativo espandibile, 374-376
Codice Reed-Solomon, 89
Codici condizione, 346
Codici correttori, 78-82
Codifica, 8b/10b, 233
Codifica dei caratteri, 141-145
Codifica hash, 544
Coerenza della cache, 616
Collegamento, 554-558
  a tempo del binding, 551-554
  dinamico, 554-558
  MULTICS, 463, 554
  UNIX, 557
  Windows, 555
Collegamento, file, 503
Collegamento a festone (daisy chaining), 203
Collegamento dinamico, 554-558
  MULTICS, 554-557
  UNIX, 557
  Windows, 557
Collegamento esplicito, 557
Collegamento implicito, 557
Collettore, 152
Collettore aperto, 196
COLOSSUS, 14, 17-18
COMA (*vedi* Cache Only Memory Access computer)
COMA semplice, 624
Commodity Off The Shelf (COTS), 38
Commutatori crossbar, 611
Commutazione a livello, latch, 175
Commutazione di pacchetto, 584
Commutazione sul fronte, flip-flop, 176
Compact Disc-Read Only Memory (CD-ROM), traccia, 107
Compact Disc-Read Only Memory (CD-ROM), XA, 107
Compagnia telefonica, 132
Comparatore, 166-167
Compilatore, 7, 526
Complemento a due, 688
Complemento a uno, 688
Complex Instruction Set Computer (CISC), 62
Compute Unified Device Architecture (CUDA), 591
Computer
  gioco, 36-37
  parallelo sui dati, 72-74
  usa e getta, 32-34
Computer, approccio strutturale, 1-13
Computer, tipologie, 29-40
Computer a valvole, 16-19
Computer con parallelismo sui dati, 70-72
Computer di generazione zero, 13-16
Computer di prima generazione, 16-19
Computer di quarta generazione, 24-27
Computer di quinta generazione, 27-29
Computer di seconda generazione, 19-22
Computer di terza generazione, 22-24
Computer invisibile, 27
Computer MIPS, 62
Computer paralleli
  prestazioni, 657-663
  tassonomia, 599
Computer per giocare, 36
Computer usa e getta, 32
Comunicatore, 647
Condivisione a soglia, 575
Condivisione ripartita delle risorse, 574
Condivisione totale delle risorse, 575
Consistenza
  debole, 605
  di cache, 602
  di processore, 604

  dopo rilascio, 605
  sequenziale, 603
  stretta, 602
Consumatore, 482
Contatore di locazioni, 726
Control Data Corporation (CDC), 21
Controller per videogiochi, 123-125
Controllo di flusso, 234
Controllore, 111
Controllore del disco, 91
Conversione tra basi, 684-687
Copia dopo scrittura, 497
Coprocessore, 582-591
  grid, 663-666
  multiprocessore, 595
  parallelismo nel chip, 562-570
Coprogettati, hardware e software, 28
Core 2 duo, 44
Core i7
  banking, 340
  formati d'istruzione, 371
  foto del chip, 44
  hyperthreading, 42, 573-575
  indirizzamento, 392-394
  introduzione, 41-50
  istruzioni, 403
  livello ISA, 366-369
  memoria virtuale, 464-468
  microarchitettura, 335-341
  modalità d'indirizzamento, 392-394
  modello di memoria, 537
  multiprocessore, 577
  pipelining, 213
  predizione dei salti, 321-326
  reorder buffer, 338
  schedulatore, 327
  tipi di dati, 367
  unità di ritiro, 340
  virtualizzazione, 471
CoreConnect, 581
Coroutine, 415, 421-423
Corsa critica, 482-486
Corsia, PCI Express, 233
Cortex A9, 341-345
Costante di rilocazione, 549
COTS (*vedi* Commodity Off The Shelf)
COW (*vedi* Cluster of Workstations)
CP/M, 24
CPU (*vedi* Central Processing Unit)
CPU, chip, 191-193
Cray, Seymour, 21
CRAY-1, 14
CRC (*vedi* Cyclic Redundancy Check)
Creazione di un processo, 481
Crittografia,
  a chiave pubblica, 594
  a chiave simmetrica, 594
Crittoprocessori, 594
CRT (*vedi* Tubo catodico)
Cubo, 629
CUDA (*vedi* Compute Unified Device Architecture)
Cut through virtuale, 634
Cyclic Redundancy Check, 233

D

Data center, 39
Dati scaduti, 607
DDR (*vedi* Double Data Rate RAM)
DEC (*vedi* Digital Equipment Corporation)
DEC Alpha, 14, 26
DEC PDP-1, 14, 20,76
DEC PDP-11, 14, 24
DEC PDP-8, 14, 20
DEC VAX, 14, 61
Decodifica dell'indirizzo, 243
Decodifica parziale dell'indirizzo, 242
Decodificatore, 165-166
Demultiplexer, 165
Descrittore di file, 502, 724
Descrittore di sicurezza, 510
Destinatario, 227
Determinazione del destinatario, 589
Device bus dei registri di periferica, 582
Diametro, rete, 627
Difference engine (macchina differenziale), 15
Digital Equipment Corporation, 14, 20
Digital Subscriber Line, 132-134
Digital Subscriber Line Access Multiplexer (DSLAM), 134
Dimensionalità, 628
DIMM (*vedi* Dual Inline Memory Module)

DIP (*vedi* Dual Inline Package)
Dipendenza, 307, 329
Dipendenza effettiva, 307
Dipendenza RAW, 307
Dipendenza WAR, 329
Direct Memory Access (DMA), 212, 406
Directory, 471
Directory delle pagine, 466
Directory di lavoro, 502
Directory radice, 502
Direttive dell'assemblatore, 530
Disallineamento del bus, 115, 197
Disambiguazione, 341
Dischi magnetici, 87-99
Dischi ottici, 101-105
Disco
  ATAPI, 93
  CD-ROM, 102-105
  DVD, 108-110
  IDE, 92-94
  magnetico, 88-92
  ottico, 102-111
  RAID, 95-99
  SCSI, 94-95
  SSD, 97-99
  Winchester, 90
Disco, ATA-3, 93
Disco, ATAPI-4, 93
Disco, ATAPI-5, 93
Disco, Winchester, 90
Disco a stato solido (SSD), 99
Disco digitale versatile, 109
Disco video digitale, 109
Dispositivi di Input/Output
  apparecchiatura per le telecomunicazioni, 130-138
  controller per videogiochi, 123
  fotocamera digitale, 139
  modem, 130
  mouse, 121
  schermo piatto, 118
  stampante a colori, 127
  stampante a getto d'inchiostro, 128
  stampante laser, 125
  stampanti speciali, 129
  tastiera, 116
  touch screen, 116
Dispositivo a tre stati, 183
Disposizione fisica dei contatti (pinout), 49
Distanza di Hamming, 79
Divisore, 134
DLL (*vedi* Dynamic Link Library)
DMA (*vedi* Direct Memory Access)
Dot, 726
Double Data Rate RAM (DDR), RAM, 188
Dpi (punti per pollice), 126
DRAM (*vedi* Dynamic RAM)
Driver del bus, 195
DSL (*vedi* Digital Subscriber Line)
DSLAM (*vedi* Digital Subscriber Line Access Multiplexer)
DSM (*vedi* Memoria condivisa distribuita) hardware, 615
Dual Inline Memory Module (DIMM), 86
Dual Inline Package (DIP), 162
Duali, 159
Dump di memoria, 10
DVD (*vedi* disco video digitale)
Dynamic Link Library (DLL), 555

E

Eckert, J. Presper, 17
ECL (*vedi* Emitter-Coupled Logic)
EDO (*vedi* Extended Data Output)
EDSAC, 14
EDVAC, 17
EEPROM (*vedi* Electrically Erasable PROM)
EHCI (*vedi* Enhanced Host Controller Interface)
EIDE (*vedi* Extended IDE)
EISA bus (*vedi* Extended ISA bus)
Elaborazione di pacchetti, 588
Elaborazione in entrata, 589
Elaborazione in uscita, 588
Elaborazione parallela, 481
Electrically Erasable PROM (EEPROM), 189
Emettitore, 152
Emitter-Coupled Logic (ECL), 154
Emulazione, 23
Endian Endian
  big, 77-78
  little, 77-78

Enhanced Host Controller Interface (EHCI), 238
ENIAC, 14, 17
ENIGMA, 16
EPIC (*vedi* Explicitly Parallel Instruction Computing)
Epilogo della procedura, 420
EPROM (*vedi* Erasable PROM)
EPT (*vedi* Extended Page Table)
Equivalenza tra circuiti, 158-162
Erasable PROM (EPROM), 188
Errore di pagina, 453
Errore relativo, 695
Esadecimale, 683
Esecuzione condizionata, 437
Esecuzione dell'istruzione, 58-61
Esecuzione fuori sequenza, 326-332
Esecuzione speculativa, 332
Esponente, 694
Estensione del segno, 256
Estrazione di campi, 589
Estridge, Philip, 25
Ethernet, 583
Etichetta, 705
  linguaggio assemblatore, 726
Etichetta globale, 729
Evento, 517
Evoluzione delle macchine multilivello, 8-13
Explicitly Parallel Instruction Computing (EPIC), 433
Extended Data Output (EDO), 187
Extended IDE (EIDE), 93
Extended ISA (EISA), 113
Extended Page Table (EPT), 473

F

Falsa condivisione, 651
Famiglie di computer, esempi, 40
Fanout, 627
Far call, 722
Fast Page Mode (FPM), 187
FAT (*vedi* File Allocation Table)
Fat tree, 629
Field Programmable Gate Array (FPGA), 26, 190-191, 586
FIFO, algoritmo (*vedi* Algoritmo First-In First-Out)
File, 474-475
File Allocation Table (FAT), 507
File binario eseguibile, 726
File immediato, 512
Filtro, 502
Filtro di Bayer, 139
Firewall, 586
Flag della modalità d'incremento, 716
Flag di parità, 719
Flag di riporto ausiliario, 715
Flip-flop, 176
Floppy disk, 89
Flusso, 415-417
  coroutine, 421-423
  diramazioni, 415
  interrupt, 424-427
  procedure, 416-421
  trap, 423
Flusso di controllo sequenziale, 415-416
Formati d'istruzione, 371-381
  ARM, 378-380
  ATmega168, 380
  Core i7, 377
  criteri progettuali, 372-374
Forrester, Jay, 19
FORTRAN, 10, 404
FORTRAN Monitor System (FMS), 10
Fotocamera digitale, 139-141
FPGA (*vedi* Field Programmable Gate Array)
FPM (*vedi* Fast Page Mode)
Frame, 103
Frammentazione, 456
  esterna, 457
  interna, 461
Frame pointer (FP), 362
Frazione, 694
Frequency shift keying, 131
Frequenza del retino dei mezzitoni, 127
Frequenza di fallimento (miss ratio), 85
Frequenza di successi (hit ratio), 84
Full handshake, 202
Full-duplex, 132
Funzione booleana, 154-155
Furto di cicli, 113, 407

G

Gamut (gamma dei colori), 128
GDT (*vedi* Global Descriptor Table)
General Purpose GPU (GPGPU), 592
Generazione del codice, 727
Generazione della checksum, 590
Gerarchia di memoria, 87
Gestione dei processi, 512
  UNIX, 512
  Windows 7, 515
Gestione dell'intestazione, 590
Gestione della coda, 590
Gestore dell'interrupt, 112
Gestore di trap, 423
Gettone di accesso, 509
Ghosting, 117
Global Descriptor Table (GDT), 464
Goldstine, Herman, 18
GPGPU (*vedi* General Purpose GPU)
GPU (*vedi* Graphics Processing Unit)
GPU Fermi, 70-71, 591-594
Graphical User Interface (GUI), 25, 493
Graphics Processing Unit (GPU), 591
Grid computing, 663-666
GridPad, 14
Gruppi d'istruzioni, Itanium 2, 434
GUI (*vedi* Graphical User Interface)

H

Half adder (sommatore), 170
Half-duplex, 132
Hamming, Richard, 31
Handle, 496
Hardware, 8
  in relazione al software, 8
Harvard Mark I, 16
Hashing, 544
Hawkins, Jeff, 28
Hazard (rischio), 307
Headless workstation, 640
Hello World, esempio, 740
High Sierra, 105
Hit ratio, 84
Hoagland, Al, 31
Hoff, Ted, 41
HTTP (*vedi* HyperText Transfer Protocol), 584
Hub principale, 236
HyperText Transfer Protocol (HTTP)
Hyperthreading, Core i7, 573-576
Hypervisor, 472

I

I/O (*vedi* Input/Output)
I/O mappato in memoria, 241
I/O programmato, 404
IA-64, 431-439
IAS, 14
IAS, macchina, 18
IBM 360, 14, 22-23
IBM 701, 19
IBM 704, 19
IBM 709, 10, 19
IBM 7094, 14, 20-22
IBM 801, 62
IBM 1401, 14, 20, 21-22
IBM CoreConnect, 581
IBM Corporation, 19-21
IBM PC, 14, 24-25
IBM POWER4, 14, 27
IBM PS/2, 42
IBM RS6000, 14
IBM Simon, 14
IC (*vedi* Circuiti integrati)
Identificatore dello spazio degli indirizzi, 470
IFU (*vedi* Unità di prelievo dell'istruzione)
IJVM, 297
  codice Java, 273-274
  implementazione, 279-291
  insieme d'istruzioni, 268-273
  modello della memoria, 260-268
  stack, 264-267
ILC (*vedi* Instruction Location Counter)
ILLIAC, 17, 70
Implementazione, 449-452
Implementazione della funzionalità macro, 536
Inchiostro
  a base di coloranti, 129
  a pigmenti, 129
  solido, 1296

Indice destinazione, 710
Indice sorgente, 710
Indirizzamento, 371, 381-396
  8088, 706-710
  a registro, 382
  a registro con indice, 714
  a registro con indice e spiazzamento, 714
  a registro indiretto, 382
  a stack, 386
  ARM, 394
  ATmega 219, 394-395
  con spiazzamento, 714
  Core i7, 392-394
  diretto, 382
  immediato, 381
  implicito, 715
  indicizzato, 384-398
  indicizzato esteso, 385
  istruzioni di salto, 389
Indirizzo, 75
Indirizzo di memoria, 75
Indirizzo effettivo, 716
Indirizzo lineare, 466
Industry Standard Architecture (ISA), 113
Infisso, 386
Iniziatore, bus PCI, 225
I-node, 505
Input/Output (I/O), 111-146
Input/Output, istruzioni, 404-407
Insiemi d'istruzioni a confronto, 414
Instradamento, 589
Instruction Location Counter (ILC), 538
Instruction Pointer, 706, 710
Instruction Register (IR), 56
Integrated drive electronics (IDE), 92
Intel 386, 14
Intel 4004, 40
Intel 8080, 14, 42
Intel 8086, 41, 42
Intel 8088, 706-710
  indirizzamento e memoria, 710-715
  programmi d'esempio, 736-741
  salto corto, 720
  salto lungo, 720
  segmenti, 710
  insieme d'istruzioni, 715-726
Intel 80286, 42
Intel 80386, 42
Intel 80486, 42
Intel Core i7 (*vedi* Core i7)
Intel Xeon, 45
Interconnessione completa, 629
Interconnessione programmabile, 190
Interfacce, 239-243
Interfaccia a scambio di messaggi (MPI), 646
Interfaccia di I/O, 240-243
Interfaccia sincrona di memoria, 215
Internet Service Provider (ISP), 584
Internet via cavo, 134-138
Intero double, 697
Interpretazione, 2
Interprete, 2, 58, 704
Interrupt, 112, 424-427
  impreciso, 330
  preciso, 330
  trasparente, 425
Introduzione, 726
Invalidazione, strategia, 609
Invertitore, 152
IP, intestazione, 585
IP, protocollo, 585
Ipercubo, 630
IR (*vedi* Instruction Register)
Iron Oxide Valley, 31
ISA, bus (*vedi* Industry Standard Architecture)
ISP (*vedi* Internet Service Provider)
Istruzioni
  ARM 410-412
  ATmega168, 412-414
  binarie, 397
  Core i7, 407-410
  di ciclo, 403
  di confronto, 400
  di Input/Output, 404-407
  di salto condizionato, 400-402
  di trasferimento dati, 386
  livello ISA, 351
  unarie, 398
Istruzioni del linguaggio assemblativo, 528-532
Istruzioni di chiamata di procedura, 402
Istruzioni di confronto, 400-402

Istruzioni di un byte, 712
Istruzioni di una parola, 712
Istruzioni per la gestione di directory, 479-480
Itanium 2, 431-439
Itanium 2, scheduling delle istruzioni, 2, 434

J

Java Virtual Machine, 249, 267
Jobs, Steve, 25
Johnniac, 17
Joint Photographic Experts Group (JPEG), 140
Joint Test Action Group (JTAG), 213
JPEG (*vedi* Joint Photographic Experts Group)
JTAG (*vedi* Joint Test Action Group)

K

Kilby, Jack, 22
Kildall, Gary, 24
Kinect, 123

L

LAN (*vedi* Local Area Network)
Land Grid Array (LGA), 162
Land, 110
Larghezza di banda complessiva, 658
Larghezza di banda del processore, 67
Larghezza di banda di bisezione, 627
Latch, 175-177
Latch D, 175
Latch D temporizzato, 170-171, 177
Latch SR, 175
Latenza rotazionale, 90
Latin-1, 143
LBA (*vedi* Logical Block Addressing)
LCD (*vedi* Schermo a cristalli liquidi)
LDT (*vedi* Local Descriptor Table)
LED (*vedi* Light Emitting Diode)
Legge di Amdah, 660
Legge di De Morgan, 159
Legge di Moore, 29
Legge di Nathan, 30
Leibniz, Gottfried Wilhelm von, 13
Letterali, 540
LGA (*vedi* Land Grid Array)
Libreria condivisa, 557
Libreria destinazione, 558
Libreria importata, 557
Libreria statica, 558
Libro arancione, 107
Libro giallo, 102
Libro rosso, 101
Libro verde, 105
Light Emitting Diode (LED), 122
Limitazione termica (Thermal throttling), 213
Linda, 652
Linea di cache, 85, 316
Lines Per Inch (LPI), 127
Linguaggi ad alto livello, 7
Linguaggio, 1
Linguaggio assemblativo, 526-529, 704-706
Linguaggio assemblativo, istruzioni, 528
Linguaggio destinazione, 525
Linguaggio macchina, 1, 704
Linguaggio sorgente, 525
Linkage editor, 546
Linker, 547, 726
  compiti, 547-549
Linking loader, 546
Lisa, 14
Lista delle locazioni libere, 478
Little endian, memoria, 77
Livello, 3
  dei dispositivi, 4
  del linguaggio assemblativo, 525-558
  di microarchitettura, 6, 249-350
  ISA, 6, 353-440
  logico digitale, 151-244
Livello applicativo, 649, 665
Livello dei dispositivi, 4
Livello del linguaggio assemblativo, 527-558
Livello di architettura dell'insieme d'istruzioni, 353-440
Livello di astrazione hardware, 495
Livello di microarchitettura, 6, 249-350
  esempi, 334-341
  progettazione, 291-301
Livello di raccolta, 664

Livello di transazione, PCI Express, 234
Livello ISA, 6, 355
  ARM, 363-365
  ATmega168, 366-367
  Core i7, 361-363
  formati d'istruzione, 371-381
  indirizzamento, 381-396
  panoramica, 355-367
  tipi d'istruzioni, 396-414
  tipi di dati, 367-371
Livello ISA, proprietà, 355
Livello logico digitale, 5
  bus, 194-196, 214-232
  bus PCI, 215-223
  bus PCI express, 231-235
  chip della CPU, 192-193
  circuiti, 162-172
  interfacce di I/O, 239-240
  memoria, 169-185
  porte logiche, 152-162
Livello macchina del sistema operativo, 6, 437-510
Livello risorse, 664
Livello software, 234
Livello struttura, 664
Local Descriptor Table (LDT), 464
Local Area Network (LAN), 583
Località spaziale, 316
Località temporale, 316
Locazione di memoria, 75
Logica negativa, 161
Logica positiva, 161
Logical Block Addressing (LBA), 93
Long, 708
Lovelace, Ada, 15
LPI (*vedi* Lines Per Inch)
LRU, algoritmo (*vedi* Algoritmo Least Recently Used)
Lunghezza del percorso, 292
LUT, 190

M

M.I.T.
  TX-0, 20
  TX-2, 20
Macchina del sistema operativo, 7, 445-518
Macchina di Von Neumann, 18
Macchina fotografica digitale, 139-141
Macchina virtuale, 2, 472
Macchine multilivello, 4-12
  evoluzione, 8
Macintosh, Apple, 25
Macro, chiamata 534
Macro, con parametri, 535
Macro, definizione, 533
Macro, espansione, 534
Macro, linguaggio assemblativo, 532-537
Macro del sistema operativo, 11
Macroarchitettura, 254
Mainframe, 40
MAL (*vedi* Micro Assembly Language)
MANIAC, 17
Mantissa, 694
Mappa di memoria, 449
MAR (*vedi* Memory Address Register)
Mark I, 14
Maschera, 387
  BlueGene, 631-635
  Red Storm, 635-638
Massively Parallel Processor, 602
Master File Table (MFT), 510
Masuoka, Fujio, 99
Mauchley, John, 16
MDR (*vedi* Memory Data Register)
Memoria, 74-87, 174-191
  associativa, 463, 544
  big endian, 77
  cache, 82-85
  CD-ROM, 101, 103
  COMA, 623
  d'attrazione, 624
  flash, 189
  FPM, 187
  interlacciata, 614
  little endian, 77
  non volatile, 188
  RAM, 186-191
  secondaria, 87-111
  virtuale, 446-471
Memoria, condivisa a livello applicativo, 649-657
Memoria condivisa distribuita, 598, 650-652
Memoria d'attrazione, 624

Memoria e indirizzamento, 8088, 710-715
Memoria secondaria, 87-111
Memoria virtuale, 447-471
  ARM, 468-471
  Core i7, 464-468
  e caching, 471
  UNIX, 496-497
  Windows 7, 498-500
Memorie di controllo, 61, 259
Memory Address Register (MAR), 255
Memory Data Register (MDR), 255
Memory Management Unit (MMU), 451
Mescolamento perfetto, 613
Mesh, 629
MESI, protocollo, 610
Metal Oxide Semiconductor (MOS), 154
Metodo, 402
Mezzitoni, 127
MFT (*vedi* Master File Table)
Mic-1, 259-264, 279
Mic-1, notazione 1, 274-279
Mic-2, 301
Mic-3, 305
Mic-4, 310
Mickey, 122
Micro Assembly Language (MAL), 275
Microarchitettura
  a tre bus, 296
  ARM, 341-345
  ATmega168, 346-348
  Core i7, 338-341
  Mic-1, 274-291
  Mic-2, 301-304
  Mic-3, 305-310
  Mic-4, 310-313
  OMAP 4430, 341-345
Microarchitettura della CPU Core i7, 335
Microcodice, 11-13
Microcontrollore, 34
Microdrive, 140
MicroInstruction Register (MIR), 261
Microistruzione, 61, 257
Microprogram counter (MPC), 261
Microprogramma, 6, 707
Microprogrammazione, 8, 12
Microsoft assembler (MASM), 528
Miglioramento delle prestazioni, 661-663
MIMD (*vedi* Multiple Instruction-stream Multiple Data-stream)
Minislot, 137
MIPS, 14
MIR (*vedi* MicroInstruction Register)
MMU (*vedi* Memory Management Unit)
MMX (*vedi* MultiMedia eXtensions)
Modalità di indirizzamento, 381
  analisi, 395-396
  ARM, 394, 395
  istruzioni di salto, 389-390
Modalità kernel, 357
Modalità reale, 361
Modalità utente, 357
Modalità virtuale 8086, 362
Modello di consistenza, 602
Modello di memoria, 357, 731
  Core i7, 358
  grande, 731
  minuscolo, 731
  piccolo, 731
Modem, 130
Modulazione, 131
Modulazione a coppia di bit, 132
Modulazione d'ampiezza, 131
Modulazione di fase, 131
Modulazione di frequenza, 131
Modulo e segno, 688
Moore, Gordon, 29
MOS (*vedi* Metal Oxide Semiconductor)
Motif, 493
Motion Picture Experts Group (MPEG), 579
Motore di elaborazione di pacchetti (PPE), 587
Motore di elaborazione programmabile, 589
Motore di stampa, 126
Motorola 68000, 61
Mouse, 121-123
MPC (*vedi* MicroProgram Counter)
MPEG-2, 579
MPI (*vedi* Interfaccia a scambio di messaggi)
MPP (*vedi* Massively Parallel Processor)
MS-DOS, 26

Multicomputer, 73, 596-602, 625, 645-648
  BlueGene, 631-635
  cluster di Google, 641-645
  MPP, 602
  Red Storm, 635
Multicomputer, prestazioni, 657-663
Multicomputer, software, 645, 658
Multicomputer a scambio di messaggi, 625-657
MULTICS (*vedi* MULTiplexed Information and Computing Service)
MultiMedia eXtensions (MMX), 43
Multiple Instruction-stream Multiple Data-stream (MIMD), 600
MULTiplexed Information and Computing Service (MULTICS), 463, 554
Multiplexer, 164
Multiprocessore, 72, 595-599
  COMA, 623-625
  Core i7, 577-578
  e multicomputer, 594-602
  eterogeneo, 578-582
  NUMA, 600, 614-623
  simmetrico, 606-614
Multiprocessore basato su directory, 617
Multiprocessori eterogenei, 578-582
Multiprocessori in un solo chip, 576-582
Multiprocessori omogenei, 577-578
Multiprogrammazione, 23
Multithreading a grana fine, 570
Multithreading a grana grossa, 571
Multithreading nel chip, 570-576
Multithreading simultaneo, 209, 572
Mutex, 514
Mutua capacitanza, 117
Myhrvold, Nathan, 30

N

NaN (*vedi* Not a Number)
NC-NUMA (*vedi* No Cache NUMA)
NEON, 343
Network Interface Device (NID), 134
Network of Workstations, 602
Newton, 14, 47
Nibble, 408
NID (*vedi* Network Interface Device)
No Cache NUMA (NC-NUMA), 615
No Remote Memory Access (NORMA), 602
Nome simbolico, 704
Nomi mnemonici, 704, 726
NonUniform Memory Access (NUMA), 600, 615
NORMA (*vedi* No Remote Memory Access)
Not a Number (NaN), 700
Notazione in eccesso, 688
Notazione polacca, 386
Notazione polacca inversa, 386
Notazione postfissa, 386
NOW (*vedi* Network of Workstations)
Noyce, Robert, 22
NT file system (NTFS), 507
NTFS (*vedi* NT file system)
NTOS, livello esecutivo, 495
NUMA, 614-623 (*vedi* Nonuniform Memory Access)
Numeri binari
  conversione tra basi, 684-687
  negativi, 688-689
  somma, 690
Numero a precisione doppia, 368
Numero a precisione finita, 681
Numero decimale, 683
Numero denormalizzato, 696
Numero esadecimale, 728
Numero in virgola mobile, 695-696
Numero in virgola mobile, normalizzato, 696
Numero ottale, 728
Nvidia Fermi GPU, 591-594
Nvidia Tegra, 36, 47

O

OCP-IP (*vedi* Open Core Protocol-International Partnership)
Oggetto, file, 726
OGSA (*vedi* Open Grid Services Architecture)
OHCI (*vedi* Open Host Controller Interface)
OLED (*vedi* Organic Light Emitting Diode)
Olsen, Kenneth, 20
OMAP 4430, 342
  bi-endian, 363
  cache dati, 343

formati d'istruzione, 378-380
indirizzamento, 394
tipi di dati, 370
Omnibus, PDP-8, 20
Open Core Protocol-International Partnership (OCP-IP), 582
Open Grid Services Architecture (OGSA), 666
Open Host Controller Interface (OHCI), 238
Operando destinazione, 721
Operando immediato, 381
Operando sorgente, 708
Operazioni binarie, 397-398
Operazioni unarie, 398-400
Operazioni del bus, 205-208
Operazioni della memoria, 255
Opposto, 400
Orca, 654-656
OR-cablata, 196
Ordinamento dei byte, 76-78
Organic Light Emitting Diode (OLED), 120
Organizzazione della CPU, 56
Organizzazione della memoria, 180-183
8088, 706-710
Organizzazione virtuale, 663
OS/2, 26
Osborne-1, 14
Ottale, 683
8088, insieme d'istruzioni, 705-726
Ottetto, 76, 357
Overlay, 447

P

pacchetti d'istruzioni (bundle), 435
caricamenti speculativi, 429
EPIC, modello, 433
scheduling delle istruzioni, 434-436
Pacchetto, 581, 584, 618
PCI, 222 PCI
Pacchetto di ackowledgment, 233
Pagina, 448
Pagina gemella, 651
Pagina impegnata (committed), 498
Pagina libera, 498
Pagina riservata, 498
Paginazione, 447-449
Paginazione a richiesta, 452-456
Palm PDA, 28
Parallel Input/Output (PIO), 239
Parallel Virtual Machine (PVM), 646
Parallelismo a livello d'istruzione, 65-69, 563-570
Parallelismo a livello di processore, 69-73
Parallelismo, nel chip, 562-570
Parametri attuali, 535
Parametri delle macro, 535
Parametri di valutazione software, 658
Parametri formali, 535
Parola, 76
Parola di codice (codeword), 79
Pascal, Blaise, 13
Payload, pacchetto PCI, 237, 585
PBGA (ball grid array), 218
PC (*vedi* Program Counter)
PCI, bus (*vedi* Peripheral Component Interconnect, bus)
PCI Express, 230-235
architettura, 231-239
PCI Express, bus (PCIe, bus), 113
PCI Express, livello fisico, 233
PCI Express, pila di protocolli, 232
PCIe (*vedi* PCI Express)
PCIe, bus (*vedi* PCI Express, bus)
PDA (see Personal Digital AssistantPDA (*vedi* Personal Digital Assistant)
PDP-1, 14, 20
PDP-11, 14, 24
PDP-8, 14, 20
Pentium 4, 26, 42, 43
Percorso assoluto, 502
Percorso dati, 6, 56-57, 250-260
Mic-1, 259
Mic-2, 301
Mic-3, 305
Mic-4, 310
Percorso dati, temporizzazione, 253-255
Percorso relativo, 503
Peripheral Component Interconnect (PCI), bus, 113, 222
arbitraggio, 226
segnali, 227
transazioni, 229

Personal computer, 24-27, 37
Personal Digital Assistant (PDA), 28
Pervasive computing (computazione pervasiva), 29
PGA (*vedi* Pin Grid Array)
Pietre miliari nell'architettura dei computer, 13-28
Pin Grid Array (PGA), 162
PIO (*vedi* Parallel Input/Output)
Pipe, 236, 512
Pipeline, 65, 305
a sette stadi, 310
Mic-3, 305
Mic-4, 310
OMAP, 341
Pipeline, memoria del Core i7, 213
Pipeline, stadi, 65
Pipeline in stallo, 308
Pipeline U, 67
Pipeline V, 67
Pit, 102
Pixel, 120
Plain Old Telephone Service (POTS), 133
PlayStation 3, 36
Poison bit, 334
Politica di sostituzione delle pagine, 454-456
Politica write-allocate, 609
Porta, 5, 152-154
Portable Operating System-IX (POSIX), 491
Portante, 130
Porzione costante di memoria, 267-270
POSIX (*vedi* Portable Operating System-IX)
Posizione di ritardo, 322
POTS (*vedi* Plain Old Telephone Service)
Power gating, 216
PPE (*vedi* Motore di elaborazione di pacchetti)
Preambolo, 89
Predicati, 436-438
Predittore dei salti, Core i7, 338
Predizione dei salti, 321-326
dinamica, 312-315, 323-326
statica, 321
Prefetching, 65, 662
Prefisso, 410
Prestazioni
della rete, miglioramento, 590
miglioramento, 661
parametri di valutazione hardware, 657
parametri di valutazione software, 658
Prestazioni, miglioramento, 661
Prestazioni dei computer paralleli, 657-663
Principi di progettazione, 63-65
Principio di località, 84, 454
Problema dei riferimenti in avanti, 537
Problema della rilocazione, 549
Procedura, 402, 416-421
Procedura ricorsiva, 416
Processo di assemblaggio, 537-544
Processo figlio, 512
Processo genitore, 512
Processore di rete, 583-594
Processore grafico, 591-594
Processore, 55-74
Processore, bi-endian, 363
Processore vettoriale, 72
Processori di rete, incremento delle prestazioni, 590
Progetto giapponese di quinta generazione, 27
Program Counter (PC), 56, 364, 706
Program Status Word (PSW), 360, 467
Programma, 1
Programma binario, 704
Programma eseguibile binario, 526, 546
Programma indipendente dalla posizione, 553
Programmable ROM, 188
Programmatore di sistema, 7
Programmazione in linguaggio assemblativo, 726
Prologo della procedura, 420
PROM (*vedi* Programmable ROM)
Protocollo, 584
bus, 191
IP, 585
MESI, 606
PCI Express, 230-235
Protocollo di coerenza delle cache, 607
Protocollo internet, 584
Protocollo write-back, 609
Protocollo write-once, 609
Pseudoistruzione, 530-532, 692, 718

PSW (*vedi* Program Status Word)
Pthread, 514
Puntatore, 382
Puntatore allo stack, 709
Puntatore base, 709
Puntatore di lettura/scrittura, 724
Punto d'ingresso, 550
PVM (*vedi* Parallel Virtual Machine)

R

Radio Frequency IDentification (RFID), 32
RAID (*vedi* Redundant Array of Inexpensive Disks)
RAM (*vedi* Random Access Memory)
RAM della scheda video, 120-121
RAM dinamica, 187
RAM statica (SRAM), 187
Random Access Memory (RAM), 74-87, 186-191
  DDR, 188
  RAM dinamica, 187
  SDRAM, 187
Ranging, 137
Read Only Memory (ROM), 188
Record d'attivazione, 362, 709
Record logico, 475
Red Storm, 635-640
Reduced Instruction set Computer (RISC), 62
  e CISC, 62-63
  principi di progettazione, 63-65
Redundant Array of Inexpensive Disks (RAID), 96
Refresh (ricarica), 187
Registrazione perpendicolare, 90
Registri di segmento, 710
Registri, 5, 180, 360, 708
  di flag, 360
  PSW, 360
Registro, di una parola, 712
Registro, indice, 709
Registro a scorrimento, 169
Registro a scorrimento della storia dei salti, 326
Registro base, 708
Registro contatore, 708
Registro dati, 708
Registro dei codici di condizione, 716
Registro di flag, 360 710
Registro di un byte, 712
Registro generale, 708-708
Registro H, 293
Registro puntatore, 709-710
Registro vettoriale, 72
Registro virtuale, 264
ReOrder buffer (ROB), Core i7, 338
Replicated worker model, 653
Rete
  anello, 577
  Ethernet, 583
  LAN, 583
  store-and-forward, 584
  wan, 583
Rete a commutazione multilivello, 612-614
Rete bloccante, 614
Rete non bloccante, 612
Rete omega, 613
Rete store-and-forward, 584
Reti, introduzione, 586-588
Reti combinatorie, 163-168
Reti d'interconnessione, 626-630
  larghezza di banda di bisezione, 627
  topologia, 627
Retrocompatibile, 354
RFID (*vedi* Radio Frequency IDentification)
Ricevitore del bus, 195
Ricorsione, 402
Riduzione del percorso, 294-301
Riferimento esterno, 549
Rinomina dei registri, 326-332
RISC (*vedi* Reduced Instruction Set Computer)
RISC e CISC, 62-63
Ritardo della porta, 163
ROB (*vedi* ReOrder Buffer)
ROM (*vedi* Read Only Memory)
Route Instradamento, 589
Router, 583

S

Salto condizionato, 720-721
Sandy Bridge, 208, 335

Scala dinamica di tensione, 217
Scale, Index, Base byte (SIB), 378, 393
Scambio di messaggi bufferizzato, 646
Scambio di messaggi non bloccante, 646
Scambio sincrono di messaggi, 646
Scanner delle pagine, 616
Scheda madre, 111
Schedulatore, Core i7, 337
Scheduling, multicomputer, 648-649
Schermi a matrice passiva, 119
Schermo, 118
Schermo a cristalli liquidi (LCD), 118
Schermo a matrice attiva, 120
Schermo multitouch, 117
Schermo piatto, 118
Schermo TFT, 120
Scoreboard, 328
SCSI (*vedi* Small Computer System Interface)
SDRAM (*vedi* Synchronous DRAM)
Seastar, 636
Secchielli (bucket), 545
Seconda passata, assemblatore, 727
Security ID (SID), 509
Seek (ricerca), 90
Segment override, 725
Segmentazione, 457-471
  implementazione, 461
Segmento, 458, 710
  8088, 706
Segmento dati, 711, 720
Segmento di codice, 706
Segmento di collegamento, 554
Segmento extra, 720
Segnale asserito, 185
Segnale di controllo, 255
Segnale negato, 185
Selezione del percorso, 589
Semaforo, 487
Semantica della memoria, 602-606
  consistenza debole, 604
  consistenza della cache, 607
  consistenza di processore, 604
  consistenza dopo rilascio, 605
  consistenza sequenziale, 603
  consistenza stretta, 602
Sequenzializzatore, 259
Serial ATA, 93
Server, 37
Sessione, CD-ROM, 107
Settore, 89
Sezione dati, 727
Sezione informativa, 356
Sezione testo, 727
Shard, 642
Shell, 493
Shockley, William, 19
SIB (*vedi* Scale, Index, Base byte)
SID (*vedi* Security ID)
Significando, 698
Simbolo esterno, 550
SIMD (*vedi* Single Instruction Multiple Data)
SIMM (*vedi* Single Inline Memory Modules)
Simon, smartphone, 28
Simplex, 132
Sincronizzazione dei processi, 486-489
Single Inline Memory Modules (SIMM), 86
Single Instruction Multiple Data (SIMD), 86, 591
Single Large Expensive Disk (SLED), 96
Sistema a memoria distribuita, 596
Sistema batch, 11
Sistema con legame lasco (loosy coupled), 73, 562
Sistema con legame stretto (tightly coupled), 562
Sistema little endian, 713
Sistema operativo, 445
  a condivisione di tempo, 11
  CP/M, 24
  OS/2, 26
  storia, 9-11
  UNIX, 490-507, 512-515
  Windows 7, 493-496, 498-500, 507-512, 515-518
  Windows, 498-500
Sistema operativo ospite, 473
Sistema scalabile, 661
Sistemi a condivisione di tempo, 11
Sistemi di numerazione in base fissa, 683-684
Situazione di stallo, 308
Skew, bus, 114
Slave, 195